

RTI Connext Micro C++ API
Version 4.3.0 ER1

1 RTI Connex Micro C++ API	1
2 Topic Index	2
2.1 Reference Manual	2
3 Hierarchical Index	8
3.1 Class Hierarchy	8
4 Class Index	14
4.1 Class List	14
5 File Index	24
5.1 File List	24
6 Topic Documentation	25
6.1 Documentation Guide	25
6.1.1 Unsupported Features	25
6.1.2 API Naming Conventions	25
6.1.3 API Documentation Terms	26
6.1.4 Stereotypes	26
6.2 DDS API	27
6.2.1 Detailed Description	28
6.2.2 Overview	28
6.2.3 Conceptual Model	29
6.2.4 Modules	30
6.2.5 Content Filter Plugin	31
6.2.6 Domain Module	31
6.2.7 Topic	42
6.2.8 Publication Module	54
6.2.9 Subscription Module	63
6.2.10 Infrastructure Module	74
6.2.11 General Utilities and Compliance Configuration	134
6.3 DPSE Static Discovery API	135
6.3.1 Detailed Description	135
6.3.2 Macro Definition Documentation	135
6.4 DPDE Dynamic Discovery API	136
6.4.1 Detailed Description	136
6.4.2 Macro Definition Documentation	136
6.5 CDR Stream API	136
6.5.1 Detailed Description	137
6.5.2 Function Documentation	137

6.6 RT Run-Time API	140
6.6.1 Detailed Description	140
6.7 RHSM Reader History API	141
6.7.1 Detailed Description	141
6.8 WHSM Writer History API	141
6.8.1 Detailed Description	141
6.9 NETIO API	141
6.9.1 Detailed Description	142
6.9.2 NETIO Packet API	142
6.9.3 NETIO Interface API	142
6.9.4 NETIO Address API	144
6.10 UDP Transport API	146
6.10.1 Detailed Description	147
6.10.2 Macro Definition Documentation	147
6.10.3 Function Documentation	147
6.11 UDP Transform API	149
6.12 RTPS Transport API	149
6.12.1 Detailed Description	150
6.12.2 Typedef Documentation	150
6.12.3 Enumeration Type Documentation	151
6.12.4 Variable Documentation	152
6.13 Shared Memory Transport API	153
6.13.1 Detailed Description	154
6.13.2 NETIO Shared Memory API	154
6.14 NETIO DGRAM API	173
6.14.1 Detailed Description	174
6.14.2 Macro Definition Documentation	175
6.14.3 Typedef Documentation	177
6.14.4 Function Documentation	178
6.14.5 Variable Documentation	179
6.15 Zero Copy v2 API	179
6.15.1 Detailed Description	181
6.15.2 Macro Definition Documentation	181
6.15.3 Typedef Documentation	181
6.15.4 Notification Interface	183
6.16 OS API	184
6.16.1 Detailed Description	185
6.16.2 OSAPI Heap API	185
6.16.3 Trace API	190

6.16.4 OSAPI Mutex API	190
6.16.5 OSAPI Process API	192
6.16.6 OSAPI Semaphore API	193
6.16.7 OSAPI Memory API	195
6.16.8 OSAPI String API	195
6.16.9 OSAPI System API	195
6.16.10 OSAPI Thread API	200
6.16.11 OSAPI time API	204
6.16.12 OSAPI Timer API	204
6.16.13 OSAPI Types	205
6.16.14 OSAPI_SharedMemoryMonitor	208
6.16.15 OSAPI_SharedMemorySegment	211
6.16.16 Log API	218
6.17 Application Generation API	221
6.17.1 Detailed Description	221
6.17.2 Function Documentation	222
6.18 Lightweight Security API	223
6.18.1 Detailed Description	223
6.18.2 Macro Definition Documentation	223
6.18.3 Variable Documentation	224
6.19 Resource Limits	224
6.19.1 Detailed Description	224
6.19.2 Introduction	224
6.19.3 Dynamic Memory Allocation	224
6.19.4 DomainParticipantFactoryQos	225
6.19.5 DomainParticipantQos	225
6.19.6 DataReaderQos	231
6.19.7 DataWriterQos	236
6.19.8 OSAPI	238
6.19.9 UDP Transport	238
6.19.10 Dynamic Participant Static Endpoint (DPSE)	239
6.19.11 Dynamic Participant Dynamic Endpoint (DPDE)	240
6.20 Log Codes	242
6.20.1 Detailed Description	242
6.20.2 OSAPI	242
6.20.3 REDA	266
6.20.4 DB	272
6.20.5 RT	278
6.20.6 CDR	280

6.20.7 NETIO	281
6.20.8 RTPS	309
6.20.9 DDS_C	340
6.20.10 ZeroCopy	452
6.20.11 WHSQ	462
6.20.12 RH	464
6.20.13 WH	469
6.20.14 DPSE	471
6.20.15 DPDE	484
6.20.16 APPGEN	496
6.20.17 DDS_FILTER	499
7 Class Documentation	510
7.1 CDR_Stream_t Struct Reference	510
7.1.1 Detailed Description	510
7.2 DDS_BuiltinTopicKey_t Struct Reference	510
7.2.1 Detailed Description	511
7.2.2 Member Data Documentation	511
7.3 DDS_ChecksumProperty_t Struct Reference	511
7.3.1 Detailed Description	512
7.3.2 Member Data Documentation	512
7.4 DDS_ContentFilterProperty Struct Reference	512
7.4.1 Detailed Description	513
7.4.2 Member Data Documentation	513
7.5 DDS_ContentFilterQosPolicy Struct Reference	514
7.5.1 Detailed Description	514
7.5.2 Usage	515
7.5.3 Member Data Documentation	515
7.6 DDS_DataReaderInstanceReplacedStatus Struct Reference	516
7.6.1 Detailed Description	517
7.6.2 Member Data Documentation	517
7.7 DDS_DataReaderProtocolQosPolicy Struct Reference	518
7.7.1 Detailed Description	518
7.7.2 Member Data Documentation	518
7.8 DDS_DataReaderQos Struct Reference	519
7.8.1 Detailed Description	520
7.8.2 Member Data Documentation	521
7.9 DDS_DataReaderResourceLimitsQosPolicy Struct Reference	523
7.9.1 Detailed Description	524

7.9.2 Member Data Documentation	524
7.10 DDS_DataRepresentationIdSeq Struct Reference	527
7.10.1 Detailed Description	527
7.11 DDS_DataRepresentationQosPolicy Struct Reference	527
7.11.1 Detailed Description	528
7.11.2 Member Data Documentation	528
7.12 DDS_DataWriterProtocolQosPolicy Struct Reference	528
7.12.1 Detailed Description	529
7.12.2 Member Data Documentation	529
7.13 DDS_DataWriterQos Struct Reference	530
7.13.1 Detailed Description	532
7.13.2 Member Data Documentation	532
7.14 DDS_DataWriterResourceLimitsQosPolicy Struct Reference	535
7.14.1 Detailed Description	535
7.14.2 Member Data Documentation	536
7.15 DDS_DataWriterShmemRefTransferModeSettings Struct Reference	538
7.15.1 Detailed Description	538
7.15.2 Member Data Documentation	538
7.16 DDS_DataWriterTransferModeQosPolicy Struct Reference	538
7.16.1 Detailed Description	539
7.16.2 Member Data Documentation	539
7.17 DDS_DeadlineQosPolicy Struct Reference	539
7.17.1 Detailed Description	540
7.17.2 Usage	540
7.17.3 Compatibility	540
7.17.4 Member Data Documentation	541
7.18 DDS_DestinationOrderQosPolicy Struct Reference	541
7.18.1 Detailed Description	541
7.18.2 Member Data Documentation	541
7.19 DDS_DiscoveryComponent Struct Reference	542
7.19.1 Detailed Description	542
7.19.2 Member Data Documentation	542
7.20 DDS_DiscoveryQosPolicy Struct Reference	543
7.20.1 Detailed Description	543
7.20.2 Usage	543
7.20.3 Member Data Documentation	544
7.21 DDS_DomainParticipantFactoryQos Struct Reference	546
7.21.1 Detailed Description	547
7.21.2 Member Data Documentation	547

7.22 DDS_DomainParticipantQos Struct Reference	547
7.22.1 Detailed Description	548
7.22.2 Member Data Documentation	548
7.23 DDS_DomainParticipantResourceLimitsQosPolicy Struct Reference	550
7.23.1 Detailed Description	552
7.23.2 Member Data Documentation	552
7.24 DDS_DurabilityQosPolicy Struct Reference	561
7.24.1 Detailed Description	561
7.24.2 Member Data Documentation	562
7.25 DDS_Duration_t Struct Reference	562
7.25.1 Detailed Description	562
7.25.2 Member Data Documentation	562
7.26 DDS_EntityFactoryQosPolicy Struct Reference	563
7.26.1 Detailed Description	563
7.26.2 Usage	563
7.26.3 Member Data Documentation	564
7.27 DDS_EntityNameQosPolicy Struct Reference	564
7.27.1 Detailed Description	564
7.27.2 Usage	565
7.27.3 Member Data Documentation	565
7.28 DDS_FilterQosPolicy Struct Reference	565
7.28.1 Detailed Description	566
7.28.2 Usage	566
7.28.3 Member Data Documentation	566
7.29 DDS_FilterResourceLimits Struct Reference	567
7.29.1 Detailed Description	568
7.29.2 Member Data Documentation	568
7.30 DDS_FlowControllerProperty_t Struct Reference	570
7.30.1 Detailed Description	570
7.30.2 Member Data Documentation	571
7.31 DDS_FlowControllerTokenBucketProperty_t Struct Reference	571
7.31.1 Detailed Description	572
7.31.2 Member Data Documentation	572
7.32 DDS_GroupDataQosPolicy Struct Reference	574
7.32.1 Detailed Description	574
7.32.2 Usage	575
7.32.3 Member Data Documentation	575
7.33 DDS_GUID_t Struct Reference	575
7.33.1 Detailed Description	575

7.33.2 Member Data Documentation	576
7.34 DDS_HistoryQosPolicy Struct Reference	576
7.34.1 Detailed Description	576
7.34.2 Usage	577
7.34.3 Consistency	577
7.34.4 Member Data Documentation	577
7.35 DDS_InconsistentTopicStatus Struct Reference	577
7.35.1 Detailed Description	578
7.36 DDS_InstanceHandleSeq Struct Reference	578
7.36.1 Detailed Description	578
7.37 DDS_LatencyBudgetQosPolicy Struct Reference	578
7.37.1 Detailed Description	579
7.37.2 Usage	579
7.37.3 Compatibility	579
7.37.4 Member Data Documentation	580
7.38 DDS_LivelinessChangedStatus Struct Reference	580
7.38.1 Detailed Description	580
7.38.2 Member Data Documentation	580
7.39 DDS_LivelinessLostStatus Struct Reference	581
7.39.1 Detailed Description	582
7.39.2 Member Data Documentation	582
7.40 DDS_LivelinessQosPolicy Struct Reference	582
7.40.1 Detailed Description	583
7.40.2 Usage	584
7.40.3 Compatibility	584
7.40.4 Member Data Documentation	585
7.41 DDS_Locator_t Struct Reference	585
7.41.1 Detailed Description	585
7.41.2 Member Data Documentation	586
7.42 DDS_LocatorSeq Struct Reference	586
7.42.1 Detailed Description	586
7.43 DDS_OfferedDeadlineMissedStatus Struct Reference	586
7.43.1 Detailed Description	587
7.43.2 Member Data Documentation	587
7.44 DDS_OfferedIncompatibleQosStatus Struct Reference	588
7.44.1 Detailed Description	588
7.44.2 Member Data Documentation	588
7.45 DDS_OwnershipQosPolicy Struct Reference	589
7.45.1 Detailed Description	589

7.45.2 Usage	590
7.45.3 Compatibility	591
7.45.4 Member Data Documentation	592
7.46 DDS_OwnershipStrengthQosPolicy Struct Reference	593
7.46.1 Detailed Description	593
7.46.2 Member Data Documentation	593
7.47 DDS_ParticipantBuiltinTopicData Struct Reference	594
7.47.1 Detailed Description	594
7.47.2 Member Data Documentation	595
7.48 DDS_PartitionQosPolicy Struct Reference	597
7.48.1 Detailed Description	597
7.48.2 Member Data Documentation	599
7.49 DDS_PresentationQosPolicy Struct Reference	599
7.49.1 Detailed Description	600
7.49.2 Compatibility	602
7.49.3 Member Data Documentation	602
7.50 DDS_ProductVersion_t Struct Reference	603
7.50.1 Detailed Description	604
7.50.2 Member Data Documentation	604
7.51 DDS_Property_t Struct Reference	604
7.51.1 Detailed Description	605
7.51.2 Member Data Documentation	605
7.52 DDS_PropertyQosPolicy Struct Reference	605
7.52.1 Detailed Description	606
7.52.2 Member Data Documentation	606
7.53 DDS_PropertySeq Struct Reference	606
7.53.1 Detailed Description	606
7.54 DDS_ProtocolVersion_t Struct Reference	607
7.54.1 Detailed Description	607
7.54.2 Member Data Documentation	607
7.55 DDS_PskPassTracker_Interface Struct Reference	607
7.55.1 Detailed Description	608
7.55.2 Member Data Documentation	608
7.56 DDS_PskServiceFactoryProperty Struct Reference	608
7.56.1 Detailed Description	609
7.56.2 Member Data Documentation	609
7.57 DDS_PublicationBuiltinTopicData Struct Reference	611
7.57.1 Detailed Description	612
7.57.2 Member Data Documentation	612

7.58 DDS_PublicationMatchedStatus Struct Reference	615
7.58.1 Detailed Description	615
7.58.2 Member Data Documentation	616
7.59 DDS_PublisherQos Struct Reference	616
7.59.1 Detailed Description	617
7.59.2 Member Data Documentation	617
7.60 DDS_PublishModeQosPolicy Struct Reference	618
7.60.1 Detailed Description	618
7.60.2 Usage	619
7.60.3 Member Data Documentation	619
7.61 DDS_QosPolicyCount Struct Reference	620
7.61.1 Detailed Description	620
7.61.2 Member Data Documentation	620
7.62 DDS_ReliabilityQosPolicy Struct Reference	621
7.62.1 Detailed Description	621
7.62.2 Usage	621
7.62.3 Compatibility	622
7.62.4 Member Data Documentation	622
7.63 DDS_ReliableReaderActivityChangedStatus Struct Reference	623
7.63.1 Detailed Description	623
7.63.2 Member Data Documentation	623
7.64 DDS_ReliableSampleUnacknowledgedStatus Struct Reference	624
7.64.1 Detailed Description	625
7.64.2 Member Data Documentation	625
7.65 DDS_RequestedDeadlineMissedStatus Struct Reference	625
7.65.1 Detailed Description	626
7.65.2 Member Data Documentation	626
7.66 DDS_RequestedIncompatibleQosStatus Struct Reference	626
7.66.1 Detailed Description	627
7.66.2 Member Data Documentation	627
7.67 DDS_ResourceLimitsQosPolicy Struct Reference	627
7.67.1 Detailed Description	628
7.67.2 Usage	628
7.67.3 Consistency	629
7.67.4 Member Data Documentation	629
7.68 DDS_RtpsReliableReaderProtocol_t Struct Reference	631
7.68.1 Detailed Description	632
7.68.2 Member Data Documentation	632
7.69 DDS_RtpsReliableWriterProtocol_t Struct Reference	632

7.69.1 Detailed Description	633
7.69.2 Member Data Documentation	633
7.70 DDS_RtpsWellKnownPorts_t Struct Reference	634
7.70.1 Detailed Description	635
7.70.2 Member Data Documentation	636
7.71 DDS_SampleIdentity_t Struct Reference	639
7.71.1 Detailed Description	639
7.71.2 Member Data Documentation	639
7.72 DDS_SampleInfo Struct Reference	639
7.72.1 Detailed Description	640
7.72.2 Interpretation of the SampleInfo	640
7.72.3 Member Data Documentation	641
7.73 DDS_SampleInfoSeq Struct Reference	643
7.73.1 Detailed Description	643
7.74 DDS_SampleLostStatus Struct Reference	643
7.74.1 Detailed Description	643
7.74.2 Member Data Documentation	644
7.75 DDS_SampleRejectedStatus Struct Reference	644
7.75.1 Detailed Description	645
7.75.2 Member Data Documentation	645
7.76 DDS_SecTransform_Configuration Struct Reference	645
7.76.1 Detailed Description	646
7.76.2 Member Data Documentation	646
7.77 DDS_SequenceNumber_t Struct Reference	646
7.77.1 Detailed Description	647
7.77.2 Member Data Documentation	647
7.78 DDS_SubscriberQos Struct Reference	647
7.78.1 Detailed Description	647
7.78.2 Member Data Documentation	648
7.79 DDS_SubscriptionBuiltinTopicData Struct Reference	648
7.79.1 Detailed Description	650
7.79.2 Member Data Documentation	650
7.80 DDS_SubscriptionMatchedStatus Struct Reference	653
7.80.1 Detailed Description	654
7.80.2 Member Data Documentation	654
7.81 DDS_SystemResourceLimitsQosPolicy Struct Reference	655
7.81.1 Detailed Description	655
7.81.2 Member Data Documentation	655
7.82 DDS_Time_t Struct Reference	656

7.82.1 Detailed Description	656
7.82.2 Member Data Documentation	657
7.83 DDS_TopicDataQosPolicy Struct Reference	657
7.83.1 Detailed Description	657
7.83.2 Usage	658
7.83.3 Member Data Documentation	658
7.84 DDS_TopicQos Struct Reference	658
7.84.1 Detailed Description	658
7.84.2 Member Data Documentation	658
7.85 DDS_TransportPriorityQosPolicy Struct Reference	659
7.85.1 Detailed Description	659
7.85.2 Member Data Documentation	659
7.86 DDS_TransportQosPolicy Struct Reference	659
7.86.1 Detailed Description	660
7.86.2 Member Data Documentation	660
7.87 DDS_TrustQosPolicy Struct Reference	660
7.87.1 Detailed Description	661
7.87.2 Member Data Documentation	661
7.88 DDS_UserDataQosPolicy Struct Reference	661
7.88.1 Detailed Description	662
7.88.2 Usage	662
7.88.3 Member Data Documentation	662
7.89 DDS_UserTrafficQosPolicy Struct Reference	663
7.89.1 Detailed Description	663
7.89.2 Member Data Documentation	663
7.90 DDS_VendorId_t Struct Reference	664
7.90.1 Detailed Description	664
7.90.2 Member Data Documentation	664
7.91 DDS_WireProtocolQosPolicy Struct Reference	664
7.91.1 Detailed Description	665
7.91.2 Usage	665
7.91.3 Member Data Documentation	665
7.92 DDS_WriteParams_t Struct Reference	668
7.92.1 Detailed Description	669
7.92.2 Member Data Documentation	669
7.93 DDSCondition Class Reference	669
7.93.1 Detailed Description	669
7.94 DDSConditionSeq Class Reference	670
7.94.1 Detailed Description	670

7.95 DDSDataReader Class Reference	670
7.95.1 Detailed Description	672
7.95.2 Member Function Documentation	673
7.95.3 SEQUENCES USAGE IN TAKE AND READ	677
7.96 DDSDataReaderListener Class Reference	688
7.96.1 Detailed Description	689
7.96.2 Member Function Documentation	689
7.97 DDSDataWriter Class Reference	692
7.97.1 Detailed Description	693
7.97.2 Member Function Documentation	694
7.98 DDSDataWriterListener Class Reference	710
7.98.1 Detailed Description	711
7.98.2 Member Function Documentation	711
7.99 DDSDomainEntity Class Reference	714
7.99.1 Detailed Description	715
7.100 DDSDomainParticipant Class Reference	715
7.100.1 Detailed Description	717
7.100.2 Member Function Documentation	718
7.101 DDSDomainParticipantFactory Class Reference	739
7.101.1 Detailed Description	740
7.101.2 Member Function Documentation	741
7.102 DDSDomainParticipantListener Class Reference	748
7.102.1 Detailed Description	749
7.102.2 Member Function Documentation	750
7.103 DDSEntity Class Reference	756
7.103.1 Detailed Description	756
7.103.2 Abstract operations	757
7.103.3 Member Function Documentation	758
7.104 DDSFlowController Class Reference	761
7.104.1 Detailed Description	761
7.104.2 Member Function Documentation	761
7.105 DDSGuardCondition Class Reference	763
7.105.1 Detailed Description	764
7.105.2 Member Function Documentation	764
7.106 DDSListener Class Reference	764
7.106.1 Detailed Description	765
7.106.2 Access to Plain Communication Status	765
7.106.3 Access to Read Communication Status	766
7.107 DDSPropertyQosPolicyHelper Class Reference	767

7.107.1 Detailed Description	767
7.107.2 Member Function Documentation	767
7.108 DDSPublisher Class Reference	770
7.108.1 Detailed Description	771
7.108.2 Member Function Documentation	771
7.109 DDSPublisherListener Class Reference	777
7.109.1 Detailed Description	778
7.110 DDSStatusCondition Class Reference	779
7.110.1 Detailed Description	779
7.110.2 Member Function Documentation	780
7.111 DDSSubscriber Class Reference	781
7.111.1 Detailed Description	782
7.111.2 Member Function Documentation	783
7.112 DDSSubscriberListener Class Reference	789
7.112.1 Detailed Description	790
7.112.2 Member Function Documentation	790
7.113 DDSTopic Class Reference	790
7.113.1 Detailed Description	792
7.113.2 Member Function Documentation	792
7.114 DDSTopicDescription Class Reference	797
7.114.1 Detailed Description	797
7.114.2 Member Function Documentation	797
7.115 DDSTopicListener Class Reference	799
7.115.1 Detailed Description	799
7.115.2 Member Function Documentation	799
7.116 DDSWaitSet Class Reference	800
7.116.1 Detailed Description	800
7.116.2 Usage	800
7.116.3 Trigger State of a ::DDSStatusCondition	801
7.116.4 Trigger State of a ::DDSGuardCondition	801
7.116.5 Member Function Documentation	802
7.117 DPDE_DiscoveryPluginProperty Struct Reference	804
7.117.1 Detailed Description	805
7.117.2 Member Data Documentation	805
7.118 DPDEDiscoveryFactory Class Reference	809
7.118.1 Detailed Description	809
7.118.2 Member Function Documentation	809
7.119 DPSE_DiscoveryPluginProperty Struct Reference	809
7.119.1 Detailed Description	810

7.119.2 Member Data Documentation	810
7.120 DPSEDiscoveryFactory Class Reference	812
7.120.1 Detailed Description	812
7.120.2 Member Function Documentation	812
7.121 DPSEDiscoveryPlugin Class Reference	813
7.121.1 Detailed Description	813
7.121.2 Member Function Documentation	813
7.122 FooDataReader Class Reference	814
7.122.1 Detailed Description	815
7.122.2 Member Function Documentation	815
7.122.3 SEQUENCES USAGE IN TAKE AND READ	820
7.123 FooDataWriter Class Reference	826
7.123.1 Detailed Description	827
7.123.2 Member Function Documentation	827
7.124 FooSeq Struct Reference	841
7.124.1 Detailed Description	842
7.124.2 Constructor & Destructor Documentation	843
7.124.3 Member Function Documentation	844
7.124.4 Member Data Documentation	852
7.125 FooTypeSupport Class Reference	854
7.125.1 Detailed Description	854
7.125.2 Member Function Documentation	854
7.126 NDDS_Discovery_Property Struct Reference	860
7.126.1 Detailed Description	860
7.127 NDDS_Type_PluginVersion Struct Reference	860
7.127.1 Detailed Description	860
7.127.2 Member Data Documentation	860
7.128 NETIO_Address Struct Reference	861
7.128.1 Detailed Description	861
7.128.2 Member Data Documentation	861
7.129 NETIO_AddressSeq Struct Reference	862
7.129.1 Detailed Description	862
7.130 NETIO_AddressUInt32 Struct Reference	862
7.130.1 Detailed Description	862
7.130.2 Member Data Documentation	862
7.131 NETIO_AddressValue Union Reference	862
7.131.1 Detailed Description	863
7.131.2 Member Data Documentation	863
7.132 NETIO_DGRAM_Interfacel Struct Reference	863

7.132.1 Detailed Description	864
7.132.2 Member Data Documentation	864
7.133 NETIO_DGRAM_InterfaceMultiCastGroup Struct Reference	867
7.133.1 Detailed Description	867
7.133.2 Member Data Documentation	867
7.134 NETIO_DGRAM_InterfaceRouteEntry Struct Reference	868
7.134.1 Detailed Description	868
7.134.2 Member Data Documentation	868
7.135 NETIO_DGRAM_InterfaceTableEntry Struct Reference	869
7.135.1 Detailed Description	869
7.135.2 Member Data Documentation	869
7.136 NETIO_DGRAM_InterfaceTableEntrySeq Struct Reference	870
7.136.1 Detailed Description	871
7.137 NETIO_Interface Struct Reference	871
7.137.1 Detailed Description	871
7.138 NETIO_InterfaceI Struct Reference	871
7.138.1 Detailed Description	871
7.139 NETIO_Netmask Struct Reference	871
7.139.1 Detailed Description	872
7.139.2 Member Data Documentation	872
7.140 NETIO_NetmaskSeq Struct Reference	872
7.140.1 Detailed Description	872
7.141 NETIO_Packet Struct Reference	872
7.141.1 Detailed Description	872
7.142 NETIO_SharedMemorySegmentHeader Struct Reference	873
7.142.1 Detailed Description	873
7.143 NETIO_SHMEMInterfaceFactoryProperty Struct Reference	873
7.143.1 Detailed Description	873
7.143.2 Member Data Documentation	873
7.144 OSAPI_LogProperty Struct Reference	874
7.144.1 Detailed Description	874
7.144.2 Member Data Documentation	875
7.145 OSAPI_SharedMemorySegmentHandle Struct Reference	875
7.145.1 Detailed Description	875
7.145.2 Member Data Documentation	876
7.146 OSAPI_SharedMemorySegmentHeader Struct Reference	876
7.146.1 Detailed Description	876
7.146.2 Member Data Documentation	876
7.147 OSAPI_SystemI Struct Reference	877

7.147.1 Detailed Description	877
7.147.2 Member Data Documentation	877
7.148 OSAPI_SystemProperty Struct Reference	878
7.148.1 Detailed Description	879
7.148.2 Member Data Documentation	879
7.149 OSAPI_SystemTime Struct Reference	880
7.149.1 Detailed Description	880
7.149.2 Member Data Documentation	881
7.150 OSAPI_TaskProperty Struct Reference	881
7.150.1 Detailed Description	881
7.150.2 Member Data Documentation	881
7.151 OSAPI_ThreadProperty Struct Reference	882
7.151.1 Detailed Description	882
7.151.2 Member Data Documentation	882
7.152 OSAPI_TimerProperty Struct Reference	883
7.152.1 Detailed Description	883
7.152.2 Member Data Documentation	883
7.153 PSK_ErrListener Struct Reference	883
7.153.1 Detailed Description	884
7.153.2 Member Data Documentation	884
7.154 PSK_FilePassTrackerErrListener Struct Reference	884
7.154.1 Detailed Description	884
7.154.2 Member Data Documentation	885
7.155 PSK_PassTrackerConfiguration Struct Reference	885
7.155.1 Detailed Description	885
7.155.2 Member Data Documentation	885
7.156 RHSMHistoryFactory Class Reference	886
7.156.1 Detailed Description	886
7.156.2 Member Function Documentation	886
7.157 RTPS_Checksum Union Reference	887
7.157.1 Detailed Description	887
7.158 RTPS_ChecksumClass Struct Reference	887
7.158.1 Detailed Description	887
7.159 RTPS_ChecksumProperty Struct Reference	887
7.159.1 Detailed Description	888
7.160 RTPS_InterfaceFactoryProperty Struct Reference	888
7.160.1 Detailed Description	888
7.161 RTRegistry Class Reference	888
7.161.1 Detailed Description	889

7.161.2 Member Function Documentation	889
7.162 SHMEMInterfaceFactory Class Reference	890
7.162.1 Detailed Description	890
7.162.2 Member Function Documentation	890
7.163 UDP_InterfaceFactoryProperty Struct Reference	891
7.163.1 Detailed Description	892
7.163.2 Member Data Documentation	892
7.164 UDP_InterfaceTableEntry Struct Reference	898
7.164.1 Detailed Description	898
7.164.2 Member Data Documentation	898
7.165 UDP_InterfaceTableEntrySeq Struct Reference	899
7.165.1 Detailed Description	899
7.166 UDP_NatEntry Struct Reference	899
7.166.1 Detailed Description	899
7.167 UDP_NatEntrySeq Struct Reference	899
7.167.1 Detailed Description	899
7.168 UDPInterfaceFactory Class Reference	900
7.168.1 Detailed Description	900
7.168.2 Member Function Documentation	900
7.169 UDPInterfaceTable Class Reference	900
7.169.1 Detailed Description	901
7.170 WHSMHistoryFactory Class Reference	901
7.170.1 Detailed Description	901
7.170.2 Member Function Documentation	901
7.171 ZCOPY_NotifInterfaceFactoryProperty Struct Reference	901
7.171.1 Detailed Description	902
7.171.2 Member Data Documentation	902
7.172 ZCOPY_NotifMechanismProperty Struct Reference	902
7.172.1 Detailed Description	903
7.172.2 Member Data Documentation	903
7.173 ZCOPY_NotifUserInterfaceI Struct Reference	904
7.173.1 Detailed Description	905
7.173.2 Member Data Documentation	905
8 File Documentation	910
8.1 app_gen_log.h File Reference	910
8.2 cdr_log.h File Reference	911
8.3 db_log.h File Reference	912
8.3.1 Detailed Description	913

8.4 dds_c_config.h File Reference	913
8.4.1 Detailed Description	913
8.5 dds_c_flowcontroller.h File Reference	913
8.5.1 Detailed Description	914
8.6 dds_c_log.h File Reference	914
8.6.1 Detailed Description	941
8.7 dds_filter_log.h File Reference	941
8.7.1 Detailed Description	943
8.7.2 Macro Definition Documentation	944
8.8 dds_psk_log.h File Reference	944
8.8.1 Detailed Description	945
8.8.2 Macro Definition Documentation	945
8.9 disc_dpde_log.h File Reference	945
8.10 disc_dpse_log.h File Reference	948
8.10.1 Macro Definition Documentation	952
8.11 netio_log.h File Reference	954
8.11.1 Detailed Description	959
8.11.2 Macro Definition Documentation	960
8.12 netio_shmem_log.h File Reference	961
8.12.1 Detailed Description	963
8.13 netio_zcopy_log.h File Reference	964
8.13.1 Detailed Description	966
8.14 netio_zcopy_notif_interface.h File Reference	966
8.14.1 Function Documentation	967
8.15 netio_zcopy_whsq_log.h File Reference	968
8.15.1 Detailed Description	968
8.15.2 Macro Definition Documentation	968
8.16 osapi_log.h File Reference	969
8.16.1 Detailed Description	976
8.16.2 Macro Definition Documentation	976
8.16.3 Function Documentation	979
8.17 osapi_process.h File Reference	983
8.17.1 Detailed Description	983
8.18 osapi_semaphore.h File Reference	983
8.18.1 Detailed Description	984
8.18.2 Function Documentation	984
8.19 osapi_stdio.h File Reference	986
8.19.1 Detailed Description	986
8.20 osapi_task.h File Reference	986

8.20.1 Detailed Description	986
8.21 osapi_time.h File Reference	986
8.21.1 Detailed Description	987
8.22 reda_log.h File Reference	987
8.22.1 Detailed Description	988
8.23 rh_sm_log.h File Reference	988
8.23.1 Detailed Description	990
8.23.2 Macro Definition Documentation	990
8.24 rt_log.h File Reference	992
8.25 rtps_config.h File Reference	993
8.25.1 Detailed Description	993
8.26 rtps_dll.h File Reference	993
8.26.1 Detailed Description	993
8.27 rtps_log.h File Reference	994
8.27.1 Detailed Description	1001
8.27.2 Macro Definition Documentation	1001
8.28 rtps_rtps_impl.h File Reference	1001
8.28.1 Detailed Description	1001
8.29 wh_sm_log.h File Reference	1001
8.29.1 Detailed Description	1002
8.29.2 Macro Definition Documentation	1002
Index	1005

1 RTI Connex Micro C++ API

Real-Time Innovations, Inc.

RTI Connex DDS Micro provides a small-footprint, modular messaging solution for resource-limited devices that have limited memory and CPU power, and may not even be running an operating system. It provides the communications services that developers need to distribute time-critical data. Additionally, RTI Connex DDS Micro is designed as a certifiable component in high-assurance systems.

RTI Connex DDS Micro benefits include:

- Accommodations for resource-constrained environments.
- Modular and user extensible architecture.
- Designed to be a certifiable component for safety-critical systems.
- Seamless interoperability with Connex DDS and Connex Messaging.

2 Topic Index

2.1 Reference Manual

Here is a list of all topics with brief descriptions:

Documentation Guide	25
DDS API	27
Content Filter Plugin	31
Domain Module	31
DomainParticipantFactory	32
DomainParticipant	33
Discovery	35
Participant Built-in Topic	40
Publication Built-in Topic	41
Subscription Built-in Topic	42
Topic	42
DDS-Specific Primitive Types	43
Topic	49
FlatData Topic-Types	49
Zero Copy Transfer Over Shared Memory	50
User Data Type Support	50
Publication Module	54
Flow Controllers	54
Publisher	59
DataWriter	61
Subscription Module	63
Subscriber	65
DataReader	65
Data Sample	68
Sample States	69
View States	70

Instance States	72
Infrastructure Module	74
Time Support	76
GUID Support	80
Sequence Number Support	81
Return Codes	82
Status Kinds	83
QoS Policies	92
FILTER	97
CONTENT_FILTER	99
DISCOVERY	99
USER_TRAFFIC	99
TRUST	100
DEADLINE	100
LATENCY_BUDGET	100
OWNERSHIP	101
OWNERSHIP_STRENGTH	102
LIVELINESS	102
RELIABILITY	103
HISTORY	104
DURABILITY	106
DATA_REPRESENTATION	106
RESOURCE_LIMITS	108
PRESENTATION	109
DESTINATION_ORDER	110
ENTITY_FACTORY	112
Extended Qos Support	112
DataReader Resource Limits	113
DataWriter Resource Limits	114
SYSTEM_RESOURCE_LIMITS	114

WIRE_PROTOCOL	115
DATA_READER_PROTOCOL	119
DATA_WRITER_PROTOCOL	119
TRANSPORT	119
DOMAIN_PARTICIPANT_RESOURCE_LIMITS	120
ENTITY_NAME	120
PUBLISH_MODE	121
PARTITION	123
PROPERTY	123
TRANSPORT_PRIORITY	124
USER_DATA	124
GROUP_DATA	125
TOPIC_DATA	125
DATA_WRITER_TRANSFER_MODE	126
Conditions and WaitSets	126
Entity Support	127
Sequence Support	128
Exception codes	128
Type Support	129
String Support	130
General Utilities and Compliance Configuration	134
Compliance Configuration	134
Property Reference	134
DPSE Static Discovery API	135
DPDE Dynamic Discovery API	136
CDR Stream API	136
RT Run-Time API	140
RHSM Reader History API	141
WHSM Writer History API	141
NETIO API	141

NETIO Packet API	142
NETIO Interface API	142
NETIO Address API	144
UDP Transport API	146
UDP Transform API	149
RTPS Transport API	149
Shared Memory Transport API	153
NETIO Shared Memory API	154
Shared Memory Mutex API	155
Shared Memory Segment API	161
Shared Memory Signaling Semaphore	167
NETIO DGRAM API	173
Zero Copy v2 API	179
Notification Interface	183
OS API	184
OSAPI Heap API	185
Trace API	190
OSAPI Mutex API	190
OSAPI Process API	192
OSAPI Semaphore API	193
OSAPI Memory API	195
OSAPI String API	195
OSAPI System API	195
OSAPI Thread API	200
OSAPI time API	204
OSAPI Timer API	204
OSAPI Types	205
OSAPI_SharedMemoryMonitor	208
OSAPI_SharedMemorySegment	211
Log API	218

Application Generation API	221
Lightweight Security API	223
Resource Limits	224
DomainParticipantFactoryQos	225
max_components	225
max_participants	225
DomainParticipantQos	225
local_topic_allocation	226
local_type_allocation	226
local_publisher_allocation	226
local_subscriber_allocation	227
local_reader_allocation	227
local_writer_allocation	227
matching_writer_reader_pair_allocation	227
remote_participant_allocation	228
remote_writer_allocation	229
remote_reader_allocation	229
max_destination_ports	229
max_receive_ports	230
DataReaderQos	231
max_instances	232
max_remote_writers_per_instance	232
max_samples	233
max_samples_per_instance	233
max_remote_writers	234
max_samples_per_remote_writer	234
max_routes_per_writer	234
max_outstanding_reads	235
max_fragmented_samples	235
max_fragmented_samples_per_remote_writer	235

DataWriterQos	236
max_samples	236
max_samples_per_instance	237
max_instances	237
max_routes_per_reader	237
max_remote_readers	238
OSAPI	238
max_buffer_size	238
UDP Transport	238
max_message_size	238
Dynamic Participant Static Endpoint (DPSE)	239
max_participant_locators	239
max_locators_per_discovered_participant	240
Dynamic Participant Dynamic Endpoint (DPDE)	240
max_participant_locators	240
max_locators_per_discovered_participant	241
max_samples_per_builtin_endpoint_reader	241
Log Codes	242
OSAPI	242
REDA	266
DB	272
RT	278
CDR	280
NETIO	281
RTPS	309
DDS_C	340
ZeroCopy	452
WHSQ	462
RH	464
WH	469

DPSE	471
DPDE	484
APPGEN	496
DDS_FILTER	499

3 Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

CDR_Stream_t	510
DDS_BuiltinTopicKey_t	510
DDS_ChecksumProperty_t	511
DDS_ContentFilterProperty	512
DDS_ContentFilterQosPolicy	514
DDS_DataReaderInstanceReplacedStatus	516
DDS_DataReaderProtocolQosPolicy	518
DDS_DataReaderQos	519
DDS_DataReaderResourceLimitsQosPolicy	523
DDS_DataRepresentationIdSeq	527
DDS_DataRepresentationQosPolicy	527
DDS_DataWriterProtocolQosPolicy	528
DDS_DataWriterQos	530
DDS_DataWriterResourceLimitsQosPolicy	535
DDS_DataWriterShmemRefTransferModeSettings	538
DDS_DataWriterTransferModeQosPolicy	538
DDS_DeadlineQosPolicy	539
DDS_DestinationOrderQosPolicy	541
DDS_DiscoveryComponent	542
DDS_DiscoveryQosPolicy	543
DDS_DomainParticipantFactoryQos	546

DDS_DomainParticipantQos	547
DDS_DomainParticipantResourceLimitsQosPolicy	550
DDS_DurabilityQosPolicy	561
DDS_Duration_t	562
DDS_EntityFactoryQosPolicy	563
DDS_EntityNameQosPolicy	564
DDS_FilterQosPolicy	565
DDS_FilterResourceLimits	567
DDS_FlowControllerProperty_t	570
DDS_FlowControllerTokenBucketProperty_t	571
DDS_GroupDataQosPolicy	574
DDS_GUID_t	575
DDS_HistoryQosPolicy	576
DDS_InconsistentTopicStatus	577
DDS_InstanceHandleSeq	578
DDS_LatencyBudgetQosPolicy	578
DDS_LivelinessChangedStatus	580
DDS_LivelinessLostStatus	581
DDS_LivelinessQosPolicy	582
DDS_Locator_t	585
DDS_LocatorSeq	586
DDS_OfferedDeadlineMissedStatus	586
DDS_OfferedIncompatibleQosStatus	588
DDS_OwnershipQosPolicy	589
DDS_OwnershipStrengthQosPolicy	593
DDS_ParticipantBuiltinTopicData	594
DDS_PartitionQosPolicy	597
DDS_PresentationQosPolicy	599
DDS_ProductVersion_t	603
DDS_Property_t	604

DDS_PropertyQosPolicy	605
DDS_PropertySeq	606
DDS_ProtocolVersion_t	607
DDS_PskPassTracker_InterfaceI	607
DDS_PskServiceFactoryProperty	608
DDS_PublicationBuiltinTopicData	611
DDS_PublicationMatchedStatus	615
DDS_PublisherQos	616
DDS_PublishModeQosPolicy	618
DDS_QosPolicyCount	620
DDS_ReliabilityQosPolicy	621
DDS_ReliableReaderActivityChangedStatus	623
DDS_ReliableSampleUnacknowledgedStatus	624
DDS_RequestedDeadlineMissedStatus	625
DDS_RequestedIncompatibleQosStatus	626
DDS_ResourceLimitsQosPolicy	627
DDS_RtpsReliableReaderProtocol_t	631
DDS_RtpsReliableWriterProtocol_t	632
DDS_RtpsWellKnownPorts_t	634
DDS_SampleIdentity_t	639
DDS_SampleInfo	639
DDS_SampleInfoSeq	643
DDS_SampleLostStatus	643
DDS_SampleRejectedStatus	644
DDS_SecTransform_Configuration	645
DDS_SequenceNumber_t	646
DDS_SubscriberQos	647
DDS_SubscriptionBuiltinTopicData	648
DDS_SubscriptionMatchedStatus	653
DDS_SystemResourceLimitsQosPolicy	655

DDS_Time_t	656
DDS_TopicDataQosPolicy	657
DDS_TopicQos	658
DDS_TransportPriorityQosPolicy	659
DDS_TransportQosPolicy	659
DDS_TrustQosPolicy	660
DDS_UserDataQosPolicy	661
DDS_UserTrafficQosPolicy	663
DDS_VendorId_t	664
DDS_WireProtocolQosPolicy	664
DDS_WriteParams_t	668
DDSCondition	669
DDSGuardCondition	763
DDSStatusCondition	779
DDSConditionSeq	670
DDSDomainParticipantFactory	739
DDSDomainParticipantListener	748
DDSEntity	756
DDSDataReader	670
DDSDataWriter	692
DDSDomainEntity	714
DDSDomainParticipant	715
DDSPublisher	770
DDSSubscriber	781
DDSTopic	790
DDSFlowController	761
DDSListener	764
DDSDataReaderListener	688
DDSSubscriberListener	789
DDSDataWriterListener	710

DDSPublisherListener	777
DDSTopicListener	799
DDSPROPERTYQOSPolicyHelper	767
DDSTopicDescription	797
DDSTopic	790
DDSWaitSet	800
DPDE_DiscoveryPluginProperty	804
DPDEDiscoveryFactory	809
DPSE_DiscoveryPluginProperty	809
DPSEDiscoveryFactory	812
DPSEDiscoveryPlugin	813
FooSeq	841
FooTypeSupport	854
NDDS_Discovery_Property	860
NDDS_Type_PluginVersion	860
NETIO_Address	861
NETIO_AddressSeq	862
NETIO_AddressUInt32	862
NETIO_AddressValue	862
NETIO_DGRAM_InterfaceI	863
NETIO_DGRAM_InterfaceMultiCastGroup	867
NETIO_DGRAM_InterfaceRouteEntry	868
NETIO_DGRAM_InterfaceTableEntry	869
NETIO_DGRAM_InterfaceTableEntrySeq	870
NETIO_Interface	871
NETIO_InterfaceI	871
NETIO_Netmask	871
NETIO_NetmaskSeq	872
NETIO_Packet	872
NETIO_SharedMemorySegmentHeader	873

NETIO_SHMEMInterfaceFactoryProperty	873
OSAPI_LogProperty	874
OSAPI_SharedMemorySegmentHandle	875
OSAPI_SharedMemorySegmentHeader	876
OSAPI_SystemI	877
OSAPI_SystemProperty	878
OSAPI_SystemTime	880
OSAPI_TaskProperty	881
OSAPI_ThreadProperty	882
OSAPI_TimerProperty	883
PSK_ErrListener	883
PSK_FilePassTrackerErrListener	884
PSK_PassTrackerConfiguration	885
RHSMHistoryFactory	886
RTPS_Checksum	887
RTPS_ChecksumClass	887
RTPS_ChecksumProperty	887
RTPS_InterfaceFactoryProperty	888
RTRegistry	888
SHMEMInterfaceFactory	890
UDP_InterfaceFactoryProperty	891
UDP_InterfaceTableEntry	898
UDP_InterfaceTableEntrySeq	899
UDP_NatEntry	899
UDP_NatEntrySeq	899
UDPInterfaceFactory	900
UDPInterfaceTable	900
WHSMHistoryFactory	901
ZCOPY_NotifInterfaceFactoryProperty	901
ZCOPY_NotifMechanismProperty	902

ZCOPY_NotifUserInterfaceI	904
---------------------------	-----

4 Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

CDR_Stream_t	Byte stream abstraction for CDR serialization and deserialization	510
DDS_BuiltinTopicKey_t	The key type of the built-in topic types	510
DDS_ChecksumProperty_t	<< <i>eXtension</i> >> Type to define the checksum properties of the participant RTI Connexx DDS Micro	511
DDS_ContentFilterProperty	All of the information necessary for a DDSDDataWriter to perform writer-side filtering for a DDSDDataReader	512
DDS_ContentFilterQosPolicy	Filter that allows a DDSDDataReader to specify that it is interested only in (potentially) a subset of the samples on its topic	514
DDS_DataReaderInstanceReplacedStatus	<< <i>eXtension</i> >> DDS_INSTANCE_REPLACED_STATUS	516
DDS_DataReaderProtocolQosPolicy	<< <i>eXtension</i> >> Protocol that applies only to DDSDDataReader instances	518
DDS_DataReaderQos	QoS policies supported by a DDSDDataReader entity	519
DDS_DataReaderResourceLimitsQosPolicy	Resource limits that apply only to DDSDDataReader instances	523
DDS_DataRepresentationIdSeq	Declares IDL sequence < DDS_DataRepresentationId_t >	527
DDS_DataRepresentationQosPolicy	This QoS policy contains a list of representation identifiers used by DDSDDataWriter and DDSDDataReader entities to negotiate which data representation to use	527
DDS_DataWriterProtocolQosPolicy	<< <i>eXtension</i> >> Protocol that applies only to DDSDDataWriter instances	528
DDS_DataWriterQos	QoS policies supported by a DDSDDataWriter entity	530
DDS_DataWriterResourceLimitsQosPolicy	Resource limits that apply only to DDSDDataWriter instances	535

DDS_DataWriterShmemRefTransferModeSettings	
Settings related to transferring data using shared memory references	538
DDS_DataWriterTransferModeQosPolicy	
<< <i>eXtension</i> >> QoS related to transferring data	538
DDS_DeadlineQosPolicy	
Expresses the maximum duration or deadline within which an instance is expected to be updated	539
DDS_DestinationOrderQosPolicy	
Destination order policies that affect the ordering of subscribed data	541
DDS_DiscoveryComponent	
Specifies the discovery plugin the DDSDomainParticipant should use	542
DDS_DiscoveryQosPolicy	
Specifies the attributes required to discover participants in the domain	543
DDS_DomainParticipantFactoryQos	
QoS policies supported by a DDSDomainParticipantFactory	546
DDS_DomainParticipantQos	
QoS policies supported by a DDSDomainParticipant entity	547
DDS_DomainParticipantResourceLimitsQosPolicy	
Resource limits that apply only to DDSDomainParticipant instances	550
DDS_DurabilityQosPolicy	
Durability properties that affect the life-cycle of published data	561
DDS_Duration_t	
Type for <i>duration</i> representation	562
DDS_EntityFactoryQosPolicy	
A QoS policy for all DDSEntity types that can act as factories for one or more other DDSEntity types	563
DDS_EntityNameQosPolicy	
Policy used to name an entity	564
DDS_FilterQosPolicy	
Configures the content filter capabilities and resource limits for a DDSDomainParticipant and all of its child entities	565
DDS_FilterResourceLimits	
Resource limits for content filtering on a DDSDomainParticipant and all of its child entities	567
DDS_FlowControllerProperty_t	
Determines the flow control characteristics of the DDSFlowController	570
DDS_FlowControllerTokenBucketProperty_t	
DDSFlowController uses the popular token bucket approach for open loop network flow control. The flow control characteristics are determined by the token bucket properties	571
DDS_GroupDataQosPolicy	
Attaches a buffer of opaque data to a DDSEntity which is distributed to other applications as part of the discovery process	574

DDS_GUID_t	
Type for <i>GUID</i> (Global Unique Identifier) representation	575
DDS_HistoryQosPolicy	
Specifies the behavior of RTI Connexx DDS Micro when a sample value changes one or more times before it can be successfully communicated to existing subscribers	576
DDS_InconsistentTopicStatus	
DDS_INCONSISTENT_TOPIC_STATUS	577
DDS_InstanceHandleSeq	
Instantiates <i>FooSeq</i> < DDS_InstanceHandle_t >	578
DDS_LatencyBudgetQosPolicy	
Provides a hint as to the maximum acceptable delay from the time the data is written to the time it is received by the subscribing applications	578
DDS_LivelinessChangedStatus	
DDS_LIVELINESS_CHANGED_STATUS	580
DDS_LivelinessLostStatus	
DDS_LIVELINESS_LOST_STATUS	581
DDS_LivelinessQosPolicy	
Determines the mechanism and parameters used by the application to determine whether a <i>DDSEntity</i> is alive	582
DDS_Locator_t	
<< <i>eXtension</i> >> Type used to represent the addressing information needed to send a message to an RTPS Endpoint using one of the supported transports	585
DDS_LocatorSeq	
Declares IDL sequence < DDS_Locator_t >	586
DDS_OfferedDeadlineMissedStatus	
DDS_OFFERED_DEADLINE_MISSED_STATUS	586
DDS_OfferedIncompatibleQosStatus	
DDS_OFFERED_INCOMPATIBLE_QOS_STATUS	588
DDS_OwnershipQosPolicy	
Specifies whether it is allowed for multiple <i>::DDSDDataWriter(s)</i> to write the same instance of the data and if so, how these modifications should be arbitrated	589
DDS_OwnershipStrengthQosPolicy	
Specifies the value of the strength used to arbitrate among multiple <i>DDSDDataWriter</i> objects that attempt to modify the same instance of a data type (identified by <i>DDSTopic</i> + key)	593
DDS_ParticipantBuiltinTopicData	
Object representing a remote DomainParticipant	594
DDS_PartitionQosPolicy	
A set of strings that introduces logical PARTITIONS	597
DDS_PresentationQosPolicy	
Specifies how the samples representing changes to data instances are presented to a subscribing application	599

DDS_ProductVersion_t	
<< <i>eXtension</i> >> Type used to represent the current version of RTI Connex DDS Micro	603
DDS_Property_t	
Properties are name/value pair objects	604
DDS_PropertyQosPolicy	
Stores name/value(string) pairs that can be used to configure certain parameters of RTI Connex DDS Micro that are not exposed through formal QoS policies	605
DDS_PropertySeq	
Declares IDL sequence < DDS_Property_t >	606
DDS_ProtocolVersion_t	
<< <i>eXtension</i> >> Type used to represent the version of the RTPS protocol	607
DDS_PskPassTracker_Interfacel	
PSK passphrase tracker user interface	607
DDS_PskServiceFactoryProperty	
Property used to register the Lightweight Security Plugin	608
DDS_PublicationBuiltinTopicData	
Object describing a remote Publication	611
DDS_PublicationMatchedStatus	
DDS_PUBLICATION_MATCHED_STATUS	615
DDS_PublisherQos	
QoS policies supported by a DDSPublisher entity	616
DDS_PublishModeQosPolicy	
Specifies how RTI Connex DDS Micro sends application data on the network. This QoS policy can be used to tell RTI Connex DDS Micro to use its <i>own</i> thread to send data, instead of the user thread	618
DDS_QosPolicyCount	
Type to hold a counter for a DDS_QosPolicyId_t	620
DDS_ReliabilityQosPolicy	
Indicates the level of reliability offered or requested by RTI Connex DDS Micro	621
DDS_ReliableReaderActivityChangedStatus	
<< <i>eXtension</i> >> Describes the activity (i.e. are acknowledgements forthcoming) of reliable readers matched to a reliable writer	623
DDS_ReliableSampleUnacknowledgedStatus	
Status about an unacknowledged sample that is removed from the DDSDataWriter cache before being acknowledged by all matched DDSDataReader entities	624
DDS_RequestedDeadlineMissedStatus	
DDS_REQUESTED_DEADLINE_MISSED_STATUS	625
DDS_RequestedIncompatibleQosStatus	
DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS	626

DDS_ResourceLimitsQosPolicy	
Specifies the resources that RTI Connex DDS Micro can consume in order to meet the requested QoS	627
DDS_RtpsReliableReaderProtocol_t	
QoS related to reliable reader protocol defined in RTPS	631
DDS_RtpsReliableWriterProtocol_t	
QoS related to the reliable writer protocol defined in RTPS	632
DDS_RtpsWellKnownPorts_t	
RTPS well-known port mapping configuration	634
DDS_SampleIdentity_t	
Data structure used to provide explicit identity information about a published data sample	639
DDS_SampleInfo	
Information that accompanies each sample that is read or taken	639
DDS_SampleInfoSeq	
Declares IDL sequence < DDS_SampleInfo >	643
DDS_SampleLostStatus	
DDS_SAMPLE_LOST_STATUS	643
DDS_SampleRejectedStatus	
DDS_SAMPLE_REJECTED_STATUS	644
DDS_SecTransform_Configuration	
A structure for configuration this Transform PSL	645
DDS_SequenceNumber_t	
Type for <i>sequence</i> number representation	646
DDS_SubscriberQos	
QoS policies supported by a DDSSubscriber entity	647
DDS_SubscriptionBuiltinTopicData	
Entry created when a DDSDataReader is discovered in association with its Subscriber	648
DDS_SubscriptionMatchedStatus	
DDS_SUBSCRIPTION_MATCHED_STATUS	653
DDS_SystemResourceLimitsQosPolicy	
Resource limits that apply only to DDSDomainParticipantFactory	655
DDS_Time_t	
Type for <i>time</i> representation	656
DDS_TopicDataQosPolicy	
Attaches a buffer of opaque data to a DDSTopic which is distributed to other applications as part of the discovery process	657
DDS_TopicQos	
QoS policies supported by a DDSTopic entity	658

DDS_TransportPriorityQosPolicy	
This QoS policy allows the application to take advantage of transports that are capable of sending messages with different priorities	659
DDS_TransportQosPolicy	
<<eXtension>> Settings for available transports	659
DDS_TrustQosPolicy	
<<eXtension>> Configures the security plugin suite used by a DDSDomainParticipant	660
DDS_UserDataQosPolicy	
Attaches a buffer of opaque data to a DDSEntity which is distributed to other applications as part of the discovery process	661
DDS_UserTrafficQosPolicy	
<<eXtension>> Configures the default transports enabled to transmit and receive user data traffic	663
DDS_VendorId_t	
<<eXtension>> Type used to represent the vendor of the service implementing the RTPS protocol	664
DDS_WireProtocolQosPolicy	
Specifies the wire-protocol-related attributes for the DDSDomainParticipant	664
DDS_WriteParams_t	
Data structure used to provide custom parameters to a DDSDataWriter 's write operation	668
DDSCondition	
<<interface>> Root class for all the conditions that may be attached to a DDSWaitSet	669
DDSConditionSeq	
A sequence object containing references to one or more DDSCondition objects	670
DDSDataReader	
<<interface>> Allows the application to: (1) declare the data it wishes to receive (i.e. make a subscription) and (2) access the data received by the attached DDSSubscriber	670
DDSDataReaderListener	
<<interface>> DDSListener for reader status	688
DDSDataWriter	
<<interface>> Allows an application to set the value of the data to be published under a given DDSTopic	692
DDSDataWriterListener	
<<interface>> DDSListener for writer status	710
DDSDomainEntity	
<<interface>> Abstract base class for all DDS entities except for the DDSDomainParticipant	714
DDSDomainParticipant	
<<interface>> Container for all DDSDomainEntity objects	715
DDSDomainParticipantFactory	
<<singleton>> <<interface>> Allows creation and destruction of DDSDomainParticipant objects	739

DDSDomainParticipantListener	
<<interface>> Listener for a DDSDomainParticipant and all of its child entities	748
DDSEntity	
<<interface>> Abstract base class for all the DDS objects that support QoS policies, and a listener	756
DDSFlowController	
<<interface>> A flow controller is the object responsible for shaping the network traffic by determining when attached asynchronous DDSDataWriter instances are allowed to write data	761
DDSGuardCondition	
<<interface>> A specific DDSCondition whose <code>trigger_value</code> is completely under the control of the application	763
DDSListener	
<<interface>> Abstract base class for all Listener interfaces	764
DDSPROPERTYQOSPolicyHelper	
Policy helpers that facilitate management of the properties in the input policy	767
DDSPublisher	
<<interface>> A publisher is the object responsible for the actual dissemination of publications	770
DDSPublisherListener	
<<interface>> DDSListener for DDSPublisher status	777
DDSStatusCondition	
<<interface>> A specific DDSCondition that is associated with each DDSEntity	779
DDSSubscriber	
<<interface>> A subscriber is the object responsible for actually receiving data from a subscription	781
DDSSubscriberListener	
<<interface>> DDSListener for status about a subscriber	789
DDSTopic	
<<interface>> The most basic description of the data to be published and subscribed	790
DDSTopicDescription	
<<interface>> Base class for DDSTopic	797
DDSTopicListener	
<<interface>> DDSListener for DDSTopic entities	799
DDSWaitSet	
<<interface>> Allows an application to wait until one or more of the attached DDSCondition objects has a <code>trigger_value</code> of DDS_BOOLEAN_TRUE or else until the timeout expires	800
DPDE_DiscoveryPluginProperty	
<<extension>> Properties for the Dynamic Participant/Dynamic Endpoint (DPDE) discovery plugin. This includes all discovery timing properties for participant discovery	804
DPDEDiscoveryFactory	
<<extension>> Discovery plug-in used for asserting remote entities	809

DPSE_DiscoveryPluginProperty	
<<eXtension>> Properties for the Dynamic Participant/Static Endpoint (DPSE) discovery plugin. These properties can be configured while registering the plugin to control its behavior	809
DPSEDiscoveryFactory	
<<eXtension>> A discovery factory that will be used by the DDSDomainParticipantFactory to create a discovery plugin	812
DPSEDiscoveryPlugin	
<<eXtension>> Discovery plug-in used for asserting remote entities	813
FooDataReader	
Declares the interface required to support a user data type-specific data reader	814
FooDataWriter	
Declares the interface required to support a user data type-specific data writer	826
FooSeq	
<<interface>> <<generic>> A type-safe, ordered collection of elements. The type of these elements is referred to in this documentation as "Foo"	841
FooTypeSupport	
<<interface>> <<generic>> User data type specific interface	854
NDDS_Discovery_Property	
Specifies internal information used to store discovery information about a DDSDomainParticipant in the registry	860
NDDS_Type_PluginVersion	
Data type used to keep track of versioning information about a ::DDS_TypePluginI	860
NETIO_Address	861
NETIO_AddressSeq	
<<eXtension>> The NETIO_AddressSeq sequence type	862
NETIO_AddressUInt32	
<<eXtension>> NETIO_AddressUInt32	862
NETIO_AddressValue	
<<eXtension>> NETIO_AddressValue	862
NETIO_DGRAM_InterfaceI	
<<eXtension>> Definition of the NETIO_DGRAM Interface	863
NETIO_DGRAM_InterfaceMultiCastGroup	
<<eXtension>> Definition of a valid multicast group address range	867
NETIO_DGRAM_InterfaceRouteEntry	
<<eXtension>> Definition of a route entry	868
NETIO_DGRAM_InterfaceTableEntry	
<<eXtension>> Definition of an interface	869
NETIO_DGRAM_InterfaceTableEntrySeq	
A sequence of NETIO_DGRAMInterfaceTableEntry elements	870

NETIO_Interface		
Base-class definition for all NETIO interfaces		871
NETIO_Interfacel		871
NETIO_Netmask		
<<eXtension>> NETIO_Netmask		871
NETIO_NetmaskSeq		
<<eXtension>> The NETIO_NetmaskSeq sequence type		872
NETIO_Packet		
Implementation of the abstract NETIO_Packet type		872
NETIO_SharedMemorySegmentHeader		873
NETIO_SHMEMInterfaceFactoryProperty		
<<eXtension>> Properties for the SHMEM Transport		873
OSAPI_LogProperty		
Configuration of logging functionality		874
OSAPI_SharedMemorySegmentHandle		875
OSAPI_SharedMemorySegmentHeader		876
OSAPI_SystemI		
System API		877
OSAPI_SystemProperty		
The System Properties		878
OSAPI_SystemTime		
System Time		880
OSAPI_TaskProperty		
Task properties		881
OSAPI_ThreadProperty		
Properties for thread creation		882
OSAPI_TimerProperty		
Timer properties		883
PSK_ErrListener		
A structure for holding callback pointers in case of an error		883
PSK_FilePassTrackerErrListener		
A structure for holding callback pointers in case of an error		884
PSK_PassTrackerConfiguration		
Configuration structure for file-based PSK pass tracker		885
RHSMHistoryFactory		
<<eXtension>> A reader history factory that will be used by the DataReader to create a history cache		886

RTPS_Checksum	
<< <i>eXtension</i> >> RTPS checksum type	887
RTPS_ChecksumClass	
<< <i>eXtension</i> >> Definition of a checksum function	887
RTPS_ChecksumProperty	
<< <i>eXtension</i> >> Checksum properties for the RTPS plugin	887
RTPS_InterfaceFactoryProperty	
<< <i>eXtension</i> >> Properties for the RTPS Transport	888
RTRRegistry	
<< <i>eXtension</i> >> A run-time component factory	888
SHMEMInterfaceFactory	
<< <i>eXtension</i> >> A SHMEM transport factory that will be used by a DomainParticipant to create a SHMEM interface. Only one SHMEM interface can be registered with the DomainParticipantFactory	890
UDP_InterfaceFactoryProperty	
<< <i>eXtension</i> >> Properties for the UDP Transport	891
UDP_InterfaceTableEntry	
Generic structure to describe a network interface	898
UDP_InterfaceTableEntrySeq	
<< <i>eXtension</i> >> Declares IDL sequence< UDP_InterfaceTableEntry >	899
UDP_NatEntry	
<< <i>eXtension</i> >> Properties for a Network Address Translation entry	899
UDP_NatEntrySeq	
<< <i>eXtension</i> >> Declares IDL sequence< UDP_NatEntry >	899
UDPInterfaceFactory	
<< <i>eXtension</i> >> A UDP transport factory that will be used by a DomainParticipant to create a UDP interface. Only one UDP interface can be registered with the DomainParticipantFactory	900
UDPInterfaceTable	900
WHSMHistoryFactory	
<< <i>eXtension</i> >> A writer history factory that will be used by the DataWriter to create a history cache	901
ZCOPY_NotifInterfaceFactoryProperty	
Notification interface property	901
ZCOPY_NotifMechanismProperty	
Notification mechanism property for the default implementation	902
ZCOPY_NotifUserInterfaceI	
Notification mechanism user interface	904

5 File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

app_gen_log.h	910
cdr_log.h	911
db_log.h DB module log codes	912
dds_c_config.h DDS C configuration file	913
dds_c_flowcontroller.h DDS Flow Controller	913
dds_c_log.h DDS_C module log codes	914
dds_filter_log.h DDS Filter module log codes	941
dds_psk_log.h DDS_PSK module log codes	944
disc_dpde_log.h	945
disc_dpse_log.h	948
netio_log.h NETIO module log codes	954
netio_shmem_log.h NETIO Zero Copy Shared Data module log codes	961
netio_zcopy_log.h ZeroCopy module log codes	964
netio_zcopy_notif_interface.h	966
netio_zcopy_whsq_log.h WHSQ module log codes	968
osapi_log.h OSAPI Log API	969
osapi_process.h Process related utility	983
osapi_semaphore.h Semaphore interface definition	983

osapi_stdio.h	Standard I/O interface definition	986
osapi_task.h	Task interface definition	986
osapi_time.h	Time API definition	986
reda_log.h	REDA module log codes	987
rh_sm_log.h	RH module log codes	988
rt_log.h		992
rtps_config.h	RTPS Configuration	993
rtps_dll.h	RTPS Configuration	993
rtps_log.h	RTPS module log codes	994
rtps_rtps_impl.h	Implementation of RTPS interface functions and types	1001
wh_sm_log.h	WH module log codes	1001

6 Topic Documentation

6.1 Documentation Guide

This section describes the conventions used in the API documentation.

6.1.1 Unsupported Features

[Not supported.] This note means that the feature from the DDS specification is not supported in the current release.

6.1.2 API Naming Conventions

6.1.2.1 Structure & Class Names

RTI Connext DDS Micro makes a distinction between *value types* and *interface types*. Value types are types such as primitives, enumerations, strings, and structures whose identity and equality are determined solely by explicit state.

Interface types are those abstract opaque data types that conceptually have an identity apart from their explicit state. Examples include all of the [DDSEntity](#) subtypes. Instances of value types are frequently transitory and are declared on the stack. Instances of interface types typically have longer lifecycles, are accessible by pointer only, and may be managed by a factory object.

Value type structures are made more explicit through the use of C's structure tag syntax. For example, a [DDS_Duration_t](#) object must be declared as being of type "struct DDS_Duration_t," not simply "DDS_Duration_t." Interface types, by contrast, are always of typedef'ed types; their underlying representations are opaque.

6.1.3 API Documentation Terms

In the API documentation, the term module refers to a logical grouping of documentation and elements in the API.

At this time, typedefs that occur in the API, such as [DDS_ReturnCode_t](#) do not show up in the compound list or indices. This is a known limitation in the generated HTML.

6.1.4 Stereotypes

Commonly used stereotypes in the API documentation include the following.

6.1.4.1 Extensions

- [`<<eXtension>>`](#)
 - An RTI Connex Micro DDS product extension to the DDS standard specification.
 - The extension APIs complement the standard APIs specified by the OMG DDS specification. They are provided to improve product usability and enable access to product-specific features.

6.1.4.2 Experimental

- [`<<experimental>>`](#)
 - This software may contain experimental features. These are used to evaluate potential new features and obtain customer feedback.
 - Experimental APIs are marked as [`<<experimental>>`](#) in this documentation.
 - Experimental features may be only available in a subset of the supported languages and for a subset of the supported platforms.
 - Experimental features may change in the future.
 - Experimental features may or may not appear in future product releases.
 - Experimental features should not be used in production.
 - Please submit your comments and suggestions about experimental features to support@rti.com or via the RTI Customer Portal (<https://support.rti.com/>).

6.1.4.3 Types

- `<<interface>>`
 - Pure interface type with *no state*.
- `<<generic>>`
 - A *generic* type is a *skeleton* class written in terms of generic parameters. Type-specific instantiations of such types are conventionally referred to in this documentation in terms of the hypothetical type "Foo"; for example: `FooSeq`, `FooDataType`, `FooDataWriter`, and `FooDataReader`.
- `<<singleton>>`
 - Singleton class. There is a single instance of the class.
 - Generally accessed via a `get_instance()` static method.

6.1.4.4 Method Parameters

- `<<in>>`
 - An *input* parameter.
- `<<out>>`
 - An *output* parameter.
- `<<inout>>`
 - An *input* and *output* parameter.

6.1.4.5 Cert API

- – Supported in safety certified version of RTI Connex DDS Micro. Note, specific fields of data structures may not be supported and will be described in documentation as unsupported.

6.2 DDS API

RTI Connex DDS Micro modules following the DDS module definitions.

Topics

- [Content Filter Plugin](#)
<<experimental>> Content Filter Plugin
- [Domain Module](#)
Contains the [DDSDomainParticipant](#) class that acts as an endpoint of RTI Connext DDS Micro and acts as a factory for many of the classes. The [DDSDomainParticipant](#) also acts as a container for the other objects that make up RTI Connext DDS Micro.
- [Topic](#)
Contains the [DDSTopic](#), the [DDSTopicListener](#) interface, and more generally, all that is needed by an application to define [DDSTopic](#) objects and the data types that are associated with [DDSTopic](#) objects.
- [Publication Module](#)
Contains the [DDSPublisher](#), and [DDSDataWriter](#) classes, and more generally, all that is needed on the publication side.
- [Subscription Module](#)
Contains the [DDSSubscriber](#) and [DDSDataReader](#) classes, as well as the [DDSSubscriberListener](#) and [DDSDataReaderListener](#) interfaces, and more generally, all that is needed on the subscription side.
- [Infrastructure Module](#)
Defines the abstract classes and the interfaces that are refined by the other modules. Contains common definitions such as return codes, status values, and QoS policies.
- [General Utilities and Compliance Configuration](#)

6.2.1 Detailed Description

RTI Connext DDS Micro modules following the DDS module definitions.

6.2.2 Overview

Information flows with the aid of the following constructs: [DDSPublisher](#) and [DDSDataWriter](#) on the sending side, [DDSSubscriber](#) and [DDSDataReader](#) on the receiving side.

- A [DDSPublisher](#) is an object responsible for data distribution. It may publish data of different data types. A [DDSDataWriter](#) acts as an accessor to a publisher - each [DDSDataWriter](#) object is dedicated to one application data type. A [DDSDataWriter](#) is the object the application must use to communicate to a publisher the existence and value of data-objects of a given type. When data-object values have been communicated to the publisher through the appropriate data-writer, it is the publisher's responsibility to perform the distribution (the publisher will do this according to its own QoS, or the QoS attached to the corresponding data-writer). A *publication* is defined by the association of a data-writer to a publisher. This association expresses the intent of the application to publish the data described by the data-writer in the context provided by the publisher.
- A [DDSSubscriber](#) is an object responsible for receiving published data and making it available (according to the Subscriber's QoS) to the receiving application. It may receive and dispatch data of different specified types. To access the received data, the application must use a [DDSDataReader](#) attached to the subscriber. Thus, a *subscription* is defined by the association of a data-reader with a subscriber. This association expresses the intent of the application to subscribe to the data described by the data-reader in the context provided by the subscriber.

DDSTopic objects conceptually fit between publications and subscriptions. Publications must be known in such a way that subscriptions can refer to them unambiguously. A **DDSTopic** is meant to fulfill that purpose: it associates a name (unique in the domain i.e. the set of applications that are communicating with each other), a data type, and QoS related to the data itself. In addition to the topic QoS, the QoS of the **DDSDataWriter** associated with that Topic and the QoS of the **DDSPublisher** associated to the **DDSDataWriter** control the behavior on the publisher's side, while the corresponding **DDSTopic**, **DDSDataReader** and **DDSSubscriber** QoS control the behavior on the subscriber's side.

When an application wishes to publish data of a given type, it must create a **DDSPublisher** (or reuse an already created one) and a **DDSDataWriter** with all the characteristics of the desired publication. Similarly, when an application wishes to receive data, it must create a **DDSSubscriber** (or reuse an already created one) and a **DDSDataReader** to define the subscription.

6.2.3 Conceptual Model

The overall conceptual model is shown below.

Notice that all the main communication objects (the specializations of Entity) follow unified patterns of:

- Supporting QoS (made up of several QoSPolicy); QoS provides a generic mechanism for the application to control the behavior of the Service and tailor it to its needs. Each **DDSEntity** supports its own specialized kind of QoS policies (see [QoS Policies](#)).
- Accepting a **DDSListener**; listeners provide a generic mechanism for the middleware to notify the application of relevant asynchronous events, such as arrival of data corresponding to a subscription, violation of a QoS setting, etc. Each **DDSEntity** supports its own specialized kind of listener. Listeners are related to changes in status conditions (see [Status Kinds](#)).

Note that only one Listener per entity is allowed (instead of a list of them). The reason for that choice is that this allows a much simpler (and, thus, more efficient) implementation as far as the middleware is concerned. Moreover, if it were required, the application could easily implement a listener that, when triggered, triggers in return attached 'sub-listeners'.

All DCPS entities are attached to a **DDSDomainParticipant**. A domain participant represents the local membership of the application in a domain. A *domain* is a distributed concept that links all the applications able to communicate with each other. It represents a communication plane: only the publishers and the subscribers attached to the same domain may interact.

DDSDomainEntity is an intermediate object whose only purpose is to state that a DomainParticipant cannot contain other domain participants.

At the DCPS level, data types represent information that is sent atomically. For performance reasons, only plain data structures are handled by this level.

By definition, a **DDSTopic** corresponds to a single data type. However, several topics may refer to the same data type. Therefore, a **DDSTopic** identifies data of a single type, ranging from one single instance to a whole collection of instances of that given type. This is shown below for the hypothetical data type `Foo`.

In case a set of instances is gathered under the same topic, different instances must be distinguishable. This is achieved by means of the values of some data fields that form the **key** to that data set. The *key description* (i.e., the list of data fields whose value forms the key) has to be indicated to the middleware. The rule is simple: *different data samples with the same key value represent successive values for the same instance, while different data samples with different key*

values represent different instances. If no key is provided, the data set associated with the [DDSTopic](#) is restricted to a *single instance*.

Topics need to be known by the middleware and potentially propagated. Topic objects are created using the create operations provided by [DDSDomainParticipant](#).

The interaction style is straightforward on the publisher's side: when the application decides that it wants to make data available for publication, it calls the appropriate operation on the related [DDSDataWriter](#) (this, in turn, will trigger its [DDSPublisher](#)).

On the subscriber's side however, there are more choices: relevant information may arrive when the application is busy doing something else or when the application is just waiting for that information. Therefore, depending on the way the application is designed, asynchronous notifications or synchronous access may be more appropriate. Both interaction modes are allowed, a [DDSListener](#) is used to provide a callback for asynchronous access, and polling allows for synchronous access.

6.2.4 Modules

DCPS consists of five modules:

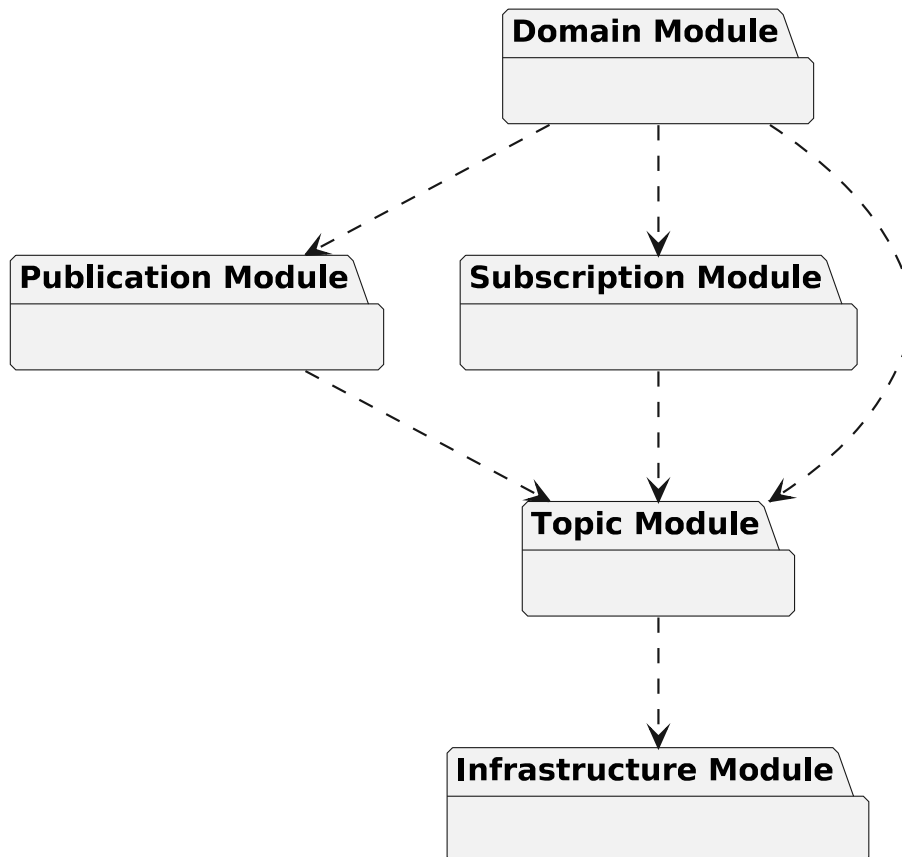


Figure 1 DCPS modules and their dependencies

- [Infrastructure module](#) defines the abstract classes and the interfaces that are refined by the other modules. It also provides support for the notification-based interaction style with the middleware.

- [Domain module](#) contains the [DDSDomainParticipant](#) class that acts as an entrypoint of the Service and acts as a factory for many of the classes. The [DDSDomainParticipant](#) also acts as a container for the other objects that make up the Service.
- [Topic module](#) contains the [DDSTopic](#) class, and more generally, all that is needed by the application to define [DDSTopic](#) objects and attach QoS policies to them.
- [Publication module](#) contains the [DDSPublisher](#) and [DDSDataWriter](#) classes, and more generally, all that is needed on the publication side.
- [Subscription module](#) contains the [DDSSubscriber](#), and [DDSDataReader](#), and more generally, all that is needed on the subscription side.

6.2.5 Content Filter Plugin

<<[experimental](#)>> Content Filter Plugin

Macros

- `#define DDS_SQL_CONTENT_FILTER_CLASS "DDSSQL"`
The name of the SQL content filter class.

6.2.5.1 Detailed Description

<<[experimental](#)>> Content Filter Plugin

This module provides the content filter plugin to enable content filtering functionality in RTI Connex DDS Micro.

6.2.5.2 Macro Definition Documentation

DDS_SQL_CONTENT_FILTER_CLASS

```
#define DDS_SQL_CONTENT_FILTER_CLASS "DDSSQL"
```

The name of the SQL content filter class.

6.2.6 Domain Module

Contains the [DDSDomainParticipant](#) class that acts as an entrypoint of RTI Connex DDS Micro and acts as a factory for many of the classes. The [DDSDomainParticipant](#) also acts as a container for the other objects that make up RTI Connex DDS Micro.

Topics

- [DomainParticipantFactory](#)
DDSDomainParticipantFactory entity and associated elements
- [DomainParticipant](#)
DDSDomainParticipant entity and associated elements
- [Discovery](#)
Descriptions of remote entities used for discovery.

Typedefs

- typedef [DDS_Long](#) [DDS_DomainId_t](#)
An integer that indicates in which domain a [DDSDomainParticipant](#) communicates.

6.2.6.1 Detailed Description

Contains the [DDSDomainParticipant](#) class that acts as an entrypoint of RTI Connex DDS Micro and acts as a factory for many of the classes. The [DDSDomainParticipant](#) also acts as a container for the other objects that make up RTI Connex DDS Micro.

Defines the DDS domain package.

6.2.6.2 Typedef Documentation

DDS_DomainId_t

```
typedef DDS\_Long DDS\_DomainId\_t
```

An integer that indicates in which domain a [DDSDomainParticipant](#) communicates.

Participants with the same [DDS_DomainId_t](#) are said to be in the same domain, and can thus communicate with one another.

The lower limit for a domain ID is 0. The upper limit for a domain ID is determined by the guidelines of the RTPS specification of the well-known ports on which a Participant will listen for discovery metatraffic from other Participants.

6.2.6.3 DomainParticipantFactory

[DDSDomainParticipantFactory](#) entity and associated elements

Classes

- struct [DDS_DomainParticipantFactoryQos](#)
QoS policies supported by a [DDSDomainParticipantFactory](#).
- class [DDSDomainParticipantFactory](#)
<<singleton>> <<interface>> Allows creation and destruction of [DDSDomainParticipant](#) objects.

Variables

- const struct [DDS_DomainParticipantQos DDS_PARTICIPANT_QOS_DEFAULT](#)
Special value for creating domain participant with default QoS.

6.2.6.3.1 Detailed Description

[DDSDomainParticipantFactory](#) entity and associated elements

6.2.6.3.2 Variable Documentation

DDS_PARTICIPANT_QOS_DEFAULT

```
const struct DDS_DomainParticipantQos DDS_PARTICIPANT_QOS_DEFAULT [extern]
```

Special value for creating domain participant with default QoS.

When used in [DDSDomainParticipantFactory::create_participant](#), this special value is used to indicate that the [DDSDomainParticipant](#) should be created with the default [DDSDomainParticipant](#) QoS.

NOTE: You cannot use this value to *get* the default QoS for a DomainParticipant; for this purpose, use [DDSDomainParticipantFactory::get_default_participant_qos](#).

See also

[DDSDomainParticipantFactory::create_participant\(\)](#)

6.2.6.4 DomainParticipant

[DDSDomainParticipant](#) entity and associated elements

Classes

- struct [DDS_DomainParticipantQos](#)
QoS policies supported by a [DDSDomainParticipant](#) entity.
- class [DDSDomainParticipantListener](#)
<<interface>> Listener for a [DDSDomainParticipant](#) and all of its child entities.
- class [DDSDomainParticipant](#)
<<interface>> Container for all [DDSDomainEntity](#) objects.

Variables

- const struct [DDS_TopicQos DDS_TOPIC_QOS_DEFAULT](#)
Special value for creating a [DDSTopic](#) with default QoS. Note that this is the only supported parameter in RTI Connex DDS Micro when creating a topic.
- const struct [DDS_PublisherQos DDS_PUBLISHER_QOS_DEFAULT](#)
Special value for creating a [DDSPublisher](#) with default QoS.
- const struct [DDS_SubscriberQos DDS_SUBSCRIBER_QOS_DEFAULT](#)
Special value for creating a [DDSSubscriber](#) with default QoS.
- const struct [DDS_FlowControllerProperty_t DDS_FLOW_CONTROLLER_PROPERTY_DEFAULT](#)
<<eXtension>> Special value for creating a [DDSFlowController](#) with default property.

6.2.6.4.1 Detailed Description

[DDSDomainParticipant](#) entity and associated elements

6.2.6.4.2 Variable Documentation

DDS_TOPIC_QOS_DEFAULT

```
const struct DDS\_TopicQos DDS_TOPIC_QOS_DEFAULT [extern]
```

Special value for creating a [DDSTopic](#) with default QoS. Note that this is the only supported parameter in RTI Connex DDS Micro when creating a topic.

When used in [DDSDomainParticipant::create_topic](#), this special value indicates that the [DDSTopic](#) should be created with the default [DDSTopic](#) QoS.

NOTE: You cannot use this value to *get* the default QoS for a Topic; for this purpose, use [DDSDomainParticipant::get_default_topic_qos](#).

See also

[DDSDomainParticipant::create_topic](#)

DDS_PUBLISHER_QOS_DEFAULT

```
const struct DDS\_PublisherQos DDS_PUBLISHER_QOS_DEFAULT [extern]
```

Special value for creating a [DDSPublisher](#) with default QoS.

When used in [DDSDomainParticipant::create_publisher](#), this special value is used to indicate that the [DDSPublisher](#) should be created with the default [DDSPublisher](#) QoS.

NOTE: You cannot use this value to *get* the default QoS for a Publisher; for this purpose, use [DDSDomainParticipant::get_default_publisher](#).

See also

[DDSDomainParticipant::create_publisher](#)

DDS_SUBSCRIBER_QOS_DEFAULT

```
const struct DDS_SubscriberQos DDS_SUBSCRIBER_QOS_DEFAULT [extern]
```

Special value for creating a [DDSSubscriber](#) with default QoS.

When used in [DDSDomainParticipant::create_subscriber](#), this special value is used to indicate that the [DDSSubscriber](#) should be created with the default [DDSSubscriber](#) QoS.

NOTE: You cannot use this value to *get* the default QoS for a Subscriber; for this purpose, use [DDSDomainParticipant::get_default_subscrib](#)

See also

[DDSDomainParticipant::create_subscriber](#)

DDS_FLOW_CONTROLLER_PROPERTY_DEFAULT

```
const struct DDS_FlowControllerProperty_t DDS_FLOW_CONTROLLER_PROPERTY_DEFAULT [extern]
```

<<*eXtension*>> Special value for creating a [DDSFlowController](#) with default property.

When used in [DDSDomainParticipant::create_flowcontroller](#), this special value is used to indicate that the [DDSFlowController](#) should be created with the default [DDSFlowController](#) property by means of the operation `get_↵ default_flowcontroller_property` and using the resulting QoS to create the [DDS_FlowControllerProperty_t](#).

Note: You cannot use this value to *get* the default properties for a FlowController; for this purpose, use [DDSDomainParticipant::get_default_flowcontroller_property](#).

See also

[DDSDomainParticipant::create_flowcontroller](#)

[DDSDomainParticipant::set_default_flowcontroller_property](#)

[DDSFlowController::set_property](#)

6.2.6.5 Discovery

Descriptions of remote entities used for discovery.

Topics

- [Participant Built-in Topic](#)

Builtin topic for accessing information about the DomainParticipants discovered by RTI Connex DDS Micro.

- [Publication Built-in Topic](#)

Builtin topic for configuring information about the Publications to be discovered by RTI Connex DDS Micro.

- [Subscription Built-in Topic](#)

Builtin topic for configuring information about the Subscriptions to be discovered by RTI Connex DDS Micro.

Classes

- struct [DDS_ContentFilterProperty](#)
All of the information necessary for a [DDSDDataWriter](#) to perform writer-side filtering for a [DDSDDataReader](#).
- struct [DDS_ChecksumProperty_t](#)
<<eXtension>> Type to define the checksum properties of the participant RTI Connex DDS Micro.
- struct [DDS_Locator_t](#)
<<eXtension>> Type used to represent the addressing information needed to send a message to an RTPS Endpoint using one of the supported transports.
- struct [DDS_LocatorSeq](#)
Declares IDL sequence < [DDS_Locator_t](#) >
- struct [DDS_ProtocolVersion_t](#)
<<eXtension>> Type used to represent the version of the RTPS protocol.
- struct [DDS_VendorId_t](#)
<<eXtension>> Type used to represent the vendor of the service implementing the RTPS protocol.
- struct [DDS_ProductVersion_t](#)
<<eXtension>> Type used to represent the current version of RTI Connex DDS Micro.
- struct [DDS_BuiltinTopicKey_t](#)
The key type of the built-in topic types.

Macros

- #define [DDS_VENDOR_ID_LENGTH_MAX](#) 2
Length of vendor id.
- #define [DDS_PRODUCTVERSION_UNKNOWN](#) { 0, 0, '0', 0 }
The value used when the product version is unknown.

Typedefs

- typedef struct [DDS_BuiltinTopicKey_t](#) [DDS_BuiltinTopicKey_t](#)
The key type of the built-in topic types.

Variables

- const [DDS_Long](#) [DDS_LOCATOR_ADDRESS_LENGTH_MAX](#)
Declares length of address field in locator.
- const struct [DDS_Locator_t](#) [DDS_LOCATOR_INVALID](#)
An invalid locator.
- const [DDS_Long](#) [DDS_LOCATOR_KIND_UDPv4](#)
A locator for a UDPv4 address.
- const [DDS_Long](#) [DDS_LOCATOR_KIND_UDPv6](#)
A locator for a UDPv6 address.
- const [DDS_Long](#) [DDS_LOCATOR_KIND_RESERVED](#)
Locator of this kind is reserved.
- const [DDS_Long](#) [DDS_LOCATOR_KIND_SHMEM](#)
A locator for an address accessed via shared memory.

- const [DDS_ProtocolVersion_t DDS_PROTOCOLVERSION_1_0](#)
The protocol version 1.0.
- const [DDS_ProtocolVersion_t DDS_PROTOCOLVERSION_1_1](#)
The protocol version 1.1.
- const [DDS_ProtocolVersion_t DDS_PROTOCOLVERSION_1_2](#)
The protocol version 1.2.
- const [DDS_ProtocolVersion_t DDS_PROTOCOLVERSION_2_0](#)
The protocol version 2.0.
- const [DDS_ProtocolVersion_t DDS_PROTOCOLVERSION_2_1](#)
The protocol version 2.1.
- const [DDS_ProtocolVersion_t DDS_PROTOCOLVERSION](#)
The most recent protocol version. Currently 2.5.

6.2.6.5.1 Detailed Description

Descriptions of remote entities used for discovery.

Given a DDS entity, a remote entity is any other entity that can be discovered and matched. A built-in object is a description of a remote entity, including all the QoS which are important for discovery and matching of remote readers and writers with local readers and writers.

RTI Connex DDS Micro must discover and keep track of the remote entities, such as new participants in the domain. Depending on the type of discovery used, this information about remote entities is gathered in three ways:

1. No data is sent over the network (static discovery)
2. A subset is sent over the network (dynamic participant/static endpoint discovery), while the rest is statically asserted.
3. All data about DomainParticipants DataReaders and DataWriters is over the network (simple discovery)

In the first case, all information about remote DomainParticipants DataReaders and DataWriters is statically configured by the user before starting the middleware. To do this static configuration, the user must fill in the built-in structures for all remote entities it wishes to discover, and must assert them to the middleware. This type of discovery is not supported in this version of RTI Connex DDS Micro.

In the second case, the participant data is sent over the network, but the endpoint data is statically configured. So, the local application fills in builtin structures representing the QoS and object IDs all the remote readers and writers it wishes to discover. For this type of discovery the user does not configure builtin data for remote participants. This type of discovery is faster than Simple Discovery, and does not require reliable communication. This is supported by RTI Connex DDS Micro and is known as Dynamic Participant, Static Endpoint (DPSE) discovery.

In the last case, the user does not need to fill in any data about remote entities, because it is automatically sent over the network by the middleware. The user can access the data through special listeners for discovery traffic.

This is supported by RTI Connex DDS Micro and is known as Dynamic Participant, Dynamic Endpoint (DPDE) discovery. Refer to [DDS_ParticipantBuiltinTopicData](#), [DDS_SubscriptionBuiltinTopicData](#) and [DDS_PublicationBuiltinTopicData](#) for a description of all the built-in topics and their contents.

[<<eXtension>>](#) Dynamic Participant, Static Endpoint Discovery plugin mechanism for RTI Connex DDS Micro.

6.2.6.5.2 Macro Definition Documentation

DDS_VENDOR_ID_LENGTH_MAX

```
#define DDS_VENDOR_ID_LENGTH_MAX 2
```

Length of vendor id.

DDS_PRODUCTVERSION_UNKNOWN

```
#define DDS_PRODUCTVERSION_UNKNOWN { 0, 0, '0', 0 }
```

The value used when the product version is unknown.

6.2.6.5.3 Typedef Documentation

DDS_BuiltinTopicKey_t

```
typedef struct DDS_BuiltinTopicKey_t DDS_BuiltinTopicKey_t
```

The key type of the built-in topic types.

Each remote [DDSEntity](#) to be discovered can be uniquely identified by this key. This is the key of all the built-in topic data types.

The value of element `DDS_BUILTIN_TOPIC_KEY_OBJECT_ID` is the object ID of the remote entity. In Dynamic Participant/Static Endpoint discovery, this must be set correctly for each remote [DDSDataReader](#) and [DDSDataWriter](#) that an application intends to discover.

For example, to configure a remote [DDSDataWriter](#) with object ID 100 by setting the key in the [DDS_PublicationBuiltinTopicData](#)↔:

```
remote_pub_data.key.value[DDS_BUILTIN_TOPIC_KEY_OBJECT_ID] = 100;
```

See also

[DDS_ParticipantBuiltinTopicData](#)

[DDS_PublicationBuiltinTopicData](#)

[DDS_SubscriptionBuiltinTopicData](#)

6.2.6.5.4 Variable Documentation

DDS_LOCATOR_ADDRESS_LENGTH_MAX

```
const DDS\_Long DDS_LOCATOR_ADDRESS_LENGTH_MAX
```

Declares length of address field in locator.

DDS_LOCATOR_INVALID

```
const struct DDS_Locator_t DDS_LOCATOR_INVALID
```

An invalid locator.

DDS_LOCATOR_KIND_UDPv4

```
const DDS_Long DDS_LOCATOR_KIND_UDPv4
```

A locator for a UDPv4 address.

DDS_LOCATOR_KIND_UDPv6

```
const DDS_Long DDS_LOCATOR_KIND_UDPv6
```

A locator for a UDPv6 address.

DDS_LOCATOR_KIND_RESERVED

```
const DDS_Long DDS_LOCATOR_KIND_RESERVED
```

Locator of this kind is reserved.

DDS_LOCATOR_KIND_SHMEM

```
const DDS_Long DDS_LOCATOR_KIND_SHMEM
```

A locator for an address accessed via shared memory.

DDS_PROTOCOLVERSION_1_0

```
const DDS_ProtocolVersion_t DDS_PROTOCOLVERSION_1_0
```

The protocol version 1.0.

DDS_PROTOCOLVERSION_1_1

```
const DDS_ProtocolVersion_t DDS_PROTOCOLVERSION_1_1
```

The protocol version 1.1.

DDS_PROTOCOLVERSION_1_2

```
const DDS_ProtocolVersion_t DDS_PROTOCOLVERSION_1_2
```

The protocol version 1.2.

DDS_PROTOCOLVERSION_2_0

```
const DDS_ProtocolVersion_t DDS_PROTOCOLVERSION_2_0
```

The protocol version 2.0.

DDS_PROTOCOLVERSION_2_1

```
const DDS_ProtocolVersion_t DDS_PROTOCOLVERSION_2_1
```

The protocol version 2.1.

DDS_PROTOCOLVERSION

```
const DDS_ProtocolVersion_t DDS_PROTOCOLVERSION
```

The most recent protocol version. Currently 2.5.

6.2.6.5.5 Participant Built-in Topic

Builtin topic for accessing information about the DomainParticipants discovered by RTI Connex DDS Micro.

Classes

- struct [DDS_ParticipantBuiltinTopicData](#)
Object representing a remote DomainParticipant.

Variables

- const char *const [DDS_PARTICIPANT_TOPIC_NAME](#)
Participant topic name

Detailed Description

Builtin topic for accessing information about the DomainParticipants discovered by RTI Connex DDS Micro.

Variable Documentation

DDS_PARTICIPANT_TOPIC_NAME

```
const char* const DDS_PARTICIPANT_TOPIC_NAME
```

Participant topic name

Topic name of participant builtin topic data [DDSDataReader](#).

See also

[DDS_ParticipantBuiltinTopicData](#)

6.2.6.5.6 Publication Built-in Topic

Builtin topic for configuring information about the Publications to be discovered by RTI Connex DDS Micro.

Classes

- struct [DDS_PublicationBuiltinTopicData](#)
Object describing a remote Publication.

Variables

- const char *const [DDS_PUBLICATION_TOPIC_NAME](#)
Publication topic name

Detailed Description

Builtin topic for configuring information about the Publications to be discovered by RTI Connex DDS Micro.

Variable Documentation

DDS_PUBLICATION_TOPIC_NAME

```
const char* const DDS_PUBLICATION_TOPIC_NAME
```

Publication topic name

Topic name of publication builtin topic data [DDSDataReader](#).

See also

[DDS_PublicationBuiltinTopicData](#)

6.2.6.5.7 Subscription Built-in Topic

Builtin topic for configuring information about the Subscriptions to be discovered by RTI Connex DDS Micro.

Classes

- struct [DDS_SubscriptionBuiltinTopicData](#)
Entry created when a [DDSDataReader](#) is discovered in association with its Subscriber.

Variables

- const char *const [DDS_SUBSCRIPTION_TOPIC_NAME](#)
Subscription topic name

Detailed Description

Builtin topic for configuring information about the Subscriptions to be discovered by RTI Connex DDS Micro.

Variable Documentation

DDS_SUBSCRIPTION_TOPIC_NAME

```
const char* const DDS_SUBSCRIPTION_TOPIC_NAME
```

Subscription topic name

Topic name of subscription builtin topic data [DDSDataReader](#).

See also

[DDS_SubscriptionBuiltinTopicData](#)

6.2.7 Topic

Contains the [DDSTopic](#), the [DDSTopicListener](#) interface, and more generally, all that is needed by an application to define [DDSTopic](#) objects and the data types that are associated with [DDSTopic](#) objects.

Topics

- [DDS-Specific Primitive Types](#)
Basic DDS value types for use in user data types.
- [Topic](#)
DDSTopic entity and associated elements
- [FlatData Topic-Types](#)
<<eXtension>> FlatData Language Binding for IDL topic-types
- [Zero Copy Transfer Over Shared Memory](#)
<<eXtension>> Zero Copy transfer over shared memory
- [User Data Type Support](#)
Defines generic classes and macros to support user data types.

6.2.7.1 Detailed Description

Contains the [DDSTopic](#), the [DDSTopicListener](#) interface, and more generally, all that is needed by an application to define [DDSTopic](#) objects and the data types that are associated with [DDSTopic](#) objects.

6.2.7.2 DDS-Specific Primitive Types

Basic DDS value types for use in user data types.

Macros

- `#define DDS\_BOOLEAN\_TRUE`
Defines TRUE value of IDL boolean data type.
- `#define DDS\_BOOLEAN\_FALSE`
Defines FALSE value of IDL boolean data type.

Typedefs

- `typedef CDR_Char DDS\_Char`
Defines IDL char data type.
- `typedef RTI_UINT16 DDS\_Wchar`
Defines IDL wchar data type.
- `typedef CDR_Octet DDS\_Octet`
Defines IDL octet data type.
- `typedef CDR_Short DDS\_Short`
Defines IDL short data type.
- `typedef CDR_UnsignedShort DDS\_UnsignedShort`
Defines IDL unsigned short data type.
- `typedef CDR_Long DDS\_Long`
Defines IDL long data type.
- `typedef CDR_UnsignedLong DDS\_UnsignedLong`

- Defines IDL unsigned long data type.*
- typedef CDR_LongLong [DDS_LongLong](#)
Defines IDL long long data type.
- typedef CDR_UnsignedLongLong [DDS_UnsignedLongLong](#)
Defines IDL unsigned long long data type.
- typedef CDR_Float [DDS_Float](#)
Defines IDL float data type.
- typedef CDR_Double [DDS_Double](#)
Defines IDL double data type.
- typedef CDR_LongDouble [DDS_LongDouble](#)
Defines IDL long double data type.
- typedef CDR_Boolean [DDS_Boolean](#)
Defines IDL boolean data type.
- typedef CDR_Enum [DDS_Enum](#)
Defines IDL enum data type.
- typedef CDR_String [DDS_String](#)
IDL string type representation.
- typedef [DDS_Wchar](#) * [DDS_Wstring](#)
Defines IDL wstring data type.

6.2.7.2.1 Detailed Description

Basic DDS value types for use in user data types.

As part of the finalization of the DDS standard, a number of DDS-specific primitive types will be introduced. By using these types, you will ensure that your data is serialized consistently across platforms even if the built-in types have different sizes on those platforms.

In this version of RTI Connex Micro DDS, the DDS primitive types are defined using the OMG's Common Data Representation (CDR) standard.

6.2.7.2.2 Macro Definition Documentation

DDS_BOOLEAN_TRUE

```
#define DDS_BOOLEAN_TRUE
```

Defines TRUE value of IDL boolean data type.

DDS_BOOLEAN_FALSE

```
#define DDS_BOOLEAN_FALSE
```

Defines FALSE value of IDL boolean data type.

6.2.7.2.3 Typedef Documentation

DDS_Char

```
typedef CDR_Char DDS_Char
```

Defines IDL char data type.

An 8-bit quantity that encodes a single byte character from any byte-oriented code set.

See also

For sequence support, see [FooSeq](#).

DDS_Wchar

```
typedef RTI_UINT16 DDS_Wchar
```

Defines IDL wchar data type.

A 16-bit quantity that encodes a wide character from any character set.

See also

For sequence support, see [FooSeq](#).

DDS_Octet

```
typedef CDR_Octet DDS_Octet
```

Defines IDL octet data type.

An 8-bit quantity that is guaranteed not to undergo any conversion when transmitted by the middleware.

See also

For sequence support, see [FooSeq](#).

DDS_Short

```
typedef CDR_Short DDS_Short
```

Defines IDL short data type.

A 16-bit signed short integer value.

See also

For sequence support, see [FooSeq](#).

DDS_UnsignedShort

```
typedef CDR_UnsignedShort DDS_UnsignedShort
```

Defines IDL unsigned short data type.

A 16-bit unsigned short integer value.

See also

For sequence support, see [FooSeq](#).

DDS_Long

```
typedef CDR_Long DDS_Long
```

Defines IDL long data type.

A 32-bit signed long integer value.

See also

For sequence support, see [FooSeq](#).

DDS_UnsignedLong

```
typedef CDR_UnsignedLong DDS_UnsignedLong
```

Defines IDL unsigned long data type.

A 32-bit unsigned long integer value.

See also

For sequence support, see [FooSeq](#).

DDS_LongLong

```
typedef CDR_LongLong DDS_LongLong
```

Defines IDL long long data type.

A 64-bit signed long long integer value.

Since some architectures do not support long long (8 bytes long), RTI has defined character arrays that match the expected size of these types.

See also

For sequence support, see [FooSeq](#).

DDS_UnsignedLongLong

```
typedef CDR_UnsignedLongLong DDS_UnsignedLongLong
```

Defines IDL unsigned long long data type.

A 64-bit unsigned long long integer value.

Since some architectures do not support (unsigned) long long (8 byte long), RTI has defined character arrays that match the expected size of these types.

See also

For sequence support, see [FooSeq](#).

DDS_Float

```
typedef CDR_Float DDS_Float
```

Defines IDL float data type.

A 32-bit floating point value.

See also

For sequence support, see [FooSeq](#).

DDS_Double

```
typedef CDR_Double DDS_Double
```

Defines IDL double data type.

A 64-bit floating point value.

See also

For sequence support, see [FooSeq](#).

DDS_LongDouble

```
typedef CDR_LongDouble DDS_LongDouble
```

Defines IDL long double data type.

A 16-byte floating point value.

Since some architectures do not support long double, RTI has defined character arrays that match the expected size of this type. On systems that do have native long double, you have to define `RTI_CDR_SIZEOF_LONG_DOUBLE` as 16 to map them to native types.

See also

For sequence support, see [FooSeq](#).

DDS_Boolean

```
typedef CDR_Boolean DDS_Boolean
```

Defines IDL boolean data type.

An 8-bit boolean value that is used to denote a data item that can only take one of the values [DDS_BOOLEAN_TRUE](#) (1) or [DDS_BOOLEAN_FALSE](#) (0).

See also

For sequence support, see [FooSeq](#).

DDS_Enum

```
typedef CDR_Enum DDS_Enum
```

Defines IDL enum data type.

Encoded as unsigned long value. The first enum identifier has the numeric value zero (0). Successive enum identifiers take ascending numeric values, in order of declaration from left to right.

See also

For sequence support, see [FooSeq](#).

DDS_String

```
typedef CDR_String DDS_String
```

IDL `string` type representation.

Has a range of ISO Latin-1 (except ASCII NULL) and a variable length.

See also

[String Support](#)

For sequence support, see [FooSeq](#).

DDS_Wstring

```
typedef DDS_Wchar* DDS_Wstring
```

Defines IDL wstring data type.

Each character composing the string is a 16-bit quantity that encodes a wide character from any character set.

See also

For sequence support, see [FooSeq](#).

6.2.7.3 Topic

[DDSTopic](#) entity and associated elements

Classes

- struct [DDS_InconsistentTopicStatus](#)
DDS_INCONSISTENT_TOPIC_STATUS
- struct [DDS_TopicQos](#)
QoS policies supported by a [DDSTopic](#) entity.
- class [DDSTopicDescription](#)
<<interface>> Base class for [DDSTopic](#).
- class [DDSTopic](#)
<<interface>> The most basic description of the data to be published and subscribed.
- class [DDSTopicListener](#)
<<interface>> [DDSTopicListener](#) for [DDSTopic](#) entities.

6.2.7.3.1 Detailed Description

[DDSTopic](#) entity and associated elements

6.2.7.4 FlatData Topic-Types

<<eXtension>> FlatData Language Binding for IDL topic-types

<<eXtension>> FlatData Language Binding for IDL topic-types

Warning

The FlatData language binding is only available in the Traditional C++ API. RTI Connex DDS Micro does not support Modern C++. It is not available in the C API.

FlatData is a language binding for IDL types in which the in-memory representation of a sample matches the wire representation. Therefore, the cost of serialization/deserialization is zero.

Note

For a complete description of the FlatData language binding and its benefits, and a tutorial, see the "Sending Large Data" chapter in the **RTI Connex DDS Micro User's Manual**.

For buildable **code examples**, see <https://community.rti.com/kb/flatdata-and-zero-copy-examples>.

6.2.7.5 Zero Copy Transfer Over Shared Memory

<<eXtension>> Zero Copy transfer over shared memory

<<eXtension>> Zero Copy transfer over shared memory

Note

For a description of Zero Copy transfer over shared memory and its benefits, and a tutorial, see the "Sending Large Data" chapter in the **RTI Connext DDS Micro User's Manual**.

For buildable **code examples**, see <https://community.rti.com/kb/flatdata-and-zerocopy-examples>.

Zero Copy transfer over shared memory is available in both the C API and the C++ API.

6.2.7.6 User Data Type Support

Defines generic classes and macros to support user data types.

Classes

- struct `DDS_InstanceHandleSeq`
Instantiates `FooSeq` < `DDS_InstanceHandle_t` > .
- class `FooTypeSupport`
<<interface>> <<generic>> User data type specific interface.

Macros

- `#define DDS_DATAWRITER_CPP(TDataWriter, TData)`
Declares the interface required to support a user data type specific data writer.
- `#define DDS_DATAREADER_CPP(TDataReader, TData)`
Declares the interface required to support a user data type-specific data reader.
- `#define DDS_TYPESUPPORT_CPP(TTypeSupport, TData)`
Declares the interface required to support a user data type.

Typedefs

- `typedef DDS_HANDLE_TYPE_NATIVE DDS_InstanceHandle_t`
Type definition for an instance handle.

Functions

- `DDS_Boolean DDS_InstanceHandle_equals` (const `DDS_InstanceHandle_t` *self, const `DDS_InstanceHandle_t` *other)
Compares this instance handle with another handle for equality.
- `DDS_Boolean DDS_InstanceHandle_is_nil` (const `DDS_InstanceHandle_t` *self)
Compare this handle to `DDS_HANDLE_NIL`

Variables

- const [DDS_InstanceHandle_t DDS_HANDLE_NIL](#)
The NIL instance handle.

6.2.7.6.1 Detailed Description

Defines generic classes and macros to support user data types.

Defines the DDS user data type support.

DDS specifies strongly typed interfaces to read and write user data. For each data class defined by the application, there is a number of specialized classes that are required to facilitate the type-safe interaction of the application with RTI Connex DDS Micro.

RTI Connex DDS Micro provides an automatic means to generate all these type-specific classes with the RTI IDL Compiler User Manual utility. The complete set of automatic classes created for a hypothetical user data type named `Foo` are shown below.

The macros defined here declare the strongly typed APIs needed to support an arbitrary user defined data of type `Foo`.

6.2.7.6.2 Macro Definition Documentation

DDS_DATAWRITER_CPP

```
#define DDS_DATAWRITER_CPP (
    TDataWriter,
    TData)
```

Declares the interface required to support a user data type specific data writer.

Uses:

[FooTypeSupport](#) user data type, `Foo`

Defines:

[FooDataWriter DDSDataWriter](#) of type `Foo`, i.e. [FooDataWriter](#)

DDS_DATAREADER_CPP

```
#define DDS_DATAREADER_CPP (
    TDataReader,
    TData)
```

Declares the interface required to support a user data type-specific data reader.

Uses:

[FooTypeSupport](#) user data type, `Foo` [FooSeq](#) sequence of user data type, `sequence<Foo>`

Defines:

[FooDataReader](#) [DDSDataReader](#) of type `Foo`, i.e. [FooDataReader](#)

See also

[FooSeq](#)

DDS_TYPESUPPORT_CPP

```
#define DDS_TYPESUPPORT_CPP (
    TTypeSupport,
    TData)
```

Declares the interface required to support a user data type.

Defines:

[FooTypeSupport](#) `TypeSupport` of type `Foo`, i.e. [FooTypeSupport](#)

6.2.7.6.3 Typedef Documentation

DDS_InstanceHandle_t

```
typedef DDS_HANDLE_TYPE_NATIVE DDS_InstanceHandle_t
```

Type definition for an instance handle.

Handle to identify different instances of the same [DDSTopic](#) of a certain type.

See also

[FooDataWriter::register_instance](#)
[DDS_SampleInfo::instance_handle](#)

6.2.7.6.4 Function Documentation

DDS_InstanceHandle_equals()

```
DDS_Boolean DDS_InstanceHandle_equals (
    const DDS_InstanceHandle_t * self,
    const DDS_InstanceHandle_t * other)
```

Compares this instance handle with another handle for equality.

Parameters

<i>self</i>	<< <i>in</i> >> This handle. Cannot be NULL.
<i>other</i>	<< <i>in</i> >> The other handle to be compared with this handle. Cannot be NULL.

Returns

[DDS_BOOLEAN_TRUE](#) if the two handles have equal values, or [DDS_BOOLEAN_FALSE](#) otherwise.

MT Safety:

UNSAFE. This operation does not protect against the left or right side from being modified by another thread while the comparison is made.

See also

[DDS_InstanceHandle_is_nil](#)

DDS_InstanceHandle_is_nil()

```
DDS_Boolean DDS_InstanceHandle_is_nil (
    const DDS_InstanceHandle_t * self)
```

Compare this handle to [DDS_HANDLE_NIL](#)

Returns

[DDS_BOOLEAN_TRUE](#) if the given instance handle is equal to [DDS_HANDLE_NIL](#) or [DDS_BOOLEAN_FALSE](#) otherwise.

See also

[DDS_InstanceHandle_equals](#)

6.2.7.6.5 Variable Documentation**DDS_HANDLE_NIL**

```
const DDS_InstanceHandle_t DDS_HANDLE_NIL [extern]
```

The NIL instance handle.

Special [DDS_InstanceHandle_t](#) value

See also

[DDS_InstanceHandle_is_nil](#)

6.2.8 Publication Module

Contains the [DDSPublisher](#), and [DDSDataWriter](#) classes, and more generally, all that is needed on the publication side.

Topics

- [Flow Controllers](#)
<<eXtension>> [DDSFlowController](#) and associated elements
- [Publisher](#)
[DDSPublisher](#) entity and associated elements
- [DataWriter](#)
[DDSDataWriter](#) entity and associated elements

6.2.8.1 Detailed Description

Contains the [DDSPublisher](#), and [DDSDataWriter](#) classes, and more generally, all that is needed on the publication side.

Defines the DDS publication package.

Please read the information on the individual types to determine whether a method is supported.

6.2.8.2 Flow Controllers

<<eXtension>> [DDSFlowController](#) and associated elements

Classes

- struct [DDS_FlowControllerTokenBucketProperty_t](#)
[DDSFlowController](#) uses the popular token bucket approach for open loop network flow control. The flow control characteristics are determined by the token bucket properties.
- struct [DDS_FlowControllerProperty_t](#)
Determines the flow control characteristics of the [DDSFlowController](#).
- class [DDSFlowController](#)
<<interface>> A flow controller is the object responsible for shaping the network traffic by determining when attached asynchronous [DDSDataWriter](#) instances are allowed to write data.

Enumerations

- enum [DDS_FlowControllerSchedulingPolicy](#) { [DDS_RR_FLOW_CONTROLLER_SCHED_POLICY](#), [DDS_EDF_FLOW_CONTROLLER_SCHED_POLICY](#), [DDS_HPF_FLOW_CONTROLLER_SCHED_POLICY](#) }
Kinds of flow controller scheduling policy.

Variables

- const char *const [DDS_DEFAULT_FLOW_CONTROLLER_NAME](#)
[default] Special value of [DDS_PublishModeQosPolicy::flow_controller_name](#) that refers to the built-in default flow controller.
- const char *const [DDS_FIXED_RATE_FLOW_CONTROLLER_NAME](#)
Special value of [DDS_PublishModeQosPolicy::flow_controller_name](#) that refers to the built-in fixed-rate flow controller.
- const char *const [DDS_ON_DEMAND_FLOW_CONTROLLER_NAME](#)
Special value of [DDS_PublishModeQosPolicy::flow_controller_name](#) that refers to the built-in on-demand flow controller.

6.2.8.2.1 Detailed Description

<<*eXtension*>> [DDSFlowController](#) and associated elements

[DDSFlowController](#) provides the network traffic shaping capability to asynchronous [DDSDataWriter](#) instances. For use cases and advantages of publishing asynchronously, please refer to [DDS_PublishModeQosPolicy](#) of [DDS_DataWriterQos](#).

See also

[DDS_PublishModeQosPolicy](#)
[DDS_DataWriterQos::publish_mode](#)

6.2.8.2.2 Enumeration Type Documentation

DDS_FlowControllerSchedulingPolicy

enum [DDS_FlowControllerSchedulingPolicy](#)

Kinds of flow controller scheduling policy.

QoS:

[DDS_FlowControllerProperty_t](#)

Samples written by an asynchronous [DDSDataWriter](#) are not sent in the context of the [FooDataWriter::write](#) call. Instead, the middleware puts the samples in a queue for future processing. The [DDSFlowController](#) associated with each asynchronous DataWriter instance determines when the samples are actually sent.

Each [DDSFlowController](#) maintains a separate FIFO queue for each unique destination (remote application). Samples written by asynchronous [DDSDataWriter](#) instances associated with the flow controller, are placed in the queues that correspond to the intended destinations of the sample.

When tokens become available, a flow controller must decide which queue(s) to grant tokens first. This is determined by the flow controller's scheduling policy. Once a queue has been granted tokens, it is serviced by the asynchronous publishing thread. The queued up samples will be coalesced and sent to the corresponding destination. The number of samples sent depends on the data size and the number of tokens granted.

Enumerator

DDS_RR_FLOW_CONTROLLER_SCHED_POLICY	Indicates to flow control in a round-robin fashion. Whenever tokens become available, the flow controller distributes the tokens uniformly across all of its (non-empty) destination queues. No destinations are prioritized. Instead, all destinations are treated equally and are serviced in a round-robin fashion.
DDS_EDF_FLOW_CONTROLLER_SCHED_POLICY	Indicates to flow control in an earliest-deadline-first fashion. A sample's deadline is determined by the time it was written plus the latency budget of the <code>DataWriter</code> at the time of the write call (as specified in the DDS_LatencyBudgetQosPolicy). The relative priority of a flow controller's destination queue is determined by the earliest deadline across all samples it contains. When tokens become available, the DDSFlowController distributes tokens to the destination queues in order of their deadline priority. In other words, the queue containing the sample with the earliest deadline is serviced first. The number of tokens granted equals the number of tokens required to send the first sample in the queue. Note that the priority of a queue may change as samples are sent (i.e. removed from the queue). If a sample must be sent to multiple destinations or two samples have an equal deadline value, the corresponding destination queues are serviced in a round-robin fashion. Hence, under the default DDS_LatencyBudgetQosPolicy::duration setting, an <code>EDF_FLOW_CONTROLLER_SCHED_POLICY</code> DDSFlowController preserves the order in which the user calls FooDataWriter::write across the <code>DataWriters</code> associated with the flow controller. In other words, the priority of a destination queue is always determined by the earliest deadline among all samples contained in the queue. This priority inheritance approach is required in order to both honor the updated DDS_LatencyBudgetQosPolicy::duration and adhere to the DDSDataWriter in-order data delivery guarantee. [default] for DDSDataWriter

Enumerator

DDS_HPF_FLOW_CONTROLLER_SCHED_POLICY	Indicates to flow control in a highest-priority-first fashion. Determines the next destination queue to service as determined by the publication priority of the DDSDDataWriter, channel of multi-channel DataWriter, or individual sample. The relative priority of a flow controller's destination queue is determined by the highest publication priority of all samples it contains. When tokens become available, the DDSFlowController distributes tokens to the destination queues in order of their publication priority. In other words, the queue containing the sample with the highest publication priority is serviced first. The number of tokens granted equals the number of tokens required to send the first sample in the queue. Note that the priority of a queue may change as samples are sent (i.e. removed from the queue). If a sample must be sent to multiple destinations or two samples have an equal publication priority, the corresponding destination queues are serviced in a round-robin fashion. This priority inheritance approach is required in order to both honor the designated publication priority and adhere to the DDSDDataWriter in-order data delivery guarantee.
--------------------------------------	---

6.2.8.2.3 Variable Documentation

DDS_DEFAULT_FLOW_CONTROLLER_NAME

```
const char* const DDS_DEFAULT_FLOW_CONTROLLER_NAME [extern]
```

[default] Special value of [DDS_PublishModeQosPolicy::flow_controller_name](#) that refers to the built-in default flow controller.

RTI Connex DDS Micro provides several built-in [DDSFlowController](#) for use with an asynchronous [DDSDDataWriter](#). The user can choose to use the built-in flow controllers and optionally modify their properties or can create a custom flow controller.

By default, flow control is disabled. That is, the built-in [DDS_DEFAULT_FLOW_CONTROLLER_NAME](#) flow controller does not apply any flow control. Instead, it allows data to be sent asynchronously as soon as it is written by the [DDSDDataWriter](#).

Essentially, this is equivalent to a user-created [DDSFlowController](#) with the following [DDS_FlowControllerProperty_t](#) settings:

- [DDS_FlowControllerProperty_t::scheduling_policy](#) = [DDS_EDF_FLOW_CONTROLLER_SCHED_POLICY](#)
- [DDS_FlowControllerProperty_t::token_bucket](#) max_tokens = [DDS_LENGTH_UNLIMITED](#)
- [DDS_FlowControllerProperty_t::token_bucket](#) tokens_added_per_period = [DDS_LENGTH_UNLIMITED](#)
- [DDS_FlowControllerProperty_t::token_bucket](#) tokens_leaked_per_period = 0
- [DDS_FlowControllerProperty_t::token_bucket](#) period = 1 second
- [DDS_FlowControllerProperty_t::token_bucket](#) bytes_per_token = [DDS_LENGTH_UNLIMITED](#)

See also

[DDSDomainParticipant::lookup_flowcontroller](#)
[DDSFlowController::set_property](#)
[DDS_PublishModeQosPolicy](#)

DDS_FIXED_RATE_FLOW_CONTROLLER_NAME

```
const char* const DDS_FIXED_RATE_FLOW_CONTROLLER_NAME [extern]
```

Special value of [DDS_PublishModeQosPolicy::flow_controller_name](#) that refers to the built-in fixed-rate flow controller.

RTI Connex DDS Micro provides several builtin [DDSFlowController](#) for use with an asynchronous [DDSDataWriter](#). The user can choose to use the built-in flow controllers and optionally modify their properties or can create a custom flow controller.

The built-in [DDS_FIXED_RATE_FLOW_CONTROLLER_NAME](#) flow controller shapes the network traffic by allowing data to be sent only once every second. Any accumulated samples destined for the same destination are coalesced into as few network packets as possible.

Essentially, this is equivalent to a user-created [DDSFlowController](#) with the following [DDS_FlowControllerProperty_t](#) settings:

- [DDS_FlowControllerProperty_t::scheduling_policy](#) = [DDS_EDF_FLOW_CONTROLLER_SCHED_POLICY](#)
- [DDS_FlowControllerProperty_t::token_bucket](#) `max_tokens` = [DDS_LENGTH_UNLIMITED](#)
- [DDS_FlowControllerProperty_t::token_bucket](#) `tokens_added_per_period` = [DDS_LENGTH_UNLIMITED](#)
- [DDS_FlowControllerProperty_t::token_bucket](#) `tokens_leaked_per_period` = [DDS_LENGTH_UNLIMITED](#)
- [DDS_FlowControllerProperty_t::token_bucket](#) `period` = 1 second
- [DDS_FlowControllerProperty_t::token_bucket](#) `bytes_per_token` = [DDS_LENGTH_UNLIMITED](#)

See also

[DDSDomainParticipant::lookup_flowcontroller](#)
[DDSFlowController::set_property](#)
[DDS_PublishModeQosPolicy](#)

DDS_ON_DEMAND_FLOW_CONTROLLER_NAME

```
const char* const DDS_ON_DEMAND_FLOW_CONTROLLER_NAME [extern]
```

Special value of [DDS_PublishModeQosPolicy::flow_controller_name](#) that refers to the built-in on-demand flow controller.

RTI Connex DDS Micro provides several builtin [DDSFlowController](#) for use with an asynchronous [DDSDataWriter](#). The user can choose to use the built-in flow controllers and optionally modify their properties or can create a custom flow controller.

The built-in [DDS_ON_DEMAND_FLOW_CONTROLLER_NAME](#) allows data to be sent only when the user calls [DDSFlowController::trigger_flow](#). With each trigger, all accumulated data since the previous trigger is sent (across all [DDSPublisher](#) or [DDSDataWriter](#) instances). In other words, the network traffic shape is fully controlled by the user. Any accumulated samples destined for the same destination are coalesced into as few network packets as possible.

Essentially, this is equivalent to a user-created [DDSFlowController](#) with the following [DDS_FlowControllerProperty_t](#) settings:

- [DDS_FlowControllerProperty_t::scheduling_policy](#) = [DDS_EDF_FLOW_CONTROLLER_SCHED_POLICY](#)
- [DDS_FlowControllerProperty_t::token_bucket](#) [max_tokens](#) = [DDS_LENGTH_UNLIMITED](#)
- [DDS_FlowControllerProperty_t::token_bucket](#) [tokens_added_per_period](#) = [DDS_LENGTH_UNLIMITED](#)
- [DDS_FlowControllerProperty_t::token_bucket](#) [tokens_leaked_per_period](#) = [DDS_LENGTH_UNLIMITED](#) -
[DDS_FlowControllerProperty_t::token_bucket](#) [period](#) = [DDS_DURATION_INFINITE](#)
- [DDS_FlowControllerProperty_t::token_bucket](#) [bytes_per_token](#) = [DDS_LENGTH_UNLIMITED](#)

See also

```
DDSDomainParticipant::lookup_flowcontroller
DDSFlowController::set_property
DDS_PublishModeQosPolicy;
```

6.2.8.3 Publisher

[DDSPublisher](#) entity and associated elements

Classes

- struct [DDS_PublisherQos](#)
QoS policies supported by a [DDSPublisher](#) entity.
- class [DDSPublisherListener](#)
<<interface>> [DDSListener](#) for [DDSPublisher](#) status.
- class [DDSPublisher](#)
<<interface>> A publisher is the object responsible for the actual dissemination of publications.

Typedefs

- typedef struct DDS_PublisherImpl [DDS_Publisher](#)
<<interface>> A publisher is the object responsible for the actual dissemination of publications.

Variables

- const struct [DDS_DataWriterQos](#) [DDS_DATAWRITER_QOS_DEFAULT](#)
Special value for creating [DDSDataWriter](#) with default QoS

6.2.8.3.1 Detailed Description

[DDSPublisher](#) entity and associated elements

6.2.8.3.2 Typedef Documentation

DDS_Publisher

```
typedef struct DDS_PublisherImpl DDS\_Publisher
```

<<interface>> A publisher is the object responsible for the actual dissemination of publications.

QoS:

[DDS_PublisherQos](#)

Listener:

[DDSPublisherListener](#)

A publisher acts on the behalf of one or several [DDSDataWriter](#) objects that belong to it. When it is informed of a change to the data associated with one of its [DDSDataWriter](#) objects, it decides when it is appropriate to actually send the data-update message. In making this decision, it considers any extra information that goes with the data (timestamp, writer, etc.) as well as the QoS of the [DDSPublisher](#) and the [DDSDataWriter](#).

The following operations may be called even if the [DDSPublisher](#) is not enabled. Other operations will fail with the value [DDS_RETCODE_NOT_ENABLED](#) if called on a disabled [DDSPublisher](#):

- The base-class operations [DDSEntity::enable](#)
- [DDSPublisher::create_datawriter](#), [DDSPublisher::delete_datawriter](#), [DDSPublisher::delete_contained_entities](#)

6.2.8.3.3 Variable Documentation

DDS_DATAWRITER_QOS_DEFAULT

```
const struct DDS_DataWriterQos DDS_DATAWRITER_QOS_DEFAULT [extern]
```

Special value for creating [DDSDDataWriter](#) with default QoS

When used in [DDSPublisher::create_datawriter](#), this special value is used to indicate that the [DDSDDataWriter](#) should be created with the default [DDSDDataWriter](#) QoS.

NOTE: You cannot use this value to *get* the default QoS for a [DDSDDataWriter](#); for this purpose, use [DDSPublisher::get_default_datawriter_qos](#).

See also

[DDSPublisher::create_datawriter](#)

6.2.8.4 DataWriter

[DDSDDataWriter](#) entity and associated elements

Classes

- struct [DDS_OfferedDeadlineMissedStatus](#)
[DDS_OFFERED_DEADLINE_MISSED_STATUS](#)
- struct [DDS_LivelinessLostStatus](#)
[DDS_LIVELINESS_LOST_STATUS](#)
- struct [DDS_OfferedIncompatibleQosStatus](#)
[DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#)
- struct [DDS_PublicationMatchedStatus](#)
[DDS_PUBLICATION_MATCHED_STATUS](#)
- struct [DDS_ReliableReaderActivityChangedStatus](#)
<<extension>> Describes the activity (i.e. are acknowledgements forthcoming) of reliable readers matched to a reliable writer.
- struct [DDS_ReliableSampleUnacknowledgedStatus](#)
Status about an unacknowledged sample that is removed from the [DDSDDataWriter](#) cache before being acknowledged by all matched [DDSDDataReader](#) entities.
- struct [DDS_DataWriterQos](#)
QoS policies supported by a [DDSDDataWriter](#) entity.
- class [FooDataWriter](#)
Declares the interface required to support a user data type-specific data writer.
- class [DDSDDataWriterListener](#)
<<interface>> [DDSListener](#) for writer status.
- class [DDSDDataWriter](#)
<<interface>> Allows an application to set the value of the data to be published under a given [DDSTopic](#).

Macros

- `#define DDS_ReliableReaderActivityChangedStatus_INITIALIZER`
Initializer for new status instances.

Typedefs

- `typedef void(* DDS_DataWriterListener_ReliableReaderActivityChangedCallback) (void *listener_data, DDS_DataWriter *writer, const struct DDS_ReliableReaderActivityChangedStatus *status)`
<<eXtension>> A matched reliable reader has become active or become inactive.
- `typedef void(* DDS_DataWriterListener_ReliableSampleUnacknowledgedCallback) (void *listener_data, DDS_DataWriter *writer, const struct DDS_ReliableSampleUnacknowledgedStatus *status)`
<<eXtension>> Unacknowledged samples are removed from the writer cache.
- `typedef void(* DDS_DataWriterListener_SampleRemovedCallback) (void *listener_data, DDS_DataWriter *writer, const struct DDS_Cookie_t *cookie)`
Prototype of a [DDSDDataWriterListener](#) on_sample_removed function.

6.2.8.4.1 Detailed Description

[DDSDDataWriter](#) entity and associated elements

6.2.8.4.2 Macro Definition Documentation

DDS_ReliableReaderActivityChangedStatus_INITIALIZER

```
#define DDS_ReliableReaderActivityChangedStatus_INITIALIZER
```

Initializer for new status instances.

New [DDS_ReliableReaderActivityChangedStatus](#) instances stored on the stack should be initialized with this value before they are passed to any method. This step ensures that those fields that use dynamic memory are properly initialized. This does not allocate memory.

The simplest way to create a status structure is to initialize it on the stack at the time of its creation.

6.2.8.4.3 Typedef Documentation

DDS_DataWriterListener_ReliableReaderActivityChangedCallback

```
typedef void(* DDS_DataWriterListener_ReliableReaderActivityChangedCallback) (void *listener_data, DDS_DataWriter *writer, const struct DDS_ReliableReaderActivityChangedStatus *status)
```

<<eXtension>> A matched reliable reader has become active or become inactive.

Parameters

<i>listener_data</i>	<<out>> Data associated with the listener when the listener is set
<i>writer</i>	<<out>> Locally created DDSDataWriter that triggers the listener callback
<i>status</i>	<<out>> Current reliable reader activity changed status of the locally created DDSDataWriter

DDS_DataWriterListener_ReliableSampleUnacknowledgedCallback

```
typedef void(* DDS_DataWriterListener_ReliableSampleUnacknowledgedCallback) (void *listener_data,
DDS_DataWriter *writer, const struct DDS\_ReliableSampleUnacknowledgedStatus *status)
```

<<eXtension>> Unacknowledged samples are removed from the writer cache.

Parameters

<i>listener_data</i>	<<out>> Data associated with the listener when the listener is set
<i>writer</i>	<<out>> Locally created DDSDataWriter that triggers the listener callback
<i>status</i>	<<out>> Unacknowledged sample status for DDSDataWriter

DDS_DataWriterListener_SampleRemovedCallback

```
typedef void(* DDS_DataWriterListener_SampleRemovedCallback) (void *listener_data, DDS_DataWriter
*writer, const struct DDS_Cookie_t *cookie)
```

Prototype of a [DDSDataWriterListener](#) on_sample_removed function.

Parameters

<i>listener_data</i>	<<out>> Data associated with the listener when the listener is set
<i>writer</i>	<<out>> Locally created DDSDataWriter that triggers the listener callback
<i>cookie</i>	<<out>> Cookie uniquely identifying the sample that was removed

6.2.9 Subscription Module

Contains the [DDSSubscriber](#) and [DDSDataReader](#) classes, as well as the [DDSSubscriberListener](#) and [DDSDataReaderListener](#) interfaces, and more generally, all that is needed on the subscription side.

Topics

- [Subscriber](#)
[DDSSubscriber](#) entity and associated elements
- [DataReader](#)
[DDSDataReader](#) entity and associated elements
- [Data Sample](#)
[DDS_SampleInfo](#), [DDS_SampleStateKind](#), [DDS_ViewStateKind](#), [DDS_InstanceStateKind](#) and associated elements

6.2.9.1 Detailed Description

Contains the [DDSSubscriber](#) and [DDSDataReader](#) classes, as well as the [DDSSubscriberListener](#) and [DDSDataReaderListener](#) interfaces, and more generally, all that is needed on the subscription side.

Defines the DDS subscription package.

6.2.9.2 Access to data samples

Data is made available to the application by the following operations on [DDSDataReader](#) objects: [FooDataReader::read](#), [FooDataReader::take](#), [FooDataReader::read_next_sample](#), [FooDataReader::take_next_sample](#), and the other variants of `read()` and `take()`.

The general semantics of the `read()` operation is that the application only gets access to the corresponding data (i.e. a precise instance value); the data remains the responsibility of RTI Connext DDS Micro and can be read again.

The semantics of the `take()` operations is that the application takes full responsibility for the data; that data will no longer be available locally to RTI Connext DDS Micro. Consequently, it is possible to access the same information multiple times only if all previous accesses were `read()` operations, not `take()`.

Each of these operations returns a collection of `Data` values and associated [DDS_SampleInfo](#) objects. Each data value represents an atom of data information (i.e., a value for one instance). This collection may contain samples related to the same or different instances (identified by the key). Multiple samples can refer to the same instance if the settings of the [HISTORY](#) QoS allow for it.

To return the memory back to the middleware, every `read()` or `take()` that retrieves a sequence of samples must be followed with a call to [FooDataReader::return_loan](#).

See also

[Interpretation of the SampleInfo](#)

6.2.9.2.1 Data access patterns

The application accesses data by means of the operations `read` or `take` on the [DDSDataReader](#). These operations return an ordered collection of `DataSamples` consisting of a [DDS_SampleInfo](#) part and a `Data` part.

The way RTI Connext DDS Micro builds the collection depends on QoS policies set on the [DDSDataReader](#) and [DDSSubscriber](#), as well as the `source_timestamp` of the samples, and the parameters passed to the `read()` / `take()` operations, namely:

- the desired sample states (any combination of [DDS_SampleStateKind](#))
- the desired view states (any combination of [DDS_ViewStateKind](#))
- the desired instance states (any combination of [DDS_InstanceStateKind](#))

The `read()` and `take()` operations are non-blocking and just deliver what is currently available that matches the specified states.

Once the data samples are available to the data readers, they can be read or taken by the application. The basic rule is that the application may do this in any order it wishes. This approach is very flexible and allows the application ultimate control.

6.2.9.3 Subscriber

[DDSSubscriber](#) entity and associated elements

Classes

- struct [DDS_SubscriberQos](#)
QoS policies supported by a [DDSSubscriber](#) entity.
- class [DDSSubscriberListener](#)
<<interface>> [DDSListener](#) for status about a subscriber.
- class [DDSSubscriber](#)
<<interface>> A subscriber is the object responsible for actually receiving data from a subscription.

Variables

- const struct [DDS_DataReaderQos](#) [DDS_DATAREADER_QOS_DEFAULT](#)
Special value for creating data reader with default QoS.

6.2.9.3.1 Detailed Description

[DDSSubscriber](#) entity and associated elements

6.2.9.3.2 Variable Documentation

DDS_DATAREADER_QOS_DEFAULT

```
const struct DDS\_DataReaderQos DDS_DATAREADER_QOS_DEFAULT [extern]
```

Special value for creating data reader with default QoS.

When used in [DDSSubscriber::create_datareader](#), this special value is used to indicate that the [DDSDataReader](#) should be created with the default [DDSDataReader](#) QoS.

NOTE: You cannot use this value to *get* the default QoS for a [DDSDataReader](#); for this purpose, use [DDSSubscriber::get_default_datareader_qos](#).

See also

[DDSSubscriber::create_datareader](#)

6.2.9.4 DataReader

[DDSDataReader](#) entity and associated elements

Classes

- struct `DDS_RequestedDeadlineMissedStatus`
`DDS_REQUESTED_DEADLINE_MISSED_STATUS`
- struct `DDS_LivelinessChangedStatus`
`DDS_LIVELINESS_CHANGED_STATUS`
- struct `DDS_RequestedIncompatibleQosStatus`
`DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS`
- struct `DDS_SampleRejectedStatus`
`DDS_SAMPLE_REJECTED_STATUS`
- struct `DDS_DataReaderInstanceReplacedStatus`
<<eXtension>> `DDS_INSTANCE_REPLACED_STATUS`.
- struct `DDS_SubscriptionMatchedStatus`
`DDS_SUBSCRIPTION_MATCHED_STATUS`
- struct `DDS_SampleLostStatus`
`DDS_SAMPLE_LOST_STATUS`
- struct `DDS_DataReaderQos`
QoS policies supported by a `DDSDataReader` entity.
- class `FooDataReader`
Declares the interface required to support a user data type-specific data reader.
- class `DDSDataReaderListener`
<<interface>> `DDSListener` for reader status.
- class `DDSDataReader`
<<interface>> *Allows the application to: (1) declare the data it wishes to receive (i.e. make a subscription) and (2) access the data received by the attached `DDSSubscriber`.*

Enumerations

- enum `DDS_SampleRejectedStatusKind` {
`DDS_NOT_REJECTED` , `DDS_REJECTED_BY_INSTANCES_LIMIT` , `DDS_REJECTED_BY_SAMPLES_LIMIT`
 , `DDS_REJECTED_BY_SAMPLES_PER_INSTANCE_LIMIT` ,
`DDS_REJECTED_BY_REMOTE_WRITERS_LIMIT` , `DDS_REJECTED_BY_SAMPLES_PER_REMOTE_WRITER_LIMIT`
 }
Kinds of reasons for rejecting a sample.
- enum `DDS_SampleLostStatusKind` {
`DDS_SAMPLE_LOST_NOT_LOST` , `DDS_SAMPLE_LOST_BY_DATAWRITER` , `DDS_SAMPLE_LOST_BY_MAX_SAMPLES_LIMIT`
 , `DDS_SAMPLE_LOST_BY_HISTORY_DEPTH_LIMIT` ,
`DDS_SAMPLE_LOST_BY_META_SAMPLE_LIMIT` , `DDS_SAMPLE_LOST_BY_NOT_READ_ON_CACHE_DELETION`
 , `DDS_SAMPLE_LOST_BY_MANAGEDPOOL_OVERWRITE` }
<<eXtension>> *Kinds of reasons a sample was lost.*

6.2.9.4.1 Detailed Description

`DDSDataReader` entity and associated elements

6.2.9.4.2 Enumeration Type Documentation

`DDS_SampleRejectedStatusKind`

enum `DDS_SampleRejectedStatusKind`

Kinds of reasons for rejecting a sample.

Enumerator

DDS_NOT_REJECTED	Samples are never rejected. See also DDS_ResourceLimitsQosPolicy
DDS_REJECTED_BY_INSTANCES_LIMIT	A resource limit on the number of instances was reached. See also DDS_ResourceLimitsQosPolicy
DDS_REJECTED_BY_SAMPLES_LIMIT	A resource limit on the number of samples was reached. See also DDS_ResourceLimitsQosPolicy
DDS_REJECTED_BY_SAMPLES_PER_INSTANCE↔ _LIMIT	A resource limit on the number of samples per instance was reached. See also DDS_ResourceLimitsQosPolicy
DDS_REJECTED_BY_REMOTE_WRITERS_LIMIT	A resource limit on the number of remote writers from which a DDSDataReader may read was reached. See also DDS_DataReaderResourceLimitsQosPolicy
DDS_REJECTED_BY_SAMPLES_PER_REMOTE↔ WRITER_LIMIT	A resource limit on the number of samples from a given remote writer that a DDSDataReader may store was reached. See also DDS_DataReaderResourceLimitsQosPolicy

DDS_SampleLostStatusKindenum [DDS_SampleLostStatusKind](#)<<*eXtension*>> Kinds of reasons a sample was lost.

Enumerator

DDS_SAMPLE_LOST_NOT_LOST	Sample was not lost, normal removal. See also DDS_ResourceLimitsQosPolicy
--------------------------	---

Enumerator

DDS_SAMPLE_LOST_BY_DATAWRITER	Sample was lost by the DDSDataWriter, it was never received by the DDSDataReader. See also DDS_ResourceLimitsQosPolicy
DDS_SAMPLE_LOST_BY_MAX_SAMPLES_LIMIT	A sample was received by the DDSDataReader but lost because the max_samples resource was exceeded and no samples could be freed to receive it. See also DDS_ResourceLimitsQosPolicy
DDS_SAMPLE_LOST_BY_HISTORY_DEPTH_LIMIT	A sample was received by the DataReader but lost because the max depth was reached for the instance and no samples could be removed from the history. For example, if all the samples in the DataReader cache are loaned by the application, they cannot be removed. See also DDS_ResourceLimitsQosPolicy
DDS_SAMPLE_LOST_BY_META_SAMPLE_LIMIT	A meta sample was received by the DataReader but lost because the meta sample for the instance was loaned by the application and could not be reused. See also DDS_ResourceLimitsQosPolicy
DDS_SAMPLE_LOST_BY_NOT_READ_ON_CACHE_DELETION ↵	The DataReader has samples in the cache that were not seen by the application and were lost when the DataReader was deleted. See also DDS_ResourceLimitsQosPolicy
DDS_SAMPLE_LOST_BY_MANAGEDPOOL_OVERWRITE ↵	The DataReader has samples that may have been overwritten. See also DDS_ResourceLimitsQosPolicy

6.2.9.5 Data Sample

[DDS_SampleInfo](#), [DDS_SampleStateKind](#), [DDS_ViewStateKind](#), [DDS_InstanceStateKind](#) and associated elements

Topics

- [Sample States](#)

- [DDS_SampleStateKind](#) and associated elements
- View States
 - [DDS_ViewStateKind](#) and associated elements
- Instance States
 - [DDS_InstanceStateKind](#) and associated elements

Classes

- struct [DDS_SampleInfo](#)
Information that accompanies each sample that is read or taken.
- struct [DDS_SampleInfoSeq](#)
Declares IDL sequence `< DDS_SampleInfo >` .

6.2.9.5.1 Detailed Description

[DDS_SampleInfo](#), [DDS_SampleStateKind](#), [DDS_ViewStateKind](#), [DDS_InstanceStateKind](#) and associated elements

6.2.9.5.2 Sample States

[DDS_SampleStateKind](#) and associated elements

Typedefs

- typedef [DDS_UnsignedLong](#) [DDS_SampleStateMask](#)
A bit-mask (list) of sample states, i.e. [DDS_SampleStateKind](#).

Enumerations

- enum [DDS_SampleStateKind](#) { [DDS_READ_SAMPLE_STATE](#) = 0x0001 << 0 , [DDS_NOT_READ_SAMPLE_STATE](#) = 0x0001 << 1 }
- Indicates whether or not a sample has ever been read.

Variables

- const [DDS_SampleStateMask](#) [DDS_ANY_SAMPLE_STATE](#)
Any sample state [DDS_READ_SAMPLE_STATE](#) | [DDS_NOT_READ_SAMPLE_STATE](#).

Detailed Description

[DDS_SampleStateKind](#) and associated elements

Typedef Documentation

DDS_SampleStateMask

```
typedef DDS_UnsignedLong DDS_SampleStateMask
```

A bit-mask (list) of sample states, i.e. [DDS_SampleStateKind](#).

Enumeration Type Documentation

DDS_SampleStateKind

```
enum DDS_SampleStateKind
```

Indicates whether or not a sample has ever been read.

For each sample received, the middleware internally maintains a sample_state relative to each [DDSDataReader](#). The sample state can be either:

- [DDS_READ_SAMPLE_STATE](#)
- [DDS_NOT_READ_SAMPLE_STATE](#)

The sample state will, in general, be different for each sample in the collection returned by read or take.

Enumerator

DDS_READ_SAMPLE_STATE	Sample has been read. Indicates that the DDSDataReader has already accessed that sample by means of a read or take operation.
DDS_NOT_READ_SAMPLE_STATE	Sample has not been read. Indicates that the DDSDataReader has not accessed that sample before.

Variable Documentation

DDS_ANY_SAMPLE_STATE

```
const DDS_SampleStateMask DDS_ANY_SAMPLE_STATE [extern]
```

Any sample state [DDS_READ_SAMPLE_STATE](#) | [DDS_NOT_READ_SAMPLE_STATE](#).

6.2.9.5.3 View States

[DDS_ViewStateKind](#) and associated elements

Typedefs

- typedef [DDS_UnsignedLong](#) [DDS_ViewStateMask](#)
A bit-mask (list) of view states, i.e. [DDS_ViewStateKind](#).

Enumerations

- enum [DDS_ViewStateKind](#) { [DDS_NEW_VIEW_STATE](#) = 0x0001 << 0 , [DDS_NOT_NEW_VIEW_STATE](#) = 0x0001 << 1 }
- Indicates whether or not an instance is new.

Variables

- const [DDS_ViewStateMask](#) [DDS_ANY_VIEW_STATE](#)
Any view state [DDS_NEW_VIEW_STATE](#) | [DDS_NOT_NEW_VIEW_STATE](#).

Detailed Description

[DDS_ViewStateKind](#) and associated elements

Typedef Documentation

DDS_ViewStateMask

```
typedef DDS\_UnsignedLong DDS\_ViewStateMask
```

A bit-mask (list) of view states, i.e. [DDS_ViewStateKind](#).

Enumeration Type Documentation

DDS_ViewStateKind

```
enum DDS\_ViewStateKind
```

Indicates whether or not an instance is new.

For each instance (identified by the key), the middleware internally maintains a view state relative to each [DDSDataReader](#). The view state can be either:

- [DDS_NEW_VIEW_STATE](#)
- [DDS_NOT_NEW_VIEW_STATE](#)

The view_state available in the [DDS_SampleInfo](#) is a snapshot of the view state of the instance relative to the [DDSDataReader](#) used to access the samples at the time the collection was obtained (i.e. at the time read or take was called). The view_state is therefore the same for all samples in the returned collection that refer to the same instance.

Once an instance has been detected as not having any "live" writers and all the samples associated with the instance are "taken" from the [DDSDataReader](#), the middleware can reclaim all local resources regarding the instance. Future samples will be treated as "never seen."

Enumerator

DDS_NEW_VIEW_STATE	New instance. This latest generation of the instance has not previously been accessed. Indicates that either this is the first time that the DDSDataReader has ever accessed samples of that instance, or else that the DDSDataReader has accessed previous samples of the instance, but the instance has since been reborn (i.e. become "not alive" and then "alive" again).
DDS_NOT_NEW_VIEW_STATE	Not a new instance. This latest generation of the instance has previously been accessed. Indicates that the DDSDataReader has already accessed samples of the same instance and that the instance has not been reborn since.

Variable Documentation

DDS_ANY_VIEW_STATE

```
const DDS\_ViewStateMask DDS_ANY_VIEW_STATE [extern]
```

Any view state [DDS_NEW_VIEW_STATE](#) | [DDS_NOT_NEW_VIEW_STATE](#).

6.2.9.5.4 Instance States

[DDS_InstanceStateKind](#) and associated elements

Typedefs

- typedef [DDS_UnsignedLong](#) [DDS_InstanceStateMask](#)
A bit-mask (list) of instance states, i.e. [DDS_InstanceStateKind](#).

Enumerations

- enum [DDS_InstanceStateKind](#) { [DDS_ALIVE_INSTANCE_STATE](#) = 0x0001 << 0, [DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE](#) = 0x0001 << 1, [DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE](#) = 0x0001 << 2 }
- Indicates if the samples are from a live [DDSDataWriter](#) or not.

Variables

- const [DDS_InstanceStateMask](#) [DDS_ANY_INSTANCE_STATE](#)
Any instance state [ALIVE_INSTANCE_STATE](#) | [NOT_ALIVE_DISPOSED_INSTANCE_STATE](#) | [NOT_ALIVE_NO_WRITERS_INSTANCE_STATE](#).
- const [DDS_InstanceStateMask](#) [DDS_NOT_ALIVE_INSTANCE_STATE](#)
Not alive instance state [NOT_ALIVE_DISPOSED_INSTANCE_STATE](#) | [NOT_ALIVE_NO_WRITERS_INSTANCE_STATE](#).

Detailed Description

[DDS_InstanceStateKind](#) and associated elements

Typedef Documentation

DDS_InstanceStateMask

```
typedef DDS_UnsignedLong DDS_InstanceStateMask
```

A bit-mask (list) of instance states, i.e. [DDS_InstanceStateKind](#).

Enumeration Type Documentation

DDS_InstanceStateKind

```
enum DDS_InstanceStateKind
```

Indicates if the samples are from a live [DDSDataWriter](#) or not.

For each instance, the middleware internally maintains an instance state. The instance state can be:

- [DDS_ALIVE_INSTANCE_STATE](#)
- [DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE](#)
- [DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE](#)

The precise behavior events that cause the instance state to change depends on the setting of the OWNERSHIP QoS:

- If [OWNERSHIP](#) is set to [DDS_EXCLUSIVE_OWNERSHIP_QOS](#), then the instance state becomes [DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE](#) only if the [DDSDataWriter](#) that "owns" the instance explicitly disposes it. The instance state becomes [DDS_ALIVE_INSTANCE_STATE](#) again only if the [DDSDataWriter](#) that owns the instance writes it.
- If [OWNERSHIP](#) is set to [DDS_SHARED_OWNERSHIP_QOS](#), then the instance state becomes [DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE](#) if any [DDSDataWriter](#) explicitly disposes the instance. The instance state becomes [DDS_ALIVE_INSTANCE_STATE](#) as soon as any [DDSDataWriter](#) writes the instance again.

The instance state available in the [DDS_SampleInfo](#) is a snapshot of the instance state of the instance at the time the collection was obtained (i.e. at the time read or take was called). The instance state is therefore the same for all samples in the returned collection that refer to the same instance.

Enumerator

DDS_ALIVE_INSTANCE_STATE	Instance is currently in existence. Indicates that (a) samples have been received for the instance, (b) there are live DDSDDataWriter entities writing the instance, and (c) the instance has not been explicitly disposed (or else more samples have been received after it was disposed).
DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE	Not alive disposed instance. The instance has been disposed by a DDSDDataWriter . Indicates the instance was explicitly disposed by a DDSDDataWriter by means of the dispose operation.
DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE	Not alive no writers for instance. None of the DDSDDataWriter objects are currently alive (according to the LIVELINESS) are writing the instance. Indicates the instance has been declared as "not alive" by the DDSDDataReader because it detected that there are no live DDSDDataWriter entities writing that instance.

Variable Documentation

DDS_ANY_INSTANCE_STATE

```
const DDS\_InstanceStateMask DDS_ANY_INSTANCE_STATE [extern]
```

Any instance state [ALIVE_INSTANCE_STATE](#) | [NOT_ALIVE_DISPOSED_INSTANCE_STATE](#) | [NOT_ALIVE_NO_WRITERS_INSTANCE_STATE](#).

DDS_NOT_ALIVE_INSTANCE_STATE

```
const DDS\_InstanceStateMask DDS_NOT_ALIVE_INSTANCE_STATE [extern]
```

Not alive instance state [NOT_ALIVE_DISPOSED_INSTANCE_STATE](#) | [NOT_ALIVE_NO_WRITERS_INSTANCE_STATE](#).

6.2.10 Infrastructure Module

Defines the abstract classes and the interfaces that are refined by the other modules. Contains common definitions such as return codes, status values, and QoS policies.

Topics

- [Time Support](#)
Time and duration types and defines.
- [GUID Support](#)
GUID type and defines.
- [Sequence Number Support](#)
Sequence number type and defines.
- [Return Codes](#)
Types of return codes.
- [Status Kinds](#)
Kinds of communication status.
- [QoS Policies](#)
Quality of Service (QoS) policies.
- [Conditions and WaitSets](#)
[DDSCondition](#) and [DDSWaitSet](#) and related items.
- [Entity Support](#)
[DDSEntity](#), [DDSListener](#) and related items.
- [Sequence Support](#)
Defines sequence interface and primitive data types sequences.
- [Exception codes](#)
Exception codes.
- [Type Support](#)
Type Plugin.
- [String Support](#)
String creation, cloning, assignment, and deletion.

Classes

- struct [DDS_SampleIdentity_t](#)
Data structure used to provide explicit identity information about a published data sample.
- struct [DDS_WriteParams_t](#)
Data structure used to provide custom parameters to a [DDSDataWriter](#)'s write operation.

Macros

- #define [DDS_WRITEPARAMS_DEFAULT](#)
Default values used to initialize instances of [DDS_WriteParams_t](#).

6.2.10.1 Detailed Description

Defines the abstract classes and the interfaces that are refined by the other modules. Contains common definitions such as return codes, status values, and QoS policies.

Defines the DDS infrastructure package.

6.2.10.2 Macro Definition Documentation

DDS_WRITEPARAMS_DEFAULT

```
#define DDS_WRITEPARAMS_DEFAULT
```

Value:

```
{ \
    DDS_SAMPLE_IDENTITY_UNKNOWN, \
    DDS_SAMPLE_IDENTITY_UNKNOWN, \
    DDS_TIME_ZERO, \
    DDS_HANDLE_NIL_NATIVE \
}
```

Default values used to initialize instances of [DDS_WriteParams_t](#).

6.2.10.3 Time Support

Time and duration types and defines.

Classes

- struct [DDS_Time_t](#)
Type for time representation.
- struct [DDS_Duration_t](#)
Type for duration representation.

Macros

- #define [DDS_TIME_ZERO](#)
The default instant in time: zero seconds and zero nanoseconds.
- #define [DDS_TIME_INVALID_SEC](#)
A sentinel indicating an invalid second of time.
- #define [DDS_TIME_INVALID_NSEC](#)
A sentinel indicating an invalid nanosecond of time.
- #define [DDS_TIME_INVALID](#)
A sentinel indicating an invalid time.

Functions

- [DDS_Boolean DDS_Time_is_zero](#) (const struct [DDS_Time_t](#) *time)
Check if time is zero.
- [DDS_Boolean DDS_Duration_is_infinite](#) (const struct [DDS_Duration_t](#) *duration)
- [DDS_Boolean DDS_Duration_equal](#) (const struct [DDS_Duration_t](#) *self, const struct [DDS_Duration_t](#) *other)
- [DDS_Boolean DDS_Duration_is_zero](#) (const struct [DDS_Duration_t](#) *duration)

Variables

- const `DDS_Long DDS_DURATION_INFINITE_SEC`
An infinite second period of time.
- const `DDS_UnsignedLong DDS_DURATION_INFINITE_NSEC`
An infinite nanosecond period of time.
- const struct `DDS_Duration_t DDS_DURATION_INFINITE`
An infinite period of time.
- const `DDS_Long DDS_DURATION_ZERO_SEC`
A zero-length second period of time.
- const `DDS_UnsignedLong DDS_DURATION_ZERO_NSEC`
A zero-length nanosecond period of time.
- const struct `DDS_Duration_t DDS_DURATION_ZERO`
A zero-length period of time.

6.2.10.3.1 Detailed Description

Time and duration types and defines.

6.2.10.3.2 Macro Definition Documentation

DDS_TIME_ZERO

```
#define DDS_TIME_ZERO
```

The default instant in time: zero seconds and zero nanoseconds.

DDS_TIME_INVALID_SEC

```
#define DDS_TIME_INVALID_SEC
```

A sentinel indicating an invalid second of time.

DDS_TIME_INVALID_NSEC

```
#define DDS_TIME_INVALID_NSEC
```

A sentinel indicating an invalid nanosecond of time.

DDS_TIME_INVALID

```
#define DDS_TIME_INVALID
```

A sentinel indicating an invalid time.

6.2.10.3.3 Function Documentation

DDS_Time_is_zero()

```
DDS_Boolean DDS_Time_is_zero (  
    const struct DDS_Time_t * time)
```

Check if time is zero.

Returns

[DDS_BOOLEAN_TRUE](#) if the given time is equal to [DDS_TIME_ZERO](#) or [DDS_BOOLEAN_FALSE](#) otherwise.

DDS_Duration_is_infinite()

```
DDS_Boolean DDS_Duration_is_infinite (  
    const struct DDS_Duration_t * duration)
```

Returns

[DDS_BOOLEAN_TRUE](#) if the given duration is of infinite length.

References [RTI_INT32](#), and [RTI_UINT32](#).

DDS_Duration_equal()

```
DDS_Boolean DDS_Duration_equal (  
    const struct DDS_Duration_t * self,  
    const struct DDS_Duration_t * other)
```

Parameters

<i>self</i>	<<in>> Duration to compare. Cannot be NULL.
<i>other</i>	<<in>> Duration to be compared to this one. Cannot be NULL.

Returns

[DDS_BOOLEAN_TRUE](#) if the two objects contain the same value, [DDS_BOOLEAN_FALSE](#) otherwise.

References [DDS_DURATION_ZERO](#), [DDS_DURATION_ZERO_NSEC](#), and [DDS_DURATION_ZERO_SEC](#).

DDS_Duration_is_zero()

```
DDS_Boolean DDS_Duration_is_zero (
    const struct DDS_Duration_t * duration)
```

Returns

[DDS_BOOLEAN_TRUE](#) if the given duration is of zero length.

6.2.10.3.4 Variable Documentation

DDS_DURATION_INFINITE_SEC

```
const DDS_Long DDS_DURATION_INFINITE_SEC [extern]
```

An infinite second period of time.

DDS_DURATION_INFINITE_NSEC

```
const DDS_UnsignedLong DDS_DURATION_INFINITE_NSEC [extern]
```

An infinite nanosecond period of time.

DDS_DURATION_INFINITE

```
const struct DDS_Duration_t DDS_DURATION_INFINITE [extern]
```

An infinite period of time.

DDS_DURATION_ZERO_SEC

```
const DDS_Long DDS_DURATION_ZERO_SEC [extern]
```

A zero-length second period of time.

Referenced by [DDS_Duration_equal\(\)](#).

DDS_DURATION_ZERO_NSEC

```
const DDS_UnsignedLong DDS_DURATION_ZERO_NSEC [extern]
```

A zero-length nanosecond period of time.

Referenced by [DDS_Duration_equal\(\)](#).

DDS_DURATION_ZERO

```
const struct DDS_Duration_t DDS_DURATION_ZERO [extern]
```

A zero-length period of time.

Referenced by [DDS_Duration_equal\(\)](#).

6.2.10.4 GUID Support

GUID type and defines.

Classes

- struct [DDS_GUID_t](#)
Type for GUID (Global Unique Identifier) representation.

Typedefs

- typedef struct DDS_GUID_t [DDS_GUID_t](#)
Type for GUID (Global Unique Identifier) representation.

Variables

- const struct [DDS_GUID_t](#) [DDS_GUID_UNKNOWN](#)
Unknown GUID.

6.2.10.4.1 Detailed Description

GUID type and defines.

6.2.10.4.2 Typedef Documentation

DDS_GUID_t

```
typedef struct DDS_GUID_t DDS_GUID_t
```

Type for *GUID* (Global Unique Identifier) representation.

Represents a 128 bit GUID.

6.2.10.4.3 Variable Documentation

DDS_GUID_UNKNOWN

```
const struct DDS_GUID_t DDS_GUID_UNKNOWN [extern]
```

Unknown GUID.

6.2.10.5 Sequence Number Support

Sequence number type and defines.

Classes

- struct `DDS_SequenceNumber_t`
Type for sequence number representation.

Functions

- int `DDS_SequenceNumber_compare` (const struct `DDS_SequenceNumber_t` *sn1, const struct `DDS_SequenceNumber_t` *sn2)
Compares two sequence numbers.

6.2.10.5.1 Detailed Description

Sequence number type and defines.

6.2.10.5.2 Function Documentation

DDS_SequenceNumber_compare()

```
int DDS_SequenceNumber_compare (
    const struct DDS_SequenceNumber_t * sn1,
    const struct DDS_SequenceNumber_t * sn2)
```

Compares two sequence numbers.

Parameters

<i>sn1</i>	<<in>> Sequence number to compare. Cannot be NULL.
<i>sn2</i>	<<in>> Sequence number to compare. Cannot be NULL.

Returns

If the two sequence numbers are equal, the method returns 0. If sn1 is greater than sn2 the method returns a positive number; otherwise, it returns a negative number.

6.2.10.6 Return Codes

Types of return codes.

Enumerations

- `enum DDS_ReturnCode_t {
 DDS_RETCODE_OK , DDS_RETCODE_ERROR , DDS_RETCODE_UNSUPPORTED , DDS_RETCODE_BAD_PARAMETER
 ,
 DDS_RETCODE_PRECONDITION_NOT_MET , DDS_RETCODE_OUT_OF_RESOURCES , DDS_RETCODE_NOT_ENABLED
 , DDS_RETCODE_IMMUTABLE_POLICY ,
 DDS_RETCODE_INCONSISTENT_POLICY , DDS_RETCODE_ALREADY_DELETED , DDS_RETCODE_TIMEOUT
 , DDS_RETCODE_NO_DATA ,
 DDS_RETCODE_ILLEGAL_OPERATION , DDS_RETCODE_NOT_ALLOWED_BY_SEC }`

Type for return codes.

6.2.10.6.1 Detailed Description

Types of return codes.

6.2.10.6.2 Standard Return Codes

Any operation with return type `DDS_ReturnCode_t` may return `DDS_RETCODE_OK`, `DDS_RETCODE_ERROR` or `DDS_RETCODE_ILLEGAL_OPERATION`. Any operation that takes one or more input parameters may additionally return `DDS_RETCODE_BAD_PARAMETER`. Any operation on an object created from any of the factories may additionally return `DDS_RETCODE_ALREADY_DELETED`. Any operation that is stated as optional may additionally return `DDS_RETCODE_UNSUPPORTED`.

Thus, the standard return codes are:

- `DDS_RETCODE_ERROR`
- `DDS_RETCODE_ILLEGAL_OPERATION`
- `DDS_RETCODE_ALREADY_DELETED`
- `DDS_RETCODE_BAD_PARAMETER`
- `DDS_RETCODE_UNSUPPORTED`

Operations that may return any of the additional return codes will state so explicitly.

6.2.10.6.3 Enumeration Type Documentation

`DDS_ReturnCode_t`

`enum DDS_ReturnCode_t`

Type for return codes.

Errors are modeled as operation return codes of this type.

Enumerator

DDS_RETCODE_OK	Successful return.
DDS_RETCODE_ERROR	Generic, unspecified error.
DDS_RETCODE_UNSUPPORTED	Unsupported operation. Can only be returned by operations that are unsupported.
DDS_RETCODE_BAD_PARAMETER	Illegal parameter value. The value of the parameter that is passed in has an illegal value. Things that fall into this category include NULL parameters and parameter values that are out of range.
DDS_RETCODE_PRECONDITION_NOT_MET	A pre-condition for the operation was not met. The system is not in the expected state when the method is called, or the parameter itself is not in the expected state when the method is called.
DDS_RETCODE_OUT_OF_RESOURCES	RTI Connex DDS Micro ran out of the resources needed to complete the operation.
DDS_RETCODE_NOT_ENABLED	Operation invoked on a DDSEntity that is not yet enabled.
DDS_RETCODE_IMMUTABLE_POLICY	Application attempted to modify an immutable QoS policy.
DDS_RETCODE_INCONSISTENT_POLICY	Application specified a set of QoS policies that are not consistent with each other.
DDS_RETCODE_ALREADY_DELETED	The object target of this operation has already been deleted.
DDS_RETCODE_TIMEOUT	The operation timed out.
DDS_RETCODE_NO_DATA	Indicates a transient situation where the operation did not return any data but there is no inherent error.
DDS_RETCODE_ILLEGAL_OPERATION	The operation was called under improper circumstances. An operation was invoked on an inappropriate object or at an inappropriate time. This return code is similar to DDS_RETCODE_PRECONDITION_NOT_MET , except that there is no precondition that could be changed to make the operation succeed.
DDS_RETCODE_NOT_ALLOWED_BY_SEC	An operation on the DDS API that fails because the security plugins do not allow it. An operation on the DDS API that fails because the security plugins do not allow it.

6.2.10.7 Status Kinds

Kinds of communication status.

Macros

- `#define DDS_STATUS_MASK_NONE`
No bits are set.
- `#define DDS_STATUS_MASK_ALL`
All bits are set.

Typedefs

- typedef [DDS_UnsignedLong](#) [DDS_StatusMask](#)
A bit-mask (list) of concrete status types, i.e. [DDS_StatusKind](#)[].

Enumerations

- enum [DDS_StatusKind](#) {
 [DDS_INCONSISTENT_TOPIC_STATUS](#) , [DDS_OFFERED_DEADLINE_MISSED_STATUS](#) , [DDS_REQUESTED_DEADLINE_MISSED_STATUS](#) ,
 [DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#) ,
 [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#) , [DDS_SAMPLE_LOST_STATUS](#) , [DDS_SAMPLE_REJECTED_STATUS](#) ,
 [DDS_DATA_ON_READERS_STATUS](#) ,
 [DDS_DATA_AVAILABLE_STATUS](#) , [DDS_LIVELINESS_LOST_STATUS](#) , [DDS_LIVELINESS_CHANGED_STATUS](#) ,
 [DDS_PUBLICATION_MATCHED_STATUS](#) ,
 [DDS_SUBSCRIPTION_MATCHED_STATUS](#) , [DDS_INSTANCE_REPLACED_STATUS](#) , [DDS_RELIABLE_READER_ACTIVITY_CHANGED_STATUS](#) ,
 [DDS_DATA_WRITER_SAMPLE_REMOVED_STATUS](#) }
Type for status kinds.

6.2.10.7.1 Detailed Description

Kinds of communication status.

Entity:

[DDSEntity](#)

QoS:

[QoS Policies](#)

Listener:

[DDSListener](#)

Each concrete [DDSEntity](#) is associated with a set of Status objects whose value represents the communication status of that entity. Each status value can be accessed with a corresponding method on the [DDSEntity](#).

RTI Connext DDS Micro supports a subset of these statuses.

When these status values change, the proper [DDSListener](#) objects are invoked to asynchronously inform the application.

An application is notified of communication status by means of the [DDSListener](#) mechanism.

The statuses may be classified into:

- *read communication statuses*: i.e., those that are related to arrival of data, namely [DDS_DATA_ON_READERS_STATUS](#) and [DDS_DATA_AVAILABLE_STATUS](#).
- *plain communication statuses*: i.e., all the others.

Read communication statuses are treated slightly differently than the others because they don't change independently. In other words, at least two changes will appear at the same time ([DDS_DATA_ON_READERS_STATUS](#) and [DDS_DATA_AVAILABLE_STATUS](#)) and even several of the last kind may be part of the set. This 'grouping' has to be communicated to the application.

For each plain communication status, there is a corresponding structure to hold the status value. These values contain the information related to the change of status, as well as information related to the statuses themselves (e.g., contains cumulative counts).

6.2.10.7.2 Changes in Status

Associated with each one of an [DDSEntity](#)'s communication status is a logical `StatusChangedFlag`. This flag indicates whether that particular communication status has changed since the last time the status was read by the application. The way the status changes is slightly different for the Plain Communication Status and the Read Communication status.

Changes in plain communication status

For the plain communication status, the `StatusChangedFlag` flag is initially set to `FALSE`. It becomes `TRUE` whenever the plain communication status changes and it is reset to `DDS_BOOLEAN_FALSE` each time the application accesses the plain communication status via the proper `get_<plain communication status>()` operation on the [DDSEntity](#).

The communication status is also reset to `FALSE` whenever the associated listener operation is called as the listener implicitly accesses the status which is passed as a parameter to the operation. The fact that the status is reset prior to calling the listener means that if the application calls the `get_<plain communication status>` from inside the listener it will see the status already reset.

An exception to this rule is when the associated listener is the 'nil' listener. The 'nil' listener is treated as a NOOP and the act of calling the 'nil' listener does not reset the communication status.

Changes in read communication status

For the read communication status, the `StatusChangedFlag` flag is initially set to `FALSE`. The `StatusChangedFlag` becomes `TRUE` when either a data-sample arrives or else the [DDS_ViewStateKind](#), [DDS_SampleStateKind](#), or [DDS_InstanceStateKind](#) of any existing sample changes for any reason other than a call to [FooDataReader::read_next_sample](#), or [FooDataReader::take_next_sample](#). Specifically, any of the following events will cause the `StatusChangedFlag` to become `TRUE`:

- The arrival of new data.
- A change in the [DDS_InstanceStateKind](#) of a contained instance. This can be caused by either:
 - The arrival of the notification that an instance has been disposed by:
 - * the [DDSDDataWriter](#) that owns it if `OWNERSHIP` QoS kind= `DDS_EXCLUSIVE_OWNERSHIP_QOS`
 - * or by any [DDSDDataWriter](#) if `OWNERSHIP` QoS kind= `DDS_SHARED_OWNERSHIP_QOS`
 - The loss of liveliness of the [DDSDDataWriter](#) of an instance for which there is no other [DDSDDataWriter](#).
 - The arrival of the notification that an instance has been unregistered by the only [DDSDDataWriter](#) that is known to be writing the instance.

Depending on the kind of `StatusChangedFlag`, the flag transitions to `FALSE` again as follows:

- The [DDS_DATA_AVAILABLE_STATUS](#) `StatusChangedFlag` becomes `FALSE` when either the corresponding listener operation (`on_data_available`) is called or the `read_next_sample` or `take_next_sample` operation is called on the associated [DDSDDataReader](#).

- The `DDS_DATA_ON_READERS_STATUS` `StatusChangedFlag` becomes FALSE when any of the following events occurs:
 - The corresponding listener operation (`on_data_on_readers`) is called.
 - The `on_data_available` listener operation is called on any `DDSDataReader` belonging to the `DDSSubscriber`.
 - The read or take operation (or their variants) is called on any `DDSDataReader` belonging to the `DDSSubscriber`.

"Changes in `StatusChangedFlag` for read communication status"

See also

[DDSListener](#)

6.2.10.7.3 Macro Definition Documentation

DDS_STATUS_MASK_NONE

```
#define DDS_STATUS_MASK_NONE
```

No bits are set.

DDS_STATUS_MASK_ALL

```
#define DDS_STATUS_MASK_ALL
```

All bits are set.

6.2.10.7.4 Typedef Documentation

DDS_StatusMask

```
typedef DDS_UnsignedLong DDS_StatusMask
```

A bit-mask (list) of concrete status types, i.e. `DDS_StatusKind`[].

The bit-mask is an efficient and compact representation of a fixed-length list of `DDS_StatusKind` values.

Bits in the mask correspond to different statuses. You can choose which changes in status will trigger a callback by setting the corresponding status bits in this bit-mask and installing callbacks for each of those statuses.

The bits that are true indicate that the listener will be called back for changes in the corresponding status.

For example:

```
DDS_StatusMask mask = DDS_REQUESTED_DEADLINE_MISSED_STATUS |  
                     DDS_DATA_AVAILABLE_STATUS;  
datareader->set_listener(listener, mask);
```

or

```
DDS_StatusMask mask = DDS_REQUESTED_DEADLINE_MISSED_STATUS |  
                     DDS_DATA_AVAILABLE_STATUS;  
datareader = subscriber->create_datareader(topic,  
DDS_DATAREADER_QOS_DEFAULT,  
listener, mask);
```

6.2.10.7.5 Enumeration Type Documentation

DDS_StatusKind

enum [DDS_StatusKind](#)

Type for *status* kinds.

Each concrete [DDSEntity](#) is associated with a set of *Status objects whose values represent the communication status of that [DDSEntity](#).

The communication statuses whose changes can be communicated to the application depend on the [DDSEntity](#).

Each status value can be accessed with a corresponding method on the [DDSEntity](#). The changes on these status values cause activation of the corresponding trigger invocation of the proper [DDSListener](#) objects to asynchronously inform the application.

See also

[DDSEntity](#), [DDSListener](#)

Enumerator

DDS_INCONSISTENT_TOPIC_STATUS	Another topic exists with the same name but different characteristics. Entity: DDSTopic Status: DDS_InconsistentTopicStatus Listener: DDSTopicListener
DDS_OFFERED_DEADLINE_MISSED_STATUS	The deadline that the DDSDDataWriter has committed to through its DDS_DeadlineQoSPolicy was not respected for a specific instance. Entity: DDSDDataWriter QoS: DEADLINE Status: DDS_OfferedDeadlineMissedStatus Listener: DDSDDataWriterListener
RTI Connext Micro C++ API Version 4.3.0 ER1 Copyright © Mon Mar 30 2026 Real-Time Innovations, Inc	

Enumerator

DDS_REQUESTED_DEADLINE_MISSED_STATUS	The deadline that the DDSDataReader was expecting through its DDS_DeadlineQosPolicy was not respected for a specific instance. Entity: DDSDataReader QoS: DEADLINE Status: DDS_RequestedDeadlineMissedStatus Listener: DDSDataReaderListener
DDS_OFFERED_INCOMPATIBLE_QOS_STATUS	A QosPolicy value was incompatible with what was requested. Entity: DDSDataWriter Status: DDS_OfferedIncompatibleQosStatus Listener: DDSDataWriterListener
DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS	A QosPolicy value was incompatible with what is offered. Entity: DDSDataReader Status: DDS_RequestedIncompatibleQosStatus Listener: DDSDataReaderListener

Enumerator

DDS_SAMPLE_LOST_STATUS	A sample has been lost and was never received. Entity: DDSDataReader Status: DDS_SampleLostStatus Listener: DDSDataReaderListener
DDS_SAMPLE_REJECTED_STATUS	A received sample has been rejected. Entity: DDSDataReader QoS: RESOURCE_LIMITS Status: DDS_SampleRejectedStatus Listener: DDSDataReaderListener
DDS_DATA_ON_READERS_STATUS	New data is available. Entity: DDSSubscriber Listener: DDSSubscriberListener
DDS_DATA_AVAILABLE_STATUS	One or more new data samples have been received. Entity: DDSDataReader Listener: DDSDataReaderListener

Enumerator

DDS_LIVELINESS_LOST_STATUS	The liveliness commitment of the DDSDataWriter through its DDS_LivelinessQosPolicy was not respected, so DDSDataReader entities will consider the DDSDataWriter as no longer alive. Entity: DDSDataWriter QoS: LIVELINESS Status: DDS_LivelinessLostStatus Listener: DDSDataWriterListener
DDS_LIVELINESS_CHANGED_STATUS	The liveliness of one or more DDSDataWriter entities that were writing instances read through the DDSDataReader has changed. Some DDSDataWriter entities have become alive or not_alive. Entity: DDSDataReader QoS: LIVELINESS Status: DDS_LivelinessChangedStatus Listener: DDSDataReaderListener

Enumerator

DDS_PUBLICATION_MATCHED_STATUS	The DDSDataWriter has found a DDSDataReader that matches the DDSTopic and has compatible QoS. Entity: DDSDataWriter Status: DDS_PublicationMatchedStatus Listener: DDSDataWriterListener
DDS_SUBSCRIPTION_MATCHED_STATUS	The DDSDataReader has found a DDSDataWriter that matches the DDSTopic and has compatible QoS. Entity: DDSDataReader Status: DDS_SubscriptionMatchedStatus Listener: DDSDataReaderListener
DDS_INSTANCE_REPLACED_STATUS	The DDSDataReader has replaced an instance. Entity: DDSDataReader Status: DDS_DataReaderInstanceReplacedStatus Listener: DDSDataReaderListener

Enumerator

DDS_RELIABLE_READER_ACTIVITY_CHANGED_STATUS	The DDSDDataWriter has detected a change of activity in a reliable DDSDDataReader. Entity: DDSDDataWriter Status: DDS_ReliableReaderActivityChangedStatus Listener: DDSDDataWriterListener
DDS_DATA_WRITER_SAMPLE_REMOVED_STATUS	The DDSDDataWriter has detected a change of activity in a reliable DDSDDataReader. Entity: DDSDDataWriter Status: DDS_ReliableReaderActivityChangedStatus Listener: DDSDDataWriterListener

6.2.10.8 QoS Policies

Quality of Service (QoS) policies.

Topics

- [FILTER](#)
<<experimental>> The FILTER QoS policy allows you to configure the content filter capabilities and resource limits for a [DDSDomainParticipant](#) and all of its child entities.
- [CONTENT_FILTER](#)
<<experimental>> <<eXtension>> Filter that allows a [DDSDDataReader](#) to specify that it is interested only in (potentially) a subset of the samples on its topic.
- [DISCOVERY](#)
<<eXtension>> Specifies the attributes required to discover participants in the domain.
- [USER_TRAFFIC](#)
<<eXtension>> Specifies user-traffic transport settings
- [TRUST](#)
<<eXtension>> Specify trust settings for secure communication.

- **DEADLINE**

Expresses the maximum duration or deadline within which an instance is expected to be updated.

- **LATENCY_BUDGET**

Provides a hint as to the maximum acceptable delay from the time the data is written to the time it is received by the subscribing applications.

- **OWNERSHIP**

Specifies whether it is allowed for multiple `DDSDataWriter`(s) to write the same instance of the data and if so, how these modifications should be arbitrated.

- **OWNERSHIP_STRENGTH**

Specifies the value of the strength used to arbitrate among multiple `DDSDataWriter` objects that attempt to modify the same instance of a data type (identified by `DDSTopic` + key).

- **LIVELINESS**

Determines the mechanism and parameters used by the application to determine whether a `DDSEntity` is alive.

- **RELIABILITY**

Indicates the level of reliability offered or requested by RTI Connex DDS Micro.

- **HISTORY**

Specifies the behavior of RTI Connex DDS Micro in the case where the value of an instance changes (one or more times) before it can be successfully communicated to one or more existing subscribers.

- **DURABILITY**

<<eXtension>> Specifies the Durability properties used by a `DDSDataWriter` and/or `DDSDataReader`.

- **DATA_REPRESENTATION**

A list of data representations supported by a `DDSDataWriter` or `DDSDataReader`.

- **RESOURCE_LIMITS**

Specifies the resources that RTI Connex DDS Micro can consume in order to meet the requested QoS.

- **PRESENTATION**

Specifies how the samples representing changes to data instances are presented to a subscribing application.

- **DESTINATION_ORDER**

<<eXtension>> Specifies the DestinationOrder policy.

- **ENTITY_FACTORY**

A QoS policy for all `DDSEntity` types that can act as factories for one or more other `DDSEntity` types.

- **Extended Qos Support**

<<eXtension>> Types and defines used in extended QoS policies.

- **SYSTEM_RESOURCE_LIMITS**

<<eXtension>> Specifies the system-specific resources that RTI Connex DDS Micro can use.

- **WIRE_PROTOCOL**

<<eXtension>> Specifies the wire protocol related attributes for the `DDSDomainParticipant`.

- **DATA_READER_PROTOCOL**

<<eXtension>> Specifies the `DDSDataReader` specific protocol QoS.

- **DATA_WRITER_PROTOCOL**

<<eXtension>> Specifies the `DDSDataWriter` specific protocol QoS.

- **TRANSPORT**

<<eXtension>> Specifies transport settings

- **DOMAIN_PARTICIPANT_RESOURCE_LIMITS**

<<eXtension>> Specifies the DomainParticipant-specific resources that RTI Connex DDS Micro can use.

- **ENTITY_NAME**

<<eXtension>> Name of the entity.

- **PUBLISH_MODE**

<<eXtension>> Specifies how RTI Connex DDS Micro sends application data on the network. This QoS policy can be used to tell RTI Connex DDS Micro to use its own thread to send data, instead of the user thread.

- **PARTITION**

The **PARTITION** QoS policy provides a method to prevent Entities that have otherwise compatible QoS policies from matching with each other and thus communicating with each other.

- **PROPERTY**

<<eXtension>> Stores (name, value) pairs that can be used to configure certain parameters of RTI Connex DDS Micro that are not exposed through formal QoS policies.

- **TRANSPORT_PRIORITY**

This QoS policy allows the application to take advantage of transports that are capable of sending messages with different priorities.

- **USER_DATA**

The **USER_DATA** QoS policy allows the user to attach a buffer of opaque data to a [DDSDomainParticipant](#), [DDSDataWriter](#), or [DDSDataReader](#). How this information is used will be up to the user code; RTI Connex DDS Micro distributes this information to other applications as part of the discovery process, but RTI Connex DDS Micro does not interpret the information.

- **GROUP_DATA**

The **GROUP_DATA** QoS policy allows you to attach a buffer of opaque data to a [DDSPublisher](#) or [DDSSubscriber](#). The use of this information is up to user code; RTI Connex DDS Micro distributes this information to other applications as part of the discovery process, but RTI Connex DDS Micro does not interpret the information.

- **TOPIC_DATA**

The **TOPIC_DATA** QoS policy allows the user to attach a buffer of opaque data to a [DDSTopic](#). How this information is used will be up to the user code; RTI Connex DDS Micro distributes this information to other applications as part of the discovery process, but RTI Connex DDS Micro does not interpret the information.

- **DATA_WRITER_TRANSFER_MODE**

<<eXtension>> Specifies the [DDSDataWriter](#) transfer mode QoS.

Classes

- struct [DDS_QosPolicyCount](#)

Type to hold a counter for a [DDS_QosPolicyId_t](#)

Enumerations

- enum [DDS_QosPolicyId_t](#) {
[DDS_INVALID_QOS_POLICY_ID](#) , [DDS_USERDATA_QOS_POLICY_ID](#) = 1 , [DDS_DURABILITY_QOS_POLICY_ID](#) = 2 , [DDS_PRESENTATION_QOS_POLICY_ID](#) = 3 ,
[DDS_DEADLINE_QOS_POLICY_ID](#) , [DDS_LATENCYBUDGET_QOS_POLICY_ID](#) = 5 , [DDS_OWNERSHIP_QOS_POLICY_ID](#) ,
[DDS_OWNERSHIPSTRENGTH_QOS_POLICY_ID](#) ,
[DDS_LIVELINESS_QOS_POLICY_ID](#) , [DDS_TIMEBASEDFILTER_QOS_POLICY_ID](#) = 9 , [DDS_PARTITION_QOS_POLICY_ID](#) = 10 , [DDS_RELIABILITY_QOS_POLICY_ID](#) ,
[DDS_DESTINATIONORDER_QOS_POLICY_ID](#) = 12 , [DDS_HISTORY_QOS_POLICY_ID](#) , [DDS_RESOURCELIMITS_QOS_POLICY_ID](#) = 14 , [DDS_ENTITYFACTORY_QOS_POLICY_ID](#) ,
[DDS_WRITERDATALIFECYCLE_QOS_POLICY_ID](#) = 16 , [DDS_READERDATALIFECYCLE_QOS_POLICY_ID](#) = 17 , [DDS_TOPICDATA_QOS_POLICY_ID](#) = 18 , [DDS_GROUPDATA_QOS_POLICY_ID](#) = 19 ,
[DDS_TRANSPORTPRIORITY_QOS_POLICY_ID](#) = 20 , [DDS_LIFESPAN_QOS_POLICY_ID](#) = 21 , [DDS_DURABILITYSERVICE_QOS_POLICY_ID](#) = 22 , [DDS_DATA_REPRESENTATION_QOS_POLICY_ID](#) = 23 ,
[DDS_PROPERTY_QOS_POLICY_ID](#) = 24 }

Type to identify *QosPolicies*.

6.2.10.8.1 Detailed Description

Quality of Service (QoS) policies.

Data Distribution Service (DDS) relies on the use of QoS. A QoS is a set of characteristics that controls some aspect of the behavior of DDS. A QoS is comprised of individual QoS policies (objects conceptually deriving from an *abstract QosPolicy* class).

RTI Connex DDS Micro supports a subset of these QoS.

The QosPolicy provides the basic mechanism for an application to specify quality of service parameters. A QosPolicy may have additional properties associated with it:

- **RxOproperty**

The QosPolicy objects that need to be set in a compatible manner between the **publisher** and **subscriber** end are indicated by the setting of the RxO property:

- RxO = **YES** indicates that the policy can be set both at the publishing and subscribing ends and the values must be set in a compatible manner. In this case the compatible values are explicitly defined.
- RxO = **NO** indicates that the policy can be set both at the publishing and subscribing ends but the two settings are independent. That is, all combinations of values are compatible.
- RxO = **N/A** indicates that the policy can only be specified at either the publishing or the subscribing end, but not at both ends. So compatibility does not apply.

- **Changeableproperty**

Determines whether a QosPolicy can be changed.

NO -- policy can only be specified at *DDSEntity* creation time.

UNTIL ENABLE -- policy can only be changed before the *DDSEntity* is enabled.

YES -- policy can be changed at any time.

QosPolicy implementation is comprised of a name, an ID, and a type. The type of a *QosPolicy* value may be atomic, such as an integer or float, or compound structures. Compound types are used whenever multiple parameters must be set coherently to define a consistent value for a QosPolicy.

QoS, which is a list of *QosPolicy* objects, may be associated with all *DDSEntity* objects in the system such as *DDSTopic*, *DDSDataWriter*, *DDSDDataReader*, *DDSPublisher*, *DDSSubscriber*, and *DDSDomainParticipant*.

6.2.10.8.2 Specifying QoS on entities

QoS Policies can be set programmatically when an *DDSEntity* is created, or modified with the *DDSEntity*'s *set_qos (abstract)* method.

To customize a *DDSEntity*'s QoS before creating the entity, the correct pattern is:

- First, initialize a QoS object with the appropriate *INITIALIZER* constructor.
- Call the relevant *get_<entity>_default_qos()* method.
- Modify the QoS values as desired.

- Finally, create the entity.

Each `QosPolicy` is treated independently from the others. This approach has the advantage of being very extensible. However, there may be cases where several policies are in conflict. Consistency checking is performed each time the policies are modified via the `set_qos (abstract)` operation, or when the `DDSEntity` is created.

When a policy is changed after being set to a given value, it is not required that the new value be applied instantaneously; RTI Connex DDS Micro is allowed to apply it after a transition phase. In addition, some `QosPolicy` have immutable semantics, meaning that they can only be specified either at `DDSEntity` creation time or else prior to calling the `DDSEntity::enable` operation on the entity.

Each `DDSEntity` can be configured with a list of `QosPolicy` objects. However, not all `QosPolicies` are supported by each `DDSEntity`. For instance, a `DDSDomainParticipant` supports a different set of `QosPolicies` than a `DDSTopic` or a `DDSPublisher`.

6.2.10.8.3 QoS compatibility

In several cases, for communications to occur properly or efficiently, a `QosPolicy` on the publisher side must be compatible with a corresponding policy on the subscriber side. For example, if a `DDSSubscriber` requests to receive data reliably while the corresponding `DDSPublisher` defines a best-effort policy, communication will not happen as requested.

To address this issue and maintain the desirable decoupling of publication and subscription as much as possible, the `QosPolicy` specification follows the **subscriber-requested, publisher-offered pattern**.

In this pattern, the subscriber side can specify a "requested" value for a particular `QosPolicy`. The publisher side specifies an "offered" value for that `QosPolicy`. RTI Connex DDS Micro will then determine whether the value requested by the subscriber side is compatible with what is offered by the publisher side. If the two policies are compatible, then communication will be established. If the two policies are not compatible, RTI Connex DDS Micro will not establish communications between the two `DDSEntity` objects and will record this fact by means of the `DDS_OFFERED_INCOMPATIBLE_QOS_STATUS` on the publisher end and `DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS` on the subscriber end. The application can detect this fact by means of a `DDSListener`.

6.2.10.8.4 Enumeration Type Documentation

DDS_QosPolicyId_t

enum `DDS_QosPolicyId_t`

Type to identify `QosPolicies`.

Enumerator

<code>DDS_INVALID_QOS_POLICY_ID</code>	Identifier for an invalid QoS policy.
<code>DDS_DEADLINE_QOS_POLICY_ID</code>	Identifier for <code>DDS_DeadlineQosPolicy</code> .
<code>DDS_OWNERSHIP_QOS_POLICY_ID</code>	Identifier for <code>DDS_OwnershipQosPolicy</code> .
<code>DDS_OWNERSHIPSTRENGTH_QOS_POLICY_ID</code>	Identifier for <code>DDS_OwnershipStrengthQosPolicy</code> .
<code>DDS_LIVELINESS_QOS_POLICY_ID</code>	Identifier for <code>DDS_LivelinessQosPolicy</code> .
<code>DDS_RELIABILITY_QOS_POLICY_ID</code>	Identifier for <code>DDS_ReliabilityQosPolicy</code> .
<code>DDS_HISTORY_QOS_POLICY_ID</code>	Identifier for <code>DDS_HistoryQosPolicy</code> .
<code>DDS_ENTITYFACTORY_QOS_POLICY_ID</code>	Identifier for <code>DDS_EntityFactoryQosPolicy</code> .

6.2.10.8.5 FILTER

<<experimental>> The FILTER QoS policy allows you to configure the content filter capabilities and resource limits for a [DDSDomainParticipant](#) and all of its child entities.

Classes

- struct [DDS_FilterResourceLimits](#)
Resource limits for content filtering on a [DDSDomainParticipant](#) and all of its child entities.
- struct [DDS_FilterQosPolicy](#)
Configures the content filter capabilities and resource limits for a [DDSDomainParticipant](#) and all of its child entities.

Macros

- #define [DDS_CONTENT_FILTER_NAME_MAX_LENGTH](#) 63
The absolute maximum length of the content filter name.
- #define [DDS_CONTENT_FILTER_CLASS_NAME_MAX_LENGTH](#) 15
The absolute maximum length of the content filter class name.
- #define [DDS_CONTENT_FILTER_EXPRESSION_MAX_LENGTH](#) 255
The absolute maximum length of the content filter expression.
- #define [DDS_CONTENT_FILTER_MAX_PARAMETERS](#) 16
The absolute maximum number of parameters in a content filter expression.
- #define [DDS_FILTER_RESOURCE_LIMITS_DEFAULT](#)
Default resource limits for content filtering.
- #define [DDS_FILTER_PLUGIN_FACTORY_DEFAULT_NAME](#) "filter"
Default name for the filter plugin factory.

Detailed Description

<<experimental>> The FILTER QoS policy allows you to configure the content filter capabilities and resource limits for a [DDSDomainParticipant](#) and all of its child entities.

Macro Definition Documentation

DDS_CONTENT_FILTER_NAME_MAX_LENGTH

```
#define DDS_CONTENT_FILTER_NAME_MAX_LENGTH 63
```

The absolute maximum length of the content filter name.

A lower maximum length can be configured in [DDS_FilterResourceLimits::filter_class_max_length](#) to reduce memory usage, but this value is the absolute maximum.

DDS_CONTENT_FILTER_CLASS_NAME_MAX_LENGTH

```
#define DDS_CONTENT_FILTER_CLASS_NAME_MAX_LENGTH 15
```

The absolute maximum length of the content filter class name.

A lower maximum length can be configured in [DDS_FilterResourceLimits::filter_class_max_length](#) to reduce memory usage, but this value is the absolute maximum.

DDS_CONTENT_FILTER_EXPRESSION_MAX_LENGTH

```
#define DDS_CONTENT_FILTER_EXPRESSION_MAX_LENGTH 255
```

The absolute maximum length of the content filter expression.

A lower maximum length can be configured in [DDS_FilterResourceLimits::filter_expression_max_length](#) to reduce memory usage, but this value is the absolute maximum.

DDS_CONTENT_FILTER_MAX_PARAMETERS

```
#define DDS_CONTENT_FILTER_MAX_PARAMETERS 16
```

The absolute maximum number of parameters in a content filter expression.

A lower maximum can be configured in [DDS_FilterResourceLimits::filter_parameter_max_count_per_expression](#) to reduce memory usage, but this value is the absolute maximum.

DDS_FILTER_RESOURCE_LIMITS_DEFAULT

```
#define DDS_FILTER_RESOURCE_LIMITS_DEFAULT
```

Value:

```
{\
    7,          /* filter_class_max_length */      \
    1,          /* filter_class_max_count */  \
    63,         /* filter_expression_max_length */ \
    DDS_LENGTH_AUTO, /* filter_expression_max_count */ \
    4,          /* filter_parameter_max_count_per_expression */ \
    15,         /* filter_parameter_max_length */ \
    4           /* sql_predicate_max_count_per_expression */ \
}
```

Default resource limits for content filtering.

DDS_FILTER_PLUGIN_FACTORY_DEFAULT_NAME

```
#define DDS_FILTER_PLUGIN_FACTORY_DEFAULT_NAME "filter"
```

Default name for the filter plugin factory.

6.2.10.8.6 CONTENT_FILTER

<<experimental>> <<eXtension>> Filter that allows a [DDSDataReader](#) to specify that it is interested only in (potentially) a subset of the samples on its topic.

Classes

- struct [DDS_ContentFilterQosPolicy](#)

Filter that allows a [DDSDataReader](#) to specify that it is interested only in (potentially) a subset of the samples on its topic.

Detailed Description

<<experimental>> <<eXtension>> Filter that allows a [DDSDataReader](#) to specify that it is interested only in (potentially) a subset of the samples on its topic.

6.2.10.8.7 DISCOVERY

<<eXtension>> Specifies the attributes required to discover participants in the domain.

Classes

- struct [DDS_DiscoveryQosPolicy](#)

Specifies the attributes required to discover participants in the domain.

Detailed Description

<<eXtension>> Specifies the attributes required to discover participants in the domain.

6.2.10.8.8 USER_TRAFFIC

<<eXtension>> Specifies user-traffic transport settings

Classes

- struct [DDS_UserTrafficQosPolicy](#)

<<eXtension>> *Configures the default transports enabled to transmit and receive user data traffic.*

Detailed Description

<<eXtension>> Specifies user-traffic transport settings

6.2.10.8.9 TRUST

<<eXtension>> Specify trust settings for secure communication.

Classes

- struct [DDS_TrustQosPolicy](#)
<<eXtension>> *Configures the security plugin suite used by a [DDSDomainParticipant](#).*

Detailed Description

<<eXtension>> Specify trust settings for secure communication.

6.2.10.8.10 DEADLINE

Expresses the maximum duration or deadline within which an instance is expected to be updated.

Classes

- struct [DDS_DeadlineQosPolicy](#)
Expresses the maximum duration or deadline within which an instance is expected to be updated.

Detailed Description

Expresses the maximum duration or deadline within which an instance is expected to be updated.

6.2.10.8.11 LATENCY_BUDGET

Provides a hint as to the maximum acceptable delay from the time the data is written to the time it is received by the subscribing applications.

Classes

- struct [DDS_LatencyBudgetQosPolicy](#)
Provides a hint as to the maximum acceptable delay from the time the data is written to the time it is received by the subscribing applications.

Detailed Description

Provides a hint as to the maximum acceptable delay from the time the data is written to the time it is received by the subscribing applications.

6.2.10.8.12 OWNERSHIP

Specifies whether it is allowed for multiple `::DDSDataWriter(s)` to write the same instance of the data and if so, how these modifications should be arbitrated.

Classes

- struct `DDS_OwnershipQosPolicy`

Specifies whether it is allowed for multiple `::DDSDataWriter(s)` to write the same instance of the data and if so, how these modifications should be arbitrated.

Enumerations

- enum `DDS_OwnershipQosPolicyKind` { `DDS_SHARED_OWNERSHIP_QOS` , `DDS_EXCLUSIVE_OWNERSHIP_QOS` }

Kinds of ownership.

Detailed Description

Specifies whether it is allowed for multiple `::DDSDataWriter(s)` to write the same instance of the data and if so, how these modifications should be arbitrated.

Enumeration Type Documentation

DDS_OwnershipQosPolicyKind

enum `DDS_OwnershipQosPolicyKind`

Kinds of ownership.

QoS:

`DDS_OwnershipQosPolicy`

Enumerator

DDS_SHARED_OWNERSHIP_QOS	[default] Indicates shared ownership for each instance. Multiple writers may update the same instance and the readers receive all the updates. In other words there is no concept of an owner for the instances. This is the default behavior if the <code>OWNERSHIP</code> policy is not specified or supported.
DDS_EXCLUSIVE_OWNERSHIP_QOS	Indicates each instance can only be owned by one <code>DDSDataWriter</code> , but the owner of an instance can change dynamically. The setting of the <code>OWNERSHIP_STRENGTH</code> policy controls the selection of the owner. The owner is always the highest-strength <code>DDSDataWriter</code> object among the ones currently active (as the <code>LIVELINESS</code> determines).

6.2.10.8.13 OWNERSHIP_STRENGTH

Specifies the value of the strength used to arbitrate among multiple [DDSDataWriter](#) objects that attempt to modify the same instance of a data type (identified by [DDSTopic](#) + key).

Classes

- struct [DDS_OwnershipStrengthQosPolicy](#)
Specifies the value of the strength used to arbitrate among multiple [DDSDataWriter](#) objects that attempt to modify the same instance of a data type (identified by [DDSTopic](#) + key).

Detailed Description

Specifies the value of the strength used to arbitrate among multiple [DDSDataWriter](#) objects that attempt to modify the same instance of a data type (identified by [DDSTopic](#) + key).

6.2.10.8.14 LIVELINESS

Determines the mechanism and parameters used by the application to determine whether a [DDSEntity](#) is alive.

Classes

- struct [DDS_LivelinessQosPolicy](#)
Determines the mechanism and parameters used by the application to determine whether a [DDSEntity](#) is alive.

Enumerations

- enum [DDS_LivelinessQosPolicyKind](#) { [DDS_AUTOMATIC_LIVELINESS_QOS](#) , [DDS_MANUAL_BY_PARTICIPANT_LIVELINESS_QOS](#) , [DDS_MANUAL_BY_TOPIC_LIVELINESS_QOS](#) }
Kinds of liveliness.

Detailed Description

Determines the mechanism and parameters used by the application to determine whether a [DDSEntity](#) is alive.

Enumeration Type Documentation

DDS_LivelinessQosPolicyKind

enum [DDS_LivelinessQosPolicyKind](#)

Kinds of liveliness.

QoS:

[DDS_LivelinessQosPolicy](#)

Enumerator

DDS_AUTOMATIC_LIVELINESS_QOS	[default] The infrastructure will automatically signal liveliness for DDSDataWriter entities at least as often as required by the DDSDataWriter 's <code>lease_duration</code> . A DDSDataWriter with this setting does not need to take any specific action to be considered alive. The DDSDataWriter is only 'not alive' when the participant terminates or when a network problem prevents the current participant from contacting that remote participant.
DDS_MANUAL_BY_PARTICIPANT_LIVELINESS_QOS	RTI Connext DDS Micro will assume that as long as at least one DDSDataWriter belonging to the DDSDomainParticipant (or the DDSDomainParticipant itself) has asserted its liveliness, then the other DDSDataWriter entities belonging to that same DDSDomainParticipant are also alive. The user application takes responsibility to signal liveliness to RTI Connext DDS Micro either by calling DDSDomainParticipant::assert_liveliness , or by calling DDSDataWriter::assert_liveliness , or FooDataWriter::write on any DDSDataWriter belonging to the DDSDomainParticipant .
DDS_MANUAL_BY_TOPIC_LIVELINESS_QOS	RTI Connext DDS Micro will only assume liveliness of the DDSDataWriter if the application has asserted liveliness of that DDSDataWriter itself. The user application takes responsibility to signal liveliness to RTI Connext DDS Micro using the DDSDataWriter::assert_liveliness method, or by writing some data.

6.2.10.8.15 RELIABILITY

Indicates the level of reliability offered or requested by RTI Connext DDS Micro.

Classes

- struct [DDS_ReliabilityQosPolicy](#)

Indicates the level of reliability offered or requested by RTI Connext DDS Micro.

Enumerations

- enum [DDS_ReliabilityQosPolicyKind](#) { [DDS_BEST_EFFORT_RELIABILITY_QOS](#), [DDS_RELIABLE_RELIABILITY_QOS](#) }

Kinds of reliability.

Detailed Description

Indicates the level of reliability offered or requested by RTI Connext DDS Micro.

Enumeration Type Documentation

DDS_ReliabilityQosPolicyKind

enum [DDS_ReliabilityQosPolicyKind](#)

Kinds of reliability.

QoS:

[DDS_ReliabilityQosPolicy](#)

Enumerator

DDS_BEST_EFFORT_RELIABILITY_QOS	Indicates that it is acceptable not to retry the propagation of any samples. Presumably new values for the samples are generated often enough that it is not necessary to re-send or acknowledge any samples. [default]
DDS_RELIABLE_RELIABILITY_QOS	Specifies that RTI Connext DDS Micro will attempt to deliver all samples in its history. Missed samples may be retried. In steady state, when no modifications are communicated via the DDSDataWriter , the middleware guarantees that all samples in the DDSDataWriter history will eventually be delivered to all the DDSDataReader objects, subject to timeouts that indicate a loss of communication with a particular DDSSubscriber . Outside steady state the HISTORY and RESOURCE_LIMITS policies will determine how samples become part of the history and whether samples can be discarded from it. Note The RTPS spec defines reliability as 0x03 to comply with RTPS spec. However, RTI Connext Core uses 0x3, as well as others.

6.2.10.8.16 HISTORY

Specifies the behavior of RTI Connext DDS Micro in the case where the value of an instance changes (one or more times) before it can be successfully communicated to one or more existing subscribers.

Classes

- struct [DDS_HistoryQosPolicy](#)

Specifies the behavior of RTI Connex DDS Micro when a sample value changes one or more times before it can be successfully communicated to existing subscribers.

Enumerations

- enum [DDS_HistoryQosPolicyKind](#) { [DDS_KEEP_LAST_HISTORY_QOS](#) , [DDS_KEEP_ALL_HISTORY_QOS](#) }

Kinds of history.

Detailed Description

Specifies the behavior of RTI Connex DDS Micro in the case where the value of an instance changes (one or more times) before it can be successfully communicated to one or more existing subscribers.

Enumeration Type Documentation

DDS_HistoryQosPolicyKind

enum [DDS_HistoryQosPolicyKind](#)

Kinds of history.

QoS:

[DDS_HistoryQosPolicy](#)

Enumerator

DDS_KEEP_LAST_HISTORY_QOS	[default] Keep the last <code>depth</code> samples. On the publishing side, RTI Connex DDS Micro will only attempt to keep the most recent <code>depth</code> samples of each instance of data (identified by its key) managed by the DDSDDataWriter. On the subscribing side, the DDSDDataReader will only attempt to keep the most recent <code>depth</code> samples received for each instance identified by its key, until the application takes them via the DDSDDataReader's take() operation.
DDS_KEEP_ALL_HISTORY_QOS	Keep <i>all</i> the samples. On the publishing side, RTI Connex DDS Micro will attempt to keep all samples of each instance of data identified by its key, which are managed by the DDSDDataWriter until they can be delivered to all subscribers. On the subscribing side, RTI Connex DDS Micro will attempt to keep all samples of each instance of data identified by its key and managed by the DDSDDataReader. These samples are kept until the application takes them from RTI Connex DDS Micro via the take() operation. NOTE: Not supported for a DDSDDataWriter communicating via INTRA transport. See the User's Manual for more details.

6.2.10.8.17 DURABILITY

<<*eXtension*>> Specifies the Durability properties used by a [DDSDataWriter](#) and/or [DDSDataReader](#).

Classes

- struct [DDS_DurabilityQosPolicy](#)
Durability properties that affect the life-cycle of published data.

Enumerations

- enum [DDS_DurabilityQosPolicyKind](#) { [DDS_VOLATILE_DURABILITY_QOS](#) , [DDS_TRANSIENT_LOCAL_DURABILITY_QOS](#) , [DDS_TRANSIENT_DURABILITY_QOS](#) , [DDS_PERSISTENT_DURABILITY_QOS](#) }
Enumerated values used to describe the durability of published data.

Detailed Description

<<*eXtension*>> Specifies the Durability properties used by a [DDSDataWriter](#) and/or [DDSDataReader](#).

Enumeration Type Documentation

DDS_DurabilityQosPolicyKind

enum [DDS_DurabilityQosPolicyKind](#)

Enumerated values used to describe the durability of published data.

Enumerator

DDS_VOLATILE_DURABILITY_QOS	Volatile durability.
DDS_TRANSIENT_LOCAL_DURABILITY_QOS	Transient local durability. NOTE: Not supported when using the INTRA transport because a DDSDataReader cannot receive historical samples via INTRA. See the User's Manual for more details.

6.2.10.8.18 DATA_REPRESENTATION

A list of data representations supported by a [DDSDataWriter](#) or [DDSDataReader](#).

Classes

- struct [DDS_DataRepresentationIdSeq](#)
Declares IDL sequence < DDS_DataRepresentationId_t >
- struct [DDS_DataRepresentationQosPolicy](#)
This QoS policy contains a list of representation identifiers used by [DDSDataWriter](#) and [DDSDataReader](#) entities to negotiate which data representation to use.

Macros

- `#define DDS_XCDR_DATA_REPRESENTATION ((DDS_DataRepresentationId_t)0)`
Corresponds to the Extended Common Data Representation encoding version 1.
- `#define DDS_XML_DATA_REPRESENTATION ((DDS_DataRepresentationId_t)1)`
Corresponds to the XML Data Representation (unsupported).
- `#define DDS_XCDR2_DATA_REPRESENTATION ((DDS_DataRepresentationId_t)2)`
Corresponds to the Extended Common Data Representation encoding version 2.
- `#define DDS_AUTO_DATA_REPRESENTATION ((DDS_DataRepresentationId_t)-1)`
Representation is automatically chosen based on the type.

Typedefs

- typedef [DDS_Short DDS_DataRepresentationId_t](#)
A two-byte signed integer that identifies a data representation.

Detailed Description

A list of data representations supported by a [DDSDataWriter](#) or [DDSDataReader](#).

Macro Definition Documentation

DDS_XCDR_DATA_REPRESENTATION

```
#define DDS_XCDR_DATA_REPRESENTATION ((DDS_DataRepresentationId_t)0)
```

Corresponds to the Extended Common Data Representation encoding version 1.

DDS_XML_DATA_REPRESENTATION

```
#define DDS_XML_DATA_REPRESENTATION ((DDS_DataRepresentationId_t)1)
```

Corresponds to the XML Data Representation (unsupported).

Note

This value is currently not supported.

DDS_XCDR2_DATA_REPRESENTATION

```
#define DDS_XCDR2_DATA_REPRESENTATION ((DDS_DataRepresentationId_t)2)
```

Corresponds to the Extended Common Data Representation encoding version 2.

DDS_AUTO_DATA_REPRESENTATION

```
#define DDS_AUTO_DATA_REPRESENTATION ((DDS_DataRepresentationId_t)-1)
```

Representation is automatically chosen based on the type.

For plain language binding, if the `allow_data_representation` annotation is not specified or if it contains the value XCDR, RTI Connex DDS Micro translates this field to [DDS_XCDR_DATA_REPRESENTATION](#). Otherwise, it translates this field to [DDS_XCDR2_DATA_REPRESENTATION](#).

For the [FlatData Topic-Types](#) "FlatData language binding", RTI Connex DDS Micro translates this field to [DDS_XCDR2_DATA_REPRESENTATION](#).

Typedef Documentation

DDS_DataRepresentationId_t

```
typedef DDS_Short DDS_DataRepresentationId_t
```

A two-byte signed integer that identifies a data representation.

6.2.10.8.19 RESOURCE_LIMITS

Specifies the resources that RTI Connex DDS Micro can consume in order to meet the requested QoS.

Classes

- struct [DDS_ResourceLimitsQosPolicy](#)
Specifies the resources that RTI Connex DDS Micro can consume in order to meet the requested QoS.

Variables

- const [DDS_Long](#) [DDS_LENGTH_UNLIMITED](#)
A special value indicating an unlimited quantity.
- const [DDS_Long](#) [DDS_LENGTH_AUTO](#)
A special value indicating the resource limit is determined based on other settings.
- const [DDS_Long](#) [DDS_SIZE_AUTO](#)
A special value indicating the resource limit is determined based on other settings.

Detailed Description

Specifies the resources that RTI Connex DDS Micro can consume in order to meet the requested QoS.

Variable Documentation

DDS_LENGTH_UNLIMITED

```
const DDS_Long DDS_LENGTH_UNLIMITED [extern]
```

A special value indicating an unlimited quantity.

DDS_LENGTH_AUTO

```
const DDS_Long DDS_LENGTH_AUTO [extern]
```

A special value indicating the resource limit is determined based on other settings.

Some resource limits can be derived from other limits using rules. However, the rules are usually conservative and typically the resulting limit is larger than needed. Thus, while this is a useful value when starting development, it should be adjusted properly when the exact limits are known.

DDS_SIZE_AUTO

```
const DDS_Long DDS_SIZE_AUTO [extern]
```

A special value indicating the resource limit is determined based on other settings.

Some resource limits can be derived from other limits. However, the rules for this are conservative, and the resulting limit is usually larger than needed. This is a useful value when starting development, but it should be adjusted properly when the exact limits are known.

6.2.10.8.20 PRESENTATION

Specifies how the samples representing changes to data instances are presented to a subscribing application.

Classes

- struct [DDS_PresentationQosPolicy](#)

Specifies how the samples representing changes to data instances are presented to a subscribing application.

Enumerations

- enum [DDS_PresentationQosPolicyAccessScopeKind](#) { [DDS_INSTANCE_PRESENTATION_QOS](#), [DDS_TOPIC_PRESENTATION_QOS](#), [DDS_GROUP_PRESENTATION_QOS](#), [DDS_HIGHEST_OFFERED_PRESENTATION_QOS](#) }

Kinds of presentation "access scope".

Detailed Description

Specifies how the samples representing changes to data instances are presented to a subscribing application.

Enumeration Type Documentation

DDS_PresentationQosPolicyAccessScopeKind

enum [DDS_PresentationQosPolicyAccessScopeKind](#)

Kinds of presentation "access scope".

Access scope determines the largest scope spanning the entities for which the order and coherency of changes can be preserved.

QoS:

[DDS_PresentationQosPolicy](#)

Enumerator

DDS_INSTANCE_PRESENTATION_QOS	[default] for DDSDataReader . Scope spans only a single instance. Indicates that changes to one instance need not be coherent nor ordered with respect to changes to any other instance. In other words, order and coherent changes apply to each instance separately.
DDS_TOPIC_PRESENTATION_QOS	[default] for DDSDataWriter . Scope spans all instances within the same DDSDataWriter or DDSDataReader , but not across instances in different DDSDataWriter or DDSDataReader entities.
DDS_GROUP_PRESENTATION_QOS	Scope spans all instances belonging to DDSDataWriter or DDSDataReader entities within the same DDSPublisher or DDSSubscriber .

6.2.10.8.21 DESTINATION_ORDER

<< *eXtension* >> Specifies the DestinationOrder policy.

Classes

- struct [DDS_DestinationOrderQosPolicy](#)
Destination order policies that affect the ordering of subscribed data.

Enumerations

- enum [DDS_DestinationOrderQosPolicyKind](#) { [DDS_BY_RECEPTION_TIMESTAMP_DESTINATIONORDER_QOS](#), [DDS_BY_SOURCE_TIMESTAMP_DESTINATIONORDER_QOS](#) }
Enumerated values used to describe the destination order of published data.

Detailed Description

<<*eXtension*>> Specifies the DestinationOrder policy.

Enumeration Type Documentation

DDS_DestinationOrderQosPolicyKind

enum [DDS_DestinationOrderQosPolicyKind](#)

Enumerated values used to describe the destination order of published data.

Enumerator

DDS_BY_RECEPTION_TIMESTAMP_DESTINATIONORDER_QOS	destination ordering by reception timestamp. Entity: DDSDataReader, DDSDataWriter Properties: RxO = YES; Changeable = NO [default] Indicates that data is ordered based on the reception time at each DDSDataReader. Since each subscriber may receive the data at different times there is no guarantee that the changes will be seen in the same order. Consequently, it is possible for each subscriber to end up with a different final value for the data.
--	--

Enumerator

DDS_BY_SOURCE_TIMESTAMP_↔ DESTINATIONORDER_QOS	destination ordering by source timestamp. Entity: DDSDDataWriter Properties: RxO = YES; Changeable = NO Indicates that data is ordered based on the source time provided by the source application. A consistent final value is guaranteed for the data in all subscribers.
---	--

6.2.10.8.22 ENTITY_FACTORY

A QoS policy for all [DDSEntity](#) types that can act as factories for one or more other [DDSEntity](#) types.

Classes

- struct [DDS_EntityFactoryQosPolicy](#)
A QoS policy for all [DDSEntity](#) types that can act as factories for one or more other [DDSEntity](#) types.

Detailed Description

A QoS policy for all [DDSEntity](#) types that can act as factories for one or more other [DDSEntity](#) types.

6.2.10.8.23 Extended Qos Support

<<[eXtension](#)>> Types and defines used in extended QoS policies.

Topics

- [DataReader Resource Limits](#)
 <<[eXtension](#)>> Specifies the [DDSDDataReader](#) specific resources that RTI Connex DDS Micro can use.
- [DataWriter Resource Limits](#)
 <<[eXtension](#)>> Specifies the [DDSDDataWriter](#)-specific resources that RTI Connex DDS Micro can use.

Classes

- struct [DDS_RtpsReliableReaderProtocol_t](#)
QoS related to reliable reader protocol defined in RTPS.
- struct [DDS_RtpsReliableWriterProtocol_t](#)
QoS related to the reliable writer protocol defined in RTPS.

Detailed Description

<<*eXtension*>> Types and defines used in extended QoS policies.

DataReader Resource Limits

<<*eXtension*>> Specifies the [DDSDataReader](#) specific resources that RTI Connex DDS Micro can use.

Classes

- struct [DDS_DataReaderResourceLimitsQosPolicy](#)
Resource limits that apply only to [DDSDataReader](#) instances.

Enumerations

- enum [DDS_DataReaderResourceLimitsInstanceReplacementKind](#) { [DDS_NO_INSTANCE_REPLACEMENT_QOS](#), [DDS_REPLACE_OLDEST_INSTANCE_REPLACEMENT_QOS](#) }
Kinds of instance replacement.

Detailed Description

<<*eXtension*>> Specifies the [DDSDataReader](#) specific resources that RTI Connex DDS Micro can use.

Enumeration Type Documentation

DDS_DataReaderResourceLimitsInstanceReplacementKind

```
enum DDS\_DataReaderResourceLimitsInstanceReplacementKind
```

Kinds of instance replacement.

QoS:

[DDS_DataReaderResourceLimitsQosPolicy](#)

Enumerator

DDS_NO_INSTANCE_REPLACEMENT_QOS	[default] No instances will be replaced when a new instance is detected and resources are exhausted.
DDS_REPLACE_OLDEST_INSTANCE_↔ REPLACEMENT_QOS	Replace the oldest instance when out of instance resources. When a new instance is detected and resources are exhausted the oldest instance will be removed even if samples with an unread state exist for the instance. Thus, this is a forceful removal of samples and the instance. The freed instance resources will be re-used for the newly detected instance. If there are outstanding loans for the replaced instance, the samples cannot be freed until returned. However, when the loan is returned the samples are returned back to the free sample pool. Note that the instance will not be rejected.

DataWriter Resource Limits

<<eXtension>> Specifies the [DDSDDataWriter](#)-specific resources that RTI Connex DDS Micro can use.

Classes

- struct [DDS_DataWriterResourceLimitsQosPolicy](#)
Resource limits that apply only to [DDSDDataWriter](#) instances.

Detailed Description

<<eXtension>> Specifies the [DDSDDataWriter](#)-specific resources that RTI Connex DDS Micro can use.

6.2.10.8.24 SYSTEM_RESOURCE_LIMITS

<<eXtension>> Specifies the system-specific resources that RTI Connex DDS Micro can use.

Classes

- struct [DDS_SystemResourceLimitsQosPolicy](#)
Resource limits that apply only to [DDSDomainParticipantFactory](#)

Detailed Description

<<eXtension>> Specifies the system-specific resources that RTI Connex DDS Micro can use.

6.2.10.8.25 WIRE_PROTOCOL

<<eXtension>> Specifies the wire protocol related attributes for the [DDSDomainParticipant](#).

Classes

- struct [DDS_RtpsWellKnownPorts_t](#)
RTPS well-known port mapping configuration.
- struct [DDS_WireProtocolQosPolicy](#)
Specifies the wire-protocol-related attributes for the [DDSDomainParticipant](#).

Macros

- #define [DDS_CHECKSUM_NONE](#) (0)
Bitmap indicating no checksum selection.
- #define [DDS_CHECKSUM_AUTO](#) (0xffffU)
Bitmap indicating automatic checksum selection.
- #define [DDS_CHECKSUM_BUILTIN32](#) (0x1U)
Bitmap for selecting the builtin 32bit checksum.
- #define [DDS_CHECKSUM_BUILTIN64](#) (0x2U)
Bitmap for selecting the builtin 64bit checksum.
- #define [DDS_CHECKSUM_BUILTIN128](#) (0x4U)
Bitmap for selecting the builtin 128bit checksum.

Typedefs

- typedef [DDS_UnsignedShort DDS_ChecksumKind_t](#)
Datatype for checksum kind.
- typedef [DDS_UnsignedShort DDS_ChecksumKindMask_t](#)
Datatype for an OR'ing for [DDS_ChecksumKind_t](#).

Variables

- struct [DDS_RtpsWellKnownPorts_t DDS_RTI_BACKWARDS_COMPATIBLE RTPS_WELL_KNOWN_PORTS](#)
Assign to use well-known port mappings which are compatible with previous versions of the RTI Connex DDS Micro middleware.
- struct [DDS_RtpsWellKnownPorts_t DDS_INTEROPERABLE RTPS_WELL_KNOWN_PORTS](#)
Assign to use well-known port mappings which are compliant with OMG's DDS Interoperability Wire Protocol.

Detailed Description

<<eXtension>> Specifies the wire protocol related attributes for the [DDSDomainParticipant](#).

Macro Definition Documentation

DDS_CHECKSUM_NONE

```
#define DDS_CHECKSUM_NONE (0)
```

Bitmap indicating no checksum selection.

QoS:

[DDS_WireProtocolQosPolicy](#)

DDS_CHECKSUM_AUTO

```
#define DDS_CHECKSUM_AUTO (0xffffU)
```

Bitmap indicating automatic checksum selection.

QoS:

[DDS_WireProtocolQosPolicy](#)

DDS_CHECKSUM_BUILTIN32

```
#define DDS_CHECKSUM_BUILTIN32 (0x1U)
```

Bitmap for selecting the builtin 32bit checksum.

QoS:

[DDS_WireProtocolQosPolicy](#)

DDS_CHECKSUM_BUILTIN64

```
#define DDS_CHECKSUM_BUILTIN64 (0x2U)
```

Bitmap for selecting the builtin 64bit checksum.

QoS:

[DDS_WireProtocolQosPolicy](#)

DDS_CHECKSUM_BUILTIN128

```
#define DDS_CHECKSUM_BUILTIN128 (0x4U)
```

Bitmap for selecting the builtin 128bit checksum.

QoS:

[DDS_WireProtocolQosPolicy](#)

Typedef Documentation**DDS_ChecksumKind_t**

```
typedef DDS_UnsignedShort DDS_ChecksumKind_t
```

Datatype for checksum kind.

QoS:

[DDS_WireProtocolQosPolicy](#)

DDS_ChecksumKindMask_t

```
typedef DDS_UnsignedShort DDS_ChecksumKindMask_t
```

Datatype for an OR'ing for [DDS_ChecksumKind_t](#).

QoS:

[DDS_WireProtocolQosPolicy](#)

Variable Documentation

DDS_RTI_BACKWARDS_COMPATIBLE_RTPS_WELL_KNOWN_PORTS

```
struct DDS_RtpsWellKnownPorts_t DDS_RTI_BACKWARDS_COMPATIBLE_RTPS_WELL_KNOWN_PORTS [extern]
```

Assign to use well-known port mappings which are compatible with previous versions of the RTI Connex DDS Micro middleware.

Assign [DDS_WireProtocolQosPolicy::rtps_well_known_ports](#) to this value to remain compatible with previous versions of the RTI Connex DDS Micro middleware that used fixed port mappings.

The following are the `rtps_well_known_ports` values for [DDS_RTI_BACKWARDS_COMPATIBLE_RTPS_WELL_KNOWN_PORTS](#):

```
port_base = 7400
domain_id_gain = 10
participant_id_gain = 1000
builtin_multicast_port_offset = 2
builtin_unicast_port_offset = 0
user_multicast_port_offset = 1
user_unicast_port_offset = 3
```

These settings are *not* compliant with OMG's DDS Interoperability Wire Protocol. To comply with the specification, please use [DDS_INTEROPERABLE_RTPS_WELL_KNOWN_PORTS](#).

See also

[DDS_WireProtocolQosPolicy::rtps_well_known_ports](#)
[DDS_INTEROPERABLE_RTPS_WELL_KNOWN_PORTS](#)

DDS_INTEROPERABLE_RTPS_WELL_KNOWN_PORTS

```
struct DDS_RtpsWellKnownPorts_t DDS_INTEROPERABLE_RTPS_WELL_KNOWN_PORTS [extern]
```

Assign to use well-known port mappings which are compliant with OMG's DDS Interoperability Wire Protocol.

Assign [DDS_WireProtocolQosPolicy::rtps_well_known_ports](#) to this value to use well-known port mappings which are compliant with OMG's DDS Interoperability Wire Protocol.

The following are the `rtps_well_known_ports` values for [DDS_INTEROPERABLE_RTPS_WELL_KNOWN_PORTS](#):

```
port_base = 7400
domain_id_gain = 250
participant_id_gain = 2
builtin_multicast_port_offset = 0
builtin_unicast_port_offset = 10
user_multicast_port_offset = 1
user_unicast_port_offset = 11
```

Assuming a maximum port number of 65535 (UDPv4), the above settings enable the use of about 230 domains with up to 120 Participants per node per domain.

These settings are *not* backwards compatible with previous versions of the RTI Connex DDS Micro middleware that used fixed port mappings. For backwards compatibility, please use [DDS_RTI_BACKWARDS_COMPATIBLE_RTPS_WELL_KNOWN_PORTS](#).

See also

[DDS_WireProtocolQosPolicy::rtps_well_known_ports](#)
[DDS_RTI_BACKWARDS_COMPATIBLE_RTPS_WELL_KNOWN_PORTS](#)

6.2.10.8.26 DATA_READER_PROTOCOL

<<eXtension>> Specifies the [DDSDataReader](#) specific protocol QoS.

Classes

- struct [DDS_DataReaderProtocolQosPolicy](#)
<<eXtension>> *Protocol that applies only to [DDSDataReader](#) instances.*

Detailed Description

<<eXtension>> Specifies the [DDSDataReader](#) specific protocol QoS.

6.2.10.8.27 DATA_WRITER_PROTOCOL

<<eXtension>> Specifies the [DDSDataWriter](#) specific protocol QoS.

Classes

- struct [DDS_DataWriterProtocolQosPolicy](#)
<<eXtension>> *Protocol that applies only to [DDSDataWriter](#) instances.*

Detailed Description

<<eXtension>> Specifies the [DDSDataWriter](#) specific protocol QoS.

6.2.10.8.28 TRANSPORT

<<eXtension>> Specifies transport settings

Classes

- struct [DDS_TransportQosPolicy](#)
<<eXtension>> *Settings for available transports*

Detailed Description

<<eXtension>> Specifies transport settings

6.2.10.8.29 DOMAIN_PARTICIPANT_RESOURCE_LIMITS

<<*eXtension*>> Specifies the DomainParticipant-specific resources that RTI Connext DDS Micro can use.

Classes

- struct [DDS_DomainParticipantResourceLimitsQosPolicy](#)
Resource limits that apply only to [DDSDomainParticipant](#) instances.

Detailed Description

<<*eXtension*>> Specifies the DomainParticipant-specific resources that RTI Connext DDS Micro can use.

6.2.10.8.30 ENTITY_NAME

<<*eXtension*>> Name of the entity.

Classes

- struct [DDS_EntityNameQosPolicy](#)
Policy used to name an entity.

Macros

- #define [DDS_ENTITYNAME_QOS_NAME_MAX](#) 255
The maximum length of the name string in an [DDS_EntityNameQosPolicy](#) instance.

Functions

- bool [DDS_EntityNameQosPolicy::set_name](#) (const char *const [name](#))
Set the EntityName policy

Detailed Description

<<*eXtension*>> Name of the entity.

Macro Definition Documentation

DDS_ENTITYNAME_QOS_NAME_MAX

```
#define DDS_ENTITYNAME_QOS_NAME_MAX 255
```

The maximum length of the name string in an [DDS_EntityNameQosPolicy](#) instance.

Function Documentation

set_name()

```
bool DDS_EntityNameQosPolicy::set_name (
    const char *const name)
```

Set the EntityName policy

Set the entity name QoS policy. The maximum allowed string length is [DDS_ENTITYNAME_QOS_NAME_MAX](#) (excluding the NUL character).

Parameters

<i>name</i>	<< <i>in</i> >> The entity name to set
-------------	--

Returns

DDS_BOOLEAN_TRUE on success, DDS_BOOLEAN_FALSE on failure

References [name](#).

6.2.10.8.31 PUBLISH_MODE

<<*eXtension*>> Specifies how RTI Connex DDS Micro sends application data on the network. This QoS policy can be used to tell RTI Connex DDS Micro to use its *own* thread to send data, instead of the user thread.

Classes

- struct [DDS_PublishModeQosPolicy](#)

Specifies how RTI Connex DDS Micro sends application data on the network. This QoS policy can be used to tell RTI Connex DDS Micro to use its own thread to send data, instead of the user thread.

Macros

- #define [DDS_PUBLICATION_PRIORITY_UNDEFINED](#)
Initializer value for [DDS_PublishModeQosPolicy::priority](#).
- #define [DDS_PUBLICATION_PRIORITY_AUTOMATIC](#)
Constant value for [DDS_PublishModeQosPolicy::priority](#).

Enumerations

- enum [DDS_PublishModeQosPolicyKind](#) { [DDS_SYNCHRONOUS_PUBLISH_MODE_QOS](#), [DDS_ASYNCHRONOUS_PUBLISH_MODE_QOS](#), [DDS_AUTOMATIC_PUBLISH_MODE_QOS](#) }
- Kinds of publishing mode.*

Detailed Description

<<[eXtension](#)>> Specifies how RTI Connex DDS Micro sends application data on the network. This QoS policy can be used to tell RTI Connex DDS Micro to use its *own* thread to send data, instead of the user thread.

Macro Definition Documentation

DDS_PUBLICATION_PRIORITY_UNDEFINED

```
#define DDS_PUBLICATION_PRIORITY_UNDEFINED
```

Initializer value for [DDS_PublishModeQosPolicy::priority](#).

When assigned this value, the publication priority of the data writer will be set to the lowest possible value.

DDS_PUBLICATION_PRIORITY_AUTOMATIC

```
#define DDS_PUBLICATION_PRIORITY_AUTOMATIC
```

Constant value for [DDS_PublishModeQosPolicy::priority](#).

When assigned this value, the publication priority of the data writer will be set to the largest priority value of any sample currently queued for publication by the [DDSDataWriter](#) or [DDSDataWriter](#) channel.

Enumeration Type Documentation

DDS_PublishModeQosPolicyKind

```
enum DDS_PublishModeQosPolicyKind
```

Kinds of publishing mode.

QoS:

[DDS_PublishModeQosPolicy](#)

Enumerator

DDS_SYNCHRONOUS_PUBLISH_MODE_QOS	Indicates to send data synchronously. Data is sent immediately in the context of the user thread, no flow control is applied. [default] for DDSDataWriter
----------------------------------	---

Enumerator

DDS_ASYNCHRONOUS_PUBLISH_MODE_QOS	Indicates to send data asynchronously. Configures the DDSDDataWriter to delegate the task of data transmission to a separate thread. The FooDataWriter::write call does not send the data, but instead schedules the data to be sent later. See also DDSFlowController DDS_HistoryQosPolicy
DDS_AUTOMATIC_PUBLISH_MODE_QOS	Automatically configures the publishing mode. Configures the DDSDDataWriter to automatically determine how data transmission is performed. If the minimum data size required for the serialized type requires fragmentation, then data is handled in the DDS_ASYNCHRONOUS_PUBLISH_MODE_QOS mode. If the minimum data size does not require fragmentation, then data is sent in the DDS_SYNCHRONOUS_PUBLISH_MODE_QOS mode. See also DDSFlowController DDS_HistoryQosPolicy [default] for DDSDDataWriter

6.2.10.8.32 PARTITION

The PARTITION QoS policy provides a method to prevent Entities that have otherwise compatible QoS policies from matching with each other and thus communicating with each other.

Classes

- struct [DDS_PartitionQosPolicy](#)
A set of strings that introduces logical PARTITIONS.

Detailed Description

The PARTITION QoS policy provides a method to prevent Entities that have otherwise compatible QoS policies from matching with each other and thus communicating with each other.

6.2.10.8.33 PROPERTY

<<*eXtension*>> Stores (name, value) pairs that can be used to configure certain parameters of RTI Connex DDS Micro that are not exposed through formal QoS policies.

Classes

- struct [DDS_Property_t](#)
Properties are name/value pair objects.
- struct [DDS_PropertySeq](#)
Declares IDL sequence < [DDS_Property_t](#) >
- struct [DDS_PropertyQosPolicy](#)
Stores name/value(string) pairs that can be used to configure certain parameters of RTI Connex DDS Micro that are not exposed through formal QoS policies.
- class [DDSPROPERTYQOSPolicyHelper](#)
Policy helpers that facilitate management of the properties in the input policy.

Detailed Description

<<[eXtension](#)>> Stores (name, value) pairs that can be used to configure certain parameters of RTI Connex DDS Micro that are not exposed through formal QoS policies.

6.2.10.8.34 TRANSPORT_PRIORITY

This QoS policy allows the application to take advantage of transports that are capable of sending messages with different priorities.

Classes

- struct [DDS_TransportPriorityQosPolicy](#)
This QoS policy allows the application to take advantage of transports that are capable of sending messages with different priorities.

Detailed Description

This QoS policy allows the application to take advantage of transports that are capable of sending messages with different priorities.

6.2.10.8.35 USER_DATA

The USER_DATA QoS policy allows the user to attach a buffer of opaque data to a [DDSDomainParticipant](#), [DDSDataWriter](#), or [DDSDataReader](#). How this information is used will be up to the user code; RTI Connex DDS Micro distributes this information to other applications as part of the discovery process, but RTI Connex DDS Micro does not interpret the information.

Classes

- struct [DDS_UserDataQosPolicy](#)
Attaches a buffer of opaque data to a [DDSEntity](#) which is distributed to other applications as part of the discovery process.

Detailed Description

The USER_DATA QoS policy allows the user to attach a buffer of opaque data to a [DDSDomainParticipant](#), [DDSDataWriter](#), or [DDSDataReader](#). How this information is used will be up to the user code; RTI Connex DDS Micro distributes this information to other applications as part of the discovery process, but RTI Connex DDS Micro does not interpret the information.

6.2.10.8.36 GROUP_DATA

The GROUP_DATA QoS policy allows you to attach a buffer of opaque data to a [DDSPublisher](#) or [DDSSubscriber](#). The use of this information is up to user code; RTI Connex DDS Micro distributes this information to other applications as part of the discovery process, but RTI Connex DDS Micro does not interpret the information.

Classes

- struct [DDS_GroupDataQosPolicy](#)

Attaches a buffer of opaque data to a [DDSEntity](#) which is distributed to other applications as part of the discovery process.

Detailed Description

The GROUP_DATA QoS policy allows you to attach a buffer of opaque data to a [DDSPublisher](#) or [DDSSubscriber](#). The use of this information is up to user code; RTI Connex DDS Micro distributes this information to other applications as part of the discovery process, but RTI Connex DDS Micro does not interpret the information.

6.2.10.8.37 TOPIC_DATA

The TOPIC_DATA QoS policy allows the user to attach a buffer of opaque data to a [DDSTopic](#). How this information is used will be up to the user code; RTI Connex DDS Micro distributes this information to other applications as part of the discovery process, but RTI Connex DDS Micro does not interpret the information.

Classes

- struct [DDS_TopicDataQosPolicy](#)

Attaches a buffer of opaque data to a [DDSTopic](#) which is distributed to other applications as part of the discovery process.

Detailed Description

The TOPIC_DATA QoS policy allows the user to attach a buffer of opaque data to a [DDSTopic](#). How this information is used will be up to the user code; RTI Connex DDS Micro distributes this information to other applications as part of the discovery process, but RTI Connex DDS Micro does not interpret the information.

6.2.10.8.38 DATA_WRITER_TRANSFER_MODE

<<*eXtension*>> Specifies the [DDSDDataWriter](#) transfer mode QoS.

Classes

- struct [DDS_DataWriterShmemRefTransferModeSettings](#)
Settings related to transferring data using shared memory references.
- struct [DDS_DataWriterTransferModeQosPolicy](#)
<<*eXtension*>> *QoS related to transferring data*

Variables

- struct [DDS_DataWriterTransferModeQosPolicy](#) [DDS_DataWriterQos::transfer_mode](#)
<<*eXtension*>> *QoS related to transferring data*

Detailed Description

<<*eXtension*>> Specifies the [DDSDDataWriter](#) transfer mode QoS.

Variable Documentation

transfer_mode

```
struct DDS\_DataWriterTransferModeQosPolicy DDS\_DataWriterQos::transfer\_mode
```

<<*eXtension*>> QoS related to transferring data

It contains qualitative settings related to the actions a [DDSDDataWriter](#) performs while transferring its data.

Entity:

[DDSDDataWriter](#)

Properties:

[RxO](#) = N/A
[Changeable](#) = NO

6.2.10.9 Conditions and WaitSets

[DDSDCondition](#) and [DDSDWaitSet](#) and related items.

Classes

- class [DDSCondition](#)
<<interface>> Root class for all the conditions that may be attached to a [DDSWaitSet](#).
- class [DDSGuardCondition](#)
<<interface>> A specific [DDSCondition](#) whose *trigger_value* is completely under the control of the application.
- class [DDSStatusCondition](#)
<<interface>> A specific [DDSCondition](#) that is associated with each [DDSEntity](#).
- class [DDSWaitSet](#)
<<interface>> Allows an application to wait until one or more of the attached [DDSCondition](#) objects has a *trigger_value* of [DDS_BOOLEAN_TRUE](#) or else until the timeout expires.

6.2.10.9.1 Detailed Description

[DDSCondition](#) and [DDSWaitSet](#) and related items.

6.2.10.10 Entity Support

[DDSEntity](#), [DDSListener](#) and related items.

Classes

- class [DDSListener](#)
<<interface>> Abstract base class for all Listener interfaces.
- class [DDSDomainEntity](#)
<<interface>> Abstract base class for all DDS entities except for the [DDSDomainParticipant](#).
- class [DDSEntity](#)
<<interface>> Abstract base class for all the DDS objects that support QoS policies, and a listener.

Enumerations

- enum [DDS_EntityKind_t](#) {
[DDS_UNKNOWN_ENTITY_KIND](#) = 0 , [DDS_PARTICIPANT_ENTITY_KIND](#) = 1 , [DDS_PUBLISHER_ENTITY_KIND](#) = 2 , [DDS_SUBSCRIBER_ENTITY_KIND](#) = 3 ,
[DDS_TOPIC_ENTITY_KIND](#) = 4 , [DDS_DATAREADER_ENTITY_KIND](#) = 5 , [DDS_DATAWRITER_ENTITY_KIND](#) = 6 }
Type of a DDS entity.

6.2.10.10.1 Detailed Description

[DDSEntity](#), [DDSListener](#) and related items.

[DDSEntity](#) subtypes are created and destroyed by factory objects. With the exception of [DDSDomainParticipant](#), whose factory is [DDSDomainParticipantFactory](#), all [DDSEntity](#) factory objects are themselves [DDSEntity](#) subtypes as well.

Important: all [DDSEntity](#) delete operations are inherently thread-unsafe. The user must take extreme care that a given [DDSEntity](#) is not destroyed in one thread while being used concurrently (including being deleted concurrently) in another thread. An operation's effect in the presence of the concurrent deletion of the operation's target [DDSEntity](#) is undefined.

6.2.10.10.2 Enumeration Type Documentation

DDS_EntityKind_t

enum [DDS_EntityKind_t](#)

Type of a DDS entity.

Enumerator

DDS_UNKNOWN_ENTITY_KIND	Entity is of an unknown type.
DDS_PARTICIPANT_ENTITY_KIND	Entity is a DDSDomainParticipant .
DDS_PUBLISHER_ENTITY_KIND	Entity is a DDSPublisher .
DDS_SUBSCRIBER_ENTITY_KIND	Entity is a DDSSubscriber .
DDS_TOPIC_ENTITY_KIND	Entity is a DDSTopic .
DDS_DATAREADER_ENTITY_KIND	Entity is a DDSDataReader .
DDS_DATAWRITER_ENTITY_KIND	Entity is a DDSDataWriter .

6.2.10.11 Sequence Support

Defines sequence interface and primitive data types sequences.

Classes

- struct [FooSeq](#)

<<interface>> <<generic>> A type-safe, ordered collection of elements. The type of these elements is referred to in this documentation as "Foo".

6.2.10.11.1 Detailed Description

Defines sequence interface and primitive data types sequences.

6.2.10.12 Exception codes

Exception codes.

6.2.10.12.1 Detailed Description

Exception codes.

6.2.10.13 Type Support

Type Plugin.

Classes

- struct [NDDS_Type_PluginVersion](#)
Data type used to keep track of versioning information about a ::DDS_TypePluginI.

Typedefs

- typedef struct [DDS_BuiltinTopicKey_t](#) [DDS_InstanceId_t](#)
The data structure by which RTI Connex DDS Micro identifies keys in user data types.
- typedef struct NDDS_Type_PluginVersion [NDDS_Type_PluginVersion](#)
Data type used to keep track of versioning information about a ::DDS_TypePluginI.

Enumerations

- enum [NDDS_TypePluginKeyKind](#) { [NDDS_TYPEPLUGIN_NO_KEY](#) , [NDDS_TYPEPLUGIN_GUID_KEY](#) , [NDDS_TYPEPLUGIN_USER_KEY](#) }
Type key kind: either keyed or unkeyed.

6.2.10.13.1 Detailed Description

Type Plugin.

6.2.10.13.2 Typedef Documentation

DDS_InstanceId_t

```
typedef struct DDS_BuiltinTopicKey_t DDS_InstanceId_t
```

The data structure by which RTI Connex DDS Micro identifies keys in user data types.

RTI Connex DDS Micro propagates and differentiates between instances of keyed types using [DDS_InstanceId_t](#) . Before a sample of a keyed type can be written, its key must be translated into one of these.

Users who do not edit the code generated by `rtiddsgen` do not need to deal with this type.

This type is structurally identical to the key type of the built-in topic types. A one-to-one mapping must exist from the key fields of a user data sample to the values of this type's fields.

See also

[DDS_BuiltinTopicKey_t](#)

NDDS_Type_PluginVersion

```
typedef struct NDDS_Type_PluginVersion NDDS_Type_PluginVersion
```

Data type used to keep track of versioning information about a `::DDS_TypePluginI`.

6.2.10.13.3 Enumeration Type Documentation

NDDS_TypePluginKeyKind

```
enum NDDS_TypePluginKeyKind
```

Type key kind: either keyed or unkeyed.

This type defines whether a user data type is keyed or unkeyed.

Enumerator

NDDS_TYPEPLUGIN_NO_KEY	Data is not keyed.
NDDS_TYPEPLUGIN_USER_KEY	Data is keyed.

6.2.10.14 String Support

String creation, cloning, assignment, and deletion.

Functions

- `char * DDS_String_alloc (DDS_UnsignedLong length)`
Create a new empty string that can hold up to `length` characters.
- `char * DDS_String_dup (const char *str)`
Clone a string. Creates a new string that duplicates the value of `string`.
- `void DDS_String_free (char *str)`
Delete a string.
- `RTI_INT32 DDS_String_cmp (const char *s1, const char *s2)`
Compare two strings.
- `RTI_INT32 DDS_String_ncmp (const char *s1, const char *s2, DDS_UnsignedLong num)`
Compare two strings.
- `RTI_INT32 DDS_String_length (const char *string)`
Return the length of a string.

6.2.10.14.1 Detailed Description

String creation, cloning, assignment, and deletion.

The methods in this class ensure consistent cross-platform implementations for string creation (`DDS_String_alloc()`), deletion (`DDS_String_free()`), and cloning (`DDS_String_dup()`) that preserve the mutable value type semantics. These are to be viewed as methods that define a `string` class whose data is represented by a `'char*'`.

6.2.10.14.2 Conventions

The following conventions govern the memory management of strings in RTI Connex DDS Micro.

- The DDS implementation ensures that when value types containing strings are passed back and forth to the DDS APIs, the strings are created/deleted/assigned/cloned using the `string` class methods.
 - Value types containing strings have ownership of the contained string. Thus, when a value type is deleted, the contained string field is also deleted.
- The user must ensure that when value types containing strings are passed back and forth to the DDS APIs, the strings are created/deleted/assigned/cloned using the `String` class methods.

The representation of a string in C/C++ unfortunately does not allow programs to detect how much memory has been allocated for a string. RTI Connex DDS Micro must therefore make some assumptions when a user requests that a string be copied into. The following rules apply when RTI Connex DDS Micro is copying into a string or string sequence:

- If the 'char*' is NULL, RTI Connex DDS Micro will log a warning and allocate a new string on behalf of the user. *To avoid leaking memory, you must ensure that the string will be freed (see [Usage](#) below).*
- If the 'char*' is not NULL, RTI Connex DDS Micro will assume that you are managing the string's memory yourself and have allocated enough memory to store the string to be copied. *RTI Connex DDS Micro will copy into your memory; to avoid memory corruption, be sure to allocate enough of it. Also, do not pass structures containing junk pointers into RTI Connex DDS Micro; you are likely to crash.*

6.2.10.14.3 Usage

This requirement can generally be assured by adhering to the following *idiom* for manipulating strings.

Always use
 `DDS_String_alloc()` to create,
 `DDS_String_dup()` to clone,
 `DDS_String_free()` to delete
 a string 'char*' that is passed back and forth between
 user code and the DDS C/C++ APIs.

Not adhering to this idiom can result in bad pointers, and incorrect memory being freed.

In addition, the user code should be vigilant to avoid memory leaks. It is good practice to:

- Balance occurrences of `DDS_String_alloc()`, `DDS_String_dup()`, with matching occurrences of `DDS_String_free()` in the code.
- Finalize value types containing strings. In C++ the destructor accomplishes this automatically. In C, explicit "destructor" functions are provided; these functions are typically called "finalize."

6.2.10.14.4 Function Documentation

DDS_String_alloc()

```
char * DDS_String_alloc (
    DDS_UnsignedLong length)
```

Create a new empty string that can hold up to `length` characters.

A string created by this method must be deleted using `DDS_String_free()`.

This function will allocate enough memory to hold a string of `length` characters, **plus** one additional byte to hold the NULL terminating character.

Parameters

<i>length</i>	<< <i>in</i> >> Capacity of the string.
---------------	---

Returns

A newly created non-NULL string upon success or NULL upon failure.

DDS_String_dup()

```
char * DDS_String_dup (  
    const char * str)
```

Clone a string. Creates a new string that duplicates the value of *string*.

A string created by this method must be deleted using [DDS_String_free\(\)](#)

Parameters

<i>str</i>	<< <i>in</i> >> The string to duplicate.
------------	--

Returns

If *string* == NULL, this method always returns NULL. Otherwise, upon success it returns a newly created string whose value is *string*; upon failure it returns NULL.

DDS_String_free()

```
void DDS_String_free (  
    char * str)
```

Delete a string.

Precondition

string must be either NULL, or must have been created using [DDS_String_alloc\(\)](#), [DDS_String_dup\(\)](#)

Parameters

<i>str</i>	<< <i>in</i> >> The string to delete.
------------	---------------------------------------

References [RTI_INT32](#).

DDS_String_cmp()

```
RTI\_INT32 DDS_String_cmp (  
    const char * s1,  
    const char * s2)
```

Compare two strings.

Precondition

s1 and *s2* can be NULL or non-NULL.

Parameters

<i>s1</i>	<< <i>in</i> >> String 1 to compare.
<i>s2</i>	<< <i>in</i> >> String 2 to compare.

Returns

0 if *s1* equals *s2*, positive integer if *s1* is greater than *s2*, negative integer if *s1* is less than *s2*

References [RTI_INT32](#).

DDS_String_ncmp()

```
RTI_INT32 DDS_String_ncmp (
    const char * s1,
    const char * s2,
    DDS_UnsignedLong num)
```

Compare two strings.

This function compares two strings for lexicographic equivalence, but only up to *num* bytes.

Parameters

in	<i>s1</i>	<< <i>in</i> >> Left side of comparison
in	<i>s2</i>	<< <i>in</i> >> Right side of comparison
in	<i>num</i>	Maximum number of bytes to compare

Returns

0 if *s1* equals *s2*, positive integer if *s1* is greater than *s2*, negative integer if *s1* is less than *s2*

References [RTI_INT32](#).

DDS_String_length()

```
RTI_INT32 DDS_String_length (
    const char * string)
```

Return the length of a string.

Precondition

string must be non-NULL

Parameters

<i>string</i>	<< <i>in</i> >> String to return length of
---------------	--

Returns

length of string. Does not include the NUL-termination character. If a NULL value is passed as argument, RTI_↵ SIZE_INVALID will be returned.

References [RTI_INT32](#), and [RTI_UINT32](#).

6.2.11 General Utilities and Compliance Configuration

Topics

- [Compliance Configuration](#)
APIs to configure compliance with certain standard specifications.
- [Property Reference](#)
This is the property reference for RTI Connex DDS Micro.

6.2.11.1 Detailed Description

6.2.11.2 Compliance Configuration

APIs to configure compliance with certain standard specifications.

APIs to configure compliance with certain standard specifications.

6.2.11.3 Property Reference

This is the property reference for RTI Connex DDS Micro.

Variables

- `const char * DDS\_XTYPES\_COMPLIANCE\_MASK\_PROPERTY`
Set the ::NDDS_Config_XTypesComplianceMask.

6.2.11.3.1 Detailed Description

This is the property reference for RTI Connex DDS Micro.

The goal of this Property Reference Guide is to gather all RTI Connex DDS Micro properties in one place and provide the main characteristics of each.

All the properties that RTI Connex DDS Micro provides (which begin with `dds.`, `com.rti.`, or `rti.`) are validated when the applicable entity is created. Please refer to [PROPERTY](#) for information about properties and the Property API.

6.2.11.3.2 Variable Documentation

DDS_XTYPES_COMPLIANCE_MASK_PROPERTY

```
const char* DDS_XTYPES_COMPLIANCE_MASK_PROPERTY [extern]
```

Set the ::NDDS_Config_XTypesComplianceMask.

This property sets the ::NDDS_Config_XTypesComplianceMask on a [DDSDomainParticipant](#) or [DDSDataWriter](#). If it is set on the [DDSDomainParticipant](#) and [DDSDataWriter](#), the value set on the [DDSDataWriter](#) takes precedence.

6.3 DPSE Static Discovery API

Classes

- struct [DPSE_DiscoveryPluginProperty](#)
<<eXtension>> Properties for the Dynamic Participant/Static Endpoint (DPSE) discovery plugin. These properties can be configured while registering the plugin to control its behavior.
- class [DPSEDiscoveryFactory](#)
<<eXtension>> A discovery factory that will be used by the [DDSDomainParticipantFactory](#) to create a discovery plugin.
- class [DPSEDiscoveryPlugin](#)
<<eXtension>> Discovery plug-in used for asserting remote entities.

Macros

- #define [DPSE_DiscoveryPluginProperty_INITIALIZER](#)
Initializer for the [DPSE_DiscoveryPluginProperty](#).

6.3.1 Detailed Description

<<eXtension>> The RTI Connex Micro support for DPSE discovery of remote entities. Please refer to the Discovery chapter in the User's Manual for an example of how to register and use the DPSE plugin.

6.3.2 Macro Definition Documentation

DPSE_DiscoveryPluginProperty_INITIALIZER

```
#define DPSE_DiscoveryPluginProperty_INITIALIZER
```

Value:

```
{ \
    RT_ComponentFactoryProperty_INITIALIZER, \
    {30,0}, \
    {100,0}, \
    5, \
    {1,0}, \
    DDS_LENGTH_AUTO, \
    4 \
    DDS_PARTICIPANT_MESSAGE_READER_RELIABILITY_KIND_INITIALIZER \
}
```

Initializer for the [DPSE_DiscoveryPluginProperty](#).

6.4 DPDE Dynamic Discovery API

Classes

- struct [DPDE_DiscoveryPluginProperty](#)
<<eXtension>> Properties for the Dynamic Participant/Dynamic Endpoint (DPDE) discovery plugin. This includes all discovery timing properties for participant discovery.
- class [DPDEDiscoveryFactory](#)
<<eXtension>> Discovery plug-in used for asserting remote entities.

Macros

- #define [DPDE_DiscoveryPluginProperty_INITIALIZER](#)
Initializer for the [DPDE_DiscoveryPluginProperty](#).

6.4.1 Detailed Description

<<eXtension>> The RTI Connex DDS Micro support for DPDE discovery of remote entities. Please refer to Discovery chapter in the User's Manual, and the Getting Started Guide, for an example of how to register and use the DPDE plugin.

6.4.2 Macro Definition Documentation

DPDE_DiscoveryPluginProperty_INITIALIZER

```
#define DPDE_DiscoveryPluginProperty_INITIALIZER
```

Value:

```
{ \
    RT_ComponentFactoryProperty_INITIALIZER, \
    {30,0}, \
    {100,0}, \
    5, \
    {1,0}, \
    DDS_BOOLEAN_FALSE, \
    DDS_LENGTH_AUTO, \
    4, \
    8, \
    DDS_MAX_UNLIMITED, \
    DDS_LENGTH_UNLIMITED, \
    {0,100000000}, \
    DDS_LENGTH_UNLIMITED, \
    DDS RTPS_NACK_PERIOD_DEFAULT \
    DDS_PARTICIPANT_MESSAGE_READER_RELIABILITY_KIND_INITIALIZER \
}
```

Initializer for the [DPDE_DiscoveryPluginProperty](#).

6.5 CDR Stream API

<<eXtension>> CDR Stream API of RTI Connex DDS Micro.

Classes

- struct [CDR_Stream_t](#)
Byte stream abstraction for CDR serialization and deserialization.

Functions

- [RTI_UINT32 CDR_Stream_get_current_position_offset](#) (struct [CDR_Stream_t](#) *cdrs)
Return current offset of stream pointer relative to buffer
- [RTI_BOOL CDR_Stream_set_current_position_offset](#) (struct [CDR_Stream_t](#) *cdrs, [RTI_UINT32](#) num)
Set current offset of stream pointer relative to buffer
- [RTI_BOOL CDR_Stream_increment_current_position](#) (struct [CDR_Stream_t](#) *me, [RTI_INT32](#) amount)
Increase stream pointer by specified number of bytes
- [RTI_BOOL CDR_Stream_check_size](#) (struct [CDR_Stream_t](#) *me, [RTI_UINT32](#) size)
Verifies whether remaining space in stream buffer is larger than specified size
- char * [CDR_Stream_get_current_position_ptr](#) (struct [CDR_Stream_t](#) *me)
Returns a pointer to the current offset in the stream
- [RTI_BOOL CDR_Stream_is_byte_swapped](#) (struct [CDR_Stream_t](#) *me)
Whether a stream is byte-swapped

6.5.1 Detailed Description

<<*eXtension*>> CDR Stream API of RTI Connex DDS Micro.

The CDR Stream API provides functions to interacting with byte streams for CDR serialization and deserialization.

6.5.2 Function Documentation**CDR_Stream_get_current_position_offset()**

```
RTI\_UINT32 CDR_Stream_get_current_position_offset (
    struct CDR\_Stream\_t * cdrs)
```

Return current offset of stream pointer relative to buffer

Parameters

in	<i>cdrs</i>	Self
----	-------------	------

Returns

The number of bytes between the stream buffer and the current stream pointer.

CDR_Stream_set_current_position_offset()

```
RTI\_BOOL CDR_Stream_set_current_position_offset (
    struct CDR\_Stream\_t * cdrs,
    RTI\_UINT32 num)
```

Set current offset of stream pointer relative to buffer

Parameters

in	<i>cdrs</i>	Self
in	<i>num</i>	Offset in bytes to set stream pointer ahead of buffer

Returns

RTI_TRUE on success with stream pointer updated. Otherwise, RTI_FALSE on failure with stream pointer unchanged.

References [RTI_UINT32](#).

CDR_Stream_increment_current_position()

```
RTI_BOOL CDR_Stream_increment_current_position (  
    struct CDR_Stream_t * me,  
    RTI_INT32 amount)
```

Increase stream pointer by specified number of bytes

Parameters

in	<i>me</i>	Self
in	<i>amount</i>	Number of bytes to increase stream pointer

Returns

RTI_TRUE on success with stream pointer incremented. RTI_FALSE on failure with stream pointer unchanged.

References [RTI_INT32](#), and [RTI_UINT32](#).

CDR_Stream_check_size()

```
RTI_BOOL CDR_Stream_check_size (  
    struct CDR_Stream_t * me,  
    RTI_UINT32 size)
```

Verifies whether remaining space in stream buffer is larger than specified size

Parameters

in	<i>me</i>	Self
in	<i>size</i>	Number of bytes to check whether fits in stream

Returns

RTI_TRUE if size fits within stream, otherwise RTI_FALSE.

References [RTI_UINT32](#).

CDR_Stream_get_current_position_ptr()

```
char * CDR_Stream_get_current_position_ptr (  
    struct CDR_Stream_t * me)
```

Returns a pointer to the current offset in the stream

Parameters

<code>in</code>	<code>me</code>	<code>Self</code>
-----------------	-----------------	-------------------

Returns

Pointer to current offset in stream on success, NULL on failure.

References [RTI_UINT32](#).

CDR_Stream_is_byte_swapped()

```
RTI_BOOL CDR_Stream_is_byte_swapped (  
    struct CDR_Stream_t * me)
```

Whether a stream is byte-swapped

This function verifies whether a stream is set to serialize or deserialize in byte-swapped order, relative to its host endianness.

Parameters

<code>in</code>	<code>me</code>	<code>Self</code>
-----------------	-----------------	-------------------

Returns

RTI_TRUE if stream is byte-swapped

References [RTI_UINT32](#), and [RTI_UINT8](#).

6.6 RT Run-Time API

Classes

- class [RTRegistry](#)
<<eXtension>> A run-time component factory.

6.6.1 Detailed Description

The run-time module (RT) manages RTI Connex Micro modules. It provides a common interface to add, remove, and lookup components.

6.7 RHSM Reader History API

Classes

- class [RHSMHistoryFactory](#)
<<eXtension>> A reader history factory that will be used by the DataReader to create a history cache.

6.7.1 Detailed Description

The RHSM plugin is an implementation of the RTI Connex DDS Micro ReaderHistory plugin interface and implements the DataReader reader cache as supported by RTI Connex DDS Micro. RTI Connex DDS Micro requires that a ReaderHistory plugin is registered with the name "rh" in the DomainParticipantFactory before a DataReader can be created. Each instance of a DataReader creates its own instance of the ReaderHistory plugin. Please refer to User Manual for an example on how to register the writer history factory.

6.8 WHSM Writer History API

Classes

- class [WHSMHistoryFactory](#)
<<eXtension>> A writer history factory that will be used by the DataWriter to create a history cache.

6.8.1 Detailed Description

The WHSM plugin is an implementation of the RTI Connex DDS Micro WriterHistory plugin interface and implements the DataWriter writer cache as supported by RTI Connex DDS Micro. RTI Connex DDS Micro requires that a WriterHistory plugin is registered with the name "wh" in the DomainParticipantFactory before a DataWriter can be created. Each instance of a DataWriter creates its own instance of the WriterHistory plugin. Please refer to User Manual for an example on how to register the writer history factory.

6.9 NETIO API

<<eXtension>> The NETIO API.

Topics

- [NETIO Packet API](#)
NETIO_Packet implementation.
- [NETIO Interface API](#)
- [NETIO Address API](#)

6.9.1 Detailed Description

<<*eXtension*>> The NETIO API.

6.9.2 NETIO Packet API

[NETIO_Packet](#) implementation.

Classes

- struct [NETIO_Packet](#)
Implementation of the abstract [NETIO_Packet](#) type.

Typedefs

- typedef struct [NETIO_Packet](#) [NETIO_Packet_T](#)
A NETIO Packet.

6.9.2.1 Detailed Description

[NETIO_Packet](#) implementation.

6.9.2.2 Typedef Documentation

NETIO_Packet_T

```
typedef struct NETIO\_Packet NETIO\_Packet\_T
```

A NETIO Packet.

The NETIO layers passed [NETIO_Packet](#) structures upstream/downstream.

6.9.3 NETIO Interface API

Classes

- struct [NETIO_Interface](#)
Base-class definition for all NETIO interfaces.
- struct [NETIO_InterfaceI](#)

Typedefs

- typedef struct [NETIO_Interface](#) [NETIO_Interface_T](#)
- typedef [RTI_BOOL](#)(* [NETIO_Interface_receiveFunc](#)) ([NETIO_Interface_T](#) *netio_intf, struct [NETIO_Address](#) *src_addr, struct [NETIO_Address](#) *dst_addr, [NETIO_Packet_T](#) *packet)
Definition of the ::DDSNETIO_Interface receive method.
- typedef [RTI_BOOL](#)(* [NETIO_Interface_reserve_addressFunc](#)) ([NETIO_Interface_T](#) *self, struct [NETIO_AddressSeq](#) *req_address, struct [NETIO_AddressSeq](#) *rsvd_address, struct [NETIOBindProperty](#) *property)
<<eXtension>> Definition of the ::DDSNETIO_Interface reserve_address method

6.9.3.1 Detailed Description

6.9.3.2 Typedef Documentation

NETIO_Interface_T

```
typedef struct NETIO\_Interface NETIO\_Interface\_T
```

<<eXtension>> Abstract NETIO Interface type.

NETIO_Interface_receiveFunc

```
typedef RTI\_BOOL(* NETIO\_Interface\_receiveFunc) (NETIO\_Interface\_T *netio_intf, struct NETIO\_Address *src_addr, struct NETIO\_Address *dst_addr, NETIO\_Packet\_T *packet)
```

Definition of the ::DDSNETIO_Interface receive method.

<<eXtension>> [NETIO_Interface](#) send function.

Parameters

in	<i>netio_intf</i>	NETIO interface to receive in
in	<i>src_addr</i>	The source address of the packet
in	<i>dst_addr</i>	The destination address for the packet
in	<i>packet</i>	The forwarded packet from downstream

Returns

RTI_TRUE on success, RTI_FALSE on failure

See Also [::DDSNETIO_Interface::receive](#), [::DDSNETIO_Interface](#)

NETIO_Interface_reserve_addressFunc

```
typedef RTI\_BOOL(* NETIO\_Interface\_reserve\_addressFunc) (NETIO\_Interface\_T *self, struct NETIO\_AddressSeq *req_address, struct NETIO\_AddressSeq *rsvd_address, struct NETIOBindProperty *property)
```

<<eXtension>> Definition of the ::DDSNETIO_Interface reserve_address method

Parameters

in	<i>self</i>	NETIO interface to reserve addresses on
in	<i>req_address</i>	List of requested addresses
in, out	<i>rsvd_address</i>	The downstream interface
in	<i>property</i>	Properties to use to listen on the reserved addresses

Returns

RTI_TRUE on success, RTI_FALSE on failure.

6.9.4 NETIO Address API

Classes

- struct [NETIO_AddressUInt32](#)
<<eXtension>> NETIO_AddressUInt32
- union [NETIO_AddressValue](#)
<<eXtension>> NETIO_AddressValue
- struct [NETIO_Address](#)
- struct [NETIO_Netmask](#)
<<eXtension>> NETIO_Netmask.
- struct [NETIO_AddressSeq](#)
<<eXtension>> The NETIO_AddressSeq sequence type.
- struct [NETIO_NetmaskSeq](#)
<<eXtension>> The NETIO_NetmaskSeq sequence type.

Macros

- #define [NETIO_ADDRESS_KIND_UDPv4](#) 1
<<eXtension>> The UDPv4 NETIO Address.
- #define [NETIO_ADDRESS_KIND_UDPv6](#) 2
<<eXtension>> The UDPv6 NETIO Address.
- #define [NETIO_ADDRESS_RTPS_RESERVED_LOW](#) 0x02000000
<<eXtension>> Addresses in the range [NETIO_ADDRESS_RTPS_RESERVED_LOW, NETIO_ADDRESS_RTPS_RESERVED_HIGH] are reserved by the RTPS standard.
- #define [NETIO_ADDRESS_RTPS_RESERVED_HIGH](#) 0x02ffffff
<<eXtension>> Addresses in the range [NETIO_ADDRESS_RTPS_RESERVED_LOW, NETIO_ADDRESS_RTPS_RESERVED_HIGH] are reserved by the RTPS standard.
- #define [NETIO_Netmask_INITIALIZER](#)
<<eXtension>> Initialize a NETIO_Netmask.
- #define [NETIO_Address_INITIALIZER](#)
<<eXtension>> Initialize a NETIO_Address.

6.9.4.1 Detailed Description

6.9.4.2 Macro Definition Documentation

NETIO_ADDRESS_KIND_UDPv4

```
#define NETIO_ADDRESS_KIND_UDPv4 1
```

<<*eXtension*>> The UDPv4 NETIO Address.

NETIO_ADDRESS_KIND_UDPv6

```
#define NETIO_ADDRESS_KIND_UDPv6 2
```

<<*eXtension*>> The UDPv6 NETIO Address.

NETIO_ADDRESS_RTPS_RESERVED_LOW

```
#define NETIO_ADDRESS_RTPS_RESERVED_LOW 0x02000000
```

<<*eXtension*>> Addresses in the range [NETIO_ADDRESS_RTPS_RESERVED_LOW, NETIO_ADDRESS_RTPS_RESERVED_HIGH] are reserved by the RTPS standard.

NETIO_ADDRESS_RTPS_RESERVED_HIGH

```
#define NETIO_ADDRESS_RTPS_RESERVED_HIGH 0x02ffffff
```

<<*eXtension*>> Addresses in the range [NETIO_ADDRESS_RTPS_RESERVED_LOW, NETIO_ADDRESS_RTPS_RESERVED_HIGH] are reserved by the RTPS standard.

NETIO_Netmask_INITIALIZER

```
#define NETIO_Netmask_INITIALIZER
```

Value:

```
{ \
    0, \
    {0, 0, 0, 0} \
}
```

<<*eXtension*>> Initialize a NETIO_Netmask.

NETIO_Address_INITIALIZER

```
#define NETIO_Address_INITIALIZER
```

Value:

```
{ \
    NETIO_ADDRESS_KIND_RESERVED, /* kind */ \
    0, /* port */ \
    {{0,0,0,0}}, /* value */ \
    {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0} \
}
```

<<eXtension>> Initialize a `NETIO_Address`.

6.10 UDP Transport API

Classes

- struct `UDP_NatEntry`
 <<eXtension>> Properties for a Network Address Translation entry
- struct `UDP_NatEntrySeq`
 <<eXtension>> Declares IDL sequence<`UDP_NatEntry`>
- struct `UDP_InterfaceTableEntry`
 Generic structure to describe a network interface.
- struct `UDP_InterfaceTableEntrySeq`
 <<eXtension>> Declares IDL sequence<`UDP_InterfaceTableEntry`>
- struct `UDP_InterfaceFactoryProperty`
 <<eXtension>> Properties for the UDP Transport.
- class `UDPInterfaceFactory`
 <<eXtension>> A UDP transport factory that will be used by a `DomainParticipant` to create a UDP interface. Only one UDP interface can be registered with the `DomainParticipantFactory`.
- class `UDPInterfaceTable`

Macros

- #define `UDP_INTERFACE_MAX_IFNAME` 64
 Maximum length of a UDP network interface.
- #define `UDP_INTERFACE_INTERFACE_UP_FLAG` 0x1U
 Generic flag to indicate if a network stack is up.
- #define `UDP_INTERFACE_INTERFACE_MULTICAST_FLAG` 0x2U
 Generic flag to indicate if a network stack supports multicast.
- #define `UDP_INTERFACE_MAX_NETMASK_BITS` (32U)
 The maximum number of bits in a UDP netmask.

Functions

- NETIOPSLDIIExport `RTI_BOOL UDP_InterfaceFactoryProperty_initialize` (struct `UDP_InterfaceFactoryProperty *self`)
Initialize a UDP factory property.
- NETIOPSLDIIExport `RTI_BOOL UDP_InterfaceFactoryProperty_finalize` (struct `UDP_InterfaceFactoryProperty *self`)
Finalize a UDP factory property.
- static bool `UDPInterfaceTable::add_entry` (struct `UDP_InterfaceTableEntrySeq *seq`, `RTI_UINT32` address, `RTI_UINT32` netmask, const char *ifname, `RTI_UINT32` flags)
Add an entry to the list of available interfaces.

6.10.1 Detailed Description

<<eXtension>> The UDP transport implements the RTI Connex DDS Micro NETIO interface and supports UDPv4. Please refer to the User Manual for information about how to use the UDP transport.

6.10.2 Macro Definition Documentation

UDP_INTERFACE_MAX_IFNAME

```
#define UDP_INTERFACE_MAX_IFNAME 64
```

Maximum length of a UDP network interface.

UDP_INTERFACE_INTERFACE_UP_FLAG

```
#define UDP_INTERFACE_INTERFACE_UP_FLAG 0x1U
```

Generic flag to indicate if a network stack is up.

UDP_INTERFACE_INTERFACE_MULTICAST_FLAG

```
#define UDP_INTERFACE_INTERFACE_MULTICAST_FLAG 0x2U
```

Generic flag to indicate if a network stack supports multicast.

UDP_INTERFACE_MAX_NETMASK_BITS

```
#define UDP_INTERFACE_MAX_NETMASK_BITS (32U)
```

The maximum number of bits in a UDP netmask.

6.10.3 Function Documentation

UDP_InterfaceFactoryProperty_initialize()

```
NETIOPSLDIIExport RTI_BOOL UDP_InterfaceFactoryProperty_initialize (
    struct UDP_InterfaceFactoryProperty * self)
```

Initialize a UDP factory property.

Parameters

in	<i>self</i>	Property to initialize
----	-------------	------------------------

Returns

RTI_TRUE on success, RTI_FALSE on failure

UDP_InterfaceFactoryProperty_finalize()

```
NETIOPSLD11Export RTI_BOOL UDP_InterfaceFactoryProperty_finalize (
    struct UDP_InterfaceFactoryProperty * self)
```

Finalize a UDP factory property.

Parameters

in	<i>self</i>	Property to finalize
----	-------------	----------------------

Returns

RTI_TRUE on success, RTI_FALSE on failure

add_entry()

```
static bool UDPInterfaceTable::add_entry (
    struct UDP_InterfaceTableEntrySeq * seq,
    RTI_UINT32 address,
    RTI_UINT32 netmask,
    const char * ifname,
    RTI_UINT32 flags) [static]
```

Add an entry to the list of available interfaces.

This function can be used to add an interface to the if_table in the [UDP_InterfaceFactoryProperty](#).

Parameters

in	<i>seq</i>	The sequence to add the entry to
in	<i>address</i>	The UDPv4 interface address to add in host order
in	<i>netmask</i>	The interface netmask, maximum of UDP_INTERFACE_MAX_NETMASK_BITS bits.
in	<i>ifname</i>	The interface name. Cannot exceed UDP_INTERFACE_MAX_IFNAME including NUL.
in	<i>flags</i>	Bitmap of UDP_INTERFACE_INTERFACE_UP_FLAG and/or UDP_INTERFACE_INTERFACE_MULTICAST_FLAG

Returns

RTI_TRUE on success, RTI_FALSE on failure

References [RTI_UINT32](#).

6.11 UDP Transform API

<<eXtension>> The UDP transform API enables custom transformation of UDP incoming and outgoing UDP datagrams. Please refer to User Manual for the User Manual and how to use the UDP transforms.

6.12 RTPS Transport API

Classes

- union [RTPS_Checksum](#)
<<eXtension>> RTPS checksum type
- struct [RTPS_ChecksumClass](#)
<<eXtension>> Definition of a checksum function
- struct [RTPS_ChecksumProperty](#)
<<eXtension>> Checksum properties for the RTPS plugin
- struct [RTPS_InterfaceFactoryProperty](#)
<<eXtension>> Properties for the RTPS Transport.

Typedefs

- typedef [RTI_INT16 RTPS_ChecksumClassId_T](#)
<<eXtension>> A checksum class ID
- typedef union [RTPS_Checksum RTPS_Checksum_T](#)
<<eXtension>> RTPS checksum type
- typedef [RTI_BOOL\(* RTPS_ChecksumCalculate_T\)](#) (void *context, const struct REDA_Buffer *buf, [RTI_UINT32](#) buf_length, [RTPS_Checksum_T](#) *checksum)
<<eXtension>> The checksum calculation function
- typedef struct [RTPS_ChecksumClass RTPS_ChecksumClass_T](#)
<<eXtension>> Definition of a checksum function
- typedef struct [RTPS_InterfaceFactoryProperty RTPS_InterfaceFactoryProperty_T](#)
<<eXtension>> Properties for the RTPS Transport.

Enumerations

- enum [RTPS_ChecksumTxMode_T](#) { [RTPS_CHECKSUM_TXMODE_RTICRC32](#), [RTPS_CHECKSUM_TXMODE_OMG](#) }
<<eXtension>> Transmit mode for checksums

Variables

- `RTI_UINT32 RTPS_Checksum::checksum32`
A 32-bit unsigned checksum in host endianness order.
- `RTI_UINT64 RTPS_Checksum::checksum64`
A 64-bit unsigned integer checksum in host endianness order.
- `RTI_UINT8 RTPS_Checksum::checksum128 [16]`
A 128-bit checksum as a 16-byte array.
- `RTPS_ChecksumClassId_T RTPS_ChecksumClass::class_id`
The checksum class ID.
- `void * RTPS_ChecksumClass::context`
User defined context.
- `RTPS_ChecksumCalculate_T RTPS_ChecksumClass::checksum_calculate`
User defined function to calculate a checksum.
- `RTPS_ChecksumClass_T RTPS_ChecksumProperty::builtin_checksum32_class`
The builtin 32-bit checksum class.
- `RTPS_ChecksumClass_T RTPS_ChecksumProperty::builtin_checksum64_class`
The builtin 64-bit checksum class.
- `RTPS_ChecksumClass_T RTPS_ChecksumProperty::builtin_checksum128_class`
The builtin 128-bit checksum class.
- `RTPS_ChecksumTxMode_T RTPS_ChecksumProperty::checksum_tx_mode`
The transmit mode for checksums.
- `RTI_BOOL RTPS_ChecksumProperty::allow_builtin_override`
If TRUE, allow overriding the builtin checksum functions.
- `struct RTPS_ChecksumProperty RTPS_InterfaceFactoryProperty::checksum`
The checksum properties for RTPS.
- `const struct RTPS_InterfaceFactoryProperty RTPS_INTERFACE_FACTORY_DEFAULT`
<<eXtension>> Default properties for the RTPS Transport.

6.12.1 Detailed Description

<<eXtension>> The RTPS Plugin implements the RTI Connex DDS Micro RTPS protocol as an RTI Connex DDS Micro NETIO interface.

6.12.2 Typedef Documentation

RTPS_ChecksumClassId_T

```
typedef RTI_INT16 RTPS_ChecksumClassId_T
```

<<eXtension>> A checksum class ID

RTPS_Checksum_T

```
typedef union RTPS_Checksum RTPS_Checksum_T
```

<<eXtension>> RTPS checksum type

RTPS_ChecksumCalculate_T

```
typedef RTI_BOOL(* RTPS_ChecksumCalculate_T) (void *context, const struct REDA_Buffer *buf, RTI_UINT32
buf_length, RTPS_Checksum_T *checksum)
```

<<*eXtension*>> The checksum calculation function

Parameters

in	<i>context</i>	Context passed as property
in	<i>buf</i>	Vector of buffers to calculate the checksum over
in	<i>buf_length</i>	Number of elements in buf
out	<i>checksum</i>	Calculated checksum

Returns

RTI_TRUE on success, RTI_FALSE on failure

RTPS_ChecksumClass_T

```
typedef struct RTPS_ChecksumClass RTPS_ChecksumClass_T
```

<<*eXtension*>> Definition of a checksum function

RTPS_InterfaceFactoryProperty_T

```
typedef struct RTPS_InterfaceFactoryProperty RTPS_InterfaceFactoryProperty_T
```

<<*eXtension*>> Properties for the RTPS Transport.

6.12.3 Enumeration Type Documentation**RTPS_ChecksumTxMode_T**

```
enum RTPS_ChecksumTxMode_T
```

<<*eXtension*>> Transmit mode for checksums

Enumerator

RTPS_CHECKSUM_TXMODE_RTICRC32	Always use CRC32, fail if not possible.
RTPS_CHECKSUM_TXMODE_OMG	Always use OMG's header extension [Default].

6.12.4 Variable Documentation

checksum32

`RTI_UINT32` RTPS_Checksum::checksum32

A 32-bit unsigned checksum in host endianness order.

checksum64

`RTI_UINT64` RTPS_Checksum::checksum64

A 64-bit unsigned integer checksum in host endianness order.

checksum128

`RTI_UINT8` RTPS_Checksum::checksum128[16]

A 128-bit checksum as a 16-byte array.

class_id

`RTPS_ChecksumClassId_T` RTPS_ChecksumClass::class_id

The checksum class ID.

context

`void*` RTPS_ChecksumClass::context

User defined context.

checksum_calculate

`RTPS_ChecksumCalculate_T` RTPS_ChecksumClass::checksum_calculate

User defined function to calculate a checksum.

builtin_checksum32_class

`RTPS_ChecksumClass_T` RTPS_ChecksumProperty::builtin_checksum32_class

The builtin 32-bit checksum class.

builtin_checksum64_class

```
RTPS_ChecksumClass_T RTPS_ChecksumProperty::builtin_checksum64_class
```

The builtin 64-bit checksum class.

builtin_checksum128_class

```
RTPS_ChecksumClass_T RTPS_ChecksumProperty::builtin_checksum128_class
```

The builtin 128-bit checksum class.

checksum_tx_mode

```
RTPS_ChecksumTxMode_T RTPS_ChecksumProperty::checksum_tx_mode
```

The transmit mode for checksums.

allow_builtin_override

```
RTI_BOOL RTPS_ChecksumProperty::allow_builtin_override
```

If TRUE, allow overriding the builtin checksum functions.

checksum

```
struct RTPS_ChecksumProperty RTPS_InterfaceFactoryProperty::checksum
```

The checksum properties for RTPS.

RTPS_INTERFACE_FACTORY_DEFAULT

```
const struct RTPS_InterfaceFactoryProperty RTPS_INTERFACE_FACTORY_DEFAULT [extern]
```

<<*eXtension*>> Default properties for the RTPS Transport.

6.13 Shared Memory Transport API**Topics**

- [NETIO Shared Memory API](#)

OS independent way to setup and access shared-memory segments, shared-memory mutexes and shared memory binary semaphores.

Classes

- struct [NETIO_SHMEMInterfaceFactoryProperty](#)
<<eXtension>> Properties for the SHMEM Transport.
- class [SHMEMInterfaceFactory](#)
<<eXtension>> A SHMEM transport factory that will be used by a DomainParticipant to create a SHMEM interface. Only one SHMEM interface can be registered with the DomainParticipantFactory.

6.13.1 Detailed Description

<<eXtension>> The shared memory (SHMEM) transport implements the RTI Connex DDS Micro NETIO interface and uses a message queue in shared memory for communication between DomainParticipants. Please refer to the User's Manual for information about how to use the SHMEM transport.

6.13.2 NETIO Shared Memory API

OS independent way to setup and access shared-memory segments, shared-memory mutexes and shared memory binary semaphores.

Topics

- [Shared Memory Mutex API](#)
- [Shared Memory Segment API](#)
Allow access to a shared memory segment (memory block) so that multiple processes can read-and write a common memory area.
- [Shared Memory Signaling Semaphore](#)

6.13.2.1 Detailed Description

OS independent way to setup and access shared-memory segments, shared-memory mutexes and shared memory binary semaphores.

This module consist of three constructs: SharedMemorySegment, SharedMemoryBinarySemaphore, SharedMemoryMutex.

These objects, represent operating-system resources that are accessible from different processes running on the same computer. These three objects are manipulated in very similar ways and have also very similar lifecycles.

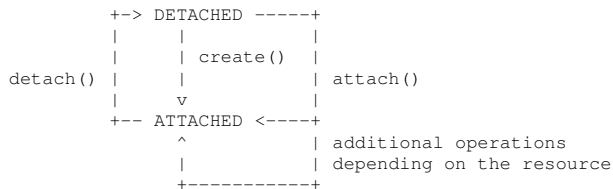
The model is as follows. The underlying shared memory resource (segment, semaphore, and mutex) is identified using an integer "key". Processes wishing to create or interact with the resource objects must do it via the appropriate handles. To be portable across operating systems the application should consider keys to be unique even across different kinds of shared memory resources, that is if a key is specified for a shared-memory segment it cannot be re-used to specify a shared-memory semaphore or mutex. Different keys must be used for different resources even if they are of different kinds.

The thing to keep in mind when looking at the lifecycles is that there are potentially many shared-memory handles referencing the same underlying shared-memory resource. That is in fact the whole point of using shared memory resource.

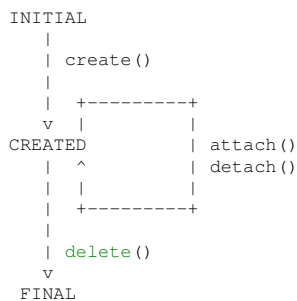
A shared-memory object can be in one of the following states: CREATED, OPEN, CLOSED.

A shared-memory handle can be in one of the following states: DETACHED, ATTACHED.

The state transitions of the shared-memory object are surrogate to those of corresponding handles because processes can only manipulate shared memory using the handles. The following charts describe this:



State of the shared-memory resource:



No control is performed on the permission of the delete() function. Once a resource is deleted, it is permanently destroyed and any subsequent call to attach() should fail.

The shared-memory objects used in this module represent named operating-system resources. Many OSs provide utilities to view and administer these operating system resources from an independent API. For example, in Unix/Linux the command-line utility "ipcs" can be used to see a listing of the current shared memory segments, semaphores and mutexes. Similarly "ipcrm" can be used to remove shared-memory resources.

6.13.2.2 Shared Memory Mutex API

Functions

- RTIBool [NETIO_SharedMemoryMutex_create](#) (struct NETIO_SharedMemoryMutexHandle *handle, RTI_INT32 *status_out, RTI_INT32 key)
Create a shared memory mutex.
- RTIBool [NETIO_SharedMemoryMutex_attach](#) (struct NETIO_SharedMemoryMutexHandle *handle, RTI_INT32 *status_out, RTI_INT32 key)
Attach to an existing shared memory binary semaphore with the provided key and sets up the handle to access it.
- RTIBool [NETIO_SharedMemoryMutex_create_or_attach](#) (struct NETIO_SharedMemoryMutexHandle *handle, RTI_INT32 *status_out, RTI_INT32 key)
Attempts to create a new binary semaphore with the provided key and if the semaphore already exist, attach to it.
- RTIBool [NETIO_SharedMemoryMutex_unlock](#) (struct NETIO_SharedMemoryMutexHandle *handle, RTI_INT32 *status_out)

Give the semaphore.

- RTIBool [NETIO_SharedMemoryMutex_lock](#) (struct NETIO_SharedMemoryMutexHandle *handle, [RTI_INT32](#) *status_out)

Take the semaphore.

- RTIBool [NETIO_SharedMemoryMutex_detach](#) (struct NETIO_SharedMemoryMutexHandle *handle)

Detach from the shared memory binary semaphore.

- RTIBool [NETIO_SharedMemoryMutex_delete](#) (struct NETIO_SharedMemoryMutexHandle *handle)

Detach from the shared memory semaphore and deletes it.

6.13.2.2.1 Detailed Description

6.13.2.2.2 Function Documentation

NETIO_SharedMemoryMutex_create()

```
RTIBool NETIO_SharedMemoryMutex_create (
    struct NETIO_SharedMemoryMutexHandle * handle,
    RTI_INT32 * status_out,
    RTI_INT32 key)
```

Create a shared memory mutex.

Parameters

<i>handle</i>	out Pointer to handle on successful return
<i>status_out</i>	out. Optional: if not NULL is set to the reason why the call has succeeded or failed. In particular: • OSAPI_SHARED_MEMORY_FAIL_REASON_PRECONDITION = precondition check failed. • OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN = an error occurred in the os-specific shared memory implementation (see error log for details). • OSAPI_SHARED_MEMORY_FAIL_REASON_NO_ENTRY= this error should never happen and is caused by a failed create() followed by a failed attach(). • OSAPI_SHARED_MEMORY_CREATED = the semaphore has been successfully created.
<i>key</i>	in. Key to identify the shared memory semaphore across processes.

Returns

RTI_TRUE on success. RTI_FALSE on error.

References [RTI_INT32](#).

NETIO_SharedMemoryMutex_attach()

```
RTIBool NETIO_SharedMemoryMutex_attach (
    struct NETIO_SharedMemoryMutexHandle * handle,
    RTI_INT32 * status_out,
    RTI_INT32 key)
```

Attach to an existing shared memory binary semaphore with the provided key and sets up the handle to access it.

Precondition

Handle in the DETACHED state. Signaling semaphore in the CREATED state. Some other process has called [NETIO_SharedMemoryMutex_create](#) using the same key in the shared memory semaphore

Postcondition

handle in the ATTACHED state, Semaphore exists and is ready to be used.

Parameters

<i>handle</i>	out. The shared memory semaphore handle to initialize.
<i>status_out</i>	out. Optional. If not NULL, will be set to: • OSAPI_SHARED_MEMORY_FAIL_REASON_PRECONDITION = precondition check failed. • OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN = an error occurred in the os-specific shared memory implementation (see error log for details). • OSAPI_SHARED_MEMORY_FAIL_REASON_NO_ENTRY = no such a semaphore. • OSAPI_SHARED_MEMORY_ATTACHED = semaphore attached successfully.
<i>key</i>	in. Key to identify the shared memory semaphore across processes.

Returns

RTI_TRUE on success. RTI_FALSE on error.

Note

On some platforms this method makes sure that the given key represent a shared memory semaphore object (to avoid situations like the same key is used to create a mutex and then used to attach it as a semaphore). If such a situation is detected, the function will fail and set status_out to: OSAPI_SHARED_MEMORYFAIL_REASON_↵ NO_ENTRY. Not all the platforms support this feature.

See also

[NETIO_SharedMemoryMutex_detach](#)

References [RTI_INT32](#).

NETIO_SharedMemoryMutex_create_or_attach()

```
RTIBool NETIO_SharedMemoryMutex_create_or_attach (
    struct NETIO_SharedMemoryMutexHandle * handle,
    RTI_INT32 * status_out,
    RTI_INT32 key)
```

Attempts to create a new binary semaphore with the provided key and if the semaphore already exist, attach to it.

Precondition

Handle in the DETACHED state. Signaling semaphore in the INITIAL, that is, Nobody has called create() before with that same key.

Postcondition

handle in the ATTACHED state, Semaphore is in the CREATED state and ready to be used.

Parameters

<i>handle</i>	inOut. The shared memory semaphore handle to initialize.
<i>status_out</i>	out. Optional: if not NULL is set to the reason why the call has succeeded or failed. In particular: • OSAPI_SHARED_MEMORY_FAIL_REASON_PRECONDITION = precondition check failed. • OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN = an error occurred in the os-specific shared memory implementation (see error log for details). • OSAPI_SHARED_MEMORY_FAIL_REASON_NO_ENTRY= this error should never happen and is caused by a failed create() followed by a failed attach(). • OSAPI_SHARED_MEMORY_CREATED = the semaphore has been successfully created.
<i>key</i>	in. Key to identify the shared memory binary semaphore across processes.

Returns

RTI_TRUE on success. RTI_FALSE on error.

References [RTI_INT32](#).

NETIO_SharedMemoryMutex_unlock()

```
RTIBool NETIO_SharedMemoryMutex_unlock (
    struct NETIO_SharedMemoryMutexHandle * handle,
    RTI_INT32 * status_out)
```

Give the semaphore.

Precondition

Handle in the ATTACHED state.

Postcondition

Semaphore count is set to 1 and any thread blocked on this semaphore is awoken.

Parameters

<i>handle</i>	in. Handle to the semaphore. Must match the one returned by NETIO_SharedMemoryMutex_create
<i>status_out</i>	out. Optional: if not NULL, will be set to: • OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN = an error occurred in the os-specific shared memory implementation (see error log for details). • OSAPI_SHARED_MEMORY_FAIL_REASON_NO_ENTRY = the creator of the mutex has shutdown and removed this mutex. This is expected to happen routinely. • OSAPI_SHARED_MEMORY_SUCCESS = Generic no error condition.

Returns

RTI_TRUE if the semaphore is given. RTI_FALSE if an error occurs. If an error occurs the handle should not be used anymore. In particular detach() should not be called either. An error may occur in certain OSs if the handle that created the semaphore calls [NETIO_SharedMemoryMutex_detach](#).

See also

[NETIO_SharedMemorySignalingSemaphore_wait](#)

References [RTI_INT32](#).

NETIO_SharedMemoryMutex_lock()

```
RTIBool NETIO_SharedMemoryMutex_lock (
    struct NETIO_SharedMemoryMutexHandle * handle,
    RTI_INT32 * status_out)
```

Take the semaphore.

Precondition

Handle in the ATTACHED state.

Postcondition

Semaphore count is decremented.

Parameters

<i>handle</i>	in. Handle to the semaphore. Must match the one returned by NETIO_SharedMemoryMutex_create
<i>status_out</i>	out. Optional: if not NULL, will be set to: • OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN = an error occurred in the os-specific shared memory implementation (see error log for details). • OSAPI_SHARED_MEMORY_FAIL_REASON_NO_ENTRY = the creator of the mutex has shutdown and removed this mutex. This is expected to happen routinely. • OSAPI_SHARED_MEMORY_SUCCESS = Generic no error condition.

Returns

RTI_TRUE if the semaphore is taken. RTI_FALSE if an error occurs. If an error occurs the handle should not be used anymore. In particular [NETIO_SharedMemoryMutex_detach](#) should not be called either.

See also

[NETIO_SharedMemorySignalingSemaphore_signal](#)

References [RTI_INT32](#).

NETIO_SharedMemoryMutex_detach()

```
RTIBool NETIO_SharedMemoryMutex_detach (  
    struct NETIO_SharedMemoryMutexHandle * handle)
```

Detach from the shared memory binary semaphore.

Precondition

Handle in the ATTACHED state.

Postcondition

handle in the DETACHED state.

Parameters

<i>handle</i>	in handle. Handle to the shared memory semaphore obtained from NETIO_SharedMemoryMutex_attach
---------------	---

See also

[NETIO_SharedMemoryMutex_attach](#)

NETIO_SharedMemoryMutex_delete()

```
RTIBool NETIO_SharedMemoryMutex_delete (  
    struct NETIO_SharedMemoryMutexHandle * handle)
```

Detach from the shared memory semaphore and deletes it.

The exact behavior of this may be OS dependent. In some OSs calling this will immediately cause the semaphore to be removed, and other threads waiting on it to be woken up. This is the typical behavior on Unix. In other OSs (windows) this call just requests the semaphore to be removed but delay the removal until all handles are detached.

The only way to ensure the same behavior across multiple OSs is to only call this method on the handle that created the semaphore after all the other handles have been closed. In this scenario no handles will get an error. Alternatively the code could be written to always expect the [NETIO_SharedMemorySignalingSemaphore_wait](#) or [NETIO_SharedMemorySignalingSemaphore_signal](#) call to fail and when that occurs close the handle.

Precondition

Handle in the ATTACHED state.

Postcondition

handle in the DETACHED state. The numHandles to the semaphore is decremented. When the last handle is detached, the semaphore is removed by the OS.

Parameters

<i>handle</i>	in handle. Handle to the shared memory semaphore obtained from NETIO_SharedMemoryMutex_create
---------------	---

See also

[NETIO_SharedMemoryMutex_create](#)

References [RTI_INT32](#).

6.13.2.3 Shared Memory Segment API

Allow access to a shared memory segment (memory block) so that multiple processes can read-and write a common memory area.

Classes

- struct [NETIO_SharedMemorySegmentHeader](#)

Functions

- RTIBool [NETIO_SharedMemorySegment_create_or_attach](#) (struct NETIO_SharedMemorySegmentHandle *handle, [RTI_INT32](#) *status_out, [RTI_INT32](#) key, [RTI_UINT32](#) size, OSAPI_ProcessId pid_in)
Attempts to create a new shared memory segment. If the segment already exists and the creator process is dead, attach and recycle it.
- RTIBool [NETIO_SharedMemorySegment_create](#) (struct NETIO_SharedMemorySegmentHandle *handle, [RTI_INT32](#) *status_out, [RTI_INT32](#) key, [RTI_UINT32](#) size, OSAPI_ProcessId pid_in)
Creates a new shared memory segment with the provided key and returns the handle to access it.
- RTIBool [NETIO_SharedMemorySegment_delete](#) (struct NETIO_SharedMemorySegmentHandle *handle)
Deletes the shared memory segment previously create with [NETIO_SharedMemorySegment_create](#). If the segment is still attached, it is detached first.
- RTIBool [NETIO_SharedMemorySegment_attach](#) (struct NETIO_SharedMemorySegmentHandle *handle, [RTI_INT32](#) *status_out, [RTI_INT32](#) key)
Attach to the shared memory previously initialized with [NETIO_SharedMemorySegment_create_or_attach](#) and returns a handle to it.
- RTIBool [NETIO_SharedMemorySegment_detach](#) (struct NETIO_SharedMemorySegmentHandle *handle)
Detach from the shared memory previously created with [NETIO_SharedMemorySegment_attach](#).
- char * [NETIO_SharedMemorySegment_get_address](#) (struct NETIO_SharedMemorySegmentHandle *handle)
Get the address of the shared memory attached by the handle.
- NETIOPSLDIExport [RTI_INT32 NETIO_SharedMemorySegment_get_size](#) (struct NETIO_SharedMemorySegmentHandle *handle)
Get the size (available for user data) of the shared memory segment.
- NETIOPSLDIExport [RTI_UINT32 NETIO_SharedMemorySegment_get_max_size](#) (void)
Maximum allowable size of a shared memory segment.

6.13.2.3.1 Detailed Description

Allow access to a shared memory segment (memory block) so that multiple processes can read-and write a common memory area.

6.13.2.3.2 Function Documentation

NETIO_SharedMemorySegment_create_or_attach()

```
RTIBool NETIO_SharedMemorySegment_create_or_attach (
    struct NETIO_SharedMemorySegmentHandle * handle,
    RTI_INT32 * status_out,
    RTI_INT32 key,
    RTI_UINT32 size,
    OSAPI_ProcessId pid_in)
```

Attempts to create a new shared memory segment. If the segment already exists and the creator process is dead, attach and recycle it.

Parameters

in, out	<i>handle</i>	The shared memory segment handle to initialize.
out	<i>status_out</i>	Optional: if not NULL is set to the reason why the call has succeeded or failed. In particular, • OSAPI_SHARED_MEMORY_FAIL_REASON_PRECONDITION = precondition check failed. • OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN = an error occurred in the os-specific shared memory implementation (see error log for details). • OSAPI_SHARED_MEMORY_FAIL_REASON_ALREADY_CLAIMED = someone is already sitting on this address. • OSAPI_SHARED_MEMORY_CREATED = the segment has been created new. • OSAPI_SHARED_MEMORY_ATTACHED = the segment was reused from an existing one.
in	<i>key</i>	Key to identify the shared memory segment Across processes. Other processes should call NETIO_SharedMemorySegment_attach with the same key to attach to this segment.
in	<i>size</i>	Size of memory to be initialized.
in	<i>pid_in</i>	PID of the process that calls this method.

Returns

RTI_TRUE if success, or RTI_FALSE if something failed.

References [RTI_INT32](#), and [RTI_UINT32](#).

NETIO_SharedMemorySegment_create()

```
RTIBool NETIO_SharedMemorySegment_create (
    struct NETIO_SharedMemorySegmentHandle * handle,
    RTI_INT32 * status_out,
    RTI_INT32 key,
    RTI_UINT32 size,
    OSAPI_ProcessId pid_in)
```

Creates a new shared memory segment with the provided key and returns the handle to access it.

Parameters

in, out	<i>handle</i>	The shared memory segment handle to initialize.
out	<i>status_out</i>	Optional: if not NULL is set to the reason why the call has succeeded or failed. In particular, • OSAPI_SHARED_MEMORY_FAIL_REASON_PRECONDITION = precondition check failed. • OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN = an error occurred in the os-specific shared memory implementation (see error log for details). • OSAPI_SHARED_MEMORY_FAIL_REASON_ALREADY_CLAIMED = segment already exist (no checks is performed whether the creator is still alive or not). • OSAPI_SHARED_MEMORY_CREATED = the segment has been successfully created.
in	<i>key</i>	Key to identify the shared memory segment across processes. Other processes should call NETIO_SharedMemorySegment_attach with the same key to attach to this segment.
in	<i>size</i>	Size of memory to be initialized.
in	<i>pid_in</i>	PID of the process that calls this method.

Returns

RTI_TRUE if success, or RTI_FALSE if something failed.

Note

Depending on the underlying OS, the key may need to be transformed to the actual representation needed by the operating system

References [RTI_INT32](#), and [RTI_UINT32](#).

NETIO_SharedMemorySegment_delete()

```
RTIBool NETIO_SharedMemorySegment_delete (
    struct NETIO_SharedMemorySegmentHandle * handle)
```

deletes the shared memory segment previously create with [NETIO_SharedMemorySegment_create](#). If the segment is still attached, it is detached first.

Parameters

in	<i>handle</i>	The shared memory segment handle identifying the segment to detach
----	---------------	--

Returns

RTI_TRUE on success. RTI_FALSE if either the precondition was not met or the shared memory segment couldn't be removed (i.e. the user running this app is not the owner or creator).

Note

This function does not perform any control on the ownership of the segment. If you attach to a segment, then when you call this function the segment will be destroyed (if you have enough permission to complete the operation)

When you create a segment with [NETIO_SharedMemorySegment_create](#) or with [NETIO_SharedMemorySegment_create_or_attach](#) you can delete it by calling directly [NETIO_SharedMemorySegment_delete](#) without calling [NETIO_SharedMemorySegment_detach](#).

Once called this function (even if it returns RTI_FALSE) you cannot use anymore this shared memory segment.

NETIO_SharedMemorySegment_attach()

```
RTIBool NETIO_SharedMemorySegment_attach (
    struct NETIO_SharedMemorySegmentHandle * handle,
    RTI_INT32 * status_out,
    RTI_INT32 key)
```

Attach to the shared memory previously initialized with [NETIO_SharedMemorySegment_create_or_attach](#) and returns a handle to it.

Parameters

in, out	<i>handle</i>	The shared memory segment handle to initialize.
out	<i>status_out</i>	Optional. If not NULL, will be set to: • OSAPI_SHARED_MEMORY_FAIL_REASON_PRECONDITION = precondition check failed. • OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN = an error occurred in the os-specific shared memory implementation (see error log for details). • OSAPI_SHARED_MEMORY_FAIL_REASON_NO_ENTRY = no such a segment. • OSAPI_SHARED_MEMORY_FAIL_REASON_OUT_OF_MEMORY = out of memory allocating the handle structure. • OSAPI_SHARED_MEMORY_ATTACHED = segment attached successfully.
in	<i>key</i>	Key to identify the shared memory segment across processes. Must match that used by other processes calling NETIO_SharedMemorySegment_create_or_attach or NETIO_SharedMemorySegment_attach() .

Returns

RTI_TRUE on success. RTI_FALSE if an error occurred.

References [RTI_INT32](#).

NETIO_SharedMemorySegment_detach()

```
RTIBool NETIO_SharedMemorySegment_detach (  
    struct NETIO_SharedMemorySegmentHandle * handle)
```

Detach from the shared memory previously created with [NETIO_SharedMemorySegment_attach](#).

Parameters

in	<i>handle</i>	The shared memory segment handle identifying the segment to detach
----	---------------	--

Returns

RTI_TRUE on success. RTI_FALSE if the precondition was not met.

The shared memory segment is always detached. Once the precondition is met, no errors will be generated.

Once called this function (even if it returns RTI_FALSE) you cannot use anymore this shared memory segment, except for destroying it.

NETIO_SharedMemorySegment_get_address()

```
char * NETIO_SharedMemorySegment_get_address (  
    struct NETIO_SharedMemorySegmentHandle * handle)
```

Get the address of the shared memory attached by the handle.

Precondition

Handle in the ATTACHED state.

Postcondition

none

Parameters

in	<i>handle</i>	Handle to the shared memory segment obtained from an NETIO_SharedMemorySegment_attach or a NETIO_SharedMemorySegment_create_or_attach
----	---------------	---

Returns

the address that can be used to read and write the memory. For performance this function never checks pre-conditions. It is the caller's responsibility to ensure the handle is NON null and ATTACHED. A NULL handle will cause a seg-fault. An un-attached handle may give invalid values for the [NETIO_SharedMemorySegment_get_size](#)

References [RTI_INT32](#).

NETIO_SharedMemorySegment_get_size()

```
NETIOPSLDllExport RTI_INT32 NETIO_SharedMemorySegment_get_size (  
    struct NETIO_SharedMemorySegmentHandle * handle)
```

Get the size (available for user data) of the shared memory segment.

This size will match the one passed to the [NETIO_SharedMemorySegment_create_or_attach](#) call

Precondition

Handle in the ATTACHED state.

Postcondition

none

Parameters

in	<i>handle</i>	Handle to the shared memory segment obtained from an NETIO_SharedMemorySegment_attach or a NETIO_SharedMemorySegment_create_or_attach call.
----	---------------	---

Returns

the number of bytes that can be read/written starting at the address provided by [NETIO_SharedMemorySegment_get_address](#). For performance this function never checks pre-conditions. It is the caller's responsibility to ensure the handle is NON null and ATTACHED. A NULL handle will cause a seg-fault. An un-attached handle may give invalid values for the size.

References [RTI_UINT32](#).

NETIO_SharedMemorySegment_get_max_size()

```
NETIOPSLDllExport RTI_UINT32 NETIO_SharedMemorySegment_get_max_size (  
    void )
```

Maximum allowable size of a shared memory segment.

References [RTI_INT32](#).

6.13.2.4 Shared Memory Signaling Semaphore

Functions

- RTIBool [NETIO_SharedMemorySignalingSemaphore_create](#) (struct NETIO_SharedMemorySignalingSemaphoreHandle *handle, RTI_INT32 *status_out, RTI_INT32 key)
Initializes a shared memory binary semaphore with the provided key and sets up the handle to access it.
- RTIBool [NETIO_SharedMemorySignalingSemaphore_attach](#) (struct NETIO_SharedMemorySignalingSemaphoreHandle *handle, RTI_INT32 *status_out, RTI_INT32 key)
Attach to an existing shared memory binary semaphore with the provided key and sets up the handle to access it.
- RTIBool [NETIO_SharedMemorySignalingSemaphore_create_or_attach](#) (struct NETIO_SharedMemorySignalingSemaphoreHandle *handle, RTI_INT32 *status_out, RTI_INT32 key)
Attempts to create a new binary semaphore with the provided key and if the semaphore already exist, attach to it.
- RTIBool [NETIO_SharedMemorySignalingSemaphore_signal](#) (struct NETIO_SharedMemorySignalingSemaphoreHandle *handle, RTI_INT32 *status_out)
Give the semaphore.
- RTIBool [NETIO_SharedMemorySignalingSemaphore_wait](#) (struct NETIO_SharedMemorySignalingSemaphoreHandle *handle, RTI_INT32 *status_out)
Take the semaphore.
- RTIBool [NETIO_SharedMemorySignalingSemaphore_detach](#) (struct NETIO_SharedMemorySignalingSemaphoreHandle *handle)
Detach from the shared memory binary semaphore.
- RTIBool [NETIO_SharedMemorySignalingSemaphore_delete](#) (struct NETIO_SharedMemorySignalingSemaphoreHandle *handle)
Detach from the shared memory semaphore and deletes it.

6.13.2.4.1 Detailed Description

6.13.2.4.2 Function Documentation

NETIO_SharedMemorySignalingSemaphore_create()

```
RTIBool NETIO_SharedMemorySignalingSemaphore_create (
    struct NETIO_SharedMemorySignalingSemaphoreHandle * handle,
    RTI_INT32 * status_out,
    RTI_INT32 key)
```

Initializes a shared memory binary semaphore with the provided key and sets up the handle to access it.

Precondition

Handle in the DETACHED state. Signaling semaphore in the INITIAL, that is, Nobody has called create() before with that same key

Postcondition

handle in the ATTACHED state, Semaphore exists and ready to be used.

Parameters

in, out	<i>handle</i>	The shared memory semaphore handle to initialize.
out	<i>status_out</i>	Optional: if not NULL is set to the reason why the call has succeeded or failed. In particular: • OSAPI_SHARED_MEMORY_FAIL_REASON_PRECONDITION = precondition check failed. • OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN = an error occurred in the os-specific shared memory implementation (see error log for details). • OSAPI_SHARED_MEMORY_FAIL_REASON_ALREADY_CLAIMED = semaphore already exist (no checks is performed whether the creator is still alive or not). • OSAPI_SHARED_MEMORY_CREATED = the semaphore has been successfully created.
in	<i>key</i>	Key to identify the shared memory semaphore across processes. Other processes should call NETIO_SharedMemorySignalingSemaphore_attach with the same key to attach to this semaphore.

Returns

RTI_TRUE on success. RTI_FALSE on error.

References [RTI_INT32](#).

NETIO_SharedMemorySignalingSemaphore_attach()

```
RTIBool NETIO_SharedMemorySignalingSemaphore_attach (
    struct NETIO_SharedMemorySignalingSemaphoreHandle * handle,
    RTI_INT32 * status_out,
    RTI_INT32 key)
```

Attach to an existing shared memory binary semaphore with the provided key and sets up the handle to access it.

Precondition

Handle in the DETACHED state. Signaling semaphore in the CREATED state. Some other process has called [NETIO_SharedMemorySignalingSemaphore_create](#) using the same key in the shared memory semaphore

Postcondition

handle in the ATTACHED state, Semaphore exists and is ready to be used.

Parameters

out	<i>handle</i>	The shared memory semaphore handle to initialize.
-----	---------------	---

Parameters

out	<i>status_out</i>	Optional. If not NULL, will be set to: • OSAPI_SHARED_MEMORY_FAIL_REASON_PRECONDITION = precondition check failed. • OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN = an error occurred in the os-specific shared memory implementation (see error log for details). • OSAPI_SHARED_MEMORY_FAIL_REASON_NO_ENTRY = no such a semaphore. • OSAPI_SHARED_MEMORY_ATTACHED = semaphore attached successfully.
in	<i>key</i>	Key to identify the shared memory semaphore across processes.

Returns

RTI_TRUE on success. RTI_FALSE on error.

Note

On some platforms this method makes sure that the given key represent a shared memory semaphore object (to avoid situations like the same key is used to create a mutex and then used to attach it as a semaphore). If such a situation is detected, the function will fail and set status_out to: OSAPI_SHARED_MEMORYFAIL_REASON_↵ NO_ENTRY. Not all the platforms support this feature.

See also

[NETIO_SharedMemorySignalingSemaphore_detach](#)

References [RTI_INT32](#).

NETIO_SharedMemorySignalingSemaphore_create_or_attach()

```
RTIBool NETIO_SharedMemorySignalingSemaphore_create_or_attach (
    struct NETIO_SharedMemorySignalingSemaphoreHandle * handle,
    RTI_INT32 * status_out,
    RTI_INT32 key)
```

Attempts to create a new binary semaphore with the provided key and if the semaphore already exist, attach to it.

Precondition

Handle in the DETACHED state. Signaling semaphore in the INITIAL, that is, Nobody has called create() before with that same key.

Postcondition

handle in the ATTACHED state, Semaphore is in the CREATED state and ready to be used.

Parameters

in, out	<i>handle</i>	The shared memory semaphore handle to initialize.
out	<i>status_out</i>	Optional: if not NULL is set to the reason why the call has succeeded or failed. In particular: • OSAPI_SHARED_MEMORY_FAIL_REASON_PRECONDITION = precondition check failed. • OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN = an error occurred in the os-specific shared memory implementation (see error log for details). • OSAPI_SHARED_MEMORY_FAIL_REASON_NO_ENTRY = this error should never happen and is caused by a failed create() followed by a failed attach(). • OSAPI_SHARED_MEMORY_CREATED = the semaphore has been successfully created.
in	<i>key</i>	Key to identify the shared memory binary semaphore across processes.

Returns

RTI_TRUE on success. RTI_FALSE on error.

References [RTI_INT32](#).

NETIO_SharedMemorySignalingSemaphore_signal()

```
RTIBool NETIO_SharedMemorySignalingSemaphore_signal (
    struct NETIO_SharedMemorySignalingSemaphoreHandle * handle,
    RTI_INT32 * status_out)
```

Give the semaphore.

Precondition

Handle in the ATTACHED state.

Postcondition

Semaphore count is set to 1 and any thread blocked on this semaphore is awoken.

Parameters

in	<i>handle</i>	Handle to the semaphore. Must match the one returned by NETIO_SharedMemorySignalingSemaphore_create
out	<i>status_out</i>	Optional: if not NULL, will be set to: • OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN = an error occurred in the os-specific shared memory implementation (see error log for details). • OSAPI_SHARED_MEMORY_FAIL_REASON_NO_ENTRY = the creator of the mutex has shutdown and removed this mutex. This is expected to happen routinely. • OSAPI_SHARED_MEMORY_SUCCESS = Generic no error condition.

Returns

RTI_TRUE if the semaphore is given. RTI_FALSE if an error occurs. If an error occurs the handle should not be used anymore. In particular detach() should not be called either. An error may occur in certain OSs if the handle that created the semaphore calls [NETIO_SharedMemorySignalingSemaphore_detach](#).

See also

[NETIO_SharedMemorySignalingSemaphore_wait](#)

References [RTI_INT32](#).

NETIO_SharedMemorySignalingSemaphore_wait()

```
RTIBool NETIO_SharedMemorySignalingSemaphore_wait (
    struct NETIO_SharedMemorySignalingSemaphoreHandle * handle,
    RTI_INT32 * status_out)
```

Take the semaphore.

Precondition

Handle in the ATTACHED state.

Postcondition

Semaphore count is decremented.

Parameters

in	<i>handle</i>	Handle to the semaphore. Must match the one returned by NETIO_SharedMemorySignalingSemaphore_create
out	<i>status_out</i>	Optional: if not NULL, will be set to: • OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN = an error occurred in the os-specific shared memory implementation (see error log for details). • OSAPI_SHARED_MEMORY_FAIL_REASON_NO_ENTRY = the creator of the mutex has shutdown and removed this mutex. This is expected to happen routinely. • OSAPI_SHARED_MEMORY_SUCCESS = Generic no error condition.

Returns

RTI_TRUE if the semaphore is taken. RTI_FALSE if an error occurs. If an error occurs the handle should not be used anymore. In particular [NETIO_SharedMemorySignalingSemaphore_detach](#) should not be called either.

See also

[NETIO_SharedMemorySignalingSemaphore_signal](#)

References [RTI_INT32](#).

NETIO_SharedMemorySignalingSemaphore_detach()

```
RTIBool NETIO_SharedMemorySignalingSemaphore_detach (  
    struct NETIO_SharedMemorySignalingSemaphoreHandle * handle)
```

Detach from the shared memory binary semaphore.

Precondition

Handle in the ATTACHED state.

Postcondition

handle in the DETACHED state.

Parameters

in	<i>handle</i>	Handle to the shared memory semaphore obtained from NETIO_SharedMemorySignalingSemaphore_attach
----	---------------	---

See also

[NETIO_SharedMemorySignalingSemaphore_attach](#)

NETIO_SharedMemorySignalingSemaphore_delete()

```
RTIBool NETIO_SharedMemorySignalingSemaphore_delete (  
    struct NETIO_SharedMemorySignalingSemaphoreHandle * handle)
```

Detach from the shared memory semaphore and deletes it.

The exact behavior of this may be OS dependent. In some OSs calling this will immediately cause the semaphore to be removed, and other threads waiting on it to be woken up. This is the typical behavior on Unix. In other OSs (windows) this call just requests the semaphore to be removed but delay the removal until all handles are detached.

The only way to ensure the same behavior across multiple OSs is to only call this method on the handle that created the semaphore after all the other handles have been closed. In this scenario no handles will get an error. Alternatively the code could be written to always expect the [NETIO_SharedMemorySignalingSemaphore_wait](#) or [NETIO_SharedMemorySignalingSemaphore_signal](#) call to fail and when that occurs close the handle.

Precondition

Handle in the ATTACHED state.

Postcondition

handle in the DETACHED state. The numHandles to the semaphore is decremented. When the last handle is detached, the semaphore is removed by the OS.

Parameters

in	handle	Handle to the shared memory semaphore obtained from NETIO_SharedMemorySignalingSemaphore_create
----	--------	---

See also

[NETIO_SharedMemorySignalingSemaphore_create](#)

6.14 NETIO DGRAM API

NETIO DGRAM Interface.

Classes

- struct [NETIO_DGRAM_InterfaceMultiCastGroup](#)
<<eXtension>> Definition of a valid multicast group address range.
- struct [NETIO_DGRAM_InterfaceTableEntry](#)
<<eXtension>> Definition of an interface.
- struct [NETIO_DGRAM_InterfaceRouteEntry](#)
<<eXtension>> Definition of a route entry.
- struct [NETIO_DGRAM_InterfaceTableEntrySeq](#)
A sequence of NETIO_DGRAMInterfaceTableEntry elements.
- struct [NETIO_DGRAM_Interfaceel](#)
<<eXtension>> Definition of the NETIO_DGRAM Interface.

Macros

- #define [NETIO_DGRAM_INTERFACE_SHARED_PORT_FLAG](#) (0x1U)
Flag that indicates that an interface uses shared ports.
- #define [NETIO_DGRAM_INTERFACE_MAX_INTERFACES](#) (8)
The maximum number of network interfaces a NETIO_DGRAM Transport instance can manage.
- #define [NETIO_DGRAM_INTERFACE_MAX_ADDRESSES](#) (8)
The maximum number of addresses which can be returned in an address reservation request.
- #define [NETIO_DGRAM_InterfaceMultiCastGroup_INITIALIZER](#)
Constant to initialize a NETIO_DGRAM_InterfaceMultiCastGroup attribute as empty.
- #define [NETIO_DGRAM_InterfaceMultiCastGroup_Invalid](#)
Constant to initialize a NETIO_DGRAM_InterfaceMultiCastGroup attribute as invalid.
- #define [NETIO_DGRAM_InterfaceMultiCastGroup_UDPv4](#)
Constant to initialize a NETIO_DGRAM_InterfaceMultiCastGroup attribute as a UDPv4.
- #define [NETIO_DGRAM_InterfaceMultiCastGroup_UDPv6](#)
Constant to initialize a NETIO_DGRAM_InterfaceMultiCastGroup attribute as a UDPv6 multicast group.
- #define [NETIO_DGRAM_InterfaceTableEntry_INITIALIZER](#)
Constant to initialize an NETIO_DGRAM_InterfaceTableEntry attribute as an empty entry.
- #define [NETIO_DGRAM_InterfaceTableEntrySeq_INITIALIZER](#) REDA_DEFINE_SEQUENCE_INITIALIZER(struct [NETIO_DGRAM_InterfaceTableEntry](#))
Constant to initialize a NETIO_DGRAM_InterfaceTableEntry as empty.

Typedefs

- typedef [NETIO_Interface_T](#) **(*[* NETIO_DGRAM_Interface_createFunc](#)*)* ([NETIO_Interface_T](#) *upstream, void *property)
The NETIO_DGRAM create interface function required by [NETIO_DGRAM_InterfaceI](#).
- typedef [RTI_BOOL](#)*(*[* NETIO_DGRAMUserInterface_get_interface_listFunc](#)*)* ([NETIO_Interface_T](#) *user_intf, struct [NETIO_DGRAM_InterfaceTableEntrySeq](#) *if_table)
NETIO_DGRAM get interface list function required by [NETIO_DGRAM_InterfaceI](#).
- typedef [RTI_BOOL](#)*(*[* NETIO_DGRAMUserInterface_resolve_addressFunc](#)*)* ([NETIO_Interface_T](#) *netio_intf, const struct [NETIO_DGRAM_InterfaceTableEntry](#) *if_entry, const char *address_string, struct [NETIO_Address](#) *address_value, [RTI_BOOL](#) *is_invalid)
NETIO_DGRAM address resolve function.
- typedef [RTI_BOOL](#)*(*[* NETIO_DGRAMUserInterface_bind_addressFunc](#)*)* ([NETIO_Interface_T](#) *user_intf, struct [NETIO_Address](#) *source)
NETIO_DGRAM bind address function. by [NETIO_DGRAM_InterfaceI](#).
- typedef void*(*[* NETIO_DGRAMUserInterface_deleteFunc](#)*)* ([NETIO_Interface_T](#) *user_intf)
The NETIO_DGRAM delete interface function. NOT in CERT.
- typedef struct [NETIO_DGRAM_InterfaceI](#) [NETIO_DGRAM_InterfaceI](#)
<<eXtension>> Definition of the NETIO_DGRAM Interface.

Functions

- [RTI_BOOL NETIO_DGRAM_InterfaceFactory_register](#) ([RT_Registry_T](#) *registry, const char *name, [NETIO_DGRAM_InterfaceI](#) *user_intf, void *user_property)
Register the DGRAM interface.

Variables

- const struct [NETIO_DGRAM_InterfaceRouteEntry](#) [NETIO_ADDRESS_UDPv4_ANY_UNICAST_ROUTE](#)
Constant to initialize a UDPv4 unicast route.
- const struct [NETIO_DGRAM_InterfaceRouteEntry](#) [NETIO_ADDRESS_UDPv4_ANY_MULTICAST_ROUTE](#)
Constant to initialize UDPv4 multicast route.
- const struct [NETIO_DGRAM_InterfaceRouteEntry](#) [NETIO_ADDRESS_UDPv6_ANY_UNICAST_ROUTE](#)
Constant to initialize a UDPv6 unicast route.
- const struct [NETIO_DGRAM_InterfaceRouteEntry](#) [NETIO_ADDRESS_UDPv6_ANY_MULTICAST_ROUTE](#)
Constant to initialize UDPv6 multicast route.

6.14.1 Detailed Description

NETIO DGRAM Interface.

<<eXtension>> The NETIO DGRAM implements the RTI Connex DDS Micro NETIO interface and exposes a subset of the interface enabling users to write an integration with concrete network interface.

6.14.2 Macro Definition Documentation

NETIO_DGRAM_INTERFACE_SHARED_PORT_FLAG

```
#define NETIO_DGRAM_INTERFACE_SHARED_PORT_FLAG (0x1U)
```

Flag that indicates that an interface uses shared ports.

RTI Connex DDS Micro uses Locators to specify transport addresses to send and receive data. A Locator consists of a kind, port, and a transport address. The kind indicates the type of transport, such as UDPv4, the port is used to reach a DDS DomainParticipant, and the address is used to reach the destination transport. RTI Connex DDS Micro can work with two different types of transports, one that uses shared ports and one that does not.

When a transport uses shared ports, it does not matter which transport address a message was received on; only the port matters. For example, if a computer has two network interfaces A and B and is listening for messages on port P, it does not matter if the message is received on A or B. That is, as long as the message is received on any network interface capable of receiving on port P, the message is accepted.

When a transport does not use shared ports it means it does matter which transport address a message was received on. For example, if a computer has two network interfaces A and B and is listening for messages on port P, but has only specified that the A should receive on port P, then messages received on interface B and port P are ignored.

RTI Connex DDS Micro support this flag on per RTI Connex DDS Micro transport basis.

It is important to note that when a message is accepted, it is routed to all relevant DDS datareaders and datawriters. Thus, this feature cannot be used to control that some DDS topics should only be accepted when received on a specific transport interface. However, this feature could be useful to allow different DDS DomainParticipants to use the same port, but with different network interfaces.

NETIO_DGRAM_INTERFACE_MAX_INTERFACES

```
#define NETIO_DGRAM_INTERFACE_MAX_INTERFACES (8)
```

The maximum number of network interfaces a NETIO_DGRAM Transport instance can manage.

Each NETIO_DGRAM Transport instance is associated with network interface as returned by the [NETIO_DGRAM_InterfaceTableEntrySeq](#). The maximum number of interfaces that can be managed by NETIO_DGRAM is NETIO_DGRAMINTERFACE_MAX↵_INTERFACES.

NETIO_DGRAM_INTERFACE_MAX_ADDRESSES

```
#define NETIO_DGRAM_INTERFACE_MAX_ADDRESSES (8)
```

The maximum number of addresses which can be returned in an address reservation request.

Each NETIO_DGRAM Transport instance receives address reservation requests to reserve addresses to receive discovery and user data based on. The maximum number of addresses that can be reserved in a single request is NETIO_DGRAM_INTERFACE_MAX_ADDRESSES.

NETIO_DGRAM_InterfaceMultiCastGroup_INITIALIZER

```
#define NETIO_DGRAM_InterfaceMultiCastGroup_INITIALIZER
```

Value:

```
{ \
    NETIO_Address_INITIALIZER, \
    NETIO_Address_INITIALIZER, \
    NETIO_Netmask_INITIALIZER \
}
```

Constant to initialize a [NETIO_DGRAM_InterfaceMultiCastGroup](#) attribute as empty.

NETIO_DGRAM_InterfaceMultiCastGroup_Invalid

```
#define NETIO_DGRAM_InterfaceMultiCastGroup_Invalid
```

Value:

```
{ \
    {-1, 0, {{0, 0, 0, 0}}, {0}}, \
    {-1, 0, {{0, 0, 0, 0}}, {0}}, \
    {0, {0, 0, 0, 0}} \
}
```

Constant to initialize a [NETIO_DGRAM_InterfaceMultiCastGroup](#) attribute as invalid.

NETIO_DGRAM_InterfaceMultiCastGroup_UDPv4

```
#define NETIO_DGRAM_InterfaceMultiCastGroup_UDPv4
```

Value:

```
{ \
    {NETIO_ADDRESS_KIND_UDPv4, 0, {{NETIO_htonl(0xe0000000), 0, 0, 0}}, {0}}, \
    {NETIO_ADDRESS_KIND_UDPv4, 0, {{NETIO_htonl(0xffffffff), 0, 0, 0}}, {0}}, \
    {4, {0, 0, 0, 0}} \
}
```

Constant to initialize a [NETIO_DGRAM_InterfaceMultiCastGroup](#) attribute as a UDPv4.

NETIO_DGRAM_InterfaceMultiCastGroup_UDPv6

```
#define NETIO_DGRAM_InterfaceMultiCastGroup_UDPv6
```

Value:

```
{ \
    {NETIO_ADDRESS_KIND_UDPv6, 0, {{NETIO_htonl(0xff000000), 0, 0, 0}}, {0}}, \
    {NETIO_ADDRESS_KIND_UDPv6, 0, {{0xffffffff, 0xffffffff, 0xffffffff, 0xffffffff}}, {0}}, \
    {8, {0, 0, 0, 0}} \
}
```

Constant to initialize a [NETIO_DGRAM_InterfaceMultiCastGroup](#) attribute as a UDPv6 multicast group.

NETIO_DGRAM_InterfaceTableEntry_INITIALIZER

```
#define NETIO_DGRAM_InterfaceTableEntry_INITIALIZER
```

Value:

```
{ \
    OU, \
    OU, \
    OU, \
    NETIO_Address_INITIALIZER, \
    NETIO_DGRAM_InterfaceMultiCastGroup_INITIALIZER, \
    NULL \
}
```

Constant to initialize an [NETIO_DGRAM_InterfaceTableEntry](#) attribute as an empty entry.

NETIO_DGRAM_InterfaceTableEntrySeq_INITIALIZER

```
#define NETIO_DGRAM_InterfaceTableEntrySeq_INITIALIZER REDA_DEFINE_SEQUENCE_INITIALIZER(struct \
NETIO_DGRAM_InterfaceTableEntry)
```

Constant to initialize a [NETIO_DGRAM_InterfaceTableEntry](#) as empty.

6.14.3 Typedef Documentation**NETIO_DGRAM_Interface_createFunc**

```
typedef NETIO_Interface_T *(* NETIO_DGRAM_Interface_createFunc) (NETIO_Interface_T *upstream, void \
*property)
```

The NETIO_DGRAM create interface function required by [NETIO_DGRAM_Interface](#).

NETIO_DGRAMUserInterface_get_interface_listFunc

```
typedef RTI_BOOL(* NETIO_DGRAMUserInterface_get_interface_listFunc) (NETIO_Interface_T *user_intf, \
struct NETIO_DGRAM_InterfaceTableEntrySeq *if_table)
```

NETIO_DGRAM get interface list function required by [NETIO_DGRAM_Interface](#).

NETIO_DGRAMUserInterface_resolve_addressFunc

```
typedef RTI_BOOL(* NETIO_DGRAMUserInterface_resolve_addressFunc) (NETIO_Interface_T *netio_↵ \
intf, const struct NETIO_DGRAM_InterfaceTableEntry *if_entry, const char *address_string, struct \
NETIO_Address *address_value, RTI_BOOL *is_invalid)
```

NETIO_DGRAM address resolve function.

NETIO_DGRAMUserInterface_bind_addressFunc

```
typedef RTI_BOOL(* NETIO_DGRAMUserInterface_bind_addressFunc) (NETIO_Interface_T *user_intf, struct  
NETIO_Address *source)
```

NETIO_DGRAM bind address function. by [NETIO_DGRAM_InterfaceI](#).

NETIO_DGRAMUserInterface_deleteFunc

```
typedef void(* NETIO_DGRAMUserInterface_deleteFunc) (NETIO_Interface_T *user_intf)
```

The NETIO_DGRAM delete interface function. NOT in CERT.

NETIO_DGRAM_InterfaceI

```
typedef struct NETIO_DGRAM_InterfaceI NETIO_DGRAM_InterfaceI
```

<<*eXtension*>> Definition of the NETIO_DGRAM Interface.

6.14.4 Function Documentation

NETIO_DGRAM_InterfaceFactory_register()

```
RTI_BOOL NETIO_DGRAM_InterfaceFactory_register (  
    RT_Registry_T * registry,  
    const char * name,  
    NETIO_DGRAM_InterfaceI * user_intf,  
    void * user_property)
```

Register the DGRAM interface.

Registers the DGRAM interface component factory, sets the interface name, sets the instance's properties, and the user interface operations.

The interface operations are the supporting network layer functions that the DGRAM uses for it's routing, binding, and communication with peer NETIO.

Parameters

in	<i>registry</i>	Registry instance.
in	<i>name</i>	Interface name to register.
in	<i>user_intf</i>	User interface.
in	<i>user_property</i>	User defined property.

Returns

RTI_TRUE on success, RTI_FALSE on failure.

MT Safety:

This operation is not thread safe.

6.14.5 Variable Documentation

NETIO_ADDRESS_UDPv4_ANY_UNICAST_ROUTE

```
const struct NETIO_DGRAM_InterfaceRouteEntry NETIO_ADDRESS_UDPv4_ANY_UNICAST_ROUTE [extern]
```

Constant to initialize a UDPv4 unicast route.

RTI Connex DDS Micro uses an internal route table to determine which RTI Connex DDS Micro transport to use to send outgoing messages on (refer to `::get_route_table` for details). The `NETIO_ADDRESS_UDPv4_ANY_UNICAST_ROUTE` can be used to configure a transport as being able to send messages to any UDPv4 address.

NETIO_ADDRESS_UDPv4_ANY_MULTICAST_ROUTE

```
const struct NETIO_DGRAM_InterfaceRouteEntry NETIO_ADDRESS_UDPv4_ANY_MULTICAST_ROUTE [extern]
```

Constant to initialize UDPv4 multicast route.

RTI Connex DDS Micro uses an internal route table to determine which RTI Connex DDS Micro transport to use to send outgoing messages on (refer to `::get_route_table` for details). The `NETIO_ADDRESS_UDPv4_ANY_MULTICAST_ROUTE` can be used to configure a transport as being able to send messages to any UDPv4 multicast address.

NETIO_ADDRESS_UDPv6_ANY_UNICAST_ROUTE

```
const struct NETIO_DGRAM_InterfaceRouteEntry NETIO_ADDRESS_UDPv6_ANY_UNICAST_ROUTE [extern]
```

Constant to initialize a UDPv6 unicast route.

RTI Connex DDS Micro uses an internal route table to determine which RTI Connex DDS Micro transport to use to send outgoing messages on (refer to `::get_route_table` for details). The `NETIO_ADDRESS_UDPv6_ANY_UNICAST_ROUTE` can be used to configure a transport as being able to send messages to any UDPv6 address.

NETIO_ADDRESS_UDPv6_ANY_MULTICAST_ROUTE

```
const struct NETIO_DGRAM_InterfaceRouteEntry NETIO_ADDRESS_UDPv6_ANY_MULTICAST_ROUTE [extern]
```

Constant to initialize UDPv6 multicast route.

RTI Connex DDS Micro uses an internal route table to determine which RTI Connex DDS Micro transport to use to send outgoing messages on (refer to `::get_route_table` for details). The `NETIO_ADDRESS_UDPv6_ANY_MULTICAST_ROUTE` can be used to configure a transport as being able to send messages to any UDPv6 multicast address.

6.15 Zero Copy v2 API

NETIO Zero Copy v2 interface.

Topics

- [Notification Interface](#)
ZCOPY Notification Interface.

Classes

- struct [ZCOPY_NotifMechanismProperty](#)
Notification mechanism property for the default implementation.
- struct [ZCOPY_NotifInterfaceFactoryProperty](#)
Notification interface property.
- struct [ZCOPY_NotifUserInterfaceI](#)
Notification mechanism user interface.

Macros

- `#define ZCOPY\_NotifInterfaceFactoryProperty\_INITIALIZER`
Constant to initialize a [ZCOPY_NotifInterfaceFactoryProperty](#).

Typedefs

- typedef [NETIO_Interface_T](#) `*(ZCOPY\_NotifUserInterface\_createFunc) (NETIO\_Interface\_T *upstream, void *user_property)`
Create an instance of a user interface with upstream as the owner.
- typedef void `*(ZCOPY\_NotifUserInterface\_deleteFunc) (NETIO\_Interface\_T *user_intf)`
Delete an instance of a user interface.
- typedef `RTI_BOOL` `*(ZCOPY\_NotifUserInterface\_reserve\_addressFunc) (NETIO\_Interface\_T *user_intf, struct NETIO\_Address *src_addr, void **port_entry_out)`
Reserve an address on a user interface.
- typedef `RTI_BOOL` `*(ZCOPY\_NotifUserInterface\_release\_addressFunc) (NETIO\_Interface\_T *user_intf, struct NETIO\_Address *src_addr, void *port_entry)`
Release an address on a user interface.
- typedef `RTI_BOOL` `*(ZCOPY\_NotifUserInterface\_add\_routeFunc) (NETIO\_Interface\_T *user_intf, struct NETIO\_Address *source, struct NETIO\_Address *destination, void **route_entry_out)`
Add a route on a user interface.
- typedef `RTI_BOOL` `*(ZCOPY\_NotifUserInterface\_delete\_routeFunc) (NETIO\_Interface\_T *user_intf, struct NETIO\_Address *source, struct NETIO\_Address *destination, void *route_entry)`
Delete a route on a user interface.
- typedef `RTI_BOOL` `*(ZCOPY\_NotifUserInterface\_sendFunc) (NETIO\_Interface\_T *self, struct NETIO\_Address *source, struct NETIO\_Address *destination, void *route_entry)`
Send a notification on a user interface.
- typedef `RTI_BOOL` `*(ZCOPY\_NotifUserInterface\_bindFunc) (NETIO\_Interface\_T *user_intf, struct NETIO\_Address *src_addr, struct NETIO\_Address *dst_addr, void *port_entry, void **bind_entry_out)`
Bind an address on a user interface.
- typedef `RTI_BOOL` `*(ZCOPY\_NotifUserInterface\_unbindFunc) (NETIO\_Interface\_T *user_intf, struct NETIO\_Address *src_addr, struct NETIO\_Address *dst_addr, void *port_entry, void *bind_entry)`
Unbind an address on a user interface.
- typedef `RTI_BOOL` `*(ZCOPY\_NotifUserInterface\_notify\_portFunc) (NETIO\_Interface\_T *user_intf, void *port←_entry)`
Notify a specific receive port.
- typedef struct [ZCOPY_NotifUserInterfaceI](#) [ZCOPY_NotifUserInterfaceI](#)
Notification mechanism user interface.

6.15.1 Detailed Description

NETIO Zero Copy v2 interface.

<<[eXtension](#)>> The NETIO Zero Copy v2 module implements the RTI Connex DDS Micro Zero Copy v2 interface.

6.15.2 Macro Definition Documentation

ZCOPY_NotifInterfaceFactoryProperty_INITIALIZER

```
#define ZCOPY_NotifInterfaceFactoryProperty_INITIALIZER
```

Value:

```
{
    NETIO_InterfaceFactoryProperty_INITIALIZER, /* _parent */
    1, /* max_samples_per_notif */
    NULL, /* user_intf */
    NULL, /* user_property */
}
```

Constant to initialize a [ZCOPY_NotifInterfaceFactoryProperty](#).

6.15.3 Typedef Documentation

ZCOPY_NotifUserInterface_createFunc

```
typedef NETIO\_Interface\_T *(* ZCOPY_NotifUserInterface_createFunc) (NETIO\_Interface\_T *upstream,
void *user_property)
```

Create an instance of a user interface with upstream as the owner.

ZCOPY_NotifUserInterface_deleteFunc

```
typedef void(* ZCOPY_NotifUserInterface_deleteFunc) (NETIO\_Interface\_T *user_intf)
```

Delete an instance of a user interface.

ZCOPY_NotifUserInterface_reserve_addressFunc

```
typedef RTI\_BOOL(* ZCOPY_NotifUserInterface_reserve_addressFunc) (NETIO\_Interface\_T *user_intf,
struct NETIO\_Address *src_addr, void **port_entry_out)
```

Reserve an address on a user interface.

Instruct the notification mechanism interface specified by user_intf to setup resources for listening to messages on the address src_addr and return a port_entry_out. The port_entry_out will be provided to a bind call.

ZCOPY_NotifUserInterface_release_addressFunc

```
typedef RTI_BOOL(* ZCOPY_NotifUserInterface_release_addressFunc) (NETIO_Interface_T *user_intf,  
struct NETIO_Address *src_addr, void *port_entry)
```

Release an address on a user interface.

Instruct the notification mechanism interface specified by user_intf to release resources for listening to messages on the address src_addr.

ZCOPY_NotifUserInterface_add_routeFunc

```
typedef RTI_BOOL(* ZCOPY_NotifUserInterface_add_routeFunc) (NETIO_Interface_T *user_intf, struct  
NETIO_Address *source, struct NETIO_Address *destination, void **route_entry_out)
```

Add a route on a user interface.

Instruct the notification mechanism interface specified by user_intf to add a route from the source to the destination. The route_entry_out will be provided to the user later when calling send.

ZCOPY_NotifUserInterface_delete_routeFunc

```
typedef RTI_BOOL(* ZCOPY_NotifUserInterface_delete_routeFunc) (NETIO_Interface_T *user_intf, struct  
NETIO_Address *source, struct NETIO_Address *destination, void *route_entry)
```

Delete a route on a user interface.

Not available in CERT.

Instruct the notification mechanism interface specified by user_intf to remove a route from the source to the destination.

ZCOPY_NotifUserInterface_sendFunc

```
typedef RTI_BOOL(* ZCOPY_NotifUserInterface_sendFunc) (NETIO_Interface_T *self, struct NETIO_Address  
*source, struct NETIO_Address *destination, void *route_entry)
```

Send a notification on a user interface.

Instruct the notification mechanism interface specified by user_intf to send a notification to the destination using the route_entry.

ZCOPY_NotifUserInterface_bindFunc

```
typedef RTI_BOOL(* ZCOPY_NotifUserInterface_bindFunc) (NETIO_Interface_T *user_intf, struct NETIO_Address  
*src_addr, struct NETIO_Address *dst_addr, void *port_entry, void **bind_entry_out)
```

Bind an address on a user interface.

Instruct the notification mechanism interface specified by user_intf to start listening for messages on the address specified by src_addr on the given port_entry.

ZCOPY_NotifUserInterface_unbindFunc

```
typedef RTI_BOOL(* ZCOPY_NotifUserInterface_unbindFunc) (NETIO_Interface_T *user_intf, struct
NETIO_Address *src_addr, struct NETIO_Address *dst_addr, void *port_entry, void *bind_entry)
```

Unbind an address on a user interface.

Instruct the notification mechanism interface specified by user_intf to stop listening for messages on the address specified by src_addr.

ZCOPY_NotifUserInterface_notify_portFunc

```
typedef RTI_BOOL(* ZCOPY_NotifUserInterface_notify_portFunc) (NETIO_Interface_T *user_intf, void
*port_entry)
```

Notify a specific receive port.

Instruct the notification mechanism interface specified by user_intf to notify the receive port specified by port_entry.

ZCOPY_NotifUserInterfaceI

```
typedef struct ZCOPY_NotifUserInterfaceI ZCOPY_NotifUserInterfaceI
```

Notification mechanism user interface.

Interface functions for the notification mechanism user interface. This allows the user to specify custom implementations of the notification mechanism. The Default Notification Mechanism provided with /ndds can be accessed using ::DDSZCOPY_NotifUserInterface::get_interface.

See also

::DDSZCOPY_NotifMechanism::register

6.15.4 Notification Interface

ZCOPY Notification Interface.

Variables

- NETIO_ZCOPYDIIExport const char *const [NETIO_DEFAULT_NOTIF_NAME](#)
Default name for notification interface.

6.15.4.1 Detailed Description

ZCOPY Notification Interface.

The Notification Interface is implemented as a NETIO interface and NETIO interface factory.

6.15.4.2 Variable Documentation

NETIO_DEFAULT_NOTIF_NAME

```
NETIO_ZCOPYDllExport const char* const NETIO_DEFAULT_NOTIF_NAME [extern]
```

Default name for notification interface.

6.16 OS API

Topics

- [OSAPI Heap API](#)
Abstract Heap API.
- [Trace API](#)
OSAPI Trace.
- [OSAPI Mutex API](#)
Abstract Mutex API.
- [OSAPI Process API](#)
Abstract Process API.
- [OSAPI Semaphore API](#)
Abstract Semaphore API.
- [OSAPI Memory API](#)
Memory Interface.
- [OSAPI String API](#)
String Interface.
- [OSAPI System API](#)
Abstract System API.
- [OSAPI Thread API](#)
OSAPI Thread API.
- [OSAPI time API](#)
Abstract Time API.
- [OSAPI Timer API](#)
Abstract Timer API.
- [OSAPI Types](#)
OSAPI Types.
- [OSAPI_SharedMemoryMonitor](#)
A shared memory monitor to provide mutual exclusion between processes and inter-process signaling.
- [OSAPI_SharedMemorySegment](#)
Allow access to a shared memory segment (memory block) so that multiple processes can read-and write a common memory area.
- [Log API](#)
OSAPI Log API.

6.16.1 Detailed Description

6.16.2 OSAPI Heap API

Abstract Heap API.

Macros

- #define [OSAPI_get_alignment_of](#)(testType)
Gets alignment of a type.
- #define [OSAPI_ALIGNMENT_DEFAULT](#) (-1)
Certain methods allow a default alignment: this should be an alignment that follows the "malloc" alignment of the architecture (aligned sufficiently to store any C-structure efficiently).
- #define [OSAPI_Heap_is_address_aligned](#)(location, alignment)
Check if a particular address is aligned to an alignment.
- #define [OSAPI_Heap_align_size_up](#)(size, alignment)
Aligns a particular address to an alignment alignment.

Functions

- void * [OSAPI_Heap_allocate](#) (RTI_SIZE_T count, RTI_SIZE_T size)
Allocates zero-initialized memory from the heap.
- void * [OSAPI_Heap_realloc](#) (void *ptr, RTI_SIZE_T size)
Reallocate memory from the heap.
- void [OSAPI_Heap_free](#) (void *ptr)
Frees memory allocated from the heap.
- void [OSAPI_Heap_allocate_struct](#) (MACRO_TYPE **pointer, MACRO_TYPE type)
Allocates space on the heap for a C structure and initializes the structure with 0's.
- void [OSAPI_Heap_free_struct](#) (MACRO_TYPE *pointer)
Returns previously allocated.
- void [OSAPI_Heap_allocate_buffer](#) (char **buffer, RTI_SIZE_T size, OSAPI_Alignment_T alignment)
Allocate a block of memory of the provided size from the heap.
- void [OSAPI_Heap_free_buffer](#) (void *buffer)
A static method to return a block (that was previously allocated with [OSAPI_Heap_allocate_buffer](#)) to the heap.

6.16.2.1 Detailed Description

Abstract Heap API.

6.16.2.2 Macro Definition Documentation

OSAPI_get_alignment_of

```
#define OSAPI_get_alignment_of(  
    testType)
```

Value:

```
((\n    struct S_ { char c; testType member; } ; \n    (int)((char*)&(((struct S_*)NULL)->member) - (char*)NULL); \n))
```

Gets alignment of a type.

OSAPI_ALIGNMENT_DEFAULT

```
#define OSAPI_ALIGNMENT_DEFAULT (-1)
```

Certain methods allow a default alignment: this should be an alignment that follows the "malloc" alignment of the architecture (aligned sufficiently to store any C-structure efficiently).

OSAPI_Heap_is_address_aligned

```
#define OSAPI_Heap_is_address_aligned(  
    location,  
    alignment)
```

Value:

```
((RTI_UINT32)((char*)(location) - (char*)OSAPI_CC_NullPtr) % (alignment)) == 0)
```

Check if a particular address is aligned to an alignment.

OSAPI_Heap_align_size_up

```
#define OSAPI_Heap_align_size_up(  
    size,  
    alignment)
```

Value:

```
((size) + ((alignment) - 1U)) & (~((alignment) - 1U))
```

Aligns a particular address to an alignment alignment.

6.16.2.3 Function Documentation

OSAPI_Heap_allocate()

```
void * OSAPI_Heap_allocate (
    RTI_SIZE_T count,
    RTI_SIZE_T size)
```

Allocates zero-initialized memory from the heap.

The function allocates `count` elements of size `size` and sets the memory region to 0.

NOTE: This operation assumes the specified size to be > 0 ; this operation is never used directly but it is only accessed by using one of the other allocate operations of OSAPI_Heap (e.g. `allocate_struct`, `allocate_array`, `allocate_string...`); these operations are implemented as macros and accept a "type" argument which is always converted to a size > 0 using the `sizeof` operator.

Parameters

in	<i>count</i>	The number of elements to allocate.
in	<i>size</i>	The size of the element; size is assumed to be always greater than 0.

Returns

. A pointer to a contiguous memory area guaranteed to be large enough store `count` elements of size `size`.

See Also [OSAPI_Heap_free](#)

OSAPI_Heap_realloc()

```
void * OSAPI_Heap_realloc (  
    void * ptr,  
    RTI_SIZE_T size)
```

Reallocate memory from the heap.

Parameters

in	<i>ptr</i>	Currently allocated buffer
in	<i>size</i>	The new desired size

Returns

. A pointer to a contiguous memory area guaranteed to be large enough store `size` bytes, NULL if the reallocation failed.

See Also [OSAPI_Heap_free](#)

OSAPI_Heap_free()

```
void OSAPI_Heap_free (  
    void * ptr)
```

Frees memory allocated from the heap.

The function frees memory previously allocated with [OSAPI_Heap_allocate](#) back to the heap.

Parameters

in	<i>ptr</i>	Pointer to region previous allocated with OSAPI_Heap_allocate
----	------------	---

See Also [OSAPI_Heap_allocate](#)

OSAPI_Heap_allocate_struct()

```
void OSAPI_Heap_allocate_struct (
    MACRO_TYPE ** pointer,
    MACRO_TYPE type)
```

Allocates space on the heap for a C structure and initializes the structure with 0's.

The returned space must be freed with [OSAPI_Heap_free_struct](#).

This operation is implemented as a macro in order to support the specification of a "type" argument (converted to a size value using the sizeof operator).

Parameters

out	<i>pointer</i>	Pointer to the allocated structure.
in	<i>type</i>	Type of the structure to allocate.

See Also [OSAPI_Heap_free_struct](#)

OSAPI_Heap_free_struct()

```
void OSAPI_Heap_free_struct (
    MACRO_TYPE * pointer)
```

Returns previously allocated.

space (that was previously allocated with [OSAPI_Heap_allocate_struct](#)) to the heap.

This operation is implemented as a macro in order to support the specification of a "type" argument.

Parameters

in	<i>pointer</i>	Pointer to region previous allocated
----	----------------	--------------------------------------

See Also [OSAPI_Heap_allocate_struct](#)

References [RTI_UINT32](#).

OSAPI_Heap_allocate_buffer()

```
void OSAPI_Heap_allocate_buffer (
    char ** buffer,
    RTI_SIZE_T size,
    OSAPI_Alignment_T alignment)
```

Allocate a block of memory of the provided size from the heap.

Parameters

<i>buffer</i>	<< <i>out</i> >> Cannot be NULL.
<i>size</i>	<< <i>in</i> >> Must be > 0.
<i>alignment</i>	<< <i>in</i> >> Power of 2 and >0 or OSAPI_ALIGNMENT_DEFAULT.

If the alignment is OSAPI_ALIGNMENT_DEFAULT, the returned pointer must have "default alignment" (meaning that any C-structure can start at the beginning of the buffer).

OSAPI_Heap_free_buffer()

```
void OSAPI_Heap_free_buffer (
    void * buffer)
```

A static method to return a block (that was previously allocated with [OSAPI_Heap_allocate_buffer](#)) to the heap.

Parameters

<i>in</i>	<i>buffer</i>	If NULL, no op.
-----------	---------------	-----------------

6.16.3 Trace API

OSAPI Trace.

OSAPI Trace.

6.16.4 OSAPI Mutex API

Abstract Mutex API.

Typedefs

- typedef struct OSAPI_Mutex [OSAPI_Mutex_T](#)

Functions

- [RTI_BOOL OSAPI_Mutex_delete](#) ([OSAPI_Mutex_T](#) *mutex)
Delete a mutex.
- [OSAPI_Mutex_T * OSAPI_Mutex_new](#) (void)
Create a mutex.
- [RTI_BOOL OSAPI_Mutex_take](#) ([OSAPI_Mutex_T](#) *self)
Take a mutex.
- [RTI_BOOL OSAPI_Mutex_give](#) ([OSAPI_Mutex_T](#) *self)
Give a mutex.

6.16.4.1 Detailed Description

Abstract Mutex API.

6.16.4.2 Typedef Documentation

OSAPI_Mutex_T

```
typedef struct OSAPI_Mutex OSAPI_Mutex_T
```

Abstract Mutex type

6.16.4.3 Function Documentation

OSAPI_Mutex_delete()

```
RTI_BOOL OSAPI_Mutex_delete (
    OSAPI_Mutex_T * mutex)
```

Delete a mutex.

Parameters

in	<i>mutex</i>	Delete a mutex created with OSAPI_Mutex_new .
----	--------------	---

Returns

RTI_TRUE on success, RTI_FALSE on failure

OSAPI_Mutex_new()

```
OSAPI_Mutex_T * OSAPI_Mutex_new (
    void )
```

Create a mutex.

Returns

Pointer to a mutex in a not taken condition. NULL on failure.

See Also [OSAPI_Mutex_delete](#)

OSAPI_Mutex_take()

```
RTI_BOOL OSAPI_Mutex_take (
    OSAPI_Mutex_T * self)
```

Take a mutex.

A mutex can only be taken if it is not currently already taken by another thread; it CAN be taken if is it already taken by the same thread. In order to release a mutex, it must be given as many times a it has been taken. Note that `take` will block indefinitely.

Note: the mutex is not required to support priority inversion; there is no protection against deadlocks or starvation.

Parameters

<code>in</code>	<code>self</code>	Take a mutex previously created with OSAPI_Mutex_new .
-----------------	-------------------	--

Returns

RTI_TRUE on success, RTI_FALSE on failure

See Also [OSAPI_Mutex_give](#), [OSAPI_Mutex_new](#)

OSAPI_Mutex_give()

```
RTI_BOOL OSAPI_Mutex_give (  
    OSAPI_Mutex_T * self)
```

Give a mutex.

A mutex can only be given if it is owned by the calling thread. Furthermore, it must be given as many times as it has been taken.

Parameters

<code>in</code>	<code>self</code>	Take a mutex previously created with OSAPI_Mutex_new .
-----------------	-------------------	--

Returns

RTI_TRUE on success, RTI_FALSE on failure

See Also [OSAPI_Mutex_take](#), [OSAPI_Mutex_new](#)

References [RTI_UINT32](#).

6.16.5 OSAPI Process API

Abstract Process API.

Functions

- [RTI_BOOL OSAPI_Process_is_alive](#) (OSAPI_ProcessId pid)
Returns true if a process is alive.
- OSAPI_ProcessId [OSAPI_Process_getpid](#) (void)
Return the process ID, which is unique for the application.

6.16.5.1 Detailed Description

Abstract Process API.

6.16.5.2 Function Documentation

OSAPI_Process_is_alive()

```
RTI_BOOL OSAPI_Process_is_alive (
    OSAPI_ProcessId pid)
```

Returns true if a process is alive.

When passed in PID 0, the function will always return RTI_TRUE

Returns

RTI_BOOL

OSAPI_Process_getpid()

```
OSAPI_ProcessId OSAPI_Process_getpid (
    void )
```

Return the process ID, which is unique for the application.

Return the process ID.

Returns

Process id

6.16.6 OSAPI Semaphore API

Abstract Semaphore API.

Macros

- #define `OSAPI_SEMAPHORE_TIMEOUT_INFINITE` -1
If `OSAPI_Semaphore_take` is called with `OSAPI_SEMAPHORE_TIMEOUT_INFINITE` as timeout, `OSAPI_Semaphore_take` will not return until the semaphore is signaled.
- #define `OSAPI_SEMAPHORE_RESULT_OK` 0
If `OSAPI_Semaphore_take` succeeds and the semaphore is signaled within the timeout period, `OSAPI_Semaphore_take` returns `TRUE` and sets the failure reason to `OSAPI_SEMAPHORE_RESULT_OK`.
- #define `OSAPI_SEMAPHORE_RESULT_TIMEOUT` 1
If `OSAPI_Semaphore_take` was called with a timeout value different from `OSAPI_SEMAPHORE_TIMEOUT_INFINITE`, and the semaphore was not signaled before the timeout expired, `OSAPI_Semaphore_take` returns `TRUE` and sets the failure reason to `OSAPI_SEMAPHORE_RESULT_TIMEOUT`.
- #define `OSAPI_SEMAPHORE_RESULT_ERROR` 2
If `OSAPI_Semaphore_take` fails for an unknown reason, `OSAPI_Semaphore_take` returns `FALSE` and sets the failure reason to `OSAPI_SEMAPHORE_RESULT_ERROR`.

Typedefs

- typedef struct OSAPI_Semaphore [OSAPI_Semaphore_T](#)
Abstract Semaphore type.

Functions

- [RTI_BOOL OSAPI_Semaphore_delete](#) ([OSAPI_Semaphore_T](#) *self)
Delete a semaphore.

6.16.6.1 Detailed Description

Abstract Semaphore API.

6.16.6.2 Macro Definition Documentation

OSAPI_SEMAPHORE_TIMEOUT_INFINITE

```
#define OSAPI_SEMAPHORE_TIMEOUT_INFINITE -1
```

If [OSAPI_Semaphore_take](#) is called with OSAPI_SEMAPHORE_TIMEOUT_INFINITE as timeout, [OSAPI_Semaphore_take](#) will not return until the semaphore is signaled.

OSAPI_SEMAPHORE_RESULT_OK

```
#define OSAPI_SEMAPHORE_RESULT_OK 0
```

If [OSAPI_Semaphore_take](#) succeeds and the semaphore is signaled within the timeout period, [OSAPI_Semaphore_take](#) returns TRUE and sets the failure reason to OSAPI_SEMAPHORE_RESULT_OK.

OSAPI_SEMAPHORE_RESULT_TIMEOUT

```
#define OSAPI_SEMAPHORE_RESULT_TIMEOUT 1
```

If [OSAPI_Semaphore_take](#) was called with a timeout value different from OSAPI_SEMAPHORE_TIMEOUT_INFINITE, and the semaphore was not signaled before the timeout expired, [OSAPI_Semaphore_take](#) returns TRUE and sets the failure reason to OSAPI_SEMAPHORE_RESULT_TIMEOUT.

OSAPI_SEMAPHORE_RESULT_ERROR

```
#define OSAPI_SEMAPHORE_RESULT_ERROR 2
```

If [OSAPI_Semaphore_take](#) fails for an unknown reason, [OSAPI_Semaphore_take](#) returns FALSE and sets the failure reason to OSAPI_SEMAPHORE_RESULT_ERROR.

6.16.6.3 Typedef Documentation

OSAPI_Semaphore_T

```
typedef struct OSAPI_Semaphore OSAPI_Semaphore_T
```

Abstract Semaphore type.

6.16.6.4 Function Documentation

OSAPI_Semaphore_delete()

```
RTI_BOOL OSAPI_Semaphore_delete (  
    OSAPI_Semaphore_T * self)
```

Delete a semaphore.

Parameters

in	self	Semaphore created with OSAPI_Semaphore_new .
----	------	--

Returns

RTI_TRUE on success, RTI_FALSE on failure.

See Also [OSAPI_Semaphore_new](#)

6.16.7 OSAPI Memory API

Memory Interface.

Memory Interface.

6.16.8 OSAPI String API

String Interface.

String Interface.

6.16.9 OSAPI System API

Abstract System API.

Classes

- struct [OSAPI_SystemI](#)
System API.
- struct [OSAPI_TaskProperty](#)
Task properties.
- struct [OSAPI_SystemProperty](#)
The System Properties.

Macros

- #define [OSAPI_SYSTEM_MAX_HOSTNAME](#) (64)
Maximum hostname length.
- #define [OSAPI_TaskProperty_INITIALIZER](#)
Initializer for [OSAPI_TaskProperty](#).
- #define [OSAPI_SystemProperty_INITIALIZER](#)
Initializer for [OSAPI_SystemProperty](#).

Typedefs

- typedef [RTI_BOOL](#)(* [OSAPI_System_start_timer_T](#)) ([OSAPI_Timer_T](#) self, [OSAPI_TimerTickHandlerFunction](#) tick_handler)
Start a OSAPI timer.
- typedef [RTI_INT32](#)(* [OSAPI_System_get_timer_resolution_T](#)) (void)
Get the resolution of the clock driving the timer in nano seconds.
- typedef [RTI_BOOL](#)(* [OSAPI_System_get_time_T](#)) ([OSAPI_SystemTime](#) *now)
Get the current system time.
- typedef [RTI_BOOL](#)(* [OSAPI_System_initialize_T](#)) (void)
Initialize the system.
- typedef [RTI_BOOL](#)(* [OSAPI_System_get_hostname_T](#)) (char *const hostname)
Get the hostname.
- typedef [RTI_BOOL](#)(* [OSAPI_System_generate_uuid_T](#)) (struct [OSAPI_SystemUUID](#) *uuid_out)
Generate a unique universal identifier (UUID).
- typedef [RTI_BOOL](#)(* [OSAPI_System_get_ticktime_T](#)) ([RTI_INT32](#) *sec, [RTI_UINT32](#) *nanosec)
Get current tick time.
- typedef struct [OSAPI_SystemI](#) [OSAPI_SystemI](#)
System API.

6.16.9.1 Detailed Description

Abstract System API.

The System is defined as the physical or virtual execution environment and implements functions that are not necessarily provided by the operating system. For example, a custom embedded board may include a special real-time clock which is synchronized via GPS, but is not available via a regular OS system call.

The following functions are not thread-safe:

- `OSAPI_System_set_interface`
- `OSAPI_System_get_property`
- `OSAPI_System_set_property`
- `OSAPI_System_get_native_interface`
- `OSAPI_System_clock_tick`
- `OSAPI_System_clock_tick_from_time`

6.16.9.2 Macro Definition Documentation

OSAPI_SYSTEM_MAX_HOSTNAME

```
#define OSAPI_SYSTEM_MAX_HOSTNAME (64)
```

Maximum hostname length.

OSAPI_TaskProperty_INITIALIZER

```
#define OSAPI_TaskProperty_INITIALIZER
```

Value:

```
{ \
    { /* OSAPI_ThreadProperty */ \
      OSAPI_THREAD_USE_OSDEFAULT_STACKSIZE, \
      OSAPI_THREAD_PRIORITY_DEFAULT, \
      OSAPI_THREAD_DEFAULT_OPTIONS \
    }, \
    100 \
}
```

Initializer for [OSAPI_TaskProperty](#).

OSAPI_SystemProperty_INITIALIZER

```
#define OSAPI_SystemProperty_INITIALIZER
```

Value:

```
{\
    OSAPI_TimerProperty_INITIALIZER,\
    {0},\
    32,\
    8,\
    OSAPI_TaskProperty_INITIALIZER \
}
```

Initializer for [OSAPI_SystemProperty](#).

6.16.9.3 Typedef Documentation

OSAPI_System_start_timer_T

```
typedef RTI_BOOL(* OSAPI_System_start_timer_T) (OSAPI_Timer_T self, OSAPI_TimerTickHandlerFunction
tick_handler)
```

Start a OSAPI timer.

Parameters

<i>self</i>	<<in>> Timer object.
<i>tick_handler</i>	<<in>> Timer handle.

Returns

RTI_TRUE on success, RTI_FALSE on failure.

OSAPI_System_get_timer_resolution_T

```
typedef RTI_INT32(* OSAPI_System_get_timer_resolution_T) (void)
```

Get the resolution of the clock driving the timer in nano seconds.

This function must return the frequency of the system timer used to implement [OSAPI_SystemImpl::start_timer](#) and [OSAPI_SystemImpl::stop_timer](#) API.

Returns

timer resolution in nanoseconds.

OSAPI_System_get_time_T

```
typedef RTI_BOOL(* OSAPI_System_get_time_T) (OSAPI_SystemTime *now)
```

Get the current system time.

In general, the system time is used by components to correlate both internal and external events, such as data reception and ordering. Thus, it is recommended that this function returns the real time. However, it is not strictly required.

Notes: - It is assumed that the time returned by `get_time` is monotonically increasing. It is up to the implementation of this function to ensure this holds true.

- It is ok to return the same time as the last call.

- The clock used to report real-time can be different than the clock used to support `start_timer` and `stop_timer`.

Parameters

<i>now</i>	<<out>> Time.
------------	---------------

Returns

RTI_TRUE on success, RTI_FALSE on failure.

OSAPI_System_initialize_T

```
typedef RTI_BOOL(* OSAPI_System_initialize_T) (void)
```

Initialize the system.

This function initializes the system and calls the port specific initialize method first. The port specific initialization method must return RTI_TRUE on success and RTI_FALSE on failure. A system can only be initialized once.

Returns

RTI_TRUE on success, RTI_FALSE on failure.

OSAPI_System_get_hostname_T

```
typedef RTI_BOOL(* OSAPI_System_get_hostname_T) (char *const hostname)
```

Get the hostname.

Parameters

<i>hostname</i>	<<out>> The buffer to store the hostname. Must be at least OSAPI_SYSTEM_MAX_HOSTNAME bytes. If the actual hostname is longer than OSAPI_SYSTEM_MAX_HOSTNAME bytes (including the NUL termination) the hostname is truncated.
-----------------	--

Returns

RTI_TRUE on success, RTI_FALSE on failure.

OSAPI_System_generate_uuid_T

```
typedef RTI_BOOL(* OSAPI_System_generate_uuid_T) (struct OSAPI_SystemUUID *uuid_out)
```

Generate a unique universal identifier (UUID).

Parameters

<i>uuid_out</i>	<<out>> The generated UUID value
-----------------	----------------------------------

Returns

RTI_TRUE on success, RTI_FALSE on failure

OSAPI_System_get_ticktime_T

```
typedef RTI_BOOL(* OSAPI_System_get_ticktime_T) (RTI_INT32 *sec, RTI_UINT32 *nanosec)
```

Get current tick time.

The ticktime is a time measurement used by Micro to determine how much time has elapsed in a period. It does not have to be an absolute time, but it *must* be monotonically increasing. For this reason it is important to choose the time source with care. For example, the system time may mbe adjusted backward or forward and this could affect the measure time lapse. The resolution of the tick time is expected to be no less than the system timer, although it is not a requirement.

Parameters

<i>sec</i>	<<out>> The current ticktime in seconds
<i>nanosec</i>	<<out>> Additional nanoseconds in the current ticktime

Returns

RTI_TRUE on success, RTI_FALSE on failure.

OSAPI_SystemI

```
typedef struct OSAPI_SystemI OSAPI_SystemI
```

System API.

The methods in this interface are called by the corresponding OSAPI_System function. For example, OSAPI_System↵_get_time() calls system->get_time().

6.16.10 OSAPI Thread API

OSAPI Thread API.

Classes

- struct [OSAPI_ThreadProperty](#)
Properties for thread creation.

Macros

- #define `OSAPI_THREAD_PRIORITY_LOW` -1
Low priority.
- #define `OSAPI_THREAD_PRIORITY_BELOW_NORMAL` -2
Below normal priority.
- #define `OSAPI_THREAD_PRIORITY_NORMAL` -3
Normal priority.
- #define `OSAPI_THREAD_PRIORITY_ABOVE_NORMAL` -4
Above normal priority.
- #define `OSAPI_THREAD_PRIORITY_HIGH` -5
High priority.
- #define `OSAPI_THREAD_PRIORITY_INHERIT` -6
Use the same priority as the spawning thread. Only applies to POSIX.
- #define `OSAPI_THREAD_USE_OSDEFAULT_STACKSIZE` 0
Use the default stack size when creating a thread.
- #define `OSAPI_THREAD_DEFAULT_OPTIONS` 0x00
Use only the default options the OS provides when creating a thread.
- #define `OSAPI_THREAD_FLOATING_POINT` 0x01
Hint to the OS to create a thread that supports floating point.
- #define `OSAPI_THREAD_STDIO` 0x02
Hint to the OS to create a thread that supports standard I/O.
- #define `OSAPI_THREAD_REALTIME_PRIORITY` 0x08
Hint to the OS to create a thread that runs in real-time priority mode.
- #define `OSAPI_THREAD_PROPERTY_DEFAULT`
Default thread properties.
- #define `OSAPI_ThreadProperty_INITIALIZER OSAPI_THREAD_PROPERTY_DEFAULT`
Initializer for `OSAPI_ThreadProperty`.

Typedefs

- typedef `RTI_UINT32 OSAPI_ThreadOptions`
Options for thread creation.

6.16.10.1 Detailed Description

OSAPI Thread API.

6.16.10.2 Macro Definition Documentation

`OSAPI_THREAD_PRIORITY_LOW`

```
#define OSAPI_THREAD_PRIORITY_LOW -1
```

Low priority.

OSAPI_THREAD_PRIORITY_BELOW_NORMAL

```
#define OSAPI_THREAD_PRIORITY_BELOW_NORMAL -2
```

Below normal priority.

OSAPI_THREAD_PRIORITY_NORMAL

```
#define OSAPI_THREAD_PRIORITY_NORMAL -3
```

Normal priority.

OSAPI_THREAD_PRIORITY_ABOVE_NORMAL

```
#define OSAPI_THREAD_PRIORITY_ABOVE_NORMAL -4
```

Above normal priority.

OSAPI_THREAD_PRIORITY_HIGH

```
#define OSAPI_THREAD_PRIORITY_HIGH -5
```

High priority.

OSAPI_THREAD_PRIORITY_INHERIT

```
#define OSAPI_THREAD_PRIORITY_INHERIT -6
```

Use the same priority as the spawning thread. Only applies to POSIX.

OSAPI_THREAD_USE_OSDEFAULT_STACKSIZE

```
#define OSAPI_THREAD_USE_OSDEFAULT_STACKSIZE 0
```

Use the default stack size when creating a thread.

OSAPI_THREAD_DEFAULT_OPTIONS

```
#define OSAPI_THREAD_DEFAULT_OPTIONS 0x00
```

Use only the default options the OS provides when creating a thread.

OSAPI_THREAD_FLOATING_POINT

```
#define OSAPI_THREAD_FLOATING_POINT 0x01
```

Hint to the OS to create a thread that supports floating point.

OSAPI_THREAD_STDIO

```
#define OSAPI_THREAD_STDIO 0x02
```

Hint to the OS to create a thread that supports standard I/O.

OSAPI_THREAD_REALTIME_PRIORITY

```
#define OSAPI_THREAD_REALTIME_PRIORITY 0x08
```

Hint to the OS to create a thread that runs in real-time priority mode.

OSAPI_THREAD_PROPERTY_DEFAULT

```
#define OSAPI_THREAD_PROPERTY_DEFAULT
```

Value:

```
{ \
    OSAPI_THREAD_USE_OSDEFAULT_STACKSIZE, \
    OSAPI_THREAD_PRIORITY_DEFAULT, \
    OSAPI_THREAD_DEFAULT_OPTIONS \
}
```

Default thread properties.

OSAPI_ThreadProperty_INITIALIZER

```
#define OSAPI_ThreadProperty_INITIALIZER OSAPI_THREAD_PROPERTY_DEFAULT
```

Initializer for [OSAPI_ThreadProperty](#).

6.16.10.3 Typedef Documentation**OSAPI_ThreadOptions**

```
typedef RTI_UINT32 OSAPI_ThreadOptions
```

Options for thread creation.

6.16.11 OSAPI time API

Abstract Time API.

Classes

- struct [OSAPI_SystemTime](#)
System Time.

Typedefs

- typedef struct OSAPI_SystemTime [OSAPI_SystemTime](#)
System Time.

6.16.11.1 Detailed Description

Abstract Time API.

6.16.11.2 Typedef Documentation

OSAPI_SystemTime

```
typedef struct OSAPI_SystemTime OSAPI_SystemTime
```

System Time.

Expresses time in seconds and nanoseconds. Assumes that Nanoseconds is less than 1000000000.

6.16.12 OSAPI Timer API

Abstract Timer API.

Classes

- struct [OSAPI_TimerProperty](#)
Timer properties.

6.16.12.1 Detailed Description

Abstract Timer API.

6.16.13 OSAPI Types

OSAPI Types.

Macros

- `#define RTI_INT8`
A 2's complement 8-bit signed integer.
- `#define RTI_UINT8`
A 8-bit unsigned integer.
- `#define RTI_INT16`
A 2's complement 16-bit signed integer.
- `#define RTI_UINT16`
A 2's complement 16-bit unsigned integer.
- `#define RTI_INT32`
A 2's complement 32-bit signed integer.
- `#define RTI_UINT32`
A 32-bit unsigned integer.
- `#define RTI_INT64`
A 64-bit signed integer.
- `#define RTI_UINT64`
A 64-bit unsigned integer.
- `#define RTI_FALSE ((RTI_BOOL) 0)`
Boolean false value.
- `#define RTI_TRUE ((RTI_BOOL) 1)`
Boolean true value.

Typedefs

- `typedef RTI_INT32 RTI_BOOL`
- `typedef RTI_UINT32 RTI_SIZE_T`
Platform-specific unsigned integer type for representing sizes.

6.16.13.1 Detailed Description

OSAPI Types.

6.16.13.2 Macro Definition Documentation

RTI_INT8

```
#define RTI_INT8
```

A 2's complement 8-bit signed integer.

Referenced by [CDR_Stream_is_byte_swapped\(\)](#).

A 2's complement 16-bit signed integer.

A 2's complement 16-bit unsigned integer.

A 2's complement 32-bit signed integer.

Referenced by [CDR_Stream_increment_current_position\(\)](#), [DDS_Duration_is_infinite\(\)](#), [DDS_String_cmp\(\)](#), [DDS_String_free\(\)](#), [DDS_String_length\(\)](#), [DDS_String_ncmp\(\)](#), [NETIO_SharedMemoryMutex_attach\(\)](#), [NETIO_SharedMemoryMutex_create\(\)](#), [NETIO_SharedMemoryMutex_create_or_attach\(\)](#), [NETIO_SharedMemoryMutex_delete\(\)](#), [NETIO_SharedMemoryMutex_lock\(\)](#), [NETIO_SharedMemoryMutex_unlock\(\)](#), [NETIO_SharedMemorySegment_attach\(\)](#), [NETIO_SharedMemorySegment_create\(\)](#), [NETIO_SharedMemorySegment_create_or_attach\(\)](#), [NETIO_SharedMemorySegment_get_address\(\)](#), [NETIO_SharedMemorySegment_get_size\(\)](#), [NETIO_SharedMemorySignalingSemaphore_attach\(\)](#), [NETIO_SharedMemorySignalingSemaphore_create\(\)](#), [NETIO_SharedMemorySignalingSemaphore_detach\(\)](#), [NETIO_SharedMemorySignalingSemaphore_signal\(\)](#), [NETIO_SharedMemorySignalingSemaphore_wait\(\)](#), [OSAPI_Log_get_last_error_code\(\)](#), [OSAPI_Log_get_verbosity\(\)](#), [OSAPI_Semaphore_give\(\)](#), and [OSAPI_Semaphore_take\(\)](#).

A 32-bit unsigned integer.

Referenced by [UDPInterfaceTable::add_entry\(\)](#), [CDR_Stream_check_size\(\)](#), [CDR_Stream_get_current_position_ptr\(\)](#), [CDR_Stream_increment_current_position\(\)](#), [CDR_Stream_is_byte_swapped\(\)](#), [CDR_Stream_set_current_position_offset\(\)](#), [DDS_Duration_is_infinite\(\)](#), [DDS_String_length\(\)](#), [NETIO_SharedMemorySegment_create\(\)](#), [NETIO_SharedMemorySegment_create_or](#), [NETIO_SharedMemorySegment_get_size\(\)](#), [OSAPI_Heap_free_struct\(\)](#), [OSAPI_Log_get_last_error_code\(\)](#), [OSAPI_Log_get_verbosity\(\)](#), [OSAPI_Mutex_give\(\)](#), [OSAPI_Semaphore_give\(\)](#), and [OSAPI_Trace_set_trace_mask\(\)](#).

RTI_INT64

```
#define RTI_INT64
```

A 64-bit signed integer.

Referenced by [OSAPI_Log_get_last_error_code\(\)](#).

RTI_UINT64

```
#define RTI_UINT64
```

A 64-bit unsigned integer.

Referenced by [OSAPI_SharedMemorySegment_create_impl\(\)](#).

RTI_FALSE

```
#define RTI_FALSE ((RTI_BOOL) 0)
```

Boolean false value.

Functions that return [RTI_BOOL](#) return RTI_FALSE as logical false.

RTI_TRUE

```
#define RTI_TRUE ((RTI_BOOL) 1)
```

Boolean true value.

Functions that return [RTI_BOOL](#) return RTI_TRUE as logical true.

6.16.13.3 Typedef Documentation**RTI_BOOL**

```
typedef RTI_INT32 RTI_BOOL
```

The type [RTI_BOOL](#) uses [RTI_TRUE](#) as logical true and [RTI_FALSE](#) as logical false. In general, a zero value is considered false, and a non-zero value is considered true.

RTI_SIZE_T

```
typedef RTI_UINT32 RTI_SIZE_T
```

Platform-specific unsigned integer type for representing sizes.

6.16.14 OSAPI_SharedMemoryMonitor

A shared memory monitor to provide mutual exclusion between processes and inter-process signaling.

Functions

- NETIOPSLDIIExport [RTI_SIZE_T OSAPI_SharedMemoryMonitor_get_size](#) (void)
Get the amount of memory required to store OSAPI_SharedMemoryMonitor on the current platform.
- NETIOPSLDIIExport [RTI_BOOL OSAPI_SharedMemoryMonitor_initialize](#) ([RTI_SIZE_T](#) mem_len, void *mem, struct OSAPI_SharedMemoryMonitor **self_out)
Initialize a shared memory monitor in a provided memory segment.
- NETIOPSLDIIExport [RTI_BOOL OSAPI_SharedMemoryMonitor_finalize](#) (struct OSAPI_SharedMemoryMonitor *self)
Finalize a shared memory monitor.
- NETIOPSLDIIExport [RTI_BOOL OSAPI_SharedMemoryMonitor_acquire](#) (struct OSAPI_SharedMemoryMonitor *self, [OSAPI_SHMEM_STATUS](#) *status_out)
Acquire the lock protecting the monitor.
- NETIOPSLDIIExport [RTI_BOOL OSAPI_SharedMemoryMonitor_release](#) (struct OSAPI_SharedMemoryMonitor *self)
Release the lock protecting the monitor.
- NETIOPSLDIIExport [RTI_BOOL OSAPI_SharedMemoryMonitor_wait](#) (struct OSAPI_SharedMemoryMonitor *self, [OSAPI_SHMEM_STATUS](#) *status_out)
Wait for the monitor to be signaled.
- NETIOPSLDIIExport [RTI_BOOL OSAPI_SharedMemoryMonitor_signal](#) (struct OSAPI_SharedMemoryMonitor *self)
Signal a thread waiting on a monitor to wake up.
- NETIOPSLDIIExport [RTI_BOOL OSAPI_SharedMemoryMonitor_mark_consistent](#) (struct OSAPI_SharedMemoryMonitor *self)
Mark the shared state protected by the monitor as consistent after a process died while holding the lock on the monitor.

6.16.14.1 Detailed Description

A shared memory monitor to provide mutual exclusion between processes and inter-process signaling.

6.16.14.2 Function Documentation

OSAPI_SharedMemoryMonitor_get_size()

```
NETIOPSLDIIExport RTI\_SIZE\_T OSAPI_SharedMemoryMonitor_get_size (
    void )
```

Get the amount of memory required to store OSAPI_SharedMemoryMonitor on the current platform.

Returns

The number of bytes required to store a shared memory monitor.

OSAPI_SharedMemoryMonitor_initialize()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemoryMonitor_initialize (
    RTI_SIZE_T mem_len,
    void * mem,
    struct OSAPI_SharedMemoryMonitor ** self_out)
```

Initialize a shared memory monitor in a provided memory segment.

Parameters

in	<i>mem_len</i>	Size of provided memory.
in	<i>mem</i>	Memory segment.
out	<i>self_out</i>	Pointer to initialized monitor.

Returns

RTI_TRUE on success. RTI_FALSE if an error occurred.

OSAPI_SharedMemoryMonitor_finalize()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemoryMonitor_finalize (
    struct OSAPI_SharedMemoryMonitor * self)
```

Finalize a shared memory monitor.

Parameters

in	<i>self</i>	Monitor to finalize.
----	-------------	----------------------

Returns

RTI_TRUE on success. RTI_FALSE if an error occurred.

Once this function is called (even if it returns RTI_FALSE) you can no longer use this monitor.

OSAPI_SharedMemoryMonitor_acquire()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemoryMonitor_acquire (
    struct OSAPI_SharedMemoryMonitor * self,
    OSAPI_SHMEM_STATUS * status_out)
```

Acquire the lock protecting the monitor.

If this function returns RTI_TRUE, the calling thread is guaranteed mutual exclusion by the monitor. No other thread can acquire the lock until the lock is returned with [OSAPI_SharedMemoryMonitor_release](#). This function shall not be called by a thread which currently holds the lock.

If this function returns status_out=OSAPI_SHMEM_STATUS_OWNER_DEAD, then any shared data protected by the monitor may be in an inconsistent state. If the shared data can be restored to a consistent state, then it can be marked as consistent with [OSAPI_SharedMemoryMonitor_mark_consistent](#). Otherwise, any subsequent attempts to acquire or wait on the monitor will fail.

Parameters

in	<i>self</i>	Monitor to acquire.
out	<i>status_out</i>	If successful, the status of the protected state.

Returns

RTI_TRUE on success. RTI_FALSE if an error occurred.

OSAPI_SharedMemoryMonitor_release()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemoryMonitor_release (  
    struct OSAPI_SharedMemoryMonitor * self)
```

Release the lock protecting the monitor.

Parameters

in	<i>self</i>	Monitor to release.
----	-------------	---------------------

Returns

RTI_TRUE on success. RTI_FALSE if an error occurred.

This function can only be called on a monitor which is currently locked by the calling thread.

OSAPI_SharedMemoryMonitor_wait()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemoryMonitor_wait (  
    struct OSAPI_SharedMemoryMonitor * self,  
    OSAPI_SHMEM_STATUS * status_out)
```

Wait for the monitor to be signaled.

Only one thread can wait on a monitor at any one time. This function can only be called on a monitor which is currently locked by the calling thread. The function will return successfully only when another thread has called [OSAPI_SharedMemoryMonitor_signal](#) and when the calling thread has acquired the lock.

If this function returns `status_out=OSAPI_SHMEM_STATUS_OWNER_DEAD`, then any shared data protected by the monitor may be in an inconsistent state. If the shared data can be restored to a consistent state, then it can be marked as consistent with [OSAPI_SharedMemoryMonitor_mark_consistent](#). Otherwise, any subsequent attempts to acquire or wait on the monitor will fail.

Parameters

in	<i>self</i>	Monitor to wait on.
out	<i>status_out</i>	If successful, the status of the protected state.

Returns

RTI_TRUE on success. RTI_FALSE if an error occurred.

OSAPI_SharedMemoryMonitor_signal()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemoryMonitor_signal (
    struct OSAPI_SharedMemoryMonitor * self)
```

Signal a thread waiting on a monitor to wake up.

Parameters

in	self	Monitor to signal.
----	------	--------------------

Returns

RTI_TRUE on success. RTI_FALSE if an error occurred.

This function can only be called on a monitor which is currently locked by the calling thread.

OSAPI_SharedMemoryMonitor_mark_consistent()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemoryMonitor_mark_consistent (
    struct OSAPI_SharedMemoryMonitor * self)
```

Mark the shared state protected by the monitor as consistent after a process died while holding the lock on the monitor.

This function can only be called on a monitor which is currently locked by the calling thread and currently inconsistent. The function shall return the monitor to a consistent state and allow other functions to be called on it. This function may not be supported on all platforms.

Parameters

in	self	Monitor to mark consistent.
----	------	-----------------------------

Returns

RTI_TRUE on success. RTI_FALSE if an error occurred.

6.16.15 OSAPI_SharedMemorySegment

Allow access to a shared memory segment (memory block) so that multiple processes can read-and write a common memory area.

Classes

- struct [OSAPI_SharedMemorySegmentHeader](#)
- struct [OSAPI_SharedMemorySegmentHandle](#)

Enumerations

- enum `OSAPI_SHMEM_MODE` {
`OSAPI_SHMEM_MODE_UNKNOWN` , `OSAPI_SHMEM_MODE_READ` , `OSAPI_SHMEM_MODE_WRITE` ,
`OSAPI_SHMEM_MODE_LOCKABLE` ,
`OSAPI_SHMEM_MODE_ROBUST` }
- enum `OSAPI_SHMEM_STATUS` { `OSAPI_SHMEM_STATUS_OK` , `OSAPI_SHMEM_STATUS_OWNER_DEAD` }
- enum `OSAPI_SHMEM_ATTACH_STATUS` { `OSAPI_SHMEM_ATTACH_STATUS_OK` , `OSAPI_SHMEM_ATTACH_STATUS_SEGMENT_DEAD` }

Functions

- `RTI_UINT64 OSAPI_SharedMemorySegment_get_max_size` (void)
Get maximum size of a shared memory segment on the current platform.
- `RTI_BOOL OSAPI_SharedMemorySegment_is_owner_alive` (struct `OSAPI_SharedMemorySegmentHandle` *handle, `RTI_BOOL` *alive)
Check if the process which created the shared memory segment is still alive.
- `NETIOPSLDIIExport RTI_SIZE_T OSAPI_SharedMemorySegmentHandle_get_size` (`OSAPI_SHMEM_MODE` mode)
Get the amount of memory required to store a `OSAPI_SharedMemorySegmentHandle` on the current platform for a given mode.
- `NETIOPSLDIIExport RTI_SIZE_T OSAPI_SharedMemorySegmentHeader_get_size` (`OSAPI_SHMEM_MODE` mode)
Get the amount of memory, aligned to the maximum alignment, required to store a `OSAPI_SharedMemorySegmentHeader` on the current platform for a given mode.
- `NETIOPSLDIIExport RTI_BOOL OSAPI_SharedMemorySegment_create_impl` (struct `OSAPI_SharedMemorySegmentHandle` *handle, const char *name, `RTI_UINT64` size, `OSAPI_SHMEM_MODE` mode, `RTI_BOOL` *exists)
Creates a new shared memory segment with the provided key and initialize a handle to it.
- `NETIOPSLDIIExport RTI_BOOL OSAPI_SharedMemorySegment_delete_impl` (struct `OSAPI_SharedMemorySegmentHandle` *handle)
Delete a shared memory segment.
- `NETIOPSLDIIExport RTI_BOOL OSAPI_SharedMemorySegment_attach_impl` (struct `OSAPI_SharedMemorySegmentHandle` *handle, const char *name, `OSAPI_SHMEM_MODE` mode, `OSAPI_SHMEM_ATTACH_STATUS` *status_out)
Attach to the shared memory and initialize a handle to it.
- `NETIOPSLDIIExport RTI_BOOL OSAPI_SharedMemorySegment_detach_impl` (struct `OSAPI_SharedMemorySegmentHandle` *handle)
detach the shared memory segment.
- `NETIOPSLDIIExport RTI_BOOL OSAPI_SharedMemorySegment_lock_impl` (struct `OSAPI_SharedMemorySegmentHandle` *handle, `OSAPI_SHMEM_STATUS` *status_out)
Lock a shared memory segment.
- `NETIOPSLDIIExport RTI_BOOL OSAPI_SharedMemorySegment_unlock_impl` (struct `OSAPI_SharedMemorySegmentHandle` *handle)
Unlock a shared memory segment previously locked with `OSAPI_SharedMemorySegment_lock_impl`.
- `NETIOPSLDIIExport RTI_BOOL OSAPI_SharedMemorySegment_mark_consistent_impl` (struct `OSAPI_SharedMemorySegmentHandle` *handle)
Mark a shared memory segment as consistent. This function can only be called if a previous call to `OSAPI_SharedMemorySegment_lock_impl` returned `OSAPI_SHMEM_FAIL_REASON_OWNER_DEAD`.
- `NETIOPSLDIIExport RTI_BOOL OSAPI_SharedMemory_is_robust_mutex_supported` (void)
Checks whether robust mutex is supported or not.

6.16.15.1 Detailed Description

Allow access to a shared memory segment (memory block) so that multiple processes can read-and write a common memory area.

6.16.15.2 Enumeration Type Documentation

OSAPI_SHMEM_MODE

enum [OSAPI_SHMEM_MODE](#)

Mode used to create or attach to a shared memory segment. Each mode defines specific capabilities of the segment.

Enumerator

OSAPI_SHMEM_MODE_UNKNOWN	Invalid mode
OSAPI_SHMEM_MODE_READ	Read permission
OSAPI_SHMEM_MODE_WRITE	Write permission, implies OSAPI_SHMEM_MODE_READ
OSAPI_SHMEM_MODE_LOCKABLE	Locking capability, implies OSAPI_SHMEM_MODE_WRITE
OSAPI_SHMEM_MODE_ROBUST	Robust locking capability, implies OSAPI_SHMEM_MODE_LOCKABLE

OSAPI_SHMEM_STATUS

enum [OSAPI_SHMEM_STATUS](#)

Status used to convey the state of a shared memory object.

Enumerator

OSAPI_SHMEM_STATUS_OK	Success
OSAPI_SHMEM_STATUS_OWNER_DEAD	Previous owner of lock died while holding it

OSAPI_SHMEM_ATTACH_STATUS

enum [OSAPI_SHMEM_ATTACH_STATUS](#)

Status used to convey the state of a shared memory object's attachment to a shared memory segment.

Enumerator

OSAPI_SHMEM_ATTACH_STATUS_OK	Success
OSAPI_SHMEM_ATTACH_STATUS_SEGMENT_NOT_FOUND	Segment was not found when trying to attach

6.16.15.3 Function Documentation

OSAPI_SharedMemorySegment_get_max_size()

```
RTI_UINT64 OSAPI_SharedMemorySegment_get_max_size (
    void )
```

Get maximum size of a shared memory segment on the current platform.

Returns

RTI_TRUE on success. RTI_FALSE if an error occurred.

OSAPI_SharedMemorySegment_is_owner_alive()

```
RTI_BOOL OSAPI_SharedMemorySegment_is_owner_alive (
    struct OSAPI_SharedMemorySegmentHandle * handle,
    RTI_BOOL * alive)
```

Check if the process which created the shared memory segment is still alive.

Parameters

in	<i>handle</i>	The shared memory segment handle identifying the segment.
out	<i>alive</i>	If the process is alive.

Returns

RTI_TRUE on success. RTI_FALSE if an error occurred.

The behavior of this function is undefined if the segment is not currently attached.

OSAPI_SharedMemorySegmentHandle_get_size()

```
NETIOPSLDllExport RTI_SIZE_T OSAPI_SharedMemorySegmentHandle_get_size (
    OSAPI_SHMEM_MODE mode)
```

Get the amount of memory required to store a [OSAPI_SharedMemorySegmentHandle](#) on the current platform for a given mode.

Parameters

in	<i>mode</i>	The creation or attachment mode of the handle.
----	-------------	--

Returns

Number of bytes required to store a handle.

OSAPI_SharedMemorySegmentHeader_get_size()

```
NETIOPSLDllExport RTI_SIZE_T OSAPI_SharedMemorySegmentHeader_get_size (
    OSAPI_SHMEM_MODE mode)
```

Get the amount of memory, aligned to the maximum alignment, required to store a [OSAPI_SharedMemorySegmentHeader](#) on the current platform for a given mode.

Parameters

<i>in</i>	<i>mode</i>	The creation mode of the segment containing the header.
-----------	-------------	---

Returns

Number of bytes required to store a header.

The number of bytes returned by this function shall be aligned to the maximum alignment of the platform.

OSAPI_SharedMemorySegment_create_impl()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemorySegment_create_impl (
    struct OSAPI_SharedMemorySegmentHandle * handle,
    const char * name,
    RTI_UINT64 size,
    OSAPI_SHMEM_MODE mode,
    RTI_BOOL * exists)
```

Creates a new shared memory segment with the provided key and initialize a handle to it.

Parameters

<i>in, out</i>	<i>handle</i>	Handle to be initialized.
<i>in</i>	<i>name</i>	Name to identify the shared memory segment.
<i>in</i>	<i>size</i>	Size of memory to be allocated.
<i>in</i>	<i>mode</i>	Mode used to create the shared memory segment. The mode determines the capabilities of the segment. The mode must be $\geq$ OSAPI_SHMEM_MODE_WRITE.
<i>out</i>	<i>exists</i>	If the segment already exists

Returns

RTI_TRUE on success. RTI_FALSE if an error occurred.

The new shared memory segment shall be initialized with null bytes ('\0').

References [RTI_UINT64](#).

OSAPI_SharedMemorySegment_delete_impl()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemorySegment_delete_impl (
    struct OSAPI_SharedMemorySegmentHandle * handle)
```

Delete a shared memory segment.

Parameters

in	<i>handle</i>	The shared memory segment handle identifying the segment to delete.
----	---------------	---

Returns

RTI_TRUE on success. RTI_FALSE if either the precondition was not met or the shared memory segment could not be deleted.

Once this function is called (even if it returns RTI_FALSE) you can no longer use this shared memory segment.

OSAPI_SharedMemorySegment_attach_impl()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemorySegment_attach_impl (  
    struct OSAPI_SharedMemorySegmentHandle * handle,  
    const char * name,  
    OSAPI_SHMEM_MODE mode,  
    OSAPI_SHMEM_ATTACH_STATUS * status_out)
```

Attach to the shared memory and initialize a handle to it.

Parameters

in, out	<i>handle</i>	Handle to be initialized.
in	<i>name</i>	Name to identify the shared memory segment.
in	<i>mode</i>	Mode used to attach the shared memory segment.
out	<i>status_out</i>	If successful, the attachment status of the shared memory segment.

Returns

RTI_TRUE on success. Note that the attachment status may still be different from OK, so checking the value of status_out is mandatory. RTI_FALSE if an error occurred.

OSAPI_SharedMemorySegment_detach_impl()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemorySegment_detach_impl (  
    struct OSAPI_SharedMemorySegmentHandle * handle)
```

detach the shared memory segment.

Parameters

in	<i>handle</i>	The shared memory segment handle identifying the segment to detach.
----	---------------	---

Returns

RTI_TRUE on success. RTI_FALSE if either the precondition was not met or the shared memory segment could not be detached.

Once this function is called (even if it returns RTI_FALSE) you can no longer use this shared memory segment handle.

OSAPI_SharedMemorySegment_lock_impl()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemorySegment_lock_impl (
    struct OSAPI_SharedMemorySegmentHandle * handle,
    OSAPI_SHMEM_STATUS * status_out)
```

Lock a shared memory segment.

Parameters

in	<i>handle</i>	The shared memory segment handle identifying the segment to lock.
out	<i>status_out</i>	Success or failure reason of acquiring the lock.

Returns

RTI_TRUE on success. RTI_FALSE if an error occurred.

OSAPI_SharedMemorySegment_unlock_impl()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemorySegment_unlock_impl (
    struct OSAPI_SharedMemorySegmentHandle * handle)
```

Unlock a shared memory segment previously locked with [OSAPI_SharedMemorySegment_lock_impl](#).

Parameters

in	<i>handle</i>	The shared memory segment handle identifying the segment to unlock.
----	---------------	---

Returns

RTI_TRUE on success. RTI_FALSE if an error occurred.

The behavior of this method is undefined if the calling process has not previously acquired the lock with [OSAPI_SharedMemorySegment_lock_impl](#).

OSAPI_SharedMemorySegment_mark_consistent_impl()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemorySegment_mark_consistent_impl (
    struct OSAPI_SharedMemorySegmentHandle * handle)
```

Mark a shared memory segment as consistent. This function can only be called if a previous call to [OSAPI_SharedMemorySegment_lock_impl](#) returned OSAPI_SHMEM_FAIL_REASON_OWNER_DEAD.

Parameters

<code>in</code>	<code>handle</code>	The shared memory segment handle identifying the segment to mark consistent.
-----------------	---------------------	--

Returns

RTI_TRUE on success. RTI_FALSE if an error occurred.

OSAPI_SharedMemory_is_robust_mutex_supported()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemory_is_robust_mutex_supported (
    void )
```

Checks whether robust mutex is supported or not .

Returns

RTI_TRUE is supported. RTI_FALSE if not supported.

6.16.16 Log API

OSAPI Log API.

Classes

- struct [OSAPI_LogProperty](#)
Configuration of logging functionality.

Macros

- #define [OSAPI_LOG_DETAIL_MODULENAME](#) (0x80000000UL)
Bitmap for enabling saving the module name in the log buffer.
- #define [OSAPI_LOG_DETAIL_SOURCEFILE](#) (0x40000000UL)
Bitmap for enabling saving the file name in the log buffer.
- #define [OSAPI_LOG_DETAIL_LINENUMBER](#) (0x20000000UL)
Bitmap for enabling saving the line number in the log buffer.
- #define [OSAPI_LOG_DETAIL_FUNCTIONNAME](#) (0x10000000UL)
Bitmap for enabling saving the function name in the log buffer.
- #define [OSAPI_LogProperty_INITIALIZER](#)
Initializer for the [OSAPI_LogProperty](#).

Typedefs

- typedef void(* [OSAPI_Log_write_buffer_T](#)) (const char *buffer, [RTI_SIZE_T](#) length)
Optional user-defined function to output a log/trace buffer.

Enumerations

- enum [OSAPI_LogVerbosity_T](#) { [OSAPI_LOG_VERBOSITY_SILENT](#) = 0 , [OSAPI_LOG_VERBOSITY_ERROR](#) = 1 , [OSAPI_LOG_VERBOSITY_WARNING](#) = 2 , [OSAPI_LOG_VERBOSITY_DEBUG](#) = 3 }

Logging verbosity.

Functions

- [RTI_INT32 OSAPI_Log_get_last_error_code](#) (void)

Returns the error code for a function that failed.

6.16.16.1 Detailed Description

OSAPI Log API.

6.16.16.2 Macro Definition Documentation

OSAPI_LOG_DETAIL_MODULENAME

```
#define OSAPI_LOG_DETAIL_MODULENAME (0x80000000UL)
```

Bitmap for enabling saving the module name in the log buffer.

See Also [OSAPI_LogProperty::log_detail](#)

OSAPI_LOG_DETAIL_SOURCEFILE

```
#define OSAPI_LOG_DETAIL_SOURCEFILE (0x40000000UL)
```

Bitmap for enabling saving the file name in the log buffer.

See Also [OSAPI_LogProperty::log_detail](#)

OSAPI_LOG_DETAIL_LINENUMBER

```
#define OSAPI_LOG_DETAIL_LINENUMBER (0x20000000UL)
```

Bitmap for enabling saving the line number in the log buffer.

See Also [OSAPI_LogProperty::log_detail](#)

OSAPI_LOG_DETAIL_FUNCTIONNAME

```
#define OSAPI_LOG_DETAIL_FUNCTIONNAME (0x10000000UL)
```

Bitmap for enabling saving the function name in the log buffer.

See Also [OSAPI_LogProperty::log_detail](#)

OSAPI_LogProperty_INIIALIZER

```
#define OSAPI_LogProperty_INIIALIZER
```

Value:

```
{ \
    OSAPI_LOG_BUFFER_SIZE, \
    OSAPI_LOG_DETAIL_ALL, \
    NULL \
}
```

Initializer for the [OSAPI_LogProperty](#).

6.16.16.3 Typedef Documentation

OSAPI_Log_write_buffer_T

```
typedef void(* OSAPI_Log_write_buffer_T) (const char *buffer, RTI\_SIZE\_T length)
```

Optional user-defined function to output a log/trace buffer.

The handler is set by `::OSAPI_Log::set_property`.

Parameters

<i>buffer</i>	<< <i>in</i> >> Pointer to buffer with log to write
<i>length</i>	<< <i>in</i> >> Length of the buffer to write

6.16.16.4 Enumeration Type Documentation

OSAPI_LogVerbosity_T

```
enum OSAPI\_LogVerbosity\_T
```

Logging verbosity.

Enumerator

OSAPI_LOG_VERBOSITY_SILENT	Logs are not written.
OSAPI_LOG_VERBOSITY_ERROR	Only error logs are written.
OSAPI_LOG_VERBOSITY_WARNING	Error and warning logs are written.
OSAPI_LOG_VERBOSITY_DEBUG	All logs are written.

6.16.16.5 Function Documentation

OSAPI_Log_get_last_error_code()

```
RTI_INT32 OSAPI_Log_get_last_error_code (
    void )
```

Returns the error code for a function that failed.

Many functions returns RTI_FALSE or NULL on failure. In order to provide additional information about reason for the failure functions may set an additional error code. This function returns the last error-code recorded for the calling thread.

Returns

Last error-code recorded for this thread

References [RTI_INT32](#), [RTI_INT64](#), and [RTI_UINT32](#).

6.17 Application Generation API

Functions

- APPGENDllExport struct RT_ComponentFactoryI * [APPGEN_Factory_get_interface](#) (void)
Gets the singleton instance of the Application Generation factory.
- APPGENDllExport [RTI_BOOL APPGEN_Factory_register](#) (RT_Registry_T *registry, struct APPGEN_Factory↵
Property *property)
Registers the Micro Application Generation factory.
- APPGENDllExport [RTI_BOOL APPGEN_Factory_unregister](#) (RT_Registry_T *registry, struct APPGEN_↵
FactoryProperty **property)
Unregisters the Micro Application Generation factory.

6.17.1 Detailed Description

The AppGen plugin is an implementation of the RTI Connex DDS Micro DDS_AppGen plugin interface and implements the application generation supported by RTI Connex DDS Micro.

RTI Connex DDS Micro requires that a DDS_AppGen plugin is registered with the name "_mag" in the Domain↵ ParticipantFactory before a participant can be created.

Each call to [DDSDomainParticipantFactory::create_participant_from_config](#) creates its own instance of the DDS_↵ AppGen plugin. Please refer to the Getting Started Guide for an example of how to register the application generation factory.

6.17.2 Function Documentation

APPGEN_Factory_get_interface()

```
APPGENDllExport struct RT_ComponentFactoryI * APPGEN_Factory_get_interface (
    void )
```

Gets the singleton instance of the Application Generation factory.

This function gets the singleton instance of the Application Generation plugin factory that is used by the middleware to create an AppGen plugin. The singleton instance of the factory must be registered with the name "_mag" in the DomainParticipantFactory.

Returns

Pointer to Application Generation plugin factory

APPGEN_Factory_register()

```
APPGENDllExport RTI_BOOL APPGEN_Factory_register (
    RT_Registry_T * registry,
    struct APPGEN_FactoryProperty * property)
```

Registers the Micro Application Generation factory.

Parameters

<i>registry</i>	Registry with which to register the component factory. Cannot be NULL.
<i>property</i>	Pointer to factory properties

Returns

BOOLEAN_TRUE if factory is successfully registered BOOLEAN_FALSE in case of error.

APPGEN_Factory_unregister()

```
APPGENDllExport RTI_BOOL APPGEN_Factory_unregister (
    RT_Registry_T * registry,
    struct APPGEN_FactoryProperty ** property)
```

Unregisters the Micro Application Generation factory.

Parameters

<i>registry</i>	Registry with which to register the component factory. Cannot be NULL.
<i>property</i>	Pointer where to copy the pointer to the factory properties passed in call to function AppGenFactory_register().

Returns

BOOLEAN_TRUE if factory is successfully unregistered. BOOLEAN_FALSE in case of error.

6.18 Lightweight Security API

Lightweight Security Plugin API.

Classes

- struct [DDS_PskPassTracker_InterfaceI](#)
PSK passphrase tracker user interface.
- struct [DDS_PskServiceFactoryProperty](#)
Property used to register the Lightweight Security Plugin.

Macros

- #define [DDS_PskServiceFactoryProperty_INITIALIZER](#)
Constant to initialize a [DDS_PskServiceFactoryProperty](#).

Variables

- DDSPSKDllExport const char *const [DDS_PSK_DEFAULT_PLUGIN_NAME](#)
Default name used for the Lightweight Security Plugin.

6.18.1 Detailed Description

Lightweight Security Plugin API.

<<eXtension>> This module provides the security solution defined by the DDS Security Specification PSK (Pre-shared Key) Builtin Plugin.

6.18.2 Macro Definition Documentation

DDS_PskServiceFactoryProperty_INITIALIZER

```
#define DDS_PskServiceFactoryProperty_INITIALIZER
```

Value:

```
{ \
    RT_ComponentFactoryProperty_INITIALIZER, \
    NULL, \
    NULL, \
    NULL, \
    NULL, \
    NULL \
}
```

Constant to initialize a [DDS_PskServiceFactoryProperty](#).

6.18.3 Variable Documentation

DDS_PSK_DEFAULT_PLUGIN_NAME

```
DDSPSKDllExport const char* const DDS_PSK_DEFAULT_PLUGIN_NAME [extern]
```

Default name used for the Lightweight Security Plugin.

6.19 Resource Limits

Topics

- [DomainParticipantFactoryQos](#)
- [DomainParticipantQos](#)
- [DataReaderQos](#)
- [DataWriterQos](#)
- [OSAPI](#)
- [UDP Transport](#)
- [Dynamic Participant Static Endpoint \(DPSE\)](#)
- [Dynamic Participant Dynamic Endpoint \(DPDE\)](#)

6.19.1 Detailed Description

6.19.2 Introduction

RTI Connex DDS Micro is designed for use in real-time systems and uses a predictable and deterministic memory manager to ensure that memory growth is not unbounded, OS memory fragmentation is eliminated and memory usage can be determined a-priori. The advantage with this design is that proper operation is ensured as soon as steady state has been reached. However, it also places an additional burden on the system designer to properly configure each resource-limit. This section describes all the resource limits in RTI Connex DDS Micro and how they impact RTI Connex DDS Micro's behavior and memory usage.

6.19.3 Dynamic Memory Allocation

RTI Connex DDS Micro allocates heap memory to create internal data-structures. It is important to know that RTI Connex DDS Micro manages memory allocated from the system heap using its own internal memory management, and only returns memory allocated from the system back to the system when something is deleted. That is, if an application never deletes anything, no memory is returned to the system.

As a rule of thumb, in RTI Connex DDS Micro only APIs that end in `_create()` or `_new()` allocate heap memory and APIs that end in `_delete()` or `_free()` free memory. An exception to this rule is dynamic discovery which allocates memory at run-time for the topic and type names.

RTI Connex DDS Micro does not support dynamically allocating resources beyond the initial configuration. That is, all resource limits must be finite. This restriction may be removed in a future version.

6.19.4 DomainParticipantFactoryQos

Topics

- [max_components](#)
- [max_participants](#)

6.19.4.1 Detailed Description

6.19.4.2 max_components

NAME

[DDS_SystemResourceLimitsQosPolicy::max_components](#)

DESCRIPTION

RTI Connex DDS Micro consists of a number of plugins and components that perform various tasks. For example, the UDP transport and the DPDE discovery plugins are RTI Connex DDS Micro components. This resource-limit limits the number of components that can be registered with RTI Connex DDS Micro. This value should only be changed if plugins and components are developed.

6.19.4.3 max_participants

NAME

[DDS_SystemResourceLimitsQosPolicy::max_participants](#)

DESCRIPTION

[DDS_SystemResourceLimitsQosPolicy::max_participants](#) limits the number of local DomainParticipants that can be created with the [DDSDomainParticipantFactory::create_participant\(\)](#) API. For the maximum number of participants that can be discovered, please refer to [remote_participant_allocation](#).

6.19.5 DomainParticipantQos

Topics

- [local_topic_allocation](#)
- [local_type_allocation](#)
- [local_publisher_allocation](#)
- [local_subscriber_allocation](#)
- [local_reader_allocation](#)
- [local_writer_allocation](#)
- [matching_writer_reader_pair_allocation](#)
- [remote_participant_allocation](#)
- [remote_writer_allocation](#)
- [remote_reader_allocation](#)
- [max_destination_ports](#)
- [max_receive_ports](#)

6.19.5.1 Detailed Description

The `DomainParticipantQos` controls resources that are applicable to the entire `DomainParticipant`. All the resources specified in the `DomainParticipantQos` are allocated when the `DomainParticipant` is created with the `DDSDomainParticipantFactory::create_participant()` call.

6.19.5.2 `local_topic_allocation`

NAME

`DDS_DomainParticipantResourceLimitsQosPolicy::local_topic_allocation`

DESCRIPTION

`DDS_DomainParticipantResourceLimitsQosPolicy::local_topic_allocation` limits the maximum number of unique `DDSTopic`'s that can be created within a single `DDSDomainParticipant` with the `DDSDomainParticipant::create_topic()` API. When a `DDSTopic` is created a string equal to the length of the topic name in bytes plus one is allocated.

6.19.5.3 `local_type_allocation`

NAME

`DDS_DomainParticipantResourceLimitsQosPolicy::local_type_allocation`

DESCRIPTION

`DDS_DomainParticipantResourceLimitsQosPolicy::local_type_allocation` limits the maximum number of unique `DDS_TypePluginI` plugins that can be registered locally within a single `DDSDomainParticipant` with the `FooTypeSupport_register_type()` API. When a `DDS_TypePluginI` is registered a string equal to the length of the type name in bytes plus one is allocated.

6.19.5.4 `local_publisher_allocation`

NAME

`DDS_DomainParticipantResourceLimitsQosPolicy::local_publisher_allocation`

DESCRIPTION

`DDS_DomainParticipantResourceLimitsQosPolicy::local_publisher_allocation` limits the maximum number of `DDSPublisher` entities that can be created within a `DomainParticipant` with the `DDSDomainParticipant::create_publisher()` API.

6.19.5.5 local_subscriber_allocation

NAME

[DDS_DomainParticipantResourceLimitsQosPolicy::local_subscriber_allocation](#)

DESCRIPTION

[DDS_DomainParticipantResourceLimitsQosPolicy::local_subscriber_allocation](#) limits the maximum number of [DDSSubscriber](#) entities that can be created within a [DomainParticipant](#) with the [DDSDomainParticipant::create_subscriber\(\)](#) API.

6.19.5.6 local_reader_allocation

NAME

[DDS_DomainParticipantResourceLimitsQosPolicy::local_reader_allocation](#)

DESCRIPTION

[DDS_DomainParticipantResourceLimitsQosPolicy::local_reader_allocation](#) limits the maximum number of [DDSDataReader](#) entities that can be created with the [DDSSubscriber::create_datareader\(\)](#) API within a single [DDSDomainParticipant](#) across all DDS Subscriber entities.

6.19.5.7 local_writer_allocation

NAME

[DDS_DomainParticipantResourceLimitsQosPolicy::local_writer_allocation](#)

DESCRIPTION

[DDS_DomainParticipantResourceLimitsQosPolicy::local_writer_allocation](#) limits the maximum number of [DDSDataWriter](#) entities that can be created with the [DDSPublisher::create_datawriter\(\)](#) API within a single [DDSDomainParticipant](#) across all [DDSPublisher](#) entities.

6.19.5.8 matching_writer_reader_pair_allocation

NAME

[DDS_DomainParticipantResourceLimitsQosPolicy::matching_writer_reader_pair_allocation](#)

DESCRIPTION

When RTI Connex Micro discovers DDS entities using a DDS discovery protocol, such as DPSE (Dynamic Participant Static Endpoint) or DPDE (Dynamic Participant Dynamic Endpoint), it matches the discovered entities with the locally created DataReaders and DataWriters. For each match, either a local DataWriter matching a remote DataReader or a local DataReader matching a remote DataWriter, internal state must be managed.

A match, in this context, is defined as being able to receive packets sent from a [DDSDataReader](#) to a [DDSDataWriter](#), or from a [DDSDataWriter](#) to a [DDSDataReader](#):

- A data-writer needs one resource for each DataReader it matches.
- A data-reader needs one resource for each DataWriter it matches.

[DDS_DomainParticipantResourceLimitsQosPolicy::matching_writer_reader_pair_allocation](#) sets the limit for the maximum number of matches that can be managed. When this resource-limit is reached no further matching can occur, even if new [DDSDataReader](#)'s and [DDSDataWriter](#)'s are discovered.

A simple rule of thumb is to set this resource-limit to:

$(\text{local_reader_allocation} + \text{local_writer_allocation}) * \text{max_remote_participants}$.

This assumes that each DataReader and DataWriter matches one DataWriter and DataReader respectively for each discovered participant. However, this may over-allocate resources.

NOTES

[DDS_DomainParticipantResourceLimitsQosPolicy::matching_reader_writer_pair_allocation](#) is not used and will be deprecated in future versions.

6.19.5.9 remote_participant_allocation

NAME

[DDS_DomainParticipantResourceLimitsQosPolicy::remote_participant_allocation](#)

DESCRIPTION

[DDS_DomainParticipantResourceLimitsQosPolicy::remote_participant_allocation](#) limits the maximum number of [DDSDomainParticipant](#)'s that can be discovered by a local [DDSDomainParticipant](#).

The DPDE discovery plugin is a first discovered first served basis and the only way to limit discovery is careful use of peer addresses.

The DPSE discovery plugin limits the discovery to those that have been asserted based on the DomainParticipant name using [DPSEDiscoveryPlugin::RemoteParticipant_assert](#)

6.19.5.10 remote_writer_allocation

NAME

[DDS_DomainParticipantResourceLimitsQosPolicy::remote_writer_allocation](#)

DESCRIPTION

[DDS_DomainParticipantResourceLimitsQosPolicy::remote_writer_allocation](#) limits the maximum number of remote [DDSDataWriter](#) entities that can be discovered by a local [DDSDomainParticipant](#). Note that a remote [DDSDataWriter](#) cannot be discovered until its parent [DDSDomainParticipant](#) has been discovered.

6.19.5.11 remote_reader_allocation

NAME

[DDS_DomainParticipantResourceLimitsQosPolicy::remote_reader_allocation](#)

DESCRIPTION

[DDS_DomainParticipantResourceLimitsQosPolicy::remote_reader_allocation](#) limits the maximum number of remote [DDSDataReader](#) entities that can be discovered by a local [DomainParticipant](#). Note that a remote [DDSDataReader](#) cannot be discovered until its parent [DDSDomainParticipant](#) has been discovered.

6.19.5.12 max_destination_ports

NAME

[DDS_DomainParticipantResourceLimitsQosPolicy::max_destination_ports](#)

DESCRIPTION

RTI Connex DDS Micro maintains a table of destination addresses which it can send to based on information queried from the registered transports. The [DDS_DomainParticipantResourceLimitsQosPolicy::max_destination_ports](#) resource-limit is unfortunately difficult to give a precise rule for how to determine since it is transport dependent. However, as a rule of thumb the following is true:

- Each network that can be reached directly by the transport requires 1 resource.
- If multicast is supported, it counts as 1 resource.
- If the transport is the default transport (by default UDP is), it counts as 1 resource.

EXAMPLES

Consider a typical Linux computer with 2 network interfaces, eth0 and lo, with support for multicast.

- 1 for the eth0 network.
- 1 for the lo network.
- 1 for multicast.
- 1 for being the default transport.

This requires 4 resources.

Consider a typical embedded computer with 1 network interface, eth0, with no support for multicast:

- 1 for the eth0 network.
- 1 for being the default transport.

This requires 2 resources.

NOTES

The default value has been set to cover most cases. This resource-limit should only be changed if a lot of interfaces are supported (increase it) or if memory is extremely limited (decrease it).

6.19.5.13 max_receive_ports

NAME

[DDS_DomainParticipantResourceLimitsQosPolicy::max_receive_ports](#)

DESCRIPTION

RTI Connext DDS Micro listens for discovery data and user data on different transports as specified by the user. The [DDS_DomainParticipantResourceLimitsQosPolicy::max_receive_ports](#) resource-limit sets the maximum number of addresses that can be listened on. As a rule of thumb, this resource limit can be determined as follows:

- A user-traffic unicast port counts as 1.
- A meta-traffic unicast port counts as 1.
- A user-traffic multicast address counts as 1.
- A meta-traffic multicast address counts as 1.

EXAMPLES

Consider the typical case with the UDP transport, using unicast and multicast for discovery data and unicast for user data:

- Discovery data needs two ports.
- User data needs one port.

Thus, this requires 3 resources.

Consider a case with the UDP transport, using unicast and multicast for discovery data and user data:

- Discovery data needs two ports.
- User data needs two ports.

Thus, this requires 4 resources.

Consider a case with the UDP transport, using unicast and 2 multicast addresses each for discovery data and user data:

- Discovery data needs three ports.
- User data needs three ports.

Thus, this requires 6 resources.

NOTES

The default value has been set to cover most cases. This resource-limit should only be changed if a lot of interfaces are supported (increase it) or if memory is extremely limited (decrease it).

6.19.6 DataReaderQos**Topics**

- [max_instances](#)
- [max_remote_writers_per_instance](#)
- [max_samples](#)
- [max_samples_per_instance](#)
- [max_remote_writers](#)
- [max_samples_per_remote_writer](#)
- [max_routes_per_writer](#)
- [max_outstanding_reads](#)
- [max_fragmented_samples](#)
- [max_fragmented_samples_per_remote_writer](#)

6.19.6.1 Detailed Description

The [DDS_DataReaderQos](#) controls the resources used by the [DDSDataReader](#). Each [DDSDataReader](#) allocates its own resources, even [DDSDataReader](#)'s of the same [DDSTopic](#). For this reason is it advised to limit the number of [DDSDataReader](#)'s per [DDSTopic](#) to one.

6.19.6.2 max_instances

NAME

[DDS_ResourceLimitsQosPolicy::max_instances](#)

DESCRIPTION

[DDS_ResourceLimitsQosPolicy::max_instances](#) limits the maximum number of instances a [DDSDataReader](#) can manage. Instance state is one of the most expensive resources to maintain as well as often being the highest one.

The resources allocated by an instance is normally only released when an instance is disposed *and* all known owners have unregistered the instance, please refer to [FooDataWriter::unregister_instance](#) and [FooDataWriter::dispose](#).

The [DDS_DataReaderResourceLimitsQosPolicy::instance_replacement](#) policy is useful to maintain state for only the last N instances and when it is not necessary to keep samples in the [DDSDataReader](#) history cache. When the [DDS_DataReaderResourceLimitsQosPolicy::instance_replacement](#) policy is set to replace the oldest instance ([DDS_REPLACE_OLDEST_INSTANCE_REPLACEMENT_QOS](#)), information about the oldest instance is automatically removed and the newly discovered instance is received. The disadvantage is that [DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE](#) state cannot be maintained. The [DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE](#) is maintained for an instance that is accepted (either there is space or another instance is replaced), regardless of whether any samples have previously been received.

SEE ALSO

[DDS_DataReaderResourceLimitsInstanceReplacementKind](#) instance_replacement

6.19.6.3 max_remote_writers_per_instance

NAME

[DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers_per_instance](#)

DESCRIPTION

[DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers_per_instance](#) limits the maximum number of remote [DDSDataWriter](#) entities a [DDSDataReader](#) maintains state for per instance. If this resource-limit is lower than the actual number of remote datawriters updating an instance only state for up to [DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers_per_instance](#) is maintained. It is not guaranteed to be the last [DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers_per_instance](#) though. However, the last remote [DDSDataWriter](#) to update an instance is *always* saved. When an instance is no longer updated by a known, to the DataReader, DataWriter, [DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE](#) state is signaled on the [DDSDataReader](#). Thus, if this resource-limit is lower than the actual number of remote datawriters updating an instance, [DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE](#) may be signaled even though there may be one or more datawriters still updating an instance. In this case the instance will go back to the [DDS_ALIVE_INSTANCE_STATE](#).

6.19.6.4 max_samples

NAME

[DDS_ResourceLimitsQosPolicy::max_samples](#)

DESCRIPTION

[DDS_ResourceLimitsQosPolicy::max_samples](#) limits the maximum number of samples a [DDSDataReader](#) can allocate and cache. Samples are cached before they are made available to the application, for example when samples are received out-of-order. Thus, this resource-limit may impact throughput and latency.

What this value should be set to is determined by a number of factors:

- For best-effort it can be set to 1 since samples are not cached. However, if data is polled or samples are loaned longer than the update frequency this resource-limit should be increased to prevent samples from being reused before the application has a chance to access the data (along with [DDS_HistoryQosPolicy::depth](#)).
- For reliable data the value should take into account the data-rate, the heartbeat rate of the [DDSDataWriter](#) and the number of piggy-backed heartbeats.

SEE ALSO

[DDS_HistoryQosPolicy::depth](#), [DDS_DataWriterQos](#)

6.19.6.5 max_samples_per_instance

NAME

[DDS_ResourceLimitsQosPolicy::max_samples_per_instance](#)

DESCRIPTION

[DDS_ResourceLimitsQosPolicy::max_samples_per_instance](#) limits the maximum number of samples that can be cached for a single instance and must be less than or equal to `max_samples` and greater or equal to the history depth. When [DDS_ResourceLimitsQosPolicy::max_samples_per_instance](#) is higher than depth it makes it possible to loan samples and receive up to depth samples before returning loaned samples.

When this resource-limit or [DDS_HistoryQosPolicy::depth](#) is exceeded and the [DDS_HistoryQosPolicy::kind](#) is [DDS_KEEP_LAST_HISTORY_QOS](#), the oldest sample is removed from the cache.

6.19.6.6 max_remote_writers

NAME

[DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers](#)

DESCRIPTION

[DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers](#) limits the maximum number of remote datawriters the [DDSDataReader](#) can receive data from regardless of which instance it is.

NOTES

Resources controlled by this limit can only be reused when a remote DataWriter is either deleted or loses liveliness.

6.19.6.7 max_samples_per_remote_writer

NAME

[DDS_DataReaderResourceLimitsQosPolicy::max_samples_per_remote_writer](#)

DESCRIPTION

[DDS_DataReaderResourceLimitsQosPolicy::max_samples_per_remote_writer](#) limits the maximum number of samples that can be cached per remote [DDSDataWriter](#) the [DDSDataReader](#) is receiving data from. This resource-limit is useful in preventing one [DDSDataWriter](#) from starving other [DDSDataWriter](#). NOTE↵: The samples cached from a [DDSDataWriter](#) is allocated from the same sample pool controlled by [DDS_ResourceLimitsQosPolicy::max_samples](#).

6.19.6.8 max_routes_per_writer

NAME

[DDS_DataReaderResourceLimitsQosPolicy::max_routes_per_writer](#)

DESCRIPTION

Each [DDSDataReader](#) maintains information about the state of its peer [DDSDataWriter](#) (those it has matched with). Part of this state is which locators (or destination addresses) it should use to send data to a particular [DDSDataWriter](#). [DDS_DataReaderResourceLimitsQosPolicy::max_routes_per_writer](#) limits the maximum number of routes that can be saved per [DDSDataWriter](#).

NOTES

This resource-limit is shared across all matched [DDSDataWriter](#) per [DDSDataReader](#). Thus, if [DDS_DataReaderResourceLimitsQosPolicy::max_routes_per_writer](#) is 2 and [DDS_DataReaderResourceLimitsQosPolicy::max_routes_per_writer](#) is 4, a total of 8 routes can be saved for both datawriters. One [DDSDataWriter](#) may have 6 routes and the other 2.

6.19.6.9 max_outstanding_reads

NAME

[DDS_DataReaderResourceLimitsQosPolicy::max_outstanding_reads](#)

DESCRIPTION

[DDS_DataReaderResourceLimitsQosPolicy::max_outstanding_reads](#) limits the maximum number of simultaneous loans an application can make at any given time using the [FooDataReader::read\(\)](#) and [FooDataReader::take\(\)](#) APIs.

Typically a value of 1 is sufficient. It is only necessary to increase this value if an application thread will access the same cache before another thread has returned the loan.

6.19.6.10 max_fragmented_samples

NAME

[DDS_DataReaderResourceLimitsQosPolicy::max_fragmented_samples](#)

DESCRIPTION

[DDS_DataReaderResourceLimitsQosPolicy::max_fragmented_samples](#) limits the maximum number of fragmented samples a [DDSDataReader](#) can cache while waiting for all fragments. A fragmented sample must be fully received before it can be deserialized. If this resource limit is reached, new fragmented samples will be rejected.

A default value equal to [DDS_ResourceLimitsQosPolicy::max_samples](#) will give the best results. However, if the data type is variable-sized where some samples may be fragmented and others not, this value can be set lower.

6.19.6.11 max_fragmented_samples_per_remote_writer

NAME

[DDS_DataReaderResourceLimitsQosPolicy::max_fragmented_samples_per_remote_writer](#)

DESCRIPTION

[DDS_DataReaderResourceLimitsQosPolicy::max_fragmented_samples_per_remote_writer](#) limits the maximum number of fragmented samples per matched [DDSDataWriter](#). This is useful to prevent one [DDSDataWriter](#) from using all the samples allocated based on [DDS_DataReaderResourceLimitsQosPolicy::max_fragmented_samples](#).

This resource limit does not allocate any resources. The resources used by a matched [DDSDataWriter](#) are allocated from the [DDS_DataReaderResourceLimitsQosPolicy::max_fragmented_samples](#) resources.

6.19.7 DataWriterQos

Topics

- [max_samples](#)
- [max_samples_per_instance](#)
- [max_instances](#)
- [max_routes_per_reader](#)
- [max_remote_readers](#)

6.19.7.1 Detailed Description

The [DDS_DataWriterQos](#) controls the resources allocated by a [DDSDataWriter](#). Each [DDSDataWriter](#) allocates its own resources, even datawriters of the same [DDSTopic](#). For this reason is it advised to limit the number of datawriters per topic to one.

6.19.7.2 max_samples

NAME

[DDS_ResourceLimitsQosPolicy::max_samples](#)

DESCRIPTION

[DDS_ResourceLimitsQosPolicy::max_samples](#) limits the maximum number of samples a [DDSDataWriter](#) can allocate and maintain state for at any given time. Every time a write operation is called, a sample is allocated from the cache and filled in with user-data.

A sample allocated from the pool controlled by this resource-limit can be reused based on the following rules:

- If the [DDSDataWriter](#) is using [DDS_BEST_EFFORT_RELIABILITY_QOS](#) effort the sample is returned for reuse after the transport returns from the write call.
- If the DataWriter is using [DDS_RELIABLE_RELIABILITY_QOS](#) and [DDS_VOLATILE_DURABILITY_QOS](#) , the sample is returned when all known DataReaders have acknowledged the sample or the DataWriter has given up on delivering the sample (controlled by [DDS_RtpsReliableWriterProtocol_t::max_heartbeat_retries](#)), whichever happens first.
- If the DataWriter is using [DDS_RELIABLE_RELIABILITY_QOS](#) and not using [DDS_VOLATILE_DURABILITY_QOS](#), the sample is returned to the pool for reuse only if the history depth is exceeded and the [DDS_HistoryQosPolicy::kind](#) is set to [DDS_KEEP_LAST_HISTORY_QOS](#)

NOTES

For best-effort communication this resource-limit can be set to 1.

For reliable communication the value of this resource-limit determines the maximum number of samples a [DDSDataWriter](#) can keep in unacknowledged state before reusing potentially unacknowledged samples (for [DDS_KEEP_LAST_HISTORY_QOS](#)). For reliable data the value should take into account the data-rate, the [DDS_RtpsReliableWriterProtocol_t::heartbeat_period](#) and [DDS_RtpsReliableWriterProtocol_t::heartbeats_per_max_samples](#).

6.19.7.3 max_samples_per_instance

NAME

[DDS_ResourceLimitsQosPolicy::max_samples_per_instance](#)

DESCRIPTION

[DDS_ResourceLimitsQosPolicy::max_samples_per_instance](#) limits the maximum number of samples that can be cached for a single instance and must be less than or equal to [max_samples](#) and greater or equal to the [DDS_HistoryQosPolicy::depth](#).

[DDS_HistoryQosPolicy::depth](#) determines how many samples are kept in case of [DDS_TRANSIENT_LOCAL_DURABILITY_QOS](#) durability and made available to late joining [DDSDataReader](#), while [DDS_ResourceLimitsQosPolicy::max_samples_per_instance](#) is the maximum number of *unacknowledged* samples the [DDSDataWriter](#) can keep per instance. Thus, if $\text{max_samples_per_instance} > \text{depth}$ it is possible to keep unacknowledged samples while limiting the number of samples made available to late joining DataReaders.

6.19.7.4 max_instances

NAME

[DDS_ResourceLimitsQosPolicy::max_instances](#)

DESCRIPTION

[DDS_ResourceLimitsQosPolicy::max_instances](#) limits the maximum number of instances a [DDSDataWriter](#) can manage. An instance resource can only be reused when it is unregistered by the [DDSDataWriter](#).

6.19.7.5 max_routes_per_reader

NAME

[DDS_DataWriterResourceLimitsQosPolicy::max_routes_per_reader](#)

DESCRIPTION

Each [DDSDataWriter](#) maintains information about the state of its peer DataReaders (those it has matched with). Part of this state is which locators (or destination addresses) it should use to send data to a particular [DDSDataReader](#). [DDS_DataWriterResourceLimitsQosPolicy::max_routes_per_reader](#) limits the number of routes that can be saved per [DDSDataReader](#).

NOTES

This resource-limit is shared across all matched DataReaders per [DDSDataWriter](#). Thus, if [DDS_DataWriterResourceLimitsQosPolicy::max_routes_per_reader](#) is 2 and [DDS_DataWriterResourceLimitsQosPolicy::max_routes_per_reader](#) is 4, a total of 8 routes can be saved for both DataReaders. One [DDSDataReader](#) may have 6 routes and the other 2.

6.19.7.6 max_remote_readers

NAME

[DDS_DataWriterResourceLimitsQosPolicy::max_remote_readers](#)

DESCRIPTION

[DDS_DataWriterResourceLimitsQosPolicy::max_remote_readers](#) limits the maximum number of DataReaders the [DDSDataWriter](#) can maintain state for.

6.19.8 OSAPI

Topics

- [max_buffer_size](#)

6.19.8.1 Detailed Description

6.19.8.2 max_buffer_size

NAME

[OSAPI_LogProperty::max_buffer_size](#)

DESCRIPTION

This resource-limit limits the amount logging information that can be saved in bytes. When the log-buffer is full no more logging information is saved.

6.19.9 UDP Transport

Topics

- [max_message_size](#)

6.19.9.1 Detailed Description

6.19.9.2 max_message_size

NAME

[UDP_InterfaceFactoryProperty::max_message_size](#)

DESCRIPTION

[UDP_InterfaceFactoryProperty::max_message_size](#) limits the size of the maximum message that can be received.

NOTES

RTI Connext DDS Micro supports RTPS fragmentation. Fragmentation will be applied when sending samples larger than the `max_message_size`.

The UDP transport allocates one buffer for each resource in [DDS_DomainParticipantResourceLimitsQosPolicy::max_receive_ports](#). Thus, the total amount of memory is $(\text{max_message_size} * \text{max_receive_ports})$

6.19.10 Dynamic Participant Static Endpoint (DPSE)

Topics

- [max_participant_locators](#)
- [max_locators_per_discovered_participant](#)

6.19.10.1 Detailed Description

The Dynamic Participant Static Endpoint (DPSE) discovery plugin creates one `DataWriter` and one `DataReader` for the `ParticipantBuiltinTopicData`. The memory for the plugin includes the memory allocated by the `DataReader` and `DataWriter`. The memory allocated by the properties must be added for total memory usage.

6.19.10.2 max_participant_locators

NAME

[DPSE_DiscoveryPluginProperty::max_participant_locators](#)

DESCRIPTION

The DPSE builtin participant topic `DataWriter` allocates `max_participant_locators` to send participant announcements to. The default value `DDS_LENGTH_AUTO` calculates `max_participant_locators` as `remote_participant_allocation + (DPSE_MAX_ANON_PARTICIPANT * (peer_length + 1))` where the constant `DPSE_MAX_ANON_PARTICIPANT = 6`.

A participant locator is defined as a unique `index@address` to send to where `index` is an integer ≥ 0 and only applies to unicast addresses. For multicast address the `index` is 0. This `index` is the highest `participant_id` that will receive participant announcements at the address.

The maximum number of addresses to send to is defined by the peer list (either specified in [DDS_DiscoveryQosPolicy::initial_peers](#) or by calling [DDSDomainParticipant::add_peer](#)). The maximum number of addresses needed is determined by adding all indices for all unicast addresses and adding one for each multicast address.

If memory is limited it is advised to set this value as low as possible.

6.19.10.3 max_locators_per_discovered_participant

NAME

[DPSE_DiscoveryPluginProperty::max_locators_per_discovered_participant](#)

DESCRIPTION

When a participant discovers another participant it must keep track of which addresses to respond to for each participant. This resource-limit sets the maximum number of addresses which can be saved.

NOTES

If two participants respond with the same addresses, for example multicast, only one address is saved.

6.19.11 Dynamic Participant Dynamic Endpoint (DPDE)

Topics

- [max_participant_locators](#)
- [max_locators_per_discovered_participant](#)
- [max_samples_per_builtin_endpoint_reader](#)

6.19.11.1 Detailed Description

The Dynamic Participant Dynamic Endpoint (DPSE) discovery plugin creates one DataWriter and one DataReader for each of the ParticipantBuiltinTopicData, PublicationBuiltinTopicData, and SubscriptionBuiltinTopicData. The memory for the plugin includes the memory allocated by these DataReaders and datawriters. The memory allocated by the properties must be added for total memory usage.

6.19.11.2 max_participant_locators

NAME

[DPDE_DiscoveryPluginProperty::max_participant_locators](#)

DESCRIPTION

The DPDE builtin participant topic DataWriter allocates max_participant_locators to send participant announcements to. The default value DDS_LENGTH_AUTO calculates max_participant_locators as remote_participant_allocation + (DPDE_MAX_ANON_PARTICIPANT * (peer_length + 1)) where the constant DPDE_MAX_ANON_PARTICIPANT = 6.

A participant locator is defined as a unique index@address to send to where index is an integer ≥ 0 and only applies to unicast addresses. For multicast address the index is 0. This index is the highest participant_id that will receive participant announcements at the address.

The maximum number of addresses to send to is defined by the peer list (either specified in [DDS_DiscoveryQosPolicy::initial_peers](#) or by calling [DDSDomainParticipant::add_peer](#). The maximum number of addresses needed is determined by adding all indices for all unicast addresses and adding one for each multicast address.

If memory is limited it is advised to set this value as low as possible.

6.19.11.3 max_locators_per_discovered_participant**NAME**

[DPDE_DiscoveryPluginProperty::max_locators_per_discovered_participant](#)

DESCRIPTION

When a participant discovers another participant it must keep track of which addresses to respond to for each participant. This resource-limit sets the maximum number of addresses which can be saved.

NOTES

If two participants respond with the same addresses, for example multicast, only one address is saved.

NAME

[DPDE_DiscoveryPluginProperty::max_locators_per_discovered_participant](#)

DESCRIPTION

When a participant discovers another participant it must keep track of which addresses to respond to for each participant. This resource-limit sets the maximum number of addresses which can be saved.

NOTES

If two participants respond with the same addresses, for example multicast, only one address is saved.

6.19.11.4 max_samples_per_builtin_endpoint_reader**NAME**

[DPDE_DiscoveryPluginProperty::max_samples_per_builtin_endpoint_reader](#)

DESCRIPTION

The builtin endpoint DataReaders are reliable and may cache samples received out-of-order to improve discovery performance. This resource-limit determines the maximum number of outstanding samples at any given time.

6.20 Log Codes

Topics

- [OSAPI](#)
OSAPI. ModuleID = 0.
- [REDA](#)
Real-time Efficient Data Structures and Algorithms. ModuleID = 1.
- [DB](#)
Database. ModuleID = 2.
- [RT](#)
RT. ModuleID = 3.
- [CDR](#)
CDR. ModuleID = 5.
- [NETIO](#)
Network I/O. ModuleID = 4.
- [RTPS](#)
Real-Time Publish-Subscribe. ModuleID = 6.
- [DDS_C](#)
DDS C. ModuleID = 7.
- [ZeroCopy](#)
Zero Copy V2. ModuleID = 14.
- [WHSQ](#)
Writer History Shared Queue. ModuleID = 22.
- [RH](#)
Reader History. ModuleID = 8.
- [WH](#)
Writer History. ModuleID = 9.
- [DPSE](#)
DPSE. ModuleID = 10.
- [DPDE](#)
DPDE. ModuleID = 11.
- [APPGEN](#)
APPGEN. ModuleID = 13.
- [DDS_FILTER](#)
DDS Filter. ModuleID = 15.

6.20.1 Detailed Description

This page contains references to all log codes reported by RTI Connex DDS Micro. Please refer to the [Debugging](#) section in the [User's Manual](#) for how to interpret log messages and [Log API](#) for the Log API.

6.20.2 OSAPI

OSAPI. ModuleID = 0.

Macros

- `#define OSAPI_LOG_GET_NEXT_OBJECT_ID_EC (OSAPI_LOG_BASE + 1)`
Retrieve the next error code failed.
- `#define OSAPI_LOG_SYSTEM_SET_PROPERTY_EC (OSAPI_LOG_BASE + 2)`
An error occurred while setting the system properties.
- `#define OSAPI_LOG_SYSTEM_TIMER_START_EC (OSAPI_LOG_BASE + 4)`
An error occurred when starting the system timer.
- `#define OSAPI_LOG_SYSTEM_TIMER_STOP_EC (OSAPI_LOG_BASE + 5)`
An error occurred when stopping the system timer.
- `#define OSAPI_LOG_THREAD_NEW_EC (OSAPI_LOG_BASE + 6)`
An error when allocating the a thread object.
- `#define OSAPI_LOG_THREAD_CREATE_EC (OSAPI_LOG_BASE + 7)`
An error when creating the thread object.
- `#define OSAPI_LOG_THREAD_SEM_EC (OSAPI_LOG_BASE + 8)`
An error when creating thread sync semaphore.
- `#define OSAPI_LOG_THREAD_EXEC_CREATE_EC (OSAPI_LOG_BASE + 9)`
Failed to signal that a thread has been created.
- `#define OSAPI_LOG_THREAD_EXEC_START_EC (OSAPI_LOG_BASE + 10)`
Failed to signal the start a created thread.
- `#define OSAPI_LOG_THREAD_START_EC (OSAPI_LOG_BASE + 11)`
Failed to start a thread.
- `#define OSAPI_LOG_THREAD_DESTROY_EC (OSAPI_LOG_BASE + 12)`
Failed to destroy a thread.
- `#define OSAPI_LOG_THREAD_DESTROY_NO_START_EC (OSAPI_LOG_BASE + 13)`
Failed to start an unstarted thread being destroyed.
- `#define OSAPI_LOG_THREAD_DESTROY_NO_WAKEUP_EC (OSAPI_LOG_BASE + 14)`
Failed to wake up a thread that's being destroyed.
- `#define OSAPI_LOG_THREAD_INIT_EC (OSAPI_LOG_BASE + 15)`
Failed initializing a thread.
- `#define OSAPI_LOG_THREAD_SCHEDPARAM_EC (OSAPI_LOG_BASE + 16)`
Failed to set scheduling policy of a thread.
- `#define OSAPI_LOG_THREAD_GET_POLICY_EC (OSAPI_LOG_BASE + 17)`
Failed to get the scheduling policy of a thread.
- `#define OSAPI_LOG_THREAD_POLICY_DIFFER_EC (OSAPI_LOG_BASE + 18)`
Scheduling policy mismatch between a created thread and the application thread.
- `#define OSAPI_LOG_THREAD_PRIORITY_MAP_EC (OSAPI_LOG_BASE + 19)`
Failed to map to native thread priority values.
- `#define OSAPI_LOG_TIMER_DELETE_EC (OSAPI_LOG_BASE + 20)`
Failed to delete the Timer object.
- `#define OSAPI_LOG_TIMER_TICK_MUTEX_EC (OSAPI_LOG_BASE + 22)`
Failed taking or giving the Timer mutex.
- `#define OSAPI_LOG_TIMER_GET_USER_DATA_EPOCH_EC (OSAPI_LOG_BASE + 27)`
Failed to return user data for a timeout due to mismatched epochs.
- `#define OSAPI_LOG_TIMER_NEW_EC (OSAPI_LOG_BASE + 28)`
Failed to allocate memory for a new Timer.
- `#define OSAPI_LOG_TIMER_NEW_ENTRY_EC (OSAPI_LOG_BASE + 29)`

- Failed to allocate memory for a new Timer entry.*

 - #define `OSAPI_LOG_TIMER_NEW_WHEEL_EC` (`OSAPI_LOG_BASE` + 30)
- Failed to allocate memory for a new Timer wheel.*

 - #define `OSAPI_LOG_TIMER_NEW_MUTEX_EC` (`OSAPI_LOG_BASE` + 31)
- Failed to create a new Timer mutex.*

 - #define `OSAPI_LOG_TIMER_NEW_START_TIMER_EC` (`OSAPI_LOG_BASE` + 32)
- Failed to start a new Timer being created.*

 - #define `OSAPI_LOG_TIMER_DELETE_STOP_TIMER_EC` (`OSAPI_LOG_BASE` + 33)
- Failed to stop a Timer being deleted.*

 - #define `OSAPI_LOG_TIMER_DELETE_MUTEX_EC` (`OSAPI_LOG_BASE` + 34)
- Failed to delete the Timer mutex.*

 - #define `OSAPI_LOG_TIMER_MUTEX_EC` (`OSAPI_LOG_BASE` + 35)
- Failed to take or give a Timer mutex.*

 - #define `OSAPI_LOG_SEMAPHORE_DELETE_EC` (`OSAPI_LOG_BASE` + 36)
- Failed to delete a semaphore.*

 - #define `OSAPI_LOG_SEMAPHORE_NEW_EC` (`OSAPI_LOG_BASE` + 37)
- Failed to create a semaphore.*

 - #define `OSAPI_LOG_SEMAPHORE_NEW_INIT_EC` (`OSAPI_LOG_BASE` + 38)
- Failed to initialize a new semaphore.*

 - #define `OSAPI_LOG_SEMAPHORE_GIVE_EC` (`OSAPI_LOG_BASE` + 39)
- Failed to give a semaphore.*

 - #define `OSAPI_LOG_SEMAPHORE_TAKE_EC` (`OSAPI_LOG_BASE` + 40)
- Failed to take a semaphore.*

 - #define `OSAPI_LOG_MUTEX_DELETE_EC` (`OSAPI_LOG_BASE` + 41)
- Failed to delete a mutex.*

 - #define `OSAPI_LOG_MUTEX_NEW_EC` (`OSAPI_LOG_BASE` + 42)
- Failed to create a mutex.*

 - #define `OSAPI_LOG_MUTEX_TAKE_EC` (`OSAPI_LOG_BASE` + 43)
- Failed to take a mutex.*

 - #define `OSAPI_LOG_MUTEX_GIVE_EC` (`OSAPI_LOG_BASE` + 44)
- Failed to give a mutex.*

 - #define `OSAPI_LOG_MUTEX_INIT_EC` (`OSAPI_LOG_BASE` + 45)
- Failed to initialize a mutex.*

 - #define `OSAPI_LOG_HEAP_INTERNAL_ALLOCATE_EC` (`OSAPI_LOG_BASE` + 46)
- Failed to allocate a buffer from the heap.*

 - #define `OSAPI_LOG_SYSTEM_GET_TIME_EC` (`OSAPI_LOG_BASE` + 48)
- Failed to get current system time.*

 - #define `OSAPI_LOG_LAST_RECORDED_ERROR_EC` (`OSAPI_LOG_BASE` + 49)
- Return the last recorded error-code for the calling thread.*

 - #define `OSAPI_LOG_SET_THREAD_NAME_EC` (`OSAPI_LOG_BASE` + 50)
- Failed to set the thread name in the OS.*

 - #define `OSAPI_LOG_SEM_OPEN_EC` (`OSAPI_LOG_BASE` + 52)
- Failure during call VxWorks semOpen(...) or Posix sem_open(...) when managing shared memory segments, semaphores, or mutexes.*

 - #define `OSAPI_LOG_SEM_INFO_GET_EC` (`OSAPI_LOG_BASE` + 53)
- Failure during VxWorks call semInfoGet when managing shared memory segments, semaphores, or mutexes.*

 - #define `OSAPI_LOG_SD_UNMAP_EC` (`OSAPI_LOG_BASE` + 54)

- Failure during VxWorks call sdUnmap when managing shared memory segments, semaphores, or mutexes.*

 - #define `OSAPI_LOG_SD_OPEN_EC` (`OSAPI_LOG_BASE` + 55)
- Failure during VxWorks call sdOpen when managing shared memory segments, semaphores, or mutexes.*

 - #define `OSAPI_LOG_SEM_GIVE_EC` (`OSAPI_LOG_BASE` + 56)
- Failure during VxWorks call semGive when managing shared memory segments, semaphores, or mutexes.*

 - #define `OSAPI_LOG_SEM_TAKE_EC` (`OSAPI_LOG_BASE` + 57)
- Failure during VxWorks call semTake when managing shared memory segments, semaphores, or mutexes.*

 - #define `OSAPI_LOG_UNKNOWN_SEMAPHORE_TYPE_EC` (`OSAPI_LOG_BASE` + 58)
- Trying to perform an operation an unknown semmutex type.*

 - #define `OSAPI_LOG_SEGMENT_KEY_ALREADY_EXISTS_EC` (`OSAPI_LOG_BASE` + 59)
- Failed to create segment due to a segment already existing.*

 - #define `OSAPI_LOG_SEGMENT_KEY_DOESNT_EXIST_EC` (`OSAPI_LOG_BASE` + 60)
- Segment trying to be attached to does not exist.*

 - #define `OSAPI_LOG_SEGMENT_DETACH_FAILED_EC` (`OSAPI_LOG_BASE` + 61)
- Failed to detach from shared memory segment.*

 - #define `OSAPI_LOG_SHMEM_ATTACH_FAILED_EC` (`OSAPI_LOG_BASE` + 62)
- Attaching to shared memory segment/mutex/signaling semaphore failed.*

 - #define `OSAPI_LOG_SHMEM_GIVE_FAILED_EC` (`OSAPI_LOG_BASE` + 63)
- Failed to give shared memory mutex/signaling semaphore.*

 - #define `OSAPI_LOG_SHMEM_TAKE_FAILED_EC` (`OSAPI_LOG_BASE` + 64)
- Failed to take shared memory mutex/signaling semaphore.*

 - #define `OSAPI_LOG_SHMEM_SEM_DELETE_FAILED_EC` (`OSAPI_LOG_BASE` + 65)
- Failed to delete shared memory semaphore or mutex.*

 - #define `OSAPI_LOG_SHMEM_SEM_MUTEX_CREATED_EC` (`OSAPI_LOG_BASE` + 66)
- Error during shared memory semaphore creation/deletion.*

 - #define `OSAPI_LOG_SHMEM_SEGMENT_FILE_MAP_FAILED_EC` (`OSAPI_LOG_BASE` + 67)
- Windows specific API MapViewOfFile failed.*

 - #define `OSAPI_LOG_SHMEM_SET_DESCRIPTOR_EC` (`OSAPI_LOG_BASE` + 68)
- Windows specific error during setting security descriptor.*

 - #define `OSAPI_LOG_SHMEM_INIT_DESCRIPTOR_EC` (`OSAPI_LOG_BASE` + 69)
- Windows specific error during initialization of security descriptor.*

 - #define `OSAPI_LOG_SHMEM_FTRUNCATE_EC` (`OSAPI_LOG_BASE` + 70)
- Posix specific error during ftruncate(...) call.*

 - #define `OSAPI_LOG_SHMEM_MMAP_EC` (`OSAPI_LOG_BASE` + 71)
- Posix specific error during mmap(...) call.*

 - #define `OSAPI_LOG_SHMEM_SHM_OPEN_EC` (`OSAPI_LOG_BASE` + 72)
- Posix specific error during shm_open(...) call.*

 - #define `OSAPI_LOG_SHMEM_MUNMAP_EC` (`OSAPI_LOG_BASE` + 73)
- Posix specific error during munmap(...) call.*

 - #define `OSAPI_LOG_CLOSE_EC` (`OSAPI_LOG_BASE` + 74)
- Posix specific error during close(...) call.*

 - #define `OSAPI_LOG_SHMEM_UNLINK_EC` (`OSAPI_LOG_BASE` + 76)
- Posix specific error during shared memory unlink(...) call.*

 - #define `OSAPI_LOG_EXHAUSTED_POSIX_SEM_LIMIT_EC` (`OSAPI_LOG_BASE` + 77)
- System ran out of space to create semaphores. Consider increasing Posix sem limit.*

 - #define `OSAPI_LOG_SHMEM_SEM_WAIT_EC` (`OSAPI_LOG_BASE` + 78)
- Posix specific error during shared memory wait call.*

- #define `OSAPI_LOG_SHMEM_SEM_UNLINK_EC` (`OSAPI_LOG_BASE` + 79)
Posix specific error during shared memory sem unlink call.
- #define `OSAPI_LOG_SHMEM_POSIX_GIVE_EC` (`OSAPI_LOG_BASE` + 80)
Posix specific error during os specific implementation of a semaphore mutex give.
- #define `OSAPI_LOG_THREAD_SEMPOOL_EXCEEDED_EC` (`OSAPI_LOG_BASE` + 90)
Failed to add a semaphore to the thread semaphore pool.
- #define `OSAPI_LOG_THREAD_SEMPOOL_INVALID_SEM_EC` (`OSAPI_LOG_BASE` + 91)
Failed to delete a semaphore from the thread semaphore pool.
- #define `OSAPI_LOG_SHMEM_SIZE_EXCEEDED_EC` (`OSAPI_LOG_BASE` + 92)
Trying to create a shared memory segment that is greater than $(2^{31}) - 1$. The middleware cannot support shared memory segments created than ~ 2 GB even though the operating system may.
- #define `OSAPI_LOG_THREAD_JOIN_EC` (`OSAPI_LOG_BASE` + 95)
Failed to join a thread.
- #define `OSAPI_LOG_THREAD_GET_MAX_PRIORITY_EC` (`OSAPI_LOG_BASE` + 96)
Failed to get the maximum priority value for a scheduling policy.
- #define `OSAPI_LOG_SHMEM_FSTAT_EC` (`OSAPI_LOG_BASE` + 97)
POSIX specific error during fstat(...) call.
- #define `OSAPI_LOG_SHMEM_MUTEX_INIT_TIMEOUT_EC` (`OSAPI_LOG_BASE` + 97)
Timed out waiting for a shared memory mutex to be initialized.
- #define `OSAPI_LOG_AUTOSAR_SETEVENT_CALL_EC` (`OSAPI_LOG_BASE` + 100)
Autosar SetEvent system call failed.
- #define `OSAPI_LOG_AUTOSAR_CLEAREVENT_CALL_EC` (`OSAPI_LOG_BASE` + 101)
Autosar ClearEvent system call failed.
- #define `OSAPI_LOG_AUTOSAR_WAITEVENT_CALL_EC` (`OSAPI_LOG_BASE` + 102)
Autosar WaitEvent system call failed.
- #define `OSAPI_LOG_AUTOSAR_GETEVENT_CALL_EC` (`OSAPI_LOG_BASE` + 103)
Autosar GetEvent system call failed.
- #define `OSAPI_LOG_AUTOSAR_TERMINATETASK_CALL_EC` (`OSAPI_LOG_BASE` + 104)
Autosar TerminateTask system call failed.
- #define `OSAPI_LOG_AUTOSAR_ACTIVATETASK_CALL_EC` (`OSAPI_LOG_BASE` + 105)
Autosar ActivateTask system call failed.
- #define `OSAPI_LOG_AUTOSAR_SCHEDULE_CALL_EC` (`OSAPI_LOG_BASE` + 106)
Autosar Schedule system call failed.
- #define `OSAPI_LOG_AUTOSAR_SETRELALARM_CALL_EC` (`OSAPI_LOG_BASE` + 107)
Autosar SetRelAlarm system call failed.
- #define `OSAPI_LOG_AUTOSAR_CANCELALARM_CALL_EC` (`OSAPI_LOG_BASE` + 108)
Autosar CancelAlarm system call failed.
- #define `OSAPI_LOG_SEGMENT_ALREADY_EXISTS_EC` (`OSAPI_LOG_BASE` + 150)
Shared memory segment already exists.
- #define `OSAPI_LOG_SEGMENT_DOESNT_EXIST_EC` (`OSAPI_LOG_BASE` + 151)
Shared memory segment does not exist.
- #define `OSAPI_LOG_SHMEM_INVALID_SEGMENT_NAME_EC` (`OSAPI_LOG_BASE` + 152)
Invalid shared memory segment name.
- #define `OSAPI_LOG_SHMEM_INVALID_SEGMENT_MODE_EC` (`OSAPI_LOG_BASE` + 153)
Invalid shared memory segment mode.
- #define `OSAPI_LOG_SHMEM_SEGMENT_NOT_LOCKABLE_EC` (`OSAPI_LOG_BASE` + 154)
Shared memory segment does not support locking.

- #define `OSAPI_LOG_SHMEM_SEGMENT_NOT_ROBUST_EC` (`OSAPI_LOG_BASE` + 155)
Shared memory segment does not support robust mutexes.
- #define `OSAPI_LOG_SHMEM_ALREADY_ATTACHED_EC` (`OSAPI_LOG_BASE` + 156)
Shared memory segment is already attached.
- #define `OSAPI_LOG_SHMEM_NOT_ATTACHED_EC` (`OSAPI_LOG_BASE` + 157)
Shared memory segment is not attached.
- #define `OSAPI_LOG_SHMEM_CHECK_UNLINKED_EC` (`OSAPI_LOG_BASE` + 158)
Checking if shared memory segment has been unlinked.
- #define `OSAPI_SHMEM_SEGMENT_NOT_INITIALIZED_EC` (`OSAPI_LOG_BASE` + 159)
Shared memory segment is not initialized.
- #define `OSAPI_LOG_SHMEM_SEGMENT_NAME_ALREADY_EXISTS_EC` (`OSAPI_LOG_BASE` + 160)
Shared memory segment with specified name already exists.
- #define `OSAPI_LOG_SHMEM_SEGMENT_NAME_DOESNT_EXIST_EC` (`OSAPI_LOG_BASE` + 161)
Shared memory segment with specified name does not exist.
- #define `OSAPI_LOG_PTHREAD_MUTEXATTR_INIT_EC` (`OSAPI_LOG_BASE` + 200)
Failed to create a pthread mutex attribute.
- #define `OSAPI_LOG_PTHREAD_MUTEXATTR_SETPSHARED_EC` (`OSAPI_LOG_BASE` + 201)
Failed to set the pthread mutex attribute to be process shared.
- #define `OSAPI_LOG_PTHREAD_MUTEXATTR_SETROBUST_EC` (`OSAPI_LOG_BASE` + 202)
Failed to set the pthread mutex attribute to be robust.
- #define `OSAPI_LOG_PTHREAD_MUTEXATTR_DESTROY_EC` (`OSAPI_LOG_BASE` + 203)
Failed to destroy the pthread mutex attribute.
- #define `OSAPI_LOG_PTHREAD_MUTEX_INIT_EC` (`OSAPI_LOG_BASE` + 204)
Failed to initialize a pthread mutex.
- #define `OSAPI_LOG_PTHREAD_MUTEX_DESTROY_EC` (`OSAPI_LOG_BASE` + 205)
Failed to destroy a pthread mutex.
- #define `OSAPI_LOG_PTHREAD_MUTEX_LOCK_EC` (`OSAPI_LOG_BASE` + 206)
Failed to lock a pthread mutex.
- #define `OSAPI_LOG_PTHREAD_MUTEX_UNLOCK_EC` (`OSAPI_LOG_BASE` + 207)
Failed to unlock a pthread mutex.
- #define `OSAPI_LOG_PTHREAD_MUTEX_CONSISTENT_EC` (`OSAPI_LOG_BASE` + 208)
Failed to set the pthread mutex to be consistent.
- #define `OSAPI_LOG_PTHREAD_CONDATTR_INIT_EC` (`OSAPI_LOG_BASE` + 209)
Failed to create a pthread condition variable attribute.
- #define `OSAPI_LOG_PTHREAD_CONDATTR_SETPSHARED_EC` (`OSAPI_LOG_BASE` + 210)
Failed to set the pthread condition variable attribute to be process shared.
- #define `OSAPI_LOG_PTHREAD_CONDATTR_DESTROY_EC` (`OSAPI_LOG_BASE` + 211)
Failed to destroy the pthread condition variable attribute.
- #define `OSAPI_LOG_PTHREAD_COND_INIT_EC` (`OSAPI_LOG_BASE` + 212)
Failed to initialize a pthread condition variable.
- #define `OSAPI_LOG_PTHREAD_COND_DESTROY_EC` (`OSAPI_LOG_BASE` + 213)
Failed to destroy a pthread condition variable.
- #define `OSAPI_LOG_PTHREAD_COND_WAIT_EC` (`OSAPI_LOG_BASE` + 214)
Failed to wait on a pthread condition variable.
- #define `OSAPI_LOG_PTHREAD_COND_SIGNAL_EC` (`OSAPI_LOG_BASE` + 215)
Failed to signal a pthread condition variable.
- #define `OSAPI_LOG_PTHREAD_MUTEXATTR_SETTYPE_EC` (`OSAPI_LOG_BASE` + 216)

Failed to set the type of a pthread mutex attribute.

- #define `OSAPI_LOG_PTHREAD_MUTEXATTR_SETPROTOCOL_EC` (`OSAPI_LOG_BASE` + 217)

Failed to set the protocol of a pthread mutex attribute.

- #define `OSAPI_LOG_PTHREAD_MUTEXATTR_SETPRIOCEILING_EC` (`OSAPI_LOG_BASE` + 218)

Failed to set the priority ceiling of a pthread mutex attribute.

- #define `OSAPI_LOG_WIN_WAIT_FAILED_EC` (`OSAPI_LOG_BASE` + 300)

WaitForSingleObject returned an error while waiting for the timer semaphore to time out. Micro continues to run, but it may indicate a platform problem.

6.20.2.1 Detailed Description

OSAPI. ModuleID = 0.

6.20.2.2 Macro Definition Documentation

OSAPI_LOG_GET_NEXT_OBJECT_ID_EC

```
#define OSAPI_LOG_GET_NEXT_OBJECT_ID_EC (OSAPI_LOG_BASE + 1)
```

Retrieve the next error code failed.

OSAPI_LOG_SYSTEM_SET_PROPERTY_EC

```
#define OSAPI_LOG_SYSTEM_SET_PROPERTY_EC (OSAPI_LOG_BASE + 2)
```

An error occurred while setting the system properties.

OSAPI_LOG_SYSTEM_TIMER_START_EC

```
#define OSAPI_LOG_SYSTEM_TIMER_START_EC (OSAPI_LOG_BASE + 4)
```

An error occurred when starting the system timer.

OSAPI_LOG_SYSTEM_TIMER_STOP_EC

```
#define OSAPI_LOG_SYSTEM_TIMER_STOP_EC (OSAPI_LOG_BASE + 5)
```

An error occurred when stopping the system timer.

OSAPI_LOG_THREAD_NEW_EC

```
#define OSAPI_LOG_THREAD_NEW_EC (OSAPI_LOG_BASE + 6)
```

An error when allocating the a thread object.

OSAPI_LOG_THREAD_CREATE_EC

```
#define OSAPI_LOG_THREAD_CREATE_EC (OSAPI_LOG_BASE + 7)
```

An error when creating the thread object.

OSAPI_LOG_THREAD_SEM_EC

```
#define OSAPI_LOG_THREAD_SEM_EC (OSAPI_LOG_BASE + 8)
```

An error when creating thread sync semaphore.

OSAPI_LOG_THREAD_EXEC_CREATE_EC

```
#define OSAPI_LOG_THREAD_EXEC_CREATE_EC (OSAPI_LOG_BASE + 9)
```

Failed to signal that a thread has been created.

OSAPI_LOG_THREAD_EXEC_START_EC

```
#define OSAPI_LOG_THREAD_EXEC_START_EC (OSAPI_LOG_BASE + 10)
```

Failed to signal the start a created thread.

OSAPI_LOG_THREAD_START_EC

```
#define OSAPI_LOG_THREAD_START_EC (OSAPI_LOG_BASE + 11)
```

Failed to start a thread.

OSAPI_LOG_THREAD_DESTROY_EC

```
#define OSAPI_LOG_THREAD_DESTROY_EC (OSAPI_LOG_BASE + 12)
```

Failed to destroy a thread.

OSAPI_LOG_THREAD_DESTROY_NO_START_EC

```
#define OSAPI_LOG_THREAD_DESTROY_NO_START_EC (OSAPI_LOG_BASE + 13)
```

Failed to start an unstarted thread being destroyed.

OSAPI_LOG_THREAD_DESTROY_NO_WAKEUP_EC

```
#define OSAPI_LOG_THREAD_DESTROY_NO_WAKEUP_EC (OSAPI_LOG_BASE + 14)
```

Failed to wake up a thread that's being destroyed.

OSAPI_LOG_THREAD_INIT_EC

```
#define OSAPI_LOG_THREAD_INIT_EC (OSAPI_LOG_BASE + 15)
```

Failed initializing a thread.

OSAPI_LOG_THREAD_SCHEDPARAM_EC

```
#define OSAPI_LOG_THREAD_SCHEDPARAM_EC (OSAPI_LOG_BASE + 16)
```

Failed to set scheduling policy of a thread.

OSAPI_LOG_THREAD_GET_POLICY_EC

```
#define OSAPI_LOG_THREAD_GET_POLICY_EC (OSAPI_LOG_BASE + 17)
```

Failed to get the scheduling policy of a thread.

OSAPI_LOG_THREAD_POLICY_DIFFER_EC

```
#define OSAPI_LOG_THREAD_POLICY_DIFFER_EC (OSAPI_LOG_BASE + 18)
```

Scheduling policy mismatch between a created thread and the application thread.

OSAPI_LOG_THREAD_PRIORITY_MAP_EC

```
#define OSAPI_LOG_THREAD_PRIORITY_MAP_EC (OSAPI_LOG_BASE + 19)
```

Failed to map to native thread priority values.

OSAPI_LOG_TIMER_DELETE_EC

```
#define OSAPI_LOG_TIMER_DELETE_EC (OSAPI_LOG_BASE + 20)
```

Failed to delete the Timer object.

OSAPI_LOG_TIMER_TICK_MUTEX_EC

```
#define OSAPI_LOG_TIMER_TICK_MUTEX_EC (OSAPI_LOG_BASE + 22)
```

Failed taking or giving the Timer mutex.

OSAPI_LOG_TIMER_GET_USER_DATA_EPOCH_EC

```
#define OSAPI_LOG_TIMER_GET_USER_DATA_EPOCH_EC (OSAPI_LOG_BASE + 27)
```

Failed to return user data for a timeout due to mismatched epochs.

OSAPI_LOG_TIMER_NEW_EC

```
#define OSAPI_LOG_TIMER_NEW_EC (OSAPI_LOG_BASE + 28)
```

Failed to allocate memory for a new Timer.

OSAPI_LOG_TIMER_NEW_ENTRY_EC

```
#define OSAPI_LOG_TIMER_NEW_ENTRY_EC (OSAPI_LOG_BASE + 29)
```

Failed to allocate memory for a new Timer entry.

OSAPI_LOG_TIMER_NEW_WHEEL_EC

```
#define OSAPI_LOG_TIMER_NEW_WHEEL_EC (OSAPI_LOG_BASE + 30)
```

Failed to allocate memory for a new Timer wheel.

OSAPI_LOG_TIMER_NEW_MUTEX_EC

```
#define OSAPI_LOG_TIMER_NEW_MUTEX_EC (OSAPI_LOG_BASE + 31)
```

Failed to create a new Timer mutex.

OSAPI_LOG_TIMER_NEW_START_TIMER_EC

```
#define OSAPI_LOG_TIMER_NEW_START_TIMER_EC (OSAPI_LOG_BASE + 32)
```

Failed to start a new Timer being created.

OSAPI_LOG_TIMER_DELETE_STOP_TIMER_EC

```
#define OSAPI_LOG_TIMER_DELETE_STOP_TIMER_EC (OSAPI_LOG_BASE + 33)
```

Failed to stop a Timer being deleted.

OSAPI_LOG_TIMER_DELETE_MUTEX_EC

```
#define OSAPI_LOG_TIMER_DELETE_MUTEX_EC (OSAPI_LOG_BASE + 34)
```

Failed to delete the Timer mutex.

OSAPI_LOG_TIMER_MUTEX_EC

```
#define OSAPI_LOG_TIMER_MUTEX_EC (OSAPI_LOG_BASE + 35)
```

Failed to take or give a Timer mutex.

OSAPI_LOG_SEMAPHORE_DELETE_EC

```
#define OSAPI_LOG_SEMAPHORE_DELETE_EC (OSAPI_LOG_BASE + 36)
```

Failed to delete a semaphore.

OSAPI_LOG_SEMAPHORE_NEW_EC

```
#define OSAPI_LOG_SEMAPHORE_NEW_EC (OSAPI_LOG_BASE + 37)
```

Failed to create a semaphore.

OSAPI_LOG_SEMAPHORE_NEW_INIT_EC

```
#define OSAPI_LOG_SEMAPHORE_NEW_INIT_EC (OSAPI_LOG_BASE + 38)
```

Failed to initialize a new semaphore.

OSAPI_LOG_SEMAPHORE_GIVE_EC

```
#define OSAPI_LOG_SEMAPHORE_GIVE_EC (OSAPI_LOG_BASE + 39)
```

Failed to give a semaphore.

OSAPI_LOG_SEMAPHORE_TAKE_EC

```
#define OSAPI_LOG_SEMAPHORE_TAKE_EC (OSAPI_LOG_BASE + 40)
```

Failed to take a semaphore.

OSAPI_LOG_MUTEX_DELETE_EC

```
#define OSAPI_LOG_MUTEX_DELETE_EC (OSAPI_LOG_BASE + 41)
```

Failed to delete a mutex.

OSAPI_LOG_MUTEX_NEW_EC

```
#define OSAPI_LOG_MUTEX_NEW_EC (OSAPI_LOG_BASE + 42)
```

Failed to create a mutex.

OSAPI_LOG_MUTEX_TAKE_EC

```
#define OSAPI_LOG_MUTEX_TAKE_EC (OSAPI_LOG_BASE + 43)
```

Failed to take a mutex.

OSAPI_LOG_MUTEX_GIVE_EC

```
#define OSAPI_LOG_MUTEX_GIVE_EC (OSAPI_LOG_BASE + 44)
```

Failed to give a mutex.

OSAPI_LOG_MUTEX_INIT_EC

```
#define OSAPI_LOG_MUTEX_INIT_EC (OSAPI_LOG_BASE + 45)
```

Failed to initialize a mutex.

OSAPI_LOG_HEAP_INTERNAL_ALLOCATE_EC

```
#define OSAPI_LOG_HEAP_INTERNAL_ALLOCATE_EC (OSAPI_LOG_BASE + 46)
```

Failed to allocate a buffer from the heap.

OSAPI_LOG_SYSTEM_GET_TIME_EC

```
#define OSAPI_LOG_SYSTEM_GET_TIME_EC (OSAPI_LOG_BASE + 48)
```

Failed to get current system time.

OSAPI_LOG_LAST_RECORDED_ERROR_EC

```
#define OSAPI_LOG_LAST_RECORDED_ERROR_EC (OSAPI_LOG_BASE + 49)
```

Return the last recorded error-code for the calling thread.

This log-messages retrieves the last recorded error-code for the calling thread. It is used a function calls another function that fails.

OSAPI_LOG_SET_THREAD_NAME_EC

```
#define OSAPI_LOG_SET_THREAD_NAME_EC (OSAPI_LOG_BASE + 50)
```

Failed to set the thread name in the OS.

When OSAPI starts a thread it also calls the OS to set the thread name. If this fails this warning message indicates why.

OSAPI_LOG_SEM_OPEN_EC

```
#define OSAPI_LOG_SEM_OPEN_EC (OSAPI_LOG_BASE + 52)
```

Failure during call VxWorks semOpen(...) or Posix sem_open(...) when managing shared memory segments, semaphores, or mutexes.

OSAPI_LOG_SEM_INFO_GET_EC

```
#define OSAPI_LOG_SEM_INFO_GET_EC (OSAPI_LOG_BASE + 53)
```

Failure during VxWorks call semInfoGet when managing shared memory segments, semaphores, or mutexes.

OSAPI_LOG_SD_UNMAP_EC

```
#define OSAPI_LOG_SD_UNMAP_EC (OSAPI_LOG_BASE + 54)
```

Failure during VxWorks call sdUnmap when managing shared memory segments, semaphores, or mutexes.

OSAPI_LOG_SD_OPEN_EC

```
#define OSAPI_LOG_SD_OPEN_EC (OSAPI_LOG_BASE + 55)
```

Failure during VxWorks call sdOpen when managing shared memory segments, semaphores, or mutexes.

OSAPI_LOG_SEM_GIVE_EC

```
#define OSAPI_LOG_SEM_GIVE_EC (OSAPI_LOG_BASE + 56)
```

Failure during VxWorks call semGive when managing shared memory segments, semaphores, or mutexes.

OSAPI_LOG_SEM_TAKE_EC

```
#define OSAPI_LOG_SEM_TAKE_EC (OSAPI_LOG_BASE + 57)
```

Failure during VxWorks call semTake when managing shared memory segments, semaphores, or mutexes.

OSAPI_LOG_UNKNOWN_SEMAPHORE_TYPE_EC

```
#define OSAPI_LOG_UNKNOWN_SEMAPHORE_TYPE_EC (OSAPI_LOG_BASE + 58)
```

Trying to perform an operation an unknown semmutex type.

OSAPI_LOG_SEGMENT_KEY_ALREADY_EXISTS_EC

```
#define OSAPI_LOG_SEGMENT_KEY_ALREADY_EXISTS_EC (OSAPI_LOG_BASE + 59)
```

Failed to create segment due to a segment already existing.

OSAPI_LOG_SEGMENT_KEY_DOESNT_EXIST_EC

```
#define OSAPI_LOG_SEGMENT_KEY_DOESNT_EXIST_EC (OSAPI_LOG_BASE + 60)
```

Segment trying to be attached to does not exist.

OSAPI_LOG_SEGMENT_DETACH_FAILED_EC

```
#define OSAPI_LOG_SEGMENT_DETACH_FAILED_EC (OSAPI_LOG_BASE + 61)
```

Failed to detach from shared memory segment.

OSAPI_LOG_SHMEM_ATTACH_FAILED_EC

```
#define OSAPI_LOG_SHMEM_ATTACH_FAILED_EC (OSAPI_LOG_BASE + 62)
```

Attaching to shared memory segment/mutex/signaling semaphore failed.

OSAPI_LOG_SHMEM_GIVE_FAILED_EC

```
#define OSAPI_LOG_SHMEM_GIVE_FAILED_EC (OSAPI_LOG_BASE + 63)
```

Failed to give shared memory mutex/signaling semaphore.

OSAPI_LOG_SHMEM_TAKE_FAILED_EC

```
#define OSAPI_LOG_SHMEM_TAKE_FAILED_EC (OSAPI_LOG_BASE + 64)
```

Failed to take shared memory mutex/signaling semaphore.

OSAPI_LOG_SHMEM_SEM_DELETE_FAILED_EC

```
#define OSAPI_LOG_SHMEM_SEM_DELETE_FAILED_EC (OSAPI_LOG_BASE + 65)
```

Failed to delete shared memory semaphore or mutex.

OSAPI_LOG_SHMEM_SEM_MUTEX_CREATED_EC

```
#define OSAPI_LOG_SHMEM_SEM_MUTEX_CREATED_EC (OSAPI_LOG_BASE + 66)
```

Error during shared memory semaphore creation/deletion.

OSAPI_LOG_SHMEM_SEGMENT_FILE_MAP_FAILED_EC

```
#define OSAPI_LOG_SHMEM_SEGMENT_FILE_MAP_FAILED_EC (OSAPI_LOG_BASE + 67)
```

Windows specific API MapViewOfFile failed.

OSAPI_LOG_SHMEM_SET_DESCRIPTOR_EC

```
#define OSAPI_LOG_SHMEM_SET_DESCRIPTOR_EC (OSAPI_LOG_BASE + 68)
```

Windows specific error during setting security descriptor.

OSAPI_LOG_SHMEM_INIT_DESCRIPTOR_EC

```
#define OSAPI_LOG_SHMEM_INIT_DESCRIPTOR_EC (OSAPI_LOG_BASE + 69)
```

Windows specific error during initialization of security descriptor.

OSAPI_LOG_SHMEM_FTRUNCATE_EC

```
#define OSAPI_LOG_SHMEM_FTRUNCATE_EC (OSAPI_LOG_BASE + 70)
```

Posix specific error during ftruncate(...) call.

OSAPI_LOG_SHMEM_MMAP_EC

```
#define OSAPI_LOG_SHMEM_MMAP_EC (OSAPI_LOG_BASE + 71)
```

Posix specific error during mmap(...) call.

OSAPI_LOG_SHMEM_SHM_OPEN_EC

```
#define OSAPI_LOG_SHMEM_SHM_OPEN_EC (OSAPI_LOG_BASE + 72)
```

Posix specific error during shm_open(...) call.

OSAPI_LOG_SHMEM_MUNMAP_EC

```
#define OSAPI_LOG_SHMEM_MUNMAP_EC (OSAPI_LOG_BASE + 73)
```

Posix specific error during munmap(...) call.

OSAPI_LOG_CLOSE_EC

```
#define OSAPI_LOG_CLOSE_EC (OSAPI_LOG_BASE + 74)
```

Posix specific error during close(...) call.

OSAPI_LOG_SHMEM_UNLINK_EC

```
#define OSAPI_LOG_SHMEM_UNLINK_EC (OSAPI_LOG_BASE + 76)
```

Posix specific error during shared memory unlink(...) call.

OSAPI_LOG_EXHAUSTED_POSIX_SEM_LIMIT_EC

```
#define OSAPI_LOG_EXHAUSTED_POSIX_SEM_LIMIT_EC (OSAPI_LOG_BASE + 77)
```

System ran out of space to create semaphores. Consider increasing Posix sem limit.

OSAPI_LOG_SHMEM_SEM_WAIT_EC

```
#define OSAPI_LOG_SHMEM_SEM_WAIT_EC (OSAPI_LOG_BASE + 78)
```

Posix specific error during shared memory wait call.

OSAPI_LOG_SHMEM_SEM_UNLINK_EC

```
#define OSAPI_LOG_SHMEM_SEM_UNLINK_EC (OSAPI_LOG_BASE + 79)
```

Posix specific error during shared memory sem unlink call.

OSAPI_LOG_SHMEM_POSIX_GIVE_EC

```
#define OSAPI_LOG_SHMEM_POSIX_GIVE_EC (OSAPI_LOG_BASE + 80)
```

Posix specific error during os specific implementation of a semaphore mutex give.

OSAPI_LOG_THREAD_SEMPOOL_EXCEEDED_EC

```
#define OSAPI_LOG_THREAD_SEMPOOL_EXCEEDED_EC (OSAPI_LOG_BASE + 90)
```

Failed to add a semaphore to the thread semaphore pool.

This maximum number of semaphores is configured with ::max_user_blocking_threads. One semaphore is needed per receive thread created.

OSAPI_LOG_THREAD_SEMPOOL_INVALID_SEM_EC

```
#define OSAPI_LOG_THREAD_SEMPOOL_INVALID_SEM_EC (OSAPI_LOG_BASE + 91)
```

Failed to delete a semaphore from the thread semaphore pool.

This is error indicates an imbalanced number of add/delete calls to the thread semaphore pool.

OSAPI_LOG_SHMEM_SIZE_EXCEEDED_EC

```
#define OSAPI_LOG_SHMEM_SIZE_EXCEEDED_EC (OSAPI_LOG_BASE + 92)
```

Trying to create a shared memory segment that is greater than $(2^{31}) - 1$. The middleware cannot support shared memory segments created than ~ 2 GB even though the operating system may.

OSAPI_LOG_THREAD_JOIN_EC

```
#define OSAPI_LOG_THREAD_JOIN_EC (OSAPI_LOG_BASE + 95)
```

Failed to join a thread.

OSAPI_LOG_THREAD_GET_MAX_PRIORITY_EC

```
#define OSAPI_LOG_THREAD_GET_MAX_PRIORITY_EC (OSAPI_LOG_BASE + 96)
```

Failed to get the maximum priority value for a scheduling policy.

OSAPI_LOG_SHMEM_FSTAT_EC

```
#define OSAPI_LOG_SHMEM_FSTAT_EC (OSAPI_LOG_BASE + 97)
```

POSIX specific error during fstat(...) call.

OSAPI_LOG_SHMEM_MUTEX_INIT_TIMEOUT_EC

```
#define OSAPI_LOG_SHMEM_MUTEX_INIT_TIMEOUT_EC (OSAPI_LOG_BASE + 97)
```

Timed out waiting for a shared memory mutex to be initialized.

This condition could occur if a process dies while initializing a shared memory mutex segment. The segment containing the mutex must be manually removed before it can be reinitialized.

OSAPI_LOG_AUTOSAR_SETEVENT_CALL_EC

```
#define OSAPI_LOG_AUTOSAR_SETEVENT_CALL_EC (OSAPI_LOG_BASE + 100)
```

Autosar SetEvent system call failed.

OSAPI_LOG_AUTOSAR_CLEAREVENT_CALL_EC

```
#define OSAPI_LOG_AUTOSAR_CLEAREVENT_CALL_EC (OSAPI_LOG_BASE + 101)
```

Autosar ClearEvent system call failed.

OSAPI_LOG_AUTOSAR_WAITEVENT_CALL_EC

```
#define OSAPI_LOG_AUTOSAR_WAITEVENT_CALL_EC (OSAPI_LOG_BASE + 102)
```

Autosar WaitEvent system call failed.

OSAPI_LOG_AUTOSAR_GETEVENT_CALL_EC

```
#define OSAPI_LOG_AUTOSAR_GETEVENT_CALL_EC (OSAPI_LOG_BASE + 103)
```

Autosar GetEvent system call failed.

OSAPI_LOG_AUTOSAR_TERMINATETASK_CALL_EC

```
#define OSAPI_LOG_AUTOSAR_TERMINATETASK_CALL_EC (OSAPI_LOG_BASE + 104)
```

Autosar TerminateTask system call failed.

OSAPI_LOG_AUTOSAR_ACTIVATETASK_CALL_EC

```
#define OSAPI_LOG_AUTOSAR_ACTIVATETASK_CALL_EC (OSAPI_LOG_BASE + 105)
```

Autosar ActivateTask system call failed.

OSAPI_LOG_AUTOSAR_SCHEDULE_CALL_EC

```
#define OSAPI_LOG_AUTOSAR_SCHEDULE_CALL_EC (OSAPI_LOG_BASE + 106)
```

Autosar Schedule system call failed.

OSAPI_LOG_AUTOSAR_SETRELALARM_CALL_EC

```
#define OSAPI_LOG_AUTOSAR_SETRELALARM_CALL_EC (OSAPI_LOG_BASE + 107)
```

Autosar SetRelAlarm system call failed.

OSAPI_LOG_AUTOSAR_CANCELALARM_CALL_EC

```
#define OSAPI_LOG_AUTOSAR_CANCELALARM_CALL_EC (OSAPI_LOG_BASE + 108)
```

Autosar CancelAlarm system call failed.

OSAPI_LOG_SEGMENT_ALREADY_EXISTS_EC

```
#define OSAPI_LOG_SEGMENT_ALREADY_EXISTS_EC (OSAPI_LOG_BASE + 150)
```

Shared memory segment already exists.

Failed to create a shared memory segment because a segment with the specified name already exists in the system.

OSAPI_LOG_SEGMENT_DOESNT_EXIST_EC

```
#define OSAPI_LOG_SEGMENT_DOESNT_EXIST_EC (OSAPI_LOG_BASE + 151)
```

Shared memory segment does not exist.

Failed to attach to a shared memory segment because a segment with the specified name does not exist in the system.

OSAPI_LOG_SHMEM_INVALID_SEGMENT_NAME_EC

```
#define OSAPI_LOG_SHMEM_INVALID_SEGMENT_NAME_EC (OSAPI_LOG_BASE + 152)
```

Invalid shared memory segment name.

The specified shared memory segment name is invalid. The name may be too long, empty, or contain invalid characters for the platform.

OSAPI_LOG_SHMEM_INVALID_SEGMENT_MODE_EC

```
#define OSAPI_LOG_SHMEM_INVALID_SEGMENT_MODE_EC (OSAPI_LOG_BASE + 153)
```

Invalid shared memory segment mode.

The specified shared memory segment mode is invalid or incompatible with the requested operation.

OSAPI_LOG_SHMEM_SEGMENT_NOT_LOCKABLE_EC

```
#define OSAPI_LOG_SHMEM_SEGMENT_NOT_LOCKABLE_EC (OSAPI_LOG_BASE + 154)
```

Shared memory segment does not support locking.

Attempted to perform a lock operation on a shared memory segment that was not created with lockable mode.

OSAPI_LOG_SHMEM_SEGMENT_NOT_ROBUST_EC

```
#define OSAPI_LOG_SHMEM_SEGMENT_NOT_ROBUST_EC (OSAPI_LOG_BASE + 155)
```

Shared memory segment does not support robust mutexes.

Attempted to perform a robust mutex operation on a shared memory segment that was not created with robust mode, or the platform does not support robust mutexes.

OSAPI_LOG_SHMEM_ALREADY_ATTACHED_EC

```
#define OSAPI_LOG_SHMEM_ALREADY_ATTACHED_EC (OSAPI_LOG_BASE + 156)
```

Shared memory segment is already attached.

Cannot attach to a shared memory segment because the handle is already attached to a segment. Detach from the current segment before attaching to another.

OSAPI_LOG_SHMEM_NOT_ATTACHED_EC

```
#define OSAPI_LOG_SHMEM_NOT_ATTACHED_EC (OSAPI_LOG_BASE + 157)
```

Shared memory segment is not attached.

Cannot perform the requested operation because the handle is not currently attached to a shared memory segment.

OSAPI_LOG_SHMEM_CHECK_UNLINKED_EC

```
#define OSAPI_LOG_SHMEM_CHECK_UNLINKED_EC (OSAPI_LOG_BASE + 158)
```

Checking if shared memory segment has been unlinked.

Information message indicating that the system is checking whether the specified shared memory segment has been unlinked.

OSAPI_SHMEM_SEGMENT_NOT_INITIALIZED_EC

```
#define OSAPI_SHMEM_SEGMENT_NOT_INITIALIZED_EC (OSAPI_LOG_BASE + 159)
```

Shared memory segment is not initialized.

Failed to attach to a shared memory segment because it has not been properly initialized by the creator. This may occur if the creator process terminated prematurely or if the maximum wait time for initialization was exceeded.

OSAPI_LOG_SHMEM_SEGMENT_NAME_ALREADY_EXISTS_EC

```
#define OSAPI_LOG_SHMEM_SEGMENT_NAME_ALREADY_EXISTS_EC (OSAPI_LOG_BASE + 160)
```

Shared memory segment with specified name already exists.

Failed to create a shared memory segment because a segment with the specified name already exists in the system. This is an informational message variant that includes the segment name.

OSAPI_LOG_SHMEM_SEGMENT_NAME_DOESNT_EXIST_EC

```
#define OSAPI_LOG_SHMEM_SEGMENT_NAME_DOESNT_EXIST_EC (OSAPI_LOG_BASE + 161)
```

Shared memory segment with specified name does not exist.

Not currently used. Reserved for future use to maintain numbering consistency.

OSAPI_LOG_PTHREAD_MUTEXATTR_INIT_EC

```
#define OSAPI_LOG_PTHREAD_MUTEXATTR_INIT_EC (OSAPI_LOG_BASE + 200)
```

Failed to create a pthread mutex attribute.

OSAPI_LOG_PTHREAD_MUTEXATTR_SETPSHARED_EC

```
#define OSAPI_LOG_PTHREAD_MUTEXATTR_SETPSHARED_EC (OSAPI_LOG_BASE + 201)
```

Failed to set the pthread mutex attribute to be process shared.

OSAPI_LOG_PTHREAD_MUTEXATTR_SETROBUST_EC

```
#define OSAPI_LOG_PTHREAD_MUTEXATTR_SETROBUST_EC (OSAPI_LOG_BASE + 202)
```

Failed to set the pthread mutex attribute to be robust.

OSAPI_LOG_PTHREAD_MUTEXATTR_DESTROY_EC

```
#define OSAPI_LOG_PTHREAD_MUTEXATTR_DESTROY_EC (OSAPI_LOG_BASE + 203)
```

Failed to destroy the pthread mutex attribute.

OSAPI_LOG_PTHREAD_MUTEX_INIT_EC

```
#define OSAPI_LOG_PTHREAD_MUTEX_INIT_EC (OSAPI_LOG_BASE + 204)
```

Failed to initialize a pthread mutex.

OSAPI_LOG_PTHREAD_MUTEX_DESTROY_EC

```
#define OSAPI_LOG_PTHREAD_MUTEX_DESTROY_EC (OSAPI_LOG_BASE + 205)
```

Failed to destroy a pthread mutex.

OSAPI_LOG_PTHREAD_MUTEX_LOCK_EC

```
#define OSAPI_LOG_PTHREAD_MUTEX_LOCK_EC (OSAPI_LOG_BASE + 206)
```

Failed to lock a pthread mutex.

OSAPI_LOG_PTHREAD_MUTEX_UNLOCK_EC

```
#define OSAPI_LOG_PTHREAD_MUTEX_UNLOCK_EC (OSAPI_LOG_BASE + 207)
```

Failed to unlock a pthread mutex.

OSAPI_LOG_PTHREAD_MUTEX_CONSISTENT_EC

```
#define OSAPI_LOG_PTHREAD_MUTEX_CONSISTENT_EC (OSAPI_LOG_BASE + 208)
```

Failed to set the pthread mutex to be consistent.

OSAPI_LOG_PTHREAD_CONDATTR_INIT_EC

```
#define OSAPI_LOG_PTHREAD_CONDATTR_INIT_EC (OSAPI_LOG_BASE + 209)
```

Failed to create a pthread condition variable attribute.

OSAPI_LOG_PTHREAD_CONDATTR_SETPSHARED_EC

```
#define OSAPI_LOG_PTHREAD_CONDATTR_SETPSHARED_EC (OSAPI_LOG_BASE + 210)
```

Failed to set the pthread condition variable attribute to be process shared.

OSAPI_LOG_PTHREAD_CONDATTR_DESTROY_EC

```
#define OSAPI_LOG_PTHREAD_CONDATTR_DESTROY_EC (OSAPI_LOG_BASE + 211)
```

Failed to destroy the pthread condition variable attribute.

OSAPI_LOG_PTHREAD_COND_INIT_EC

```
#define OSAPI_LOG_PTHREAD_COND_INIT_EC (OSAPI_LOG_BASE + 212)
```

Failed to initialize a pthread condition variable.

OSAPI_LOG_PTHREAD_COND_DESTROY_EC

```
#define OSAPI_LOG_PTHREAD_COND_DESTROY_EC (OSAPI_LOG_BASE + 213)
```

Failed to destroy a pthread condition variable.

OSAPI_LOG_PTHREAD_COND_WAIT_EC

```
#define OSAPI_LOG_PTHREAD_COND_WAIT_EC (OSAPI_LOG_BASE + 214)
```

Failed to wait on a pthread condition variable.

OSAPI_LOG_PTHREAD_COND_SIGNAL_EC

```
#define OSAPI_LOG_PTHREAD_COND_SIGNAL_EC (OSAPI_LOG_BASE + 215)
```

Failed to signal a pthread condition variable.

OSAPI_LOG_PTHREAD_MUTEXATTR_SETTYPE_EC

```
#define OSAPI_LOG_PTHREAD_MUTEXATTR_SETTYPE_EC (OSAPI_LOG_BASE + 216)
```

Failed to set the type of a pthread mutex attribute.

OSAPI_LOG_PTHREAD_MUTEXATTR_SETPROTOCOL_EC

```
#define OSAPI_LOG_PTHREAD_MUTEXATTR_SETPROTOCOL_EC (OSAPI_LOG_BASE + 217)
```

Failed to set the protocol of a pthread mutex attribute.

OSAPI_LOG_PTHREAD_MUTEXATTR_SETPRIOCEILING_EC

```
#define OSAPI_LOG_PTHREAD_MUTEXATTR_SETPRIOCEILING_EC (OSAPI_LOG_BASE + 218)
```

Failed to set the priority ceiling of a pthread mutex attribute.

OSAPI_LOG_WIN_WAIT_FAILED_EC

```
#define OSAPI_LOG_WIN_WAIT_FAILED_EC (OSAPI_LOG_BASE + 300)
```

WaitForSingleObject returned an error while waiting for the timer semaphore to time out. Micro continues to run, but it may indicate a platform problem.

6.20.3 REDA

Real-time Efficient Data Structures and Algorithms. ModuleID = 1.

Macros

- #define REDA_LOG_HEAP_MGR_SAMPLE_BUFFERPOOL 1
Sample buffer pool object.
- #define REDA_LOG_HEAP_MGR_LISTNODE_BUFFERPOOL 2
Listnode buffer pool object.
- #define REDA_LOG_HEAP_MGR_OBJECT_MEMORY 3
Heap manager object.
- #define REDA_LOG_HEAP_MGR_INDEXER 4
REDA indexer object.
- #define REDA_LOG_HEAP_MGR_MUTEX 5
REDA heap manager object.
- #define REDA_LOG_BUFFERPOOL_OUT_OF_RESOURCES_EC (REDA_LOG_BASE + 1)
Not sufficient memory to allocate buffer-pool.
- #define REDA_LOG_BUFFERPOOL_BUFFER_INITIALIZATION_FAILED_EC (REDA_LOG_BASE + 2)
The buffer initialization method failed.
- #define REDA_LOG_BUFFERPOOL_NOT_EMPTY_EC (REDA_LOG_BASE + 3)
Cannot delete buffer-pool due to buffer(s) not returned to pool.
- #define REDA_LOG_BUFFERPOOL_DOUBLE_FREE_EC (REDA_LOG_BASE + 4)
A buffer was freed more than once.
- #define REDA_LOG_MEMPOOL_INCONSISTENT_PROPERTIES_EC (REDA_LOG_BASE + 5)
Inconsistent properties specified for a REDA_MemPool.
- #define REDA_LOG_MEMPOOL_OUT_OF_RESOURCES_EC (REDA_LOG_BASE + 6)
Insufficient memory to allocate a new REDA_MemPool.
- #define REDA_LOG_MEMPOOL_POOL_ALLOCATION_EC (REDA_LOG_BASE + 7)
Error during allocation of the memory pool used by a REDA_MemPool.
- #define REDA_LOG_BUFFERPOOL_NULL_POINTER_EC (REDA_LOG_BASE + 8)
A pointer in a pool's linked list was NULL.

- #define REDA_LOG_SEQUENCE_COPY_FAILED_EC (REDA_LOG_BASE + 100)
An error occurred while copying a sequence.
- #define REDA_LOG_SEQUENCE_ALLOC_FAILED_EC (REDA_LOG_BASE + 101)
An error occurred while allocating space for a sequence.
- #define REDA_LOG_SEQUENCE_SET_MAX_FAILED_EC (REDA_LOG_BASE + 102)
An error occurred while setting the maximum sequence size.
- #define REDA_LOG_SEQUENCE_REALLOCATION_EC (REDA_LOG_BASE + 103)
An attempt was made to resize an already allocated sequence.
- #define REDA_LOG_SEQUENCE_INVALID_OPERATION_EC (REDA_LOG_BASE + 104)
An error occurred while operating on two sequences (copy/compare)
- #define REDA_LOG_SEQUENCE_INVALID_LENGTH_EC (REDA_LOG_BASE + 105)
An attempt was made to set an illegal length (length < 0 or length > max_length)
- #define REDA_LOG_SEQUENCE_INDEX_OUT_OF_BOUNDS_EC (REDA_LOG_BASE + 106)
An illegal sequence index was specified (index < 0 or index > length)
- #define REDA_LOG_STRING_ALLOC_FAILED_EC (REDA_LOG_BASE + 200)
An error occurred while allocating memory for a string.
- #define REDA_LOG_INDEX_FULL_EC (REDA_LOG_BASE + 300)
An attempt was made to add an element to a full index.
- #define REDA_LOG_INDEX_ENTRY_EXISTS_EC (REDA_LOG_BASE + 301)
An attempt was made to add an existing element to an index.
- #define REDA_LOG_INVALID_LIST_NODE_EC (REDA_LOG_BASE + 302)
An invalid list node was encountered.
- #define REDA_LOG_BUFFER_OUT_OF_MEMORY_EC (REDA_LOG_BASE + 307)
Failed to allocate buffer of a specified kind.
- #define REDA_LOG_HEAP_MGR_ALLOC_EC (REDA_LOG_BASE + 308)
Allocation error in REDA Heap Manager.
- #define REDA_LOG_HEAP_MGR_GET_BUFFER_EC (REDA_LOG_BASE + 309)
Error when there are no more available loanable buffers.
- #define REDA_LOG_HEAP_MGR_MUTEX_FAILURE_EC (REDA_LOG_BASE + 310)
Error during locking or unlocking of a mutex.
- #define NETIO_SHMEM_LOG_CQ_INT_REPRESENTATION_EC (SHMEM_LOG_BASE + 200)
Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.
- #define NETIO_SHMEM_LOG_CQ_UNLINKED_EC (SHMEM_LOG_BASE + 201)
Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.
- #define NETIO_SHMEM_LOG_CQ_INVALID_SIGNATURE_EC (SHMEM_LOG_BASE + 202)
Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.
- #define NETIO_SHMEM_LOG_CQ_INCOMPATIBLE_VERSION_EC (SHMEM_LOG_BASE + 203)
Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.
- #define NETIO_SHMEM_LOG_CQ_INCONSISTENT_STATE_EC (SHMEM_LOG_BASE + 204)
Found an inconsistent concurrent queue state while performing a a read or write. The queue memory may have been corrupted or otherwise tampered with.

6.20.3.1 Detailed Description

Real-time Efficient Data Structures and Algorithms. ModuleID = 1.

6.20.3.2 Macro Definition Documentation

REDA_LOG_HEAP_MGR_SAMPLE_BUFFERPOOL

```
#define REDA_LOG_HEAP_MGR_SAMPLE_BUFFERPOOL 1
```

Sample buffer pool object.

REDA_LOG_HEAP_MGR_LISTNODE_BUFFERPOOL

```
#define REDA_LOG_HEAP_MGR_LISTNODE_BUFFERPOOL 2
```

Listnode buffer pool object.

REDA_LOG_HEAP_MGR_OBJECT_MEMORY

```
#define REDA_LOG_HEAP_MGR_OBJECT_MEMORY 3
```

Heap manager object.

REDA_LOG_HEAP_MGR_INDEXER

```
#define REDA_LOG_HEAP_MGR_INDEXER 4
```

REDA indexer object.

REDA_LOG_HEAP_MGR_MUTEX

```
#define REDA_LOG_HEAP_MGR_MUTEX 5
```

REDA heap manager object.

REDA_LOG_BUFFERPOOL_OUT_OF_RESOURCES_EC

```
#define REDA_LOG_BUFFERPOOL_OUT_OF_RESOURCES_EC (REDA_LOG_BASE + 1)
```

Not sufficient memory to allocate buffer-pool.

REDA_LOG_BUFFERPOOL_BUFFER_INITIALIZATION_FAILED_EC

```
#define REDA_LOG_BUFFERPOOL_BUFFER_INITIALIZATION_FAILED_EC (REDA_LOG_BASE + 2)
```

The buffer initialization method failed.

REDA_LOG_BUFFERPOOL_NOT_EMPTY_EC

```
#define REDA_LOG_BUFFERPOOL_NOT_EMPTY_EC (REDA_LOG_BASE + 3)
```

Cannot delete buffer-pool due to buffer(s) not returned to pool.

REDA_LOG_BUFFERPOOL_DOUBLE_FREE_EC

```
#define REDA_LOG_BUFFERPOOL_DOUBLE_FREE_EC (REDA_LOG_BASE + 4)
```

A buffer was freed more than once.

REDA_LOG_MEMPOOL_INCONSISTENT_PROPERTIES_EC

```
#define REDA_LOG_MEMPOOL_INCONSISTENT_PROPERTIES_EC (REDA_LOG_BASE + 5)
```

Inconsistent properties specified for a REDA_MemPool.

REDA_LOG_MEMPOOL_OUT_OF_RESOURCES_EC

```
#define REDA_LOG_MEMPOOL_OUT_OF_RESOURCES_EC (REDA_LOG_BASE + 6)
```

Insufficient memory to allocate a new REDA_MemPool.

REDA_LOG_MEMPOOL_POOL_ALLOCATION_EC

```
#define REDA_LOG_MEMPOOL_POOL_ALLOCATION_EC (REDA_LOG_BASE + 7)
```

Error during allocation of the memory pool used by a REDA_MemPool.

REDA_LOG_BUFFERPOOL_NULL_POINTER_EC

```
#define REDA_LOG_BUFFERPOOL_NULL_POINTER_EC (REDA_LOG_BASE + 8)
```

A pointer in a pool's linked list was NULL.

REDA_LOG_SEQUENCE_COPY_FAILED_EC

```
#define REDA_LOG_SEQUENCE_COPY_FAILED_EC (REDA_LOG_BASE + 100)
```

An error occurred while copying a sequence.

REDA_LOG_SEQUENCE_ALLOC_FAILED_EC

```
#define REDA_LOG_SEQUENCE_ALLOC_FAILED_EC (REDA_LOG_BASE + 101)
```

An error occurred while allocating space for a sequence.

REDA_LOG_SEQUENCE_SET_MAX_FAILED_EC

```
#define REDA_LOG_SEQUENCE_SET_MAX_FAILED_EC (REDA_LOG_BASE + 102)
```

An error occurred while setting the maximum sequence size.

REDA_LOG_SEQUENCE_REALLOCATION_EC

```
#define REDA_LOG_SEQUENCE_REALLOCATION_EC (REDA_LOG_BASE + 103)
```

An attempt was made to resize an already allocated sequence.

REDA_LOG_SEQUENCE_INVALID_OPERATION_EC

```
#define REDA_LOG_SEQUENCE_INVALID_OPERATION_EC (REDA_LOG_BASE + 104)
```

An error occurred while operating on two sequences (copy/compare)

REDA_LOG_SEQUENCE_INVALID_LENGTH_EC

```
#define REDA_LOG_SEQUENCE_INVALID_LENGTH_EC (REDA_LOG_BASE + 105)
```

An attempt was made to set an illegal length (length < 0 or length > max_length)

REDA_LOG_SEQUENCE_INDEX_OUT_OF_BOUNDS_EC

```
#define REDA_LOG_SEQUENCE_INDEX_OUT_OF_BOUNDS_EC (REDA_LOG_BASE + 106)
```

An illegal sequence index was specified (index < 0 or index > length)

REDA_LOG_STRING_ALLOC_FAILED_EC

```
#define REDA_LOG_STRING_ALLOC_FAILED_EC (REDA_LOG_BASE + 200)
```

An error occurred while allocating memory for a string.

REDA_LOG_INDEX_FULL_EC

```
#define REDA_LOG_INDEX_FULL_EC (REDA_LOG_BASE + 300)
```

An attempt was made to add an element to a full index.

REDA_LOG_INDEX_ENTRY_EXISTS_EC

```
#define REDA_LOG_INDEX_ENTRY_EXISTS_EC (REDA_LOG_BASE + 301)
```

An attempt was made to add an existing element to an index.

REDA_LOG_INVALID_LIST_NODE_EC

```
#define REDA_LOG_INVALID_LIST_NODE_EC (REDA_LOG_BASE + 302)
```

An invalid list node was encountered.

REDA_LOG_BUFFER_OUT_OF_MEMORY_EC

```
#define REDA_LOG_BUFFER_OUT_OF_MEMORY_EC (REDA_LOG_BASE + 307)
```

Failed to allocate buffer of a specified kind.

REDA_LOG_HEAP_MGR_ALLOC_EC

```
#define REDA_LOG_HEAP_MGR_ALLOC_EC (REDA_LOG_BASE + 308)
```

Allocation error in REDA Heap Manager.

REDA_LOG_HEAP_MGR_GET_BUFFER_EC

```
#define REDA_LOG_HEAP_MGR_GET_BUFFER_EC (REDA_LOG_BASE + 309)
```

Error when there are no more available loanable buffers.

REDA_LOG_HEAP_MGR_MUTEX_FAILURE_EC

```
#define REDA_LOG_HEAP_MGR_MUTEX_FAILURE_EC (REDA_LOG_BASE + 310)
```

Error during locking or unlocking of a mutex.

NETIO_SHMEM_LOG_CQ_INT_REPRESENTATION_EC

```
#define NETIO_SHMEM_LOG_CQ_INT_REPRESENTATION_EC (SHMEM_LOG_BASE + 200)
```

Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.

NETIO_SHMEM_LOG_CQ_UNLINKED_EC

```
#define NETIO_SHMEM_LOG_CQ_UNLINKED_EC (SHMEM_LOG_BASE + 201)
```

Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.

NETIO_SHMEM_LOG_CQ_INVALID_SIGNATURE_EC

```
#define NETIO_SHMEM_LOG_CQ_INVALID_SIGNATURE_EC (SHMEM_LOG_BASE + 202)
```

Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.

NETIO_SHMEM_LOG_CQ_INCOMPATIBLE_VERSION_EC

```
#define NETIO_SHMEM_LOG_CQ_INCOMPATIBLE_VERSION_EC (SHMEM_LOG_BASE + 203)
```

Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.

NETIO_SHMEM_LOG_CQ_INCONSISTENT_STATE_EC

```
#define NETIO_SHMEM_LOG_CQ_INCONSISTENT_STATE_EC (SHMEM_LOG_BASE + 204)
```

Found an inconsistent concurrent queue state while performing a read or write. The queue memory may have been corrupted or otherwise tampered with.

6.20.4 DB

Database. ModuleID = 2.

Macros

- #define `DB_LOG_SORTED_ALLOC_EC` (`DB_LOG_BASE` + 1)
Not sufficient memory to allocate sorted list for an index.
- #define `DB_LOG_NAME_TOO_LONG_EC` (`DB_LOG_BASE` + 2)
The specified database name is too long.
- #define `DB_LOG_ILLEGAL_TABLE_SIZE_EC` (`DB_LOG_BASE` + 3)
Illegal table size specified.
- #define `DB_LOG_ILLEGAL_LOCK_MODE_EC` (`DB_LOG_BASE` + 4)
Illegal combination of lock mode and mutex given.
- #define `DB_LOG_MUTEX_ALLOC_EC` (`DB_LOG_BASE` + 5)
Failed to allocate database mutex.
- #define `DB_LOG_ALLOC_TABLE_POOL_EC` (`DB_LOG_BASE` + 6)
Failed to allocate buffer pool.
- #define `DB_LOG_TABLES_INUSE_EC` (`DB_LOG_BASE` + 7)
Table is in use.
- #define `DB_LOG_TABLE_NAME_TOO_LONG_EC` (`DB_LOG_BASE` + 8)
Specified table name too long.
- #define `DB_LOG_ILLEGAL_RECORD_COUNT_EC` (`DB_LOG_BASE` + 9)
Illegal record count specified.
- #define `DB_LOG_TABLE_EXISTS_EC` (`DB_LOG_BASE` + 10)
Specified table already exists.
- #define `DB_LOG_OUT_OF_TABLES_EC` (`DB_LOG_BASE` + 11)
Table resources exceeded.
- #define `DB_LOG_OUT_OF_RECORDS_EC` (`DB_LOG_BASE` + 12)
Table record resources exceeded.
- #define `DB_LOG_OUT_OF_INDICES_EC` (`DB_LOG_BASE` + 13)
Table index resources exceeded.
- #define `DB_LOG_OUT_OF_CURSORS_EC` (`DB_LOG_BASE` + 14)
Table cursor resources exceeded.
- #define `DB_LOG_RECORDS_INUSE_EC` (`DB_LOG_BASE` + 15)
Cannot delete table, records are still in table.
- #define `DB_LOG_CURSORS_INUSE_EC` (`DB_LOG_BASE` + 16)
Cannot delete table, cursors are still in use.
- #define `DB_LOG_INDEX_INUSE_EC` (`DB_LOG_BASE` + 17)
Cannot delete table, indices are still in use.
- #define `DB_LOG_ALLOC_CURSOR_POOL_EC` (`DB_LOG_BASE` + 18)
Failed to allocate cursor buffer pool.
- #define `DB_LOG_ALLOC_INDEX_POOL_EC` (`DB_LOG_BASE` + 19)
Failed to allocate index buffer pool.
- #define `DB_LOG_ALLOC_RECORD_POOL_EC` (`DB_LOG_BASE` + 20)
Failed to allocate record buffer pool.
- #define `DB_LOG_ALLOC_DATABASE_EC` (`DB_LOG_BASE` + 21)
Failed to allocate database.
- #define `DB_LOG_TABLE_NOT_INUSE_EC` (`DB_LOG_BASE` + 22)
An attempt was made to delete a table not in use.
- #define `DB_LOG_RECORD_ALREADY_EXISTS_EC` (`DB_LOG_BASE` + 23)

An attempt was made to insert an already existing record.

- #define `DB_LOG_RECORD_DOES_NOT_EXIST_EC` (`DB_LOG_BASE` + 24)

An attempt was made to delete a record that does not exist.

- #define `DB_LOG_CURSOR_INVALIDATED_EC` (`DB_LOG_BASE` + 25)

An attempt was made to use a cursor that has been invalidated.

- #define `DB_LOG_LOCK_FAILURE_EC` (`DB_LOG_BASE` + 26)

Failed to lock the database.

- #define `DB_LOG_UNLOCK_FAILURE_EC` (`DB_LOG_BASE` + 27)

Failed to unlock the database.

- #define `DB_LOG_OUT_OF_CTORDTOR_EC` (`DB_LOG_BASE` + 28)

Constructor/Destructor memory could not be allocated.

6.20.4.1 Detailed Description

Database. ModuleID = 2.

6.20.4.2 Macro Definition Documentation

DB_LOG_SORTED_ALLOC_EC

```
#define DB_LOG_SORTED_ALLOC_EC (DB_LOG_BASE + 1)
```

Not sufficient memory to allocate sorted list for an index.

DB_LOG_NAME_TOO_LONG_EC

```
#define DB_LOG_NAME_TOO_LONG_EC (DB_LOG_BASE + 2)
```

The specified database name is too long.

DB_LOG_ILLEGAL_TABLE_SIZE_EC

```
#define DB_LOG_ILLEGAL_TABLE_SIZE_EC (DB_LOG_BASE + 3)
```

Illegal table size specified.

DB_LOG_ILLEGAL_LOCK_MODE_EC

```
#define DB_LOG_ILLEGAL_LOCK_MODE_EC (DB_LOG_BASE + 4)
```

Illegal combination of lock mode and mutex given.

DB_LOG_MUTEX_ALLOC_EC

```
#define DB_LOG_MUTEX_ALLOC_EC (DB_LOG_BASE + 5)
```

Failed to allocate database mutex.

DB_LOG_ALLOC_TABLE_POOL_EC

```
#define DB_LOG_ALLOC_TABLE_POOL_EC (DB_LOG_BASE + 6)
```

Failed to allocate buffer pool.

DB_LOG_TABLES_INUSE_EC

```
#define DB_LOG_TABLES_INUSE_EC (DB_LOG_BASE + 7)
```

Table is in use.

DB_LOG_TABLE_NAME_TOO_LONG_EC

```
#define DB_LOG_TABLE_NAME_TOO_LONG_EC (DB_LOG_BASE + 8)
```

Specified table name too long.

DB_LOG_ILLEGAL_RECORD_COUNT_EC

```
#define DB_LOG_ILLEGAL_RECORD_COUNT_EC (DB_LOG_BASE + 9)
```

Illegal record count specified.

DB_LOG_TABLE_EXISTS_EC

```
#define DB_LOG_TABLE_EXISTS_EC (DB_LOG_BASE + 10)
```

Specified table already exists.

DB_LOG_OUT_OF_TABLES_EC

```
#define DB_LOG_OUT_OF_TABLES_EC (DB_LOG_BASE + 11)
```

Table resources exceeded.

DB_LOG_OUT_OF_RECORDS_EC

```
#define DB_LOG_OUT_OF_RECORDS_EC (DB_LOG_BASE + 12)
```

Table record resources exceeded.

DB_LOG_OUT_OF_INDICES_EC

```
#define DB_LOG_OUT_OF_INDICES_EC (DB_LOG_BASE + 13)
```

Table index resources exceeded.

DB_LOG_OUT_OF_CURSORS_EC

```
#define DB_LOG_OUT_OF_CURSORS_EC (DB_LOG_BASE + 14)
```

Table cursor resources exceeded.

DB_LOG_RECORDS_INUSE_EC

```
#define DB_LOG_RECORDS_INUSE_EC (DB_LOG_BASE + 15)
```

Cannot delete table, records are still in table.

DB_LOG_CURSORS_INUSE_EC

```
#define DB_LOG_CURSORS_INUSE_EC (DB_LOG_BASE + 16)
```

Cannot delete table, cursors are still in use.

DB_LOG_INDEX_INUSE_EC

```
#define DB_LOG_INDEX_INUSE_EC (DB_LOG_BASE + 17)
```

Cannot delete table, indices are still in use.

DB_LOG_ALLOC_CURSOR_POOL_EC

```
#define DB_LOG_ALLOC_CURSOR_POOL_EC (DB_LOG_BASE + 18)
```

Failed to allocate cursor buffer pool.

DB_LOG_ALLOC_INDEX_POOL_EC

```
#define DB_LOG_ALLOC_INDEX_POOL_EC (DB_LOG_BASE + 19)
```

Failed to allocate index buffer pool.

DB_LOG_ALLOC_RECORD_POOL_EC

```
#define DB_LOG_ALLOC_RECORD_POOL_EC (DB_LOG_BASE + 20)
```

Failed to allocate record buffer pool.

DB_LOG_ALLOC_DATABASE_EC

```
#define DB_LOG_ALLOC_DATABASE_EC (DB_LOG_BASE + 21)
```

Failed to allocate database.

DB_LOG_TABLE_NOT_INUSE_EC

```
#define DB_LOG_TABLE_NOT_INUSE_EC (DB_LOG_BASE + 22)
```

An attempt was made to delete a table not in use.

DB_LOG_RECORD_ALREADY_EXISTS_EC

```
#define DB_LOG_RECORD_ALREADY_EXISTS_EC (DB_LOG_BASE + 23)
```

An attempt was made to insert an already existing record.

DB_LOG_RECORD_DOES_NOT_EXIST_EC

```
#define DB_LOG_RECORD_DOES_NOT_EXIST_EC (DB_LOG_BASE + 24)
```

An attempt was made to delete a record that does not exist.

DB_LOG_CURSOR_INVALIDATED_EC

```
#define DB_LOG_CURSOR_INVALIDATED_EC (DB_LOG_BASE + 25)
```

An attempt was made to use a cursor that has been invalidated.

DB_LOG_LOCK_FAILURE_EC

```
#define DB_LOG_LOCK_FAILURE_EC (DB_LOG_BASE + 26)
```

Failed to lock the database.

DB_LOG_UNLOCK_FAILURE_EC

```
#define DB_LOG_UNLOCK_FAILURE_EC (DB_LOG_BASE + 27)
```

Failed to unlock the database.

DB_LOG_OUT_OF_CTORDTOR_EC

```
#define DB_LOG_OUT_OF_CTORDTOR_EC (DB_LOG_BASE + 28)
```

Constructor/Destructor memory could not be allocated.

6.20.5 RT

RT. ModuleID = 3.

Macros

- #define **RT_LOG_SET_IMMUTABLE_PROPERTY_EC** (RT_LOG_BASE + 1)
Failed to set registry property due to registry already being enabled.
- #define **RT_LOG_REGISTRY_INIT_FAILURE_EC** (RT_LOG_BASE + 2)
Failed to initialize registry due to failed table creation.
- #define **RT_LOG_REGISTRY_FINALIZE_EC** (RT_LOG_BASE + 3)
Failed to finalize registry due to failed table deletion.
- #define **RT_LOG_REGISTRY_EXISTS_EC** (RT_LOG_BASE + 4)
An attempt was made to register a factory that already existed.
- #define **RT_LOG_REGISTRY_REGISTER_EC** (RT_LOG_BASE + 5)
Error registering a component factory. May have exceeded RT_RegistryProperty.max_factories.
- #define **RT_LOG_REGISTRY_INIT_FACTORY_EC** (RT_LOG_BASE + 7)
A registered factory failed to initialize.
- #define **RT_LOG_REGISTRY_NAME_TOO_LONG_EC** (RT_LOG_BASE + 8)
Factory name longer than maximum length of 8.
- #define **RT_LOG_REGISTRY_INCONSISTENT_CID_EC** (RT_LOG_BASE + 9)
Factory name exists, but the class ID is not of the requested type.
- #define **RT_LOG_REGISTRY_NOT_INITIALIZED_EC** (RT_LOG_BASE + 10)
An attempt was made to operate on an uninitialized registry.

6.20.5.1 Detailed Description

RT.ModuleID = 3.

6.20.5.2 Macro Definition Documentation

RT_LOG_SET_IMMUTABLE_PROPERTY_EC

```
#define RT_LOG_SET_IMMUTABLE_PROPERTY_EC (RT_LOG_BASE + 1)
```

Failed to set registry property due to registry already being enabled.

RT_LOG_REGISTRY_INIT_FAILURE_EC

```
#define RT_LOG_REGISTRY_INIT_FAILURE_EC (RT_LOG_BASE + 2)
```

Failed to initialize registry due to failed table creation.

RT_LOG_REGISTRY_FINALIZE_EC

```
#define RT_LOG_REGISTRY_FINALIZE_EC (RT_LOG_BASE + 3)
```

Failed to finalize registry due to failed table deletion.

RT_LOG_REGISTRY_EXISTS_EC

```
#define RT_LOG_REGISTRY_EXISTS_EC (RT_LOG_BASE + 4)
```

An attempt was made to register a factory that already existed.

RT_LOG_REGISTRY_REGISTER_EC

```
#define RT_LOG_REGISTRY_REGISTER_EC (RT_LOG_BASE + 5)
```

Error registering a component factory. May have exceeded RT_RegistryProperty.max_factories.

RT_LOG_REGISTRY_INIT_FACTORY_EC

```
#define RT_LOG_REGISTRY_INIT_FACTORY_EC (RT_LOG_BASE + 7)
```

A registered factory failed to initialize.

RT_LOG_REGISTRY_NAME_TOO_LONG_EC

```
#define RT_LOG_REGISTRY_NAME_TOO_LONG_EC (RT_LOG_BASE + 8)
```

Factory name longer than maximum length of 8.

RT_LOG_REGISTRY_INCONSISTENT_CID_EC

```
#define RT_LOG_REGISTRY_INCONSISTENT_CID_EC (RT_LOG_BASE + 9)
```

Factory name exists, but the class ID is not of the requested type.

RT_LOG_REGISTRY_NOT_INITIALIZED_EC

```
#define RT_LOG_REGISTRY_NOT_INITIALIZED_EC (RT_LOG_BASE + 10)
```

An attempt was made to operate on an uninitialized registry.

6.20.6 CDR

CDR. ModuleID = 5.

Macros

- `#define CDR_LOG_STREAM_ALLOC_EC (CDR_LOG_BASE + 1)`
Failed to allocate stream or stream buffer.
- `#define CDR_LOG_SET_OFFSET_EC (CDR_LOG_BASE + 2)`
Failed to set stream's current offset.
- `#define CDR_LOG_INCR_OFFSET_EC (CDR_LOG_BASE + 3)`
Failed to increment stream's current offset.
- `#define CDR_LOG_UNSUPPORTED_OPTIONS_EC (CDR_LOG_BASE + 4)`
Unsupported CDR encapsulations were received. The sample was dropped.

6.20.6.1 Detailed Description

CDR. ModuleID = 5.

6.20.6.2 Macro Definition Documentation**CDR_LOG_STREAM_ALLOC_EC**

```
#define CDR_LOG_STREAM_ALLOC_EC (CDR_LOG_BASE + 1)
```

Failed to allocate stream or stream buffer.

CDR_LOG_SET_OFFSET_EC

```
#define CDR_LOG_SET_OFFSET_EC (CDR_LOG_BASE + 2)
```

Failed to set stream's current offset.

CDR_LOG_INCR_OFFSET_EC

```
#define CDR_LOG_INCR_OFFSET_EC (CDR_LOG_BASE + 3)
```

Failed to increment stream's current offset.

CDR_LOG_UNSUPPORTED_OPTIONS_EC

```
#define CDR_LOG_UNSUPPORTED_OPTIONS_EC (CDR_LOG_BASE + 4)
```

Unsupported CDR encapsulations were received. The sample was dropped.

6.20.7 NETIO

Network I/O. ModuleID = 4.

Macros

- #define **NETIO_LOG_GETHOST_BYNAME_EC** (NETIO_LOG_BASE + 1)
Failed to get host address from a string representation.
- #define **NETIO_LOG_BIND_NEW_EC** (NETIO_LOG_BASE + 2)
Failed to allocate memory for a bind-resolver.
- #define **NETIO_LOG_BIND_NEW_RTABLE_EC** (NETIO_LOG_BASE + 3)
Failed to create a database table for routes of a bind-resolver.
- #define **NETIO_LOG_BIND_CREATE_RTABLE_EC** (NETIO_LOG_BASE + 4)
Failed to create route record.
- #define **NETIO_LOG_BIND_DELETE_RTABLE_EC** (NETIO_LOG_BASE + 5)
Failed to delete a database table for routes of a bind_resolver.
- #define **NETIO_LOG_BIND_SET_LENGTH_EC** (NETIO_LOG_BASE + 6)
Failed to set the length for a sequence of resolved addresses.
- #define **NETIO_LOG_BIND_SET_MAX_EC** (NETIO_LOG_BASE + 7)
Failed to set the maximum length of an internal sequence of addresses.
- #define **NETIO_LOG_BIND_EXTERNAL_FAILED_EC** (NETIO_LOG_BASE + 8)
An interface failed on bind_external.
- #define **NETIO_LOG_UNBIND_EXTERNAL_FAILED_EC** (NETIO_LOG_BASE + 10)
An interface failed on unbind_external.
- #define **NETIO_LOG_ROUTE_RTABLE_ALLOC_EC** (NETIO_LOG_BASE + 11)
Failed to allocate memory for a route resolver.

- #define NETIO_LOG_ROUTE_RTABLE_CREATE_EC (NETIO_LOG_BASE + 12)
Failed to create a database table for routes of a route-resolver.
- #define NETIO_LOG_ROUTE_RTABLE_DELETE_EC (NETIO_LOG_BASE + 13)
Failed to delete a database table for routes of a route-resolver.
- #define NETIO_LOG_ROUTE_RTABLE_ADD_EC (NETIO_LOG_BASE + 14)
Failed to create route resolver record.
- #define NETIO_LOG_ROUTE_RTABLE_UPDATE_EC (NETIO_LOG_BASE + 15)
Failed to find routes to update for a route-resolver.
- #define NETIO_LOG_AR_ALLOC_FAILED_EC (NETIO_LOG_BASE + 16)
Failed to allocate memory for address resolver.
- #define NETIO_LOG_AR_ADD_INVALID_INTERFACE_EC (NETIO_LOG_BASE + 17)
An attempt was made to add an existing interface with a different port resolver to the address resolver.
- #define NETIO_LOG_AR_ADDRESS_RESOLVE_FAILED_EC (NETIO_LOG_BASE + 18)
An interface failed to resolve an address.
- #define NETIO_LOG_AR_PORT_RESOLVE_FAILED_EC (NETIO_LOG_BASE + 19)
An interface failed to resolve a port.
- #define NETIO_LOG_AR_CONTEXT_UNINITIALIZED_EC (NETIO_LOG_BASE + 21)
NETIO_AddressResolver_resolve/get_next was called with an uninitialized context.
- #define NETIO_LOG_AR_INVALID_TOKEN_EC (NETIO_LOG_BASE + 22)
An invalid token was encountered in the address string.
- #define NETIO_LOG_AR_ADDRESS_STRING_EXCEEDED_EC (NETIO_LOG_BASE + 23)
The length of the address exceeded the address buffer passed in.
- #define NETIO_LOG_INVALID_ADDRESS_INDEX_EC (NETIO_LOG_BASE + 24)
An address index was not parsed correctly.
- #define NETIO_LOG_ILLEGAL_ROUTE_OPERATION_EC (NETIO_LOG_BASE + 25)
Illegal route operation attempted.
- #define NETIO_LOG_SET_NAME_EC (UDP_LOG_BASE + 26)
Failed to set the name of a runtime component interface.
- #define NETIO_LOG_PACKET_SET_HEAD_EC (UDP_LOG_BASE + 27)
Failed to set packet's head.
- #define NETIO_LOG_PACKET_SET_TAIL_EC (UDP_LOG_BASE + 28)
Failed to set packet's tail.
- #define NETIO_LOG_PACKET_INITIALIZE_EC (UDP_LOG_BASE + 29)
Failed to initialize packet.
- #define NETIO_LOG_PACKET_OVERFLOW_EC (UDP_LOG_BASE + 30)
Packet payload length overflow detected.
- #define NETIO_LOG_LOOP_DELETE_TABLE_EC (NETIO_LOG_BASE + 100)
Failed to delete a database table for a NETIO loopback interface.
- #define NETIO_LOG_LOOP_CREATE_TABLE_EC (NETIO_LOG_BASE + 101)
Failed to create a database table for a NETIO loopback interface.
- #define NETIO_LOG_LOOP_SELECT_TABLE_EC (NETIO_LOG_BASE + 102)
Failed to select a valid record when sending with the NETIO loopback interface.
- #define NETIO_LOG_LOOP_FWD_EC (NETIO_LOG_BASE + 103)
Failed to send forward with the NETIO loopback interface.
- #define NETIO_LOG_LOOP_BINDX_DB_EC (NETIO_LOG_BASE + 104)
Failed to bind an external interface with the NETIO loopback interface.
- #define NETIO_LOG_LOOP_UNBINDX_DB_EC (NETIO_LOG_BASE + 105)

- Failed to unbind an external interface with the NETIO loopback interface.*

 - #define `NETIO_LOG_LOOP_SET_LENGTH_EC` (`NETIO_LOG_BASE` + 106)
- Failed to set the length of an address sequence.*

 - #define `NETIO_LOG_LOOP_INVALID_PROPERTY_EC` (`NETIO_LOG_BASE` + 107)
- Invalid property passed in when creating a loopback interface.*

 - #define `NETIO_LOG_LOOP_INITIALIZE_FAILED_EC` (`NETIO_LOG_BASE` + 108)
- Failed to initialize the loopback interface.*

 - #define `NETIO_LOG_LOOP_CURSOR_ERROR_EC` (`NETIO_LOG_BASE` + 109)
- An error occurred with a cursor while iterating over a table.*

 - #define `NETIO_LOG_LOOP_INVALID_FACTORY_EC` (`NETIO_LOG_BASE` + 110)
- An invalid factory was passed in to create a loopback interface.*

 - #define `NETIO_LOG_LOOP_INVALID_COMPONENT_EC` (`NETIO_LOG_BASE` + 111)
- An invalid component was passed in to delete a loopback interface.*

 - #define `UDP_LOG_SOCKET_INIT_EC` (`UDP_LOG_BASE` + 200)
- Failed to initialize use of sockets.*

 - #define `UDP_LOG_GETHOSTNAME_EC` (`UDP_LOG_BASE` + 201)
- Failed to get the host address given the host name.*

 - #define `UDP_LOG_SOCKET_SET_MCAST_EC` (`UDP_LOG_BASE` + 202)
- Failed to set a socket for multicast loopback.*

 - #define `UDP_LOG_SOCKET_SET_MCASTIF_EC` (`UDP_LOG_BASE` + 203)
- Failed to set a socket for multicast bound to a specific interface.*

 - #define `UDP_LOG_SOCKET_CREATE_EC` (`UDP_LOG_BASE` + 204)
- Failed to create a new socket, the sysrc code is the OS return code and reason for failure.*

 - #define `UDP_LOG_SOCKET_SETFD_EC` (`UDP_LOG_BASE` + 205)
- Failed to set the close-on-exec flag for a socket.*

 - #define `UDP_LOG_SOCKET_SNDBUF_EC` (`UDP_LOG_BASE` + 206)
- Failed to set the socket send buffer size.*

 - #define `UDP_LOG_SOCKET_TTL_EC` (`UDP_LOG_BASE` + 207)
- Failed to set multicast TTL for a socket.*

 - #define `UDP_LOG_PACKET_INIT_EC` (`UDP_LOG_BASE` + 208)
- Failed to initialize a receive packet.*

 - #define `UDP_LOG_PACKET_HEAD_EC` (`UDP_LOG_BASE` + 209)
- Failed to set the head of a receive packet.*

 - #define `UDP_LOG_PACKET_FWD_EC` (`UDP_LOG_BASE` + 210)
- Failed reception upstream for a received packet.*

 - #define `UDP_LOG_NAT_DELETE_EC` (`UDP_LOG_BASE` + 211)
- Failed to delete a database record or index from an internal NAT.*

 - #define `UDP_LOG_FINALIZE_EC` (`UDP_LOG_BASE` + 212)
- Failed to finalize the parent NETIO interface.*

 - #define `UDP_LOG_SOCKET_REUSE_PORT_EC` (`UDP_LOG_BASE` + 213)
- Failed to set socket to reuse a port or address.*

 - #define `UDP_LOG_SOCKET_BIND_EC` (`UDP_LOG_BASE` + 214)
- Failed a socket bind.*

 - #define `UDP_LOG_SOCKET_ADDGRP_EC` (`UDP_LOG_BASE` + 215)
- Failed to add membership to a multicast group for a socket.*

 - #define `UDP_LOG_PORT_ENTRY_EC` (`UDP_LOG_BASE` + 216)
- Failed to create a bind entry for a receive port.*

- #define `UDP_LOG_PORT_ALLOC_EC` (`UDP_LOG_BASE` + 217)
Failed to allocate memory for a receive buffer of a specific port.
- #define `UDP_LOG_SOCKET_RXBUF_EC` (`UDP_LOG_BASE` + 218)
Failed to set the receive buffer size of a socket.
- #define `UDP_LOG_PORT_ADD_EC` (`UDP_LOG_BASE` + 219)
Failed to add a port entry record to a database table.
- #define `UDP_LOG_PORT_THREAD_EC` (`UDP_LOG_BASE` + 220)
Failed to create a receive thread for a receive port.
- #define `UDP_LOG_SOCKET_IFLIST_EC` (`UDP_LOG_BASE` + 221)
Failed to get a list of interface addresses.
- #define `UDP_LOG_SOCKET_IFFLAGS_EC` (`UDP_LOG_BASE` + 222)
Failed to get the network interface flags or mask of a socket.
- #define `UDP_LOG_SOCKET_CLOSE_EC` (`UDP_LOG_BASE` + 223)
Failed to close a socket.
- #define `UDP_LOG_LOADLIB_EC` (`UDP_LOG_BASE` + 224)
Failed to load a dynamic library.
- #define `UDP_LOG_GET_BUF_SIZE_EC` (`UDP_LOG_BASE` + 225)
Failed to get interface buffer size.
- #define `UDP_LOG_GET_BUF_ALLOC_EC` (`UDP_LOG_BASE` + 226)
Out of memory to allocate interface buffer.
- #define `UDP_LOG_GET_IFLIST_EC` (`UDP_LOG_BASE` + 227)
Failed to get an interface list.
- #define `UDP_LOG_GET_NAMEINFO_EC` (`UDP_LOG_BASE` + 228)
Failed getnameinfo()
- #define `UDP_LOG_MULTICAST_ENABLE_EC` (`UDP_LOG_BASE` + 229)
Failed to enable multicast loopback.
- #define `UDP_LOG_UDP_CREATE_TABLE_EC` (`UDP_LOG_BASE` + 230)
Failed to create a database table for a UDP interface.
- #define `UDP_LOG_UDP_CREATE_INDEX_EC` (`UDP_LOG_BASE` + 231)
Failed to create an index for a UDP interface.
- #define `UDP_LOG_RECORD_EC` (`UDP_LOG_BASE` + 232)
Failed to create or insert a database record for a UDP interface.
- #define `UDP_LOG_PROPERTY_FINALIZE_EC` (`UDP_LOG_BASE` + 233)
Failed to finalize UDP interface factory property.
- #define `UDP_LOG_ALLOC_EC` (`UDP_LOG_BASE` + 234)
Failed to allocate memory for a UDP interface.
- #define `UDP_LOG_SEND_PING_EC` (`UDP_LOG_BASE` + 235)
Failed sending a ping message, upon adding a route or waking up a receive thread.
- #define `UDP_LOG_DROPGRP_EC` (`UDP_LOG_BASE` + 236)
Failed to drop membership from a multicast group.
- #define `UDP_LOG_GET_LENGTH_EC` (`UDP_LOG_BASE` + 237)
Length of address sequence exceeds its maximum.
- #define `UDP_LOG_SET_LENGTH_EC` (`UDP_LOG_BASE` + 240)
Failed to set the length of an address sequence.
- #define `UDP_LOG_WSA_STARTUP_EC` (`UDP_LOG_BASE` + 241)
Failed WSStartup()
- #define `UDP_LOG_WINSOCK_INCOMPATIBLE_EC` (`UDP_LOG_BASE` + 242)

- Incompatible version of Winsock, must not be older than 2.0.*

 - #define `UDP_LOG_INCONSISTENT_MAX_MESSAGE_SIZE_EC` (`UDP_LOG_BASE` + 243)

Inconsistent max_message_size.
- #define `UDP_LOG_INITIALIZE_FAILED_EC` (`UDP_LOG_BASE` + 244)

Failed to initialize the loopback interface.
- #define `UDP_LOG_PORT_NOT_FOUND_EC` (`UDP_LOG_BASE` + 245)

Did not find the specified thread port.
- #define `UDP_LOG_CURSOR_ERROR_EC` (`UDP_LOG_BASE` + 246)

An error occurred while iterating over a cursor.
- #define `UDP_LOG_SEND_ERROR_EC` (`UDP_LOG_BASE` + 247)

Failed to send message.
- #define `UDP_LOG_MULTICAST_NOT_ENABLED_EC` (`UDP_LOG_BASE` + 248)

Multicast address ignored because multicast supported is not compiled in.
- #define `UDP_LOG_TRANSPORT_NO_INTERFACES_EC` (`UDP_LOG_BASE` + 249)

Transport does not have any interfaces.
- #define `UDP_LOG_RECV_ERROR_EC` (`UDP_LOG_BASE` + 250)

Failed to receive message.
- #define `UDP_LOG_SOCKET_PRIORITY_EC` (`UDP_LOG_BASE` + 251)

Failed to set socket priority.
- #define `UDP_LOG_TOS_NOT_SUPPORTED_EC` (`UDP_LOG_BASE` + 252)

IP_TOS is not supported.
- #define `UDP_TRANSFORM_LOG_INVALID_FACTORY_REFERENCE_EC` (`UDP_LOG_BASE` + 300)

The returned factory reference is NULL.
- #define `UDP_TRANSFORM_LOG_FACTORY_NOT_FOUND_EC` (`UDP_LOG_BASE` + 301)

Could not find the factory used as part of a transform rule.
- #define `UDP_TRANSFORM_LOG_ALLOC_EC` (`UDP_LOG_BASE` + 302)

Failed to allocate entry of the specified kind.
- #define `UDP_TRANSFORM_LOG_CREATE_EC` (`UDP_LOG_BASE` + 303)

Failed to create a transformation of the specified kind.
- #define `UDP_TRANSFORM_LOG_ADD_EC` (`UDP_LOG_BASE` + 304)

Failed to add object of specified kind.
- #define `UDP_TRANSFORM_LOG_FIND_EC` (`UDP_LOG_BASE` + 305)

Failed to find a transformation.
- #define `UDP_TRANSFORM_LOG_DUPLICATE_EC` (`UDP_LOG_BASE` + 306)

Duplicate object.
- #define `UDP_TRANSFORM_LOG_TRANSFORM_EC` (`UDP_LOG_BASE` + 307)

Source or destination transformation function failed.
- #define `UDP_TRANSFORM_LOG_INVALID_UDP_MODE_EC` (`UDP_LOG_BASE` + 308)

Invalid UDP mode specified.
- #define `UDP_TRANSFORM_LOG_INVALID_TUDP_LOCATOR_KIND_EC` (`UDP_LOG_BASE` + 309)

Invalid transformation UDP locator kind specified.
- #define `UDP_PRIORITY_MAP_ALLOC_EC` (`UDP_LOG_BASE` + 310)

Failed to allocate memory for priority map.
- #define `UDP_PRIORITY_MAP_INVALID_MAPPING_EC` (`UDP_LOG_BASE` + 311)

Invalid priority mapping specified.
- #define `UDP_PRIORITY_MAP_NEGATIVE_MAPPING_EC` (`UDP_LOG_BASE` + 312)

Negative priority mapping values specified.

- `#define NETIO_SHMEM_LOG_DELETE_TABLE_EC (SHMEM_LOG_BASE + 1)`
Failed to delete a database table for a NETIO shared memory interface.
- `#define NETIO_SHMEM_LOG_CREATE_TABLE_EC (SHMEM_LOG_BASE + 2)`
Failed to create a database table for a NETIO shared memory interface.
- `#define NETIO_SHMEM_LOG_SELECT_TABLE_EC (SHMEM_LOG_BASE + 3)`
Failed to select a valid record when sending with the NETIO shared memory.
- `#define NETIO_SHMEM_LOG_FWD_EC (SHMEM_LOG_BASE + 4)`
Failed to send forward with the NETIO shared memory interface.
- `#define NETIO_SHMEM_LOG_BINDX_DB_EC (SHMEM_LOG_BASE + 5)`
Failed to bind an external interface with the NETIO shared memory interface.
- `#define NETIO_SHMEM_LOG_UNBINDX_DB_EC (SHMEM_LOG_BASE + 6)`
Failed to unbind an external interface with the NETIO shared memory interface.
- `#define NETIO_SHMEM_LOG_SET_LENGTH_EC (SHMEM_LOG_BASE + 7)`
Failed to set the length of an address sequence.
- `#define NETIO_SHMEM_LOG_INVALID_PROPERTY_EC (SHMEM_LOG_BASE + 8)`
Invalid property passed in when creating a shared memory interface.
- `#define NETIO_SHMEM_LOG_INITIALIZE_FAILED_EC (SHMEM_LOG_BASE + 9)`
Failed to initialize the shared memory interface.
- `#define NETIO_SHMEM_LOG_CURSOR_ERROR_EC (SHMEM_LOG_BASE + 10)`
An error occurred with a cursor while iterating over a table.
- `#define NETIO_SHMEM_LOG_INVALID_FACTORY_EC (SHMEM_LOG_BASE + 11)`
An invalid factory was passed in to create a shared memory interface.
- `#define NETIO_SHMEM_LOG_INVALID_COMPONENT_EC (SHMEM_LOG_BASE + 12)`
An invalid component was passed in to delete a shared memory interface.
- `#define NETIO_SHMEM_LOG_INCOMPATIBLE_COOKIE_EC (SHMEM_LOG_BASE + 13)`
Found incompatible cookie in shared memory header.
- `#define NETIO_SHMEM_LOG_INCOMPATIBLE_VERSION_EC (SHMEM_LOG_BASE + 14)`
Found incompatible major version in shared memory header.
- `#define NETIO_SHMEM_LOG_INCOMPATIBLE_HEADER_SIZE_EC (SHMEM_LOG_BASE + 15)`
Found incompatible header size in shared memory header.
- `#define NETIO_SHMEM_LOG_INCOMPATIBLE_HEADER_EC (SHMEM_LOG_BASE + 16)`
Attempted to attach to shared memory header transport concurrent queue that is incompatible.
- `#define NETIO_SHMEM_LOG_ADDRESS_IN_USE_EC (SHMEM_LOG_BASE + 17)`
Failed to create or attach to a shared memory segment.
- `#define NETIO_SHMEM_LOG_CHECK_SYSTEM_SETTINGS_EC (SHMEM_LOG_BASE + 18)`
Possible failure due to misconfigured system settings on Darwin operating systems.
- `#define NETIO_SHMEM_LOG_SHMEM_MUTEX_LOCK_FAILED_EC (SHMEM_LOG_BASE + 19)`
Failed to lock shared memory mutex protecting the shared memory segment.
- `#define NETIO_SHMEM_LOG_SHMEM_MUTEX_UNLOCK_FAILED_EC (SHMEM_LOG_BASE + 20)`
Failed to unlock shared memory mutex protecting the shared memory segment.
- `#define NETIO_SHMEM_LOG_FAILED_TO_ALLOCATE_STRUCT_EC (SHMEM_LOG_BASE + 21)`
Failed to allocate memory needed by shared memory transport.
- `#define NETIO_SHMEM_LOG_FAILED_TO_INITIALIZE_EC (SHMEM_LOG_BASE + 22)`
Failed to initialize shared memory transport.
- `#define NETIO_SHMEM_LOG_CREATE_ATTACH_INFINITE_EC (SHMEM_LOG_BASE + 23)`
Failed to create or attach. Possible shared memory key conflict.
- `#define NETIO_SHMEM_LOG_FAILED_TO_INIT_MUTEX_EC (SHMEM_LOG_BASE + 24)`

- Failed to create a new shared memory mutex or failed to attach to an existing one.*

 - #define `NETIO_SHMEM_LOG_FAILED_TO_INIT_SIG_SEM_EC` (`SHMEM_LOG_BASE` + 25)

Failed to create signaling semaphore. A Domain Participant's shared memory transport blocks on a signaling semaphore when waiting for data.
- #define `NETIO_SHMEM_LOG_FAILED_TAKE_SIGN_SEM_EC` (`SHMEM_LOG_BASE` + 27)

Failed to take a shared memory signaling semaphore.
- #define `NETIO_SHMEM_LOG_SIG_SEM_SIGNAL_FAILED_EC` (`SHMEM_LOG_BASE` + 28)

Failed to give a shared memory signaling semaphore.
- #define `NETIO_SHMEM_LOG_FAILED_TO_INIT_SEGMENT_ADDR_EC` (`SHMEM_LOG_BASE` + 29)

Failed to attach to a shared memory segment because it doesn't exist.
- #define `NETIO_SHMEM_LOG_PACKET_INIT_EC` (`SHMEM_LOG_BASE` + 30)

Failed to initialize a receive packet.
- #define `NETIO_SHMEM_LOG_FAILED_TO_INIT_CONCURRENT_Q_EC` (`SHMEM_LOG_BASE` + 31)

Failed to create a concurrent queue.
- #define `NETIO_SHMEM_LOG_FAILED_TO_ATTACH_CONCURRENT_Q_EC` (`SHMEM_LOG_BASE` + 32)

Failed to attach to a concurrent queue.
- #define `NETIO_SHMEM_LOG_INCOMPATIBLE_CONCURRENT_Q_EC` (`SHMEM_LOG_BASE` + 33)

Attempting to attach to an incompatible concurrent queue with incompatible message_size_max.
- #define `NETIO_SHMEM_LOG_FAILED_TO_GET_TIMESTAMP_EC` (`SHMEM_LOG_BASE` + 33)

Failed to get timestamp.
- #define `NETIO_SHMEM_LOG_NONFATAL_SIG_SEM_RC_EC` (`SHMEM_LOG_BASE` + 34)

Failed to unlock shared memory mutex.
- #define `NETIO_SHMEM_LOG_INCONSISTENT_MESSAGE_SIZE_EC` (`SHMEM_LOG_BASE` + 35)

Failed to unlock shared memory mutex.
- #define `NETIO_SHMEM_LOG_RECORD_EC` (`SHMEM_LOG_BASE` + 36)

Database operation on record failed.
- #define `NETIO_SHMEM_LOG_PACKET_HEAD_EC` (`SHMEM_LOG_BASE` + 37)

Failed to set packet head.
- #define `NETIO_SHMEM_LOG_PACKET_FWD_EC` (`SHMEM_LOG_BASE` + 38)

Failed reception upstream for a received packet.
- #define `NETIO_SHMEM_LOG_QUEUE_FULL_EC` (`SHMEM_LOG_BASE` + 39)

Failed reception upstream for a received packet.
- #define `NETIO_SHMEM_LOG_ROUTE_LOOKUP_FAILED_EC` (`SHMEM_LOG_BASE` + 40)

Failed to evict find an entry in the route table.
- #define `NETIO_SHMEM_LOG_ROUTE_DELETE_FAILED_EC` (`SHMEM_LOG_BASE` + 41)

Failed to delete an entry from the route table.
- #define `NETIO_SHMEM_LOG_IGNORE_TRANSPORT_COMPATIBILITY_EC` (`SHMEM_LOG_BASE` + 42)

The shared memory transport is ignoring the requested transport minimum compatibility version because it cannot be backward compatible.
- #define `NETIO_SHMEM_LOG_UNSUPPORTED_TRANSPORT_COMPATIBILITY_EC` (`SHMEM_LOG_BASE` + 43)

Unsupported minimum compatibility version because it requires a version of the shared memory transport which is not supported.

6.20.7.1 Detailed Description

Network I/O. ModuleID = 4.

6.20.7.2 Macro Definition Documentation

NETIO_LOG_GETHOST_BYNAME_EC

```
#define NETIO_LOG_GETHOST_BYNAME_EC (NETIO_LOG_BASE + 1)
```

Failed to get host address from a string representation.

NETIO_LOG_BIND_NEW_EC

```
#define NETIO_LOG_BIND_NEW_EC (NETIO_LOG_BASE + 2)
```

Failed to allocate memory for a bind-resolver.

NETIO_LOG_BIND_NEW_RTABLE_EC

```
#define NETIO_LOG_BIND_NEW_RTABLE_EC (NETIO_LOG_BASE + 3)
```

Failed to create a database table for routes of a bind-resolver.

NETIO_LOG_BIND_CREATE_RTABLE_EC

```
#define NETIO_LOG_BIND_CREATE_RTABLE_EC (NETIO_LOG_BASE + 4)
```

Failed to create route record.

May have exceeded DomainParticipantQos.resource_limits.max_receive_ports or DomainParticipantQos.resource_↵limits.local_writer_allocation

NETIO_LOG_BIND_DELETE_RTABLE_EC

```
#define NETIO_LOG_BIND_DELETE_RTABLE_EC (NETIO_LOG_BASE + 5)
```

Failed to delete a database table for routes of a bind_resolver.

NETIO_LOG_BIND_SET_LENGTH_EC

```
#define NETIO_LOG_BIND_SET_LENGTH_EC (NETIO_LOG_BASE + 6)
```

Failed to set the length for a sequence of resolved addresses.

NETIO_LOG_BIND_SET_MAX_EC

```
#define NETIO_LOG_BIND_SET_MAX_EC (NETIO_LOG_BASE + 7)
```

Failed to set the maximum length of an internal sequence of addresses.

NETIO_LOG_BIND_EXTERNAL_FAILED_EC

```
#define NETIO_LOG_BIND_EXTERNAL_FAILED_EC (NETIO_LOG_BASE + 8)
```

An interface failed on bind_external.

NETIO_LOG_UNBIND_EXTERNAL_FAILED_EC

```
#define NETIO_LOG_UNBIND_EXTERNAL_FAILED_EC (NETIO_LOG_BASE + 10)
```

An interface failed on unbind_external.

NETIO_LOG_ROUTE_RTABLE_ALLOC_EC

```
#define NETIO_LOG_ROUTE_RTABLE_ALLOC_EC (NETIO_LOG_BASE + 11)
```

Failed to allocate memory for a route resolver.

NETIO_LOG_ROUTE_RTABLE_CREATE_EC

```
#define NETIO_LOG_ROUTE_RTABLE_CREATE_EC (NETIO_LOG_BASE + 12)
```

Failed to create a database table for routes of a route-resolver.

NETIO_LOG_ROUTE_RTABLE_DELETE_EC

```
#define NETIO_LOG_ROUTE_RTABLE_DELETE_EC (NETIO_LOG_BASE + 13)
```

Failed to delete a database table for routes of a route-resolver.

NETIO_LOG_ROUTE_RTABLE_ADD_EC

```
#define NETIO_LOG_ROUTE_RTABLE_ADD_EC (NETIO_LOG_BASE + 14)
```

Failed to create route resolver record.

May have exceeded DomainParticipantQos.resource_limits.max_destination_ports

NETIO_LOG_ROUTE_RTABLE_UPDATE_EC

```
#define NETIO_LOG_ROUTE_RTABLE_UPDATE_EC (NETIO_LOG_BASE + 15)
```

Failed to find routes to update for a route-resolver.

NETIO_LOG_AR_ALLOC_FAILED_EC

```
#define NETIO_LOG_AR_ALLOC_FAILED_EC (NETIO_LOG_BASE + 16)
```

Failed to allocate memory for address resolver.

NETIO_LOG_AR_ADD_INVALID_INTERFACE_EC

```
#define NETIO_LOG_AR_ADD_INVALID_INTERFACE_EC (NETIO_LOG_BASE + 17)
```

An attempt was made to add an existing interface with a different port resolver to the address resolver.

NETIO_LOG_AR_ADDRESS_RESOLVE_FAILED_EC

```
#define NETIO_LOG_AR_ADDRESS_RESOLVE_FAILED_EC (NETIO_LOG_BASE + 18)
```

An interface failed to resolve an address.

NETIO_LOG_AR_PORT_RESOLVE_FAILED_EC

```
#define NETIO_LOG_AR_PORT_RESOLVE_FAILED_EC (NETIO_LOG_BASE + 19)
```

An interface failed to resolve a port.

NETIO_LOG_AR_CONTEXT_UNINITIALIZED_EC

```
#define NETIO_LOG_AR_CONTEXT_UNINITIALIZED_EC (NETIO_LOG_BASE + 21)
```

NETIO_AddressResolver_resolve/get_next was called with an uninitialized context.

NETIO_LOG_AR_INVALID_TOKEN_EC

```
#define NETIO_LOG_AR_INVALID_TOKEN_EC (NETIO_LOG_BASE + 22)
```

An invalid token was encountered in the address string.

NETIO_LOG_AR_ADDRESS_STRING_EXCEEDED_EC

```
#define NETIO_LOG_AR_ADDRESS_STRING_EXCEEDED_EC (NETIO_LOG_BASE + 23)
```

The length of the address exceeded the address buffer passed in.

NETIO_LOG_INVALID_ADDRESS_INDEX_EC

```
#define NETIO_LOG_INVALID_ADDRESS_INDEX_EC (NETIO_LOG_BASE + 24)
```

An address index was not parsed correctly.

When parsing an address string the index was not specified correctly. Legal formats are: N@ [,N]@ [N]@ [M,N]@

Where N and M are integers ≥ 0

NETIO_LOG_ILLEGAL_ROUTE_OPERATION_EC

```
#define NETIO_LOG_ILLEGAL_ROUTE_OPERATION_EC (NETIO_LOG_BASE + 25)
```

Illegal route operation attempted.

NETIO_LOG_SET_NAME_EC

```
#define NETIO_LOG_SET_NAME_EC (UDP_LOG_BASE + 26)
```

Failed to set the name of a runtime component interface.

NETIO_LOG_PACKET_SET_HEAD_EC

```
#define NETIO_LOG_PACKET_SET_HEAD_EC (UDP_LOG_BASE + 27)
```

Failed to set packet's head.

NETIO_Packet_set_head() failed, due to an invalid delta that would have moved the head position out-of-bounds of the packet.

NETIO_LOG_PACKET_SET_TAIL_EC

```
#define NETIO_LOG_PACKET_SET_TAIL_EC (UDP_LOG_BASE + 28)
```

Failed to set packet's tail.

NETIO_Packet_set_tail() failed, due to an invalid delta that would have moved the tail position out-of-bounds of the packet.

NETIO_LOG_PACKET_INITIALIZE_EC

```
#define NETIO_LOG_PACKET_INITIALIZE_EC (UDP_LOG_BASE + 29)
```

Failed to initialize packet.

NETIO_Packet_initialize() failed due to invalid arguments

NETIO_LOG_PACKET_OVERFLOW_EC

```
#define NETIO_LOG_PACKET_OVERFLOW_EC (UDP_LOG_BASE + 30)
```

Packet payload length overflow detected.

NETIO_Packet_get_payload_length() detected an integer overflow while accumulating lengths from packet buffer linked list.

NETIO_LOG_LOOP_DELETE_TABLE_EC

```
#define NETIO_LOG_LOOP_DELETE_TABLE_EC (NETIO_LOG_BASE + 100)
```

Failed to delete a database table for a NETIO loopback interface.

NETIO_LOG_LOOP_CREATE_TABLE_EC

```
#define NETIO_LOG_LOOP_CREATE_TABLE_EC (NETIO_LOG_BASE + 101)
```

Failed to create a database table for a NETIO loopback interface.

NETIO_LOG_LOOP_SELECT_TABLE_EC

```
#define NETIO_LOG_LOOP_SELECT_TABLE_EC (NETIO_LOG_BASE + 102)
```

Failed to select a valid record when sending with the NETIO loopback interface.

NETIO_LOG_LOOP_FWD_EC

```
#define NETIO_LOG_LOOP_FWD_EC (NETIO_LOG_BASE + 103)
```

Failed to send forward with the NETIO loopback interface.

NETIO_LOG_LOOP_BINDX_DB_EC

```
#define NETIO_LOG_LOOP_BINDX_DB_EC (NETIO_LOG_BASE + 104)
```

Failed to bind an external interface with the NETIO loopback interface.

NETIO_LOG_LOOP_UNBINDX_DB_EC

```
#define NETIO_LOG_LOOP_UNBINDX_DB_EC (NETIO_LOG_BASE + 105)
```

Failed to unbind an external interface with the NETIO loopback interface.

NETIO_LOG_LOOP_SET_LENGTH_EC

```
#define NETIO_LOG_LOOP_SET_LENGTH_EC (NETIO_LOG_BASE + 106)
```

Failed to set the length of an address sequence.

NETIO_LOG_LOOP_INVALID_PROPERTY_EC

```
#define NETIO_LOG_LOOP_INVALID_PROPERTY_EC (NETIO_LOG_BASE + 107)
```

Invalid property passed in when creating a loopback interface.

NETIO_LOG_LOOP_INITIALIZE_FAILED_EC

```
#define NETIO_LOG_LOOP_INITIALIZE_FAILED_EC (NETIO_LOG_BASE + 108)
```

Failed to initialize the loopback interface.

NETIO_LOG_LOOP_CURSOR_ERROR_EC

```
#define NETIO_LOG_LOOP_CURSOR_ERROR_EC (NETIO_LOG_BASE + 109)
```

An error occurred with a cursor while iterating over a table.

NETIO_LOG_LOOP_INVALID_FACTORY_EC

```
#define NETIO_LOG_LOOP_INVALID_FACTORY_EC (NETIO_LOG_BASE + 110)
```

An invalid factory was passed in to create a loopback interface.

NETIO_LOG_LOOP_INVALID_COMPONENT_EC

```
#define NETIO_LOG_LOOP_INVALID_COMPONENT_EC (NETIO_LOG_BASE + 111)
```

An invalid component was passed in to delete a loopback interface.

UDP_LOG_SOCKET_INIT_EC

```
#define UDP_LOG_SOCKET_INIT_EC (UDP_LOG_BASE + 200)
```

Failed to initialize use of sockets.

UDP_LOG_GETHOSTNAME_EC

```
#define UDP_LOG_GETHOSTNAME_EC (UDP_LOG_BASE + 201)
```

Failed to get the host address given the host name.

UDP_LOG_SOCKET_SET_MCAST_EC

```
#define UDP_LOG_SOCKET_SET_MCAST_EC (UDP_LOG_BASE + 202)
```

Failed to set a socket for multicast loopback.

UDP_LOG_SOCKET_SET_MCASTIF_EC

```
#define UDP_LOG_SOCKET_SET_MCASTIF_EC (UDP_LOG_BASE + 203)
```

Failed to set a socket for multicast bound to a specific interface.

UDP_LOG_SOCKET_CREATE_EC

```
#define UDP_LOG_SOCKET_CREATE_EC (UDP_LOG_BASE + 204)
```

Failed to create a new socket, the sysrc code is the OS return code and reason for failure.

UDP_LOG_SOCKET_SETFD_EC

```
#define UDP_LOG_SOCKET_SETFD_EC (UDP_LOG_BASE + 205)
```

Failed to set the close-on-exec flag for a socket.

UDP_LOG_SOCKET_SNDBUF_EC

```
#define UDP_LOG_SOCKET_SNDBUF_EC (UDP_LOG_BASE + 206)
```

Failed to set the socket send buffer size.

UDP_LOG_SOCKET_TTL_EC

```
#define UDP_LOG_SOCKET_TTL_EC (UDP_LOG_BASE + 207)
```

Failed to set multicast TTL for a socket.

UDP_LOG_PACKET_INIT_EC

```
#define UDP_LOG_PACKET_INIT_EC (UDP_LOG_BASE + 208)
```

Failed to initialize a receive packet.

UDP_LOG_PACKET_HEAD_EC

```
#define UDP_LOG_PACKET_HEAD_EC (UDP_LOG_BASE + 209)
```

Failed to set the head of a receive packet.

UDP_LOG_PACKET_FWD_EC

```
#define UDP_LOG_PACKET_FWD_EC (UDP_LOG_BASE + 210)
```

Failed reception upstream for a received packet.

UDP_LOG_NAT_DELETE_EC

```
#define UDP_LOG_NAT_DELETE_EC (UDP_LOG_BASE + 211)
```

Failed to delete a database record or index from an internal NAT.

UDP_LOG_FINALIZE_EC

```
#define UDP_LOG_FINALIZE_EC (UDP_LOG_BASE + 212)
```

Failed to finalize the parent NETIO interface.

UDP_LOG_SOCKET_REUSE_PORT_EC

```
#define UDP_LOG_SOCKET_REUSE_PORT_EC (UDP_LOG_BASE + 213)
```

Failed to set socket to reuse a port or address.

UDP_LOG_SOCKET_BIND_EC

```
#define UDP_LOG_SOCKET_BIND_EC (UDP_LOG_BASE + 214)
```

Failed a socket bind.

UDP_LOG_SOCKET_ADDGRP_EC

```
#define UDP_LOG_SOCKET_ADDGRP_EC (UDP_LOG_BASE + 215)
```

Failed to add membership to a multicast group for a socket.

UDP_LOG_PORT_ENTRY_EC

```
#define UDP_LOG_PORT_ENTRY_EC (UDP_LOG_BASE + 216)
```

Failed to create a bind entry for a receive port.

UDP_LOG_PORT_ALLOC_EC

```
#define UDP_LOG_PORT_ALLOC_EC (UDP_LOG_BASE + 217)
```

Failed to allocate memory for a receive buffer of a specific port.

UDP_LOG_SOCKET_RXBUF_EC

```
#define UDP_LOG_SOCKET_RXBUF_EC (UDP_LOG_BASE + 218)
```

Failed to set the receive buffer size of a socket.

UDP_LOG_PORT_ADD_EC

```
#define UDP_LOG_PORT_ADD_EC (UDP_LOG_BASE + 219)
```

Failed to add a port entry record to a database table.

UDP_LOG_PORT_THREAD_EC

```
#define UDP_LOG_PORT_THREAD_EC (UDP_LOG_BASE + 220)
```

Failed to create a receive thread for a receive port.

UDP_LOG_SOCKET_IFLIST_EC

```
#define UDP_LOG_SOCKET_IFLIST_EC (UDP_LOG_BASE + 221)
```

Failed to get a list of interface addresses.

UDP_LOG_SOCKET_IFFLAGS_EC

```
#define UDP_LOG_SOCKET_IFFLAGS_EC (UDP_LOG_BASE + 222)
```

Failed to get the network interface flags or mask of a socket.

UDP_LOG_SOCKET_CLOSE_EC

```
#define UDP_LOG_SOCKET_CLOSE_EC (UDP_LOG_BASE + 223)
```

Failed to close a socket.

UDP_LOG_LOADLIB_EC

```
#define UDP_LOG_LOADLIB_EC (UDP_LOG_BASE + 224)
```

Failed to load a dynamic library.

UDP_LOG_GET_BUF_SIZE_EC

```
#define UDP_LOG_GET_BUF_SIZE_EC (UDP_LOG_BASE + 225)
```

Failed to get interface buffer size.

UDP_LOG_GET_BUF_ALLOC_EC

```
#define UDP_LOG_GET_BUF_ALLOC_EC (UDP_LOG_BASE + 226)
```

Out of memory to allocate interface buffer.

UDP_LOG_GET_IFLIST_EC

```
#define UDP_LOG_GET_IFLIST_EC (UDP_LOG_BASE + 227)
```

Failed to get an interface list.

UDP_LOG_GET_NAMEINFO_EC

```
#define UDP_LOG_GET_NAMEINFO_EC (UDP_LOG_BASE + 228)
```

Failed getnameinfo()

UDP_LOG_MULTICAST_ENABLE_EC

```
#define UDP_LOG_MULTICAST_ENABLE_EC (UDP_LOG_BASE + 229)
```

Failed to enable multicast loopback.

UDP_LOG_UDP_CREATE_TABLE_EC

```
#define UDP_LOG_UDP_CREATE_TABLE_EC (UDP_LOG_BASE + 230)
```

Failed to create a database table for a UDP interface.

UDP_LOG_UDP_CREATE_INDEX_EC

```
#define UDP_LOG_UDP_CREATE_INDEX_EC (UDP_LOG_BASE + 231)
```

Failed to create an index for a UDP interface.

UDP_LOG_RECORD_EC

```
#define UDP_LOG_RECORD_EC (UDP_LOG_BASE + 232)
```

Failed to create or insert a database record for a UDP interface.

UDP_LOG_PROPERTY_FINALIZE_EC

```
#define UDP_LOG_PROPERTY_FINALIZE_EC (UDP_LOG_BASE + 233)
```

Failed to finalize UDP interface factory property.

UDP_LOG_ALLOC_EC

```
#define UDP_LOG_ALLOC_EC (UDP_LOG_BASE + 234)
```

Failed to allocate memory for a UDP interface.

UDP_LOG_SEND_PING_EC

```
#define UDP_LOG_SEND_PING_EC (UDP_LOG_BASE + 235)
```

Failed sending a ping message, upon adding a route or waking up a receive thread.

UDP_LOG_DROPGRP_EC

```
#define UDP_LOG_DROPGRP_EC (UDP_LOG_BASE + 236)
```

Failed to drop membership from a multicast group.

UDP_LOG_GET_LENGTH_EC

```
#define UDP_LOG_GET_LENGTH_EC (UDP_LOG_BASE + 237)
```

Length of address sequence exceeds its maximum.

UDP_LOG_SET_LENGTH_EC

```
#define UDP_LOG_SET_LENGTH_EC (UDP_LOG_BASE + 240)
```

Failed to set the length of an address sequence.

UDP_LOG_WSA_STARTUP_EC

```
#define UDP_LOG_WSA_STARTUP_EC (UDP_LOG_BASE + 241)
```

Failed WSStartup()

UDP_LOG_WINSOCK_INCOMPATIBLE_EC

```
#define UDP_LOG_WINSOCK_INCOMPATIBLE_EC (UDP_LOG_BASE + 242)
```

Incompatible version of Winsock, must not be older than 2.0.

UDP_LOG_INCONSISTENT_MAX_MESSAGE_SIZE_EC

```
#define UDP_LOG_INCONSISTENT_MAX_MESSAGE_SIZE_EC (UDP_LOG_BASE + 243)
```

Inconsistent max_message_size.

UDP_InterfaceProperty.max_message_size is set to an inconsistent value, larger than UDP_InterfaceProperty.max_send_buffer_size and/or larger than UDP_InterfaceProperty.max_receive_buffer_size .

UDP_LOG_INITIALIZE_FAILED_EC

```
#define UDP_LOG_INITIALIZE_FAILED_EC (UDP_LOG_BASE + 244)
```

Failed to initialize the loopback interface.

UDP_LOG_PORT_NOT_FOUND_EC

```
#define UDP_LOG_PORT_NOT_FOUND_EC (UDP_LOG_BASE + 245)
```

Did not find the specified thread port.

UDP_LOG_CURSOR_ERROR_EC

```
#define UDP_LOG_CURSOR_ERROR_EC (UDP_LOG_BASE + 246)
```

An error occurred while iterating over a cursor.

UDP_LOG_SEND_ERROR_EC

```
#define UDP_LOG_SEND_ERROR_EC (UDP_LOG_BASE + 247)
```

Failed to send message.

A message could not be sent due to system error

UDP_LOG_MULTICAST_NOT_ENABLED_EC

```
#define UDP_LOG_MULTICAST_NOT_ENABLED_EC (UDP_LOG_BASE + 248)
```

Multicast address ignored because multicast support is not compiled in.

A multicast address was specified as an address, but multicast support has not been enabled with NETIO_CONFIG_ENABLE_MULTICAST in netio_config.h

UDP_LOG_TRANSPORT_NO_INTERFACES_EC

```
#define UDP_LOG_TRANSPORT_NO_INTERFACES_EC (UDP_LOG_BASE + 249)
```

Transport does not have any interfaces.

UDP configuration might be wrong.

UDP_LOG_RECV_ERROR_EC

```
#define UDP_LOG_RECV_ERROR_EC (UDP_LOG_BASE + 250)
```

Failed to receive message.

A message could not be received due to system error

UDP_LOG_SOCKET_PRIORITY_EC

```
#define UDP_LOG_SOCKET_PRIORITY_EC (UDP_LOG_BASE + 251)
```

Failed to set socket priority.

A socket priority could not be set due to system error

UDP_LOG_TOS_NOT_SUPPORTED_EC

```
#define UDP_LOG_TOS_NOT_SUPPORTED_EC (UDP_LOG_BASE + 252)
```

IP_TOS is not supported.

The system does not support setting the IP_TOS (Type of Service) field for UDP sockets.

UDP_TRANSFORM_LOG_INVALID_FACTORY_REFERENCE_EC

```
#define UDP_TRANSFORM_LOG_INVALID_FACTORY_REFERENCE_EC (UDP_LOG_BASE + 300)
```

The returned factory reference is NULL.

This error indicates that the system has run out of memory

UDP_TRANSFORM_LOG_FACTORY_NOT_FOUND_EC

```
#define UDP_TRANSFORM_LOG_FACTORY_NOT_FOUND_EC (UDP_LOG_BASE + 301)
```

Could not find the factory used as part of a transform rule.

This error indicates that a factory for the specified rule has not been registered before the UDP transport is registered.

UDP_TRANSFORM_LOG_ALLOC_EC

```
#define UDP_TRANSFORM_LOG_ALLOC_EC (UDP_LOG_BASE + 302)
```

Failed to allocate entry of the specified kind.

UDP_TRANSFORM_LOG_CREATE_EC

```
#define UDP_TRANSFORM_LOG_CREATE_EC (UDP_LOG_BASE + 303)
```

Failed to create a transformation of the specified kind.

UDP_TRANSFORM_LOG_ADD_EC

```
#define UDP_TRANSFORM_LOG_ADD_EC (UDP_LOG_BASE + 304)
```

Failed to add object of specified kind.

UDP_TRANSFORM_LOG_FIND_EC

```
#define UDP_TRANSFORM_LOG_FIND_EC (UDP_LOG_BASE + 305)
```

Failed to find a transformation.

UDP_TRANSFORM_LOG_DUPLICATE_EC

```
#define UDP_TRANSFORM_LOG_DUPLICATE_EC (UDP_LOG_BASE + 306)
```

Duplicate object.

UDP_TRANSFORM_LOG_TRANSFORM_EC

```
#define UDP_TRANSFORM_LOG_TRANSFORM_EC (UDP_LOG_BASE + 307)
```

Source or destination transformation function failed.

UDP_TRANSFORM_LOG_INVALID_UDP_MODE_EC

```
#define UDP_TRANSFORM_LOG_INVALID_UDP_MODE_EC (UDP_LOG_BASE + 308)
```

Invalid UDP mode specified.

UDP_TRANSFORM_LOG_INVALID_TUDP_LOCATOR_KIND_EC

```
#define UDP_TRANSFORM_LOG_INVALID_TUDP_LOCATOR_KIND_EC (UDP_LOG_BASE + 309)
```

Invalid transformation UDP locator kind specified.

[UDP_InterfaceFactoryProperty::transform_locator_kind](#) must be greater or equal than `NETIO_ADDRESS_KIND_USER` and not equal to `NETIO_ADDRESS_KIND_SHMEM` or `NETIO_ADDRESS_KIND_INTRA`.

UDP_PRIORITY_MAP_ALLOC_EC

```
#define UDP_PRIORITY_MAP_ALLOC_EC (UDP_LOG_BASE + 310)
```

Failed to allocate memory for priority map.

UDP_PRIORITY_MAP_INVALID_MAPPING_EC

```
#define UDP_PRIORITY_MAP_INVALID_MAPPING_EC (UDP_LOG_BASE + 311)
```

Invalid priority mapping specified.

The specified priority mapping is invalid, the low priority mapping must be less than or equal to the high priority mapping.

UDP_PRIORITY_MAP_NEGATIVE_MAPPING_EC

```
#define UDP_PRIORITY_MAP_NEGATIVE_MAPPING_EC (UDP_LOG_BASE + 312)
```

Negative priority mapping values specified.

The specified priority mapping values are invalid, both low and high priority mapping values must be non-negative (0-255).

NETIO_SHMEM_LOG_DELETE_TABLE_EC

```
#define NETIO_SHMEM_LOG_DELETE_TABLE_EC (SHMEM_LOG_BASE + 1)
```

Failed to delete a database table for a NETIO shared memory interface.

NETIO_SHMEM_LOG_CREATE_TABLE_EC

```
#define NETIO_SHMEM_LOG_CREATE_TABLE_EC (SHMEM_LOG_BASE + 2)
```

Failed to create a database table for a NETIO shared memory interface.

NETIO_SHMEM_LOG_SELECT_TABLE_EC

```
#define NETIO_SHMEM_LOG_SELECT_TABLE_EC (SHMEM_LOG_BASE + 3)
```

Failed to select a valid record when sending with the NETIO shared memory.

NETIO_SHMEM_LOG_FWD_EC

```
#define NETIO_SHMEM_LOG_FWD_EC (SHMEM_LOG_BASE + 4)
```

Failed to send forward with the NETIO shared memory interface.

NETIO_SHMEM_LOG_BINDX_DB_EC

```
#define NETIO_SHMEM_LOG_BINDX_DB_EC (SHMEM_LOG_BASE + 5)
```

Failed to bind an external interface with the NETIO shared memory interface.

NETIO_SHMEM_LOG_UNBINDX_DB_EC

```
#define NETIO_SHMEM_LOG_UNBINDX_DB_EC (SHMEM_LOG_BASE + 6)
```

Failed to unbind an external interface with the NETIO shared memory interface.

NETIO_SHMEM_LOG_SET_LENGTH_EC

```
#define NETIO_SHMEM_LOG_SET_LENGTH_EC (SHMEM_LOG_BASE + 7)
```

Failed to set the length of an address sequence.

NETIO_SHMEM_LOG_INVALID_PROPERTY_EC

```
#define NETIO_SHMEM_LOG_INVALID_PROPERTY_EC (SHMEM_LOG_BASE + 8)
```

Invalid property passed in when creating a shared memory interface.

NETIO_SHMEM_LOG_INITIALIZE_FAILED_EC

```
#define NETIO_SHMEM_LOG_INITIALIZE_FAILED_EC (SHMEM_LOG_BASE + 9)
```

Failed to initialize the shared memory interface.

NETIO_SHMEM_LOG_CURSOR_ERROR_EC

```
#define NETIO_SHMEM_LOG_CURSOR_ERROR_EC (SHMEM_LOG_BASE + 10)
```

An error occurred with a cursor while iterating over a table.

NETIO_SHMEM_LOG_INVALID_FACTORY_EC

```
#define NETIO_SHMEM_LOG_INVALID_FACTORY_EC (SHMEM_LOG_BASE + 11)
```

An invalid factory was passed in to create a shared memory interface.

NETIO_SHMEM_LOG_INVALID_COMPONENT_EC

```
#define NETIO_SHMEM_LOG_INVALID_COMPONENT_EC (SHMEM_LOG_BASE + 12)
```

An invalid component was passed in to delete a shared memory interface.

NETIO_SHMEM_LOG_INCOMPATIBLE_COOKIE_EC

```
#define NETIO_SHMEM_LOG_INCOMPATIBLE_COOKIE_EC (SHMEM_LOG_BASE + 13)
```

Found incompatible cookie in shared memory header.

NETIO_SHMEM_LOG_INCOMPATIBLE_VERSION_EC

```
#define NETIO_SHMEM_LOG_INCOMPATIBLE_VERSION_EC (SHMEM_LOG_BASE + 14)
```

Found incompatible major version in shared memory header.

NETIO_SHMEM_LOG_INCOMPATIBLE_HEADER_SIZE_EC

```
#define NETIO_SHMEM_LOG_INCOMPATIBLE_HEADER_SIZE_EC (SHMEM_LOG_BASE + 15)
```

Found incompatible header size in shared memory header.

NETIO_SHMEM_LOG_INCOMPATIBLE_HEADER_EC

```
#define NETIO_SHMEM_LOG_INCOMPATIBLE_HEADER_EC (SHMEM_LOG_BASE + 16)
```

Attempted to attach to shared memory header transport concurrent queue that is incompatible.

NETIO_SHMEM_LOG_ADDRESS_IN_USE_EC

```
#define NETIO_SHMEM_LOG_ADDRESS_IN_USE_EC (SHMEM_LOG_BASE + 17)
```

Failed to create or attach to a shared memory segment.

NETIO_SHMEM_LOG_CHECK_SYSTEM_SETTINGS_EC

```
#define NETIO_SHMEM_LOG_CHECK_SYSTEM_SETTINGS_EC (SHMEM_LOG_BASE + 18)
```

Possible failure due to misconfigured system settings on Darwin operating systems.

NETIO_SHMEM_LOG_SHMEM_MUTEX_LOCK_FAILED_EC

```
#define NETIO_SHMEM_LOG_SHMEM_MUTEX_LOCK_FAILED_EC (SHMEM_LOG_BASE + 19)
```

Failed to lock shared memory mutex protecting the shared memory segment.

NETIO_SHMEM_LOG_SHMEM_MUTEX_UNLOCK_FAILED_EC

```
#define NETIO_SHMEM_LOG_SHMEM_MUTEX_UNLOCK_FAILED_EC (SHMEM_LOG_BASE + 20)
```

Failed to unlock shared memory mutex protecting the shared memory segment.

NETIO_SHMEM_LOG_FAILED_TO_ALLOCATE_STRUCT_EC

```
#define NETIO_SHMEM_LOG_FAILED_TO_ALLOCATE_STRUCT_EC (SHMEM_LOG_BASE + 21)
```

Failed to allocate memory needed by shared memory transport.

NETIO_SHMEM_LOG_FAILED_TO_INITIALIZE_EC

```
#define NETIO_SHMEM_LOG_FAILED_TO_INITIALIZE_EC (SHMEM_LOG_BASE + 22)
```

Failed to initialize shared memory transport.

NETIO_SHMEM_LOG_CREATE_ATTACH_INFINITE_EC

```
#define NETIO_SHMEM_LOG_CREATE_ATTACH_INFINITE_EC (SHMEM_LOG_BASE + 23)
```

Failed to create or attach. Possible shared memory key conflict.

NETIO_SHMEM_LOG_FAILED_TO_INIT_MUTEX_EC

```
#define NETIO_SHMEM_LOG_FAILED_TO_INIT_MUTEX_EC (SHMEM_LOG_BASE + 24)
```

Failed to create a new shared memory mutex or failed to attach to an existing one.

NETIO_SHMEM_LOG_FAILED_TO_INIT_SIG_SEM_EC

```
#define NETIO_SHMEM_LOG_FAILED_TO_INIT_SIG_SEM_EC (SHMEM_LOG_BASE + 25)
```

Failed to create signaling semaphore. A Domain Participant's shared memory transport blocks on a signaling semaphore when waiting for data.

NETIO_SHMEM_LOG_FAILED_TAKE_SIGN_SEM_EC

```
#define NETIO_SHMEM_LOG_FAILED_TAKE_SIGN_SEM_EC (SHMEM_LOG_BASE + 27)
```

Failed to take a shared memory signaling semaphore.

NETIO_SHMEM_LOG_SIG_SEM_SIGNAL_FAILED_EC

```
#define NETIO_SHMEM_LOG_SIG_SEM_SIGNAL_FAILED_EC (SHMEM_LOG_BASE + 28)
```

Failed to give a shared memory signaling semaphore.

NETIO_SHMEM_LOG_FAILED_TO_INIT_SEGMENT_ADDR_EC

```
#define NETIO_SHMEM_LOG_FAILED_TO_INIT_SEGMENT_ADDR_EC (SHMEM_LOG_BASE + 29)
```

Failed to attach to a shared memory segment because it doesn't exist.

NETIO_SHMEM_LOG_PACKET_INIT_EC

```
#define NETIO_SHMEM_LOG_PACKET_INIT_EC (SHMEM_LOG_BASE + 30)
```

Failed to initialize a receive packet.

NETIO_SHMEM_LOG_FAILED_TO_INIT_CONCURRENT_Q_EC

```
#define NETIO_SHMEM_LOG_FAILED_TO_INIT_CONCURRENT_Q_EC (SHMEM_LOG_BASE + 31)
```

Failed to create a concurrent queue.

NETIO_SHMEM_LOG_FAILED_TO_ATTACH_CONCURRENT_Q_EC

```
#define NETIO_SHMEM_LOG_FAILED_TO_ATTACH_CONCURRENT_Q_EC (SHMEM_LOG_BASE + 32)
```

Failed to attach to a concurrent queue.

NETIO_SHMEM_LOG_INCOMPATIBLE_CONCURRENT_Q_EC

```
#define NETIO_SHMEM_LOG_INCOMPATIBLE_CONCURRENT_Q_EC (SHMEM_LOG_BASE + 33)
```

Attempting to attach to an incompatible concurrent queue with incompatible message_size_max.

NETIO_SHMEM_LOG_FAILED_TO_GET_TIMESTAMP_EC

```
#define NETIO_SHMEM_LOG_FAILED_TO_GET_TIMESTAMP_EC (SHMEM_LOG_BASE + 33)
```

Failed to get timestamp.

NETIO_SHMEM_LOG_NONFATAL_SIG_SEM_RC_EC

```
#define NETIO_SHMEM_LOG_NONFATAL_SIG_SEM_RC_EC (SHMEM_LOG_BASE + 34)
```

Failed to unlock shared memory mutex.

NETIO_SHMEM_LOG_INCONSISTENT_MESSAGE_SIZE_EC

```
#define NETIO_SHMEM_LOG_INCONSISTENT_MESSAGE_SIZE_EC (SHMEM_LOG_BASE + 35)
```

Failed to unlock shared memory mutex.

NETIO_SHMEM_LOG_RECORD_EC

```
#define NETIO_SHMEM_LOG_RECORD_EC (SHMEM_LOG_BASE + 36)
```

Database operation on record failed.

NETIO_SHMEM_LOG_PACKET_HEAD_EC

```
#define NETIO_SHMEM_LOG_PACKET_HEAD_EC (SHMEM_LOG_BASE + 37)
```

Failed to set packet head.

NETIO_SHMEM_LOG_PACKET_FWD_EC

```
#define NETIO_SHMEM_LOG_PACKET_FWD_EC (SHMEM_LOG_BASE + 38)
```

Failed reception upstream for a received packet.

NETIO_SHMEM_LOG_QUEUE_FULL_EC

```
#define NETIO_SHMEM_LOG_QUEUE_FULL_EC (SHMEM_LOG_BASE + 39)
```

Failed reception upstream for a received packet.

NETIO_SHMEM_LOG_ROUTE_LOOKUP_FAILED_EC

```
#define NETIO_SHMEM_LOG_ROUTE_LOOKUP_FAILED_EC (SHMEM_LOG_BASE + 40)
```

Failed to evict find an entry in the route table.

NETIO_SHMEM_LOG_ROUTE_DELETE_FAILED_EC

```
#define NETIO_SHMEM_LOG_ROUTE_DELETE_FAILED_EC (SHMEM_LOG_BASE + 41)
```

Failed to delete an entry from the route table.

NETIO_SHMEM_LOG_IGNORE_TRANSPORT_COMPATIBILITY_EC

```
#define NETIO_SHMEM_LOG_IGNORE_TRANSPORT_COMPATIBILITY_EC (SHMEM_LOG_BASE + 42)
```

The shared memory transport is ignoring the requested transport minimum compatibility version because it cannot be backward compatible.

NETIO_SHMEM_LOG_UNSUPPORTED_TRANSPORT_COMPATIBILITY_EC

```
#define NETIO_SHMEM_LOG_UNSUPPORTED_TRANSPORT_COMPATIBILITY_EC (SHMEM_LOG_BASE + 43)
```

Unsupported minimum compatibility version because it requires a version of the shared memory transport which is not supported.

6.20.8 RTPS

Real-Time Publish-Subscribe. ModuleID = 6.

Macros

- #define `RTPS_LOG_INITIALIZE_INTERFACE_EC` (`RTPS_LOG_BASE` + 1)
Failed to initialize an RTPS interface.
- #define `RTPS_LOG_ALLOCATE_EC` (`RTPS_LOG_BASE` + 2)
Failed to allocate heap memory for internal resources.
- #define `RTPS_LOG_CREATE_ROUTE_TABLE_EC` (`RTPS_LOG_BASE` + 3)
Failed to create a database table for storing RTPS route info.
- #define `RTPS_LOG_DB_CREATE_ROUTE_PEER_INDEX_EC` (`RTPS_LOG_BASE` + 4)
Failed to create database index for route-peer info.
- #define `RTPS_LOG_DB_CREATE_ROUTE_XPORT_INDEX_EC` (`RTPS_LOG_BASE` + 5)
Failed to create database index for route-transport info.
- #define `RTPS_LOG_DB_CREATE_BIND_PEER_INDEX_EC` (`RTPS_LOG_BASE` + 6)
Failed to create database index for bind-peer info.
- #define `RTPS_LOG_INDEX_ADD_ENTRY_EC` (`RTPS_LOG_BASE` + 8)
Failed to add an entry to a database index.
- #define `RTPS_LOG_DB_SELECT_ALL_EC` (`RTPS_LOG_BASE` + 9)
Failed to select all entries of a database table.
- #define `RTPS_LOG_DB_SELECT_MATCH_EC` (`RTPS_LOG_BASE` + 10)
Matching entry not found in a database table.
- #define `RTPS_LOG_DB_SELECT_RANGE_EC` (`RTPS_LOG_BASE` + 11)
No matching entries found in a database table.
- #define `RTPS_LOG_DB_CREATE_ROUTE_ENTRY_EC` (`RTPS_LOG_BASE` + 12)
Failed to create database entry for route.
- #define `RTPS_LOG_DB_INSERT_ROUTE_ENTRY_EC` (`RTPS_LOG_BASE` + 13)
Failed to insert entry into route table.
- #define `RTPS_LOG_DB_DELETE_ROUTE_ENTRY_EC` (`RTPS_LOG_BASE` + 15)
Failed to delete entry from route table.
- #define `RTPS_LOG_DB_CREATE_BIND_ENTRY_EC` (`RTPS_LOG_BASE` + 16)
Failed to create database entry for bind.
- #define `RTPS_LOG_DB_INSERT_BIND_ENTRY_EC` (`RTPS_LOG_BASE` + 17)
Failed to insert an entry into the bind table.
- #define `RTPS_LOG_DB_REMOVE_BIND_ENTRY_EC` (`RTPS_LOG_BASE` + 18)
Failed to remove an entry from the bind table.
- #define `RTPS_LOG_DB_REMOVE_EXTERNAL_BIND_ENTRY_EC` (`RTPS_LOG_BASE` + 19)
Failed to remove an entry from the external bind table.
- #define `RTPS_LOG_DB_DELETE_EXTERNAL_BIND_ENTRY_EC` (`RTPS_LOG_BASE` + 20)
Failed to delete entry from external bind table.
- #define `RTPS_LOG_SEND_EC` (`RTPS_LOG_BASE` + 21)
Failed to send a packet due to a lower module failing to send the packet.
- #define `RTPS_LOG_RECEIVE_EC` (`RTPS_LOG_BASE` + 22)
Failed to receive a packet due to a higher module failing to receive the packet.
- #define `RTPS_LOG_ROUTE_PACKET_EC` (`RTPS_LOG_BASE` + 23)
Failed to send a packet, when routing to a lower module or peer.
- #define `RTPS_LOG_FORWARD_UPSTREAM_EC` (`RTPS_LOG_BASE` + 24)
Failed to receive a packet, when forwarding to a higher upstream module.
- #define `RTPS_LOG_BAD_PARAMETER_EC` (`RTPS_LOG_BASE` + 25)

- *Bad parameter to an RTPS interface function.*
- #define `RTPS_LOG_NOT_ENABLED_EC` (`RTPS_LOG_BASE` + 26)
- *Failed because interface is not enabled.*
- #define `RTPS_LOG_READER_UNSUPPORTED_EC` (`RTPS_LOG_BASE` + 27)
- *RTPS reader does not support send.*
- #define `RTPS_LOG_INTERFACE_MISMATCH_EC` (`RTPS_LOG_BASE` + 28)
- *Upstream interface does not match source or destination of packet being sent or received.*
- #define `RTPS_LOG_FULL_SEND_WINDOW_EC` (`RTPS_LOG_BASE` + 29)
- *Failed to send due to reaching maximum send window size.*
- #define `RTPS_LOG_NETIO_PACKET_SET_TAIL_EC` (`RTPS_LOG_BASE` + 31)
- *Failed to set tail of packet to send.*
- #define `RTPS_LOG_FUNC_UNSUPPORTED_EC` (`RTPS_LOG_BASE` + 32)
- *RTPS interface does not support this function.*
- #define `RTPS_LOG_EXCEEDED_LIMIT_TRANSPORTS_EC` (`RTPS_LOG_BASE` + 33)
- *Out of transport entries.*
- #define `RTPS_LOG_EXCEEDED_LIMIT_PEERS_EC` (`RTPS_LOG_BASE` + 34)
- *Out of peer entries.*
- #define `RTPS_LOG_ASSERT_PEER_EC` (`RTPS_LOG_BASE` + 35)
- *Failed to assert a remote writer or reader.*
- #define `RTPS_LOG_ASSERT_TRANSPORT_EC` (`RTPS_LOG_BASE` + 36)
- *Failed to assert a downstream transport.*
- #define `RTPS_LOG_FIND_EXTERNAL_INTERFACE_EC` (`RTPS_LOG_BASE` + 37)
- *Failed to lookup an external interface.*
- #define `RTPS_LOG_NONEXISTENT_ROUTE_EC` (`RTPS_LOG_BASE` + 38)
- *Failed to delete a nonexistent route.*
- #define `RTPS_LOG_NONEXISTENT_BIND_EC` (`RTPS_LOG_BASE` + 39)
- *Failed to remove a nonexistent bind entry.*
- #define `RTPS_LOG_NONEXISTENT_EXTERNAL_BIND_EC` (`RTPS_LOG_BASE` + 40)
- *Failed to remove a nonexistent external bind entry.*
- #define `RTPS_LOG_STALE_ACK_EPOCH_EC` (`RTPS_LOG_BASE` + 41)
- *Dropped an ACKNACK submessage with an old epoch.*
- #define `RTPS_LOG_STALE_HB_EPOCH_EC` (`RTPS_LOG_BASE` + 42)
- *Dropped a HEARTBEAT submessage with an old epoch.*
- #define `RTPS_LOG_ACK_EC` (`RTPS_LOG_BASE` + 43)
- *Failed an acknack() upstream.*
- #define `RTPS_LOG_REQUEST_EC` (`RTPS_LOG_BASE` + 45)
- *Failed a request() upstream.*
- #define `RTPS_LOG_RETURN_LOAN_EC` (`RTPS_LOG_BASE` + 46)
- *Failed a return_loan() upstream.*
- #define `RTPS_LOG_NETIO_PACKET_SET_HEAD_EC` (`RTPS_LOG_BASE` + 47)
- *Failed to set the head of a packet.*
- #define `RTPS_LOG_SEND_HEARTBEAT_EC` (`RTPS_LOG_BASE` + 48)
- *Failed to send a HEARTBEAT submessage.*
- #define `RTPS_LOG_SHIFT_BITMAP_EC` (`RTPS_LOG_BASE` + 49)
- *Failed to shift a bitmap.*
- #define `RTPS_LOG_FULLY_ACKED_READER_EC` (`RTPS_LOG_BASE` + 50)
- *A Reader is fully acknowledged and does not need to respond to a final HEARTBEAT.*

- #define `RTPS_LOG_DATA_OUT_OF_RANGE_EC` (`RTPS_LOG_BASE` + 51)
Dropping a DATA submessage whose sequence number is outside of a Reader's receive window.
- #define `RTPS_LOG_DATA_ALREADY_RECEIVED_EC` (`RTPS_LOG_BASE` + 52)
Dropping a DATA submessage whose sequence number was previously received.
- #define `RTPS_LOG_INTERFACE_NOT_ENABLED_EC` (`RTPS_LOG_BASE` + 53)
Failed to receive a message because interface is not enabled.
- #define `RTPS_LOG_INVALID_PACKET_EC` (`RTPS_LOG_BASE` + 54)
Dropped a message with an invalid packet header.
- #define `RTPS_LOG_UNSUPPORTED_INTERFACE_EC` (`RTPS_LOG_BASE` + 55)
Received a message on an unsupported interface.
- #define `RTPS_LOG_UNKNOWN_SUBMESSAGE_EC` (`RTPS_LOG_BASE` + 56)
Received a submessage with an unknown ID.
- #define `RTPS_LOG_PROCESS_ACKNACK_EC` (`RTPS_LOG_BASE` + 57)
Failed to process received ACKNACK submessage.
- #define `RTPS_LOG_PROCESS_DATA_EC` (`RTPS_LOG_BASE` + 58)
Failed to process received DATA submessage.
- #define `RTPS_LOG_PROCESS_GAP_EC` (`RTPS_LOG_BASE` + 59)
Failed to process received GAP submessage.
- #define `RTPS_LOG_PROCESS_HEARTBEAT_EC` (`RTPS_LOG_BASE` + 60)
Failed to process received HEARTBEAT submessage.
- #define `RTPS_LOG_CREATE_HB_EVENT_EC` (`RTPS_LOG_BASE` + 61)
Failed to create periodic HEARTBEAT event.
- #define `RTPS_LOG_CREATE_EVENT_TIMER_EC` (`RTPS_LOG_BASE` + 62)
Failed to create periodic HEARTBEAT event timer.
- #define `RTPS_LOG_SEQ_NUM_OUT_OF_RANGE_EC` (`RTPS_LOG_BASE` + 63)
Failed to set bitmap bit due to out-of-range sequence number.
- #define `RTPS_LOG_FIRST_SN_GREATER_LAST_SN_EC` (`RTPS_LOG_BASE` + 65)
Invalid range of sequence numbers to fill in bitmap.
- #define `RTPS_LOG_BITCOUNT_OUT_OF_BOUNDS_EC` (`RTPS_LOG_BASE` + 66)
Deserialized an invalid out-of-bounds bitcount for a bitmap.
- #define `RTPS_LOG_SHIFT_SN_OUT_OF_RANGE_EC` (`RTPS_LOG_BASE` + 67)
Failed to shift bitmap due to invalid starting sequence number.
- #define `RTPS_LOG_SERIALIZE_HOST_ID_EC` (`RTPS_LOG_BASE` + 68)
Failed to serialize host ID of GUID.
- #define `RTPS_LOG_SERIALIZE_APP_ID_EC` (`RTPS_LOG_BASE` + 69)
Failed to serialize app ID of GUID.
- #define `RTPS_LOG_SERIALIZE_INSTANCE_ID_EC` (`RTPS_LOG_BASE` + 70)
Failed to serialize instance ID of GUID.
- #define `RTPS_LOG_SERIALIZE_OBJECT_ID_EC` (`RTPS_LOG_BASE` + 71)
Failed to serialize object ID of GUID.
- #define `RTPS_LOG_DESERIALIZE_HOST_ID_EC` (`RTPS_LOG_BASE` + 72)
Failed to deserialize host ID of GUID.
- #define `RTPS_LOG_DESERIALIZE_APP_ID_EC` (`RTPS_LOG_BASE` + 73)
Failed to deserialize app ID of GUID.
- #define `RTPS_LOG_DESERIALIZE_INSTANCE_ID_EC` (`RTPS_LOG_BASE` + 74)
Failed to deserialize instance ID of GUID.
- #define `RTPS_LOG_DESERIALIZE_OBJECT_ID_EC` (`RTPS_LOG_BASE` + 75)

- Failed to deserialize object ID of GUID.*

 - #define `RTPS_LOG_DB_CREATE_EXT_BIND_ENTRY_EC` (`RTPS_LOG_BASE` + 76)

Failed to create database entry for external bind table.
- #define `RTPS_LOG_BITMAP_SET_BIT_EC` (`RTPS_LOG_BASE` + 77)

Failed to set bit in bitmap.
- #define `RTPS_LOG_UNSUPPORTED_INFO_REPLY_EC` (`RTPS_LOG_BASE` + 78)

Received and dropped currently unsupported INFO_REPLY submessage.
- #define `RTPS_LOG_UNSUPPORTED_INFO_REPLY_IP4_EC` (`RTPS_LOG_BASE` + 79)

Received and dropped currently unsupported INFO_REPLY_IP4 submessage.
- #define `RTPS_LOG_UNSUPPORTED_INFO_SRC_EC` (`RTPS_LOG_BASE` + 80)

Received and dropped currently unsupported INFO_SRC submessage.
- #define `RTPS_LOG_DELETE_INDEX_EC` (`RTPS_LOG_BASE` + 81)

Failed to delete index of a database table.
- #define `RTPS_LOG_DELETE_TABLE_EC` (`RTPS_LOG_BASE` + 82)

Failed to delete a database table.
- #define `RTPS_LOG_DB_LOCK_EC` (`RTPS_LOG_BASE` + 83)

Failed to take database lock.
- #define `RTPS_LOG_DB_UNLOCK_EC` (`RTPS_LOG_BASE` + 84)

Failed to give database lock.
- #define `RTPS_LOG_CREATE_BIND_TABLE_EC` (`RTPS_LOG_BASE` + 85)

Failed to create a database table for storing RTPS bind info.
- #define `RTPS_LOG_STATUS_CHANGE_EC` (`RTPS_LOG_BASE` + 86)

Failed to update status for reliable reader activity changed.
- #define `RTPS_LOG_SET_GAP_EC` (`RTPS_LOG_BASE` + 87)

Failed to update status for reliable reader activity changed.
- #define `RTPS_LOG_WINDOW_POOL_ALLOC_EC` (`RTPS_LOG_BASE` + 88)

Out of memory to allocate send window pool.
- #define `RTPS_LOG_GET_WINDOW_BUFFER_EC` (`RTPS_LOG_BASE` + 89)

Failed to get buffer from send window pool.
- #define `RTPS_LOG_INSERT_WINDOW_FULL_EC` (`RTPS_LOG_BASE` + 90)

Cannot insert sample into full window.
- #define `RTPS_LOG_WINDOW_INSERT_EC` (`RTPS_LOG_BASE` + 91)

Failed to insert sample into send window.
- #define `RTPS_LOG_TIMER_DELETE_TIMEOUT_EC` (`RTPS_LOG_BASE` + 92)

Failed to delete a timeout event.
- #define `RTPS_LOG_BUFFERPOOL_DELETE_EC` (`RTPS_LOG_BASE` + 93)

Failed to delete a buffer pool.
- #define `RTPS_LOG_DIRECT_SEND_EC` (`RTPS_LOG_BASE` + 94)

Failed to send a message to a specific peer.
- #define `RTPS_LOG_PACKET_INITIALIZE_EC` (`RTPS_LOG_BASE` + 95)

RTPS writer failed to initialize a packet.
- #define `RTPS_LOG_WRITER_REQUEST_SENT_HISTORY_EC` (`RTPS_LOG_BASE` + 96)

RTPS reliable writer failed to request and resend history.
- #define `RTPS_LOG_BITMAP_SHIFT_EC` (`RTPS_LOG_BASE` + 97)

Failed to shift bitmap to new lead sequence number.
- #define `RTPS_LOG_WINDOW_ADVANCE_EC` (`RTPS_LOG_BASE` + 98)

Send window of a remote reader failed to advance.

- #define `RTPS_LOG_WINDOW_ADVANCE_UNACKED_EC` (`RTPS_LOG_BASE` + 99)
Failed when advancing window ahead of unacknowledged sequence number.
- #define `RTPS_LOG_LOOKUP_PEER_EC` (`RTPS_LOG_BASE` + 100)
Failed to find peer that should exist.
- #define `RTPS_LOG_READER_SEND_ACKNACK_EC` (`RTPS_LOG_BASE` + 101)
RTPS reader failed to send an ACKNACK message.
- #define `RTPS_LOG_FINALIZE_INTERFACE_EC` (`RTPS_LOG_BASE` + 102)
Failed to finalize an RTPS interface.
- #define `RTPS_LOG_INDEXER_REMOVE_ENTRY_EC` (`RTPS_LOG_BASE` + 103)
Failed to remove index entry for external interface upon deleting RTPS interface.
- #define `RTPS_LOG_LOOKUP_EXT_BIND_ENTRY_EC` (`RTPS_LOG_BASE` + 104)
Failed when looking up external bind entry from database.
- #define `RTPS_LOG_SEQ_INIT_EC` (`RTPS_LOG_BASE` + 105)
Failed to initialize a local sequence.
- #define `RTPS_LOG_SEQ_MAX_EC` (`RTPS_LOG_BASE` + 106)
Failed to set maximum of a local sequence.
- #define `RTPS_LOG_SEQ_LEN_EC` (`RTPS_LOG_BASE` + 107)
Failed to set length of a local sequence.
- #define `RTPS_LOG_INTF_MODE_UNDEF_EC` (`RTPS_LOG_BASE` + 108)
Initializing an RTPS interface with undefined mode.
- #define `RTPS_LOG_DELETE_BUFFER_POOL_EC` (`RTPS_LOG_BASE` + 109)
Failed to delete buffer pool.
- #define `RTPS_LOG_CREATE_BUFFER_POOL_EC` (`RTPS_LOG_BASE` + 110)
Failed to create buffer pool.
- #define `RTPS_LOG_CREATE_PEERS_INDEX_EC` (`RTPS_LOG_BASE` + 111)
Failed to create index of peers.
- #define `RTPS_LOG_DELETE_INDEXER_EC` (`RTPS_LOG_BASE` + 112)
Failed to delete indexer.
- #define `RTPS_LOG_GET_PEER_BUFFER_EC` (`RTPS_LOG_BASE` + 113)
Failed to get buffer from peer buffer pool.
- #define `RTPS_LOG_DB_CREATE_DIRECT_INDEX_EC` (`RTPS_LOG_BASE` + 114)
Failed to create database index for direct sends.
- #define `RTPS_LOG_DB_CREATE_GROUP_INDEX_EC` (`RTPS_LOG_BASE` + 115)
Failed to create database index for group sends.
- #define `RTPS_LOG_NETIO_PACKET_INSUFFICIENT_BUFFER_EC` (`RTPS_LOG_BASE` + 116)
Insufficient buffer in packet.
- #define `RTPS_LOG_PROCESS_DATA_BATCH_EC` (`RTPS_LOG_BASE` + 136)
Failed to process received DATA BATCH submessage.
- #define `RTPS_LOG_PROCESS_HEARTBEAT_BATCH_EC` (`RTPS_LOG_BASE` + 137)
Failed to process received HEARTBEAT_BATCH submessage.
- #define `RTPS_LOG_NEGATIVE_RESERVATION_COUNT_EC` (`RTPS_LOG_BASE` + 138)
The reservation count is zero.
- #define `RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_LOOKUP_FAILED_EC` (`RTPS_LOG_BASE` + 139)
Failed to lookup a remote participant's interceptor handle.
- #define `RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_CREATE_FAILED_EC` (`RTPS_LOG_BASE` + 140)

- Failed to create a record to store a remote participant's interceptor handle.*

 - #define `RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_INSERT_FAILED_EC` (`RTPS_LOG_BASE` + 141)
- Failed to insert a record in the table storing interceptor handles of matched remote participants.*

 - #define `RTPS_LOG_TRUST_INTERCEPTOR_HANDLE_LIST_FULL_EC` (`RTPS_LOG_BASE` + 142)
- Interceptor handle list out of resources.*

 - #define `RTPS_LOG_TRUST_INCONSISTENT_INTERCEPTOR_HANDLE_EC` (`RTPS_LOG_BASE` + 143)
- Inconsistent handles found for remote peer.*

 - #define `RTPS_LOG_TRUST_BIND_FAILED_EC` (`RTPS_LOG_BASE` + 144)
- Failed to bind resources for trust behavior.*

 - #define `RTPS_LOG_TRUST_SUBMSG_CONVERSION_FAILED_EC` (`RTPS_LOG_BASE` + 145)
- Failed to bind resources for trust behavior.*

 - #define `RTPS_LOG_TRUST_INVALID_SUBMSG_EC` (`RTPS_LOG_BASE` + 146)
- A malformed trust message was received.*

 - #define `RTPS_LOG_TRUST_UNEXPECTED_READER_STATE_EC` (`RTPS_LOG_BASE` + 147)
- The RTPS reader was in an unexpected state.*

 - #define `RTPS_LOG_DB_CREATE_SELECTED_GROUP_INDEX_EC` (`RTPS_LOG_BASE` + 148)
- Failed to create database index for the selected route group.*

 - #define `RTPS_LOG_LOCK_EC` (`RTPS_LOG_BASE` + 149)
- Failed to take a lock.*

 - #define `RTPS_LOG_UNLOCK_EC` (`RTPS_LOG_BASE` + 150)
- Failed to give a lock.*

 - #define `RTPS_LOG_SCHEDULE_PACKET_EC` (`RTPS_LOG_BASE` + 151)
- Failed to schedule a packet.*

 - #define `RTPS_LOG_SEQ_FINALIZE_EC` (`RTPS_LOG_BASE` + 152)
- Failed to finalize a sequence.*

 - #define `RTPS_LOG_TRANSFORM_RTPS_EC` (`RTPS_LOG_BASE` + 153)
- Failed to transform rtps message.*

 - #define `RTPS_LOG_RESTORE_TRUST_LOAN_BUFFER_EC` (`RTPS_LOG_BASE` + 154)
- Failed to restore trust loan buffer.*

 - #define `RTPS_LOG_CREATE_REASSEMBLY_TABLE_EC` (`RTPS_LOG_BASE` + 155)
- Failed to create reassembly table.*

 - #define `RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM128_EC` (`RTPS_LOG_BASE` + 200)
- The property tries to override the BUILTIN128 checksum without setting allow_builtin_override to TRUE.*

 - #define `RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM32_EC` (`RTPS_LOG_BASE` + 201)
- The property tries to override the BUILTIN32 checksum without setting allow_builtin_override to TRUE.*

 - #define `RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM64_EC` (`RTPS_LOG_BASE` + 202)
- The property tries to override the BUILTIN64 checksum without setting allow_builtin_override to TRUE.*

 - #define `RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN128_EC` (`RTPS_LOG_BASE` + 203)
- The property override of the BUILTIN128 checksum is invalid. It is not legal to reuse any of the built-in functions.*

 - #define `RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN32_EC` (`RTPS_LOG_BASE` + 204)
- The property override of the BUILTIN32 checksum is invalid. It is not legal to reuse any of the built-in functions functions.*

 - #define `RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN64_EC` (`RTPS_LOG_BASE` + 205)
- The property override of the BUILTIN64 checksum is invalid. It is not legal to reuse any of the built-in functions functions.*

 - #define `RTPS_LOG_CHECKSUM_CHECKSUM_CALCULATION_FAILED_EC` (`RTPS_LOG_BASE` + 210)
- RTPS_Interface_calculate_checksum() failed to calculate the checksum.*

 - #define `RTPS_LOG_UNSUPPORTED_MANDTORY_HDR_EXT_PID_EC` (`RTPS_LOG_BASE` + 211)

The HeaderExtension contained a mandatory PID that was not understood.

- #define RTPS_LOG_INVALID_HDR_EXT_PID_LIST_EC (RTPS_LOG_BASE + 212)

An invalid HeaderExtension PID was detected.

- #define RTPS_LOG_INVALID_HDR_EXT_PID_ERROR_EC (RTPS_LOG_BASE + 213)

Failed to decode the HeaderExtension PID list.

- #define RTPS_LOG_INCONSISTENT_HDR_EXT_LENGTH_ERROR_EC (RTPS_LOG_BASE + 214)

The HeaderExtension length is inconsistent with the decoded encoded HeaderExtension length.

- #define RTPS_LOG_MESSAGE_EXCEED_LENGTH_ERROR_EC (RTPS_LOG_BASE + 215)

A RTPS message which was longer than the indicated message length in the RTPS header was received. This causes the rest of the RTPS message to be invalidated and the remaining submessages to be ignored.

- #define RTPS_LOG_CHECKSUM_INVALID_CRC32_FLAGS_EC (RTPS_LOG_BASE + 216)

The CRC32 submsg flags are invalid.

- #define RTPS_LOG_CHECKSUM_MISSING_CHECKSUM_LENGTH_EC (RTPS_LOG_BASE + 217)

A header extension was received, but the checksum length is not present.

- #define RTPS_LOG_CHECKSUM_UNSUPPORTED_EC (RTPS_LOG_BASE + 218)

Unsupported checksum bits received, message dropped.

- #define RTPS_LOG_CHECKSUM_LENGTH_DESERIALIZE_ERROR_EC (RTPS_LOG_BASE + 219)

Checksum length could not be deserialized.

- #define RTPS_LOG_CHECKSUM_INCONSISTENT_LENGTH_EC (RTPS_LOG_BASE + 220)

The payload length is less than the deserialized checksum length.

- #define RTPS_LOG_CHECKSUM_CHECKSUM_DESERIALIZE_ERROR_EC (RTPS_LOG_BASE + 221)

Checksum length could not be deserialized.

- #define RTPS_LOG_CHECKSUM_CHECKSUM_ERROR_EC (RTPS_LOG_BASE + 222)

The checksum is invalid.

- #define RTPS_LOG_CHECKSUM_TOO_MANY_SG_BUFFERS_EC (RTPS_LOG_BASE + 223)

Found too many scatter-gather buffers when calculating checksum.

- #define RTPS_LOG_SEND_LIVELINESS_EC (RTPS_LOG_BASE + 224)

Failure to send Liveliness packets.

- #define RTPS_LOG_DELETE_RECORD_EC (RTPS_LOG_BASE + 225)

Failure to delete a record.

- #define RTPS_LOG_APPLY_CONTENT_FILTER_EC (RTPS_LOG_BASE + 300)

Failed to apply a content filter to a message.

- #define RTPS_LOG_UPDATE_RELIABLE_PEERS_FILTER_EC (RTPS_LOG_BASE + 301)

Failed to update the state of reliable peers while applying filtering.

- #define RTPS_LOG_ADD_FILTERED_ROUTE_EC (RTPS_LOG_BASE + 302)

Failed to add a filtered route.

- #define RTPS_LOG_DELETE_FILTERED_ROUTE_EC (RTPS_LOG_BASE + 303)

Failed to delete a filtered route.

6.20.8.1 Detailed Description

Real-Time Publish-Subscribe. ModuleID = 6.

6.20.8.2 Macro Definition Documentation

RTPS_LOG_INITIALIZE_INTERFACE_EC

```
#define RTPS_LOG_INITIALIZE_INTERFACE_EC (RTPS_LOG_BASE + 1)
```

Failed to initialize an RTPS interface.

RTPS_LOG_ALLOCATE_EC

```
#define RTPS_LOG_ALLOCATE_EC (RTPS_LOG_BASE + 2)
```

Failed to allocate heap memory for internal resources.

RTPS_LOG_CREATE_ROUTE_TABLE_EC

```
#define RTPS_LOG_CREATE_ROUTE_TABLE_EC (RTPS_LOG_BASE + 3)
```

Failed to create a database table for storing RTPS route info.

RTPS_LOG_DB_CREATE_ROUTE_PEER_INDEX_EC

```
#define RTPS_LOG_DB_CREATE_ROUTE_PEER_INDEX_EC (RTPS_LOG_BASE + 4)
```

Failed to create database index for route-peer info.

RTPS_LOG_DB_CREATE_ROUTE_XPORT_INDEX_EC

```
#define RTPS_LOG_DB_CREATE_ROUTE_XPORT_INDEX_EC (RTPS_LOG_BASE + 5)
```

Failed to create database index for route-transport info.

RTPS_LOG_DB_CREATE_BIND_PEER_INDEX_EC

```
#define RTPS_LOG_DB_CREATE_BIND_PEER_INDEX_EC (RTPS_LOG_BASE + 6)
```

Failed to create database index for bind-peer info.

RTPS_LOG_INDEX_ADD_ENTRY_EC

```
#define RTPS_LOG_INDEX_ADD_ENTRY_EC (RTPS_LOG_BASE + 8)
```

Failed to add an entry to a database index.

RTPS_LOG_DB_SELECT_ALL_EC

```
#define RTPS_LOG_DB_SELECT_ALL_EC (RTPS_LOG_BASE + 9)
```

Failed to select all entries of a database table.

Failure reason given by database return code (dbrc)

RTPS_LOG_DB_SELECT_MATCH_EC

```
#define RTPS_LOG_DB_SELECT_MATCH_EC (RTPS_LOG_BASE + 10)
```

Matching entry not found in a database table.

Failure reason given by database return code (dbrc)

RTPS_LOG_DB_SELECT_RANGE_EC

```
#define RTPS_LOG_DB_SELECT_RANGE_EC (RTPS_LOG_BASE + 11)
```

No matching entries found in a database table.

RTPS_LOG_DB_CREATE_ROUTE_ENTRY_EC

```
#define RTPS_LOG_DB_CREATE_ROUTE_ENTRY_EC (RTPS_LOG_BASE + 12)
```

Failed to create database entry for route.

May have exceeded DomainParticipantQos.resource_limits.matching_writer_reader_pair_allocation, DataWriterQos.writer_resource_limits.max_remote_readers, or DataReaderQos.reader_resource_limits.max_remote_writers

RTPS_LOG_DB_INSERT_ROUTE_ENTRY_EC

```
#define RTPS_LOG_DB_INSERT_ROUTE_ENTRY_EC (RTPS_LOG_BASE + 13)
```

Failed to insert entry into route table.

Failure reason given by database return code (dbrc)

RTPS_LOG_DB_DELETE_ROUTE_ENTRY_EC

```
#define RTPS_LOG_DB_DELETE_ROUTE_ENTRY_EC (RTPS_LOG_BASE + 15)
```

Failed to delete entry from route table.

Failure reason given by database return code (dbrc)

RTPS_LOG_DB_CREATE_BIND_ENTRY_EC

```
#define RTPS_LOG_DB_CREATE_BIND_ENTRY_EC (RTPS_LOG_BASE + 16)
```

Failed to create database entry for bind.

If DomainParticipant, may have exceeded DomainParticipantQos.resource_limits.matching_reader_writer_pair_↔ allocation. Failure reason given by database return code (dbrc).

RTPS_LOG_DB_INSERT_BIND_ENTRY_EC

```
#define RTPS_LOG_DB_INSERT_BIND_ENTRY_EC (RTPS_LOG_BASE + 17)
```

Failed to insert an entry into the bind table.

Failure reason given by database return code (dbrc)

RTPS_LOG_DB_REMOVE_BIND_ENTRY_EC

```
#define RTPS_LOG_DB_REMOVE_BIND_ENTRY_EC (RTPS_LOG_BASE + 18)
```

Failed to remove an entry from the bind table.

Failure reason given by database return code (dbrc)

RTPS_LOG_DB_REMOVE_EXTERNAL_BIND_ENTRY_EC

```
#define RTPS_LOG_DB_REMOVE_EXTERNAL_BIND_ENTRY_EC (RTPS_LOG_BASE + 19)
```

Failed to remove an entry from the external bind table.

Failure reason given by database return code (dbrc)

RTPS_LOG_DB_DELETE_EXTERNAL_BIND_ENTRY_EC

```
#define RTPS_LOG_DB_DELETE_EXTERNAL_BIND_ENTRY_EC (RTPS_LOG_BASE + 20)
```

Failed to delete entry from external bind table.

Failure reason given by database return code (dbrc)

RTPS_LOG_SEND_EC

```
#define RTPS_LOG_SEND_EC (RTPS_LOG_BASE + 21)
```

Failed to send a packet due to a lower module failing to send the packet.

RTPS_LOG_RECEIVE_EC

```
#define RTPS_LOG_RECEIVE_EC (RTPS_LOG_BASE + 22)
```

Failed to receive a packet due to a higher module failing to receive the packet.

RTPS_LOG_ROUTE_PACKET_EC

```
#define RTPS_LOG_ROUTE_PACKET_EC (RTPS_LOG_BASE + 23)
```

Failed to send a packet, when routing to a lower module or peer.

RTPS_LOG_FORWARD_UPSTREAM_EC

```
#define RTPS_LOG_FORWARD_UPSTREAM_EC (RTPS_LOG_BASE + 24)
```

Failed to receive a packet, when forwarding to a higher upstream module.

RTPS_LOG_BAD_PARAMETER_EC

```
#define RTPS_LOG_BAD_PARAMETER_EC (RTPS_LOG_BASE + 25)
```

Bad parameter to an RTPS interface function.

RTPS_LOG_NOT_ENABLED_EC

```
#define RTPS_LOG_NOT_ENABLED_EC (RTPS_LOG_BASE + 26)
```

Failed because interface is not enabled.

RTPS_LOG_READER_UNSUPPORTED_EC

```
#define RTPS_LOG_READER_UNSUPPORTED_EC (RTPS_LOG_BASE + 27)
```

RTPS reader does not support send.

RTPS_LOG_INTERFACE_MISMATCH_EC

```
#define RTPS_LOG_INTERFACE_MISMATCH_EC (RTPS_LOG_BASE + 28)
```

Upstream interface does not match source or destination of packet being sent or received.

RTPS_LOG_FULL_SEND_WINDOW_EC

```
#define RTPS_LOG_FULL_SEND_WINDOW_EC (RTPS_LOG_BASE + 29)
```

Failed to send due to reaching maximum send window size.

RTPS_LOG_NETIO_PACKET_SET_TAIL_EC

```
#define RTPS_LOG_NETIO_PACKET_SET_TAIL_EC (RTPS_LOG_BASE + 31)
```

Failed to set tail of packet to send.

RTPS_LOG_FUNC_UNSUPPORTED_EC

```
#define RTPS_LOG_FUNC_UNSUPPORTED_EC (RTPS_LOG_BASE + 32)
```

RTPS interface does not support this function.

RTPS_LOG_EXCEEDED_LIMIT_TRANSPORTS_EC

```
#define RTPS_LOG_EXCEEDED_LIMIT_TRANSPORTS_EC (RTPS_LOG_BASE + 33)
```

Out of transport entries.

Exceeded either `DataWriterQos.writer_resource_limits.max_remote_readers` or `DataReaderQos.reader_resource_↔limits.max_remote_writers`

RTPS_LOG_EXCEEDED_LIMIT_PEERS_EC

```
#define RTPS_LOG_EXCEEDED_LIMIT_PEERS_EC (RTPS_LOG_BASE + 34)
```

Out of peer entries.

Exceeded either `DataWriterQos.writer_resource_limits.max_remote_readers` or `DataReaderQos.reader_resource_↔limits.max_remote_writers`

RTPS_LOG_ASSERT_PEER_EC

```
#define RTPS_LOG_ASSERT_PEER_EC (RTPS_LOG_BASE + 35)
```

Failed to assert a remote writer or reader.

RTPS_LOG_ASSERT_TRANSPORT_EC

```
#define RTPS_LOG_ASSERT_TRANSPORT_EC (RTPS_LOG_BASE + 36)
```

Failed to assert a downstream transport.

RTPS_LOG_FIND_EXTERNAL_INTERFACE_EC

```
#define RTPS_LOG_FIND_EXTERNAL_INTERFACE_EC (RTPS_LOG_BASE + 37)
```

Failed to lookup an external interface.

RTPS_LOG_NONEXISTENT_ROUTE_EC

```
#define RTPS_LOG_NONEXISTENT_ROUTE_EC (RTPS_LOG_BASE + 38)
```

Failed to delete a nonexistent route.

RTPS_LOG_NONEXISTENT_BIND_EC

```
#define RTPS_LOG_NONEXISTENT_BIND_EC (RTPS_LOG_BASE + 39)
```

Failed to remove a nonexistent bind entry.

RTPS_LOG_NONEXISTENT_EXTERNAL_BIND_EC

```
#define RTPS_LOG_NONEXISTENT_EXTERNAL_BIND_EC (RTPS_LOG_BASE + 40)
```

Failed to remove a nonexistent external bind entry.

RTPS_LOG_STALE_ACK_EPOCH_EC

```
#define RTPS_LOG_STALE_ACK_EPOCH_EC (RTPS_LOG_BASE + 41)
```

Dropped an ACKNACK submessage with an old epoch.

RTPS_LOG_STALE_HB_EPOCH_EC

```
#define RTPS_LOG_STALE_HB_EPOCH_EC (RTPS_LOG_BASE + 42)
```

Dropped a HEARTBEAT submessage with an old epoch.

RTPS_LOG_ACK_EC

```
#define RTPS_LOG_ACK_EC (RTPS_LOG_BASE + 43)
```

Failed an acknack() upstream.

RTPS_LOG_REQUEST_EC

```
#define RTPS_LOG_REQUEST_EC (RTPS_LOG_BASE + 45)
```

Failed a request() upstream.

RTPS_LOG_RETURN_LOAN_EC

```
#define RTPS_LOG_RETURN_LOAN_EC (RTPS_LOG_BASE + 46)
```

Failed a return_loan() upstream.

RTPS_LOG_NETIO_PACKET_SET_HEAD_EC

```
#define RTPS_LOG_NETIO_PACKET_SET_HEAD_EC (RTPS_LOG_BASE + 47)
```

Failed to set the head of a packet.

RTPS_LOG_SEND_HEARTBEAT_EC

```
#define RTPS_LOG_SEND_HEARTBEAT_EC (RTPS_LOG_BASE + 48)
```

Failed to send a HEARTBEAT submessage.

RTPS_LOG_SHIFT_BITMAP_EC

```
#define RTPS_LOG_SHIFT_BITMAP_EC (RTPS_LOG_BASE + 49)
```

Failed to shift a bitmap.

RTPS_LOG_FULLY_ACKED_READER_EC

```
#define RTPS_LOG_FULLY_ACKED_READER_EC (RTPS_LOG_BASE + 50)
```

A Reader is fully acknowledged and does not need to respond to a final HEARTBEAT.

RTPS_LOG_DATA_OUT_OF_RANGE_EC

```
#define RTPS_LOG_DATA_OUT_OF_RANGE_EC (RTPS_LOG_BASE + 51)
```

Dropping a DATA submessage whose sequence number is outside of a Reader's receive window.

RTPS_LOG_DATA_ALREADY_RECEIVED_EC

```
#define RTPS_LOG_DATA_ALREADY_RECEIVED_EC (RTPS_LOG_BASE + 52)
```

Dropping a DATA submessage whose sequence number was previously received.

RTPS_LOG_INTERFACE_NOT_ENABLED_EC

```
#define RTPS_LOG_INTERFACE_NOT_ENABLED_EC (RTPS_LOG_BASE + 53)
```

Failed to receive a message because interface is not enabled.

RTPS_LOG_INVALID_PACKET_EC

```
#define RTPS_LOG_INVALID_PACKET_EC (RTPS_LOG_BASE + 54)
```

Dropped a message with an invalid packet header.

RTPS_LOG_UNSUPPORTED_INTERFACE_EC

```
#define RTPS_LOG_UNSUPPORTED_INTERFACE_EC (RTPS_LOG_BASE + 55)
```

Received a message on an unsupported interface.

RTPS_LOG_UNKNOWN_SUBMESSAGE_EC

```
#define RTPS_LOG_UNKNOWN_SUBMESSAGE_EC (RTPS_LOG_BASE + 56)
```

Received a submessage with an unknown ID.

RTPS_LOG_PROCESS_ACKNACK_EC

```
#define RTPS_LOG_PROCESS_ACKNACK_EC (RTPS_LOG_BASE + 57)
```

Failed to process received ACKNACK submessage.

RTPS_LOG_PROCESS_DATA_EC

```
#define RTPS_LOG_PROCESS_DATA_EC (RTPS_LOG_BASE + 58)
```

Failed to process received DATA submessage.

RTPS_LOG_PROCESS_GAP_EC

```
#define RTPS_LOG_PROCESS_GAP_EC (RTPS_LOG_BASE + 59)
```

Failed to process received GAP submessage.

RTPS_LOG_PROCESS_HEARTBEAT_EC

```
#define RTPS_LOG_PROCESS_HEARTBEAT_EC (RTPS_LOG_BASE + 60)
```

Failed to process received HEARTBEAT submessage.

RTPS_LOG_CREATE_HB_EVENT_EC

```
#define RTPS_LOG_CREATE_HB_EVENT_EC (RTPS_LOG_BASE + 61)
```

Failed to create periodic HEARTBEAT event.

RTPS_LOG_CREATE_EVENT_TIMER_EC

```
#define RTPS_LOG_CREATE_EVENT_TIMER_EC (RTPS_LOG_BASE + 62)
```

Failed to create periodic HEARTBEAT event timer.

RTPS_LOG_SEQ_NUM_OUT_OF_RANGE_EC

```
#define RTPS_LOG_SEQ_NUM_OUT_OF_RANGE_EC (RTPS_LOG_BASE + 63)
```

Failed to set bitmap bit due to out-of-range sequence number.

RTPS_LOG_FIRST_SN_GREATER_LAST_SN_EC

```
#define RTPS_LOG_FIRST_SN_GREATER_LAST_SN_EC (RTPS_LOG_BASE + 65)
```

Invalid range of sequence numbers to fill in bitmap.

RTPS_LOG_BITCOUNT_OUT_OF_BOUNDS_EC

```
#define RTPS_LOG_BITCOUNT_OUT_OF_BOUNDS_EC (RTPS_LOG_BASE + 66)
```

Deserialized an invalid out-of-bounds bitcount for a bitmap.

RTPS_LOG_SHIFT_SN_OUT_OF_RANGE_EC

```
#define RTPS_LOG_SHIFT_SN_OUT_OF_RANGE_EC (RTPS_LOG_BASE + 67)
```

Failed to shift bitmap due to invalid starting sequence number.

RTPS_LOG_SERIALIZE_HOST_ID_EC

```
#define RTPS_LOG_SERIALIZE_HOST_ID_EC (RTPS_LOG_BASE + 68)
```

Failed to serialize host ID of GUID.

RTPS_LOG_SERIALIZE_APP_ID_EC

```
#define RTPS_LOG_SERIALIZE_APP_ID_EC (RTPS_LOG_BASE + 69)
```

Failed to serialize app ID of GUID.

RTPS_LOG_SERIALIZE_INSTANCE_ID_EC

```
#define RTPS_LOG_SERIALIZE_INSTANCE_ID_EC (RTPS_LOG_BASE + 70)
```

Failed to serialized instance ID of GUID.

RTPS_LOG_SERIALIZE_OBJECT_ID_EC

```
#define RTPS_LOG_SERIALIZE_OBJECT_ID_EC (RTPS_LOG_BASE + 71)
```

Failed to serialize object ID of GUID.

RTPS_LOG_DESERIALIZE_HOST_ID_EC

```
#define RTPS_LOG_DESERIALIZE_HOST_ID_EC (RTPS_LOG_BASE + 72)
```

Failed to deserialize host ID of GUID.

RTPS_LOG_DESERIALIZE_APP_ID_EC

```
#define RTPS_LOG_DESERIALIZE_APP_ID_EC (RTPS_LOG_BASE + 73)
```

Failed to deserialize app ID of GUID.

RTPS_LOG_DESERIALIZE_INSTANCE_ID_EC

```
#define RTPS_LOG_DESERIALIZE_INSTANCE_ID_EC (RTPS_LOG_BASE + 74)
```

Failed to deserialize instance ID of GUID.

RTPS_LOG_DESERIALIZE_OBJECT_ID_EC

```
#define RTPS_LOG_DESERIALIZE_OBJECT_ID_EC (RTPS_LOG_BASE + 75)
```

Failed to deserialize object ID of GUID.

RTPS_LOG_DB_CREATE_EXT_BIND_ENTRY_EC

```
#define RTPS_LOG_DB_CREATE_EXT_BIND_ENTRY_EC (RTPS_LOG_BASE + 76)
```

Failed to create database entry for external bind table.

If DomainParticipant, may have exceeded DomainParticipantQos.resource_limits.matching_reader_writer_pair_↔ allocation.

RTPS_LOG_BITMAP_SET_BIT_EC

```
#define RTPS_LOG_BITMAP_SET_BIT_EC (RTPS_LOG_BASE + 77)
```

Failed to set bit in bitmap.

RTPS_LOG_UNSUPPORTED_INFO_REPLY_EC

```
#define RTPS_LOG_UNSUPPORTED_INFO_REPLY_EC (RTPS_LOG_BASE + 78)
```

Received and dropped currently unsupported INFO_REPLY submessage.

RTPS_LOG_UNSUPPORTED_INFO_REPLY_IP4_EC

```
#define RTPS_LOG_UNSUPPORTED_INFO_REPLY_IP4_EC (RTPS_LOG_BASE + 79)
```

Received and dropped currently unsupported INFO_REPLY_IP4 submessage.

RTPS_LOG_UNSUPPORTED_INFO_SRC_EC

```
#define RTPS_LOG_UNSUPPORTED_INFO_SRC_EC (RTPS_LOG_BASE + 80)
```

Received and dropped currently unsupported INFO_SRC submessage.

RTPS_LOG_DELETE_INDEX_EC

```
#define RTPS_LOG_DELETE_INDEX_EC (RTPS_LOG_BASE + 81)
```

Failed to delete index of a database table.

RTPS_LOG_DELETE_TABLE_EC

```
#define RTPS_LOG_DELETE_TABLE_EC (RTPS_LOG_BASE + 82)
```

Failed to delete a database table.

RTPS_LOG_DB_LOCK_EC

```
#define RTPS_LOG_DB_LOCK_EC (RTPS_LOG_BASE + 83)
```

Failed to take database lock.

RTPS_LOG_DB_UNLOCK_EC

```
#define RTPS_LOG_DB_UNLOCK_EC (RTPS_LOG_BASE + 84)
```

Failed to give database lock.

RTPS_LOG_CREATE_BIND_TABLE_EC

```
#define RTPS_LOG_CREATE_BIND_TABLE_EC (RTPS_LOG_BASE + 85)
```

Failed to create a database table for storing RTPS bind info.

RTPS_LOG_STATUS_CHANGE_EC

```
#define RTPS_LOG_STATUS_CHANGE_EC (RTPS_LOG_BASE + 86)
```

Failed to update status for reliable reader activity changed.

RTPS_LOG_SET_GAP_EC

```
#define RTPS_LOG_SET_GAP_EC (RTPS_LOG_BASE + 87)
```

Failed to update status for reliable reader activity changed.

RTPS_LOG_WINDOW_POOL_ALLOC_EC

```
#define RTPS_LOG_WINDOW_POOL_ALLOC_EC (RTPS_LOG_BASE + 88)
```

Out of memory to allocate send window pool.

RTPS_LOG_GET_WINDOW_BUFFER_EC

```
#define RTPS_LOG_GET_WINDOW_BUFFER_EC (RTPS_LOG_BASE + 89)
```

Failed to get buffer from send window pool.

RTPS_LOG_INSERT_WINDOW_FULL_EC

```
#define RTPS_LOG_INSERT_WINDOW_FULL_EC (RTPS_LOG_BASE + 90)
```

Cannot insert sample into full window.

RTPS_LOG_WINDOW_INSERT_EC

```
#define RTPS_LOG_WINDOW_INSERT_EC (RTPS_LOG_BASE + 91)
```

Failed to insert sample into send window.

RTPS_LOG_TIMER_DELETE_TIMEOUT_EC

```
#define RTPS_LOG_TIMER_DELETE_TIMEOUT_EC (RTPS_LOG_BASE + 92)
```

Failed to delete a timeout event.

RTPS_LOG_BUFFERPOOL_DELETE_EC

```
#define RTPS_LOG_BUFFERPOOL_DELETE_EC (RTPS_LOG_BASE + 93)
```

Failed to delete a buffer pool.

RTPS_LOG_DIRECT_SEND_EC

```
#define RTPS_LOG_DIRECT_SEND_EC (RTPS_LOG_BASE + 94)
```

Failed to send a message to a specific peer.

RTPS_LOG_PACKET_INITIALIZE_EC

```
#define RTPS_LOG_PACKET_INITIALIZE_EC (RTPS_LOG_BASE + 95)
```

RTPS writer failed to initialize a packet.

RTPS_LOG_WRITER_REQUEST_SENT_HISTORY_EC

```
#define RTPS_LOG_WRITER_REQUEST_SENT_HISTORY_EC (RTPS_LOG_BASE + 96)
```

RTPS reliable writer failed to request and resend history.

RTPS_LOG_BITMAP_SHIFT_EC

```
#define RTPS_LOG_BITMAP_SHIFT_EC (RTPS_LOG_BASE + 97)
```

Failed to shift bitmap to new lead sequence number.

RTPS_LOG_WINDOW_ADVANCE_EC

```
#define RTPS_LOG_WINDOW_ADVANCE_EC (RTPS_LOG_BASE + 98)
```

Send window of a remote reader failed to advance.

RTPS_LOG_WINDOW_ADVANCE_UNACKED_EC

```
#define RTPS_LOG_WINDOW_ADVANCE_UNACKED_EC (RTPS_LOG_BASE + 99)
```

Failed when advancing window ahead of unacknowledged sequence number.

RTPS_LOG_LOOKUP_PEER_EC

```
#define RTPS_LOG_LOOKUP_PEER_EC (RTPS_LOG_BASE + 100)
```

Failed to find peer that should exist.

RTPS_LOG_READER_SEND_ACKNACK_EC

```
#define RTPS_LOG_READER_SEND_ACKNACK_EC (RTPS_LOG_BASE + 101)
```

RTPS reader failed to send an ACKNACK message.

RTPS_LOG_FINALIZE_INTERFACE_EC

```
#define RTPS_LOG_FINALIZE_INTERFACE_EC (RTPS_LOG_BASE + 102)
```

Failed to finalize an RTPS interface.

RTPS_LOG_INDEXER_REMOVE_ENTRY_EC

```
#define RTPS_LOG_INDEXER_REMOVE_ENTRY_EC (RTPS_LOG_BASE + 103)
```

Failed to remove index entry for external interface upon deleting RTPS interface.

RTPS_LOG_LOOKUP_EXT_BIND_ENTRY_EC

```
#define RTPS_LOG_LOOKUP_EXT_BIND_ENTRY_EC (RTPS_LOG_BASE + 104)
```

Failed when looking up external bind entry from database.

RTPS_LOG_SEQ_INIT_EC

```
#define RTPS_LOG_SEQ_INIT_EC (RTPS_LOG_BASE + 105)
```

Failed to initialize a local sequence.

RTPS_LOG_SEQ_MAX_EC

```
#define RTPS_LOG_SEQ_MAX_EC (RTPS_LOG_BASE + 106)
```

Failed to set maximum of a local sequence.

RTPS_LOG_SEQ_LEN_EC

```
#define RTPS_LOG_SEQ_LEN_EC (RTPS_LOG_BASE + 107)
```

Failed to set length of a local sequence.

RTPS_LOG_INTF_MODE_UNDEF_EC

```
#define RTPS_LOG_INTF_MODE_UNDEF_EC (RTPS_LOG_BASE + 108)
```

Initializing an RTPS interface with undefined mode.

RTPS_LOG_DELETE_BUFFER_POOL_EC

```
#define RTPS_LOG_DELETE_BUFFER_POOL_EC (RTPS_LOG_BASE + 109)
```

Failed to delete buffer pool.

May still have buffers in use, thus cannot delete pool.

RTPS_LOG_CREATE_BUFFER_POOL_EC

```
#define RTPS_LOG_CREATE_BUFFER_POOL_EC (RTPS_LOG_BASE + 110)
```

Failed to create buffer pool.

May have insufficient memory to allocate pool.

RTPS_LOG_CREATE_PEERS_INDEX_EC

```
#define RTPS_LOG_CREATE_PEERS_INDEX_EC (RTPS_LOG_BASE + 111)
```

Failed to create index of peers.

System may have insufficient memory to allocate new index.

RTPS_LOG_DELETE_INDEXER_EC

```
#define RTPS_LOG_DELETE_INDEXER_EC (RTPS_LOG_BASE + 112)
```

Failed to delete indexer.

RTPS_LOG_GET_PEER_BUFFER_EC

```
#define RTPS_LOG_GET_PEER_BUFFER_EC (RTPS_LOG_BASE + 113)
```

Failed to get buffer from peer buffer pool.

RTPS_LOG_DB_CREATE_DIRECT_INDEX_EC

```
#define RTPS_LOG_DB_CREATE_DIRECT_INDEX_EC (RTPS_LOG_BASE + 114)
```

Failed to create database index for direct sends.

RTPS_LOG_DB_CREATE_GROUP_INDEX_EC

```
#define RTPS_LOG_DB_CREATE_GROUP_INDEX_EC (RTPS_LOG_BASE + 115)
```

Failed to create database index for group sends.

RTPS_LOG_NETIO_PACKET_INSUFFICIENT_BUFFER_EC

```
#define RTPS_LOG_NETIO_PACKET_INSUFFICIENT_BUFFER_EC (RTPS_LOG_BASE + 116)
```

Insufficient buffer in packet.

Receive buffer does not have enough space for a valid RTPS message

RTPS_LOG_PROCESS_DATA_BATCH_EC

```
#define RTPS_LOG_PROCESS_DATA_BATCH_EC (RTPS_LOG_BASE + 136)
```

Failed to process received DATA BATCH submessage.

This indicates an invalid DATA_BATCH submessage

RTPS_LOG_PROCESS_HEARTBEAT_BATCH_EC

```
#define RTPS_LOG_PROCESS_HEARTBEAT_BATCH_EC (RTPS_LOG_BASE + 137)
```

Failed to process received HEARTBEAT_BATCH submessage.

This indicates an invalid HEARTBEAT_BATCH submessage

RTPS_LOG_NEGATIVE_RESERVATION_COUNT_EC

```
#define RTPS_LOG_NEGATIVE_RESERVATION_COUNT_EC (RTPS_LOG_BASE + 138)
```

The reservation count is zero.

The reservation count should never be less than 0, this is an internal error.

RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_LOOKUP_FAILED_EC

```
#define RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_LOOKUP_FAILED_EC (RTPS_LOG_BASE + 139)
```

Failed to lookup a remote participant's interceptor handle.

An interceptor handle must be always supplied for a remote participant, if any encoding and/or decoding are to be performed. This is an internal error.

RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_CREATE_FAILED_EC

```
#define RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_CREATE_FAILED_EC (RTPS_LOG_BASE + 140)
```

Failed to create a record to store a remote participant's interceptor handle.

Interceptor handles for remote participants are stored in an indexed table for quicker access. This is an internal error.

RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_INSERT_FAILED_EC

```
#define RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_INSERT_FAILED_EC (RTPS_LOG_BASE + 141)
```

Failed to insert a record in the table storing interceptor handles of matched remote participants.

RTPS_LOG_TRUST_INTERCEPTOR_HANDLE_LIST_FULL_EC

```
#define RTPS_LOG_TRUST_INTERCEPTOR_HANDLE_LIST_FULL_EC (RTPS_LOG_BASE + 142)
```

Interceptor handle list out of resources.

RTPS_LOG_TRUST_INCONSISTENT_INTERCEPTOR_HANDLE_EC

```
#define RTPS_LOG_TRUST_INCONSISTENT_INTERCEPTOR_HANDLE_EC (RTPS_LOG_BASE + 143)
```

Inconsistent handles found for remote peer.

RTPS_LOG_TRUST_BIND_FAILED_EC

```
#define RTPS_LOG_TRUST_BIND_FAILED_EC (RTPS_LOG_BASE + 144)
```

Failed to bind resources for trust behavior.

RTPS_LOG_TRUST_SUBMSG_CONVERSION_FAILED_EC

```
#define RTPS_LOG_TRUST_SUBMSG_CONVERSION_FAILED_EC (RTPS_LOG_BASE + 145)
```

Failed to bind resources for trust behavior.

RTPS_LOG_TRUST_INVALID_SUBMSG_EC

```
#define RTPS_LOG_TRUST_INVALID_SUBMSG_EC (RTPS_LOG_BASE + 146)
```

A malformed trust message was received.

RTPS_LOG_TRUST_UNEXPECTED_READER_STATE_EC

```
#define RTPS_LOG_TRUST_UNEXPECTED_READER_STATE_EC (RTPS_LOG_BASE + 147)
```

The RTPS reader was in an unexpected state.

RTPS_LOG_DB_CREATE_SELECTED_GROUP_INDEX_EC

```
#define RTPS_LOG_DB_CREATE_SELECTED_GROUP_INDEX_EC (RTPS_LOG_BASE + 148)
```

Failed to create database index for the selected route group.

RTPS_LOG_LOCK_EC

```
#define RTPS_LOG_LOCK_EC (RTPS_LOG_BASE + 149)
```

Failed to take a lock.

RTPS_LOG_UNLOCK_EC

```
#define RTPS_LOG_UNLOCK_EC (RTPS_LOG_BASE + 150)
```

Failed to give a lock.

RTPS_LOG_SCHEDULE_PACKET_EC

```
#define RTPS_LOG_SCHEDULE_PACKET_EC (RTPS_LOG_BASE + 151)
```

Failed to schedule a packet.

RTPS_LOG_SEQ_FINALIZE_EC

```
#define RTPS_LOG_SEQ_FINALIZE_EC (RTPS_LOG_BASE + 152)
```

Failed to finalize a sequence.

RTPS_LOG_TRANSFORM_RTPS_EC

```
#define RTPS_LOG_TRANSFORM_RTPS_EC (RTPS_LOG_BASE + 153)
```

Failed to transform rtps message.

RTPS_LOG_RESTORE_TRUST_LOAN_BUFFER_EC

```
#define RTPS_LOG_RESTORE_TRUST_LOAN_BUFFER_EC (RTPS_LOG_BASE + 154)
```

Failed to restore trust loan buffer.

RTPS_LOG_CREATE_REASSEMBLY_TABLE_EC

```
#define RTPS_LOG_CREATE_REASSEMBLY_TABLE_EC (RTPS_LOG_BASE + 155)
```

Failed to create reassembly table.

RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM128_EC

```
#define RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM128_EC (RTPS_LOG_BASE + 200)
```

The property tries to override the BUILTIN128 checksum without setting allow_builtin_override to TRUE.

RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM32_EC

```
#define RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM32_EC (RTPS_LOG_BASE + 201)
```

The property tries to override the BUILTIN32 checksum without setting allow_builtin_override to TRUE.

RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM64_EC

```
#define RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM64_EC (RTPS_LOG_BASE + 202)
```

The property tries to override the BUILTIN64 checksum without setting allow_builtin_override to TRUE.

RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN128_EC

```
#define RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN128_EC (RTPS_LOG_BASE + 203)
```

The property override of the BUILTIN128 checksum is invalid. It is not legal to reuse any of the built-in functions.

RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN32_EC

```
#define RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN32_EC (RTPS_LOG_BASE + 204)
```

The property override of the BUILTIN32 checksum is invalid. It is not legal to reuse any of the built-in functions functions.

RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN64_EC

```
#define RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN64_EC (RTPS_LOG_BASE + 205)
```

The property override of the BUILTIN64 checksum is invalid. It is not legal to reuse any of the built-in functions functions.

RTPS_LOG_CHECKSUM_CHECKSUM_CALCULATION_FAILED_EC

```
#define RTPS_LOG_CHECKSUM_CHECKSUM_CALCULATION_FAILED_EC (RTPS_LOG_BASE + 210)
```

RTPS_Interface_calculate_checksum() failed to calculate the checksum.

RTPS_LOG_UNSUPPORTED_MANDTORY_HDR_EXT_PID_EC

```
#define RTPS_LOG_UNSUPPORTED_MANDTORY_HDR_EXT_PID_EC (RTPS_LOG_BASE + 211)
```

The HeaderExtension contained a mandatory PID that was not understood.

RTPS_LOG_INVALID_HDR_EXT_PID_LIST_EC

```
#define RTPS_LOG_INVALID_HDR_EXT_PID_LIST_EC (RTPS_LOG_BASE + 212)
```

An invalid HeaderExtension PID was detected.

RTPS_LOG_INVALID_HDR_EXT_PID_ERROR_EC

```
#define RTPS_LOG_INVALID_HDR_EXT_PID_ERROR_EC (RTPS_LOG_BASE + 213)
```

Failed to decode the HeaderExtension PID list.

RTPS_LOG_INCONSISTENT_HDR_EXT_LENGTH_ERROR_EC

```
#define RTPS_LOG_INCONSISTENT_HDR_EXT_LENGTH_ERROR_EC (RTPS_LOG_BASE + 214)
```

The HeaderExtension length is inconsistent with the decoded encoded HeaderExtension length.

RTPS_LOG_MESSAGE_EXCEED_LENGTH_ERROR_EC

```
#define RTPS_LOG_MESSAGE_EXCEED_LENGTH_ERROR_EC (RTPS_LOG_BASE + 215)
```

A RTPS message which was longer than the indicated message length in the RTPS header was received. This causes the rest of the RTPS message to be invalidated and the remaining submessages to be ignored.

RTPS_LOG_CHECKSUM_INVALID_CRC32_FLAGS_EC

```
#define RTPS_LOG_CHECKSUM_INVALID_CRC32_FLAGS_EC (RTPS_LOG_BASE + 216)
```

The CRC32 submsg flags are invalid.

RTPS_LOG_CHECKSUM_MISSING_CHECKSUM_LENGTH_EC

```
#define RTPS_LOG_CHECKSUM_MISSING_CHECKSUM_LENGTH_EC (RTPS_LOG_BASE + 217)
```

A header extension was received, but the checksum length is not present.

RTPS_LOG_CHECKSUM_UNSUPPORTED_EC

```
#define RTPS_LOG_CHECKSUM_UNSUPPORTED_EC (RTPS_LOG_BASE + 218)
```

Unsupported checksum bits received, message dropped.

RTPS_LOG_CHECKSUM_LENGTH_DESERIALIZE_ERROR_EC

```
#define RTPS_LOG_CHECKSUM_LENGTH_DESERIALIZE_ERROR_EC (RTPS_LOG_BASE + 219)
```

Checksum length could not be deserialized.

RTPS_LOG_CHECKSUM_INCONSISTENT_LENGTH_EC

```
#define RTPS_LOG_CHECKSUM_INCONSISTENT_LENGTH_EC (RTPS_LOG_BASE + 220)
```

The payload length is less than the deserialized checksum length.

RTPS_LOG_CHECKSUM_CHECKSUM_DESERIALIZE_ERROR_EC

```
#define RTPS_LOG_CHECKSUM_CHECKSUM_DESERIALIZE_ERROR_EC (RTPS_LOG_BASE + 221)
```

Checksum length could not be deserialized.

RTPS_LOG_CHECKSUM_CHECKSUM_ERROR_EC

```
#define RTPS_LOG_CHECKSUM_CHECKSUM_ERROR_EC (RTPS_LOG_BASE + 222)
```

The checksum is invalid.

RTPS_LOG_CHECKSUM_TOO_MANY_SG_BUFFERS_EC

```
#define RTPS_LOG_CHECKSUM_TOO_MANY_SG_BUFFERS_EC (RTPS_LOG_BASE + 223)
```

Found too many scatter-gather buffers when calculating checksum.

RTPS_LOG_SEND_LIVELINESS_EC

```
#define RTPS_LOG_SEND_LIVELINESS_EC (RTPS_LOG_BASE + 224)
```

Failure to send Liveliness packets.

RTPS_LOG_DELETE_RECORD_EC

```
#define RTPS_LOG_DELETE_RECORD_EC (RTPS_LOG_BASE + 225)
```

Failure to delete a record.

RTPS_LOG_APPLY_CONTENT_FILTER_EC

```
#define RTPS_LOG_APPLY_CONTENT_FILTER_EC (RTPS_LOG_BASE + 300)
```

Failed to apply a content filter to a message.

RTPS_LOG_UPDATE_RELIABLE_PEERS_FILTER_EC

```
#define RTPS_LOG_UPDATE_RELIABLE_PEERS_FILTER_EC (RTPS_LOG_BASE + 301)
```

Failed to update the state of reliable peers while applying filtering.

RTPS_LOG_ADD_FILTERED_ROUTE_EC

```
#define RTPS_LOG_ADD_FILTERED_ROUTE_EC (RTPS_LOG_BASE + 302)
```

Failed to add a filtered route.

RTPS_LOG_DELETE_FILTERED_ROUTE_EC

```
#define RTPS_LOG_DELETE_FILTERED_ROUTE_EC (RTPS_LOG_BASE + 303)
```

Failed to delete a filtered route.

6.20.9 DDS_C

DDS C. ModuleID = 7.

Macros

- #define **DDSC_LOG_INVALID_DURATION_EC** (DDSC_LOG_BASE + 1)
The specified duration is not valid.
- #define **DDSC_LOG_INVALID_PARTICIPANT_NAME_EC** (DDSC_LOG_BASE + 2)
An invalid participant name was specified with an unknown GUID.
- #define **DDSC_LOG_INVALID_PARTICIPANT_GUID_PREFIX_EC** (DDSC_LOG_BASE + 3)
An invalid participant GUID prefix was specified, typically the GUID prefix does not match an already detected participant.
- #define **DDSC_LOG_SYS_GETTIME_EC** (DDSC_LOG_BASE + 4)
A call to OSAPI_System_get_time failed.
- #define **DDSC_LOG_GET_NEXT_OBJECT_ID_EC** (DDSC_LOG_BASE + 5)
Failed to get the next automatically generated object ID for an entity's GUID. This typically means the object id pool has been exhausted.
- #define **DDSC_LOG_SET_ENTITY_NAME_EC** (DDSC_LOG_BASE + 6)
Failed to set name string for a DDS entity in the DomainParticipantQos entity_name policy.
- #define **DDSC_LOG_SYS_GET_HOSTNAME_EC** (DDSC_LOG_BASE + 7)
A call to OSAPI_System_get_hostname failed.
- #define **DDSC_LOG_IO_SNPRINTF_FAILED_EC** (DDSC_LOG_BASE + 8)
A call to OSAPI_Stdio_snprintf failed. Typically this means the destination buffer was too small.
- #define **DDSC_LOG_TOPIC_FIND_EC** (DDSC_LOG_BASE + 9)
Failed to find a topic created by a DomainParticipant.
- #define **DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_NAME_EC** (DDSC_LOG_BASE + 10)
Endpoint discovery failed because the name of the remote participant parent for an endpoint was not found.
- #define **DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_KEY_EC** (DDSC_LOG_BASE + 11)
Endpoint discovery failed because the key of the remote participant parent for an endpoint was not found. Note that the key is logged as 4 integers in host endianness format.
- #define **DDSC_LOG_REMOTE_PARTICIPANT_KEY_NOT_EQUAL_EC** (DDSC_LOG_BASE + 12)
Failed endpoint discovery when key does not match the remote participant.
- #define **DDSC_LOG_INVALID_ENDPOINT_GUID_EC** (DDSC_LOG_BASE + 13)

- Failed endpoint discovery due to an invalid or unknown endpoint GUID.*

 - #define `DDSC_LOG_ENDPOINT_NOT_CHILD_OF_PARTICIPANT_EC` (`DDSC_LOG_BASE` + 14)
- Failed endpoint discovery when an endpoint is determined to belong to a different remote participant.*

 - #define `DDSC_LOG_PARTICIPANT_DOES_NOT_EXIST_EC` (`DDSC_LOG_BASE` + 15)
- Failed participant discovery because a remote participant that should have been already asserted locally was not found.*

 - #define `DDSC_LOG_REFRESH_REM_PARTICIPANT_EC` (`DDSC_LOG_BASE` + 16)
- Did not find a remote participant when asserting participant liveness for it. Note that the key is logged as 4 integers in host endianness format.*

 - #define `DDSC_LOG_REFRESH_REM_PARTICIPANT_TIMEOUT_EC` (`DDSC_LOG_BASE` + 17)
- Failed to assert participant liveness to a remote participant.*

 - #define `DDSC_LOG_PARTICIPANT_LOOKUP_EC` (`DDSC_LOG_BASE` + 18)
- Failed to find a remote participant as previously discovered.*

 - #define `DDSC_LOG_REMOVE_PUBLICATION_EC` (`DDSC_LOG_BASE` + 19)
- Failed to remove resources for a remote publication.*

 - #define `DDSC_LOG_REMOVE_SUBSCRIPTION_EC` (`DDSC_LOG_BASE` + 20)
- Failed to remove resources for a remote subscription.*

 - #define `DDSC_LOG_FIND_PUBLICATION_PARENT_EC` (`DDSC_LOG_BASE` + 21)
- Cannot determine the participant of a remote publication.*

 - #define `DDSC_LOG_FIND_SUBSCRIPTION_PARENT_EC` (`DDSC_LOG_BASE` + 22)
- Cannot determine the participant of a remote subscription.*

 - #define `DDSC_LOG_MAX_PARTICIPANT_ID_REACHED_EC` (`DDSC_LOG_BASE` + 23)
- Failed to create DomainParticipant due to running out of participant IDs. This is typically caused by all UDP ports being used.*

 - #define `DDSC_LOG_RESERVE_LOCATORS_EC` (`DDSC_LOG_BASE` + 24)
- Failed to reserve endpoint locators.*

 - #define `DDSC_LOG_TIMER_CREATE_TIMEOUT_EC` (`DDSC_LOG_BASE` + 25)
- Failed to create a timeout.*

 - #define `DDSC_LOG_ILLEGAL_OBJECTID_EC` (`DDSC_LOG_BASE` + 26)
- Illegal object id specified.*

 - #define `DDSC_LOG_TOPIC_NARROW_EC` (`DDSC_LOG_BASE` + 27)
- Failed to narrow a TopicDescription to the named Topic.*

 - #define `DDSC_LOG_FAILED_UPDATE_STATUS_CONDITION_EC` (`DDSC_LOG_BASE` + 28)
- Failed to update state of StatusCondition.*

 - #define `DDSC_LOG_WS_REMOVE_COND_REFERENCE_EC` (`DDSC_LOG_BASE` + 29)
- Failed to remove a condition reference from a waitset.*

 - #define `DDSC_LOG_WS_ADD_COND_REFERENCE_EC` (`DDSC_LOG_BASE` + 30)
- Failed to add a condition reference to a waitset.*

 - #define `DDSC_LOG_RELEASE_META_MC_EC` (`DDSC_LOG_BASE` + 31)
- Failed to release resources for multicast discovery locators.*

 - #define `DDSC_LOG_RELEASE_META_UC_EC` (`DDSC_LOG_BASE` + 32)
- Failed to release resources for unicast discovery locators.*

 - #define `DDSC_LOG_RELEASE_USER_MC_EC` (`DDSC_LOG_BASE` + 33)
- Failed to release resources for multicast user locators.*

 - #define `DDSC_LOG_RELEASE_USER_UC_EC` (`DDSC_LOG_BASE` + 34)
- Failed to release resources for unicast user locators.*

 - #define `DDSC_LOG_INVALID_DOMAINID_EC` (`DDSC_LOG_BASE` + 35)
- The domain ID specified exceeds what is allowed based on the parameters specified in `dp_qos.protocol.rtps_well_known_ports`.*

- #define DDSC_LOG_ON_TYPE_REGISTERED_FAILURE_EC (DDSC_LOG_BASE + 36)
Error occurred during the on_type_registered call back.
- #define DDSC_LOG_ON_TYPE_UNREGISTERED_FAILURE_EC (DDSC_LOG_BASE + 37)
Error occurred during the on_type_unregistered call back.
- #define DDSC_LOG_GET_SERIALIZED_KEY_SIZE_EC (DDSC_LOG_BASE + 38)
Error occurred during the on_type_unregistered call back.
- #define DDSC_LOG_MIN_DISCOVERY_MTU_EC (DDSC_LOG_BASE + 39)
Minimum MTU across all Discovery transports must be greater or equal to a packet containing serialized Participant↵BuiltinTopicData.
- #define DDSC_LOG_MIN_MTU_SIZE_EC (DDSC_LOG_BASE + 40)
MTU for a transport set lower or equal to Protocol Overhead of 448 bytes.
- #define DDSC_LOG_GET_MTU_NO_ROUTES_EC (DDSC_LOG_BASE + 41)
MTU could not be found because of no routes.
- #define DDSC_LOG_RESOLVE_LOCATORS_PRIORITY_EC (DDSC_LOG_BASE + 42)
Failed to resolve locators based on priority.
- #define DDSC_LOG_REMOTE_PUBLICATION_DATA_CHANGED_EC (DDSC_LOG_BASE + 43)
The publication builtin topic data for a remote publication changed after the remote publication was asserted.
- #define DDSC_LOG_REMOTE_SUBSCRIPTION_DATA_CHANGED_EC (DDSC_LOG_BASE + 44)
Fields in subscription builtin topic data for a remote subscription which are not supported to dynamically change were updated after the remote subscription was asserted.
- #define DDSC_LOG_GET_MTU_EC (DDSC_LOG_BASE + 45)
Error getting the mtu from the route resolver.
- #define DDSC_LOG_DATABASE_CREATE_EC (DDSC_LOG_BASE + 100)
Failed to create database.
- #define DDSC_LOG_DATABASE_DELETE_EC (DDSC_LOG_BASE + 101)
Failed to delete database.
- #define DDSC_LOG_TABLE_CREATE_EC (DDSC_LOG_BASE + 102)
Failed to create database table of the specified name.
- #define DDSC_LOG_TABLE_INUSE_EC (DDSC_LOG_BASE + 103)
Failed to delete a database table because it is not empty.
- #define DDSC_LOG_TABLE_DELETE_EC (DDSC_LOG_BASE + 104)
Failed to delete a database table.
- #define DDSC_LOG_TABLE_SELECT_EC (DDSC_LOG_BASE + 105)
A selection operation failed on the specified database table.
- #define DDSC_LOG_CREATE_INDEX_EC (DDSC_LOG_BASE + 106)
Failed to create an index on a database table.
- #define DDSC_LOG_DELETE_INDEX_EC (DDSC_LOG_BASE + 107)
Failed to delete an index on a database table.
- #define DDSC_LOG_DB_CURSOR_INVALIDATED_EC (DDSC_LOG_BASE + 108)
A database table cursor was invalidated while in use.
- #define DDSC_LOG_SUBSCRIPTION_RECORD 1
Database record for a Subscription.
- #define DDSC_LOG_PUBLICATION_RECORD 2
Database record for a Publication.
- #define DDSC_LOG_PARTICIPANT_RECORD 3
Database record for a Remote Participant.
- #define DDSC_LOG_TOPIC_RECORD 4

- *Database record for a Topic.*
- #define `DDSC_LOG_DATAWRITER_RECORD` 5
- *Database record for a DataWriter.*
- #define `DDSC_LOG_DATAREADER_RECORD` 6
- *Database record for a DataReader.*
- #define `DDSC_LOG_PUBLISHER_RECORD` 7
- *Database record for a Publisher.*
- #define `DDSC_LOG_SUBSCRIBER_RECORD` 8
- *Database record for a Subscriber.*
- #define `DDSC_LOG_ROUTE_RECORD` 9
- *Database record for a route record.*
- #define `DDSC_LOG_BIND_RECORD` 10
- *Database record for a bind record.*
- #define `DDSC_LOG_TYPE_RECORD` 11
- *Database record for a type plugin.*
- #define `DDSC_LOG_REMOTE_ENDPOINT_TRUST_STATE_RECORD` 12
- *Database record for a remote trust endpoint.*
- #define `DDSC_LOG_MATCHED_DW_RECORD` 13
- *Database record for a matched remote data writer.*
- #define `DDSC_LOG_STRING_RECORD` 14
- *Database record for a string property.*
- #define `DDSC_LOG_RECORD_CREATE_EC` (`DDSC_LOG_BASE` + 109)
- *Failed to create a database record of the specified kind.*
- #define `DDSC_LOG_RECORD_DELETE_EC` (`DDSC_LOG_BASE` + 110)
- *Failed to delete a database record of the specified kind.*
- #define `DDSC_LOG_RECORD_INSERT_EC` (`DDSC_LOG_BASE` + 111)
- *Failed to insert a database record of the specified kind.*
- #define `DDSC_LOG_RECORD_ERROR_EC` (`DDSC_LOG_BASE` + 112)
- *Unknown error for database record of the specified kind.*
- #define `DDSC_LOG_RECORD_EXISTS_EC` (`DDSC_LOG_BASE` + 113)
- *A database record of the specified kind already exists.*
- #define `DDSC_LOG_RECORD_LOOKUP_EC` (`DDSC_LOG_BASE` + 114)
- *A lookup of a database record of the specified kind failed.*
- #define `DDSC_LOG_RECORD_NOT_EXISTS_EC` (`DDSC_LOG_BASE` + 115)
- *A database record of the specified kind does not exist.*
- #define `DDSC_LOG_RECORD_SELECT_EC` (`DDSC_LOG_BASE` + 116)
- *A database select on the specified record kind failed.*
- #define `DDSC_LOG_RECORD_REMOVE_EC` (`DDSC_LOG_BASE` + 117)
- *Removal a database record of the specified kind failed.*
- #define `DDSC_LOG_RECORD_INITIALIZE_EC` (`DDSC_LOG_BASE` + 118)
- *A database record of the specified kind could not be initialized.*
- #define `DDSC_LOG_RECORD_FINALIZE_EC` (`DDSC_LOG_BASE` + 119)
- *A database record of the specified kind could not be finalized.*
- #define `DDSC_LOG_RECORD_RESET_EC` (`DDSC_LOG_BASE` + 120)
- *A database record of the specified kind could not be reset.*
- #define `DDSC_LOG_PUBLICATIONDATA_OBJECT` 1
- *DDS PublicationBuiltinTopicData object.*

- #define [DDSC_LOG_SUBSCRIPTIONDATA_OBJECT](#) 2
DDS SubscriptionBuiltinTopicData object.
- #define [DDSC_LOG_PARTICIPANTDATA_OBJECT](#) 3
DDS ParticipantBuiltinTopicData object.
- #define [DDSC_LOG_DATAREADERQOS_OBJECT](#) 4
DDS DataReaderQos object.
- #define [DDSC_LOG_DATAWRITERQOS_OBJECT](#) 5
DDS DataWriterQos object.
- #define [DDSC_LOG_TOPICQOS_OBJECT](#) 6
DDS TopicQos object.
- #define [DDSC_LOG_PUBLISHERQOS_OBJECT](#) 7
DDS PublisherQos object.
- #define [DDSC_LOG_SUBSCRIBERQOS_OBJECT](#) 8
DDS SubscriberQos object.
- #define [DDSC_LOG_PARTICIPANTQOS_OBJECT](#) 9
DDS DomainParticipantQos object.
- #define [DDSC_LOG_ENTITY_OBJECT](#) 10
DDS Entity object.
- #define [DDSC_LOG_PARTICIPANTFACTORYQOS_OBJECT](#) 11
DDS DomainParticipantFactoryQos object.
- #define [DDSC_LOG_TYPE_OBJECT](#) 12
A DDS Type object.
- #define [DDSC_LOG_BINDRESOLVER_OBJECT](#) 13
A NETIO BindResolver object.
- #define [DDSC_LOG_ROUTERESOLVER_OBJECT](#) 14
A NETIO RouteResolver object.
- #define [DDSC_LOG_ADDRESSRESOLVER_OBJECT](#) 15
A NETIO AddressResolver object.
- #define [DDSC_LOG_DATAWRITERIO_OBJECT](#) 16
A NETIO DataWriterInterface object.
- #define [DDSC_LOG_DATAREADERIO_OBJECT](#) 17
A NETIO DataReaderInterface object.
- #define [DDSC_LOG_LOG_OBJECT](#) 18
A OSAPI Log object.
- #define [DDSC_LOG_SYSTEM_OBJECT](#) 19
A OSAPI system object.
- #define [DDSC_LOG_MUTEX_OBJECT](#) 20
A OSAPI mutex object.
- #define [DDSC_LOG_CONDREF_OBJECT](#) 21
A DDS Waitset condition reference object.
- #define [DDSC_LOG_WSREF_OBJECT](#) 22
A DDS Condition waitset reference object.
- #define [DDSC_LOG_MD5STREAM_OBJECT](#) 23
A MD5 stream object.
- #define [DDSC_LOG_CONDITION_OBJECT](#) 24
A DDS Condition object.
- #define [DDSC_LOG_RT_OBJECT](#) 25

- *A RT Condition object.*
- #define `DDSC_LOG_PARTICIPANT_POOL_OBJECT` 26
- *A Participant pool object.*
- #define `DDSC_LOG_TIMER_OBJECT` 27
- *A OSAPI_Timer object.*
- #define `DDSC_LOG_TIMEROUT_OBJECT` 28
- *A OSAPI_Timer timeout object.*
- #define `DDSC_LOG_TOPIC_OBJECT` 29
- *A DDS Topic object.*
- #define `DDSC_LOG_WAITSET_OBJECT` 30
- *A DDS WaitSet object.*
- #define `DDSC_LOG_UUID_OBJECT` 31
- *A UUID object.*
- #define `DDSC_LOG_MEMPOOL_OBJECT` 32
- *A REDA_MemPool object.*
- #define `DDSC_LOG_PARTICIPANT_PROPERTIES` 33
- *A DomainParticipant's string properties sequence.*
- #define `DDSC_LOG_PARTICIPANT_BINARY_PROPERTIES` 34
- *A DomainParticipant's binary properties sequence.*
- #define `DDSC_LOG_PARTICIPANT_BUILTIN_PUBLISHER` 35
- *A DomainParticipant's built-in Publisher.*
- #define `DDSC_LOG_PARTICIPANT_BUILTIN_SUBSCRIBER` 36
- *A DomainParticipant's built-in Subscriber.*
- #define `DDSC_LOG_PARTICIPANT_GENERIC_MESSAGE_OBJECT` 37
- *A built-in Inter-Participant Channel data type object.*
- #define `DDSC_LOG_DATA HOLDER_SEQ_OBJECT` 38
- *A Trust data holder sequence object.*
- #define `DDSC_LOG_PROPERTYQOSPOLICY_OBJECT` 39
- *A Property QoS policy object.*
- #define `DDSC_LOG_STRING_OBJECT` 40
- *A String object.*
- #define `DDSC_LOG_AUTHPOOL_OBJECT` 41
- *An authentication pool object.*
- #define `DDSC_LOG_BUFFER_OBJECT` 42
- *A Log buffer object.*
- #define `DDSC_LOG_STRINGMANAGER_OBJECT` 43
- *A String Manager object.*
- #define `DDSC_LOG_OBJECT_INITIALIZE_EC` (`DDSC_LOG_BASE` + 200)
- *Out of resources to initialize object of the specified kind.*
- #define `DDSC_LOG_OBJECT_ALLOCATE_EC` (`DDSC_LOG_BASE` + 201)
- *Out of resources to allocate an object of the specified kind.*
- #define `DDSC_LOG_OBJECT_FINALIZE_EC` (`DDSC_LOG_BASE` + 202)
- *Failed to finalize object of specified kind.*
- #define `DDSC_LOG_OBJECT_DELETE_EC` (`DDSC_LOG_BASE` + 203)
- *Failed to delete object of specified kind.*
- #define `DDSC_LOG_OBJECT_COPY_EC` (`DDSC_LOG_BASE` + 204)
- *Failed to copy object of specified kind.*

- `#define DDSC_LOG_OBJECT_REFCOUNT_EC (DDSC_LOG_BASE + 205)`
Failed to delete/finalize an object because other objects are referencing it.
- `#define DDSC_LOG_OBJECT_GET_PROPERTY_EC (DDSC_LOG_BASE + 206)`
Failed to get the object properties.
- `#define DDSC_LOG_OBJECT_SET_PROPERTY_EC (DDSC_LOG_BASE + 207)`
Failed to set the object properties.
- `#define DDSC_LOG_OBJECT_EMPTY_EC (DDSC_LOG_BASE + 208)`
An object is empty, typically applies only to buffer-pool objects.
- `#define DDSC_LOG_OBJECT_INUSECOUNT_EC (DDSC_LOG_BASE + 209)`
Failed to delete/finalize an object because other objects are using it.
- `#define DDSC_LOG_OBJECT_CREATE_EC (DDSC_LOG_BASE + 210)`
Failed to create an object.
- `#define DDSC_LOG_NETMASK_SEQUENCE 1`
A NETIO Netmask sequence.
- `#define DDSC_LOG_ROUTE_SEQUENCE 2`
A NETIO Route sequence.
- `#define DDSC_LOG_RESERVED_SEQUENCE 3`
A NETIO Reserved address sequence.
- `#define DDSC_LOG_ENABLED_USER_TRANSPORT_SEQUENCE 4`
A DDS sequence of enabled user NETIO transports.
- `#define DDSC_LOG_ENABLED_TRANSPORT_SEQUENCE 5`
A DDS sequence of enabled transports.
- `#define DDSC_LOG_ENABLED_DISCOVERY_TRANSPORT_SEQUENCE 6`
A DDS sequence of enabled discovery transports.
- `#define DDSC_LOG_INITIAL_PEER_SEQUENCE 7`
A DDS sequence of initial peers.
- `#define DDSC_LOG_DESTINATION_SEQUENCE 8`
A NETIO sequence of destinations to send to.
- `#define DDSC_LOG_METAUNICAST_SEQUENCE 9`
A DDS sequence of meta-traffic unicast locators.
- `#define DDSC_LOG_METAMULTICAST_SEQUENCE 10`
A DDS sequence of meta-traffic multicast locators.
- `#define DDSC_LOG_USERUNICAST_SEQUENCE 11`
A DDS sequence of user-traffic unicast locators.
- `#define DDSC_LOG_USERMULTICAST_SEQUENCE 12`
A DDS sequence of user-traffic multicast locators.
- `#define DDSC_LOG_READTAKE_SEQUENCE 13`
A DDS sequence used in a read/take call.
- `#define DDSC_LOG_DATAHOLDER_SEQUENCE 14`
A DDS sequence of DataHolder objects.
- `#define DDSC_LOG_COOKIE_PAYLOAD_SEQUENCE 15`
A cookie octet sequence.
- `#define DDSC_LOG_REPRESENTATION_ID_SEQUENCE 16`
a sequence for data representation IDs
- `#define DDSC_LOG_SEQUENCE_SETMAX_EC (DDSC_LOG_BASE + 300)`
Failed to set the maximum length of a sequence of the specified kind.
- `#define DDSC_LOG_SEQUENCE_SETLENGTH_EC (DDSC_LOG_BASE + 301)`

- Failed to set the length of a sequence of the specified kind.*

 - #define `DDSC_LOG_SEQUENCE_GETREF_EC` (`DDSC_LOG_BASE` + 302)
- Failed to get a reference at the specified index for a sequence of the specified kind.*

 - #define `DDSC_LOG_SEQUENCE_INITIALIZE_EC` (`DDSC_LOG_BASE` + 303)
- Failed to initialize a sequence of the specified kind.*

 - #define `DDSC_LOG_SEQUENCE_FINALIZE_EC` (`DDSC_LOG_BASE` + 304)
- Failed to finalize a sequence of the specified kind.*

 - #define `DDSC_LOG_SEQUENCE_COPY_EC` (`DDSC_LOG_BASE` + 305)
- Failed to copy a sequence of the specified kind.*

 - #define `DDSC_LOG_SEQUENCE_INVALID_EC` (`DDSC_LOG_BASE` + 306)
- The sequence of the specified kind was invalid in the context it is used.*

 - #define `DDSC_LOG_DISCOVERY_COMPONENT` 1
- A component of discovery kind.*

 - #define `DDSC_LOG_RTPS_COMPONENT` 2
- A component of RTPS kind.*

 - #define `DDSC_LOG_READERHISTORY_COMPONENT` 3
- A component of reader-history kind.*

 - #define `DDSC_LOG_WRITERHISTORY_COMPONENT` 4
- A component of writer-history kind.*

 - #define `DDSC_LOG_DATAREADERIO_COMPONENT` 5
- A component of DataReaderInterface kind.*

 - #define `DDSC_LOG_DATAWRITERIO_COMPONENT` 6
- A component of DataWriterInterface kind.*

 - #define `DDSC_LOG_NETIO_COMPONENT` 7
- A component of NETIO kind.*

 - #define `DDSC_LOG_TRANSPORT_COMPONENT` 8
- A component of transport kind.*

 - #define `DDSC_LOG_APPGEN_COMPONENT` 9
- A component of application generation kind.*

 - #define `DDSC_LOG_COMPONENT_LOOKUP_EC` (`DDSC_LOG_BASE` + 400)
- Did not find a component factory with the given name in the registry.*

 - #define `DDSC_LOG_COMPONENT_CREATE_EC` (`DDSC_LOG_BASE` + 401)
- Could not create a component of the specified kind using the specified factory.*

 - #define `DDSC_LOG_COMPONENT_DELETE_EC` (`DDSC_LOG_BASE` + 402)
- Could not delete a component of the specified kind using the specified factory.*

 - #define `DDSC_LOG_HISTORY_RESOURCE` 1
- Exceeded resource limits for writer history queue.*

 - #define `DDSC_LOG_SAMPLE_RESOURCES` 2
- Failed to get resource for new sample due to resource limit.*

 - #define `DDSC_LOG_PARTICIPANT_RESOURCES` 3
- Failed to allocate a new participant.*

 - #define `DDSC_LOG_RESOURCE_EXCEEDED_EC` (`DDSC_LOG_BASE` + 500)
- Could not allocate a resource of the specified kind.*

 - #define `DDSC_LOG_TOPIC_QOS` 1
- The TopicQos kind.*

 - #define `DDSC_LOG_DATAREADER_QOS` 2
- The DataReaderQos kind.*

- #define [DDSC_LOG_DATAWRITER_QOS](#) 3
The DataWriterQos kind.
- #define [DDSC_LOG_SUBSCRIBER_QOS](#) 4
The SubscriberQos kind.
- #define [DDSC_LOG_PUBLISHER_QOS](#) 5
The PublisherQos kind.
- #define [DDSC_LOG_PARTICIPANT_QOS](#) 6
The DomainParticipantQos kind.
- #define [DDSC_LOG_PARTICIPANTFACTORY_QOS](#) 7
The DomainParticipantFactoryQos kind.
- #define [DDSC_LOG_DEFAULTTOPIC_QOS](#) 8
The default TopicQos kind.
- #define [DDSC_LOG_DEFAULTDATAREADER_QOS](#) 9
The default DataReaderQos kind.
- #define [DDSC_LOG_DEFAULTDATAWRITER_QOS](#) 10
The default DataWriterQos kind.
- #define [DDSC_LOG_DEFAULTSUBSCRIBER_QOS](#) 11
The default SubscriberQos kind.
- #define [DDSC_LOG_DEFAULTPUBLISHER_QOS](#) 12
The default PublisherQos kind.
- #define [DDSC_LOG_DEFAULTPARTICIPANT_QOS](#) 13
The default PublisherQos kind.
- #define [DDSC_LOG_DEADLINE_QOS_POLICY](#) 14
The deadline qos policy kind.
- #define [DDSC_LOG_LIVELINESS_QOS_POLICY](#) 15
The liveliness qos policy kind.
- #define [DDSC_LOG_HISTORY_QOS_POLICY](#) 16
The history qos policy kind.
- #define [DDSC_LOG_RESOURCE_LIMIT_QOS_POLICY](#) 17
The resource limits qos policy kind.
- #define [DDSC_LOG_PROTOCOL_QOS_POLICY](#) 18
The protocol qos policy kind.
- #define [DDSC_LOG_TYPE_SUPPORT_QOS_POLICY](#) 19
The type-support qos policy kind.
- #define [DDSC_LOG_RELIABILITY_QOS_POLICY](#) 20
The reliability qos policy kind.
- #define [DDSC_LOG_DURABILITY_QOS_POLICY](#) 21
The durability qos policy kind.
- #define [DDSC_LOG_OWNERSHIP_QOS_POLICY](#) 22
The ownership qos policy kind.
- #define [DDSC_LOG_OWNERSHIP_STRENGTH_QOS_POLICY](#) 23
The ownership-strength qos policy kind.
- #define [DDSC_LOG_TRANSPORT_QOS_POLICY](#) 24
The transport qos policy kind.
- #define [DDSC_LOG_PARTICIPANT_ID_QOS_POLICY](#) 25
The participant id qos policy kind.
- #define [DDSC_LOG_HEARTBEATS_QOS_POLICY](#) 26

- The heartbeat qos policy kind.*

 - #define [DDSC_LOG_DATAWRITER_RESOURCE_QOS_POLICY](#) 27
- The DataWriterQos writer_resource_limits policy.*

 - #define [DDSC_LOG_DATAREADER_RESOURCE_QOS_POLICY](#) 28
- The DataReaderQos reader_resource_limits policy.*

 - #define [DDSC_LOG_DESTINATION_ORDER_POLICY](#) 29
- The DestinationOrderQos destination_order policy.*

 - #define [DDSC_LOG_PROPERTY_QOS_POLICY](#) 30
- The PropertyQos policy.*

 - #define [DDSC_LOG_PUBLISH_MODE_QOS_POLICY](#) 31
- The PublishModeQos policy.*

 - #define [DDSC_LOG_DATA_REPRESENTATION_QOS_POLICY](#) 33
- The DataRepresentationQos policy.*

 - #define [DDSC_LOG_LATENCY_BUDGET_QOS_POLICY](#) 34
- The LatencyBudgetQos policy.*

 - #define [DDSC_LOG_CONTENT_FILTER_QOS_POLICY](#) 35
- The ContentFilterQos policy.*

 - #define [DDSC_LOG_QOS_INCONSISTENT_POLICY_EC](#) ([DDSC_LOG_BASE](#) + 501)
- An inconsistent Qos policy for the specified Qos kind was found.*

 - #define [DDSC_LOG_QOS_INCONSISTENT_POLICIES_EC](#) ([DDSC_LOG_BASE](#) + 502)
- Inconsistency between two Qos policies for the specified Qos kind was found.*

 - #define [DDSC_LOG_QOS_INCONSISTENT_EC](#) ([DDSC_LOG_BASE](#) + 503)
- Failed to create an entity or set a qos due to inconsistent policy.*

 - #define [DDSC_LOG_QOS_COPY_EC](#) ([DDSC_LOG_BASE](#) + 504)
- Failed to copy a Qos of the specified kind.*

 - #define [DDSC_LOG_QOS_INITIALIZE_EC](#) ([DDSC_LOG_BASE](#) + 505)
- Failed to initialize a Qos of the specified kind.*

 - #define [DDSC_LOG_QOS_FINALIZE_EC](#) ([DDSC_LOG_BASE](#) + 506)
- Failed to finalize a Qos of the specified kind.*

 - #define [DDSC_LOG_QOS_SET_EC](#) ([DDSC_LOG_BASE](#) + 507)
- Failed to set a Qos of the specified kind.*

 - #define [DDSC_LOG_QOS_SET_ON_ENABLED_EC](#) ([DDSC_LOG_BASE](#) + 508)
- Failed to set a Qos of the specified kind because the entity is already enabled.*

 - #define [DDSC_LOG_QOS_IMMUTABLE_EC](#) ([DDSC_LOG_BASE](#) + 509)
- Failed to set a Qos of the specified kind the immutable Qos policies have been changed.*

 - #define [DDSC_LOG_QOS_CHANGED_EC](#) ([DDSC_LOG_BASE](#) + 510)
- A discovered Qos changed (the entity already existed)*

 - #define [DDSC_LOG_QOS_GET_EC](#) ([DDSC_LOG_BASE](#) + 511)
- Failed to get a Qos of the specified kind.*

 - #define [DDSC_LOG_TOPIC_LISTENER](#) 1
- The topic listener kind.*

 - #define [DDSC_LOG_DATAREADER_LISTENER](#) 2
- The datareader listener kind.*

 - #define [DDSC_LOG_DATAWRITER_LISTENER](#) 3
- The datawriter listener kind.*

 - #define [DDSC_LOG_SUBSCRIBER_LISTENER](#) 4
- The subscriber listener kind.*

- #define DDSC_LOG_PUBLISHER_LISTENER 5
The publisher listener kind.
- #define DDSC_LOG_PARTICIPANT_LISTENER 6
The participant listener kind.
- #define DDSC_LOG_PARTICIPANTFACTORY_LISTENER 7
The participant-factory listener kind.
- #define DDSC_LOG_LISTENER_INCONSISTENT_EC (DDSC_LOG_BASE + 512)
Failed to create an entity due to inconsistent listener and status mask.
- #define DDSC_LOG_LISTENER_SET_EC (DDSC_LOG_BASE + 513)
Failed to set the listener of the specified kind.
- #define DDSC_LOG_LISTENER_GET_EC (DDSC_LOG_BASE + 514)
Failed to get the listener of the specified kind.
- #define DDSC_LOG_LISTENER_SET_ILLEGAL_NULL_EC (DDSC_LOG_BASE + 515)
Illegal combination of NULL listener and non-NONE status mask when setting a listener for an Entity.
- #define DDSC_LOG_TOPIC_ENTITY 1
The topic entity kind.
- #define DDSC_LOG_DATAWRITER_ENTITY 2
The datawriter entity kind.
- #define DDSC_LOG_DATAREADER_ENTITY 3
The datareader entity kind.
- #define DDSC_LOG_PUBLISHER_ENTITY 4
The publisher entity kind.
- #define DDSC_LOG_SUBSCRIBER_ENTITY 5
The subscriber entity kind.
- #define DDSC_LOG_PARTICIPANT_ENTITY 6
The participant entity kind.
- #define DDSC_LOG_PUBLICATION_ENTITY 7
The publication entity kind.
- #define DDSC_LOG_SUBSCRIPTION_ENTITY 8
The subscription entity kind.
- #define DDSC_LOG_ENTITY_ENABLE_EC (DDSC_LOG_BASE + 600)
Failed to enable an entity of the specified kind.
- #define DDSC_LOG_ENTITY_NOT_EMPTY_EC (DDSC_LOG_BASE + 601)
Failed to an delete/finalize an entity of the specified kind because it is not empty.
- #define DDSC_LOG_ENTITY_FINALIZE_EC (DDSC_LOG_BASE + 602)
Failed to finalize an entity of the specified kind.
- #define DDSC_LOG_ENTITY_INITIALIZE_EC (DDSC_LOG_BASE + 603)
Failed to initialize an entity of the specified kind.
- #define DDSC_LOG_ENTITY_NOT_ENABLED_EC (DDSC_LOG_BASE + 604)
An operation was attempted on an entity that is not enabled.
- #define DDSC_LOG_ENTITY_DIFFERENT_FACTORY_EC (DDSC_LOG_BASE + 605)
Entities are in different factories.
- #define DDSC_LOG_ENTITY_INVALID_PROPERTY_EC (DDSC_LOG_BASE + 606)
An invalid property was specified for the entity.
- #define DDSC_LOG_DATAWRITER_CDR 1
The datawriter CDR kind.
- #define DDSC_LOG_DATAREADER_CDR 2

- *The datareader CDR kind.*
- #define `DDSC_LOG_DATAWRITER_INLINE` 3
- *The datawriter inline kind.*
- #define `DDSC_LOG_CDR_POOL_ALLOC_EC` (`DDSC_LOG_BASE` + 700)
- *Failed to allocate a pool of the specified kind.*
- #define `DDSC_LOG_CDR_BUFFER_SET_EC` (`DDSC_LOG_BASE` + 701)
- *Failed to set the CDR buffer for a packet.*
- #define `DDSC_LOG_CDR_POOL_DELETE_EC` (`DDSC_LOG_BASE` + 702)
- *Failed to delete the CDR pool.*
- #define `DDSC_LOG_CDR_SERIALIZE_PID_EC` (`DDSC_LOG_BASE` + 703)
- *Failed to serialize a parameter ID.*
- #define `DDSC_LOG_CDR_SERIALIZE_PID_LENGTH_EC` (`DDSC_LOG_BASE` + 704)
- *Failed to serialize a parameter length.*
- #define `DDSC_LOG_CDR_SERIALIZE_KEYHASH_EC` (`DDSC_LOG_BASE` + 705)
- *Failed to serialize a key-hash.*
- #define `DDSC_LOG_CDR_SERIALIZE_DATA_EC` (`DDSC_LOG_BASE` + 706)
- *Failed to serialize payload data.*
- #define `DDSC_LOG_DESERIALIZE_BAD_PID_LENGTH_EC` (`DDSC_LOG_BASE` + 707)
- *Deserialized an invalid parameter length for a specific parameter ID.*
- #define `DDSC_LOG_CDR_DESERIALIZE_PID_EC` (`DDSC_LOG_BASE` + 708)
- *Failed to deserialize the ID of an inline parameter.*
- #define `DDSC_LOG_CDR_DESERIALIZE_PID_LENGTH_EC` (`DDSC_LOG_BASE` + 709)
- *Failed to deserialize the length of an inline parameter.*
- #define `DDSC_LOG_CDR_INCREMENT_POS_EC` (`DDSC_LOG_BASE` + 710)
- *Failed to increment to the position of the next inline parameter.*
- #define `DDSC_LOG_CDR_SET_POS_EC` (`DDSC_LOG_BASE` + 711)
- *Failed to set the reception stream position.*
- #define `DDSC_LOG_CDR_DESERIALIZE_HEADER_EC` (`DDSC_LOG_BASE` + 712)
- *Failed to deserialize the encapsulation header.*
- #define `DDSC_LOG_CDR_DESERIALIZE_DATA_EC` (`DDSC_LOG_BASE` + 713)
- *Failed to deserialize CDR payload data.*
- #define `DDSC_LOG_CDR_INITIALIZE_SAMPLE_EC` (`DDSC_LOG_BASE` + 714)
- *Failed to initialize CDR sample.*
- #define `DDSC_LOG_CDR_FINALIZE_SAMPLE_EC` (`DDSC_LOG_BASE` + 715)
- *Failed to finalize sample.*
- #define `DDSC_LOG_CDR_SERIALIZE_STATUS_INFO_EC` (`DDSC_LOG_BASE` + 716)
- *Failed to serialize the status info parameter.*
- #define `DDSC_LOG_CDR_DESERIALIZE_KEYHASH_EC` (`DDSC_LOG_BASE` + 717)
- *Failed to deserialize a key-hash.*
- #define `DDSC_LOG_CDR_DESERIALIZE_KEY_EC` (`DDSC_LOG_BASE` + 718)
- *Failed to deserialize CDR payload key.*
- #define `DDSC_LOG_NETIO_ADD_ANON_TOPIC_ROUTE_EC` (`DDSC_LOG_BASE` + 800)
- *Failed to add a route to an anonymous participant discovery datawriter.*
- #define `DDSC_LOG_NETIO_ADD_TOPIC_ROUTE_EC` (`DDSC_LOG_BASE` + 801)
- *Failed to add a route to a topic from a datawriter.*
- #define `DDSC_LOG_NETIO_DELETE_TOPIC_ROUTE_EC` (`DDSC_LOG_BASE` + 802)
- *Failed to delete a route to a topic.*

- #define `DDSC_LOG_NETIO_FORWARD_TOPIC_EC` (`DDSC_LOG_BASE` + 803)
Failed to forward a topic.
- #define `DDSC_LOG_NETIO_NETIO_KIND` 1
Any NETIO interface kind, typically UDP.
- #define `DDSC_LOG_INTRA_NETIO_KIND` 2
The intra interface kind.
- #define `DDSC_LOG_RTPS_NETIO_KIND` 3
The RTPS interface kind.
- #define `DDSC_LOG_DATAREADER_NETIO_KIND` 4
The DataReader interface kind.
- #define `DDSC_LOG_DATAWRITER_NETIO_KIND` 5
The DataWriter interface kind.
- #define `DDSC_LOG_NETIO_BIND_EXTERNAL_EC` (`DDSC_LOG_BASE` + 804)
Failed to bind two external interface of the specified kind.
- #define `DDSC_LOG_NETIO_UNBIND_EXTERNAL_EC` (`DDSC_LOG_BASE` + 805)
Failed to unbind two external interfaces of the specified kind.
- #define `DDSC_LOG_NETIO_BIND_EC` (`DDSC_LOG_BASE` + 806)
Failed to bind an interface to a peer interface.
- #define `DDSC_LOG_NETIO_UNBIND_EC` (`DDSC_LOG_BASE` + 807)
Failed to unbind an interface from a peer interface.
- #define `DDSC_LOG_NETIO_ADD_ROUTE_EC` (`DDSC_LOG_BASE` + 808)
Failed to add a route from an interface to a peer interface.
- #define `DDSC_LOG_NETIO_DELETE_ROUTE_EC` (`DDSC_LOG_BASE` + 809)
Failed to delete a route from an interface to a peer interface.
- #define `DDSC_LOG_NETIO_GET_EXTERNAL_INTF_EC` (`DDSC_LOG_BASE` + 810)
Failed to get an external interface for the specified interface kind.
- #define `DDSC_LOG_NETIO_NO_ROUTE_EC` (`DDSC_LOG_BASE` + 811)
A DataReader failed a bind due to no existing route.
- #define `DDSC_LOG_NETIO_ROUTE_LOOKUP_FAILED_EC` (`DDSC_LOG_BASE` + 812)
Lookup a route to a destination failed.
- #define `DDSC_LOG_NETIO_GET_ROUTE_TABLE_FAILED_EC` (`DDSC_LOG_BASE` + 813)
Failed to get the route table for an interface.
- #define `DDSC_LOG_NETIO_SEND_FAILED_EC` (`DDSC_LOG_BASE` + 814)
Failure when sending on an interface.
- #define `DDSC_LOG_NETIO_SET_STATE_EC` (`DDSC_LOG_BASE` + 815)
Failed to set an interface state.
- #define `DDSC_LOG_NETIO_PEER_LOOKUP_EC` (`DDSC_LOG_BASE` + 816)
Datawriter did not find a peer.
- #define `DDSC_LOG_NETIO_FORCED_REMOVE_EC` (`DDSC_LOG_BASE` + 817)
Forced removal of sample downstream failed.
- #define `DDSC_LOG_PACKET_INIT_EC` (`DDSC_LOG_BASE` + 818)
Failed to initialize a packet.
- #define `DDSC_LOG_PACKET_SET_HEAD_EC` (`DDSC_LOG_BASE` + 819)
Failed to set the head of a packet.
- #define `DDSC_LOG_PACKET_SET_TAIL_EC` (`DDSC_LOG_BASE` + 820)
Failed to set the tail of a packet.
- #define `DDSC_LOG_NETIO_DISCOVERY_ENABLED_TRANSPORT_EC` (`DDSC_LOG_BASE` + 821)

- Configured discovery enabled transport is not valid.*

 - #define DDSC_LOG_NETIO_PACKETPOOL_RETURN_EC (DDSC_LOG_BASE + 822)
Failed to return a packet to the packet pool.
 - #define DDSC_LOG_NETIO_PACKETPOOL_GET_EC (DDSC_LOG_BASE + 823)
Failed to get a packet from the packet pool.
 - #define DDSC_LOG_NETIO_PACKETPOOL_CREATE_EC (DDSC_LOG_BASE + 824)
Failed to create packet pool.
 - #define DDSC_LOG_NETIO_PACKETPOOL_DELETE_EC (DDSC_LOG_BASE + 825)
Failed to delete packet pool.
 - #define DDSC_LOG_DW_ACKNACK_FAILED_EC (DDSC_LOG_BASE + 900)
Failed to ACKNACK sample in the writer history.
 - #define DDSC_LOG_DW_COMMIT_EC (DDSC_LOG_BASE + 901)
Failed to commit a sample to the writer queue.
 - #define DDSC_LOG_DW_KEYHASH_CREATE_EC (DDSC_LOG_BASE + 902)
Failed to create keyhash of instance handle.
 - #define DDSC_LOG_DW_ILLEGAL_KEY_KIND_EC (DDSC_LOG_BASE + 903)
Failed a write due to an invalid key kind for the type being written.
 - #define DDSC_LOG_DW_CREATE_TYPED_WRITER_EC (DDSC_LOG_BASE + 904)
Failed to create a typed writer.
 - #define DDSC_LOG_DW_HISTORY_REGISTER_KEY_EC (DDSC_LOG_BASE + 905)
Failed to register the key of an instance.
 - #define DDSC_LOG_DW_UNKNOWN_FLOW_CONTROLLER_EC (DDSC_LOG_BASE + 906)
Failed to create a flow-controller.
 - #define DDSC_LOG_DW_FLOW_CONTROLLER_REQUIRED_EC (DDSC_LOG_BASE + 907)
The data-sample is larger then supported by the transport, but the flow-controller has been compiled out.
 - #define DDSC_LOG_DW_INSTANCE_ASSERTION_FAILED_EC (DDSC_LOG_BASE + 908)
Failed to reserve an instance to publish discovery data for a user created data writer. This typically means Domain← ParticipantQos.resource_limits.remote_writer_allocation is too small.
 - #define DDSC_LOG_DW_INCOMPATIBLE_PADDING_EC (DDSC_LOG_BASE + 909)
A DataReader incompatible with padding bits in encapsulation header was discovered.
 - #define DDSC_LOG_DR_CREATE_TYPED_READER_EC (DDSC_LOG_BASE + 1000)
Failed to create a typed datareader.
 - #define DDSC_LOG_DR_COPY_DATA_SAMPLE_EC (DDSC_LOG_BASE + 1001)
Failed to copy a sample upon reception, read, or take.
 - #define DDSC_LOG_DR_COMMIT_SAMPLE_EC (DDSC_LOG_BASE + 1002)
Failed to commit a sample to be made available to be read or taken.
 - #define DDSC_LOG_DR_FILTER_ERROR_EC (DDSC_LOG_BASE + 1003)
A datareader filter function failed.
 - #define DDSC_LOG_DR_DESERIALIZE_KEYHASH_EC (DDSC_LOG_BASE + 1004)
Failed to deserialize a key-hash parameter.
 - #define DDSC_LOG_DR_GET_ENTRY_FAILED_EC (DDSC_LOG_BASE + 1005)
Failed to get a Reader History entry for a received sample.
 - #define DDSC_LOG_DR_COMMIT_ENTRY_EC (DDSC_LOG_BASE + 1006)
Failed to commit a receive sample to Reader History to be read or taken.
 - #define DDSC_LOG_DR_UNREGISTER_KEY_EC (DDSC_LOG_BASE + 1007)
A DataReader failed to unregister an instance.
 - #define DDSC_LOG_DR_DISPOSE_KEY_EC (DDSC_LOG_BASE + 1008)

- *A DataReader failed to dispose an instance.*
- #define DDSC_LOG_DR_READ_TAKE_FAILURE_EC (DDSC_LOG_BASE + 1009)
- *A call to a reader/take function failed.*
- #define DDSC_LOG_DR_INSTANCE_TO_KEYHASH_EC (DDSC_LOG_BASE + 1010)
- *Failed to create keyhash of instance handle.*
- #define DDSC_LOG_DR_INSTANCE_MAPPING_EXHAUSTED_EC (DDSC_LOG_BASE + 1011)
- *Failed to create keyhash of instance handle.*
- #define DDSC_LOG_DR_INSTANCE_ASSERTION_FAILED_EC (DDSC_LOG_BASE + 1012)
- *Failed to reserve an instance to publish discovery data for a user created data reader. This typically means DomainParticipantQos.resource_limits.remote_reader_allocation is too small.*
- #define DDSC_LOG_DR_ON_SAMPLE_REMOVED_EC (DDSC_LOG_BASE + 1013)
- *Failed to remove a sample when it was being pushed out from the Reader History.*
- #define DDSC_LOG_TYPE_NAME_CMP_EC (DDSC_LOG_BASE + 1100)
- *Two type names are incompatible.*
- #define DDSC_LOG_TOPIC_NAME_CMP_EC (DDSC_LOG_BASE + 1101)
- *Two topic names are incompatible.*
- #define DDSC_LOG_TYPE_FUNCTION_GET_SERIALIZED_SAMPLE_MAX_SIZE 1
- *The get_serialized_sample_max_size function kind.*
- #define DDSC_LOG_TYPE_FUNCTION_SERIALIZE_DATA 2
- *The serialize_data function kind.*
- #define DDSC_LOG_TYPE_FUNCTION_DESERIALIZE_DATA 3
- *The deserialize_data function kind.*
- #define DDSC_LOG_TYPE_FUNCTION_CREATE_SAMPLE 4
- *The create_sample function kind.*
- #define DDSC_LOG_TYPE_FUNCTION_COPY_SAMPLE 5
- *The copy_sample function kind.*
- #define DDSC_LOG_TYPE_FUNCTION_DELETE_SAMPLE 6
- *The delete_sample function kind.*
- #define DDSC_LOG_TYPE_FUNCTION_GET_KEY_KIND 7
- *The get_key_kind function kind.*
- #define DDSC_LOG_TYPE_FUNCTION_INSTANCE_TO_KEYHASH 8
- *The instance_to_keyhash function kind.*
- #define DDSC_LOG_TYPE_FUNCTION_NULL_EC (DDSC_LOG_BASE + 1102)
- *Invalid type plugin, The specified function pointer is NULL.*
- #define DDSC_LOG_TOPIC_TOO_LONG_EC (DDSC_LOG_BASE + 1103)
- *Failed to create a topic because the name exceeded the maximum length of 255 octets (excluding the terminating NUL)*
- #define DDSC_LOG_TYPE_TOO_LONG_EC (DDSC_LOG_BASE + 1104)
- *Failed to create a type because the name exceeded the maximum length of 255 octets (excluding the terminating NUL)*
- #define DDSC_LOG_LOOKUP_TYPE_PLUGIN_EC (DDSC_LOG_BASE + 1105)
- *A type-plugin for the given type could not be found.*
- #define DDSC_LOG_TYPE_KEY_TYPE_EC (DDSC_LOG_BASE + 1106)
- *Two types have incompatible keys.*
- #define DDSC_LOG_TYPE_PLUGIN_GET_BUFFER_EC (DDSC_LOG_BASE + 1107)
- *Type-plugin cannot return a buffer.*
- #define DDSC_LOG_TYPE_PLUGIN_INCONSISTENT_DR_EC (DDSC_LOG_BASE + 1108)
- *Inconsistent type-plugin DataReader.*
- #define DDSC_LOG_DUPLICATE_GUID_PREFIX_EC (DDSC_LOG_BASE + 1109)

- A DomainParticipant with the specified GUID already exists in the DomainParticipant factory.*

 - #define `DDSC_LOG_TYPE_MANAGED_POOL_FAILURE_EC` (`DDSC_LOG_BASE` + 1110)

Managed pool function failed.
- #define `DDSC_LOG_TYPE_GET_SAMPLE_ID_EC` (`DDSC_LOG_BASE` + 1111)

Managed pool failed to get sample id.
- #define `DDSC_LOG_PARTICIPANT_NAME_EMPTY_EC` (`DDSC_LOG_BASE` + 1112)

A DomainParticipant with enable_participant_discovery_by_name set to to TRUE but an empty name is created.
- #define `DDSC_LOG_DUPLICATE_PARTICIPANT_NAME_EC` (`DDSC_LOG_BASE` + 1113)

A DomainParticipant with enable_participant_discovery_by_name set to to TRUE, but a local participant with the same DomainParticipantQos.participant_name.name already exists.
- #define `DDSC_LOG_DUPLICATE_REMOTE_PARTICIPANT_NAME_EC` (`DDSC_LOG_BASE` + 1114)

A remote participant is discovered with the same name as the a local participant name in the same domain ID.
- #define `DDSC_LOG_REMOTE_PARTICIPANT_NAME_EMPTY_EC` (`DDSC_LOG_BASE` + 1115)

A remote participant without a name is discovered.
- #define `DDSC_LOG_REMOTE_PARTICIPANT_RESTART_EC` (`DDSC_LOG_BASE` + 1116)

Failed to reset a remote participant.
- #define `DDSC_LOG_DISC_LOCAL_PARTICIPANT_ENABLED_EC` (`DDSC_LOG_BASE` + 1200)

Discovery plugin failed its update after a local DomainParticipant was enabled.
- #define `DDSC_LOG_DISC_BEFORE_LOCAL_PARTICIPANT_CREATED_EC` (`DDSC_LOG_BASE` + 1201)

Discovery plugin failed its update before a local DomainParticipant was created.
- #define `DDSC_LOG_DISC_AFTER_LOCAL_PARTICIPANT_CREATED_EC` (`DDSC_LOG_BASE` + 1202)

Discovery plugin failed its update after a local DomainParticipant was created.
- #define `DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_ENABLED_EC` (`DDSC_LOG_BASE` + 1203)

Discovery plugin failed its update after a local DataReader was enabled.
- #define `DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_DELETED_EC` (`DDSC_LOG_BASE` + 1204)

Discovery plugin failed its update after a local DataReader was deleted.
- #define `DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_ENABLED_EC` (`DDSC_LOG_BASE` + 1205)

Discovery plugin failed its update after a local DataWriter was enabled.
- #define `DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_DELETED_EC` (`DDSC_LOG_BASE` + 1206)

Discovery plugin failed its update after a local DataWriter was deleted.
- #define `DDSC_LOG_DISC_BEFORE_REMOTE_PARTICIPANT_DELETED_EC` (`DDSC_LOG_BASE` + 1207)

Discovery plugin failed its update after being notified that a remote DomainParticipant was about to be deleted.
- #define `DDSC_LOG_DISC_ADD_PEER_EC` (`DDSC_LOG_BASE` + 1208)

Failed to add a peer with discovery plugin.
- #define `DDSC_LOG_DESERIALIZE_UNKNOWN_PID_EC` (`DDSC_LOG_BASE` + 1209)

Deserialized a parameter of unknown ID.
- #define `DDSC_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC` (`DDSC_LOG_BASE` + 1210)

Failed to serialize Builtin Endpoint Mask parameter.
- #define `DDSC_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC` (`DDSC_LOG_BASE` + 1211)

Failed to deserialize Builtin Endpoint Mask parameter.
- #define `DDSC_LOG_SERIALIZE_TOPIC_NAME_EC` (`DDSC_LOG_BASE` + 1212)

Failed to serialize Topic Name parameter.
- #define `DDSC_LOG_CDR_SET_OFFSET_EC` (`DDSC_LOG_BASE` + 1213)

Failed to set current offset of stream.
- #define `DDSC_LOG_SERIALIZE_ENTITY_NAME_EC` (`DDSC_LOG_BASE` + 1214)

Failed to serialize Entity Name parameter.
- #define `DDSC_LOG_SERIALIZE_TYPE_NAME_EC` (`DDSC_LOG_BASE` + 1215)

- Failed to serialize Type Name parameter.*

 - #define DDSC_LOG_SERIALIZE_GUID_EC (DDSC_LOG_BASE + 1216)
- Failed to serialize GUID key.*

 - #define DDSC_LOG_SERIALIZE_DEFAULT_UNICAST_EC (DDSC_LOG_BASE + 1217)
- Failed to serialize Default Unicast Locator parameter.*

 - #define DDSC_LOG_DESERIALIZE_LOCATOR_EC (DDSC_LOG_BASE + 1218)
- Failed to deserialize a locator of the specified kind.*

 - #define DDSC_LOG_LOCATORS_FULL_EC (DDSC_LOG_BASE + 1219)
- A locator sequence is full, the remaining locators of the specified kind are dropped.*

 - #define DDSC_LOG_UNSUPPORTED_LOCATOR_EC (DDSC_LOG_BASE + 1220)
- The locator kind is not supported by any transport registered with the domain participant and is dropped.*

 - #define DDSC_LOG_SERIALIZE_PROTOCOL_VERSION_EC (DDSC_LOG_BASE + 1221)
- Failed to serialize Protocol Version parameter.*

 - #define DDSC_LOG_DESERIALIZE_PROTOCOL_VERSION_EC (DDSC_LOG_BASE + 1222)
- Failed to deserialize Protocol Version parameter.*

 - #define DDSC_LOG_DESERIALIZE_VENDOR_ID_EC (DDSC_LOG_BASE + 1223)
- Failed to deserialize Vendor ID parameter.*

 - #define DDSC_LOG_SERIALIZE_VENDOR_ID_EC (DDSC_LOG_BASE + 1224)
- Failed to serialize Vendor ID parameter.*

 - #define DDSC_LOG_SERIALIZE_PRODUCT_VERSION_EC (DDSC_LOG_BASE + 1225)
- Failed to serialize Product Version parameter.*

 - #define DDSC_LOG_SERIALIZE_LEASE_DURATION_EC (DDSC_LOG_BASE + 1226)
- Failed to serialize Lease Duration parameter.*

 - #define DDSC_LOG_DATAWRITER_ADD_PEER_FAILED_EC (DDSC_LOG_BASE + 1227)
- Failed to add a remote peer to a local DataWriter.*

 - #define DDSC_LOG_DATAREADER_ADD_PEER_FAILED_EC (DDSC_LOG_BASE + 1228)
- Failed to add a remote peer to a local DataReader.*

 - #define DDSC_LOG_DISC_REMOTE_PARTICIPANT_REMOVE_EC (DDSC_LOG_BASE + 1229)
- Failed to remote/reset remote participant after liveliness expired.*

 - #define DDSC_LOG_DESERIALIZE_PARTITION_SEQ_EC (DDSC_LOG_BASE + 1230)
- Failed to deserialize partition seq.*

 - #define DDSC_LOG_INITIALIZE_DISCOVERY_QUEUE_EC (DDSC_LOG_BASE + 1231)
- Failed to initialize discovery queue.*

 - #define DDSC_LOG_QUEUE_PUBLICATION_DISCOVERY_EC (DDSC_LOG_BASE + 1232)
- Failed to queue a publication discovery message.*

 - #define DDSC_LOG_QUEUE_SUBSCRIPTION_DISCOVERY_EC (DDSC_LOG_BASE + 1233)
- Failed to queue a subscription discovery message.*

 - #define DDSC_LOG_QUEUE_FINALIZE_EC (DDSC_LOG_BASE + 1234)
- Failed finalize discovery queue.*

 - #define DDSC_LOG_IPC_INIT_FAILED_EC (DDSC_LOG_BASE + 1300)
- Failed to initialize Builtin IPC entities.*

 - #define DDSC_LOG_IPC_ALLOC_FAILED_EC (DDSC_LOG_BASE + 1301)
- Failed to allocate memory for one of the Builtin IPC entities.*

 - #define DDSC_LOG_IPC_FINALIZE_FAILED_EC (DDSC_LOG_BASE + 1302)
- Failed to finalize all resources A potential memory leak might have been caused by unexpected errors during finalization of acquired resources.*

 - #define DDSC_LOG_IPC_CHANNEL_TYPE_REGISTER_EC (DDSC_LOG_BASE + 1303)

- Failed to register type for a Builtin IPC channel.*

 - #define `DDSC_LOG_IPC_CHANNEL_TOPIC_CREATE_EC` (`DDSC_LOG_BASE` + 1304)
- Failed to create topic for a Builtin IPC channel.*

 - #define `DDSC_LOG_IPC_CHANNEL_READER_CREATE_EC` (`DDSC_LOG_BASE` + 1305)
- Failed to create reader for a Builtin IPC channel.*

 - #define `DDSC_LOG_IPC_CHANNEL_WRITER_CREATE_EC` (`DDSC_LOG_BASE` + 1306)
- Failed to create writer for a Builtin IPC channel.*

 - #define `DDSC_LOG_IPC_CHANNEL_TYPE_PLUGIN_EC` (`DDSC_LOG_BASE` + 1307)
- Failed to find type plugin for a Builtin IPC channel.*

 - #define `DDSC_LOG_IPC_CHANNEL_UNKNOWN_EC` (`DDSC_LOG_BASE` + 1308)
- Unknown builtin IPC channel requested.*

 - #define `DDSC_LOG_IPC_UPDATE_QOS_FAILED_EC` (`DDSC_LOG_BASE` + 1309)
- Failed to update DomainParticipantQos to account for built-in IPC endpoints.*

 - #define `DDSC_LOG_IPC_CHANNEL_ADD_ANON_ROUTE_EC` (`DDSC_LOG_BASE` + 1310)
- Failed to add an anonymous route to a built-in DataWriter or DataReader.*

 - #define `DDSC_LOG_IPC_CHANNEL_DELETE_ANON_ROUTE_EC` (`DDSC_LOG_BASE` + 1311)
- Failed to delete an anonymous route to a built-in DataWriter or DataReader.*

 - #define `DDSC_LOG_IPC_CHANNEL_ADD_PEER_EC` (`DDSC_LOG_BASE` + 1312)
- Failed to add an inter-participant channel peer.*

 - #define `DDSC_LOG_IPC_CHANNEL_REMOVE_PEER_EC` (`DDSC_LOG_BASE` + 1313)
- Failed to remove an inter-participant channel peer.*

 - #define `DDSC_LOG_IPC_CHANNEL_RETURN_LOAN_EC` (`DDSC_LOG_BASE` + 1314)
- Failed to return a loaned inter-participant sample.*

 - #define `DDSC_LOG_IPC_CHANNEL_PROCESS_SAMPLE_EC` (`DDSC_LOG_BASE` + 1315)
- Failed to process inter-participant message.*

 - #define `DDS_LOG_IPC_ASSERT_ROUTES_EC` (`DDSC_LOG_BASE` + 1316)
- Failed to assert inter-participant routes.*

 - #define `DDSC_LOG_IPC_CHANNEL_WRITE_SAMPLE_EC` (`DDSC_LOG_BASE` + 1317)
- Failed to write inter-participant message.*

 - #define `DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_AUTH_HANDSHAKE_FAILED_EC` (`DDSC_LOG_BASE` + 1318)
- Failed to process handshake authentication.*

 - #define `DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_PARTICIPANT_INTERCEPTOR_STATE_FAILED_EC` (`DDSC_LOG_BASE` + 1319)
- Failed to process participant interceptor tokens.*

 - #define `DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_ENDPOINT_INTERCEPTOR_STATE_FAILED_EC` (`DDSC_LOG_BASE` + 1320)
- Failed to process endpoint interceptor tokens.*

 - #define `DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_AUTH_REQUEST_FAILED_EC` (`DDSC_LOG_BASE` + 1321)
- Failed to process authentication request.*

 - #define `DDSC_LOG_IPC_CHANNEL_CONFIG_INITIALIZE_FAILED_EC` (`DDSC_LOG_BASE` + 1322)
- Failed to initialize inter-participant channel.*

 - #define `DDSC_LOG_IPC_CHANNEL_CONFIG_FINALIZE_FAILED_EC` (`DDSC_LOG_BASE` + 1323)
- Failed to get inter-participant channel configuration.*

 - #define `DDSC_LOG_IPC_CHANNEL_CONFIG_SET_FAILED_EC` (`DDSC_LOG_BASE` + 1324)
- Failed to set inter-participant channel listeners.*

 - #define `DDSC_LOG_IPC_CHANNEL_SET_LISTENERS_FAILED_EC` (`DDSC_LOG_BASE` + 1325)

- `#define DDSC_LOG_IPC_CHANNEL_ENABLE_FAILED_EC (DDSC_LOG_BASE + 1326)`
Failed to enable inter-participant channel entities.
- `#define DDSC_LOG_IPC_CHANNEL_CREATE_FAILED_EC (DDSC_LOG_BASE + 1327)`
Failed to create inter-participant channel.
- `#define DDSC_LOG_TRUST_UNKNOWN_GMCLASSID_EC (DDSC_LOG_BASE + 1401)`
Unknown generic message class id.
- `#define DDSC_LOG_TRUST_UNKNOWN_SERVICEID_EC (DDSC_LOG_BASE + 1402)`
Unknown builtin service id.
- `#define DDSC_LOG_TRUST_IGNORED_REMOTE_PARTICIPANT_EC (DDSC_LOG_BASE + 1403)`
Ignored remote participant.
- `#define DDSC_LOG_TRUST_IGNORED_HANDSHAKE_MESSAGE_EC (DDSC_LOG_BASE + 1404)`
Ignored handshake message from remote participant.
- `#define DDSC_LOG_TRUST_UNEXPECTED_VALIDATION_RESULT_EC (DDSC_LOG_BASE + 1405)`
Unexpected validation result.
- `#define DDSC_LOG_TRUST_UNAUTHORIZED_REMOTE_PARTICIPANT_EC (DDSC_LOG_BASE + 1406)`
A remote participant failed the authorization process.
- `#define DDSC_LOG_TRUST_FAILED_VALIDATE_LOCAL_IDENTITY_EC (DDSC_LOG_BASE + 1407)`
Failed to validate local identity.
- `#define DDSC_LOG_TRUST_INVALID_IDENTITY_HANDLE_EC (DDSC_LOG_BASE + 1408)`
Invalid local trust identity.
- `#define DDSC_LOG_TRUST_GET_IDENTITY_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1409)`
Failed to get identity token.
- `#define DDSC_LOG_TRUST_INVALID_PERMISSIONS_HANDLE_EC (DDSC_LOG_BASE + 1410)`
Invalid local trust permissions.
- `#define DDSC_LOG_TRUST_INVALID_PARTICIPANT_GUID_EC (DDSC_LOG_BASE + 1411)`
Invalid DomainParticipant GUID configuration.
- `#define DDSC_LOG_TRUST_GET_PARTICIPANT_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1412)`
Failed to get DomainParticipant trust attributes.
- `#define DDSC_LOG_TRUST_CHECK_CREATE_PARTICIPANT_FAILED_EC (DDSC_LOG_BASE + 1413)`
Failed to create access control DomainParticipant.
- `#define DDSC_LOG_TRUST_GET_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1414)`
Failed to get permissions credential token.
- `#define DDSC_LOG_TRUST_GET_PERMISSIONS_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1415)`
Failed to get permissions token.
- `#define DDSC_LOG_TRUST_INVALID_INTERCEPTOR_HANDLE_EC (DDSC_LOG_BASE + 1416)`
Invalid interceptor handle.
- `#define DDSC_LOG_TRUST_REGISTER_LOCAL_PARTICIPANT_FAILED_EC (DDSC_LOG_BASE + 1417)`
Failed to register local DomainParticipant.
- `#define DDSC_LOG_TRUST_RETURN_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1418)`
Failed to return permissions credential token.
- `#define DDSC_LOG_TRUST_RETURN_PERMISSIONS_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1419)`
Failed to return permissions token.
- `#define DDSC_LOG_TRUST_RETURN_IDENTITY_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1420)`
Failed to return identity token.

- #define DDSC_LOG_TRUST_UNREGISTER_PARTICIPANT_FAILED_EC (DDSC_LOG_BASE + 1421)
Failed to unregister local participant.
- #define DDSC_LOG_TRUST_RETURN_PERMISSIONS_HANDLE_FAILED_EC (DDSC_LOG_BASE + 1422)
Failed to return permissions handle.
- #define DDSC_LOG_TRUST_RETURN_IDENTITY_HANDLE_FAILED_EC (DDSC_LOG_BASE + 1423)
Failed to return identity handle.
- #define DDSC_LOG_TRUST_RETURN_SHAREDSECRET_HANDLE_FAILED_EC (DDSC_LOG_BASE + 1424)
Failed to return shared secret handle.
- #define DDSC_LOG_TRUST_GET_AUTHENTICATED_PEER_CREDENTIAL_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1425)
Failed to get authenticated peer credential token.
- #define DDSC_LOG_TRUST_CHECK_REMOTE_PARTICIPANT_FAILED_EC (DDSC_LOG_BASE + 1426)
Failed to check access control remote DomainParticipant.
- #define DDSC_LOG_TRUST_GET_SHARED_SECRET_FAILED_EC (DDSC_LOG_BASE + 1427)
Failed to get shared secret.
- #define DDSC_LOG_TRUST_PREPARE_LOCAL_PARTICIPANT_STATE_FAILED_EC (DDSC_LOG_BASE + 1428)
Failed to validate local DomainParticipant trust.
- #define DDSC_LOG_TRUST_DELETE_TRUST_PLUGINS_EC (DDSC_LOG_BASE + 1429)
Failed to delete trust plugins.
- #define DDSC_LOG_TRUST_SET_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1430)
Failed to set permissions credential and token.
- #define DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_PARTICIPANT_FAILED_EC (DDSC_LOG_BASE + 1431)
Failed to register matched remote DomainParticipant.
- #define DDSC_LOG_TRUST_GET_TOPIC_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1432)
Failed to get topic trust attributes.
- #define DDSC_LOG_TRUST_CHECK_CREATE_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1433)
Failed to create trust check DataWriter.
- #define DDSC_LOG_TRUST_RETURN_DATAWRITER_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1434)
Failed to return DataWriter trust attributes.
- #define DDSC_LOG_TRUST_GET_DATAWRITER_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1435)
Failed to get DataWriter trust attributes.
- #define DDSC_LOG_TRUST_GET_DATAREADER_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1436)
Failed to get DataReader trust attributes.
- #define DDSC_LOG_TRUST_CHECK_CREATE_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1437)
Failed to create trust check DataReader.
- #define DDSC_LOG_TRUST_BEGIN_HANDSHAKE_REPLY_EC (DDSC_LOG_BASE + 1438)
Handshake begin reply failed.
- #define DDSC_LOG_TRUST_CREATE_LOCAL_PARTICIPANT_INTERCEPTOR_TOKENS_EC (DDSC_LOG_BASE + 1439)
Failed to create local DomainParticipant interceptor tokens.
- #define DDSC_LOG_TRUST_CREATE_LOCAL_DW_TOKEN_EC (DDSC_LOG_BASE + 1440)

- Failed to create local DataWriter interceptor token.*

 - #define DDSC_LOG_TRUST_CREATE_LOCAL_DR_TOKEN_EC (DDSC_LOG_BASE + 1441)
- Failed to create local DataReader interceptor token.*

 - #define DDSC_LOG_TRUST_RETURN_HANDSHAKE_HANDLE_FAILED_EC (DDSC_LOG_BASE + 1442)
- Failed to return handshake handle.*

 - #define DDSC_LOG_TRUST_SET_REMOTE_PARTICIPANT_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1443)
- Failed to set remote DomainParticipant interceptor tokens.*

 - #define DDSC_LOG_TRUST_SET_REMOTE_DATAREADER_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1444)
- Failed to set remote DataReader interceptor tokens.*

 - #define DDSC_LOG_TRUST_SET_REMOTE_DATAWRITER_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1445)
- Failed to set remote DataWriter interceptor tokens.*

 - #define DDSC_LOG_TRUST_REGISTER_LOCAL_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1446)
- Failed to register local DataReader.*

 - #define DDSC_LOG_TRUST_REGISTER_LOCAL_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1447)
- Failed to register local DataWriter.*

 - #define DDSC_LOG_TRUST_CHECK_REMOTE_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1448)
- Failed to check remote DataWriter.*

 - #define DDSC_LOG_TRUST_CHECK_REMOTE_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1449)
- Failed to check remote DataReader.*

 - #define DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1450)
- Failed to register matched remote DataWriter.*

 - #define DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1451)
- Failed to register matched remote DataReader.*

 - #define DDSC_LOG_TRUST_UNREGISTER_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1452)
- Failed to unregister DataReader.*

 - #define DDSC_LOG_TRUST_UNREGISTER_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1453)
- Failed to unregister DataWriter.*

 - #define DDSC_LOG_TRUST_VALIDATE_REMOTE_IDENTITY_EC (DDSC_LOG_BASE + 1454)
- Failed to validate remote identity.*

 - #define DDSC_LOG_TRUST_BEGIN_HANDSHAKE_REQUEST_EC (DDSC_LOG_BASE + 1455)
- Handshake begin request failed.*

 - #define DDSC_LOG_TRUST_PROCESS_HANDSHAKE_EC (DDSC_LOG_BASE + 1456)
- Handshake process failed.*

 - #define DDSC_LOG_TRUST_CREATE_TRUST_PLUGINS_FAILED_EC (DDSC_LOG_BASE + 1457)
- Failed to create trust plugins.*

 - #define DDSC_LOG_TRUST_PARSE_PROPERTY_FAILED_EC (DDSC_LOG_BASE + 1458)
- Failed to parse trust property.*

 - #define DDSC_LOG_TRUST_AFTER_REMOTE_PARTICIPANT_READY_FAILED_EC (DDSC_LOG_BASE + 1459)
- Error after remote trust DomainParticipant is ready.*

 - #define DDSC_LOG_TRUST_REMOVE_REMOTE_PARTICIPANT_EC (DDSC_LOG_BASE + 1460)
- Failed to remove remote DomainParticipant.*

- #define `DDS_LOG_TRUST_ASSERT_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC` (`DDSC_LOG_BASE + 1461`)
Failed to assert DomainParticipant generic message.
- #define `DDSC_LOG_TRUST_SEND_HANDSHAKE_EC` (`DDSC_LOG_BASE + 1462`)
Failed to send handshake message.
- #define `DDSC_LOG_VALIDATE_REMOTE_PARTICIPANT_TRUST_FAILED_EC` (`DDSC_LOG_BASE + 1463`)
Failed to validate trust remote DomainParticipant.
- #define `DDSC_LOG_TRUST_BIND_REMOTE_DATAREADER_FAILED_EC` (`DDSC_LOG_BASE + 1464`)
Failed to get remote DataReader bind properties.
- #define `DDSC_LOG_TRUST_REGISTER_REMOTE_DATAREADER_FAILED_EC` (`DDSC_LOG_BASE + 1465`)
Failed to register remote DataReader.
- #define `DDSC_LOG_TRUST_BIND_REMOTE_DATAWRITER_FAILED_EC` (`DDSC_LOG_BASE + 1466`)
Failed to get remote DataWriter bind properties.
- #define `DDSC_LOG_TRUST_REGISTER_REMOTE_DATAWRITER_FAILED_EC` (`DDSC_LOG_BASE + 1467`)
Failed to register remote DataWriter.
- #define `DDSC_LOG_TRUST_VALIDATE_REMOTE_DATAWRITER_TRUST_FAILED_EC` (`DDSC_LOG_BASE + 1468`)
Failed to validate remote DataWriter.
- #define `DDSC_LOG_TRUST_VALIDATE_REMOTE_DATAREADER_TRUST_FAILED_EC` (`DDSC_LOG_BASE + 1469`)
Failed to validate remote DataReader.
- #define `DDSC_LOG_TRUST_VALIDATE_LOCAL_DATAWRITER_TRUST_FAILED_EC` (`DDSC_LOG_BASE + 1470`)
Failed to validate local DataWriter.
- #define `DDSC_TRUST_INVALID_PARTICIPANT_GENERIC_MESSAGE_EC` (`DDSC_LOG_BASE + 1471`)
Invalid DomainParticipant trust generic message.
- #define `DDSC_LOG_TRUST_PREPARE_AUTH_REQUEST_EC` (`DDSC_LOG_BASE + 1472`)
Failed to prepare authentication request.
- #define `DDSC_LOG_TRUST_PREPARE_AUTH_HANDSHAKE_MESSAGE_EC` (`DDSC_LOG_BASE + 1473`)
Failed to prepare authentication handshake message.
- #define `DDSC_LOG_TRUST_PREPARE_ENDPOINT_INTERCEPTOR_MESSAGE_EC` (`DDSC_LOG_BASE + 1474`)
Failed to prepare endpoint interceptor message.
- #define `DDSC_LOG_TRUST_INVALID_GENERIC_MESSAGE_REPLY_EC` (`DDSC_LOG_BASE + 1475`)
Invalid trust generic message reply.
- #define `DDSC_LOG_FINALIZE_TRUST_FAILED_EC` (`DDSC_LOG_BASE + 1476`)
Failed to finalize trust.
- #define `DDSC_LOG_TRUST_DATA HOLDER_COPY_FAILED_EC` (`DDSC_LOG_BASE + 1477`)
Failed to copy trust data holder.
- #define `DDSC_LOG_TRUST_INVALID_SAMPLE_MAX_SERIALIZED_SIZE_EC` (`DDSC_LOG_BASE + 1478`)
Invalid sample serialized size.
- #define `DDSC_LOG_TRUST_ALLOC_FAILED_EC` (`DDSC_LOG_BASE + 1479`)
Failed to allocate memory.
- #define `DDSC_LOG_TRUST_SERIALIZE_DATA_FAILED_EC` (`DDSC_LOG_BASE + 1480`)
Failed to serialize trust data.
- #define `DDSC_LOG_TRUST_INITIALIZE_PARTICIPANT_TRUST_FAILED_EC` (`DDSC_LOG_BASE + 1481`)
Failed to initialize trust.

- `#define DDSC_LOG_TRUST_INITIALIZE_PARTICIPANT_BUILTIN_DATA_FAILED_EC (DDSC_LOG_BASE + 1482)`
Failed to initialize DomainParticipant trust data.
- `#define DDSC_LOG_TRUST_INVALID_PARTICIPANT_GENERIC_MESSAGE_EC (DDSC_LOG_BASE + 1483)`
Invalid DomainParticipant trust generic message.
- `#define DDSC_LOG_TRUST_ADVANCE_SN_FAILED_EC (DDSC_LOG_BASE + 1484)`
Failed to advance trust sequence number.
- `#define DDSC_LOG_TRUST_WRITE_SAMPLE_FAILED_EC (DDSC_LOG_BASE + 1485)`
Failed to write sample.
- `#define DDSC_LOG_TRUST_RESEND_HANDSHAKE_IN_PROGRESS_EC (DDSC_LOG_BASE + 1486)`
Failed to resend handshake because it is already in progress.
- `#define DDSC_LOG_TRUST_RESEND_HANDSHAKE_STOP_FAILED_EC (DDSC_LOG_BASE + 1487)`
Failed to stop periodic handshake.
- `#define DDSC_LOG_TRUST_RELEASE_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC (DDSC_LOG_BASE + 1488)`
Failed to release DomainParticipant trust generic message.
- `#define DDSC_LOG_TRUST_RESEND_HANDSHAKE_MESSAGE_START_FAILED_EC (DDSC_LOG_BASE + 1489)`
Failed to start periodic handshake.
- `#define DDSC_LOG_TRUST_UNEXPECTED_REMOTE_PARTICIPANT_STATUS_EC (DDSC_LOG_BASE + 1490)`
Invalid remote DomainParticipant trust status.
- `#define DDSC_LOG_TRUST_SEND_PARTICIPANT_INTERCEPTOR_STATE_FAILED_EC (DDSC_LOG_BASE + 1491)`
Failed to send DomainParticipant interceptor state.
- `#define DDSC_LOG_TRUST_PARTICIPANT_AUTHENTICATION_FAILED_EC (DDSC_LOG_BASE + 1492)`
DomainParticipant authentication failed.
- `#define DDSC_LOG_TRUST_ASSERT_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC (DDSC_LOG_BASE + 1493)`
Failed to assert DomainParticipant generic message.
- `#define DDSC_LOG_TRUST_SEND_HANDSHAKE_MESSAGE_FAILED_EC (DDSC_LOG_BASE + 1494)`
Failed to send handshake message.
- `#define DDSC_LOG_TRUST_INVALID_INTERCEPTOR_TOKENS_EC (DDSC_LOG_BASE + 1495)`
Invalid interceptor tokens.
- `#define DDSC_LOG_TRUST_INVALID_INTERCEPTOR_TOKENS_DESTINATION_EC (DDSC_LOG_BASE + 1496)`
Invalid interceptor token destination.
- `#define DDSC_LOG_TRUST_ENDPOINT_INTERNAL_ATTRIBUTES_CREATE_FAILED_EC (DDSC_LOG_BASE + 1497)`
Failed to copy endpoint trust attributes.
- `#define DDSC_LOG_TRUST_SEND_ENDPOINT_INTERCEPTOR_STATE_FAILED_EC (DDSC_LOG_BASE + 1498)`
Failed to send tokens to remote DomainParticipant.
- `#define DDSC_LOG_TRUST_CREATE_LOCAL_DATAWRITER_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1499)`
Failed to create tokens for the remote DataReader.
- `#define DDSC_LOG_TRUST_CREATE_LOCAL_DATAREADER_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1500)`

- Failed to create tokens for the remote DataWriter.*

 - #define `DDSC_LOG_TRUST_RETURN_INTERCEPTOR_TOKENS_FAILED_EC` (`DDSC_LOG_BASE` + 1501)
- Failed to return interceptor tokens.*

 - #define `DDSC_LOG_TRUST_RETURN_AUTHENTICATED_PEER_CREDENTIAL_TOKEN_FAILED_EC` (`DDSC_LOG_BASE` + 1502)
- Failed to return authenticated peer credential token.*

 - #define `DDSC_LOG_TRUST_PROCESS_REMOTE_INTERCEPTOR_TOKENS_FAILED_EC` (`DDSC_LOG_BASE` + 1503)
- Failed to process interceptor tokens.*

 - #define `DDSC_LOG_TRUST_UNREGISTER_REMOTE_DATAWRITER_FAILED_EC` (`DDSC_LOG_BASE` + 1504)
- Failed to unregister remote DataWriter.*

 - #define `DDSC_LOG_TRUST_UNREGISTER_REMOTE_DATAREADER_FAILED_EC` (`DDSC_LOG_BASE` + 1505)
- Failed to unregister remote DataReader.*

 - #define `DDSC_LOG_TRUST_INVALID_SHARED_SECRET_EC` (`DDSC_LOG_BASE` + 1506)
- Invalid shared secret.*

 - #define `DDSC_LOG_TRUST_PARTICIPANT_UNEXPECTED_AUTHENTICATION_ERROR_EC` (`DDSC_LOG_BASE` + 1507)
- Unexpected DomainParticipant authentication error.*

 - #define `DDSC_LOG_TRUST_PREPARE_LOCAL_DP_TOKENS_EC` (`DDSC_LOG_BASE` + 1508)
- Failed to prepare local DomainParticipant interceptor tokens.*

 - #define `DDSC_LOG_TRUST_ASSERT_PROPERTY_FAILED_EC` (`DDSC_LOG_BASE` + 1509)
- Failed to assert trust property.*

 - #define `DDSC_LOG_TRUST_PARTICIPANT_INTERNAL_ATTRIBUTES_CREATE_FAILED_EC` (`DDSC_LOG_BASE` + 1510)
- Failed to copy DomainParticipant trust attributes.*

 - #define `DDSC_LOG_TRUST_INVALID_PARTICIPANT_INTERNAL_ATTRIBUTES_EC` (`DDSC_LOG_BASE` + 1511)
- Invalid DomainParticipant trust attributes.*

 - #define `DDSC_LOG_TRUST_TRANSFORM_OUTGOING_SERIALIZED_PAYLOAD_FAILED_EC` (`DDSC_LOG_BASE` + 1512)
- Failed to encode outgoing serialized payload.*

 - #define `DDSC_LOG_TRUST_TRANSFORM_INCOMING_SERIALIZED_PAYLOAD_FAILED_EC` (`DDSC_LOG_BASE` + 1513)
- Failed to decode incoming serialized payload.*

 - #define `DDSC_LOG_TRUST_FINALIZE_AC_PLUGIN_PROPERTIES_FAILED_EC` (`DDSC_LOG_BASE` + 1514)
- Failed to finalize trust properties.*

 - #define `DDSC_LOG_TRUST_AFTER_REMOTE_PARTICIPANT_AUTHENTICATED_FAILED_EC` (`DDSC_LOG_BASE` + 1515)
- Error after remote trust DomainParticipant is authenticated.*

 - #define `DDSC_LOG_TRUST_CACHE_REMOTE_PARTICIPANT_SAMPLE_FAILED_EC` (`DDSC_LOG_BASE` + 1516)
- Failed to copy remote DomainParticipant sample.*

 - #define `DDSC_LOG_TRUST_FINALIZE_AUTH_HANDSHAKE_STATE_FAILED_EC` (`DDSC_LOG_BASE` + 1517)
- Failed to finalize authentication handshake.*

- #define `DDSC_LOG_TRUST_FINALIZE_REMOTE_PARTICIPANT_TRUST_FAILED_EC` (`DDSC_LOG_BASE` + 1518)
Failed to finalize remote trust DomainParticipant record.
- #define `DDSC_LOG_TRUST_RESET_REMOTE_PARTICIPANT_TRUST_FAILED_EC` (`DDSC_LOG_BASE` + 1519)
Failed to finalize remote trust DomainParticipant.
- #define `DDSC_LOG_TRUST_ASSERT_AUTH_HANDSHAKE_STATE_FAILED_EC` (`DDSC_LOG_BASE` + 1520)
Failed to assert authentication handshake state.
- #define `DDSC_LOG_TRUST_INVALID_AUTH_HANDSHAKE_STATE_EC` (`DDSC_LOG_BASE` + 1521)
Invalid authentication handshake state.
- #define `DDSC_LOG_TRUST_SEND_AUTH_REQUEST_FAILED_EC` (`DDSC_LOG_BASE` + 1522)
Failed to send authentication request.
- #define `DDSC_LOG_TRUST_GET_AUTH_HANDSHAKE_STATE_FAILED_EC` (`DDSC_LOG_BASE` + 1523)
Failed to get authentication handshake state.
- #define `DDSC_LOG_TRUST_SET_AUTH_HANDSHAKE_STATE_FAILED_EC` (`DDSC_LOG_BASE` + 1524)
Failed to set authentication handshake state.
- #define `DDSC_LOG_TRUST_REAUTH_REMOTE_PARTICIPANT_FAILED_EC` (`DDSC_LOG_BASE` + 1525)
Failed to reauthenticate a remote DomainParticipant.
- #define `DDSC_LOG_TRUST_RESEND_HANDSHAKE_MESSAGE_FAILED_EC` (`DDSC_LOG_BASE` + 1526)
Failed to resend handshake message.
- #define `DDSC_LOG_TRUST_PBUFS_MAX_EXCEEDED_EC` (`DDSC_LOG_BASE` + 1527)
Maximum number of buffers exceeded.
- #define `DDSC_LOG_TRUST_CREATE_TRUST_PLUGIN_EC` (`DDSC_LOG_BASE` + 1528)
Failed to create trust plugin.
- #define `DDSC_LOG_TRUST_LOOKUP_FACTORY_FAILED_EC` (`DDSC_LOG_BASE` + 1529)
Failed to lookup trust plugin factory.
- #define `DDSC_LOG_NAME_LOOKUP_EC` (`DDSC_LOG_BASE` + 1700)
Fully qualified name does not contain substring "::.".
- #define `DDSC_LOG_ENTITY_NAME_TOO_LONG_EC` (`DDSC_LOG_BASE` + 1701)
Failed to get an entity because the specified name exceeds the maximum length of 255 octets (excluding the terminating NUL)
- #define `DDSC_LOG_MESSAGE_DATA_UNKNOWN_LIVELINESS_KIND_EC` (`DDSC_LOG_BASE` + 1800)
Unknown inter-participant message liveliness kind received.
- #define `DDSC_LOG_MESSAGE_DATA_WRONG_LIVELINESS_KIND_EC` (`DDSC_LOG_BASE` + 1801)
Wrong inter-participant message liveliness kind received.
- #define `DDSC_LOG_UPDATE_LEASE_DURATION_TIMER_EC` (`DDSC_LOG_BASE` + 1802)
Failed to update DataWriter lease duration.
- #define `DDSC_LOG_PARTMESSAGE_WRITE_SAMPLE_FAILED_EC` (`DDSC_LOG_BASE` + 1803)
Failed to write inter-participant liveliness message.
- #define `DDSC_LOG_SEND_LIVELINESS_FAILED_EC` (`DDSC_LOG_BASE` + 1804)
Failed to send inter-participant liveliness message.
- #define `DDSC_LOG_UPDATE_LIVELINESS_FAILED_EC` (`DDSC_LOG_BASE` + 1805)
Failed to update DataWriter liveliness.
- #define `DDSC_LOG_REFRESH_LIVELINESS_FAILED_EC` (`DDSC_LOG_BASE` + 1806)
Failed to refresh endpoint liveliness.
- #define `DDSC_LOG_TRUST_INCOMPATIBLE_ENDPOINT_TRUST_ATTRIBUTES_EC` (`DDSC_LOG_BASE` + 1807)

- Two security endpoints are incompatible.*

 - #define `DDSC_LOG_FLOW_CONTROLLER_CREATE_EC` (`DDSC_LOG_BASE` + 1900)

Failed to create flow controller.
- #define `DDSC_LOG_FLOW_CONTROLLER_DELETE_EC` (`DDSC_LOG_BASE` + 1901)

Failed to delete flow controller.
- #define `DDSC_LOG_XTYPES_LOANED_SAMPLE_STATE_ERROR_EC` (`DDSC_LOG_BASE` + 2000)

Extensible types sample state error.
- #define `DDSC_LOG_INTERPRETER_SUPPORT_UNEXPECTED_ERROR_EC` (`DDSC_LOG_BASE` + 2001)

Interpreter unexpected error.
- #define `DDSC_LOG_MEMORY_MANAGER_OUT_OF_RESOURCES_EC` (`DDSC_LOG_BASE` + 2002)

Memory manager out of resources.
- #define `DDSC_LOG_MEMORY_MANAGER_NOT_OWNER_EC` (`DDSC_LOG_BASE` + 2003)

Memory manager is not the owner of the data passed in.
- #define `DDSC_LOG_MEMORY_MANAGER_DELETE_EC` (`DDSC_LOG_BASE` + 2004)

The wire manager was not able to delete a resource.
- #define `DDSC_LOG_BUFFER_MAX_EXCEEDED_EC` (`DDSC_LOG_BASE` + 2101)

Maximum size of buffer exceeded.
- #define `DDSC_LOG_INVALID_BUFFER_LENGTH_EC` (`DDSC_LOG_BASE` + 2102)

Invalid buffer length.
- #define `DDSC_LOG_INCOMPATIBLE_PRESENTATION_QOS_EC` (`DDSC_LOG_BASE` + 2201)

Matching error because Presentation QoS is not compatible.
- #define `DDSC_LOG_INCOMPATIBLE_PARTITION_QOS_EC` (`DDSC_LOG_BASE` + 2202)

Two endpoints did not match because of incompatible partition QoS.
- #define `DDSC_LOG_CHECKSUM_REQUIRED_EC` (`DDSC_LOG_BASE` + 2301)

A participant requires checksum, but the discovered participant does not send a checksum.
- #define `DDSC_LOG_CHECKSUM_INCOMPATIBLE_EC` (`DDSC_LOG_BASE` + 2302)

A participant does not support checksum's sent by a discovered participant.
- #define `DDSC_LOG_CHECKSUM_INCONSISTENT_CLASS_EC` (`DDSC_LOG_BASE` + 2303)

The checksum class id for a custom checksum does match.
- #define `DDSC_LOG_CHECKSUM_INCONSISTENT_COMPUTE_EC` (`DDSC_LOG_BASE` + 2304)

The computed_crc_kind is inconsistent with what is allowed or supported.
- #define `DDSC_LOG_CHECKSUM_INCONSISTENT_ALLOW_EC` (`DDSC_LOG_BASE` + 2305)

The allowed_crc_mask is inconsistent with what is supported.
- #define `DDSC_LOG_FLOWCONTROLLER_INCONSISTENT_PROP_EC` (`DDSC_LOG_BASE` + 2306)

The token bucket properties for the flow controller are inconsistent.
- #define `DDSC_LOG_LOCK_EC` (`DDSC_LOG_BASE` + 2401)

Failed to take a lock.
- #define `DDSC_LOG_UNLOCK_EC` (`DDSC_LOG_BASE` + 2402)

Failed to give a lock.
- #define `DDSC_LOG_FILTER_PLUGIN_FACTORY_NOT_FOUND_EC` (`DDSC_LOG_BASE` + 2501)

Failed to find the filter plugin factory specified in a DomainParticipant's QoS.
- #define `DDSC_LOG_ENABLE_WRITER_FILTERING_EC` (`DDSC_LOG_BASE` + 2502)

A local writer failed to enable writer filtering for a reader.
- #define `DDSC_LOG_CREATE_WRITER_FILTER_EC` (`DDSC_LOG_BASE` + 2503)

A local writer failed to create its writer filter.
- #define `DDSC_LOG_EVALUATE_WRITER_FILTER_EC` (`DDSC_LOG_BASE` + 2504)

A local writer failed to evaluate a sample against the filters of its matched readers.

- `#define DDSC_LOG_APPLY_READER_FILTER_EC (DDSC_LOG_BASE + 2505)`
A local writer failed to check if a reader had filtered a sample.
- `#define DDSC_LOG_COMPILE_CONTENT_FILTER_EC (DDSC_LOG_BASE + 2506)`
Failed to compile the content filter configured in a DataReader's QoS.
- `#define DDSC_LOG_CONTENT_FILTERING_NOT_ENABLED_EC (DDSC_LOG_BASE + 2507)`
Reader configured a content filter but content filtering is not enabled.
- `#define DDSC_LOG_EVALUATE_CONTENT_FILTER_EC (DDSC_LOG_BASE + 2508)`
Failed to evaluate a sample against a reader's content filter.
- `#define DDSC_LOG_DESERIALIZE_CONTENT_FILTER_INFO_EC (DDSC_LOG_BASE + 2509)`
Failed to deserialize the content filter info from the in-line QoS of a sample.
- `#define DDSC_LOG_CHANGE_CONTENT_FILTER_EC (DDSC_LOG_BASE + 2510)`
Failed to change the configured content filter of a DataReader.
- `#define DDSC_LOG_UPDATE_REMOTE_CONTENT_FILTER_EC (DDSC_LOG_BASE + 2511)`
Failed to update the content filter of a remote subscription.
- `#define DDSC_LOG_CREATE_FILTER_PLUGIN_EC (DDSC_LOG_BASE + 2512)`
DomainParticipant failed to create its filter plugin.
- `#define DDSC_LOG_FINALIZE_FILTER_PLUGIN_EC (DDSC_LOG_BASE + 2513)`
DomainParticipant failed to finalize its filter plugin.
- `#define DDSC_LOG_DESERIALIZE_REMOTE_CONTENT_FILTER_EC (DDSC_LOG_BASE + 2514)`
Failed to deserialize the content filter property in the subscription built-in topic data for a remote subscription.

6.20.9.1 Detailed Description

DDS C. ModuleID = 7.

6.20.9.2 Macro Definition Documentation

DDSC_LOG_INVALID_DURATION_EC

```
#define DDSC_LOG_INVALID_DURATION_EC (DDSC_LOG_BASE + 1)
```

The specified duration is not valid.

DDSC_LOG_INVALID_PARTICIPANT_NAME_EC

```
#define DDSC_LOG_INVALID_PARTICIPANT_NAME_EC (DDSC_LOG_BASE + 2)
```

An invalid participant name was specified with an unknown GUID.

DDSC_LOG_INVALID_PARTICIPANT_GUID_PREFIX_EC

```
#define DDSC_LOG_INVALID_PARTICIPANT_GUID_PREFIX_EC (DDSC_LOG_BASE + 3)
```

An invalid participant GUID prefix was specified, typically the GUID prefix does not match an already detected participant.

DDSC_LOG_SYS_GETTIME_EC

```
#define DDSC_LOG_SYS_GETTIME_EC (DDSC_LOG_BASE + 4)
```

A call to OSAPI_System_get_time failed.

DDSC_LOG_GET_NEXT_OBJECT_ID_EC

```
#define DDSC_LOG_GET_NEXT_OBJECT_ID_EC (DDSC_LOG_BASE + 5)
```

Failed to get the next automatically generated object ID for an entity's GUID. This typically means the object id pool has been exhausted.

DDSC_LOG_SET_ENTITY_NAME_EC

```
#define DDSC_LOG_SET_ENTITY_NAME_EC (DDSC_LOG_BASE + 6)
```

Failed to set name string for a DDS entity in the DomainParticipantQos entity_name policy.

DDSC_LOG_SYS_GET_HOSTNAME_EC

```
#define DDSC_LOG_SYS_GET_HOSTNAME_EC (DDSC_LOG_BASE + 7)
```

A call to OSAPI_System_get_hostname failed.

DDSC_LOG_IO_SNPRINTF_FAILED_EC

```
#define DDSC_LOG_IO_SNPRINTF_FAILED_EC (DDSC_LOG_BASE + 8)
```

A call to OSAPI_stdio_snprintf failed. Typically this means the destination buffer was too small.

DDSC_LOG_TOPIC_FIND_EC

```
#define DDSC_LOG_TOPIC_FIND_EC (DDSC_LOG_BASE + 9)
```

Failed to find a topic created by a DomainParticipant.

DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_NAME_EC

```
#define DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_NAME_EC (DDSC_LOG_BASE + 10)
```

Endpoint discovery failed because the name of the remote participant parent for an endpoint was not found.

DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_KEY_EC

```
#define DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_KEY_EC (DDSC_LOG_BASE + 11)
```

Endpoint discovery failed because the key of the remote participant parent for an endpoint was not found. Note that the key is logged as 4 integers in host endianness format.

DDSC_LOG_REMOTE_PARTICIPANT_KEY_NOT_EQUAL_EC

```
#define DDSC_LOG_REMOTE_PARTICIPANT_KEY_NOT_EQUAL_EC (DDSC_LOG_BASE + 12)
```

Failed endpoint discovery when key does not match the remote participant.

DDSC_LOG_INVALID_ENDPOINT_GUID_EC

```
#define DDSC_LOG_INVALID_ENDPOINT_GUID_EC (DDSC_LOG_BASE + 13)
```

Failed endpoint discovery due to an invalid or unknown endpoint GUID.

DDSC_LOG_ENDPOINT_NOT_CHILD_OF_PARTICIPANT_EC

```
#define DDSC_LOG_ENDPOINT_NOT_CHILD_OF_PARTICIPANT_EC (DDSC_LOG_BASE + 14)
```

Failed endpoint discovery when an endpoint is determined to belong to a different remote participant.

DDSC_LOG_PARTICIPANT_DOES_NOT_EXIST_EC

```
#define DDSC_LOG_PARTICIPANT_DOES_NOT_EXIST_EC (DDSC_LOG_BASE + 15)
```

Failed participant discovery because a remote participant that should have been already asserted locally was not found.

DDSC_LOG_REFRESH_REM_PARTICIPANT_EC

```
#define DDSC_LOG_REFRESH_REM_PARTICIPANT_EC (DDSC_LOG_BASE + 16)
```

Did not find a remote participant when asserting participant liveliness for it. Note that the key is logged as 4 integers in host endianness format.

DDSC_LOG_REFRESH_REM_PARTICIPANT_TIMEOUT_EC

```
#define DDSC_LOG_REFRESH_REM_PARTICIPANT_TIMEOUT_EC (DDSC_LOG_BASE + 17)
```

Failed to assert participant liveliness to a remote participant.

DDSC_LOG_PARTICIPANT_LOOKUP_EC

```
#define DDSC_LOG_PARTICIPANT_LOOKUP_EC (DDSC_LOG_BASE + 18)
```

Failed to find a remote participant as previously discovered.

DDSC_LOG_REMOVE_PUBLICATION_EC

```
#define DDSC_LOG_REMOVE_PUBLICATION_EC (DDSC_LOG_BASE + 19)
```

Failed to remove resources for a remote publication.

DDSC_LOG_REMOVE_SUBSCRIPTION_EC

```
#define DDSC_LOG_REMOVE_SUBSCRIPTION_EC (DDSC_LOG_BASE + 20)
```

Failed to remove resources for a remote subscription.

DDSC_LOG_FIND_PUBLICATION_PARENT_EC

```
#define DDSC_LOG_FIND_PUBLICATION_PARENT_EC (DDSC_LOG_BASE + 21)
```

Cannot determine the participant of a remote publication.

DDSC_LOG_FIND_SUBSCRIPTION_PARENT_EC

```
#define DDSC_LOG_FIND_SUBSCRIPTION_PARENT_EC (DDSC_LOG_BASE + 22)
```

Cannot determine the participant of a remote subscription.

DDSC_LOG_MAX_PARTICIPANT_ID_REACHED_EC

```
#define DDSC_LOG_MAX_PARTICIPANT_ID_REACHED_EC (DDSC_LOG_BASE + 23)
```

Failed to create DomainParticipant due to running out of participant IDs. This is typically caused by all UDP ports being used.

DDSC_LOG_RESERVE_LOCATORS_EC

```
#define DDSC_LOG_RESERVE_LOCATORS_EC (DDSC_LOG_BASE + 24)
```

Failed to reserve endpoint locators.

DDSC_LOG_TIMER_CREATE_TIMEOUT_EC

```
#define DDSC_LOG_TIMER_CREATE_TIMEOUT_EC (DDSC_LOG_BASE + 25)
```

Failed to create a timeout.

DDSC_LOG_ILLEGAL_OBJECTID_EC

```
#define DDSC_LOG_ILLEGAL_OBJECTID_EC (DDSC_LOG_BASE + 26)
```

Illegal object id specified.

Manually specified object id must be in the range [0,0xfffff]

DDSC_LOG_TOPIC_NARROW_EC

```
#define DDSC_LOG_TOPIC_NARROW_EC (DDSC_LOG_BASE + 27)
```

Failed to narrow a TopicDescription to the named Topic.

DDSC_LOG_FAILED_UPDATE_STATUS_CONDITION_EC

```
#define DDSC_LOG_FAILED_UPDATE_STATUS_CONDITION_EC (DDSC_LOG_BASE + 28)
```

Failed to update state of StatusCondition.

An error occurred when trying to update the state (i.e. the trigger value) of a StatusCondition.

DDSC_LOG_WS_REMOVE_COND_REFERENCE_EC

```
#define DDSC_LOG_WS_REMOVE_COND_REFERENCE_EC (DDSC_LOG_BASE + 29)
```

Failed to remove a condition reference from a waitset.

DDSC_LOG_WS_ADD_COND_REFERENCE_EC

```
#define DDSC_LOG_WS_ADD_COND_REFERENCE_EC (DDSC_LOG_BASE + 30)
```

Failed to add a condition reference to a waitset.

DDSC_LOG_RELEASE_META_MC_EC

```
#define DDSC_LOG_RELEASE_META_MC_EC (DDSC_LOG_BASE + 31)
```

Failed to release resources for multicast discovery locators.

DDSC_LOG_RELEASE_META_UC_EC

```
#define DDSC_LOG_RELEASE_META_UC_EC (DDSC_LOG_BASE + 32)
```

Failed to release resources for unicast discovery locators.

DDSC_LOG_RELEASE_USER_MC_EC

```
#define DDSC_LOG_RELEASE_USER_MC_EC (DDSC_LOG_BASE + 33)
```

Failed to release resources for multicast user locators.

DDSC_LOG_RELEASE_USER_UC_EC

```
#define DDSC_LOG_RELEASE_USER_UC_EC (DDSC_LOG_BASE + 34)
```

Failed to release resources for unicast user locators.

DDSC_LOG_INVALID_DOMAINID_EC

```
#define DDSC_LOG_INVALID_DOMAINID_EC (DDSC_LOG_BASE + 35)
```

The domain ID specified exceeds what is allowed based on the parameters specified in `dp_qos.protocol.rtps_well_known_ports`.

DDSC_LOG_ON_TYPE_REGISTERED_FAILURE_EC

```
#define DDSC_LOG_ON_TYPE_REGISTERED_FAILURE_EC (DDSC_LOG_BASE + 36)
```

Error occurred during the `on_type_registered` call back.

DDSC_LOG_ON_TYPE_UNREGISTERED_FAILURE_EC

```
#define DDSC_LOG_ON_TYPE_UNREGISTERED_FAILURE_EC (DDSC_LOG_BASE + 37)
```

Error occurred during the `on_type_unregistered` call back.

DDSC_LOG_GET_SERIALIZED_KEY_SIZE_EC

```
#define DDSC_LOG_GET_SERIALIZED_KEY_SIZE_EC (DDSC_LOG_BASE + 38)
```

Error occurred during the `on_type_unregistered` call back.

DDSC_LOG_MIN_DISCOVERY_MTU_EC

```
#define DDSC_LOG_MIN_DISCOVERY_MTU_EC (DDSC_LOG_BASE + 39)
```

Minimum MTU across all Discovery transports must be greater or equal to a packet containing serialized Participant↔BuiltinTopicData.

DDSC_LOG_MIN_MTU_SIZE_EC

```
#define DDSC_LOG_MIN_MTU_SIZE_EC (DDSC_LOG_BASE + 40)
```

MTU for a transport set lower or equal to Protocol Overhead of 448 bytes.

DDSC_LOG_GET_MTU_NO_ROUTES_EC

```
#define DDSC_LOG_GET_MTU_NO_ROUTES_EC (DDSC_LOG_BASE + 41)
```

MTU could not be found because of no routes.

DDSC_LOG_RESOLVE_LOCATORS_PRIORITY_EC

```
#define DDSC_LOG_RESOLVE_LOCATORS_PRIORITY_EC (DDSC_LOG_BASE + 42)
```

Failed to resolve locators based on priority.

DDSC_LOG_REMOTE_PUBLICATION_DATA_CHANGED_EC

```
#define DDSC_LOG_REMOTE_PUBLICATION_DATA_CHANGED_EC (DDSC_LOG_BASE + 43)
```

The publication builtin topic data for a remote publication changed after the remote publication was asserted.

No dynamic changes are supported to the publication builtin topic data after the remote publication is asserted. The previous data will continue to be used.

DDSC_LOG_REMOTE_SUBSCRIPTION_DATA_CHANGED_EC

```
#define DDSC_LOG_REMOTE_SUBSCRIPTION_DATA_CHANGED_EC (DDSC_LOG_BASE + 44)
```

Fields in subscription builtin topic data for a remote subscription which are not supported to dynamically change were updated after the remote subscription was asserted.

The only supported dynamic changes to the subscription builtin topic data are the configured content filter on the remote subscription. Any other changes are not supported and will be ignored.

DDSC_LOG_GET_MTU_EC

```
#define DDSC_LOG_GET_MTU_EC (DDSC_LOG_BASE + 45)
```

Error getting the mtu from the route resolver.

DDSC_LOG_DATABASE_CREATE_EC

```
#define DDSC_LOG_DATABASE_CREATE_EC (DDSC_LOG_BASE + 100)
```

Failed to create database.

DDSC_LOG_DATABASE_DELETE_EC

```
#define DDSC_LOG_DATABASE_DELETE_EC (DDSC_LOG_BASE + 101)
```

Failed to delete database.

DDSC_LOG_TABLE_CREATE_EC

```
#define DDSC_LOG_TABLE_CREATE_EC (DDSC_LOG_BASE + 102)
```

Failed to create database table of the specified name.

DDSC_LOG_TABLE_INUSE_EC

```
#define DDSC_LOG_TABLE_INUSE_EC (DDSC_LOG_BASE + 103)
```

Failed to delete a database table because it is not empty.

DDSC_LOG_TABLE_DELETE_EC

```
#define DDSC_LOG_TABLE_DELETE_EC (DDSC_LOG_BASE + 104)
```

Failed to delete a database table.

DDSC_LOG_TABLE_SELECT_EC

```
#define DDSC_LOG_TABLE_SELECT_EC (DDSC_LOG_BASE + 105)
```

A selection operation failed on the specified database table.

DDSC_LOG_CREATE_INDEX_EC

```
#define DDSC_LOG_CREATE_INDEX_EC (DDSC_LOG_BASE + 106)
```

Failed to create an index on a database table.

DDSC_LOG_DELETE_INDEX_EC

```
#define DDSC_LOG_DELETE_INDEX_EC (DDSC_LOG_BASE + 107)
```

Failed to delete an index on a database table.

DDSC_LOG_DB_CURSOR_INVALIDATED_EC

```
#define DDSC_LOG_DB_CURSOR_INVALIDATED_EC (DDSC_LOG_BASE + 108)
```

A database table cursor was invalidated while in use.

DDSC_LOG_SUBSCRIPTION_RECORD

```
#define DDSC_LOG_SUBSCRIPTION_RECORD 1
```

Database record for a Subscription.

DDSC_LOG_PUBLICATION_RECORD

```
#define DDSC_LOG_PUBLICATION_RECORD 2
```

Database record for a Publication.

DDSC_LOG_PARTICIPANT_RECORD

```
#define DDSC_LOG_PARTICIPANT_RECORD 3
```

Database record for a Remote Participant.

DDSC_LOG_TOPIC_RECORD

```
#define DDSC_LOG_TOPIC_RECORD 4
```

Database record for a Topic.

DDSC_LOG_DATAWRITER_RECORD

```
#define DDSC_LOG_DATAWRITER_RECORD 5
```

Database record for a DataWriter.

DDSC_LOG_DATAREADER_RECORD

```
#define DDSC_LOG_DATAREADER_RECORD 6
```

Database record for a DataReader.

DDSC_LOG_PUBLISHER_RECORD

```
#define DDSC_LOG_PUBLISHER_RECORD 7
```

Database record for a Publisher.

DDSC_LOG_SUBSCRIBER_RECORD

```
#define DDSC_LOG_SUBSCRIBER_RECORD 8
```

Database record for a Subscriber.

DDSC_LOG_ROUTE_RECORD

```
#define DDSC_LOG_ROUTE_RECORD 9
```

Database record for a route record.

DDSC_LOG_BIND_RECORD

```
#define DDSC_LOG_BIND_RECORD 10
```

Database record for a bind record.

DDSC_LOG_TYPE_RECORD

```
#define DDSC_LOG_TYPE_RECORD 11
```

Database record for a type plugin.

DDSC_LOG_REMOTE_ENDPOINT_TRUST_STATE_RECORD

```
#define DDSC_LOG_REMOTE_ENDPOINT_TRUST_STATE_RECORD 12
```

Database record for a remote trust endpoint.

DDSC_LOG_MATCHED_DW_RECORD

```
#define DDSC_LOG_MATCHED_DW_RECORD 13
```

Database record for a matched remote data writer.

DDSC_LOG_STRING_RECORD

```
#define DDSC_LOG_STRING_RECORD 14
```

Database record for a string property.

DDSC_LOG_RECORD_CREATE_EC

```
#define DDSC_LOG_RECORD_CREATE_EC (DDSC_LOG_BASE + 109)
```

Failed to create a database record of the specified kind.

The creation of an internal database record failed. The failure may have been caused by insufficient resources based on the record kind. The following resource-limits may apply:

DomainParticipantQos.resource_limit.local_publisher_allocation limits the number of DDS Publishers.

DomainParticipantQos.resource_limit.local_subscriber_allocation limits the number of DDS Subscribers.

DomainParticipantQos.resource_limit.local_topic_allocation limits the number of DDS Topics.

DomainParticipantQos.resource_limits.local_reader_allocation limits the number of DDS DataReader records.

DomainParticipantQos.resource_limits.local_writer_allocation limits the number of DDS DataWriter records.

DomainParticipantQos.resource_limits.remote_writer_allocation limits the number of remote publication records.

DomainParticipantQos.resource_limits.remote_reader_allocation limits the number of remote subscription records.

DomainParticipantQos.resource_limits.remote_participant_allocation limits the number of remote participant records.

DataWriterQos.writer_resource_limits.max_remote_readers limits the number of DDS DataReaders a DDS Data↔Writer can communicate with.

DataReaderQos.reader_resource_limits.max_remote_writers limits the number of DDS DataWriter a DDS Data↔Reader can communicate with.

DDSC_LOG_RECORD_DELETE_EC

```
#define DDSC_LOG_RECORD_DELETE_EC (DDSC_LOG_BASE + 110)
```

Failed to delete a database record of the specified kind.

DDSC_LOG_RECORD_INSERT_EC

```
#define DDSC_LOG_RECORD_INSERT_EC (DDSC_LOG_BASE + 111)
```

Failed to insert a database record of the specified kind.

DDSC_LOG_RECORD_ERROR_EC

```
#define DDSC_LOG_RECORD_ERROR_EC (DDSC_LOG_BASE + 112)
```

Unknown error for database record of the specified kind.

DDSC_LOG_RECORD_EXISTS_EC

```
#define DDSC_LOG_RECORD_EXISTS_EC (DDSC_LOG_BASE + 113)
```

A database record of the specified kind already exists.

DDSC_LOG_RECORD_LOOKUP_EC

```
#define DDSC_LOG_RECORD_LOOKUP_EC (DDSC_LOG_BASE + 114)
```

A lookup of a database record of the specified kind failed.

DDSC_LOG_RECORD_NOT_EXISTS_EC

```
#define DDSC_LOG_RECORD_NOT_EXISTS_EC (DDSC_LOG_BASE + 115)
```

A database record of the specified kind does not exist.

DDSC_LOG_RECORD_SELECT_EC

```
#define DDSC_LOG_RECORD_SELECT_EC (DDSC_LOG_BASE + 116)
```

A database select on the specified record kind failed.

DDSC_LOG_RECORD_REMOVE_EC

```
#define DDSC_LOG_RECORD_REMOVE_EC (DDSC_LOG_BASE + 117)
```

Removal a database record of the specified kind failed.

DDSC_LOG_RECORD_INITIALIZE_EC

```
#define DDSC_LOG_RECORD_INITIALIZE_EC (DDSC_LOG_BASE + 118)
```

A database record of the specified kind could not be initialized.

DDSC_LOG_RECORD_FINALIZE_EC

```
#define DDSC_LOG_RECORD_FINALIZE_EC (DDSC_LOG_BASE + 119)
```

A database record of the specified kind could not be finalized.

DDSC_LOG_RECORD_RESET_EC

```
#define DDSC_LOG_RECORD_RESET_EC (DDSC_LOG_BASE + 120)
```

A database record of the specified kind could not be reset.

A database record is "reset" when all its attributes are reverted to a previous state (typically the one it had at creation/insertion), without removing it from the database.

The actions performed by the "reset" depend on the type of record.

DDSC_LOG_PUBLICATIONDATA_OBJECT

```
#define DDSC_LOG_PUBLICATIONDATA_OBJECT 1
```

DDS PublicationBuiltinTopicData object.

DDSC_LOG_SUBSCRIPTIONDATA_OBJECT

```
#define DDSC_LOG_SUBSCRIPTIONDATA_OBJECT 2
```

DDS SubscriptionBuiltinTopicData object.

DDSC_LOG_PARTICIPANTDATA_OBJECT

```
#define DDSC_LOG_PARTICIPANTDATA_OBJECT 3
```

DDS ParticipantBuiltinTopicData object.

DDSC_LOG_DATAREADERQOS_OBJECT

```
#define DDSC_LOG_DATAREADERQOS_OBJECT 4
```

DDS DataReaderQos object.

DDSC_LOG_DATAWRITERQOS_OBJECT

```
#define DDSC_LOG_DATAWRITERQOS_OBJECT 5
```

DDS DataWriterQos object.

DDSC_LOG_TOPICQOS_OBJECT

```
#define DDSC_LOG_TOPICQOS_OBJECT 6
```

DDS TopicQos object.

DDSC_LOG_PUBLISHERQOS_OBJECT

```
#define DDSC_LOG_PUBLISHERQOS_OBJECT 7
```

DDS PublisherQos object.

DDSC_LOG_SUBSCRIBERQOS_OBJECT

```
#define DDSC_LOG_SUBSCRIBERQOS_OBJECT 8
```

DDS SubscriberQos object.

DDSC_LOG_PARTICIPANTQOS_OBJECT

```
#define DDSC_LOG_PARTICIPANTQOS_OBJECT 9
```

DDS DomainParticipantQos object.

DDSC_LOG_ENTITY_OBJECT

```
#define DDSC_LOG_ENTITY_OBJECT 10
```

DDS Entity object.

DDSC_LOG_PARTICIPANTFACTORYQOS_OBJECT

```
#define DDSC_LOG_PARTICIPANTFACTORYQOS_OBJECT 11
```

DDS DomainParticipantFactoryQos object.

DDSC_LOG_TYPE_OBJECT

```
#define DDSC_LOG_TYPE_OBJECT 12
```

A DDS Type object.

DDSC_LOG_BINDRESOLVER_OBJECT

```
#define DDSC_LOG_BINDRESOLVER_OBJECT 13
```

A NETIO BindResolver object.

DDSC_LOG_ROUTERESOLVER_OBJECT

```
#define DDSC_LOG_ROUTERESOLVER_OBJECT 14
```

A NETIO RouteResolver object.

DDSC_LOG_ADDRESSRESOLVER_OBJECT

```
#define DDSC_LOG_ADDRESSRESOLVER_OBJECT 15
```

A NETIO AddressResolver object.

DDSC_LOG_DATAWRITERIO_OBJECT

```
#define DDSC_LOG_DATAWRITERIO_OBJECT 16
```

A NETIO DataWriterInterface object.

DDSC_LOG_DATAREADERIO_OBJECT

```
#define DDSC_LOG_DATAREADERIO_OBJECT 17
```

A NETIO DataReaderInterface object.

DDSC_LOG_LOG_OBJECT

```
#define DDSC_LOG_LOG_OBJECT 18
```

A OSAPI Log object.

DDSC_LOG_SYSTEM_OBJECT

```
#define DDSC_LOG_SYSTEM_OBJECT 19
```

A OSAPI system object.

DDSC_LOG_MUTEX_OBJECT

```
#define DDSC_LOG_MUTEX_OBJECT 20
```

A OSAPI mutex object.

DDSC_LOG_CONDREF_OBJECT

```
#define DDSC_LOG_CONDREF_OBJECT 21
```

A DDS Waitset condition reference object.

DDSC_LOG_WSREF_OBJECT

```
#define DDSC_LOG_WSREF_OBJECT 22
```

A DDS Condition waitset reference object.

DDSC_LOG_MD5STREAM_OBJECT

```
#define DDSC_LOG_MD5STREAM_OBJECT 23
```

A MD5 stream object.

DDSC_LOG_CONDITION_OBJECT

```
#define DDSC_LOG_CONDITION_OBJECT 24
```

A DDS Condition object.

DDSC_LOG_RT_OBJECT

```
#define DDSC_LOG_RT_OBJECT 25
```

A RT Condition object.

DDSC_LOG_PARTICIPANT_POOL_OBJECT

```
#define DDSC_LOG_PARTICIPANT_POOL_OBJECT 26
```

A Participant pool object.

DDSC_LOG_TIMER_OBJECT

```
#define DDSC_LOG_TIMER_OBJECT 27
```

A OSAPI_Timer object.

DDSC_LOG_TIMEROUT_OBJECT

```
#define DDSC_LOG_TIMEROUT_OBJECT 28
```

A OSAPI_Timer timeout object.

DDSC_LOG_TOPIC_OBJECT

```
#define DDSC_LOG_TOPIC_OBJECT 29
```

A DDS Topic object.

DDSC_LOG_WAITSET_OBJECT

```
#define DDSC_LOG_WAITSET_OBJECT 30
```

A DDS WaitSet object.

DDSC_LOG_UUID_OBJECT

```
#define DDSC_LOG_UUID_OBJECT 31
```

A UUID object.

DDSC_LOG_MEMPOOL_OBJECT

```
#define DDSC_LOG_MEMPOOL_OBJECT 32
```

A REDA_MemPool object.

DDSC_LOG_PARTICIPANT_PROPERTIES

```
#define DDSC_LOG_PARTICIPANT_PROPERTIES 33
```

A DomainParticipant's string properties sequence.

DDSC_LOG_PARTICIPANT_BINARY_PROPERTIES

```
#define DDSC_LOG_PARTICIPANT_BINARY_PROPERTIES 34
```

A DomainParticipant's binary properties sequence.

DDSC_LOG_PARTICIPANT_BUILTIN_PUBLISHER

```
#define DDSC_LOG_PARTICIPANT_BUILTIN_PUBLISHER 35
```

A DomainParticipant's built-in Publisher.

DDSC_LOG_PARTICIPANT_BUILTIN_SUBSCRIBER

```
#define DDSC_LOG_PARTICIPANT_BUILTIN_SUBSCRIBER 36
```

A DomainParticipant's built-in Subscriber.

DDSC_LOG_PARTICIPANT_GENERIC_MESSAGE_OBJECT

```
#define DDSC_LOG_PARTICIPANT_GENERIC_MESSAGE_OBJECT 37
```

A built-in Inter-Participant Channel data type object.

DDSC_LOG_DATA_HOLDER_SEQ_OBJECT

```
#define DDSC_LOG_DATA_HOLDER_SEQ_OBJECT 38
```

A Trust data holder sequence object.

DDSC_LOG_PROPERTYQOSPOLICY_OBJECT

```
#define DDSC_LOG_PROPERTYQOSPOLICY_OBJECT 39
```

A Property QoS policy object.

DDSC_LOG_STRING_OBJECT

```
#define DDSC_LOG_STRING_OBJECT 40
```

A String object.

DDSC_LOG_AUTHPOOL_OBJECT

```
#define DDSC_LOG_AUTHPOOL_OBJECT 41
```

An authentication pool object.

DDSC_LOG_BUFFER_OBJECT

```
#define DDSC_LOG_BUFFER_OBJECT 42
```

A Log buffer object.

DDSC_LOG_STRINGMANAGER_OBJECT

```
#define DDSC_LOG_STRINGMANAGER_OBJECT 43
```

A String Manager object.

DDSC_LOG_OBJECT_INITIALIZE_EC

```
#define DDSC_LOG_OBJECT_INITIALIZE_EC (DDSC_LOG_BASE + 200)
```

Out of resources to initialize object of the specified kind.

DDSC_LOG_OBJECT_ALLOCATE_EC

```
#define DDSC_LOG_OBJECT_ALLOCATE_EC (DDSC_LOG_BASE + 201)
```

Out of resources to allocate an object of the specified kind.

DDSC_LOG_OBJECT_FINALIZE_EC

```
#define DDSC_LOG_OBJECT_FINALIZE_EC (DDSC_LOG_BASE + 202)
```

Failed to finalize object of specified kind.

DDSC_LOG_OBJECT_DELETE_EC

```
#define DDSC_LOG_OBJECT_DELETE_EC (DDSC_LOG_BASE + 203)
```

Failed to delete object of specified kind.

DDSC_LOG_OBJECT_COPY_EC

```
#define DDSC_LOG_OBJECT_COPY_EC (DDSC_LOG_BASE + 204)
```

Failed to copy object of specified kind.

DDSC_LOG_OBJECT_REFCOUNT_EC

```
#define DDSC_LOG_OBJECT_REFCOUNT_EC (DDSC_LOG_BASE + 205)
```

Failed to delete/finalize an object because other objects are referencing it.

DDSC_LOG_OBJECT_GET_PROPERTY_EC

```
#define DDSC_LOG_OBJECT_GET_PROPERTY_EC (DDSC_LOG_BASE + 206)
```

Failed to get the object properties.

DDSC_LOG_OBJECT_SET_PROPERTY_EC

```
#define DDSC_LOG_OBJECT_SET_PROPERTY_EC (DDSC_LOG_BASE + 207)
```

Failed to set the object properties.

DDSC_LOG_OBJECT_EMPTY_EC

```
#define DDSC_LOG_OBJECT_EMPTY_EC (DDSC_LOG_BASE + 208)
```

An object is empty, typically applies only to buffer-pool objects.

DDSC_LOG_OBJECT_INUSECOUNT_EC

```
#define DDSC_LOG_OBJECT_INUSECOUNT_EC (DDSC_LOG_BASE + 209)
```

Failed to delete/finalize an object because other objects are using it.

DDSC_LOG_OBJECT_CREATE_EC

```
#define DDSC_LOG_OBJECT_CREATE_EC (DDSC_LOG_BASE + 210)
```

Failed to create an object.

DDSC_LOG_NETMASK_SEQUENCE

```
#define DDSC_LOG_NETMASK_SEQUENCE 1
```

A NETIO Netmask sequence.

DDSC_LOG_ROUTE_SEQUENCE

```
#define DDSC_LOG_ROUTE_SEQUENCE 2
```

A NETIO Route sequence.

DDSC_LOG_RESERVED_SEQUENCE

```
#define DDSC_LOG_RESERVED_SEQUENCE 3
```

A NETIO Reserved address sequence.

DDSC_LOG_ENABLED_USER_TRANSPORT_SEQUENCE

```
#define DDSC_LOG_ENABLED_USER_TRANSPORT_SEQUENCE 4
```

A DDS sequence of enabled user NETIO transports.

DDSC_LOG_ENABLED_TRANSPORT_SEQUENCE

```
#define DDSC_LOG_ENABLED_TRANSPORT_SEQUENCE 5
```

A DDS sequence of enabled transports.

DDSC_LOG_ENABLED_DISCOVERY_TRANSPORT_SEQUENCE

```
#define DDSC_LOG_ENABLED_DISCOVERY_TRANSPORT_SEQUENCE 6
```

A DDS sequence of enabled discovery transports.

DDSC_LOG_INITIAL_PEER_SEQUENCE

```
#define DDSC_LOG_INITIAL_PEER_SEQUENCE 7
```

A DDS sequence of initial peers.

DDSC_LOG_DESTINATION_SEQUENCE

```
#define DDSC_LOG_DESTINATION_SEQUENCE 8
```

A NETIO sequence of destinations to send to.

DDSC_LOG_METAUNICAST_SEQUENCE

```
#define DDSC_LOG_METAUNICAST_SEQUENCE 9
```

A DDS sequence of meta-traffic unicast locators.

DDSC_LOG_METAMULTICAST_SEQUENCE

```
#define DDSC_LOG_METAMULTICAST_SEQUENCE 10
```

A DDS sequence of meta-traffic multicast locators.

DDSC_LOG_USERUNICAST_SEQUENCE

```
#define DDSC_LOG_USERUNICAST_SEQUENCE 11
```

A DDS sequence of user-traffic unicast locators.

DDSC_LOG_USERMULTICAST_SEQUENCE

```
#define DDSC_LOG_USERMULTICAST_SEQUENCE 12
```

A DDS sequence of user-traffic multicast locators.

DDSC_LOG_READTAKE_SEQUENCE

```
#define DDSC_LOG_READTAKE_SEQUENCE 13
```

A DDS sequence used in a read/take call.

DDSC_LOG_DATAHOLDER_SEQUENCE

```
#define DDSC_LOG_DATAHOLDER_SEQUENCE 14
```

A DDS sequence of DataHolder objects.

DDSC_LOG_COOKIE_PAYLOAD_SEQUENCE

```
#define DDSC_LOG_COOKIE_PAYLOAD_SEQUENCE 15
```

A cookie octet sequence.

DDSC_LOG_REPRESENTATION_ID_SEQUENCE

```
#define DDSC_LOG_REPRESENTATION_ID_SEQUENCE 16
```

a sequence for data representation IDs

DDSC_LOG_SEQUENCE_SETMAX_EC

```
#define DDSC_LOG_SEQUENCE_SETMAX_EC (DDSC_LOG_BASE + 300)
```

Failed to set the maximum length of a sequence of the specified kind.

DDSC_LOG_SEQUENCE_SETLENGTH_EC

```
#define DDSC_LOG_SEQUENCE_SETLENGTH_EC (DDSC_LOG_BASE + 301)
```

Failed to set the length of a sequence of the specified kind.

DDSC_LOG_SEQUENCE_GETREF_EC

```
#define DDSC_LOG_SEQUENCE_GETREF_EC (DDSC_LOG_BASE + 302)
```

Failed to get a reference at the specified index for a sequence of the specified kind.

DDSC_LOG_SEQUENCE_INITIALIZE_EC

```
#define DDSC_LOG_SEQUENCE_INITIALIZE_EC (DDSC_LOG_BASE + 303)
```

Failed to initialize a sequence of the specified kind.

DDSC_LOG_SEQUENCE_FINALIZE_EC

```
#define DDSC_LOG_SEQUENCE_FINALIZE_EC (DDSC_LOG_BASE + 304)
```

Failed to finalize a sequence of the specified kind.

DDSC_LOG_SEQUENCE_COPY_EC

```
#define DDSC_LOG_SEQUENCE_COPY_EC (DDSC_LOG_BASE + 305)
```

Failed to copy a sequence of the specified kind.

DDSC_LOG_SEQUENCE_INVALID_EC

```
#define DDSC_LOG_SEQUENCE_INVALID_EC (DDSC_LOG_BASE + 306)
```

The sequence of the specified kind was invalid in the context it is used.

DDSC_LOG_DISCOVERY_COMPONENT

```
#define DDSC_LOG_DISCOVERY_COMPONENT 1
```

A component of discovery kind.

DDSC_LOG_RTPS_COMPONENT

```
#define DDSC_LOG_RTPS_COMPONENT 2
```

A component of RTPS kind.

DDSC_LOG_READERHISTORY_COMPONENT

```
#define DDSC_LOG_READERHISTORY_COMPONENT 3
```

A component of reader-history kind.

DDSC_LOG_WRITERHISTORY_COMPONENT

```
#define DDSC_LOG_WRITERHISTORY_COMPONENT 4
```

A component of writer-history kind.

DDSC_LOG_DATAREADERIO_COMPONENT

```
#define DDSC_LOG_DATAREADERIO_COMPONENT 5
```

A component of DataReaderInterface kind.

DDSC_LOG_DATAWRITERIO_COMPONENT

```
#define DDSC_LOG_DATAWRITERIO_COMPONENT 6
```

A component of DataWriterInterface kind.

DDSC_LOG_NETIO_COMPONENT

```
#define DDSC_LOG_NETIO_COMPONENT 7
```

A component of NETIO kind.

DDSC_LOG_TRANSPORT_COMPONENT

```
#define DDSC_LOG_TRANSPORT_COMPONENT 8
```

A component of transport kind.

DDSC_LOG_APPGEN_COMPONENT

```
#define DDSC_LOG_APPGEN_COMPONENT 9
```

A component of application generation kind.

DDSC_LOG_COMPONENT_LOOKUP_EC

```
#define DDSC_LOG_COMPONENT_LOOKUP_EC (DDSC_LOG_BASE + 400)
```

Did not find a component factory with the given name in the registry.

DDSC_LOG_COMPONENT_CREATE_EC

```
#define DDSC_LOG_COMPONENT_CREATE_EC (DDSC_LOG_BASE + 401)
```

Could not create a component of the specified kind using the specified factory.

DDSC_LOG_COMPONENT_DELETE_EC

```
#define DDSC_LOG_COMPONENT_DELETE_EC (DDSC_LOG_BASE + 402)
```

Could not delete a component of the specified kind using the specified factory.

DDSC_LOG_HISTORY_RESOURCE

```
#define DDSC_LOG_HISTORY_RESOURCE 1
```

Exceeded resource limits for writer history queue.

A DataWriter failed to get a queue entry for a new sample it is attempting to send, due to exceeding a limit:

- When the sample is for new instance: DataWriterQos.resource_limits.max_instances may have been exceeded
- When the sample is for an existing instance, and History kind is KEEP_ALL: DataWriterQos.resource_limits.↔ max_samples_per_instance may have been exceeded.
- The limit on the total number of samples in the queue: DataWriterQos.resource_limits.max_samples, may have been exceeded.

DDSC_LOG_SAMPLE_RESOURCES

```
#define DDSC_LOG_SAMPLE_RESOURCES 2
```

Failed to get resource for new sample due to resource limit.

A DataWriter failed to get a buffer for a new sample being written because the limit DataWriterQos.resource_limits.↔ max_samples was exceeded.

A DataReader failed in getting a buffer for a newly received sample because DataReaderQos.resource_limits.max_↔ samples was exceeded.

DDSC_LOG_PARTICIPANT_RESOURCES

```
#define DDSC_LOG_PARTICIPANT_RESOURCES 3
```

Failed to allocate a new participant.

The DomainParticipantFactory failed to get a buffer for a new DomainParticipant because DomainParticipantFactory←
Qos.system_resource.max_participants because was exceeded.

DDSC_LOG_RESOURCE_EXCEEDED_EC

```
#define DDSC_LOG_RESOURCE_EXCEEDED_EC (DDSC_LOG_BASE + 500)
```

Could not allocate a resource of the specified kind.

DDSC_LOG_TOPIC_QOS

```
#define DDSC_LOG_TOPIC_QOS 1
```

The TopicQos kind.

DDSC_LOG_DATAREADER_QOS

```
#define DDSC_LOG_DATAREADER_QOS 2
```

The DataReaderQos kind.

DDSC_LOG_DATAWRITER_QOS

```
#define DDSC_LOG_DATAWRITER_QOS 3
```

The DataWriterQos kind.

DDSC_LOG_SUBSCRIBER_QOS

```
#define DDSC_LOG_SUBSCRIBER_QOS 4
```

The SubscriberQos kind.

DDSC_LOG_PUBLISHER_QOS

```
#define DDSC_LOG_PUBLISHER_QOS 5
```

The PublisherQos kind.

DDSC_LOG_PARTICIPANT_QOS

```
#define DDSC_LOG_PARTICIPANT_QOS 6
```

The DomainParticipantQos kind.

DDSC_LOG_PARTICIPANTFACTORY_QOS

```
#define DDSC_LOG_PARTICIPANTFACTORY_QOS 7
```

The DomainParticipantFactoryQos kind.

DDSC_LOG_DEFAULTTOPIC_QOS

```
#define DDSC_LOG_DEFAULTTOPIC_QOS 8
```

The default TopicQos kind.

DDSC_LOG_DEFAULTDATAREADER_QOS

```
#define DDSC_LOG_DEFAULTDATAREADER_QOS 9
```

The default DataReaderQos kind.

DDSC_LOG_DEFAULTDATAWRITER_QOS

```
#define DDSC_LOG_DEFAULTDATAWRITER_QOS 10
```

The default DataWriterQos kind.

DDSC_LOG_DEFAULTSUBSCRIBER_QOS

```
#define DDSC_LOG_DEFAULTSUBSCRIBER_QOS 11
```

The default SubscriberQos kind.

DDSC_LOG_DEFAULTPUBLISHER_QOS

```
#define DDSC_LOG_DEFAULTPUBLISHER_QOS 12
```

The default PublisherQos kind.

DDSC_LOG_DEFAULTPARTICIPANT_QOS

```
#define DDSC_LOG_DEFAULTPARTICIPANT_QOS 13
```

The default PublisherQos kind.

DDSC_LOG_DEADLINE_QOS_POLICY

```
#define DDSC_LOG_DEADLINE_QOS_POLICY 14
```

The deadline qos policy kind.

DDSC_LOG_LIVELINESS_QOS_POLICY

```
#define DDSC_LOG_LIVELINESS_QOS_POLICY 15
```

The liveliness qos policy kind.

DDSC_LOG_HISTORY_QOS_POLICY

```
#define DDSC_LOG_HISTORY_QOS_POLICY 16
```

The history qos policy kind.

DDSC_LOG_RESOURCE_LIMIT_QOS_POLICY

```
#define DDSC_LOG_RESOURCE_LIMIT_QOS_POLICY 17
```

The resource limits qos policy kind.

DDSC_LOG_PROTOCOL_QOS_POLICY

```
#define DDSC_LOG_PROTOCOL_QOS_POLICY 18
```

The protocol qos policy kind.

DDSC_LOG_TYPE_SUPPORT_QOS_POLICY

```
#define DDSC_LOG_TYPE_SUPPORT_QOS_POLICY 19
```

The type-support qos policy kind.

DDSC_LOG_RELIABILITY_QOS_POLICY

```
#define DDSC_LOG_RELIABILITY_QOS_POLICY 20
```

The reliability qos policy kind.

DDSC_LOG_DURABILITY_QOS_POLICY

```
#define DDSC_LOG_DURABILITY_QOS_POLICY 21
```

The durability qos policy kind.

DDSC_LOG_OWNERSHIP_QOS_POLICY

```
#define DDSC_LOG_OWNERSHIP_QOS_POLICY 22
```

The ownership qos policy kind.

DDSC_LOG_OWNERSHIP_STRENGTH_QOS_POLICY

```
#define DDSC_LOG_OWNERSHIP_STRENGTH_QOS_POLICY 23
```

The ownership-strength qos policy kind.

DDSC_LOG_TRANSPORT_QOS_POLICY

```
#define DDSC_LOG_TRANSPORT_QOS_POLICY 24
```

The transport qos policy kind.

DDSC_LOG_PARTICIPANT_ID_QOS_POLICY

```
#define DDSC_LOG_PARTICIPANT_ID_QOS_POLICY 25
```

The participant id qos policy kind.

DDSC_LOG_HEARTBEATS_QOS_POLICY

```
#define DDSC_LOG_HEARTBEATS_QOS_POLICY 26
```

The heartbeat qos policy kind.

When configured for reliable communication, `heartbeats_per_max_samples` must be fit within `max_samples`

DDSC_LOG_DATAWRITER_RESOURCE_QOS_POLICY

```
#define DDSC_LOG_DATAWRITER_RESOURCE_QOS_POLICY 27
```

The DataWriterQos writer_resource_limits policy.

An invalid value has been set for a limit of DataWriterQos.writer_resource_limits. Each value must be positive and finite. May be logged by Discovery writers with no initial_peers set.

DDSC_LOG_DATAREADER_RESOURCE_QOS_POLICY

```
#define DDSC_LOG_DATAREADER_RESOURCE_QOS_POLICY 28
```

The DataReaderQos reader_resource_limits policy.

DDSC_LOG_DESTINATION_ORDER_POLICY

```
#define DDSC_LOG_DESTINATION_ORDER_POLICY 29
```

The DestinationOrderQos destination_order policy.

DDSC_LOG_PROPERTY_QOS_POLICY

```
#define DDSC_LOG_PROPERTY_QOS_POLICY 30
```

The PropertyQos policy.

DDSC_LOG_PUBLISH_MODE_QOS_POLICY

```
#define DDSC_LOG_PUBLISH_MODE_QOS_POLICY 31
```

The PublishModeQos policy.

DDSC_LOG_DATA_REPRESENTATION_QOS_POLICY

```
#define DDSC_LOG_DATA_REPRESENTATION_QOS_POLICY 33
```

The DataRepresentationQos policy.

DDSC_LOG_LATENCY_BUDGET_QOS_POLICY

```
#define DDSC_LOG_LATENCY_BUDGET_QOS_POLICY 34
```

The LatencyBudgetQos policy.

DDSC_LOG_CONTENT_FILTER_QOS_POLICY

```
#define DDSC_LOG_CONTENT_FILTER_QOS_POLICY 35
```

The ContentFilterQos policy.

DDSC_LOG_QOS_INCONSISTENT_POLICY_EC

```
#define DDSC_LOG_QOS_INCONSISTENT_POLICY_EC (DDSC_LOG_BASE + 501)
```

An inconsistent Qos policy for the specified Qos kind was found.

DDSC_LOG_QOS_INCONSISTENT_POLICIES_EC

```
#define DDSC_LOG_QOS_INCONSISTENT_POLICIES_EC (DDSC_LOG_BASE + 502)
```

Inconsistency between two Qos policies for the specified Qos kind was found.

DDSC_LOG_QOS_INCONSISTENT_EC

```
#define DDSC_LOG_QOS_INCONSISTENT_EC (DDSC_LOG_BASE + 503)
```

Failed to create an entity or set a qos due to inconsistent policy.

DDSC_LOG_QOS_COPY_EC

```
#define DDSC_LOG_QOS_COPY_EC (DDSC_LOG_BASE + 504)
```

Failed to copy a Qos of the specified kind.

DDSC_LOG_QOS_INITIALIZE_EC

```
#define DDSC_LOG_QOS_INITIALIZE_EC (DDSC_LOG_BASE + 505)
```

Failed to initialize a Qos of the specified kind.

DDSC_LOG_QOS_FINALIZE_EC

```
#define DDSC_LOG_QOS_FINALIZE_EC (DDSC_LOG_BASE + 506)
```

Failed to finalize a Qos of the specified kind.

DDSC_LOG_QOS_SET_EC

```
#define DDSC_LOG_QOS_SET_EC (DDSC_LOG_BASE + 507)
```

Failed to set a Qos of the specified kind.

DDSC_LOG_QOS_SET_ON_ENABLED_EC

```
#define DDSC_LOG_QOS_SET_ON_ENABLED_EC (DDSC_LOG_BASE + 508)
```

Failed to set a Qos of the specified kind because the entity is already enabled.

DDSC_LOG_QOS_IMMUTABLE_EC

```
#define DDSC_LOG_QOS_IMMUTABLE_EC (DDSC_LOG_BASE + 509)
```

Failed to set a Qos of the specified kind the immutable Qos policies have been changed.

DDSC_LOG_QOS_CHANGED_EC

```
#define DDSC_LOG_QOS_CHANGED_EC (DDSC_LOG_BASE + 510)
```

A discovered Qos changed (the entity already existed)

DDSC_LOG_QOS_GET_EC

```
#define DDSC_LOG_QOS_GET_EC (DDSC_LOG_BASE + 511)
```

Failed to get a Qos of the specified kind.

DDSC_LOG_TOPIC_LISTENER

```
#define DDSC_LOG_TOPIC_LISTENER 1
```

The topic listener kind.

DDSC_LOG_DATAREADER_LISTENER

```
#define DDSC_LOG_DATAREADER_LISTENER 2
```

The datareader listener kind.

DDSC_LOG_DATAWRITER_LISTENER

```
#define DDSC_LOG_DATAWRITER_LISTENER 3
```

The datawriter listener kind.

DDSC_LOG_SUBSCRIBER_LISTENER

```
#define DDSC_LOG_SUBSCRIBER_LISTENER 4
```

The subscriber listener kind.

DDSC_LOG_PUBLISHER_LISTENER

```
#define DDSC_LOG_PUBLISHER_LISTENER 5
```

The publisher listener kind.

DDSC_LOG_PARTICIPANT_LISTENER

```
#define DDSC_LOG_PARTICIPANT_LISTENER 6
```

The participant listener kind.

DDSC_LOG_PARTICIPANTFACTORY_LISTENER

```
#define DDSC_LOG_PARTICIPANTFACTORY_LISTENER 7
```

The participant-factory listener kind.

DDSC_LOG_LISTENER_INCONSISTENT_EC

```
#define DDSC_LOG_LISTENER_INCONSISTENT_EC (DDSC_LOG_BASE + 512)
```

Failed to create an entity due to inconsistent listener and status mask.

DDSC_LOG_LISTENER_SET_EC

```
#define DDSC_LOG_LISTENER_SET_EC (DDSC_LOG_BASE + 513)
```

Failed to set the listener of the specified kind.

DDSC_LOG_LISTENER_GET_EC

```
#define DDSC_LOG_LISTENER_GET_EC (DDSC_LOG_BASE + 514)
```

Failed to get the listener of the specified kind.

DDSC_LOG_LISTENER_SET_ILLEGAL_NULL_EC

```
#define DDSC_LOG_LISTENER_SET_ILLEGAL_NULL_EC (DDSC_LOG_BASE + 515)
```

Illegal combination of NULL listener and non-NONE status mask when setting a listener for an Entity.

DDSC_LOG_TOPIC_ENTITY

```
#define DDSC_LOG_TOPIC_ENTITY 1
```

The topic entity kind.

DDSC_LOG_DATAWRITER_ENTITY

```
#define DDSC_LOG_DATAWRITER_ENTITY 2
```

The datawriter entity kind.

DDSC_LOG_DATAREADER_ENTITY

```
#define DDSC_LOG_DATAREADER_ENTITY 3
```

The datareader entity kind.

DDSC_LOG_PUBLISHER_ENTITY

```
#define DDSC_LOG_PUBLISHER_ENTITY 4
```

The publisher entity kind.

DDSC_LOG_SUBSCRIBER_ENTITY

```
#define DDSC_LOG_SUBSCRIBER_ENTITY 5
```

The subscriber entity kind.

DDSC_LOG_PARTICIPANT_ENTITY

```
#define DDSC_LOG_PARTICIPANT_ENTITY 6
```

The participant entity kind.

DDSC_LOG_PUBLICATION_ENTITY

```
#define DDSC_LOG_PUBLICATION_ENTITY 7
```

The publication entity kind.

DDSC_LOG_SUBSCRIPTION_ENTITY

```
#define DDSC_LOG_SUBSCRIPTION_ENTITY 8
```

The subscription entity kind.

DDSC_LOG_ENTITY_ENABLE_EC

```
#define DDSC_LOG_ENTITY_ENABLE_EC (DDSC_LOG_BASE + 600)
```

Failed to enable an entity of the specified kind.

DDSC_LOG_ENTITY_NOT_EMPTY_EC

```
#define DDSC_LOG_ENTITY_NOT_EMPTY_EC (DDSC_LOG_BASE + 601)
```

Failed to an delete/finalize an entity of the specified kind because it is not empty.

DDSC_LOG_ENTITY_FINALIZE_EC

```
#define DDSC_LOG_ENTITY_FINALIZE_EC (DDSC_LOG_BASE + 602)
```

Failed to finalize an entity of the specified kind.

DDSC_LOG_ENTITY_INITIALIZE_EC

```
#define DDSC_LOG_ENTITY_INITIALIZE_EC (DDSC_LOG_BASE + 603)
```

Failed to initialize an entity of the specified kind.

DDSC_LOG_ENTITY_NOT_ENABLED_EC

```
#define DDSC_LOG_ENTITY_NOT_ENABLED_EC (DDSC_LOG_BASE + 604)
```

An operation was attempted on an entity that is not enabled.

DDSC_LOG_ENTITY_DIFFERENT_FACTORY_EC

```
#define DDSC_LOG_ENTITY_DIFFERENT_FACTORY_EC (DDSC_LOG_BASE + 605)
```

Entities are in different factories.

A entity tried to use an entity created by a different factory. For example, it is illegal to create a datawriter using a topic from a different participant.

DDSC_LOG_ENTITY_INVALID_PROPERTY_EC

```
#define DDSC_LOG_ENTITY_INVALID_PROPERTY_EC (DDSC_LOG_BASE + 606)
```

An invalid property was specified for the entity.

DDSC_LOG_DATAWRITER_CDR

```
#define DDSC_LOG_DATAWRITER_CDR 1
```

The datawriter CDR kind.

DDSC_LOG_DATAREADER_CDR

```
#define DDSC_LOG_DATAREADER_CDR 2
```

The datareader CDR kind.

DDSC_LOG_DATAWRITER_INLINE

```
#define DDSC_LOG_DATAWRITER_INLINE 3
```

The datawriter inline kind.

DDSC_LOG_CDR_POOL_ALLOC_EC

```
#define DDSC_LOG_CDR_POOL_ALLOC_EC (DDSC_LOG_BASE + 700)
```

Failed to allocate a pool of the specified kind.

DDSC_LOG_CDR_BUFFER_SET_EC

```
#define DDSC_LOG_CDR_BUFFER_SET_EC (DDSC_LOG_BASE + 701)
```

Failed to set the CDR buffer for a packet.

DDSC_LOG_CDR_POOL_DELETE_EC

```
#define DDSC_LOG_CDR_POOL_DELETE_EC (DDSC_LOG_BASE + 702)
```

Failed to delete the CDR pool.

DDSC_LOG_CDR_SERIALIZE_PID_EC

```
#define DDSC_LOG_CDR_SERIALIZE_PID_EC (DDSC_LOG_BASE + 703)
```

Failed to serialize a parameter ID.

DDSC_LOG_CDR_SERIALIZE_PID_LENGTH_EC

```
#define DDSC_LOG_CDR_SERIALIZE_PID_LENGTH_EC (DDSC_LOG_BASE + 704)
```

Failed to serialize a parameter length.

DDSC_LOG_CDR_SERIALIZE_KEYHASH_EC

```
#define DDSC_LOG_CDR_SERIALIZE_KEYHASH_EC (DDSC_LOG_BASE + 705)
```

Failed to serialize a key-hash.

DDSC_LOG_CDR_SERIALIZE_DATA_EC

```
#define DDSC_LOG_CDR_SERIALIZE_DATA_EC (DDSC_LOG_BASE + 706)
```

Failed to serialize payload data.

DDSC_LOG_DESERIALIZE_BAD_PID_LENGTH_EC

```
#define DDSC_LOG_DESERIALIZE_BAD_PID_LENGTH_EC (DDSC_LOG_BASE + 707)
```

Deserialized an invalid parameter length for a specific parameter ID.

DDSC_LOG_CDR_DESERIALIZE_PID_EC

```
#define DDSC_LOG_CDR_DESERIALIZE_PID_EC (DDSC_LOG_BASE + 708)
```

Failed to deserialize the ID of an inline parameter.

DDSC_LOG_CDR_DESERIALIZE_PID_LENGTH_EC

```
#define DDSC_LOG_CDR_DESERIALIZE_PID_LENGTH_EC (DDSC_LOG_BASE + 709)
```

Failed to deserialize the length of an inline parameter.

DDSC_LOG_CDR_INCREMENT_POS_EC

```
#define DDSC_LOG_CDR_INCREMENT_POS_EC (DDSC_LOG_BASE + 710)
```

Failed to increment to the position of the next inline parameter.

DDSC_LOG_CDR_SET_POS_EC

```
#define DDSC_LOG_CDR_SET_POS_EC (DDSC_LOG_BASE + 711)
```

Failed to set the reception stream position.

DDSC_LOG_CDR_DESERIALIZE_HEADER_EC

```
#define DDSC_LOG_CDR_DESERIALIZE_HEADER_EC (DDSC_LOG_BASE + 712)
```

Failed to deserialize the encapsulation header.

DDSC_LOG_CDR_DESERIALIZE_DATA_EC

```
#define DDSC_LOG_CDR_DESERIALIZE_DATA_EC (DDSC_LOG_BASE + 713)
```

Failed to deserialize CDR payload data.

DDSC_LOG_CDR_INITIALIZE_SAMPLE_EC

```
#define DDSC_LOG_CDR_INITIALIZE_SAMPLE_EC (DDSC_LOG_BASE + 714)
```

Failed to initialize CDR sample.

DDSC_LOG_CDR_FINALIZE_SAMPLE_EC

```
#define DDSC_LOG_CDR_FINALIZE_SAMPLE_EC (DDSC_LOG_BASE + 715)
```

Failed to finalize sample.

DDSC_LOG_CDR_SERIALIZE_STATUS_INFO_EC

```
#define DDSC_LOG_CDR_SERIALIZE_STATUS_INFO_EC (DDSC_LOG_BASE + 716)
```

Failed to serialize the status info parameter.

DDSC_LOG_CDR_DESERIALIZE_KEYHASH_EC

```
#define DDSC_LOG_CDR_DESERIALIZE_KEYHASH_EC (DDSC_LOG_BASE + 717)
```

Failed to deserialize a key-hash.

DDSC_LOG_CDR_DESERIALIZE_KEY_EC

```
#define DDSC_LOG_CDR_DESERIALIZE_KEY_EC (DDSC_LOG_BASE + 718)
```

Failed to deserialize CDR payload key.

DDSC_LOG_NETIO_ADD_ANON_TOPIC_ROUTE_EC

```
#define DDSC_LOG_NETIO_ADD_ANON_TOPIC_ROUTE_EC (DDSC_LOG_BASE + 800)
```

Failed to add a route to an anonymous participant discovery datawriter.

DDSC_LOG_NETIO_ADD_TOPIC_ROUTE_EC

```
#define DDSC_LOG_NETIO_ADD_TOPIC_ROUTE_EC (DDSC_LOG_BASE + 801)
```

Failed to add a route to a topic from a datawriter.

DDSC_LOG_NETIO_DELETE_TOPIC_ROUTE_EC

```
#define DDSC_LOG_NETIO_DELETE_TOPIC_ROUTE_EC (DDSC_LOG_BASE + 802)
```

Failed to delete a route to a topic.

DDSC_LOG_NETIO_FORWARD_TOPIC_EC

```
#define DDSC_LOG_NETIO_FORWARD_TOPIC_EC (DDSC_LOG_BASE + 803)
```

Failed to forward a topic.

DDSC_LOG_NETIO_NETIO_KIND

```
#define DDSC_LOG_NETIO_NETIO_KIND 1
```

Any NETIO interface kind, typically UDP.

DDSC_LOG_INTRA_NETIO_KIND

```
#define DDSC_LOG_INTRA_NETIO_KIND 2
```

The intra interface kind.

DDSC_LOG_RTPS_NETIO_KIND

```
#define DDSC_LOG_RTPS_NETIO_KIND 3
```

The RTPS interface kind.

DDSC_LOG_DATAREADER_NETIO_KIND

```
#define DDSC_LOG_DATAREADER_NETIO_KIND 4
```

The DataReader interface kind.

DDSC_LOG_DATAWRITER_NETIO_KIND

```
#define DDSC_LOG_DATAWRITER_NETIO_KIND 5
```

The DataWriter interface kind.

DDSC_LOG_NETIO_BIND_EXTERNAL_EC

```
#define DDSC_LOG_NETIO_BIND_EXTERNAL_EC (DDSC_LOG_BASE + 804)
```

Failed to bind two external interface of the specified kind.

DDSC_LOG_NETIO_UNBIND_EXTERNAL_EC

```
#define DDSC_LOG_NETIO_UNBIND_EXTERNAL_EC (DDSC_LOG_BASE + 805)
```

Failed to unbind two external interfaces of the specified kind.

DDSC_LOG_NETIO_BIND_EC

```
#define DDSC_LOG_NETIO_BIND_EC (DDSC_LOG_BASE + 806)
```

Failed to bind an interface to a peer interface.

DDSC_LOG_NETIO_UNBIND_EC

```
#define DDSC_LOG_NETIO_UNBIND_EC (DDSC_LOG_BASE + 807)
```

Failed to unbind an interface from a peer interface.

DDSC_LOG_NETIO_ADD_ROUTE_EC

```
#define DDSC_LOG_NETIO_ADD_ROUTE_EC (DDSC_LOG_BASE + 808)
```

Failed to add a route from an interface to a peer interface.

DDSC_LOG_NETIO_DELETE_ROUTE_EC

```
#define DDSC_LOG_NETIO_DELETE_ROUTE_EC (DDSC_LOG_BASE + 809)
```

Failed to delete a route from an interface to a peer interface.

DDSC_LOG_NETIO_GET_EXTERNAL_INTF_EC

```
#define DDSC_LOG_NETIO_GET_EXTERNAL_INTF_EC (DDSC_LOG_BASE + 810)
```

Failed to get an external interface for the specified interface kind.

DDSC_LOG_NETIO_NO_ROUTE_EC

```
#define DDSC_LOG_NETIO_NO_ROUTE_EC (DDSC_LOG_BASE + 811)
```

A DataReader failed a bind due to no existing route.

DDSC_LOG_NETIO_ROUTE_LOOKUP_FAILED_EC

```
#define DDSC_LOG_NETIO_ROUTE_LOOKUP_FAILED_EC (DDSC_LOG_BASE + 812)
```

Lookup a route to a destination failed.

A failure was encountered when trying to lookup a route to a destination, this log message is preceded by a more specific message. Failure to lookup a route does not mean the route does not exist, it means it failed to determine if a route did exist.

DDSC_LOG_NETIO_GET_ROUTE_TABLE_FAILED_EC

```
#define DDSC_LOG_NETIO_GET_ROUTE_TABLE_FAILED_EC (DDSC_LOG_BASE + 813)
```

Failed to get the route table for an interface.

DDSC_LOG_NETIO_SEND_FAILED_EC

```
#define DDSC_LOG_NETIO_SEND_FAILED_EC (DDSC_LOG_BASE + 814)
```

Failure when sending on an interface.

DDSC_LOG_NETIO_SET_STATE_EC

```
#define DDSC_LOG_NETIO_SET_STATE_EC (DDSC_LOG_BASE + 815)
```

Failed to set an interface state.

DDSC_LOG_NETIO_PEER_LOOKUP_EC

```
#define DDSC_LOG_NETIO_PEER_LOOKUP_EC (DDSC_LOG_BASE + 816)
```

Datawriter did not find a peer.

DDSC_LOG_NETIO_FORCED_REMOVE_EC

```
#define DDSC_LOG_NETIO_FORCED_REMOVE_EC (DDSC_LOG_BASE + 817)
```

Forced removal of sample downstream failed.

The datawriter failed to force a sample removal downstream

DDSC_LOG_PACKET_INIT_EC

```
#define DDSC_LOG_PACKET_INIT_EC (DDSC_LOG_BASE + 818)
```

Failed to initialize a packet.

DDSC_LOG_PACKET_SET_HEAD_EC

```
#define DDSC_LOG_PACKET_SET_HEAD_EC (DDSC_LOG_BASE + 819)
```

Failed to set the head of a packet.

DDSC_LOG_PACKET_SET_TAIL_EC

```
#define DDSC_LOG_PACKET_SET_TAIL_EC (DDSC_LOG_BASE + 820)
```

Failed to set the tail of a packet.

DDSC_LOG_NETIO_DISCOVERY_ENABLED_TRANSPORT_EC

```
#define DDSC_LOG_NETIO_DISCOVERY_ENABLED_TRANSPORT_EC (DDSC_LOG_BASE + 821)
```

Configured discovery enabled transport is not valid.

DDSC_LOG_NETIO_PACKETPOOL_RETURN_EC

```
#define DDSC_LOG_NETIO_PACKETPOOL_RETURN_EC (DDSC_LOG_BASE + 822)
```

Failed to return a packet to the packet pool.

DDSC_LOG_NETIO_PACKETPOOL_GET_EC

```
#define DDSC_LOG_NETIO_PACKETPOOL_GET_EC (DDSC_LOG_BASE + 823)
```

Failed to get a packet from the packet pool.

DDSC_LOG_NETIO_PACKETPOOL_CREATE_EC

```
#define DDSC_LOG_NETIO_PACKETPOOL_CREATE_EC (DDSC_LOG_BASE + 824)
```

Failed to create packet pool.

DDSC_LOG_NETIO_PACKETPOOL_DELETE_EC

```
#define DDSC_LOG_NETIO_PACKETPOOL_DELETE_EC (DDSC_LOG_BASE + 825)
```

Failed to delete packet pool.

DDSC_LOG_DW_ACKNACK_FAILED_EC

```
#define DDSC_LOG_DW_ACKNACK_FAILED_EC (DDSC_LOG_BASE + 900)
```

Failed to ACKNACK sample in the writer history.

DDSC_LOG_DW_COMMIT_EC

```
#define DDSC_LOG_DW_COMMIT_EC (DDSC_LOG_BASE + 901)
```

Failed to commit a sample to the writer queue.

DDSC_LOG_DW_KEYHASH_CREATE_EC

```
#define DDSC_LOG_DW_KEYHASH_CREATE_EC (DDSC_LOG_BASE + 902)
```

Failed to create keyhash of instance handle.

DDSC_LOG_DW_ILLEGAL_KEY_KIND_EC

```
#define DDSC_LOG_DW_ILLEGAL_KEY_KIND_EC (DDSC_LOG_BASE + 903)
```

Failed a write due to an invalid key kind for the type being written.

DDSC_LOG_DW_CREATE_TYPED_WRITER_EC

```
#define DDSC_LOG_DW_CREATE_TYPED_WRITER_EC (DDSC_LOG_BASE + 904)
```

Failed to create a typed writer.

DDSC_LOG_DW_HISTORY_REGISTER_KEY_EC

```
#define DDSC_LOG_DW_HISTORY_REGISTER_KEY_EC (DDSC_LOG_BASE + 905)
```

Failed to register the key of an instance.

A DataWriter failed to register the key of a new instance because DataWriterQos.resource_limits.max_instances was exceeded.

DDSC_LOG_DW_UNKNOWN_FLOW_CONTROLLER_EC

```
#define DDSC_LOG_DW_UNKNOWN_FLOW_CONTROLLER_EC (DDSC_LOG_BASE + 906)
```

Failed to create a flow-controller.

A DataWriter failed to create the flow-controller specified in the Qos policy.

DDSC_LOG_DW_FLOW_CONTROLLER_REQUIRED_EC

```
#define DDSC_LOG_DW_FLOW_CONTROLLER_REQUIRED_EC (DDSC_LOG_BASE + 907)
```

The data-sample is larger than supported by the transport, but the flow-controller has been compiled out.

DDSC_LOG_DW_INSTANCE_ASSERTION_FAILED_EC

```
#define DDSC_LOG_DW_INSTANCE_ASSERTION_FAILED_EC (DDSC_LOG_BASE + 908)
```

Failed to reserve an instance to publish discovery data for a user created data writer. This typically means DomainParticipantQos.resource_limits.remote_writer_allocation is too small.

DDSC_LOG_DW_INCOMPATIBLE_PADDING_EC

```
#define DDSC_LOG_DW_INCOMPATIBLE_PADDING_EC (DDSC_LOG_BASE + 909)
```

A DataReader incompatible with padding bits in encapsulation header was discovered.

Discovered an incompatible Micro DataReader that cannot parse the padding bits set in the encapsulation options of a sample payload by the Micro DataWriter. Resolve by configuring the Micro DataWriter to omit padding bits or upgrade the Micro DataReader to a version that can interpret them.

Disable padding bits in the Micro DataWriter by setting the property [DDS_XTYPES_COMPLIANCE_MASK_PROPERTY](#) to a value that removes the encapsulation option padding bit. See the `::NDDS_Config_XTypesComplianceMask` section in RTI Connext Micro documentation for more information.

The version number of RTI Connext DDS Micro is printed as 4 hexadecimal digits:

- byte 3: Major
- byte 2: Minor
- byte 1: Release
- byte 0: Revision

DDSC_LOG_DR_CREATE_TYPED_READER_EC

```
#define DDSC_LOG_DR_CREATE_TYPED_READER_EC (DDSC_LOG_BASE + 1000)
```

Failed to create a typed datareader.

DDSC_LOG_DR_COPY_DATA_SAMPLE_EC

```
#define DDSC_LOG_DR_COPY_DATA_SAMPLE_EC (DDSC_LOG_BASE + 1001)
```

Failed to copy a sample upon reception, read, or take.

DDSC_LOG_DR_COMMIT_SAMPLE_EC

```
#define DDSC_LOG_DR_COMMIT_SAMPLE_EC (DDSC_LOG_BASE + 1002)
```

Failed to commit a sample to be made available to be read or taken.

DDSC_LOG_DR_FILTER_ERROR_EC

```
#define DDSC_LOG_DR_FILTER_ERROR_EC (DDSC_LOG_BASE + 1003)
```

A datareader filter function failed.

DDSC_LOG_DR_DESERIALIZE_KEYHASH_EC

```
#define DDSC_LOG_DR_DESERIALIZE_KEYHASH_EC (DDSC_LOG_BASE + 1004)
```

Failed to deserialize a key-hash parameter.

DDSC_LOG_DR_GET_ENTRY_FAILED_EC

```
#define DDSC_LOG_DR_GET_ENTRY_FAILED_EC (DDSC_LOG_BASE + 1005)
```

Failed to get a Reader History entry for a received sample.

DDSC_LOG_DR_COMMIT_ENTRY_EC

```
#define DDSC_LOG_DR_COMMIT_ENTRY_EC (DDSC_LOG_BASE + 1006)
```

Failed to commit a receive sample to Reader History to be read or taken.

DDSC_LOG_DR_UNREGISTER_KEY_EC

```
#define DDSC_LOG_DR_UNREGISTER_KEY_EC (DDSC_LOG_BASE + 1007)
```

A DataReader failed to unregister an instance.

DDSC_LOG_DR_DISPOSE_KEY_EC

```
#define DDSC_LOG_DR_DISPOSE_KEY_EC (DDSC_LOG_BASE + 1008)
```

A DataReader failed to dispose an instance.

DDSC_LOG_DR_READ_TAKE_FAILURE_EC

```
#define DDSC_LOG_DR_READ_TAKE_FAILURE_EC (DDSC_LOG_BASE + 1009)
```

A call to a reader/take function failed.

DDSC_LOG_DR_INSTANCE_TO_KEYHASH_EC

```
#define DDSC_LOG_DR_INSTANCE_TO_KEYHASH_EC (DDSC_LOG_BASE + 1010)
```

Failed to create keyhash of instance handle.

DDSC_LOG_DR_INSTANCE_MAPPING_EXHAUSTED_EC

```
#define DDSC_LOG_DR_INSTANCE_MAPPING_EXHAUSTED_EC (DDSC_LOG_BASE + 1011)
```

Failed to create keyhash of instance handle.

DDSC_LOG_DR_INSTANCE_ASSERTION_FAILED_EC

```
#define DDSC_LOG_DR_INSTANCE_ASSERTION_FAILED_EC (DDSC_LOG_BASE + 1012)
```

Failed to reserve an instance to publish discovery data for a user created data reader. This typically means Domain↔ ParticipantQos.resource_limits.remote_reader_allocation is too small.

DDSC_LOG_DR_ON_SAMPLE_REMOVED_EC

```
#define DDSC_LOG_DR_ON_SAMPLE_REMOVED_EC (DDSC_LOG_BASE + 1013)
```

Failed to remove a sample when it was being pushed out from the Reader History.

DDSC_LOG_TYPE_NAME_CMP_EC

```
#define DDSC_LOG_TYPE_NAME_CMP_EC (DDSC_LOG_BASE + 1100)
```

Two type names are incompatible.

DDSC_LOG_TOPIC_NAME_CMP_EC

```
#define DDSC_LOG_TOPIC_NAME_CMP_EC (DDSC_LOG_BASE + 1101)
```

Two topic names are incompatible.

DDSC_LOG_TYPE_FUNCTION_GET_SERIALIZED_SAMPLE_MAX_SIZE

```
#define DDSC_LOG_TYPE_FUNCTION_GET_SERIALIZED_SAMPLE_MAX_SIZE 1
```

The `get_serialized_sample_max_size` function kind.

DDSC_LOG_TYPE_FUNCTION_SERIALIZE_DATA

```
#define DDSC_LOG_TYPE_FUNCTION_SERIALIZE_DATA 2
```

The `serialize_data` function kind.

DDSC_LOG_TYPE_FUNCTION_DESERIALIZE_DATA

```
#define DDSC_LOG_TYPE_FUNCTION_DESERIALIZE_DATA 3
```

The `deserialize_data` function kind.

DDSC_LOG_TYPE_FUNCTION_CREATE_SAMPLE

```
#define DDSC_LOG_TYPE_FUNCTION_CREATE_SAMPLE 4
```

The `create_sample` function kind.

DDSC_LOG_TYPE_FUNCTION_COPY_SAMPLE

```
#define DDSC_LOG_TYPE_FUNCTION_COPY_SAMPLE 5
```

The `copy_sample` function kind.

DDSC_LOG_TYPE_FUNCTION_DELETE_SAMPLE

```
#define DDSC_LOG_TYPE_FUNCTION_DELETE_SAMPLE 6
```

The delete_sample function kind.

DDSC_LOG_TYPE_FUNCTION_GET_KEY_KIND

```
#define DDSC_LOG_TYPE_FUNCTION_GET_KEY_KIND 7
```

The get_key_kind function kind.

DDSC_LOG_TYPE_FUNCTION_INSTANCE_TO_KEYHASH

```
#define DDSC_LOG_TYPE_FUNCTION_INSTANCE_TO_KEYHASH 8
```

The instance_to_keyhash function kind.

DDSC_LOG_TYPE_FUNCTION_NULL_EC

```
#define DDSC_LOG_TYPE_FUNCTION_NULL_EC (DDSC_LOG_BASE + 1102)
```

Invalid type plugin, The specified function pointer is NULL.

DDSC_LOG_TOPIC_TOO_LONG_EC

```
#define DDSC_LOG_TOPIC_TOO_LONG_EC (DDSC_LOG_BASE + 1103)
```

Failed to create a topic because the name exceeded the maximum length of 255 octets (excluding the terminating NUL)

DDSC_LOG_TYPE_TOO_LONG_EC

```
#define DDSC_LOG_TYPE_TOO_LONG_EC (DDSC_LOG_BASE + 1104)
```

Failed to create a type because the name exceeded the maximum length of 255 octets (excluding the terminating NUL)

DDSC_LOG_LOOKUP_TYPE_PLUGIN_EC

```
#define DDSC_LOG_LOOKUP_TYPE_PLUGIN_EC (DDSC_LOG_BASE + 1105)
```

A type-plugin for the given type could not be found.

DDSC_LOG_TYPE_KEY_TYPE_EC

```
#define DDSC_LOG_TYPE_KEY_TYPE_EC (DDSC_LOG_BASE + 1106)
```

Two types have incompatible keys.

DDSC_LOG_TYPE_PLUGIN_GET_BUFFER_EC

```
#define DDSC_LOG_TYPE_PLUGIN_GET_BUFFER_EC (DDSC_LOG_BASE + 1107)
```

Type-plugin cannot return a buffer.

DDSC_LOG_TYPE_PLUGIN_INCONSISTENT_DR_EC

```
#define DDSC_LOG_TYPE_PLUGIN_INCONSISTENT_DR_EC (DDSC_LOG_BASE + 1108)
```

Inconsistent type-plugin DataReader.

DDSC_LOG_DUPLICATE_GUID_PREFIX_EC

```
#define DDSC_LOG_DUPLICATE_GUID_PREFIX_EC (DDSC_LOG_BASE + 1109)
```

A DomainParticipant with the specified GUID already exists in the DomainParticipant factory.

A DomainParticipant with the specified GUID already exists in the domain in the DomainParticipant factory. The GUID is either generated (default) or specified manually in the DomainParticipantQos.protocol policy. This error typically indicates that the DomainParticipantQos.protocol is manually specified and is a duplicate.

DDSC_LOG_TYPE_MANAGED_POOL_FAILURE_EC

```
#define DDSC_LOG_TYPE_MANAGED_POOL_FAILURE_EC (DDSC_LOG_BASE + 1110)
```

Managed pool function failed.

DDSC_LOG_TYPE_GET_SAMPLE_ID_EC

```
#define DDSC_LOG_TYPE_GET_SAMPLE_ID_EC (DDSC_LOG_BASE + 1111)
```

Managed pool failed to get sample id.

DDSC_LOG_PARTICIPANT_NAME_EMPTY_EC

```
#define DDSC_LOG_PARTICIPANT_NAME_EMPTY_EC (DDSC_LOG_BASE + 1112)
```

A DomainParticipant with enable_participant_discovery_by_name set to TRUE but an empty name is created.

If DomainParticipantQos.discovery.enable_participant_discovery_by_name is set to TRUE, but DomainParticipantQos.participant_name.name is empty. This is not allowed.

If DomainParticipantQos.discovery.enable_participant_discovery_by_name is set to TRUE, the requirement is that all DomainParticipant's in the domain are uniquely named. If this is not true the system may experience undefined behavior.

Note that it is not possible For RTI Connext DDS Micro to verify that all DomainParticipant's are uniquely named. This is the responsibility of the applications.

DDSC_LOG_DUPLICATE_PARTICIPANT_NAME_EC

```
#define DDSC_LOG_DUPLICATE_PARTICIPANT_NAME_EC (DDSC_LOG_BASE + 1113)
```

A DomainParticipant with enable_participant_discovery_by_name set to TRUE, but a local participant with the same DomainParticipantQos.participant_name.name already exists.

If DomainParticipantQos.discovery.enable_participant_discovery_by_name is set to TRUE, but an local DomainParticipant in the same DomainParticipantQos.participant_name.name already exists in the same process in the same domain. This is not allowed.

If DomainParticipantQos.discovery.enable_participant_discovery_by_name is set to TRUE, the requirement is that all DomainParticipant's in the domain are uniquely named. If this is not true the system may experience undefined behavior.

Note that it is not possible For RTI Connext DDS Micro to verify that all DomainParticipant's are uniquely named. This is the responsibility of the applications.

DDSC_LOG_DUPLICATE_REMOTE_PARTICIPANT_NAME_EC

```
#define DDSC_LOG_DUPLICATE_REMOTE_PARTICIPANT_NAME_EC (DDSC_LOG_BASE + 1114)
```

A remote participant is discovered with the same name as the a local participant name in the same domain ID.

If DomainParticipantQos.discovery.enable_participant_discovery_by_name is set to TRUE, but a remote participant is discovered with the same name as a local participant name in the same domain ID.

If DomainParticipantQos.discovery.enable_participant_discovery_by_name is set to TRUE, the requirement is that all DomainParticipant's in the domain are uniquely named. If this is not true the system may experience undefined behavior.

Note that it is not possible For RTI Connext DDS Micro to verify that all DomainParticipant's are uniquely named. This is the responsibility of the applications.

DDSC_LOG_REMOTE_PARTICIPANT_NAME_EMPTY_EC

```
#define DDSC_LOG_REMOTE_PARTICIPANT_NAME_EMPTY_EC (DDSC_LOG_BASE + 1115)
```

A remote participant without a name is discovered.

If `DomainParticipantQos.discovery.enable_participant_discovery_by_name` is set to `TRUE`, but a discovered participant's name is empty. This is not allowed.

If `DomainParticipantQos.discovery.enable_participant_discovery_by_name` is set to `TRUE`, the requirement is that all `DomainParticipant`'s in the domain are uniquely named. If this is not true the system may experience undefined behavior.

Note that it is not possible For RTI Connex Micro to verify that all `DomainParticipant`'s are uniquely named. This is the responsibility of the applications.

DDSC_LOG_REMOTE_PARTICIPANT_RESTART_EC

```
#define DDSC_LOG_REMOTE_PARTICIPANT_RESTART_EC (DDSC_LOG_BASE + 1116)
```

Failed to reset a remote participant.

If `DomainParticipantQos.discovery.enable_participant_discovery_by_name` is set to `TRUE`, but a discovered participant's name is empty. This is not allowed.

If `DomainParticipantQos.discovery.enable_participant_discovery_by_name` is set to `TRUE`, the requirement is that all `DomainParticipant`'s in the domain are uniquely named. If this is not true the system may experience undefined behavior.

Note that it is not possible For RTI Connex Micro to verify that all `DomainParticipant`'s are uniquely named. This is the responsibility of the applications.

DDSC_LOG_DISC_LOCAL_PARTICIPANT_ENABLED_EC

```
#define DDSC_LOG_DISC_LOCAL_PARTICIPANT_ENABLED_EC (DDSC_LOG_BASE + 1200)
```

Discovery plugin failed its update after a local `DomainParticipant` was enabled.

DDSC_LOG_DISC_BEFORE_LOCAL_PARTICIPANT_CREATED_EC

```
#define DDSC_LOG_DISC_BEFORE_LOCAL_PARTICIPANT_CREATED_EC (DDSC_LOG_BASE + 1201)
```

Discovery plugin failed its update before a local `DomainParticipant` was created.

DDSC_LOG_DISC_AFTER_LOCAL_PARTICIPANT_CREATED_EC

```
#define DDSC_LOG_DISC_AFTER_LOCAL_PARTICIPANT_CREATED_EC (DDSC_LOG_BASE + 1202)
```

Discovery plugin failed its update after a local `DomainParticipant` was created.

DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_ENABLED_EC

```
#define DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_ENABLED_EC (DDSC_LOG_BASE + 1203)
```

Discovery plugin failed its update after a local DataReader was enabled.

DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_DELETED_EC

```
#define DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_DELETED_EC (DDSC_LOG_BASE + 1204)
```

Discovery plugin failed its update after a local DataReader was deleted.

DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_ENABLED_EC

```
#define DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_ENABLED_EC (DDSC_LOG_BASE + 1205)
```

Discovery plugin failed its update after a local DataWriter was enabled.

DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_DELETED_EC

```
#define DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_DELETED_EC (DDSC_LOG_BASE + 1206)
```

Discovery plugin failed its update after a local DataWriter was deleted.

DDSC_LOG_DISC_BEFORE_REMOTE_PARTICIPANT_DELETED_EC

```
#define DDSC_LOG_DISC_BEFORE_REMOTE_PARTICIPANT_DELETED_EC (DDSC_LOG_BASE + 1207)
```

Discovery plugin failed its update after being notified that a remote DomainParticipant was about to be deleted.

DDSC_LOG_DISC_ADD_PEER_EC

```
#define DDSC_LOG_DISC_ADD_PEER_EC (DDSC_LOG_BASE + 1208)
```

Failed to add a peer with discovery plugin.

DDSC_LOG_DESERIALIZE_UNKNOWN_PID_EC

```
#define DDSC_LOG_DESERIALIZE_UNKNOWN_PID_EC (DDSC_LOG_BASE + 1209)
```

Deserialized a parameter of unknown ID.

DDSC_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC

```
#define DDSC_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC (DDSC_LOG_BASE + 1210)
```

Failed to serialize Builtin Endpoint Mask parameter.

DDSC_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC

```
#define DDSC_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC (DDSC_LOG_BASE + 1211)
```

Failed to deserialize Builtin Endpoint Mask parameter.

DDSC_LOG_SERIALIZE_TOPIC_NAME_EC

```
#define DDSC_LOG_SERIALIZE_TOPIC_NAME_EC (DDSC_LOG_BASE + 1212)
```

Failed to serialize Topic Name parameter.

DDSC_LOG_CDR_SET_OFFSET_EC

```
#define DDSC_LOG_CDR_SET_OFFSET_EC (DDSC_LOG_BASE + 1213)
```

Failed to set current offset of stream.

DDSC_LOG_SERIALIZE_ENTITY_NAME_EC

```
#define DDSC_LOG_SERIALIZE_ENTITY_NAME_EC (DDSC_LOG_BASE + 1214)
```

Failed to serialize Entity Name parameter.

DDSC_LOG_SERIALIZE_TYPE_NAME_EC

```
#define DDSC_LOG_SERIALIZE_TYPE_NAME_EC (DDSC_LOG_BASE + 1215)
```

Failed to serialize Type Name parameter.

DDSC_LOG_SERIALIZE_GUID_EC

```
#define DDSC_LOG_SERIALIZE_GUID_EC (DDSC_LOG_BASE + 1216)
```

Failed to serialize GUID key.

DDSC_LOG_SERIALIZE_DEFAULT_UNICAST_EC

```
#define DDSC_LOG_SERIALIZE_DEFAULT_UNICAST_EC (DDSC_LOG_BASE + 1217)
```

Failed to serialize Default Unicast Locator parameter.

DDSC_LOG_DESERIALIZE_LOCATOR_EC

```
#define DDSC_LOG_DESERIALIZE_LOCATOR_EC (DDSC_LOG_BASE + 1218)
```

Failed to deserialize a locator of the specified kind.

DDSC_LOG_LOCATORS_FULL_EC

```
#define DDSC_LOG_LOCATORS_FULL_EC (DDSC_LOG_BASE + 1219)
```

A locator sequence is full, the remaining locators of the specified kind are dropped.

DDSC_LOG_UNSUPPORTED_LOCATOR_EC

```
#define DDSC_LOG_UNSUPPORTED_LOCATOR_EC (DDSC_LOG_BASE + 1220)
```

The locator kind is not supported by any transport registered with the domain participant and is dropped.

DDSC_LOG_SERIALIZE_PROTOCOL_VERSION_EC

```
#define DDSC_LOG_SERIALIZE_PROTOCOL_VERSION_EC (DDSC_LOG_BASE + 1221)
```

Failed to serialize Protocol Version parameter.

DDSC_LOG_DESERIALIZE_PROTOCOL_VERSION_EC

```
#define DDSC_LOG_DESERIALIZE_PROTOCOL_VERSION_EC (DDSC_LOG_BASE + 1222)
```

Failed to deserialize Protocol Version parameter.

DDSC_LOG_DESERIALIZE_VENDOR_ID_EC

```
#define DDSC_LOG_DESERIALIZE_VENDOR_ID_EC (DDSC_LOG_BASE + 1223)
```

Failed to deserialize Vendor ID parameter.

DDSC_LOG_SERIALIZE_VENDOR_ID_EC

```
#define DDSC_LOG_SERIALIZE_VENDOR_ID_EC (DDSC_LOG_BASE + 1224)
```

Failed to serialize Vendor ID parameter.

DDSC_LOG_SERIALIZE_PRODUCT_VERSION_EC

```
#define DDSC_LOG_SERIALIZE_PRODUCT_VERSION_EC (DDSC_LOG_BASE + 1225)
```

Failed to serialize Product Version parameter.

DDSC_LOG_SERIALIZE_LEASE_DURATION_EC

```
#define DDSC_LOG_SERIALIZE_LEASE_DURATION_EC (DDSC_LOG_BASE + 1226)
```

Failed to serialize Lease Duration parameter.

DDSC_LOG_DATAWRITER_ADD_PEER_FAILED_EC

```
#define DDSC_LOG_DATAWRITER_ADD_PEER_FAILED_EC (DDSC_LOG_BASE + 1227)
```

Failed to add a remote peer to a local DataWriter.

DDSC_LOG_DATAREADER_ADD_PEER_FAILED_EC

```
#define DDSC_LOG_DATAREADER_ADD_PEER_FAILED_EC (DDSC_LOG_BASE + 1228)
```

Failed to add a remote peer to a local DataReader.

DDSC_LOG_DISC_REMOTE_PARTICIPANT_REMOVE_EC

```
#define DDSC_LOG_DISC_REMOTE_PARTICIPANT_REMOVE_EC (DDSC_LOG_BASE + 1229)
```

Failed to remote/reset remote participant after liveliness expired.

DDSC_LOG_DESERIALIZE_PARTITION_SEQ_EC

```
#define DDSC_LOG_DESERIALIZE_PARTITION_SEQ_EC (DDSC_LOG_BASE + 1230)
```

Failed to deserialize partition seq.

DDSC_LOG_INITIALIZE_DISCOVERY_QUEUE_EC

```
#define DDSC_LOG_INITIALIZE_DISCOVERY_QUEUE_EC (DDSC_LOG_BASE + 1231)
```

Failed to initialize discovery queue.

DDSC_LOG_QUEUE_PUBLICATION_DISCOVERY_EC

```
#define DDSC_LOG_QUEUE_PUBLICATION_DISCOVERY_EC (DDSC_LOG_BASE + 1232)
```

Failed to queue a publication discovery message.

DDSC_LOG_QUEUE_SUBSCRIPTION_DISCOVERY_EC

```
#define DDSC_LOG_QUEUE_SUBSCRIPTION_DISCOVERY_EC (DDSC_LOG_BASE + 1233)
```

Failed to queue a subscription discovery message.

DDSC_LOG_QUEUE_FINALIZE_EC

```
#define DDSC_LOG_QUEUE_FINALIZE_EC (DDSC_LOG_BASE + 1234)
```

Failed finalize discovery queue.

DDSC_LOG_IPC_INIT_FAILED_EC

```
#define DDSC_LOG_IPC_INIT_FAILED_EC (DDSC_LOG_BASE + 1300)
```

Failed to initialize Builtin IPC entities.

DDSC_LOG_IPC_ALLOC_FAILED_EC

```
#define DDSC_LOG_IPC_ALLOC_FAILED_EC (DDSC_LOG_BASE + 1301)
```

Failed to allocate memory for one of the Builtin IPC entities.

DDSC_LOG_IPC_FINALIZE_FAILED_EC

```
#define DDSC_LOG_IPC_FINALIZE_FAILED_EC (DDSC_LOG_BASE + 1302)
```

Failed to finalize all resources A potential memory leak might have been caused by unexpected errors during finalization of acquired resources.

DDSC_LOG_IPC_CHANNEL_TYPE_REGISTER_EC

```
#define DDSC_LOG_IPC_CHANNEL_TYPE_REGISTER_EC (DDSC_LOG_BASE + 1303)
```

Failed to register type for a Builtin IPC channel.

DDSC_LOG_IPC_CHANNEL_TOPIC_CREATE_EC

```
#define DDSC_LOG_IPC_CHANNEL_TOPIC_CREATE_EC (DDSC_LOG_BASE + 1304)
```

Failed to create topic for a Builtin IPC channel.

DDSC_LOG_IPC_CHANNEL_READER_CREATE_EC

```
#define DDSC_LOG_IPC_CHANNEL_READER_CREATE_EC (DDSC_LOG_BASE + 1305)
```

Failed to create reader for a Builtin IPC channel.

DDSC_LOG_IPC_CHANNEL_WRITER_CREATE_EC

```
#define DDSC_LOG_IPC_CHANNEL_WRITER_CREATE_EC (DDSC_LOG_BASE + 1306)
```

Failed to create writer for a Builtin IPC channel.

DDSC_LOG_IPC_CHANNEL_TYPE_PLUGIN_EC

```
#define DDSC_LOG_IPC_CHANNEL_TYPE_PLUGIN_EC (DDSC_LOG_BASE + 1307)
```

Failed to find type plugin for a Builtin IPC channel.

DDSC_LOG_IPC_CHANNEL_UNKNOWN_EC

```
#define DDSC_LOG_IPC_CHANNEL_UNKNOWN_EC (DDSC_LOG_BASE + 1308)
```

Unknown builtin IPC channel requested.

DDSC_LOG_IPC_UPDATE_QOS_FAILED_EC

```
#define DDSC_LOG_IPC_UPDATE_QOS_FAILED_EC (DDSC_LOG_BASE + 1309)
```

Failed to update DomainParticipantQos to account for built-in IPC endpoints.

DDSC_LOG_IPC_CHANNEL_ADD_ANON_ROUTE_EC

```
#define DDSC_LOG_IPC_CHANNEL_ADD_ANON_ROUTE_EC (DDSC_LOG_BASE + 1310)
```

Failed to add an anonymous route to a built-in DataWriter or DataReader.

DDSC_LOG_IPC_CHANNEL_DELETE_ANON_ROUTE_EC

```
#define DDSC_LOG_IPC_CHANNEL_DELETE_ANON_ROUTE_EC (DDSC_LOG_BASE + 1311)
```

Failed to delete an anonymous route to a built-in DataWriter or DataReader.

DDSC_LOG_IPC_CHANNEL_ADD_PEER_EC

```
#define DDSC_LOG_IPC_CHANNEL_ADD_PEER_EC (DDSC_LOG_BASE + 1312)
```

Failed to add an inter-participant channel peer.

DDSC_LOG_IPC_CHANNEL_REMOVE_PEER_EC

```
#define DDSC_LOG_IPC_CHANNEL_REMOVE_PEER_EC (DDSC_LOG_BASE + 1313)
```

Failed to remove an inter-participant channel peer.

DDSC_LOG_IPC_CHANNEL_RETURN_LOAN_EC

```
#define DDSC_LOG_IPC_CHANNEL_RETURN_LOAN_EC (DDSC_LOG_BASE + 1314)
```

Failed to return a loaned inter-participant sample.

DDSC_LOG_IPC_CHANNEL_PROCESS_SAMPLE_EC

```
#define DDSC_LOG_IPC_CHANNEL_PROCESS_SAMPLE_EC (DDSC_LOG_BASE + 1315)
```

Failed to process inter-participant message.

DDS_LOG_IPC_ASSERT_ROUTES_EC

```
#define DDS_LOG_IPC_ASSERT_ROUTES_EC (DDSC_LOG_BASE + 1316)
```

Failed to assert inter-participant routes.

DDSC_LOG_IPC_CHANNEL_WRITE_SAMPLE_EC

```
#define DDSC_LOG_IPC_CHANNEL_WRITE_SAMPLE_EC (DDSC_LOG_BASE + 1317)
```

Failed to write inter-participant message.

DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_AUTH_HANDSHAKE_FAILED_EC

```
#define DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_AUTH_HANDSHAKE_FAILED_EC (DDSC_LOG_BASE + 1318)
```

Failed to process handshake authentication.

DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_PARTICIPANT_INTERCEPTOR_STATE_FAILED_EC

```
#define DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_PARTICIPANT_INTERCEPTOR_STATE_FAILED_EC (DDSC_LOG_BASE + 1319)
```

Failed to process participant interceptor tokens.

DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_ENDPOINT_INTERCEPTOR_STATE_FAILED_EC

```
#define DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_ENDPOINT_INTERCEPTOR_STATE_FAILED_EC (DDSC_LOG_BASE + 1320)
```

Failed to process endpoint interceptor tokens.

DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_AUTH_REQUEST_FAILED_EC

```
#define DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_AUTH_REQUEST_FAILED_EC (DDSC_LOG_BASE + 1321)
```

Failed to process authentication request.

DDSC_LOG_IPC_CHANNEL_CONFIG_INITIALIZE_FAILED_EC

```
#define DDSC_LOG_IPC_CHANNEL_CONFIG_INITIALIZE_FAILED_EC (DDSC_LOG_BASE + 1322)
```

Failed to initialize inter-participant channel.

DDSC_LOG_IPC_CHANNEL_CONFIG_FINALIZE_FAILED_EC

```
#define DDSC_LOG_IPC_CHANNEL_CONFIG_FINALIZE_FAILED_EC (DDSC_LOG_BASE + 1323)
```

Failed to finalize inter-participant channel

DDSC_LOG_IPC_CHANNEL_CONFIG_SET_FAILED_EC

```
#define DDSC_LOG_IPC_CHANNEL_CONFIG_SET_FAILED_EC (DDSC_LOG_BASE + 1324)
```

Failed to get inter-participant channel configuration.

DDSC_LOG_IPC_CHANNEL_SET_LISTENERS_FAILED_EC

```
#define DDSC_LOG_IPC_CHANNEL_SET_LISTENERS_FAILED_EC (DDSC_LOG_BASE + 1325)
```

Failed to set inter-participant channel listeners.

DDSC_LOG_IPC_CHANNEL_ENABLE_FAILED_EC

```
#define DDSC_LOG_IPC_CHANNEL_ENABLE_FAILED_EC (DDSC_LOG_BASE + 1326)
```

Failed to enable inter-participant channel entities.

DDSC_LOG_IPC_CHANNEL_CREATE_FAILED_EC

```
#define DDSC_LOG_IPC_CHANNEL_CREATE_FAILED_EC (DDSC_LOG_BASE + 1327)
```

Failed to create inter-participant channel.

DDSC_LOG_TRUST_UNKNOWN_GMCLASSID_EC

```
#define DDSC_LOG_TRUST_UNKNOWN_GMCLASSID_EC (DDSC_LOG_BASE + 1401)
```

Unknown generic message class id.

DDSC_LOG_TRUST_UNKNOWN_SERVICEID_EC

```
#define DDSC_LOG_TRUST_UNKNOWN_SERVICEID_EC (DDSC_LOG_BASE + 1402)
```

Unknown builtin service id.

DDSC_LOG_TRUST_IGNORED_REMOTE_PARTICIPANT_EC

```
#define DDSC_LOG_TRUST_IGNORED_REMOTE_PARTICIPANT_EC (DDSC_LOG_BASE + 1403)
```

Ignored remote participant.

DDSC_LOG_TRUST_IGNORED_HANDSHAKE_MESSAGE_EC

```
#define DDSC_LOG_TRUST_IGNORED_HANDSHAKE_MESSAGE_EC (DDSC_LOG_BASE + 1404)
```

Ignored handshake message from remote participant.

DDSC_LOG_TRUST_UNEXPECTED_VALIDATION_RESULT_EC

```
#define DDSC_LOG_TRUST_UNEXPECTED_VALIDATION_RESULT_EC (DDSC_LOG_BASE + 1405)
```

Unexpected validation result.

DDSC_LOG_TRUST_UNAUTHORIZED_REMOTE_PARTICIPANT_EC

```
#define DDSC_LOG_TRUST_UNAUTHORIZED_REMOTE_PARTICIPANT_EC (DDSC_LOG_BASE + 1406)
```

A remote participant failed the authorization process.

DDSC_LOG_TRUST_FAILED_VALIDATE_LOCAL_IDENTITY_EC

```
#define DDSC_LOG_TRUST_FAILED_VALIDATE_LOCAL_IDENTITY_EC (DDSC_LOG_BASE + 1407)
```

Failed to validate local identity.

DDSC_LOG_TRUST_INVALID_IDENTITY_HANDLE_EC

```
#define DDSC_LOG_TRUST_INVALID_IDENTITY_HANDLE_EC (DDSC_LOG_BASE + 1408)
```

Invalid local trust identity.

DDSC_LOG_TRUST_GET_IDENTITY_TOKEN_FAILED_EC

```
#define DDSC_LOG_TRUST_GET_IDENTITY_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1409)
```

Failed to get identity token.

DDSC_LOG_TRUST_INVALID_PERMISSIONS_HANDLE_EC

```
#define DDSC_LOG_TRUST_INVALID_PERMISSIONS_HANDLE_EC (DDSC_LOG_BASE + 1410)
```

Invalid local trust permissions.

DDSC_LOG_TRUST_INVALID_PARTICIPANT_GUID_EC

```
#define DDSC_LOG_TRUST_INVALID_PARTICIPANT_GUID_EC (DDSC_LOG_BASE + 1411)
```

Invalid DomainParticipant GUID configuration.

DDSC_LOG_TRUST_GET_PARTICIPANT_TRUST_ATTRIBUTES_FAILED_EC

```
#define DDSC_LOG_TRUST_GET_PARTICIPANT_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1412)
```

Failed to get DomainParticipant trust attributes.

DDSC_LOG_TRUST_CHECK_CREATE_PARTICIPANT_FAILED_EC

```
#define DDSC_LOG_TRUST_CHECK_CREATE_PARTICIPANT_FAILED_EC (DDSC_LOG_BASE + 1413)
```

Failed to create access control DomainParticipant.

DDSC_LOG_TRUST_GET_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC

```
#define DDSC_LOG_TRUST_GET_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1414)
```

Failed to get permissions credential token.

DDSC_LOG_TRUST_GET_PERMISSIONS_TOKEN_FAILED_EC

```
#define DDSC_LOG_TRUST_GET_PERMISSIONS_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1415)
```

Failed to get permissions token.

DDSC_LOG_TRUST_INVALID_INTERCEPTOR_HANDLE_EC

```
#define DDSC_LOG_TRUST_INVALID_INTERCEPTOR_HANDLE_EC (DDSC_LOG_BASE + 1416)
```

Invalid interceptor handle.

DDSC_LOG_TRUST_REGISTER_LOCAL_PARTICIPANT_FAILED_EC

```
#define DDSC_LOG_TRUST_REGISTER_LOCAL_PARTICIPANT_FAILED_EC (DDSC_LOG_BASE + 1417)
```

Failed to register local DomainParticipant.

DDSC_LOG_TRUST_RETURN_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC

```
#define DDSC_LOG_TRUST_RETURN_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1418)
```

Failed to return permissions credential token.

DDSC_LOG_TRUST_RETURN_PERMISSIONS_TOKEN_FAILED_EC

```
#define DDSC_LOG_TRUST_RETURN_PERMISSIONS_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1419)
```

Failed to return permissions token.

DDSC_LOG_TRUST_RETURN_IDENTITY_TOKEN_FAILED_EC

```
#define DDSC_LOG_TRUST_RETURN_IDENTITY_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1420)
```

Failed to return identity token.

DDSC_LOG_TRUST_UNREGISTER_PARTICIPANT_FAILED_EC

```
#define DDSC_LOG_TRUST_UNREGISTER_PARTICIPANT_FAILED_EC (DDSC_LOG_BASE + 1421)
```

Failed to unregister local participant.

DDSC_LOG_TRUST_RETURN_PERMISSIONS_HANDLE_FAILED_EC

```
#define DDSC_LOG_TRUST_RETURN_PERMISSIONS_HANDLE_FAILED_EC (DDSC_LOG_BASE + 1422)
```

Failed to return permissions handle.

DDSC_LOG_TRUST_RETURN_IDENTITY_HANDLE_FAILED_EC

```
#define DDSC_LOG_TRUST_RETURN_IDENTITY_HANDLE_FAILED_EC (DDSC_LOG_BASE + 1423)
```

Failed to return identity handle.

DDSC_LOG_TRUST_RETURN_SHAREDSECRET_HANDLE_FAILED_EC

```
#define DDSC_LOG_TRUST_RETURN_SHAREDSECRET_HANDLE_FAILED_EC (DDSC_LOG_BASE + 1424)
```

Failed to return shared secret handle.

DDSC_LOG_TRUST_GET_AUTHENTICATED_PEER_CREDENTIAL_TOKEN_FAILED_EC

```
#define DDSC_LOG_TRUST_GET_AUTHENTICATED_PEER_CREDENTIAL_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1425)
```

Failed to get authenticated peer credential token.

DDSC_LOG_TRUST_CHECK_REMOTE_PARTICIPANT_FAILED_EC

```
#define DDSC_LOG_TRUST_CHECK_REMOTE_PARTICIPANT_FAILED_EC (DDSC_LOG_BASE + 1426)
```

Failed to check access control remote DomainParticipant.

DDSC_LOG_TRUST_GET_SHARED_SECRET_FAILED_EC

```
#define DDSC_LOG_TRUST_GET_SHARED_SECRET_FAILED_EC (DDSC_LOG_BASE + 1427)
```

Failed to get shared secret.

DDSC_LOG_TRUST_PREPARE_LOCAL_PARTICIPANT_STATE_FAILED_EC

```
#define DDSC_LOG_TRUST_PREPARE_LOCAL_PARTICIPANT_STATE_FAILED_EC (DDSC_LOG_BASE + 1428)
```

Failed to validate local DomainParticipant trust.

DDSC_LOG_TRUST_DELETE_TRUST_PLUGINS_EC

```
#define DDSC_LOG_TRUST_DELETE_TRUST_PLUGINS_EC (DDSC_LOG_BASE + 1429)
```

Failed to delete trust plugins.

DDSC_LOG_TRUST_SET_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC

```
#define DDSC_LOG_TRUST_SET_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1430)
```

Failed to set permissions credential and token.

DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_PARTICIPANT_FAILED_EC

```
#define DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_PARTICIPANT_FAILED_EC (DDSC_LOG_BASE + 1431)
```

Failed to register matched remote DomainParticipant.

DDSC_LOG_TRUST_GET_TOPIC_TRUST_ATTRIBUTES_FAILED_EC

```
#define DDSC_LOG_TRUST_GET_TOPIC_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1432)
```

Failed to get topic trust attributes.

DDSC_LOG_TRUST_CHECK_CREATE_DATAWRITER_FAILED_EC

```
#define DDSC_LOG_TRUST_CHECK_CREATE_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1433)
```

Failed to create trust check DataWriter.

DDSC_LOG_TRUST_RETURN_DATAWRITER_TRUST_ATTRIBUTES_FAILED_EC

```
#define DDSC_LOG_TRUST_RETURN_DATAWRITER_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1434)
```

Failed to return DataWriter trust attributes.

DDSC_LOG_TRUST_GET_DATAWRITER_TRUST_ATTRIBUTES_FAILED_EC

```
#define DDSC_LOG_TRUST_GET_DATAWRITER_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1435)
```

Failed to get DataWriter trust attributes.

DDSC_LOG_TRUST_GET_DATAREADER_TRUST_ATTRIBUTES_FAILED_EC

```
#define DDSC_LOG_TRUST_GET_DATAREADER_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1436)
```

Failed to get DataReader trust attributes.

DDSC_LOG_TRUST_CHECK_CREATE_DATAREADER_FAILED_EC

```
#define DDSC_LOG_TRUST_CHECK_CREATE_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1437)
```

Failed to create trust check DataReader.

DDSC_LOG_TRUST_BEGIN_HANDSHAKE_REPLY_EC

```
#define DDSC_LOG_TRUST_BEGIN_HANDSHAKE_REPLY_EC (DDSC_LOG_BASE + 1438)
```

Handshake begin reply failed.

DDSC_LOG_TRUST_CREATE_LOCAL_PARTICIPANT_INTERCEPTOR_TOKENS_EC

```
#define DDSC_LOG_TRUST_CREATE_LOCAL_PARTICIPANT_INTERCEPTOR_TOKENS_EC (DDSC_LOG_BASE + 1439)
```

Failed to create local DomainParticipant interceptor tokens.

DDSC_LOG_TRUST_CREATE_LOCAL_DW_TOKEN_EC

```
#define DDSC_LOG_TRUST_CREATE_LOCAL_DW_TOKEN_EC (DDSC_LOG_BASE + 1440)
```

Failed to create local DataWriter interceptor token.

DDSC_LOG_TRUST_CREATE_LOCAL_DR_TOKEN_EC

```
#define DDSC_LOG_TRUST_CREATE_LOCAL_DR_TOKEN_EC (DDSC_LOG_BASE + 1441)
```

Failed to create local DataReader interceptor token.

DDSC_LOG_TRUST_RETURN_HANDSHAKE_HANDLE_FAILED_EC

```
#define DDSC_LOG_TRUST_RETURN_HANDSHAKE_HANDLE_FAILED_EC (DDSC_LOG_BASE + 1442)
```

Failed to return handshake handle.

DDSC_LOG_TRUST_SET_REMOTE_PARTICIPANT_INTERCEPTOR_TOKENS_FAILED_EC

```
#define DDSC_LOG_TRUST_SET_REMOTE_PARTICIPANT_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1443)
```

Failed to set remote DomainParticipant interceptor tokens.

DDSC_LOG_TRUST_SET_REMOTE_DATAREADER_INTERCEPTOR_TOKENS_FAILED_EC

```
#define DDSC_LOG_TRUST_SET_REMOTE_DATAREADER_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1444)
```

Failed to set remote DataReader interceptor tokens.

DDSC_LOG_TRUST_SET_REMOTE_DATAWRITER_INTERCEPTOR_TOKENS_FAILED_EC

```
#define DDSC_LOG_TRUST_SET_REMOTE_DATAWRITER_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1445)
```

Failed to set remote DataWriter interceptor tokens.

DDSC_LOG_TRUST_REGISTER_LOCAL_DATAREADER_FAILED_EC

```
#define DDSC_LOG_TRUST_REGISTER_LOCAL_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1446)
```

Failed to register local DataReader.

DDSC_LOG_TRUST_REGISTER_LOCAL_DATAWRITER_FAILED_EC

```
#define DDSC_LOG_TRUST_REGISTER_LOCAL_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1447)
```

Failed to register local DataWriter.

DDSC_LOG_TRUST_CHECK_REMOTE_DATAWRITER_FAILED_EC

```
#define DDSC_LOG_TRUST_CHECK_REMOTE_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1448)
```

Failed to check remote DataWriter.

DDSC_LOG_TRUST_CHECK_REMOTE_DATAREADER_FAILED_EC

```
#define DDSC_LOG_TRUST_CHECK_REMOTE_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1449)
```

Failed to check remote DataReader.

DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_DATAWRITER_FAILED_EC

```
#define DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1450)
```

Failed to register matched remote DataWriter.

DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_DATAREADER_FAILED_EC

```
#define DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1451)
```

Failed to register matched remote DataReader.

DDSC_LOG_TRUST_UNREGISTER_DATAREADER_FAILED_EC

```
#define DDSC_LOG_TRUST_UNREGISTER_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1452)
```

Failed to unregister DataReader.

DDSC_LOG_TRUST_UNREGISTER_DATAWRITER_FAILED_EC

```
#define DDSC_LOG_TRUST_UNREGISTER_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1453)
```

Failed to unregister DataWriter.

DDSC_LOG_TRUST_VALIDATE_REMOTE_IDENTITY_EC

```
#define DDSC_LOG_TRUST_VALIDATE_REMOTE_IDENTITY_EC (DDSC_LOG_BASE + 1454)
```

Failed to validate remote identity.

DDSC_LOG_TRUST_BEGIN_HANDSHAKE_REQUEST_EC

```
#define DDSC_LOG_TRUST_BEGIN_HANDSHAKE_REQUEST_EC (DDSC_LOG_BASE + 1455)
```

Handshake begin request failed.

DDSC_LOG_TRUST_PROCESS_HANDSHAKE_EC

```
#define DDSC_LOG_TRUST_PROCESS_HANDSHAKE_EC (DDSC_LOG_BASE + 1456)
```

Handshake process failed.

DDSC_LOG_TRUST_CREATE_TRUST_PLUGINS_FAILED_EC

```
#define DDSC_LOG_TRUST_CREATE_TRUST_PLUGINS_FAILED_EC (DDSC_LOG_BASE + 1457)
```

Failed to create trust plugins.

DDSC_LOG_TRUST_PARSE_PROPERTY_FAILED_EC

```
#define DDSC_LOG_TRUST_PARSE_PROPERTY_FAILED_EC (DDSC_LOG_BASE + 1458)
```

Failed to parse trust property.

DDSC_LOG_TRUST_AFTER_REMOTE_PARTICIPANT_READY_FAILED_EC

```
#define DDSC_LOG_TRUST_AFTER_REMOTE_PARTICIPANT_READY_FAILED_EC (DDSC_LOG_BASE + 1459)
```

Error after remote trust DomainParticipant is ready.

DDSC_LOG_TRUST_REMOVE_REMOTE_PARTICIPANT_EC

```
#define DDSC_LOG_TRUST_REMOVE_REMOTE_PARTICIPANT_EC (DDSC_LOG_BASE + 1460)
```

Failed to remove remote DomainParticipant.

DDS_LOG_TRUST_ASSERT_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC

```
#define DDS_LOG_TRUST_ASSERT_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC (DDSC_LOG_BASE + 1461)
```

Failed to assert DomainParticipant generic message.

DDSC_LOG_TRUST_SEND_HANDSHAKE_EC

```
#define DDSC_LOG_TRUST_SEND_HANDSHAKE_EC (DDSC_LOG_BASE + 1462)
```

Failed to send handshake message.

DDSC_LOG_VALIDATE_REMOTE_PARTICIPANT_TRUST_FAILED_EC

```
#define DDSC_LOG_VALIDATE_REMOTE_PARTICIPANT_TRUST_FAILED_EC (DDSC_LOG_BASE + 1463)
```

Failed to validate trust remote DomainParticipant.

DDSC_LOG_TRUST_BIND_REMOTE_DATAREADER_FAILED_EC

```
#define DDSC_LOG_TRUST_BIND_REMOTE_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1464)
```

Failed to get remote DataReader bind properties.

DDSC_LOG_TRUST_REGISTER_REMOTE_DATAREADER_FAILED_EC

```
#define DDSC_LOG_TRUST_REGISTER_REMOTE_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1465)
```

Failed to register remote DataReader.

DDSC_LOG_TRUST_BIND_REMOTE_DATAWRITER_FAILED_EC

```
#define DDSC_LOG_TRUST_BIND_REMOTE_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1466)
```

Failed to get remote DataWriter bind properties.

DDSC_LOG_TRUST_REGISTER_REMOTE_DATAWRITER_FAILED_EC

```
#define DDSC_LOG_TRUST_REGISTER_REMOTE_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1467)
```

Failed to register remote DataWriter.

DDSC_LOG_TRUST_VALIDATE_REMOTE_DATAWRITER_TRUST_FAILED_EC

```
#define DDSC_LOG_TRUST_VALIDATE_REMOTE_DATAWRITER_TRUST_FAILED_EC (DDSC_LOG_BASE + 1468)
```

Failed to validate remote DataWriter.

DDSC_LOG_TRUST_VALIDATE_REMOTE_DATAREADER_TRUST_FAILED_EC

```
#define DDSC_LOG_TRUST_VALIDATE_REMOTE_DATAREADER_TRUST_FAILED_EC (DDSC_LOG_BASE + 1469)
```

Failed to validate remote DataReader.

DDSC_LOG_TRUST_VALIDATE_LOCAL_DATAWRITER_TRUST_FAILED_EC

```
#define DDSC_LOG_TRUST_VALIDATE_LOCAL_DATAWRITER_TRUST_FAILED_EC (DDSC_LOG_BASE + 1470)
```

Failed to validate local DataWriter.

DDSC_TRUST_INVALID_PARTICIPANT_GENERIC_MESSAGE_EC

```
#define DDSC_TRUST_INVALID_PARTICIPANT_GENERIC_MESSAGE_EC (DDSC_LOG_BASE + 1471)
```

Invalid DomainParticipant trust generic message.

DDSC_LOG_TRUST_PREPARE_AUTH_REQUEST_EC

```
#define DDSC_LOG_TRUST_PREPARE_AUTH_REQUEST_EC (DDSC_LOG_BASE + 1472)
```

Failed to prepare authentication request.

DDSC_LOG_TRUST_PREPARE_AUTH_HANDSHAKE_MESSAGE_EC

```
#define DDSC_LOG_TRUST_PREPARE_AUTH_HANDSHAKE_MESSAGE_EC (DDSC_LOG_BASE + 1473)
```

Failed to prepare authentication handshake message.

DDSC_LOG_TRUST_PREPARE_ENDPOINT_INTERCEPTOR_MESSAGE_EC

```
#define DDSC_LOG_TRUST_PREPARE_ENDPOINT_INTERCEPTOR_MESSAGE_EC (DDSC_LOG_BASE + 1474)
```

Failed to prepare endpoint interceptor message.

DDSC_LOG_TRUST_INVALID_GENERIC_MESSAGE_REPLY_EC

```
#define DDSC_LOG_TRUST_INVALID_GENERIC_MESSAGE_REPLY_EC (DDSC_LOG_BASE + 1475)
```

Invalid trust generic message reply.

DDSC_LOG_FINALIZE_TRUST_FAILED_EC

```
#define DDSC_LOG_FINALIZE_TRUST_FAILED_EC (DDSC_LOG_BASE + 1476)
```

Failed to finalize trust.

DDSC_LOG_TRUST_DATA HOLDER_COPY_FAILED_EC

```
#define DDSC_LOG_TRUST_DATA_HOLDER_COPY_FAILED_EC (DDSC_LOG_BASE + 1477)
```

Failed to copy trust data holder.

DDSC_LOG_TRUST_INVALID_SAMPLE_MAX_SERIALIZED_SIZE_EC

```
#define DDSC_LOG_TRUST_INVALID_SAMPLE_MAX_SERIALIZED_SIZE_EC (DDSC_LOG_BASE + 1478)
```

Invalid sample serialized size.

DDSC_LOG_TRUST_ALLOC_FAILED_EC

```
#define DDSC_LOG_TRUST_ALLOC_FAILED_EC (DDSC_LOG_BASE + 1479)
```

Failed to allocate memory.

DDSC_LOG_TRUST_SERIALIZE_DATA_FAILED_EC

```
#define DDSC_LOG_TRUST_SERIALIZE_DATA_FAILED_EC (DDSC_LOG_BASE + 1480)
```

Failed to serialize trust data.

DDSC_LOG_TRUST_INITIALIZE_PARTICIPANT_TRUST_FAILED_EC

```
#define DDSC_LOG_TRUST_INITIALIZE_PARTICIPANT_TRUST_FAILED_EC (DDSC_LOG_BASE + 1481)
```

Failed to initialize trust.

DDSC_LOG_TRUST_INITIALIZE_PARTICIPANT_BUILTIN_DATA_FAILED_EC

```
#define DDSC_LOG_TRUST_INITIALIZE_PARTICIPANT_BUILTIN_DATA_FAILED_EC (DDSC_LOG_BASE + 1482)
```

Failed to initialize DomainParticipant trust data.

DDSC_LOG_TRUST_INVALID_PARTICIPANT_GENERIC_MESSAGE_EC

```
#define DDSC_LOG_TRUST_INVALID_PARTICIPANT_GENERIC_MESSAGE_EC (DDSC_LOG_BASE + 1483)
```

Invalid DomainParticipant trust generic message.

DDSC_LOG_TRUST_ADVANCE_SN_FAILED_EC

```
#define DDSC_LOG_TRUST_ADVANCE_SN_FAILED_EC (DDSC_LOG_BASE + 1484)
```

Failed to advance trust sequence number.

DDSC_LOG_TRUST_WRITE_SAMPLE_FAILED_EC

```
#define DDSC_LOG_TRUST_WRITE_SAMPLE_FAILED_EC (DDSC_LOG_BASE + 1485)
```

Failed to write sample.

DDSC_LOG_TRUST_RESEND_HANDSHAKE_IN_PROGRESS_EC

```
#define DDSC_LOG_TRUST_RESEND_HANDSHAKE_IN_PROGRESS_EC (DDSC_LOG_BASE + 1486)
```

Failed to resend handshake because it is already in progress.

DDSC_LOG_TRUST_RESEND_HANDSHAKE_STOP_FAILED_EC

```
#define DDSC_LOG_TRUST_RESEND_HANDSHAKE_STOP_FAILED_EC (DDSC_LOG_BASE + 1487)
```

Failed to stop periodic handshake.

DDSC_LOG_TRUST_RELEASE_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC

```
#define DDSC_LOG_TRUST_RELEASE_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC (DDSC_LOG_BASE + 1488)
```

Failed to release DomainParticipant trust generic message.

DDSC_LOG_TRUST_RESEND_HANDSHAKE_MESSAGE_START_FAILED_EC

```
#define DDSC_LOG_TRUST_RESEND_HANDSHAKE_MESSAGE_START_FAILED_EC (DDSC_LOG_BASE + 1489)
```

Failed to start periodic handshake.

DDSC_LOG_TRUST_UNEXPECTED_REMOTE_PARTICIPANT_STATUS_EC

```
#define DDSC_LOG_TRUST_UNEXPECTED_REMOTE_PARTICIPANT_STATUS_EC (DDSC_LOG_BASE + 1490)
```

Invalid remote DomainParticipant trust status.

DDSC_LOG_TRUST_SEND_PARTICIPANT_INTERCEPTOR_STATE_FAILED_EC

```
#define DDSC_LOG_TRUST_SEND_PARTICIPANT_INTERCEPTOR_STATE_FAILED_EC (DDSC_LOG_BASE + 1491)
```

Failed to send DomainParticipant interceptor state.

DDSC_LOG_TRUST_PARTICIPANT_AUTHENTICATION_FAILED_EC

```
#define DDSC_LOG_TRUST_PARTICIPANT_AUTHENTICATION_FAILED_EC (DDSC_LOG_BASE + 1492)
```

DomainParticipant authentication failed.

DDSC_LOG_TRUST_ASSERT_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC

```
#define DDSC_LOG_TRUST_ASSERT_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC (DDSC_LOG_BASE + 1493)
```

Failed to assert DomainParticipant generic message.

DDSC_LOG_TRUST_SEND_HANDSHAKE_MESSAGE_FAILED_EC

```
#define DDSC_LOG_TRUST_SEND_HANDSHAKE_MESSAGE_FAILED_EC (DDSC_LOG_BASE + 1494)
```

Failed to send handshake message.

DDSC_LOG_TRUST_INVALID_INTERCEPTOR_TOKENS_EC

```
#define DDSC_LOG_TRUST_INVALID_INTERCEPTOR_TOKENS_EC (DDSC_LOG_BASE + 1495)
```

Invalid interceptor tokens.

DDSC_LOG_TRUST_INVALID_INTERCEPTOR_TOKENS_DESTINATION_EC

```
#define DDSC_LOG_TRUST_INVALID_INTERCEPTOR_TOKENS_DESTINATION_EC (DDSC_LOG_BASE + 1496)
```

Invalid interceptor token destination.

DDSC_LOG_TRUST_ENDPOINT_INTERNAL_ATTRIBUTES_CREATE_FAILED_EC

```
#define DDSC_LOG_TRUST_ENDPOINT_INTERNAL_ATTRIBUTES_CREATE_FAILED_EC (DDSC_LOG_BASE + 1497)
```

Failed to copy endpoint trust attributes.

DDSC_LOG_TRUST_SEND_ENDPOINT_INTERCEPTOR_STATE_FAILED_EC

```
#define DDSC_LOG_TRUST_SEND_ENDPOINT_INTERCEPTOR_STATE_FAILED_EC (DDSC_LOG_BASE + 1498)
```

Failed to send tokens to remote DomainParticipant.

DDSC_LOG_TRUST_CREATE_LOCAL_DATAWRITER_INTERCEPTOR_TOKENS_FAILED_EC

```
#define DDSC_LOG_TRUST_CREATE_LOCAL_DATAWRITER_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1499)
```

Failed to create tokens for the remote DataReader.

DDSC_LOG_TRUST_CREATE_LOCAL_DATAREADER_INTERCEPTOR_TOKENS_FAILED_EC

```
#define DDSC_LOG_TRUST_CREATE_LOCAL_DATAREADER_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1500)
```

Failed to create tokens for the remote DataWriter.

DDSC_LOG_TRUST_RETURN_INTERCEPTOR_TOKENS_FAILED_EC

```
#define DDSC_LOG_TRUST_RETURN_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1501)
```

Failed to return interceptor tokens.

DDSC_LOG_TRUST_RETURN_AUTHENTICATED_PEER_CREDENTIAL_TOKEN_FAILED_EC

```
#define DDSC_LOG_TRUST_RETURN_AUTHENTICATED_PEER_CREDENTIAL_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1502)
```

Failed to return authenticated peer credential token.

DDSC_LOG_TRUST_PROCESS_REMOTE_INTERCEPTOR_TOKENS_FAILED_EC

```
#define DDSC_LOG_TRUST_PROCESS_REMOTE_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1503)
```

Failed to process interceptor tokens.

DDSC_LOG_TRUST_UNREGISTER_REMOTE_DATAWRITER_FAILED_EC

```
#define DDSC_LOG_TRUST_UNREGISTER_REMOTE_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1504)
```

Failed to unregister remote DataWriter.

DDSC_LOG_TRUST_UNREGISTER_REMOTE_DATAREADER_FAILED_EC

```
#define DDSC_LOG_TRUST_UNREGISTER_REMOTE_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1505)
```

Failed to unregister remote DataReader.

DDSC_LOG_TRUST_INVALID_SHARED_SECRET_EC

```
#define DDSC_LOG_TRUST_INVALID_SHARED_SECRET_EC (DDSC_LOG_BASE + 1506)
```

Invalid shared secret.

DDSC_LOG_TRUST_PARTICIPANT_UNEXPECTED_AUTHENTICATION_ERROR_EC

```
#define DDSC_LOG_TRUST_PARTICIPANT_UNEXPECTED_AUTHENTICATION_ERROR_EC (DDSC_LOG_BASE + 1507)
```

Unexpected DomainParticipant authentication error.

DDSC_LOG_TRUST_PREPARE_LOCAL_DP_TOKENS_EC

```
#define DDSC_LOG_TRUST_PREPARE_LOCAL_DP_TOKENS_EC (DDSC_LOG_BASE + 1508)
```

Failed to prepare local DomainParticipant interceptor tokens.

DDSC_LOG_TRUST_ASSERT_PROPERTY_FAILED_EC

```
#define DDSC_LOG_TRUST_ASSERT_PROPERTY_FAILED_EC (DDSC_LOG_BASE + 1509)
```

Failed to assert trust property.

DDSC_LOG_TRUST_PARTICIPANT_INTERNAL_ATTRIBUTES_CREATE_FAILED_EC

```
#define DDSC_LOG_TRUST_PARTICIPANT_INTERNAL_ATTRIBUTES_CREATE_FAILED_EC (DDSC_LOG_BASE + 1510)
```

Failed to copy DomainParticipant trust attributes.

DDSC_LOG_TRUST_INVALID_PARTICIPANT_INTERNAL_ATTRIBUTES_EC

```
#define DDSC_LOG_TRUST_INVALID_PARTICIPANT_INTERNAL_ATTRIBUTES_EC (DDSC_LOG_BASE + 1511)
```

Invalid DomainParticipant trust attributes.

DDSC_LOG_TRUST_TRANSFORM_OUTGOING_SERIALIZED_PAYLOAD_FAILED_EC

```
#define DDSC_LOG_TRUST_TRANSFORM_OUTGOING_SERIALIZED_PAYLOAD_FAILED_EC (DDSC_LOG_BASE + 1512)
```

Failed to encode outgoing serialized payload.

DDSC_LOG_TRUST_TRANSFORM_INCOMING_SERIALIZED_PAYLOAD_FAILED_EC

```
#define DDSC_LOG_TRUST_TRANSFORM_INCOMING_SERIALIZED_PAYLOAD_FAILED_EC (DDSC_LOG_BASE + 1513)
```

Failed to decode incoming serialized payload.

DDSC_LOG_TRUST_FINALIZE_AC_PLUGIN_PROPERTIES_FAILED_EC

```
#define DDSC_LOG_TRUST_FINALIZE_AC_PLUGIN_PROPERTIES_FAILED_EC (DDSC_LOG_BASE + 1514)
```

Failed to finalize trust properties.

DDSC_LOG_TRUST_AFTER_REMOTE_PARTICIPANT_AUTHENTICATED_FAILED_EC

```
#define DDSC_LOG_TRUST_AFTER_REMOTE_PARTICIPANT_AUTHENTICATED_FAILED_EC (DDSC_LOG_BASE + 1515)
```

Error after remote trust DomainParticipant is authenticated.

DDSC_LOG_TRUST_CACHE_REMOTE_PARTICIPANT_SAMPLE_FAILED_EC

```
#define DDSC_LOG_TRUST_CACHE_REMOTE_PARTICIPANT_SAMPLE_FAILED_EC (DDSC_LOG_BASE + 1516)
```

Failed to copy remote DomainParticipant sample.

DDSC_LOG_TRUST_FINALIZE_AUTH_HANDSHAKE_STATE_FAILED_EC

```
#define DDSC_LOG_TRUST_FINALIZE_AUTH_HANDSHAKE_STATE_FAILED_EC (DDSC_LOG_BASE + 1517)
```

Failed to finalize authentication handshake.

DDSC_LOG_TRUST_FINALIZE_REMOTE_PARTICIPANT_TRUST_FAILED_EC

```
#define DDSC_LOG_TRUST_FINALIZE_REMOTE_PARTICIPANT_TRUST_FAILED_EC (DDSC_LOG_BASE + 1518)
```

Failed to finalize remote trust DomainParticipant record.

DDSC_LOG_TRUST_RESET_REMOTE_PARTICIPANT_TRUST_FAILED_EC

```
#define DDSC_LOG_TRUST_RESET_REMOTE_PARTICIPANT_TRUST_FAILED_EC (DDSC_LOG_BASE + 1519)
```

Failed to finalize remote trust DomainParticipant.

DDSC_LOG_TRUST_ASSERT_AUTH_HANDSHAKE_STATE_FAILED_EC

```
#define DDSC_LOG_TRUST_ASSERT_AUTH_HANDSHAKE_STATE_FAILED_EC (DDSC_LOG_BASE + 1520)
```

Failed to assert authentication handshake state.

DDSC_LOG_TRUST_INVALID_AUTH_HANDSHAKE_STATE_EC

```
#define DDSC_LOG_TRUST_INVALID_AUTH_HANDSHAKE_STATE_EC (DDSC_LOG_BASE + 1521)
```

Invalid authentication handshake state.

DDSC_LOG_TRUST_SEND_AUTH_REQUEST_FAILED_EC

```
#define DDSC_LOG_TRUST_SEND_AUTH_REQUEST_FAILED_EC (DDSC_LOG_BASE + 1522)
```

Failed to send authentication request.

DDSC_LOG_TRUST_GET_AUTH_HANDSHAKE_STATE_FAILED_EC

```
#define DDSC_LOG_TRUST_GET_AUTH_HANDSHAKE_STATE_FAILED_EC (DDSC_LOG_BASE + 1523)
```

Failed to get authentication handshake state.

DDSC_LOG_TRUST_SET_AUTH_HANDSHAKE_STATE_FAILED_EC

```
#define DDSC_LOG_TRUST_SET_AUTH_HANDSHAKE_STATE_FAILED_EC (DDSC_LOG_BASE + 1524)
```

Failed to set authentication handshake state.

DDSC_LOG_TRUST_REAUTH_REMOTE_PARTICIPANT_FAILED_EC

```
#define DDSC_LOG_TRUST_REAUTH_REMOTE_PARTICIPANT_FAILED_EC (DDSC_LOG_BASE + 1525)
```

Failed to reauthenticate a remote DomainParticipant.

DDSC_LOG_TRUST_RESEND_HANDSHAKE_MESSAGE_FAILED_EC

```
#define DDSC_LOG_TRUST_RESEND_HANDSHAKE_MESSAGE_FAILED_EC (DDSC_LOG_BASE + 1526)
```

Failed to resend handshake message.

DDSC_LOG_TRUST_PBUFS_MAX_EXCEEDED_EC

```
#define DDSC_LOG_TRUST_PBUFS_MAX_EXCEEDED_EC (DDSC_LOG_BASE + 1527)
```

Maximum number of buffers exceeded.

DDSC_LOG_TRUST_CREATE_TRUST_PLUGIN_EC

```
#define DDSC_LOG_TRUST_CREATE_TRUST_PLUGIN_EC (DDSC_LOG_BASE + 1528)
```

Failed to create trust plugin.

DDSC_LOG_TRUST_LOOKUP_FACTORY_FAILED_EC

```
#define DDSC_LOG_TRUST_LOOKUP_FACTORY_FAILED_EC (DDSC_LOG_BASE + 1529)
```

Failed to lookup trust plugin factory.

DDSC_LOG_NAME_LOOKUP_EC

```
#define DDSC_LOG_NAME_LOOKUP_EC (DDSC_LOG_BASE + 1700)
```

Fully qualified name does not contain substring "::".

DDSC_LOG_ENTITY_NAME_TOO_LONG_EC

```
#define DDSC_LOG_ENTITY_NAME_TOO_LONG_EC (DDSC_LOG_BASE + 1701)
```

Failed to get an entity because the specified name exceeds the maximum length of 255 octets (excluding the terminating NUL)

DDSC_LOG_MESSAGE_DATA_UNKNOWN_LIVELINESS_KIND_EC

```
#define DDSC_LOG_MESSAGE_DATA_UNKNOWN_LIVELINESS_KIND_EC (DDSC_LOG_BASE + 1800)
```

Unknown inter-participant message liveliness kind received.

DDSC_LOG_MESSAGE_DATA_WRONG_LIVELINESS_KIND_EC

```
#define DDSC_LOG_MESSAGE_DATA_WRONG_LIVELINESS_KIND_EC (DDSC_LOG_BASE + 1801)
```

Wrong inter-participant message liveliness kind received.

DDSC_LOG_UPDATE_LEASE_DURATION_TIMER_EC

```
#define DDSC_LOG_UPDATE_LEASE_DURATION_TIMER_EC (DDSC_LOG_BASE + 1802)
```

Failed to update DataWriter lease duration.

DDSC_LOG_PARTMESSAGE_WRITE_SAMPLE_FAILED_EC

```
#define DDSC_LOG_PARTMESSAGE_WRITE_SAMPLE_FAILED_EC (DDSC_LOG_BASE + 1803)
```

Failed to write inter-participant liveliness message.

DDSC_LOG_SEND_LIVELINESS_FAILED_EC

```
#define DDSC_LOG_SEND_LIVELINESS_FAILED_EC (DDSC_LOG_BASE + 1804)
```

Failed to send inter-participant liveliness message.

DDSC_LOG_UPDATE_LIVELINESS_FAILED_EC

```
#define DDSC_LOG_UPDATE_LIVELINESS_FAILED_EC (DDSC_LOG_BASE + 1805)
```

Failed to update DataWriter liveliness.

DDSC_LOG_REFRESH_LIVELINESS_FAILED_EC

```
#define DDSC_LOG_REFRESH_LIVELINESS_FAILED_EC (DDSC_LOG_BASE + 1806)
```

Failed to refresh endpoint liveliness.

DDSC_LOG_TRUST_INCOMPATIBLE_ENDPOINT_TRUST_ATTRIBUTES_EC

```
#define DDSC_LOG_TRUST_INCOMPATIBLE_ENDPOINT_TRUST_ATTRIBUTES_EC (DDSC_LOG_BASE + 1807)
```

Two security endpoints are incompatible.

DDSC_LOG_FLOW_CONTROLLER_CREATE_EC

```
#define DDSC_LOG_FLOW_CONTROLLER_CREATE_EC (DDSC_LOG_BASE + 1900)
```

Failed to create flow controller.

DDSC_LOG_FLOW_CONTROLLER_DELETE_EC

```
#define DDSC_LOG_FLOW_CONTROLLER_DELETE_EC (DDSC_LOG_BASE + 1901)
```

Failed to delete flow controller.

DDSC_LOG_XTYPES_LOANED_SAMPLE_STATE_ERROR_EC

```
#define DDSC_LOG_XTYPES_LOANED_SAMPLE_STATE_ERROR_EC (DDSC_LOG_BASE + 2000)
```

Extensible types sample state error.

DDSC_LOG_INTERPRETER_SUPPORT_UNEXPECTED_ERROR_EC

```
#define DDSC_LOG_INTERPRETER_SUPPORT_UNEXPECTED_ERROR_EC (DDSC_LOG_BASE + 2001)
```

Interpreter unexpected error.

DDSC_LOG_MEMORY_MANAGER_OUT_OF_RESOURCES_EC

```
#define DDSC_LOG_MEMORY_MANAGER_OUT_OF_RESOURCES_EC (DDSC_LOG_BASE + 2002)
```

Memory manager out of resources.

DDSC_LOG_MEMORY_MANAGER_NOT_OWNER_EC

```
#define DDSC_LOG_MEMORY_MANAGER_NOT_OWNER_EC (DDSC_LOG_BASE + 2003)
```

Memory manager is not the owner of the data passed in.

DDSC_LOG_MEMORY_MANAGER_DELETE_EC

```
#define DDSC_LOG_MEMORY_MANAGER_DELETE_EC (DDSC_LOG_BASE + 2004)
```

The wire manager was not able to delete a resource.

DDSC_LOG_BUFFER_MAX_EXCEEDED_EC

```
#define DDSC_LOG_BUFFER_MAX_EXCEEDED_EC (DDSC_LOG_BASE + 2101)
```

Maximum size of buffer exceeded.

DDSC_LOG_INVALID_BUFFER_LENGTH_EC

```
#define DDSC_LOG_INVALID_BUFFER_LENGTH_EC (DDSC_LOG_BASE + 2102)
```

Invalid buffer length.

DDSC_LOG_INCOMPATIBLE_PRESENTATION_QOS_EC

```
#define DDSC_LOG_INCOMPATIBLE_PRESENTATION_QOS_EC (DDSC_LOG_BASE + 2201)
```

Matching error because Presentation QoS is not compatible.

DDSC_LOG_INCOMPATIBLE_PARTITION_QOS_EC

```
#define DDSC_LOG_INCOMPATIBLE_PARTITION_QOS_EC (DDSC_LOG_BASE + 2202)
```

Two endpoints did not match because of incompatible partition QoS.

DDSC_LOG_CHECKSUM_REQUIRED_EC

```
#define DDSC_LOG_CHECKSUM_REQUIRED_EC (DDSC_LOG_BASE + 2301)
```

A participant requires checksum, but the discovered participant does not send a checksum.

DDSC_LOG_CHECKSUM_INCOMPATIBLE_EC

```
#define DDSC_LOG_CHECKSUM_INCOMPATIBLE_EC (DDSC_LOG_BASE + 2302)
```

A participant does not support checksum's sent by a discovered participant.

DDSC_LOG_CHECKSUM_INCONSISTENT_CLASS_EC

```
#define DDSC_LOG_CHECKSUM_INCONSISTENT_CLASS_EC (DDSC_LOG_BASE + 2303)
```

The checksum class id for a custom checksum does match.

DDSC_LOG_CHECKSUM_INCONSISTENT_COMPUTE_EC

```
#define DDSC_LOG_CHECKSUM_INCONSISTENT_COMPUTE_EC (DDSC_LOG_BASE + 2304)
```

The computed_crc_kind is inconsistent with what is allowed or supported.

DDSC_LOG_CHECKSUM_INCONSISTENT_ALLOW_EC

```
#define DDSC_LOG_CHECKSUM_INCONSISTENT_ALLOW_EC (DDSC_LOG_BASE + 2305)
```

The allowed_crc_mask is inconsistent with what is supported.

DDSC_LOG_FLOWCONTROLLER_INCONSISTENT_PROP_EC

```
#define DDSC_LOG_FLOWCONTROLLER_INCONSISTENT_PROP_EC (DDSC_LOG_BASE + 2306)
```

The token bucket properties for the flow controller are inconsistent.

DDSC_LOG_LOCK_EC

```
#define DDSC_LOG_LOCK_EC (DDSC_LOG_BASE + 2401)
```

Failed to take a lock.

DDSC_LOG_UNLOCK_EC

```
#define DDSC_LOG_UNLOCK_EC (DDSC_LOG_BASE + 2402)
```

Failed to give a lock.

DDSC_LOG_FILTER_PLUGIN_FACTORY_NOT_FOUND_EC

```
#define DDSC_LOG_FILTER_PLUGIN_FACTORY_NOT_FOUND_EC (DDSC_LOG_BASE + 2501)
```

Failed to find the filter plugin factory specified in a DomainParticipant's QoS.

DDSC_LOG_ENABLE_WRITER_FILTERING_EC

```
#define DDSC_LOG_ENABLE_WRITER_FILTERING_EC (DDSC_LOG_BASE + 2502)
```

A local writer failed to enable writer filtering for a reader.

DDSC_LOG_CREATE_WRITER_FILTER_EC

```
#define DDSC_LOG_CREATE_WRITER_FILTER_EC (DDSC_LOG_BASE + 2503)
```

A local writer failed to create its writer filter.

DDSC_LOG_EVALUATE_WRITER_FILTER_EC

```
#define DDSC_LOG_EVALUATE_WRITER_FILTER_EC (DDSC_LOG_BASE + 2504)
```

A local writer failed to evaluate a sample against the filters of its matched readers.

DDSC_LOG_APPLY_READER_FILTER_EC

```
#define DDSC_LOG_APPLY_READER_FILTER_EC (DDSC_LOG_BASE + 2505)
```

A local writer failed to check if a reader had filtered a sample.

DDSC_LOG_COMPILE_CONTENT_FILTER_EC

```
#define DDSC_LOG_COMPILE_CONTENT_FILTER_EC (DDSC_LOG_BASE + 2506)
```

Failed to compile the content filter configured in a DataReader's QoS.

DDSC_LOG_CONTENT_FILTERING_NOT_ENABLED_EC

```
#define DDSC_LOG_CONTENT_FILTERING_NOT_ENABLED_EC (DDSC_LOG_BASE + 2507)
```

Reader configured a content filter but content filtering is not enabled.

DDSC_LOG_EVALUATE_CONTENT_FILTER_EC

```
#define DDSC_LOG_EVALUATE_CONTENT_FILTER_EC (DDSC_LOG_BASE + 2508)
```

Failed to evaluate a sample against a reader's content filter.

DDSC_LOG_DESERIALIZE_CONTENT_FILTER_INFO_EC

```
#define DDSC_LOG_DESERIALIZE_CONTENT_FILTER_INFO_EC (DDSC_LOG_BASE + 2509)
```

Failed to deserialize the content filter info from the in-line QoS of a sample.

DDSC_LOG_CHANGE_CONTENT_FILTER_EC

```
#define DDSC_LOG_CHANGE_CONTENT_FILTER_EC (DDSC_LOG_BASE + 2510)
```

Failed to change the configured content filter of a DataReader.

DDSC_LOG_UPDATE_REMOTE_CONTENT_FILTER_EC

```
#define DDSC_LOG_UPDATE_REMOTE_CONTENT_FILTER_EC (DDSC_LOG_BASE + 2511)
```

Failed to update the content filter of a remote subscription.

DDSC_LOG_CREATE_FILTER_PLUGIN_EC

```
#define DDSC_LOG_CREATE_FILTER_PLUGIN_EC (DDSC_LOG_BASE + 2512)
```

DomainParticipant failed to create its filter plugin.

DDSC_LOG_FINALIZE_FILTER_PLUGIN_EC

```
#define DDSC_LOG_FINALIZE_FILTER_PLUGIN_EC (DDSC_LOG_BASE + 2513)
```

DomainParticipant failed to finalize its filter plugin.

DDSC_LOG_DESERIALIZE_REMOTE_CONTENT_FILTER_EC

```
#define DDSC_LOG_DESERIALIZE_REMOTE_CONTENT_FILTER_EC (DDSC_LOG_BASE + 2514)
```

Failed to deserialize the content filter property in the subscription built-in topic data for a remote subscription.

This warning is expected if the remote subscription has configured a content filter which exceeds the resource limits of the local participant. No writers on this participant will be able perform writer filtering for this remote subscription. The resource limits can be increased to allow the filter to be stored locally if writer filtering is desired.

6.20.10 ZeroCopy

Zero Copy V2. ModuleID = 14.

Macros

- #define ZCOPY_LOG_NOTIF_GUID_BUFFER_TOO_SMALL_EC (NETIO_ZCOPY_LOG_BASE + 1)
Failed to convert GUID to string representation.
- #define ZCOPY_LOG_NOTIF_ALREADY_CONNECTED_EC (NETIO_ZCOPY_LOG_BASE + 2)
Failed to connect because already connected.
- #define ZCOPY_LOG_NOTIF_NOT_CONNECTED_EC (NETIO_ZCOPY_LOG_BASE + 3)
Failed to perform operation because notif is not connected.
- #define ZCOPY_LOG_NOTIF_INCOMPATIBLE_VERSION_EC (NETIO_ZCOPY_LOG_BASE + 4)
Failed to connect because of incompatible ZCV2 version.
- #define ZCOPY_LOG_NOTIF_SLOT_ALREADY_ASSIGNED_EC (NETIO_ZCOPY_LOG_BASE + 5)
Failed to assign a slot because it is already assigned.
- #define ZCOPY_LOG_NOTIF_NO_FREE_SLOT_EC (NETIO_ZCOPY_LOG_BASE + 6)
Failed to assign a slot because there is no free slot.
- #define ZCOPY_LOG_NOTIF_INVALID_NUM_SLOTS_EC (NETIO_ZCOPY_LOG_BASE + 7)
Failed to connect because of invalid number of slots.
- #define ZCOPY_LOG_NOTIF_INVALID_PORT_NUMBER_EC (NETIO_ZCOPY_LOG_BASE + 8)
Failed to convert GUID to string representation because of an invalid port number.
- #define ZCOPY_LOG_NOTIF_OPEN_FAILED_EC (NETIO_ZCOPY_LOG_BASE + 9)
Failed to open notification interface.
- #define ZCOPY_LOG_NOTIF_DB_SELECT_RECORD_EC (NETIO_ZCOPY_LOG_BASE + 10)
Notification interface failed to select a database record.
- #define ZCOPY_LOG_NOTIF_DB_CREATE_RECORD_EC (NETIO_ZCOPY_LOG_BASE + 11)
Notification interface failed to create a database record.
- #define ZCOPY_LOG_NOTIF_DB_INSERT_RECORD_EC (NETIO_ZCOPY_LOG_BASE + 12)
Notification interface failed to insert a database record.
- #define ZCOPY_LOG_NOTIF_DB_REMOVE_RECORD_EC (NETIO_ZCOPY_LOG_BASE + 13)
Notification interface failed to remove a database record.
- #define ZCOPY_LOG_NOTIF_DB_DELETE_RECORD_EC (NETIO_ZCOPY_LOG_BASE + 14)
Notification interface failed to delete a database record.
- #define ZCOPY_LOG_NOTIF_DB_CURSOR_NEXT_EC (NETIO_ZCOPY_LOG_BASE + 15)
Notification interface failed to get the next database record from a cursor.

- #define ZCOPY_LOG_NOTIF_DB_LOCK_EC (NETIO_ZCOPY_LOG_BASE + 16)
Notification interface failed to lock or unlock its database.
- #define ZCOPY_LOG_NOTIF_DB_TABLE_CREATE_EC (NETIO_ZCOPY_LOG_BASE + 17)
Notification interface failed to create a table.
- #define ZCOPY_LOG_NOTIF_DB_TABLE_DELETE_EC (NETIO_ZCOPY_LOG_BASE + 18)
Notification interface failed to delete a table.
- #define ZCOPY_LOG_NOTIF_INVALID_PROPERTY_EC (NETIO_ZCOPY_LOG_BASE + 19)
Notification interface initialized with an invalid property.
- #define ZCOPY_LOG_NOTIF_SQ_OPEN_EC (NETIO_ZCOPY_LOG_BASE + 20)
Notification interface failed to open a shared queue reader.
- #define ZCOPY_LOG_NOTIF_SQ_CLOSE_EC (NETIO_ZCOPY_LOG_BASE + 21)
Notification interface failed to close a shared queue reader.
- #define ZCOPY_LOG_NOTIF_NO_MORE_NOTIFIERS_EC (NETIO_ZCOPY_LOG_BASE + 22)
Notification interface ran out of notifiers in its pool of notifiers.
- #define ZCOPY_LOG_NOTIF_NOTIFIER_CONNECT_EC (NETIO_ZCOPY_LOG_BASE + 23)
Notification interface failed to connect a notifier to a notifyee.
- #define ZCOPY_LOG_NOTIF_NOTIFIER_DISCONNECT_EC (NETIO_ZCOPY_LOG_BASE + 24)
Notification interface failed to disconnect a notifier from a notifyee.
- #define ZCOPY_LOG_NOTIF_NOTIFIER_NOTIFY_EC (NETIO_ZCOPY_LOG_BASE + 25)
Notification interface failed to notify with a notifier.
- #define ZCOPY_LOG_NOTIF_NOTIFIER_ALLOW_EC (NETIO_ZCOPY_LOG_BASE + 26)
Notification interface failed to allow a notifier to notify.
- #define ZCOPY_LOG_NOTIF_NOTIFIEE_DESTROY_EC (NETIO_ZCOPY_LOG_BASE + 27)
Notification interface failed to destroy a notifyee.
- #define ZCOPY_LOG_NOTIF_NOTIFIEE_RAISE_FLAG_EC (NETIO_ZCOPY_LOG_BASE + 28)
Notification interface failed to raise a flag.
- #define ZCOPY_LOG_NOTIF_NOTIFIEE_NEXT_EC (NETIO_ZCOPY_LOG_BASE + 29)
Notification interface failed to get next notification.
- #define ZCOPY_LOG_NOTIF_THREAD_CREATE_EC (NETIO_ZCOPY_LOG_BASE + 30)
Notification interface failed to create a receive thread.
- #define ZCOPY_LOG_NOTIF_THREAD_DESTROY_EC (NETIO_ZCOPY_LOG_BASE + 31)
Notification interface failed to destroy a receive thread.
- #define ZCOPY_LOG_NOTIF_INTF_RECEIVE_EC (NETIO_ZCOPY_LOG_BASE + 32)
Notification interface failed to forward a packet upstream to a DRI.
- #define ZCOPY_LOG_NOTIF_PORT_IN_USE_EC (NETIO_ZCOPY_LOG_BASE + 33)
Notification interface tried to release a port that was still in use.
- #define ZCOPY_LOG_NOTIF_ALLOC_EC (NETIO_ZCOPY_LOG_BASE + 34)
Notification interface factory failed to allocate memory.
- #define ZCOPY_LOG_NOTIF_INTF_INIT_EC (NETIO_ZCOPY_LOG_BASE + 35)
Notification interface factory failed to initialize a notification interface.
- #define ZCOPY_LOG_NOTIF_INTF_FINALIZE_EC (NETIO_ZCOPY_LOG_BASE + 36)
Notification interface factory failed to finalize a notification interface.
- #define ZCOPY_LOG_NOTIF_FACTORY_IN_USE_EC (NETIO_ZCOPY_LOG_BASE + 37)
Tried to finalize a notification interface factory which was still in use.
- #define ZCOPY_LOG_NOTIF_MECHANISM_CREATE_INST_EC (NETIO_ZCOPY_LOG_BASE + 38)
Notification mechanism failed to create instance.
- #define ZCOPY_LOG_NOTIF_MECHANISM_RESERVE_ADDR_EC (NETIO_ZCOPY_LOG_BASE + 39)

- *Notification mechanism failed to reserve an address.*
• #define ZCOPY_LOG_NOTIF_MECHANISM_RELEASE_ADDR_EC (NETIO_ZCOPY_LOG_BASE + 40)
- *Notification mechanism failed to release an address.*
• #define ZCOPY_LOG_NOTIF_MECHANISM_ADD_ROUTE_EC (NETIO_ZCOPY_LOG_BASE + 41)
- *Notification mechanism failed to add route.*
• #define ZCOPY_LOG_NOTIF_MECHANISM_DELETE_ROUTE_EC (NETIO_ZCOPY_LOG_BASE + 42)
- *Notification mechanism failed to delete route.*
• #define ZCOPY_LOG_NOTIF_MECHANISM_BIND_EC (NETIO_ZCOPY_LOG_BASE + 43)
- *Notification mechanism failed to bind.*
• #define ZCOPY_LOG_NOTIF_MECHANISM_UNBIND_EC (NETIO_ZCOPY_LOG_BASE + 44)
- *Notification mechanism failed to unbind.*
• #define ZCOPY_LOG_NOTIF_MECHANISM_SEND_EC (NETIO_ZCOPY_LOG_BASE + 45)
- *Notification mechanism failed to send.*
• #define ZCOPY_LOG_WH_REGISTER_FAILED_EC (NETIO_ZCOPY_LOG_BASE + 46)
- *Writer history wrap failed.*
• #define ZCOPY_LOG_NOTIF_TIMER_CREATE_EC (NETIO_ZCOPY_LOG_BASE + 47)
- *Timer creation failed.*
• #define ZCOPY_LOG_NOTIF_RECEIVE_ON_EVENT_EC (NETIO_ZCOPY_LOG_BASE + 48)
- *Receive on Writer match event failed.*
• #define ZCOPY_LOG_NOTIF_TIMEOUT_DELETE_EC (NETIO_ZCOPY_LOG_BASE + 49)
- *Failed to delete timeout.*
• #define ZCOPY_LOG_NOTIF_TIMEOUT_CREATE_EC (NETIO_ZCOPY_LOG_BASE + 50)
- *Timeout creation failed.*
• #define ZCOPY_LOG_ADD_ENTRY_EC (NETIO_ZCOPY_LOG_BASE + 51)
- *failed to add an indexer entry*
• #define ZCOPY_LOG_REMOVE_ENTRY_EC (NETIO_ZCOPY_LOG_BASE + 52)
- *failed to remove an indexer entry*
• #define ZCOPY_LOG_DELETE_POOL_EC (NETIO_ZCOPY_LOG_BASE + 53)
- *The memory manager was not able to delete a resource.*
• #define ZCOPY_LOG_SHARED_MEM_POOL_OWNER_DEAD_EC (NETIO_ZCOPY_LOG_BASE + 54)
- *A shared memory pool is no longer usable because a process died while holding the pool's lock.*

6.20.10.1 Detailed Description

Zero Copy V2. ModuleID = 14.

6.20.10.2 Macro Definition Documentation

ZCOPY_LOG_NOTIF_GUID_BUFFER_TOO_SMALL_EC

```
#define ZCOPY_LOG_NOTIF_GUID_BUFFER_TOO_SMALL_EC (NETIO_ZCOPY_LOG_BASE + 1)
```

Failed to convert GUID to string representation.

ZCOPY_LOG_NOTIF_ALREADY_CONNECTED_EC

```
#define ZCOPY_LOG_NOTIF_ALREADY_CONNECTED_EC (NETIO_ZCOPY_LOG_BASE + 2)
```

Failed to connect because already connected.

ZCOPY_LOG_NOTIF_NOT_CONNECTED_EC

```
#define ZCOPY_LOG_NOTIF_NOT_CONNECTED_EC (NETIO_ZCOPY_LOG_BASE + 3)
```

Failed to perform operation because notif is not connected.

ZCOPY_LOG_NOTIF_INCOMPATIBLE_VERSION_EC

```
#define ZCOPY_LOG_NOTIF_INCOMPATIBLE_VERSION_EC (NETIO_ZCOPY_LOG_BASE + 4)
```

Failed to connect because of incompatible ZCV2 version.

ZCOPY_LOG_NOTIF_SLOT_ALREADY_ASSIGNED_EC

```
#define ZCOPY_LOG_NOTIF_SLOT_ALREADY_ASSIGNED_EC (NETIO_ZCOPY_LOG_BASE + 5)
```

Failed to assign a slot because it is already assigned.

ZCOPY_LOG_NOTIF_NO_FREE_SLOT_EC

```
#define ZCOPY_LOG_NOTIF_NO_FREE_SLOT_EC (NETIO_ZCOPY_LOG_BASE + 6)
```

Failed to assign a slot because there is no free slot.

ZCOPY_LOG_NOTIF_INVALID_NUM_SLOTS_EC

```
#define ZCOPY_LOG_NOTIF_INVALID_NUM_SLOTS_EC (NETIO_ZCOPY_LOG_BASE + 7)
```

Failed to connect because of invalid number of slots.

ZCOPY_LOG_NOTIF_INVALID_PORT_NUMBER_EC

```
#define ZCOPY_LOG_NOTIF_INVALID_PORT_NUMBER_EC (NETIO_ZCOPY_LOG_BASE + 8)
```

Failed to convert GUID to string representation because of an invalid port number.

ZCOPY_LOG_NOTIF_OPEN_FAILED_EC

```
#define ZCOPY_LOG_NOTIF_OPEN_FAILED_EC (NETIO_ZCOPY_LOG_BASE + 9)
```

Failed to open notification interface.

ZCOPY_LOG_NOTIF_DB_SELECT_RECORD_EC

```
#define ZCOPY_LOG_NOTIF_DB_SELECT_RECORD_EC (NETIO_ZCOPY_LOG_BASE + 10)
```

Notification interface failed to select a database record.

ZCOPY_LOG_NOTIF_DB_CREATE_RECORD_EC

```
#define ZCOPY_LOG_NOTIF_DB_CREATE_RECORD_EC (NETIO_ZCOPY_LOG_BASE + 11)
```

Notification interface failed to create a database record.

ZCOPY_LOG_NOTIF_DB_INSERT_RECORD_EC

```
#define ZCOPY_LOG_NOTIF_DB_INSERT_RECORD_EC (NETIO_ZCOPY_LOG_BASE + 12)
```

Notification interface failed to insert a database record.

ZCOPY_LOG_NOTIF_DB_REMOVE_RECORD_EC

```
#define ZCOPY_LOG_NOTIF_DB_REMOVE_RECORD_EC (NETIO_ZCOPY_LOG_BASE + 13)
```

Notification interface failed to remove a database record.

ZCOPY_LOG_NOTIF_DB_DELETE_RECORD_EC

```
#define ZCOPY_LOG_NOTIF_DB_DELETE_RECORD_EC (NETIO_ZCOPY_LOG_BASE + 14)
```

Notification interface failed to delete a database record.

ZCOPY_LOG_NOTIF_DB_CURSOR_NEXT_EC

```
#define ZCOPY_LOG_NOTIF_DB_CURSOR_NEXT_EC (NETIO_ZCOPY_LOG_BASE + 15)
```

Notification interface failed to get the next database record from a cursor.

ZCOPY_LOG_NOTIF_DB_LOCK_EC

```
#define ZCOPY_LOG_NOTIF_DB_LOCK_EC (NETIO_ZCOPY_LOG_BASE + 16)
```

Notification interface failed to lock or unlock its database.

ZCOPY_LOG_NOTIF_DB_TABLE_CREATE_EC

```
#define ZCOPY_LOG_NOTIF_DB_TABLE_CREATE_EC (NETIO_ZCOPY_LOG_BASE + 17)
```

Notification interface failed to create a table.

ZCOPY_LOG_NOTIF_DB_TABLE_DELETE_EC

```
#define ZCOPY_LOG_NOTIF_DB_TABLE_DELETE_EC (NETIO_ZCOPY_LOG_BASE + 18)
```

Notification interface failed to delete a table.

ZCOPY_LOG_NOTIF_INVALID_PROPERTY_EC

```
#define ZCOPY_LOG_NOTIF_INVALID_PROPERTY_EC (NETIO_ZCOPY_LOG_BASE + 19)
```

Notification interface initialized with an invalid property.

ZCOPY_LOG_NOTIF_SQ_OPEN_EC

```
#define ZCOPY_LOG_NOTIF_SQ_OPEN_EC (NETIO_ZCOPY_LOG_BASE + 20)
```

Notification interface failed to open a shared queue reader.

ZCOPY_LOG_NOTIF_SQ_CLOSE_EC

```
#define ZCOPY_LOG_NOTIF_SQ_CLOSE_EC (NETIO_ZCOPY_LOG_BASE + 21)
```

Notification interface failed to close a shared queue reader.

ZCOPY_LOG_NOTIF_NO_MORE_NOTIFIERS_EC

```
#define ZCOPY_LOG_NOTIF_NO_MORE_NOTIFIERS_EC (NETIO_ZCOPY_LOG_BASE + 22)
```

Notification interface ran out of notifiers in its pool of notifiers.

ZCOPY_LOG_NOTIF_NOTIFIER_CONNECT_EC

```
#define ZCOPY_LOG_NOTIF_NOTIFIER_CONNECT_EC (NETIO_ZCOPY_LOG_BASE + 23)
```

Notification interface failed to connect a notifier to a notifiee.

ZCOPY_LOG_NOTIF_NOTIFIER_DISCONNECT_EC

```
#define ZCOPY_LOG_NOTIF_NOTIFIER_DISCONNECT_EC (NETIO_ZCOPY_LOG_BASE + 24)
```

Notification interface failed to disconnect a notifier from a notifiee.

ZCOPY_LOG_NOTIF_NOTIFIER_NOTIFY_EC

```
#define ZCOPY_LOG_NOTIF_NOTIFIER_NOTIFY_EC (NETIO_ZCOPY_LOG_BASE + 25)
```

Notification interface failed to notify with a notifier.

ZCOPY_LOG_NOTIF_NOTIFIER_ALLOW_EC

```
#define ZCOPY_LOG_NOTIF_NOTIFIER_ALLOW_EC (NETIO_ZCOPY_LOG_BASE + 26)
```

Notification interface failed to allow a notifier to notify.

ZCOPY_LOG_NOTIF_NOTIFIEE_DESTROY_EC

```
#define ZCOPY_LOG_NOTIF_NOTIFIEE_DESTROY_EC (NETIO_ZCOPY_LOG_BASE + 27)
```

Notification interface failed to destroy a notifiee.

ZCOPY_LOG_NOTIF_NOTIFIEE_RAISE_FLAG_EC

```
#define ZCOPY_LOG_NOTIF_NOTIFIEE_RAISE_FLAG_EC (NETIO_ZCOPY_LOG_BASE + 28)
```

Notification interface failed to raise a flag.

ZCOPY_LOG_NOTIF_NOTIFIEE_NEXT_EC

```
#define ZCOPY_LOG_NOTIF_NOTIFIEE_NEXT_EC (NETIO_ZCOPY_LOG_BASE + 29)
```

Notification interface failed to get next notification.

ZCOPY_LOG_NOTIF_THREAD_CREATE_EC

```
#define ZCOPY_LOG_NOTIF_THREAD_CREATE_EC (NETIO_ZCOPY_LOG_BASE + 30)
```

Notification interface failed to create a receive thread.

ZCOPY_LOG_NOTIF_THREAD_DESTROY_EC

```
#define ZCOPY_LOG_NOTIF_THREAD_DESTROY_EC (NETIO_ZCOPY_LOG_BASE + 31)
```

Notification interface failed to destroy a receive thread.

ZCOPY_LOG_NOTIF_INTF_RECEIVE_EC

```
#define ZCOPY_LOG_NOTIF_INTF_RECEIVE_EC (NETIO_ZCOPY_LOG_BASE + 32)
```

Notification interface failed to forward a packet upstream to a DRI.

ZCOPY_LOG_NOTIF_PORT_IN_USE_EC

```
#define ZCOPY_LOG_NOTIF_PORT_IN_USE_EC (NETIO_ZCOPY_LOG_BASE + 33)
```

Notification interface tried to release a port that was still in use.

ZCOPY_LOG_NOTIF_ALLOC_EC

```
#define ZCOPY_LOG_NOTIF_ALLOC_EC (NETIO_ZCOPY_LOG_BASE + 34)
```

Notification interface factory failed to allocate memory.

ZCOPY_LOG_NOTIF_INTF_INIT_EC

```
#define ZCOPY_LOG_NOTIF_INTF_INIT_EC (NETIO_ZCOPY_LOG_BASE + 35)
```

Notification interface factory failed to initialize a notification interface.

ZCOPY_LOG_NOTIF_INTF_FINALIZE_EC

```
#define ZCOPY_LOG_NOTIF_INTF_FINALIZE_EC (NETIO_ZCOPY_LOG_BASE + 36)
```

Notification interface factory failed to finalize a notification interface.

ZCOPY_LOG_NOTIF_FACTORY_IN_USE_EC

```
#define ZCOPY_LOG_NOTIF_FACTORY_IN_USE_EC (NETIO_ZCOPY_LOG_BASE + 37)
```

Tried to finalize a notification interface factory which was still in use.

ZCOPY_LOG_NOTIF_MECHANISM_CREATE_INST_EC

```
#define ZCOPY_LOG_NOTIF_MECHANISM_CREATE_INST_EC (NETIO_ZCOPY_LOG_BASE + 38)
```

Notification mechanism failed to create instance.

ZCOPY_LOG_NOTIF_MECHANISM_RESERVE_ADDR_EC

```
#define ZCOPY_LOG_NOTIF_MECHANISM_RESERVE_ADDR_EC (NETIO_ZCOPY_LOG_BASE + 39)
```

Notification mechanism failed to reserve an address.

ZCOPY_LOG_NOTIF_MECHANISM_RELEASE_ADDR_EC

```
#define ZCOPY_LOG_NOTIF_MECHANISM_RELEASE_ADDR_EC (NETIO_ZCOPY_LOG_BASE + 40)
```

Notification mechanism failed to release an address.

ZCOPY_LOG_NOTIF_MECHANISM_ADD_ROUTE_EC

```
#define ZCOPY_LOG_NOTIF_MECHANISM_ADD_ROUTE_EC (NETIO_ZCOPY_LOG_BASE + 41)
```

Notification mechanism failed to add route.

ZCOPY_LOG_NOTIF_MECHANISM_DELETE_ROUTE_EC

```
#define ZCOPY_LOG_NOTIF_MECHANISM_DELETE_ROUTE_EC (NETIO_ZCOPY_LOG_BASE + 42)
```

Notification mechanism failed to delete route.

ZCOPY_LOG_NOTIF_MECHANISM_BIND_EC

```
#define ZCOPY_LOG_NOTIF_MECHANISM_BIND_EC (NETIO_ZCOPY_LOG_BASE + 43)
```

Notification mechanism failed to bind.

ZCOPY_LOG_NOTIF_MECHANISM_UNBIND_EC

```
#define ZCOPY_LOG_NOTIF_MECHANISM_UNBIND_EC (NETIO_ZCOPY_LOG_BASE + 44)
```

Notification mechanism failed to unbind.

ZCOPY_LOG_NOTIF_MECHANISM_SEND_EC

```
#define ZCOPY_LOG_NOTIF_MECHANISM_SEND_EC (NETIO_ZCOPY_LOG_BASE + 45)
```

Notification mechanism failed to send.

ZCOPY_LOG_WH_REGISTER_FAILED_EC

```
#define ZCOPY_LOG_WH_REGISTER_FAILED_EC (NETIO_ZCOPY_LOG_BASE + 46)
```

Writer history wrap failed.

ZCOPY_LOG_NOTIF_TIMER_CREATE_EC

```
#define ZCOPY_LOG_NOTIF_TIMER_CREATE_EC (NETIO_ZCOPY_LOG_BASE + 47)
```

Timer creation failed.

ZCOPY_LOG_NOTIF_RECEIVE_ON_EVENT_EC

```
#define ZCOPY_LOG_NOTIF_RECEIVE_ON_EVENT_EC (NETIO_ZCOPY_LOG_BASE + 48)
```

Receive on Writer match event failed.

ZCOPY_LOG_NOTIF_TIMEOUT_DELETE_EC

```
#define ZCOPY_LOG_NOTIF_TIMEOUT_DELETE_EC (NETIO_ZCOPY_LOG_BASE + 49)
```

Failed to delete timeout.

ZCOPY_LOG_NOTIF_TIMEOUT_CREATE_EC

```
#define ZCOPY_LOG_NOTIF_TIMEOUT_CREATE_EC (NETIO_ZCOPY_LOG_BASE + 50)
```

Timeout creation failed.

ZCOPY_LOG_ADD_ENTRY_EC

```
#define ZCOPY_LOG_ADD_ENTRY_EC (NETIO_ZCOPY_LOG_BASE + 51)
```

failed to add an indexer entry

ZCOPY_LOG_REMOVE_ENTRY_EC

```
#define ZCOPY_LOG_REMOVE_ENTRY_EC (NETIO_ZCOPY_LOG_BASE + 52)
```

failed to remove an indexer entry

ZCOPY_LOG_DELETE_POOL_EC

```
#define ZCOPY_LOG_DELETE_POOL_EC (NETIO_ZCOPY_LOG_BASE + 53)
```

The memory manager was not able to delete a resource.

ZCOPY_LOG_SHARED_MEM_POOL_OWNER_DEAD_EC

```
#define ZCOPY_LOG_SHARED_MEM_POOL_OWNER_DEAD_EC (NETIO_ZCOPY_LOG_BASE + 54)
```

A shared memory pool is no longer usable because a process died while holding the pool's lock.

6.20.11 WHSQ

Writer History Shared Queue. ModuleID = 22.

Macros

- #define [WHSQ_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC](#) (WHSQ_LOG_BASE + 1)
KEEP_ALL History kind is not supported.
- #define [WHSQ_LOG_UNLIMITED_HISTORY_NOT_SUPPORTED_EC](#) (WHSQ_LOG_BASE + 2)
Unlimited length resource limits unsupported.
- #define [WHSQ_LOG_MAX_SAMPLES_TOO_SMALL_EC](#) (WHSQ_LOG_BASE + 3)
DataWriterQos.resource_limits.max_samples set too small.
- #define [WHSQ_LOG_OBJECT_ALLOCATE_EC](#) (WHSQ_LOG_BASE + 4)
Failed to allocate object of the specified kind.
- #define [WHSQ_LOG_PURGE_FAILED_EC](#) (WHSQ_LOG_BASE + 5)
Failed to purge a sample from the shared queue.
- #define [WHSQ_LOG_WH_CREATE_FAILED_EC](#) (WHSQ_LOG_BASE + 6)
Failed to create the underlying writer history.
- #define [WHSQ_LOG_OBJECT_DELETE_EC](#) (WHSQ_LOG_BASE + 7)
Failed to delete object of the specified kind.
- #define [WHSQ_LOG_CONFIGURATION_EC](#) (WHSQ_LOG_BASE + 8)
Listener or Property not configured when creating the component.
- #define [WHSQ_LOG_NO_DDS_FACTORY_EC](#) (WHSQ_LOG_BASE + 9)
No factory for forwarding when creating a writer history instance.
- #define [WHSQ_LOG_COMMIT_FAILED_EC](#) (WHSQ_LOG_BASE + 10)
Committing a Sample failed.

6.20.11.1 Detailed Description

Writer History Shared Queue. ModuleID = 22.

6.20.11.2 Macro Definition Documentation

WHSQ_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC

```
#define WHSQ_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC (WHSQ_LOG_BASE + 1)
```

KEEP_ALL History kind is not supported.

WHSQ_LOG_UNLIMITED_HISTORY_NOT_SUPPORTED_EC

```
#define WHSQ_LOG_UNLIMITED_HISTORY_NOT_SUPPORTED_EC (WHSQ_LOG_BASE + 2)
```

Unlimited length resource limits unsupported.

Ensure that max_samples, max_samples_per_instance, and max_instances resource limits are all finite values

WHSQ_LOG_MAX_SAMPLES_TOO_SMALL_EC

```
#define WHSQ_LOG_MAX_SAMPLES_TOO_SMALL_EC (WHSQ_LOG_BASE + 3)
```

DataWriterQos.resource_limits.max_samples set too small.

DataWriterQos.resource_limits.max_samples must be no less than max_instances * max_samples_per_instance

WHSQ_LOG_OBJECT_ALLOCATE_EC

```
#define WHSQ_LOG_OBJECT_ALLOCATE_EC (WHSQ_LOG_BASE + 4)
```

Failed to allocate object of the specified kind.

WHSQ_LOG_PURGE_FAILED_EC

```
#define WHSQ_LOG_PURGE_FAILED_EC (WHSQ_LOG_BASE + 5)
```

Failed to purge a sample from the shared queue.

WHSQ_LOG_WH_CREATE_FAILED_EC

```
#define WHSQ_LOG_WH_CREATE_FAILED_EC (WHSQ_LOG_BASE + 6)
```

Failed to create the underlying writer history.

WHSQ_LOG_OBJECT_DELETE_EC

```
#define WHSQ_LOG_OBJECT_DELETE_EC (WHSQ_LOG_BASE + 7)
```

Failed to delete object of the specified kind.

WHSQ_LOG_CONFIGURATION_EC

```
#define WHSQ_LOG_CONFIGURATION_EC (WHSQ_LOG_BASE + 8)
```

Listener or Property not configured when creating the component.

WHSQ_LOG_NO_DDS_FACTORY_EC

```
#define WHSQ_LOG_NO_DDS_FACTORY_EC (WHSQ_LOG_BASE + 9)
```

No factory for forwarding when creating a writer history instance.

WHSQ_LOG_COMMIT_FAILED_EC

```
#define WHSQ_LOG_COMMIT_FAILED_EC (WHSQ_LOG_BASE + 10)
```

Committing a Sample failed.

6.20.12 RH

Reader History. ModuleID = 8.

Macros

- #define `RHSM_LOG_OUTSTANDING_SAMPLE_EC` (`RHSM_LOG_BASE` + 1)
Failed to delete Reader History due to outstanding, unremoved sample.
- #define `RHSM_LOG_NO_PROPERTY_QOS_EC` (`RHSM_LOG_BASE` + 2)
Failed to create Reader History due to unspecified property Qos.
- #define `RHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC` (`RHSM_LOG_BASE` + 3)
KEEP_ALL History kind is not supported.
- #define `RHSM_LOG_MAX_SAMPLE_TOO_SMALL_EC` (`RHSM_LOG_BASE` + 4)
DataReaderQos.resource_limits.max_samples set too small.
- #define `RHSM_LOG_TBF_NOT_SUPPORTED_EC` (`RHSM_LOG_BASE` + 5)
Time-based filtering is not supported.
- #define `RHSM_LOG_SAMPLE_INFO_POOL_EC` (`RHSM_LOG_BASE` + 6)
Failed to allocate a buffer pool for sample info.
- #define `RHSM_LOG_SAMPLE_POOL_EC` (`RHSM_LOG_BASE` + 7)
Failed to allocate a buffer pool for sample pointers.
- #define `RHSM_LOG_SAMPLE_PTR_ARRAY_EC` (`RHSM_LOG_BASE` + 8)
Failed to get sample buffer.
- #define `RHSM_LOG_INFO_ARRAY_EC` (`RHSM_LOG_BASE` + 9)
Failed to get sample info buffer.
- #define `RHSM_LOG_GET_RECEPTION_TIMESTAMP_EC` (`RHSM_LOG_BASE` + 10)
Failed to get time for reception timestamp.
- #define `RHSM_LOG_INVALID_INSTANCE_REPLACEMENT_EC` (`RHSM_LOG_BASE` + 11)
An invalid instance replacement policy was specified.
- #define `RHSM_LOG_FAILED_TO_REMOVE_OLDEST_EC` (`RHSM_LOG_BASE` + 12)
Failed to remove the oldest sample.
- #define `RHSM_LOG_SAMPLE_POOL_EMPTY_EC` (`RHSM_LOG_BASE` + 13)
Sample pool empty, may have exceeded.
- #define `RHSM_LOG_RW_PRUNE_FAILED_EC` (`RHSM_LOG_BASE` + 14)
Failed to prune remote writer entry.
- #define `RHSM_LOG_ENTRY_RESERVATION_FAILED_EC` (`RHSM_LOG_BASE` + 15)
Failed to reserve an entry.
- #define `RHSM_LOG_RETURN_SAMPLE_EC` (`RHSM_LOG_BASE` + 16)
Failed to return a sample.
- #define `RHSM_LOG_HISTORY_UPDATE_EC` (`RHSM_LOG_BASE` + 17)
Failed to update the history queue.
- #define `RHSM_LOG_GET_TIME_EC` (`RHSM_LOG_BASE` + 18)
Failed to get the system time.
- #define `RHSM_LOG_OBJECT_ALLOCATE_EC` (`RHSM_LOG_BASE` + 19)
Failed to allocate object of the specified kind.
- #define `RHSM_LOG_OBJECT_DELETE_EC` (`RHSM_LOG_BASE` + 20)
Failed to delete object of the specified kind.
- #define `RHSM_LOG_OBJECT_INDEX_EC` (`RHSM_LOG_BASE` + 21)
Failed to index an object of the specified kind.
- #define `RHSM_LOG_OBJECT_INVALID_EC` (`RHSM_LOG_BASE` + 22)
Invalid object specified.
- #define `RHSM_LOG_NO_PROPERTY_EC` (`RHSM_LOG_BASE` + 23)

No property when creating a reader history instance.

- #define `RHSM_LOG_DESTINATION_BY_SOURCE_NOT_SUPPORTED_EC` (`RHSM_LOG_BASE` + 24)

Destination ordering by SOURCE_TIMESTAMP ordering is not supported.

- #define `RHSM_LOG_RECOMMIT_FAILURE_EC` (`RHSM_LOG_BASE` + 25)

Failed to recommit sample.

6.20.12.1 Detailed Description

Reader History. ModuleID = 8.

6.20.12.2 Macro Definition Documentation

RHSM_LOG_OUTSTANDING_SAMPLE_EC

```
#define RHSM_LOG_OUTSTANDING_SAMPLE_EC (RHSM_LOG_BASE + 1)
```

Failed to delete Reader History due to outstanding, unremoved sample.

RHSM_LOG_NO_PROPERTY_QOS_EC

```
#define RHSM_LOG_NO_PROPERTY_QOS_EC (RHSM_LOG_BASE + 2)
```

Failed to create Reader History due to unspecified property Qos.

RHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC

```
#define RHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC (RHSM_LOG_BASE + 3)
```

KEEP_ALL History kind is not supported.

RHSM_LOG_MAX_SAMPLE_TOO_SMALL_EC

```
#define RHSM_LOG_MAX_SAMPLE_TOO_SMALL_EC (RHSM_LOG_BASE + 4)
```

DataReaderQos.resource_limits.max_samples set too small.

DataReaderQos.resource_limits.max_samples too small, must be at least DataReaderQos.resource_limits.max_instances * DataReaderQos.resource_limits.max_samples_per_instance

RHSM_LOG_TBF_NOT_SUPPORTED_EC

```
#define RHSM_LOG_TBF_NOT_SUPPORTED_EC (RHSM_LOG_BASE + 5)
```

Time-based filtering is not supported.

RHSM_LOG_SAMPLE_INFO_POOL_EC

```
#define RHSM_LOG_SAMPLE_INFO_POOL_EC (RHSM_LOG_BASE + 6)
```

Failed to allocate a buffer pool for sample info.

RHSM_LOG_SAMPLE_POOL_EC

```
#define RHSM_LOG_SAMPLE_POOL_EC (RHSM_LOG_BASE + 7)
```

Failed to allocate a buffer pool for sample pointers.

RHSM_LOG_SAMPLE_PTR_ARRAY_EC

```
#define RHSM_LOG_SAMPLE_PTR_ARRAY_EC (RHSM_LOG_BASE + 8)
```

Failed to get sample buffer.

May have exceeded DataReaderQos.reader_resource_limits.max_outstanding_reads

RHSM_LOG_INFO_ARRAY_EC

```
#define RHSM_LOG_INFO_ARRAY_EC (RHSM_LOG_BASE + 9)
```

Failed to get sample info buffer.

May have exceeded DataReaderQos.reader_resource_limits.max_outstanding_reads

RHSM_LOG_GET_RECEPTION_TIMESTAMP_EC

```
#define RHSM_LOG_GET_RECEPTION_TIMESTAMP_EC (RHSM_LOG_BASE + 10)
```

Failed to get time for reception timestamp.

RHSM_LOG_INVALID_INSTANCE_REPLACEMENT_EC

```
#define RHSM_LOG_INVALID_INSTANCE_REPLACEMENT_EC (RHSM_LOG_BASE + 11)
```

An invalid instance replacement policy was specified.

RHSM_LOG_FAILED_TO_REMOVE_OLDEST_EC

```
#define RHSM_LOG_FAILED_TO_REMOVE_OLDEST_EC (RHSM_LOG_BASE + 12)
```

Failed to remove the oldest sample.

RHSM_LOG_SAMPLE_POOL_EMPTY_EC

```
#define RHSM_LOG_SAMPLE_POOL_EMPTY_EC (RHSM_LOG_BASE + 13)
```

Sample pool empty, may have exceeded.

The sample pool is empty, may have exceeded `DataReaderQos.resource_limits.max_samples`

RHSM_LOG_RW_PRUNE_FAILED_EC

```
#define RHSM_LOG_RW_PRUNE_FAILED_EC (RHSM_LOG_BASE + 14)
```

Failed to prune remote writer entry.

RHSM_LOG_ENTRY_RESERVATION_FAILED_EC

```
#define RHSM_LOG_ENTRY_RESERVATION_FAILED_EC (RHSM_LOG_BASE + 15)
```

Failed to reserve an entry.

An entry reservation failed beyond normal resource settings

RHSM_LOG_RETURN_SAMPLE_EC

```
#define RHSM_LOG_RETURN_SAMPLE_EC (RHSM_LOG_BASE + 16)
```

Failed to return a sample.

RHSM_LOG_HISTORY_UPDATE_EC

```
#define RHSM_LOG_HISTORY_UPDATE_EC (RHSM_LOG_BASE + 17)
```

Failed to update the history queue.

RHSM_LOG_GET_TIME_EC

```
#define RHSM_LOG_GET_TIME_EC (RHSM_LOG_BASE + 18)
```

Failed to get the system time.

RHSM_LOG_OBJECT_ALLOCATE_EC

```
#define RHSM_LOG_OBJECT_ALLOCATE_EC (RHSM_LOG_BASE + 19)
```

Failed to allocate object of the specified kind.

RHSM_LOG_OBJECT_DELETE_EC

```
#define RHSM_LOG_OBJECT_DELETE_EC (RHSM_LOG_BASE + 20)
```

Failed to delete object of the specified kind.

RHSM_LOG_OBJECT_INDEX_EC

```
#define RHSM_LOG_OBJECT_INDEX_EC (RHSM_LOG_BASE + 21)
```

Failed to index an object of the specified kind.

RHSM_LOG_OBJECT_INVALID_EC

```
#define RHSM_LOG_OBJECT_INVALID_EC (RHSM_LOG_BASE + 22)
```

Invalid object specified.

RHSM_LOG_NO_PROPERTY_EC

```
#define RHSM_LOG_NO_PROPERTY_EC (RHSM_LOG_BASE + 23)
```

No property when creating a reader history instance.

RHSM_LOG_DESTINATION_BY_SOURCE_NOT_SUPPORTED_EC

```
#define RHSM_LOG_DESTINATION_BY_SOURCE_NOT_SUPPORTED_EC (RHSM_LOG_BASE + 24)
```

Destination ordering by SOURCE_TIMESTAMP ordering is not supported.

RHSM_LOG_RECOMMIT_FAILURE_EC

```
#define RHSM_LOG_RECOMMIT_FAILURE_EC (RHSM_LOG_BASE + 25)
```

Failed to recommit sample.

6.20.13 WH

Writer History. ModuleID = 9.

Macros

- #define `WHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC` (`WHSM_LOG_BASE` + 1)
KEEP_ALL History kind is not supported.
- #define `WHSM_LOG_UNLIMITED_HISTORY_NOT_SUPPORTED_EC` (`WHSM_LOG_BASE` + 2)
Unlimited length resource limits unsupported.
- #define `WHSM_LOG_MAX_SAMPLES_TOO_SMALL_EC` (`WHSM_LOG_BASE` + 3)
DataWriterQos.resource_limits.max_samples set too small.
- #define `WHSM_LOG_OBJECT_ALLOCATE_EC` (`WHSM_LOG_BASE` + 4)
Failed to allocate object of the specified kind.
- #define `WHSM_LOG_OBJECT_DELETE_EC` (`WHSM_LOG_BASE` + 5)
Failed to delete object of the specified kind.
- #define `WHSM_LOG_OBJECT_INDEX_EC` (`WHSM_LOG_BASE` + 6)
Failed to index an object of the specified kind.
- #define `WHSM_LOG_NO_PROPERTY_EC` (`WHSM_LOG_BASE` + 7)
No property when creating a writer history instance.
- #define `WHSM_LOG_INVALID_INDEX_OBJECT_EC` (`WHSM_LOG_BASE` + 8)
The sample removed from an index did not match sample's key.

6.20.13.1 Detailed Description

Writer History. ModuleID = 9.

6.20.13.2 Macro Definition Documentation

WHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC

```
#define WHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC (WHSM_LOG_BASE + 1)
```

KEEP_ALL History kind is not supported.

WHSM_LOG_UNLIMITED_HISTORY_NOT_SUPPORTED_EC

```
#define WHSM_LOG_UNLIMITED_HISTORY_NOT_SUPPORTED_EC (WHSM_LOG_BASE + 2)
```

Unlimited length resource limits unsupported.

Ensure that max_samples, max_samples_per_instance, and max_instances resource limits are all finite values

WHSM_LOG_MAX_SAMPLES_TOO_SMALL_EC

```
#define WHSM_LOG_MAX_SAMPLES_TOO_SMALL_EC (WHSM_LOG_BASE + 3)
```

DataWriterQos.resource_limits.max_samples set too small.

DataWriterQos.resource_limits.max_samples must be no less than max_instances * max_samples_per_instance

WHSM_LOG_OBJECT_ALLOCATE_EC

```
#define WHSM_LOG_OBJECT_ALLOCATE_EC (WHSM_LOG_BASE + 4)
```

Failed to allocate object of the specified kind.

WHSM_LOG_OBJECT_DELETE_EC

```
#define WHSM_LOG_OBJECT_DELETE_EC (WHSM_LOG_BASE + 5)
```

Failed to delete object of the specified kind.

WHSM_LOG_OBJECT_INDEX_EC

```
#define WHSM_LOG_OBJECT_INDEX_EC (WHSM_LOG_BASE + 6)
```

Failed to index an object of the specified kind.

WHSM_LOG_NO_PROPERTY_EC

```
#define WHSM_LOG_NO_PROPERTY_EC (WHSM_LOG_BASE + 7)
```

No property when creating a writer history instance.

WHSM_LOG_INVALID_INDEX_OBJECT_EC

```
#define WHSM_LOG_INVALID_INDEX_OBJECT_EC (WHSM_LOG_BASE + 8)
```

The sample removed from an index did not match sample's key.

6.20.14 DPSE

DPSE. ModuleID = 10.

Macros

- `#define DPSE_LOG_INVALID_PEER_ADDRESS_EC (DPSE_LOG_BASE + 1)`
A peer address string specifies an invalid address.
- `#define DPSE_LOG_CREATE_DISCOVERY_PUBLISHER_EC (DPSE_LOG_BASE + 2)`
Failed to create the publisher for a participant discovery writer.
- `#define DPSE_LOG_CREATE_DISCOVERY_SUBSCRIBER_EC (DPSE_LOG_BASE + 3)`
Failed to create the subscriber for a participant discovery datareader.
- `#define DPSE_LOG_REGISTER_TYPE_EC (DPSE_LOG_BASE + 4)`
Failed to register a built-in participant discovery type.
- `#define DPSE_LOG_CREATE_TOPIC_EC (DPSE_LOG_BASE + 5)`
Failed to create a topic for the built-in participant discovery topic.
- `#define DPSE_LOG_CREATE_WRITER_EC (DPSE_LOG_BASE + 6)`
Failed to create a DataWriter for participant discovery.
- `#define DPSE_LOG_CREATE_READER_EC (DPSE_LOG_BASE + 7)`
Failed to create a DataReader for participant discovery.
- `#define DPSE_LOG_DISPOSE_EC (DPSE_LOG_BASE + 8)`
Failed to dispose a participant discovery instance.
- `#define DPSE_LOG_ANNOUNCE_LOCAL_PARTICIPANT_DELETION_EC (DPSE_LOG_BASE + 9)`
Failed to dispose a participant discovery instance upon deletion.
- `#define DPSE_LOG_DELETE_WRITER_EC (DPSE_LOG_BASE + 10)`
Failed to delete a participant discovery DataWriter.
- `#define DPSE_LOG_DELETE_PUBLISHER_EC (DPSE_LOG_BASE + 11)`
Failed to delete a participant discovery Publisher.
- `#define DPSE_LOG_DELETE_READER_EC (DPSE_LOG_BASE + 12)`
Failed to delete a participant discovery DataReader.
- `#define DPSE_LOG_DELETE_TOPIC_EC (DPSE_LOG_BASE + 13)`
Failed to delete a participant discovery Topic.
- `#define DPSE_LOG_DELETE_SUBSCRIBER_EC (DPSE_LOG_BASE + 14)`
Failed to delete a participant discovery Subscriber.
- `#define DPSE_LOG_OBJECT_ALLOCATE_EC (DPSE_LOG_BASE + 15)`
Failed to allocate an object of the specified kind.
- `#define DPSE_LOG_OBJECT_INITIALIZE_EC (DPSE_LOG_BASE + 16)`
Failed to initialize an object of the specified kind.
- `#define DPSE_LOG_OBJECT_FINALIZE_EC (DPSE_LOG_BASE + 17)`
Failed to finalize an object of the specified kind.
- `#define DPSE_LOG_OBJECT_DELETE_EC (DPSE_LOG_BASE + 18)`
Failed to delete an object of the specified kind.
- `#define DPSE_LOG_OBJECT_INVALID_EC (DPSE_LOG_BASE + 19)`
The object was invalid in the context it was used.
- `#define DPSE_LOG_CDR_SET_POSITION_EC (DPSE_LOG_BASE + 20)`
Failed to set the CDR stream position.
- `#define DPSE_LOG_CDR_SERIALIZE_EC (DPSE_LOG_BASE + 21)`
Failed to serialize the specified kind.
- `#define DPSE_LOG_CDR_DESERIALIZE_EC (DPSE_LOG_BASE + 22)`
Failed to deserialize the specified kind.
- `#define DPSE_LOG_SERIALIZE_GUID_EC (DPSE_LOG_BASE + 23)`

- Failed to serialize a GUID key parameter.*

 - #define `DPSE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC` (`DPSE_LOG_BASE` + 24)
- Failed to serialize a Builtin Endpoint Mask parameter.*

 - #define `DPSE_LOG_SERIALIZE_PROTOCOL_VERSION_EC` (`DPSE_LOG_BASE` + 25)
- Failed to serialize a Protocol Version parameter.*

 - #define `DPSE_LOG_SERIALIZE_VENDOR_ID_EC` (`DPSE_LOG_BASE` + 26)
- Failed to serialize a Vendor ID parameter.*

 - #define `DPSE_LOG_SERIALIZE_DEFAULT_UNICAST_EC` (`DPSE_LOG_BASE` + 27)
- Failed to serialize a Default Unicast Locator parameter.*

 - #define `DPSE_LOG_SERIALIZE_META_UNICAST_EC` (`DPSE_LOG_BASE` + 28)
- Failed to serialize a Meta Unicast Locator parameter.*

 - #define `DPSE_LOG_SERIALIZE_META_MULTICAST_EC` (`DPSE_LOG_BASE` + 29)
- Failed to serialize a Meta Multicast Locator parameter.*

 - #define `DPSE_LOG_SERIALIZE_LEASE_DURATION_EC` (`DPSE_LOG_BASE` + 30)
- Failed to serialize a Lease Duration parameter.*

 - #define `DPSE_LOG_SERIALIZE_PRODUCT_VERSION_EC` (`DPSE_LOG_BASE` + 31)
- Failed to serialize a Product Version parameter.*

 - #define `DPSE_LOG_DESERIALIZE_PRODUCT_VERSION_EC` (`DPSE_LOG_BASE` + 32)
- Failed to serialize a Product Version parameter.*

 - #define `DPSE_LOG_SERIALIZE_PARTICIPANT_NAME_EC` (`DPSE_LOG_BASE` + 33)
- Failed to serialize a Participant Name parameter.*

 - #define `DPSE_LOG_DESERIALIZE_GUID_EC` (`DPSE_LOG_BASE` + 34)
- Failed to deserialize a GUID key parameter.*

 - #define `DPSE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC` (`DPSE_LOG_BASE` + 35)
- Failed to deserialize a Builtin Endpoint Mask parameter.*

 - #define `DPSE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC` (`DPSE_LOG_BASE` + 36)
- Failed to deserialize a Protocol Version parameter.*

 - #define `DPSE_LOG_DESERIALIZE_VENDOR_ID_EC` (`DPSE_LOG_BASE` + 37)
- Failed to deserialize a Vendor ID parameter.*

 - #define `DPSE_LOG_DESERIALIZE_TOO_MANY_LOCATORS_EC` (`DPSE_LOG_BASE` + 38)
- Cannot deserialize another locator parameter, having reached maximum of 4 unicast or 4 multicast locators.*

 - #define `DPSE_LOG_DESERIALIZE_PARTICIPANT_NAME_EC` (`DPSE_LOG_BASE` + 39)
- Failed to deserialize a Participant Name parameter.*

 - #define `DPSE_LOG_DESERIALIZE_UNKNOWN_PID_EC` (`DPSE_LOG_BASE` + 40)
- Failed to deserialize a parameter with an unknown ID.*

 - #define `DPSE_LOG_ANNOUNCEMENT_EC` (`DPSE_LOG_BASE` + 41)
- Failed to send a participant discovery announcement.*

 - #define `DPSE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC` (`DPSE_LOG_BASE` + 42)
- Failed to schedule an event to send the next participant discovery announcement.*

 - #define `DPSE_LOG_ADVANCE_SN_EC` (`DPSE_LOG_BASE` + 43)
- Failed to advance the sequence number of a participant discovery announcement.*

 - #define `DPSE_LOG_ANNOUNCE_WRITE_EC` (`DPSE_LOG_BASE` + 44)
- Failed to write a participant discovery announcement message.*

 - #define `DPSE_LOG_SCHEDULE_FAST_ASSERTION_EC` (`DPSE_LOG_BASE` + 45)
- Failed to schedule an event to send the next participant discovery announcement.*

 - #define `DPSE_LOG_PARTICIPANT_TAKE_EC` (`DPSE_LOG_BASE` + 46)
- Failed to take a sample from a participant discovery DataReader.*

- #define DPSE_LOG_ASSERT_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 47)
Failed to assert remote participant.
- #define DPSE_LOG_ON_ASSERT_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 48)
Failed to assert and complete discovery of a remote participant.
- #define DPSE_LOG_ADD_ANONYMOUS_ROUTE_EC (DPSE_LOG_BASE + 49)
Failed to add an anonymous route.
- #define DPSE_LOG_DELETE_ANONYMOUS_ROUTE_EC (DPSE_LOG_BASE + 50)
Failed to delete an anonymous route.
- #define DPSE_LOG_MAX_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 51)
Exceeded resource limit, remote_participant_allocation.
- #define DPSE_LOG_REFRESH_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 52)
Failed to refresh liveness for a remote participant.
- #define DPSE_LOG_RESET_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 53)
Failed to reset liveness for a remote participant.
- #define DPSE_LOG_INVALID_DISCOVERY_SAMPLE_EC (DPSE_LOG_BASE + 54)
Received a participant discovery announcement with invalid state.
- #define DPSE_LOG_RETURN_DISCOVERY_SAMPLE_EC (DPSE_LOG_BASE + 55)
Failed to return a loan on a participant discovery announcement.
- #define DPSE_LOG_SERIALIZE_MULTICAST_EC (DPSE_LOG_BASE + 56)
Failed to serialize a Multicast Locator parameter.
- #define DPSE_LOG_GET_DDS_PROPERTIES_EC (DPSE_LOG_BASE + 57)
Failed to serialize a Multicast Locator parameter.
- #define DPSE_LOG_ENTITY_ENABLE_EC (DPSE_LOG_BASE + 58)
Failed to enable the specified entity kind.
- #define DPSE_LOG_SEQUENCE_SETMAX_EC (DPSE_LOG_BASE + 59)
Failed to set the maximum length of a sequence of the specified kind.
- #define DPSE_LOG_SEQUENCE_SETLENGTH_EC (DPSE_LOG_BASE + 60)
Failed to set the length of a sequence of the specified kind.
- #define DPSE_LOG_SEQUENCE_GETREF_EC (DPSE_LOG_BASE + 61)
Failed to get a reference at the specified index for a sequence of the specified kind.
- #define DPSE_LOG_SEQUENCE_INITIALIZE_EC (DPSE_LOG_BASE + 62)
Failed to initialize a sequence of the specified kind.
- #define DPSE_LOG_SEQUENCE_FINALIZE_EC (DPSE_LOG_BASE + 63)
Failed to finalize a sequence of the specified kind.
- #define DPSE_LOG_SEQUENCE_COPY_EC (DPSE_LOG_BASE + 64)
Failed to copy a sequence of the specified kind.
- #define DPSE_LOG_GET_TIMER_EC (DPSE_LOG_BASE + 65)
Failed to get timer.
- #define DPSE_LOG_UNSUPPORTED_LOCATOR_EC (DPSE_LOG_BASE + 66)
The locator kind is not supported by any transport registered with the domain participant and is dropped.
- #define DPSE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC (DPSE_LOG_BASE + 67)
A participant with incompatible checksum was discovered and ignored.
- #define DPSE_LOG_SERIALIZE_PROP_EC (DPSE_LOG_BASE + 68)
A Participant failed to serialize its properties.

6.20.14.1 Detailed Description

DPSE. ModuleID = 10.

6.20.14.2 Macro Definition Documentation

DPSE_LOG_INVALID_PEER_ADDRESS_EC

```
#define DPSE_LOG_INVALID_PEER_ADDRESS_EC (DPSE_LOG_BASE + 1)
```

A peer address string specifies an invalid address.

DPSE_LOG_CREATE_DISCOVERY_PUBLISHER_EC

```
#define DPSE_LOG_CREATE_DISCOVERY_PUBLISHER_EC (DPSE_LOG_BASE + 2)
```

Failed to create the publisher for a participant discovery writer.

DPSE_LOG_CREATE_DISCOVERY_SUBSCRIBER_EC

```
#define DPSE_LOG_CREATE_DISCOVERY_SUBSCRIBER_EC (DPSE_LOG_BASE + 3)
```

Failed to create the subscriber for a participant discovery datareader.

DPSE_LOG_REGISTER_TYPE_EC

```
#define DPSE_LOG_REGISTER_TYPE_EC (DPSE_LOG_BASE + 4)
```

Failed to register a built-in participant discovery type.

DPSE_LOG_CREATE_TOPIC_EC

```
#define DPSE_LOG_CREATE_TOPIC_EC (DPSE_LOG_BASE + 5)
```

Failed to create a topic for the built-in participant discovery topic.

DPSE_LOG_CREATE_WRITER_EC

```
#define DPSE_LOG_CREATE_WRITER_EC (DPSE_LOG_BASE + 6)
```

Failed to create a DataWriter for participant discovery.

DPSE_LOG_CREATE_READER_EC

```
#define DPSE_LOG_CREATE_READER_EC (DPSE_LOG_BASE + 7)
```

Failed to create a DataReader for participant discovery.

DPSE_LOG_DISPOSE_EC

```
#define DPSE_LOG_DISPOSE_EC (DPSE_LOG_BASE + 8)
```

Failed to dispose a participant discovery instance.

DPSE_LOG_ANNOUNCE_LOCAL_PARTICIPANT_DELETION_EC

```
#define DPSE_LOG_ANNOUNCE_LOCAL_PARTICIPANT_DELETION_EC (DPSE_LOG_BASE + 9)
```

Failed to dispose a participant discovery instance upon deletion.

DPSE_LOG_DELETE_WRITER_EC

```
#define DPSE_LOG_DELETE_WRITER_EC (DPSE_LOG_BASE + 10)
```

Failed to delete a participant discovery DataWriter.

DPSE_LOG_DELETE_PUBLISHER_EC

```
#define DPSE_LOG_DELETE_PUBLISHER_EC (DPSE_LOG_BASE + 11)
```

Failed to delete a participant discovery Publisher.

DPSE_LOG_DELETE_READER_EC

```
#define DPSE_LOG_DELETE_READER_EC (DPSE_LOG_BASE + 12)
```

Failed to delete a participant discovery DataReader.

DPSE_LOG_DELETE_TOPIC_EC

```
#define DPSE_LOG_DELETE_TOPIC_EC (DPSE_LOG_BASE + 13)
```

Failed to delete a participant discovery Topic.

DPSE_LOG_DELETE_SUBSCRIBER_EC

```
#define DPSE_LOG_DELETE_SUBSCRIBER_EC (DPSE_LOG_BASE + 14)
```

Failed to delete a participant discovery Subscriber.

DPSE_LOG_OBJECT_ALLOCATE_EC

```
#define DPSE_LOG_OBJECT_ALLOCATE_EC (DPSE_LOG_BASE + 15)
```

Failed to allocate an object of the specified kind.

DPSE_LOG_OBJECT_INITIALIZE_EC

```
#define DPSE_LOG_OBJECT_INITIALIZE_EC (DPSE_LOG_BASE + 16)
```

Failed to initialize an object of the specified kind.

DPSE_LOG_OBJECT_FINALIZE_EC

```
#define DPSE_LOG_OBJECT_FINALIZE_EC (DPSE_LOG_BASE + 17)
```

Failed to finalize an object of the specified kind.

DPSE_LOG_OBJECT_DELETE_EC

```
#define DPSE_LOG_OBJECT_DELETE_EC (DPSE_LOG_BASE + 18)
```

Failed to delete an object of the specified kind.

DPSE_LOG_OBJECT_INVALID_EC

```
#define DPSE_LOG_OBJECT_INVALID_EC (DPSE_LOG_BASE + 19)
```

The object was invalid in the context it was used.

DPSE_LOG_CDR_SET_POSITION_EC

```
#define DPSE_LOG_CDR_SET_POSITION_EC (DPSE_LOG_BASE + 20)
```

Failed to set the CDR stream position.

DPSE_LOG_CDR_SERIALIZE_EC

```
#define DPSE_LOG_CDR_SERIALIZE_EC (DPSE_LOG_BASE + 21)
```

Failed to serialize the specified kind.

DPSE_LOG_CDR_DESERIALIZE_EC

```
#define DPSE_LOG_CDR_DESERIALIZE_EC (DPSE_LOG_BASE + 22)
```

Failed to deserialize the specified kind.

DPSE_LOG_SERIALIZE_GUID_EC

```
#define DPSE_LOG_SERIALIZE_GUID_EC (DPSE_LOG_BASE + 23)
```

Failed to serialize a GUID key parameter.

DPSE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC

```
#define DPSE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC (DPSE_LOG_BASE + 24)
```

Failed to serialize a Builtin Endpoint Mask parameter.

DPSE_LOG_SERIALIZE_PROTOCOL_VERSION_EC

```
#define DPSE_LOG_SERIALIZE_PROTOCOL_VERSION_EC (DPSE_LOG_BASE + 25)
```

Failed to serialize a Protocol Version parameter.

DPSE_LOG_SERIALIZE_VENDOR_ID_EC

```
#define DPSE_LOG_SERIALIZE_VENDOR_ID_EC (DPSE_LOG_BASE + 26)
```

Failed to serialize a Vendor ID parameter.

DPSE_LOG_SERIALIZE_DEFAULT_UNICAST_EC

```
#define DPSE_LOG_SERIALIZE_DEFAULT_UNICAST_EC (DPSE_LOG_BASE + 27)
```

Failed to serialize a Default Unicast Locator parameter.

DPSE_LOG_SERIALIZE_META_UNICAST_EC

```
#define DPSE_LOG_SERIALIZE_META_UNICAST_EC (DPSE_LOG_BASE + 28)
```

Failed to serialize a Meta Unicast Locator parameter.

DPSE_LOG_SERIALIZE_META_MULTICAST_EC

```
#define DPSE_LOG_SERIALIZE_META_MULTICAST_EC (DPSE_LOG_BASE + 29)
```

Failed to serialize a Meta Multicast Locator parameter.

DPSE_LOG_SERIALIZE_LEASE_DURATION_EC

```
#define DPSE_LOG_SERIALIZE_LEASE_DURATION_EC (DPSE_LOG_BASE + 30)
```

Failed to serialize a Lease Duration parameter.

DPSE_LOG_SERIALIZE_PRODUCT_VERSION_EC

```
#define DPSE_LOG_SERIALIZE_PRODUCT_VERSION_EC (DPSE_LOG_BASE + 31)
```

Failed to serialize a Product Version parameter.

DPSE_LOG_DESERIALIZE_PRODUCT_VERSION_EC

```
#define DPSE_LOG_DESERIALIZE_PRODUCT_VERSION_EC (DPSE_LOG_BASE + 32)
```

Failed to serialize a Product Version parameter.

DPSE_LOG_SERIALIZE_PARTICIPANT_NAME_EC

```
#define DPSE_LOG_SERIALIZE_PARTICIPANT_NAME_EC (DPSE_LOG_BASE + 33)
```

Failed to serialize a Participant Name parameter.

DPSE_LOG_DESERIALIZE_GUID_EC

```
#define DPSE_LOG_DESERIALIZE_GUID_EC (DPSE_LOG_BASE + 34)
```

Failed to deserialize a GUID key parameter.

DPSE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC

```
#define DPSE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC (DPSE_LOG_BASE + 35)
```

Failed to deserialize a Builtin Endpoint Mask parameter.

DPSE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC

```
#define DPSE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC (DPSE_LOG_BASE + 36)
```

Failed to deserialize a Protocol Version parameter.

DPSE_LOG_DESERIALIZE_VENDOR_ID_EC

```
#define DPSE_LOG_DESERIALIZE_VENDOR_ID_EC (DPSE_LOG_BASE + 37)
```

Failed to deserialize a Vendor ID parameter.

DPSE_LOG_DESERIALIZE_TOO_MANY_LOCATORS_EC

```
#define DPSE_LOG_DESERIALIZE_TOO_MANY_LOCATORS_EC (DPSE_LOG_BASE + 38)
```

Cannot deserialize another locator parameter, having reached maximum of 4 unicast or 4 multicast locators.

DPSE_LOG_DESERIALIZE_PARTICIPANT_NAME_EC

```
#define DPSE_LOG_DESERIALIZE_PARTICIPANT_NAME_EC (DPSE_LOG_BASE + 39)
```

Failed to deserialize a Participant Name parameter.

DPSE_LOG_DESERIALIZE_UNKNOWN_PID_EC

```
#define DPSE_LOG_DESERIALIZE_UNKNOWN_PID_EC (DPSE_LOG_BASE + 40)
```

Failed to deserialize a parameter with an unknown ID.

DPSE_LOG_ANNOUNCEMENT_EC

```
#define DPSE_LOG_ANNOUNCEMENT_EC (DPSE_LOG_BASE + 41)
```

Failed to send a participant discovery announcement.

DPSE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC

```
#define DPSE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC (DPSE_LOG_BASE + 42)
```

Failed to schedule an event to send the next participant discovery announcement.

DPSE_LOG_ADVANCE_SN_EC

```
#define DPSE_LOG_ADVANCE_SN_EC (DPSE_LOG_BASE + 43)
```

Failed to advance the sequence number of a participant discovery announcement.

DPSE_LOG_ANNOUNCE_WRITE_EC

```
#define DPSE_LOG_ANNOUNCE_WRITE_EC (DPSE_LOG_BASE + 44)
```

Failed to write a participant discovery announcement message.

DPSE_LOG_SCHEDULE_FAST_ASSERTION_EC

```
#define DPSE_LOG_SCHEDULE_FAST_ASSERTION_EC (DPSE_LOG_BASE + 45)
```

Failed to schedule an event to send the next participant discovery announcement.

DPSE_LOG_PARTICIPANT_TAKE_EC

```
#define DPSE_LOG_PARTICIPANT_TAKE_EC (DPSE_LOG_BASE + 46)
```

Failed to take a sample from a participant discovery DataReader.

DPSE_LOG_ASSERT_REMOTE_PARTICIPANT_EC

```
#define DPSE_LOG_ASSERT_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 47)
```

Failed to assert remote participant.

DPSE_LOG_ON_ASSERT_REMOTE_PARTICIPANT_EC

```
#define DPSE_LOG_ON_ASSERT_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 48)
```

Failed to assert and complete discovery of a remote participant.

DPSE_LOG_ADD_ANONYMOUS_ROUTE_EC

```
#define DPSE_LOG_ADD_ANONYMOUS_ROUTE_EC (DPSE_LOG_BASE + 49)
```

Failed to add an anonymous route.

Results in a DomainParticipant not having a destination to which it should have sent participant discovery announcements

DPSE_LOG_DELETE_ANONYMOUS_ROUTE_EC

```
#define DPSE_LOG_DELETE_ANONYMOUS_ROUTE_EC (DPSE_LOG_BASE + 50)
```

Failed to delete an anonymous route.

DPSE_LOG_MAX_REMOTE_PARTICIPANT_EC

```
#define DPSE_LOG_MAX_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 51)
```

Exceeded resource limit, remote_participant_allocation.

DPSE_LOG_REFRESH_REMOTE_PARTICIPANT_EC

```
#define DPSE_LOG_REFRESH_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 52)
```

Failed to refresh liveliness for a remote participant.

DPSE_LOG_RESET_REMOTE_PARTICIPANT_EC

```
#define DPSE_LOG_RESET_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 53)
```

Failed to reset liveliness for a remote participant.

DPSE_LOG_INVALID_DISCOVERY_SAMPLE_EC

```
#define DPSE_LOG_INVALID_DISCOVERY_SAMPLE_EC (DPSE_LOG_BASE + 54)
```

Received a participant discovery announcement with invalid state.

DPSE_LOG_RETURN_DISCOVERY_SAMPLE_EC

```
#define DPSE_LOG_RETURN_DISCOVERY_SAMPLE_EC (DPSE_LOG_BASE + 55)
```

Failed to return a loan on a participant discovery announcement.

DPSE_LOG_SERIALIZE_MULTICAST_EC

```
#define DPSE_LOG_SERIALIZE_MULTICAST_EC (DPSE_LOG_BASE + 56)
```

Failed to serialize a Multicast Locator parameter.

DPSE_LOG_GET_DDS_PROPERTIES_EC

```
#define DPSE_LOG_GET_DDS_PROPERTIES_EC (DPSE_LOG_BASE + 57)
```

Failed to serialize a Multicast Locator parameter.

DPSE_LOG_ENTITY_ENABLE_EC

```
#define DPSE_LOG_ENTITY_ENABLE_EC (DPSE_LOG_BASE + 58)
```

Failed to enable the specified entity kind.

DPSE_LOG_SEQUENCE_SETMAX_EC

```
#define DPSE_LOG_SEQUENCE_SETMAX_EC (DPSE_LOG_BASE + 59)
```

Failed to set the maximum length of a sequence of the specified kind.

DPSE_LOG_SEQUENCE_SETLENGTH_EC

```
#define DPSE_LOG_SEQUENCE_SETLENGTH_EC (DPSE_LOG_BASE + 60)
```

Failed to set the length of a sequence of the specified kind.

DPSE_LOG_SEQUENCE_GETREF_EC

```
#define DPSE_LOG_SEQUENCE_GETREF_EC (DPSE_LOG_BASE + 61)
```

Failed to get a reference at the specified index for a sequence of the specified kind.

DPSE_LOG_SEQUENCE_INITIALIZE_EC

```
#define DPSE_LOG_SEQUENCE_INITIALIZE_EC (DPSE_LOG_BASE + 62)
```

Failed to initialize a sequence of the specified kind.

DPSE_LOG_SEQUENCE_FINALIZE_EC

```
#define DPSE_LOG_SEQUENCE_FINALIZE_EC (DPSE_LOG_BASE + 63)
```

Failed to finalize a sequence of the specified kind.

DPSE_LOG_SEQUENCE_COPY_EC

```
#define DPSE_LOG_SEQUENCE_COPY_EC (DPSE_LOG_BASE + 64)
```

Failed to copy a sequence of the specified kind.

DPSE_LOG_GET_TIMER_EC

```
#define DPSE_LOG_GET_TIMER_EC (DPSE_LOG_BASE + 65)
```

Failed to get timer.

DPSE_LOG_UNSUPPORTED_LOCATOR_EC

```
#define DPSE_LOG_UNSUPPORTED_LOCATOR_EC (DPSE_LOG_BASE + 66)
```

The locator kind is not supported by any transport registered with the domain participant and is dropped.

DPSE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC

```
#define DPSE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC (DPSE_LOG_BASE + 67)
```

A participant with incompatible checksum was discovered and ignored.

DPSE_LOG_SERIALIZE_PROP_EC

```
#define DPSE_LOG_SERIALIZE_PROP_EC (DPSE_LOG_BASE + 68)
```

A Participant failed to serialize its properties.

6.20.15 DPDE

DPDE. ModuleID = 11.

Macros

- #define `DPDE_LOG_CDR_SET_OFFSET_EC` (`DPDE_LOG_BASE` + 1)
Failed to set current offset of stream.
- #define `DPDE_LOG_SERIALIZE_ENTITY_NAME_EC` (`DPDE_LOG_BASE` + 2)
Failed to serialize Entity Name parameter.
- #define `DPDE_LOG_SERIALIZE_TOPIC_NAME_EC` (`DPDE_LOG_BASE` + 3)
Failed to serialize Topic Name parameter.
- #define `DPDE_LOG_SERIALIZE_TYPE_NAME_EC` (`DPDE_LOG_BASE` + 4)
Failed to serialize Type Name parameter.
- #define `DPDE_LOG_SERIALIZE_GUID_EC` (`DPDE_LOG_BASE` + 5)
Failed to serialize GUID key.
- #define `DPDE_LOG_SERIALIZE_DEFAULT_UNICAST_EC` (`DPDE_LOG_BASE` + 6)
Failed to serialize Default Unicast Locator parameter.
- #define `DPDE_LOG_SERIALIZE_PROTOCOL_VERSION_EC` (`DPDE_LOG_BASE` + 7)
Failed to serialize Protocol Version parameter.
- #define `DPDE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC` (`DPDE_LOG_BASE` + 8)
Failed to deserialize Protocol Version parameter.
- #define `DPDE_LOG_DESERIALIZE_VENDOR_ID_EC` (`DPDE_LOG_BASE` + 9)
Failed to deserialize Vendor ID parameter.
- #define `DPDE_LOG_SERIALIZE_VENDOR_ID_EC` (`DPDE_LOG_BASE` + 10)
Failed to serialize Vendor ID parameter.
- #define `DPDE_LOG_SERIALIZE_PRODUCT_VERSION_EC` (`DPDE_LOG_BASE` + 11)
Failed to serialize Product Version parameter.
- #define `DPDE_LOG_SERIALIZE_LEASE_DURATION_EC` (`DPDE_LOG_BASE` + 12)
Failed to serialize Lease Duration parameter.
- #define `DPDE_LOG_CREATE_PUBLISHER_EC` (`DPDE_LOG_BASE` + 13)
Failed to create Publisher for built-in endpoint DataWriters.
- #define `DPDE_LOG_CREATE_SUBSCRIBER_EC` (`DPDE_LOG_BASE` + 14)
Failed to create Subscriber for built-in endpoint DataReaders.
- #define `DPDE_LOG_CREATE_PARTICIPANT_DISCOVERY_EC` (`DPDE_LOG_BASE` + 15)
Failed to create built-in endpoint for participant discovery after creating local user DomainParticipant.
- #define `DPDE_LOG_CREATE_PUBLICATION_DISCOVERY_EC` (`DPDE_LOG_BASE` + 16)
Failed to create built-in publication endpoint after creating local user DomainParticipant.
- #define `DPDE_LOG_CREATE_SUBSCRIPTION_DISCOVERY_EC` (`DPDE_LOG_BASE` + 17)
Failed to create built-in subscription endpoint after creating local DomainParticipant.
- #define `DPDE_LOG_DELETE_SUBSCRIPTION_TOPIC_EC` (`DPDE_LOG_BASE` + 18)
Failed to delete subscription built-in topic.
- #define `DPDE_LOG_DELETE_PUBLICATION_TOPIC_EC` (`DPDE_LOG_BASE` + 19)
Failed to delete publication built-in topic.
- #define `DPDE_LOG_DELETE_PARTICIPANT_TOPIC_EC` (`DPDE_LOG_BASE` + 20)
Failed to delete participant built-in topic.
- #define `DPDE_LOG_DISPOSE_PARTICIPANT_EC` (`DPDE_LOG_BASE` + 21)
Failed to dispose built-in participant when deleting DomainParticipant.
- #define `DPDE_LOG_ALLOCATE_DPDE_EC` (`DPDE_LOG_BASE` + 22)
Out of memory to allocate discovery plugin.
- #define `DPDE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC` (`DPDE_LOG_BASE` + 23)

- Failed to serialize Builtin Endpoint Mask parameter.*

 - #define `DPDE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC` (`DPDE_LOG_BASE` + 24)
- Failed to deserialize Builtin Endpoint Mask parameter.*

 - #define `DPDE_LOG_DESERIALIZE_UNKNOWN_PID_EC` (`DPDE_LOG_BASE` + 25)
- Failed to deserialize a parameter of unknown ID.*

 - #define `DPDE_LOG_ANNOUNCEMENT_EC` (`DPDE_LOG_BASE` + 26)
- Failed to write a dynamic participant discovery message.*

 - #define `DPDE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC` (`DPDE_LOG_BASE` + 27)
- Failed to schedule an event to assert the next participant discovery announcement.*

 - #define `DPDE_LOG_ADVANCE_SN_EC` (`DPDE_LOG_BASE` + 28)
- Failed to advance the sequence number of a participant discovery announcement.*

 - #define `DPDE_LOG_ANNOUNCE_WRITE_EC` (`DPDE_LOG_BASE` + 29)
- Failed to write initial participant discovery announcement.*

 - #define `DPDE_LOG_SCHEDULE_FAST_ASSERTION_EC` (`DPDE_LOG_BASE` + 30)
- Failed to schedule event for asserting participant discovery announcements.*

 - #define `DPDE_LOG_REGISTER_TYPE_EC` (`DPDE_LOG_BASE` + 32)
- Failed to register a built-in type for discovery.*

 - #define `DPDE_LOG_CREATE_TOPIC_EC` (`DPDE_LOG_BASE` + 33)
- Failed to create a topic for discovery.*

 - #define `DPDE_LOG_CREATE_WRITER_EC` (`DPDE_LOG_BASE` + 34)
- Failed to create a built-in DataWriter for discovery.*

 - #define `DPDE_LOG_CREATE_READER_EC` (`DPDE_LOG_BASE` + 35)
- Failed to create a built-in DataReader for discovery.*

 - #define `DPDE_LOG_ASSERT_REMOTE_PARTICIPANT_EC` (`DPDE_LOG_BASE` + 36)
- Failed to assert and complete discovery of a remote participant.*

 - #define `DPDE_LOG_ENABLE_REMOTE_PARTICIPANT_EC` (`DPDE_LOG_BASE` + 37)
- Failed to enable and complete discovery of a remote participant.*

 - #define `DPDE_LOG_REFRESH_REMOTE_PARTICIPANT_EC` (`DPDE_LOG_BASE` + 38)
- Failed to refresh liveness for a discovered remote participant.*

 - #define `DPDE_LOG_TAKE_PARTICIPANT_SAMPLE_EC` (`DPDE_LOG_BASE` + 39)
- Failed to take a sample from a participant discovery DataReader.*

 - #define `DPDE_LOG_INVALID_PARTICIPANT_SAMPLE_EC` (`DPDE_LOG_BASE` + 40)
- Received a participant discovery announcement with invalid state.*

 - #define `DPDE_LOG_RETURN_PARTICIPANT_SAMPLE_EC` (`DPDE_LOG_BASE` + 41)
- Failed to return a loan on a participant discovery sample.*

 - #define `DPDE_LOG_DISPOSE_PUBLICATION_EC` (`DPDE_LOG_BASE` + 42)
- Failed to dispose an instance for a publication.*

 - #define `DPDE_LOG_DISPOSE_SUBSCRIPTION_EC` (`DPDE_LOG_BASE` + 43)
- Failed to dispose an instance of a subscription.*

 - #define `DPDE_LOG_ASSERT_REMOTE_PUBLICATION_EC` (`DPDE_LOG_BASE` + 44)
- Failed to assert and complete discovery of a remote publication.*

 - #define `DPDE_LOG_ASSERT_REMOTE_SUBSCRIPTION_EC` (`DPDE_LOG_BASE` + 45)
- Failed to assert and complete discovery of a remote subscription. Logs the topic name.*

 - #define `DPDE_LOG_TAKE_PUBLICATION_SAMPLE_EC` (`DPDE_LOG_BASE` + 46)
- Failed to take a sample from a publication discovery DataReader.*

 - #define `DPDE_LOG_REMOVE_REMOTE_PUBLICATION_EC` (`DPDE_LOG_BASE` + 47)
- Failed to dispose or unregister a remote publication instance.*

- `#define DPDE_LOG_INVALID_PUBLICATION_SAMPLE_EC (DPDE_LOG_BASE + 48)`
Received a publication discovery announcement with invalid state.
- `#define DPDE_LOG_RETURN_PUBLICATION_SAMPLE_EC (DPDE_LOG_BASE + 49)`
Failed to return a loan on a publication discovery sample.
- `#define DPDE_LOG_TAKE_SUBSCRIPTION_SAMPLE_EC (DPDE_LOG_BASE + 51)`
Failed to take a sample from a subscription discovery DataReader.
- `#define DPDE_LOG_INVALID_SUBSCRIPTION_SAMPLE_EC (DPDE_LOG_BASE + 52)`
Received a subscription discovery sample with invalid state.
- `#define DPDE_LOG_RETURN_SUBSCRIPTION_SAMPLE_EC (DPDE_LOG_BASE + 53)`
Failed to return a loan on a subscription discovery sample.
- `#define DPDE_LOG_REMOVE_REMOTE_SUBSCRIPTION_EC (DPDE_LOG_BASE + 54)`
Failed to dispose or unregister a remote subscription instance.
- `#define DPDE_LOG_LOCATORS_FULL_EC (DPDE_LOG_BASE + 55)`
A locator sequence is full, the remaining locators of the specified kind are dropped.
- `#define DPDE_LOG_DESERIALIZE_LOCATOR_EC (DPDE_LOG_BASE + 56)`
Failed to deserialize a locator of the specified kind.
- `#define DPDE_LOG_UNSUPPORTED_LOCATOR_EC (DPDE_LOG_BASE + 57)`
The locator kind is not supported by any transport registered with the domain participant and is dropped.
- `#define DPDE_LOG_ADD_PEER_EC (DPDE_LOG_BASE + 58)`
Failed to add the specified entity as a peer.
- `#define DPDE_LOG_REMOVE_PEER_EC (DPDE_LOG_BASE + 59)`
Failed to add the specified entity as a peer.
- `#define DPDE_LOG_SERIALIZE_PRESENTATION_EC (DPDE_LOG_BASE + 60)`
Failed to serialize Presentation parameter.
- `#define DPDE_LOG_UNREGISTER_TYPE_EC (DPDE_LOG_BASE + 61)`
Failed to unregister a built-in type for discovery.
- `#define DPDE_LOG_ADD_REMOTE_PARTICIPANT_ROUTES_FAILED_EC (DPDE_LOG_BASE + 62)`
Failed to add routes to a remote participant.
- `#define DPDE_LOG_DELETE_REMOTE_PARTICIPANT_ROUTES_FAILED_EC (DPDE_LOG_BASE + 63)`
Failed to delete routes to a remote participant.
- `#define DPDE_LOG_REMOVE_REMOTE_PARTICIPANT_EC (DPDE_LOG_BASE + 64)`
Failed remove a remote participant.
- `#define DPDE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC (DPDE_LOG_BASE + 65)`
A participant with incompatible checksum was discovered and ignored.

6.20.15.1 Detailed Description

DPDE. ModuleID = 11.

6.20.15.2 Macro Definition Documentation

DPDE_LOG_CDR_SET_OFFSET_EC

```
#define DPDE_LOG_CDR_SET_OFFSET_EC (DPDE_LOG_BASE + 1)
```

Failed to set current offset of stream.

DPDE_LOG_SERIALIZE_ENTITY_NAME_EC

```
#define DPDE_LOG_SERIALIZE_ENTITY_NAME_EC (DPDE_LOG_BASE + 2)
```

Failed to serialize Entity Name parameter.

DPDE_LOG_SERIALIZE_TOPIC_NAME_EC

```
#define DPDE_LOG_SERIALIZE_TOPIC_NAME_EC (DPDE_LOG_BASE + 3)
```

Failed to serialize Topic Name parameter.

DPDE_LOG_SERIALIZE_TYPE_NAME_EC

```
#define DPDE_LOG_SERIALIZE_TYPE_NAME_EC (DPDE_LOG_BASE + 4)
```

Failed to serialize Type Name parameter.

DPDE_LOG_SERIALIZE_GUID_EC

```
#define DPDE_LOG_SERIALIZE_GUID_EC (DPDE_LOG_BASE + 5)
```

Failed to serialize GUID key.

DPDE_LOG_SERIALIZE_DEFAULT_UNICAST_EC

```
#define DPDE_LOG_SERIALIZE_DEFAULT_UNICAST_EC (DPDE_LOG_BASE + 6)
```

Failed to serialize Default Unicast Locator parameter.

DPDE_LOG_SERIALIZE_PROTOCOL_VERSION_EC

```
#define DPDE_LOG_SERIALIZE_PROTOCOL_VERSION_EC (DPDE_LOG_BASE + 7)
```

Failed to serialize Protocol Version parameter.

DPDE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC

```
#define DPDE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC (DPDE_LOG_BASE + 8)
```

Failed to deserialize Protocol Version parameter.

DPDE_LOG_DESERIALIZE_VENDOR_ID_EC

```
#define DPDE_LOG_DESERIALIZE_VENDOR_ID_EC (DPDE_LOG_BASE + 9)
```

Failed to deserialize Vendor ID parameter.

DPDE_LOG_SERIALIZE_VENDOR_ID_EC

```
#define DPDE_LOG_SERIALIZE_VENDOR_ID_EC (DPDE_LOG_BASE + 10)
```

Failed to serialize Vendor ID parameter.

DPDE_LOG_SERIALIZE_PRODUCT_VERSION_EC

```
#define DPDE_LOG_SERIALIZE_PRODUCT_VERSION_EC (DPDE_LOG_BASE + 11)
```

Failed to serialize Product Version parameter.

DPDE_LOG_SERIALIZE_LEASE_DURATION_EC

```
#define DPDE_LOG_SERIALIZE_LEASE_DURATION_EC (DPDE_LOG_BASE + 12)
```

Failed to serialize Lease Duration parameter.

DPDE_LOG_CREATE_PUBLISHER_EC

```
#define DPDE_LOG_CREATE_PUBLISHER_EC (DPDE_LOG_BASE + 13)
```

Failed to create Publisher for built-in endpoint DataWriters.

DPDE_LOG_CREATE_SUBSCRIBER_EC

```
#define DPDE_LOG_CREATE_SUBSCRIBER_EC (DPDE_LOG_BASE + 14)
```

Failed to create Subscriber for built-in endpoint DataReaders.

DPDE_LOG_CREATE_PARTICIPANT_DISCOVERY_EC

```
#define DPDE_LOG_CREATE_PARTICIPANT_DISCOVERY_EC (DPDE_LOG_BASE + 15)
```

Failed to create built-in endpoint for participant discovery after creating local user DomainParticipant.

DPDE_LOG_CREATE_PUBLICATION_DISCOVERY_EC

```
#define DPDE_LOG_CREATE_PUBLICATION_DISCOVERY_EC (DPDE_LOG_BASE + 16)
```

Failed to create built-in publication endpoint after creating local user DomainParticipant.

DPDE_LOG_CREATE_SUBSCRIPTION_DISCOVERY_EC

```
#define DPDE_LOG_CREATE_SUBSCRIPTION_DISCOVERY_EC (DPDE_LOG_BASE + 17)
```

Failed to create built-in subscription endpoint after creating local DomainParticipant.

DPDE_LOG_DELETE_SUBSCRIPTION_TOPIC_EC

```
#define DPDE_LOG_DELETE_SUBSCRIPTION_TOPIC_EC (DPDE_LOG_BASE + 18)
```

Failed to delete subscription built-in topic.

DPDE_LOG_DELETE_PUBLICATION_TOPIC_EC

```
#define DPDE_LOG_DELETE_PUBLICATION_TOPIC_EC (DPDE_LOG_BASE + 19)
```

Failed to delete publication built-in topic.

DPDE_LOG_DELETE_PARTICIPANT_TOPIC_EC

```
#define DPDE_LOG_DELETE_PARTICIPANT_TOPIC_EC (DPDE_LOG_BASE + 20)
```

Failed to delete participant built-in topic.

DPDE_LOG_DISPOSE_PARTICIPANT_EC

```
#define DPDE_LOG_DISPOSE_PARTICIPANT_EC (DPDE_LOG_BASE + 21)
```

Failed to dispose built-in participant when deleting DomainParticipant.

DPDE_LOG_ALLOCATE_DPDE_EC

```
#define DPDE_LOG_ALLOCATE_DPDE_EC (DPDE_LOG_BASE + 22)
```

Out of memory to allocate discovery plugin.

DPDE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC

```
#define DPDE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC (DPDE_LOG_BASE + 23)
```

Failed to serialize Builtin Endpoint Mask parameter.

DPDE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC

```
#define DPDE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC (DPDE_LOG_BASE + 24)
```

Failed to deserialize Builtin Endpoint Mask parameter.

DPDE_LOG_DESERIALIZE_UNKNOWN_PID_EC

```
#define DPDE_LOG_DESERIALIZE_UNKNOWN_PID_EC (DPDE_LOG_BASE + 25)
```

Failed to deserialize a parameter of unknown ID.

DPDE_LOG_ANNOUNCEMENT_EC

```
#define DPDE_LOG_ANNOUNCEMENT_EC (DPDE_LOG_BASE + 26)
```

Failed to write a dynamic participant discovery message.

DPDE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC

```
#define DPDE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC (DPDE_LOG_BASE + 27)
```

Failed to schedule an event to assert the next participant discovery announcement.

DPDE_LOG_ADVANCE_SN_EC

```
#define DPDE_LOG_ADVANCE_SN_EC (DPDE_LOG_BASE + 28)
```

Failed to advance the sequence number of a participant discovery announcement.

DPDE_LOG_ANNOUNCE_WRITE_EC

```
#define DPDE_LOG_ANNOUNCE_WRITE_EC (DPDE_LOG_BASE + 29)
```

Failed to write initial participant discovery announcement.

DPDE_LOG_SCHEDULE_FAST_ASSERTION_EC

```
#define DPDE_LOG_SCHEDULE_FAST_ASSERTION_EC (DPDE_LOG_BASE + 30)
```

Failed to schedule event for asserting participant discovery announcements.

DPDE_LOG_REGISTER_TYPE_EC

```
#define DPDE_LOG_REGISTER_TYPE_EC (DPDE_LOG_BASE + 32)
```

Failed to register a built-in type for discovery.

DPDE_LOG_CREATE_TOPIC_EC

```
#define DPDE_LOG_CREATE_TOPIC_EC (DPDE_LOG_BASE + 33)
```

Failed to create a topic for discovery.

DPDE_LOG_CREATE_WRITER_EC

```
#define DPDE_LOG_CREATE_WRITER_EC (DPDE_LOG_BASE + 34)
```

Failed to create a built-in DataWriter for discovery.

DPDE_LOG_CREATE_READER_EC

```
#define DPDE_LOG_CREATE_READER_EC (DPDE_LOG_BASE + 35)
```

Failed to create a built-in DataReader for discovery.

DPDE_LOG_ASSERT_REMOTE_PARTICIPANT_EC

```
#define DPDE_LOG_ASSERT_REMOTE_PARTICIPANT_EC (DPDE_LOG_BASE + 36)
```

Failed to assert and complete discovery of a remote participant.

DPDE_LOG_ENABLE_REMOTE_PARTICIPANT_EC

```
#define DPDE_LOG_ENABLE_REMOTE_PARTICIPANT_EC (DPDE_LOG_BASE + 37)
```

Failed to enable and complete discovery of a remote participant.

DPDE_LOG_REFRESH_REMOTE_PARTICIPANT_EC

```
#define DPDE_LOG_REFRESH_REMOTE_PARTICIPANT_EC (DPDE_LOG_BASE + 38)
```

Failed to refresh liveliness for a discovered remote participant.

DPDE_LOG_TAKE_PARTICIPANT_SAMPLE_EC

```
#define DPDE_LOG_TAKE_PARTICIPANT_SAMPLE_EC (DPDE_LOG_BASE + 39)
```

Failed to take a sample from a participant discovery DataReader.

DPDE_LOG_INVALID_PARTICIPANT_SAMPLE_EC

```
#define DPDE_LOG_INVALID_PARTICIPANT_SAMPLE_EC (DPDE_LOG_BASE + 40)
```

Received a participant discovery announcement with invalid state.

DPDE_LOG_RETURN_PARTICIPANT_SAMPLE_EC

```
#define DPDE_LOG_RETURN_PARTICIPANT_SAMPLE_EC (DPDE_LOG_BASE + 41)
```

Failed to return a loan on a participant discovery sample.

DPDE_LOG_DISPOSE_PUBLICATION_EC

```
#define DPDE_LOG_DISPOSE_PUBLICATION_EC (DPDE_LOG_BASE + 42)
```

Failed to dispose an instance for a publication.

DPDE_LOG_DISPOSE_SUBSCRIPTION_EC

```
#define DPDE_LOG_DISPOSE_SUBSCRIPTION_EC (DPDE_LOG_BASE + 43)
```

Failed to dispose an instance of a subscription.

DPDE_LOG_ASSERT_REMOTE_PUBLICATION_EC

```
#define DPDE_LOG_ASSERT_REMOTE_PUBLICATION_EC (DPDE_LOG_BASE + 44)
```

Failed to assert and complete discovery of a remote publication.

DPDE_LOG_ASSERT_REMOTE_SUBSCRIPTION_EC

```
#define DPDE_LOG_ASSERT_REMOTE_SUBSCRIPTION_EC (DPDE_LOG_BASE + 45)
```

Failed to assert and complete discovery of a remote subscription. Logs the topic name.

DPDE_LOG_TAKE_PUBLICATION_SAMPLE_EC

```
#define DPDE_LOG_TAKE_PUBLICATION_SAMPLE_EC (DPDE_LOG_BASE + 46)
```

Failed to take a sample from a publication discovery DataReader.

DPDE_LOG_REMOVE_REMOTE_PUBLICATION_EC

```
#define DPDE_LOG_REMOVE_REMOTE_PUBLICATION_EC (DPDE_LOG_BASE + 47)
```

Failed to dispose or unregister a remote publication instance.

DPDE_LOG_INVALID_PUBLICATION_SAMPLE_EC

```
#define DPDE_LOG_INVALID_PUBLICATION_SAMPLE_EC (DPDE_LOG_BASE + 48)
```

Received a publication discovery announcement with invalid state.

DPDE_LOG_RETURN_PUBLICATION_SAMPLE_EC

```
#define DPDE_LOG_RETURN_PUBLICATION_SAMPLE_EC (DPDE_LOG_BASE + 49)
```

Failed to return a loan on a publication discovery sample.

DPDE_LOG_TAKE_SUBSCRIPTION_SAMPLE_EC

```
#define DPDE_LOG_TAKE_SUBSCRIPTION_SAMPLE_EC (DPDE_LOG_BASE + 51)
```

Failed to take a sample from a subscription discovery DataReader.

DPDE_LOG_INVALID_SUBSCRIPTION_SAMPLE_EC

```
#define DPDE_LOG_INVALID_SUBSCRIPTION_SAMPLE_EC (DPDE_LOG_BASE + 52)
```

Received a subscription discovery sample with invalid state.

DPDE_LOG_RETURN_SUBSCRIPTION_SAMPLE_EC

```
#define DPDE_LOG_RETURN_SUBSCRIPTION_SAMPLE_EC (DPDE_LOG_BASE + 53)
```

Failed to return a loan on a subscription discovery sample.

DPDE_LOG_REMOVE_REMOTE_SUBSCRIPTION_EC

```
#define DPDE_LOG_REMOVE_REMOTE_SUBSCRIPTION_EC (DPDE_LOG_BASE + 54)
```

Failed to dispose or unregister a remote subscription instance.

DPDE_LOG_LOCATORS_FULL_EC

```
#define DPDE_LOG_LOCATORS_FULL_EC (DPDE_LOG_BASE + 55)
```

A locator sequence is full, the remaining locators of the specified kind are dropped.

DPDE_LOG_DESERIALIZE_LOCATOR_EC

```
#define DPDE_LOG_DESERIALIZE_LOCATOR_EC (DPDE_LOG_BASE + 56)
```

Failed to deserialize a locator of the specified kind.

DPDE_LOG_UNSUPPORTED_LOCATOR_EC

```
#define DPDE_LOG_UNSUPPORTED_LOCATOR_EC (DPDE_LOG_BASE + 57)
```

The locator kind is not supported by any transport registered with the domain participant and is dropped.

DPDE_LOG_ADD_PEER_EC

```
#define DPDE_LOG_ADD_PEER_EC (DPDE_LOG_BASE + 58)
```

Failed to add the specified entity as a peer.

DPDE_LOG_REMOVE_PEER_EC

```
#define DPDE_LOG_REMOVE_PEER_EC (DPDE_LOG_BASE + 59)
```

Failed to add the specified entity as a peer.

DPDE_LOG_SERIALIZE_PRESENTATION_EC

```
#define DPDE_LOG_SERIALIZE_PRESENTATION_EC (DPDE_LOG_BASE + 60)
```

Failed to serialize Presentation parameter.

DPDE_LOG_UNREGISTER_TYPE_EC

```
#define DPDE_LOG_UNREGISTER_TYPE_EC (DPDE_LOG_BASE + 61)
```

Failed to unregister a built-in type for discovery.

DPDE_LOG_ADD_REMOTE_PARTICIPANT_ROUTES_FAILED_EC

```
#define DPDE_LOG_ADD_REMOTE_PARTICIPANT_ROUTES_FAILED_EC (DPDE_LOG_BASE + 62)
```

Failed to add routes to a remote participant.

DPDE_LOG_DELETE_REMOTE_PARTICIPANT_ROUTES_FAILED_EC

```
#define DPDE_LOG_DELETE_REMOTE_PARTICIPANT_ROUTES_FAILED_EC (DPDE_LOG_BASE + 63)
```

Failed to delete routes to a remote participant.

DPDE_LOG_REMOVE_REMOTE_PARTICIPANT_EC

```
#define DPDE_LOG_REMOVE_REMOTE_PARTICIPANT_EC (DPDE_LOG_BASE + 64)
```

Failed remove a remote participant.

DPDE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC

```
#define DPDE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC (DPDE_LOG_BASE + 65)
```

A participant with incompatible checksum was discovered and ignored.

6.20.16 APPGEN

APPGEN. ModuleID = 13.

Macros

- #define APPGEN_LOG_ALLOCATE_EC (APPGEN_LOG_BASE + 1)
Failed to allocate memory for an Application Generator interface.
- #define APPGEN_LOG_COPY_ENTITY_NAME_EC (APPGEN_LOG_BASE + 2)
Cannot copy a name into an entity name. Name too long.
- #define APPGEN_LOG_CREATE_ENTITY_NAME_EC (APPGEN_LOG_BASE + 3)
Cannot generate an entity name. Name too long.
- #define APPGEN_MULTIPPLICITY_EC (APPGEN_LOG_BASE + 4)
Cannot create entity name because multiplicity is wrong (0)
- #define APPGEN_FACTORY_REGISTER_EC (APPGEN_LOG_BASE + 5)
Cannot register a factory.
- #define APPGEN_LOG_FACTORY_TOPIC_NOT_FOUND_EC (APPGEN_LOG_BASE + 6)
Topic not found. Wrong topic name in writer configuration.
- #define APPGEN_LOG_DDS_ENTITY_EC (APPGEN_LOG_BASE + 7)
Failed to create/delete/lookup DDS entity.
- #define APPGEN_LOG_API_ERROR_EC (APPGEN_LOG_BASE + 8)
A call to a DDS api returned an error.
- #define APPGEN_LOG_APP_NAME_ERROR_EC (APPGEN_LOG_BASE + 9)
Wrong application or participant name. Not a qualified name.
- #define APPGEN_LOG_LIB_NOT_FOUND_EC (APPGEN_LOG_BASE + 10)
Participant cannot be created because the library name is not found in the configuration.
- #define APPGEN_APP_PARTICIPANT_NOT_FOUND_EC (APPGEN_LOG_BASE + 11)
Participant cannot be created because the participant name is not found in the configuration.
- #define APPGEN_LOG_APP_CONFIG_NOT_CORRECT_EC (APPGEN_LOG_BASE + 12)
Application generation configuration is not correct or not consistent.
- #define APPGEN_LOG_APP_CONFIG_POINTER_NOT_CORRECT_EC (APPGEN_LOG_BASE + 13)
Application generation configuration is not correct or not consistent.
- #define APPGEN_FACTORY_UNREGISTER_EC (APPGEN_LOG_BASE + 14)
Cannot unregister a factory.

6.20.16.1 Detailed Description

APPGEN. ModuleID = 13.

6.20.16.2 Macro Definition Documentation

APPGEN_LOG_ALLOCATE_EC

```
#define APPGEN_LOG_ALLOCATE_EC (APPGEN_LOG_BASE + 1)
```

Failed to allocate memory for an Application Generator interface.

APPGEN_LOG_COPY_ENTITY_NAME_EC

```
#define APPGEN_LOG_COPY_ENTITY_NAME_EC (APPGEN_LOG_BASE + 2)
```

Cannot copy a name into an entity name. Name too long.

APPGEN_LOG_CREATE_ENTITY_NAME_EC

```
#define APPGEN_LOG_CREATE_ENTITY_NAME_EC (APPGEN_LOG_BASE + 3)
```

Cannot generate an entity name. Name too long.

APPGEN_MULTIPLICITY_EC

```
#define APPGEN_MULTIPLICITY_EC (APPGEN_LOG_BASE + 4)
```

Cannot create entity name because multiplicity is wrong (0)

APPGEN_FACTORY_REGISTER_EC

```
#define APPGEN_FACTORY_REGISTER_EC (APPGEN_LOG_BASE + 5)
```

Cannot register a factory.

APPGEN_LOG_FACTORY_TOPIC_NOT_FOUND_EC

```
#define APPGEN_LOG_FACTORY_TOPIC_NOT_FOUND_EC (APPGEN_LOG_BASE + 6)
```

Topic not found. Wrong topic name in writer configuration.

APPGEN_LOG_DDS_ENTITY_EC

```
#define APPGEN_LOG_DDS_ENTITY_EC (APPGEN_LOG_BASE + 7)
```

Failed to create/delete/lookup DDS entity.

APPGEN_LOG_API_ERROR_EC

```
#define APPGEN_LOG_API_ERROR_EC (APPGEN_LOG_BASE + 8)
```

A call to a DDS api returned an error.

APPGEN_LOG_APP_NAME_ERROR_EC

```
#define APPGEN_LOG_APP_NAME_ERROR_EC (APPGEN_LOG_BASE + 9)
```

Wrong application or participant name. Not a qualified name.

APPGEN_LOG_LIB_NOT_FOUND_EC

```
#define APPGEN_LOG_LIB_NOT_FOUND_EC (APPGEN_LOG_BASE + 10)
```

Participant cannot be created because the library name is not found in the configuration.

APPGEN_APP_PARTICIPANT_NOT_FOUND_EC

```
#define APPGEN_APP_PARTICIPANT_NOT_FOUND_EC (APPGEN_LOG_BASE + 11)
```

Participant cannot be created because the participant name is not found in the configuration.

APPGEN_LOG_APP_CONFIG_NOT_CORRECT_EC

```
#define APPGEN_LOG_APP_CONFIG_NOT_CORRECT_EC (APPGEN_LOG_BASE + 12)
```

Application generation configuration is not correct or not consistent.

APPGEN_LOG_APP_CONFIG_POINTER_NOT_CORRECT_EC

```
#define APPGEN_LOG_APP_CONFIG_POINTER_NOT_CORRECT_EC (APPGEN_LOG_BASE + 13)
```

Application generation configuration is not correct or not consistent.

APPGEN_FACTORY_UNREGISTER_EC

```
#define APPGEN_FACTORY_UNREGISTER_EC (APPGEN_LOG_BASE + 14)
```

Cannot unregister a factory.

6.20.17 DDS_FILTER

DDS Filter. ModuleID = 15.

Macros

- #define [DDS_FILTER_LOG_FILTER_CLASS_RECORD](#) 1
Database record for a Filter class.
- #define [DDS_FILTER_LOG_COMPILED_FILTER_RECORD](#) 2
Database record for a compiled filter.
- #define [DDS_FILTER_LOG_READER_RECORD](#) 3
Database record for a reader entry.
- #define [DDS_FILTER_LOG_ROUTE_RECORD](#) 4
Database record for a route entry.
- #define [DDS_FILTER_LOG_RECORD_SELECT_EC](#) (DDS_FILTER_LOG_BASE + 1)
A database select on the specified record kind failed.
- #define [DDS_FILTER_LOG_RECORD_CREATE_EC](#) (DDS_FILTER_LOG_BASE + 2)
Failed to create a database record of the specified kind.
- #define [DDS_FILTER_LOG_RECORD_DELETE_EC](#) (DDS_FILTER_LOG_BASE + 3)
Failed to delete a database record of the specified kind.
- #define [DDS_FILTER_LOG_RECORD_INSERT_EC](#) (DDS_FILTER_LOG_BASE + 4)
Failed to insert a database record of the specified kind.
- #define [DDS_FILTER_LOG_RECORD_REMOVE_EC](#) (DDS_FILTER_LOG_BASE + 5)
Failed to remove a database record of the specified kind.
- #define [DDS_FILTER_LOG_TABLE_CREATE_EC](#) (DDS_FILTER_LOG_BASE + 6)
Failed to create a database table of the specified kind.
- #define [DDS_FILTER_LOG_TABLE_DELETE_EC](#) (DDS_FILTER_LOG_BASE + 7)
Failed to delete a database table of the specified kind.
- #define [DDS_FILTER_LOG_FILTER_CREATE_INSTANCE_EC](#) (DDS_FILTER_LOG_BASE + 10)
Failed to create an instance of the specified filter class.
- #define [DDS_FILTER_LOG_FILTER_DELETE_INSTANCE_EC](#) (DDS_FILTER_LOG_BASE + 11)
Failed to delete an instance of the specified filter class.
- #define [DDS_FILTER_LOG_FILTER_COMPILE_EC](#) (DDS_FILTER_LOG_BASE + 12)
Failed to compile a filter of the specified class.
- #define [DDS_FILTER_LOG_FILTER_EVALUATE_EC](#) (DDS_FILTER_LOG_BASE + 13)
Failed to evaluate a filter of the specified class.
- #define [DDS_FILTER_LOG_WRITER_DETACH_EC](#) (DDS_FILTER_LOG_BASE + 14)
Failed to detach a writer because it still has remote readers attached.
- #define [DDS_FILTER_LOG_FILTER_EXPRESSION_MAX_COUNT_EC](#) (DDS_FILTER_LOG_BASE + 15)
No more filters can be compiled because the configured maximum number of filter expressions has been reached.
- #define [DDS_FILTER_LOG_FILTER_CLASS_REGISTER_EC](#) (DDS_FILTER_LOG_BASE + 20)
Failed to register a filter class because one with the same name has already been registered or because the provided interface did not contain the required function pointers.
- #define [DDS_FILTER_LOG_FILTER_CLASS_NOT_FOUND_EC](#) (DDS_FILTER_LOG_BASE + 21)
Failed to find a filter class registered with the specified name.
- #define [DDS_FILTER_LOG_FILTER_CLASS_UNREGISTER_EC](#) (DDS_FILTER_LOG_BASE + 22)
Failed to unregister a filter class because it is still in use.
- #define [DDS_FILTER_LOG_CONTENT_FILTERED_TOPIC_NAME_STRING](#) 1
Content filter topic name string.
- #define [DDS_FILTER_LOG_RELATED_TOPIC_NAME_STRING](#) 2
Content filter related topic name string.

- `#define DDS_FILTER_LOG_FILTER_CLASS_NAME_STRING 3`
Filter class name string.
- `#define DDS_FILTER_LOG_FILTER_EXPRESSION_STRING 4`
Filter expression string.
- `#define DDS_FILTER_LOG_FILTER_EXPRESSION_PARAMETER_STRING 5`
Filter expression parameter string.
- `#define DDS_FILTER_LOG_CONTENT_FILTER_NAME_STRING 6`
Content filter QoS expression name string.
- `#define DDS_FILTER_LOG_STRING_ASSERT_EC (DDS_FILTER_LOG_BASE + 30)`
Failed to assert a string in the string manager of a specified kind.
- `#define DDS_FILTER_LOG_STRING_DELETE_EC (DDS_FILTER_LOG_BASE + 31)`
Failed to delete a string in the string manager of a specified kind.
- `#define DDS_FILTER_LOG_CONTENT_FILTER_NAME_TOO_LONG_EC (DDS_FILTER_LOG_BASE + 32)`
Failed to construct a content filter topic name from the combination of the related topic name and the filter name because it exceeded the maximum topic name length.
- `#define DDS_FILTER_LOG_STRING_MANAGER_CREATE_EC (DDS_FILTER_LOG_BASE + 33)`
Failed to create a string manager for a specified string kind, max_string_size, and memory_allocation.
- `#define DDS_FILTER_LOG_FILTER_PARAMETER_COUNT_EC (DDS_FILTER_LOG_BASE + 34)`
Failed to deserialize a content filter property because it contained too many expression parameters.
- `#define DDS_FILTER_LOG_HEAP_ALLOC_EC (DDS_FILTER_LOG_BASE + 50)`
Failed to allocate heap memory for a buffer of a given type.
- `#define DDS_FILTER_LOG_FILTER_PLUGIN_INIT_EC (DDS_FILTER_LOG_BASE + 51)`
Failed to initialize a filter plugin.
- `#define DDS_FILTER_LOG_FILTER_PLUGIN_FINALIZE_EC (DDS_FILTER_LOG_BASE + 52)`
Failed to finalize a filter plugin.
- `#define DDS_FILTER_LOG_FILTER_PLUGIN_FACTORY_IN_USE_EC (DDS_FILTER_LOG_BASE + 53)`
Failed to finalize a filter plugin factory because it is still in use.
- `#define DDS_FILTER_LOG_WRITER_FILTER_COMPILE_EC (DDS_FILTER_LOG_BASE + 61)`
A writer filter failed to compile a filter for a discovered reader.
- `#define DDS_FILTER_LOG_WRITER_FILTER_REMOVE_READER_EC (DDS_FILTER_LOG_BASE + 62)`
Failed to remove a reader from a writer filter because it still has active routes.
- `#define DDS_FILTER_LOG_WRITER_FILTER_SEND_GAP_EC (DDS_FILTER_LOG_BASE + 63)`
Writer failed to send a reader a GAP to indicate filtered data.
- `#define DDS_FILTER_LOG_WRITER_SERIALIZE_INLINE_QOS_EC (DDS_FILTER_LOG_BASE + 64)`
Writer failed to serialize the content filter info into the in-line QoS.
- `#define DDS_FILTER_LOG_WRITER_FILTER_EVALUATE_OUT_OF_ORDER_EC (DDS_FILTER_LOG_BASE + 65)`
Writer filter was evaluated out of order which requires invalidating a previously results.
- `#define DDS_FILTER_LOG_WRITER_FILTER_STORE_RESULT_EC (DDS_FILTER_LOG_BASE + 66)`
Writer filter unexpectedly failed to store the result of a filter evaluation.
- `#define DDS_FILTER_LOG_SQL_UNSUPPORTED_TYPE_CODE_EC (DDS_FILTER_LOG_BASE + 100)`
Unsupported type code for SQL filtering.
- `#define DDS_FILTER_LOG_SQL_UNEXPECTED_TOKEN_EC (DDS_FILTER_LOG_BASE + 101)`
Found an unexpected token while parsing an SQL filter expression.
- `#define DDS_FILTER_LOG_SQL_MAX_PREDICATES_EXCEEDED_EC (DDS_FILTER_LOG_BASE + 102)`
Failed to parse an SQL filter expression because the number of predicates in the expression exceeded the configured maximum.
- `#define DDS_FILTER_LOG_SQL_INVALID_TOKEN_EC (DDS_FILTER_LOG_BASE + 103)`

Found an invalid token while parsing an SQL filter expression.

- #define `DDS_FILTER_LOG_SQL_PARSE_PARAMETER_EC` (`DDS_FILTER_LOG_BASE` + 104)

Failed to parse a parameter in an SQL filter expression.

- #define `DDS_FILTER_LOG_SQL_FIELD_NOT_FOUND_EC` (`DDS_FILTER_LOG_BASE` + 105)

Failed to parse an SQL filter expression because a given field name did not match any field in the type code of the data being filtered.

- #define `DDS_FILTER_LOG_SQL_UNSUPPORTED_FIELD_TYPE_EC` (`DDS_FILTER_LOG_BASE` + 106)

A field specified in an SQL filter expression is not a supported type for use in an SQL filter.

- #define `DDS_FILTER_LOG_SQL_ENUM_LABEL_NOT_FOUND_EC` (`DDS_FILTER_LOG_BASE` + 107)

Failed to find a given enum label in an SQL filter expression.

- #define `DDS_FILTER_LOG_SQL_INCOMPATIBLE_TYPES_EC` (`DDS_FILTER_LOG_BASE` + 108)

Comparison between two incompatible types in an SQL filter expression.

- #define `DDS_FILTER_LOG_SQL_INTEGER_OUT_OF_RANGE_EC` (`DDS_FILTER_LOG_BASE` + 109)

Integer constant in an SQL expression is out of range for the type.

6.20.17.1 Detailed Description

DDS Filter. ModuleID = 15.

6.20.17.2 Macro Definition Documentation

DDS_FILTER_LOG_FILTER_CLASS_RECORD

```
#define DDS_FILTER_LOG_FILTER_CLASS_RECORD 1
```

Database record for a Filter class.

DDS_FILTER_LOG_COMPILED_FILTER_RECORD

```
#define DDS_FILTER_LOG_COMPILED_FILTER_RECORD 2
```

Database record for a compiled filter.

DDS_FILTER_LOG_READER_RECORD

```
#define DDS_FILTER_LOG_READER_RECORD 3
```

Database record for a reader entry.

DDS_FILTER_LOG_ROUTE_RECORD

```
#define DDS_FILTER_LOG_ROUTE_RECORD 4
```

Database record for a route entry.

DDS_FILTER_LOG_RECORD_SELECT_EC

```
#define DDS_FILTER_LOG_RECORD_SELECT_EC (DDS_FILTER_LOG_BASE + 1)
```

A database select on the specified record kind failed.

DDS_FILTER_LOG_RECORD_CREATE_EC

```
#define DDS_FILTER_LOG_RECORD_CREATE_EC (DDS_FILTER_LOG_BASE + 2)
```

Failed to create a database record of the specified kind.

DDS_FILTER_LOG_RECORD_DELETE_EC

```
#define DDS_FILTER_LOG_RECORD_DELETE_EC (DDS_FILTER_LOG_BASE + 3)
```

Failed to delete a database record of the specified kind.

DDS_FILTER_LOG_RECORD_INSERT_EC

```
#define DDS_FILTER_LOG_RECORD_INSERT_EC (DDS_FILTER_LOG_BASE + 4)
```

Failed to insert a database record of the specified kind.

DDS_FILTER_LOG_RECORD_REMOVE_EC

```
#define DDS_FILTER_LOG_RECORD_REMOVE_EC (DDS_FILTER_LOG_BASE + 5)
```

Failed to remove a database record of the specified kind.

DDS_FILTER_LOG_TABLE_CREATE_EC

```
#define DDS_FILTER_LOG_TABLE_CREATE_EC (DDS_FILTER_LOG_BASE + 6)
```

Failed to create a database table of the specified kind.

DDS_FILTER_LOG_TABLE_DELETE_EC

```
#define DDS_FILTER_LOG_TABLE_DELETE_EC (DDS_FILTER_LOG_BASE + 7)
```

Failed to delete a database table of the specified kind.

DDS_FILTER_LOG_FILTER_CREATE_INSTANCE_EC

```
#define DDS_FILTER_LOG_FILTER_CREATE_INSTANCE_EC (DDS_FILTER_LOG_BASE + 10)
```

Failed to create an instance of the specified filter class.

DDS_FILTER_LOG_FILTER_DELETE_INSTANCE_EC

```
#define DDS_FILTER_LOG_FILTER_DELETE_INSTANCE_EC (DDS_FILTER_LOG_BASE + 11)
```

Failed to delete an instance of the specified filter class.

DDS_FILTER_LOG_FILTER_COMPILE_EC

```
#define DDS_FILTER_LOG_FILTER_COMPILE_EC (DDS_FILTER_LOG_BASE + 12)
```

Failed to compile a filter of the specified class.

For a reader, it is an error to not be able to compile a filter which was explicitly configured for it. For a writer performing writer-side filtering, it is only a warning that it will not be able to filter the data for the reader because it was not able to compile the filter.

DDS_FILTER_LOG_FILTER_EVALUATE_EC

```
#define DDS_FILTER_LOG_FILTER_EVALUATE_EC (DDS_FILTER_LOG_BASE + 13)
```

Failed to evaluate a filter of the specified class.

DDS_FILTER_LOG_WRITER_DETACH_EC

```
#define DDS_FILTER_LOG_WRITER_DETACH_EC (DDS_FILTER_LOG_BASE + 14)
```

Failed to detach a writer because it still has remote readers attached.

DDS_FILTER_LOG_FILTER_EXPRESSION_MAX_COUNT_EC

```
#define DDS_FILTER_LOG_FILTER_EXPRESSION_MAX_COUNT_EC (DDS_FILTER_LOG_BASE + 15)
```

No more filters can be compiled because the configured maximum number of filter expressions has been reached.

DDS_FILTER_LOG_FILTER_CLASS_REGISTER_EC

```
#define DDS_FILTER_LOG_FILTER_CLASS_REGISTER_EC (DDS_FILTER_LOG_BASE + 20)
```

Failed to register a filter class because one with the same name has already been registered or because the provided interface did not contain the required function pointers.

DDS_FILTER_LOG_FILTER_CLASS_NOT_FOUND_EC

```
#define DDS_FILTER_LOG_FILTER_CLASS_NOT_FOUND_EC (DDS_FILTER_LOG_BASE + 21)
```

Failed to find a filter class registered with the specified name.

DDS_FILTER_LOG_FILTER_CLASS_UNREGISTER_EC

```
#define DDS_FILTER_LOG_FILTER_CLASS_UNREGISTER_EC (DDS_FILTER_LOG_BASE + 22)
```

Failed to unregister a filter class because it is still in use.

DDS_FILTER_LOG_CONTENT_FILTERED_TOPIC_NAME_STRING

```
#define DDS_FILTER_LOG_CONTENT_FILTERED_TOPIC_NAME_STRING 1
```

Content filter topic name string.

DDS_FILTER_LOG_RELATED_TOPIC_NAME_STRING

```
#define DDS_FILTER_LOG_RELATED_TOPIC_NAME_STRING 2
```

Content filter related topic name string.

DDS_FILTER_LOG_FILTER_CLASS_NAME_STRING

```
#define DDS_FILTER_LOG_FILTER_CLASS_NAME_STRING 3
```

Filter class name string.

DDS_FILTER_LOG_FILTER_EXPRESSION_STRING

```
#define DDS_FILTER_LOG_FILTER_EXPRESSION_STRING 4
```

Filter expression string.

DDS_FILTER_LOG_FILTER_EXPRESSION_PARAMETER_STRING

```
#define DDS_FILTER_LOG_FILTER_EXPRESSION_PARAMETER_STRING 5
```

Filter expression parameter string.

DDS_FILTER_LOG_CONTENT_FILTER_NAME_STRING

```
#define DDS_FILTER_LOG_CONTENT_FILTER_NAME_STRING 6
```

Content filter QoS expression name string.

DDS_FILTER_LOG_STRING_ASSERT_EC

```
#define DDS_FILTER_LOG_STRING_ASSERT_EC (DDS_FILTER_LOG_BASE + 30)
```

Failed to assert a string in the string manager of a specified kind.

For string that are configured by the user in their QoS, this is an error that indicates that the resource limits configured were not sufficient to hold the string. For content filter's received through discovery, this is a warning that the content filter will be dropped because of inadequate resources. This could either be because the string is too long or because too many strings of that kind have been configured.

DDS_FILTER_LOG_STRING_DELETE_EC

```
#define DDS_FILTER_LOG_STRING_DELETE_EC (DDS_FILTER_LOG_BASE + 31)
```

Failed to delete a string in the string manager of a specified kind.

DDS_FILTER_LOG_CONTENT_FILTER_NAME_TOO_LONG_EC

```
#define DDS_FILTER_LOG_CONTENT_FILTER_NAME_TOO_LONG_EC (DDS_FILTER_LOG_BASE + 32)
```

Failed to construct a content filter topic name from the combination of the related topic name and the filter name because it exceeded the maximum topic name length.

To maintain compatibility with the RTPS specification, an internal name is constructed to represent a content filter on the wire. This name is constructed by concatenating the related topic name, two colons, and the filter name. If the resulting name exceeds the maximum topic name length, then either the related topic name or the filter name must be shortened.

DDS_FILTER_LOG_STRING_MANAGER_CREATE_EC

```
#define DDS_FILTER_LOG_STRING_MANAGER_CREATE_EC (DDS_FILTER_LOG_BASE + 33)
```

Failed to create a string manager for a specified string kind, max_string_size, and memory_allocation.

DDS_FILTER_LOG_FILTER_PARAMETER_COUNT_EC

```
#define DDS_FILTER_LOG_FILTER_PARAMETER_COUNT_EC (DDS_FILTER_LOG_BASE + 34)
```

Failed to deserialize a content filter property because it contained too many expression parameters.

DDS_FILTER_LOG_HEAP_ALLOC_EC

```
#define DDS_FILTER_LOG_HEAP_ALLOC_EC (DDS_FILTER_LOG_BASE + 50)
```

Failed to allocate heap memory for a buffer of a given type.

DDS_FILTER_LOG_FILTER_PLUGIN_INIT_EC

```
#define DDS_FILTER_LOG_FILTER_PLUGIN_INIT_EC (DDS_FILTER_LOG_BASE + 51)
```

Failed to initialize a filter plugin.

DDS_FILTER_LOG_FILTER_PLUGIN_FINALIZE_EC

```
#define DDS_FILTER_LOG_FILTER_PLUGIN_FINALIZE_EC (DDS_FILTER_LOG_BASE + 52)
```

Failed to finalize a filter plugin.

DDS_FILTER_LOG_FILTER_PLUGIN_FACTORY_IN_USE_EC

```
#define DDS_FILTER_LOG_FILTER_PLUGIN_FACTORY_IN_USE_EC (DDS_FILTER_LOG_BASE + 53)
```

Failed to finalize a filter plugin factory because it is still in use.

DDS_FILTER_LOG_WRITER_FILTER_COMPILE_EC

```
#define DDS_FILTER_LOG_WRITER_FILTER_COMPILE_EC (DDS_FILTER_LOG_BASE + 61)
```

A writer filter failed to compile a filter for a discovered reader.

This is a warning that the writer will not be able to filter the data for the reader because it was not able to compile the filter.

DDS_FILTER_LOG_WRITER_FILTER_REMOVE_READER_EC

```
#define DDS_FILTER_LOG_WRITER_FILTER_REMOVE_READER_EC (DDS_FILTER_LOG_BASE + 62)
```

Failed to remove a reader from a writer filter because it still has active routes.

DDS_FILTER_LOG_WRITER_FILTER_SEND_GAP_EC

```
#define DDS_FILTER_LOG_WRITER_FILTER_SEND_GAP_EC (DDS_FILTER_LOG_BASE + 63)
```

Writer failed to send a reader a GAP to indicate filtered data.

DDS_FILTER_LOG_WRITER_SERIALIZE_INLINE_QOS_EC

```
#define DDS_FILTER_LOG_WRITER_SERIALIZE_INLINE_QOS_EC (DDS_FILTER_LOG_BASE + 64)
```

Writer failed to serialize the content filter info into the in-line QoS.

DDS_FILTER_LOG_WRITER_FILTER_EVALUATE_OUT_OF_ORDER_EC

```
#define DDS_FILTER_LOG_WRITER_FILTER_EVALUATE_OUT_OF_ORDER_EC (DDS_FILTER_LOG_BASE + 65)
```

Writer filter was evaluated out of order which requires invalidating a previously results.

DDS_FILTER_LOG_WRITER_FILTER_STORE_RESULT_EC

```
#define DDS_FILTER_LOG_WRITER_FILTER_STORE_RESULT_EC (DDS_FILTER_LOG_BASE + 66)
```

Writer filter unexpectedly failed to store the result of a filter evaluation.

DDS_FILTER_LOG_SQL_UNSUPPORTED_TYPE_CODE_EC

```
#define DDS_FILTER_LOG_SQL_UNSUPPORTED_TYPE_CODE_EC (DDS_FILTER_LOG_BASE + 100)
```

Unsupported type code for SQL filtering.

The DDS SQL filter only supports filtering on structs with the C or C++ type binding. The flat data language binding is not supported.

DDS_FILTER_LOG_SQL_UNEXPECTED_TOKEN_EC

```
#define DDS_FILTER_LOG_SQL_UNEXPECTED_TOKEN_EC (DDS_FILTER_LOG_BASE + 101)
```

Found an unexpected token while parsing an SQL filter expression.

This error indicate that the SQL parser found a token that it did not expect at the given index in the expression. This could occur for various syntax errors in the expression. Check that the expression follows the supported SQL syntax.

DDS_FILTER_LOG_SQL_MAX_PREDICATES_EXCEEDED_EC

```
#define DDS_FILTER_LOG_SQL_MAX_PREDICATES_EXCEEDED_EC (DDS_FILTER_LOG_BASE + 102)
```

Failed to parse an SQL filter expression because the number of predicates in the expression exceeded the configured maximum.

The maximum number of predicates in an SQL filter expression is configured with `sql_predicate_max_count_per_` expression in the DomainParticipant's [DDS_DomainParticipantQos](#) filter resource limits.

DDS_FILTER_LOG_SQL_INVALID_TOKEN_EC

```
#define DDS_FILTER_LOG_SQL_INVALID_TOKEN_EC (DDS_FILTER_LOG_BASE + 103)
```

Found an invalid token while parsing an SQL filter expression.

This error indicates that the SQL parser found a token that it did not recognize at the given index in the expression. This could occur for various syntax errors in the expression. Check that the expression follows the supported SQL syntax.

DDS_FILTER_LOG_SQL_PARSE_PARAMETER_EC

```
#define DDS_FILTER_LOG_SQL_PARSE_PARAMETER_EC (DDS_FILTER_LOG_BASE + 104)
```

Failed to parse a parameter in an SQL filter expression.

DDS_FILTER_LOG_SQL_FIELD_NOT_FOUND_EC

```
#define DDS_FILTER_LOG_SQL_FIELD_NOT_FOUND_EC (DDS_FILTER_LOG_BASE + 105)
```

Failed to parse an SQL filter expression because a given field name did not match any field in the type code of the data being filtered.

This error indicates a field name in an SQL filter expression did not correspond to any field in the type code of the data being filtered. This could also occur if the specified field is not a field which can be used in an SQL filter. Only primitive, top-level, non-optional fields can be used in an SQL expression.

DDS_FILTER_LOG_SQL_UNSUPPORTED_FIELD_TYPE_EC

```
#define DDS_FILTER_LOG_SQL_UNSUPPORTED_FIELD_TYPE_EC (DDS_FILTER_LOG_BASE + 106)
```

A field specified in an SQL filter expression is not a supported type for use in an SQL filter.

This error indicates that a field in an SQL filter expression is not a supported type for use in an SQL filter. Only primitive fields can be used in an SQL filter expression and strings are not supported.

DDS_FILTER_LOG_SQL_ENUM_LABEL_NOT_FOUND_EC

```
#define DDS_FILTER_LOG_SQL_ENUM_LABEL_NOT_FOUND_EC (DDS_FILTER_LOG_BASE + 107)
```

Failed to find a given enum label in an SQL filter expression.

This error indicates that an enum label specified in an SQL filter expression did not correspond to any label in type code of the enum type.

DDS_FILTER_LOG_SQL_INCOMPATIBLE_TYPES_EC

```
#define DDS_FILTER_LOG_SQL_INCOMPATIBLE_TYPES_EC (DDS_FILTER_LOG_BASE + 108)
```

Comparison between two incompatible types in an SQL filter expression.

DDS_FILTER_LOG_SQL_INTEGER_OUT_OF_RANGE_EC

```
#define DDS_FILTER_LOG_SQL_INTEGER_OUT_OF_RANGE_EC (DDS_FILTER_LOG_BASE + 109)
```

Integer constant in an SQL expression is out of range for the type.

The default integer type in an SQL expression is 32-bit signed integer. If additional range is needed, then the constant can be suffixed with "L" to indicate that it is a 64-bit signed integer.

7 Class Documentation

7.1 CDR_Stream_t Struct Reference

Byte stream abstraction for CDR serialization and deserialization.

```
#include <cdr_stream.h>
```

7.1.1 Detailed Description

Byte stream abstraction for CDR serialization and deserialization.

7.2 DDS_BuiltinTopicKey_t Struct Reference

The key type of the built-in topic types.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- DDS_BUILTIN_TOPIC_KEY_TYPE_NATIVE [value](#) [DDS_BUILTIN_TOPIC_KEY_TYPE_NATIVE_LENGTH]
An array of four integers that uniquely represents a remote [DDSEntity](#).

7.2.1 Detailed Description

The key type of the built-in topic types.

Each remote [DDSEntity](#) to be discovered can be uniquely identified by this key. This is the key of all the built-in topic data types.

The value of element DDS_BUILTIN_TOPIC_KEY_OBJECT_ID is the object ID of the remote entity. In Dynamic Participant/Static Endpoint discovery, this must be set correctly for each remote [DDSDataReader](#) and [DDSDataWriter](#) that an application intends to discover.

For example, to configure a remote [DDSDataWriter](#) with object ID 100 by setting the key in the [DDS_PublicationBuiltinTopicData](#)↔:

```
remote_pub_data.key.value[DDS_BUILTIN_TOPIC_KEY_OBJECT_ID] = 100;
```

See also

[DDS_ParticipantBuiltinTopicData](#)
[DDS_PublicationBuiltinTopicData](#)
[DDS_SubscriptionBuiltinTopicData](#)

7.2.2 Member Data Documentation**value**

```
DDS_BUILTIN_TOPIC_KEY_TYPE_NATIVE DDS_BuiltinTopicKey_t::value[DDS_BUILTIN_TOPIC_KEY_TYPE_NATIVE↔_LENGTH]
```

An array of four integers that uniquely represents a remote [DDSEntity](#).

7.3 DDS_ChecksumProperty_t Struct Reference

<<[eXtension](#)>> Type to define the checksum properties of the participant RTI Connex DDS Micro.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_ChecksumKind_t](#) [computed_crc_kind](#)
The checksum kind for the computed and sent checksum.
- [DDS_ChecksumKindMask_t](#) [allowed_crc_mask](#)
The allowed checksum for checked checksum.
- [DDS_Boolean](#) [require_crc](#)
Whether the discovered participant requires checksum or not.

7.3.1 Detailed Description

<<[eXtension](#)>> Type to define the checksum properties of the participant RTI Connex DDS Micro.

7.3.2 Member Data Documentation

computed_crc_kind

[DDS_ChecksumKind_t](#) [DDS_ChecksumProperty_t::computed_crc_kind](#)

The checksum kind for the computed and sent checksum.

allowed_crc_mask

[DDS_ChecksumKindMask_t](#) [DDS_ChecksumProperty_t::allowed_crc_mask](#)

The allowed checksum for checked checksum.

require_crc

[DDS_Boolean](#) [DDS_ChecksumProperty_t::require_crc](#)

Whether the discovered participant requires checksum or not.

7.4 DDS_ContentFilterProperty Struct Reference

All of the information necessary for a [DDSDDataWriter](#) to perform writer-side filtering for a [DDSDDataReader](#).

```
#include <dds_c_content_filter.h>
```

Public Attributes

- [DDS_String content_filtered_topic_name](#)
Name of the content-filtered topic.
- [DDS_String related_topic_name](#)
Name of the related (base) topic.
- [DDS_String filter_class_name](#)
Name of the filter class to use.
- [DDS_String filter_expression](#)
The filter expression string.
- struct [DDS_StringSeq expression_parameters](#)
Parameters for the filter expression.

7.4.1 Detailed Description

All of the information necessary for a [DDSDataWriter](#) to perform writer-side filtering for a [DDSDataReader](#).

See also

[DDS_SubscriptionBuiltinTopicData](#)

7.4.2 Member Data Documentation

content_filtered_topic_name

[DDS_String](#) DDS_ContentFilterProperty::content_filtered_topic_name

Name of the content-filtered topic.

For a Connex Professional [DDSDataReader](#), this is the name of the content-filtered topic that the [DDSDataReader](#) is associated with.

For interoperability with Connex Professional and the DDS standard, an RTI Connex DDS Micro [DDSDataReader](#) will include a content-filtered topic name in its discovery information even though RTI Connex DDS Micro does not support content-filtered topics. RTI Connex DDS Micro automatically derives this name for local DataReaders from the [DDS_ContentFilterProperty::related_topic_name](#) and [DDS_ContentFilterQosPolicy::filter_name](#). Specifically, the name will be formatted as:

```
printf(content_filtered_topic_name, "%s::%s", related_topic_name, filter_name);
```

If asserting a remote RTI Connex DDS Micro [DDSDataReader](#) using DPSE, you can specify the content-filtered topic name in that format to achieve writer-side filtering.

related_topic_name

[DDS_String](#) DDS_ContentFilterProperty::related_topic_name

Name of the related (base) topic.

Specifies the name of the base topic from which the content-filtered topic is derived, or in RTI Connex DDS Micro simply the topic that the reader is associated with.

filter_class_name

[DDS_String](#) DDS_ContentFilterProperty::filter_class_name

Name of the filter class to use.

Specifies which filter class should be used to evaluate the filter expression. See [DDS_ContentFilterQosPolicy::filter_class_name](#) for additional information.

filter_expression

`DDS_String DDS_ContentFilterProperty::filter_expression`

The filter expression string.

Contains the filter expression that determines which samples are accepted by the content filter. See [DDS_ContentFilterQosPolicy::filter_expression](#) for additional information.

expression_parameters

`struct DDS_StringSeq DDS_ContentFilterProperty::expression_parameters`

Parameters for the filter expression.

A sequence of string parameters that can be referenced in the filter expression. See [DDS_ContentFilterQosPolicy::expression_parameters](#) for additional information.

7.5 DDS_ContentFilterQosPolicy Struct Reference

Filter that allows a [DDSDataReader](#) to specify that it is interested only in (potentially) a subset of the samples on its topic.

```
#include <dds_c_content_filter.h>
```

Public Attributes

- [DDS_String filter_name](#)
Name of the content filter.
- [DDS_String filter_class_name](#)
Name of the filter class.
- [DDS_String filter_expression](#)
Filter expression.
- `struct DDS_StringSeq` [expression_parameters](#)
Filter expression parameters.

7.5.1 Detailed Description

Filter that allows a [DDSDataReader](#) to specify that it is interested only in (potentially) a subset of the samples on its topic.

Entity:

[DDSDataReader](#)

Properties:

[RxO](#) = NO

[Changeable](#) = YES

7.5.2 Usage

The CONTENT_FILTER QoS policy allows a [DDSDataReader](#) to specify that it is interested only in (potentially) a subset of the samples on its topic. Once a content filter is configured, the [DDSDataReader](#) will only receive samples that pass the filter. If content filtering is supported by the [DDSDataWriter](#), samples that do not pass the filter may not be sent at all, thus reducing the network bandwidth usage.

A [DDSDataReader](#) will fail to be created if it specifies a [DDS_ContentFilterQosPolicy](#) and the content filtering feature is not enabled for its parent [DDSDomainParticipant](#). It will also fail to be created if the [DDS_ContentFilterQosPolicy](#) specifies a filter that is not compatible with the resource limits specified in the [DDS_FilterQosPolicy](#) for its parent [DDSDomainParticipant](#).

Generally, RTI Connex DDS Micro does not support changing the QoS of DDS entities after they are enabled. [DDS_ContentFilterQosPolicy](#) is an exception to this rule, since it can be updated at runtime using [DDSDataReader::set_qos](#).

For all strings contained in this QoS, be sure to follow the [Conventions](#) for allocating and freeing memory. In particular, if you use [::DDS_DataReaderQos::finalize](#) to finalize the [DDS_DataReaderQos](#), it will expect that all strings in the [DDS_ContentFilterQosPolicy](#) are owned by the policy and will attempt to free them when finalizing the QoS. Therefore, it is recommended to use [DDS_String_dup](#) to duplicate all strings that you want to use in the [DDS_ContentFilterQosPolicy](#).

As an example, a content filter could be configured as follows:

```
dr_qos.content_filter.filter_name = DDS_String_dup("MyFilter");
dr_qos.content_filter.filter_class_name = DDS_String_dup(DDS_SQL_CONTENT_FILTER_CLASS);
dr_qos.content_filter.filter_expression = DDS_String_dup("a < %0");
if (!DDS_StringSeq_set_maximum(&dr_qos.content_filter.expression_parameters, 1))
{
    /* error handling */
}
if (!DDS_StringSeq_set_length(&dr_qos.content_filter.expression_parameters, 1))
{
    /* error handling */
}
*DDS_StringSeq_get_reference(&dr_qos.content_filter.expression_parameters, 0) = DDS_String_dup("10");
```

See also

[::DDSFilterPluginFactory::register](#) for how to enable the content filtering feature
[DDS_FilterQosPolicy](#) for how to configure the content filtering feature

7.5.3 Member Data Documentation

filter_name

[DDS_String](#) [DDS_ContentFilterQosPolicy::filter_name](#)

Name of the content filter.

A name to uniquely identify this content filter within a DomainParticipant. For interoperability with Connex Professional, it is equivalent to the *filter_name* when configuring a content filter with XML application creation.

filter_class_name

```
DDS_String DDS_ContentFilterQosPolicy::filter_class_name
```

Name of the filter class.

The name that identifies the type of filter expression. RTI Connex DDS Micro currently only supports the "DDSSQL" filter class.

filter_expression

```
DDS_String DDS_ContentFilterQosPolicy::filter_expression
```

Filter expression.

A logical expression on the contents of the [DDSDataReader](#)'s Topic. If the expression evaluates to TRUE, a DDS sample is received; otherwise, it is discarded. The notation for this expression depends on the filter that you are using (specified by the [DDS_ContentFilterQosPolicy::filter_class_name](#)).

expression_parameters

```
struct DDS_StringSeq DDS_ContentFilterQosPolicy::expression_parameters
```

Filter expression parameters.

A string sequence of filter expression parameters. Each parameter corresponds to a positional argument in the filter expression: element 0 corresponds to positional argument 0, element 1 to positional argument 1, and so forth.

7.6 DDS_DataReaderInstanceReplacedStatus Struct Reference

```
<<eXtension>> DDS_INSTANCE_REPLACED_STATUS.
```

```
#include <dds_c_subscription.h>
```

Public Attributes

- [DDS_Long total_count](#)
The total number of instances that have been replaced during the life-span of the [DDSDataReader](#).
- [DDS_Long total_count_change](#)
The incremental number of instances replaced since the last time the listener was called or the status was read.
- [DDS_InstanceHandle_t last_instance_handle](#)
The instance that is being replaced.
- [DDS_InstanceHandle_t last_replacement_instance](#)
Handle to the instance which last replaced an instance.
- [DDS_InstanceHandle_t publication_handle](#)
Handle to the publisher which last replaced an instance.
- [DDS_Long lost_samples](#)
The number of samples which were lost due to the instance being replaced. Note that this number is an approximation because the middleware does not keep track of how many samples were removed prior to the instance being replaced (or the state of the samples being replaced).

7.6.1 Detailed Description

<<*eXtension*>> DDS_INSTANCE_REPLACED_STATUS.

A status structure describing the current instance replacement status for the reader.

7.6.2 Member Data Documentation

total_count

DDS_Long DDS_DataReaderInstanceReplacedStatus::total_count

The total number of instances that have been replaced during the life-span of the [DDSDataReader](#).

total_count_change

DDS_Long DDS_DataReaderInstanceReplacedStatus::total_count_change

The incremental number of instances replaced since the last time the listener was called or the status was read.

last_instance_handle

DDS_InstanceHandle_t DDS_DataReaderInstanceReplacedStatus::last_instance_handle

The instance that is being replaced.

last_replacement_instance

DDS_InstanceHandle_t DDS_DataReaderInstanceReplacedStatus::last_replacement_instance

Handle to the instance which last replaced an instance.

publication_handle

DDS_InstanceHandle_t DDS_DataReaderInstanceReplacedStatus::publication_handle

Handle to the publisher which last replaced an instance.

lost_samples

DDS_Long DDS_DataReaderInstanceReplacedStatus::lost_samples

The number of samples which were lost due to the instance being replaced. Note that this number is an approximation because the middleware does not keep track of how many samples were removed prior to the instance being replaced (or the state of the samples being replaced).

7.7 DDS_DataReaderProtocolQosPolicy Struct Reference

<<eXtension>> Protocol that applies only to [DDSDataReader](#) instances.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_UnsignedLong](#) `rtps_object_id`
The RTPS Object ID.
- struct [DDS_RtpsReliableReaderProtocol_t](#) `rtps_reliable_reader`
RTPS protocol-related configuration settings for the RTPS reliable reader associated with a [DDSDataReader](#). This parameter only has an effect if the reader is configured with [DDS_RELIABLE_RELIABILITY_QOS](#).
- [DDS_Boolean](#) `propagate_dispose_of_unregistered_instances`
<<eXtension>> Indicates whether or not an instance can move to the [DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE](#) state without being in the [DDS_ALIVE_INSTANCE_STATE](#) state.

7.7.1 Detailed Description

<<eXtension>> Protocol that applies only to [DDSDataReader](#) instances.

This QoS policy is an extension to the DDS standard.

Entity:

[DDSDataReader](#)

Properties:

[RxO](#) = N/A
[Changeable](#) = NO

7.7.2 Member Data Documentation

`rtps_object_id`

[DDS_UnsignedLong](#) `DDS_DataReaderProtocolQosPolicy::rtps_object_id`

The RTPS Object ID.

This value is used to determine the RTPS object ID of a [DDSDataReader](#) according to the DDS-RTPS Interoperability Wire Protocol.

Only the last 3 bytes are used; the most significant byte is ignored.

[default] [DDS_RTPS_AUTO_ID](#)

[range] [0,0x00ffffff]

rtps_reliable_reader

```
struct DDS_RtpsReliableReaderProtocol_t DDS_DataReaderProtocolQosPolicy::rtps_reliable_reader
```

RTPS protocol-related configuration settings for the RTPS reliable reader associated with a [DDSDataReader](#). This parameter only has an effect if the reader is configured with [DDS_RELIABLE_RELIABILITY_QOS](#).

For details, refer to the [DDS_RtpsReliableReaderProtocol_t](#).

[default]

nack_period 50 ms;

propagate_dispose_of_unregistered_instances

```
DDS_Boolean DDS_DataReaderProtocolQosPolicy::propagate_dispose_of_unregistered_instances
```

<< *eXtension* >> Indicates whether or not an instance can move to the [DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE](#) state without being in the [DDS_ALIVE_INSTANCE_STATE](#) state.

This field only applies to keyed readers.

When the field is set to [DDS_BOOLEAN_TRUE](#), the [DDSDataReader](#) will receive dispose notifications even if the instance has received zero samples.

When the field is set to [DDS_BOOLEAN_FALSE](#), the [DDSDataReader](#) will not receive dispose notifications unless the instance has received at least one sample.

NOTE: The default behavior in Connex Professional is [DDS_BOOLEAN_FALSE](#).

[default] DDS_BOOLEAN_TRUE

immutableinstance_lifecycle

7.8 DDS_DataReaderQos Struct Reference

QoS policies supported by a [DDSDataReader](#) entity.

```
#include <dds_c_subscription.h>
```

Public Attributes

- struct [DDS_DeadlineQosPolicy](#) deadline
Deadline policy, [DEADLINE](#).
- struct [DDS_LivelinessQosPolicy](#) liveliness
Liveliness policy, [LIVELINESS](#).
- struct [DDS_HistoryQosPolicy](#) history
History policy, [HISTORY](#).
- struct [DDS_ResourceLimitsQosPolicy](#) resource_limits
Resource limits policy, [RESOURCE_LIMITS](#).
- struct [DDS_OwnershipQosPolicy](#) ownership
Ownership policy, [OWNERSHIP](#).
- struct [DDS_ReliabilityQosPolicy](#) reliability
Reliability policy, [RELIABILITY](#).
- struct [DDS_DurabilityQosPolicy](#) durability
<<eXtension>> Durability policy, [DURABILITY](#).
- struct [DDS_DestinationOrderQosPolicy](#) destination_order
<<eXtension>> Destination Order policy, [DESTINATION_ORDER](#).
- struct [DDS_DataRepresentationQosPolicy](#) representation
Data representation policy, [DATA_REPRESENTATION](#).
- struct [DDS_DataReaderProtocolQosPolicy](#) protocol
<<eXtension>> [DDSDataReader](#) protocol policy, [DATA_READER_PROTOCOL](#).
- struct [DDS_TransportQosPolicy](#) transport
<<eXtension>> Transport policy, [TRANSPORT](#).
- struct [DDS_DataReaderResourceLimitsQosPolicy](#) reader_resource_limits
<<eXtension>> [DDSDataReader](#) resource limits policy, [DataReader Resource Limits](#).
- struct [DDS_UserDataQosPolicy](#) user_data
User data policy, [USER_DATA](#).
- struct [DDS_PropertyQosPolicy](#) property
<<eXtension>> The [DDSDataReader](#) properties.
- struct [DDS_ContentFilterQosPolicy](#) content_filter
<<experimental>> <<eXtension>> Content filter policy, [CONTENT_FILTER](#).
- struct [DDS_EntityNameQosPolicy](#) subscription_name
<<eXtension>> The [DDSDataReader](#) name. [ENTITY_NAME](#).
- struct [DDS_TransportPriorityQosPolicy](#) transport_priority
Transport priority policy, [TRANSPORT_PRIORITY](#).

7.8.1 Detailed Description

QoS policies supported by a [DDSDataReader](#) entity.

You must set the members in a consistent manner. If not, [DDSDataReader::set_qos](#) will fail with [DDS_RETCODE_INCONSISTENT_POLICY](#) and [DDSSubscriber::create_datareader](#) will fail.

7.8.2 Member Data Documentation

deadline

```
struct DDS_DeadlineQosPolicy DDS_DataReaderQos::deadline
```

Deadline policy, [DEADLINE](#).

liveliness

```
struct DDS_LivelinessQosPolicy DDS_DataReaderQos::liveliness
```

Liveliness policy, [LIVELINESS](#).

history

```
struct DDS_HistoryQosPolicy DDS_DataReaderQos::history
```

History policy, [HISTORY](#).

resource_limits

```
struct DDS_ResourceLimitsQosPolicy DDS_DataReaderQos::resource_limits
```

Resource limits policy, [RESOURCE_LIMITS](#).

ownership

```
struct DDS_OwnershipQosPolicy DDS_DataReaderQos::ownership
```

Ownership policy, [OWNERSHIP](#).

reliability

```
struct DDS_ReliabilityQosPolicy DDS_DataReaderQos::reliability
```

Reliability policy, [RELIABILITY](#).

durability

```
struct DDS_DurabilityQosPolicy DDS_DataReaderQos::durability
```

<<*eXtension*>> Durability policy, [DURABILITY](#).

destination_order

```
struct DDS_DestinationOrderQosPolicy DDS_DataReaderQos::destination_order
```

<<*eXtension*>> Destination Order policy, [DESTINATION_ORDER](#).

representation

```
struct DDS_DataRepresentationQosPolicy DDS_DataReaderQos::representation
```

Data representation policy, [DATA_REPRESENTATION](#).

protocol

```
struct DDS_DataReaderProtocolQosPolicy DDS_DataReaderQos::protocol
```

<<*eXtension*>> [DDSDataReader](#) protocol policy, [DATA_READER_PROTOCOL](#)

transport

```
struct DDS_TransportQosPolicy DDS_DataReaderQos::transport
```

<<*eXtension*>> Transport policy, [TRANSPORT](#).

reader_resource_limits

```
struct DDS_DataReaderResourceLimitsQosPolicy DDS_DataReaderQos::reader_resource_limits
```

<<*eXtension*>> [DDSDataReader](#) resource limits policy, [DataReader Resource Limits](#).

user_data

```
struct DDS_UserDataQosPolicy DDS_DataReaderQos::user_data
```

User data policy, [USER_DATA](#).

property

```
struct DDS_PropertyQosPolicy DDS_DataReaderQos::property
```

<<*eXtension*>> The [DDSDataReader](#) properties.

content_filter

```
struct DDS_ContentFilterQosPolicy DDS_DataReaderQos::content_filter
```

<<experimental>> <<eXtension>> Content filter policy, [CONTENT_FILTER](#).

If a content filter is specified, then the content filtering feature must be enabled or the [DDSDataReader](#) will fail to be created.

See also

[::DDSFilterPluginFactory::register](#) for how to enable the content filtering feature

subscription_name

```
struct DDS_EntityNameQosPolicy DDS_DataReaderQos::subscription_name
```

<<eXtension>> The [DDSDataReader](#) name. [ENTITY_NAME](#).

transport_priority

```
struct DDS_TransportPriorityQosPolicy DDS_DataReaderQos::transport_priority
```

Transport priority policy, [TRANSPORT_PRIORITY](#).

7.9 DDS_DataReaderResourceLimitsQosPolicy Struct Reference

Resource limits that apply only to [DDSDataReader](#) instances.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_Long max_remote_writers](#)
The maximum number of remote writers for which the reader will maintain state and communication.
- [DDS_Long max_remote_writers_per_instance](#)
The maximum number of remote writers for a unique instance for which the reader will maintain state and communication.
- [DDS_Long max_samples_per_remote_writer](#)
The maximum number of untaken samples from a given remote [DDSDataWriter](#) that a [DDSDataReader](#) may store.
- [DDS_Long max_outstanding_reads](#)
The maximum number of outstanding read or take calls, or one of their variants, on the same [DDSDataReader](#) for which the memory has not been returned by calling [FooDataReader::return_loan](#).
- [DDS_DataReaderResourceLimitsInstanceReplacementKind instance_replacement](#)
The instance replacement policy when the [DDS_ResourceLimitsQosPolicy::max_samples](#) limit has been reached and a new instance is detected.
- [DDS_Long max_routes_per_writer](#)
The maximum number of routes the reader will send to per matched writer.
- [DDS_Long max_fragmented_samples](#)
The maximum number of samples for which the built-in topic [DDSDataReader](#) may store fragments at a given point in time.
- [DDS_Long max_fragmented_samples_per_remote_writer](#)
The maximum number of samples per remote writer for which a [DDSDataReader](#) may store fragments.
- [DDS_Long shmем_ref_transfer_mode_attached_segment_allocation](#)
Number of shared memory segments a Zero Copy transfer over shared memory [DDSDataReader](#) can attach to at any given time.

7.9.1 Detailed Description

Resource limits that apply only to [DDSDataReader](#) instances.

This QoS policy is an extension to the DDS standard.

Entity:

[DDSDataReader](#)

Properties:

[RxO](#) = N/A

[Changeable](#) = NO

7.9.2 Member Data Documentation

max_remote_writers

[DDS_Long](#) [DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers](#)

The maximum number of remote writers for which the reader will maintain state and communication.

The [DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers](#) resource-limit limits the maximum number of remote [DDSDataWriter](#) entities that the [DDSDataReader](#) can receive data from regardless of which instance it is.

Resources controlled by this limit can only be reused when a remote [DDSDataWriter](#) is either deleted or loses liveliness.
[default] 1

[range] [1, 1000000], >= max_remote_writers_per_instance

max_remote_writers_per_instance

[DDS_Long](#) [DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers_per_instance](#)

The maximum number of remote writers for a unique instance for which the reader will maintain state and communication.

The [DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers_per_instance](#) limits the maximum number of remote [DDSDataWriter](#) entities a [DDSDataReader](#) maintains state for per instance. If this resource-limit is lower than the actual number of remote [DDSDataWriter](#) entities updating an instance, only state for up to [DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers_per_instance](#) is maintained. It is not guaranteed to be the last [DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers_per_instance](#) though. However, the last remote [DDSDataWriter](#) to update an instance is *always* saved. When an instance is no longer updated by a [DDSDataWriter](#) known to the [DDSDataReader](#), [DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE](#) state is signaled on the [DDSDataReader](#). Thus, if this resource-limit is lower than the actual number of remote [DDSDataWriter](#) entities updating an instance, [DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE](#) may be signaled even though there may be one or more [DDSDataWriter](#) entities still updating an instance. In this case the instance will go back to the [DDS_ALIVE_INSTANCE_STATE](#).

[default] 1

[range] [1, 2147483647], <= max_remote_writers

max_samples_per_remote_writer

DDS_Long DDS_DataReaderResourceLimitsQosPolicy::max_samples_per_remote_writer

The maximum number of untaken samples from a given remote [DDSDataWriter](#) that a [DDSDataReader](#) may store.

This resource-limit limits the maximum number of samples that can be cached for each remote [DDSDataWriter](#) that the [DDSDataReader](#) is receiving data from. This resource-limit is useful in preventing one [DDSDataWriter](#) from starving another [DDSDataWriter](#). NOTE: The samples cached from a [DDSDataWriter](#) are allocated from the same sample pool controlled by [DDS_ResourceLimitsQosPolicy::max_samples](#).

[default] 1

[range] [1, 100000000], <= [DDS_ResourceLimitsQosPolicy::max_samples](#)

max_outstanding_reads

DDS_Long DDS_DataReaderResourceLimitsQosPolicy::max_outstanding_reads

The maximum number of outstanding read or take calls, or one of their variants, on the same [DDSDataReader](#) for which the memory has not been returned by calling [FooDataReader::return_loan](#).

The [DDS_DataReaderResourceLimitsQosPolicy::max_outstanding_reads](#) resource-limit limits the maximum number of simultaneous loans an application can make at any given time using the [FooDataReader::read\(\)](#) and [FooDataReader::take\(\)](#) APIs.

Typically a value of 1 is sufficient. It is only necessary to increase this value if an application thread will access the same cache before another thread has returned the loan.

[default] 1

[range] [1, 2147483647]

instance_replacement

DDS_DataReaderResourceLimitsInstanceReplacementKind DDS_DataReaderResourceLimitsQosPolicy::instance←_replacement

The instance replacement policy when the [DDS_ResourceLimitsQosPolicy::max_samples](#) limit has been reached and a new instance is detected.

[default] [DDS_NO_INSTANCE_REPLACEMENT_QOS](#)

max_routes_per_writer

`DDS_Long DDS_DataReaderResourceLimitsQosPolicy::max_routes_per_writer`

The maximum number of routes the reader will send to per matched writer.

Each [DDSDDataReader](#) maintains information about the state of its peer [DDSDDataWriter](#), those it has matched with. Part of this state is which locators, or destination addresses, it should use to send data to a particular [DDSDDataWriter](#). The `DDS_DataReaderResourceLimitsQosPolicy::max_routes_per_writer` resource-limit limits the maximum number of routes that can be saved per [DDSDDataWriter](#).

This resource-limit is shared across all matched [DDSDDataWriter](#) entities per [DDSDDataReader](#). Thus, if `DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers` is 2 and `DDS_DataReaderResourceLimitsQosPolicy::max_routes_per_writer` is 4, a total of 8 routes can be saved for both [DDSDDataWriter](#) entities. One [DDSDDataWriter](#) entity may have 6 routes and the other 2.

[default] 4

[range] [1, 2000]

max_fragmented_samples

`DDS_Long DDS_DataReaderResourceLimitsQosPolicy::max_fragmented_samples`

The maximum number of samples for which the built-in topic [DDSDDataReader](#) may store fragments at a given point in time.

At any given time, a built-in topic [DDSDDataReader](#) may store fragments for up to `max_fragmented_samples` samples while waiting for the remaining fragments. These samples need not have consecutive sequence numbers and may have been sent by different built-in topic [DDSDDataWriter](#) instances.

Once all fragments of a sample have been received, the sample is treated as a regular sample and becomes subject to standard QoS settings such as `DDS_ResourceLimitsQosPolicy::max_samples`.

The middleware will drop fragments if the `max_fragmented_samples` limit has been reached. For best-effort communication, the middleware will accept a fragment for a new sample, but drop the oldest fragmented sample from the same remote writer. For reliable communication, the middleware will not drop fragments for any new samples until all fragments have been received.

max_fragmented_samples_per_remote_writer

`DDS_Long DDS_DataReaderResourceLimitsQosPolicy::max_fragmented_samples_per_remote_writer`

The maximum number of samples per remote writer for which a [DDSDDataReader](#) may store fragments.

Logical limit so a single remote writer cannot consume all available resources.

[default] 256

[range] [1, 1 million], $\leq$ `max_fragmented_samples`

shmem_ref_transfer_mode_attached_segment_allocation

`DDS_Long DDS_DataReaderResourceLimitsQosPolicy::shmem_ref_transfer_mode_attached_segment_allocation`

Number of shared memory segments a Zero Copy transfer over shared memory [DDSDataReader](#) can attach to at any given time.

The `max_count` does not limit the total number of shared memory segments used by the [DDSDataReader](#). When this limit is hit, the [DDSDataReader](#) will try to detach from a segment that doesn't contain any loaned samples and attach to a new segment. If samples are loaned from all attached segments, then the [DDSDataReader](#) will fail to attach to the new segment. This scenario will result in the sample being rejected.

[default] `DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers` **[range]** [0, 1000000]

7.10 DDS_DataRepresentationIdSeq Struct Reference

Declares IDL sequence < [DDS_DataRepresentationId_t](#) >

```
#include <dds_c_infrastructure.h>
```

7.10.1 Detailed Description

Declares IDL sequence < [DDS_DataRepresentationId_t](#) >

See also

[DDS_DataRepresentationId_t](#)

7.11 DDS_DataRepresentationQosPolicy Struct Reference

This QoS policy contains a list of representation identifiers used by [DDSDataWriter](#) and [DDSDataReader](#) entities to negotiate which data representation to use.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- struct [DDS_DataRepresentationIdSeq](#) value
Sequence of representation identifiers.

7.11.1 Detailed Description

This QoS policy contains a list of representation identifiers used by [DDSDataWriter](#) and [DDSDataReader](#) entities to negotiate which data representation to use.

Entity:

[DDSTopic](#), [DDSDataReader](#), [DDSDataWriter](#)

Status:

[DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#), [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#)

Properties:

[RxO](#) = YES

[Changeable](#) = UNTIL ENABLE

See also

[DATA_REPRESENTATION](#)

This policy has request-offer semantics. A [DDSDataWriter](#) may only offer a single representation. Attempting to put multiple representations in a [DDSDataWriter](#) will result in [DDS_RETCODE_INCONSISTENT_POLICY](#). A [DDSDataWriter](#) will use its offered policy to communicate with its matched [DDSDataReader](#) entities. A [DDSDataReader](#) requests one or more representations. If a [DDSDataWriter](#) offers a representation that is contained within the sequence of the [DDSDataReader](#), the offer satisfies the request and the policies are compatible. Otherwise, they are incompatible.

7.11.2 Member Data Documentation

value

```
struct DDS\_DataRepresentationIdSeq DDS_DataRepresentationQosPolicy::value
```

Sequence of representation identifiers.

[default] For [DDSTopic](#), [DDSDataWriter](#), and [DDSDataReader](#), a list with one element: [DDS_AUTO_DATA_REPRESENTATION](#).

Note

An empty sequence is equivalent to a list with one element: [DDS_XCDR_DATA_REPRESENTATION](#).

7.12 DDS_DataWriterProtocolQosPolicy Struct Reference

[*<<eXtension>>*](#) Protocol that applies only to [DDSDataWriter](#) instances.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_UnsignedLong](#) `rtps_object_id`
The RTPS Object ID.
- struct [DDS_RtpsReliableWriterProtocol_t](#) `rtps_reliable_writer`
The reliable protocol defined in RTPS.
- [DDS_Boolean](#) `serialize_on_write`
Indicate whether a sample should be serialized when it is written (TRUE) or when it is sent (FALSE).

7.12.1 Detailed Description

<<*eXtension*>> Protocol that applies only to [DDSDataWriter](#) instances.

This QoS policy is an extension to the DDS standard.

Entity:

[DDSDataWriter](#)

Properties:

[RxO](#) = N/A

[Changeable](#) = NO

7.12.2 Member Data Documentation**rtps_object_id**

[DDS_UnsignedLong](#) `DDS_DataWriterProtocolQosPolicy::rtps_object_id`

The RTPS Object ID.

This value is used to determine the RTPS object ID of a [DDSDataWriter](#) according to the DDS-RTPS Interoperability Wire Protocol.

Only the last 3 bytes are used; the most significant byte is ignored.

[default] [DDS_RTPS_AUTO_ID](#)

[range] [0,0x00ffffff]

rtps_reliable_writer

```
struct DDS_RtpsReliableWriterProtocol_t DDS_DataWriterProtocolQosPolicy::rtps_reliable_writer
```

The reliable protocol defined in RTPS.

[default] heartbeat_period 3.0 seconds;
heartbeats_per_max_samples: 1;
max_send_window: minimum(256, ::DDS_HistoryQosPolicy::depth * ::DDS_ResourceLimitsQosPolicy::max_instances);
max_heartbeat_retries: DDS_LENGTH_UNLIMITED;

serialize_on_write

```
DDS_Boolean DDS_DataWriterProtocolQosPolicy::serialize_on_write
```

Indicate whether a sample should be serialized when it is written (TRUE) or when it is sent (FALSE).

To send a sample over the network it must first be serialized. The `serialize_on_write` attribute determines when the sample is serialized.

If set to `DDS_BOOLEAN_TRUE`, the sample is serialized when it is written using `FooDataWriter::write` or a similar API. This is the default behavior and results in the sample being stored in serialized form in case it needs to be resent.

If set to `DDS_BOOLEAN_FALSE`, the sample is not serialized when using `FooDataWriter::write` or a similar API. Instead it is serialized when it is needed and never stored in serialized form. This requires the unserialized sample to be available for the life-span of the `DDSDDataWriter`. If this is not the case, this option must not be used. Use of this option will also require a specialized type-plugin which is not supported by the IDL compiler.

[default] TRUE

7.13 DDS_DataWriterQos Struct Reference

QoS policies supported by a `DDSDDataWriter` entity.

```
#include <dds_c_publication.h>
```

Public Attributes

- struct [DDS_DeadlineQosPolicy](#) deadline
Deadline policy, [DEADLINE](#).
- struct [DDS_LivelinessQosPolicy](#) liveliness
Liveliness policy, [LIVELINESS](#).
- struct [DDS_HistoryQosPolicy](#) history
History policy, [HISTORY](#).
- struct [DDS_ResourceLimitsQosPolicy](#) resource_limits
Resource limits policy, [RESOURCE_LIMITS](#).
- struct [DDS_OwnershipQosPolicy](#) ownership
Ownership policy, [OWNERSHIP](#).
- struct [DDS_OwnershipStrengthQosPolicy](#) ownership_strength
Ownership strength policy, [OWNERSHIP_STRENGTH](#).
- struct [DDS_ReliabilityQosPolicy](#) reliability
Reliability policy, [RELIABILITY](#).
- struct [DDS_DurabilityQosPolicy](#) durability
<<eXtension>> Durability policy, [DURABILITY](#).
- struct [DDS_DestinationOrderQosPolicy](#) destination_order
<<eXtension>> Destination Order policy, [DESTINATION_ORDER](#).
- struct [DDS_DataRepresentationQosPolicy](#) representation
Data representation policy, [DATA_REPRESENTATION](#).
- struct [DDS_DataWriterProtocolQosPolicy](#) protocol
<<eXtension>> [DDSDataWriter](#) protocol policy, [DATA_WRITER_PROTOCOL](#).
- struct [DDS_TransportQosPolicy](#) transport
Transport policy, [TRANSPORT](#). Only unicast transports are supported.
- struct [DDS_DataWriterResourceLimitsQosPolicy](#) writer_resource_limits
<<eXtension>> Writer resource limits policy, [DataWriter Resource Limits](#).
- struct [DDS_PublishModeQosPolicy](#) publish_mode
<<eXtension>> Publish mode policy, [PUBLISH_MODE](#).
- struct [DDS_UserDataQosPolicy](#) user_data
User data policy, [USER_DATA](#).
- struct [DDS_PropertyQosPolicy](#) property
<<eXtension>> The [DDSDataWriter](#) properties. Please refer to the [Property Reference](#) for available properties.
- struct [DDS_EntityNameQosPolicy](#) publication_name
<<eXtension>> The [DDSDataWriter](#) name. [ENTITY_NAME](#).
- struct [DDS_DataWriterTransferModeQosPolicy](#) transfer_mode
<<eXtension>> QoS related to transferring data
- struct [DDS_TransportPriorityQosPolicy](#) transport_priority
Transport priority policy, [TRANSPORT_PRIORITY](#).

7.13.1 Detailed Description

QoS policies supported by a [DDSDataWriter](#) entity.

The [DDS_DataWriterQos](#) controls the resources allocated by a [DDSDataWriter](#). Each [DDSDataWriter](#) allocates its own resources, even DataWriters of the same [DDSTopic](#). For this reason is it advised to limit the number of DataWriters per topic.

You must set certain members in a consistent manner:

- [DDS_DataWriterQos::DDS_HistoryQosPolicy::depth](#) $\leq$ [DDS_DataWriterQos::DDS_ResourceLimitsQosPolicy::max_samples_per_instance](#)
- [DDS_DataWriterQos::DDS_ResourceLimitsQosPolicy::max_samples_per_instance](#) $\leq$ [DDS_DataWriterQos::DDS_ResourceLimitsQosPolicy::max_samples](#)

If any of the above are not true, [DDSDataWriter::set_qos](#) and [DDSPublisher::set_default_datawriter_qos](#) will fail with [DDS_RETCODE_INCONSISTENT_POLICY](#) and [DDSPublisher::create_datawriter](#) and will return NULL.

Entity:

[DDSDataWriter](#)

See also

[QoS Policies](#) allowed ranges within each Qos.

7.13.2 Member Data Documentation

deadline

```
struct DDS\_DeadlineQosPolicy DDS_DataWriterQos::deadline
```

Deadline policy, [DEADLINE](#).

liveliness

```
struct DDS\_LivelinessQosPolicy DDS_DataWriterQos::liveliness
```

Liveliness policy, [LIVELINESS](#).

history

```
struct DDS\_HistoryQosPolicy DDS_DataWriterQos::history
```

History policy, [HISTORY](#).

resource_limits

```
struct DDS_ResourceLimitsQosPolicy DDS_DataWriterQos::resource_limits
```

Resource limits policy, [RESOURCE_LIMITS](#).

ownership

```
struct DDS_OwnershipQosPolicy DDS_DataWriterQos::ownership
```

Ownership policy, [OWNERSHIP](#).

ownership_strength

```
struct DDS_OwnershipStrengthQosPolicy DDS_DataWriterQos::ownership_strength
```

Ownership strength policy, [OWNERSHIP_STRENGTH](#).

reliability

```
struct DDS_ReliabilityQosPolicy DDS_DataWriterQos::reliability
```

Reliability policy, [RELIABILITY](#).

durability

```
struct DDS_DurabilityQosPolicy DDS_DataWriterQos::durability
```

<<*eXtension*>> Durability policy, [DURABILITY](#).

destination_order

```
struct DDS_DestinationOrderQosPolicy DDS_DataWriterQos::destination_order
```

<<*eXtension*>> Destination Order policy, [DESTINATION_ORDER](#).

representation

```
struct DDS_DataRepresentationQosPolicy DDS_DataWriterQos::representation
```

Data representation policy, [DATA_REPRESENTATION](#).

protocol

```
struct DDS_DataWriterProtocolQosPolicy DDS_DataWriterQos::protocol
```

<<*eXtension*>> [DDSDataWriter](#) protocol policy, [DATA_WRITER_PROTOCOL](#)

transport

```
struct DDS_TransportQosPolicy DDS_DataWriterQos::transport
```

Transport policy, [TRANSPORT](#). Only unicast transports are supported.

writer_resource_limits

```
struct DDS_DataWriterResourceLimitsQosPolicy DDS_DataWriterQos::writer_resource_limits
```

<<*eXtension*>> Writer resource limits policy, [DataWriter Resource Limits](#).

publish_mode

```
struct DDS_PublishModeQosPolicy DDS_DataWriterQos::publish_mode
```

<<*eXtension*>> Publish mode policy, [PUBLISH_MODE](#).

Determines whether the [DDSDataWriter](#) publishes data synchronously or asynchronously and how.

user_data

```
struct DDS_UserDataQosPolicy DDS_DataWriterQos::user_data
```

User data policy, [USER_DATA](#).

property

```
struct DDS_PropertyQosPolicy DDS_DataWriterQos::property
```

<<*eXtension*>> The [DDSDataWriter](#) properties. Please refer to the [Property Reference](#) for available properties.

publication_name

```
struct DDS_EntityNameQosPolicy DDS_DataWriterQos::publication_name
```

<<*eXtension*>> The [DDSDataWriter](#) name. [ENTITY_NAME](#).

transport_priority

```
struct DDS_TransportPriorityQosPolicy DDS_DataWriterQos::transport_priority
```

Transport priority policy, [TRANSPORT_PRIORITY](#).

7.14 DDS_DataWriterResourceLimitsQosPolicy Struct Reference

Resource limits that apply only to [DDSDataWriter](#) instances.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_Long max_remote_readers](#)
The maximum number of remote readers for which the writer will maintain state and communication.
- [DDS_Long max_routes_per_reader](#)
The maximum number of routes the writer will send per matched reader.
- [DDS_Long writer_loaned_sample_allocation](#)
Represents the amount of loanable samples managed by a [DDSDataWriter](#).
- [DDS_Boolean initialize_writer_loaned_sample](#)
Whether to initialize loaned samples returned by a [DDSDataWriter](#).
- [DDS_Long max_remote_reader_filters](#)
<<experimental>> The maximum number of remote readers for which a [DDSDataWriter](#) will simultaneously perform writer-side filtering.

7.14.1 Detailed Description

Resource limits that apply only to [DDSDataWriter](#) instances.

This QoS policy is an extension to the DDS standard.

Entity:

[DDSDataWriter](#)

Properties:

[RxO](#) = N/A

[Changeable](#) = NO

7.14.2 Member Data Documentation

max_remote_readers

`DDS_Long DDS_DataWriterResourceLimitsQosPolicy::max_remote_readers`

The maximum number of remote readers for which the writer will maintain state and communication.

The `DDS_DataWriterResourceLimitsQosPolicy::max_remote_readers` resource-limit limits the maximum number of remote `DDSDataReader` entities that the `DDSDataWriter` will maintain state and communication.

[default] 16

[range] [1, 100000000]

max_routes_per_reader

`DDS_Long DDS_DataWriterResourceLimitsQosPolicy::max_routes_per_reader`

The maximum number of routes the writer will send per matched reader.

Each `DDSDataWriter` maintains information about the state of its peer `DDSDataReader` entities, those it has matched with. Part of this state is which locators or destination addresses it should use to send data to a particular `DDSDataReader`. The `DDS_DataWriterResourceLimitsQosPolicy::max_routes_per_reader` resource-limit limits the number of routes that can be saved per `DDSDataReader`.

This resource-limit is shared across all matched `DDSDataReader` entities per `DDSDataWriter`. Thus, if `DDS_DataWriterResourceLimitsQosPolicy::max_remote_readers` is 2 and `DDS_DataWriterResourceLimitsQosPolicy::max_routes_per_reader` is 4, a total of 8 routes can be saved for both `DDSDataReader` entities. One `DDSDataReader` may have 6 routes and the other 2.

[default] 4

[range] [1, 2000]

writer_loaned_sample_allocation

`DDS_Long DDS_DataWriterResourceLimitsQosPolicy::writer_loaned_sample_allocation`

Represents the amount of loanable samples managed by a `DDSDataWriter`.

The number of samples loaned by a `DDSDataWriter` via `FooDataWriter::get_loan` is limited by `DDS_DataWriterResourceLimitsQosPolicy::writer_loaned_sample_allocation`. `FooDataWriter::get_loan` returns NULL if and only if `DDS_DataWriterResourceLimitsQosPolicy::writer_loaned_sample_allocation` samples have been loaned, and none of those samples have been written with `FooDataWriter::write` or discarded via `FooDataWriter::discard_loan`. If the default value of `DDS_SIZE_AUTO` is used the `DDS_DataWriterResourceLimitsQosPolicy::writer_loaned_sample_allocation` is set to `DDS_ResourceLimitsQosPolicy::max_samples + 1`.

[default] `DDS_SIZE_AUTO`

[range] [1, 1000000]

See also

[FooDataWriter::get_loan](#)

[FooDataWriter::discard_loan](#)

initialize_writer_loaned_sample

`DDS_Boolean DDS_DataWriterResourceLimitsQosPolicy::initialize_writer_loaned_sample`

Whether to initialize loaned samples returned by a [DDSDataWriter](#).

[default] `DDS_BOOLEAN_FALSE`

See also

[FooDataWriter::get_loan](#)

max_remote_reader_filters

`DDS_Long DDS_DataWriterResourceLimitsQosPolicy::max_remote_reader_filters`

<<experimental>> The maximum number of remote readers for which a [DDSDataWriter](#) will simultaneously perform writer-side filtering.

This QoS policy has no effect if the content filtering feature is not enabled or if [DDS_FilterQosPolicy::disable_writer_filtering](#) is set to `DDS_BOOLEAN_TRUE`.

This resource limit controls how many remote [DDSDataReader](#) entities a [DDSDataWriter](#) will simultaneously perform writer-side content filtering for. The [DDSDataWriter](#) will store the filter result per sample per filtered [DDSDataReader](#).

The behavior depends on the configured value:

- 0: The [DDSDataWriter](#) will not perform writer-side filtering for any [DDSDataReader](#). All content filtering will be performed on the reader side.
- 1 to $(2^{31})-1$: The [DDSDataWriter](#) will perform writer-side filtering for up to the specified number of [DDSDataReader](#) entities.
- `DDS_LENGTH_UNLIMITED`: The [DDSDataWriter](#) will perform writer-side filtering for up to [DDS_DataWriterResourceLimitsQosPolicy::max_remote_reader_filters](#) [DDSDataReader](#) entities.

If a [DDSDataWriter](#) is already filtering the maximum number of [DDSDataReader](#) entities specified by this limit and a new filtered [DDSDataReader](#) is discovered, the newly discovered [DDSDataReader](#) will not be filtered by the [DDSDataWriter](#), and the [DDSDataReader](#) will instead apply the content filter itself.

Once a [DDSDataReader](#) is excluded from writer-side filtering due to this resource limit, it will remain excluded even if one of the previously filtered [DDSDataReader](#) entities goes away. However, any subsequently created [DDSDataReader](#) entities will be eligible for writer-side filtering as long as the number of currently filtered [DDSDataReader](#) entities does not exceed this limit.

No matter the value of this QoS policy, a [DDSDataWriter](#) will only ever be able to apply writer-side filtering for up to 4 unique filters on the same locator.

[default] `DDS_LENGTH_UNLIMITED`

[range] `[0,  $(2^{31})-1$ ]` or `DDS_LENGTH_UNLIMITED`

See also

[::DDSFilterPluginFactory::register](#) for how to enable the content filtering feature

7.15 DDS_DataWriterShmemRefTransferModeSettings Struct Reference

Settings related to transferring data using shared memory references.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_Boolean enable_data_consistency_check](#)
Controls if samples can be checked for consistency.

7.15.1 Detailed Description

Settings related to transferring data using shared memory references.

Properties:

[RxO](#) = N/A
[Changeable](#) = NO

QoS:

[DDS_DataWriterTransferModeQosPolicy](#)

7.15.2 Member Data Documentation

enable_data_consistency_check

[DDS_Boolean](#) [DDS_DataWriterShmemRefTransferModeSettings::enable_data_consistency_check](#)

Controls if samples can be checked for consistency.

When this setting is true, the [DDSDDataWriter](#) sends an incrementing sequence number as an inline QoS with every sample. This sequence number allows a [DDSDDataReader](#) to use the [FooDataReader::is_data_consistent](#) API to detect if the [DDSDDataWriter](#) overwrote the sample before the [DDSDDataReader](#) could complete processing the sample.

[default] [DDS_BOOLEAN_TRUE](#)

7.16 DDS_DataWriterTransferModeQosPolicy Struct Reference

<<[eXtension](#)>> QoS related to transferring data

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- struct [DDS_DataWriterShmemRefTransferModeSettings](#) `shmem_ref_settings`
Settings related to transferring data using shared memory references.

7.16.1 Detailed Description

<<*eXtension*>> QoS related to transferring data

It contains qualitative settings related to the actions a [DDSDDataWriter](#) performs while transferring its data.

Entity:

[DDSDDataWriter](#)

Properties:

[RxO](#) = N/A
[Changeable](#) = NO

7.16.2 Member Data Documentation**shmem_ref_settings**

```
struct DDS\_DataWriterShmemRefTransferModeSettings DDS_DataWriterTransferModeQosPolicy::shmem_ref←
_settings
```

Settings related to transferring data using shared memory references.

For details, refer to the [DDS_DataWriterShmemRefTransferModeSettings](#).

7.17 DDS_DeadlineQosPolicy Struct Reference

Expresses the maximum duration or deadline within which an instance is expected to be updated.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- struct [DDS_Duration_t](#) `period`
Duration of the deadline period.

7.17.1 Detailed Description

Expresses the maximum duration or deadline within which an instance is expected to be updated.

[DDSDataReader](#) expects a new sample updating the value of each instance at least once every `period`. [DDSDataWriter](#) indicates that the application commits to write a new value using the [DDSDataWriter](#) for each instance managed by the [DDSDataWriter](#) at least once every `period`.

If a [DDSDataReader](#) stops receiving samples, it will take at most $(period + (period / 8))$ to detect that the deadline has been missed. The deadline period starts again when the next sample is received.

Entity:

[DDSTopic](#), [DDSDataReader](#), [DDSDataWriter](#)

Status:

[DDS_OFFERED_DEADLINE_MISSED_STATUS](#), [DDS_REQUESTED_DEADLINE_MISSED_STATUS](#), [DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#), [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#)

Properties:

[Rxo](#) = YES
[Changeable](#) = NO

7.17.2 Usage

This policy is useful for cases where a [DDSTopic](#) is expected to have each instance updated periodically. On the publishing side this setting establishes a contract that the application must meet. On the subscribing side the setting establishes a minimum requirement for the remote publishers that are expected to supply the data values.

When RTI Connex DDS Micro 'matches' a [DDSDataWriter](#) and a [DDSDataReader](#) it checks whether the settings are compatible; that is, *offered deadline* $\leq$ *requested deadline*; if they are not, the two entities are informed via the [DDSListener](#) mechanism of the incompatibility of the QoS settings and communication will not occur.

Assuming that the reader and writer have compatible settings, the fulfillment of this contract is monitored by RTI Connex DDS Micro and the application is informed of any violations by means of the proper [DDSListener](#).

7.17.3 Compatibility

The value offered is considered compatible with the value requested if and only if the inequality *offered period* $\leq$ *requested period* evaluates to 'TRUE'.

7.17.4 Member Data Documentation

period

```
struct DDS_Duration_t DDS_DeadlineQosPolicy::period
```

Duration of the deadline period.

[default] [DDS_DURATION_INFINITE](#)

[range] [1 nanosec, 1 year] or [DDS_DURATION_INFINITE](#)

7.18 DDS_DestinationOrderQosPolicy Struct Reference

Destination order policies that affect the ordering of subscribed data.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_DestinationOrderQosPolicyKind](#) kind
The kind of destination ordering that applies to subscribed data.
- struct [DDS_Duration_t](#) [source_timestamp_tolerance](#)
<<eXtension>> Allowed tolerance between source timestamps of consecutive samples.

7.18.1 Detailed Description

Destination order policies that affect the ordering of subscribed data.

7.18.2 Member Data Documentation

kind

```
DDS_DestinationOrderQosPolicyKind DDS_DestinationOrderQosPolicy::kind
```

The kind of destination ordering that applies to subscribed data.

[default] [DDS_BY_RECEPTION_TIMESTAMP_DESTINATIONORDER_QOS](#)

source_timestamp_tolerance

```
struct DDS_Duration_t DDS_DestinationOrderQosPolicy::source_timestamp_tolerance
```

<<*eXtension*>> Allowed tolerance between source timestamps of consecutive samples.

When a [DDSDDataWriter](#) sets [DDS_DestinationOrderQosPolicy](#) to [DDS_BY_SOURCE_TIMESTAMP_DESTINATIONORDER_QOS](#), when writing a sample, its timestamp must not be less than the timestamp of the previously written sample. However, if it is less than the timestamp of the previously written sample but the difference is less than this tolerance, the sample will use the previously written sample's timestamp. Otherwise, if the difference is greater than this tolerance, the write will fail.

[DDS_BY_SOURCE_TIMESTAMP_DESTINATIONORDER_QOS](#) is not supported on the [DDSDDataReader](#).

[default] 100 milliseconds for [DDSDDataWriter](#)

[range] [1 nanosec, 1 year]

7.19 DDS_DiscoveryComponent Struct Reference

Specifies the discovery plugin the [DDSDomainParticipant](#) should use.

```
#include <dds_c_domain.h>
```

Public Attributes

- [RT_ComponentFactoryId_T](#) [name](#)

Specifies the name of a registered discovery plugin the [DDSDomainParticipant](#) should use.

7.19.1 Detailed Description

Specifies the discovery plugin the [DDSDomainParticipant](#) should use.

7.19.2 Member Data Documentation

name

```
RT_ComponentFactoryId_T DDS_DiscoveryComponent::name
```

Specifies the name of a registered discovery plugin the [DDSDomainParticipant](#) should use.

This attribute specifies the name of the plug-in to be used for DDS Discovery. The maximum length of the name is [RT_MAX_FACTORY_NAME](#) excluding the NUL termination.

7.20 DDS_DiscoveryQosPolicy Struct Reference

Specifies the attributes required to discover participants in the domain.

```
#include <dds_c_domain.h>
```

Public Attributes

- struct DDS_StringSeq [initial_peers](#)
Determines the initial list of peers that will be contacted by the Discovery mechanism to send announcements about the presence of this participant.
- struct DDS_StringSeq [enabled_transports](#)
The transports available for use by the Discovery mechanism.
- struct DDS_DiscoveryComponent [discovery](#)
The name of the discovery plugin that will be used to create a discovery plugin for this [DDSDomainParticipant](#).
- [DDS_Boolean accept_unknown_peers](#)
Whether to accept a new participant that is not in the initial peer list.
- [DDS_Boolean enable_participant_discovery_by_name](#)
Enable Participant discovery by name.
- [DDS_Boolean enable_endpoint_discovery_queue](#)
Enable the endpoint discovery queue and queue endpoint discovery messages if there are no resources to assert the endpoint at the time of discovery.
- [DDS_Long metatraffic_transport_priority](#)
Transport priority for builtin endpoint DataWriters and DataReaders.

7.20.1 Detailed Description

Specifies the attributes required to discover participants in the domain.

Entity:

[DDSDomainParticipant](#)

Properties:

[RxO](#) = N/A

[Changeable](#) = NO

7.20.2 Usage

This extended QoS is used to define how participants find each other on a network.

7.20.3 Member Data Documentation

initial_peers

```
struct DDS_StringSeq DDS_DiscoveryQosPolicy::initial_peers
```

Determines the initial list of peers that will be contacted by the Discovery mechanism to send announcements about the presence of this participant.

If there is a remote peer [DDSDomainParticipant](#) such as is described this list, it will become aware of this participant and will engage in the Discovery protocol to exchange meta-data with this participant.

Each element of this list must be a `peer` descriptor in the proper format.

[default] Empty sequence

[range] Sequence of arbitrary length.

enabled_transports

```
struct DDS_StringSeq DDS_DiscoveryQosPolicy::enabled_transports
```

The transports available for use by the Discovery mechanism.

Only these transports can be used by the discovery mechanism to send meta-traffic via the builtin endpoints (built-in [DDSDataReader](#) and [DDSDataWriter](#)).

Also determines the locators on which the Discovery mechanism will listen for meta-traffic. These along with the `domain_id` and `participant_id` determine the locators on which the Discovery mechanism can receive meta-data.

The memory for the strings in this sequence is managed according to the conventions described in [String Support](#). In particular, be careful to avoid a situation in which RTI Connex DDS Micro allocates a string on your behalf and you then reuse that string in such a way that RTI Connex DDS Micro believes it to have more memory allocated to it than it actually does.

[default] Empty sequence.

When an empty sequence is passed to [DDSDomainParticipantFactory::create_participant](#) in the [DDS_DomainParticipantQos](#) and the UDP transport has been registered, the domain participant will automatically add the following locators to listen for discovery traffic on `"_udp://"` and `"_udp://127.0.0.1"`.

[range] Sequence of non-null, non-empty strings.

discovery

```
struct DDS_DiscoveryComponent DDS_DiscoveryQosPolicy::discovery
```

The name of the discovery plugin that will be used to create a discovery plugin for this [DDSDomainParticipant](#).

The [DDSDomainParticipantFactory](#) will search through a list of registered discovery plugins. If it finds a registered discovery factory with a name matching this factory name, it will use that discovery plugin to create a new instance of the discovery plugin and register the plugin with the [DDSDomainParticipant](#).

If it does not find a registered discovery factory with this name, it will return [DDS_RETCODE_PRECONDITION_NOT_MET](#)

Precondition

One or more discovery plugins must have been previously registered with the [DDSDomainParticipantFactory](#), each with an associated name.

accept_unknown_peers

```
DDS_Boolean DDS_DiscoveryQosPolicy::accept_unknown_peers
```

Whether to accept a new participant that is not in the initial peer list.

If [DDS_BOOLEAN_FALSE](#), the participant will only communicate with those in the initial peer list

If [DDS_BOOLEAN_TRUE](#), the participant will also communicate with all discovered remote participants.

[default] [DDS_BOOLEAN_TRUE](#)

enable_participant_discovery_by_name

```
DDS_Boolean DDS_DiscoveryQosPolicy::enable_participant_discovery_by_name
```

Enable Participant discovery by name.

If [DDS_BOOLEAN_FALSE](#), a [DDSDomainParticipant](#) is only identified by its GUID.

If [DDS_BOOLEAN_TRUE](#), a [DDSDomainParticipant](#) is identified by its [DDS_DomainParticipantQos::participant_name](#). Thus, each [DDSDomainParticipant](#) in a domain must be uniquely named in the DDS domain. Failure to uniquely name each [DDSDomainParticipant](#) may result in undefined behavior.

Note that [DDS_DomainParticipantQos::participant_name](#) cannot be empty when set to [DDS_BOOLEAN_TRUE](#).

[default] [DDS_BOOLEAN_FALSE](#)

enable_endpoint_discovery_queue

`DDS_Boolean DDS_DiscoveryQosPolicy::enable_endpoint_discovery_queue`

Enable the endpoint discovery queue and queue endpoint discovery messages if there are no resources to assert the endpoint at the time of discovery.

When a remote [DDSDataReader](#) or [DDSDataWriter](#) is discovered, the discovery information is saved, and the discovered entity is matched with local endpoints.

However, if there are insufficient resources to save the information or perform the matching, the discovery message is discarded by default. That is, all information about the discovered endpoint is lost, even if resources are later freed up.

This behavior can be changed by setting this value to [DDS_BOOLEAN_TRUE](#). If there are insufficient resources to store the discovery information, the message is queued and processed when resources becomes available.

Discovery messages that are queued are queued until the [DDSDomainParticipant](#) that published the discovery messages is no longer available.

[default] [DDS_BOOLEAN_FALSE](#)

metatraffic_transport_priority

`DDS_Long DDS_DiscoveryQosPolicy::metatraffic_transport_priority`

Transport priority for builtin endpoint DataWriters and DataReaders.

This value is used to set the transport priority for all builtin endpoint DataWriters and DataReaders created by the discovery plugins.

[default] 0

7.21 DDS_DomainParticipantFactoryQos Struct Reference

QoS policies supported by a [DDSDomainParticipantFactory](#).

```
#include <dds_c_domain.h>
```

Public Attributes

- struct [DDS_EntityFactoryQosPolicy](#) `entity_factory`
Entity factory policy, [ENTITY_FACTORY](#).
- struct [DDS_SystemResourceLimitsQosPolicy](#) `resource_limits`
<<eXtension>> System resource limits, [SYSTEM_RESOURCE_LIMITS](#).

7.21.1 Detailed Description

QoS policies supported by a [DDSDomainParticipantFactory](#).

Entity:

[DDSDomainParticipantFactory](#)

See also

[QoS Policies](#) and allowed ranges within each Qos.

7.21.2 Member Data Documentation

entity_factory

```
struct DDS\_EntityFactoryQosPolicy DDS_DomainParticipantFactoryQos::entity_factory
```

Entity factory policy, [ENTITY_FACTORY](#).

resource_limits

```
struct DDS\_SystemResourceLimitsQosPolicy DDS_DomainParticipantFactoryQos::resource_limits
```

<<*eXtension*>> System resource limits, [SYSTEM_RESOURCE_LIMITS](#).

7.22 DDS_DomainParticipantQos Struct Reference

QoS policies supported by a [DDSDomainParticipant](#) entity.

```
#include <dds_c_domain.h>
```

Public Attributes

- struct [DDS_EntityFactoryQosPolicy](#) `entity_factory`
Entity factory policy, [ENTITY_FACTORY](#).
- struct [DDS_DiscoveryQosPolicy](#) `discovery`
<<eXtension>> Discovery policy, [DISCOVERY](#).
- struct [DDS_DomainParticipantResourceLimitsQosPolicy](#) `resource_limits`
<<eXtension>> Domain participant resource limits policy, [DOMAIN_PARTICIPANT_RESOURCE_LIMITS](#).
- struct [DDS_EntityNameQosPolicy](#) `participant_name`
<<eXtension>> The participant name. [ENTITY_NAME](#)
- struct [DDS_WireProtocolQosPolicy](#) `protocol`
<<eXtension>> Wire Protocol policy, [WIRE_PROTOCOL](#).
- struct [DDS_TransportQosPolicy](#) `transports`
<<eXtension>> Available transports for all communication to and from this DomainParticipant.
- struct [DDS_UserTrafficQosPolicy](#) `user_traffic`
<<eXtension>> Transports enabled for user traffic.
- struct [DDS_TrustQosPolicy](#) `trust`
<<eXtension>> The name of the Lightweight Security Plugin that will be used for Trust communications.
- struct [DDS_PropertyQosPolicy](#) `property`
<<eXtension>> The [DDSDomainParticipant](#) properties. Please refer to the [Property Reference](#) for available properties.
- struct [DDS_UserDataQosPolicy](#) `user_data`
User data policy, [USER_DATA](#).
- struct [DDS_FilterQosPolicy](#) `filter`
<<experimental>> <<eXtension>> Filter policy, [FILTER](#).

7.22.1 Detailed Description

QoS policies supported by a [DDSDomainParticipant](#) entity.

The Qos policy must be set in a consistent manner, otherwise [DDSDomainParticipant::set_qos](#) and [DDSDomainParticipantFactory::set_de](#) will fail with [DDS_RETCODE_INCONSISTENT_POLICY](#), and [DDSDomainParticipantFactory::create_participant](#) will fail.

Entity:

[DDSDomainParticipant](#)

See also

[QoS Policies](#) and allowed ranges within each Qos.

7.22.2 Member Data Documentation

`entity_factory`

```
struct DDS\_EntityFactoryQosPolicy DDS_DomainParticipantQos::entity_factory
```

Entity factory policy, [ENTITY_FACTORY](#).

discovery

```
struct DDS_DiscoveryQosPolicy DDS_DomainParticipantQos::discovery
```

<<*eXtension*>> Discovery policy, [DISCOVERY](#).

resource_limits

```
struct DDS_DomainParticipantResourceLimitsQosPolicy DDS_DomainParticipantQos::resource_limits
```

<<*eXtension*>> Domain participant resource limits policy, [DOMAIN_PARTICIPANT_RESOURCE_LIMITS](#).

participant_name

```
struct DDS_EntityNameQosPolicy DDS_DomainParticipantQos::participant_name
```

<<*eXtension*>> The participant name. [ENTITY_NAME](#)

protocol

```
struct DDS_WireProtocolQosPolicy DDS_DomainParticipantQos::protocol
```

<<*eXtension*>> Wire Protocol policy, [WIRE_PROTOCOL](#).

The wire protocol (RTPS) attributes associated with the participant.

transports

```
struct DDS_TransportQosPolicy DDS_DomainParticipantQos::transports
```

<<*eXtension*>> Available transports for all communication to and from this DomainParticipant.

user_traffic

```
struct DDS_UserTrafficQosPolicy DDS_DomainParticipantQos::user_traffic
```

<<*eXtension*>> Transports enabled for user traffic.

trust

```
struct DDS_TrustQosPolicy DDS_DomainParticipantQos::trust
```

<<*eXtension*>> The name of the Lightweight Security Plugin that will be used for Trust communications.

The [DDSDomainParticipantFactory](#) will search through a list of registered plugins. If it finds a registered Lightweight Security factory with a name matching this factory name, it will use that plugin to create a new instance and register the plugin with the [DDSDomainParticipant](#). If it does not find a registered Lightweight Security factory with this name, [DDSDomainParticipant](#) creation will fail.

Precondition

One or more Lightweight Security plugins must have been previously registered with the [DDSDomainParticipantFactory](#), each with an associated service name using `::DDSDDS_PskLibrary::register`.

property

```
struct DDS_PropertyQosPolicy DDS_DomainParticipantQos::property
```

<<*eXtension*>> The [DDSDomainParticipant](#) properties. Please refer to the [Property Reference](#) for available properties.

user_data

```
struct DDS_UserDataQosPolicy DDS_DomainParticipantQos::user_data
```

User data policy, [USER_DATA](#).

filter

```
struct DDS_FilterQosPolicy DDS_DomainParticipantQos::filter
```

<<*experimental*>> <<*eXtension*>> Filter policy, [FILTER](#).

7.23 DDS_DomainParticipantResourceLimitsQosPolicy Struct Reference

Resource limits that apply only to [DDSDomainParticipant](#) instances.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_Long local_writer_allocation](#)
The maximum number of local Writers that can be created.
- [DDS_Long local_reader_allocation](#)
The maximum number of local Readers that can be created.
- [DDS_Long local_publisher_allocation](#)
The maximum number of local Publishers that can be created.
- [DDS_Long local_subscriber_allocation](#)
The maximum number of local Subscribers that can be created.
- [DDS_Long local_topic_allocation](#)
The maximum number of topics that can be created.
- [DDS_Long local_type_allocation](#)
The maximum number of local types that can be registered with this DomainParticipant.
- [DDS_Long remote_participant_allocation](#)
The maximum number of remote participants that can be discovered by this DomainParticipant.
- [DDS_Long remote_writer_allocation](#)
The maximum number of remote writers that can be discovered by this DomainParticipant.
- [DDS_Long remote_reader_allocation](#)
The maximum number of remote readers that can be discovered by this DomainParticipant.
- [DDS_Long matching_writer_reader_pair_allocation](#)
The maximum number of matching local writer and remote/local reader pairs.
- [DDS_Long matching_reader_writer_pair_allocation](#)
The maximum number of matching local reader and remote/local writer pairs contained in this DomainParticipant.
- [DDS_Long max_receive_ports](#)
Maximum number of addresses that can be listened on.
- [DDS_Long max_destination_ports](#)
Maximum number of unique destination addresses that a participant can reach.
- [DDS_UnsignedLong shmем_ref_transfer_mode_max_segments](#)
Maximum number of segments created by all [DDSDataWriter](#) entities belonging to a [DDSDomainParticipant](#).
- [DDS_Long participant_user_data_max_length](#)
Maximum length of user data in [DDS_DomainParticipantQos](#) and [DDS_ParticipantBuiltinTopicData](#).
- [DDS_Long participant_user_data_max_count](#)
Maximum number of unique user data sequences allowed across DomainParticipants (the local [DDS_DomainParticipantQos](#) and discovered participants).
- [DDS_Long topic_data_max_length](#)
Maximum length of topic data in [DDS_TopicQos](#), [DDS_PublicationBuiltinTopicData](#), and [DDS_SubscriptionBuiltinTopicData](#).
- [DDS_Long topic_data_max_count](#)
Maximum number of unique topic data sequences allowed in the [DDS_TopicQos](#) across all local topics, discovered publications, and discovered subscriptions.
- [DDS_Long publisher_group_data_max_length](#)
Maximum length of group data in [DDS_PublisherQos](#) and [DDS_PublicationBuiltinTopicData](#).
- [DDS_Long publisher_group_data_max_count](#)
Maximum number of unique group data sequences allowed in the [DDS_PublisherQos](#) across all local publishers and discovered publications.
- [DDS_Long subscriber_group_data_max_length](#)
Maximum length of group data in [DDS_SubscriberQos](#) and [DDS_SubscriptionBuiltinTopicData](#).
- [DDS_Long subscriber_group_data_max_count](#)

Maximum number of unique group data sequences allowed in the [DDS_SubscriberQos](#) across all local subscribers and discovered subscriptions.

- [DDS_Long writer_user_data_max_length](#)

Maximum length of user data in [DDS_DataWriterQos](#) and [DDS_PublicationBuiltinTopicData](#).

- [DDS_Long writer_user_data_max_count](#)

Maximum number of unique user data sequences allowed in the [DDS_DataWriterQos](#) across all local [DDSDataReader](#) entities and discovered publications.

- [DDS_Long reader_user_data_max_length](#)

Maximum length of user data in [DDS_DataReaderQos](#) and [DDS_SubscriptionBuiltinTopicData](#).

- [DDS_Long reader_user_data_max_count](#)

Maximum number of unique user data sequences allowed in the [DDS_DataReaderQos](#) across all local [DDSDataReader](#) entities and discovered subscriptions.

- [DDS_Long max_partitions](#)

The maximum number of elements in the [DDS_PartitionQosPolicy](#).

- [DDS_Long max_partition_cumulative_characters](#)

The maximum number of combined characters across all [DDS_PartitionQosPolicy](#) [PARTITION.name](#) sequence elements, including the NUL characters.

- [DDS_Long max_partition_string_size](#)

The maximum number of 8-bit characters in a [PARTITION.name](#) element, including the NUL termination.

- [DDS_Long max_partition_string_allocation](#)

The maximum number of 8-bit characters reserved for [PARTITION.name](#) strings to allocate for the [PARTITION](#) [DDS_PartitionQosPolicy](#).

7.23.1 Detailed Description

Resource limits that apply only to [DDSDomainParticipant](#) instances.

This QoS policy is an extension to the DDS standard.

Entity:

[DDSDomainParticipant](#)

Properties:

[RxO](#) = N/A

[Changeable](#) = NO

7.23.2 Member Data Documentation

local_writer_allocation

[DDS_Long](#) [DDS_DomainParticipantResourceLimitsQosPolicy::local_writer_allocation](#)

The maximum number of local Writers that can be created.

This resource-limit limits the maximum number of [DDSDataWriter](#) entities that can be created with the [DDSPublisher::create_datawriter\(\)](#) API within a single [DDSDomainParticipant](#) across all [DDSPublisher](#) entities.

[default] 1

local_reader_allocation

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::local_reader_allocation`

The maximum number of local Readers that can be created.

This resource-limit limits the maximum number of [DDSDataReader](#) entities that can be created with the [DDSSubscriber::create_datareader\(\)](#) API within a single [DDSDomainParticipant](#) across all [DDSSubscriber](#) entities.

[default] 1

local_publisher_allocation

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::local_publisher_allocation`

The maximum number of local Publishers that can be created.

This resource-limit limits the maximum number of [DDSPublisher](#) entities that can be created within a [DDSDomainParticipant](#) with the [DDSDomainParticipant::create_publisher\(\)](#) API.

[default] 1

local_subscriber_allocation

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::local_subscriber_allocation`

The maximum number of local Subscribers that can be created.

This resource-limit limits the maximum number of [DDSSubscriber](#) entities that can be created within a [DDSDomainParticipant](#) with the [DDSDomainParticipant::create_subscriber\(\)](#) API.

[default] 1

local_topic_allocation

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::local_topic_allocation`

The maximum number of topics that can be created.

This resource-limit limits the maximum number of unique [DDSTopic](#) instances that can be created within a single [DDSDomainParticipant](#) with the [DDSDomainParticipant::create_topic\(\)](#) API.

[default] 1

local_type_allocation

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::local_type_allocation`

The maximum number of local types that can be registered with this DomainParticipant.

This resource-limit limits the maximum number of unique `::DDS_TypePluginI` instances that can be registered locally within a single `DDSDomainParticipant` with the `FooTypeSupport::register_type()` API.

[default] 1

remote_participant_allocation

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::remote_participant_allocation`

The maximum number of remote participants that can be discovered by this DomainParticipant.

This resource-limit limits the maximum number of `DDSDomainParticipant` instances that can be discovered by a local `DDSDomainParticipant`.

The DPDE discovery plugin is on a first-discovered, first-served basis, and the only way to limit discovery is careful use of peer addresses. The DPSE discovery plugin limits the discovery to those that have been asserted based on the DomainParticipant name using `DPSEDiscoveryPlugin::RemoteParticipant_assert`

[default] 1

remote_writer_allocation

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::remote_writer_allocation`

The maximum number of remote writers that can be discovered by this DomainParticipant.

This resource-limit limits the maximum number of remote `DDSDataWriter` entities that can be discovered by a local `DDSDomainParticipant`. Note that a remote `DDSDataWriter` cannot be discovered until its parent `DDSDomainParticipant` has been discovered.

[default] 1

remote_reader_allocation

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::remote_reader_allocation`

The maximum number of remote readers that can be discovered by this DomainParticipant.

This resource-limit limits the maximum number of remote `DDSDataReader` entities that can be discovered by a local DomainParticipant. Note that a remote `DDSDataReader` cannot be discovered until its parent `DDSDomainParticipant` has been discovered.

[default] 1

matching_writer_reader_pair_allocation

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::matching_writer_reader_pair_allocation`

The maximum number of matching local writer and remote/local reader pairs.

When RTI Connext DDS Micro discovers DDS entities using a DDS discovery protocol, such as DPSE (Dynamic Participant Static Endpoint) or DPDE (Dynamic Participant Dynamic Endpoint), it matches the discovered entities with the locally created [DDSDataReader](#) entities and [DDSDataWriter](#) entities. For each match, either a local [DDSDataWriter](#) matching a remote [DDSDataReader](#) or a local [DDSDataReader](#) matching a remote [DDSDataWriter](#), internal state must be managed.

A match, in this context, is defined as being able to receive packets sent from a [DDSDataReader](#) to a [DDSDataWriter](#), or from a [DDSDataWriter](#) to a [DDSDataReader](#):

- A data-writer needs one resource for each [DDSDataReader](#) it matches.
- A data-reader needs one resource for each [DDSDataWriter](#) it matches.

This resource-limit limits the maximum number of matches that can be managed. When this resource limit is reached, no further matching can occur, even if new [DDSDataReader](#) and [DDSDataWriter](#) entities are discovered.

A simple rule of thumb is to set this resource limit to:

- $(\text{local_reader_allocation} + \text{local_writer_allocation}) * \text{max_remote_participants}$.

This assumes that each [DDSDataReader](#) and [DDSDataWriter](#) matches one [DDSDataWriter](#) and [DDSDataReader](#) respectively for each discovered participant. However, this may over-allocate resources.

[default] 32

matching_reader_writer_pair_allocation

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::matching_reader_writer_pair_allocation`

The maximum number of matching local reader and remote/local writer pairs contained in this DomainParticipant.

A resource is used for each discovery match between a reader of the participant and a writer of this or another participant.

[default] 32

max_receive_ports

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::max_receive_ports`

Maximum number of addresses that can be listened on.

RTI Connex DDS Micro listens for discovery data and user data on different transports as specified by the user. This resource-limit limits the maximum number of addresses that can be listened on. As a rule of thumb, this resource limit can be determined as follows: - A user-traffic unicast port counts as 1. - A meta-traffic unicast port counts as 1. - A user-traffic multicast address counts as 1. - A meta-traffic multicast address counts as 1.

Consider the typical case with the UDP transport, using unicast and multicast for discovery data and unicast for user data:

- Discovery data needs two ports.
- User data needs one port.

Thus, this requires 3 resources.

Consider a case with the UDP transport, using unicast and multicast for discovery data and user data: - Discovery data needs two ports. - User data needs two ports.

Thus, this requires 4 resources.

Consider a case with the UDP transport, using unicast and 2 multicast addresses each for discovery data and user data:

- Discovery data needs three ports.
- User data needs three ports.

Thus, this requires 6 resources.

The default value has been set to cover most cases. This resource-limit should only be changed if a lot of interfaces are supported (increase it) or if memory is extremely limited (decrease it).

[default] 8

max_destination_ports

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::max_destination_ports`

Maximum number of unique destination addresses that a participant can reach.

RTI Connex DDS Micro maintains a table of destination addresses that it can send to based on information queried from the registered transports. This resource limit is difficult to give a precise rule for how to determine, since it is transport dependent. However, as a rule of thumb, the following is true: - Each network that can be reached directly by the transport requires 1 resource. - If multicast is supported, it counts as 1 resource. - If the transport is the default transport (by default, UDP is), it counts as one address.

Consider a typical Linux computer with 2 network interfaces, eth0 and lo, with support for multicast:

- 1 for the eth0 network.
- 1 for the lo network.
- 1 for multicast.
- 1 for being the default transport.

This requires 4 resources.

Consider a typical embedded computer with 1 network interface, eth0, with no support for multicast:

- 1 for the eth0 network.
- 1 for multicast.
- 1 for being the default transport.

This requires 3 resources.

The default value has been set to cover most cases. This resource-limit should only be changed if a lot of interfaces are supported (increase it) or if memory is extremely limited (decrease it).

[default] 8

shmem_ref_transfer_mode_max_segments

`DDS_UnsignedLong DDS_DomainParticipantResourceLimitsQosPolicy::shmem_ref_transfer_mode_max_↔ segments`

Maximum number of segments created by all [DDSDataWriter](#) entities belonging to a [DDSDomainParticipant](#).

[default] 500

[range] [0,100000] but in practice, this value will be limited by the system-wide maximum number of shared memory segments. On a Linux machine, this value is provided by the kernel parameter `shmni`.

participant_user_data_max_length

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::participant_user_data_max_length`

Maximum length of user data in `DDS_DomainParticipantQos` and `DDS_ParticipantBuiltinTopicData`.

[default] 0

participant_user_data_max_count

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::participant_user_data_max_count`

Maximum number of unique user data sequences allowed across DomainParticipants (the local `DDS_DomainParticipantQos` and discovered participants).

For example, if a discovered participant has the same user data as the local participant, then that unique sequence only counts once against this resource-limit.

[default] DDS_SIZE_AUTO

topic_data_max_length

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::topic_data_max_length`

Maximum length of topic data in `DDS_TopicQos`, `DDS_PublicationBuiltinTopicData`, and `DDS_SubscriptionBuiltinTopicData`.

[default] 0

topic_data_max_count

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::topic_data_max_count`

Maximum number of unique topic data sequences allowed in the `DDS_TopicQos` across all local topics, discovered publications, and discovered subscriptions.

For example, if a local topic has the same topic data as the topic of a discovered publication or subscription, then that unique sequence only counts once against this resource-limit.

[default] DDS_SIZE_AUTO

publisher_group_data_max_length

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::publisher_group_data_max_length`

Maximum length of group data in `DDS_PublisherQos` and `DDS_PublicationBuiltinTopicData`.

[default] 0

publisher_group_data_max_count

[DDS_Long](#) DDS_DomainParticipantResourceLimitsQosPolicy::publisher_group_data_max_count

Maximum number of unique group data sequences allowed in the [DDS_PublisherQos](#) across all local publishers and discovered publications.

For example, if a local publisher has the same group data as a discovered publication, then that unique sequence only counts once against this resource-limit.

[default] DDS_SIZE_AUTO

subscriber_group_data_max_length

[DDS_Long](#) DDS_DomainParticipantResourceLimitsQosPolicy::subscriber_group_data_max_length

Maximum length of group data in [DDS_SubscriberQos](#) and [DDS_SubscriptionBuiltinTopicData](#).

[default] 0

subscriber_group_data_max_count

[DDS_Long](#) DDS_DomainParticipantResourceLimitsQosPolicy::subscriber_group_data_max_count

Maximum number of unique group data sequences allowed in the [DDS_SubscriberQos](#) across all local subscribers and discovered subscriptions.

For example, if a local subscriber has the same group data as a discovered subscription, then that unique sequence only counts once against this resource-limit.

[default] DDS_SIZE_AUTO

writer_user_data_max_length

[DDS_Long](#) DDS_DomainParticipantResourceLimitsQosPolicy::writer_user_data_max_length

Maximum length of user data in [DDS_DataWriterQos](#) and [DDS_PublicationBuiltinTopicData](#).

[default] 0

writer_user_data_max_count

[DDS_Long](#) DDS_DomainParticipantResourceLimitsQosPolicy::writer_user_data_max_count

Maximum number of unique user data sequences allowed in the [DDS_DataWriterQos](#) across all local [DDSDDataReader](#) entities and discovered publications.

For example, if a local [DDSDDataWriter](#) has the same user data as a discovered publication, then that unique sequence only counts once against this resource-limit.

[default] DDS_SIZE_AUTO

reader_user_data_max_length

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::reader_user_data_max_length`

Maximum length of user data in `DDS_DataReaderQos` and `DDS_SubscriptionBuiltinTopicData`.

[default] 0

reader_user_data_max_count

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::reader_user_data_max_count`

Maximum number of unique user data sequences allowed in the `DDS_DataReaderQos` across all local `DDSDataReader` entities and discovered subscriptions.

For example, if a local `DDSDataReader` has the same user data as a discovered subscription, then that unique sequence only counts once against this resource-limit.

[default] `DDS_SIZE_AUTO`

max_partitions

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::max_partitions`

The maximum number of elements in the `DDS_PartitionQosPolicy`.

This setting is made on a per DomainParticipant basis; it cannot be set individually on a per Publisher/Subscriber basis. However, the limit is enforced and applies per Publisher/Subscriber.

This value cannot exceed 64.

[default] 64

[range] [0,64]

max_partition_cumulative_characters

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::max_partition_cumulative_characters`

The maximum number of combined characters across all `DDS_PartitionQosPolicy` `PARTITION.name` sequence elements, including the NUL characters.

The maximum number of combined characters should account for a terminating NULL ('\0') character for each partition name string.

This setting is made on a per DomainParticipant basis; it cannot be set individually on a per Publisher/Subscriber basis. However, the limit is enforced and applies per Publisher/Subscriber.

This value cannot exceed 256.

[default] 256

[range] [0,256]

max_partition_string_size

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::max_partition_string_size`

The maximum number of 8-bit characters in a PARTITION.name element, including the NUL termination.

[default] DDS_LENGTH_UNLIMITED

[range] DDS_LENGTH_UNLIMITED or greater than 0

max_partition_string_allocation

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::max_partition_string_allocation`

The maximum number of 8-bit characters reserved for PARTITION.name strings to allocate for the PARTITION [DDS_PartitionQosPolicy](#).

If max_partition_string_allocation is DDS_LENGTH_UNLIMITED then memory will be allocated and freed during operation.

This setting is made on a per DomainParticipant basis; it cannot be set individually on a per Publisher/Subscriber basis. However, the limit is enforced and applies per Publisher/Subscriber.

[default] DDS_LENGTH_UNLIMITED

[range] DDS_LENGTH_UNLIMITED or greater than 1

7.24 DDS_DurabilityQosPolicy Struct Reference

Durability properties that affect the life-cycle of published data.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_DurabilityQosPolicyKind kind](#)
The kind of durability that applies to published data.

7.24.1 Detailed Description

Durability properties that affect the life-cycle of published data.

7.24.2 Member Data Documentation

kind

`DDS_DurabilityQosPolicyKind DDS_DurabilityQosPolicy::kind`

The kind of durability that applies to published data.

7.25 DDS_Duration_t Struct Reference

Type for *duration* representation.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- `DDS_Long sec`
seconds
- `DDS_UnsignedLong nanosec`
nanoseconds

7.25.1 Detailed Description

Type for *duration* representation.

Represents a time interval.

7.25.2 Member Data Documentation

sec

`DDS_Long DDS_Duration_t::sec`

seconds

nanosec

`DDS_UnsignedLong DDS_Duration_t::nanosec`

nanoseconds

7.26 DDS_EntityFactoryQosPolicy Struct Reference

A QoS policy for all [DDSEntity](#) types that can act as factories for one or more other [DDSEntity](#) types.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_Boolean autoenable_created_entities](#)

Specifies whether the entity acting as a factory automatically enables the instances it creates.

7.26.1 Detailed Description

A QoS policy for all [DDSEntity](#) types that can act as factories for one or more other [DDSEntity](#) types.

Entity:

[DDSDomainParticipantFactory](#), [DDSDomainParticipant](#), [DDSPublisher](#), [DDSSubscriber](#)

Properties:

[RxO](#) = NO

[Changeable](#) = YES

7.26.2 Usage

This policy controls the behavior of the [DDSEntity](#) as a factory for other entities.

This policy is mutable. A change in the policy affects only the entities created after the change, not the previously created entities.

The setting of `autoenable_created_entities` to [DDS_BOOLEAN_TRUE](#) indicates that the factory `create_<entity>` operation will automatically invoke the [DDSEntity::enable](#) operation each time a new [DDSEntity](#) is created. Therefore, the [DDSEntity](#) returned by `create_<entity>` will already be enabled. A setting of [DDS_BOOLEAN_FALSE](#) indicates that the [DDSEntity](#) will not be automatically enabled. The application can enable entities explicitly using the [DDSEntity::enable](#) operation.

By default, `autoenable_created_entities` is set to [DDS_BOOLEAN_TRUE](#), so newly created entities are automatically enabled without needing to call [DDSEntity::enable](#).

7.26.3 Member Data Documentation

autoenable_created_entities

`DDS_Boolean DDS_EntityFactoryQosPolicy::autoenable_created_entities`

Specifies whether the entity acting as a factory automatically enables the instances it creates.

If `DDS_BOOLEAN_TRUE`, the factory will automatically enable each created entity. Otherwise, it will not automatically enable created entities.

[default] `DDS_BOOLEAN_TRUE`

7.27 DDS_EntityNameQosPolicy Struct Reference

Policy used to name an entity.

```
#include <dds_c_infrastructure.h>
```

Public Member Functions

- bool `set_name` (const char *const `name`)
Set the EntityName policy

Public Attributes

- char `name` [`DDS_ENTITYNAME_QOS_NAME_MAX+1`]
The actual name string.

7.27.1 Detailed Description

Policy used to name an entity.

Entity:

`DDSDomainParticipant`

Properties:

`RxO` = NO;
`Changeable` = NO

7.27.2 Usage

The purpose of this QoS is to allow the user to attach naming information to created [DDSEntity](#) objects.

The name must not exceed 255 characters in length.

The name must be a NULL-terminated string allocated with [DDS_String_alloc](#) or [DDS_String_dup](#).

When getting the QoS with a non-NULL pointer, ensure it points to valid memory. This prevents potential failures.

Use [DDS_EntityNameQosPolicy_set_name](#) to set the name.

7.27.3 Member Data Documentation

name

```
char DDS_EntityNameQosPolicy::name[DDS_ENTITYNAME_QOS_NAME_MAX+1]
```

The actual name string.

[default] "[ENTITY]"

[range] Null-terminated string with maximum length of 255 characters, or NULL.

Referenced by [set_name\(\)](#).

7.28 DDS_FilterQosPolicy Struct Reference

Configures the content filter capabilities and resource limits for a [DDSDomainParticipant](#) and all of its child entities.

```
#include <dds_c_content_filter.h>
```

Public Attributes

- [RT_ComponentFactoryId_T](#) [name](#)
Name of the filter plugin factory to use.
- [struct DDS_FilterResourceLimits](#) [resource_limits](#)
Resource limits for content filtering.
- [DDS_Boolean](#) [disable_writer_filtering](#)
Controls whether writer-side filtering is disabled.
- [DDS_Boolean](#) [disable_builtin_sql_filter](#)
Controls whether the built-in SQL filter is disabled.

7.28.1 Detailed Description

Configures the content filter capabilities and resource limits for a [DDSDomainParticipant](#) and all of its child entities.

Entity:

[DDSDomainParticipant](#)

Properties:

[RxO](#) = NO

[Changeable](#) = NO

7.28.2 Usage

By default if the content filtering feature is enabled, then every [DDSDomainParticipant](#) and its child entities will support content filtering. This policy allows the feature or parts of the feature to be disabled for a specific [DDSDomainParticipant](#).

It is also used to configure the resource limits for all content filters that are created by the child entities of the [DDSDomainParticipant](#). Reducing the resource limits can be useful to reduce the memory usage of the filter, or the limits can be increased to allow for more complex filters to be used.

A [DDSDataReader](#) will fail to be created if it specifies a [DDS_ContentFilterQosPolicy](#) that is not compatible with the resource limits specified in this policy for its parent [DDSDomainParticipant](#).

A [DDSDataWriter](#) will be unable to perform writer-side filtering for any [DDSDataReader](#) that specifies a content filter that is not compatible with the resource limits specified in this policy for its parent [DDSDomainParticipant](#).

See also

`::DDSTFilterPluginFactory::register` for how to enable the content filtering feature

7.28.3 Member Data Documentation

name

`RT_ComponentFactoryId_T DDS_FilterQosPolicy::name`

Name of the filter plugin factory to use.

[default] [DDS_FILTER_PLUGIN_FACTORY_DEFAULT_NAME](#)

If the `FilterPluginFactory` is registered with this name, then the [DDSDomainParticipant](#) will use that filter plugin to support content filtering.

If it is cleared (set to an empty string), then content filtering is disabled for the [DDSDomainParticipant](#) and all of its child entities.

If it is set to a different name than [DDS_FILTER_PLUGIN_FACTORY_DEFAULT_NAME](#), and that factory cannot be found, then the [DDSDomainParticipant](#) will fail to be created with this policy.

resource_limits

```
struct DDS_FilterResourceLimits DDS_FilterQosPolicy::resource_limits
```

Resource limits for content filtering.

[default] [DDS_FILTER_RESOURCE_LIMITS_DEFAULT](#)

Defines the resource constraints for the content filters created by the child entities of a [DDSDomainParticipant](#).

disable_writer_filtering

```
DDS_Boolean DDS_FilterQosPolicy::disable_writer_filtering
```

Controls whether writer-side filtering is disabled.

[default] [DDS_BOOLEAN_FALSE](#)

When set to [DDS_BOOLEAN_TRUE](#), prevents all child [DDSDataWriter](#) entities of a [DDSDomainParticipant](#) from performing writer-side filtering. This reduces the memory usage of the content filtering feature, but also prevents the [DDSDataWriter](#) from filtering data before it is sent to a [DDSDataReader](#) that has a content filter.

disable_builtin_sql_filter

```
DDS_Boolean DDS_FilterQosPolicy::disable_builtin_sql_filter
```

Controls whether the built-in SQL filter is disabled.

[default] [DDS_BOOLEAN_FALSE](#)

By default, a built-in SQL content filter is enabled that allows filtering based on SQL-like expressions.

When set to [DDS_BOOLEAN_TRUE](#), this disables the built-in SQL content filter from being automatically enabled.

7.29 DDS_FilterResourceLimits Struct Reference

Resource limits for content filtering on a [DDSDomainParticipant](#) and all of its child entities.

```
#include <dds_c_content_filter.h>
```

Public Attributes

- [DDS_Long filter_class_max_length](#)
Maximum length of a filter class name.
- [DDS_Long filter_class_max_count](#)
Maximum number of filter classes supported.
- [DDS_Long filter_expression_max_length](#)
Maximum length of a filter expression.
- [DDS_Long filter_expression_max_count](#)
Maximum number of unique filter expressions supported.
- [DDS_Long filter_parameter_max_count_per_expression](#)
Maximum number of filter parameters per filter expression.
- [DDS_Long filter_parameter_max_length](#)
Maximum length of a filter parameter.
- [DDS_Long sql_predicate_max_count_per_expression](#)
Maximum number of SQL predicates per filter expression.

7.29.1 Detailed Description

Resource limits for content filtering on a [DDSDomainParticipant](#) and all of its child entities.

The resources used for content filtering are all pre-allocated and shared by all the child entities of a [DDSDomainParticipant](#). This includes the filters specified by a [DDSDataReader](#) in its [DDS_ContentFilterQosPolicy](#) and the filters discovered by a [DDSDataWriter](#) for remote [DDSDataReader](#) entities.

7.29.2 Member Data Documentation

filter_class_max_length

[DDS_Long](#) `DDS_FilterResourceLimits::filter_class_max_length`

Maximum length of a filter class name.

[default] 7

[range] [1, [DDS_CONTENT_FILTER_CLASS_NAME_MAX_LENGTH](#)]

filter_class_max_count

[DDS_Long](#) `DDS_FilterResourceLimits::filter_class_max_count`

Maximum number of filter classes supported.

[default] 1

[range] [1, 2147483647]

filter_expression_max_length

`DDS_Long DDS_FilterResourceLimits::filter_expression_max_length`

Maximum length of a filter expression.

[default] 63

[range] [1, [DDS_CONTENT_FILTER_EXPRESSION_MAX_LENGTH](#)]

filter_expression_max_count

`DDS_Long DDS_FilterResourceLimits::filter_expression_max_count`

Maximum number of unique filter expressions supported.

[default] DDS_LENGTH_AUTO

[range] [1, 2147483647]

If set to DDS_LENGTH_AUTO, then enough resources will be allocated for every local and remote [DDSDataReader](#) to have a unique filter expression.

filter_parameter_max_count_per_expression

`DDS_Long DDS_FilterResourceLimits::filter_parameter_max_count_per_expression`

Maximum number of filter parameters per filter expression.

[default] 4

[range] [1, [DDS_CONTENT_FILTER_MAX_PARAMETERS](#)]

This is the maximum number of parameters that can be specified in a single filter expression.

filter_parameter_max_length

`DDS_Long DDS_FilterResourceLimits::filter_parameter_max_length`

Maximum length of a filter parameter.

[default] 15

[range] [1, 2147483647]

sql_predicate_max_count_per_expression

`DDS_Long DDS_FilterResourceLimits::sql_predicate_max_count_per_expression`

Maximum number of SQL predicates per filter expression.

[default] 4

[range] [1, 2147483647]

When using the built-in SQL content filter, this is the maximum number of SQL predicates that can be specified in a single filter expression.

When not using the built-in SQL content filter, this value may or may not be used, depending on the filter implementation.

7.30 DDS_FlowControllerProperty_t Struct Reference

Determines the flow control characteristics of the [DDSFlowController](#).

```
#include <dds_c_flowcontroller.h>
```

Public Attributes

- [DDS_FlowControllerSchedulingPolicy](#) `scheduling_policy`
Scheduling policy.
- struct [DDS_FlowControllerTokenBucketProperty_t](#) `token_bucket`
Settings for the token bucket.

7.30.1 Detailed Description

Determines the flow control characteristics of the [DDSFlowController](#).

The flow control characteristics shape the network traffic by determining how often and in what order associated asynchronous [DDSDataWriter](#) instances are serviced and how much data they are allowed to send.

Note that these settings apply directly to the [DDSFlowController](#), and do not depend on the number of [DDSDataWriter](#) instances the [DDSFlowController](#) is servicing. For instance, the specified flow rate does *not* double simply because two [DDSDataWriter](#) instances are waiting to write.

Entity:

[DDSFlowController](#)

Properties:

`RxO` = N/A

`Changeable` = NO for `DDS_FlowControllerProperty_t::scheduling_policy`, YES for `DDS_FlowControllerProperty_t::token_bucket`. However, the special value of `DDS_DURATION_INFINITE` as `DDS_FlowControllerTokenBucketProperty_t::period` is strictly used to create an *on-demand* [DDSFlowController](#). The token period cannot toggle from an infinite to finite value (or vice versa). It can, however, change from one finite value to another.

7.30.2 Member Data Documentation

scheduling_policy

[DDS_FlowControllerSchedulingPolicy](#) DDS_FlowControllerProperty_t::scheduling_policy

Scheduling policy.

Determines the scheduling policy for servicing the [DDSDataWriter](#) instances associated with the [DDSFlowController](#).

[default] [DDS_EDF_FLOW_CONTROLLER_SCHED_POLICY](#)

token_bucket

struct [DDS_FlowControllerTokenBucketProperty_t](#) DDS_FlowControllerProperty_t::token_bucket

Settings for the token bucket.

7.31 DDS_FlowControllerTokenBucketProperty_t Struct Reference

[DDSFlowController](#) uses the popular token bucket approach for open loop network flow control. The flow control characteristics are determined by the token bucket properties.

```
#include <dds_c_flowcontroller.h>
```

Public Attributes

- [DDS_Long max_tokens](#)
Maximum number of tokens than can accumulate in the token bucket.
- [DDS_Long tokens_added_per_period](#)
The number of tokens added to the token bucket per specified period.
- [DDS_Long tokens_leaked_per_period](#)
The number of tokens removed from the token bucket per specified period.
- struct [DDS_Duration_t period](#)
Period for adding tokens to and removing tokens from the bucket.
- [DDS_Long bytes_per_token](#)
Maximum number of bytes allowed to send for each token available.

7.31.1 Detailed Description

[DDSFlowController](#) uses the popular token bucket approach for open loop network flow control. The flow control characteristics are determined by the token bucket properties.

Asynchronously published samples are queued up and transmitted based on the token bucket flow control scheme. The token bucket contains tokens, each of which represents a number of bytes. Samples can be sent only when there are sufficient tokens in the bucket. As samples are sent, tokens are consumed. The number of tokens consumed is proportional to the size of the data being sent. Tokens are replenished on a periodic basis.

The rate at which tokens become available and other token bucket properties determine the network traffic flow.

Note that if the same sample must be sent to multiple destinations, separate tokens are required for each destination. Only when multiple samples are destined to the same destination will they be co-alesced and sent using the same token(s). In other words, each token can only contribute to a single network packet.

Entity:

[DDSFlowController](#)

Properties:

[RxO](#) = N/A

[Changeable](#) = YES. However, the special value of [DDS_DURATION_INFINITE](#) as [DDS_FlowControllerTokenBucketProperty_t::period](#) is strictly used to create an *on-demand* [DDSFlowController](#). The token period cannot toggle from an infinite to finite value (or vice versa). It can, however, change from one finite value to another.

7.31.2 Member Data Documentation

max_tokens

[DDS_Long](#) [DDS_FlowControllerTokenBucketProperty_t::max_tokens](#)

Maximum number of tokens than can accumulate in the token bucket.

The number of tokens in the bucket will never exceed this value. Any excess tokens are discarded. This property value, combined with [DDS_FlowControllerTokenBucketProperty_t::bytes_per_token](#), determines the maximum allowable data burst.

Use [DDS_LENGTH_UNLIMITED](#) to allow accumulation of an unlimited amount of tokens (and therefore potentially an unlimited burst size).

[default] [DDS_LENGTH_UNLIMITED](#)

[range] [1,[DDS_LENGTH_UNLIMITED](#)]

tokens_added_per_period

`DDS_Long DDS_FlowControllerTokenBucketProperty_t::tokens_added_per_period`

The number of tokens added to the token bucket per specified period.

[DDSFlowController](#) transmits data only when tokens are available. Tokens are periodically replenished. This field determines the number of tokens added to the token bucket with each periodic replenishment.

Available tokens are distributed to associated [DDSDDataWriter](#) instances based on the [DDS_FlowControllerProperty_t::scheduling_policy](#).

Use `DDS_LENGTH_UNLIMITED` to add the maximum number of tokens allowed by [DDS_FlowControllerTokenBucketProperty_t::max_tokens](#).

[default] `DDS_LENGTH_UNLIMITED`

[range] `[1, DDS_LENGTH_UNLIMITED]`

tokens_leaked_per_period

`DDS_Long DDS_FlowControllerTokenBucketProperty_t::tokens_leaked_per_period`

The number of tokens removed from the token bucket per specified period.

[DDSFlowController](#) transmits data only when tokens are available. When tokens are replenished and there are sufficient tokens to send all samples in the queue, this property determines whether any or all of the leftover tokens remain in the bucket.

Use `DDS_LENGTH_UNLIMITED` to remove all excess tokens from the token bucket once all samples have been sent. In other words, no token accumulation is allowed. When new samples are written after tokens were purged, the earliest point in time at which they can be sent is at the next periodic replenishment.

[default] `0`

[range] `[0, DDS_LENGTH_UNLIMITED]`

period

`struct DDS_Duration_t DDS_FlowControllerTokenBucketProperty_t::period`

Period for adding tokens to and removing tokens from the bucket.

[DDSFlowController](#) transmits data only when tokens are available. This field determines the period by which tokens are added or removed from the token bucket.

The special value [DDS_DURATION_INFINITE](#) can be used to create an *on-demand* [DDSFlowController](#), for which tokens are no longer replenished periodically. Instead, tokens must be added explicitly by calling [DDSFlowController::trigger_flow](#). This external trigger adds [DDS_FlowControllerTokenBucketProperty_t::tokens_added_per_period](#) tokens each time it is called (subject to the other property settings).

[default] 1 second

[range] `[1 nanosec, 1 year]` or [DDS_DURATION_INFINITE](#)

bytes_per_token

`DDS_Long DDS_FlowControllerTokenBucketProperty_t::bytes_per_token`

Maximum number of bytes allowed to send for each token available.

`DDSFlowController` transmits data only when tokens are available. This field determines the number of bytes that can actually be transmitted based on the number of tokens.

Tokens are always consumed in whole by each `DDSDDataWriter`. That is, in cases where `DDS_FlowControllerTokenBucketProperty_t::bytes_per_token` is greater than the sample size, multiple samples may be sent to the same destination using a single token (regardless of `DDS_FlowControllerProperty_t::scheduling_policy`).

Where fragmentation is required, the fragment size will be `DDS_FlowControllerTokenBucketProperty_t::bytes_per_token` or the minimum largest message size across all transports installed with the `DDSDDataWriter`, whichever is less.

Use `DDS_LENGTH_UNLIMITED` to indicate that an unlimited number of bytes can be transmitted per token. In other words, a single token allows the recipient `DDSDDataWriter` to transmit all its queued samples to a single destination. A separate token is required to send to each additional destination.

[default] `DDS_LENGTH_UNLIMITED`

[range] `[1024, DDS_LENGTH_UNLIMITED]`

7.32 DDS_GroupDataQosPolicy Struct Reference

Attaches a buffer of opaque data to a `DDSEntity` which is distributed to other applications as part of the discovery process.

```
#include <dds_c_user_data_qos.h>
```

Public Attributes

- struct `DDS_OctetSeq` [value](#)

The opaque data to be attached to the `DDSEntity`.

7.32.1 Detailed Description

Attaches a buffer of opaque data to a `DDSEntity` which is distributed to other applications as part of the discovery process.

Entity:

[DDSPublisher](#), [DDSSubscriber](#)

Properties:

[RxO](#) = NO

[Changeable](#) = NO

7.32.2 Usage

The GROUP_DATA QoS policy allows an application to attach additional information to [DDSEntity](#) objects so that when a remote application discovers them, it can access that information and use it for its own purposes. This information is not interpreted by RTI Connex DDS Micro. Use cases are usually application-to-application identification, authentication, authorization, and encryption.

The group data for a remote entity is received through the discovery process. The received group data can be accessed through the functions for accessing the discovered entity data: [DDSDataWriter::get_matched_subscription_data](#) and [DDSDataReader::get_matched_publication_data](#).

Important: RTI Connex DDS Micro stores the data placed in this policy in pre-allocated pools. You must configure RTI Connex DDS Micro with the maximum size and number of the data sequences that will be stored in policies of this type. These settings are configured in the [DDS_DomainParticipantResourceLimitsQosPolicy](#).

7.32.3 Member Data Documentation

value

```
struct DDS_OctetSeq DDS_GroupDataQosPolicy::value
```

The opaque data to be attached to the [DDSEntity](#).

[default] empty (zero length)

7.33 DDS_GUID_t Struct Reference

Type for *GUID* (Global Unique Identifier) representation.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_Octet value](#) [DDS_GUID_LENGTH]
A 16 byte array containing the GUID value.

7.33.1 Detailed Description

Type for *GUID* (Global Unique Identifier) representation.

Represents a 128 bit GUID.

7.33.2 Member Data Documentation

value

[DDS_Octet](#) DDS_GUID_t::value[DDS_GUID_LENGTH]

A 16 byte array containing the GUID value.

7.34 DDS_HistoryQosPolicy Struct Reference

Specifies the behavior of RTI Connexx DDS Micro when a sample value changes one or more times before it can be successfully communicated to existing subscribers.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_HistoryQosPolicyKind](#) kind
Specifies the kind of history to be kept.
- [DDS_Long](#) depth
*Specifies the number of samples to be kept when the *kind* is [DDS_KEEP_LAST_HISTORY_QOS](#).*

7.34.1 Detailed Description

Specifies the behavior of RTI Connexx DDS Micro when a sample value changes one or more times before it can be successfully communicated to existing subscribers.

This `QosPolicy` controls whether RTI Connexx DDS Micro should deliver only the most recent value, attempt to deliver all intermediate values, or do something in between.

On the publishing side this policy controls the samples that should be maintained by the [DDSDataWriter](#) on behalf of existing [DDSDataReader](#) entities.

On the subscribing side it controls the samples that should be maintained until the application takes them from RTI Connexx DDS Micro.

RTI Connexx DDS Micro supports the [DDS_KEEP_LAST_HISTORY_QOS](#) kind.

Entity:

[DDSTopic](#), [DDSDataReader](#), [DDSDataWriter](#)

Properties:

[RxO](#) = NO
[Changeable](#) = NO

See also

[DDS_ReliabilityQosPolicy](#)
[DDS_HistoryQosPolicy](#)

7.34.2 Usage

This policy controls the behavior of RTI Connex DDS Micro when an instance value changes before it is communicated to its existing [DDSDataReader](#) entities.

When the kind is set to [DDS_KEEP_LAST_HISTORY_QOS](#), RTI Connex DDS Micro will only keep the latest values of the instance and discard older ones. The value of `depth` regulates the maximum number of values that RTI Connex DDS Micro will maintain and deliver. The default for `depth` is 1, which means only the most recent value is delivered.

7.34.3 Consistency

The `depth` setting must be consistent with the [RESOURCE_LIMITS](#) `max_samples_per_instance` setting. This requires the following constraint: $depth \leq max_samples_per_instance$.

See also

[DDS_ResourceLimitsQosPolicy](#)

7.34.4 Member Data Documentation

kind

[DDS_HistoryQosPolicyKind](#) `DDS_HistoryQosPolicy::kind`

Specifies the kind of history to be kept.

[default] [DDS_KEEP_LAST_HISTORY_QOS](#)

depth

[DDS_Long](#) `DDS_HistoryQosPolicy::depth`

Specifies the number of samples to be kept when the `kind` is [DDS_KEEP_LAST_HISTORY_QOS](#).

If a value other than 1 is specified, ensure it is consistent with the [RESOURCE_LIMITS](#) policy:

$depth \leq \text{DDS_ResourceLimitsQosPolicy::max_samples_per_instance}$

[default] 1

[range] [1,2147483647]

7.35 DDS_InconsistentTopicStatus Struct Reference

[DDS_INCONSISTENT_TOPIC_STATUS](#)

```
#include <dds_c_topic.h>
```

7.35.1 Detailed Description

DDS_INCONSISTENT_TOPIC_STATUS

Entity:

[DDSTopic](#)

Listener:

[DDSTopicListener](#)

A remote [DDSTopic](#) will be inconsistent with the locally created [DDSTopic](#) if the type name of the two topics are different.

7.36 DDS_InstanceHandleSeq Struct Reference

Instantiates [FooSeq](#) < [DDS_InstanceHandle_t](#) > .

```
#include <dds_c_infrastructure.h>
```

7.36.1 Detailed Description

Instantiates [FooSeq](#) < [DDS_InstanceHandle_t](#) > .

Instantiates:

[<<generic>> FooSeq](#)

See also

[DDS_InstanceHandle_t](#)

[FooSeq](#)

7.37 DDS_LatencyBudgetQosPolicy Struct Reference

Provides a hint as to the maximum acceptable delay from the time the data is written to the time it is received by the subscribing applications.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- struct [DDS_Duration_t](#) [duration](#)
Duration of the maximum acceptable delay.

7.37.1 Detailed Description

Provides a hint as to the maximum acceptable delay from the time the data is written to the time it is received by the subscribing applications.

This policy is a *hint* to a DDS implementation; it can be used to change how it processes and sends data that has low latency requirements. The DDS specification does not mandate whether or how this policy is used.

Entity:

[DDSTopic](#), [DDSDataReader](#), [DDSDataWriter](#)

Status:

[DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#), [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#)

Properties:

[RxO](#) = YES
[Changeable](#) = YES

See also

[DDS_PublishModeQosPolicy](#)
[DDSFlowController](#)

7.37.2 Usage

This policy provides a means for the application to indicate to the middleware the urgency of the data communication. By having a non-zero `duration`, RTI Connex DDS Micro can optimize its internal operation.

RTI Connex DDS Micro uses this policy only in conjunction with [DDS_ASYNCHRONOUS_PUBLISH_MODE_QOS](#) [DDSDataWriter](#) instances associated with a [DDS_EDF_FLOW_CONTROLLER_SCHED_POLICY](#) [DDSFlowController](#). Together with the time of write, [DDS_LatencyBudgetQosPolicy::duration](#) determines the deadline of each individual sample. RTI Connex DDS Micro uses this information to prioritize the sending of asynchronously published data.

7.37.3 Compatibility

The value offered is considered compatible with the value requested if and only if the inequality *offered duration* ≤ *requested duration* evaluates to 'TRUE'.

7.37.4 Member Data Documentation

duration

```
struct DDS_Duration_t DDS_LatencyBudgetQosPolicy::duration
```

Duration of the maximum acceptable delay.

[default] 0 (meaning minimize the delay)

7.38 DDS_LivelinessChangedStatus Struct Reference

DDS_LIVELINESS_CHANGED_STATUS

```
#include <dds_c_subscription.h>
```

Public Attributes

- [DDS_Long alive_count](#)
The total count of currently alive [DDSDDataWriter](#) entities that write the [DDSTopic](#) the [DDSDDataReader](#) reads.
- [DDS_Long not_alive_count](#)
The total count of currently not_alive [DDSDDataWriter](#) entities that write the [DDSTopic](#) the [DDSDDataReader](#) reads.
- [DDS_Long alive_count_change](#)
The change in the alive_count since the last time the listener was called or the status was read.
- [DDS_Long not_alive_count_change](#)
The change in the not_alive_count since the last time the listener was called or the status was read.
- [DDS_InstanceHandle_t last_publication_handle](#)
An instance handle to the last remote writer to change its liveliness.

7.38.1 Detailed Description

DDS_LIVELINESS_CHANGED_STATUS

7.38.2 Member Data Documentation

alive_count

```
DDS_Long DDS_LivelinessChangedStatus::alive_count
```

The total count of currently alive [DDSDDataWriter](#) entities that write the [DDSTopic](#) the [DDSDDataReader](#) reads.

not_alive_count

DDS_Long DDS_LivelinessChangedStatus::not_alive_count

The total count of currently not_alive [DDSDDataWriter](#) entities that write the [DDSTopic](#) the [DDSDDataReader](#) reads.

alive_count_change

DDS_Long DDS_LivelinessChangedStatus::alive_count_change

The change in the alive_count since the last time the listener was called or the status was read.

not_alive_count_change

DDS_Long DDS_LivelinessChangedStatus::not_alive_count_change

The change in the not_alive_count since the last time the listener was called or the status was read.

last_publication_handle

DDS_InstanceHandle_t DDS_LivelinessChangedStatus::last_publication_handle

An instance handle to the last remote writer to change its liveliness.

7.39 DDS_LivelinessLostStatus Struct Reference

DDS_LIVELINESS_LOST_STATUS

```
#include <dds_c_publication.h>
```

Public Attributes

- [DDS_Long total_count](#)

Total cumulative number of times that a previously-alive [DDSDDataWriter](#) became not alive due to a failure to to actively signal its liveliness within the offered liveliness period.

- [DDS_Long total_count_change](#)

The incremental changees in total_count since the last time the listener was called or the status was read.

7.39.1 Detailed Description

DDS_LIVELINESS_LOST_STATUS

Entity:

[DDSDataWriter](#)

Listener:

[DDSDataWriterListener](#)

The liveliness that the [DDSDataWriter](#) has committed through its [DDS_LivelinessQosPolicy](#) was not respected; thus [DDSDataReader](#) entities will consider the [DDSDataWriter](#) as no longer "alive/active".

7.39.2 Member Data Documentation

total_count

[DDS_Long](#) [DDS_LivelinessLostStatus::total_count](#)

Total cumulative number of times that a previously-alive [DDSDataWriter](#) became not alive due to a failure to to actively signal its liveliness within the offered liveliness period.

This count does not change when an already not alive [DDSDataWriter](#) simply remains not alive for another liveliness period.

total_count_change

[DDS_Long](#) [DDS_LivelinessLostStatus::total_count_change](#)

The incremental changes in total_count since the last time the listener was called or the status was read.

7.40 DDS_LivelinessQosPolicy Struct Reference

Determines the mechanism and parameters used by the application to determine whether a [DDSEntity](#) is alive.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_LivelinessQosPolicyKind](#) kind
The kind of liveliness desired.
- struct [DDS_Duration_t](#) lease_duration
The duration within which a [DDSEntity](#) must be asserted, or else it is assumed to be not alive.

7.40.1 Detailed Description

Determines the mechanism and parameters used by the application to determine whether a [DDSEntity](#) is alive.

Liveliness must be asserted at least once every `lease_duration` otherwise RTI Connex DDS Micro will assume the corresponding [DDSEntity](#) is no longer alive.

The liveliness status of a [DDSEntity](#) is used to maintain instance ownership in combination with the setting of the [OWNERSHIP](#) policy. The application also receives notification via [DDSListener](#) when an [DDSEntity](#) is no longer alive.

A [DDSDataReader](#) requests that writers maintain liveliness using the requested [DDS_LivelinessQosPolicy::kind](#). Liveliness loss must be detected within the `lease_duration`.

A [DDSDataWriter](#) commits to signalling its liveliness using the requested [DDS_LivelinessQosPolicy::kind](#) at intervals that do not exceed the `lease_duration`. A failure by the [DDSDataWriter](#) to assert liveliness within its offered `lease_duration` will result in a [DDS_LIVELINESS_LOST_STATUS](#) event.

If the [DDSDataReader](#)'s requested `lease_duration` is greater than or equal to the `lease_duration` offered by the [DDSDataWriter](#) and the offered `kind` is equal to or better than the requested `kind`, they will match. The [DDSDataReader](#) uses its own `lease_duration` as the check period to determine if a [DDSDataWriter](#) is active. A [DDSDataWriter](#) may be undetected as inactive for up to twice the [DDSDataReader](#)'s `lease_duration`. The frequency and timing of prior activity do not affect whether the writer is considered inactive.

For example: If a [DDSDataReader](#) must detect that a [DDSDataWriter](#) was inactive within at most 100ms, both the [DDSDataWriter](#) and [DDSDataReader](#) must specify a `lease_duration` of 50ms.

Listeners notify a [DDSDataReader](#) when liveliness is lost and notify a [DDSDataWriter](#) when it violates the liveliness contract. The `on_liveliness_lost()` callback is called *once*, the first time the `lease_duration` is exceeded.

Applications can use the liveliness policy to verify that systems meet design specifications during integration and at run-time. Applications can detect when systems operate outside specifications and respond appropriately to disconnected [DDSDataWriter](#) entities.

Entity:

[DDSTopic](#), [DDSDataReader](#), [DDSDataWriter](#)

Status:

[DDS_LIVELINESS_LOST_STATUS](#), [DDS_LivelinessLostStatus](#);
[DDS_LIVELINESS_CHANGED_STATUS](#), [DDS_LivelinessChangedStatus](#);
[DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#), [DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#)

Properties:

[RxO](#) = YES
[Changeable](#) = NO

7.40.2 Usage

The liveliness policy controls the mechanism and parameters used by RTI Connex DDS Micro to ensure that particular entities on the network are still alive. The liveliness policy can also affect the ownership of a particular instance, as determined by the [OWNERSHIP](#) policy.

This policy has several settings to support both data types that are updated periodically as well as those that are changed sporadically. It also allows customization for different application requirements in terms of the kinds of failures that will be detected by the liveliness mechanism.

The [DDS_AUTOMATIC_LIVELINESS_QOS](#) liveliness setting is most appropriate for applications that only need to detect process-level failures, not application-logic failures within a process. RTI Connex DDS Micro takes responsibility for renewing the leases at the required rates and thus, as long as the local process where a [DDSDomainParticipant](#) is running and the link connecting it to remote participants remain connected, RTI Connex DDS Micro considers the entities within the [DDSDomainParticipant](#) alive. This requires the lowest overhead.

The manual settings include both [DDS_MANUAL_BY_PARTICIPANT_LIVELINESS_QOS](#) and [DDS_MANUAL_BY_TOPIC_LIVELINESS_QOS](#). These require the application on the publishing side to periodically assert liveliness before the lease expires to indicate the corresponding [DDSEntity](#) is still alive. The action can be explicit by calling [DDSDataWriter::assert_liveliness](#), or implicit by writing data.

The two possible manual settings control the granularity at which the application must assert liveliness.

- The setting [DDS_MANUAL_BY_PARTICIPANT_LIVELINESS_QOS](#) requires only that one [DDSEntity](#) within a participant is asserted to be alive to infer that all other [DDSEntity](#) objects within the same [DDSDomainParticipant](#) are also alive.
- The setting [DDS_MANUAL_BY_TOPIC_LIVELINESS_QOS](#) requires that at least one instance within the [DDSDataWriter](#) is asserted.

The service must detect changes in [LIVELINESS](#) with time granularity greater than or equal to the `lease_duration`. This ensures that RTI Connex DDS Micro updates [DDS_LivelinessChangedStatus](#) at least once per `lease_duration` and notifies Listeners within a `lease_duration` of the configuration change.

7.40.3 Compatibility

A [DDSDataWriter](#)'s offered liveliness policy matches a [DDSDataReader](#)'s requested liveliness policy if and only if the following conditions are true:

- the inequality *offered kind* $\geq$ *requested kind* evaluates to true. For this comparison, [DDS_LivelinessQosPolicyKind](#) values are ordered as follows: [DDS_AUTOMATIC_LIVELINESS_QOS](#) $<$ [DDS_MANUAL_BY_PARTICIPANT_LIVELINESS_QOS](#) $<$ [DDS_MANUAL_BY_TOPIC_LIVELINESS_QOS](#).
- the inequality *offered lease_duration* $\leq$ *requested lease_duration* evaluates to [DDS_BOOLEAN_TRUE](#).

7.40.4 Member Data Documentation

kind

`DDS_LivelinessQosPolicyKind DDS_LivelinessQosPolicy::kind`

The kind of liveliness desired.

[default] `DDS_AUTOMATIC_LIVELINESS_QOS`

lease_duration

`struct DDS_Duration_t DDS_LivelinessQosPolicy::lease_duration`

The duration within which a `DDSEntity` must be asserted, or else it is assumed to be not alive.

[default] `DDS_DURATION_INFINITE`

[range] [0,1 year] or `DDS_DURATION_INFINITE`

7.41 DDS_Locator_t Struct Reference

`<<eXtension>>` Type used to represent the addressing information needed to send a message to an RTPS Endpoint using one of the supported transports.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- `DDS_Long kind`
The kind of locator.
- `DDS_UnsignedLong port`
the port number
- `DDS_Octet address [DDS_LOCATOR_ADDRESS_LENGTH_MAX]`
An array of octets with maximum length of `DDS_LOCATOR_ADDRESS_LENGTH_MAX` to hold the address.

7.41.1 Detailed Description

`<<eXtension>>` Type used to represent the addressing information needed to send a message to an RTPS Endpoint using one of the supported transports.

7.41.2 Member Data Documentation

kind

`DDS_Long DDS_Locator_t::kind`

The kind of locator.

If the `Locator_t` kind is `DDS_LOCATOR_KIND_UDPv4`, the address contains an IPv4 address. In this case, the leading 12 octets of the `DDS_Locator_t::address` must be zero. The last 4 octets of `DDS_Locator_t::address` are used to store the IPv4 address in big-endian order.

If the `Locator_t` kind is not `DDS_LOCATOR_KIND_UDPv4`, the address is not interpreted and is treated as an array of 16 octets.

port

`DDS_UnsignedLong DDS_Locator_t::port`

the port number

address

`DDS_Octet DDS_Locator_t::address[DDS_LOCATOR_ADDRESS_LENGTH_MAX]`

An array of octets with maximum length of `DDS_LOCATOR_ADDRESS_LENGTH_MAX` to hold the address.

7.42 DDS_LocatorSeq Struct Reference

Declares IDL sequence `< DDS_Locator_t >`

```
#include <dds_c_infrastructure.h>
```

7.42.1 Detailed Description

Declares IDL sequence `< DDS_Locator_t >`

See also

[`DDS\_Locator\_t`](#)

7.43 DDS_OfferedDeadlineMissedStatus Struct Reference

`DDS_OFFERED_DEADLINE_MISSED_STATUS`

```
#include <dds_c_publication.h>
```

Public Attributes

- [DDS_Long total_count](#)
Total cumulative count of the number of times the [DDSDataWriter](#) failed to write within its offered deadline.
- [DDS_Long total_count_change](#)
The incremental changes in total_count since the last time the listener was called or the status was read.
- [DDS_InstanceHandle_t last_instance_handle](#)
Handle to the last instance in the [DDSDataWriter](#) for which an offered deadline was missed.

7.43.1 Detailed Description[DDS_OFFERED_DEADLINE_MISSED_STATUS](#)

Entity:

[DDSDataWriter](#)

Listener:

[DDSDataWriterListener](#)

The deadline that the [DDSDataWriter](#) has committed through its [DDS_DeadlineQosPolicy](#) was not respected for a specific instance.

7.43.2 Member Data Documentation**total_count**[DDS_Long](#) [DDS_OfferedDeadlineMissedStatus::total_count](#)

Total cumulative count of the number of times the [DDSDataWriter](#) failed to write within its offered deadline.

Missed deadlines accumulate; that is, each deadline period the `total_count` will be incremented by one.

total count

total_count_change[DDS_Long](#) [DDS_OfferedDeadlineMissedStatus::total_count_change](#)

The incremental changes in `total_count` since the last time the listener was called or the status was read.

last_instance_handle

`DDS_InstanceHandle_t DDS_OfferedDeadlineMissedStatus::last_instance_handle`

Handle to the last instance in the [DDSDDataWriter](#) for which an offered deadline was missed.

7.44 DDS_OfferedIncompatibleQosStatus Struct Reference

DDS_OFFERED_INCOMPATIBLE_QOS_STATUS

```
#include <dds_c_publication.h>
```

Public Attributes

- [DDS_Long total_count](#)
Total cumulative number of times the concerned [DDSDDataWriter](#) discovered a [DDSDDataReader](#) for the same [DDSTopic](#), common partition with a requested QoS that is incompatible with that offered by the [DDSDDataWriter](#).
- [DDS_Long total_count_change](#)
The incremental changes in `total_count` since the last time the listener was called or the status was read.
- [DDS_QosPolicyId_t last_policy_id](#)
The [DDS_QosPolicyId_t](#) of one of the policies that was found to be incompatible the last time an incompatibility was detected.
- struct [DDS_QosPolicyCountSeq policies](#)
Not used in RTI Connext DDS Micro.

7.44.1 Detailed Description

DDS_OFFERED_INCOMPATIBLE_QOS_STATUS

Entity:

[DDSDDataWriter](#)

Listener:

[DDSDDataWriterListener](#)

The qos policy value was incompatible with what was requested.

7.44.2 Member Data Documentation

total_count

`DDS_Long DDS_OfferedIncompatibleQosStatus::total_count`

Total cumulative number of times the concerned [DDSDDataWriter](#) discovered a [DDSDDataReader](#) for the same [DDSTopic](#), common partition with a requested QoS that is incompatible with that offered by the [DDSDDataWriter](#).

total_count_change

[DDS_Long](#) DDS_OfferedIncompatibleQosStatus::total_count_change

The incremental changes in total_count since the last time the listener was called or the status was read.

last_policy_id

[DDS_QosPolicyId_t](#) DDS_OfferedIncompatibleQosStatus::last_policy_id

The [DDS_QosPolicyId_t](#) of one of the policies that was found to be incompatible the last time an incompatibility was detected.

policies

struct DDS_QosPolicyCountSeq DDS_OfferedIncompatibleQosStatus::policies

Not used in RTI Connex DDS Micro.

7.45 DDS_OwnershipQosPolicy Struct Reference

Specifies whether it is allowed for multiple ::DDSDataWriter(s) to write the same instance of the data and if so, how these modifications should be arbitrated.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_OwnershipQosPolicyKind](#) kind
The kind of ownership.

7.45.1 Detailed Description

Specifies whether it is allowed for multiple ::DDSDataWriter(s) to write the same instance of the data and if so, how these modifications should be arbitrated.

Entity:

[DDSTopic](#), [DDSDataReader](#), [DDSDataWriter](#)

Status:

[DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#), [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#)

Properties:

[RxO](#) = YES
[Changeable](#) = NO

See also

[OWNERSHIP_STRENGTH](#)

7.45.2 Usage

This policy controls whether RTI Connext DDS Micro allows multiple [DDSDataWriter](#) objects to update the same instance (identified by [DDSTopic](#) + key) of a data type.

There are two kinds of ownership selected by the setting of the `kind`: SHARED and EXCLUSIVE.

7.45.2.1 SHARED ownership

[DDS_SHARED_OWNERSHIP_QOS](#) indicates that RTI Connext DDS Micro does not enforce unique ownership for each instance. In this case, multiple writers can update the same data type instance. The subscriber to the [DDSTopic](#) will be able to access modifications from all [DDSDataWriter](#) objects, subject to the settings of other QoS that may filter particular samples (e.g. the [HISTORY](#) policy). In any case there is no "filtering" of modifications made based on the identity of the [DDSDataWriter](#) that causes the modification.

7.45.2.2 EXCLUSIVE ownership

[DDS_EXCLUSIVE_OWNERSHIP_QOS](#) indicates that each instance of a data type can only be modified by one [DDSDataWriter](#). In other words, at any point in time a single [DDSDataWriter](#) owns each instance and is the only one whose modifications will be visible to the [DDSDataReader](#) objects. The system determines the owner by selecting the [DDSDataWriter](#) with the highest value of the [DDS_OwnershipStrengthQosPolicy::value](#) that is currently alive as defined by the [LIVELINESS](#) policy and has not violated its [DEADLINE](#) contract with regards to the data-instance.

Ownership can therefore change as a result of:

- a [DDSDataWriter](#) in the system with a higher value of the strength that modifies the instance,
- a change in the strength value of the [DDSDataWriter](#) that owns the instance, and
- a change in the liveliness of the [DDSDataWriter](#) that owns the instance.
- a deadline with regard to the instance that the [DDSDataWriter](#) that owns the instance misses.

The behavior of the system is as if each [DDSDataReader](#) made the determination independently. Each [DDSDataReader](#) may detect the change of ownership at a different time. It is not a requirement that at a particular point in time all the [DDSDataReader](#) objects for that [DDSTopic](#) have a consistent picture of who owns each instance.

It is also not a requirement that the [DDSDataWriter](#) objects are aware of whether they own a particular instance. The system gives no error or notification to a [DDSDataWriter](#) that modifies an instance it does not currently own.

These requirements (a) preserve the decoupling of publishers and subscribers, and (b) allow the policy to be implemented efficiently.

It is possible that multiple [DDSDataWriter](#) objects with the same strength modify the same instance. If this occurs RTI Connext DDS Micro will pick one of the [DDSDataWriter](#) objects as the owner. The specification does not define how to select the owner. However, the policy used to select the owner must be such that all [DDSDataReader](#) objects will make the same choice of the particular [DDSDataWriter](#) that is the owner. The owner must remain the same until there is a change in strength, liveliness, the owner misses a deadline on the instance, or a new [DDSDataWriter](#) with higher or same strength that becomes the owner according to the policy of the Service, modifies the instance.

Exclusive ownership is on an instance-by-instance basis. That is, a subscriber can receive values written by a lower strength [DDSDataWriter](#) as long as they affect instances whose values the higher-strength [DDSDataWriter](#) has not set.

7.45.3 Compatibility

The value of the [DDS_OwnershipQosPolicyKind](#) offered must exactly match the one requested or else the service considers them incompatible.

7.45.3.1 Ownership Resolution on Redundant Systems

Users are expected to use DDS to set up redundant systems where multiple [DDSDataWriter](#) entities are "capable" of writing the same instance. In this situation, the [DDSDataWriter](#) entities are configured such that:

- Either both are writing the instance "constantly"
- Or else they use some mechanism to classify each other as "primary" and "secondary", such that the primary is the only one writing, and the secondary monitors the primary and only writes when it detects that the primary "writer" is no longer writing.

Both cases above use the [DDS_EXCLUSIVE_OWNERSHIP_QOS](#) and arbitrate themselves by means of the [DDS_OwnershipStrengthQosPolicy](#). Regardless of the scheme, the desired behavior from the [DDSDataReader](#) point of view is that [DDSDataReader](#) normally receives data from the primary unless the "primary" writer stops writing, in which case the [DDSDataReader](#) starts to receive data from the secondary [DDSDataWriter](#).

This approach requires some mechanism to detect that a [DDSDataWriter](#) (the primary) is no longer "writing" the data as it should. There are several reasons why this may happen and the system must detect all of them (but not necessarily distinguish them):

- [crash] The writing process is no longer running (e.g. the whole application has crashed)
- [connectivity loss] The writing application has lost connectivity (e.g. network disconnection)
- [application fault] The application logic that was writing the data is faulty and has stopped calling [FooDataWriter::write](#).

Arbitrating from a [DDSDataWriter](#) to one of higher strength is simple and the [DDSDataReader](#) can make the decision autonomously. Switching ownership from a higher strength [DDSDataWriter](#) to one of a lower strength [DDSDataWriter](#) requires that the [DDSDataReader](#) can make a determination that the stronger [DDSDataWriter](#) is "no longer writing the instance".

7.45.3.1.1 Case where the data is periodically updated

This determination is reasonably simple when a [DDSDataWriter](#) writes the data periodically at some rate. The [DDSDataWriter](#) simply states its offered [DDS_DeadlineQosPolicy](#) (maximum interval between updates) and the [DDSDataReader](#) automatically monitors that the [DDSDataWriter](#) indeed updates the instance at least once per [DDS_DeadlineQosPolicy::period](#). If the [DDSDataWriter](#) misses the deadline, the [DDSDataReader](#) considers the [DDSDataWriter](#) "not alive" and automatically gives ownership to the next highest-strength [DDSDataWriter](#) that *is* alive.

7.45.3.1.2 Case where data is not periodically updated

The case where the `DDSDDataWriter` is not writing data periodically is also a very important use-case. Since the `DDSDDataWriter` is not updating the instance at any fixed period, the "deadline" mechanism cannot be used to determine ownership. The liveliness solves this situation. The `DDSDDataWriter` maintains ownership while it is "alive" and to be alive the `DDSDDataWriter` must fulfill its `DDS_LivelinessQosPolicy` contract. The different means to renew liveliness (automatic, manual) combined with the implied renewal each time data is written handle the three conditions above [crash], [connectivity loss], and [application fault]. Note that to handle [application fault], LIVENESS must be `DDS_MANUAL_BY_TOPIC_LIVENESS_QOS`. The `DDSDDataWriter` can retain ownership by periodically writing data or else calling `assert_liveliness` if it has no data to write. Alternatively if the application desires only protection against [crash] or [connectivity loss], it is sufficient that some task on the `DDSDDataWriter` process periodically writes data. However, this scenario requires that the `DDSDDataReader` knows what instances the `DDSDDataWriter` "writes". That is the only way that the `DDSDDataReader` deduces the ownership of specific instances from the fact that the `DDSDDataWriter` is still "alive".

7.45.3.2 Detection of Loss in Topological Connectivity

There are applications that have a design such that their correct operation requires some minimal topological connectivity, that is, the writer needs to have a minimum number of readers or alternatively the reader must have a minimum number of writers.

A common scenario is that the application does not start doing its logic until it knows that some specific writers have the minimum configured readers (e.g. the alarm monitor is up).

A *more* common scenario is that the application logic will wait until some writers appear that can provide some needed source of information (e.g. the raw sensor data that must be processed).

Furthermore, once the application is running it is a requirement that this minimal connectivity (from the source of the data) is monitored and informs the application if it is ever lost. For the case where a `DDSDDataWriter` writes data periodically, the `DDS_DeadlineQosPolicy` and the `on_deadline_missed` listener provides the notification. The case where a `DDSDDataWriter` does not periodically update data requires the use of the `DDS_LivelinessQosPolicy` to detect whether the "connectivity" has been lost, and the system provides notification by means of `DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE`.

In terms of the required mechanisms, the scenario is very similar to the case of maintaining ownership. In both cases, the reader needs to know whether a writer is still "managing the current value of an instance" even though it is not continually writing it and this knowledge requires the writer to keep its liveliness.

For a `DDSTopic` with `DDS_EXCLUSIVE_OWNERSHIP_QOS`, if the current owner of an instance *disposes* it, the readers accessing the instance will see the `instance_state` as being "DISPOSED" and not see the values being written by the weaker writer (even after the stronger one has disposed the instance). This is because the `DDSDDataWriter` that owns the instance is saying that the instance no longer exists (e.g. the master of the database is saying that a record has been deleted) and thus the readers should see it as such.

7.45.4 Member Data Documentation

kind

`DDS_OwnershipQosPolicyKind` `DDS_OwnershipQosPolicy::kind`

The kind of ownership.

[default] `DDS_SHARED_OWNERSHIP_QOS`

7.46 DDS_OwnershipStrengthQosPolicy Struct Reference

Specifies the value of the strength used to arbitrate among multiple [DDSDataWriter](#) objects that attempt to modify the same instance of a data type (identified by [DDSTopic](#) + key).

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_Long](#) value
The strength value used to arbitrate among multiple writers.

7.46.1 Detailed Description

Specifies the value of the strength used to arbitrate among multiple [DDSDataWriter](#) objects that attempt to modify the same instance of a data type (identified by [DDSTopic](#) + key).

This policy only applies if the [OWNERSHIP](#) policy is of kind [DDS_EXCLUSIVE_OWNERSHIP_QOS](#).

Entity:

[DDSDataWriter](#)

Properties:

[RxO](#) = N/A
[Changeable](#) = NO

The value of the [OWNERSHIP_STRENGTH](#) is used to determine the ownership of a data-instance (identified by the key). The arbitration is performed by the [DDSDataReader](#).

See also

[EXCLUSIVE ownership](#)

7.46.2 Member Data Documentation

value

[DDS_Long](#) [DDS_OwnershipStrengthQosPolicy::value](#)

The strength value used to arbitrate among multiple writers.

[default] 0

[range] [0, 1 million]

7.47 DDS_ParticipantBuiltinTopicData Struct Reference

Object representing a remote DomainParticipant.

```
#include <dds_c_discovery.h>
```

Public Attributes

- struct [DDS_BuiltinTopicKey_t](#) `key`
DCPS key to distinguish entries.
- struct [DDS_EntityNameQosPolicy](#) `participant_name`
<<eXtension>> The participant name.
- [DDS_UnsignedLong](#) `dds_builtin_endpoints`
<<eXtension>> Bitmap of builtin endpoints supported by the participant.
- [DDS_ProtocolVersion_t](#) `rtps_protocol_version`
<<eXtension>> Version number of the RTPS wire protocol used.
- struct [DDS_VendorId_t](#) `rtps_vendor_id`
<<eXtension>> ID of vendor implementing the RTPS wire protocol.
- struct [DDS_LocatorSeq](#) `default_unicast_locators`
<<eXtension>> Unicast locators used when individual entities do not specify unicast locators.
- struct [DDS_LocatorSeq](#) `default_multicast_locators`
<<eXtension>> Multicast locators used when individual entities do not specify multicast locators.
- struct [DDS_LocatorSeq](#) `metatraffic_unicast_locators`
<<eXtension>> Unicast locators used for discovery metatraffic.
- struct [DDS_LocatorSeq](#) `metatraffic_multicast_locators`
<<eXtension>> Multicast locators used for discovery metatraffic.
- struct [DDS_Duration_t](#) `liveliness_lease_duration`
<<eXtension>> Liveliness lease duration of participant.
- struct [DDS_ProductVersion_t](#) `product_version`
<<eXtension>> This is a vendor specific parameter. It gives the current version for RTI Connexx DDS Micro.
- struct [DDS_ChecksumProperty_t](#) `checksum`
<<eXtension>> Checksum properties for the participant.
- struct [DDS_PropertyQosPolicy](#) `property`
<<eXtension>> Name value pair properties to be stored with domain participant
- struct [DDS_UserDataQosPolicy](#) `user_data`
Policy of the corresponding DomainParticipant.

7.47.1 Detailed Description

Object representing a remote DomainParticipant.

Data associated with the built-in topic [DDS_PARTICIPANT_TOPIC_NAME](#). It contains QoS policies and additional information that apply to the remote [DDSDomainParticipant](#).

7.47.2 Member Data Documentation

key

```
struct DDS_BuiltinTopicKey_t DDS_ParticipantBuiltinTopicData::key
```

DCPS key to distinguish entries.

participant_name

```
struct DDS_EntityNameQosPolicy DDS_ParticipantBuiltinTopicData::participant_name
```

<<eXtension>> The participant name.

This is the name of the discovered participant.

dds_builtin_endpoints

```
DDS_UnsignedLong DDS_ParticipantBuiltinTopicData::dds_builtin_endpoints
```

<<eXtension>> Bitmap of builtin endpoints supported by the participant.

Each bit indicates a builtin endpoint that may be available on the participant for use in discovery.

rtps_protocol_version

```
DDS_ProtocolVersion_t DDS_ParticipantBuiltinTopicData::rtps_protocol_version
```

<<eXtension>> Version number of the RTPS wire protocol used.

rtps_vendor_id

```
struct DDS_VendorId_t DDS_ParticipantBuiltinTopicData::rtps_vendor_id
```

<<eXtension>> ID of vendor implementing the RTPS wire protocol.

default_unicast_locators

```
struct DDS_LocatorSeq DDS_ParticipantBuiltinTopicData::default_unicast_locators
```

<<eXtension>> Unicast locators used when individual entities do not specify unicast locators.

default_multicast_locators

```
struct DDS_LocatorSeq DDS_ParticipantBuiltinTopicData::default_multicast_locators
```

<<eXtension>> Multicast locators used when individual entities do not specify multicast locators.

metatraffic_unicast_locators

```
struct DDS_LocatorSeq DDS_ParticipantBuiltinTopicData::metatraffic_unicast_locators
```

<<eXtension>> Unicast locators used for discovery metatraffic.

metatraffic_multicast_locators

```
struct DDS_LocatorSeq DDS_ParticipantBuiltinTopicData::metatraffic_multicast_locators
```

<<eXtension>> Multicast locators used for discovery metatraffic.

liveliness_lease_duration

```
struct DDS_Duration_t DDS_ParticipantBuiltinTopicData::liveliness_lease_duration
```

<<eXtension>> Liveliness lease duration of participant.

product_version

```
struct DDS_ProductVersion_t DDS_ParticipantBuiltinTopicData::product_version
```

<<eXtension>> This is a vendor specific parameter. It gives the current version for RTI Connex DDS Micro.

checksum

```
struct DDS_ChecksumProperty_t DDS_ParticipantBuiltinTopicData::checksum
```

<<eXtension>> Checksum properties for the participant.

property

```
struct DDS_PropertyQosPolicy DDS_ParticipantBuiltinTopicData::property
```

<<eXtension>> Name value pair properties to be stored with domain participant

user_data

```
struct DDS_UserDataQosPolicy DDS_ParticipantBuiltinTopicData::user_data
```

Policy of the corresponding DomainParticipant.

7.48 DDS_PartitionQosPolicy Struct Reference

A set of strings that introduces logical PARTITIONS.

```
#include <dds_c_partition_qos.h>
```

Public Attributes

- struct DDS_StringSeq [name](#)
The PARTITION string Sequence.

7.48.1 Detailed Description

A set of strings that introduces logical PARTITIONS.

Only entities that belong to the same PARTITION can communicate with each other.

The PARTITION QoS controls which entities will match which other entities. PARTITIONS prevent entities from communicating. Only entities that belong to the same PARTITION can communicate with each other. The PARTITION QoS policy applies directly to Publishers and Subscribers. [DDSDDataWriter](#) entities and [DDSDDataReader](#) entities belong to the partitions of the Publishers and Subscribers that created them.

PARTITION names are propagated with the discovery traffic for Publishers and Subscribers.

[DDSDDataWriter](#) entities will make their data available on all PARTITIONS that correspond to their Publisher's PARTITION names, and the [DDSDDataReader](#) entities will see the data on all PARTITIONS that correspond to their Subscriber's PARTITION names. For a [DDSDDataWriter](#) and [DDSDDataReader](#) to communicate, the [DDSDDataWriter](#)'s Publisher must match the [DDSDDataReader](#)'s Subscriber's PARTITION.

[DDSDDataWriter](#) entities and [DDSDDataReader](#) entities will only match with each other if their Publisher's QoS and Subscriber's QoS contain a compatible PARTITION.

The PARTITION sequence must be set before enabling the entity. The user must create a Publisher QoS [DDS_PublisherQos::partition](#) or a Subscriber QoS [DDS_SubscriberQos::partition](#) for PARTITION configuration.

The default value for a PARTITION is the empty sequence. When an entity is created with a [DDS_PublisherQos::partition](#) that does not have any concrete strings, meaning it only has regular expressions, then the partition is considered a member of the empty string "".

Publishers and Subscribers without PARTITIONS are considered members of the empty string "".

The regular expression matching rules are as follows:

- Two PARTITIONS will match if they are lexicographically identical.
- Two PARTITIONS will match if one is a string that is not a regular expression and the other is a regular expression that completely matches.
- Two regular expressions cannot match.

[DDS_PartitionQosPolicy](#) supports the following regular expressions:

* The '*' matches any sequence of characters, including the empty string.

? The '?' character matches the preceding character once. Note that the '?' does not conform with the POSIX standard where it is allowed to match zero times.

\ The '\' escape character **shall** cause the next character to be interpreted literally.

[] Opening and closing brackets '[' ']' matches one of the characters enclosed in the brackets: A range "[C0 '-' C1]" matches one character between the collating elements C0 and C1, inclusive.

The "^" and "!" characters placed first in opening and closing brackets matches a single character if the character is not enclosed in the brackets.

A range "['!' C0 '-' C1]" matches a character not in the lexicographically closed interval C0 to C1.

The [DDS_PartitionQosPolicy](#) resource limits are made on a per DomainParticipant basis. The resource limits cannot be set individually on a per Publisher/Subscriber basis.

[DDS_DomainParticipantResourceLimitsQosPolicy::max_partitions](#) controls the maximum number of PARTITIONS that can be assigned per Publisher/Subscriber.

[DDS_DomainParticipantResourceLimitsQosPolicy::max_partition_cumulative_characters](#) controls the maximum number of combined characters in all PARTITION names in a [DDS_PartitionQosPolicy](#).

The maximum number of combined characters should account for a terminating NUL ('\0') character for each partition name string.

[DDS_DomainParticipantResourceLimitsQosPolicy::max_partition_string_allocation](#) The maximum number of 8-bit characters reserved for PARTITION.name strings to allocate for the PARTITION. When this is set to DDS_LENGTH_UNLIMITED, no memory is preallocated. Memory is allocated during local and remote entity creation even if the creation occurs during runtime. When max_partition_string_allocation is greater than 0, that amount of memory is preallocated for storing created and discovered PARTITION names. [DDS_PartitionQosPolicy](#).

[DDS_DomainParticipantResourceLimitsQosPolicy::max_partition_string_size](#) max_partition_string_size configures the PARTITION name element max string length in local and remote Publishers and Subscribers. All applications in the DDS domain must use the same max_partition_string_size in order to communicate.

When creating a Publisher or Subscriber, all PARTITION elements in the Publisher or Subscriber QoS must not exceed max_partition_string_size or the creation will fail. max_partition_string_size also controls the max length for any discovered PARTITION names from remote Publishers and Subscribers. Remote entities that have PARTITION name lengths that exceed max_partition_string_size will not be discovered.

When max_partition_string_allocation is DDS_LENGTH_UNLIMITED max_partition_string_size should be set to DDS_LENGTH_UNLIMITED. This configuration allows each PARTITION name size to be any length up to max_partition_cumulative_characters. In this configuration, PARTITION strings are allocated and freed during runtime. To prevent allocation during runtime, when max_partition_string_allocation is DDS_LENGTH_UNLIMITED, the users can assert all unique PARTITION names during creation. When any additional instance of that name is encountered, during entity creation or discovery, no further memory is used for storing the existing PARTITION names.

Use the following configurations to completely control memory utilization or to avoid any possibility of runtime allocation.

When `max_partition_string_allocation` is not `DDS_LENGTH_UNLIMITED` and `max_partition_string_size` is `DDS_LENGTH_UNLIMITED` then `PARTITION` names can be any length up to `max_partition_cumulative_characters`. Memory for `PARTITION` names will be preallocated and will never be freed or reused. The user must have enough `max_partition_string_allocation` to store every unique instance of each `PARTITION` name to be discovered.

When `max_partition_string_allocation` is not `DDS_LENGTH_UNLIMITED` then `max_partition_string_size` can be set to a value greater than 0. In this configuration, memory for `PARTITION` names are preallocated and will never be freed during operation. `PARTITION` names are stored in memory and consume `max_partition_string_size` sized memory segments up to `max_partition_string_allocation`. Memory is reused for `PARTITION` names that are added to memory, internally deleted, and are no longer needed. As an example, if the application creates a Publisher with a unique `PARTITION` name instance and then deletes that Publisher, if there are no further uses of that partition name, the application will reuse the memory that was storing the unique `PARTITION` name. Reuse occurs in the same way for remote entities. The user must have enough `max_partition_string_allocation` to store every unique instance of each `PARTITION` name to be created and discovered.

7.48.2 Member Data Documentation

name

```
struct DDS_StringSeq DDS_PartitionQosPolicy::name
```

The `PARTITION` string Sequence.

Sequence of strings that introduce a logical `PARTITION` among the topics visible by the Publisher and Subscriber.

7.49 DDS_PresentationQosPolicy Struct Reference

Specifies how the samples representing changes to data instances are presented to a subscribing application.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_PresentationQosPolicyAccessScopeKind access_scope](#)
Determines the largest scope spanning the entities for which the order and coherency of changes can be preserved.
- [DDS_Boolean coherent_access](#)
Specifies support for coherent access. Controls whether coherent access is supported within the scope `access_scope`.
- [DDS_Boolean ordered_access](#)
Specifies support for ordered access to the samples received at the subscription end. Controls whether ordered access is supported within the scope `access_scope`.

7.49.1 Detailed Description

Specifies how the samples representing changes to data instances are presented to a subscribing application.

NOTE: This QoS policy can only be set when using the static discovery API on the [DDS_SubscriptionBuiltinTopicData](#).

A RTI Connex DDS Micro [DDSPublisher](#) always offers:

- `access_scope = DDS_TOPIC_PRESENTATION_QOS`
- `ordered_access = DDS_BOOLEAN_TRUE`
- `coherent_access = DDS_BOOLEAN_FALSE`

These settings enable a [DDSPublisher](#) to match with a [DDSSubscriber](#) that requests either `DDS_INSTANCE_PRESENTATION_QOS` or `DDS_TOPIC_PRESENTATION_QOS`, increasing interoperability.

A RTI Connex DDS Micro [DDSSubscriber](#) always requests:

- `access_scope = DDS_INSTANCE_PRESENTATION_QOS`
- `ordered_access = DDS_BOOLEAN_FALSE`
- `coherent_access = DDS_BOOLEAN_FALSE`

The following is a general description of the [DDS_PresentationQosPolicy](#).

This QoS policy controls the extent to which changes to data instances can be made dependent on each other and also the kind of dependencies that can be propagated and maintained by RTI Connex DDS Micro. Specifically, this policy affects the application's ability to:

- specify and receive coherent changes to instances
- specify the relative order in which changes are presented

Entity:

[DDSPublisher](#), [DDSSubscriber](#)

Status:

[DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#), [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#)

A [DDSDataReader](#) will usually receive data in the order that it was sent by a [DDSDataWriter](#), and the data is presented to the [DDSDataReader](#) as soon as the application receives the next expected value. However, sometimes you may want a set of data for the same [DDSDataWriter](#) to be presented to the [DDSDataReader](#) only after *all* of the elements of the set have been received. Or you may want the data to be presented in a different order than that in which it was received. Specifically for keyed data, you may want the middleware to present the data in keyed – or *instance* – order, such that samples pertaining to the same instance are presented together.

The Presentation QoS policy allows you to specify different *scopes* of presentation: within a [DDSDataWriter](#), across instances of a single [DDSDataWriter](#), and even across multiple [DDSDataWriter](#) entities used by different writers of a publisher. It also controls whether or not a set of changes within the scope is delivered at the same time or can be delivered as soon as each element is received.

- `coherent_access` controls whether RTI Connex DDS Micro will preserve the groupings of changes made by a publishing application by means of the operations `begin_coherent_changes()` and `end_coherent_changes()`. These operations are not supported.
- `ordered_access` controls whether RTI Connex DDS Micro will preserve the order of changes.
- `access_scope` controls the granularity of the other settings. See below:

If `coherent_access` is set, then the `access_scope` controls the maximum extent of coherent changes. The behavior is as follows:

- If `access_scope` is set to [DDS_INSTANCE_PRESENTATION_QOS](#), the default, the use of `begin_coherent_changes()` and `end_coherent_changes()` has no effect on how the subscriber can access the data, because with the scope limited to each instance, changes to separate instances are considered independent and thus cannot be grouped into a coherent set.
- If `access_scope` is set to [DDS_TOPIC_PRESENTATION_QOS](#), then coherent changes indicated by their enclosure within calls to `begin_coherent_changes()` and `end_coherent_changes()` will be made available as such to each remote [DDSDataReader](#) independently. That is, changes made to instances within each individual [DDSDataWriter](#) will be available as coherent with respect to other changes to instances in that same [DDSDataWriter](#), but will not be grouped with changes made to instances belonging to a different [DDSDataWriter](#).
- If `access_scope` is set to [DDS_GROUP_PRESENTATION_QOS](#), then coherent changes made to instances through a [DDSDataWriter](#) attached to a common [DDSPublisher](#) are made available as a unit to remote subscribers.

If `ordered_access` is set, then the `access_scope` controls the maximum extent for which order will be preserved by RTI Connex DDS Micro.

- If `access_scope` is set to [DDS_INSTANCE_PRESENTATION_QOS](#), the lowest level, then changes to each instance are considered unordered relative to changes to any other instance. That means that changes, creations, deletions, or modifications made to two instances are not necessarily seen in the order they occur. This is the case even if it is the same application thread making the changes using the same [DDSDataWriter](#).

- If `access_scope` is set to `DDS_TOPIC_PRESENTATION_QOS`, changes, creations, deletions, modifications, made by a single `DDSDataWriter` are made available to subscribers in the same order they occur. Changes made to instances through different `DDSDataWriter` entities are not necessarily seen in the order they occur. This is the case, even if the changes are made by a single application thread using `DDSDataWriter` objects attached to the same `DDSPublisher`.
- Finally, if `access_scope` is set to `DDS_GROUP_PRESENTATION_QOS`, changes made to instances via `DDSDataWriter` entities attached to the same `DDSPublisher` object are made available to subscribers in the same order they occur.

Note that this QoS policy controls the scope at which related changes are made available to the subscriber. This means the subscriber **can** access the changes in a coherent manner and in the proper order; however, it does not necessarily imply that the `DDSSubscriber` **will** indeed access the changes in the correct order. For that to occur, the application at the subscriber end must use the proper logic in reading the `DDSDataReader` objects.

7.49.2 Compatibility

The value offered is considered compatible with the value requested if and only if the following conditions are met:

- the inequality `offered access_scope >= requested access_scope` evaluates to 'TRUE'. For the purposes of this inequality, the values of `access_scope` are considered ordered such that `DDS_INSTANCE_PRESENTATION_QOS < DDS_TOPIC_PRESENTATION_QOS < DDS_GROUP_PRESENTATION_QOS`.
- `requested coherent_access` is `DDS_BOOLEAN_FALSE`, or else both offered and requested `coherent_access` are `DDS_BOOLEAN_TRUE`.
- `requested ordered_access` is `DDS_BOOLEAN_FALSE`, or else both offered and requested `ordered_access` are `DDS_BOOLEAN_TRUE`.

7.49.3 Member Data Documentation

`access_scope`

`DDS_PresentationQosPolicyAccessScopeKind DDS_PresentationQosPolicy::access_scope`

Determines the largest scope spanning the entities for which the order and coherency of changes can be preserved.

NOTE: This QoS setting can only be set when using the static discovery API on the `DDS_SubscriptionBuiltinTopicData`.

[default] `DDSPublisher DDS_TOPIC_PRESENTATION_QOS`

[default] `DDSSubscriber DDS_INSTANCE_PRESENTATION_QOS`

coherent_access

`DDS_Boolean DDS_PresentationQosPolicy::coherent_access`

Specifies support for *coherent* access. Controls whether coherent access is supported within the scope `access_scope`.

That is, the ability to group a set of changes as a unit on the publishing end such that they are received as a unit at the subscribing end.

To use this feature, the `DDSDataWriter` must be configured for RELIABLE communication, see `DDS_RELIABLE_RELIABILITY_QOS`.

NOTE: This QoS setting can only be set when using the static discovery API on the `DDS_SubscriptionBuiltinTopicData`.

[default] `DDS_BOOLEAN_FALSE`

ordered_access

`DDS_Boolean DDS_PresentationQosPolicy::ordered_access`

Specifies support for *ordered* access to the samples received at the subscription end. Controls whether ordered access is supported within the scope `access_scope`.

That is, the ability of the subscriber to see changes in the same order as they occurred on the publishing end.

NOTE: This QoS setting can only be set when using the static discovery API on the `DDS_SubscriptionBuiltinTopicData`.

[default] `DDS_BOOLEAN_TRUE` for `DDSDataWriter` entities, `DDS_BOOLEAN_FALSE` for `DDSDataReader` entities

7.50 DDS_ProductVersion_t Struct Reference

`<<eXtension>>` Type used to represent the current version of RTI Connex DDS Micro.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- `DDS_Char` major
Major product version.
- `DDS_Char` minor
Minor product version.
- `DDS_Char` release
Release letter for product version.
- `DDS_Char` revision
Revision number of product.

7.50.1 Detailed Description

<<*eXtension*>> Type used to represent the current version of RTI Connex DDS Micro.

7.50.2 Member Data Documentation

major

`DDS_Char DDS_ProductVersion_t::major`

Major product version.

minor

`DDS_Char DDS_ProductVersion_t::minor`

Minor product version.

release

`DDS_Char DDS_ProductVersion_t::release`

Release letter for product version.

revision

`DDS_Char DDS_ProductVersion_t::revision`

Revision number of product.

7.51 DDS_Property_t Struct Reference

Properties are name/value pair objects.

```
#include <dds_c_property_qos.h>
```

Public Attributes

- `DDS_String name`
Property name.
- `DDS_String value`
Property value.
- `DDS_Boolean propagate`
Indicates whether the property is propagated on discovery. Must be set to DDS_BOOLEAN_FALSE.

7.51.1 Detailed Description

Properties are name/value pair objects.

7.51.2 Member Data Documentation

name

`DDS_String DDS_Property_t::name`

Property name.

It must be a NULL-terminated string allocated with `DDS_String_alloc` or `DDS_String_dup`.

value

`DDS_String DDS_Property_t::value`

Property value.

It must be a NULL-terminated string allocated with `DDS_String_alloc` or `DDS_String_dup`.

propagate

`DDS_Boolean DDS_Property_t::propagate`

Indicates whether the property is propagated on discovery. Must be set to `DDS_BOOLEAN_FALSE`.

7.52 DDS_PropertyQosPolicy Struct Reference

Stores name/value(string) pairs that can be used to configure certain parameters of RTI Connex DDS Micro that are not exposed through formal QoS policies.

```
#include <dds_c_property_qos.h>
```

Public Attributes

- struct `DDS_PropertySeq` value
Sequence of properties.

7.52.1 Detailed Description

Stores name/value(string) pairs that can be used to configure certain parameters of RTI Connexx DDS Micro that are not exposed through formal QoS policies.

Entity:

[DDSDomainParticipant](#) [DDSDataReader](#) [DDSDataWriter](#)

Properties:

[RxO](#) = N/A;
[Changeable](#) = NO

The PROPERTY QoS policy can be used to associate a set of properties in the form of (name, value) pairs with a [DDSDataReader](#), [DDSDataWriter](#), or [DDSDomainParticipant](#).

You can find a complete list of predefined properties in the [Property Reference](#).

There are helper functions to facilitate working with properties, see the [PROPERTY](#) page.

7.52.2 Member Data Documentation

value

```
struct DDS\_PropertySeq DDS_PropertyQosPolicy::value
```

Sequence of properties.

[default] An empty list.

7.53 DDS_PropertySeq Struct Reference

Declares IDL sequence < [DDS_Property_t](#) >

```
#include <dds_c_property_qos.h>
```

7.53.1 Detailed Description

Declares IDL sequence < [DDS_Property_t](#) >

See also

[DDS_Property_t](#)

7.54 DDS_ProtocolVersion_t Struct Reference

<<eXtension>> Type used to represent the version of the RTPS protocol.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_Octet major](#)
Major protocol version number.
- [DDS_Octet minor](#)
Minor protocol version number.

7.54.1 Detailed Description

<<eXtension>> Type used to represent the version of the RTPS protocol.

7.54.2 Member Data Documentation

major

[DDS_Octet](#) DDS_ProtocolVersion_t::major

Major protocol version number.

minor

[DDS_Octet](#) DDS_ProtocolVersion_t::minor

Minor protocol version number.

7.55 DDS_PskPassTracker_InterfaceI Struct Reference

PSK passphrase tracker user interface.

```
#include <dds_psk_pass_tracker.h>
```

Public Attributes

- [DDS_PskPassTracker_initializeFunc_T * initialize_func](#)
Create an instance of the passphrase tracker interface with upstream as the owner.
- [DDS_PskPassTracker_finalizeFunc_T * finalize_func](#)
Finalize and clean up the passphrase tracker instance.

7.55.1 Detailed Description

PSK passphrase tracker user interface.

Interface functions for the passphrase tracker user interface. This allows the user to specify custom implementations of the passphrase tracker. The default passphrase tracker provided with /ndds can be accessed using `PSK_File↵PassTracker_get_interface`.

7.55.2 Member Data Documentation

initialize_func

`DDS_PskPassTracker_initializeFunc_T* DDS_PskPassTracker_InterfaceI::initialize_func`

Create an instance of the passphrase tracker interface with upstream as the owner.

Parameters

out	<i>tracker_data</i>	Pointer to store tracker-specific data.
in	<i>tracker_config</i>	Configuration properties for the tracker.
in	<i>provider_arg</i>	User-provided initialization arguments.
in	<i>listener</i>	Listener structure for passphrase updates.

Returns

RTI_TRUE on successful initialization, RTI_FALSE on failure.

finalize_func

`DDS_PskPassTracker_finalizeFunc_T* DDS_PskPassTracker_InterfaceI::finalize_func`

Finalize and clean up the passphrase tracker instance.

Parameters

in	<i>tracker_data</i>	Pointer to tracker-specific data.
----	---------------------	-----------------------------------

Returns

RTI_TRUE on successful finalization, RTI_FALSE on failure.

7.56 DDS_PskServiceFactoryProperty Struct Reference

Property used to register the Lightweight Security Plugin.

```
#include <dds_psk_service.h>
```

Public Attributes

- `const char * service\_name`
The name the plugin is registered with.
- `DDS_PskTransform_get_interfaceFunc_T * psl\_get\_interface\_func`
A function pointer to retrieve the Transform PSL implementation interface.
- `struct DDS\_SecTransform\_Configuration * psl\_config`
Pass-through opaque pointer to the Transform PSL implementation, to provide the implementation with custom configuration settings.
- `DDS_PskPassTracker_get_interfaceFunc_T * pass\_tracker\_get\_interface\_func`
A function pointer to retrieve the passphrase tracker implementation interface.
- `struct PSK\_PassTrackerConfiguration * pass\_tracker\_config`
Pass-through opaque pointer to the passphrase tracker implementation to provide the implementation with custom configuration settings.

7.56.1 Detailed Description

Property used to register the Lightweight Security Plugin.

7.56.2 Member Data Documentation**service_name**

```
const char* DDS_PskServiceFactoryProperty::service_name
```

The name the plugin is registered with.

Specifies the unique identifier used when registering the plugin. Multiple plugins can be registered under distinct names. This identifier is referenced within [DDS_DomainParticipantQos::trust](#) to determine which plugin is applied. If NULL is provided, the plugin is registered with [DDS_PSK_DEFAULT_PLUGIN_NAME](#).

[default] NULL

psl_get_interface_func

```
DDS_PskTransform_get_interfaceFunc_T* DDS_PskServiceFactoryProperty::psl_get_interface_func
```

A function pointer to retrieve the Transform PSL implementation interface.

The Lightweight Security Plugin calls the function this to retrieve the Transform PSL implementation. When using the RTI-provided OpenSSL-based PSL implementation, this should be set to 'PSK_OSSL_get_interface'.

Note

include "pskpsl/psk_ssl_transform.h" when using the RTI provided OpenSSL-based PSL implementation.

psl_config

```
struct DDS_SecTransform_Configuration* DDS_PskServiceFactoryProperty::psl_config
```

Pass-through opaque pointer to the Transform PSL implementation, to provide the implementation with custom configuration settings.

When using the RTI-provided OpenSSL-based implementation, this resolves to [DDS_SecTransform_Configuration](#).

[default] NULL

Note

include "pskpsl/psk_ossI_transform.h" when using the RTI provided OpenSSL-based PSL implementation.

pass_tracker_get_interface_func

```
DDS_PskPassTracker_get_interfaceFunc_T* DDS_PskServiceFactoryProperty::pass_tracker_get_interface↔  
_func
```

A function pointer to retrieve the passphrase tracker implementation interface.

The Lightweight Security Plugin calls this function to retrieve the passphrase tracker implementation. When using the RTI-provided file-based passphrase tracker implementation, this should be set to [PSK_FilePassTracker_get↔_interface](#).

Note

Include `psk_file_tracker/psk_file_pass_tracker.h` when using the RTI-provided file-based passphrase tracker implementation.

pass_tracker_config

```
struct PSK_PassTrackerConfiguration* DDS_PskServiceFactoryProperty::pass_tracker_config
```

Pass-through opaque pointer to the passphrase tracker implementation to provide the implementation with custom configuration settings.

When using the RTI-provided file-based implementation, this resolves to [PSK_PassTrackerConfiguration](#).

[default] NULL

Note

Include `psk_file_tracker/psk_file_pass_tracker.h` when using the RTI-provided file-based passphrase tracker implementation.

7.57 DDS_PublicationBuiltinTopicData Struct Reference

Object describing a remote Publication.

```
#include <dds_c_discovery.h>
```

Public Attributes

- struct [DDS_BuiltinTopicKey_t](#) `key`
DCPS key to distinguish entries. This is used to configure the object ID of the remote [DDSDDataWriter](#).
- struct [DDS_BuiltinTopicKey_t](#) `participant_key`
DCPS key of the participant to which the [DDSDDataWriter](#) belongs.
- char * `topic_name`
Name of the related [DDSTopic](#).
- char * `type_name`
Name of the type attached to the [DDSTopic](#).
- struct [DDS_DeadlineQosPolicy](#) `deadline`
Policy of the corresponding [DataWriter](#).
- struct [DDS_OwnershipQosPolicy](#) `ownership`
Policy of the corresponding [DataWriter](#).
- struct [DDS_OwnershipStrengthQosPolicy](#) `ownership_strength`
Policy of the corresponding [DataWriter](#).
- struct [DDS_LatencyBudgetQosPolicy](#) `latency_budget`
Policy of the corresponding [DataWriter](#).
- struct [DDS_ReliabilityQosPolicy](#) `reliability`
Policy of the corresponding [DDSDDataWriter](#).
- struct [DDS_LivelinessQosPolicy](#) `liveliness`
Policy of the corresponding [DataWriter](#).
- struct [DDS_DurabilityQosPolicy](#) `durability`
Durability policy of the corresponding [DDSDDataWriter](#).
- struct [DDS_DestinationOrderQosPolicy](#) `destination_order`
Policy of the corresponding [DDSDDataWriter](#).
- struct [DDS_LocatorSeq](#) `unicast_locator`
Custom unicast locator sequence that the endpoint can specify. The default locator sequence from the [DDSDomainParticipant](#) will be used if this is not specified.
- struct [DDS_DataRepresentationQosPolicy](#) `representation`
Data representation policy, [DATA_REPRESENTATION](#).
- struct [DDS_PartitionQosPolicy](#) `partition`
Policy of the Publisher to which the [DataWriter](#) belongs.
- struct [DDS_UserDataQosPolicy](#) `user_data`
Policy of the corresponding [DDSDDataWriter](#).
- struct [DDS_GroupDataQosPolicy](#) `group_data`
Policy of the Publisher to which the [DataWriter](#) belongs.
- struct [DDS_TopicDataQosPolicy](#) `topic_data`
Policy of the related [Topic](#).

7.57.1 Detailed Description

Object describing a remote Publication.

This object contains all of the information about a remote [DDSDataWriter](#) and its [DDSPublisher](#) that is necessary for matching the remote [DDSDataWriter](#) to a local [DDSDataReader](#) during discovery.

This object is used to pre-configure remote entities for Dynamic Participant/Static Endpoint discovery.

For example: Application A is designed to discover a remote [DDSDataWriter](#) in application B, and that remote [DDSDataWriter](#) has an object ID of 100, and a deadline of three seconds. In Dynamic Participant/Static Endpoint discovery, this discovery data is not sent over the network, so the user must create an instance of a [DDS_PublicationBuiltinTopicData](#) structure in application A. The user then must fill in the QoS describing this remote [DDSDataWriter](#), including the writer's object ID and three-second deadline. Then the user must assert this [DDS_PublicationBuiltinTopicData](#) describing the remote entity with the discovery plugin. The pre-configured QoS must match the actual QoS of the remote entity, or communication will not occur.

7.57.2 Member Data Documentation

key

```
struct DDS\_BuiltinTopicKey\_t DDS_PublicationBuiltinTopicData::key
```

DCPS key to distinguish entries. This is used to configure the object ID of the remote [DDSDataWriter](#).

participant_key

```
struct DDS\_BuiltinTopicKey\_t DDS_PublicationBuiltinTopicData::participant_key
```

DCPS key of the participant to which the [DDSDataWriter](#) belongs.

topic_name

```
char* DDS_PublicationBuiltinTopicData::topic_name
```

Name of the related [DDSTopic](#).

The length of this string is limited to 255 characters.

The memory for this field is managed as described in [String Support](#).

See also

[String Support](#)

type_name

```
char* DDS_PublicationBuiltinTopicData::type_name
```

Name of the type attached to the [DDSTopic](#).

The length of this string is limited to 255 characters.

The memory for this field is managed as described in [String Support](#).

See also

[String Support](#)

deadline

```
struct DDS_DeadlineQosPolicy DDS_PublicationBuiltinTopicData::deadline
```

Policy of the corresponding DataWriter.

ownership

```
struct DDS_OwnershipQosPolicy DDS_PublicationBuiltinTopicData::ownership
```

Policy of the corresponding DataWriter.

ownership_strength

```
struct DDS_OwnershipStrengthQosPolicy DDS_PublicationBuiltinTopicData::ownership_strength
```

Policy of the corresponding DataWriter.

latency_budget

```
struct DDS_LatencyBudgetQosPolicy DDS_PublicationBuiltinTopicData::latency_budget
```

Policy of the corresponding DataWriter.

reliability

```
struct DDS_ReliabilityQosPolicy DDS_PublicationBuiltinTopicData::reliability
```

Policy of the corresponding [DDSDDataWriter](#).

liveliness

```
struct DDS_LivelinessQosPolicy DDS_PublicationBuiltinTopicData::liveliness
```

Policy of the corresponding DataWriter.

durability

```
struct DDS_DurabilityQosPolicy DDS_PublicationBuiltinTopicData::durability
```

Durability policy of the corresponding [DDSDDataWriter](#).

destination_order

```
struct DDS_DestinationOrderQosPolicy DDS_PublicationBuiltinTopicData::destination_order
```

Policy of the corresponding [DDSDDataWriter](#).

unicast_locator

```
struct DDS_LocatorSeq DDS_PublicationBuiltinTopicData::unicast_locator
```

Custom unicast locator sequence that the endpoint can specify. The default locator sequence from the [DDSDomainParticipant](#) will be used if this is not specified.

representation

```
struct DDS_DataRepresentationQosPolicy DDS_PublicationBuiltinTopicData::representation
```

Data representation policy, [DATA_REPRESENTATION](#).

partition

```
struct DDS_PartitionQosPolicy DDS_PublicationBuiltinTopicData::partition
```

Policy of the Publisher to which the DataWriter belongs.

user_data

```
struct DDS_UserDataQosPolicy DDS_PublicationBuiltinTopicData::user_data
```

Policy of the corresponding [DDSDDataWriter](#).

group_data

```
struct DDS_GroupDataQosPolicy DDS_PublicationBuiltinTopicData::group_data
```

Policy of the Publisher to which the DataWriter belongs.

topic_data

```
struct DDS_TopicDataQosPolicy DDS_PublicationBuiltinTopicData::topic_data
```

Policy of the related Topic.

7.58 DDS_PublicationMatchedStatus Struct Reference**DDS_PUBLICATION_MATCHED_STATUS**

```
#include <dds_c_publication.h>
```

Public Attributes

- **DDS_Long total_count**
The total cumulative number of times the concerned [DDSDDataWriter](#) discovered a "match" with a [DDSDDataReader](#).
- **DDS_Long total_count_change**
The incremental changes in total_count since the last time the listener was called or the status was read.
- **DDS_Long current_count**
The current number of readers with which the [DDSDDataWriter](#) is matched.
- **DDS_Long current_count_change**
The change in current_count since the last time the listener was called or the status was read.
- **DDS_InstanceHandle_t last_subscription_handle**
A handle to the last [DDSDDataReader](#) that caused the the [DDSDDataWriter](#)'s status to change.

7.58.1 Detailed Description**DDS_PUBLICATION_MATCHED_STATUS**

A "match" happens when the [DDSDDataWriter](#) finds a [DDSDDataReader](#) for the same [DDSTopic](#) and common partition with a requested QoS that is compatible with that offered by the [DDSDDataWriter](#).

This status is also changed (and the listener, if any, called) when a match is ended. A local [DDSDDataWriter](#) will become "unmatched" from a remote [DDSDDataReader](#) when that [DDSDDataReader](#) goes away for any reason.

7.58.2 Member Data Documentation

total_count

`DDS_Long DDS_PublicationMatchedStatus::total_count`

The total cumulative number of times the concerned [DDSDataWriter](#) discovered a "match" with a [DDSDataReader](#).

This number increases whenever a new match is discovered. It does not change when an existing match goes away.

total_count_change

`DDS_Long DDS_PublicationMatchedStatus::total_count_change`

The incremental changes in `total_count` since the last time the listener was called or the status was read.

current_count

`DDS_Long DDS_PublicationMatchedStatus::current_count`

The current number of readers with which the [DDSDataWriter](#) is matched.

This number increases when a new match is discovered and decreases when an existing match goes away.

current_count_change

`DDS_Long DDS_PublicationMatchedStatus::current_count_change`

The change in `current_count` since the last time the listener was called or the status was read.

last_subscription_handle

`DDS_InstanceHandle_t DDS_PublicationMatchedStatus::last_subscription_handle`

A handle to the last [DDSDataReader](#) that caused the the [DDSDataWriter](#)'s status to change.

7.59 DDS_PublisherQos Struct Reference

QoS policies supported by a [DDSPublisher](#) entity.

```
#include <dds_c_publication.h>
```

Public Attributes

- struct [DDS_EntityFactoryQosPolicy](#) entity_factory
Entity factory policy, [ENTITY_FACTORY](#).
- struct [DDS_PartitionQosPolicy](#) partition
The [DDSPublisher](#) [PARTITION](#) policy.
- struct [DDS_GroupDataQosPolicy](#) group_data
Group data policy, [GROUP_DATA](#).
- struct [DDS_EntityNameQosPolicy](#) publisher_name
<<eXtension>> The [DDSPublisher](#) name. [ENTITY_NAME](#).

7.59.1 Detailed Description

QoS policies supported by a [DDSPublisher](#) entity.

If the call to [DDSPublisher::set_qos](#) attempts to change the [PARTITION](#) QoS then it will return [DDS_RETCODE_INCONSISTENT_POLICY](#).

7.59.2 Member Data Documentation**entity_factory**

```
struct DDS\_EntityFactoryQosPolicy DDS_PublisherQos::entity_factory
```

Entity factory policy, [ENTITY_FACTORY](#).

partition

```
struct DDS\_PartitionQosPolicy DDS_PublisherQos::partition
```

The [DDSPublisher](#) [PARTITION](#) policy.

A set of strings that introduces logical [PARTITIONS](#).

Only entities that belong to the same [PARTITION](#) can communicate to each other. The [PARTITION](#) QoS policy applies directly to [DDSPublisher.DDSDataWriter](#) belong to the [PARTITIONS](#) of the [DDSPublisher](#) that created them.

The default [PARTITION](#) during initialization is the empty string, "".

[PARTITION](#)**group_data**

```
struct DDS\_GroupDataQosPolicy DDS_PublisherQos::group_data
```

Group data policy, [GROUP_DATA](#).

publisher_name

```
struct DDS_EntityNameQosPolicy DDS_PublisherQos::publisher_name
```

<<*eXtension*>> The [DDSPublisher](#) name. [ENTITY_NAME](#).

7.60 DDS_PublishModeQosPolicy Struct Reference

Specifies how RTI Connex DDS Micro sends application data on the network. This QoS policy can be used to tell RTI Connex DDS Micro to use its *own* thread to send data, instead of the user thread.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_PublishModeQosPolicyKind](#) kind
Publishing mode.
- const char * [flow_controller_name](#)
Name of the associated flow controller.
- [DDS_Long](#) priority
Publication priority.

7.60.1 Detailed Description

Specifies how RTI Connex DDS Micro sends application data on the network. This QoS policy can be used to tell RTI Connex DDS Micro to use its *own* thread to send data, instead of the user thread.

The publishing mode of a [DDSDataWriter](#) determines whether data is written synchronously in the context of the user thread when calling [FooDataWriter::write](#) or asynchronously in the context of a separate thread internal to the middle-ware.

See also

[DDS_HistoryQosPolicy](#)
[DDSFlowController](#)

Entity:

[DDSDataWriter](#)

Properties:

[RxO](#) = N/A
[Changeable](#) = NO

7.60.2 Usage

The fastest way for RTI Connex DDS Micro to send data is for the user thread to execute the middleware code that actually sends the data itself. However, there are times when user applications may need or want an internal middleware thread to send the data instead. For instance, to send large data reliably, you must use an asynchronous thread.

When data is written asynchronously, a [DDSFlowController](#), identified by `flow_controller_name`, can be used to shape the network traffic. Shaping a data flow usually means limiting the maximum data rates at which the middleware will send data for a [DDSDataWriter](#). The flow controller will buffer any excess data and only send it when the send rate drops below the maximum rate. The flow controller's properties determine when the asynchronous publishing thread is allowed to send data and how much data.

Asynchronous publishing may increase latency, but offers the following advantages:

- The [FooDataWriter::write](#) call does not make any network calls and is therefore faster and more deterministic. This becomes important when the user thread is executing time-critical code.
- When data is written in bursts or when sending large data types as multiple fragments, a flow controller can throttle the send rate of the asynchronous publishing thread to avoid flooding the network.

The middleware must queue samples until they can be sent by the publishing thread as determined by the corresponding [DDSFlowController](#). The number of samples that will be queued is determined by the [DDS_HistoryQosPolicy](#). When using [DDS_KEEP_LAST_HISTORY_QOS](#), only the most recent [DDS_HistoryQosPolicy::depth](#) samples are kept in the queue. Once unsent samples are removed from the queue, they are no longer available to the asynchronous publishing thread and will therefore never be sent.

7.60.3 Member Data Documentation

kind

```
DDS_PublishModeQosPolicyKind DDS_PublishModeQosPolicy::kind
```

Publishing mode.

[default] [DDS_SYNCHRONOUS_PUBLISH_MODE_QOS](#)

flow_controller_name

```
const char* DDS_PublishModeQosPolicy::flow_controller_name
```

Name of the associated flow controller.

NULL value or zero-length string refers to [DDS_DEFAULT_FLOW_CONTROLLER_NAME](#).

Unless `flow_controller_name` points to a built-in flow controller, finalizing the [DDS_DataWriterQos](#) will also free the string pointed to by `flow_controller_name`. Therefore, use [DDS_String_dup](#) before passing the string to `flow_controller_name`, or reset `flow_controller_name` to NULL before finalizing the QoS.

Please refer to [String Support](#) for more information.

See also

[DDSDomainParticipant::create_flowcontroller](#)
[DDS_DEFAULT_FLOW_CONTROLLER_NAME](#)
[DDS_FIXED_RATE_FLOW_CONTROLLER_NAME](#)
[DDS_ON_DEMAND_FLOW_CONTROLLER_NAME](#)

[default] [DDS_DEFAULT_FLOW_CONTROLLER_NAME](#)

priority

`DDS_Long DDS_PublishModeQosPolicy::priority`

Publication priority.

A positive integer value designating the relative priority of the [DDSDataWriter](#), used to determine the transmission order of pending writes.

Use of publication priorities requires the asynchronous publisher [DDS_ASYNCHRONOUS_PUBLISH_MODE_QOS](#) with [DDS_FlowControllerProperty_t::scheduling_policy](#) set to [DDS_HPF_FLOW_CONTROLLER_SCHED_POLICY](#).

Larger numbers have higher priority.

[default] [DDS_PUBLICATION_PRIORITY_UNDEFINED](#)

7.61 DDS_QosPolicyCount Struct Reference

Type to hold a counter for a [DDS_QosPolicyId_t](#)

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_QosPolicyId_t](#) `policy_id`
The QosPolicy ID.
- [DDS_Long](#) `count`
a counter

7.61.1 Detailed Description

Type to hold a counter for a [DDS_QosPolicyId_t](#)

7.61.2 Member Data Documentation

policy_id

`DDS_QosPolicyId_t DDS_QosPolicyCount::policy_id`

The QosPolicy ID.

count

`DDS_Long DDS_QosPolicyCount::count`

a counter

7.62 DDS_ReliabilityQosPolicy Struct Reference

Indicates the level of reliability offered or requested by RTI Connex DDS Micro.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_ReliabilityQosPolicyKind](#) kind
Kind of reliability.
- struct [DDS_Duration_t](#) max_blocking_time
For Readers: The maximum time a receive thread may block if the reader is out of resources.

7.62.1 Detailed Description

Indicates the level of reliability offered or requested by RTI Connex DDS Micro.

Entity:

[DDSTopic](#), [DDSDataReader](#), [DDSDataWriter](#)

Status:

[DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#), [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#)

Properties:

[RxO](#) = YES
[Changeable](#) = NO

7.62.2 Usage

This policy indicates the level of reliability requested by a [DDSDataReader](#) or offered by a [DDSDataWriter](#).

These levels are ordered: [DDS_BEST_EFFORT_RELIABILITY_QOS](#) < [DDS_RELIABLE_RELIABILITY_QOS](#). A [DDSDataWriter](#) offering a level is implicitly offering all levels below.

If the [DDS_ReliabilityQosPolicy::kind](#) is set to [DDS_RELIABLE_RELIABILITY_QOS](#), data samples originating from a single [DDSDataWriter](#) cannot be made available to the [DDSDataReader](#) if there are previous data samples that have not yet been received due to a communication error. In other words, the service will repair the error and re-transmit data samples as needed in order to reconstruct a correct snapshot of the [DDSDataWriter](#) history before it is accessible by the [DDSDataReader](#).

If the [DDS_ReliabilityQosPolicy::kind](#) is set to [DDS_BEST_EFFORT_RELIABILITY_QOS](#), the service will not re-transmit missing data-samples. However for data-samples originating from any one [DDSDataWriter](#) the service will ensure they are stored in the [DDSDataReader](#) history in the same order they originated in the [DDSDataWriter](#). In other words, the [DDSDataReader](#) may miss some data samples, but it will never see the value of a data object change from a newer value to an older value.

See also

[DDS_HistoryQosPolicy](#)
[DDS_ResourceLimitsQosPolicy](#)

7.62.3 Compatibility

The value offered is considered compatible with the value requested if and only if the inequality *offered kind* $\geq$ *requested kind* evaluates to TRUE. For the purposes of this inequality, the values of [DDS_ReliabilityQosPolicy::kind](#) are considered ordered such that [DDS_BEST_EFFORT_RELIABILITY_QOS](#) < [DDS_RELIABLE_RELIABILITY_QOS](#).

7.62.4 Member Data Documentation

kind

```
DDS_ReliabilityQosPolicyKind DDS_ReliabilityQosPolicy::kind
```

Kind of reliability.

[default] [DDS_BEST_EFFORT_RELIABILITY_QOS](#)

max_blocking_time

```
struct DDS_Duration_t DDS_ReliabilityQosPolicy::max_blocking_time
```

For Readers: The maximum time a receive thread may block if the reader is out of resources.

For Writers: The maximum time a writer may block on a write call.

For a [DDSDataReader](#): A non-zero maximum blocking time enables the receive thread to block up to the configured maximum blocking time if a [DDSDataReader](#) is out of resources when new data arrives. This is useful when the [DDS_HistoryQosPolicy::depth](#) is equal to `max_samples_per_instance` and the application is using a combination of loans and waitsets or polling. With a value of 0, the receive thread will not block.

For a [DDSDataWriter](#): This setting applies only to the case where [DDS_ReliabilityQosPolicy::kind](#) = [DDS_RELIABLE_RELIABILITY_QOS](#). [FooDataWriter::write](#) is allowed to block if the [DDSDataWriter](#) does not have space to store the written value.

[default] 0ms on a [DDSDataReader](#) **[default]** 100ms on a [DDSDataWriter](#)

[range] [0,1 year] for [DDSDataWriter](#) and [DDSDataReader](#)

See also

[DDS_ResourceLimitsQosPolicy](#)

7.63 DDS_ReliableReaderActivityChangedStatus Struct Reference

<<*eXtension*>> Describes the activity (i.e. are acknowledgements forthcoming) of reliable readers matched to a reliable writer.

```
#include <dds_c_publication.h>
```

Public Attributes

- [DDS_Long active_count](#)
The current number of reliable readers currently matched with this reliable writer.
- [DDS_Long inactive_count](#)
The number of reliable readers that have been dropped by this reliable writer because they failed to send acknowledgements in a timely fashion.
- [DDS_Long active_count_change](#)
The most recent change in the number of active remote reliable readers.
- [DDS_Long inactive_count_change](#)
The most recent change in the number of inactive remote reliable readers.
- [DDS_InstanceHandle_t last_instance_handle](#)
The instance handle of the last reliable remote reader to be determined inactive.

7.63.1 Detailed Description

<<*eXtension*>> Describes the activity (i.e. are acknowledgements forthcoming) of reliable readers matched to a reliable writer.

Entity:

[DDSDDataWriter](#)

Listener:

[DDSDDataWriterListener](#)

This status is the reciprocal status to the [DDS_LivelinessChangedStatus](#) on the reader. It is different than the [DDS_LivelinessLostStatus](#) on the writer in that the latter informs the writer about its own liveliness; this status informs the writer about the "liveliness" (activity) of its matched readers.

All counts in this status will remain at zero for best effort writers.

7.63.2 Member Data Documentation

active_count

[DDS_Long](#) [DDS_ReliableReaderActivityChangedStatus::active_count](#)

The current number of reliable readers currently matched with this reliable writer.

inactive_count

`DDS_Long DDS_ReliableReaderActivityChangedStatus::inactive_count`

The number of reliable readers that have been dropped by this reliable writer because they failed to send acknowledgements in a timely fashion.

A reader is considered to be inactive after is has been sent heartbeats `DDS_RtpsReliableWriterProtocol_t::max_heartbeat_retries` times, each heartbeat having been separated from the previous by the current heartbeat period.

active_count_change

`DDS_Long DDS_ReliableReaderActivityChangedStatus::active_count_change`

The most recent change in the number of active remote reliable readers.

inactive_count_change

`DDS_Long DDS_ReliableReaderActivityChangedStatus::inactive_count_change`

The most recent change in the number of inactive remote reliable readers.

last_instance_handle

`DDS_InstanceHandle_t DDS_ReliableReaderActivityChangedStatus::last_instance_handle`

The instance handle of the last reliable remote reader to be determined inactive.

7.64 DDS_ReliableSampleUnacknowledgedStatus Struct Reference

Status about an unacknowledged sample that is removed from the `DDSDataWriter` cache before being acknowledged by all matched `DDSDataReader` entities.

```
#include <dds_c_publication.h>
```

Public Attributes

- struct `DDS_SequenceNumber_t` `sequence_number`
The sequence number of the unacknowledged sample.
- `DDS_Long` `unacknowledged_count`
The number of matched `DDSDataReader` entities that did not acknowledge the sample before it was removed.
- `DDS_InstanceHandle_t` `instance_handle`
The instance_handle for the sample. If the `DDSDataWriter` is not keyed the value is `DDS_HANDLE_NIL`.

7.64.1 Detailed Description

Status about an unacknowledged sample that is removed from the [DDSDataWriter](#) cache before being acknowledged by all matched [DDSDataReader](#) entities.

Entity:

[DDSDataWriter](#)

Listener:

[DDSDataWriterListener](#)

7.64.2 Member Data Documentation

sequence_number

```
struct DDS_SequenceNumber_t DDS_ReliableSampleUnacknowledgedStatus::sequence_number
```

The sequence number of the unacknowledged sample.

unacknowledged_count

```
DDS_Long DDS_ReliableSampleUnacknowledgedStatus::unacknowledged_count
```

The number of matched [DDSDataReader](#) entities that did not acknowledge the sample before it was removed.

instance_handle

```
DDS_InstanceHandle_t DDS_ReliableSampleUnacknowledgedStatus::instance_handle
```

The instance_handle for the sample. If the [DDSDataWriter](#) is not keyed the value is DDS_HANDLE_NIL.

7.65 DDS_RequestedDeadlineMissedStatus Struct Reference

DDS_REQUESTED_DEADLINE_MISSED_STATUS

```
#include <dds_c_subscription.h>
```

Public Attributes

- [DDS_Long total_count](#)
Total cumulative count of the deadlines detected for any instance read by the [DDSDataReader](#).
- [DDS_Long total_count_change](#)
The incremental number of deadlines detected since the last time the listener was called or the status was read.
- [DDS_InstanceHandle_t last_instance_handle](#)
Handle to the last instance in the [DDSDataReader](#) for which a deadline was detected.

7.65.1 Detailed Description

DDS_REQUESTED_DEADLINE_MISSED_STATUS

7.65.2 Member Data Documentation

total_count

`DDS_Long DDS_RequestedDeadlineMissedStatus::total_count`

Total cumulative count of the deadlines detected for any instance read by the [DDSDataReader](#).

total_count_change

`DDS_Long DDS_RequestedDeadlineMissedStatus::total_count_change`

The incremental number of deadlines detected since the last time the listener was called or the status was read.

last_instance_handle

`DDS_InstanceHandle_t DDS_RequestedDeadlineMissedStatus::last_instance_handle`

Handle to the last instance in the [DDSDataReader](#) for which a deadline was detected.

7.66 DDS_RequestedIncompatibleQosStatus Struct Reference

DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS

```
#include <dds_c_subscription.h>
```

Public Attributes

- [DDS_Long total_count](#)
Total cumulative count of how many times the concerned [DDSDataReader](#) discovered a [DDSDataWriter](#) for the same [DDSTopic](#) with an offered QoS that is incompatible with that requested by the [DDSDataReader](#).
- [DDS_Long total_count_change](#)
The change in `total_count` since the last time the listener was called or the status was read.
- [DDS_QosPolicyId_t last_policy_id](#)
The `PolicyId_t` of one of the policies that was found to be incompatible the last time an incompatibility was detected.
- struct `DDS_QosPolicyCountSeq` [policies](#)
Not used in RTI Connext DDS Micro.

7.66.1 Detailed Description

DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS

See also

[RELIABILITY](#)
[OWNERSHIP](#)
[LIVELINESS](#)
[DEADLINE](#)

7.66.2 Member Data Documentation

total_count

[DDS_Long](#) DDS_RequestedIncompatibleQosStatus::total_count

Total cumulative count of how many times the concerned [DDSDataReader](#) discovered a [DDSDataWriter](#) for the same [DDSTopic](#) with an offered QoS that is incompatible with that requested by the [DDSDataReader](#).

total_count_change

[DDS_Long](#) DDS_RequestedIncompatibleQosStatus::total_count_change

The change in `total_count` since the last time the listener was called or the status was read.

last_policy_id

[DDS_QosPolicyId_t](#) DDS_RequestedIncompatibleQosStatus::last_policy_id

The `PolicyId_t` of one of the policies that was found to be incompatible the last time an incompatibility was detected.

policies

```
struct DDS_QosPolicyCountSeq DDS_RequestedIncompatibleQosStatus::policies
```

Not used in RTI Connex DDS Micro.

7.67 DDS_ResourceLimitsQosPolicy Struct Reference

Specifies the resources that RTI Connex DDS Micro can consume in order to meet the requested QoS.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_Long max_samples](#)
Represents the maximum samples the middleware can store for any one [DDSDDataWriter](#) or [DDSDDataReader](#).
- [DDS_Long max_instances](#)
Represents the maximum number of instances a [DDSDDataWriter](#) or [DDSDDataReader](#) can manage.
- [DDS_Long max_samples_per_instance](#)
Represents the maximum number of samples of any one instance a [DDSDDataWriter](#) or [DDSDDataReader](#) can manage.

7.67.1 Detailed Description

Specifies the resources that RTI Connex DDS Micro can consume in order to meet the requested QoS.

Entity:

[DDSTopic](#), [DDSDDataReader](#), [DDSDDataWriter](#)

Status:

[DDS_SAMPLE_REJECTED_STATUS](#), [DDS_SampleRejectedStatus](#)

Properties:

[RxO](#) = NO
[Changeable](#) = NO

7.67.2 Usage

This policy controls the resources that RTI Connex DDS Micro can use in order to meet the requirements imposed by the application and other QoS settings.

If [DDSDDataWriter](#) objects are communicating samples faster than they are ultimately taken by the [DDSDDataReader](#) objects, the middleware will eventually hit against some of the QoS-imposed resource limits. Note that this may occur when just a single [DDSDDataReader](#) cannot keep up with its corresponding [DDSDDataWriter](#). The behavior in this case depends on the setting for the [RELIABILITY](#). If reliability is [DDS_BEST_EFFORT_RELIABILITY_QOS](#) then RTI Connex DDS Micro is allowed to drop samples. If the reliability is [DDS_RELIABLE_RELIABILITY_QOS](#), RTI Connex DDS Micro will block the [DDSDDataWriter](#) or discard the sample at the [DDSDDataReader](#) in order not to lose existing samples.

See also

[DDS_ReliabilityQosPolicy](#)
[DDS_HistoryQosPolicy](#)

7.67.3 Consistency

All limits must be set to positive finite values. Furthermore, `max_samples` must be large enough to contain all samples for all instances:

```
max_samples >= max_instances * max_samples_per_instance
```

The setting of [RESOURCE_LIMITS](#) `max_samples_per_instance` must be consistent with the [HISTORY](#) `depth`. For these two QoS to be consistent, they must verify that *depth* ≤ *max_samples_per_instance*.

See also

[DDS_HistoryQosPolicy](#)

7.67.4 Member Data Documentation

`max_samples`

```
DDS_Long DDS_ResourceLimitsQosPolicy::max_samples
```

Represents the maximum samples the middleware can store for any one [DDSDDataWriter](#) or [DDSDDataReader](#).

Specifies the maximum number of data samples a [DDSDDataWriter](#) or [DDSDDataReader](#) can manage across all the instances associated with it.

For unkeyed types this value has to be equal to `max_samples_per_instance`.

[DDSDDataReader](#)

The [DDS_ResourceLimitsQosPolicy::max_samples](#) resource limit limits the maximum number of samples a [DDSDDataReader](#) can allocate and cache. Samples are cached before they are made available to the application; for example, when samples are received out of order. Thus, this resource limit may impact throughput and latency.

What this value should be set to is determined by a number of factors:

- For best effort, it can be set to 1 since samples are not cached. However, if data is polled or samples are loaned longer than the update frequency, this resource-limit should be increased to prevent samples from being reused before the application has a chance to access the data (along with [DDS_HistoryQosPolicy::depth](#)).
- For reliable data, the value should take into account the data rate, the heartbeat rate of the [DDSDDataWriter](#), and the number of piggybacked heartbeats.

[DDSDDataWriter](#)

The [DDS_ResourceLimitsQosPolicy::max_samples](#) resource limit limits the maximum number of samples a [DDSDDataWriter](#) can allocate and maintain state for at any given time. Every time a write operation is called, a sample is allocated from the cache and filled in with user data.

A sample allocated from the pool controlled by this resource limit can be reused based on the following rules:

- If the [DDSDataWriter](#) is using [DDS_BEST_EFFORT_RELIABILITY_QOS](#) effort, the sample is returned for reuse after the transport returns from the write call.
- If the [DDSDataWriter](#) is using [DDS_RELIABLE_RELIABILITY_QOS](#) and [DDS_VOLATILE_DURABILITY_QOS](#), the sample is returned when all known [DDSDataReader](#) entities have acknowledged the sample or the [DDSDataWriter](#) has given up on delivering the sample, controlled by [DDS_RtpsReliableWriterProtocol_t::max_heartbeat_retries](#), whichever happens first.
- If the [DDSDataWriter](#) is using [DDS_RELIABLE_RELIABILITY_QOS](#) and not using [DDS_VOLATILE_DURABILITY_QOS](#), the sample is returned to the pool for reuse only if the history depth is exceeded and the [DDS_HistoryQosPolicy::kind](#) is set to [DDS_KEEP_LAST_HISTORY_QOS](#).

For best effort communication, this resource limit can be set to 1.

For reliable communication, the value of this resource limit determines the maximum number of samples a [DDSDataWriter](#) can keep in unacknowledged state before reusing potentially unacknowledged samples for [DDS_KEEP_LAST_HISTORY_QOS](#). For reliable data, the value should take into account the data rate, the [DDS_RtpsReliableWriterProtocol_t::heartbeat_period](#), and [DDS_RtpsReliableWriterProtocol_t::heartbeats_per_max_samples](#).

See also

[DDS_HistoryQosPolicy::depth](#), [DDS_DataWriterQos](#)

[default] 1

[range] [1, 100 million] $\geq$ max_samples_per_instance * max_instances

max_instances

[DDS_Long](#) [DDS_ResourceLimitsQosPolicy::max_instances](#)

Represents the maximum number of instances a [DDSDataWriter](#) or [DDSDataReader](#) can manage.

DDSDataReader

The max_instance resource-limit limits the maximum number of instances a [DDSDataReader](#) can manage. Instance state is one of the most expensive resources to maintain, as well as often being the highest one.

The resources allocated by an instance are normally only released when an instance is disposed *and* all known owners have unregistered the instance; please refer to [FooDataWriter::unregister_instance](#) and [FooDataWriter::dispose](#).

The [DDS_DataReaderResourceLimitsQosPolicy::instance_replacement](#) policy is useful to maintain state for only the last N instances, and when it is not necessary to keep samples in the [DDSDataReader](#) history cache. When the [DDS_DataReaderResourceLimitsQosPolicy::instance_replacement](#) policy is set to replace the oldest instance, [DDS_REPLACE_OLDEST_INSTANCE_REPLACEMENT_QOS](#), information about the oldest instance is automatically removed and the newly discovered instance is received. The disadvantage is that [DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE](#) state cannot be maintained. The [DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE](#) is maintained for an instance that is accepted, either when there is space or another instance is replaced, regardless of whether any samples have previously been received.

See also

[DDS_DataReaderResourceLimitsInstanceReplacementKind](#) instance_replacement

DDSDataWriter

The [DDS_ResourceLimitsQosPolicy::max_instances](#) resource limit limits the maximum number of instances a [DDSDataWriter](#) can manage. An instance resource can only be reused when it is unregistered by the [DDSDataWriter](#).

[default] 1

[range] [1, 1 million]

max_samples_per_instance

DDS_Long DDS_ResourceLimitsQosPolicy::max_samples_per_instance

Represents the maximum number of samples of any one instance a [DDSDataWriter](#) or [DDSDataReader](#) can manage.

[DDSDataReader](#)

This resource limit limits the maximum number of samples that can be cached for a single instance; it must be less than or equal to max_samples and greater or equal to the history depth. When [DDS_ResourceLimitsQosPolicy::max_samples_per_instance](#) is higher than depth, it makes it possible to loan samples and receive up to depth samples before returning loaned samples.

When this resource-limit or [DDS_HistoryQosPolicy::depth](#) is exceeded and the [DDS_HistoryQosPolicy::kind](#) is [DDS_KEEP_LAST_HISTORY_QOS](#), the oldest sample is removed from the cache.

Note that it is possible to exceed the max_samples_per_instance limit while samples are being received and processed, but before they are added to the reader queue. This typically happens when there are holes in the sequence numbers that must be filled before the samples can be committed to the reader queue. If a sample cannot be committed due to the max_samples_per_instance limit, it is not dropped. Instead, it is recommitted when space is made available, such as the application taking samples from the reader queue or older samples being removed due to [KEEP_LAST](#).

For unkeyed types this value has to be equal to max_samples.

[DDSDataWriter](#)

This resource limit limits the maximum number of samples that can be cached for a single instance; it must be less than or equal to max_samples and greater or equal to the [DDS_HistoryQosPolicy::depth](#).

[DDS_HistoryQosPolicy::depth](#) determines how many samples are kept in case of [DDS_TRANSIENT_LOCAL_DURABILITY_QOS](#) durability and made available to late joining [DDSDataReader](#), while [DDS_ResourceLimitsQosPolicy::max_samples_per_instance](#) is the maximum number of *unacknowledged* samples the [DDSDataWriter](#) can keep per instance. Thus, if max_samples_per_instance > depth, it is possible to keep additional unacknowledged samples.

[default] 1

[range] [1, 100 million], ≤ max_samples, ≥ [DDS_HistoryQosPolicy::depth](#)

7.68 DDS_RtpsReliableReaderProtocol_t Struct Reference

Qos related to reliable reader protocol defined in RTPS.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- struct [DDS_Duration_t nack_period](#)
The period at which to send NACKs.

7.68.1 Detailed Description

Qos related to reliable reader protocol defined in RTPS.

It is used to configure the reliable reader according to RTPS protocol.

Properties:

RxO = N/A
Changeable = NO

QoS:

[DDS_DataReaderProtocolQosPolicy](#)

7.68.2 Member Data Documentation

nack_period

```
struct DDS\_Duration\_t DDS_RtpsReliableReaderProtocol_t::nack_period
```

The period at which to send NACKs.

A reliable reader will send periodic NACKs at this rate when it first matches with a reliable writer. The reader will stop sending NACKs when it has received all available historical data from the writer. Note that this NACK period only determines the rate at which NACKs are sent until the first HB is received from the writer. When the first HB is received, the writer's HB period determines how fast historical data is sent.

[default] 50 ms

[range] [1 nanosec, 1 year]

7.69 DDS_RtpsReliableWriterProtocol_t Struct Reference

QoS related to the reliable writer protocol defined in RTPS.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- struct [DDS_Duration_t](#) heartbeat_period
The period at which to send heartbeats.
- [DDS_Long](#) heartbeats_per_max_samples
The number of heartbeats per send queue.
- [DDS_Long](#) max_send_window
The maximum size of a reliable send window.
- [DDS_Long](#) max_heartbeat_retries
The maximum number of heartbeat retries before marking a remote reader as inactive.

7.69.1 Detailed Description

QoS related to the reliable writer protocol defined in RTPS.

It is used to configure a reliable writer according to RTPS protocol.

Properties:

RxO = N/A
Changeable = NO

QoS:

[DDS_DataWriterProtocolQosPolicy](#)

7.69.2 Member Data Documentation

heartbeat_period

```
struct DDS\_Duration\_t DDS_RtpsReliableWriterProtocol_t::heartbeat_period
```

The period at which to send heartbeats.

A reliable writer will send periodic heartbeats at this rate.

[default] 3.0 seconds

[range] [1 nanosec, 1 year]

heartbeats_per_max_samples

```
DDS\_Long DDS_RtpsReliableWriterProtocol_t::heartbeats_per_max_samples
```

The number of heartbeats per send queue.

A piggyback heartbeat will be sent every [[DDS_ResourceLimitsQosPolicy::max_samples/heartbeats_per_max_↵](#) samples] samples.

If set to zero, no piggyback heartbeat will be sent. If [DDS_ResourceLimitsQosPolicy::max_samples](#) is [DDS_LENGTH_↵](#) _UNLIMITED, 100 million is assumed as the maximum value in the calculation.

[default] 1

[range] [0, 100 million]

- For [DDS_DataWriterProtocolQosPolicy::rtps_reliable_writer](#):
heartbeats_per_max_samples ≤ [DDS_ResourceLimitsQosPolicy::max_samples](#)

max_send_window

`DDS_Long DDS_RtpsReliableWriterProtocol_t::max_send_window`

The maximum size of a reliable send window.

A reliable [DDSDataWriter](#) may maintain a send window of samples in-flight that have not been acknowledged by its [DDSDataReader](#)'s. The `max_send_window` is the maximum number of unacknowledged samples in-flight. When the window is at `max_send_window`, subsequent writes will pass but not publish any data over the network.

[default] `DDS_LENGTH_UNLIMITED`, meaning the window will be as large as the resource limits allow

[range] [1, 256] or `DDS_LENGTH_UNLIMITED`

max_heartbeat_retries

`DDS_Long DDS_RtpsReliableWriterProtocol_t::max_heartbeat_retries`

The maximum number of heartbeat retries before marking a remote reader as inactive.

When a remote reader has not acked all the samples the reliable writer has in its queue, and `max_heartbeat_retries` number of periodic heartbeats has been sent without receiving any ack/nack back, the remote reader will be marked as inactive (not alive) and be ignored until it resumes sending ack/nack.

[default] `DDS_LENGTH_UNLIMITED`

[range] [1, 1 million] or `DDS_LENGTH_UNLIMITED`

7.70 DDS_RtpsWellKnownPorts_t Struct Reference

RTPS well-known port mapping configuration.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_Long port_base](#)
The base port offset.
- [DDS_Long domain_id_gain](#)
Tunable domain gain parameter.
- [DDS_Long participant_id_gain](#)
Tunable participant gain parameter.
- [DDS_Long builtin_multicast_port_offset](#)
*Additional offset for **metatraffic** multicast port.*
- [DDS_Long builtin_unicast_port_offset](#)
*Additional offset for **metatraffic** unicast port.*
- [DDS_Long user_multicast_port_offset](#)
*Additional offset for **user** traffic multicast port.*
- [DDS_Long user_unicast_port_offset](#)
*Additional offset for **user** traffic unicast port.*

7.70.1 Detailed Description

RTPS well-known port mapping configuration.

RTI Connex DDS Micro uses the RTPS wire protocol. The discovery protocols defined by RTPS rely on well-known ports to initiate discovery. These well-known ports define the multicast and unicast ports on which a Participant will listen for discovery **metatraffic** from other Participants. The discovery metatraffic contains all the information required to establish the presence of remote DDS entities in the network.

The well-known ports are defined by RTPS in terms of port mapping expressions with several tunable parameters, which allow the user to customize what network ports are used by the middleware. These parameters are exposed in [DDS_RtpsWellKnownPorts_t](#). In order for all Participants in a system to correctly discover each other, it is important that they all use the same port mapping expressions.

The actual port mapping expressions, as defined by the RTPS specification, can be found below. In addition to the parameters listed in [DDS_RtpsWellKnownPorts_t](#), the port numbers depend on:

- domain_id, as specified in [DDSDomainParticipantFactory::create_participant](#)
- participant_id, as specified using [DDS_WireProtocolQosPolicy::participant_id](#)

The domain_id parameter ensures no port conflicts exist between Participants belonging to different domains. This also means that discovery metatraffic in one domain is not visible to Participants in a different domain. The participant_id parameter ensures that unique unicast port numbers are assigned to Participants belonging to the same domain on a given host.

The *metatraffic_unicast_port* is used to exchange discovery metatraffic using unicast.

```
metatraffic_unicast_port = port_base + (domain_id_gain * domain_id) + (participant_id_gain * participant_id) + bu
```

The *metatraffic_multicast_port* is used to exchange discovery metatraffic using multicast.

```
metatraffic_multicast_port = port_base + (domain_id_gain * domain_id) + builtin_multicast_port_offset
```

RTPS also defines the *default* multicast and unicast ports on which [DDSDataReader](#) and [DDSDataWriter](#) entities receive **user** traffic.

The *usertraffic_unicast_port* is used to exchange user data using unicast.

```
usertraffic_unicast_port = port_base + (domain_id_gain * domain_id) + (participant_id_gain * participant_id) + us
```

The *usertraffic_multicast_port* is used to exchange user data using multicast. The corresponding multicast group addresses can be configured using

```
usertraffic_multicast_port = port_base + (domain_id_gain * domain_id) + user_multicast_port_offset
```

By default, the port mapping parameters are configured to be compliant with OMG's DDS Interoperability Wire Protocol (see also [DDS_INTEROPERABLE_RTPS_WELL_KNOWN_PORTS](#)).

The OMG's DDS Interoperability Wire Protocol compliant port mapping parameters are *not* backwards compatible with previous versions of the RTI Connex DDS Micro middleware.

When modifying the port mapping parameters, care must be taken to avoid port aliasing. This would result in undefined discovery behavior. The chosen parameter values will also determine the maximum possible number of domains in the system and the maximum number of participants per domain. Additionally, any resulting mapped port number must be within the range imposed by the underlying transport. For example, for UDPv4, this range typically equals [1024 - 65535].

QoS:

[DDS_WireProtocolQosPolicy](#)

7.70.2 Member Data Documentation

port_base

`DDS_Long DDS_RtpsWellKnownPorts_t::port_base`

The base port offset.

All mapped well-known ports are offset by this value.

[default] 7400

[range] [≥ 1], but resulting ports must be within the range imposed by the underlying transport.

domain_id_gain

`DDS_Long DDS_RtpsWellKnownPorts_t::domain_id_gain`

Tunable domain gain parameter.

Multiplier of the `domain_id`. Together with `participant_id_gain`, it determines the highest `domain_id` and `participant_id` allowed on this network.

In general, there are two ways to setup `domain_id_gain` and `participant_id_gain` parameters.

If `domain_id_gain > participant_id_gain`, it results in a port mapping layout where all [DDSDomainParticipant](#) instances within a single domain occupy a consecutive range of `domain_id_gain` ports. Precisely, all ports occupied by the domain fall within:

```
(port_base + (domain_id_gain * domain_id))
```

and:

```
(port_base + (domain_id_gain * (domain_id + 1)) - 1)
```

Under such a case, the highest `domain_id` is limited only by the underlying transport's maximum port. The highest `participant_id`, however, must satisfy:

```
max_participant_id < (domain_id_gain / participant_id_gain)
```

On the contrary, if `domain_id_gain <= participant_id_gain`, it results in a port mapping layout where a given domain's [DDSDomainParticipant](#) instances occupy ports spanned across the entire valid port range allowed by the underlying transport. For instance, it results in the following potential mapping:

Mapped Port	Domain Id	Participant ID
higher port number	Domain Id = 1	Participant ID = 2
	Domain Id = 0	Participant ID = 2
	Domain Id = 1	Participant ID = 1
	Domain Id = 0	Participant ID = 1
	Domain Id = 1	Participant ID = 0
lower port number	Domain Id = 0	Participant ID = 0

Under this case, the highest `participant_id` is limited only by the underlying transport's maximum port. The highest `domain_id`, however, must satisfy:

```
max_domain_id < (participant_id_gain / domain_id_gain)
```

Additionally, `domain_id_gain` also determines the range of the port-specific offsets.

```
domain_id_gain > abs(builtin_multicast_port_offset - user_multicast_port_offset)
```

```
domain_id_gain > abs(builtin_unicast_port_offset - user_unicast_port_offset)
```

Violating this may result in port aliasing and undefined discovery behavior.

[default] 250

[range] [> 0], but resulting ports must be within the range imposed by the underlying transport.

participant_id_gain

`DDS_Long DDS_RtpsWellKnownPorts_t::participant_id_gain`

Tunable participant gain parameter.

Multiplier of the `participant_id`. See `DDS_RtpsWellKnownPorts_t::domain_id_gain` for its implications on the highest `domain_id` and `participant_id` allowed on this network.

Additionally, `participant_id_gain` also determines the range of `builtin_unicast_port_offset` and `user_unicast_port_offset`.

```
participant_id_gain > abs(builtin_unicast_port_offset - user_unicast_port_offset)
```

[default] 2

[range] [> 0], but resulting ports must be within the range imposed by the underlying transport.

builtin_multicast_port_offset

`DDS_Long DDS_RtpsWellKnownPorts_t::builtin_multicast_port_offset`

Additional offset for **metatraffic** multicast port.

It must be unique from other port-specific offsets.

[default] 0

[range] [≥ 0], but resulting ports must be within the range imposed by the underlying transport.

builtin_unicast_port_offset

`DDS_Long DDS_RtpsWellKnownPorts_t::builtin_unicast_port_offset`

Additional offset for **metatraffic** unicast port.

It must be unique from other port-specific offsets.

[default] 10

[range] [≥ 0], but resulting ports must be within the range imposed by the underlying transport.

user_multicast_port_offset

`DDS_Long DDS_RtpsWellKnownPorts_t::user_multicast_port_offset`

Additional offset for **user** traffic multicast port.

It must be unique from other port-specific offsets.

[default] 1

[range] [≥ 0], but resulting ports must be within the range imposed by the underlying transport.

user_unicast_port_offset

`DDS_Long DDS_RtpsWellKnownPorts_t::user_unicast_port_offset`

Additional offset for **user** traffic unicast port.

It must be unique from other port-specific offsets.

[default] 11

[range] [≥ 0], but resulting ports must be within the range imposed by the underlying transport.

7.71 DDS_SampleIdentity_t Struct Reference

Data structure used to provide explicit identity information about a published data sample.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- struct [DDS_GUID_t](#) `writer_guid`
The GUID of the [DDSDataWriter](#) that published the sample.
- struct [DDS_SequenceNumber_t](#) `sequence_number`
Sequence number of the sample.

7.71.1 Detailed Description

Data structure used to provide explicit identity information about a published data sample.

7.71.2 Member Data Documentation

`writer_guid`

```
struct DDS\_GUID\_t DDS_SampleIdentity_t::writer_guid
```

The GUID of the [DDSDataWriter](#) that published the sample.

`sequence_number`

```
struct DDS\_SequenceNumber\_t DDS_SampleIdentity_t::sequence_number
```

Sequence number of the sample.

7.72 DDS_SampleInfo Struct Reference

Information that accompanies each sample that is `read` or `taken`.

```
#include <dds_c_subscription.h>
```

Public Attributes

- [DDS_SampleStateKind sample_state](#)
The sample state of the sample.
- [DDS_ViewStateKind view_state](#)
The view state of the instance.
- [DDS_InstanceStateKind instance_state](#)
The instance state of the instance.
- [DDS_InstanceHandle_t instance_handle](#)
Identifies locally the corresponding instance.
- [DDS_InstanceHandle_t publication_handle](#)
Identifies locally the DataWriter that modified the instance.
- [DDS_Boolean valid_data](#)
Indicates whether the DataSample contains data or else it is only used to communicate a change in the instance↔_state of the instance.
- struct [DDS_Time_t source_timestamp](#)
The timestamp when the sample was written by a DataWriter.
- struct [DDS_Time_t reception_timestamp](#)
<<eXtension>> The timestamp when the sample was committed by a DataReader.
- struct [DDS_SequenceNumber_t publication_sequence_number](#)
<<eXtension>> The publication sequence number.

7.72.1 Detailed Description

Information that accompanies each sample that is `read` or `taken`.

7.72.2 Interpretation of the SampleInfo

The [DDS_SampleInfo](#) contains information pertaining to the associated `Data` instance sample including:

- the `sample_state` of the `Data` value (i.e., if it has already been read or not)
- the `view_state` of the related instance (i.e., if the instance is new or not)
- the `instance_state` of the related instance (i.e., if the instance is alive or not)
- the `valid_data` flag. This flag indicates whether there is data associated with the sample. Some samples do not contain data indicating only a change on the `instance_state` of the corresponding instance.
- The `source_timestamp` of the sample. This is the timestamp provided by the [DDSDDataWriter](#) at the time the sample was produced.

See also

[DDS_SampleStateKind](#), [DDS_InstanceStateKind](#), [DDS_ViewStateKind](#), [DDS_SampleInfo::valid_data](#)

7.72.3 Member Data Documentation

sample_state

`DDS_SampleStateKind DDS_SampleInfo::sample_state`

The sample state of the sample.

Indicates whether or not the corresponding data sample has already been read.

See also

[DDS_SampleStateKind](#)

view_state

`DDS_ViewStateKind DDS_SampleInfo::view_state`

The view state of the instance.

Indicates whether the [DDSDataReader](#) has already seen samples for the most-current generation of the related instance.

See also

[DDS_ViewStateKind](#)

instance_state

`DDS_InstanceStateKind DDS_SampleInfo::instance_state`

The instance state of the instance.

Indicates whether the instance is currently in existence or, if it has been disposed, the reason why it was disposed.

See also

[DDS_InstanceStateKind](#)

instance_handle

`DDS_InstanceHandle_t DDS_SampleInfo::instance_handle`

Identifies locally the corresponding instance.

publication_handle

```
DDS_InstanceHandle_t DDS_SampleInfo::publication_handle
```

Identifies locally the `DataWriter` that modified the instance.

The `publication_handle` is the same `DDS_InstanceHandle_t` that is returned by the operation `DDSDataReader::get_matched_publication_handle` and can also be used as a parameter to the operation `DDSDataReader::get_matched_publication_data`.

valid_data

```
DDS_Boolean DDS_SampleInfo::valid_data
```

Indicates whether the `DataSample` contains data or else it is only used to communicate a change in the `instance_state` of the instance.

Normally each `DataSample` contains both a `DDS_SampleInfo` and some Data. However there are situations where a `DataSample` contains only the `DDS_SampleInfo` and does not have any associated data. This occurs when the RTI Connext DDS Micro notifies the application of a change of state for an instance that was caused by some internal mechanism (such as a timeout) for which there is no associated data. An example of this situation is when the RTI Connext DDS Micro detects that an instance has no writers and changes the corresponding `instance_state` to `DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE`.

The application can distinguish whether a particular `DataSample` has data by examining the value of the `valid_data` flag. If this flag is set to `DDS_BOOLEAN_TRUE`, then the `DataSample` contains valid Data. If the flag is set to `DDS_BOOLEAN_FALSE`, the `DataSample` contains no Data.

To ensure correctness and portability, the `valid_data` flag must be examined by the application prior to accessing the Data associated with the `DataSample` and if the flag is set to `DDS_BOOLEAN_FALSE`, the application should not access the Data associated with the `DataSample`, that is, the application should access only the `DDS_SampleInfo`.

source_timestamp

```
struct DDS_Time_t DDS_SampleInfo::source_timestamp
```

The timestamp when the sample was written by a `DataWriter`.

reception_timestamp

```
struct DDS_Time_t DDS_SampleInfo::reception_timestamp
```

<<eXtension>> The timestamp when the sample was committed by a `DataReader`.

The `reception_timestamp` is platform dependent and depends on the port of RTI Connext DDS Micro to the platform. Typically, it represents the current local time, but that may not always be the case. E.g, some platforms may not have a real-time clock and the system time is the time since the system was started.

A `reception_timestamp` equal to zero may mean any of the following:

- RTI Connext DDS Micro was unable to read the current time
- The system time is not incrementing
- Zero time has elapsed between the system was started and the first sample was committed.

publication_sequence_number

```
struct DDS_SequenceNumber_t DDS_SampleInfo::publication_sequence_number
```

<<*eXtension*>> The publication sequence number.

7.73 DDS_SampleInfoSeq Struct Reference

Declares IDL sequence < [DDS_SampleInfo](#) > .

```
#include <dds_c_subscription.h>
```

7.73.1 Detailed Description

Declares IDL sequence < [DDS_SampleInfo](#) > .

See also

[FooSeq](#)

7.74 DDS_SampleLostStatus Struct Reference[DDS_SAMPLE_LOST_STATUS](#)

```
#include <dds_c_subscription.h>
```

Public Attributes

- [DDS_Long total_count](#)
Total cumulative count of all samples lost across all instances of data published under the [DDSTopic](#).
- [DDS_Long total_count_change](#)
The incremental number of samples lost since the last time the listener was called or the status was read.
- [DDS_SampleLostStatusKind reason](#)
The reason a sample was lost.
- struct [DDS_SampleInfo sample_info](#)
Additional information about the sample lost for reasons other than ::DDS_BY_DATAWRITER and ::DDS_NOT_LOST.

7.74.1 Detailed Description[DDS_SAMPLE_LOST_STATUS](#)

7.74.2 Member Data Documentation

total_count

`DDS_Long DDS_SampleLostStatus::total_count`

Total cumulative count of all samples lost across all instances of data published under the [DDSTopic](#).

total_count_change

`DDS_Long DDS_SampleLostStatus::total_count_change`

The incremental number of samples lost since the last time the listener was called or the status was read.

reason

`DDS_SampleLostStatusKind DDS_SampleLostStatus::reason`

The reason a sample was lost.

sample_info

`struct DDS_SampleInfo DDS_SampleLostStatus::sample_info`

Additional information about the sample lost for reasons other than `::DDS_BY_DATAWRITER` and `::DDS_NOT_LOST`.

7.75 DDS_SampleRejectedStatus Struct Reference

[DDS_SAMPLE_REJECTED_STATUS](#)

```
#include <dds_c_subscription.h>
```

Public Attributes

- [DDS_Long total_count](#)
Total cumulative count of samples rejected by the [DDSDataReader](#).
- [DDS_Long total_count_change](#)
The incremental number of samples rejected since the last time the listener was called or the status was read.
- [DDS_SampleRejectedStatusKind last_reason](#)
Reason for rejecting the last sample rejected.
- [DDS_InstanceHandle_t last_instance_handle](#)
Handle to the instance being updated by the last sample that was rejected.

7.75.1 Detailed Description

DDS_SAMPLE_REJECTED_STATUS

A status structure describing the samples rejected by the reader.

7.75.2 Member Data Documentation

total_count

`DDS_Long DDS_SampleRejectedStatus::total_count`

Total cumulative count of samples rejected by the [DDSDataReader](#).

total_count_change

`DDS_Long DDS_SampleRejectedStatus::total_count_change`

The incremental number of samples rejected since the last time the listener was called or the status was read.

last_reason

`DDS_SampleRejectedStatusKind DDS_SampleRejectedStatus::last_reason`

Reason for rejecting the last sample rejected.

See also

[DDS_SampleRejectedStatusKind](#)

last_instance_handle

`DDS_InstanceHandle_t DDS_SampleRejectedStatus::last_instance_handle`

Handle to the instance being updated by the last sample that was rejected.

7.76 DDS_SecTransform_Configuration Struct Reference

A structure for configuration this Transform PSL.

```
#include <psk_oss1_transform.h>
```


Public Attributes

- `const char * provider_name`
A 0-terminated string with the name of the crypto provider to use. or NULL for the default provider.
- `struct PSK_ErrListener err_listener`
Configuration items for an error callback mechanism.

7.76.1 Detailed Description

A structure for configuration this Transform PSL.

The values of the fields in this structure influence how the Transform PSL behaves. The details of what the impact is, is known to the implementation of the PSL only.

The configuration has to be provided at creation time and is immutable.

See Also `PSK_OSSL_get_default_configuration`

7.76.2 Member Data Documentation

`provider_name`

```
const char* DDS_SecTransform_Configuration::provider_name
```

A 0-terminated string with the name of the crypto provider to use. or NULL for the default provider.

`err_listener`

```
struct PSK_ErrListener DDS_SecTransform_Configuration::err_listener
```

Configuration items for an error callback mechanism.

7.77 DDS_SequenceNumber_t Struct Reference

Type for *sequence* number representation.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- `DDS_Long high`
The most significant part of the sequence number.
- `DDS_UnsignedLong low`
The least significant part of the sequence number.

7.77.1 Detailed Description

Type for *sequence* number representation.

Represents a 64-bit sequence number.

7.77.2 Member Data Documentation

high

`DDS_Long DDS_SequenceNumber_t::high`

The most significant part of the sequence number.

low

`DDS_UnsignedLong DDS_SequenceNumber_t::low`

The least significant part of the sequence number.

7.78 DDS_SubscriberQos Struct Reference

QoS policies supported by a `DDSSubscriber` entity.

```
#include <dds_c_subscription.h>
```

Public Attributes

- struct `DDS_EntityFactoryQosPolicy` `entity_factory`
Entity factory policy, ENTITY_FACTORY.
- struct `DDS_PartitionQosPolicy` `partition`
The DDSSubscriber PARTITION policy.
- struct `DDS_GroupDataQosPolicy` `group_data`
Group data policy, GROUP_DATA.
- struct `DDS_EntityNameQosPolicy` `subscriber_name`
<<eXtension>> The DDSSubscriber name. ENTITY_NAME.

7.78.1 Detailed Description

QoS policies supported by a `DDSSubscriber` entity.

You must set the members in a consistent manner. If not, `DDSSubscriber::set_qos` and `DDSDomainParticipant::set_default_subscriber_qos` will fail with `DDS_RETCODE_INCONSISTENT_POLICY`, and `DDSDomainParticipant::create_subscriber` will fail.

7.78.2 Member Data Documentation

entity_factory

```
struct DDS_EntityFactoryQosPolicy DDS_SubscriberQos::entity_factory
```

Entity factory policy, [ENTITY_FACTORY](#).

partition

```
struct DDS_PartitionQosPolicy DDS_SubscriberQos::partition
```

The [DDSSubscriber](#) PARTITION policy.

A set of strings that introduces logical PARTITIONS.

Only entities that belong to the same PARTITION can communicate to each other. The PARTITION QoS policy applies directly to the [DDSSubscriber](#). [DDSDataReader](#) belong to the PARTITIONS of the [DDSSubscriber](#) that created them.

The default PARTITION during initialization is the empty string, "".

[PARTITION](#)

group_data

```
struct DDS_GroupDataQosPolicy DDS_SubscriberQos::group_data
```

Group data policy, [GROUP_DATA](#).

subscriber_name

```
struct DDS_EntityNameQosPolicy DDS_SubscriberQos::subscriber_name
```

[<<eXtension>>](#) The [DDSSubscriber](#) name. [ENTITY_NAME](#).

7.79 DDS_SubscriptionBuiltinTopicData Struct Reference

Entry created when a [DDSDataReader](#) is discovered in association with its Subscriber.

```
#include <dds_c_discovery.h>
```

Public Attributes

- struct [DDS_BuiltinTopicKey_t](#) `key`
DCPS key to distinguish entries. This is used to configure the object ID of the remote [DDSDataReader](#).
- struct [DDS_BuiltinTopicKey_t](#) `participant_key`
DCPS key of the participant to which the [DDSDataReader](#) belongs.
- char * [topic_name](#)
Name of the related [DDSTopic](#).
- char * [type_name](#)
Name of the type attached to the [DDSTopic](#).
- struct [DDS_DeadlineQosPolicy](#) `deadline`
Policy of the corresponding [DataReader](#).
- struct [DDS_OwnershipQosPolicy](#) `ownership`
Policy of the corresponding [DataReader](#).
- struct [DDS_LatencyBudgetQosPolicy](#) `latency_budget`
Policy of the corresponding [DataReader](#).
- struct [DDS_ReliabilityQosPolicy](#) `reliability`
Policy of the corresponding [DataReader](#).
- struct [DDS_LivelinessQosPolicy](#) `liveliness`
Policy of the corresponding [DataReader](#).
- struct [DDS_DurabilityQosPolicy](#) `durability`
Policy of the corresponding [DataReader](#).
- struct [DDS_DestinationOrderQosPolicy](#) `destination_order`
Policy of the corresponding [DataReader](#).
- struct [DDS_LocatorSeq](#) `unicast_locator`
Custom unicast locator sequence that the endpoint can specify. The default locator sequence from the [DDSDomainParticipant](#) will be used if this is not specified.
- struct [DDS_LocatorSeq](#) `multicast_locator`
Custom multicast locator that the endpoint can specify. The default multicast locator sequence from the [DDSDomainParticipant](#) will be used if this is not specified.
- struct [DDS_PresentationQosPolicy](#) `presentation`
Policy of the Subscriber to which the [DDSDataReader](#) belongs.
- struct [DDS_DataRepresentationQosPolicy](#) `representation`
Data representation policy, [DATA_REPRESENTATION](#).
- struct [DDS_PartitionQosPolicy](#) `partition`
Policy of the Subscriber to which the [DataReader](#) belongs.
- struct [DDS_UserDataQosPolicy](#) `user_data`
Policy of the corresponding [DDSDataReader](#).
- struct [DDS_GroupDataQosPolicy](#) `group_data`
Policy of the Subscriber to which the [DataReader](#) belongs.
- struct [DDS_TopicDataQosPolicy](#) `topic_data`
Policy of the related Topic.
- struct [DDS_ContentFilterProperty](#) `content_filter`
<<experimental>> All of the information required for a [DataWriter](#) to perform writer-side filtering for a [DataReader](#).

7.79.1 Detailed Description

Entry created when a [DDSDataReader](#) is discovered in association with its Subscriber.

Data associated with the built-in topic [DDS_SUBSCRIPTION_TOPIC_NAME](#). It contains QoS policies and additional information that apply to the remote [DDSDataReader](#) and the related [DDSSubscriber](#).

See also

[DDS_SUBSCRIPTION_TOPIC_NAME](#)

7.79.2 Member Data Documentation

key

```
struct DDS\_BuiltinTopicKey\_t DDS_SubscriptionBuiltinTopicData::key
```

DCPS key to distinguish entries. This is used to configure the object ID of the remote [DDSDataReader](#).

Key GUID must be first

participant_key

```
struct DDS\_BuiltinTopicKey\_t DDS_SubscriptionBuiltinTopicData::participant_key
```

DCPS key of the participant to which the [DDSDataReader](#) belongs.

topic_name

```
char* DDS_SubscriptionBuiltinTopicData::topic_name
```

Name of the related [DDSTopic](#).

The length of this string is limited to 255 characters.

The memory for this field is managed as described in [String Support](#).

See also

[String Support](#)

type_name

```
char* DDS_SubscriptionBuiltinTopicData::type_name
```

Name of the type attached to the [DDSTopic](#).

The length of this string is limited to 255 characters.

The memory for this field is managed as described in [String Support](#).

See also

[String Support](#)

deadline

```
struct DDS_DeadlineQosPolicy DDS_SubscriptionBuiltinTopicData::deadline
```

Policy of the corresponding DataReader.

ownership

```
struct DDS_OwnershipQosPolicy DDS_SubscriptionBuiltinTopicData::ownership
```

Policy of the corresponding DataReader.

latency_budget

```
struct DDS_LatencyBudgetQosPolicy DDS_SubscriptionBuiltinTopicData::latency_budget
```

Policy of the corresponding DataReader.

reliability

```
struct DDS_ReliabilityQosPolicy DDS_SubscriptionBuiltinTopicData::reliability
```

Policy of the corresponding DataReader.

liveliness

```
struct DDS_LivelinessQosPolicy DDS_SubscriptionBuiltinTopicData::liveliness
```

Policy of the corresponding DataReader.

durability

```
struct DDS_DurabilityQosPolicy DDS_SubscriptionBuiltinTopicData::durability
```

Policy of the corresponding DataReader.

destination_order

```
struct DDS_DestinationOrderQosPolicy DDS_SubscriptionBuiltinTopicData::destination_order
```

Policy of the corresponding DataReader.

unicast_locator

```
struct DDS_LocatorSeq DDS_SubscriptionBuiltinTopicData::unicast_locator
```

Custom unicast locator sequence that the endpoint can specify. The default locator sequence from the [DDSDomainParticipant](#) will be used if this is not specified.

multicast_locator

```
struct DDS_LocatorSeq DDS_SubscriptionBuiltinTopicData::multicast_locator
```

Custom multicast locator that the endpoint can specify. The default multicast locator sequence from the [DDSDomainParticipant](#) will be used if this is not specified.

presentation

```
struct DDS_PresentationQosPolicy DDS_SubscriptionBuiltinTopicData::presentation
```

Policy of the Subscriber to which the [DDSDataReader](#) belongs.

representation

```
struct DDS_DataRepresentationQosPolicy DDS_SubscriptionBuiltinTopicData::representation
```

Data representation policy, [DATA_REPRESENTATION](#).

partition

```
struct DDS_PartitionQosPolicy DDS_SubscriptionBuiltinTopicData::partition
```

Policy of the Subscriber to which the DataReader belongs.

user_data

```
struct DDS_UserDataQosPolicy DDS_SubscriptionBuiltinTopicData::user_data
```

Policy of the corresponding [DDSDataReader](#).

group_data

```
struct DDS_GroupDataQosPolicy DDS_SubscriptionBuiltinTopicData::group_data
```

Policy of the Subscriber to which the DataReader belongs.

topic_data

```
struct DDS_TopicDataQosPolicy DDS_SubscriptionBuiltinTopicData::topic_data
```

Policy of the related Topic.

content_filter

```
struct DDS_ContentFilterProperty DDS_SubscriptionBuiltinTopicData::content_filter
```

<<experimental>> All of the information required for a DataWriter to perform writer-side filtering for a DataReader.

If the content filtering feature is not enabled, then this field is unused.

See also

[::DDSFilterPluginFactory::register](#) for how to enable the content filtering feature

7.80 DDS_SubscriptionMatchedStatus Struct Reference**DDS_SUBSCRIPTION_MATCHED_STATUS**

```
#include <dds_c_subscription.h>
```

Public Attributes

- [DDS_Long total_count](#)
The total cumulative number of times the concerned [DDSDataReader](#) discovered a "match" with a [DDSDataWriter](#).
- [DDS_Long total_count_change](#)
The change in total_count since the last time the listener was called or the status was read.
- [DDS_Long current_count](#)
The current number of writers with which the [DDSDataReader](#) is matched.
- [DDS_Long current_count_change](#)
The change in current_count since the last time the listener was called or the status was read.
- [DDS_InstanceHandle_t last_publication_handle](#)
A handle to the last [DDSDataWriter](#) that caused the status to change.

7.80.1 Detailed Description

DDS_SUBSCRIPTION_MATCHED_STATUS

A "match" happens when the [DDSDataReader](#) finds a [DDSDataWriter](#) for the same [DDSTopic](#) with an offered QoS that is compatible with that requested by the [DDSDataReader](#).

This status is also changed (and the listener, if any, called) when a match is ended. A local [DDSDataReader](#) will become "unmatched" from a remote [DDSDataWriter](#) when that [DDSDataWriter](#) goes away for any reason.

7.80.2 Member Data Documentation

total_count

[DDS_Long](#) `DDS_SubscriptionMatchedStatus::total_count`

The total cumulative number of times the concerned [DDSDataReader](#) discovered a "match" with a [DDSDataWriter](#).

This number increases whenever a new match is discovered. It does not change when an existing match goes away.

total_count_change

[DDS_Long](#) `DDS_SubscriptionMatchedStatus::total_count_change`

The change in `total_count` since the last time the listener was called or the status was read.

current_count

[DDS_Long](#) `DDS_SubscriptionMatchedStatus::current_count`

The current number of writers with which the [DDSDataReader](#) is matched.

This number increases when a new match is discovered and decreases when an existing match goes away.

current_count_change

[DDS_Long](#) `DDS_SubscriptionMatchedStatus::current_count_change`

The change in `current_count` since the last time the listener was called or the status was read.

last_publication_handle

[DDS_InstanceHandle_t](#) `DDS_SubscriptionMatchedStatus::last_publication_handle`

A handle to the last [DDSDataWriter](#) that caused the status to change.

7.81 DDS_SystemResourceLimitsQosPolicy Struct Reference

Resource limits that apply only to [DDSDomainParticipantFactory](#)

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_Long max_participants](#)
The maximum number of DomainParticipants ([DDSDomainParticipant](#)) the [DDSDomainParticipantFactory](#) can manage.
- [DDS_Long max_components](#)
The maximum number of components that may be registered on the [DDSDomainParticipantFactory](#). Note that changing this value can cause a [DDSDomainParticipantFactory](#) to free and reallocate memory.

7.81.1 Detailed Description

Resource limits that apply only to [DDSDomainParticipantFactory](#)

This QoS policy is an extension to the DDS standard.

Entity:

[DDSDomainParticipantFactory](#)

Properties:

[RxO](#) = N/A
[Changeable](#) = NO

7.81.2 Member Data Documentation

max_participants

[DDS_Long](#) [DDS_SystemResourceLimitsQosPolicy::max_participants](#)

The maximum number of DomainParticipants ([DDSDomainParticipant](#)) the [DDSDomainParticipantFactory](#) can manage.

The [DDSDomainParticipantFactory](#) can manage up to the number of DomainParticipants specified by this limit. However, the **actual** number of DomainParticipants that can be **created** with [DDSDomainParticipantFactory::create_participant](#) is further limited by the following constraints:

- Available memory and system resources such as network sockets and shared memory resources.
- The maximum number of [::max_timers](#) available. Each [DDSDomainParticipant](#) requires one timer plus one timer if Zero Copy v2 is enabled as a transport in [DDS_TransportQosPolicy::enabled_transports](#).

- The maximum number of `::max_user_blocking_threads`. Each receive-thread requires one blocking resource.
- DDS uses the [DDS_WireProtocolQosPolicy::rtps_well_known_ports](#) to discover other participants. The number of DomainParticipants that can be created is limited by port aliasing where ports calculated for different DDS domain IDs may overlap. RTI Connex DDS Micro does not determine if port aliasing may occur and the application must ensure that port aliasing does not occur.

Note that this value can only be modified until the first [DDSDomainParticipant](#) is created and may cause memory reallocation.

[default] 1

[range] [1, INT_MAX] with default QoS settings. Additional [DDSDomainParticipant](#) objects can be allocated by changing the [DDS_WireProtocolQosPolicy::rtps_well_known_ports](#) QoS policy on all [DDSDomainParticipant](#)'s on a node.

max_components

[DDS_Long](#) `DDS_SystemResourceLimitsQosPolicy::max_components`

The maximum number of components that may be registered on the [DDSDomainParticipantFactory](#). Note that changing this value can cause a [DDSDomainParticipantFactory](#) to free and reallocate memory.

RTI Connex DDS Micro consists of a number of plugins and components that perform various tasks. For example, the UDP transport and the DPDE discovery plugins are RTI Connex DDS Micro components. This resource-limit limits the number of components that can be registered with RTI Connex DDS Micro. This value should only be changed if plugins and components are developed.

[default] 16

7.82 DDS_Time_t Struct Reference

Type for *time* representation.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_LongLong](#) `sec`
seconds
- [DDS_UnsignedLong](#) `nanosec`
nanoseconds

7.82.1 Detailed Description

Type for *time* representation.

A [DDS_Time_t](#) represents a moment in time.

7.82.2 Member Data Documentation

sec

`DDS_LongLong DDS_Time_t::sec`

seconds

nanosec

`DDS_UnsignedLong DDS_Time_t::nanosec`

nanoseconds

7.83 DDS_TopicDataQosPolicy Struct Reference

Attaches a buffer of opaque data to a [DDSTopic](#) which is distributed to other applications as part of the discovery process.

```
#include <dds_c_user_data_qos.h>
```

Public Attributes

- struct DDS_OctetSeq [value](#)

The opaque data to be attached to the [DDSEntity](#).

7.83.1 Detailed Description

Attaches a buffer of opaque data to a [DDSTopic](#) which is distributed to other applications as part of the discovery process.

Entity:

[DDSTopic](#)

Properties:

[RxO](#) = NO

[Changeable](#) = NO

7.83.2 Usage

The TOPIC_DATA QoS policy allows an application to attach additional information to [DDSTopic](#) objects so that when a remote application discovers them, it can access that information and use it for its own purposes. This information is not interpreted by RTI Connex DDS Micro. Use cases are usually application-to-application identification, authentication, authorization, and encryption.

The topic data for a remote entity is received through the discovery process. The received topic data can be accessed through the functions for accessing the discovered entity data: [DDSDataWriter::get_matched_subscription_data](#) and [DDSDataReader::get_matched_publication_data](#).

Important: RTI Connex DDS Micro stores the data placed in this policy in pre-allocated pools. You must configure RTI Connex DDS Micro with the maximum size and number of the data sequences that will be stored in policies of this type. These settings are configured in the [DDS_DomainParticipantResourceLimitsQosPolicy](#).

7.83.3 Member Data Documentation

value

```
struct DDS_OctetSeq DDS_TopicDataQosPolicy::value
```

The opaque data to be attached to the [DDSEntity](#).

[default] empty (zero length)

7.84 DDS_TopicQos Struct Reference

QoS policies supported by a [DDSTopic](#) entity.

```
#include <dds_c_topic.h>
```

Public Attributes

- struct [DDS_TopicDataQosPolicy](#) [topic_data](#)
Topic data policy, TOPIC_DATA.

7.84.1 Detailed Description

QoS policies supported by a [DDSTopic](#) entity.

7.84.2 Member Data Documentation

topic_data

```
struct DDS_TopicDataQosPolicy DDS_TopicQos::topic_data
```

Topic data policy, [TOPIC_DATA](#).

7.85 DDS_TransportPriorityQosPolicy Struct Reference

This QoS policy allows the application to take advantage of transports that are capable of sending messages with different priorities.

```
#include <dds_c_transport_priority_qos.h>
```

Public Attributes

- [DDS_Long](#) value

This policy is a hint to the infrastructure as to how to set the priority of the underlying transport used to send the data.

7.85.1 Detailed Description

This QoS policy allows the application to take advantage of transports that are capable of sending messages with different priorities.

The Transport Priority QoS policy is optional and only supported on certain OSs and transports. It allows you to specify on a per [DDSDataWriter](#) or [DDSDataReader](#) basis that the data sent by that endpoint is of a different priority. The DDS specification does not indicate how a DDS implementation should treat data of different priorities. It is often difficult or impossible for DDS implementations to treat data of higher priority differently than data of lower priority, especially when data is being sent (delivered to a physical transport) directly by the thread that called `FooDataWriter_write`. Also, many physical network transports themselves do not have an end-user controllable level of data packet priority.-->

7.85.2 Member Data Documentation

value

[DDS_Long](#) DDS_TransportPriorityQosPolicy::value

This policy is a hint to the infrastructure as to how to set the priority of the underlying transport used to send the data.

You may choose any value within the range of a 32-bit signed integer; higher values indicate higher priority. However, any further interpretation of this policy is specific to a particular transport.

[default] 0

7.86 DDS_TransportQosPolicy Struct Reference

<<[eXtension](#)>> Settings for available transports

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- struct DDS_StringSeq [enabled_transports](#)
The transports available for use.

7.86.1 Detailed Description

<<[eXtension](#)>> Settings for available transports

Entity:

[DDSDomainParticipant](#), [DDSDataWriter](#), [DDSDataReader](#)

Properties:

[RxO](#) = N/A
[Changeable](#) = NO

7.86.2 Member Data Documentation

enabled_transports

```
struct DDS_StringSeq DDS_TransportQosPolicy::enabled_transports
```

The transports available for use.

A sequence of transport identifiers that defines the set of transports available for use by the associated DDS entity.

See also

[DDS_DomainParticipantQos](#), [DDS_DataWriterQos](#), [DDS_DataReaderQos](#)

[default] Empty sequence.

[range] Sequence of non-null,non-empty strings.

7.87 DDS_TrustQosPolicy Struct Reference

<<[eXtension](#)>> Configures the security plugin suite used by a [DDSDomainParticipant](#).

```
#include <dds_c_domain.h>
```

Public Attributes

- `RT_ComponentFactoryId_T` [suite](#)

The name of the registered security plugin suite to use.

7.87.1 Detailed Description

<<*eXtension*>> Configures the security plugin suite used by a [DDSDomainParticipant](#).

Entity:

[DDSDomainParticipant](#)

Properties:

`RxO` = N/A

`Changeable` = NO

7.87.2 Member Data Documentation

suite

```
RT_ComponentFactoryId_T DDS_TrustQosPolicy::suite
```

The name of the registered security plugin suite to use.

Identifies which registered security plugin suite the [DDSDomainParticipant](#) will use for secure communication. The name must match the name used when registering the plugin suite with the `RT_Registry`.

Use `RT_ComponentFactoryId_set_name` to set this field.

See also

[DDS_DomainParticipantQos](#)

[default] Empty (no security plugin suite).

7.88 DDS_UserDataQosPolicy Struct Reference

Attaches a buffer of opaque data to a [DDSEntity](#) which is distributed to other applications as part of the discovery process.

```
#include <dds_c_user_data_qos.h>
```


Public Attributes

- struct DDS_OctetSeq [value](#)

The opaque data to be attached to the [DDSEntity](#).

7.88.1 Detailed Description

Attaches a buffer of opaque data to a [DDSEntity](#) which is distributed to other applications as part of the discovery process.

Entity:

[DDSDomainParticipant](#), [DDSDataWriter](#), [DDSDataReader](#)

Properties:

[RxO](#) = NO

[Changeable](#) = NO

7.88.2 Usage

The USER_DATA QoS policy allows an application to attach additional information to [DDSEntity](#) objects so that when a remote application discovers them, it can access that information and use it for its own purposes. This information is not interpreted by RTI Connex DDS Micro. Use cases are usually application-to-application identification, authentication, authorization, and encryption.

The user data for a remote entity is received through the discovery process. The received user data can be accessed through the functions for accessing the discovered entity data: [DDSDomainParticipant::get_discovered_participant_data](#), [DDSDataWriter::get_matched_subscription_data](#), and [DDSDataReader::get_matched_publication_data](#) operations.

Important: RTI Connex DDS Micro stores the data placed in this policy in pre-allocated pools. You must configure RTI Connex DDS Micro with the maximum size and number of the data sequences that will be stored in policies of this type. These settings are configured in the [DDS_DomainParticipantResourceLimitsQosPolicy](#).

7.88.3 Member Data Documentation

value

```
struct DDS_OctetSeq DDS_UserDataQosPolicy::value
```

The opaque data to be attached to the [DDSEntity](#).

[default] empty (zero length)

7.89 DDS_UserTrafficQosPolicy Struct Reference

<<[eXtension](#)>> Configures the default transports enabled to transmit and receive user data traffic.

```
#include <dds_c_domain.h>
```

Public Attributes

- struct DDS_StringSeq [enabled_transports](#)
Configures the default transports enabled to transmit and receive user data traffic.

7.89.1 Detailed Description

<<[eXtension](#)>> Configures the default transports enabled to transmit and receive user data traffic.

Entity:

[DDSDomainParticipant](#)

Properties:

[RxO](#) = N/A
[Changeable](#) = NO

7.89.2 Member Data Documentation

enabled_transports

```
struct DDS_StringSeq DDS_UserTrafficQosPolicy::enabled_transports
```

Configures the default transports enabled to transmit and receive user data traffic.

A sequence of transport identifiers that defines the set of transports available for use by the associated DDS entity.

See also

[DDS_DomainParticipantQos](#)

[default] Empty sequence. When an empty sequence is passed to [DDSDomainParticipantFactory::create_participant](#) in the [DDS_DomainParticipantQos](#) and the UDP transport is registered the domain participant will automatically use the "_intra://" and "_udp://" transports for user-traffic. If the UDP transport is not registered, only the "_intra://" transport is used.

[range] Sequence of non-null, non-empty strings.

7.90 DDS_VendorId_t Struct Reference

<<eXtension>> Type used to represent the vendor of the service implementing the RTPS protocol.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_Octet vendorId \[DDS_VENDOR_ID_LENGTH_MAX\]](#)
The vendor Id.

7.90.1 Detailed Description

<<eXtension>> Type used to represent the vendor of the service implementing the RTPS protocol.

7.90.2 Member Data Documentation

vendorId

```
DDS_Octet DDS_VendorId_t::vendorId[DDS_VENDOR_ID_LENGTH_MAX]
```

The vendor Id.

7.91 DDS_WireProtocolQosPolicy Struct Reference

Specifies the wire-protocol-related attributes for the [DDSDomainParticipant](#).

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- [DDS_Long participant_id](#)
A value used to distinguish between different participants belonging to the same domain on the same host.
- [DDS_UnsignedLong rtps_host_id](#)
The RTPS Host ID of the domain participant.
- [DDS_UnsignedLong rtps_app_id](#)
The RTPS App ID of the domain participant.
- [DDS_UnsignedLong rtps_instance_id](#)
The RTPS Instance ID of the domain participant.
- struct [DDS_RtpsWellKnownPorts_t rtps_well_known_ports](#)
Configures the RTPS well-known port mappings.
- [DDS_Boolean compute_crc](#)
Adds RTPS checksum submessage to every message when this field is set to [DDS_BOOLEAN_TRUE](#).
- [DDS_Boolean check_crc](#)
Checks if the received RTPS message is valid by comparing the computed checksum with the received RTPS checksum submessage when this field is set to [DDS_BOOLEAN_TRUE](#).
- [DDS_Boolean require_crc](#)
Require that discovered participants send one of the checksums enabled in [DDS_WireProtocolQosPolicy::allowed_crc_mask](#).
- [DDS_ChecksumKind_t computed_crc_kind](#)
A bitmask indicating which checksum is used for sent messages when compute_crc is [DDS_BOOLEAN_TRUE](#). Only one mask is supported.
- [DDS_ChecksumKindMask_t allowed_crc_mask](#)
The checksums which are enabled for validating incoming packets.

7.91.1 Detailed Description

Specifies the wire-protocol-related attributes for the [DDSDomainParticipant](#).

Entity:

[DDSDomainParticipant](#)

Properties:

[RxO](#) = N/A

[Changeable](#) = NO

7.91.2 Usage

The default QoS policies returned by RTI Connex DDS Micro contain the correctly initialized wire protocol attributes.

The defaults are not expected to be modified normally, but are available to the advanced user customizing the implementation behavior.

The default values should not be modified without an understanding of the underlying Real-Time Publish Subscribe (RTPS) wire protocol.

7.91.3 Member Data Documentation

participant_id

[DDS_Long](#) [DDS_WireProtocolQosPolicy::participant_id](#)

A value used to distinguish between different participants belonging to the same domain on the same host.

Determines the unicast port on which meta-traffic is received. Also defines the *default* unicast port for receiving user-traffic for [DDSDataReader](#) and [DDSDataWriter](#) entities .

For more information on port mapping, please refer to [DDS_RtpsWellKnownPorts_t](#).

Each [DDSDomainParticipant](#) in the same domain, running on the same host, must have a unique `participant_id`. The participants may be in the same address space or in distinct address spaces.

A negative number (-1) means that RTI Connex DDS Micro will *automatically* resolve the participant ID as follows.

- RTI Connex DDS Micro will pick the *smallest* participant ID, based on the unicast ports available on the transports enabled for discovery.
- RTI Connex DDS Micro will attempt to resolve an automatic port index either when a [DDSDomainParticipant](#) is enabled, or when a [DDSDataReader](#) or a [DDSDataWriter](#) is created. Therefore, all the transports enabled for discovery must have been registered by this time. Otherwise, the discovery transports registered after resolving the automatic port index may produce port conflicts when the [DDSDomainParticipant](#) is enabled.

[default] -1 [automatic], i.e. RTI Connex DDS Micro will automatically pick the `participant_id`, as described above.

[range] [≥ 0], or -1, and does not violate guidelines stated in [DDS_RtpsWellKnownPorts_t](#).

See also

[DDSEntity::enable\(\)](#)

rtps_host_id

`DDS_UnsignedLong DDS_WireProtocolQosPolicy::rtps_host_id`

The RTPS Host ID of the domain participant.

Either all of `DDS_WireProtocolQosPolicy::rtps_host_id`, `DDS_WireProtocolQosPolicy::rtps_app_id`, and `DDS_WireProtocolQosPolicy::rtps_instance_id` for a DomainParticipant are set to `DDS_RTPS_AUTO_ID`, or none of them are set to `DDS_RTPS_AUTO_ID`.

[default] `DDS_RTPS_AUTO_ID`.

[range] `[0,0xffffffff]`

rtps_app_id

`DDS_UnsignedLong DDS_WireProtocolQosPolicy::rtps_app_id`

The RTPS App ID of the domain participant.

Either all of `DDS_WireProtocolQosPolicy::rtps_host_id`, `DDS_WireProtocolQosPolicy::rtps_app_id`, and `DDS_WireProtocolQosPolicy::rtps_instance_id` for a DomainParticipant are set to `DDS_RTPS_AUTO_ID`, or none of them are set to `DDS_RTPS_AUTO_ID`.

[default] `DDS_RTPS_AUTO_ID`.

[range] `[0,0xffffffff]`

rtps_instance_id

`DDS_UnsignedLong DDS_WireProtocolQosPolicy::rtps_instance_id`

The RTPS Instance ID of the domain participant.

Either all of `DDS_WireProtocolQosPolicy::rtps_host_id`, `DDS_WireProtocolQosPolicy::rtps_app_id`, and `DDS_WireProtocolQosPolicy::rtps_instance_id` for a DomainParticipant are set to `DDS_RTPS_AUTO_ID`, or none of them are set to `DDS_RTPS_AUTO_ID`.

[default] `DDS_RTPS_AUTO_ID`.

[range] `[0,0xffffffff]`

rtps_well_known_ports

`struct DDS_RtpsWellKnownPorts_t DDS_WireProtocolQosPolicy::rtps_well_known_ports`

Configures the RTPS well-known port mappings.

Determines the well-known multicast and unicast port mappings for discovery (meta) traffic and user traffic.

[default] `DDS_INTEROPERABLE_RTPS_WELL_KNOWN_PORTS`

compute_crc

DDS_Boolean DDS_WireProtocolQosPolicy::compute_crc

Adds RTPS checksum submessage to every message when this field is set to [DDS_BOOLEAN_TRUE](#).

The computed checksum covers the entire RTPS message including the RTPS header itself.

[default] [DDS_BOOLEAN_FALSE](#)

check_crc

DDS_Boolean DDS_WireProtocolQosPolicy::check_crc

Checks if the received RTPS message is valid by comparing the computed checksum with the received RTPS checksum submessage when this field is set to [DDS_BOOLEAN_TRUE](#).

[DDS_WireProtocolQosPolicy::compute_crc](#) must be enabled at the publishing application for validating the message at the subscribing application.

If a message is received with an invalid checksum, the message is discarded. If no checksum is received or found, the message is accepted but may still be corrupted.

[default] [DDS_BOOLEAN_FALSE](#)

require_crc

DDS_Boolean DDS_WireProtocolQosPolicy::require_crc

Require that discovered participants send one of the checksums enabled in [DDS_WireProtocolQosPolicy::allowed_crc_mask](#).

If an invalid checksum is received, the message is dropped.

If the value is [DDS_BOOLEAN_TRUE](#) the participant requires that discovered participants send one of the checksums enabled in [DDS_WireProtocolQosPolicy::allowed_crc_mask](#). If this is not the case the participant is ignored and not matching will take place.

This is a stronger check than [DDS_WireProtocolQosPolicy::check_crc](#). If an invalid checksum is received or not found, the message is dropped.

[default] [DDS_BOOLEAN_FALSE](#)

computed_crc_kind

`DDS_ChecksumKind_t DDS_WireProtocolQosPolicy::computed_crc_kind`

A bitmask indicating which checksum is used for sent messages when `compute_crc` is `DDS_BOOLEAN_TRUE`. Only one mask is supported.

- `DDS_CHECKSUM_AUTO`
- `DDS_CHECKSUM_BUILTIN32`
- `DDS_CHECKSUM_BUILTIN64`
- `DDS_CHECKSUM_BUILTIN128`

[default] `DDS_CHECKSUM_AUTO`.

allowed_crc_mask

`DDS_ChecksumKindMask_t DDS_WireProtocolQosPolicy::allowed_crc_mask`

The checksums which are enabled for validating incoming packets.

The following bitmask can be OR'ed for all enabled checksums:

- `DDS_CHECKSUM_AUTO`
- `DDS_CHECKSUM_BUILTIN32`
- `DDS_CHECKSUM_BUILTIN64`
- `DDS_CHECKSUM_BUILTIN128`

Note that the selected checksums must be a subset of what is supported. If a bitmask that is not supported is selected participant creation will fail.

[default] `DDS_CHECKSUM_AUTO`.

7.92 DDS_WriteParams_t Struct Reference

Data structure used to provide custom parameters to a `DDSDDataWriter`'s write operation.

```
#include <dds_c_infrastructure.h>
```

Public Attributes

- struct `DDS_Time_t` `source_timestamp`
Source time-stamp of the published sample.
- `DDS_InstanceHandle_t` `handle`
DDS_InstanceHandle_t value identifying the instance the sample is being written to.

7.92.1 Detailed Description

Data structure used to provide custom parameters to a [DDSDataWriter](#)'s write operation.

7.92.2 Member Data Documentation

source_timestamp

```
struct DDS_Time_t DDS_WriteParams_t::source_timestamp
```

Source time-stamp of the published sample.

handle

```
DDS_InstanceHandle_t DDS_WriteParams_t::handle
```

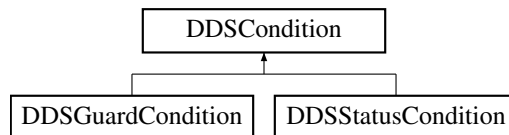
[DDS_InstanceHandle_t](#) value identifying the instance the sample is being written to.

7.93 DDSCondition Class Reference

<<interface>> Root class for all the conditions that may be attached to a [DDSWaitSet](#).

```
#include <dds_cpp_infrastructure.hxx>
```

Inheritance diagram for DDSCondition:



Friends

- class [DDSWaitSet](#)

7.93.1 Detailed Description

<<interface>> Root class for all the conditions that may be attached to a [DDSWaitSet](#).

This basic class is specialized in two classes:

[DDSGuardCondition](#) and [DDSStatusCondition](#).

A [DDSCondition](#) has a `trigger_value` that can be [DDS_BOOLEAN_TRUE](#) or [DDS_BOOLEAN_FALSE](#) and is set automatically by RTI Connex DDS Micro.

See also

[DDSWaitSet](#)

7.94 DDSConditionSeq Class Reference

A sequence object containing references to one or more [DDSCondition](#) objects.

```
#include <dds_cpp_infrastructure.hxx>
```

Friends

- class [DDSWaitSet](#)

7.94.1 Detailed Description

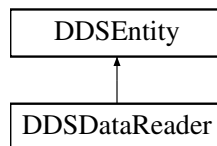
A sequence object containing references to one or more [DDSCondition](#) objects.

7.95 DDSDataReader Class Reference

[<<interface>>](#) Allows the application to: (1) declare the data it wishes to receive (i.e. make a subscription) and (2) access the data received by the attached [DDSSubscriber](#).

```
#include <dds_cpp_subscription.hxx>
```

Inheritance diagram for DDSDataReader:



Public Member Functions

- virtual [DDSEntity](#) * [as_entity](#) ()=0
Accesses the [DDSDataReader](#)'s supertype instance.
- virtual [DDSTopicDescription](#) * [get_topicdescription](#) ()=0
Returns the [DDSTopicDescription](#) associated with the [DDSDataReader](#).
- virtual [DDSSubscriber](#) * [get_subscriber](#) ()=0
Returns the [DDSSubscriber](#) to which the [DDSDataReader](#) belongs.
- virtual [DDS_ReturnCode_t](#) [set_qos](#) (const [DDS_DataReaderQos](#) &qos)=0
Sets the reader QoS.
- virtual [DDS_ReturnCode_t](#) [get_qos](#) ([DDS_DataReaderQos](#) &qos)=0
Gets the reader QoS.
- virtual [DDS_ReturnCode_t](#) [set_listener](#) (const [DDSDataReaderListener](#) *listener, [DDS_StatusMask](#) mask)=0
Sets the reader listener.
- virtual [DDSDataReaderListener](#) * [get_listener](#) ()=0
Get the reader listener.

- virtual `DDS_ReturnCode_t read_untyped` (`DDS_UntypedSampleSeq *received_data`, `DDS_SampleInfoSeq &info_seq`, `DDS_Long max_samples`, `DDS_SampleStateMask sample_states`, `DDS_ViewStateMask view_states`, `DDS_InstanceStateMask instance_states`)=0
Access a collection of data samples from the [DDSDataReader](#).
- virtual `DDS_ReturnCode_t take_untyped` (`DDS_UntypedSampleSeq *received_data`, `DDS_SampleInfoSeq &info_seq`, `DDS_Long max_samples`, `DDS_SampleStateMask sample_states`, `DDS_ViewStateMask view_states`, `DDS_InstanceStateMask instance_states`)=0
Access a collection of data-samples from the [DDSDataReader](#).
- virtual `DDS_ReturnCode_t read_instance_untyped` (`DDS_UntypedSampleSeq *received_data`, `DDS_SampleInfoSeq &info_seq`, `DDS_Long max_samples`, `const DDS_InstanceHandle_t &a_handle`, `DDS_SampleStateMask sample_states`, `DDS_ViewStateMask view_states`, `DDS_InstanceStateMask instance_states`)=0
Access a collection of data samples from the [DDSDataReader](#), all of which belong to the same instance .
- virtual `DDS_ReturnCode_t take_instance_untyped` (`DDS_UntypedSampleSeq *received_data`, `DDS_SampleInfoSeq &info_seq`, `DDS_Long max_samples`, `const DDS_InstanceHandle_t &a_handle`, `DDS_SampleStateMask sample_states`, `DDS_ViewStateMask view_states`, `DDS_InstanceStateMask instance_states`)=0
Access a collection of data samples from the [DDSDataReader](#), all of which belong to the same instance .
- virtual `DDS_ReturnCode_t read_next_sample_untyped` (`void *received_data`, `DDS_SampleInfo &sample_info`)=0
Copies the next not-previously-accessed data value from the [DDSDataReader](#).
- virtual `DDS_ReturnCode_t take_next_sample_untyped` (`void *received_data`, `DDS_SampleInfo &sample_info`)=0
Copies the next not-previously-accessed data value from the [DDSDataReader](#).
- virtual `DDS_InstanceHandle_t lookup_instance_untyped` (`const void *key_holder`)=0
Retrieve the [DDS_InstanceHandle_t](#) identifying a specific instance.
- virtual `DDS_ReturnCode_t is_data_consistent_untyped` (`DDS_Boolean &is_data_consistent`, `const void *sample`, `const DDS_SampleInfo &sample_info`)=0
Checks if the sample has been overwritten by the [DDSDataWriter](#).
- virtual `DDS_ReturnCode_t return_loan_untyped` (`DDS_UntypedSampleSeq *received_data`, `DDS_SampleInfoSeq &info_seq`)=0
Indicates to the [DDSDataReader](#) that the application is done accessing the collection of `received_data` and `info_seq` obtained by some earlier invocation of `read` or `take` on the [DDSDataReader](#).
- virtual `DDS_ReturnCode_t get_subscription_matched_status` (`DDS_SubscriptionMatchedStatus &status`)=0
Accesses the [DDS_SUBSCRIPTION_MATCHED_STATUS](#) communication status.
- virtual `DDS_ReturnCode_t get_liveliness_changed_status` (`DDS_LivelinessChangedStatus &status`)=0
Accesses the [DDS_LIVELINESS_CHANGED_STATUS](#) communication status.
- virtual `DDS_ReturnCode_t get_matched_publications` (`DDS_InstanceHandleSeq &publication_handles`)=0
Retrieve the list of publications currently "associated" with this [DDSDataReader](#).
- virtual `DDS_ReturnCode_t get_matched_publication_data` (`DDS_PublicationBuiltinTopicData &publication_data`, `const DDS_InstanceHandle_t &publication_handle`)=0
This operation retrieves the information on a publication that is currently "associated" with the [DDSDataReader](#).
- virtual `DDS_ReturnCode_t get_sample_rejected_status` (`DDS_SampleRejectedStatus &status`)=0
Accesses the [DDS_SAMPLE_REJECTED_STATUS](#) communication status.
- virtual `DDS_ReturnCode_t get_sample_lost_status` (`DDS_SampleLostStatus &status`)=0
Accesses the [DDS_SAMPLE_LOST_STATUS](#) communication status.
- virtual `DDS_ReturnCode_t get_requested_deadline_missed_status` (`DDS_RequestedDeadlineMissedStatus &status`)=0
Accesses the [DDS_REQUESTED_DEADLINE_MISSED_STATUS](#) communication status.
- virtual `DDS_ReturnCode_t get_requested_incompatible_qos_status` (`DDS_RequestedIncompatibleQosStatus &status`)=0
Accesses the [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#) communication status.
- virtual `DDS_ReturnCode_t get_instance_replaced_missed_status` (`DDS_DataReaderInstanceReplacedStatus &status`)=0
Accesses the [DDS_INSTANCE_REPLACED_STATUS](#) communication status.

Public Member Functions inherited from [DDSEntity](#)

- [DDS_ReturnCode_t enable \(\)](#)
Enables the [DDSEntity](#).
- [DDSStatusCondition * get_statuscondition \(\)](#)
Returns the [DDSStatusCondition](#) associated with a particular [DDSEntity](#).
- [DDS_StatusMask get_status_changes \(\)](#)
Retrieves the list of communication statuses in the [DDSEntity](#) that are triggered.
- [DDS_InstanceHandle_t get_instance_handle \(\)](#)
Allows access to the [DDS_InstanceHandle_t](#) associated with the [DDSEntity](#).

7.95.1 Detailed Description

<<[interface](#)>> Allows the application to: (1) declare the data it wishes to receive (i.e. make a subscription) and (2) access the data received by the attached [DDSSubscriber](#).

QoS:

[DDS_DataReaderQos](#)

Status:

[DDS_DATA_AVAILABLE_STATUS](#);
[DDS_LIVELINESS_CHANGED_STATUS](#), [DDS_LivelinessChangedStatus](#);
[DDS_REQUESTED_DEADLINE_MISSED_STATUS](#), [DDS_RequestedDeadlineMissedStatus](#);
[DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#), [DDS_RequestedIncompatibleQosStatus](#);
[DDS_SAMPLE_LOST_STATUS](#), [DDS_SampleLostStatus](#);
[DDS_SAMPLE_REJECTED_STATUS](#), [DDS_SampleRejectedStatus](#);
[DDS_SUBSCRIPTION_MATCHED_STATUS](#), [DDS_SubscriptionMatchedStatus](#);
[DDS_INSTANCE_REPLACED_STATUS](#), [DDS_DataReaderInstanceReplacedStatus](#);

Listener:

[DDSDDataReaderListener](#)

A [DDSDDataReader](#) refers to exactly one [DDSTopicDescription](#) (a [DDSTopic](#)) that identifies the data to be read.

The subscription has a unique resulting type. The data-reader may give access to several instances of the resulting type, which can be distinguished from each other by their `key`.

The following operations may be called even if the [DDSDDataReader](#) is not enabled. Other operations will fail with the value [DDS_RETCODE_NOT_ENABLED](#) if called on a disabled [DDSDDataReader](#):

- The base-class operations [DDSEntity::enable](#) , [DDSDDataReader::get_qos](#) , [DDSDDataReader::get_listener](#) , [DDSDDataReader::set_qos](#) , and [DDSDDataReader::set_listener](#)

7.95.2 Member Function Documentation

as_entity()

```
virtual DDSEntity * DDSDataReader::as_entity () [pure virtual]
```

Accesses the [DDSDDataReader](#)'s supertype instance.

get_topicdescription()

```
virtual DDSTopicDescription * DDSDataReader::get_topicdescription () [pure virtual]
```

Returns the [DDSTopicDescription](#) associated with the [DDSDDataReader](#).

Returns that same [DDSTopicDescription](#) that was used to create the [DDSDDataReader](#).

Returns

[DDSTopicDescription](#) associated with the [DDSDDataReader](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

get_subscriber()

```
virtual DDSSubscriber * DDSDataReader::get_subscriber () [pure virtual]
```

Returns the [DDSSubscriber](#) to which the [DDSDDataReader](#) belongs.

Returns

[DDSSubscriber](#) to which the [DDSDDataReader](#) belongs.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

set_qos()

```
virtual DDS\_ReturnCode\_t DDSDataReader::set_qos (  
    const DDS\_DataReaderQos & qos) [pure virtual]
```

Sets the reader QoS.

This operation modifies the QoS of the [DDSDDataReader](#).

The [DDS_DataReaderQos::content_filter](#) can be changed. The other policies are immutable.

Parameters

<i>qos</i>	<< <i>in</i> >> The DDS_DataReaderQos to be set to. Policies must be consistent. Policies cannot be changed after DDSDataReader is enabled. The special value DDS_DATAREADER_QOS_DEFAULT can be used to indicate that the QoS of the DDSDataReader should be changed to match the current default DDS_DataReaderQos set in the DDSSubscriber .
------------	--

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_IMMUTABLE_POLICY](#), or [DDS_RETCODE_INCONSISTENT_POLICY](#).

See also

[DDS_DataReaderQos](#) for rules on consistency among QoS
[set_qos](#) (abstract)

get_qos()

```
virtual DDS_ReturnCode_t DDSDataReader::get_qos (
    DDS_DataReaderQos & qos) [pure virtual]
```

Gets the reader QoS.

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

Parameters

<i>qos</i>	<< <i>inout</i> >> The DDS_DataReaderQos to be filled up.
------------	---

Returns

One of the [Standard Return Codes](#)

See also

[get_qos](#) (abstract)

set_listener()

```
virtual DDS_ReturnCode_t DDSDataReader::set_listener (
    const DDSDataReaderListener * listener,
    DDS_StatusMask mask) [pure virtual]
```

Sets the reader listener.

Parameters

<i>listener</i>	<< <i>in</i> >> DDSDataReaderListener to set to
<i>mask</i>	<< <i>in</i> >> DDS_StatusMask associated with the DDSDataReaderListener .

Returns

One of the [Standard Return Codes](#)

See also

[set_listener](#) (abstract)

get_listener()

```
virtual DDSDataReaderListener * DDSDataReader::get_listener () [pure virtual]
```

Get the reader listener.

Returns

[DDSDataReaderListener](#) of the [DDSDataReader](#).

read_untyped()

```
virtual DDS\_ReturnCode\_t DDSDataReader::read_untyped (
    DDSUntypedSampleSeq * received_data,
    DDS\_SampleInfoSeq & info_seq,
    DDS\_Long max_samples,
    DDS\_SampleStateMask sample_states,
    DDS\_ViewStateMask view_states,
    DDS\_InstanceStateMask instance_states) [pure virtual]
```

Access a collection of data samples from the [DDSDataReader](#).

This operation offers the same functionality and API as [DDSDataReader::take_untyped](#) except that the samples returned remain in the [DDSDataReader](#) such that they can be retrieved again by means of a read or take operation.

Please refer to the documentation of [DDSDataReader::take_untyped\(\)](#) for details on the number of samples returned within the `received_data` and `info_seq` as well as the order in which the samples appear in these sequences.

The act of reading a sample changes its `sample_state` to [DDS_READ_SAMPLE_STATE](#). If the sample belongs to the most recent generation of the instance, it will also set the `view_state` of the instance to be [DDS_NOT_NEW_VIEW_STATE](#). It will not affect the `instance_state` of the instance.

Important: If the samples "returned" by this method are loaned from RTI Connex Micro DDS (see [DDSDataReader::take_untyped](#) for more information on memory loaning), it is important that their contents not be changed. Because the memory in which the data is stored belongs to the middleware, any modifications made to the data will be seen the next time the same samples are read or taken; the samples will no longer reflect the state that was received from the network.

Parameters

<i>received_data</i>	<< <i>inout</i> >> User data untyped pointer where the received data samples will be returned. Must be a valid non-NULL pointer. The method will fail with DDS_RETCODE_BAD_PARAMETER if it is NULL.
<i>info_seq</i>	<< <i>inout</i> >> A DDS_SampleInfoSeq object where the received sample info will be returned.
<i>max_samples</i>	<< <i>in</i> >> The maximum number of samples to be returned. If the special value DDS_LENGTH_UNLIMITED is provided, as many samples will be returned as are available.
<i>sample_states</i>	<< <i>in</i> >> Data samples matching one of these <i>sample_states</i> are returned.
<i>view_states</i>	<< <i>in</i> >> Data samples matching one of these <i>view_state</i> are returned.
<i>instance_states</i>	<< <i>in</i> >> Data samples matching ones of these <i>instance_state</i> are returned.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

See also

[DDSDataReader::take_untyped](#),
[DDS_LENGTH_UNLIMITED](#)

take_untyped()

```
virtual DDS\_ReturnCode\_t DDSDataReader::take_untyped (
    DDS\_UntypedSampleSeq * received_data,
    DDS\_SampleInfoSeq & info_seq,
    DDS\_Long max_samples,
    DDS\_SampleStateMask sample_states,
    DDS\_ViewStateMask view_states,
    DDS\_InstanceStateMask instance_states) [pure virtual]
```

Access a collection of data-samples from the [DDSDataReader](#).

The operation will return the list of samples received by the [DDSDataReader](#) since the last [DDSDataReader::take_untyped](#) operation that match the specified [DDS_SampleStateMask](#), [DDS_ViewStateMask](#) and [DDS_InstanceStateMask](#).

This operation may fail with [DDS_RETCODE_ERROR](#) if [DDS_DataReaderResourceLimitsQosPolicy::max_outstanding_reads](#) limit has been exceeded.

The actual number of samples returned depends on the information that has been received by the middleware as well as the [DDS_HistoryQosPolicy](#), [DDS_ResourceLimitsQosPolicy](#), [DDS_DataReaderResourceLimitsQosPolicy](#) and the characteristics of the data-type that is associated with the [DDSDataReader](#):

- In the case where the [DDS_HistoryQosPolicy::kind](#) is [DDS_KEEP_LAST_HISTORY_QOS](#), the call will return at most [DDS_HistoryQosPolicy::depth](#) samples per instance.

- The maximum number of samples returned is limited by `DDS_ResourceLimitsQosPolicy::max_samples`.
- For multiple instances, the number of samples returned is additionally limited by the product (`DDS_ResourceLimitsQosPolicy::max_s`
* `DDS_ResourceLimitsQosPolicy::max_instances`)

If the read or take succeeds and the number of samples returned has been limited (by means of a maximum limit, as listed above, or insufficient `DDS_SampleInfo` resources), the call will complete successfully and provide those samples the reader is able to return. The user may need to make additional calls, or return outstanding loaned buffers in the case of insufficient resources, in order to access remaining samples.

Note that in the case where the `DDSTopic` associated with the `DDSDataReader` is bound to a data-type that has no key definition, then there will be at most one instance in the `DDSDataReader`. So the per-sample limits will apply.

The act of *taking* a sample removes it from RTI Connex DDS Micro so it cannot be read or taken again. If the sample belongs to the most recent generation of the instance, it will also set the `view_state` of the sample's instance to `DDS_NOT_NEW_VIEW_STATE`. It will not affect the `instance_state` of the sample's instance.

After `DDSDataReader::take_untyped` completes, `received_data` and `info_seq` will be of the same length and contain the received data.

If the sequences are empty (maximum size equal 0) when the `DDSDataReader::take_untyped` is called, the samples returned in the `received_data` and the corresponding `info_seq` are 'loaned' to the application from buffers provided by the `DDSDataReader`. The application can use them as desired and has guaranteed exclusive access to them.

Once the application completes its use of the samples it must 'return the loan' to the `DDSDataReader` by calling the `DDSDataReader::return_loan_untyped` operation.

Note: While you must call `DDSDataReader::return_loan_untyped` at some point, you do *not* have to do so before the next `DDSDataReader::take_untyped` call. However, failure to return the loan will eventually deplete the `DDSDataReader` of the buffers it needs to receive new samples and eventually samples will start to be lost. The total number of buffers available to the `DDSDataReader` is specified by the `DDS_ResourceLimitsQosPolicy` and the `DDS_DataReaderResourceLimitsQosPolicy`.

If the sequences are not empty (maximum size not equal 0) when the `DDSDataReader::take_untyped` is called, samples are copied to `received_data` and `info_seq`. The application will not need to call `DDSDataReader::return_loan`.

If the `DDSDataReader` has no samples that meet the constraints, the method will fail with `DDS_RETCODE_NO_DATA`.

In addition to the collection of samples, the read and take operations also use a collection of `DDS_SampleInfo` structures.

7.95.3 SEQUENCES USAGE IN TAKE AND READ

The initial (input) properties of the `received_data` and `info_seq` collections will determine the precise behavior of the read or take operation. For the purposes of this description, the collections are modeled as having these properties:

- whether the collection container owns the memory of the elements within (`owns`, see `::FooSeq::has_ownership`)
- the current-length (`len`, see `::FooSeq::length()`)
- the maximum length (`max_len`, see `::FooSeq::maximum()`)

The initial values of the `owns`, `len` and `max_len` properties for the `received_data` and `info_seq` collections govern the behavior of the read and take operations as specified by the following rules:

1. The values of `owns`, `len` and `max_len` properties for the two collections must be identical. Otherwise read/take will fail with `DDS_RETCODE_PRECONDITION_NOT_MET`.
2. On successful output, the values of `owns`, `len` and `max_len` will be the same for both collections.
3. If the initial `max_len==0`, then the `received_data` and `info_seq` collections will be filled with elements that are loaned by the `DDSDataReader`. On output, `owns` will be `FALSE`, `len` will be set to the number of values returned, and `max_len` will be set to a value verifying `max_len >= len`. The use of this variant allows for zero-copy access to the data and the application will need to return the loan to the `DDSDataReader` using `FooDataReader::return_loan`.
4. If the initial `max_len>0` and `owns==FALSE`, then the read or take operation will fail with `DDS_RETCODE_PRECONDITION_NOT_MET`. This avoids the potential hard-to-detect memory leaks caused by an application forgetting to return the loan.
5. If initial `max_len>0` and `owns==TRUE`, then the read or take operation will copy the `received_data` values and `DDS_SampleInfo` values into the elements already inside the collections. On output, `owns` will be `TRUE`, `len` will be set to the number of values copied and `max_len` will remain unchanged. The use of this variant forces a copy but the application can control where the copy is placed and the application will not need to return the loan. The number of samples copied depends on the relative values of `max_len` and `max_samples`:
 - If `max_samples == LENGTH_UNLIMITED`, then at most `max_len` values will be copied. The use of this variant lets the application limit the number of samples returned to what the sequence can accommodate.
 - If `max_samples <= max_len`, then at most `max_samples` values will be copied. The use of this variant lets the application limit the number of samples returned to fewer than what the sequence can accommodate.
 - If `max_samples > max_len`, then the read or take operation will fail with `DDS_RETCODE_PRECONDITION_NOT_MET`. This avoids the potential confusion where the application expects to be able to access up to `max_samples`, but that number can never be returned, even if they are available in the `DDSDataReader`, because the output sequence cannot accommodate them.

As described above, upon completion, the `received_data` and `info_seq` collections may contain elements loaned from the `DDSDataReader`. If this is the case, the application will need to use `DDSDataReader::return_loan_untyped` to return the loan once it is no longer using the `received_data` in the collection. When `DDSDataReader::return_loan_untyped` completes, the collection will have `owns=FALSE` and `max_len=0`. The application can determine whether it is necessary to return the loan or not based on how the state of the collections when the read/take operation was called or by accessing the `owns` property. However, in many cases it may be simpler to always call `DDSDataReader::return_loan_untyped`, as this operation is harmless (i.e., it leaves all elements unchanged) if the collection does not have a loan.

To avoid potential memory leaks, the implementation of the `Foo` and `DDS_SampleInfo` collections should disallow changing the length of a collection for which `owns==FALSE`. Furthermore, deleting a collection for which `owns==FALSE` should be considered an error.

On output, the collection of `Foo` values and the collection of `DDS_SampleInfo` structures are of the same length and are in a one-to-one correspondence. Each `DDS_SampleInfo` provides information, such as the `source_timestamp`, the `sample_state`, `view_state`, and `instance_state`, etc., about the corresponding sample.

Some elements in the returned collection may not have valid data. If the `instance_state` in the `DDS_SampleInfo` is `DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE` or `DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE`, then the last sample for that instance in the collection (that is, the one whose `DDS_SampleInfo` has `sample_rank==0`) does not contain valid data.

Samples that contain no data do not count towards the limits imposed by the [DDS_ResourceLimitsQosPolicy](#). The act of reading/taking a sample sets its `sample_state` to [DDS_READ_SAMPLE_STATE](#).

If the sample belongs to the most recent generation of the instance, it will also set the `view_state` of the instance to [DDS_NOT_NEW_VIEW_STATE](#). It will not affect the `instance_state` of the instance.

This operation must be provided on the specialized class that is generated for the particular application data-type that is being read . If the [DDSDataReader](#) has no samples that meet the constraints, the operations fails with [DDS_RETCODE_NO_DATA](#).

Parameters

<i>received_data</i>	<< <i>inout</i> >> User data untyped pointer where the received data samples will be returned.
----------------------	--

Parameters

<i>info_seq</i>	<< <i>inout</i> >> A DDS_SampleInfoSeq object where the received sample info will be returned.
<i>max_samples</i>	<< <i>in</i> >> The maximum number of samples to be returned. If the special value DDS_LENGTH_UNLIMITED is provided, as many samples will be returned as are available.
<i>sample_states</i>	<< <i>in</i> >> Data samples matching one of these <code>sample_states</code> are returned.
<i>view_states</i>	<< <i>in</i> >> Data samples matching one of these <code>view_state</code> are returned.
<i>instance_states</i>	<< <i>in</i> >> Data samples matching one of these <code>instance_state</code> are returned.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

See also

[DDSDataReader::read_untyped](#)

read_instance_untyped()

```
virtual DDS_ReturnCode_t DDSDataReader::read_instance_untyped (
    DDS_UntypedSampleSeq * received_data,
    DDS_SampleInfoSeq & info_seq,
    DDS_Long max_samples,
    const DDS_InstanceHandle_t & a_handle,
    DDS_SampleStateMask sample_states,
    DDS_ViewStateMask view_states,
    DDS_InstanceStateMask instance_states) [pure virtual]
```

Access a collection of data samples from the [DDSDataReader](#), all of which belong to the same instance .

Parameters

<i>received_data</i>	<< <i>inout</i> >> User data untyped pointer where the received data samples will be returned. Must be a valid non-NULL pointer. The method will fail with DDS_RETCODE_BAD_PARAMETER if it is NULL.
<i>info_seq</i>	<< <i>inout</i> >> A DDS_SampleInfoSeq object where the received sample info will be returned.
<i>max_samples</i>	<< <i>in</i> >> The maximum number of samples to be returned. If the special value DDS_LENGTH_UNLIMITED is provided, as many samples will be returned as are available.
<i>a_handle</i>	<< <i>in</i> >> Instance for which data samples should be read
<i>sample_states</i>	<< <i>in</i> >> Data samples matching one of these <i>sample_states</i> are returned.
<i>view_states</i>	<< <i>in</i> >> Data samples matching one of these <i>view_state</i> are returned.
<i>instance_states</i>	<< <i>in</i> >> Data samples matching ones of these <i>instance_state</i> are returned.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

See also

[DDSDataReader::read_untyped](#),
[DDS_LENGTH_UNLIMITED](#)

take_instance_untyped()

```
virtual DDS\_ReturnCode\_t DDSDataReader::take_instance_untyped (
    DDSUntypedSampleSeq * received_data,
    DDS\_SampleInfoSeq & info_seq,
    DDS\_Long max_samples,
    const DDS\_InstanceHandle\_t & a_handle,
    DDS\_SampleStateMask sample_states,
    DDS\_ViewStateMask view_states,
    DDS\_InstanceStateMask instance_states) [pure virtual]
```

Access a collection of data samples from the [DDSDataReader](#), all of which belong to the same instance .

Parameters

<i>received_data</i>	<< <i>inout</i> >> User data untyped pointer where the received data samples will be returned. Must be a valid non-NULL pointer. The method will fail with DDS_RETCODE_BAD_PARAMETER if it is NULL.
<i>info_seq</i>	<< <i>inout</i> >> A DDS_SampleInfoSeq object where the received sample info will be returned.
<i>max_samples</i>	<< <i>in</i> >> The maximum number of samples to be returned. If the special value DDS_LENGTH_UNLIMITED is provided, as many samples will be returned as are available.
<i>a_handle</i>	<< <i>in</i> >> Instance for which data samples should be read
<i>sample_states</i>	<< <i>in</i> >> Data samples matching one of these <i>sample_states</i> are returned.
<i>view_states</i>	<< <i>in</i> >> Data samples matching one of these <i>view_state</i> are returned.
<i>instance_states</i>	<< <i>in</i> >> Data samples matching ones of these <i>instance_state</i> are returned.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

See also

[DDSDataReader::take_untyped](#),
[DDS_LENGTH_UNLIMITED](#)

read_next_sample_untyped()

```
virtual DDS_ReturnCode_t DDSDataReader::read_next_sample_untyped (
    void * received_data,
    DDS_SampleInfo & sample_info) [pure virtual]
```

Copies the next not-previously-accessed data value from the [DDSDataReader](#).

This operation copies the next not-previously-accessed data value from the [DDSDataReader](#). This operation also copies the corresponding [DDS_SampleInfo](#). The implied order among the samples stored in the [DDSDataReader](#) is the same as for the [DDSDataReader::read_untyped](#) operation.

The [DDSDataReader::read_next_sample_untyped](#) operation is semantically equivalent to the [DDSDataReader::read_untyped](#) operation, where the input data sequences has max_len=1, the sample_states=NOT_READ, the view_states=ANY_↔VIEW_STATE, and the instance_states=ANY_INSTANCE_STATE.

The [DDSDataReader::read_next_sample_untyped](#) operation provides a simplified API to 'read' samples, avoiding the need for the application to manage sequences and specify states.

If there is no unread data in the [DDSDataReader](#), the operation will fail with [DDS_RETCODE_NO_DATA](#) and nothing is copied.

Parameters

<i>received_data</i>	<< <i>inout</i> >> user data type-specific data object where the next received data sample will be returned. The received_data must have been fully allocated. Otherwise, this operation may fail.
<i>sample_info</i>	<< <i>inout</i> >> a DDS_SampleInfo object where the next received sample info will be returned.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

See also

[DDSDataReader::read_untyped](#)

take_next_sample_untyped()

```
virtual DDS_ReturnCode_t DDSDataReader::take_next_sample_untyped (
    void * received_data,
    DDS_SampleInfo & sample_info) [pure virtual]
```

Copies the next not-previously-accessed data value from the [DDSDDataReader](#).

This operation copies the next not-previously-accessed data value from the [DDSDDataReader](#) and 'removes' it from the [DDSDDataReader](#) so that it is no longer accessible. This operation also copies the corresponding [DDS_SampleInfo](#). This operation is analogous to the [DDSDDataReader::read_next_sample_untyped](#) except for the fact that the sample is removed from the [DDSDDataReader](#).

The [DDSDDataReader::take_next_sample_untyped](#) operation is semantically equivalent to the [DDSDDataReader::take_untyped](#) operation, where the input data sequences has max_len=1, the sample_states=NOT_READ, the view_states=ANY_↔VIEW_STATE, and the instance_states=ANY_INSTANCE_STATE.

The [DDSDDataReader::take_next_sample_untyped](#) operation provides a simplified API to 'take' samples, avoiding the need for the application to manage sequences and specify states.

If there is no unread data in the [DDSDDataReader](#), the operation will fail with [DDS_RETCODE_NO_DATA](#) and nothing is copied.

Parameters

<i>received_data</i>	<<inout>> user data type-specific Data object where the next received data sample will be returned. The received_data must have been fully allocated. Otherwise, this operation may fail.
<i>sample_info</i>	<<inout>> a DDS_SampleInfo object where the next received sample info will be returned.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

See also

[DDSDDataReader::take_untyped](#)

lookup_instance_untyped()

```
virtual DDS_InstanceHandle_t DDSDataReader::lookup_instance_untyped (
    const void * key_holder) [pure virtual]
```

Retrieve the [DDS_InstanceHandle_t](#) identifying a specific instance.

Parameters

<i>key_holder</i>	<<in>> A data sample containing the desired values for key attributes.
-------------------	--

Returns

The [DDS_InstanceHandle_t](#) for the selected instance or [DDS_HANDLE_NIL](#) if the instance was not found or an error occurred.

is_data_consistent_untyped()

```
virtual DDS_ReturnCode_t DDSDDataReader::is_data_consistent_untyped (
    DDS_Boolean & is_data_consistent,
    const void * sample,
    const DDS_SampleInfo & sample_info) [pure virtual]
```

Checks if the sample has been overwritten by the [DDSDDataWriter](#).

When a sample is received via Zero Copy transfer over shared memory, the sample can be reused by the [DDSDDataWriter](#) once it is removed from the DataWriter's send queue. Since there is no synchronization between the [DDSDDataReader](#) and the [DDSDDataWriter](#), the sample could be overwritten by the [DDSDDataWriter](#) before it is processed by the [DDSDDataReader](#). The [FooDataReader::is_data_consistent](#) operation can be used after processing the sample to check if it was overwritten by the [DDSDDataWriter](#).

A precondition for using this operation is to set [DDS_DataWriterShmemRefTransferModeSettings::enable_data_consistency_check](#) to true.

Parameters

<i>is_data_consistent</i>	<< <i>inout</i> >> Set to true if the sample is consistent (i.e., the sample has not been overwritten by the DataWriter)
<i>sample</i>	<< <i>in</i> >> Sample to be validated
<i>sample_info</i>	<< <i>in</i> >> DDS_SampleInfo object received with the sample

Returns

One of the [Standard Return Codes](#) or [DDS_RETCODE_PRECONDITION_NOT_MET](#).

return_loan_untyped()

```
virtual DDS_ReturnCode_t DDSDDataReader::return_loan_untyped (
    DDS_UntypedSampleSeq * received_data,
    DDS_SampleInfoSeq & info_seq) [pure virtual]
```

Indicates to the [DDSDDataReader](#) that the application is done accessing the collection of `received_data` and `info_seq` obtained by some earlier invocation of read or take on the [DDSDDataReader](#).

This operation indicates to the [DDSDDataReader](#) that the application is done accessing the collection of `received_data` and `info_seq` obtained by some earlier invocation of read or take on the [DDSDDataReader](#).

The `received_data` and `info_seq` must belong to a single related "pair"; that is, they should correspond to a pair returned from a single call to read or take. The `received_data` and `info_seq` must also have been obtained from the same [DDSDDataReader](#) to which they are returned. If either of these conditions is not met, the operation will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#).

The operation [DDSDDataReader::return_loan_untyped](#) allows implementations of the read and take operations to "loan" buffers from the [DDSDDataReader](#) to the application and in this manner provide "zerocopy" access to the data. During the loan, the [DDSDDataReader](#) will guarantee that the data and sample-information are not modified.

It is not necessary for an application to return the loans immediately after the read or take calls. However, as these buffers correspond to internal resources inside the [DDSDataReader](#), the application should not retain them indefinitely.

The use of [DDSDataReader::return_loan_untyped](#) is only necessary if the read or take calls "loaned" buffers to the application. This only occurs if the `received_data` and `info_seq` collections had `max_len=0` at the time read or take was called.

The application may also examine the "owns" property of the collection to determine where there is an outstanding loan. However, calling [DDSDataReader::return_loan_untyped](#) on a collection that does not have a loan is safe and has no side effects.

If the collections had a loan, upon completion of [DDSDataReader::return_loan_untyped](#), the collections will have `max_len=0`.

Parameters

<i>received_data</i>	<<in>> user data type-specific FooSeq object where the received data samples was obtained from earlier invocation of read or take on the DDSDataReader .
----------------------	--

Parameters

<i>info_seq</i>	<<in>> a DDS_SampleInfoSeq object where the received sample info was obtained from earlier invocation of read or take on the DDSDataReader .
-----------------	--

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#) or [DDS_RETCODE_NOT_ENABLED](#).

get_subscription_matched_status()

```
virtual DDS_ReturnCode_t DDSDataReader::get_subscription_matched_status (
    DDS_SubscriptionMatchedStatus & status) [pure virtual]
```

Accesses the [DDS_SUBSCRIPTION_MATCHED_STATUS](#) communication status.

Parameters

<i>status</i>	<<inout>> DDS_SubscriptionMatchedStatus to be filled in.
---------------	--

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

get_liveliness_changed_status()

```
virtual DDS_ReturnCode_t DDSDataReader::get_liveliness_changed_status (
    DDS_LivelinessChangedStatus & status) [pure virtual]
```

Accesses the [DDS_LIVELINESS_CHANGED_STATUS](#) communication status.

Parameters

<i>status</i>	<<inout>> DDS_LivelinessChangedStatus to be filled in.
---------------	--

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

get_matched_publications()

```
virtual DDS_ReturnCode_t DDSDataReader::get_matched_publications (
    DDS_InstanceHandleSeq & publication_handles) [pure virtual]
```

Retrieve the list of publications currently "associated" with this [DDSDataReader](#).

Matching publications are those in the same domain that have a matching [DDSTopic](#), and compatible QoS common partition.

The handles returned in the `publication_handles` list are the ones that are used by the DDS implementation to locally identify the corresponding matched [DDSDataWriter](#) entities. These handles match the ones that appear in the `instance_handle` field of the [DDS_SampleInfo](#) when reading the DCPSPublication built-in topic.

Parameters

<i>publication_handles</i>	<<inout>>
----------------------------	-----------

The sequence will be grown if the sequence has ownership and the system has the corresponding resources. Use a sequence without ownership to avoid dynamic memory allocation. If the sequence is too small to store all the matches and the system can not resize the sequence, this method will fail with [DDS_RETCODE_OUT_OF_RESOURCES](#).

The maximum number of matches possible is configured with [DDS_DomainParticipantResourceLimitsQosPolicy](#). You can use a zero-maximum sequence without ownership to quickly check whether there are any matches without allocating any memory. .

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_OUT_OF_RESOURCES](#) if the sequence is too small and the system can not resize it, or [DDS_RETCODE_NOT_ENABLED](#)

get_matched_publication_data()

```
virtual DDS_ReturnCode_t DDSDataReader::get_matched_publication_data (
    DDS_PublicationBuiltinTopicData & publication_data,
    const DDS_InstanceHandle_t & publication_handle) [pure virtual]
```

This operation retrieves the information on a publication that is currently "associated" with the [DDSDataReader](#).

Publication with a matching [DDSTopic](#), compatible QoS and common partition.

The `publication_handle` must correspond to a publication currently associated with the [DDSDataReader](#). Otherwise, the operation will fail with [DDS_RETCODE_BAD_PARAMETER](#). Use the operation [DDSDataReader::get_matched_publications](#) to find the publications that are currently matched with the [DDSDataReader](#).

Parameters

<i>publication_data</i>	<<inout>>. The information to be filled in on the associated publication. Cannot be NULL.
<i>publication_handle</i>	<<in>>. Handle to a specific publication associated with the DDSDataReader . . Must correspond to a publication currently associated with the DDSDataReader .

Returns

One of the [Standard Return Codes](#) or [DDS_RETCODE_NOT_ENABLED](#)

get_sample_rejected_status()

```
virtual DDS_ReturnCode_t DDSDataReader::get_sample_rejected_status (
    DDS_SampleRejectedStatus & status) [pure virtual]
```

Accesses the [DDS_SAMPLE_REJECTED_STATUS](#) communication status.

Parameters

<i>status</i>	<<inout>> DDS_SampleRejectedStatus to be filled in.
---------------	---

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

get_sample_lost_status()

```
virtual DDS_ReturnCode_t DDSDataReader::get_sample_lost_status (
    DDS_SampleLostStatus & status) [pure virtual]
```

Accesses the [DDS_SAMPLE_LOST_STATUS](#) communication status.

Parameters

<i>status</i>	<<inout>> DDS_SampleLostStatus to be filled in.
---------------	---

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

get_requested_deadline_missed_status()

```
virtual DDS_ReturnCode_t DDSDataReader::get_requested_deadline_missed_status (
    DDS_RequestedDeadlineMissedStatus & status) [pure virtual]
```

Accesses the [DDS_REQUESTED_DEADLINE_MISSED_STATUS](#) communication status.

Parameters

<i>status</i>	<<inout>> DDS_RequestedDeadlineMissedStatus to be filled in.
---------------	--

Returns

One of the [Standard Return Codes](#)

get_requested_incompatible_qos_status()

```
virtual DDS_ReturnCode_t DDSDataReader::get_requested_incompatible_qos_status (
    DDS_RequestedIncompatibleQosStatus & status) [pure virtual]
```

Accesses the [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#) communication status.

Parameters

<i>status</i>	<<inout>> DDS_RequestedIncompatibleQosStatus to be filled in.
---------------	---

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

get_instance_replaced_missed_status()

```
virtual DDS_ReturnCode_t DDSDataReader::get_instance_replaced_missed_status (
    DDS_DataReaderInstanceReplacedStatus & status) [pure virtual]
```

Accesses the [DDS_INSTANCE_REPLACED_STATUS](#) communication status.

Parameters

<i>status</i>	<<inout>> DDS_DataReaderInstanceReplacedStatus to be filled in.
---------------	---

Returns

One of the [Standard Return Codes](#)

MT Safety:

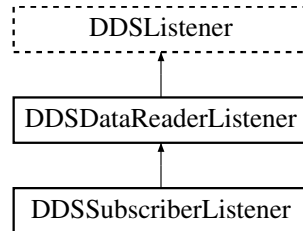
This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.96 DDSDataReaderListener Class Reference

<<interface>> [DDSListener](#) for reader status.

```
#include <dds_cpp_subscription.hxx>
```

Inheritance diagram for DDSDataReaderListener:



Public Member Functions

- virtual void [on_data_available](#) (DDSDataReader *)
Handles the [DDS_DATA_AVAILABLE_STATUS](#) communication status.
- virtual void [on_requested_deadline_missed](#) (DDSDataReader *, const [DDS_RequestedDeadlineMissedStatus](#) &)
Handles the [DDS_REQUESTED_DEADLINE_MISSED_STATUS](#) communication status.
- virtual void [on_liveliness_changed](#) (DDSDataReader *, const [DDS_LivelinessChangedStatus](#) &)
Handles the [DDS_LIVELINESS_CHANGED_STATUS](#) communication status.
- virtual void [on_requested_incompatible_qos](#) (DDSDataReader *, const [DDS_RequestedIncompatibleQosStatus](#) &)
Handles the [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#) communication status.
- virtual void [on_sample_rejected](#) (DDSDataReader *, const [DDS_SampleRejectedStatus](#) &)
Handles the [DDS_SAMPLE_REJECTED_STATUS](#) communication status.
- virtual void [on_subscription_matched](#) (DDSDataReader *, const [DDS_SubscriptionMatchedStatus](#) &)
Handles the [DDS_SUBSCRIPTION_MATCHED_STATUS](#) communication status.
- virtual void [on_sample_lost](#) (DDSDataReader *, const [DDS_SampleLostStatus](#) &)
Handles the [DDS_SAMPLE_LOST_STATUS](#) communication status.
- virtual void [on_instance_replaced](#) (DDSDataReader *, const [DDS_DataReaderInstanceReplacedStatus](#) &)
Handles the [DDS_INSTANCE_REPLACED_STATUS](#) communication status.
- virtual [DDS_Boolean](#) [on_before_sample_deserialize](#) (DDSDataReader *, [NDDS_Type_Plugin](#) *, [CDR_Stream_t](#) *, [DDS_Boolean](#) *)
Callback to filter a received sample based on serialized data
- virtual [DDS_Boolean](#) [on_before_sample_commit](#) (DDSDataReader *, const void *const, const struct [DDS_SampleInfo](#) *const, [DDS_Boolean](#) *)
Callback to filter a received sample based on deserialized data

7.96.1 Detailed Description

<<[*interface*](#)>> [DDSListener](#) for reader status.

Entity:

[DDSDDataReader](#)

Status:

[DDS_DATA_AVAILABLE_STATUS](#);
[DDS_LIVELINESS_CHANGED_STATUS](#), [DDS_LivelinessChangedStatus](#);
[DDS_REQUESTED_DEADLINE_MISSED_STATUS](#), [DDS_RequestedDeadlineMissedStatus](#);
[DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#), [DDS_RequestedIncompatibleQosStatus](#);
[DDS_SAMPLE_LOST_STATUS](#), [DDS_SampleLostStatus](#);
[DDS_SAMPLE_REJECTED_STATUS](#), [DDS_SampleRejectedStatus](#);
[DDS_SUBSCRIPTION_MATCHED_STATUS](#), [DDS_SubscriptionMatchedStatus](#);
[DDS_INSTANCE_REPLACED_STATUS](#), [DDS_DataReaderInstanceReplacedStatus](#);

See also

[Status Kinds](#)

7.96.2 Member Function Documentation

on_data_available()

```
virtual void DDSDataReaderListener::on_data_available (
    DDSDDataReader * ) [inline], [virtual]
```

Handles the [DDS_DATA_AVAILABLE_STATUS](#) communication status.

on_requested_deadline_missed()

```
virtual void DDSDataReaderListener::on_requested_deadline_missed (
    DDSDDataReader * ,
    const DDS\_RequestedDeadlineMissedStatus & ) [inline], [virtual]
```

Handles the [DDS_REQUESTED_DEADLINE_MISSED_STATUS](#) communication status.

on_liveliness_changed()

```
virtual void DDSDataReaderListener::on_liveliness_changed (
    DDSDDataReader * ,
    const DDS\_LivelinessChangedStatus & ) [inline], [virtual]
```

Handles the [DDS_LIVELINESS_CHANGED_STATUS](#) communication status.

on_requested_incompatible_qos()

```
virtual void DDSDataReaderListener::on_requested_incompatible_qos (
    DDSDataReader * ,
    const DDS_RequestedIncompatibleQosStatus & ) [inline], [virtual]
```

Handles the [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#) communication status.

on_sample_rejected()

```
virtual void DDSDataReaderListener::on_sample_rejected (
    DDSDataReader * ,
    const DDS_SampleRejectedStatus & ) [inline], [virtual]
```

Handles the [DDS_SAMPLE_REJECTED_STATUS](#) communication status.

on_subscription_matched()

```
virtual void DDSDataReaderListener::on_subscription_matched (
    DDSDataReader * ,
    const DDS_SubscriptionMatchedStatus & ) [inline], [virtual]
```

Handles the [DDS_SUBSCRIPTION_MATCHED_STATUS](#) communication status.

on_sample_lost()

```
virtual void DDSDataReaderListener::on_sample_lost (
    DDSDataReader * ,
    const DDS_SampleLostStatus & ) [inline], [virtual]
```

Handles the [DDS_SAMPLE_LOST_STATUS](#) communication status.

on_instance_replaced()

```
virtual void DDSDataReaderListener::on_instance_replaced (
    DDSDataReader * ,
    const DDS_DataReaderInstanceReplacedStatus & ) [inline], [virtual]
```

Handles the [DDS_INSTANCE_REPLACED_STATUS](#) communication status.

This listener may only be called when the instance replacement policy is set to [DDS_REPLACE_OLDEST_INSTANCE](#) ↔ [_REPLACEMENT_QOS](#). When a new instance is detected and resources are exhausted the oldest instance will be removed even if samples with a not read state exists for the instance. Thus, this is a forceful removal of samples and the instance. The freed instance resources will be re-used for the newly detected instance. If there are outstanding loans for the replaced instance the samples can not be freed until returned. However, when the loan is returned the samples are automatically freed. Note that the instance will not be rejected.

on_before_sample_deserialize()

```
virtual DDS_Boolean DDSDataReaderListener::on_before_sample_deserialize (
    DDSDataReader * ,
    NDDS_Type_Plugin * ,
    CDR_Stream_t * ,
    DDS_Boolean * ) [inline], [virtual]
```

Callback to filter a received sample based on serialized data

When a [DDSDataReader](#) receives a (serialized) sample, it will deserialize it before storing it in its queue. `On_before_sample_deserialize()` is called before deserialization. It allows the application to determine whether or not to drop the sample, before it is read or taken.

The callback controls whether the sample is filtered out of the DataReader's queue, with the `sample_dropped` parameter. If user code within the callback determines that the sample should be dropped, the user should set `sample_dropped` to true; consequently, when the callback returns, the [DDSDataReader](#) will drop the sample before it is committed to the queue. Otherwise, returning the callback with `sample_dropped` as false will allow the sample to be read or taken.

Compared to `on_before_sample_commit()`, the sample would be dropped with less processing (without deserialization) by the subscribing application, but with the added complexity of filtering on serialized data.

The callback is not associated with a DDS Status. It is enabled when the callback in the listener is assigned, to a non-NULL callback.

on_before_sample_commit()

```
virtual DDS_Boolean DDSDataReaderListener::on_before_sample_commit (
    DDSDataReader * ,
    const void * const ,
    const struct DDS_SampleInfo * const ,
    DDS_Boolean * ) [inline], [virtual]
```

Callback to filter a received sample based on deserialized data

When a [DDSDataReader](#) receives a (serialized) sample, it will deserialize it before storing it in its queue. `On_before_sample_commit()` is called after deserialization, but before the sample is stored into the DataReader's queue. It allows the application to determine whether or not to drop the sample, before it is read or taken.

The callback controls whether the sample is filtered out of the DataReader's queue, with the `sample_dropped` parameter. If user code within the callback determines that the sample should be dropped, the user should set `sample_dropped` to true; consequently, when the callback returns, the [DDSDataReader](#) will drop the sample before it is committed to the queue. Otherwise, returning the callback with `sample_dropped` as false will allow the sample to be read or taken.

Compared to `on_before_sample_deserialize()`, the sample would be dropped with more processing (with deserialization) by the subscribing application, but with the lesser complexity of filtering on deserialized data.

The callback is not associated with a DDS Status. It is enabled when the callback in the listener is assigned, to a non-NULL callback.

The following fields are valid in the [DDS_SampleInfo](#) structure:

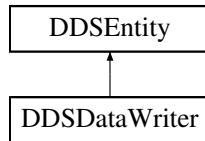
- `source_timestamp`
- `instance_handle`
- `publication_handle`
- `valid_data`
- `encapsulation_id`

7.97 DDSDataWriter Class Reference

<<interface>> Allows an application to set the value of the data to be published under a given [DDSTopic](#).

```
#include <dds_cpp_publication.hxx>
```

Inheritance diagram for DDSDataWriter:



Public Member Functions

- virtual [DDS_ReturnCode_t](#) [write_untyped](#) (const void *instance_data, const [DDS_InstanceHandle_t](#) &handle)=0
Modifies the value of a data instance.
- virtual [DDS_ReturnCode_t](#) [assert_liveliness](#) ()=0
This operation manually asserts the liveliness of this [DDSDataWriter](#).
- virtual [DDSTopic](#) * [get_topic](#) ()=0
This operation returns the [DDSTopic](#) associated with the [DDSDataWriter](#).
- virtual [DDSPublisher](#) * [get_publisher](#) ()=0
This operation returns the [DDSPublisher](#) to which the [DDSDataWriter](#) belongs.
- virtual [DDS_ReturnCode_t](#) [get_qos](#) ([DDS_DataWriterQos](#) &qos)=0
Gets the writer QoS.
- virtual [DDS_ReturnCode_t](#) [set_qos](#) (const [DDS_DataWriterQos](#) &qos)=0
Sets the writer QoS.
- virtual [DDS_ReturnCode_t](#) [set_listener](#) (const [DDSDataWriterListener](#) *listener, [DDS_StatusMask](#) mask)=0
Sets the writer listener.
- virtual [DDSDataWriterListener](#) * [get_listener](#) ()=0
Get the writer listener.
- virtual [DDS_InstanceHandle_t](#) [register_instance_untyped](#) (const void *instance_data)=0
Informs RTI Connext DDS Micro that the application will be modifying a particular instance.
- virtual [DDS_InstanceHandle_t](#) [register_instance_w_timestamp_untyped](#) (const void *instance_data, const [DDS_Time_t](#) &source_timestamp)=0
Performs the same function as [DDSDataWriter::register_instance_untyped](#) except that it also provides the value for the source_timestamp.
- virtual [DDS_ReturnCode_t](#) [unregister_instance_untyped](#) (const void *instance_data, const [DDS_InstanceHandle_t](#) &handle)=0
Reverses the action of [DDSDataWriter::register_instance_untyped](#).
- virtual [DDS_ReturnCode_t](#) [unregister_instance_w_timestamp_untyped](#) (const void *instance_data, const [DDS_InstanceHandle_t](#) &handle, const [DDS_Time_t](#) &source_timestamp)=0
Performs the same function as [DDSDataWriter::unregister_instance_untyped](#) except that it also provides the value for the source_timestamp.
- virtual [DDS_ReturnCode_t](#) [dispose_untyped](#) (const void *instance_data, const [DDS_InstanceHandle_t](#) &handle)=0
Requests the middleware to delete the data.

- virtual [DDS_ReturnCode_t dispose_w_timestamp_untyped](#) (const void *instance_data, const [DDS_InstanceHandle_t](#) &handle, const [DDS_Time_t](#) &source_timestamp)=0
Performs the same function as [DDSDataWriter::dispose_untyped](#) except that it also provides the value for the source_timestamp.
- virtual [DDS_ReturnCode_t write_w_timestamp_untyped](#) (const void *instance_data, const [DDS_InstanceHandle_t](#) &handle, const [DDS_Time_t](#) &source_timestamp)=0
Performs the same function as [DDSDataWriter::write_untyped](#) except that it also provides the value for the source_timestamp.
- virtual [DDS_ReturnCode_t get_publication_matched_status](#) ([DDS_PublicationMatchedStatus](#) &status)=0
Accesses the [DDS_PUBLICATION_MATCHED_STATUS](#) communication status.
- virtual [DDS_ReturnCode_t get_liveliness_lost_status](#) ([DDS_LivelinessLostStatus](#) &status)=0
Accesses the [DDS_LIVELINESS_LOST_STATUS](#) communication status.
- virtual [DDS_ReturnCode_t get_matched_subscriptions](#) ([DDS_InstanceHandleSeq](#) &subscription_handles)=0
Returns a sequence containing [DDS_InstanceHandle_t](#) for all the remote [DDSDataReader](#) that have been matched by the [DDSDataWriter](#).
- virtual [DDS_ReturnCode_t get_matched_subscription_data](#) ([DDS_SubscriptionBuiltinTopicData](#) &subscription_data, const [DDS_InstanceHandle_t](#) &subscription_handle)=0
Returns information about a remote [DDSDataReader](#) that has been matched by the [DDSDataWriter](#).
- virtual [DDS_ReturnCode_t get_offered_deadline_missed_status](#) ([DDS_OfferedDeadlineMissedStatus](#) &status)=0
Accesses the [DDS_OFFERED_DEADLINE_MISSED_STATUS](#) communication status.
- virtual [DDS_ReturnCode_t get_offered_incompatible_qos_status](#) ([DDS_OfferedIncompatibleQosStatus](#) &status)=0
Accesses the [DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#) communication status.

Public Member Functions inherited from [DDSEntity](#)

- [DDS_ReturnCode_t enable](#) ()
Enables the [DDSEntity](#).
- [DDSStatusCondition * get_statuscondition](#) ()
Returns the [DDSStatusCondition](#) associated with a particular [DDSEntity](#).
- [DDS_StatusMask get_status_changes](#) ()
Retrieves the list of communication statuses in the [DDSEntity](#) that are triggered.
- [DDS_InstanceHandle_t get_instance_handle](#) ()
Allows access to the [DDS_InstanceHandle_t](#) associated with the [DDSEntity](#).

7.97.1 Detailed Description

<<[interface](#)>> Allows an application to set the value of the data to be published under a given [DDSTopic](#).

QoS:

[DDS_DataWriterQos](#)

Status:

```
DDS_LIVELINESS_LOST_STATUS, DDS_LivelinessLostStatus;
DDS_OFFERED_DEADLINE_MISSED_STATUS, DDS_OfferedDeadlineMissedStatus;
DDS_OFFERED_INCOMPATIBLE_QOS_STATUS, DDS_OfferedIncompatibleQosStatus;
DDS_PUBLICATION_MATCHED_STATUS, DDS_PublicationMatchedStatus;
```

Listener:

```
DDSDataWriterListener
```

A [DDSDataWriter](#) is attached to exactly one [DDSPublisher](#), that acts as a factory for it.

A [DDSDataWriter](#) is bound to exactly one [DDSTopic](#) and therefore to exactly one data type. The [DDSTopic](#) must exist prior to the [DDSDataWriter](#)'s creation.

[DDSDataWriter](#) is an abstract class. It must be specialized for each particular application data-type. The additional methods or functions that must be defined in the auto-generated class for a hypothetical application type `Foo` are specified in the example type [DDSDataWriter](#).

The following operations may be called even if the [DDSDataWriter](#) is not enabled. Other operations will fail with [DDS_RETCODE_NOT_ENABLED](#) if called on a disabled [DDSDataWriter](#):

- The base-class operations [DDSEntity::enable](#), [DDSDataWriter::get_qos](#), [DDSDataWriter::get_listener](#), and [DDSDataWriter::set_qos](#), [DDSDataWriter::set_listener](#)
- [DDSDataWriter::get_liveliness_lost_status](#), [DDSDataWriter::get_offered_deadline_missed_status](#), [DDSDataWriter::get_offered_incompatible_qos_status](#), and [DDSDataWriter::get_publication_matched_status](#)

Several [DDSDataWriter](#) may operate in different threads. If they share the same [DDSPublisher](#), the middleware guarantees that its operations are thread-safe.

7.97.2 Member Function Documentation

write_untyped()

```
virtual DDS_ReturnCode_t DDSDataWriter::write_untyped (
    const void * instance_data,
    const DDS_InstanceHandle_t & handle) [pure virtual]
```

Modifies the value of a data instance.

When this operation is used, RTI Connex Micro will automatically supply the value of the `source_timestamp` that is made available to [DDSDataReader](#) objects by means of the `source_timestamp` attribute inside the [DDS_SampleInfo](#). (Refer to [DDS_SampleInfo](#) for details).

As a side effect, this operation asserts liveliness on the [DDSDataWriter](#) itself, the [DDSPublisher](#) and the [DDSDomainParticipant](#).

Note that the special value [DDS_HANDLE_NIL](#) can be used for the parameter `handle`. This indicates the identity of the instance should be automatically deduced from the `instance_data` (by means of the `key`).

If `handle` is any value other than [DDS_HANDLE_NIL](#), then it must correspond to an instance that has been registered. If there is no correspondence, the operation will fail with [DDS_RETCODE_BAD_PARAMETER](#).

RTI Connex DDS Micro will not detect the error when the `handle` is any value other than [DDS_HANDLE_NIL](#), corresponds to an instance that has been registered, but does not correspond to the instance deduced from the `instance_data` (by means of the `key`). RTI Connex DDS Micro will treat as if the `write()` operation is for the instance as indicated by the `handle`.

If there are no instance resources left, this operation may fail with [DDS_RETCODE_OUT_OF_RESOURCES](#). Calling [DDSDataWriter::unregister_instance_untyped](#) may help freeing up some resources.

Parameters

<i>instance_data</i>	<< <i>in</i> >> The data to write.
<i>handle</i>	<< <i>in</i> >> Either the handle returned by a previous call to DDSDDataWriter::register_instance_untyped , or else the special value DDS_HANDLE_NIL . If the data <i>has</i> a key and <i>handle</i> is not DDS_HANDLE_NIL , <i>handle</i> must represent a registered instance of the data. Otherwise, this method may fail with DDS_RETCODE_BAD_PARAMETER .

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_TIMEOUT](#), [DDS_RETCODE_OUT_OF_RESOURCES](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDSDDataReader](#)

[DDSDDataWriter::write_w_timestamp_untyped](#)

assert_liveliness()

```
virtual DDS_ReturnCode_t DDSDataWriter::assert_liveliness () [pure virtual]
```

This operation manually asserts the liveliness of this [DDSDDataWriter](#).

This is used in combination with the [LIVELINESS](#) policy to indicate to RTI Connex DDS Micro that the [DDSDDataWriter](#) remains active.

You only need to use this operation if the [LIVELINESS](#) setting is [DDS_MANUAL_BY_TOPIC_LIVELINESS_QOS](#). Otherwise, it has no effect.

Note: writing data via the [FooDataWriter::write](#) or [FooDataWriter::write_w_timestamp](#) operation asserts liveliness on the [DDSDDataWriter](#) itself, and its [DDSDDomainParticipant](#). Consequently the use of [assert_liveliness\(\)](#) is only needed if the application is not writing data regularly.

Returns

One of the [Standard Return Codes](#) or [DDS_RETCODE_NOT_ENABLED](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDS_LivelinessQosPolicy](#)

get_topic()

```
virtual DDS::Topic * DDSDataWriter::get_topic () [pure virtual]
```

This operation returns the [DDS::Topic](#) associated with the [DDSDataWriter](#).

This is the same [DDS::Topic](#) that was used to create the [DDSDataWriter](#).

Returns

[DDS::Topic](#) that was used to create the [DDSDataWriter](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

get_publisher()

```
virtual DDS::Publisher * DDSDataWriter::get_publisher () [pure virtual]
```

This operation returns the [DDS::Publisher](#) to which the [DDSDataWriter](#) belongs.

Returns

[DDS::Publisher](#) to which the [DDSDataWriter](#) belongs.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

get_qos()

```
virtual DDS_ReturnCode_t DDSDataWriter::get_qos (
    DDS_DataWriterQos & qos) [pure virtual]
```

Gets the writer QoS.

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

Parameters

<i>qos</i>	<< <i>inout</i> >> The DDS_DataWriterQos to be filled up.
------------	---

Returns

One of the [Standard Return Codes](#)

See also

[get_qos \(abstract\)](#)

set_qos()

```
virtual DDS_ReturnCode_t DDSDataWriter::set_qos (
    const DDS_DataWriterQos & qos) [pure virtual]
```

Sets the writer QoS.

This operation modifies the QoS of the [DDSDataWriter](#).

The [DDS_DataWriterQos::deadline](#) and [DDS_DataWriterQos::ownership_strength](#), can be changed. The other policies are immutable.

Parameters

<i>qos</i>	<< <i>in</i> >> The DDS_DataWriterQos to be set to. Policies must be consistent. Policies cannot be changed after DDSDataWriter is enabled. The special value DDS_DATAWRITER_QOS_DEFAULT can be used to indicate that the QoS of the DDSDataWriter should be changed to match the current default DDS_DataWriterQos set in the DDSPublisher .
------------	---

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_IMMUTABLE_POLICY](#) or [DDS_RETCODE_INCONSISTENT_POLICY](#)

See also

[DDS_DataWriterQos](#) for rules on consistency among QoS
[set_qos](#) (abstract)

set_listener()

```
virtual DDS_ReturnCode_t DDSDataWriter::set_listener (
    const DDSDataWriterListener * listener,
    DDS_StatusMask mask) [pure virtual]
```

Sets the writer listener.

Parameters

<i>listener</i>	<< <i>in</i> >> DDSDataWriterListener to set to
<i>mask</i>	<< <i>in</i> >> DDS_StatusMask associated with the DDSDataWriterListener .

Returns

One of the [Standard Return Codes](#)

See also

[set_listener](#) (abstract)

get_listener()

```
virtual DDSDataWriterListener * DDSDataWriter::get_listener () [pure virtual]
```

Get the writer listener.

Returns

[DDSDataWriterListener](#) of the [DDSDataWriter](#).

register_instance_untyped()

```
virtual DDS_InstanceHandle_t DDSDataWriter::register_instance_untyped (
    const void * instance_data) [pure virtual]
```

Informs RTI Connexx DDS Micro that the application will be modifying a particular instance.

This operation is only useful for keyed data types. Using it for non-keyed types causes no effect and returns [DDS_HANDLE_NIL](#). The operation takes as a parameter an instance (of which only the key value is examined) and returns a `handle` that can be used in successive `write()` or `dispose()` operations.

The operation gives RTI Connexx DDS Micro an opportunity to pre-configure itself to improve performance.

The use of this operation by an application is optional even for keyed types. If an instance has not been pre-registered, the application can use the special value [DDS_HANDLE_NIL](#) as the [DDS_InstanceHandle_t](#) parameter to the write or dispose operation and RTI Connexx DDS Micro will auto-register the instance.

For best performance, the operation should be invoked prior to calling any operation that modifies the instance, such as [DDSDataWriter::write_untyped](#), [DDSDataWriter::write_w_timestamp_untyped](#), [DDSDataWriter::dispose_untyped](#) and the handle used in conjunction with the data for those calls.

When this operation is used, RTI Connexx DDS Micro will automatically supply the value of the `source_timestamp` that is used.

This operation may fail and return [DDS_HANDLE_NIL](#) if [DDS_ResourceLimitsQosPolicy::max_instances](#) limit has been exceeded.

The operation is **idempotent**. If it is called for an already registered instance, it just returns the already allocated handle. This may be used to lookup and retrieve the handle allocated to a given instance.

This operation can only be called after [DDSDataWriter](#) has been enabled. Otherwise, [DDS_HANDLE_NIL](#) will be returned.

Parameters

<i>instance_data</i>	<< <i>in</i> >> The instance that should be registered. Of this instance, only the fields that represent the key are examined by the method. .
----------------------	--

Returns

For keyed data type, a handle that can be used in the calls that take a [DDS_InstanceHandle_t](#), such as `write`, `dispose`, `unregister_instance`, or return [DDS_HANDLE_NIL](#) on failure. If the `instance_data` is of a data type that has no keys, this method always return [DDS_HANDLE_NIL](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDSDataWriter::unregister_instance_untyped](#)

register_instance_w_timestamp_untyped()

```
virtual DDS_InstanceHandle_t DDSDataWriter::register_instance_w_timestamp_untyped (
    const void * instance_data,
    const DDS_Time_t & source_timestamp) [pure virtual]
```

Performs the same function as [DDSDataWriter::register_instance_untyped](#) except that it also provides the value for the `source_timestamp`.

Explicitly provides the timestamp that will be available to the [DDSDataReader](#) objects by means of the `source_timestamp` attribute inside the [DDS_SampleInfo](#). (Refer to [DDS_SampleInfo](#) for details)

The constraints on the values of the parameters and the corresponding error behavior are the same specified for the [DDSDataWriter::register_instance_untyped](#) operation.

This method allows type-independent code to work with a variety of concrete [FooDataWriter](#) classes in a consistent way.

Statically type-safe code should use the appropriate [FooDataWriter::register_instance_w_timestamp](#) method instead of this one. See that method for detailed documentation.

Parameters

<i>instance_data</i>	<<in>> The instance that should be registered. Of this instance, only the fields that represent the key are examined by the method. .
<i>source_timestamp</i>	<<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers. This timestamp will be available to the DDSDataReader objects by means of the <code>source_timestamp</code> attribute inside the DDS_SampleInfo .

Returns

For keyed data type, a handle that can be used in the calls that take a [DDS_InstanceHandle_t](#), such as `write`, `dispose`, `unregister_instance`, or return [DDS_HANDLE_NIL](#) on failure. If the `instance_data` is of a data type that has no keys, this method always return [DDS_HANDLE_NIL](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDSDataWriter::unregister_instance_untyped](#)

unregister_instance_untyped()

```
virtual DDS_ReturnCode_t DDSDataWriter::unregister_instance_untyped (
    const void * instance_data,
    const DDS_InstanceHandle_t & handle) [pure virtual]
```

Reverses the action of [DDSDataWriter::register_instance_untyped](#).

This operation is useful only for keyed data types. Using it for non-keyed types causes no effect and reports no error. The operation takes as a parameter an instance (of which only the key value is examined) and a handle.

This operation should only be called on an instance that is currently registered. This includes instances that have been auto-registered by calling operations such as `write` or `dispose` as described in [DDSDataWriter::register_instance_untyped](#). Otherwise, this operation may fail with [DDS_RETCODE_BAD_PARAMETER](#).

This only need be called just once per instance, regardless of how many times `register_instance` was called for that instance.

When this operation is used, RTI Connexx DDS Micro will automatically supply the value of the `source_timestamp` that is used.

This operation informs RTI Connexx DDS Micro that the [DDSDataWriter](#) is no longer going to provide any information about the instance. This operation also indicates that RTI Connexx DDS Micro can locally remove all information regarding that instance. The application should not attempt to use the `handle` previously allocated to that instance after calling [DDSDataWriter::unregister_instance_untyped\(\)](#).

The special value [DDS_HANDLE_NIL](#) can be used for the parameter `handle`. This indicates that the identity of the instance should be automatically deduced from the `instance_data` (by means of the `key`).

If `handle` is any value other than [DDS_HANDLE_NIL](#), then it must correspond to an instance that has been registered. If there is no correspondence, the operation will fail with [DDS_RETCODE_BAD_PARAMETER](#).

RTI Connexx DDS Micro will not detect the error when the `handle` is any value other than [DDS_HANDLE_NIL](#), corresponds to an instance that has been registered, but does not correspond to the instance deduced from the `instance_data` (by means of the `key`). RTI Connexx DDS Micro will treat as if the `unregister_instance()` operation is for the instance as indicated by the `handle`.

If after a [DDSDataWriter::unregister_instance_untyped](#), the application wants to modify ([DDSDataWriter::write_untyped](#) or [DDSDataWriter::dispose_untyped](#)) an instance, it has to register it again, or else use the special `handle` value [DDS_HANDLE_NIL](#).

This operation does not indicate that the instance is deleted (that is the purpose of [DDSDataWriter::dispose_untyped](#)). The operation [DDSDataWriter::unregister_instance_untyped](#) just indicates that the [DDSDataWriter](#) no longer has anything to say about the instance. [DDSDataReader](#) entities that are reading the instance may receive a sample with [DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE](#) for the instance, unless there are other [DDSDataWriter](#) objects writing that same instance.

This operation can affect the ownership of the data instance (see [OWNERSHIP](#)). If the [DDSDataWriter](#) was the exclusive owner of the instance, then calling `unregister_instance()` will relinquish that ownership.

Parameters

<i>instance_data</i>	<< <i>in</i> >> The instance that should be unregistered. If the data <i>has</i> a key and <code>instance_handle</code> is DDS_HANDLE_NIL , only the fields that represent the key are examined by the method. Otherwise, <code>instance_data</code> is not used. If <code>instance_data</code> is used, it must represent an instance that has been registered. Otherwise, this method may fail with DDS_RETCODE_BAD_PARAMETER .
<i>handle</i>	<< <i>in</i> >> represents the instance to be unregistered. If the data <i>has</i> a key and <code>handle</code> is DDS_HANDLE_NIL , <code>handle</code> is not used and <code>instance</code> is deduced from <code>instance_data</code> . If the data has no key, <code>handle</code> is not used. If <code>handle</code> is used, it must represent an instance that has been registered. Otherwise, this method may fail with DDS_RETCODE_BAD_PARAMETER .

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_TIMEOUT](#) or [DDS_RETCODE_NOT_ENABLED](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDSDataWriter::register_instance_untyped](#)

unregister_instance_w_timestamp_untyped()

```
virtual DDS_ReturnCode_t DDSDataWriter::unregister_instance_w_timestamp_untyped (
    const void * instance_data,
    const DDS_InstanceHandle_t & handle,
    const DDS_Time_t & source_timestamp) [pure virtual]
```

Performs the same function as [DDSDataWriter::unregister_instance_untyped](#) except that it also provides the value for the `source_timestamp`.

Explicitly provides the timestamp that will be available to the [DDSDataReader](#) objects by means of the `source_timestamp` attribute inside the [DDS_SampleInfo](#). (Refer to [DDS_SampleInfo](#) for details)

The constraints on the values of the parameters and the corresponding error behavior are the same specified for the [DDSDataWriter::unregister_instance_untyped](#) operation.

Parameters

<i>instance_data</i>	<<in>> The instance that should be unregistered. If the data <i>has</i> a key and <i>instance_handle</i> is DDS_HANDLE_NIL , only the fields that represent the key are examined by the method. Otherwise, <i>instance_data</i> is not used. If <i>instance_data</i> is used, it must represent an instance that has been registered. Otherwise, this method may fail with DDS_RETCODE_BAD_PARAMETER .
<i>handle</i>	<<in>> represents the <i>instance</i> to be unregistered. If the data <i>has</i> a key and <i>handle</i> is DDS_HANDLE_NIL , <i>handle</i> is not used and <i>instance</i> is deduced from <i>instance_data</i> . If the data has no key, <i>handle</i> is not used. If <i>handle</i> is used, it must represent an instance that has been registered. Otherwise, this method may fail with DDS_RETCODE_BAD_PARAMETER .

Parameters

<i>source_timestamp</i>	<<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers. This timestamp will be available to the DDSDDataReader objects by means of the <i>source_timestamp</i> attribute inside the DDS_SampleInfo .
-------------------------	--

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_TIMEOUT](#) or [DDS_RETCODE_NOT_ENABLED](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDSDDataWriter::register_instance_untyped](#)

dispose_untyped()

```
virtual DDS\_ReturnCode\_t DDSDDataWriter::dispose_untyped (
    const void * instance_data,
    const DDS\_InstanceHandle\_t & handle) [pure virtual]
```

Requests the middleware to delete the data.

This operation is useful only for keyed data types. Using it for non-keyed types has no effect and reports no error.

The actual deletion is postponed until there is no more use for that data in the whole system.

Applications are made aware of the deletion by means of operations on the [DDSDDataReader](#) objects that already knew that instance. [DDSDDataReader](#) objects that didn't know the instance will never see it.

This operation does not modify the value of the instance. The `instance_data` parameter is passed just for the purposes of identifying the instance.

When this operation is used, RTI Connext DDS Micro will automatically supply the value of the `source_timestamp` that is made available to `DDSDataReader` objects by means of the `source_timestamp` attribute inside the `DDS_SampleInfo`.

The constraints on the values of the handle parameter and the corresponding error behavior are the same specified for the `DDSDataWriter::unregister_instance_untyped` operation.

The special value `DDS_HANDLE_NIL` can be used for the parameter `instance_handle`. This indicates the identity of the instance should be automatically deduced from the `instance_data` (by means of the key).

If `handle` is any value other than `DDS_HANDLE_NIL`, then it must correspond to an instance that has been registered. If there is no correspondence, the operation will fail with `DDS_RETCODE_BAD_PARAMETER`.

RTI Connext DDS Micro will not detect the error when the `handle` is any value other than `DDS_HANDLE_NIL`, corresponds to an instance that has been registered, but does not correspond to the instance deduced from the `instance_data` (by means of the key). RTI Connext DDS Micro will treat as if the `dispose()` operation is for the instance as indicated by the `handle`.

This operation may block and time out (`DDS_RETCODE_TIMEOUT`) under the same circumstances described for `DDSDataWriter::write_untyped()`.

If there are no instance resources left, this operation may fail with `DDS_RETCODE_OUT_OF_RESOURCES`. Calling `DDSDataWriter::unregister_instance_untyped` may help freeing up some resources.

Parameters

<i>instance_data</i>	<<in>> The data to dispose. If the data <i>has</i> a key and <code>instance_handle</code> is <code>DDS_HANDLE_NIL</code> , only the fields that represent the key are examined by the method. Otherwise, <code>instance_data</code> is not used.
<i>handle</i>	<<in>> Either the handle returned by a previous call to <code>DDSDataWriter::register_instance_untyped</code> , or else the special value <code>DDS_HANDLE_NIL</code> . If the data <i>has</i> a key and <code>instance_handle</code> is <code>DDS_HANDLE_NIL</code> , <code>instance_handle</code> is not used and <code>instance</code> is deduced from <code>instance_data</code> . If the data has no key, <code>instance_handle</code> is not used. If <code>handle</code> is used, it must represent a registered instance of the matching data type. Otherwise, this method may fail with <code>DDS_RETCODE_BAD_PARAMETER</code> .

Returns

One of the [Standard Return Codes](#), `DDS_RETCODE_TIMEOUT`, `DDS_RETCODE_OUT_OF_RESOURCES` or `DDS_RETCODE_NOT_ENABLED`.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

dispose_w_timestamp_untyped()

```
virtual DDS_ReturnCode_t DDSDDataWriter::dispose_w_timestamp_untyped (
    const void * instance_data,
    const DDS_InstanceHandle_t & handle,
    const DDS_Time_t & source_timestamp) [pure virtual]
```

Performs the same function as [DDSDDataWriter::dispose_untyped](#) except that it also provides the value for the `source_timestamp`.

Explicitly provides the timestamp that will be available to the [DDSDDataReader](#) objects by means of the `source_timestamp` attribute inside the [DDS_SampleInfo](#). (Refer to [DDS_SampleInfo](#) for details)

The constraints on the values of the parameters and the corresponding error behavior are the same specified for the [DDSDDataWriter::dispose_untyped](#) operation.

Parameters

<i>instance_data</i>	<<in>> The data to dispose. If the data <i>has</i> a key and <code>instance_handle</code> is DDS_HANDLE_NIL , only the fields that represent the key are examined by the method. Otherwise, <code>instance_data</code> is not used.
<i>handle</i>	<<in>> Either the handle returned by a previous call to DDSDDataWriter::register_instance_untyped , or else the special value DDS_HANDLE_NIL . If the data <i>has</i> a key and <code>instance_handle</code> is DDS_HANDLE_NIL , <code>instance_handle</code> is not used and <code>instance</code> is deduced from <code>instance_data</code> . If the data has no key, <code>instance_handle</code> is not used. If <code>handle</code> is used, it must represent a registered instance of the matching data type. Otherwise, this method may fail with DDS_RETCODE_BAD_PARAMETER .
<i>source_timestamp</i>	<<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers. This timestamp will be available to the DDSDDataReader objects by means of the <code>source_timestamp</code> attribute inside the DDS_SampleInfo .

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_TIMEOUT](#), [DDS_RETCODE_OUT_OF_RESOURCES](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

write_w_timestamp_untyped()

```
virtual DDS_ReturnCode_t DDSDataWriter::write_w_timestamp_untyped (
    const void * instance_data,
    const DDS_InstanceHandle_t & handle,
    const DDS_Time_t & source_timestamp) [pure virtual]
```

Performs the same function as [DDSDataWriter::write_untyped](#) except that it also provides the value for the `source_timestamp`.

Explicitly provides the timestamp that will be available to the [DDSDataReader](#) objects by means of the `source_timestamp` attribute inside the [DDS_SampleInfo](#). (Refer to [DDS_SampleInfo](#) for details)

The constraints on the values of the `handle` parameter and the corresponding error behavior are the same specified for the [DDSDataWriter::write_untyped](#) operation.

If there are no instance resources left, this operation may fail with [DDS_RETCODE_OUT_OF_RESOURCES](#). Calling [DDSDataWriter::unregister_instance_untyped](#) may help free up some resources.

This operation may fail with [DDS_RETCODE_BAD_PARAMETER](#) under the same circumstances described for the write operation.

Parameters

<i>instance_data</i>	<<in>> The data to write.
<i>handle</i>	<<in>> Either the handle returned by a previous call to DDSDataWriter::register_instance_untyped , or else the special value DDS_HANDLE_NIL . If the data has a key and <code>handle</code> is not DDS_HANDLE_NIL , <code>handle</code> must represent a registered instance of the matching data type. Otherwise, this method may fail with DDS_RETCODE_BAD_PARAMETER .
<i>source_timestamp</i>	<<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers. This timestamp will be available to the DDSDataReader objects by means of the <code>source_timestamp</code> attribute inside the DDS_SampleInfo .

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_TIMEOUT](#), [DDS_RETCODE_OUT_OF_RESOURCES](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDSDataWriter::write_untyped](#)

[DDSDataReader](#)

get_publication_matched_status()

```
virtual DDS_ReturnCode_t DDSDataWriter::get_publication_matched_status (  
    DDS_PublicationMatchedStatus & status) [pure virtual]
```

Accesses the [DDS_PUBLICATION_MATCHED_STATUS](#) communication status.

Parameters

<i>status</i>	<< <i>inout</i> >> DDS_PublicationMatchedStatus to be filled in.
---------------	--

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

get_liveliness_lost_status()

```
virtual DDS_ReturnCode_t DDSDataWriter::get_liveliness_lost_status (  
    DDS_LivelinessLostStatus & status) [pure virtual]
```

Accesses the [DDS_LIVELINESS_LOST_STATUS](#) communication status.

Parameters

<i>status</i>	<< <i>inout</i> >> DDS_LivelinessLostStatus to be filled in.
---------------	--

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

get_matched_subscriptions()

```
virtual DDS_ReturnCode_t DDSDataWriter::get_matched_subscriptions (  
    DDS_InstanceHandleSeq & subscription_handles) [pure virtual]
```

Returns a sequence containing [DDS_InstanceHandle_t](#) for all the remote [DDSDataReader](#) that have been matched by the [DDSDataWriter](#).

Matched subscriptions include all those in the same domain that have a matching [DDSTopic](#), compatible QoS and common partition .

The handles returned in the `subscription_handles` list are the ones that are used by the DDS implementation to locally identify the corresponding matched [DDSDataReader](#) entities. These handles match the ones that appear in the [DDS_SampleInfo::instance_handle](#) field of the [DDS_SampleInfo](#) when reading the [DDS_SUBSCRIPTION_TOPIC_NAME](#) builtin topic.

Parameters

<i>subscription_handles</i>	<<inout>>.
-----------------------------	------------

Handles of all the matched subscriptions.

The sequence will be grown if the sequence has ownership and the system has the corresponding resources. Use a sequence without ownership to avoid dynamic memory allocation. If the sequence is too small to store all the matches and the system can not resize the sequence, this method will fail with [DDS_RETCODE_OUT_OF_RESOURCES](#).

The maximum number of matches possible is configured with [DDS_DomainParticipantResourceLimitsQosPolicy](#). You can use a zero-maximum sequence without ownership to quickly check whether there are any matches without allocating any memory. .

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_OUT_OF_RESOURCES](#) if the sequence is too small and the system can not resize it, or [DDS_RETCODE_NOT_ENABLED](#)

get_matched_subscription_data()

```
virtual DDS_ReturnCode_t DDSDataWriter::get_matched_subscription_data (
    DDS_SubscriptionBuiltinTopicData & subscription_data,
    const DDS_InstanceHandle_t & subscription_handle) [pure virtual]
```

Returns information about a remote [DDSDDataReader](#) that has been matched by the [DDSDDataWriter](#).

The *subscription_handle* must correspond to a subscription currently associated with the [DDSDDataWriter](#). Otherwise, the operation will fail with [DDS_RETCODE_BAD_PARAMETER](#). Use [DDSDDataWriter::get_matched_subscriptions](#) to find the subscriptions that are currently matched with the [DDSDDataWriter](#).

Parameters

<i>subscription_data</i>	<<inout>>. The information to be filled in on the associated subscription. Cannot be NULL.
<i>subscription_handle</i>	<<in>>. Handle to a specific subscription associated with the DDSDDataWriter . . Must correspond to a subscription currently associated with the DDSDDataWriter .

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_NOT_ENABLED](#)

get_offered_deadline_missed_status()

```
virtual DDS_ReturnCode_t DDSDataWriter::get_offered_deadline_missed_status (
    DDS_OfferedDeadlineMissedStatus & status) [pure virtual]
```

Accesses the [DDS_OFFERED_DEADLINE_MISSED_STATUS](#) communication status.

Parameters

<i>status</i>	<< <i>inout</i> >> DDS_OfferedDeadlineMissedStatus to be filled in.
---------------	---

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

get_offered_incompatible_qos_status()

```
virtual DDS_ReturnCode_t DDSDataWriter::get_offered_incompatible_qos_status (
    DDS_OfferedIncompatibleQosStatus & status) [pure virtual]
```

Accesses the [DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#) communication status.

Parameters

<i>status</i>	<< <i>inout</i> >> DDS_OfferedIncompatibleQosStatus to be filled in.
---------------	--

Returns

One of the [Standard Return Codes](#)

MT Safety:

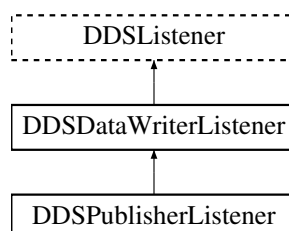
This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.98 DDSDataWriterListener Class Reference

<<*interface*>> [DDSListener](#) for writer status.

```
#include <dds_cpp_publication.hxx>
```

Inheritance diagram for DDSDataWriterListener:



Public Member Functions

- virtual void [on_offered_deadline_missed](#) (DDSDDataWriter *writer, const [DDS_OfferedDeadlineMissedStatus](#) &status)
Handles the [DDS_OFFERED_DEADLINE_MISSED_STATUS](#) status.
- virtual void [on_liveliness_lost](#) (DDSDDataWriter *writer, const [DDS_LivelinessLostStatus](#) &status)
Handles the [DDS_LIVELINESS_LOST_STATUS](#) status.
- virtual void [on_offered_incompatible_qos](#) (DDSDDataWriter *writer, const [DDS_OfferedIncompatibleQosStatus](#) &status)
Handles the [DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#) status.
- virtual void [on_publication_matched](#) (DDSDDataWriter *writer, const [DDS_PublicationMatchedStatus](#) &status)
Handles the [DDS_PUBLICATION_MATCHED_STATUS](#) status.
- virtual void [on_reliable_reader_activity_changed](#) (DDSDDataWriter *writer, const [DDS_ReliableReaderActivityChangedStatus](#) &status)
<<eXtension>> A matched reliable reader has become active or become inactive.
- virtual void [on_reliable_sample_unacknowledged](#) (DDSDDataWriter *writer, const [DDS_ReliableSampleUnacknowledgedStatus](#) &status)
<<eXtension>> A matched reliable reader has become active or become inactive.
- virtual void [on_sample_removed](#) (DDSDDataWriter *writer, const [DDS_Cookie_t](#) &cookie)
<<eXtension>> Called when a sample is removed from the DataWriter queue.

7.98.1 Detailed Description

<<interface>> [DDSLListener](#) for writer status.

Entity:

[DDSDDataWriter](#)

Status:

[DDS_LIVELINESS_LOST_STATUS](#), [DDS_LivelinessLostStatus](#);
[DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#), [DDS_OfferedIncompatibleQosStatus](#);
[DDS_PUBLICATION_MATCHED_STATUS](#), [DDS_PublicationMatchedStatus](#);

See also

[Status Kinds](#)

7.98.2 Member Function Documentation**on_offered_deadline_missed()**

```
virtual void DDSDDataWriterListener::on_offered_deadline_missed (
    DDSDDataWriter * writer,
    const DDS\_OfferedDeadlineMissedStatus & status) [inline], [virtual]
```

Handles the [DDS_OFFERED_DEADLINE_MISSED_STATUS](#) status.

This callback is called when the deadline that the [DDSDDataWriter](#) has committed through its [DEADLINE](#) qos policy was not respected for a specific instance. This callback is called for each deadline period elapsed during which the [DDSDDataWriter](#) failed to provide data for an instance.

Parameters

<i>writer</i>	<<out>> Locally created DDSDDataWriter that triggers the listener callback
<i>status</i>	<<out>> Current deadline missed status of locally created DDSDDataWriter

on_liveliness_lost()

```
virtual void DDSDataWriterListener::on_liveliness_lost (
    DDSDataWriter * writer,
    const DDS_LivelinessLostStatus & status) [inline], [virtual]
```

Handles the [DDS_LIVELINESS_LOST_STATUS](#) status.

This callback is called when the liveliness that the [DDSDDataWriter](#) has committed through its [LIVELINESS](#) qos policy was not respected; this [DDSDDataReader](#) entities will consider the [DDSDDataWriter](#) as no longer "alive/active". This callback will not be called when an already not alive [DDSDDataWriter](#) simply remains not alive for another liveliness period.

Parameters

<i>writer</i>	<<out>> Locally created DDSDDataWriter that triggers the listener callback
<i>status</i>	<<out>> Current liveliness lost status of locally created DDSDDataWriter

on_offered_incompatible_qos()

```
virtual void DDSDataWriterListener::on_offered_incompatible_qos (
    DDSDataWriter * writer,
    const DDS_OfferedIncompatibleQosStatus & status) [inline], [virtual]
```

Handles the [DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#) status.

This callback is called when the [DDS_DataWriterQos](#) of the [DDSDDataWriter](#) was incompatible with what was requested by a [DDSDDataReader](#). This callback is called when a [DDSDDataWriter](#) has discovered a [DDSDDataReader](#) for the same [DDSTopic](#) and common partition, but with a requested QoS that is incompatible with that offered by the [DDSDDataWriter](#).

Parameters

<i>writer</i>	<<out>> Locally created DDSDDataWriter that triggers the listener callback
<i>status</i>	<<out>> Current incompatible qos status of locally created DDSDDataWriter

on_publication_matched()

```
virtual void DDSDataWriterListener::on_publication_matched (
    DDSDataWriter * writer,
    const DDS_PublicationMatchedException & status) [inline], [virtual]
```

Handles the [DDS_PUBLICATION_MATCHED_STATUS](#) status.

This callback is called when the [DDSDDataWriter](#) has found a [DDSDDataReader](#) that matches the [DDSTopic](#), has a common partition and compatible QoS, or has ceased to be matched with a [DDSDDataReader](#) that was previously considered to be matched.

Parameters

<i>writer</i>	<<out>> Locally created DDSDataWriter that triggers the listener callback
<i>status</i>	<<out>> Current publication match status of locally created DDSDataWriter

on_reliable_reader_activity_changed()

```
virtual void DDSDataWriterListener::on_reliable_reader_activity_changed (
    DDSDataWriter * writer,
    const DDS_ReliableReaderActivityChangedStatus & status) [inline], [virtual]
```

<<eXtension>> A matched reliable reader has become active or become inactive.

A reliable writer considers a matched reliable reader to be active when it receives ACKNACK messages from the reader in response to its HEARTBEAT messages. The writer considers an active reader to have become inactive when it has sent a number of consecutive periodic HEARTBEATs equal to [DDS_RtpsReliableWriterProtocol_t::max_heartbeat_retries](#) without receiving an ACKNACK from the reader. An inactive reader becomes active once the writer receives an ACKNACK from it.

This callback is called when the transition from active to inactive, and vice versa, has occurred for a matched reliable reader.

Parameters

<i>writer</i>	<<out>> Locally created DDSDataWriter that triggers the listener callback
<i>status</i>	<<out>> Current reliable reader activity changed status of locally created DDSDataWriter

on_reliable_sample_unacknowledged()

```
virtual void DDSDataWriterListener::on_reliable_sample_unacknowledged (
    DDSDataWriter * writer,
    const DDS_ReliableSampleUnacknowledgedStatus & status) [inline], [virtual]
```

<<eXtension>> A matched reliable reader has become active or become inactive.

A reliable writer considers a matched reliable reader to be active when it receives ACKNACK messages from the reader in response to its HEARTBEAT messages. The writer considers an active reader to have become inactive when it has sent a number of consecutive periodic HEARTBEATs equal to [DDS_RtpsReliableWriterProtocol_t::max_heartbeat_retries](#) without receiving an ACKNACK from the reader. An inactive reader becomes active once the writer receives an ACKNACK from it.

This callback is called when the transition from active to inactive, and vice versa, has occurred for a matched reliable reader.

Parameters

<i>writer</i>	<<out>> Locally created DDSDataWriter that triggers the listener callback
<i>status</i>	<<out>> Current reliable reader activity changed status of locally created DDSDataWriter

on_sample_removed()

```
virtual void DDSDataWriterListener::on_sample_removed (
    DDSDataWriter * writer,
    const DDS_Cookie_t & cookie) [inline], [virtual]
```

<<*eXtension*>> Called when a sample is removed from the DataWriter queue.

Supported while using [Zero Copy Transfer Over Shared Memory](#) "Zero Copy transfer over shared memory" or [FlatData Topic-Types](#) "FlatData language binding".

Parameters

<i>writer</i>	<< <i>out</i> >> Locally created DDSDataWriter that triggers the listener callback
<i>cookie</i>	<< <i>out</i> >> Contains the absolute address of the sample that is removed. The address of the sample can be obtained by using

See also

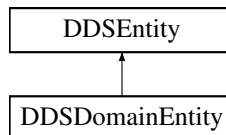
[FooDataWriter::get_loan](#)

7.99 DDSDomainEntity Class Reference

<<*interface*>> Abstract base class for all DDS entities except for the [DDSDomainParticipant](#).

```
#include <dds_cpp_infrastructure.hxx>
```

Inheritance diagram for DDSDomainEntity:

**Additional Inherited Members****Public Member Functions inherited from [DDSEntity](#)**

- [DDS_ReturnCode_t enable \(\)](#)
Enables the [DDSEntity](#).
- [DDSStatusCondition * get_statuscondition \(\)](#)
Returns the [DDSStatusCondition](#) associated with a particular [DDSEntity](#).
- [DDS_StatusMask get_status_changes \(\)](#)
Retrieves the list of communication statuses in the [DDSEntity](#) that are triggered.
- [DDS_InstanceHandle_t get_instance_handle \(\)](#)
Allows access to the [DDS_InstanceHandle_t](#) associated with the [DDSEntity](#).

7.99.1 Detailed Description

<<interface>> Abstract base class for all DDS entities except for the [DDSDomainParticipant](#).

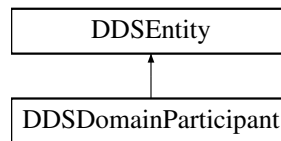
Its sole purpose is to *conceptually* express that [DDSDomainParticipant](#) is a special kind of [DDSEntity](#) that acts as a container of all other [DDSEntity](#) but itself cannot contain other [DDSDomainParticipant](#).

7.100 DDSDomainParticipant Class Reference

<<interface>> Container for all [DDSDomainEntity](#) objects.

```
#include <dds_cpp_domain.hxx>
```

Inheritance diagram for DDSDomainParticipant:



Public Member Functions

- virtual [DDSEntity](#) * [as_entity](#) ()=0
Access a [DDSDomainParticipant](#)'s supertype instance.
- virtual [DDSPublisher](#) * [create_publisher](#) (const [DDS_PublisherQos](#) &qos, [DDSPublisherListener](#) *listener, [DDS_StatusMask](#) mask)=0
Creates a [DDSPublisher](#) with the desired QoS policies and attaches to it the specified [DDSPublisherListener](#).
- virtual [DDS_ReturnCode_t](#) [delete_publisher](#) ([DDSPublisher](#) *p)=0
Deletes an existing [DDSPublisher](#).
- virtual [DDSSubscriber](#) * [create_subscriber](#) (const [DDS_SubscriberQos](#) &qos, [DDSSubscriberListener](#) *listener, [DDS_StatusMask](#) mask)=0
Creates a [DDSSubscriber](#) with the desired QoS policies and attaches to it the specified [DDSSubscriberListener](#).
- virtual [DDS_ReturnCode_t](#) [delete_subscriber](#) ([DDSSubscriber](#) *s)=0
Deletes an existing [DDSSubscriber](#).
- virtual [DDSTopic](#) * [create_topic](#) (const char *topic_name, const char *type_name, const [DDS_TopicQos](#) &qos, [DDSTopicListener](#) *listener, [DDS_StatusMask](#) mask)=0
Creates a [DDSTopic](#) with the desired QoS policies
- virtual [DDS_ReturnCode_t](#) [delete_topic](#) ([DDSTopic](#) *topic)=0
Deletes a [DDSTopic](#).
- virtual [DDSTopic](#) * [find_topic](#) (const char *topic_name, const [DDS_Duration_t](#) &timeout)=0
Finds an existing (or ready to exist) [DDSTopic](#), based on its name.
- virtual [DDS_ReturnCode_t](#) [get_qos](#) ([DDS_DomainParticipantQos](#) &qos)=0
Get the participant QoS.
- virtual [DDS_ReturnCode_t](#) [set_qos](#) (const [DDS_DomainParticipantQos](#) &qos)=0
Change the QoS of this domain participant.
- virtual [DDS_ReturnCode_t](#) [get_default_publisher_qos](#) ([DDS_PublisherQos](#) &qos)=0

- Copy the default *DDS_PublisherQos* values into the provided *DDS_PublisherQos* instance.

 - virtual *DDS_ReturnCode_t* *set_default_publisher_qos* (const *DDS_PublisherQos* &qos)=0

Set the default *DDS_PublisherQos* values for this domain participant.
- virtual *DDS_ReturnCode_t* *get_default_subscriber_qos* (*DDS_SubscriberQos* &qos)=0

Copy the default *DDS_SubscriberQos* values into the provided *DDS_SubscriberQos* instance.
- virtual *DDS_ReturnCode_t* *set_default_subscriber_qos* (const *DDS_SubscriberQos* &qos)=0

Set the default *DDS_SubscriberQos* values for this domain participant.
- virtual *DDS_ReturnCode_t* *get_default_topic_qos* (*DDS_TopicQos* &qos)=0

Copies the default *DDS_TopicQos* values for this domain participant into the given *DDS_TopicQos* instance.
- virtual *DDS_ReturnCode_t* *set_default_topic_qos* (const *DDS_TopicQos* &qos)=0

Set the default *DDS_TopicQos* values for this domain participant.
- virtual *DDSTopicDescription* * *lookup_topicdescription* (const char *topic_name)=0

Lookup an existing locally-created *DDSTopicDescription*, based on its name.
- virtual *DDSPublisher* * *lookup_publisher_by_name* (const char *publisher_name)=0

Retrieves a *DDSPublisher* by its entity name within this *DDSDomainParticipant*
- virtual *DDSSubscriber* * *lookup_subscriber_by_name* (const char *subscriber_name)=0

Retrieves a *DDSSubscriber* by its entity name within this *DDSDomainParticipant*
- virtual *DDSDataWriter* * *lookup_datawriter_by_name* (const char *datawriter_full_name)=0

Retrieves a *DDSDataWriter* by its entity name within this *DDSDomainParticipant*
- virtual *DDSDataReader* * *lookup_datareader_by_name* (const char *datareader_full_name)=0

Retrieves a *DDSDataReader* by its entity name within this *DDSDomainParticipant*
- virtual *DDS_ReturnCode_t* *assert_liveliness* ()=0

Manually asserts the liveliness of this *DDSDomainParticipant*.
- virtual *DDS_ReturnCode_t* *delete_contained_entities* ()=0

Delete all the entities that were created by means of the "create" operations on the *DDSDomainParticipant*.
- virtual *DDS_ReturnCode_t* *register_type* (const char *type_name, *DDS_TypePluginI* *plugin)=0

<<eXtension>> Allows an application to communicate to RTI Connex DDS Micro the existence of a data type.
- virtual *DDS_TypePluginI* * *unregister_type* (const char *type_name)=0

Allows an application to unregister a data type from RTI Connex DDS Micro. After calling *unregister_type*, no further communication using that type is possible.
- virtual *DDS_ReturnCode_t* *set_listener* (const *DDSDomainParticipantListener* *listener, *DDS_StatusMask* mask)=0

Sets the participant listener.
- virtual *DDSDomainParticipantListener* * *get_listener* ()=0

Get the participant listener.
- virtual *DDS_ReturnCode_t* *get_discovered_participants* (*DDS_InstanceHandleSeq* &participant_handles)=0

Returns a sequence containing *DDS_InstanceHandle_t* for all the remote *DDSDomainParticipant* that have been discovered by the *DDSDomainParticipant*.
- virtual *DDS_ReturnCode_t* *get_discovered_participant_data* (*DDS_ParticipantBuiltinTopicData* &participant_data, const *DDS_InstanceHandle_t* &participant_handle)=0

Returns information about a remote *DDSDomainParticipant* that has been discovered by the *DDSDomainParticipant*.
- virtual *DDS_ReturnCode_t* *get_current_time* (*DDS_Time_t* ¤t_time)=0

Returns the current value of the time.
- virtual *DDS_ReturnCode_t* *add_peer* (const char *peer)=0

<<eXtension>> Attempt to contact one or more additional peer participants.
- virtual *DDS_ReturnCode_t* *get_default_flowcontroller_property* (*DDS_FlowControllerProperty_t* &prop)=0

<<eXtension>> Copies the default *DDS_FlowControllerProperty_t* values for this domain participant into the given *DDS_FlowControllerProperty_t* instance.

- virtual [DDS_ReturnCode_t set_default_flowcontroller_property](#) (const [DDS_FlowControllerProperty_t](#) &prop)=0
<<eXtension>> Set the default [DDS_FlowControllerProperty_t](#) values for this domain participant.
- virtual [DDSFlowController * create_flowcontroller](#) (const char *name, const [DDS_FlowControllerProperty_t](#) &prop)=0
<<eXtension>> Creates a [DDSFlowController](#) with the desired property.
- virtual [DDS_ReturnCode_t delete_flowcontroller](#) ([DDSFlowController](#) *fc)=0
<<eXtension>> Deletes an existing [DDSFlowController](#).
- virtual [DDSFlowController * lookup_flowcontroller](#) (const char *name)=0
<<eXtension>> Looks up an existing locally-created [DDSFlowController](#), based on its name.

Public Member Functions inherited from [DDSEntity](#)

- [DDS_ReturnCode_t enable](#) ()
Enables the [DDSEntity](#).
- [DDSStatusCondition * get_statuscondition](#) ()
Returns the [DDSStatusCondition](#) associated with a particular [DDSEntity](#).
- [DDS_StatusMask get_status_changes](#) ()
Retrieves the list of communication statuses in the [DDSEntity](#) that are triggered.
- [DDS_InstanceHandle_t get_instance_handle](#) ()
Allows access to the [DDS_InstanceHandle_t](#) associated with the [DDSEntity](#).

7.100.1 Detailed Description

<<interface>> Container for all [DDSDomainEntity](#) objects.

The DomainParticipant object plays several roles:

- It acts as a container for all other [DDSEntity](#) objects.
- It acts as *factory* for the [DDSPublisher](#), [DDSSubscriber](#), [DDSTopic](#) and [DDSEntity](#) objects.
- It represents the participation of the application on a communication plane that isolates applications running on the same set of physical computers from each other. A domain establishes a virtual network linking all applications that share the same `domainId` and isolating them from applications running on different domains. In this way, several independent distributed applications can coexist in the same physical network without interfering, or even being aware of each other.

A [DDSDomainParticipant](#) may be created in a disabled or enabled state; refer to [DDSEntity::enable](#) for details. The initial state of the [DDSDomainParticipant](#) is determined by [DDS_DomainParticipantFactoryQos::entity_factory](#).

The documentation for each operation on the [DDSDomainParticipant](#) includes any restrictions with respect to whether the operation is valid only for a disabled or enabled [DDSDomainParticipant](#).

The following operations may be called even if the [DDSDomainParticipant](#) is not enabled. Other operations will fail with the value [DDS_RETCODE_NOT_ENABLED](#) if called on a disabled DomainParticipant:

- Operations defined at the base-class level namely, [get_qos\(\)](#), [get_listener\(\)](#), and [enable\(\)](#), [set_qos\(\)](#), [set_listener\(\)](#)
- Factory operations: [create_topic\(\)](#), [create_publisher\(\)](#), [create_subscriber\(\)](#), [delete_topic\(\)](#), [delete_publisher\(\)](#), [delete_subscriber\(\)](#), [delete_contained_entities\(\)](#).

QoS:

[DDS_DomainParticipantQos](#)

Status:

[Status Kinds](#)

Listener:

[DDSDomainParticipantListener](#)

7.100.2 Member Function Documentation

as_entity()

```
virtual DDSEntity * DDSDomainParticipant::as_entity () [pure virtual]
```

Access a [DDSDomainParticipant](#)'s supertype instance.

Returns

[DDSDomainParticipant](#)'s supertype [DDSEntity](#) instance

create_publisher()

```
virtual DDSPublisher * DDSDomainParticipant::create_publisher (
    const DDS\_PublisherQos & qos,
    DDSPublisherListener * listener,
    DDS\_StatusMask mask) [pure virtual]
```

Creates a [DDSPublisher](#) with the desired QoS policies and attaches to it the specified [DDSPublisherListener](#).

The specified QoS policies must be consistent, or the operation will fail and no [DDSPublisher](#) will be created.

Parameters

<i>qos</i>	<<in>> QoS to be used for creating the new DDSPublisher . The special value DDS_PUBLISHER_QOS_DEFAULT can be used to indicate that the DDSPublisher should be created with the default DDS_PublisherQos set in the DDSDomainParticipant .
<i>listener</i>	<<in>> Listener to be attached to the newly created DDSPublisher .
<i>mask</i>	<<in>> Changes of communication status to be invoked on the listener.

Returns

newly created publisher object or NULL on failure.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

ARINC 653 API Restriction:

This function *must* only be called before a partition enters *NORMAL* mode.

See also

[Specifying QoS on entities](#) for information on setting QoS before entity creation

[DDS_PublisherQos](#) for rules on consistency among QoS

[DDS_PUBLISHER_QOS_DEFAULT](#)

delete_publisher()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::delete_publisher (
    DDSPublisher * p) [pure virtual]
```

Deletes an existing [DDSPublisher](#).

Precondition

The [DDSPublisher](#) must not have any attached [DDSDataWriter](#) objects. If there are existing [DDSDataWriter](#) objects, it will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#).

[DDSPublisher](#) must have been created by this [DDSDomainParticipant](#), or else it will fail with [DDS_RETCODE_PRECONDITION_NO](#)

Postcondition

Listener installed on the [DDSPublisher](#) will not be called after this method completes successfully.

Parameters

<i>p</i>	<< <i>in</i> >> DDSPublisher to be deleted.
----------	---

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_PRECONDITION_NOT_MET](#).

create_subscriber()

```
virtual DDSSubscriber * DDSDomainParticipant::create_subscriber (
    const DDS_SubscriberQos & qos,
    DDSSubscriberListener * listener,
    DDS_StatusMask mask) [pure virtual]
```

Creates a [DDSSubscriber](#) with the desired QoS policies and attaches to it the specified [DDSSubscriberListener](#).

The specified QoS policies must be consistent, or the operation will fail and no [DDSSubscriber](#) will be created.

Parameters

<i>qos</i>	<< <i>in</i> >> QoS to be used for creating the new DDSSubscriber . The special value DDS_SUBSCRIBER_QOS_DEFAULT can be used to indicate that the DDSSubscriber should be created with the default DDS_SubscriberQos set in the DDSDomainParticipant .
<i>listener</i>	<< <i>in</i> >> Listener to be attached to the newly created DDSSubscriber .
<i>mask</i>	<< <i>in</i> >> Changes of communication status to be invoked on the listener.

Returns

newly created subscriber object or NULL on failure.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

ARINC 653 API Restriction:

This function *must* only be called before a partition enters *NORMAL* mode.

See also

[Specifying QoS on entities](#) for information on setting QoS before entity creation
[DDS_SubscriberQos](#) for rules on consistency among QoS
[DDS_SUBSCRIBER_QOS_DEFAULT](#)

delete_subscriber()

```
virtual DDS\_ReturnCode\_t DDSDomainParticipant::delete_subscriber (
    DDSSubscriber * s) [pure virtual]
```

Deletes an existing [DDSSubscriber](#).

Precondition

The [DDSSubscriber](#) must not have any attached [DDSDataReader](#) objects. If there are existing [DDSDataReader](#) objects, it will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#)

The [DDSSubscriber](#) must have been created by this [DDSDomainParticipant](#), or else it will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#).

Postcondition

A Listener installed on the [DDSSubscriber](#) will not be called after this method completes successfully.

Parameters

<i>s</i>	<< <i>in</i> >> DDSSubscriber to be deleted.
----------	--

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_PRECONDITION_NOT_MET](#).

create_topic()

```
virtual DDSTopic * DDSDomainParticipant::create_topic (
    const char * topic_name,
    const char * type_name,
    const DDS\_TopicQos & qos,
    DDSTopicListener * listener,
    DDS\_StatusMask mask) [pure virtual]
```

Creates a [DDSTopic](#) with the desired QoS policies

The specified QoS policies must be consistent, or the operation will fail and no [DDSTopic](#) will be created.

Precondition

The application is not allowed to create two [DDSTopicDescription](#) objects with the same `topic_name` attached to the same [DDSDomainParticipant](#). If the application attempts this, this method will fail and return NULL.

Prior to creating a [DDSTopic](#), the type must have been registered with RTI Connex DDS Micro. This is done using the [DDSDomainParticipant::register_type](#) operation on a derived class of the `::DDS_TypePluginI` interface.

Parameters

<i>topic_name</i>	<< <i>in</i> >> Name for the new topic, must not exceed 255 characters. Cannot be NULL.
<i>type_name</i>	<< <i>in</i> >> The type to which the new DDSTopic will be bound. Cannot be NULL.
<i>qos</i>	<< <i>in</i> >> QoS to be used for creating the new DDSTopic . The special value DDS_TOPIC_QOS_DEFAULT can be used to indicate that the DDSTopic should be created with the default DDS_TopicQos set in the DDSDomainParticipant .
<i>listener</i>	<< <i>in</i> >> Listener to be attached to the newly created DDSTopic .
<i>mask</i>	<< <i>in</i> >> Changes of communication status to be invoked on the listener.

Returns

newly created topic, or NULL on failure

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

ARINC 653 API Restriction:

This function *must* only be called before a partition enters *NORMAL* mode.

See also

[Specifying QoS on entities](#) for information on setting QoS before entity creation

[DDS_TopicQos](#) for rules on consistency among QoS

[DDS_TOPIC_QOS_DEFAULT](#)

delete_topic()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::delete_topic (
    DDSTopic * topic) [pure virtual]
```

Deletes a [DDSTopic](#).

Precondition

If the [DDSTopic](#) does not belong to the application's [DDSDomainParticipant](#), this operation fails with [DDS_RETCODE_PRECONDITION_NOT_MET](#).

Make sure no objects are using the topic. More specifically, there must be no existing [DDSDataReader](#), or [DDSDataWriter](#), objects belonging to the same [DDSDomainParticipant](#) that are using the [DDSTopic](#). If `delete_topic` is called on a [DDSTopic](#) with any of these existing objects attached to it, it will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#).

Postcondition

Listener installed on the [DDSTopic](#) will not be called after this method completes successfully.

Parameters

<i>topic</i>	<< <i>in</i> >> DDSTopic to be deleted.
--------------	---

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_PRECONDITION_NOT_MET](#)

find_topic()

```
virtual DDSTopic * DDSDomainParticipant::find_topic (
    const char * topic_name,
    const DDS_Duration_t & timeout) [pure virtual]
```

Finds an existing (or ready to exist) [DDSTopic](#), based on its name.

If a [DDSTopic](#) of the same name already exists, it gives access to it. RTI Connex DDS Micro does not support waiting for a topic to be created. This operation will fail unless the `timeout` is [DDS_DURATION_ZERO](#).

[DDSTopic](#) obtained by [DDSDomainParticipant::find_topic](#) must also be deleted by means of [DDSDomainParticipant::delete_topic](#). If [DDSTopic](#) is obtained multiple times by means of [DDSDomainParticipant::find_topic](#) or [DDSDomainParticipant::create_topic](#), it must also be deleted that same number of times using [DDSDomainParticipant::delete_topic](#)

Parameters

<i>topic_name</i>	<< <i>in</i> >> Name of the DDSTopic to search for. Cannot be NULL.
<i>timeout</i>	<< <i>in</i> >> The time to wait if the DDSTopic doesn't exist already.

Returns

the topic, if it exists, or NULL

get_qos()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::get_qos (  
    DDS_DomainParticipantQos & qos) [pure virtual]
```

Get the participant QoS.

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

Parameters

<i>qos</i>	<< <i>inout</i> >> QoS to be filled up.
------------	---

Returns

One of the [Standard Return Codes](#)

See also

[get_qos \(abstract\)](#)

set_qos()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::set_qos (  
    const DDS_DomainParticipantQos & qos) [pure virtual]
```

Change the QoS of this domain participant.

The [DDS_DomainParticipantQos::entity_factory](#) can be changed. The other policies are immutable.

Parameters

<i>qos</i>	<< <i>in</i> >> Set of policies to be applied to DDSDomainParticipant . Policies must be consistent. Immutable policies cannot be changed after DDSDomainParticipant is enabled. The special value DDS_PARTICIPANT_QOS_DEFAULT can be used to indicate that the QoS of the DDSDomainParticipant should be changed to match the current default DDS_DomainParticipantQos set in the DDSDomainParticipantFactory .
------------	--

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_IMMUTABLE_POLICY](#) if immutable policy is changed, or [DDS_RETCODE_INCONSISTENT_POLICY](#) if policies are inconsistent

See also

[DDS_DomainParticipantQos](#) for rules on consistency among QoS
[set_qos](#) (abstract)

get_default_publisher_qos()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::get_default_publisher_qos (
    DDS_PublisherQos & qos) [pure virtual]
```

Copy the default [DDS_PublisherQos](#) values into the provided [DDS_PublisherQos](#) instance.

The retrieved `qos` will match the set of values specified on the last successful call to [DDSDomainParticipant::set_default_publisher_qos](#), or else, if the call was never made, the default values listed in [DDS_PublisherQos](#).

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

If [DDS_PUBLISHER_QOS_DEFAULT](#) is specified as the `qos` parameter when [DDSDomainParticipant::create_publisher](#) is called, the default value of the QoS set in the factory, equivalent to the value obtained by calling [DDSDomainParticipant::get_default_publisher_qos](#) will be used to create the [DDSPublisher](#).

Parameters

<i>qos</i>	<< <i>out</i> >> Qos to be filled up.
------------	---------------------------------------

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDS_PUBLISHER_QOS_DEFAULT](#)
[DDSDomainParticipant::create_publisher](#)

set_default_publisher_qos()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::set_default_publisher_qos (
    const DDS_PublisherQos & qos) [pure virtual]
```

Set the default [DDS_PublisherQos](#) values for this domain participant.

This default value will be used for newly created [DDSPublisher](#) if [DDS_PUBLISHER_QOS_DEFAULT](#) is specified as the `qos` parameter when [DDSDomainParticipant::create_publisher](#) is called.

The specified QoS policies must be consistent, or else the operation will have no effect and fail with [DDS_RETCODE_INCONSISTENT_POLICY](#).

Parameters

<i>qos</i>	<< <i>in</i> >> Default qos to be set. The special value DDS_PUBLISHER_QOS_DEFAULT may be passed as <code>qos</code> to indicate that the default QoS should be reset back to the initial values the factory would have used if DDSDomainParticipant::set_default_publisher_qos had never been called.
------------	--

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_INCONSISTENT_POLICY](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDS_PUBLISHER_QOS_DEFAULT](#)
[DDSDomainParticipant::create_publisher](#)

get_default_subscriber_qos()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::get_default_subscriber_qos (
    DDS_SubscriberQos & qos) [pure virtual]
```

Copy the default [DDS_SubscriberQos](#) values into the provided [DDS_SubscriberQos](#) instance.

The retrieved `qos` will match the set of values specified on the last successful call to [DDSDomainParticipant::set_default_subscriber_qos](#), or else, if the call was never made, the default values listed in [DDS_SubscriberQos](#).

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

If [DDS_SUBSCRIBER_QOS_DEFAULT](#) is specified as the `qos` parameter when [DDSDomainParticipant::create_subscriber](#) is called, the default value of the QoS set in the factory, equivalent to the value obtained by calling [DDSDomainParticipant::get_default_subscriber_qos](#), will be used to create the [DDSSubscriber](#).

Parameters

<i>qos</i>	<<out>> Qos to be filled up.
------------	------------------------------

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDS_SUBSCRIBER_QOS_DEFAULT](#)

[DDSDomainParticipant::create_subscriber](#)

set_default_subscriber_qos()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::set_default_subscriber_qos (  
    const DDS_SubscriberQos & qos) [pure virtual]
```

Set the default [DDS_SubscriberQos](#) values for this domain participant.

This default value will be used for newly created [DDSSubscriber](#) if [DDS_SUBSCRIBER_QOS_DEFAULT](#) is specified as the *qos* parameter when [DDSDomainParticipant::create_subscriber](#) is called.

The specified QoS policies must be consistent, or else the operation will have no effect and fail with [DDS_RETCODE_INCONSISTENT_POLICY](#).

Parameters

<i>qos</i>	<<in>> Default qos to be set. The special value DDS_SUBSCRIBER_QOS_DEFAULT may be passed as <i>qos</i> to indicate that the default QoS should be reset back to the initial values the factory would have used if DDSDomainParticipant::set_default_subscriber_qos had never been called.
------------	---

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_INCONSISTENT_POLICY](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDS_SUBSCRIBER_QOS_DEFAULT](#)

[DDSDomainParticipant::create_subscriber](#)

get_default_topic_qos()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::get_default_topic_qos (
    DDS_TopicQos & qos) [pure virtual]
```

Copies the default [DDS_TopicQos](#) values for this domain participant into the given [DDS_TopicQos](#) instance.

The retrieved qos will match the set of values specified on the last successful call to [DDSDomainParticipant::set_default_topic_qos](#), or else, if the call was never made, the default values listed in [DDS_TopicQos](#).

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

Parameters

<i>qos</i>	<<out>> Default qos to be retrieved.
------------	--------------------------------------

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDS_TOPIC_QOS_DEFAULT](#)
[DDSDomainParticipant::create_topic](#)

set_default_topic_qos()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::set_default_topic_qos (
    const DDS_TopicQos & qos) [pure virtual]
```

Set the default [DDS_TopicQos](#) values for this domain participant.

This default value will be used for newly created [DDSTopic](#) if [DDS_TOPIC_QOS_DEFAULT](#) is specified as the qos parameter when [DDSDomainParticipant::create_topic](#) is called.

The specified QoS policies must be consistent, or else the operation will have no effect and fail with [DDS_RETCODE_INCONSISTENT_POLICY](#).

Parameters

<i>qos</i>	<< <i>in</i> >> Default qos to be set. The special value DDS_TOPIC_QOS_DEFAULT may be passed as <i>qos</i> to indicate that the default QoS should be reset back to the initial values the factory would have used if DDSDomainParticipant::set_default_topic_qos had never been called.
------------	--

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_INCONSISTENT_POLICY](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDS_TOPIC_QOS_DEFAULT](#)
[DDSDomainParticipant::create_topic](#)

lookup_topicdescription()

```
virtual DDSTopicDescription * DDSDomainParticipant::lookup_topicdescription (
    const char * topic_name) [pure virtual]
```

Lookup an existing locally-created [DDSTopicDescription](#), based on its name.

The returned topic can either be enabled or disabled.

The operation [DDSDomainParticipant::lookup_topicdescription](#) searches among the locally created topics and returns a pointer to a [DDSTopicDescription](#). It is possible to delete the [DDSTopicDescription](#) returned by [DDSDomainParticipant::lookup_topicdescription](#), provided it has no readers or writers, but then it is really deleted and subsequent lookups will fail.

Parameters

<i>topic_name</i>	<< <i>in</i> >> Name of DDSTopicDescription to search for. This string must be no more than 255 characters. Cannot be NULL.
-------------------	---

Returns

the topic description if it has already been created locally, or NULL otherwise.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

lookup_publisher_by_name()

```
virtual DDSPublisher * DDSDomainParticipant::lookup_publisher_by_name (
    const char * publisher_name) [pure virtual]
```

Retrieves a [DDSPublisher](#) by its entity name within this [DDSDomainParticipant](#)

This returned [DDSPublisher](#) is either enabled or disabled.

Every [DDSPublisher](#) in the system has an entity name which is configured and stored in the EntityName policy, ENTITY_NAME.

This operation retrieves a [DDSPublisher](#) within the [DDSDomainParticipant](#) given the entity's name. If there are several [DDSPublisher](#) with the same name within the [DDSDomainParticipant](#), this method returns the first matching occurrence.

Parameters

<i>publisher_name</i>	<< <i>in</i> >> Entity name of the DDSPublisher . Cannot be NULL.
-----------------------	---

Returns

The first [DDSPublisher](#) found with the specified name or NULL if it is not found.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

lookup_subscriber_by_name()

```
virtual DDSSubscriber * DDSDomainParticipant::lookup_subscriber_by_name (
    const char * subscriber_name) [pure virtual]
```

Retrieves a [DDSSubscriber](#) by its entity name within this [DDSDomainParticipant](#)

This returned [DDSSubscriber](#) is either enabled or disabled.

Every [DDSSubscriber](#) in the system has an entity name which is configured and stored in the EntityName policy, ENTITY_NAME.

This operation retrieves a [DDSSubscriber](#) within the [DDSDomainParticipant](#) given the entity's name. If there are several [DDSSubscriber](#) with the same name within the [DDSDomainParticipant](#), this method returns the first matching occurrence.

Parameters

<i>subscriber_name</i>	<< <i>in</i> >> Entity name of the DDSSubscriber . Cannot be NULL.
------------------------	--

Returns

The first [DDSSubscriber](#) found with the specified name or NULL if it is not found.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

lookup_datawriter_by_name()

```
virtual DDSDataWriter * DDSDomainParticipant::lookup_datawriter_by_name (
    const char * datawriter_full_name) [pure virtual]
```

Retrieves a [DDSDataWriter](#) by its entity name within this [DDSDomainParticipant](#)

This returned [DDSDataWriter](#) is either enabled or disabled.

Every [DDSDataWriter](#) in the system has an entity name which is configured and stored in the EntityName policy, ENTITY_NAME.

This operation retrieves a [DDSDataWriter](#) within the [DDSDomainParticipant](#) given the entity's name. If there are several [DDSDataWriter](#) with the same name within the [DDSDomainParticipant](#), this method returns the first matching occurrence.

Parameters

<i>datawriter_full_name</i>	<<in>> Entity name of the DDSDataWriter . Cannot be NULL.
-----------------------------	---

Returns

The first [DDSDataWriter](#) found with the specified name or NULL if it is not found.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

lookup_datareader_by_name()

```
virtual DDSDataReader * DDSDomainParticipant::lookup_datareader_by_name (
    const char * datareader_full_name) [pure virtual]
```

Retrieves a [DDSDataReader](#) by its entity name within this [DDSDomainParticipant](#)

This returned [DDSDataReader](#) is either enabled or disabled.

Every [DDSDataReader](#) in the system has an entity name which is configured and stored in the EntityName policy, ENTITY_NAME.

This operation retrieves a [DDSDataReader](#) within the [DDSDomainParticipant](#) given the entity's name. If there are several [DDSDataReader](#) with the same name within the [DDSDomainParticipant](#), this method returns the first matching occurrence.

Parameters

<i>datareader_full_name</i>	<<in>> Entity name of the DDSDataReader . Cannot be NULL.
-----------------------------	---

Returns

The first [DDSDataReader](#) found with the specified name or NULL if it is not found.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

assert_liveliness()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::assert_liveliness () [pure virtual]
```

Manually asserts the liveliness of this [DDSDomainParticipant](#).

This is used in combination with the [DDS_LivelinessQosPolicy](#) to indicate to RTI Connex DDS Micro that the entity remains active.

You need to use this operation if the [DDSDomainParticipant](#) contains [DDSDataWriter](#) entities with the [DDS_LivelinessQosPolicy::kind](#) set to [DDS_MANUAL_BY_PARTICIPANT_LIVELINESS_QOS](#) and it only affects the liveliness of those [DDSDataWriter](#) entities. Otherwise, it has no effect.

Note: writing data via the [FooDataWriter::write](#) or [FooDataWriter::write_w_timestamp](#) operation asserts liveliness on the [DDSDataWriter](#) itself and its [DDSDomainParticipant](#). Consequently the use of [assert_liveliness\(\)](#) is only needed if the application is not writing data regularly.

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_NOT_ENABLED](#)

See also

[DDS_LivelinessQosPolicy](#)

delete_contained_entities()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::delete_contained_entities () [pure virtual]
```

Delete all the entities that were created by means of the "create" operations on the [DDSDomainParticipant](#).

This operation deletes all contained [DDSPublisher](#), [DDSSubscriber](#), and [DDSTopic](#).

Prior to deleting each contained entity, this operation will recursively call the corresponding [delete_contained_entities](#) operation on each contained entity (if applicable). This pattern is applied recursively. In this manner the operation [delete_contained_entities\(\)](#) on the [DDSDomainParticipant](#) will end up deleting all the entities recursively contained in the [DDSDomainParticipant](#), that is also the [DDSDataWriter](#), and [DDSDataReader](#).

The operation will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#) if any of the contained entities is in a state where it cannot be deleted.

If [delete_contained_entities\(\)](#) completes successfully, the application may delete the [DDSDomainParticipant](#) knowing that it has no contained entities.

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_PRECONDITION_NOT_MET](#).

register_type()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::register_type (
    const char * type_name,
    DDS_TypePluginI * plugin) [pure virtual]
```

<<*eXtension*>> Allows an application to communicate to RTI Connex DDS Micro the existence of a data type.

The *generated* implementation of the operation embeds all the knowledge that has to be communicated to the middle-ware in order to make it able to manage the contents of data of that type. This includes in particular the key definition that will allow RTI Connex DDS Micro to distinguish different instances of the same type.

The same `::DDS_TypePluginI` can be registered multiple times with a `DDSDomainParticipant` using the same or different values for the `type_name`. If `register_type` is called multiple times on the same `::DDS_TypePluginI` with the same `DDSDomainParticipant` and `type_name`, the second (and subsequent) registrations are ignored and the operation succeeds with `DDS_RETCODE_OK`.

Precondition

Cannot use the same `type_name` to register two different `::DDS_TypePluginI` with the same `DDSDomainParticipant`, or else the operation will fail and `DDS_RETCODE_PRECONDITION_NOT_MET` will be returned.

Parameters

<i>type_name</i>	<< <i>in</i> >> the type name under which the data type <code>Foo</code> is registered with the participant; this type name is used when creating a new <code>DDSTopic</code> . (See <code>DDSDomainParticipant::create_topic</code> .) The name may not be longer than 255 characters. If name is NULL, the typename specified in IDL is used.
<i>plugin</i>	<< <i>in</i> >> the <code>Foo</code> type plugin to register the data type with. Cannot be NULL.

Returns

One of the [Standard Return Codes](#), `DDS_RETCODE_PRECONDITION_NOT_MET` or `DDS_RETCODE_OUT_OF_RESOURCES`.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDSDomainParticipant::create_topic](#)

unregister_type()

```
virtual DDS_TypePluginI * DDSDomainParticipant::unregister_type (
    const char * type_name) [pure virtual]
```

Allows an application to unregister a data type from RTI Connex DDS Micro. After calling `unregister_type`, no further communication using that type is possible.

The *generated* implementation of the operation removes all the information about a type from RTI Connex DDS Micro. No further communication using that type is possible.

Precondition

A type with `type_name` is registered with the participant and all [DDSTopic](#) objects referencing the type have been destroyed. If the type is not registered with the participant, or if any [DDSTopic](#) is associated with the type, the operation will fail with [DDS_RETCODE_ERROR](#).

Postcondition

All information about the type is removed from RTI Connex DDS Micro. No further communication using this type is possible.

Parameters

<i>type_name</i>	<< <i>in</i> >> the type name under which the data type <code>Foo</code> is registered with the participant. The name should match a name that has been previously used to register a type with the participant. Cannot be NULL.
------------------	--

Returns

The previously registered type plugin on success, or NULL on failure.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDSDomainParticipant::register_type](#)

set_listener()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::set_listener (
    const DDSDomainParticipantListener * listener,
    DDS_StatusMask mask) [pure virtual]
```

Sets the participant listener.

Parameters

<i>listener</i>	<<in>> Listener to be installed on entity.
<i>mask</i>	<<in>> Changes of communication status to be invoked on the listener.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

Returns

One of the [Standard Return Codes](#)

See also

[set_listener](#) (abstract)

get_listener()

```
virtual DDSDomainParticipantListener * DDSDomainParticipant::get_listener () [pure virtual]
```

Get the participant listener.

Returns

Existing listener attached to the [DDSDomainParticipant](#).

get_discovered_participants()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::get_discovered_participants (
    DDS_InstanceHandleSeq & participant_handles) [pure virtual]
```

Returns a sequence containing [DDS_InstanceHandle_t](#) for all the remote [DDSDomainParticipant](#) that have been discovered by the [DDSDomainParticipant](#).

Parameters

<i>participant_handles</i>	<<out>> sequence of DDS_InstanceHandle_t where the result will be stored.
----------------------------	---

Returns

One of the [Standard Return Codes](#)

get_discovered_participant_data()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::get_discovered_participant_data (
    DDS_ParticipantBuiltinTopicData & participant_data,
    const DDS_InstanceHandle_t & participant_handle) [pure virtual]
```

Returns information about a remote [DDSDomainParticipant](#) that has been discovered by the [DDSDomainParticipant](#).

Parameters

<i>participant_data</i>	<<out>> information about the remote DDSDomainParticipant .
<i>participant_handle</i>	<<in>> DDS_InstanceHandle_t identifying the remote DDSDomainParticipant .

Returns

One of the [Standard Return Codes](#)

get_current_time()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::get_current_time (
    DDS_Time_t & current_time) [pure virtual]
```

Returns the current value of the time.

The current value of the time that RTI Connex Micro uses to time-stamp [DDSDDataWriter](#) and to set the reception-timestamp for the data updates that it receives. Note that the definition of the current time is dependent on the platform and may not be the actual `wall-clock` time. Additional information for specific platforms can be found in the User's Manual.

Parameters

<i>current_time</i>	<<out>> Current time to be filled up.
---------------------	---------------------------------------

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

add_peer()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::add_peer (
    const char * peer) [pure virtual]
```

<<eXtension>> Attempt to contact one or more additional peer participants.

Add the given peer description to the list of peers with which this [DDSDomainParticipant](#) will try to communicate.

This method may be called at any time after this [DDSDomainParticipant](#) has been created, before or after it has been enabled. If it is called after [DDSEntity::enable](#), an attempt will be made to contact the new peer(s) immediately. If it is called before, the peer description will simply be added to the list that was populated by [DDS_DiscoveryQosPolicy::initial_peers](#); the first attempted contact will take place after this [DDSDomainParticipant](#) is enabled.

Adding a peer description with this method does not guarantee that any peer(s) discovered as a result will exactly correspond to those described:

- This [DDSDomainParticipant](#) will attempt to discover peer participants at the given locations, but may not succeed if no such participants are available. Such a situation will not result in an error result from this method, which will not wait for contact attempt(s) to be made.
- If remote participants such as are described by the given peer description are discovered, the distributed application is configured with asymmetric peer lists, and [DDS_DiscoveryQosPolicy::accept_unknown_peers](#) is set to [DDS_BOOLEAN_TRUE](#) this [DDSDomainParticipant](#) may actually discover *more* peers than are described in the given peer description.

Adding a peer description with this method has no effect on [DDS_DiscoveryQosPolicy::initial_peers](#).

Parameters

<i>peer</i>	<< <i>in</i> >> New peer descriptor to be added.
-------------	--

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDS_DiscoveryQosPolicy::initial_peers](#)

get_default_flowcontroller_property()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::get_default_flowcontroller_property (
    DDS_FlowControllerProperty_t & prop) [pure virtual]
```

<<*eXtension*>> Copies the default [DDS_FlowControllerProperty_t](#) values for this domain participant into the given [DDS_FlowControllerProperty_t](#) instance.

The retrieved `prop` will match the set of values specified on the last successful call to [DDSDomainParticipant::set_default_flowcontroller_p](#) or else, if the call was never made, the default values listed in [DDS_FlowControllerProperty_t](#).

MT Safety:

UNSAFE. It is not safe to retrieve the default flow controller properties from a DomainParticipant while another thread may be simultaneously calling [DDSDomainParticipant::set_default_flowcontroller_property](#)

Parameters

<i>prop</i>	<< <i>out</i> >> Default property to be retrieved.
-------------	--

Returns

One of the [Standard Return Codes](#)

See also

[DDS_FLOW_CONTROLLER_PROPERTY_DEFAULT](#)
[DDSDomainParticipant::create_flowcontroller](#)

set_default_flowcontroller_property()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::set_default_flowcontroller_property (
    const DDS_FlowControllerProperty_t & prop) [pure virtual]
```

<<*eXtension*>> Set the default [DDS_FlowControllerProperty_t](#) values for this domain participant.

This default value will be used for newly created [DDSFlowController](#) if [DDS_FLOW_CONTROLLER_PROPERTY_DEFAULT](#) is specified as the `prop` parameter when [DDSDomainParticipant::create_flowcontroller](#) is called.

The specified property values must be consistent, or else the operation will have no effect and fail with [DDS_RETCODE_INCONSISTENT_POLICY](#).

MT Safety:

UNSAFE. It is not safe to set the default flow controller properties for a DomainParticipant while another thread may be simultaneously calling [DDSDomainParticipant::set_default_flowcontroller_property](#), [DDSDomainParticipant::get_default_flowcontroller_property](#) or calling [DDSDomainParticipant::create_flowcontroller](#) with [DDS_FLOW_CONTROLLER_PROPERTY_DEFAULT](#) as the `prop` parameter.

Parameters

<i>prop</i>	<< <i>in</i> >> Default property to be set. The special value DDS_FLOW_CONTROLLER_PROPERTY_DEFAULT may be passed as <code>prop</code> to indicate that the default property should be reset to the default values the factory would use if DDSDomainParticipant::set_default_flowcontroller_property had never been called.
-------------	---

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_INCONSISTENT_POLICY](#)

See also

[DDS_FLOW_CONTROLLER_PROPERTY_DEFAULT](#)
[DDSDomainParticipant::create_flowcontroller](#)

create_flowcontroller()

```
virtual DDSFlowController * DDSDomainParticipant::create_flowcontroller (
    const char * name,
    const DDS_FlowControllerProperty_t & prop) [pure virtual]
```

<<*eXtension*>> Creates a [DDSFlowController](#) with the desired property.

MT Safety:

UNSAFE. If [DDS_FLOW_CONTROLLER_PROPERTY_DEFAULT](#) is used for `prop`, it is not safe to create the flow controller while another thread may be simultaneously calling [DDSDomainParticipant::set_default_flowcontroller_property](#) or trying to lookup that flow controller with [DDSDomainParticipant::lookup_flowcontroller](#).

Parameters

<i>name</i>	<< <i>in</i> >> name of the DDSFlowController to create. A DDSDataWriter is associated with a DDSFlowController by name. Limited to 255 characters.
<i>prop</i>	<< <i>in</i> >> property to be used for creating the new DDSFlowController . The special value DDS_FLOW_CONTROLLER_PROPERTY_DEFAULT can be used to indicate that the DDSFlowController should be created with the default DDS_FlowControllerProperty_t set in the DDSDomainParticipant .

Returns

Newly created flow controller object or NULL on failure.

See also

[DDS_FlowControllerProperty_t](#) for rules on consistency among property

[DDS_FLOW_CONTROLLER_PROPERTY_DEFAULT](#)

[DDSDomainParticipant::get_default_flowcontroller_property](#)

The created [DDSFlowController](#) is associated with a [DDSDataWriter](#) via [DDS_PublishModeQosPolicy::flow_controller_name](#). A single FlowController may service multiple [DDSDataWriter](#) instances, even if they belong to a different [DDSPublisher](#). The *prop* determines how the FlowController shapes the network traffic.

The specified *prop* must be consistent, or the operation will fail and no [DDSFlowController](#) will be created.

delete_flowcontroller()

```
virtual DDS\_ReturnCode\_t DDSDomainParticipant::delete_flowcontroller (
    DDSFlowController * fc) [pure virtual]
```

<<*eXtension*>> Deletes an existing [DDSFlowController](#).

Precondition

The [DDSFlowController](#) must not have any attached [DDSDataWriter](#) objects. If there are any attached [DDSDataWriter](#) objects, it will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#).

The [DDSFlowController](#) must have been created by this [DDSDomainParticipant](#), or else it will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#).

Postcondition

The [DDSFlowController](#) is deleted if this method completes successfully.

MT Safety:

UNSAFE. It is not safe to delete an entity while another thread may be simultaneously calling an API that uses the entity.

Parameters

<i>fc</i>	<< <i>in</i> >> The DDSFlowController to be deleted.
-----------	--

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_PRECONDITION_NOT_MET](#).

lookup_flowcontroller()

```
virtual DDSFlowController * DDSDomainParticipant::lookup_flowcontroller (
    const char * name) [pure virtual]
```

<<*eXtension*>> Looks up an existing locally-created [DDSFlowController](#), based on its name.

Looks up a previously created [DDSFlowController](#), including the built-in ones. Once a [DDSFlowController](#) has been deleted, subsequent lookups will fail.

MT Safety:

UNSAFE. It is not safe to lookup a flow controller description while another thread is creating that flow controller.

Parameters

<i>name</i>	<< <i>in</i> >> Name of DDSFlowController to search for. Limited to 255 characters. Cannot be NULL.
-------------	---

Returns

The flow controller if it has already been created locally, or NULL otherwise.

7.101 DDSDomainParticipantFactory Class Reference

<<*singleton*>> <<*interface*>> Allows creation and destruction of [DDSDomainParticipant](#) objects.

```
#include <dds_cpp_domain.hxx>
```

Public Member Functions

- virtual `DDSDomainParticipant * create_participant (DDS_DomainId_t domainId, const DDS_DomainParticipantQos &qos, DDSDomainParticipantListener *listener, DDS_StatusMask mask)=0`
Creates a new *DDSDomainParticipant* object.
- virtual `DDS_ReturnCode_t delete_participant (DDSDomainParticipant *a_participant)=0`
Deletes an existing *DDSDomainParticipant*.
- virtual `DDSDomainParticipant * create_participant_from_config (const char *configuration_name)=0`
<<eXtension>> Creates a new *DDSDomainParticipant* given its configuration name from a description provided in configuration file.
- virtual `DDSDomainParticipant * lookup_participant (DDS_DomainId_t domainId)=0`
Locates an existing *DDSDomainParticipant*.
- virtual `DDSDomainParticipant * lookup_participant_by_name (const char *participant_name)=0`
Retrieves a *DDSDomainParticipant* by its entity name within the *DDSDomainParticipantFactory*.
- virtual `DDS_ReturnCode_t set_default_participant_qos (const DDS_DomainParticipantQos &qos)=0`
Sets the default *DDS_DomainParticipantQos* values for this domain participant factory.
- virtual `DDS_ReturnCode_t get_default_participant_qos (DDS_DomainParticipantQos &qos)=0`
Initializes the *DDS_DomainParticipantQos* instance with default values.
- virtual `DDS_ReturnCode_t get_qos (DDS_DomainParticipantFactoryQos &qos)=0`
Gets the value for participant factory QoS.
- virtual `DDS_ReturnCode_t set_qos (const DDS_DomainParticipantFactoryQos &qos)=0`
Sets the value for a participant factory QoS.
- virtual `RTRegistry * get_registry ()=0`
Retrieves the singleton registry instance associated with the factory.

Static Public Member Functions

- static `DDSDomainParticipantFactory * get_instance ()`
Gets the singleton instance of this class.
- static `DDS_ReturnCode_t finalize_instance ()`
<<eXtension>> Destroys the singleton instance of this class.

7.101.1 Detailed Description

<<singleton>> <<interface>> Allows creation and destruction of *DDSDomainParticipant* objects.

The sole purpose of this class is to allow the creation and destruction of *DDSDomainParticipant* objects. This class itself is a <<singleton>>, and accessed via the `get_instance()` method, and destroyed with `finalize_instance()` method.

A single application can participate in multiple domains by instantiating multiple *DDSDomainParticipant* objects.

An application may even instantiate multiple participants in the same domain. Participants in the same domain exchange data in the same way regardless of whether they are in the same application or different applications or on the same node or different nodes; their location is transparent.

There are two important caveats:

- With multiple participants in the same domain on the same node (in the same application or different applications), each participant must be assigned its own receive ports. The receive ports are calculated based on the domain ID and the participant index set in the [DDS_WireProtocolQosPolicy](#). The default index (-1) causes the participant to automatically search for the next available index, starting at 0, when it is created. If an index larger or equal to 0 is assigned then only that value will be used. If another participant with the same index already exists the participant creation will fail.

NOTE: An exception to this rule is multicast ports. Multicast ports are shared between participants in the same domain on the same host). Thus, the index value is ignored.

- You cannot mix entities from different participants. For example, you cannot delete a topic on a different participant than you created it from, and you cannot ask a subscriber to create a reader for a topic created from a participant different than the subscriber's own participant. (Note that it is permissible for an application built on top of RTI Connex DDS Micro to know about entities from different participants. For example, an application could keep references to a reader from one domain and a writer from another and then bridge the domains by writing the data received in the reader callback.)

See also

[DDSDomainParticipant](#)

7.101.2 Member Function Documentation

get_instance()

```
static DDSDomainParticipantFactory * DDSDomainParticipantFactory::get_instance () [static]
```

Gets the singleton instance of this class.

MT Safety:

UNSAFE on the FIRST call to [DDSDomainParticipantFactory::get_instance\(\)](#). It is not safe for two threads to simultaneously make the first call to get an instance of the factory. Subsequent calls are thread safe.

DDSTheParticipantFactory can be used as an alias for the singleton factory returned by this operation.

[DDSDomainParticipantFactory::get_instance](#) also initializes the RTI Connex DDS Micro library and should be called before any other API, unless the API is explicitly mentioned as safe to call before [DDSDomainParticipantFactory::get_instance](#).

Returns

the singleton [DDSDomainParticipantFactory](#) instance.

ARINC 653 API Restriction:

This function *must* only be called before a partition enters *NORMAL* mode.

See also

DDSTheParticipantFactory

finalize_instance()

```
static DDS_ReturnCode_t DDSDomainParticipantFactory::finalize_instance () [static]
```

<<*eXtension*>> Destroys the singleton instance of this class.

Only necessary to explicitly reclaim resources used by the participant factory singleton. Note that on many OSes, these resources are automatically reclaimed by the OS when the program terminates. Some memory checker tools still flag these as unreclaimed however. So this method provides a way to clean up memory used by the participant factory.

Precondition

All participants created from the factory have been deleted.

Postcondition

All resources belonging to the factory reclaimed. Another call to [DDSDomainParticipantFactory::get_instance](#) will return a new lifecycle of the singleton.

MT Safety:

UNSAFE for two threads to simultaneously call [DDSDomainParticipantFactory::finalize_instance\(\)](#) or [DDSDomainParticipantFactory::get_instance\(\)](#) at the same time.

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_PRECONDITION_NOT_MET](#)

create_participant()

```
virtual DDSDomainParticipant * DDSDomainParticipantFactory::create_participant (
    DDS_DomainId_t domainId,
    const DDS_DomainParticipantQos & qos,
    DDSDomainParticipantListener * listener,
    DDS_StatusMask mask) [pure virtual]
```

Creates a new [DDSDomainParticipant](#) object.

The specified QoS policies must be consistent, or the operation will fail and no [DDSDomainParticipant](#) will be created.

If you want to create multiple participants on a given host in the same domain, make sure each one has a different participant index (set in the [DDS_WireProtocolQosPolicy](#)). This in turn will ensure each participant uses a different port number (since the unicast port numbers are calculated from the participant index and the domain ID).

The default participant index value (-1) causes the participant to automatically search for the first available participant index, starting at 0, when it is created. This is appropriate for most use cases, including when there are multiple participants per host in the same domain.

Parameters

<i>domain↔ id</i>	<<in>> ID of the domain that the application intends to join. [range] [≥ 0], and does not violate guidelines stated in DDS_RtpsWellKnownPorts_t .
<i>qos</i>	<<in>> the domain participant's QoS. The special value DDS_PARTICIPANT_QOS_DEFAULT can be used to indicate that the DDSDomainParticipant should be created with the default DDS_DomainParticipantQos set in the DDSDomainParticipantFactory .
<i>listener</i>	<<in>> the domain participant's listener.
<i>mask</i>	<<in>> Changes of communication status to be invoked on the listener.

Returns

domain participant or NULL on failure

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

ARINC 653 API Restriction:

This function *must* only be called before a partition enters *NORMAL* mode.

See also

[Specifying QoS on entities](#) for information on setting QoS before entity creation

[DDS_DomainParticipantQos](#) for rules on consistency among QoS

[DDS_PARTICIPANT_QOS_DEFAULT](#)

delete_participant()

```
virtual DDS_ReturnCode_t DDSDomainParticipantFactory::delete_participant (
    DDSDomainParticipant * a_participant) [pure virtual]
```

Deletes an existing [DDSDomainParticipant](#).

Precondition

All domain entities belonging to the participant must have already been deleted. Otherwise it fails with the error [DDS_RETCODE_PRECONDITION_NOT_MET](#).

Postcondition

Listener installed on the [DDSDomainParticipant](#) will not be called after this method returns successfully.

Parameters

<i>a_participant</i>	<<in>> DDSDomainParticipant to be deleted.
----------------------	--

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_PRECONDITION_NOT_MET](#).

create_participant_from_config()

```
virtual DDSDomainParticipant * DDSDomainParticipantFactory::create_participant_from_config (
    const char * configuration_name) [pure virtual]
```

<<eXtension>> Creates a new [DDSDomainParticipant](#) given its configuration name from a description provided in configuration file.

This operation creates a [DDSDomainParticipant](#) and all the contained entities ([DDSTopic](#), [DDSPublisher](#), [DDSSubscriber](#), [DDSDataWriter](#), [DDSDataReader](#)) from a description given in a configuration file.

The configuration name is the fully qualified name of the application, consisting of the name of the application library plus the name of application configuration.

For example the name "MyApplicationLibrary::PublicationParticipant" can be used to create the application from the configuration with library name "MyApplicationLibrary" and participant name "PublicationParticipant"

The entities belonging to the newly created [DDSDomainParticipant](#) can be retrieved with the help of lookup operations such as: [DDSDomainParticipant::lookup_publisher_by_name](#).

Parameters

<i>configuration_name</i>	<<in>> Name of RTI Connex Micro application configuration in the configuration file. Cannot be NULL.
---------------------------	---

Returns

Pointer to [DDSDomainParticipant](#) or NULL on failure

lookup_participant()

```
virtual DDSDomainParticipant * DDSDomainParticipantFactory::lookup_participant (
    DDS_DomainId_t domainId) [pure virtual]
```

Locates an existing [DDSDomainParticipant](#).

If no such [DDSDomainParticipant](#) exists, the operation will return NULL value.

If multiple [DDSDomainParticipant](#) entities belonging to that domainId exist, then the operation will return one of them. It is not specified which one.

Parameters

<i>domain↔ id</i>	<<in>> ID of the domain participant to lookup.
-----------------------	--

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

Returns

domain participant if it exists, or NULL

lookup_participant_by_name()

```
virtual DDSDomainParticipant * DDSDomainParticipantFactory::lookup_participant_by_name (
    const char * participant_name) [pure virtual]
```

Retrieves a [DDSDomainParticipant](#) by its entity name within the [DDSDomainParticipantFactory](#).

This returned [DDSDomainParticipant](#) is either enabled or disabled.

Every [DDSDomainParticipant](#) in the system has an entity name which is configured and stored in the EntityName policy, ENTITY_NAME.

This operation retrieves a [DDSDomainParticipant](#) within the [DDSDomainParticipantFactory](#) given the entity's name. If there are several [DDSDomainParticipant](#) with the same name within the [DDSDomainParticipantFactory](#), this method returns the first matching occurrence.

Parameters

<i>participant_name</i>	<<in>> Entity name of the DDSDomainParticipant . Cannot be NULL.
-------------------------	--

Returns

The first [DDSDomainParticipant](#) found with the specified name or NULL if it is not found.

MT Safety:

UNSAFE. It is not safe to lookup a [DDSDomainParticipant](#) in one thread while another thread is simultaneously creating or destroying that [DDSDomainParticipant](#).

set_default_participant_qos()

```
virtual DDS_ReturnCode_t DDSDomainParticipantFactory::set_default_participant_qos (
    const DDS_DomainParticipantQos & qos) [pure virtual]
```

Sets the default [DDS_DomainParticipantQos](#) values for this domain participant factory.

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

Subsequent participants created from this factory that specify [DDS_PARTICIPANT_QOS_DEFAULT](#) will use the QoS values set by this operation.

Parameters

<i>qos</i>	<< <i>in</i> >> Qos to be used by the DDSDomainParticipant . The special value DDS_PARTICIPANT_QOS_DEFAULT may be passed as <i>qos</i> to indicate that the default QoS should be reset back to the initial values the factory would have used if DDSDomainParticipantFactory::set_default_participant_qos had never been called.
------------	---

Returns

One of the [Standard Return Codes](#)

See also

[DDS_PARTICIPANT_QOS_DEFAULT](#)
[DDSDomainParticipantFactory::create_participant](#)

get_default_participant_qos()

```
virtual DDS_ReturnCode_t DDSDomainParticipantFactory::get_default_participant_qos (
    DDS_DomainParticipantQos & qos) [pure virtual]
```

Initializes the [DDS_DomainParticipantQos](#) instance with default values.

The retrieved *qos* will match the set of values specified on the last successful call to [DDSDomainParticipantFactory::set_default_participant_qos](#) or else, if the call was never made, the default values listed in [DDS_DomainParticipantQos](#).

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

Parameters

<i>qos</i>	<< <i>out</i> >> the domain participant's QoS
------------	---

Returns

One of the [Standard Return Codes](#)

See also

[DDS_PARTICIPANT_QOS_DEFAULT](#)
[DDSDomainParticipantFactory::create_participant](#)

get_qos()

```
virtual DDS_ReturnCode_t DDSDomainParticipantFactory::get_qos (
    DDS_DomainParticipantFactoryQos & qos) [pure virtual]
```

Gets the value for participant factory QoS.

Parameters

<i>qos</i>	<< <i>inout</i> >> QoS to be filled up.
------------	---

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

set_qos()

```
virtual DDS_ReturnCode_t DDSDomainParticipantFactory::set_qos (
    const DDS_DomainParticipantFactoryQos & qos) [pure virtual]
```

Sets the value for a participant factory QoS.

The [DDS_DomainParticipantFactoryQos::entity_factory](#) can be changed. The other policies are immutable.

Note that despite having QoS, the [DDSDomainParticipantFactory](#) is not an [DDSEntity](#).

Note that this method may cause RTI Connex DDS Micro to free and reallocate memory, depending on the QoS policies that are changed.

Parameters

<i>qos</i>	<< <i>in</i> >> Set of policies to be applied to DDSDomainParticipantFactory . Policies must be consistent. Immutable policies can only be changed before calling any other RTI Connex DDS Micro methods except for DDSDomainParticipantFactory::get_qos
------------	--

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_IMMUTABLE_POLICY](#) if immutable policy is changed, or [DDS_RETCODE_INCONSISTENT_POLICY](#) if policies are inconsistent

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDS_DomainParticipantFactoryQos](#) for rules on consistency among QoS

get_registry()

```
virtual RTRegistry * DDSDomainParticipantFactory::get_registry () [pure virtual]
```

Retrieves the singleton registry instance associated with the factory.

Returns

the registry instance or NULL on failure.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.102 DDSDomainParticipantListener Class Reference

<<[interface](#)>> Listener for a [DDSDomainParticipant](#) and all of its child entities.

```
#include <dds_cpp_domain.hxx>
```

Public Member Functions

- virtual void [on_data_available](#) ([DDSDataReader](#) *)
Handles the [DDS_DATA_AVAILABLE_STATUS](#) communication status.
- virtual void [on_requested_deadline_missed](#) ([DDSDataReader](#) *reader, const [DDS_RequestedDeadlineMissedStatus](#) &status)
Handles the [DDS_REQUESTED_DEADLINE_MISSED_STATUS](#) communication status.
- virtual void [on_liveliness_changed](#) ([DDSDataReader](#) *reader, const [DDS_LivelinessChangedStatus](#) &status)
Handles the [DDS_LIVELINESS_CHANGED_STATUS](#) communication status.
- virtual void [on_requested_incompatible_qos](#) ([DDSDataReader](#) *reader, const [DDS_RequestedIncompatibleQosStatus](#) &status)
Handles the [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#) communication status.
- virtual void [on_sample_rejected](#) ([DDSDataReader](#) *reader, const [DDS_SampleRejectedStatus](#) &status)
Handles the [DDS_SAMPLE_REJECTED_STATUS](#) communication status.
- virtual void [on_subscription_matched](#) ([DDSDataReader](#) *reader, const [DDS_SubscriptionMatchedStatus](#) &status)
Handles the [DDS_SUBSCRIPTION_MATCHED_STATUS](#) communication status.
- virtual void [on_sample_lost](#) ([DDSDataReader](#) *reader, const [DDS_SampleLostStatus](#) &status)
Handles the [DDS_SAMPLE_LOST_STATUS](#) communication status.
- virtual void [on_instance_replaced](#) ([DDSDataReader](#) *reader, const [DDS_DataReaderInstanceReplacedStatus](#) &status)
Handles the [DDS_INSTANCE_REPLACED_STATUS](#) communication status.
- virtual [DDS_Boolean](#) [on_before_sample_deserialize](#) ([DDSDataReader](#) *reader, [NDDS_Type_Plugin](#) *, [CDR_Stream_t](#) *stream, [DDS_Boolean](#) *)
Callback to filter a received sample based on serialized data

- virtual [DDS_Boolean](#) [on_before_sample_commit](#) ([DDSDataReader](#) *, const void *const, const struct [DDS_SampleInfo](#) *const, [DDS_Boolean](#) *)
Callback to filter a received sample based on deserialized data
- virtual void [on_data_on_readers](#) ([DDSSubscriber](#) *)
Handles the [DDS_DATA_ON_READERS_STATUS](#) communication status.
- virtual void [on_offered_deadline_missed](#) ([DDSDataWriter](#) *writer, const struct [DDS_OfferedDeadlineMissedStatus](#) &status)
Handles the [DDS_OFFERED_DEADLINE_MISSED_STATUS](#) status.
- virtual void [on_liveliness_lost](#) ([DDSDataWriter](#) *writer, const struct [DDS_LivelinessLostStatus](#) &status)
Handles the [DDS_LIVELINESS_LOST_STATUS](#) status.
- virtual void [on_offered_incompatible_qos](#) ([DDSDataWriter](#) *writer, const struct [DDS_OfferedIncompatibleQosStatus](#) &status)
Handles the [DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#) status.
- virtual void [on_publication_matched](#) ([DDSDataWriter](#) *writer, const struct [DDS_PublicationMatchedException](#) &status)
Handles the [DDS_PUBLICATION_MATCHED_STATUS](#) status.
- virtual void [on_reliable_reader_activity_changed](#) ([DDSDataWriter](#) *writer, const struct [DDS_ReliableReaderActivityChangedStatus](#) &status)
<<extension>> A matched reliable reader has become active or become inactive.
- virtual void [on_inconsistent_topic](#) ([DDSTopic](#) *topic, const [DDS_InconsistentTopicStatus](#) &status)
Handle the [DDS_INCONSISTENT_TOPIC_STATUS](#) status.

7.102.1 Detailed Description

<<*interface*>> Listener for a [DDSDomainParticipant](#) and all of its child entities.

Entity:

[DDSDomainParticipant](#)

Status:

[Status Kinds](#)

This is the interface that can be implemented by an application-provided class and then registered with the [DDSDomainParticipant](#) such that the application can be notified by RTI Connext DDS Micro of relevant status changes.

The [DDSDomainParticipantListener](#) interface extends all other Listener interfaces and has no additional operation beyond the ones defined by the more general listeners.

The purpose of the [DDSDomainParticipantListener](#) is to be the listener of last resort that is notified of all status changes not captured by more specific listeners attached to the [DDSDomainEntity](#) objects. When a relevant status change occurs, RTI Connext DDS Micro will first attempt to notify the listener attached to the concerned [DDSDomainEntity](#) if one is installed. Otherwise, RTI Connext DDS Micro will notify the Listener attached to the [DDSDomainParticipant](#).

Important: Because a [DDSDomainParticipantListener](#) may receive callbacks pertaining to many different entities, it is possible for the same listener to receive multiple callbacks simultaneously in different threads. (Such is not the case for listeners of other types.) It is therefore critical that users of this listener provide their own protection for any thread-unsafe activities undertaken in a [DDSDomainParticipantListener](#) callback.

See also

[DDSTheme](#)

[DDSDomainParticipant::set_listener](#)

RTI Connext DDS Micro C++ API avoids the use of multiple inheritance for certification purposes. For this reason, [DDSDomainParticipantListener](#) is not declared as a sub-class of [DDSPublisherListener](#), [DDSSubscriberListener](#), and [DDSTopicListener](#), but it explicitly declares all supported call-backs.

7.102.2 Member Function Documentation

on_data_available()

```
virtual void DDSDomainParticipantListener::on_data_available (
    DDSDataReader * ) [inline], [virtual]
```

Handles the [DDS_DATA_AVAILABLE_STATUS](#) communication status.

on_requested_deadline_missed()

```
virtual void DDSDomainParticipantListener::on_requested_deadline_missed (
    DDSDataReader * reader,
    const DDS_RequestedDeadlineMissedStatus & status) [inline], [virtual]
```

Handles the [DDS_REQUESTED_DEADLINE_MISSED_STATUS](#) communication status.

on_liveliness_changed()

```
virtual void DDSDomainParticipantListener::on_liveliness_changed (
    DDSDataReader * reader,
    const DDS_LivelinessChangedStatus & status) [inline], [virtual]
```

Handles the [DDS_LIVELINESS_CHANGED_STATUS](#) communication status.

on_requested_incompatible_qos()

```
virtual void DDSDomainParticipantListener::on_requested_incompatible_qos (
    DDSDataReader * reader,
    const DDS_RequestedIncompatibleQosStatus & status) [inline], [virtual]
```

Handles the [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#) communication status.

on_sample_rejected()

```
virtual void DDSDomainParticipantListener::on_sample_rejected (
    DDSDataReader * reader,
    const DDS_SampleRejectedStatus & status) [inline], [virtual]
```

Handles the [DDS_SAMPLE_REJECTED_STATUS](#) communication status.

on_subscription_matched()

```
virtual void DDSDomainParticipantListener::on_subscription_matched (
    DDSDataReader * reader,
    const DDS_SubscriptionMatchedStatus & status) [inline], [virtual]
```

Handles the [DDS_SUBSCRIPTION_MATCHED_STATUS](#) communication status.

on_sample_lost()

```
virtual void DDSDomainParticipantListener::on_sample_lost (
    DDSDataReader * reader,
    const DDS_SampleLostStatus & status) [inline], [virtual]
```

Handles the [DDS_SAMPLE_LOST_STATUS](#) communication status.

on_instance_replaced()

```
virtual void DDSDomainParticipantListener::on_instance_replaced (
    DDSDataReader * reader,
    const DDS_DataReaderInstanceReplacedStatus & status) [inline], [virtual]
```

Handles the [DDS_INSTANCE_REPLACED_STATUS](#) communication status.

This listener may only be called when the instance replacement policy is set to [DDS_REPLACE_OLDEST_INSTANCE_REPLACEMENT_QOS](#). When a new instance is detected and resources are exhausted the oldest instance will be removed even if samples with a not read state exists for the instance. Thus, this is a forceful removal of samples and the instance. The freed instance resources will be re-used for the newly detected instance. If there are outstanding loans for the replaced instance the samples can not be freed until returned. However, when the loan is returned the samples are automatically freed. Note that the instance will not be rejected.

on_before_sample_deserialize()

```
virtual DDS_Boolean DDSDomainParticipantListener::on_before_sample_deserialize (
    DDSDataReader * reader,
    NDDS_Type_Plugin * ,
    CDR_Stream_t * stream,
    DDS_Boolean * ) [inline], [virtual]
```

Callback to filter a received sample based on serialized data

When a [DDSDataReader](#) receives a (serialized) sample, it will deserialize it before storing it in its queue. `On_before_sample_deserialize()` is called before deserialization. It allows the application to determine whether or not to drop the sample, before it is read or taken.

The callback controls whether the sample is filtered out of the DataReader's queue, with the `sample_dropped` parameter. If user code within the callback determines that the sample should be dropped, the user should set `sample_dropped` to true; consequently, when the callback returns, the [DDSDataReader](#) will drop the sample before it is committed to the queue. Otherwise, returning the callback with `sample_dropped` as false will allow the sample to be read or taken.

Compared to `on_before_sample_commit()`, the sample would be dropped with less processing (without deserialization) by the subscribing application, but with the added complexity of filtering on serialized data.

The callback is not associated with a DDS Status. It is enabled when the callback in the listener is assigned, to a non-NULL callback.

on_before_sample_commit()

```
virtual DDS_Boolean DDSDomainParticipantListener::on_before_sample_commit (
    DDSDataReader * ,
    const void * const ,
    const struct DDS_SampleInfo * const ,
    DDS_Boolean * ) [inline], [virtual]
```

Callback to filter a received sample based on deserialized data

When a [DDSDataReader](#) receives a (serialized) sample, it will deserialize it before storing it in its queue. `On_before_sample_commit()` is called after deserialization, but before the sample is stored into the DataReader's queue. It allows the application to determine whether or not to drop the sample, before it is read or taken.

The callback controls whether the sample is filtered out of the DataReader's queue, with the `sample_dropped` parameter. If user code within the callback determines that the sample should be dropped, the user should set `sample_dropped` to true; consequently, when the callback returns, the [DDSDataReader](#) will drop the sample before it is committed to the queue. Otherwise, returning the callback with `sample_dropped` as false will allow the sample to be read or taken.

Compared to `on_before_sample_deserialize()`, the sample would be dropped with more processing (with deserialization) by the subscribing application, but with the lesser complexity of filtering on deserialized data.

The callback is not associated with a DDS Status. It is enabled when the callback in the listener is assigned, to a non-NULL callback.

The following fields are valid in the [DDS_SampleInfo](#) structure:

- `source_timestamp`
- `instance_handle`
- `publication_handle`
- `valid_data`
- `encapsulation_id`

on_data_on_readers()

```
virtual void DDSDomainParticipantListener::on_data_on_readers (
    DDSSubscriber * ) [inline], [virtual]
```

Handles the [DDS_DATA_ON_READERS_STATUS](#) communication status.

on_offered_deadline_missed()

```
virtual void DDSDomainParticipantListener::on_offered_deadline_missed (
    DDSDataWriter * writer,
    const struct DDS_OfferedDeadlineMissedStatus & status) [inline], [virtual]
```

Handles the [DDS_OFFERED_DEADLINE_MISSED_STATUS](#) status.

This callback is called when the deadline that the [DDSDataWriter](#) has committed through its [DEADLINE](#) qos policy was not respected for a specific instance. This callback is called for each deadline period elapsed during which the [DDSDataWriter](#) failed to provide data for an instance.

Parameters

<i>writer</i>	<<out>> Locally created DDSDataWriter that triggers the listener callback
<i>status</i>	<<out>> Current deadline missed status of locally created DDSDataWriter

on_liveliness_lost()

```
virtual void DDSDomainParticipantListener::on_liveliness_lost (
    DDSDataWriter * writer,
    const struct DDS_LivelinessLostStatus & status) [inline], [virtual]
```

Handles the [DDS_LIVELINESS_LOST_STATUS](#) status.

This callback is called when the liveliness that the [DDSDataWriter](#) has committed through its [LIVELINESS](#) qos policy was not respected; this [DDSDataReader](#) entities will consider the [DDSDataWriter](#) as no longer "alive/active". This callback will not be called when an already not alive [DDSDataWriter](#) simply remains not alive for another liveliness period.

Parameters

<i>writer</i>	<<out>> Locally created DDSDataWriter that triggers the listener callback
<i>status</i>	<<out>> Current liveliness lost status of locally created DDSDataWriter

on_offered_incompatible_qos()

```
virtual void DDSDomainParticipantListener::on_offered_incompatible_qos (
    DDSDataWriter * writer,
    const struct DDS_OfferedIncompatibleQosStatus & status) [inline], [virtual]
```

Handles the [DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#) status.

This callback is called when the [DDS_DataWriterQos](#) of the [DDSDataWriter](#) was incompatible with what was requested by a [DDSDataReader](#). This callback is called when a [DDSDataWriter](#) has discovered a [DDSDataReader](#) for the same [DDSTopic](#) and common partition, but with a requested QoS that is incompatible with that offered by the [DDSDataWriter](#).

Parameters

<i>writer</i>	<<out>> Locally created DDSDDataWriter that triggers the listener callback
<i>status</i>	<<out>> Current incompatible qos status of locally created DDSDDataWriter

on_publication_matched()

```
virtual void DDSDomainParticipantListener::on_publication_matched (
    DDSDDataWriter * writer,
    const struct DDS\_PublicationMatchedException & status) [inline], [virtual]
```

Handles the [DDS_PUBLICATION_MATCHED_STATUS](#) status.

This callback is called when the [DDSDDataWriter](#) has found a [DDSDDataReader](#) that matches the [DDSTopic](#), has a common partition and compatible QoS, or has ceased to be matched with a [DDSDDataReader](#) that was previously considered to be matched.

Parameters

<i>writer</i>	<<out>> Locally created DDSDDataWriter that triggers the listener callback
<i>status</i>	<<out>> Current publication match status of locally created DDSDDataWriter

on_reliable_reader_activity_changed()

```
virtual void DDSDomainParticipantListener::on_reliable_reader_activity_changed (
    DDSDDataWriter * writer,
    const struct DDS\_ReliableReaderActivityChangedException & status) [inline], [virtual]
```

<<eXtension>> A matched reliable reader has become active or become inactive.

A reliable writer considers a matched reliable reader to be active when it receives ACKNACK messages from the reader in response to its HEARTBEAT messages. The writer considers an active reader to have become inactive when it has sent a number of consecutive periodic HEARTBEATs equal to [DDS_RtpsReliableWriterProtocol_t::max_heartbeat_retries](#) without receiving an ACKNACK from the reader. An inactive reader becomes active once the writer receives an ACKNACK from it.

This callback is called when the transition from active to inactive, and vice versa, has occurred for a matched reliable reader.

Parameters

<i>writer</i>	<<out>> Locally created DDSDDataWriter that triggers the listener callback
<i>status</i>	<<out>> Current reliable reader activity changed status of locally created DDSDDataWriter

on_inconsistent_topic()

```
virtual void DDSDomainParticipantListener::on_inconsistent_topic (  
    DDS::Topic * topic,  
    const DDS_InconsistentTopicStatus & status) [inline], [virtual]
```

Handle the `DDS_INCONSISTENT_TOPIC_STATUS` status.

This callback is called when a remote `DDS::Topic` is discovered but is inconsistent with the locally created `DDS::Topic` of the same topic name.

Parameters

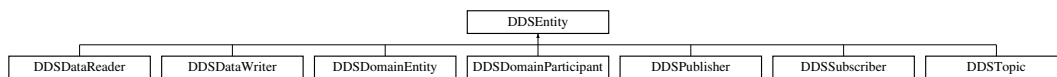
<i>topic</i>	<<out>> Locally created DDSTopic that triggers the listener callback
<i>status</i>	<<out>> Current inconsistent status of locally created DDSTopic

7.103 DDSEntity Class Reference

<<interface>> Abstract base class for all the DDS objects that support QoS policies, and a listener.

```
#include <dds_cpp_infrastructure.hxx>
```

Inheritance diagram for DDSEntity:



Public Member Functions

- [DDS_ReturnCode_t enable \(\)](#)
Enables the [DDSEntity](#).
- [DDSStatusCondition * get_statuscondition \(\)](#)
Returns the [DDSStatusCondition](#) associated with a particular [DDSEntity](#).
- [DDS_StatusMask get_status_changes \(\)](#)
Retrieves the list of communication statuses in the [DDSEntity](#) that are triggered.
- [DDS_InstanceHandle_t get_instance_handle \(\)](#)
Allows access to the [DDS_InstanceHandle_t](#) associated with the [DDSEntity](#).

7.103.1 Detailed Description

<<interface>> Abstract base class for all the DDS objects that support QoS policies, and a listener.

All operations except for `set_qos()`, `get_qos()`, `set_listener()`, `get_listener()` and [enable\(\)](#), may return the value [DDS_RETCODE_NOT_ENABLED](#).

QoS:

[QoS Policies](#)

Status:

[Status Kinds](#)

Listener:

[DDSLListener](#)

7.103.2 Abstract operations

Each derived entity provides the following operations specific to its role in RTI Connex DDS Micro.

7.103.2.1 set_qos (abstract)

This operation sets the QoS policies of the [DDSEntity](#).

This operation must be provided by each of the derived [DDSEntity](#) classes ([DDSDomainParticipant](#), [DDSTopic](#), [DDSPublisher](#), [DDSDataWriter](#), [DDSSubscriber](#), and [DDSDataReader](#)) so that the policies that are meaningful to each [DDSEntity](#) can be set.

Precondition

Certain policies are immutable (see [QoS Policies](#)): they can only be set at [DDSEntity](#) creation time or before the entity is enabled. If `set_qos()` is invoked after the [DDSEntity](#) is enabled and it attempts to change the value of an immutable policy, the operation will fail and return [DDS_RETCODE_IMMUTABLE_POLICY](#).

Certain values of QoS policies can be incompatible with the settings of the other policies. The `set_qos()` operation will also fail if it specifies a set of values that, once combined with the existing values, would result in an inconsistent set of policies. In this case, the operation will fail and return [DDS_RETCODE_INCONSISTENT_POLICY](#).

If the application supplies a non-default value for a QoS policy that is not supported by the implementation of the service, the `set_qos` operation will fail and return [DDS_RETCODE_UNSUPPORTED](#).

Postcondition

The existing set of policies is only changed if the `set_qos()` operation succeeds. This is indicated by a return code of [DDS_RETCODE_OK](#). In all other cases, none of the policies are modified.

Each derived [DDSEntity](#) class ([DDSDomainParticipant](#), [DDSTopic](#), [DDSPublisher](#), [DDSDataWriter](#), [DDSSubscriber](#), [DDSDataReader](#)) has a corresponding special value of the QoS ([DDS_PARTICIPANT_QOS_DEFAULT](#), [DDS_PUBLISHER_QOS_DEFAULT](#), [DDS_SUBSCRIBER_QOS_DEFAULT](#), [DDS_TOPIC_QOS_DEFAULT](#), [DDS_DATAWRITER_QOS_DEFAULT](#), [DDS_DATAREADER_QOS_DEFAULT](#)). This special value may be used as a parameter to the `set_qos` operation to indicate that the QoS of the [DDSEntity](#) should be changed to match the current default QoS set in the [DDSEntity](#)'s factory. The operation `set_qos` cannot modify the immutable QoS, so a successful return of the operation indicates that the mutable QoS for the Entity has been modified to match the current default for the [DDSEntity](#)'s factory.

The set of policies specified in the `qos` parameter are applied on top of the existing QoS, replacing the values of any policies previously set.

Possible error codes returned in addition to [Standard Return Codes](#) : [DDS_RETCODE_IMMUTABLE_POLICY](#), or [DDS_RETCODE_INCONSISTENT_POLICY](#).

7.103.2.2 get_qos (abstract)

This operation allows access to the existing set of QoS policies for the [DDSEntity](#). This operation must be provided by each of the derived [DDSEntity](#) classes ([DDSDomainParticipant](#), [DDSTopic](#), [DDSPublisher](#), [DDSDataWriter](#), [DDSSubscriber](#), and [DDSDataReader](#)), so that the policies that are meaningful to each [DDSEntity](#) can be retrieved.

Possible error codes are [Standard Return Codes](#).

7.103.2.3 set_listener (abstract)

This operation installs a [DDSListener](#) on the [DDSEntity](#). The listener will only be invoked on the changes of communication status indicated by the specified `mask`.

This operation must be provided by each of the derived [DDSEntity](#) classes ([DDSDomainParticipant](#), [DDSTopic](#), [DDSPublisher](#), [DDSDataWriter](#), [DDSSubscriber](#), and [DDSDataReader](#)), so that the listener is of the concrete type suitable to the particular [DDSEntity](#).

It is permitted to use NULL as the value of the listener. The NULL listener behaves as if the `mask` is [DDS_STATUS_MASK_NONE](#).

Postcondition

Only one listener can be attached to each [DDSEntity](#). If a listener was already set, the operation `set_listener()` will replace it with the new one. Consequently, if the value NULL is passed for the listener parameter to the `set_listener` operation, any existing listener will be removed.

7.103.2.4 get_listener (abstract)

This operation allows access to the existing [DDSListener](#) attached to the [DDSEntity](#).

This operation must be provided by each of the derived [DDSEntity](#) classes ([DDSDomainParticipant](#), [DDSTopic](#), [DDSPublisher](#), [DDSDataWriter](#), [DDSSubscriber](#), and [DDSDataReader](#)) so that the listener is of the concrete type suitable to the particular [DDSEntity](#).

If no listener is installed on the [DDSEntity](#), this operation will return NULL.

7.103.3 Member Function Documentation

enable()

```
DDS_ReturnCode_t DDSEntity::enable ()
```

Enables the [DDSEntity](#).

This operation enables the Entity. Entity objects can be created either enabled or disabled. This is controlled by the value of the [ENTITY_FACTORY](#) QoS policy on the corresponding factory for the [DDSEntity](#).

By default, [ENTITY_FACTORY](#) is set so that it is not necessary to explicitly call [DDSEntity::enable](#) on newly created entities.

The [DDSEntity::enable](#) operation is idempotent. Calling enable on an already enabled Entity returns OK and has no effect.

If a [DDSEntity](#) has not yet been enabled, the following kinds of operations may be invoked on it:

- set or get the QoS policies (including default QoS policies) and listener
- 'factory' operations

- 'lookup' operations

Other operations may explicitly state that they may be called on disabled entities; those that do not will return the error [DDS_RETCODE_NOT_ENABLED](#).

It is legal to delete an [DDSEntity](#) that has not been enabled by calling the proper operation on its factory.

Entities created from a factory that is disabled are created disabled, regardless of the setting of the [DDS_EntityFactoryQosPolicy](#).

Calling enable on an Entity whose factory is not enabled will fail and return [DDS_RETCODE_PRECONDITION_NOT_MET](#).

If [DDS_EntityFactoryQosPolicy::autoenable_created_entities](#) is TRUE, the enable operation on a factory will automatically enable all entities created from that factory.

Listeners associated with an entity are not called until the entity is enabled.

Returns

One of the [Standard Return Codes](#), [Standard Return Codes](#) or [DDS_RETCODE_PRECONDITION_NOT_MET](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

get_statuscondition()

```
DDSStatusCondition * DDSEntity::get_statuscondition ()
```

Returns the [DDSStatusCondition](#) associated with a particular [DDSEntity](#).

Returns

The [DDSStatusCondition](#) that belongs to the specified [DDSEntity](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

get_status_changes()

```
DDS_StatusMask DDSEntity::get_status_changes ()
```

Retrieves the list of communication statuses in the [DDSEntity](#) that are triggered.

That is, the list of statuses whose value has changed since the last time the application read the status using the `get_*_status()` method.

When the entity is first created or if the entity is not enabled, all communication statuses are in the "untriggered" state so the list returned by the `get_status_changes` operation will be empty.

The list of statuses returned by the `get_status_changes` operation refers to the statuses that are triggered on the Entity itself and does not include statuses that apply to contained entities.

Returns

list of communication statuses in the [DDSEntity](#) that are triggered.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[Status Kinds](#)

get_instance_handle()

```
DDS_InstanceHandle_t DDSEntity::get_instance_handle ()
```

Allows access to the [DDS_InstanceHandle_t](#) associated with the [DDSEntity](#).

This operation returns the [DDS_InstanceHandle_t](#) that represents the [DDSEntity](#).

Returns

the instance handle associated with this entity.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.104 DDSFlowController Class Reference

<<*interface*>> A flow controller is the object responsible for shaping the network traffic by determining when attached asynchronous [DDSDataWriter](#) instances are allowed to write data.

```
#include <dds_cpp_flowcontroller.hxx>
```

Public Member Functions

- virtual [DDS_ReturnCode_t](#) [set_property](#) (const struct [DDS_FlowControllerProperty_t](#) &prop)=0
Sets the [DDSFlowController](#) property.
- virtual [DDS_ReturnCode_t](#) [get_property](#) (struct [DDS_FlowControllerProperty_t](#) &prop)=0
Gets the [DDSFlowController](#) property.
- virtual [DDS_ReturnCode_t](#) [trigger_flow](#) ()=0
Provides an external trigger to the [DDSFlowController](#).
- virtual const char * [get_name](#) ()=0
Returns the name of the [DDSFlowController](#).
- virtual [DDSDomainParticipant](#) * [get_participant](#) ()=0
Returns the [DDSDomainParticipant](#) to which the [DDSFlowController](#) belongs.

7.104.1 Detailed Description

<<*interface*>> A flow controller is the object responsible for shaping the network traffic by determining when attached asynchronous [DDSDataWriter](#) instances are allowed to write data.

QoS:

[DDS_FlowControllerProperty_t](#)

7.104.2 Member Function Documentation

set_property()

```
virtual DDS\_ReturnCode\_t DDSFlowController::set_property (
    const struct DDS\_FlowControllerProperty\_t & prop) [pure virtual]
```

Sets the [DDSFlowController](#) property.

This operation modifies the property of the [DDSFlowController](#).

Once a [DDSFlowController](#) has been instantiated, only the [DDS_FlowControllerProperty_t::token_bucket](#) can be changed. The [DDS_FlowControllerProperty_t::scheduling_policy](#) is immutable.

A new [DDS_FlowControllerTokenBucketProperty_t::period](#) only takes effect at the next scheduled token distribution time (as determined by its previous value).

Parameters

<i>prop</i>	<<in>> The new DDS_FlowControllerProperty_t . Property must be consistent. Immutable fields cannot be changed after DDSFlowController has been created. The special value DDS_FLOW_CONTROLLER_PROPERTY_DEFAULT can be used to indicate that the property of the DDSFlowController should be changed to match the current default DDS_FlowControllerProperty_t set in the DDSDomainParticipant .
-------------	---

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_IMMUTABLE_POLICY](#), or [DDS_RETCODE_INCONSISTENT_POLICY](#).

See also

[DDS_FlowControllerProperty_t](#) for rules on consistency among property values.

get_property()

```
virtual DDS\_ReturnCode\_t DDSFlowController::get_property (
    struct DDS\_FlowControllerProperty\_t & prop) [pure virtual]
```

Gets the [DDSFlowController](#) property.

Parameters

<i>prop</i>	<<in>> DDSFlowController to be filled in.
-------------	---

Returns

One of the [Standard Return Codes](#)

trigger_flow()

```
virtual DDS\_ReturnCode\_t DDSFlowController::trigger_flow () [pure virtual]
```

Provides an external trigger to the [DDSFlowController](#).

Returns

One of the [Standard Return Codes](#)

Typically, a [DDSFlowController](#) uses an internal trigger to periodically replenish its tokens. The period by which this trigger is called is determined by the [DDS_FlowControllerTokenBucketProperty_t::period](#) property setting.

This method provides an additional, external trigger to the [DDSFlowController](#). This trigger adds [DDS_FlowControllerTokenBucketProperty_t::period](#) tokens each time it is called (subject to the other property settings of the [DDSFlowController](#)).

An *on-demand* [DDSFlowController](#) can be created with a [DDS_DURATION_INFINITE](#) as [DDS_FlowControllerTokenBucketProperty_t::period](#) in which case the only trigger source is external (i.e. the FlowController is solely triggered by the user on demand).

[DDSFlowController::trigger_flow](#) can be called on both strict *on-demand* FlowController and hybrid FlowController (internally and externally triggered).

get_name()

```
virtual const char * DDSFlowController::get_name () [pure virtual]
```

Returns the name of the [DDSFlowController](#).

Returns

The name of the [DDSFlowController](#).

get_participant()

```
virtual DDSDomainParticipant * DDSFlowController::get_participant () [pure virtual]
```

Returns the [DDSDomainParticipant](#) to which the [DDSFlowController](#) belongs.

Returns

The [DDSDomainParticipant](#) to which the [DDSFlowController](#) belongs.

MT Safety:

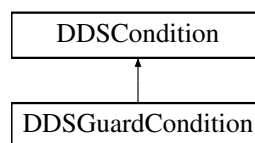
This operation is thread safe.

7.105 DDSGuardCondition Class Reference

<<*interface*>> A specific [DDSCondition](#) whose `trigger_value` is completely under the control of the application.

```
#include <dds_cpp_infrastructure.hxx>
```

Inheritance diagram for DDSGuardCondition:

**Public Member Functions**

- [DDS_ReturnCode_t set_trigger_value](#) (bool value)
Set the guard condition trigger value.

7.105.1 Detailed Description

<<interface>> A specific [DDSCondition](#) whose `trigger_value` is completely under the control of the application.

The [DDSGuardCondition](#) provides a way for an application to manually wake up a [DDSWaitSet](#). This is accomplished by attaching the [DDSGuardCondition](#) to the [DDSWaitSet](#) and then setting the `trigger_value` by means of the [DDSGuardCondition::set_trigger_value](#) operation.

See also

[DDSWaitSet](#)

7.105.2 Member Function Documentation

set_trigger_value()

```
DDS_ReturnCode_t DDSGuardCondition::set_trigger_value (
    bool value)
```

Set the guard condition trigger value.

Parameters

<i>value</i>	<<in>> the new trigger value.
--------------	--

MT Safety:

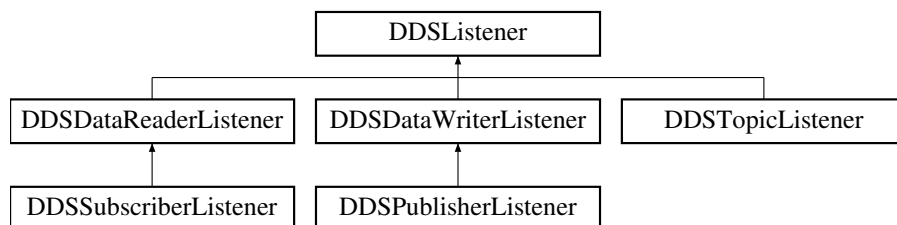
This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.106 DDSListener Class Reference

<<interface>> Abstract base class for all Listener interfaces.

```
#include <dds_cpp_infrastructure.hxx>
```

Inheritance diagram for DDSListener:



7.106.1 Detailed Description

[`<<interface>>`](#) Abstract base class for all Listener interfaces.

Entity:

[DDSEntity](#)

QoS:

[QoS Policies](#)

Status:

[Status Kinds](#)

All the supported kinds of concrete [DDSListener](#) interfaces (one per concrete [DDSEntity](#) type) derive from this root and add methods whose prototype depends on the concrete Listener.

Listeners provide a way for RTI Connex DDS Micro to asynchronously alert the application when there are relevant status changes.

Almost every application will have to implement listener interfaces.

Each dedicated listener presents a list of operations that correspond to the relevant communication status changes to which an application may respond.

The same [DDSListener](#) instance may be shared among multiple entities if you so desire. Consequently, the provided parameter contains a reference to the concerned [DDSEntity](#).

7.106.2 Access to Plain Communication Status

The general mapping between the plain communication statuses (see [Status Kinds](#)) and the listeners' operations is as follows:

- For each communication status, there is a corresponding operation whose name is `on_<communication_<_status>()`, which takes a parameter of type `<communication_status>` as listed in [Status Kinds](#).
- `on_<communication_status>` is available on the relevant [DDSEntity](#) as well as those that embed it, as expressed in the following figure:
- When the application attaches a listener on an entity, it must set a mask. The mask indicates to RTI Connex DDS Micro which operations are enabled within the listener (cf. operation [DDSEntity set_listener\(\)](#)).
- When a plain communication status changes, RTI Connex DDS Micro triggers the most specific relevant listener operation that is enabled. In case the most specific relevant listener operation corresponds to an application-installed 'nil' listener the operation will be considered handled by a NO-OP operation that does not reset the communication status.

This behavior allows the application to set a default behavior (e.g., in the listener associated with the [DDSDomainParticipant](#)) and to set dedicated behaviors only where needed.

7.106.3 Access to Read Communication Status

The two statuses related to data arrival are treated slightly differently. Since they constitute the core purpose of the Data Distribution Service, there is no need to provide a default mechanism (as is done for the plain communication statuses above).

The rule is as follows. Each time the read communication status changes:

- First, RTI Connex DDS Micro tries to trigger the [DDSSubscriberListener::on_data_on_readers](#) with a parameter of the related [DDSSubscriber](#);
- If this does not succeed (there is no listener or the operation is not enabled), RTI Connex DDS Micro tries to trigger [DDSDataReaderListener::on_data_available](#) on all the related [DDSDataReaderListener](#) objects, with a parameter of the related [DDSDataReader](#).

The rationale is that either the application is interested in relations among data arrivals and it must use the first option or it wants to treat each [DDSDataReader](#) independently and it may choose the second option (and then get the data by calling [FooDataReader::read_next_sample](#) or [FooDataReader::take_next_sample](#) on the related [DDSDataReader](#)).

Note that if [DDSSubscriberListener::on_data_on_readers](#) is called, RTI Connex DDS Micro will *not* try to call [DDSDataReaderListener::on_data_available](#).

The operations that are allowed in [DDSListener](#) callbacks depend the [DDSEntity](#) to which the [DDSListener](#) is attached -- or in the case of a [DDSDataWriter](#) or [DDSDataReader](#) listener, on the parent [DDSPublisher](#) or [DDSSubscriber](#).

The following operations are *not* allowed:

- Within any listener callback, deleting the entity to which the [DDSListener](#) is attached
- Within a [DDSTopic](#) listener callback, any operations on any subscribers, readers, publishers or writers

An attempt to call a disallowed method from within a callback will result in [DDS_RETCODE_ILLEGAL_OPERATION](#).

The following are *not* allowed:

- Within any listener callback, creating any entity
- Within any listener callback, deleting any entity
- Within any listener callback, enabling any entity
- Within any listener callback, setting the QoS of any entities
- Within a [DDSDataReader](#) or [DDSSubscriber](#) listener callback, invoking any operation on any other [DDSSubscriber](#) or on any [DDSDataReader](#) belonging to another [DDSSubscriber](#).
- Within a [DDSDataReader](#) or [DDSSubscriber](#) listener callback, invoking any operation on any [DDSPublisher](#) (or on any [DDSDataWriter](#) belonging to such a [DDSPublisher](#)).
- Within a [DDSDataWriter](#) or [DDSPublisher](#) listener callback, invoking any operation on another Publisher or on a [DDSDataWriter](#) belonging to another [DDSPublisher](#).
- Within a [DDSDataWriter](#) or [DDSPublisher](#) listener callback, invoking any operation on any [DDSSubscriber](#) or [DDSDataReader](#).

An attempt to call a disallowed method from within a callback will result in [DDS_RETCODE_ILLEGAL_OPERATION](#).

See also

[Status Kinds](#)

7.107 DDSPropertyQosPolicyHelper Class Reference

Policy helpers that facilitate management of the properties in the input policy.

```
#include <dds_cpp_infrastructure.hxx>
```

Static Public Member Functions

- static `DDS_ReturnCode_t assert_property` (`DDS_PropertyQosPolicy` &policy, const char *name, const char *value, `DDS_Boolean` propagate)
Asserts the property identified by name in the input policy.
- static `DDS_ReturnCode_t add_property` (`DDS_PropertyQosPolicy` &policy, const char *name, const char *value, `DDS_Boolean` propagate)
Adds a new property to the input policy.
- static struct `DDS_Property_t * lookup_property` (`DDS_PropertyQosPolicy` &policy, const char *name)
Searches for a property in the input policy given its name.
- static `DDS_ReturnCode_t remove_property` (`DDS_PropertyQosPolicy` &policy, const char *name)
Removes a property from the input policy.

7.107.1 Detailed Description

Policy helpers that facilitate management of the properties in the input policy.

7.107.2 Member Function Documentation

`assert_property()`

```
static DDS_ReturnCode_t DDSPropertyQosPolicyHelper::assert_property (
    DDS_PropertyQosPolicy & policy,
    const char * name,
    const char * value,
    DDS_Boolean propagate) [static]
```

Asserts the property identified by name in the input policy.

If the property already exists, the method replaces its current value with the new one.

If the property identified by name does not exist, the method adds it to the property set.

This method increases the maximum number of elements of the policy sequence when this number is not enough to store the new property.

Precondition

policy, name and value cannot be NULL.

Parameters

<i>policy</i>	<< <i>in</i> >> Input policy.
<i>name</i>	<< <i>in</i> >> Property name.
<i>value</i>	<< <i>in</i> >> Property value.
<i>propagate</i>	<< <i>in</i> >> Indicates whether the property is propagated on discovery. Must be set to DDS_BOOLEAN_FALSE.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_OUT_OF_RESOURCES](#)

add_property()

```
static DDS_ReturnCode_t DDSPropertyQosPolicyHelper::add_property (  
    DDS_PropertyQosPolicy & policy,  
    const char * name,  
    const char * value,  
    DDS_Boolean propagate) [static]
```

Adds a new property to the input policy.

This method allocates memory to store the (name, value) pair. The memory is owned by RTI Connexx DDS Micro.

If the policy sequence does not have enough capacity for the new property, this method increases it.

If the property already exists, the method fails with [DDS_RETCODE_PRECONDITION_NOT_MET](#).

Precondition

policy, name and value cannot be NULL.

The property is not in the policy.

Parameters

<i>policy</i>	<< <i>in</i> >> Input policy.
<i>name</i>	<< <i>in</i> >> Property name.
<i>value</i>	<< <i>in</i> >> Property value.
<i>propagate</i>	<< <i>in</i> >> Indicates whether the property is propagated on discovery. Must be set to DDS_BOOLEAN_FALSE.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#), [DDS_RETCODE_OUT_OF_RESOURCES](#)

lookup_property()

```
static struct DDS_Property_t * DDSPropertyQosPolicyHelper::lookup_property (
    DDS_PropertyQosPolicy & policy,
    const char * name) [static]
```

Searches for a property in the input policy given its name.

Precondition

policy and name cannot be NULL.

Parameters

<i>policy</i>	<< <i>in</i> >> Input policy.
<i>name</i>	<< <i>in</i> >> Property name.

Returns

The method returns the first property with the given name. If such a property does not exist, the method returns NULL.

remove_property()

```
static DDS_ReturnCode_t DDSPropertyQosPolicyHelper::remove_property (
    DDS_PropertyQosPolicy & policy,
    const char * name) [static]
```

Removes a property from the input policy.

If the property does not exist, the method fails with [DDS_RETCODE_PRECONDITION_NOT_MET](#).

Precondition

policy and name cannot be NULL.

The property exists in the policy.

Parameters

<i>policy</i>	<< <i>in</i> >> Input policy.
<i>name</i>	<< <i>in</i> >> Property name.

Returns

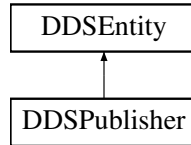
One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#)

7.108 DDSPublisher Class Reference

<<interface>> A publisher is the object responsible for the actual dissemination of publications.

```
#include <dds_cpp_publication.hxx>
```

Inheritance diagram for DDSPublisher:



Public Member Functions

- virtual [DDSDDataWriter](#) * [create_datawriter](#) ([DDSTopic](#) *topic, const [DDS_DataWriterQos](#) &qos, [DDSDDataWriterListener](#) *listener, [DDS_StatusMask](#) mask)=0
Creates a [DDSDDataWriter](#) that will be attached and belong to the [DDSPublisher](#).
- virtual [DDS_ReturnCode_t](#) [delete_datawriter](#) ([DDSDDataWriter](#) *a_datawriter)=0
Deletes a [DDSDDataWriter](#) that belongs to the [DDSPublisher](#).
- virtual [DDS_ReturnCode_t](#) [get_default_datawriter_qos](#) ([DDS_DataWriterQos](#) &qos)=0
Copies the default [DDS_DataWriterQos](#) values into the provided [DDS_DataWriterQos](#) instance.
- virtual [DDS_ReturnCode_t](#) [set_default_datawriter_qos](#) (const [DDS_DataWriterQos](#) &qos)=0
Sets the default [DDS_DataWriterQos](#) values for this publisher.
- virtual [DDSDDataWriter](#) * [lookup_datawriter](#) (const char *topic_name)=0
Retrieves the [DDSDDataWriter](#) for a specific [DDSTopic](#).
- virtual [DDSDDataWriter](#) * [lookup_datawriter_by_name](#) (const char *writer_name)=0
Retrieves a [DDSDDataWriter](#) by its entity name within this [DDSPublisher](#)
- virtual [DDSDomainParticipant](#) * [get_participant](#) ()=0
Returns the [DDSDomainParticipant](#) to which the [DDSPublisher](#) belongs.
- virtual [DDS_ReturnCode_t](#) [get_qos](#) ([DDS_PublisherQos](#) &qos)=0
Gets the publisher QoS.
- virtual [DDS_ReturnCode_t](#) [set_qos](#) (const [DDS_PublisherQos](#) &qos)=0
Sets the publisher QoS.
- virtual [DDS_ReturnCode_t](#) [delete_contained_entities](#) ()=0
Deletes all the entities that were created by means of the "create" operation on the [DDSPublisher](#).
- virtual [DDS_ReturnCode_t](#) [set_listener](#) (const [DDSPublisherListener](#) *listener, [DDS_StatusMask](#) mask)=0
Sets the publisher listener.
- virtual [DDSPublisherListener](#) * [get_listener](#) ()=0
Get the publisher listener.

Public Member Functions inherited from [DDSEntity](#)

- [DDS_ReturnCode_t enable](#) ()
Enables the [DDSEntity](#).
- [DDSStatusCondition * get_statuscondition](#) ()
Returns the [DDSStatusCondition](#) associated with a particular [DDSEntity](#).
- [DDS_StatusMask get_status_changes](#) ()
Retrieves the list of communication statuses in the [DDSEntity](#) that are triggered.
- [DDS_InstanceHandle_t get_instance_handle](#) ()
Allows access to the [DDS_InstanceHandle_t](#) associated with the [DDSEntity](#).

7.108.1 Detailed Description

[DDSPublisher](#) is the object responsible for the actual dissemination of publications.

QoS:

[DDS_PublisherQos](#)

Listener:

[DDSPublisherListener](#)

A publisher acts on the behalf of one or several [DDSDDataWriter](#) objects that belong to it. When it is informed of a change to the data associated with one of its [DDSDDataWriter](#) objects, it decides when it is appropriate to actually send the data-update message. In making this decision, it considers any extra information that goes with the data (timestamp, writer, etc.) as well as the QoS of the [DDSPublisher](#) and the [DDSDDataWriter](#).

The following operations may be called even if the [DDSPublisher](#) is not enabled. Other operations will fail with the value [DDS_RETCODE_NOT_ENABLED](#) if called on a disabled [DDSPublisher](#):

- The base-class operations [DDSEntity::enable](#)
- [DDSPublisher::create_datawriter](#), [DDSPublisher::delete_datawriter](#), [DDSPublisher::delete_contained_entities](#)

7.108.2 Member Function Documentation**[create_datawriter\(\)](#)**

```
virtual DDSDDataWriter * DDSPublisher::create\_datawriter (
    DDSTopic * topic,
    const DDS\_DataWriterQos & qos,
    DDSDDataWriterListener * listener,
    DDS\_StatusMask mask) [pure virtual]
```

Creates a [DDSDDataWriter](#) that will be attached and belong to the [DDSPublisher](#).

Precondition

If publisher is enabled, topic must have been enabled. Otherwise, this operation will fail and no [DDSDDataWriter](#) will be created.

The given [DDSTopic](#) must have been created from the same participant as this publisher. If it was created from a different participant, this method will fail.

Parameters

<i>topic</i>	<<in>> The DDSTopic that the DDSDataWriter will be associated with. Cannot be NULL.
<i>qos</i>	<<in>> QoS to be used for creating the new DDSDataWriter . The special value DDS_DATAWRITER_QOS_DEFAULT can be used to indicate that the DDSDataWriter should be created with the default DDS_DataWriterQos set in the DDSPublisher .

Parameters

<i>listener</i>	<<in>> The listener of the DDSDataWriter .
<i>mask</i>	<<in>>. Changes of communication status to be invoked on the listener.

Returns

A [DDSDataWriter](#) of a derived class specific to the data type associated with the [DDSTopic](#) or NULL if an error occurred.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

ARINC 653 API Restriction:

This function *must* only be called before a partition enters *NORMAL* mode.

See also

[Specifying QoS on entities](#) for information on setting QoS before entity creation

[DDS_DataWriterQos](#) for rules on consistency among QoS

[DDS_DATAWRITER_QOS_DEFAULT](#)

delete_datawriter()

```
virtual DDS_ReturnCode_t DDSPublisher::delete_datawriter (
    DDSDataWriter * a_datawriter) [pure virtual]
```

Deletes a [DDSDataWriter](#) that belongs to the [DDSPublisher](#).

The deletion of the [DDSDataWriter](#) will automatically unregister all instances.

Precondition

If the [DDSDataWriter](#) does not belong to the [DDSPublisher](#), the operation will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#).

Postcondition

Listener installed on the [DDSDataWriter](#) will not be called after this method completes successfully.

Parameters

<i>a_datawriter</i>	<<in>> The DDSDDataWriter to be deleted.
---------------------	--

Returns

One of the [Standard Return Codes](#) or [DDS_RETCODE_PRECONDITION_NOT_MET](#).

get_default_datawriter_qos()

```
virtual DDS_ReturnCode_t DDSPublisher::get_default_datawriter_qos (
    DDS_DataWriterQos & qos) [pure virtual]
```

Copies the default [DDS_DataWriterQos](#) values into the provided [DDS_DataWriterQos](#) instance.

The retrieved qos will match the set of values specified on the last successful call to [DDSPublisher::set_default_datawriter_qos](#) or else, if the call was never made, the default values from its owning [DDSDomainParticipant](#).

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

Returns

One of the [Standard Return Codes](#)

See also

[DDS_DATAWRITER_QOS_DEFAULT](#)
[DDSPublisher::create_datawriter](#)

set_default_datawriter_qos()

```
virtual DDS_ReturnCode_t DDSPublisher::set_default_datawriter_qos (
    const DDS_DataWriterQos & qos) [pure virtual]
```

Sets the default [DDS_DataWriterQos](#) values for this publisher.

This call causes the default values inherited from the owning [DDSDomainParticipant](#) to be overridden.

This default value will be used for newly created [DDSDDataWriter](#) if [DDS_DATAWRITER_QOS_DEFAULT](#) is specified as the qos parameter when [DDSPublisher::create_datawriter](#) is called.

Precondition

The specified QoS policies must be consistent, or else the operation will have no effect and fail with [DDS_RETCODE_INCONSISTENT_POLICY](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

Parameters

<i>qos</i>	<< <i>in</i> >> Default qos to be set. The special value DDS_DATAWRITER_QOS_DEFAULT may be passed as <i>qos</i> to indicate that the default QoS should be reset back to the initial values the factory would have used if DDSPublisher::set_default_datawriter_qos had never been called.
------------	--

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_INCONSISTENT_POLICY](#)

lookup_datawriter()

```
virtual DDSDDataWriter * DDSPublisher::lookup_datawriter (
    const char * topic_name) [pure virtual]
```

Retrieves the [DDSDDataWriter](#) for a specific [DDSTopic](#).

This returned [DDSDDataWriter](#) is either enabled or disabled.

Parameters

<i>topic_name</i>	<< <i>in</i> >> Name of the DDSTopic associated with the DDSDDataWriter that is to be looked up. Cannot be NULL.
-------------------	--

Returns

A [DDSDDataWriter](#) that belongs to the [DDSPublisher](#) attached to the [DDSTopic](#) with *topic_name*. If no such [DDSDDataWriter](#) exists, this operation returns NULL.

If more than one [DDSDDataWriter](#) is attached to the [DDSPublisher](#) with the same *topic_name*, then this operation may return any one of them.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

lookup_datawriter_by_name()

```
virtual DDSDDataWriter * DDSPublisher::lookup_datawriter_by_name (
    const char * writer_name) [pure virtual]
```

Retrieves a [DDSDDataWriter](#) by its entity name within this [DDSPublisher](#)

This returned [DDSDDataWriter](#) is either enabled or disabled.

Every [DDSDDataWriter](#) in the system has an entity name which is configured and stored in the EntityName policy, ENTITY_NAME.

This operation retrieves a [DDSDDataWriter](#) within the [DDSPublisher](#) given the entity's name. If there are several [DDSDDataWriter](#) with the same name within the [DDSPublisher](#), this method returns the first matching occurrence.

Parameters

<i>writer_name</i>	<<in>> Entity name of the DDSDDataWriter . Cannot be NULL.
--------------------	--

Returns

The first [DDSDDataWriter](#) found with the specified name or NULL if it is not found.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

get_participant()

```
virtual DDSDomainParticipant * DDSPublisher::get_participant () [pure virtual]
```

Returns the [DDSDomainParticipant](#) to which the [DDSPublisher](#) belongs.

Returns

the [DDSDomainParticipant](#) to which the [DDSPublisher](#) belongs.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

get_qos()

```
virtual DDS_ReturnCode_t DDSPublisher::get_qos (
    DDS_PublisherQos & qos) [pure virtual]
```

Gets the publisher QoS.

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

Parameters

<i>qos</i>	<<inout>> DDS_PublisherQos to be filled in.
------------	---

Returns

One of the [Standard Return Codes](#)

See also

[get_qos \(abstract\)](#)

set_qos()

```
virtual DDS_ReturnCode_t DDSPublisher::set_qos (
    const DDS_PublisherQos & qos) [pure virtual]
```

Sets the publisher QoS.

This operation modifies the QoS of the [DDSPublisher](#).

The [DDS_PublisherQos::entity_factory](#) can be changed. The other policies are immutable.

Parameters

<i>qos</i>	<< <i>in</i> >> DDS_PublisherQos to be set to. Policies must be consistent. Policies cannot be changed after DDSPublisher is enabled. The special value DDS_PUBLISHER_QOS_DEFAULT can be used to indicate that the QoS of the DDSPublisher should be changed to match the current default DDS_PublisherQos set in the DDSDomainParticipant .
------------	--

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_IMMUTABLE_POLICY](#), or [DDS_RETCODE_INCONSISTENT_POLICY](#).

See also

[DDS_PublisherQos](#) for rules on consistency among QoS
[set_qos](#) (abstract)

delete_contained_entities()

```
virtual DDS_ReturnCode_t DDSPublisher::delete_contained_entities () [pure virtual]
```

Deletes all the entities that were created by means of the "create" operation on the [DDSPublisher](#).

Deletes all contained [DDSDataWriter](#) objects. Once [DDSPublisher::delete_contained_entities](#) completes successfully, the application may delete the [DDSPublisher](#), knowing that it has no contained [DDSDataWriter](#) objects.

The operation will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#) if any of the contained entities is in a state where it cannot be deleted.

Returns

One of the [Standard Return Codes](#) or [DDS_RETCODE_PRECONDITION_NOT_MET](#).

set_listener()

```
virtual DDS_ReturnCode_t DDSPublisher::set_listener (
    const DDSPublisherListener * listener,
    DDS_StatusMask mask) [pure virtual]
```

Sets the publisher listener.

Parameters

<i>listener</i>	<<in>> DDSPublisherListener to set to.
<i>mask</i>	<<in>> DDS_StatusMask associated with the DDSPublisherListener .

Returns

One of the [Standard Return Codes](#)

See also

[set_listener](#) (abstract)

get_listener()

```
virtual DDSPublisherListener * DDSPublisher::get_listener () [pure virtual]
```

Get the publisher listener.

Returns

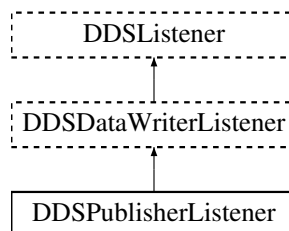
[DDSPublisherListener](#) of the [DDSPublisher](#).

7.109 DDSPublisherListener Class Reference

<<interface>> [DDSListener](#) for [DDSPublisher](#) status.

```
#include <dds_cpp_publication.hxx>
```

Inheritance diagram for DDSPublisherListener:



Additional Inherited Members

Public Member Functions inherited from [DDSDataWriterListener](#)

- virtual void [on_offered_deadline_missed](#) ([DDSDataWriter](#) *writer, const [DDS_OfferedDeadlineMissedStatus](#) &status)
Handles the [DDS_OFFERED_DEADLINE_MISSED_STATUS](#) status.
- virtual void [on_liveliness_lost](#) ([DDSDataWriter](#) *writer, const [DDS_LivelinessLostStatus](#) &status)
Handles the [DDS_LIVELINESS_LOST_STATUS](#) status.
- virtual void [on_offered_incompatible_qos](#) ([DDSDataWriter](#) *writer, const [DDS_OfferedIncompatibleQosStatus](#) &status)
Handles the [DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#) status.
- virtual void [on_publication_matched](#) ([DDSDataWriter](#) *writer, const [DDS_PublicationMatchedStatus](#) &status)
Handles the [DDS_PUBLICATION_MATCHED_STATUS](#) status.
- virtual void [on_reliable_reader_activity_changed](#) ([DDSDataWriter](#) *writer, const [DDS_ReliableReaderActivityChangedStatus](#) &status)
<<eXtension>> A matched reliable reader has become active or become inactive.
- virtual void [on_reliable_sample_unacknowledged](#) ([DDSDataWriter](#) *writer, const [DDS_ReliableSampleUnacknowledgedStatus](#) &status)
<<eXtension>> A matched reliable reader has become active or become inactive.
- virtual void [on_sample_removed](#) ([DDSDataWriter](#) *writer, const [DDS_Cookie_t](#) &cookie)
<<eXtension>> Called when a sample is removed from the DataWriter queue.

7.109.1 Detailed Description

<<*interface*>> [DDSListener](#) for [DDSPublisher](#) status.

Entity:

[DDSPublisher](#)

Status:

[DDS_LIVELINESS_LOST_STATUS](#), [DDS_LivelinessLostStatus](#);
[DDS_OFFERED_DEADLINE_MISSED_STATUS](#), [DDS_OfferedDeadlineMissedStatus](#);
[DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#), [DDS_OfferedIncompatibleQosStatus](#);
[DDS_PUBLICATION_MATCHED_STATUS](#), [DDS_PublicationMatchedStatus](#);

See also

[DDSListener](#)

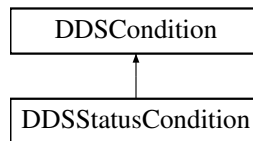
[Status Kinds](#)

7.110 DDSStatusCondition Class Reference

<<*interface*>> A specific [DDSCondition](#) that is associated with each [DDSEntity](#).

```
#include <dds_cpp_infrastructure.hxx>
```

Inheritance diagram for DDSStatusCondition:



Public Member Functions

- [DDS_StatusMask get_enabled_statuses \(\)](#)
Get the list of statuses enabled on an [DDSEntity](#).
- [DDS_ReturnCode_t set_enabled_statuses \(DDS_StatusMask mask\)](#)
*This operation defines the list of communication statuses that determine the *trigger_value* of the [DDSStatusCondition](#).*

Friends

- class [DDSEntity](#)

7.110.1 Detailed Description

<<*interface*>> A specific [DDSCondition](#) that is associated with each [DDSEntity](#).

The `trigger_value` of the [DDSStatusCondition](#) depends on the communication status of that entity (e.g., arrival of data, loss of information, etc.), 'filtered' by the set of `enabled_statuses` on the [DDSStatusCondition](#).

This release supports the following subset of communication statuses that can be associated with a [DDSStatusCondition](#):

[DDS_DATA_AVAILABLE_STATUS](#)

See also

[Status Kinds](#)
[DDSWaitSet](#), [DDSCondition](#)
[DDSListener](#)

7.110.2 Member Function Documentation

get_enabled_statuses()

```
DDS_StatusMask DDSStatusCondition::get_enabled_statuses ()
```

Get the list of statuses enabled on an [DDSEntity](#).

Returns

list of enabled statuses.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

set_enabled_statuses()

```
DDS_ReturnCode_t DDSStatusCondition::set_enabled_statuses (  
    DDS_StatusMask mask)
```

This operation defines the list of communication statuses that determine the `trigger_value` of the [DDSStatusCondition](#).

This operation may change the `trigger_value` of the [DDSStatusCondition](#).

[DDSWaitSet](#) objects' behavior depends on the changes of the `trigger_value` of their attached conditions. Therefore, any [DDSWaitSet](#) to which the [DDSStatusCondition](#) is attached is potentially affected by this operation.

If this method is not invoked, the default list of enabled statuses includes all the statuses.

Parameters

<i>mask</i>	<< <i>in</i> >> the list of enables statuses (see Status Kinds)
-------------	--

Returns

One of the [Standard Return Codes](#)

MT Safety:

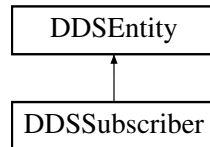
This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.111 DDSSubscriber Class Reference

<<interface>> A subscriber is the object responsible for actually receiving data from a subscription.

```
#include <dds_cpp_subscription.hxx>
```

Inheritance diagram for DDSSubscriber:



Public Member Functions

- virtual [DDSDataReader](#) * [create_datareader](#) ([DDSTopicDescription](#) *topic, const [DDS_DataReaderQos](#) &qos, [DDSDataReaderListener](#) *listener, [DDS_StatusMask](#) mask)=0
Creates a [DDSDataReader](#) that will be attached and belong to the [DDSSubscriber](#).
- virtual [DDS_ReturnCode_t](#) [delete_datareader](#) ([DDSDataReader](#) *a_datareader)=0
Deletes a [DDSDataReader](#) that belongs to the [DDSSubscriber](#).
- virtual [DDS_ReturnCode_t](#) [get_default_datareader_qos](#) ([DDS_DataReaderQos](#) &qos)=0
Copies the default [DDS_DataReaderQos](#) values into the provided [DDS_DataReaderQos](#) instance.
- virtual [DDS_ReturnCode_t](#) [set_default_datareader_qos](#) (const [DDS_DataReaderQos](#) &qos)=0
Sets the default [DDS_DataReaderQos](#) values for this subscriber.
- virtual [DDS_ReturnCode_t](#) [delete_contained_entities](#) ()=0
Deletes all the entities that were created by means of the "create" operation on the [DDSSubscriber](#).
- virtual [DDSDataReader](#) * [lookup_datareader](#) (const char *topic_name)=0
Retrieves an existing [DDSDataReader](#).
- virtual [DDSDataReader](#) * [lookup_datareader_by_name](#) (const char *reader_name)=0
Retrieves a [DDSDataReader](#) by its entity name within this [DDSSubscriber](#)
- virtual [DDSDomainParticipant](#) * [get_participant](#) ()=0
Returns the [DDSDomainParticipant](#) to which the [DDSSubscriber](#) belongs.
- virtual [DDS_ReturnCode_t](#) [set_qos](#) (const [DDS_SubscriberQos](#) &qos)=0
Sets the subscriber QoS.
- virtual [DDS_ReturnCode_t](#) [get_qos](#) ([DDS_SubscriberQos](#) &qos)=0
Gets the subscriber QoS.
- virtual [DDS_ReturnCode_t](#) [set_listener](#) (const [DDSSubscriberListener](#) *listener, [DDS_StatusMask](#) mask)=0
Sets the subscriber listener.
- virtual [DDSSubscriberListener](#) * [get_listener](#) ()=0
Get the subscriber listener.

Public Member Functions inherited from [DDSEntity](#)

- [DDS_ReturnCode_t enable](#) ()
Enables the [DDSEntity](#).
- [DDSStatusCondition * get_statuscondition](#) ()
Returns the [DDSStatusCondition](#) associated with a particular [DDSEntity](#).
- [DDS_StatusMask get_status_changes](#) ()
Retrieves the list of communication statuses in the [DDSEntity](#) that are triggered.
- [DDS_InstanceHandle_t get_instance_handle](#) ()
Allows access to the [DDS_InstanceHandle_t](#) associated with the [DDSEntity](#).

7.111.1 Detailed Description

<<[interface](#)>> A subscriber is the object responsible for actually receiving data from a subscription.

QoS:

[DDS_SubscriberQos](#)

Status:

[DDS_DATA_ON_READERS_STATUS](#)

Listener:

[DDSSubscriberListener](#)

A subscriber acts on the behalf of one or several [DDSDataReader](#) objects that are related to it. When it receives data (from the other parts of the system), it builds the list of concerned [DDSDataReader](#) objects and then indicates to the application that data is available through its listener.

The application can access the concerned [DDSDataReader](#) objects and then access the data available through operations on the [DDSDataReader](#).

The following operations may be called even if the [DDSSubscriber](#) is not enabled. Other operations will fail with the value [DDS_RETCODE_NOT_ENABLED](#) if called on a disabled [DDSSubscriber](#):

- The base-class operations [DDSEntity::enable](#)
- [DDSSubscriber::create_datareader](#) , [DDSSubscriber::delete_datareader](#) , [DDSSubscriber::delete_contained_entities](#).

All operations except for the base-class operations [enable\(\)](#) and [create_datareader\(\)](#) may fail with [DDS_RETCODE_NOT_ENABLED](#).

7.111.2 Member Function Documentation

create_datareader()

```
virtual DDSDataReader * DDSSubscriber::create_datareader (
    DDSTopicDescription * topic,
    const DDS_DataReaderQos & qos,
    DDSDataReaderListener * listener,
    DDS_StatusMask mask) [pure virtual]
```

Creates a [DDSDataReader](#) that will be attached and belong to the [DDSSubscriber](#).

Precondition

If subscriber is enabled, the topic must be enabled. If it is not, this operation will fail and no [DDSDataReader](#) will be created.

The given [DDSTopicDescription](#) must have been created from the same participant as this subscriber. If it was created from a different participant, this method will return NULL.

Parameters

<i>topic</i>	<< <i>in</i> >> The DDSTopicDescription that the DDSDataReader will be associated with. Cannot be NULL.
<i>qos</i>	<< <i>in</i> >> The qos of the DDSDataReader . The special value DDS_DATAREADER_QOS_DEFAULT can be used to indicate that the DDSDataReader should be created with the default DDS_DataReaderQos set in the DDSSubscriber .
<i>listener</i>	<< <i>in</i> >> The listener of the DDSDataReader .
<i>mask</i>	<< <i>in</i> >> Changes of communication status to be invoked on the listener.

Returns

A [DDSDataReader](#) of a derived class specific to the data-type associated with the [DDSTopic](#) or NULL if an error occurred.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

ARINC 653 API Restriction:

This function *must* only be called before a partition enters *NORMAL* mode.

See also

[Specifying QoS on entities](#) for information on setting QoS before entity creation

[DDS_DataReaderQos](#) for rules on consistency among QoS

delete_datareader()

```
virtual DDS_ReturnCode_t DDSSubscriber::delete_datareader (
    DDSDataReader * a_datareader) [pure virtual]
```

Deletes a [DDSDataReader](#) that belongs to the [DDSSubscriber](#).

Precondition

If the [DDSDataReader](#) does not belong to the [DDSSubscriber](#), or if there are any existing objects that are attached to the [DDSDataReader](#), or if there are outstanding loans on samples (as a result of a call to [read\(\)](#), [take\(\)](#), or one of the variants thereof), the operation fails with the error [DDS_RETCODE_PRECONDITION_NOT_MET](#).

Postcondition

Listener installed on the [DDSDataReader](#) will not be called after this method completes successfully.

Parameters

<i>a_datareader</i>	<< <i>in</i> >> The DDSDataReader to be deleted.
---------------------	--

Returns

One of the [Standard Return Codes](#) or [DDS_RETCODE_PRECONDITION_NOT_MET](#).

get_default_datareader_qos()

```
virtual DDS_ReturnCode_t DDSSubscriber::get_default_datareader_qos (
    DDS_DataReaderQos & qos) [pure virtual]
```

Copies the default [DDS_DataReaderQos](#) values into the provided [DDS_DataReaderQos](#) instance.

The retrieved `qos` will match the set of values specified on the last successful call to [DDSSubscriber::set_default_datareader_qos](#), or else, if the call was never made, the default values from its owning [DDSDomainParticipant](#).

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

Parameters

<i>qos</i>	<< <i>inout</i> >> DDS_DataReaderQos to be filled-up.
------------	---

Returns

One of the [Standard Return Codes](#)

See also

[DDS_DATAREADER_QOS_DEFAULT](#)
[DDSSubscriber::create_datareader](#)

set_default_datareader_qos()

```
virtual DDS_ReturnCode_t DDSSubscriber::set_default_datareader_qos (
    const DDS_DataReaderQos & qos) [pure virtual]
```

Sets the default [DDS_DataReaderQos](#) values for this subscriber.

This call causes the default values inherited from the owning [DDSDomainParticipant](#) to be overridden.

This default value will be used for newly created [DDSDataReader](#) if [DDS_DATAREADER_QOS_DEFAULT](#) is specified as the `qos` parameter when [DDSSubscriber::create_datareader](#) is called.

Precondition

The specified QoS policies must be consistent, or else the operation will have no effect and fail with [DDS_RETCODE_INCONSISTENT_POLICY](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

Parameters

<i>qos</i>	<< <i>in</i> >> The default DDS_DataReaderQos to be set to. The special value DDS_DATAREADER_QOS_DEFAULT may be passed as <code>qos</code> to indicate that the default QoS should be reset back to the initial values the factory would have used if DDSSubscriber::set_default_datareader_qos had never been called.
------------	--

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_INCONSISTENT_POLICY](#)

delete_contained_entities()

```
virtual DDS_ReturnCode_t DDSSubscriber::delete_contained_entities () [pure virtual]
```

Deletes all the entities that were created by means of the "create" operation on the [DDSSubscriber](#).

Deletes all contained [DDSDataReader](#) objects. This pattern is applied recursively. In this manner, the operation [DDSSubscriber::delete_contained_entities](#) on the [DDSSubscriber](#) will end up deleting all the entities recursively contained in the [DDSSubscriber](#), that is also the objects belonging to the contained [DDSDataReader](#).

The operation will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#) if any of the contained entities is in a state where it cannot be deleted. This will occur, for example, if a contained [DDSDataReader](#) cannot be deleted because the application has called a [FooDataReader::read](#) or [FooDataReader::take](#) operation and has not called the corresponding [FooDataReader::return_loan](#) operation to return the loaned samples.

Once [DDSSubscriber::delete_contained_entities](#) completes successfully, the application may delete the [DDSSubscriber](#), knowing that it has no contained [DDSDataReader](#) objects.

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_PRECONDITION_NOT_MET](#)

lookup_datareader()

```
virtual DDSDataReader * DDSSubscriber::lookup_datareader (
    const char * topic_name) [pure virtual]
```

Retrieves an existing [DDSDataReader](#).

Parameters

<i>topic_name</i>	<<in>> Name of the DDSTopicDescription that the retrieved DDSDataReader is attached to. Cannot be NULL.
-------------------	---

Returns

A [DDSDataReader](#) that belongs to the [DDSSubscriber](#) attached to the [DDSTopicDescription](#) with *topic_name*. If no such [DDSDataReader](#) exists, this operation returns NULL.

The returned [DDSDataReader](#) may be enabled or disabled.

If more than one [DDSDataReader](#) is attached to the [DDSSubscriber](#), this operation may return any one of them.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

lookup_datareader_by_name()

```
virtual DDSDataReader * DDSSubscriber::lookup_datareader_by_name (
    const char * reader_name) [pure virtual]
```

Retrieves a [DDSDataReader](#) by its entity name within this [DDSSubscriber](#)

This returned [DDSDataReader](#) is either enabled or disabled.

Every [DDSDataReader](#) in the system has an entity name which is configured and stored in the EntityName policy, ENTITY_NAME.

This operation retrieves a [DDSDataReader](#) within the [DDSSubscriber](#) given the entity's name. If there are several [DDSDataReader](#) with the same name within the [DDSSubscriber](#), this method returns the first matching occurrence.

Parameters

<i>reader_name</i>	<<in>> Entity name of the DDSDataReader . Cannot be NULL.
--------------------	---

Returns

The first [DDSDataReader](#) found with the specified name or NULL if it is not found.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

get_participant()

```
virtual DDSDomainParticipant * DDSSubscriber::get_participant () [pure virtual]
```

Returns the [DDSDomainParticipant](#) to which the [DDSSubscriber](#) belongs.

Returns

the [DDSDomainParticipant](#) to which the [DDSSubscriber](#) belongs.

MT Safety:

This operation is thread safe.

set_qos()

```
virtual DDS_ReturnCode_t DDSSubscriber::set_qos (
    const DDS_SubscriberQos & qos) [pure virtual]
```

Sets the subscriber QoS.

This operation modifies the QoS of the [DDSSubscriber](#).

The [DDS_SubscriberQos::entity_factory](#) can be changed. The other policies are immutable.

Parameters

<i>qos</i>	<< <i>in</i> >> DDS_SubscriberQos to be set to. Policies must be consistent. Policies cannot be changed after DDSSubscriber is enabled. The special value DDS_SUBSCRIBER_QOS_DEFAULT can be used to indicate that the QoS of the DDSSubscriber should be changed to match the current default DDS_SubscriberQos set in the DDSDomainParticipant .
------------	---

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_IMMUTABLE_POLICY](#), or [DDS_RETCODE_INCONSISTENT_POLICY](#).

See also

[DDS_SubscriberQos](#) for rules on consistency among QoS
[set_qos](#) (abstract)

get_qos()

```
virtual DDS_ReturnCode_t DDSSubscriber::get_qos (
    DDS_SubscriberQos & qos) [pure virtual]
```

Gets the subscriber QoS.

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

Parameters

<i>qos</i>	<<out>> DDS_SubscriberQos to be filled in.
------------	--

Returns

One of the [Standard Return Codes](#)

See also

[get_qos](#) (abstract)

set_listener()

```
virtual DDS\_ReturnCode\_t DDSSubscriber::set_listener (  
    const DDSSubscriberListener * listener,  
    DDS\_StatusMask mask) [pure virtual]
```

Sets the subscriber listener.

Parameters

<i>listener</i>	<<in>> DDSSubscriberListener to set to.
<i>mask</i>	<<in>> DDS_StatusMask associated with the DDSSubscriberListener .

Returns

One of the [Standard Return Codes](#)

See also

[set_listener](#) (abstract)

get_listener()

```
virtual DDSSubscriberListener * DDSSubscriber::get_listener () [pure virtual]
```

Get the subscriber listener.

Returns

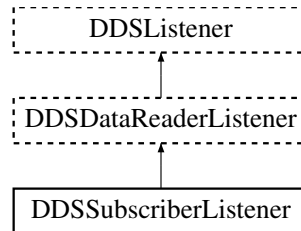
[DDSSubscriberListener](#) of the [DDSSubscriber](#).

7.112 DDSSubscriberListener Class Reference

<<*interface*>> [DDSListener](#) for status about a subscriber.

```
#include <dds_cpp_subscription.hxx>
```

Inheritance diagram for DDSSubscriberListener:



Public Member Functions

- virtual void [on_data_on_readers](#) ([DDSSubscriber](#) *)
Handles the [DDS_DATA_ON_READERS_STATUS](#) communication status.

Public Member Functions inherited from [DDSDDataReaderListener](#)

- virtual void [on_data_available](#) ([DDSDDataReader](#) *)
Handles the [DDS_DATA_AVAILABLE_STATUS](#) communication status.
- virtual void [on_requested_deadline_missed](#) ([DDSDDataReader](#) *, const [DDS_RequestedDeadlineMissedStatus](#) &)
Handles the [DDS_REQUESTED_DEADLINE_MISSED_STATUS](#) communication status.
- virtual void [on_liveliness_changed](#) ([DDSDDataReader](#) *, const [DDS_LivelinessChangedStatus](#) &)
Handles the [DDS_LIVELINESS_CHANGED_STATUS](#) communication status.
- virtual void [on_requested_incompatible_qos](#) ([DDSDDataReader](#) *, const [DDS_RequestedIncompatibleQosStatus](#) &)
Handles the [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#) communication status.
- virtual void [on_sample_rejected](#) ([DDSDDataReader](#) *, const [DDS_SampleRejectedStatus](#) &)
Handles the [DDS_SAMPLE_REJECTED_STATUS](#) communication status.
- virtual void [on_subscription_matched](#) ([DDSDDataReader](#) *, const [DDS_SubscriptionMatchedStatus](#) &)
Handles the [DDS_SUBSCRIPTION_MATCHED_STATUS](#) communication status.
- virtual void [on_sample_lost](#) ([DDSDDataReader](#) *, const [DDS_SampleLostStatus](#) &)
Handles the [DDS_SAMPLE_LOST_STATUS](#) communication status.
- virtual void [on_instance_replaced](#) ([DDSDDataReader](#) *, const [DDS_DataReaderInstanceReplacedStatus](#) &)
Handles the [DDS_INSTANCE_REPLACED_STATUS](#) communication status.
- virtual [DDS_Boolean](#) [on_before_sample_deserialize](#) ([DDSDDataReader](#) *, [NDDS_Type_Plugin](#) *, [CDR_Stream_t](#) *, [DDS_Boolean](#) *)
Callback to filter a received sample based on serialized data
- virtual [DDS_Boolean](#) [on_before_sample_commit](#) ([DDSDDataReader](#) *, const void *const, const struct [DDS_SampleInfo](#) *const, [DDS_Boolean](#) *)
Callback to filter a received sample based on deserialized data

7.112.1 Detailed Description

<<*interface*>> [DDSListener](#) for status about a subscriber.

Entity:

[DDSSubscriber](#)

Status:

```

DDS_DATA_AVAILABLE_STATUS;
DDS_DATA_ON_READERS_STATUS;
DDS_LIVELINESS_CHANGED_STATUS, DDS_LivelinessChangedStatus;
DDS_REQUESTED_DEADLINE_MISSED_STATUS, DDS_RequestedDeadlineMissedStatus;
DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS, DDS_RequestedIncompatibleQosStatus;
DDS_SAMPLE_LOST_STATUS, DDS_SampleLostStatus;
DDS_SAMPLE_REJECTED_STATUS, DDS_SampleRejectedStatus;
DDS_SUBSCRIPTION_MATCHED_STATUS, DDS_SubscriptionMatchedStatus;
DDS_INSTANCE_REPLACED_STATUS, DDS_DataReaderInstanceReplacedStatus;

```

See also

[DDSListener](#)

[Status Kinds](#)

7.112.2 Member Function Documentation

on_data_on_readers()

```

virtual void DDSSubscriberListener::on_data_on_readers (
    DDSSubscriber * ) [inline], [virtual]

```

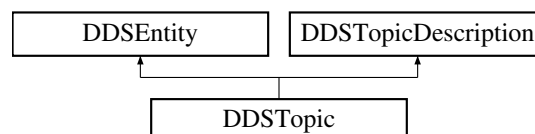
Handles the [DDS_DATA_ON_READERS_STATUS](#) communication status.

7.113 DDSTopic Class Reference

<<*interface*>> The most basic description of the data to be published and subscribed.

```
#include <dds_cpp_topic.hxx>
```

Inheritance diagram for DDSTopic:



Public Member Functions

- virtual [DDSTopicDescription](#) * [as_topicdescription](#) ()=0
Access a [DDSTopic](#)'s [DDSTopicDescription](#) supertype instance.
- virtual [DDSEntity](#) * [as_entity](#) ()=0
Access a [DDSTopic](#)'s [DDSEntity](#) supertype instance.
- virtual [DDS_ReturnCode_t](#) [set_qos](#) (const [DDS_TopicQos](#) &qos)=0
Set the topic QoS.
- virtual [DDS_ReturnCode_t](#) [get_qos](#) ([DDS_TopicQos](#) &qos)=0
Get the topic QoS.
- virtual [DDS_ReturnCode_t](#) [get_inconsistent_topic_status](#) ([DDS_InconsistentTopicStatus](#) &status)=0
Allows the application to retrieve the [DDS_INCONSISTENT_TOPIC_STATUS](#) status of a [DDSTopic](#).
- virtual [DDS_ReturnCode_t](#) [set_listener](#) (const [DDSTopicListener](#) *listener, [DDS_StatusMask](#) mask)=0
Set the topic listener.
- virtual [DDSTopicListener](#) * [get_listener](#) ()=0
Get the topic listener.
- virtual const char * [get_type_name](#) ()=0
Get the associated `type_name`.
- virtual const char * [get_name](#) ()=0
Get the name used to create this [DDSTopicDescription](#) .
- virtual [DDSDomainParticipant](#) * [get_participant](#) ()=0
Get the [DDSDomainParticipant](#) to which the [DDSTopicDescription](#) belongs.

Public Member Functions inherited from [DDSEntity](#)

- [DDS_ReturnCode_t](#) [enable](#) ()
Enables the [DDSEntity](#).
- [DDSStatusCondition](#) * [get_statuscondition](#) ()
Returns the [DDSStatusCondition](#) associated with a particular [DDSEntity](#).
- [DDS_StatusMask](#) [get_status_changes](#) ()
Retrieves the list of communication statuses in the [DDSEntity](#) that are triggered.
- [DDS_InstanceHandle_t](#) [get_instance_handle](#) ()
Allows access to the [DDS_InstanceHandle_t](#) associated with the [DDSEntity](#).

Static Public Member Functions

- static [DDSTopic](#) * [narrow](#) ([DDSTopicDescription](#) *topic_description)
Narrow the given [DDSTopicDescription](#) pointer to a [DDSTopic](#) pointer.

7.113.1 Detailed Description

<<[*interface*](#)>> The most basic description of the data to be published and subscribed.

QoS:

[DDS_TopicQos](#)

Status:

[DDS_INCONSISTENT_TOPIC_STATUS](#), [DDS_InconsistentTopicStatus](#)

Listener:

[DDSTopicListener](#)

A [DDSTopic](#) is identified by its name, which must be unique in the whole domain. In addition (by virtue of extending [DDSTopicDescription](#)) it fully specifies the type of the data that can be communicated when publishing or subscribing to the [DDSTopic](#).

[DDSTopic](#) is the only [DDSTopicDescription](#) that can be used for publications and therefore associated with a [DDSDataWriter](#).

The following operations may be called even if the [DDSTopic](#) is not enabled. Other operations will fail with the value [DDS_RETCODE_NOT_ENABLED](#) if called on a disabled [DDSTopic](#):

- [get_inconsistent_topic_status\(\)](#)

7.113.2 Member Function Documentation

narrow()

```
static DDSTopic * DDSTopic::narrow (  
    DDSTopicDescription * topic_description)    [static]
```

Narrow the given [DDSTopicDescription](#) pointer to a [DDSTopic](#) pointer.

Returns

[DDSTopic](#) if this [DDSTopicDescription](#) is a [DDSTopic](#). Otherwise, return NULL.

as_topicdescription()

```
virtual DDSTopicDescription * DDSTopic::as\_topicdescription ()    [pure virtual]
```

Access a [DDSTopic](#)'s [DDSTopicDescription](#) supertype instance.

Returns

[DDSTopic](#)'s supertype [DDSTopicDescription](#) instance

as_entity()

```
virtual DDSEntity * DDSTopic::as_entity () [pure virtual]
```

Access a [DDSTopic](#)'s [DDSEntity](#) supertype instance.

Returns

[DDSTopic](#)'s supertype [DDSEntity](#) instance

set_qos()

```
virtual DDS\_ReturnCode\_t DDSTopic::set_qos (
    const DDS\_TopicQos & qos) [pure virtual]
```

Set the topic QoS.

Parameters

<i>qos</i>	<< <i>in</i> >> Set of policies to be applied to DDSTopic .
------------	---

Policies must be consistent. Policies cannot be changed after [DDSTopic](#) is enabled. The special value [DDS_TOPIC_QOS_DEFAULT](#) can be used to indicate that the QoS of the [DDSTopic](#) should be changed to match the current default [DDS_TopicQos](#) set in the [DDSDomainParticipant](#).

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_IMMUTABLE_POLICY](#) if immutable policy is changed, or [DDS_RETCODE_INCONSISTENT_POLICY](#) if policies are inconsistent

See also

[DDS_TopicQos](#) for rules on consistency among QoS
[set_qos \(abstract\)](#)

get_qos()

```
virtual DDS\_ReturnCode\_t DDSTopic::get_qos (
    DDS\_TopicQos & qos) [pure virtual]
```

Get the topic QoS.

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

Parameters

<i>qos</i>	<< <i>inout</i> >> QoS to be filled up.
------------	---

Returns

One of the [Standard Return Codes](#)

See also

[get_qos \(abstract\)](#)

get_inconsistent_topic_status()

```
virtual DDS_ReturnCode_t DDSTopic::get_inconsistent_topic_status (  
    DDS_InconsistentTopicStatus & status) [pure virtual]
```

Allows the application to retrieve the [DDS_INCONSISTENT_TOPIC_STATUS](#) status of a [DDSTopic](#).

Retrieve the current [DDS_InconsistentTopicStatus](#)

Parameters

<i>status</i>	<< <i>inout</i> >> Status to be retrieved.
---------------	--

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDS_InconsistentTopicStatus](#)

set_listener()

```
virtual DDS_ReturnCode_t DDSTopic::set_listener (  
    const DDSTopicListener * listener,  
    DDS_StatusMask mask) [pure virtual]
```

Set the topic listener.

Parameters

<i>listener</i>	<< <i>in</i> >> Listener to be installed on entity.
<i>mask</i>	<< <i>in</i> >> Changes of communication status to be invoked on the listener.

Returns

One of the [Standard Return Codes](#)

get_listener()

```
virtual DDSTopicListener * DDSTopic::get_listener () [pure virtual]
```

Get the topic listener.

Returns

Existing listener attached to the [DDSTopic](#).

get_type_name()

```
virtual const char * DDSTopic::get_type_name () [pure virtual]
```

Get the associated `type_name`.

The type name defines a locally unique type for the publication or the subscription.

The `type_name` corresponds to a unique string used to register a type via the [DDSDomainParticipant::register_type](#) method.

Thus, the `type_name` implies an association with a corresponding type and this [DDSTopicDescription](#).

Returns

the type name. The returned type name is valid until the [DDSTopicDescription](#) is deleted.

MT Safety:

This operation is thread safe.

Postcondition

The result is NULL if self is NULL, otherwise result is non-NULL.

Implements [DDSTopicDescription](#).

get_name()

```
virtual const char * DDSTopic::get_name () [pure virtual]
```

Get the name used to create this [DDSTopicDescription](#) .

Returns

the name used to create this [DDSTopicDescription](#). The returned topic name is valid until the [DDSTopicDescription](#) is deleted.

MT Safety:

This operation is thread safe.

Postcondition

The result is NULL if self is NULL, otherwise result is non-NULL.

Implements [DDSTopicDescription](#).

get_participant()

```
virtual DDSDomainParticipant * DDSTopic::get_participant () [pure virtual]
```

Get the [DDSDomainParticipant](#) to which the [DDSTopicDescription](#) belongs.

Returns

The [DDSDomainParticipant](#) to which the [DDSTopicDescription](#) belongs.

MT Safety:

This operation is thread safe.

Postcondition

The result is NULL if self is NULL, otherwise result is non-NULL.

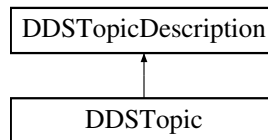
Implements [DDSTopicDescription](#).

7.114 DDSTopicDescription Class Reference

<<*interface*>> Base class for [DDSTopic](#).

```
#include <dds_cpp_topic.hxx>
```

Inheritance diagram for DDSTopicDescription:



Public Member Functions

- virtual const char * [get_type_name](#) ()=0
Get the associated `type_name`.
- virtual const char * [get_name](#) ()=0
Get the name used to create this [DDSTopicDescription](#) .
- virtual [DDSDomainParticipant](#) * [get_participant](#) ()=0
Get the [DDSDomainParticipant](#) to which the [DDSTopicDescription](#) belongs.

7.114.1 Detailed Description

<<*interface*>> Base class for [DDSTopic](#).

[DDSTopicDescription](#) represents the fact that both publications and subscriptions are tied to a single data-type. Its attribute `type_name` defines a unique resulting type for the publication or the subscription and therefore creates an implicit association with a type.

[DDSTopicDescription](#) has also a `name` that allows it to be retrieved locally.

7.114.2 Member Function Documentation

[get_type_name\(\)](#)

```
virtual const char * DDSTopicDescription::get_type_name () [pure virtual]
```

Get the associated `type_name`.

The type name defines a locally unique type for the publication or the subscription.

The `type_name` corresponds to a unique string used to register a type via the [DDSDomainParticipant::register_type](#) method.

Thus, the `type_name` implies an association with a corresponding type and this [DDSTopicDescription](#).

Returns

the type name. The returned type name is valid until the [DDSTopicDescription](#) is deleted.

MT Safety:

This operation is thread safe.

Postcondition

The result is NULL if self is NULL, otherwise result is non-NULL.

Implemented in [DDSTopic](#).

get_name()

```
virtual const char * DDSTopicDescription::get_name () [pure virtual]
```

Get the name used to create this [DDSTopicDescription](#) .

Returns

the name used to create this [DDSTopicDescription](#). The returned topic name is valid until the [DDSTopicDescription](#) is deleted.

MT Safety:

This operation is thread safe.

Postcondition

The result is NULL if self is NULL, otherwise result is non-NULL.

Implemented in [DDSTopic](#).

get_participant()

```
virtual DDSDomainParticipant * DDSTopicDescription::get_participant () [pure virtual]
```

Get the [DDSDomainParticipant](#) to which the [DDSTopicDescription](#) belongs.

Returns

The [DDSDomainParticipant](#) to which the [DDSTopicDescription](#) belongs.

MT Safety:

This operation is thread safe.

Postcondition

The result is NULL if self is NULL, otherwise result is non-NULL.

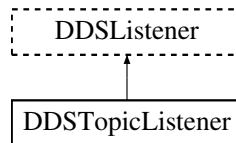
Implemented in [DDSTopic](#).

7.115 DDSTopicListener Class Reference

<<[interface](#)>> [DDSListener](#) for [DDSTopic](#) entities.

```
#include <dds_cpp_topic.hxx>
```

Inheritance diagram for DDSTopicListener:



Public Member Functions

- virtual void [on_inconsistent_topic](#) ([DDSTopic](#) *topic, const [DDS_InconsistentTopicStatus](#) &status)
Handle the [DDS_INCONSISTENT_TOPIC_STATUS](#) status.

7.115.1 Detailed Description

<<[interface](#)>> [DDSListener](#) for [DDSTopic](#) entities.

Entity:

[DDSTopic](#)

Status:

[DDS_INCONSISTENT_TOPIC_STATUS](#), [DDS_InconsistentTopicStatus](#)

This is the interface that can be implemented by an application-provided class and then registered with the [DDSTopic](#) such that the application can be notified by RTI Connex DDS Micro of relevant status changes.

See also

[Status Kinds](#)

[DDSListener](#)

7.115.2 Member Function Documentation

[on_inconsistent_topic\(\)](#)

```
virtual void DDSTopicListener::on_inconsistent_topic (
    DDSTopic * topic,
    const DDS\_InconsistentTopicStatus & status) [inline], [virtual]
```

Handle the [DDS_INCONSISTENT_TOPIC_STATUS](#) status.

This callback is called when a remote [DDSTopic](#) is discovered but is inconsistent with the locally created [DDSTopic](#) of the same topic name.

Parameters

<i>topic</i>	<<out>> Locally created DDSTopic that triggers the listener callback
<i>status</i>	<<out>> Current inconsistent status of locally created DDSTopic

7.116 DDSWaitSet Class Reference

<<[interface](#)>> Allows an application to wait until one or more of the attached [DDSCondition](#) objects has a `trigger_value` of [DDS_BOOLEAN_TRUE](#) or else until the timeout expires.

```
#include <dds_cpp_infrastructure.hxx>
```

Public Member Functions

- [DDS_ReturnCode_t](#) `wait` ([DDSConditionSeq](#) &active_conditions, const [DDS_Duration_t](#) &timeout)
Allows an application thread to wait for the occurrence of certain conditions.
- [DDS_ReturnCode_t](#) `attach_condition` ([DDSCondition](#) *cond)
Attaches a [DDSCondition](#) to the [DDSWaitSet](#).
- [DDS_ReturnCode_t](#) `detach_condition` ([DDSCondition](#) *cond)
Detaches a [DDSCondition](#) from the [DDSWaitSet](#).
- [DDS_ReturnCode_t](#) `get_conditions` ([DDSConditionSeq](#) &attached_conditions)
Retrieves the list of attached [DDSCondition](#) (s).

7.116.1 Detailed Description

<<[interface](#)>> Allows an application to wait until one or more of the attached [DDSCondition](#) objects has a `trigger_value` of [DDS_BOOLEAN_TRUE](#) or else until the timeout expires.

7.116.2 Usage

[DDSCondition](#) (s) (in conjunction with wait-sets) provide an alternative mechanism to allow the middleware to communicate communication status changes (including arrival of data) to the application.

This mechanism is wait-based. Its general use pattern is as follows:

- The application indicates which relevant information it wants to get by creating [DDSCondition](#) objects ([DDSStatusCondition](#)) and attaching them to a [DDSWaitSet](#).
- It then waits on that [DDSWaitSet](#) until the `trigger_value` of one or several [DDSCondition](#) objects become [DDS_BOOLEAN_TRUE](#).
- It then uses the result of the wait (i.e., `active_conditions`, the list of [DDSCondition](#) objects with `trigger_value == DDS\_BOOLEAN\_TRUE`) to actually get the information:

- by calling [DDSEntity::get_status_changes](#) and then `get_<communication_status>()` on the relevant [DDSEntity](#), if the condition is a [DDSStatusCondition](#) and the status changes, refer to plain communication status;
- by calling [DDSEntity::get_status_changes](#) and then [FooDataReader::read\(\)](#) or [FooDataReader::take](#) on the relevant [DDSDataReader](#), if the condition is a [DDSStatusCondition](#) and the status changes refers to [DDS_DATA_AVAILABLE_STATUS](#);

Usually the first step is done in an initialization phase, while the others are put in the application main loop.

As there is no extra information passed from the middleware to the application when a wait returns (only the list of triggered [DDSCondition](#) objects), [DDSCondition](#) objects are meant to embed all that is needed to react properly when enabled. In particular, [DDSEntity](#)-related conditions are related to exactly one [DDSEntity](#) and cannot be shared.

The result of a [DDSWaitSet::wait](#) operation depends on the state of the [DDSWaitSet](#), which in turn depends on whether at least one attached [DDSCondition](#) has a `trigger_value` of [DDS_BOOLEAN_TRUE](#). If the wait operation is called on [DDSWaitSet](#) with state BLOCKED, it will block the calling thread. If wait is called on a [DDSWaitSet](#) with state UNBLOCKED, it will return immediately. In addition, when the [DDSWaitSet](#) transitions from BLOCKED to UNBLOCKED it wakes up any threads that had called wait on it.

A key aspect of the [DDSCondition/DDSWaitSet](#) mechanism is the setting of the `trigger_value` of each [DDSCondition](#).

7.116.3 Trigger State of a ::DDSStatusCondition

The `trigger_value` of a [DDSStatusCondition](#) is the boolean OR of the `ChangedStatusFlag` of all the communication statuses (see [Status Kinds](#)) to which it is sensitive. That is, `trigger_value == DDS_BOOLEAN_FALSE` only if all the values of the `ChangedStatusFlags` are [DDS_BOOLEAN_FALSE](#).

The sensitivity of the [DDSStatusCondition](#) to a particular communication status is controlled by the list of `enabled_↵_statuses` set on the condition by means of the [DDSStatusCondition::set_enabled_statuses](#) operation.

7.116.4 Trigger State of a ::DDSGuardCondition

The `trigger_value` of a [DDSGuardCondition](#) is completely controlled by the application via the operation [DDSGuardCondition::set_trigger_value](#).

See also

[Status Kinds](#)

[DDSStatusCondition](#), [DDSGuardCondition](#)

[DDSListener](#)

7.116.5 Member Function Documentation

wait()

```
DDS_ReturnCode_t DDSWaitSet::wait (
    DDSConditionSeq & active_conditions,
    const DDS_Duration_t & timeout)
```

Allows an application thread to wait for the occurrence of certain conditions.

If none of the conditions attached to the `DDSWaitSet` have a `trigger_value` of `DDS_BOOLEAN_TRUE`, the wait operation will block suspending the calling thread.

The result of the wait operation is the list of all the attached conditions that have a `trigger_value` of `DDS_BOOLEAN_TRUE` (i.e., the conditions that unblocked the wait).

Note: The resolution of the `timeout` period is constrained by the resolution of the system clock.

The wait operation takes a `timeout` argument that specifies the maximum duration for the wait. If this duration is exceeded and none of the attached `DDSCondition` objects is `DDS_BOOLEAN_TRUE`, wait will return with the return code `DDS_RETCODE_TIMEOUT`. In this case, the resulting list of conditions will be empty.

Note: The resolution of the `timeout` period is constrained by the resolution of the system clock.

`DDS_RETCODE_TIMEOUT` will *not* be returned when the `timeout` duration is exceeded if attached `DDSCondition` objects are `DDS_BOOLEAN_TRUE`, or in the case of a `DDSWaitSet` waiting for more than one trigger event, if one or more trigger events have occurred.

It is not allowable for more than one application thread to be waiting on the same `DDSWaitSet`. If the wait operation is invoked on a `DDSWaitSet` that already has a thread blocking on it, the operation will return immediately with the value `DDS_RETCODE_PRECONDITION_NOT_MET`.

Parameters

<code>active_conditions</code>	<<inout>> a valid non-NULL <code>DDSConditionSeq</code> object. Note that RTI Connexx DDS Micro will not allocate a new object if <code>active_conditions</code> is NULL; the method will return <code>DDS_RETCODE_PRECONDITION_NOT_MET</code> .
--------------------------------	--

Parameters

<code>timeout</code>	<<in>> a wait timeout
----------------------	-----------------------

Returns

One of the [Standard Return Codes](#) or `DDS_RETCODE_PRECONDITION_NOT_MET` or `DDS_RETCODE_TIMEOUT`.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

attach_condition()

```
DDS_ReturnCode_t DDSWaitSet::attach_condition (
    DDSCondition * cond)
```

Attaches a [DDSCondition](#) to the [DDSWaitSet](#).

It is possible to attach a [DDSCondition](#) on a [DDSWaitSet](#) that is currently being waited upon (via the wait operation). In this case, if the [DDSCondition](#) has a `trigger_value` of [DDS_BOOLEAN_TRUE](#), then attaching the condition will unblock the [DDSWaitSet](#).

Parameters

<i>cond</i>	<< <i>in</i> >> Condition to be attached.
-------------	---

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_OUT_OF_RESOURCES](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

detach_condition()

```
DDS_ReturnCode_t DDSWaitSet::detach_condition (
    DDSCondition * cond)
```

Detaches a [DDSCondition](#) from the [DDSWaitSet](#).

If the [DDSCondition](#) was not attached to the [DDSWaitSet](#) the operation will return [DDS_RETCODE_BAD_PARAMETER](#).

Parameters

<i>cond</i>	<< <i>in</i> >> Condition to be detached.
-------------	---

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_PRECONDITION_NOT_MET](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

get_conditions()

```
DDS_ReturnCode_t DDSWaitSet::get_conditions (
    DDSConditionSeq & attached_conditions)
```

Retrieves the list of attached [DDSCondition](#) (s).

Parameters

<code>attached_conditions</code>	<<inout>> a DDSConditionSeq object where the list of attached conditions will be returned
----------------------------------	---

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_OUT_OF_RESOURCES](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.117 DPDE_DiscoveryPluginProperty Struct Reference

<<eXtension>> Properties for the Dynamic Participant/Dynamic Endpoint (DPDE) discovery plugin. This includes all discovery timing properties for participant discovery.

```
#include <disc_dpde_discovery_plugin.h>
```

Public Attributes

- struct [DDS_Duration_t participant_liveliness_assert_period](#)
The period to assert liveliness for the participant.
- struct [DDS_Duration_t participant_liveliness_lease_duration](#)
The liveliness lease duration for the participant.
- [DDS_Long initial_participant_announcements](#)
The number of initial announcements sent when a participant is enabled or when a remote participant is first discovered.
- struct [DDS_Duration_t initial_participant_announcement_period](#)
The period between initial announcements when a participant is first enabled or when a remote participant is newly discovered.
- [DDS_Boolean cache_serialized_samples](#)
Cache serialized discovery samples for locally created DataReaders and DataWriters.
- [DDS_Long max_participant_locators](#)
The maximum number of locators the RTI Connext DDS Micro can send to discover other participants in the same domain.
- [DDS_Long max_locators_per_discovered_participant](#)
The maximum number of locators the RTI Connext DDS Micro can keep track of per discovered participant.
- [DDS_Long max_samples_per_builtin_endpoint_reader](#)
The maximum number of samples allocated per builtin endpoint DataReader.
- [DDS_Long max_samples_per_remote_builtin_endpoint_writer](#)
The maximum number of samples allocated per remote builtin writer.
- [DDS_Long builtin_writer_max_heartbeat_retries](#)
Maximum number of heartbeat retries for built-in endpoints.
- struct [DDS_Duration_t builtin_writer_heartbeat_period](#)
Heartbeat period for built-in endpoints.
- [DDS_Long builtin_writer_heartbeats_per_max_samples](#)
Number of heartbeats per max_samples built-in endpoints.
- struct [DDS_Duration_t builtin_endpoint_reader_nack_period](#)
The period at which NACKs are sent for the builtin endpoint readers.
- [DDS_ReliabilityQosPolicyKind participant_message_reader_reliability_kind](#)
Reliability policy for a builtin endpoint DataReader.

7.117.1 Detailed Description

<<eXtension>> Properties for the Dynamic Participant/Dynamic Endpoint (DPDE) discovery plugin. This includes all discovery timing properties for participant discovery.

7.117.2 Member Data Documentation

participant_liveliness_assert_period

```
struct DDS_Duration_t DPDE_DiscoveryPluginProperty::participant_liveliness_assert_period
```

The period to assert liveliness for the participant.

Specifies how often the DomainParticipant's discovery DataWriter will assert the liveliness of the DomainParticipant to its peers.

Should be strictly less than participant_liveliness_lease_duration.

[default] 30 seconds

[range] [1 nanosec, 1 year], < participant_liveliness_lease_duration

participant_liveliness_lease_duration

```
struct DDS_Duration_t DPDE_DiscoveryPluginProperty::participant_liveliness_lease_duration
```

The liveliness lease duration for the participant.

This is the same as the expiration time of the DomainParticipant as defined in the RTPS protocol.

If the participant has not refreshed its own liveliness to other participants at least once within this period, it may be considered as stale by other participants in the network.

Should be strictly greater than participant_liveliness_assert_period.

[default] 100 seconds

[range] [1 nanosec, 1 year], > participant_liveliness_assert_period

initial_participant_announcements

```
DDS_Long DPDE_DiscoveryPluginProperty::initial_participant_announcements
```

The number of initial announcements sent when a participant is enabled or when a remote participant is first discovered.

When a participant is enabled or when a remote participant is first discovered, a participant announcement is sent immediately. The participant will continue to send initial participant announcements to peers, up to the number of times controlled by this parameter.

The first participant announcement is included in the count, and a value of 0 or 1 results in one announcement being sent.

[range] [0,2147483647]

[default] 5

initial_participant_announcement_period

```
struct DDS_Duration_t DPDE_DiscoveryPluginProperty::initial_participant_announcement_period
```

The period between initial announcements when a participant is first enabled or when a remote participant is newly discovered.

[default] 1 second

cache_serialized_samples

```
DDS_Boolean DPDE_DiscoveryPluginProperty::cache_serialized_samples
```

Cache serialized discovery samples for locally created DataReaders and DataWriters.

When a DataReader or DataWriter is enabled locally, the discovery plugin announces this to other participants in its domain. When this value is set to [DDS_BOOLEAN_TRUE](#) the serialized sample is stored in case a new participant is discovered and the information must be sent again. This may improve discovery performance, but increases memory usage. The default value is [DDS_BOOLEAN_FALSE](#) which uses less memory.

[default] DDS_BOOLEAN_FALSE

max_participant_locators

```
DDS_Long DPDE_DiscoveryPluginProperty::max_participant_locators
```

The maximum number of locators the RTI Connext DDS Micro can send to discover other participants in the same domain.

The discovery plugin sends participant announcements to a number of locators as specified in the initial peer list. The maximum number of locators to send too is specified here.

The value [DDS_LENGTH_AUTO](#) calculates the maximum number of locators as $\text{DomainParticipantQos.resource_limits.remote_participant_allocation} + (\text{DPDE_MAX_ANON_PARTICIPANT} * (\text{initial_peers length} + 1))$

The DPDE builtin participant topic [DDSDDataWriter](#) allocates `max_participant_locators` to send participant announcements to. The default value [DDS_LENGTH_AUTO](#) calculates `max_participant_locators` as $\text{remote_participant_allocation} + (\text{DPDE_MAX_ANON_PARTICIPANT} * (\text{peer_length} + 1))$ where the constant `DPDE_MAX_ANON_PARTICIPANT = 6`.

A participant locator is defined as a unique `index@address` to send to where `index` is an integer ≥ 0 and only applies to unicast addresses. For multicast address the index is 0. This index is the highest participant_id that will receive participant announcements at the address.

The maximum number of addresses to send to is defined by the peer list (either specified in [DDS_DiscoveryQosPolicy::initial_peers](#) or by calling [DDSDomainParticipant::add_peer](#). The maximum number of addresses needed is determined by adding all indices for all unicast addresses and adding one for each multicast address.

If memory is limited it is advised to set this value as low as possible.

[default] DDS_LENGTH_AUTO

max_locators_per_discovered_participant

`DDS_Long DPDE_DiscoveryPluginProperty::max_locators_per_discovered_participant`

The maximum number of locators the RTI Connex DDS Micro can keep track of per discovered participant.

When a participant discover another participant, it must keep track of which addresses to respond to for each participant.

NOTE: If two participants respond with the same addresses, for example multicast, only one address is saved.

[default] 4

max_samples_per_builtin_endpoint_reader

`DDS_Long DPDE_DiscoveryPluginProperty::max_samples_per_builtin_endpoint_reader`

The maximum number of samples allocated per builtin endpoint DataReader.

The builtin endpoint DataReaders are reliable and may cache samples received out-of-order to improve discovery performance. This resource-limit determines the maximum number of outstanding samples at any given time.

[default] 8

[range] [1, 100000000]

max_samples_per_remote_builtin_endpoint_writer

`DDS_Long DPDE_DiscoveryPluginProperty::max_samples_per_remote_builtin_endpoint_writer`

The maximum number of samples allocated per remote builtin writer.

The builtin endpoint DataReaders are reliable and may cache samples received out-of-order to improve discovery performance. This resource-limit determines the maximum number of outstanding samples at any given time per remote builtin endpoint DataWriter. Note that this limit is per builtin endpoint DataReader.

The default value `DDS_MAX_UNLIMITED` allows a remote builtin endpoint DataWriter to potentially use all samples while missing samples are being resent. This may cause samples received by other remote builtin endpoint datawriters to be rejected until the remote builtin endpoint DataWriter using all the samples frees up resources.

The following values are supported for this member:

- **DDS_MAX_UNLIMITED.** This allows each remote builtin endpoint DataWriter to use up to `DPDE_DiscoveryPluginProperty::max_sa` samples.
- [1, 256]. When a positive value is specified `DPDE_DiscoveryPluginProperty::max_samples_per_builtin_endpoint_reader` is automatically adjusted to `DPDE_DiscoveryPluginProperty::max_samples_per_remote_builtin_endpoint_writer * DDS_DomainParticipantResourceLimitsQosPolicy::remote_participant_allocation`.

Note that the value of `DPDE_DiscoveryPluginProperty::max_samples_per_builtin_endpoint_reader` is still limited to [1, 100000000] and a run-time error will occur if the value is exceeded.

[default] `DDS_MAX_UNLIMITED`

[range] `DDS_MAX_UNLIMITED`, [1, 256]

builtin_writer_max_heartbeat_retries

`DDS_Long DPDE_DiscoveryPluginProperty::builtin_writer_max_heartbeat_retries`

Maximum number of heartbeat retries for built-in endpoints.

[default] DDS_LENGTH_UNLIMITED

[range] [1, 100000000] or DDS_LENGTH_UNLIMITED.

builtin_writer_heartbeat_period

`struct DDS_Duration_t DPDE_DiscoveryPluginProperty::builtin_writer_heartbeat_period`

Heartbeat period for built-in endpoints.

[default] 100 ms.

[range] [1 nanosec, 1 year]

builtin_writer_heartbeats_per_max_samples

`DDS_Long DPDE_DiscoveryPluginProperty::builtin_writer_heartbeats_per_max_samples`

Number of heartbeats per max_samples built-in endpoints.

[default] DDS_LENGTH_UNLIMITED

[range] [1, local_writer_allocation] or DDS_LENGTH_UNLIMITED.

builtin_endpoint_reader_nack_period

`struct DDS_Duration_t DPDE_DiscoveryPluginProperty::builtin_endpoint_reader_nack_period`

The period at which NACKs are sent for the builtin endpoint readers.

Please refer to [DDS_RtpsReliableReaderProtocol_t::nack_period](#) for a detailed description.

participant_message_reader_reliability_kind

`DDS_ReliabilityQosPolicyKind DPDE_DiscoveryPluginProperty::participant_message_reader_reliability↔
_kind`

Reliability policy for a builtin endpoint DataReader.

For details, refer to the [DDS_ReliabilityQosPolicyKind](#).

[default] DDS_BEST_EFFORT_RELIABILITY_QOS

7.118 DPDEDiscoveryFactory Class Reference

<<eXtension>> Discovery plug-in used for asserting remote entities.

```
#include <dds_cpp_dpde.hxx>
```

Static Public Member Functions

- static struct RT_ComponentFactoryI * [get_interface](#) ()
Gets the component factory necessary when registering a DPDE component with RTI Connexx DDS Micro

7.118.1 Detailed Description

<<eXtension>> Discovery plug-in used for asserting remote entities.

7.118.2 Member Function Documentation

get_interface()

```
static struct RT_ComponentFactoryI * DPDEDiscoveryFactory::get_interface () [static]
```

Gets the component factory necessary when registering a DPDE component with RTI Connexx DDS Micro

7.119 DPSE_DiscoveryPluginProperty Struct Reference

<<eXtension>> Properties for the Dynamic Participant/Static Endpoint (DPSE) discovery plugin. These properties can be configured while registering the plugin to control its behavior.

```
#include <disc_dpse_dpseDiscovery.h>
```

Public Attributes

- struct [DDS_Duration_t participant_liveliness_assert_period](#)
The period to assert liveliness for the participant.
- struct [DDS_Duration_t participant_liveliness_lease_duration](#)
The liveliness lease duration for the participant.
- [DDS_Long initial_participant_announcements](#)
The number of initial announcements sent when a participant is enabled or when a remote participant is first discovered.
- struct [DDS_Duration_t initial_participant_announcement_period](#)
The period between initial announcements when a participant is first enabled or when a remote participant is newly discovered.
- [DDS_Long max_participant_locators](#)
The maximum number of locators RTI Connexx DDS Micro can send to to discover other participants in the same domain.
- [DDS_Long max_locators_per_discovered_participant](#)
The maximum number of locators RTI Connexx DDS Micro can keep track of per discovered participant.
- [DDS_ReliabilityQosPolicyKind participant_message_reader_reliability_kind](#)
Reliability policy for a built-in participant message reader.

7.119.1 Detailed Description

<<*eXtension*>> Properties for the Dynamic Participant/Static Endpoint (DPSE) discovery plugin. These properties can be configured while registering the plugin to control its behavior.

7.119.2 Member Data Documentation

participant_liveliness_assert_period

```
struct DDS_Duration_t DPSE_DiscoveryPluginProperty::participant_liveliness_assert_period
```

The period to assert liveliness for the participant.

Specifies how often the DomainParticipant's discovery DataWriter will assert the liveliness of the DomainParticipant to its peers.

Note

Should be strictly less than participant_liveliness_lease_duration.

[default] 30 seconds

[range] [1 nanosec, 1 year], < participant_liveliness_lease_duration

participant_liveliness_lease_duration

```
struct DDS_Duration_t DPSE_DiscoveryPluginProperty::participant_liveliness_lease_duration
```

The liveliness lease duration for the participant.

This is the same as the expiration time of the DomainParticipant as defined in the RTPS protocol.

If the participant has not refreshed its own liveliness to other participants at least once within this period, it may be considered as stale by other participants in the network.

Note

Should be strictly greater than participant_liveliness_assert_period.

[default] 100 seconds

[range] [1 nanosec, 1 year], > participant_liveliness_assert_period

initial_participant_announcements

`DDS_Long DPSE_DiscoveryPluginProperty::initial_participant_announcements`

The number of initial announcements sent when a participant is enabled or when a remote participant is first discovered.

When a participant is enabled or when a remote participant is first discovered, a participant announcement is sent immediately. The participant will continue to send initial participant announcements to peers, up to the number of times controlled by this parameter.

The first participant announcement is included in the count, and a value of 0 or 1 results in one announcement being sent.

[range] [0,2147483647]

[default] 5

initial_participant_announcement_period

`struct DDS_Duration_t DPSE_DiscoveryPluginProperty::initial_participant_announcement_period`

The period between initial announcements when a participant is first enabled or when a remote participant is newly discovered.

[default] 1 second

max_participant_locators

`DDS_Long DPSE_DiscoveryPluginProperty::max_participant_locators`

The maximum number of locators RTI Connex DDS Micro can send to to discover other participants in the same domain.

The discovery plugin sends participant announcements to a number of locators as specified in the initial peer list. The maximum number of locators to send too is specified here.

The value `DDS_LENGTH_AUTO` calculates the maximum number of locators as `DomainParticipantQos.resource_limits.remote_participant_allocation + (DPDE_MAX_ANON_PARTICIPANT * (initial_peers length + 1))`;

[default] `DDS_LENGTH_AUTO`;

max_locators_per_discovered_participant

`DDS_Long DPSE_DiscoveryPluginProperty::max_locators_per_discovered_participant`

The maximum number of locators RTI Connex DDS Micro can keep track of per discovered participant.

When a participant discover another participant, it must keep track of which addresses to respond to for each participant. NOTE: If two participants respond with the same addresses, for example multicast, only one address is saved.

[default] 4;

participant_message_reader_reliability_kind

`DDS_ReliabilityQosPolicyKind DPSE_DiscoveryPluginProperty::participant_message_reader_reliability←_kind`

Reliability policy for a built-in participant message reader.

For details, refer to the [DDS_ReliabilityQosPolicyKind](#).

[default] [DDS_BEST_EFFORT_RELIABILITY_QOS](#)

7.120 DPSEDiscoveryFactory Class Reference

<<*eXtension*>> A discovery factory that will be used by the [DDSDomainParticipantFactory](#) to create a discovery plugin.

```
#include <dds_cpp_dpse.hxx>
```

Static Public Member Functions

- static struct `RT_ComponentFactoryI` * [get_interface](#) ()

Gets the singleton instance of the DPSE discovery plugin factory. The singleton instance of the factory is used when creating a domain participant.

7.120.1 Detailed Description

<<*eXtension*>> A discovery factory that will be used by the [DDSDomainParticipantFactory](#) to create a discovery plugin.

7.120.2 Member Function Documentation

`get_interface()`

```
static struct RT_ComponentFactoryI * DPSEDiscoveryFactory::get_interface () [static]
```

Gets the singleton instance of the DPSE discovery plugin factory. The singleton instance of the factory is used when creating a domain participant.

This function gets the single singleton instance of the DPSE discovery plugin factory that is used by the middleware to create a DPSE plugin. In the future, as more discovery plug-ins are supported, they will be registered with the middleware by installing different discovery factory instances with the [DDSDomainParticipantFactory](#). The singleton instance of the factory is registered with a name, and that name is specified in the [DDSDomainParticipant DDS_DiscoveryQosPolicy](#). This is used to create a discovery plug-in instance when creating a domain participant.

Returns

Pointer to DPSE discovery plugin factory

See also

[DDS_DiscoveryQosPolicy](#)

7.121 DPSEDiscoveryPlugin Class Reference

<<eXtension>> Discovery plug-in used for asserting remote entities.

```
#include <dds_cpp_dpse.hxx>
```

Static Public Member Functions

- static `DDS_ReturnCode_t RemoteParticipant_assert` (`DDSDomainParticipant` *const participant, const char *rem_participant_name)
Assert remote participant by passing in the name of that participant.
- static `DDS_ReturnCode_t RemotePublication_assert` (`DDSDomainParticipant` *const participant, const char *const rem_participant_name, const struct `DDS_PublicationBuiltinTopicData` *const data, `NDDS_TypePluginKeyKind` key_kind)
Assert remote publication by passing in the builtin data that describes the type, topic, and QoS of that remote publication.
- static `DDS_ReturnCode_t RemoteSubscription_assert` (`DDSDomainParticipant` *const participant, const char *const rem_participant_name, const struct `DDS_SubscriptionBuiltinTopicData` *const data, `NDDS_TypePluginKeyKind` key_kind)
Assert remote subscription by passing in the builtin data that describes the type, topic, and QoS of that remote subscription.

7.121.1 Detailed Description

<<eXtension>> Discovery plug-in used for asserting remote entities.

7.121.2 Member Function Documentation

RemoteParticipant_assert()

```
static DDS_ReturnCode_t DPSEDiscoveryPlugin::RemoteParticipant_assert (
    DDSDomainParticipant *const participant,
    const char * rem_participant_name) [static]
```

Assert remote participant by passing in the name of that participant.

DPSE discovery requires that you pre-configure your application with information about any remote participants that you intend to discover. The only information that needs to be pre-configured is the name of the remote `DDSDomainParticipant` that your application intends to discover.

RemotePublication_assert()

```
static DDS_ReturnCode_t DPSEDiscoveryPlugin::RemotePublication_assert (
    DDSDomainParticipant *const participant,
    const char *const rem_participant_name,
    const struct DDS_PublicationBuiltinTopicData *const data,
    NDDS_TypePluginKeyKind key_kind) [static]
```

Assert remote publication by passing in the builtin data that describes the type, topic, and QoS of that remote publication.

DPSE discovery requires that you pre-configure your application with information about any remote publications that you intend to discover and receive data from. The `DDS_PublicationBuiltinTopicData` field must include the topic name, type name, and QoS of the remote `DDSDataWriter` that you intend to receive communications from.

Additionally, this method takes a key_kind argument that must match the key kind returned by the type plugin.

RemoteSubscription_assert()

```
static DDS_ReturnCode_t DPSEDiscoveryPlugin::RemoteSubscription_assert (
    DDSDomainParticipant *const participant,
    const char *const rem_participant_name,
    const struct DDS_SubscriptionBuiltinTopicData *const data,
    NDDS_TypePluginKeyKind key_kind) [static]
```

Assert remote subscription by passing in the builtin data that describes the type, topic, and QoS of that remote subscription.

DPSE discovery requires that you pre-configure your application with information about any remote subscriptions that you intend to discover and send data to. The `DDS_SubscriptionBuiltinTopicData` parameter must include the topic name, type name, and QoS of the remote `DDSDataReader` that your application intends to send data to. Additionally, if the remote `DDSDataReader` is listening on a non-default port, you must set the remote locator (including the non-default port) in the `DDS_SubscriptionBuiltinTopicData` that you intend to use to contact the remote `DDSDataReader`.

Additionally, this method takes a `key_kind` argument that must match the key kind returned by the type plugin.

7.122 FooDataReader Class Reference

Declares the interface required to support a user data type-specific data reader.

```
#include <dds_cpp_data.hxx>
```

Public Member Functions

- `DDS_ReturnCode_t return_loan` (`FooSeq` &received_data, `DDS_SampleInfoSeq` &info_seq)
Indicates to the `DDSDataReader` that the application is done accessing the collection of `received_data` and `info_seq` obtained by some earlier invocation of `read` or `take` on the `DDSDataReader`.
- `DDS_ReturnCode_t read_next_sample` (`Foo` &received_data, `DDS_SampleInfo` &sample_info)
Copies the next not-previously-accessed data value from the `DDSDataReader`.
- `DDS_ReturnCode_t take_next_sample` (`Foo` &received_data, `DDS_SampleInfo` &sample_info)
Copies the next not-previously-accessed data value from the `DDSDataReader`.
- `DDS_ReturnCode_t read` (`FooSeq` &received_data, `DDS_SampleInfoSeq` &info_seq, `DDS_Long` max_samples, `DDS_SampleStateMask` sample_states, `DDS_ViewStateMask` view_states, `DDS_InstanceStateMask` instance_states)
Access a collection of data samples from the `DDSDataReader`.
- `DDS_ReturnCode_t take` (`FooSeq` &received_data, `DDS_SampleInfoSeq` &info_seq, `DDS_Long` max_samples, `DDS_SampleStateMask` sample_states, `DDS_ViewStateMask` view_states, `DDS_InstanceStateMask` instance_states)
Access a collection of data-samples from the `DDSDataReader`.
- `DDS_ReturnCode_t read_instance` (`FooSeq` &received_data, `DDS_SampleInfoSeq` &info_seq, `DDS_Long` max_samples, const `DDS_InstanceHandle_t` &a_handle, `DDS_SampleStateMask` sample_states, `DDS_ViewStateMask` view_states, `DDS_InstanceStateMask` instance_states)
Access a collection of data samples from the `DDSDataReader`.
- `DDS_ReturnCode_t take_instance` (`FooSeq` &received_data, `DDS_SampleInfoSeq` &info_seq, `DDS_Long` max_samples, const `DDS_InstanceHandle_t` &a_handle, `DDS_SampleStateMask` sample_states, `DDS_ViewStateMask` view_states, `DDS_InstanceStateMask` instance_states)
Access a collection of data samples from the `DDSDataReader`.
- `DDS_InstanceHandle_t lookup_instance` (const `Foo` &key_holder)
Retrieve the instance `handle` that corresponds to an instance `key_holder`.
- `DDS_ReturnCode_t is_data_consistent` (`DDS_Boolean` &is_data_consistent, const `Foo` &sample, const struct `DDS_SampleInfo` &sample_info)
Checks if the sample has been overwritten by the `DataWriter`.

Static Public Member Functions

- static [FooDataReader](#) * [narrow](#) ([DDSDDataReader](#) *reader)
Narrow the given [DDSDDataReader](#) pointer to a [FooDataReader](#) pointer.

7.122.1 Detailed Description

Declares the interface required to support a user data type-specific data reader.

7.122.2 Member Function Documentation

[narrow\(\)](#)

```
static FooDataReader * FooDataReader::narrow (  
    DDSDDataReader * reader) [static]
```

Narrow the given [DDSDDataReader](#) pointer to a [FooDataReader](#) pointer.

[return_loan\(\)](#)

```
DDS\_ReturnCode\_t FooDataReader::return\_loan (  
    FooSeq & received_data,  
    DDS\_SampleInfoSeq & info_seq)
```

Indicates to the [DDSDDataReader](#) that the application is done accessing the collection of `received_data` and `info_seq` obtained by some earlier invocation of `read` or `take` on the [DDSDDataReader](#).

This operation indicates to the [DDSDDataReader](#) that the application is done accessing the collection of `received_data` and `info_seq` obtained by some earlier invocation of `read` or `take` on the [DDSDDataReader](#).

The `received_data` and `info_seq` must belong to a single related "pair"; that is, they should correspond to a pair returned from a single call to `read` or `take`. The `received_data` and `info_seq` must also have been obtained from the same [DDSDDataReader](#) to which they are returned. If either of these conditions is not met, the operation will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#).

The operation [FooDataReader::return_loan](#) allows implementations of the `read` and `take` operations to "loan" buffers from the [DDSDDataReader](#) to the application and in this manner provide "zerocopy" access to the data. During the loan, the [DDSDDataReader](#) will guarantee that the data and sample-information are not modified.

It is not necessary for an application to return the loans immediately after the `read` or `take` calls. However, as these buffers correspond to internal resources inside the [DDSDDataReader](#), the application should not retain them indefinitely.

The use of [FooDataReader::return_loan](#) is only necessary if the `read` or `take` calls "loaned" buffers to the application. This only occurs if the `received_data` and `info_seq` collections had `max_len=0` at the time `read` or `take` was called.

The application may also examine the "owns" property of the collection to determine where there is an outstanding loan. However, calling [FooDataReader::return_loan](#) on a collection that does not have a loan is safe and has no side effects.

If the collections had a loan, upon completion of [FooDataReader::return_loan](#), the collections will have `max_len=0`.

Similar to `read`, this operation must be provided on the specialized class that is generated for the particular application data-type that is being taken.

Parameters

<i>received_data</i>	<<in>> user data type-specific FooSeq object where the received data samples was obtained from earlier invocation of read or take on the DDSDDataReader .
----------------------	---

Parameters

<i>info_seq</i>	<<in>> a DDS_SampleInfoSeq object where the received sample info was obtained from earlier invocation of read or take on the DDSDDataReader .
-----------------	---

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

read_next_sample()

```
DDS_ReturnCode_t FooDataReader::read_next_sample (
    Foo & received_data,
    DDS_SampleInfo & sample_info)
```

Copies the next not-previously-accessed data value from the [DDSDDataReader](#).

This operation copies the next not-previously-accessed data value from the [DDSDDataReader](#). This operation also copies the corresponding [DDS_SampleInfo](#). The implied order among the samples stored in the [DDSDDataReader](#) is the same as for the [FooDataReader::read](#) operation.

The [FooDataReader::read_next_sample](#) operation is semantically equivalent to the [FooDataReader::read](#) operation, where the input data sequences has max_len=1, the sample_states=NOT_READ, the view_states=ANY_VIEW_STATE, and the instance_states=ANY_INSTANCE_STATE.

The [FooDataReader::read_next_sample](#) operation provides a simplified API to 'read' samples, avoiding the need for the application to manage sequences and specify states.

If there is no unread data in the [DDSDDataReader](#), the operation will fail with [DDS_RETCODE_NO_DATA](#) and nothing is copied.

Parameters

<i>received_data</i>	<<inout>> user data type-specific Foo object where the next received data sample will be returned. The received_data must have been fully allocated. Otherwise, this operation may fail.
<i>sample_info</i>	<<inout>> a DDS_SampleInfo object where the next received sample info will be returned.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataReader::read](#)

take_next_sample()

```
DDS_ReturnCode_t FooDataReader::take_next_sample (
    Foo & received_data,
    DDS_SampleInfo & sample_info)
```

Copies the next not-previously-accessed data value from the [DDSDDataReader](#).

This operation copies the next not-previously-accessed data value from the [DDSDDataReader](#) and 'removes' it from the [DDSDDataReader](#) so that it is no longer accessible. This operation also copies the corresponding [DDS_SampleInfo](#). This operation is analogous to the [FooDataReader::read_next_sample](#) except for the fact that the sample is removed from the [DDSDDataReader](#).

The [FooDataReader::take_next_sample](#) operation is semantically equivalent to the [FooDataReader::take](#) operation, where the input data sequences has max_len=1, the sample_states=NOT_READ, the view_states=ANY_VIEW_STATE, and the instance_states=ANY_INSTANCE_STATE.

The [FooDataReader::take_next_sample](#) operation provides a simplified API to 'take' samples, avoiding the need for the application to manage sequences and specify states.

If there is no unread data in the [DDSDDataReader](#), the operation will fail with [DDS_RETCODE_NO_DATA](#) and nothing is copied.

Parameters

<i>received_data</i>	<<inout>> user data type-specific Foo object where the next received data sample will be returned. The received_data must have been fully allocated. Otherwise, this operation may fail.
<i>sample_info</i>	<<inout>> a DDS_SampleInfo object where the next received sample info will be returned.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataReader::take](#)

read()

```
DDS_ReturnCode_t FooDataReader::read (
    FooSeq & received_data,
    DDS_SampleInfoSeq & info_seq,
    DDS_Long max_samples,
    DDS_SampleStateMask sample_states,
    DDS_ViewStateMask view_states,
    DDS_InstanceStateMask instance_states)
```

Access a collection of data samples from the [DDSDDataReader](#).

This operation offers the same functionality and API as [FooDataReader::take](#) except that the samples returned remain in the [DDSDDataReader](#) such that they can be retrieved again by means of a read or take operation.

Please refer to the documentation of [FooDataReader::take\(\)](#) for details on the number of samples returned within the received_data and info_seq as well as the order in which the samples appear in these sequences.

The act of reading a sample changes its sample_state to [DDS_READ_SAMPLE_STATE](#). If the sample belongs to the most recent generation of the instance, it will also set the view_state of the instance to be [DDS_NOT_NEW_VIEW_STATE](#). It will not affect the instance_state of the instance.

Important: If the samples "returned" by this method are loaned from RTI Connex Micro (see [FooDataReader::take](#) for more information on memory loaning), it is important that their contents not be changed. Because the memory in which the data is stored belongs to the middleware, any modifications made to the data will be seen the next time the same samples are read or taken; the samples will no longer reflect the state that was received from the network.

Parameters

<i>received_data</i>	<<inout>> User data type-specific FooSeq object where the received data samples will be returned.
<i>info_seq</i>	<<inout>> A DDS_SampleInfoSeq object where the received sample info will be returned.
<i>max_samples</i>	<<in>> The maximum number of samples to be returned. If the special value DDS_LENGTH_UNLIMITED is provided, as many samples will be returned as are available.
<i>sample_states</i>	<<in>> Data samples matching one of these sample_states are returned.
<i>view_states</i>	<<in>> Data samples matching one of these view_state are returned.
<i>instance_states</i>	<<in>> Data samples matching one of these instance_state are returned.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataReader::take](#),
[DDS_LENGTH_UNLIMITED](#)

take()

```
DDS_ReturnCode_t FooDataReader::take (
    FooSeq & received_data,
    DDS_SampleInfoSeq & info_seq,
    DDS_Long max_samples,
    DDS_SampleStateMask sample_states,
    DDS_ViewStateMask view_states,
    DDS_InstanceStateMask instance_states)
```

Access a collection of data-samples from the [DDSDDataReader](#).

The operation will return the list of samples received by the [DDSDDataReader](#) since the last [FooDataReader::take](#) operation that match the specified [DDS_SampleStateMask](#), [DDS_ViewStateMask](#), and [DDS_InstanceStateMask](#).

This operation may fail with [DDS_RETCODE_ERROR](#) if the [DDS_DataReaderResourceLimitsQosPolicy::max_outstanding_reads](#) limit has been exceeded.

The actual number of samples returned depends on the information that has been received by the middleware as well as the [DDS_HistoryQosPolicy](#), [DDS_ResourceLimitsQosPolicy](#), [DDS_DataReaderResourceLimitsQosPolicy](#), and the characteristics of the data-type that is associated with the [DDSDDataReader](#):

- When the [DDS_HistoryQosPolicy::kind](#) is [DDS_KEEP_LAST_HISTORY_QOS](#), the call will return a maximum of [DDS_HistoryQosPolicy::depth](#) samples for each ALIVE instance and [DDS_HistoryQosPolicy::depth](#) + 1 samples for each NOT_ALIVE instance. The extra sample is an invalid sample that indicates the instance state transition from ALIVE to NOT_ALIVE.
- When the [DDS_HistoryQosPolicy::kind](#) is [DDS_KEEP_ALL_HISTORY_QOS](#), the maximum number of samples returned is limited to [DDS_ResourceLimitsQosPolicy::max_samples](#) + 1 for each NOT_ALIVE instance.
- When [max_samples_per_instance](#) is not [DDS_LENGTH_UNLIMITED](#), the number of samples returned is also limited by the product ([DDS_ResourceLimitsQosPolicy::max_samples_per_instance](#) * [DDS_ResourceLimitsQosPolicy::max_instances](#)).

If the read or take succeeds, and the number of samples returned has been limited (by means of a maximum limit, as listed above, or insufficient [DDS_SampleInfo](#) resources), the call will complete successfully and provide those samples the reader is able to return. The user may need to make additional calls, or return outstanding loaned buffers in the case of insufficient resources, in order to access remaining samples.

Note that in the case where the [DDSTopic](#) associated with the [DDSDDataReader](#) is bound to a data-type that has no key definition, then there will be at most one instance in the [DDSDDataReader](#). So the per-sample limits will apply.

The act of *taking* a sample removes it from RTI Connex DDS Micro so it cannot be read or taken again. If the sample belongs to the most recent generation of the instance, it will also set the `view_state` of the sample's instance to [DDS_NOT_NEW_VIEW_STATE](#). It will not affect the `instance_state` of the sample's instance.

After [FooDataReader::take](#) completes, `received_data` and `info_seq` will be of the same length and contain the received data.

If the sequences are empty (maximum size equal 0) when the [FooDataReader::take](#) is called, the samples returned in the `received_data` and the corresponding `info_seq` are 'loaned' to the application from buffers provided by the [DDSDDataReader](#). The application can use them as desired and has guaranteed exclusive access to them.

Once the application completes its use of the samples it must 'return the loan' to the [DDSDataReader](#) by calling the [FooDataReader::return_loan](#) operation.

Note: While you must call [FooDataReader::return_loan](#) at some point, you do *not* have to do so before the next [FooDataReader::take](#) call. However, failure to return the loan will eventually deplete the [DDSDataReader](#) of the buffers it needs to receive new samples and eventually samples will start to be lost. The total number of buffers available to the [DDSDataReader](#) is specified by the [DDS_ResourceLimitsQosPolicy](#).

If the sequences are not empty (maximum size not equal 0) when the [FooDataReader::take](#) is called, samples are copied to `received_data` and `info_seq`. The application will not need to call [FooDataReader::return_loan](#).

The samples returned to the caller is a list where samples belonging to the same data instance are consecutive (i.e. instance access scope).

Samples belonging to the same instance will appear in the relative order in which there were received (i.e. reception timestamp destination order, where earlier samples are ahead of later samples).

If the [DDSDataReader](#) has no samples that meet the constraints, the method will fail with [DDS_RETCODE_NO_DATA](#).

In addition to the collection of samples, the read and take operations also use a collection of [DDS_SampleInfo](#) structures.

7.122.3 SEQUENCES USAGE IN TAKE AND READ

The initial (input) properties of the `received_data` and `info_seq` collections will determine the precise behavior of the read or take operation. For the purposes of this description, the collections are modeled as having these properties:

- whether the collection container owns the memory of the elements within (`owns`, see [::FooSeq::has_ownership](#))
- the current-length (`len`, see [::FooSeq::length\(\)](#))
- the maximum length (`max_len`, see [::FooSeq::maximum\(\)](#))

The initial values of the `owns`, `len` and `max_len` properties for the `received_data` and `info_seq` collections govern the behavior of the read and take operations as specified by the following rules:

1. The values of `owns`, `len` and `max_len` properties for the two collections must be identical. Otherwise read/take will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#).
2. On successful output, the values of `owns`, `len` and `max_len` will be the same for both collections.
3. If the initial `max_len==0`, then the `received_data` and `info_seq` collections will be filled with elements that are loaned by the [DDSDataReader](#). On output, `owns` will be `FALSE`, `len` will be set to the number of values returned, and `max_len` will be set to a value verifying `max_len >= len`. The use of this variant allows for zero-copy access to the data and the application will need to return the loan to the [DDSDataReader](#) using [FooDataReader::return_loan](#).
4. If the initial `max_len>0` and `owns==FALSE`, then the read or take operation will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#). This avoids the potential hard-to-detect memory leaks caused by an application forgetting to return the loan.
5. If initial `max_len>0` and `owns==TRUE`, then the read or take operation will copy the `received_data` values and [DDS_SampleInfo](#) values into the elements already inside the collections. On output, `owns` will be `TRUE`, `len` will be set to the number of values copied and `max_len` will remain unchanged. The use of this variant forces a copy but the application can control where the copy is placed and the application will not need to return the loan. The number of samples copied depends on the relative values of `max_len` and `max_samples`:

- If `max_samples == LENGTH_UNLIMITED`, then at most `max_len` values will be copied. The use of this variant lets the application limit the number of samples returned to what the sequence can accommodate.
- If `max_samples <= max_len`, then at most `max_samples` values will be copied. The use of this variant lets the application limit the number of samples returned to fewer than what the sequence can accommodate.
- If `max_samples > max_len`, then the read or take operation will fail with `DDS_RETCODE_PRECONDITION_NOT_MET`. This avoids the potential confusion where the application expects to be able to access up to `max_samples`, but that number can never be returned, even if they are available in the `DDSDataReader`, because the output sequence cannot accommodate them.

As described above, upon completion, the `received_data` and `info_seq` collections may contain elements loaned from the `DDSDataReader`. If this is the case, the application will need to use `FooDataReader::return_loan` to return the loan once it is no longer using the `received_data` in the collection. When `FooDataReader::return_loan` completes, the collection will have `owns=FALSE` and `max_len=0`. The application can determine whether it is necessary to return the loan or not based on how the state of the collections when the read/take operation was called or by accessing the `owns` property. However, in many cases it may be simpler to always call `FooDataReader::return_loan`, as this operation is harmless (i.e., it leaves all elements unchanged) if the collection does not have a loan.

To avoid potential memory leaks, the implementation of the `Foo` and `DDS_SampleInfo` collections should disallow changing the length of a collection for which `owns==FALSE`. Furthermore, deleting a collection for which `owns==FALSE` should be considered an error.

On output, the collection of `Foo` values and the collection of `DDS_SampleInfo` structures are of the same length and are in a one-to-one correspondence. Each `DDS_SampleInfo` provides information, such as the `source_timestamp`, the `sample_state`, `view_state`, and `instance_state`, etc., about the corresponding sample.

Some elements in the returned collection may not have valid data. If the `instance_state` in the `DDS_SampleInfo` is `DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE` or `DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE`, then the last sample for that instance in the collection (that is, the one whose `DDS_SampleInfo` has `sample_rank==0`) does not contain valid data.

Samples that contain no data do not count towards the limits imposed by the `DDS_ResourceLimitsQosPolicy`. The act of reading/taking a sample sets its `sample_state` to `DDS_READ_SAMPLE_STATE`.

If the sample belongs to the most recent generation of the instance, it will also set the `view_state` of the instance to `DDS_NOT_NEW_VIEW_STATE`. It will not affect the `instance_state` of the instance.

This operation must be provided on the specialized class that is generated for the particular application data-type that is being read (`Foo`). If the `DDSDataReader` has no samples that meet the constraints, the operation fails with `DDS_RETCODE_NO_DATA`.

Parameters

<i>received_data</i>	<<inout>> User data type-specific <code>FooSeq</code> object where the received data samples will be returned.
----------------------	--

Parameters

<i>info_seq</i>	<<inout>> A <code>DDS_SampleInfoSeq</code> object where the received sample info will be returned.
<i>max_samples</i>	<<in>> The maximum number of samples to be returned. If the special value <code>DDS_LENGTH_UNLIMITED</code> is provided, as many samples will be returned as are available.
<i>sample_states</i>	<<in>> Data samples matching one of these <code>sample_states</code> are returned.
<i>view_states</i>	<<in>> Data samples matching one of these <code>view_state</code> are returned.
<i>instance_states</i>	<<in>> Data samples matching one of these <code>instance_state</code> are returned.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataReader::read](#)

[DDS_LENGTH_UNLIMITED](#)

read_instance()

```
DDS_ReturnCode_t FooDataReader::read_instance (
    FooSeq & received_data,
    DDS_SampleInfoSeq & info_seq,
    DDS_Long max_samples,
    const DDS_InstanceHandle_t & a_handle,
    DDS_SampleStateMask sample_states,
    DDS_ViewStateMask view_states,
    DDS_InstanceStateMask instance_states)
```

Access a collection of data samples from the [DDSDataReader](#).

This operation accesses a collection of data values from the [DDSDataReader](#). The behavior is identical to [FooDataReader::read](#), except that all samples returned belong to the single specified instance whose handle is `a_handle`.

Upon successful completion, the data collection will contain samples all belonging to the same instance. The corresponding [DDS_SampleInfo](#) verifies [DDS_SampleInfo::instance_handle](#) == `a_handle`.

The [FooDataReader::read_instance](#) operation is semantically equivalent to the [FooDataReader::read](#) operation, except in building the collection, the [DDSDataReader](#) will check that the sample belongs to the specified instance and otherwise it will not place the sample in the returned collection.

The behavior of the [FooDataReader::read_instance](#) operation follows the same rules as the [FooDataReader::read](#) operation regarding the pre-conditions and post-conditions for the `received_data` and `sample_info`. Similar to the [FooDataReader::read](#), the [FooDataReader::read_instance](#) operation may 'loan' elements to the output collections, which must then be returned by means of [FooDataReader::return_loan](#).

Similar to the [FooDataReader::read](#), this operation must be provided on the specialized class that is generated for the particular application data-type that is being taken.

If the [DDSDataReader](#) has no samples that meet the constraints, the method will fail with [DDS_RETCODE_NO_DATA](#).

This operation may fail with [DDS_RETCODE_BAD_PARAMETER](#) if the [DDS_InstanceHandle_t](#) `a_handle` does not correspond to an existing data-object known to the [DDSDataReader](#).

Parameters

<i>received_data</i>	<< <i>inout</i> >> user data type-specific FooSeq object where the received data samples will be returned.
<i>info_seq</i>	<< <i>inout</i> >> a DDS_SampleInfoSeq object where the received sample info will be returned.
<i>max_samples</i>	<< <i>in</i> >> The maximum number of samples to be returned. If the special value DDS_LENGTH_UNLIMITED is provided, as many samples will be returned as are available.
<i>a_handle</i>	<< <i>in</i> >> The specified instance to return samples for. The method will fail with DDS_RETCODE_BAD_PARAMETER if the <code>handle</code> does not correspond to an existing data-object known to the DDSDDataReader .
<i>sample_states</i>	<< <i>in</i> >> data samples matching ones of these <code>sample_states</code> are returned
<i>view_states</i>	<< <i>in</i> >> data samples matching ones of these <code>view_state</code> are returned
<i>instance_states</i>	<< <i>in</i> >> data samples matching ones of these <code>instance_state</code> are returned

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataReader::read](#)

[DDS_LENGTH_UNLIMITED](#)

take_instance()

```
DDS_ReturnCode_t FooDataReader::take_instance (
    FooSeq & received_data,
    DDS_SampleInfoSeq & info_seq,
    DDS_Long max_samples,
    const DDS_InstanceHandle_t & a_handle,
    DDS_SampleStateMask sample_states,
    DDS_ViewStateMask view_states,
    DDS_InstanceStateMask instance_states)
```

Access a collection of data samples from the [DDSDDataReader](#).

This operation accesses a collection of data values from the [DDSDDataReader](#). The behavior is identical to [FooDataReader::take](#), except for that all samples returned belong to the single specified instance whose handle is `a_handle`.

The semantics are the same for the [FooDataReader::take](#) operation, except in building the collection, the [DDSDDataReader](#) will check that the sample belongs to the specified instance, and otherwise it will not place the sample in the returned collection.

The behavior of the [FooDataReader::take_instance](#) operation follows the same rules as the [FooDataReader::read](#) operation regarding the pre-conditions and post-conditions for the `received_data` and `sample_info`. Similar to the [FooDataReader::read](#), the [FooDataReader::take_instance](#) operation may 'loan' elements to the output collections, which must then be returned by means of [FooDataReader::return_loan](#).

Similar to the [FooDataReader::read](#), this operation must be provided on the specialized class that is generated for the particular application data-type that is being taken.

If the [DDSDDataReader](#) has no samples that meet the constraints, the method fails with [DDS_RETCODE_NO_DATA](#).

This operation may fail with [DDS_RETCODE_BAD_PARAMETER](#) if the [DDS_InstanceHandle_t](#) `a_handle` does not correspond to an existing data-object known to the [DDSDDataReader](#).

Parameters

<i>received_data</i>	<< <i>inout</i> >> user data type-specific FooSeq object where the received data samples will be returned.
<i>info_seq</i>	<< <i>inout</i> >> a DDS_SampleInfoSeq object where the received sample info will be returned.
<i>max_samples</i>	<< <i>in</i> >> The maximum number of samples to be returned. If the special value DDS_LENGTH_UNLIMITED is provided, as many samples will be returned as are available.
<i>a_handle</i>	<< <i>in</i> >> The specified instance to return samples for. The method will fail with DDS_RETCODE_BAD_PARAMETER if the <code>handle</code> does not correspond to an existing data-object known to the DDSDDataReader .
<i>sample_states</i>	<< <i>in</i> >> data samples matching ones of these <code>sample_states</code> are returned
<i>view_states</i>	<< <i>in</i> >> data samples matching ones of these <code>view_state</code> are returned
<i>instance_states</i>	<< <i>in</i> >> data samples matching ones of these <code>instance_state</code> are returned

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataReader::take](#)

[DDS_LENGTH_UNLIMITED](#)

lookup_instance()

```
DDS_InstanceHandle_t FooDataReader::lookup_instance (
    const Foo & key_holder)
```

Retrieve the instance `handle` that corresponds to an instance `key_holder`.

Useful for keyed data types.

This operation takes as a parameter an instance and returns a handle that can be used in subsequent operations that accept an instance handle as an argument. The instance parameter is only used for the purpose of examining the fields that define the key. This operation does not register the instance in question. If the instance has not been previously registered, or if for any other reason the Service is unable to provide an instance handle, the Service will return the special value `HANDLE_NIL`.

Parameters

<i>key_holder</i>	<< <i>in</i> >> a user data type specific key holder.
-------------------	---

Returns

the instance handle associated with this instance. If Foo has no key, this method has no effect and returns `DDS_HANDLE_NIL`

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

is_data_consistent()

```
DDS_ReturnCode_t FooDataReader::is_data_consistent (
    DDS_Boolean & is_data_consistent,
    const Foo & sample,
    const struct DDS_SampleInfo & sample_info)
```

Checks if the sample has been overwritten by the DataWriter.

When a sample is received via Zero Copy transfer over shared memory, the sample can be reused by the DataWriter once it is removed from the DataWriter's send queue. Since there is no synchronization between the DataReader and the DataWriter, the sample could be overwritten by the DataWriter before it is processed by the DataReader. The `FooDataReader::is_data_consistent` operation can be used after processing the sample to check if it was overwritten by the DataWriter.

A precondition for using this operation is to set `DDS_DataWriterShmemRefTransferModeSettings::enable_data_consistency_check` to true.

Parameters

<i>is_data_consistent</i>	<< <i>inout</i> >> Set to true if the sample is consistent (i.e., the sample has not been overwritten by the DataWriter)
<i>sample</i>	<< <i>in</i> >> Sample to be validated
<i>sample_info</i>	<< <i>in</i> >> DDS_SampleInfo object received with the sample

Returns

One of the [Standard Return Codes](#) or [DDS_RETCODE_PRECONDITION_NOT_MET](#).

References [is_data_consistent\(\)](#).

Referenced by [is_data_consistent\(\)](#).

7.123 FooDataWriter Class Reference

Declares the interface required to support a user data type-specific data writer.

```
#include <dds_cpp_data.hxx>
```

Public Member Functions

- [DDSDataWriter](#) * [as_datawriter](#) ()
Widen the given [FooDataWriter](#) pointer to a [DDSDataWriter](#) pointer.
- [DDS_ReturnCode_t](#) [write](#) (const Foo &instance_data, const [DDS_InstanceHandle_t](#) &handle)
Modifies the value of a data instance.
- [DDS_InstanceHandle_t](#) [register_instance](#) (const Foo &instance_data)
Informs RTI Connext DDS Micro that the application will be modifying a particular instance.
- [DDS_InstanceHandle_t](#) [register_instance_w_timestamp](#) (const Foo &instance_data, const [DDS_Time_t](#) &source_timestamp)
Performs the same functions as [register_instance](#) except that the application provides the value for the `source_timestamp`.
- [DDS_ReturnCode_t](#) [unregister_instance](#) (const Foo &instance_data, const [DDS_InstanceHandle_t](#) &handle)
Reverses the action of [FooDataWriter::register_instance](#).
- [DDS_ReturnCode_t](#) [unregister_instance_w_timestamp](#) (const Foo &instance_data, const [DDS_InstanceHandle_t](#) &handle, const [DDS_Time_t](#) &source_timestamp)
Performs the same function as [FooDataWriter::unregister_instance](#) except that it also provides the value for the `source_timestamp`.
- [DDS_ReturnCode_t](#) [dispose](#) (const Foo &instance_data, const [DDS_InstanceHandle_t](#) &handle)
Requests the middleware to delete the data.
- [DDS_ReturnCode_t](#) [dispose_w_timestamp](#) (const Foo &instance_data, const [DDS_InstanceHandle_t](#) &handle, const [DDS_Time_t](#) &source_timestamp)
Performs the same functions as [dispose](#) except that the application provides the value for the `source_timestamp` that is made available to [DDSDataReader](#) objects by means of the `source_timestamp` attribute inside the [DDS_SampleInfo](#).

- `DDS_ReturnCode_t write_w_timestamp` (const Foo &instance_data, const `DDS_InstanceHandle_t` &handle, const `DDS_Time_t` &source_timestamp)
Performs the same function as `FooDataWriter::write` except that it also provides the value for the `source_timestamp`.
- `DDS_ReturnCode_t get_loan` (Foo *&sample)
Gets a sample managed by the DataWriter.
- `DDS_ReturnCode_t discard_loan` (Foo &sample)
Returns a loaned sample back to the DataWriter.
- Foo * `create_data` ()
Creates a data sample and initializes it.
- bool `delete_data` (Foo *sample)
Destroys a user data type instance.

Static Public Member Functions

- static `FooDataWriter * narrow` (`DDSDDataWriter *writer`)
Narrow the given `DDSDDataWriter` pointer to a `FooDataWriter` pointer.

7.123.1 Detailed Description

Declares the interface required to support a user data type-specific data writer.

7.123.2 Member Function Documentation

`narrow()`

```
static FooDataWriter * FooDataWriter::narrow (  
    DDSDDataWriter * writer) [static]
```

Narrow the given `DDSDDataWriter` pointer to a `FooDataWriter` pointer.

Check if the given `writer` is of type `FooDataWriter`.

Parameters

<code>writer</code>	<<in>> Base-class <code>DDSDDataWriter</code> to be converted to the auto-generated class <code>FooDataWriter</code> that extends <code>DDSDDataWriter</code> .
---------------------	---

Returns

`FooDataWriter` if `writer` is of type Foo. Return NULL otherwise.

as_datawriter()

```
DDSDataWriter * FooDataWriter::as_datawriter ()
```

Widen the given [FooDataWriter](#) pointer to a [DDSDataWriter](#) pointer.

Returns

[DDSDataWriter](#).

write()

```
DDS_ReturnCode_t FooDataWriter::write (
    const Foo & instance_data,
    const DDS_InstanceHandle_t & handle)
```

Modifies the value of a data instance.

When this operation is used, RTI Connex DDS Micro will automatically supply the value of the `source_timestamp` that is made available to [DDSDataReader](#) objects by means of the `source_timestamp` attribute inside the [DDS_SampleInfo](#). (Refer to [DDS_SampleInfo](#) details).

As a side effect, this operation asserts liveness on the [DDSDataWriter](#) itself, the [DDSPublisher](#) and the [DDSDomainParticipant](#).

Note that the special value [DDS_HANDLE_NIL](#) can be used for the parameter `handle`. This indicates the identity of the instance should be automatically deduced from the `instance_data` (by means of the `key`).

If `handle` is any value other than [DDS_HANDLE_NIL](#), then it must correspond to an instance that has been registered. If there is no correspondence, the operation will fail with [DDS_RETCODE_BAD_PARAMETER](#).

RTI Connex DDS Micro will not detect the error when the `handle` is any value other than [DDS_HANDLE_NIL](#), corresponds to an instance that has been registered, but does not correspond to the instance deduced from the `instance_data` (by means of the `key`). RTI Connex DDS Micro will treat as if the [write\(\)](#) operation is for the instance as indicated by the `handle`.

This operation may block if the [RELIABILITY](#) kind is set to [DDS_RELIABLE_RELIABILITY_QOS](#) and the modification would cause data to be lost or else cause one of the limits specified in the [RESOURCE_LIMITS](#) to be exceeded.

Specifically, this operation may block in the following situations (note that the list may not be exhaustive), even if its [DDS_HistoryQosPolicyKind](#) is [DDS_KEEP_LAST_HISTORY_QOS](#):

- If ([DDS_ResourceLimitsQosPolicy::max_samples](#) < [DDS_ResourceLimitsQosPolicy::max_instances](#) * [DDS_HistoryQosPolicy::depth](#)), then in the situation where the `max_samples` resource limit is exhausted, RTI Connex DDS Micro is allowed to discard samples of some other instance, as long as at least one sample remains for such an instance. If it is still not possible to make space available to store the modification, the writer is allowed to block.
- If ([DDS_ResourceLimitsQosPolicy::max_samples](#) < [DDS_ResourceLimitsQosPolicy::max_instances](#)), then the DataWriter may block regardless of the [DDS_HistoryQosPolicy::depth](#).

If this operation *does* block for any of the above reasons, the [RELIABILITY](#) `max_blocking_time` configures the maximum time the write operation may block (waiting for space to become available). If `max_blocking_time` elapses before the [DDSDataWriter](#) is able to store the modification without exceeding the limits, the operation will time out ([DDS_RETCODE_TIMEOUT](#)).

If there are no instance resources left, this operation may fail with [DDS_RETCODE_OUT_OF_RESOURCES](#). Calling [FooDataWriter::unregister_instance](#) may help freeing up some resources.

Parameters

<i>instance_data</i>	<< <i>in</i> >> The data to write.
<i>handle</i>	<< <i>in</i> >> Either the handle returned by a previous call to FooDataWriter::register_instance , or else the special value DDS_HANDLE_NIL . If <i>Foo</i> has a key and <i>handle</i> is not DDS_HANDLE_NIL , <i>handle</i> must represent a registered instance of type <i>Foo</i> . Otherwise, this method may fail with DDS_RETCODE_BAD_PARAMETER .

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_TIMEOUT](#), [DDS_RETCODE_OUT_OF_RESOURCES](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDSDataReader](#)

[FooDataWriter::write_w_timestamp](#)

register_instance()

```
DDS_InstanceHandle_t FooDataWriter::register_instance (
    const Foo & instance_data)
```

Informs RTI Connex DDS Micro that the application will be modifying a particular instance.

This operation is only useful for keyed data types. Using it for non-keyed types causes no effect and returns [DDS_HANDLE_NIL](#). The operation takes as a parameter an instance (of which only the key value is examined) and returns a *handle* that can be used in successive [write\(\)](#) or [dispose\(\)](#) operations.

The operation gives RTI Connex DDS Micro an opportunity to pre-configure itself to improve performance.

The use of this operation by an application is optional even for keyed types. If an instance has not been pre-registered, the application can use the special value [DDS_HANDLE_NIL](#) as the [DDS_InstanceHandle_t](#) parameter to the write or dispose operation and RTI Connex DDS Micro will auto-register the instance.

For best performance, the operation should be invoked prior to calling any operation that modifies the instance, such as [FooDataWriter::write](#), [FooDataWriter::write_w_timestamp](#), [FooDataWriter::dispose](#) and [FooDataWriter::dispose_w_timestamp](#) and the handle used in conjunction with the data for those calls.

When this operation is used, RTI Connex DDS Micro will automatically supply the value of the *source_timestamp* that is used.

This operation may fail and return [DDS_HANDLE_NIL](#) if [DDS_ResourceLimitsQosPolicy::max_instances](#) limit has been exceeded.

The operation is **idempotent**. If it is called for an already registered instance, it just returns the already allocated handle. This may be used to lookup and retrieve the handle allocated to a given instance.

This operation can only be called after [DDSDataWriter](#) has been enabled. Otherwise, [DDS_HANDLE_NIL](#) will be returned.

Parameters

<i>instance_data</i>	<<in>> The instance that should be registered. Of this instance, only the fields that represent the key are examined by the method. .
----------------------	---

Returns

For keyed data type, a handle that can be used in the calls that take a [DDS_InstanceHandle_t](#), such as `write`, `dispose`, `unregister_instance`, or return [DDS_HANDLE_NIL](#) on failure. If the `instance_data` is of a data type that has no keys, this method always return [DDS_HANDLE_NIL](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataWriter::unregister_instance](#)

register_instance_w_timestamp()

```
DDS_InstanceHandle_t FooDataWriter::register_instance_w_timestamp (
    const Foo & instance_data,
    const DDS_Time_t & source_timestamp)
```

Performs the same functions as `register_instance` except that the application provides the value for the `source_timestamp`.

The provided `source_timestamp` potentially affects the relative order in which readers observe events from multiple writers.

This operation may fail and return [DDS_HANDLE_NIL](#) if [DDS_ResourceLimitsQosPolicy::max_instances](#) limit has been exceeded.

This operation can only be called after [DDSDDataWriter](#) has been enabled. Otherwise, [DDS_HANDLE_NIL](#) will be returned.

Parameters

<i>instance_data</i>	<<in>> The instance that should be registered. Of this instance, only the fields that represent the key are examined by the method.
<i>source_timestamp</i>	<<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers.

Returns

For keyed data type, return a handle that can be used in the calls that take a [DDS_InstanceHandle_t](#), such as [write](#), [dispose](#), [unregister_instance](#), or return [DDS_HANDLE_NIL](#) on failure. If the `instance_data` is of a data type that has no keys, this method always return [DDS_HANDLE_NIL](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataWriter::unregister_instance](#)

unregister_instance()

```
DDS_ReturnCode_t FooDataWriter::unregister_instance (
    const Foo & instance_data,
    const DDS_InstanceHandle_t & handle)
```

Reverses the action of [FooDataWriter::register_instance](#).

This operation is useful only for keyed data types. Using it for non-keyed types causes no effect and reports no error. The operation takes as a parameter an instance (of which only the key value is examined) and a handle.

This operation should only be called on an instance that is currently registered. This includes instances that have been auto-registered by calling operations such as [write](#) or [dispose](#) as described in [FooDataWriter::register_instance](#). Otherwise, this operation may fail with [DDS_RETCODE_BAD_PARAMETER](#).

This only need be called just once per instance, regardless of how many times [register_instance](#) was called for that instance.

When this operation is used, RTI Connexx DDS Micro will automatically supply the value of the `source_timestamp` that is used.

This operation informs RTI Connexx DDS Micro that the [DDSDataWriter](#) is no longer going to provide any information about the instance. This operation also indicates that RTI Connexx DDS Micro can locally remove all information regarding that instance. The application should not attempt to use the `handle` previously allocated to that instance after calling [FooDataWriter::unregister_instance\(\)](#).

The special value [DDS_HANDLE_NIL](#) can be used for the parameter `handle`. This indicates that the identity of the instance should be automatically deduced from the `instance_data` (by means of the key).

If `handle` is any value other than [DDS_HANDLE_NIL](#), then it must correspond to an instance that has been registered. If there is no correspondence, the operation will fail with [DDS_RETCODE_BAD_PARAMETER](#).

RTI Connexx DDS Micro will not detect the error when the `handle` is any value other than [DDS_HANDLE_NIL](#), corresponds to an instance that has been registered, but does not correspond to the instance deduced from the `instance_data` (by means of the key). RTI Connexx DDS Micro will treat as if the [unregister_instance\(\)](#) operation is for the instance as indicated by the `handle`.

If after a `FooDataWriter::unregister_instance`, the application wants to modify (`FooDataWriter::write` or `FooDataWriter::dispose`) an instance, it has to register it again, or else use the special `handle` value `DDS_HANDLE_NIL`.

This operation does not indicate that the instance is deleted (that is the purpose of `FooDataWriter::dispose`). The operation `FooDataWriter::unregister_instance` just indicates that the `DDSDataWriter` no longer has anything to say about the instance. `DDSDataReader` entities that are reading the instance may receive a sample with `DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE` for the instance, unless there are other `DDSDataWriter` objects writing that same instance.

This operation can affect the ownership of the data instance (see [OWNERSHIP](#)). If the `DDSDataWriter` was the exclusive owner of the instance, then calling `unregister_instance()` will relinquish that ownership.

If `DDS_ReliabilityQosPolicy::kind` is set to `DDS_RELIABLE_RELIABILITY_QOS` and the unregistration would overflow the resource limits of this writer or of a reader, this operation may block for up to `DDS_ReliabilityQosPolicy::max_blocking_time`; if this writer is still unable to unregister after that period, this method will fail with `DDS_RETCODE_TIMEOUT`.

Parameters

<i>instance_data</i>	<<in>> The instance that should be unregistered. If <i>Foo</i> has a key and <code>instance_handle</code> is <code>DDS_HANDLE_NIL</code> , only the fields that represent the key are examined by the method. Otherwise, <code>instance_data</code> is not used. If <code>instance_data</code> is used, it must represent an instance that has been registered. Otherwise, this method may fail with <code>DDS_RETCODE_BAD_PARAMETER</code> .
<i>handle</i>	<<in>> represents the instance to be unregistered. If <i>Foo</i> has a key and <code>handle</code> is <code>DDS_HANDLE_NIL</code> , <code>handle</code> is not used and <code>instance</code> is deduced from <code>instance_data</code> . If <i>Foo</i> has no key, <code>handle</code> is not used. If <code>handle</code> is used, it must represent an instance that has been registered. Otherwise, this method may fail with <code>DDS_RETCODE_BAD_PARAMETER</code> .

Returns

One of the [Standard Return Codes](#), `DDS_RETCODE_TIMEOUT` or `DDS_RETCODE_NOT_ENABLED`

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataWriter::register_instance](#)

[FooDataWriter::unregister_instance_w_timestamp](#)

unregister_instance_w_timestamp()

```
DDS_ReturnCode_t FooDataWriter::unregister_instance_w_timestamp (
    const Foo & instance_data,
    const DDS_InstanceHandle_t & handle,
    const DDS_Time_t & source_timestamp)
```

Performs the same function as [FooDataWriter::unregister_instance](#) except that it also provides the value for the `source_timestamp`.

The provided `source_timestamp` potentially affects the relative order in which readers observe events from multiple writers.

The constraints on the values of the `handle` parameter and the corresponding error behavior are the same specified for the [FooDataWriter::unregister_instance](#) operation.

This operation may block and may time out ([DDS_RETCODE_TIMEOUT](#)) under the same circumstances described for the `unregister_instance` operation.

Parameters

<i>instance_data</i>	<<in>> The instance that should be unregistered. If <i>Foo</i> has a key and <i>instance_handle</i> is DDS_HANDLE_NIL , only the fields that represent the key are examined by the method. Otherwise, <i>instance_data</i> is not used. If <i>instance_data</i> is used, it must represent an instance that has been registered. Otherwise, this method may fail with DDS_RETCODE_BAD_PARAMETER .
<i>handle</i>	<<in>> represents the <i>instance</i> to be unregistered. If <i>Foo</i> has a key and <i>handle</i> is DDS_HANDLE_NIL , <i>handle</i> is not used and <i>instance</i> is deduced from <i>instance_data</i> . If <i>Foo</i> has no key, <i>handle</i> is not used. If <i>handle</i> is used, it must represent an instance that has been registered. Otherwise, this method may fail with DDS_RETCODE_BAD_PARAMETER .
<i>source_timestamp</i>	<<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_TIMEOUT](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataWriter::register_instance](#)

[FooDataWriter::unregister_instance](#)

dispose()

```
DDS_ReturnCode_t FooDataWriter::dispose (
    const Foo & instance_data,
    const DDS_InstanceHandle_t & handle)
```

Requests the middleware to delete the data.

This operation is useful only for keyed data types. Using it for non-keyed types has no effect and reports no error.

The actual deletion is postponed until there is no more use for that data in the whole system.

Applications are made aware of the deletion by means of operations on the [DDSDataReader](#) objects that already knew that instance. [DDSDataReader](#) objects that didn't know the instance will never see it.

This operation does not modify the value of the instance. The *instance_data* parameter is passed just for the purposes of identifying the instance.

When this operation is used, RTI Connext DDS Micro will automatically supply the value of the `source_timestamp` that is made available to `DDSDataReader` objects by means of the `source_timestamp` attribute inside the `DDS_SampleInfo`.

The constraints on the values of the handle parameter and the corresponding error behavior are the same specified for the `FooDataWriter::unregister_instance` operation.

The special value `DDS_HANDLE_NIL` can be used for the parameter `instance_handle`. This indicates the identity of the instance should be automatically deduced from the `instance_data` (by means of the key).

If `handle` is any value other than `DDS_HANDLE_NIL`, then it must correspond to an instance that has been registered. If there is no correspondence, the operation will fail with `DDS_RETCODE_BAD_PARAMETER`.

RTI Connext DDS Micro will not detect the error when the `handle` is any value other than `DDS_HANDLE_NIL`, corresponds to an instance that has been registered, but does not correspond to the instance deduced from the `instance_data` (by means of the key). RTI Connext DDS Micro will treat as if the `dispose()` operation is for the instance as indicated by the `handle`.

This operation may block and time out (`DDS_RETCODE_TIMEOUT`) under the same circumstances described for `FooDataWriter::write()`.

If there are no instance resources left, this operation may fail with `DDS_RETCODE_OUT_OF_RESOURCES`. Calling `FooDataWriter::unregister_instance` may help freeing up some resources.

Parameters

<i>instance_data</i>	<<in>> The data to dispose. If Foo has a key and <code>instance_handle</code> is <code>DDS_HANDLE_NIL</code> , only the fields that represent the key are examined by the method. Otherwise, <code>instance_data</code> is not used. Foo If Foo has a key, <code>instance_data</code> can be NULL only if <code>instance_handle</code> is not <code>DDS_HANDLE_NIL</code> . Otherwise, this method will fail with <code>DDS_RETCODE_BAD_PARAMETER</code> .
<i>handle</i>	<<in>> Either the handle returned by a previous call to <code>FooDataWriter::register_instance</code> , or else the special value <code>DDS_HANDLE_NIL</code> . If Foo has a key and <code>instance_handle</code> is <code>DDS_HANDLE_NIL</code> , <code>instance_handle</code> is not used and <code>instance</code> is deduced from <code>instance_data</code> . If Foo has no key, <code>instance_handle</code> is not used. If <code>handle</code> is used, it must represent a registered instance of type Foo. Otherwise, this method fail with <code>DDS_RETCODE_BAD_PARAMETER</code> .

Returns

One of the [Standard Return Codes](#), `DDS_RETCODE_TIMEOUT`, `DDS_RETCODE_OUT_OF_RESOURCES` or `DDS_RETCODE_NOT_ENABLED`.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataWriter::dispose_w_timestamp](#)

dispose_w_timestamp()

```
DDS_ReturnCode_t FooDataWriter::dispose_w_timestamp (
    const Foo & instance_data,
    const DDS_InstanceHandle_t & handle,
    const DDS_Time_t & source_timestamp)
```

Performs the same functions as `dispose` except that the application provides the value for the `source_timestamp` that is made available to `DDSDataReader` objects by means of the `source_timestamp` attribute inside the `DDS_SampleInfo`.

The constraints on the values of the `handle` parameter and the corresponding error behavior are the same specified for the `FooDataWriter::dispose` operation.

This operation may block and time out (`DDS_RETCODE_TIMEOUT`) under the same circumstances described for `FooDataWriter::write`.

If there are no instance resources left, this operation may fail with `DDS_RETCODE_OUT_OF_RESOURCES`. Calling `FooDataWriter::unregister_instance` may help freeing up some resources.

Parameters

<i>instance_data</i>	<<in>> The data to dispose. If <i>Foo</i> has a key and <i>instance_handle</i> is <code>DDS_HANDLE_NIL</code> , only the fields that represent the key are examined by the method. Otherwise, <i>instance_data</i> is not used.
<i>handle</i>	<<in>> Either the handle returned by a previous call to <code>FooDataWriter::register_instance</code> , or else the special value <code>DDS_HANDLE_NIL</code> . If <i>Foo</i> has a key and <i>instance_handle</i> is <code>DDS_HANDLE_NIL</code> , <i>instance_handle</i> is not used and <i>instance</i> is deduced from <i>instance_data</i> . If <i>Foo</i> has no key, <i>instance_handle</i> is not used. If <i>handle</i> is used, it must represent a registered instance of type <i>Foo</i> . Otherwise, this method may fail with <code>DDS_RETCODE_BAD_PARAMETER</code>
<i>source_timestamp</i>	<<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers. This timestamp will be available to the <code>DDSDataReader</code> objects by means of the <code>source_timestamp</code> attribute inside the <code>DDS_SampleInfo</code> .

Returns

One of the [Standard Return Codes](#), `DDS_RETCODE_TIMEOUT`, `DDS_RETCODE_OUT_OF_RESOURCES` or `DDS_RETCODE_NOT_ENABLED`.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataWriter::dispose](#)

write_w_timestamp()

```
DDS_ReturnCode_t FooDataWriter::write_w_timestamp (
    const Foo & instance_data,
    const DDS_InstanceHandle_t & handle,
    const DDS_Time_t & source_timestamp)
```

Performs the same function as [FooDataWriter::write](#) except that it also provides the value for the `source_timestamp`.

Explicitly provides the timestamp that will be available to the [DDSDataReader](#) objects by means of the `source_timestamp` attribute inside the [DDS_SampleInfo](#). (Refer to [DDS_SampleInfo](#) details)

The constraints on the values of the `handle` parameter and the corresponding error behavior are the same specified for the [FooDataWriter::write](#) operation.

This operation may block and time out ([DDS_RETCODE_TIMEOUT](#)) under the same circumstances described for [FooDataWriter::write](#).

If there are no instance resources left, this operation may fail with [DDS_RETCODE_OUT_OF_RESOURCES](#). Calling [FooDataWriter::unregister_instance](#) may help free up some resources.

This operation may fail with [DDS_RETCODE_BAD_PARAMETER](#) under the same circumstances described for the write operation.

Parameters

<i>instance_data</i>	<<in>> The data to write.
<i>handle</i>	<<in>> Either the handle returned by a previous call to FooDataWriter::register_instance , or else the special value DDS_HANDLE_NIL . If <i>Foo</i> has a key and <code>handle</code> is not DDS_HANDLE_NIL , <code>handle</code> must represent a registered instance of type <i>Foo</i> . Otherwise, this method may fail with DDS_RETCODE_BAD_PARAMETER .
<i>source_timestamp</i>	<<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers. This timestamp will be available to the DDSDataReader objects by means of the <code>source_timestamp</code> attribute inside the DDS_SampleInfo .

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_TIMEOUT](#), [DDS_RETCODE_OUT_OF_RESOURCES](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataWriter::write](#)

[DDSDataReader](#)

get_loan()

```
DDS_ReturnCode_t FooDataWriter::get_loan (
    Foo *& sample)
```

Gets a sample managed by the DataWriter.

This operation is supported while using Zero Copy transfer or FlatData language binding.

[FooDataWriter::get_loan](#) fails with [DDS_RETCODE_PRECONDITION_NOT_MET](#) if the above conditions are not satisfied.

The loaned sample is obtained from a DataWriter-managed sample pool and is uninitialized by default. An initialized sample can be obtained by setting [DDS_DataWriterResourceLimitsQosPolicy::initialize_writer_loaned_sample](#) to [DDS_BOOLEAN_TRUE](#). The [DDS_DataWriterResourceLimitsQosPolicy::writer_loaned_sample_allocation](#) settings can be used to configure the DataWriter-managed sample pool.

[FooDataWriter::get_loan](#) fails with [DDS_RETCODE_OUT_OF_RESOURCES](#) if [DDS_DataWriterResourceLimitsQosPolicy::writer_loaned_sample_allocation](#) samples have been loaned, and none of those samples has been written with [FooDataWriter::write](#) or discarded via [FooDataWriter::discard_loan](#).

Samples returned from [FooDataWriter::get_loan](#) have an associated state. Due to the optimized nature of the write operation while using Zero Copy transfer over shared memory or FlatData language binding, this sample state is used to control when a sample is available for reuse after the write operation. The possible sample states are free, allocated, removed or serialized. A sample that has never been allocated is "free". [FooDataWriter::get_loan](#) takes a "free" or "removed" sample and makes it "allocated". When a sample is written, its state transitions from "allocated" to "serialized", and the DataWriter takes responsibility for returning the sample back to its sample pool. The sample remains in the "serialized" state until it is removed from the DataWriter queue.

Sample removal behavior differs slightly when using the Shared Memory transport or the Zero Copy v2 transport. When using the Shared Memory transport with a reliable DataWriter, the sample is removed from the DataWriter's queue when the sample is acknowledged by all DataReaders. For a best-effort DataWriter, the sample is removed from the queue immediately after the write operation. After the sample is removed from the DataWriter queue, the sample is put back into the sample pool, and its state transitions from "serialized" to "removed". At this time, a new call to [FooDataWriter::get_loan](#) may return the same sample.

When using Zero Copy v2 transport, Best Effort and Reliable DataWriters behave similarly. The samples are kept in the serialized state until the sample HISTORY.depth is reached. At that point they are removed from the DataWriter's queue and put back into the sample pool. Since Zero Copy v2 provides automatic sample synchronization if the sample is currently being read by a DataReader with the [FooDataReader::take](#) or [FooDataReader::read](#) APIs, the sample will not be removed from the DataWriter queue until the DataReader returns the loan.

A loaned sample should not be reused to write a new value after the first write operation. Instead, a new sample from [FooDataWriter::get_loan](#) should be used to write the new value.

A loaned sample that has not been written can be returned to the DataWriter's sample pool by using [FooDataWriter::discard_loan](#).

A DataWriter writing a type annotated with `@transfer_mode(SHMEM_REF)` or `@language_binding(FLAT_DATA)` cannot write unmanaged samples that have not been returned from [FooDataWriter::get_loan](#).

Parameters

<i>sample</i>	<<inout>> reference to a user data type pointer. The loaned sample is returned via this sample.
---------------	---

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataWriter::discard_loan](#)

discard_loan()

```
DDS_ReturnCode_t FooDataWriter::discard_loan (
    Foo & sample)
```

Returns a loaned sample back to the DataWriter.

This operation is supported while using Zero Copy transfer or the FlatData language binding.

[FooDataWriter::get_loan](#) fails with [DDS_RETCODE_PRECONDITION_NOT_MET](#) if the above conditions are not satisfied.

A loaned sample that hasn't been written can be returned to the DataWriter with this operation.

Parameters

<i>sample</i>	<< <i>in</i> >> loaned sample to be discarded.
---------------	--

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

Returns

One of the [Standard Return Codes](#)

See also

[FooDataWriter::get_loan](#)

create_data()

```
Foo * FooDataWriter::create_data ()
```

Creates a data sample and initializes it.

The behavior of this API is identical to [FooTypeSupport::create_data](#).

See also

[FooDataWriter::delete_data](#)

delete_data()

```
bool FooDataWriter::delete_data (
    Foo * sample)
```

Destroys a user data type instance.

The behavior of this API is identical to [FooTypeSupport::delete_data](#).

Parameters

<i>sample</i>	<< <i>in</i> >> Cannot be NULL.
---------------	---------------------------------

Returns

DDS_BOOLEAN_TRUE on success.

See also

[FooDataWriter::create_data](#)

7.124 FooSeq Struct Reference

<<*interface*>> <<*generic*>> A type-safe, ordered collection of elements. The type of these elements is referred to in this documentation as "Foo".

```
#include <dds_cpp_sequence_defn.hxx>
```

Public Member Functions

- [FooSeq](#) & [operator=](#) (const struct [FooSeq](#) &src_seq)
Copy elements from another sequence, resizing the sequence if necessary.
- bool [copy_no_alloc](#) (const struct [FooSeq](#) &src_seq)
Copy elements from another sequence, only if the destination sequence has enough capacity.
- bool [from_array](#) (const Foo array[], [DDS_Long](#) length)
Copy elements from an array of elements, resizing the sequence if necessary. The original contents of the sequence (if any) are replaced.
- bool [to_array](#) (Foo array[], [DDS_Long](#) length)
Copy elements to an array of elements. The original contents of the array (if any) are replaced.
- Foo & [operator\[\]](#) ([DDS_Long](#) i)
Get the i-th element of the sequence.
- const Foo & [operator\[\]](#) ([DDS_Long](#) i) const
Get the i-th element for a const sequence.
- [DDS_Long](#) [length](#) () const
Get the logical length of this sequence.
- bool [length](#) ([DDS_Long](#) new_length)
Change the length of this sequence.
- bool [ensure_length](#) ([DDS_Long](#) length, [DDS_Long](#) max)
Set the sequence to the desired length, and resize the sequence if necessary.
- [DDS_Long](#) [maximum](#) () const
Get the current maximum number of elements that can be stored in this sequence.
- bool [maximum](#) ([DDS_Long](#) new_max)
Resize this sequence to a new desired maximum.
- bool [loan_contiguous](#) (Foo *buffer, [DDS_Long](#) new_length, [DDS_Long](#) new_max)
Loan a contiguous buffer to this sequence.

- bool [unloan](#) ()
Return the loaned buffer in the sequence and set the maximum to 0.
- Foo * [get_contiguous_buffer](#) () const
Access the internal buffer of a sequence.
- bool [has_ownership](#) ()
Return the value of the owned flag.
- ~[FooSeq](#) ()
Deallocate this sequence's buffer.
- [FooSeq](#) (DDS_Long new_max=0)
Create a sequence with the given maximum.
- [FooSeq](#) (const struct [FooSeq](#) &foo_seq)
Create a sequence by copying from an existing sequence.

Public Attributes

- Foo * [_contiguous_buffer](#)
Pointer to array of contiguous or discontiguous data.
- DDS_UnsignedLong [_maximum](#)
Maximum size of the sequence.
- DDS_UnsignedLong [_length](#)
Actual length of the sequence that contains data.
- DDS_Long [_element_size](#)
Size of data element in the sequence.
- void * [_token1](#)
Reserved for internal use.
- void * [_token2](#)
Reserved for internal use.
- DDS_UnsignedLong [max_alloc_size](#)
The maximum allocation for pointer elements (string,wstring) Only in debug librarires.
- DDS_Char [_flags](#)
Reserved for internal use.

7.124.1 Detailed Description

[<<interface>>](#) [<<generic>>](#) A type-safe, ordered collection of elements. The type of these elements is referred to in this documentation as "Foo".

The sequence offers a subset of the methods defined by the standard OMG IDL to C++ mapping for sequences. We refer to an IDL sequence<Foo> as [FooSeq](#).

The state of a sequence is described by the properties 'maximum', and 'length'.

- The 'maximum' represents the size of the underlying buffer; this is the maximum number of elements it can possibly hold. It is returned by the [FooSeq::maximum\(\)](#) operation.
- The 'length' represents the actual number of elements it currently holds. It is returned by the [FooSeq::length\(\)](#) operation.

7.124.2 Constructor & Destructor Documentation

~FooSeq()

```
FooSeq::~~FooSeq ()
```

Deallocate this sequence's buffer.

Precondition

(owned == [DDS_BOOLEAN_TRUE](#)). If this precondition is not met, no memory will be freed and an error will be logged.

Postcondition

maximum == 0 and the underlying buffer is freed.

See also

[FooSeq::maximum\(\)](#), [FooSeq::unloan](#)

FooSeq() [1/2]

```
FooSeq::FooSeq (
    DDS\_Long new_max = 0)
```

Create a sequence with the given maximum.

This is a constructor for the sequence. The constructor will automatically allocate memory to hold new_max elements of type Foo.

This constructor will be used when the application creates a sequence using one of the following:

```
FooSeq mySeq(5);
// or
FooSeq mySeq;
// or
FooSeq* mySeqPtr = new FooSeq(5);
```

Postcondition

maximum == new_max
length == 0
owned == [DDS_BOOLEAN_TRUE](#),

Parameters

<i>new_max</i>	Must be >= 0. Otherwise the sequence will be initialized to a new_max=0.
----------------	--

Referenced by [copy_no_alloc\(\)](#), [FooSeq\(\)](#), and [operator=\(\)](#).

FooSeq() [2/2]

```
FooSeq::FooSeq (
    const struct FooSeq & foo_seq)
```

Create a sequence by copying from an existing sequence.

This is a constructor for the sequence. The constructor will automatically allocate memory to hold `foo_seq::maximum()` elements of type `Foo` and will copy the current contents of `foo_seq` into the new sequence.

This constructor will be used when the application creates a sequence using one of the following:

```
FooSeq mySeq(foo_seq);
// or
FooSeq mySeq = foo_seq;
// or
FooSeq *mySeqPtr = new FooSeq(foo_seq);
```

Postcondition

```
this::maximum == foo_seq::maximum
this::length == foo_seq::length
this[i] == foo_seq[i] for 0 <= i < foo_seq::length
this::owned == DDS_BOOLEAN_TRUE
```

Note

If the pre-conditions are not met, the constructor will initialize the new sequence to a maximum of zero.

References [FooSeq\(\)](#).

7.124.3 Member Function Documentation**operator=()**

```
FooSeq & FooSeq::operator= (
    const struct FooSeq & src_seq)
```

Copy elements from another sequence, resizing the sequence if necessary.

This method invokes copies the content of the specified sequence, after ensuring that this sequence has enough capacity to hold the elements to be copied.

NOTE: If this method is called on a sequence that has been created with `set_maximum_w_max` or `ensure_length_w_max`, it will automatically call `copy_w_max()`.

This operator is invoked when the following expression appears in the code:

```
target_seq = src_seq
```

Important: This method *will* allocate memory if `this::maximum < src_seq::length`.

Therefore, to programatically detect the successful completion of the operator it is recommended that the application first sets the length of this sequence to zero, makes the assignment, and then checks that the length of this sequence matches that of `src_seq`.

Parameters

<code>src_seq</code>	<< <i>in</i> >> the sequence from which to copy
----------------------	---

References [FooSeq\(\)](#).

copy_no_alloc()

```
bool FooSeq::copy_no_alloc (
    const struct FooSeq & src_seq)
```

Copy elements from another sequence, only if the destination sequence has enough capacity.

Fill the elements in this sequence by copying the corresponding elements in `src_seq`. The original contents in this sequence are replaced via the element assignment operation (`Foo_copy()` function). By default, elements are discarded; 'delete' is not invoked on the discarded elements.

Precondition

```
this::maximum >= src_seq::length
this::owned == DDS_BOOLEAN_TRUE
```

Postcondition

```
this::length == src_seq::length
this[i] == src_seq[i] for 0 <= i < target_seq::length
this::owned == DDS_BOOLEAN_TRUE
```

Parameters

<code>src_seq</code>	<< <i>in</i> >> the sequence from which to copy
----------------------	---

Returns

[DDS_BOOLEAN_TRUE](#) if the sequence was successfully copied; [DDS_BOOLEAN_FALSE](#) otherwise.

Note

If the pre-conditions are not met, the operator will print a message to stdout and leave this sequence unchanged.

See also

[FooSeq::operator=](#)

References [FooSeq\(\)](#).

from_array()

```
bool FooSeq::from_array (
    const Foo array[],
    DDS_Long length)
```

Copy elements from an array of elements, resizing the sequence if necessary. The original contents of the sequence (if any) are replaced.

Fill the elements in this sequence by copying the corresponding elements in `array`. The original contents in this sequence are replaced via the element assignment operation (`Foo_copy()` function). By default, elements are discarded; 'delete' is not invoked on the discarded elements.

Precondition

`this::owned == DDS_BOOLEAN_TRUE`

Postcondition

`this::length == length`

`this[i] == array[i]` for $0 \leq i < \text{length}$

`this::owned == DDS_BOOLEAN_TRUE`

Parameters

<i>array</i>	<<in>> The array of elements to be copy elements from
<i>length</i>	<<in>> The length of the array.

Returns

`DDS_BOOLEAN_TRUE` if the array was successfully copied; `DDS_BOOLEAN_FALSE` otherwise.

References `length()`.

to_array()

```
bool FooSeq::to_array (
    Foo array[],
    DDS_Long length)
```

Copy elements to an array of elements. The original contents of the array (if any) are replaced.

Copy the elements of this sequence to the corresponding elements in the array. The original contents of the array are replaced via the element assignment operation (`Foo_copy()` function). By default, elements are discarded; 'delete' is not invoked on the discarded elements.

Parameters

<i>array</i>	<<in>> The array of elements to be filled with elements from this sequence
<i>length</i>	<<in>> The number of elements to be copied.

Returns

[DDS_BOOLEAN_TRUE](#) if the elements of the sequence were successfully copied; [DDS_BOOLEAN_FALSE](#) otherwise.

References [length\(\)](#).

operator[]() [1/2]

```
Foo & FooSeq::operator[] (
    DDS_Long i)
```

Get the *i*-th element of the sequence.

This is the operator that is invoked when the application indexes into a non-const sequence:

```
myElement = mySequence[i];
mySequence[i] = myElement;
```

Parameters

<i>i</i>	index of element to access, must be ≥ 0 and less than FooSeq::length()
----------	---

Returns

the *i*-th element

operator[]() [2/2]

```
const Foo & FooSeq::operator[] (
    DDS_Long i) const
```

Get the *i*-th element for a `const` sequence.

This is the operator that is invoked when the application indexes into a const sequence:

```
myElement = mySequence[i];
```

Parameters

<i>i</i>	index of element to access, must be ≥ 0 and less than FooSeq::length()
----------	---

Returns

the *i*-th element

length() [1/2]

```
DDS_Long FooSeq::length () const
```

Get the logical length of this sequence.

Get the length that was last set, or zero if the length has never been set.

Returns

the length of the sequence

Referenced by [ensure_length\(\)](#), [from_array\(\)](#), and [to_array\(\)](#).

length() [2/2]

```
bool FooSeq::length (  
    DDS_Long new_length)
```

Change the length of this sequence.

This method does not allocate/deallocate memory.

The new length must not exceed the maximum of this sequence as returned by the [FooSeq::maximum\(\)](#) operation. (Note that, if necessary, the maximum of this sequence can be increased manually by using the [FooSeq::maximum\(DDS_Long\)](#) operation.)

The elements of the sequence are not modified by this operation. If the new length is larger than the original length, the new elements will be uninitialized; if the length is decreased, the old elements that are beyond the new length will physically remain in the sequence but will not be accessible.

Postcondition

length = new_length.

Parameters

<i>new_length</i>	the new desired length. This value must be non-negative and cannot exceed maximum of the sequence. In other words $0 \leq \text{new_length} \leq \text{maximum}$
-------------------	---

Returns

[DDS_BOOLEAN_TRUE](#) on success or [DDS_BOOLEAN_FALSE](#) on failure

ensure_length()

```
bool FooSeq::ensure_length (
    DDS_Long length,
    DDS_Long max)
```

Set the sequence to the desired length, and resize the sequence if necessary.

If the current maximum is greater than the desired length, then sequence is not resized.

Otherwise if this sequence owns its buffer, the sequence is resized to the new maximum by freeing and re-allocating the buffer. However, if the sequence does not own its buffer, this operation will fail.

This function allows user to avoid unnecessary buffer re-allocation.

Precondition

length <= max
owned == [DDS_BOOLEAN_TRUE](#) if sequence needs to be resized

Postcondition

length == length
maximum == max if resized

Parameters

<i>length</i>	<<in>> The new length that should be set. Must be >= 0.
<i>max</i>	<<in>> If sequence need to be resized, this is the maximum that should be set. max >= length

Returns

[DDS_BOOLEAN_TRUE](#) on success, [DDS_BOOLEAN_FALSE](#) if the preconditions are not met. In that case the sequence is not modified.

References [length\(\)](#).

maximum() [1/2]

```
DDS_Long FooSeq::maximum () const
```

Get the current maximum number of elements that can be stored in this sequence.

The `maximum` of the sequence represents the maximum number of elements that the underlying buffer can hold. It does not represent the current number of elements.

The `maximum` is a non-negative number. It is initialized when the sequence is first created and can only be set by mean of the [FooSeq::maximum\(DDS_Long\)](#) operation.

Returns

the current maximum of the sequence.

See also

[FooSeq::length\(\)](#)

maximum() [2/2]

```
bool FooSeq::maximum (
    DDS_Long new_max)
```

Resize this sequence to a new desired maximum.

This operation does nothing if the new desired maximum matches the current `maximum`.

If this sequence owns its buffer and the new maximum is not equal to the old maximum, then the existing buffer will be freed and re-allocated.

Precondition

`owned == DDS_BOOLEAN_TRUE`

Postcondition

`owned == DDS_BOOLEAN_TRUE`
`length == MINIMUM(original length, new_max)`

Parameters

<code>new_max</code>	Must be ≥ 0 .
----------------------	--------------------

Returns

`DDS_BOOLEAN_TRUE` on success, `DDS_BOOLEAN_FALSE` if the preconditions are not met. In that case the sequence is not modified.

loan_contiguous()

```
bool FooSeq::loan_contiguous (
    Foo * buffer,
    DDS_Long new_length,
    DDS_Long new_max)
```

Loan a contiguous buffer to this sequence.

This operation changes the `owned` flag of the sequence to `DDS_BOOLEAN_FALSE` and also sets the underlying buffer used by the sequence. See the user's manual for more information about sequences and memory ownership.

Use this method if you want to manage the memory used by the sequence yourself. You must provide an array of elements and integers indicating how many elements are allocated in that array (i.e. the maximum) and how many elements are valid (i.e. the length). The sequence will subsequently use the memory you provide and will not permit it to be freed by a call to `FooSeq::maximum(DDS_Long)`.

Once you have loaned a buffer to a sequence, make sure that you don't free it before calling `FooSeq::unloan`: the next time you access the sequence, you will be accessing freed memory!

You can use this method to wrap stack memory with a sequence interface, thereby avoiding dynamic memory allocation. Create a `FooSeq` and an array of type `Foo` and then loan the array to the sequence:

```
Foo fooArray[10];
::FooSeq fooSeq;
fooSeq.loan_contiguous(fooArray, 0, 10);
```

By default, a sequence you create owns its memory unless you explicitly loan memory of your own to it. In a very few cases, RTI Connex Micro DDS will return a sequence to you that has a loan; those cases are documented as such. For example, if you call `FooDataReader::read` or `FooDataReader::take` and pass in sequences with no loan and no memory allocated, RTI Connex Micro DDS will loan memory to your sequences which must be unloaned with `FooDataReader::return_loan`. See the documentation of those methods for more information.

Precondition

`FooSeq::maximum() == 0`; i.e. the sequence has no memory allocated to it.

`FooSeq::has_ownership == DDS_BOOLEAN_TRUE`; i.e. the sequence does not already have an outstanding loan

Postcondition

The sequence will store its elements in the buffer provided.

`FooSeq::has_ownership == DDS_BOOLEAN_FALSE`

`FooSeq::length() == new_length`

`FooSeq::maximum() == new_max`

Parameters

<i>buffer</i>	The new buffer that the sequence will use. Must point to enough memory to hold <code>new_max</code> elements of type <code>Foo</code> . It may be <code>NULL</code> if <code>new_max == 0</code> .
<i>new_length</i>	The desired new length for the sequence. It must be the case that that <code>0 <= new_length <= new_max</code> .
<i>new_max</i>	The allocated number of elements that could fit in the loaned buffer.

Returns

`DDS_BOOLEAN_TRUE` if `buffer` is successfully loaned to this sequence or `DDS_BOOLEAN_FALSE` otherwise. Failure only occurs due to failing to meet the pre-conditions. Upon failure the sequence remains unmodified.

See also

[FooSeq::unloan](#)

unloan()

```
bool FooSeq::unloan ()
```

Return the loaned buffer in the sequence and set the maximum to 0.

This method affects only the state of this sequence; it does not change the contents of the buffer in any way.

Only the user who originally loaned a buffer should return that loan, as the user may have dependencies on that memory known only to them. Unloaning someone else's buffer may cause unspecified problems. For example, suppose a sequence is loaning memory from a custom memory pool. A user of the sequence likely has no way to release the memory back into the pool, so unloaning the sequence buffer would result in a resource leak. If the user were to then re-loan a different buffer, the original creator of the sequence would have no way to discover, when freeing the sequence, that the loan no longer referred to its own memory and would thus not free the user's memory properly, exacerbating the situation and leading to undefined behavior.

Precondition

owned == [DDS_BOOLEAN_FALSE](#)

Postcondition

owned == [DDS_BOOLEAN_TRUE](#)
maximum == 0

Returns

[DDS_BOOLEAN_TRUE](#) if the preconditions were met. Otherwise [DDS_BOOLEAN_FALSE](#). The function only fails if the pre-conditions are not met, in which case it leaves the sequence unmodified.

See also

[FooSeq::loan_contiguous](#) [FooSeq::maximum\(DDS_Long\)](#)

get_contiguous_buffer()

```
Foo * FooSeq::get_contiguous_buffer () const
```

Access the internal buffer of a sequence.

Use this function to access the memory buffer that a sequence uses to store the values of its elements.

has_ownership()

```
bool FooSeq::has_ownership ()
```

Return the value of the owned flag.

Returns

[DDS_BOOLEAN_TRUE](#) if sequence owns the underlying buffer, or [DDS_BOOLEAN_FALSE](#) if it has an outstanding loan.

7.124.4 Member Data Documentation**_contiguous_buffer**

```
Foo* FooSeq::_contiguous_buffer
```

Pointer to array of contiguous or discontinuous data.

_maximum

```
DDS_UnsignedLong FooSeq::_maximum
```

Maximum size of the sequence.

The allocated length of this sequence. It applies to whichever of the above buffers is non-NULL, if any. If both a NULL, its value must be 0.

If `_maximum == 0`, `_owned == true`.

_length

```
DDS_UnsignedLong FooSeq::_length
```

Actual length of the sequence that contains data.

The current logical length of this sequence, i.e. the number of valid elements it contains. It applies to whichever of the above buffers is non-null, if any. If both are NULL, its value must be 0.

_element_size

```
DDS_Long FooSeq::_element_size
```

Size of data element in the sequence.

Each element in the sequence has this size.

_token1

```
void* FooSeq::_token1
```

Reserved for internal use.

_token2

```
void* FooSeq::_token2
```

Reserved for internal use.

max_alloc_size

```
DDS_UnsignedLong FooSeq::max_alloc_size
```

The maximum allocation for pointer elements (string,wstring) Only in debug libraraires.

_flags

DDS_Char FooSeq::_flags

Reserved for internal use.

7.125 FooTypeSupport Class Reference

<<interface>> <<generic>> User data type specific interface.

```
#include <dds_cpp_data.hxx>
```

Static Public Member Functions

- static bool `initialize_data` (Foo *sample)
<<eXtension>> Initialize data type.
- static bool `finalize_data` (Foo *sample)
<<eXtension>> Finalize data type.
- static Foo * `create_data` ()
<<eXtension>> Create a data type and initialize it.
- static DDS_ReturnCode_t `delete_data` (Foo *sample)
<<eXtension>> Destroy a user data type instance.
- static bool `copy_data` (Foo *dst_data, Foo *src_data)
<<eXtension>> Copy data type.
- static DDS_ReturnCode_t `register_type` (DDSDomainParticipant *participant, const char *type_name)
Allows an application to communicate to RTI Connex DDS Micro the existence of a data type.
- static DDS_ReturnCode_t `unregister_type` (DDSDomainParticipant *participant, const char *type_name)
Allows an application to unregister a data type from RTI Connex DDS Micro. After calling `unregister_type`, no further communication using that type is possible.
- static DDS_ReturnCode_t `serialize_data_to_cdr_buffer_ex` (char *buffer, unsigned int &length, const TData *a_data, DDS_DataRepresentationId_t representation)
<<eXtension>> Serializes the input sample into a buffer of octets.
- static DDS_ReturnCode_t `serialize_data_to_cdr_buffer` (char *buffer, unsigned int &length, const TData *a_data)
<<eXtension>> Serializes the input sample into a CDR buffer of octets.
- static DDS_ReturnCode_t `deserialize_data_from_cdr_buffer` (TData *a_data, const char *buffer, unsigned int length)
<<eXtension>> Deserializes a sample from a buffer of octets.

7.125.1 Detailed Description

<<interface>> <<generic>> User data type specific interface.

7.125.2 Member Function Documentation

initialize_data()

```
static bool FooTypeSupport::initialize_data (
    Foo * sample) [static]
```

<<eXtension>> Initialize data type.

The *generated* implementation of the operation knows how to initialize a data type. This method is typically called to initialize a data type that is allocated on the stack.

Parameters

<i>sample</i>	<< <i>inout</i> >> Cannot be NULL.
---------------	------------------------------------

Returns

One of the [Standard Return Codes](#)

See also

[FooTypeSupport::finalize_data](#)

finalize_data()

```
static bool FooTypeSupport::finalize_data (
    Foo * sample) [static]
```

<<*eXtension*>> Finalize data type.

The *generated* implementation of the operation knows how to finalize a data type. This method is typically called to finalize a data type that has previously been initialized.

Parameters

<i>sample</i>	<< <i>in</i> >> Cannot be NULL.
---------------	---------------------------------

Returns

One of the [Standard Return Codes](#)

See also

[FooTypeSupport::initialize_data](#)

create_data()

```
static Foo * FooTypeSupport::create_data () [static]
```

<<*eXtension*>> Create a data type and initialize it.

Memory for a data type is allocated from memory using internal RTI Connex DDS Micro memory allocation and deallocation APIs. The implementation of these APIs are specific to a particular platform. For this reason it is of the utmost importance that [FooTypeSupport::create_data](#) is not called until [DDSDomainParticipantFactory::get_instance](#) has been called successfully. The [DDSDomainParticipantFactory](#) initializes the RTI Connex DDS Micro system, including any memory management initialization.

The *generated* implementation of the operation knows how to instantiate a data type and initialize it properly.

All memory for the type is deeply allocated.

Returns

newly created data type

MT Safety:

This operation is thread safe.

API Restriction:

This function *must* only be called after [DDSDomainParticipantFactory::get_instance](#).

See also

[FooTypeSupport::delete_data](#)

delete_data()

```
static DDS_ReturnCode_t FooTypeSupport::delete_data (  
    Foo * sample) [static]
```

<<*eXtension*>> Destroy a user data type instance.

The *generated* implementation of the operation knows how to destroy a data type and return all resources.

Parameters

<i>sample</i>	<< <i>in</i> >> Cannot be NULL.
---------------	---------------------------------

Returns

One of the [Standard Return Codes](#)

See also

[FooTypeSupport::create_data](#)

copy_data()

```
static bool FooTypeSupport::copy_data (  
    Foo * dst_data,  
    Foo * src_data) [static]
```

<<*eXtension*>> Copy data type.

The *generated* implementation of the operation knows how to copy value of a data type.

Parameters

<i>dst_data</i>	<< <i>inout</i> >> Data type to copy value to. Cannot be NULL.
<i>src_data</i>	<< <i>in</i> >> Data type to copy value from. Cannot be NULL.

Returns

One of the [Standard Return Codes](#)

register_type()

```
static DDS_ReturnCode_t FooTypeSupport::register_type (
    DDSDomainParticipant * participant,
    const char * type_name) [static]
```

Allows an application to communicate to RTI Connex DDS Micro the existence of a data type.

The *generated* implementation of the operation embeds all the knowledge that has to be communicated to the middle-ware in order to make it able to manage the contents of data of that type. This includes in particular the key definition that will allow RTI Connex DDS Micro to distinguish different instances of the same type.

The same [DDS_TYPESUPPORT_CPP](#) can be registered multiple times with a [DDSDomainParticipant](#) using the same or different values for the *type_name*. If *register_type* is called multiple times on the same [DDS_TYPESUPPORT_CPP](#) with the same [DDSDomainParticipant](#) and *type_name*, the second (and subsequent) registrations return [DDS_RETCODE_OK](#). A type must be registered and unregistered an equal number of times.

Precondition

Cannot use the same *type_name* to register two different [DDS_TYPESUPPORT_CPP](#) with the same [DDSDomainParticipant](#), or else the operation will fail and [DDS_RETCODE_PRECONDITION_NOT_MET](#) will be returned.

Parameters

<i>participant</i>	<< <i>in</i> >> the DDSDomainParticipant to register the data type <code>Foo</code> with. Cannot be NULL.
<i>type_name</i>	<< <i>in</i> >> the type name under with the data type <code>Foo</code> is registered with the participant; this type name is used when creating a new DDSTopic . (See DDSDomainParticipant::create_topic .) The name may not be longer than 255 characters. If name is NULL, the typename specified in IDL is used.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#) or [DDS_RETCODE_OUT_OF_RESOURCES](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDSDomainParticipant::create_topic](#)

unregister_type()

```
static DDS_ReturnCode_t FooTypeSupport::unregister_type (
    DDSDomainParticipant * participant,
    const char * type_name) [static]
```

Allows an application to unregister a data type from RTI Connex DDS Micro. After calling `unregister_type`, no further communication using that type is possible.

The *generated* implementation of the operation removes all the information about a type from RTI Connex DDS Micro. No further communication using that type is possible.

Precondition

A type with `type_name` is registered with the participant and all [DDSTopic](#) objects referencing the type have been destroyed. If the type is not registered with the participant, or if any [DDSTopic](#) is associated with the type, the operation will fail with [DDS_RETCODE_ERROR](#).

Postcondition

All information about the type is removed from RTI Connex DDS Micro. No further communication using this type is possible.

Parameters

<i>participant</i>	<< <i>in</i> >> the DDSDomainParticipant to unregister the data type <code>Foo</code> from. Cannot be NULL.
<i>type_name</i>	<< <i>in</i> >> the type name under with the data type <code>Foo</code> is registered with the participant. The name should match a name that has been previously used to register a type with the participant. Cannot be NULL.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_BAD_PARAMETER](#) or [DDS_RETCODE_ERROR](#)

MT Safety:

SAFE.

See also

[DDSDomainParticipant::register_type](#)

serialize_data_to_cdr_buffer_ex()

```
static DDS_ReturnCode_t FooTypeSupport::serialize_data_to_cdr_buffer_ex (
    char * buffer,
    unsigned int & length,
    const TData * a_data,
    DDS_DataRepresentationId_t representation) [static]
```

<<*eXtension*>> Serializes the input sample into a buffer of octets.

This method serializes a sample into a buffer of octets using the input data representation. See [FooTypeSupport::serialize_data_to_cdr_buffer_ex](#) for details.

Parameters

<i>a_data</i>	<< <i>in</i> >>. Input sample. Cannot be NULL.
<i>buffer</i>	<< <i>out</i> >>. Serialization buffer.
<i>length</i>	<< <i>inout</i> >>. Serialization buffer length.
<i>representation</i>	<< <i>in</i> >>. Representation used to serialize the data.

Returns

One of the [Standard Return Codes](#)

serialize_data_to_cdr_buffer()

```
static DDS_ReturnCode_t FooTypeSupport::serialize_data_to_cdr_buffer (
    char * buffer,
    unsigned int & length,
    const TData * a_data) [static]
```

<<*eXtension*>> Serializes the input sample into a CDR buffer of octets.

This method serializes a sample into a buffer of octets and it uses CDR as the data representation. Calling this method is equivalent to calling `FooTypeSupport::serialize_data_to_cdr_buffer_ex` with `DDS_AUTO_DATA_REPRESENTATION` as the representation.

The input buffer must be big enough to store the serialized representation of the sample. Otherwise, the method will return an error.

To determine the minimum size of the input buffer, the user must call this method with the buffer set to NULL.

Parameters

<i>a_data</i>	<< <i>in</i> >>. Input sample. Cannot be NULL.
<i>buffer</i>	<< <i>out</i> >>. Serialization buffer.
<i>length</i>	<< <i>inout</i> >>. When <i>buffer</i> is set to NULL, after the method executes, <i>length</i> will contain a buffer size big enough to hold the serialized data. When <i>buffer</i> is not NULL, <i>length</i> must contain the size of the input buffer when the method is invoked. After the method executes, <i>length</i> will be updated to contain the actual size of the serialized content, which may be smaller than the size obtained when <i>buffer</i> is set to NULL.

Returns

One of the [Standard Return Codes](#)

deserialize_data_from_cdr_buffer()

```
static DDS_ReturnCode_t FooTypeSupport::deserialize_data_from_cdr_buffer (
    TData * a_data,
    const char * buffer,
    unsigned int length) [static]
```

<<*eXtension*>> Deserializes a sample from a buffer of octets.

This method deserializes a sample from a CDR buffer of octets.

The content of the buffer generated by the method `FooTypeSupport::serialize_data_to_cdr_buffer` can be provided to this method to get the sample back.

Parameters

<i>a_data</i>	<< <i>in</i> >>. Output sample. Cannot be NULL.
<i>buffer</i>	<< <i>in</i> >>. Deserialization buffer. Cannot be NULL.
<i>length</i>	<< <i>in</i> >>. Length of the serialized representation of the sample in the buffer.

Returns

One of the [Standard Return Codes](#)

7.126 NDDS_Discovery_Property Struct Reference

Specifies internal information used to store discovery information about a [DDSDomainParticipant](#) in the registry.

```
#include <dds_c_domain.h>
```

7.126.1 Detailed Description

Specifies internal information used to store discovery information about a [DDSDomainParticipant](#) in the registry.

7.127 NDDS_Type_PluginVersion Struct Reference

Data type used to keep track of versioning information about a `::DDS_TypePluginI`.

```
#include <dds_c_type.h>
```

Public Attributes

- char [majorRev](#)
Major version number.
- char [minorRev](#)
Minor version number.

7.127.1 Detailed Description

Data type used to keep track of versioning information about a `::DDS_TypePluginI`.

7.127.2 Member Data Documentation

majorRev

```
char NDDS_Type_PluginVersion::majorRev
```

Major version number.

minorRev

```
char NDDS_Type_PluginVersion::minorRev
```

Minor version number.

7.128 NETIO_Address Struct Reference

```
#include <netio_address.h>
```

Public Attributes

- [RTI_INT32](#) kind
The address kind.
- [RTI_UINT32](#) port
The address port.
- union [NETIO_AddressValue](#) value
The transport address.

7.128.1 Detailed Description

<<[eXtension](#)>> The NETIO Address.

7.128.2 Member Data Documentation

kind

```
RTI\_INT32 NETIO_Address::kind
```

The address kind.

port

```
RTI\_UINT32 NETIO_Address::port
```

The address port.

value

```
union NETIO\_AddressValue NETIO_Address::value
```

The transport address.

7.129 NETIO_AddressSeq Struct Reference

<<*eXtension*>> The [NETIO_AddressSeq](#) sequence type.

```
#include <netio_address.h>
```

7.129.1 Detailed Description

<<*eXtension*>> The [NETIO_AddressSeq](#) sequence type.

7.130 NETIO_AddressUInt32 Struct Reference

<<*eXtension*>> [NETIO_AddressUInt32](#)

```
#include <netio_address.h>
```

Public Attributes

- [RTI_UINT32](#) value [4]
Array of unsigned 32-bit integers.

7.130.1 Detailed Description

<<*eXtension*>> [NETIO_AddressUInt32](#)

7.130.2 Member Data Documentation

value

```
RTI\_UINT32 NETIO_AddressUInt32::value[4]
```

Array of unsigned 32-bit integers.

7.131 NETIO_AddressValue Union Reference

<<*eXtension*>> [NETIO_AddressValue](#)

```
#include <netio_address.h>
```

Public Attributes

- struct NETIO_AddressIpv6 [address](#)
Address as array of octets address.octet[0-15].
- struct NETIO_AddressUInt32 [as_uint32](#)
Address as array of unsigned 32-bit integers.

7.131.1 Detailed Description

<<*eXtension*>> [NETIO_AddressValue](#)

7.131.2 Member Data Documentation**address**

```
struct NETIO_AddressIpv6 NETIO_AddressValue::address
```

Address as array of octets address.octet[0-15].

as_uint32

```
struct NETIO_AddressUInt32 NETIO_AddressValue::as_uint32
```

Address as array of unsigned 32-bit integers.

7.132 NETIO_DGRAM_Interface Struct Reference

<<*eXtension*>> Definition of the NETIO_DGRAM Interface.

```
#include <netio_dgram.h>
```

Public Attributes

- [NETIO_DGRAM_Interface_createFunc](#) [create_instance](#)
Creates an instance of the user defined datagram Interface.
- [NETIO_DGRAMUserInterface_deleteFunc](#) [delete_instance](#)
Deletes an instance of the user defined datagram Interface. Not in CERT.
- [NETIO_DGRAMUserInterface_get_interface_listFunc](#) [get_interface_list](#)
Read available interfaces from the user DGRAM Interface.
- [NETIO_Interface_release_addressFunc](#) [release_address](#)
Release addresses to the DGRAMInterfaceUser.
- [NETIO_DGRAMUserInterface_resolve_addressFunc](#) [resolve_address](#)
Resolve an address string from DGRAM user interface.
- [NETIO_Interface_sendFunc](#) [send](#)
Send a message with the DGRAM user interface.
- [NETIO_Interface_get_route_tableFunc](#) [get_route_table](#)
Read the route table from the DGRAM user interface.
- [NETIO_DGRAMUserInterface_bind_addressFunc](#) [bind_address](#)
Bind to an address.

7.132.1 Detailed Description

<<*eXtension*>> Definition of the NETIO_DGRAM Interface.

7.132.2 Member Data Documentation

create_instance

NETIO_DGRAM_Interface_createFunc NETIO_DGRAM_InterfaceI::create_instance

Creates an instance of the user defined datagram Interface.

A user defined datagram function that is called by RTI Connex DDS Micro to create an instance of the NETIO_DGRAM transport.

Parameters

in	<i>upstream</i>	The upstream interface.
in	<i>property</i>	The user defined interface property.

Returns

The function must return a pointer to a new instance on success and NULL on failure.

delete_instance

NETIO_DGRAMUserInterface_deleteFunc NETIO_DGRAM_InterfaceI::delete_instance

Deletes an instance of the user defined datagram Interface. Not in CERT.

A user defined datagram function that is called by RTI Connex DDS Micro to create an instance of the NETIO_DGRAM transport.

Parameters

in	<i>upstream</i>	The upstream interface.
----	-----------------	-------------------------

Returns

RTI_TRUE on success, RTI_FALSE on failure.

get_interface_list

NETIO_DGRAMUserInterface_get_interface_listFunc NETIO_DGRAM_InterfaceI::get_interface_list

Read available interfaces from the user DGRAM Interface.

A user defined datagram function that is called by RTI Connex DDS Micro to read the available interfaces from a transport.

Parameters

in	<i>netio_intf</i>	The DGRAM interface to use.
in	<i>if_table</i>	The interface table.

Returns

TRUE on success, FALSE on failure.

release_address

NETIO_Interface_release_addressFunc NETIO_DGRAM_InterfaceI::release_address

Release addresses to the DGRAMInterfaceUser.

A user defined datagram function that is called by RTI Connex DDS Micro when an address is no longer in use.

Parameters

in	<i>self</i>	The DGRAM interface to use.
in	<i>src_addr</i>	The address to release.

Returns

TRUE on success, FALSE on failure.

resolve_address

NETIO_DGRAMUserInterface_resolve_addressFunc NETIO_DGRAM_InterfaceI::resolve_address

Resolve an address string from DGRAM user interface.

A user defined datagram function that is called by RTI Connex DDS Micro to convert an address in string format to a numerical value. The *if_entry* *multicast_group* decides if the address is a multicast group. The *address_value* is automatically marked as a multicast address if the *resolve_address* function returns RTI_TRUE.

If *resolve_address* is NULL when calling [NETIO_DGRAM_InterfaceFactory_register](#), RTI Connex DDS Micro will resolve addresses in the dotted IPv4 notation "a.b.c.d" where a, b, c, and d are decimal integers between 0-255.

Parameters

in	<i>netio_intf</i>	Interface.
in	<i>if_entry</i>	The interface entry requested to resolve the address.
in	<i>address_string</i>	Address to convert.
out	<i>address_value</i>	Converted address.
out	<i>is_invalid</i>	If the address is valid or not.

Returns

RTI_TRUE on success, RTI_FALSE on failure.

send

```
NETIO_Interface_sendFunc NETIO_DGRAM_InterfaceI::send
```

Send a message with the DGRAM user interface.

A user defined datagram function that is called by RTI Connex DDS Micro to forward a packet from a source interface to a destination with the user defined DGRAM interface.

Parameters

in	<i>self</i>	DGRAM interface to send from.
in	<i>source</i>	The source interface for the packet.
in	<i>destination</i>	The destination address for the packet.
in	<i>packet</i>	The packet to send.

Returns

RTI_TRUE on success, RTI_FALSE on failure.

get_route_table

```
NETIO_Interface_get_route_tableFunc NETIO_DGRAM_InterfaceI::get_route_table
```

Read the route table from the DGRAM user interface.

A user defined datagram function that will be used by RTI Connex DDS Micro to get the route table from the DGRAM interface. The route table is a sequence of address and netmask pairs that informs RTI Connex DDS Micro which locators can be reached from the transport.

Parameters

in	<i>netio_intf</i>	The NETIO interface.
in, out	<i>address</i>	Sequence of NETIO addresses this interface understands.
in, out	<i>netmask</i>	Sequence of the corresponding netmasks.

Returns

RTI_TRUE on success, RTI_FALSE on failure.

bind_address

```
NETIO_DGRAMUserInterface_bind_addressFunc NETIO_DGRAM_InterfaceI::bind_address
```

Bind to an address.

A user defined datagram function called by RTI Connex DDS Micro to starting listening on the specified address.

Parameters

in	<i>user_intf</i>	DGRAM interface to to use.
in	<i>source</i>	The address to bind.

Returns

RTI_TRUE if successful, RTI_FALSE if not.

7.133 NETIO_DGRAM_InterfaceMultiCastGroup Struct Reference

<<*eXtension*>> Definition of a valid multicast group address range.

```
#include <netio_dgram.h>
```

Public Attributes

- struct [NETIO_Address](#) *address_low*
The lowest multicast address.
- struct [NETIO_Address](#) *address_high*
The highest multicast address.
- struct [NETIO_Netmask](#) *netmask*
The netmask to apply to the group.

7.133.1 Detailed Description

<<*eXtension*>> Definition of a valid multicast group address range.

7.133.2 Member Data Documentation

address_low

```
struct NETIO\_Address NETIO_DGRAM_InterfaceMultiCastGroup::address_low
```

The lowest multicast address.

address_high

```
struct NETIO\_Address NETIO_DGRAM_InterfaceMultiCastGroup::address_high
```

The highest multicast address.

netmask

```
struct NETIO_Netmask NETIO_DGRAM_InterfaceMultiCastGroup::netmask
```

The netmask to apply to the group.

The netmask is a sequence of N consecutive 1s, starting with the left most bit. In order to determine if an address is a multicast address the netmask is first applied to the address and the high and low address of the multicast group. If the masked address is within the multicast group it is treated as a multicast address.

7.134 NETIO_DGRAM_InterfaceRouteEntry Struct Reference

<<*eXtension*>> Definition of a route entry.

```
#include <netio_dgram.h>
```

Public Attributes

- struct [NETIO_Address address](#)
The address of the entry.
- struct [NETIO_Netmask netmask](#)
The netmask of the entry used for comparing addresses.

7.134.1 Detailed Description

<<*eXtension*>> Definition of a route entry.

7.134.2 Member Data Documentation

address

```
struct NETIO_Address NETIO_DGRAM_InterfaceRouteEntry::address
```

The address of the entry.

The [NETIO_DGRAM_InterfaceRouteEntry](#) structure provides a convenient method to add routes to a NETIO_DGRAM transport. The address attribute is the addresses that can be reached after the netmask attribute has been applied.

netmask

```
struct NETIO_Netmask NETIO_DGRAM_InterfaceRouteEntry::netmask
```

The netmask of the entry used for comparing addresses.

7.135 NETIO_DGRAM_InterfaceTableEntry Struct Reference

<<*eXtension*>> Definition of an interface.

```
#include <netio_dgram.h>
```

Public Attributes

- [RTI_INT32 locator_kind](#)
Locator type supported by the interface, such as UDPv4.
- [RTI_UINT32 flags](#)
The flags attribute is a bitmap used to indicate capabilities of the transport.
- [RTI_UINT32 mtu](#)
The maximum message size that can be sent and received by the transport.
- struct [NETIO_Address address](#)
The address of the interface.
- struct [NETIO_DGRAM_InterfaceMultiCastGroup multicast_group](#)
An optional multicast group for the interface.
- const char * [ifname](#)
The interface name is not used by RTI Connex DDS Micro.

7.135.1 Detailed Description

<<*eXtension*>> Definition of an interface.

7.135.2 Member Data Documentation

locator_kind

```
RTI_INT32 NETIO_DGRAM_InterfaceTableEntry::locator_kind
```

Locator type supported by the interface, such as UDPv4.

The locator_kind describes the transport protocol. For example, a locator_kind equal to 1 is reserved for DDS UDPv4 transport. Note the locator_kind is used discovery and if a locator_kind is discovered, but unsupported, no communication will take place using locators of that kind.

Note that the locator_kind must be less than [NETIO_ADDRESS RTPS_RESERVED_LOW](#) and greater than [NETIO_ADDRESS RTPS_RESERVED_HIGH](#).

flags

```
RTI_UINT32 NETIO_DGRAM_InterfaceTableEntry::flags
```

The flags attribute is a bitmap used to indicate capabilities of the transport.

mtu

```
RTI_UINT32 NETIO_DGRAM_InterfaceTableEntry::mtu
```

The maximum message size that can be sent and received by the transport.

The mtu should be set to the maximum number of octets that can be sent and received in a message over the transport, excluding overhead required by the transport. RTI Connex DDS Micro uses the smallest mtu across all available transports to determine the maximum number of octets that can be sent in a single message.

address

```
struct NETIO_Address NETIO_DGRAM_InterfaceTableEntry::address
```

The address of the interface.

This is the address that is announced to other DDS DomainParticipants that this transport can receive messages on.

Note: This address must not be within the multicast group.

multicast_group

```
struct NETIO_DGRAM_InterfaceMultiCastGroup NETIO_DGRAM_InterfaceTableEntry::multicast_group
```

An optional multicast group for the interface.

This attribute can be used to specify a group of addresses that should be considered multicast addresses. Multicast addresses are address that is sent to once, but received by many and can be a more efficient mechanism to distribute data. RTI Connex DDS Micro will prioritize sending to multicast addresses over unicast addresses.

Note the following restrictions:

- All the multicast_group.netmask.mask array elements must be 0
- The multicast_group.netmask.bits must be in the range [0,128]

ifname

```
const char* NETIO_DGRAM_InterfaceTableEntry::ifname
```

The interface name is not used by RTI Connex DDS Micro.

7.136 NETIO_DGRAM_InterfaceTableEntrySeq Struct Reference

A sequence of NETIO_DGRAMInterfaceTableEntry elements.

```
#include <netio_dgram.h>
```

7.136.1 Detailed Description

A sequence of NETIO_DGRAMInterfaceTableEntry elements.

7.137 NETIO_Interface Struct Reference

Base-class definition for all NETIO interfaces.

```
#include <netio_interface.h>
```

7.137.1 Detailed Description

Base-class definition for all NETIO interfaces.

<<eXtension>> NETIO_Interface base-class.

All NETIO layers are RT components and are derived from RT components. All NETIO layers also share a common base class, NETIO_Interface and share common properties.

7.138 NETIO_InterfaceI Struct Reference

```
#include <netio_interface.h>
```

7.138.1 Detailed Description

<<eXtension>> NETIO_InterfaceI. This type is internal to RTI

7.139 NETIO_Netmask Struct Reference

<<eXtension>> NETIO_Netmask.

```
#include <netio_address.h>
```

Public Attributes

- RTI_UINT32 bits
Number of bits in Netmask.
- RTI_UINT32 mask [NETIO_NETMASK_MASK_LENGTH]
The Netmask as a sequence of consecutive 1s from left to right.

7.139.1 Detailed Description

<<*eXtension*>> [NETIO_Netmask](#).

7.139.2 Member Data Documentation

bits

[RTI_UINT32](#) [NETIO_Netmask::bits](#)

Number of bits in Netmask.

mask

[RTI_UINT32](#) [NETIO_Netmask::mask](#)[[NETIO_NETMASK_MASK_LENGTH](#)]

The Netmask as a sequence of consecutive 1s from left to right.

7.140 NETIO_NetmaskSeq Struct Reference

<<*eXtension*>> The [NETIO_NetmaskSeq](#) sequence type.

```
#include <netio_address.h>
```

7.140.1 Detailed Description

<<*eXtension*>> The [NETIO_NetmaskSeq](#) sequence type.

7.141 NETIO_Packet Struct Reference

Implementation of the abstract [NETIO_Packet](#) type.

```
#include <netio_interface.h>
```

7.141.1 Detailed Description

Implementation of the abstract [NETIO_Packet](#) type.

<<*eXtension*>> The NETIO Packet API.

The NETIO layers passed [NETIO_Packet](#) structures upstream/downstream. Any meta-data about the packet must be encapsulated either in the packet payload or the packet info field.

7.142 NETIO_SharedMemorySegmentHeader Struct Reference

```
#include <netio_shmem_segment.h>
```

7.142.1 Detailed Description

Opaque handle initialized by the attach methods and used to interact with a shared memory segment.

7.143 NETIO_SHMEMInterfaceFactoryProperty Struct Reference

<<eXtension>> Properties for the SHMEM Transport.

```
#include <netio_shmem.h>
```

Public Attributes

- [RTI_INT32 received_message_count_max](#)
Max number of received message sizes that can be residing inside the shared memory transport concurrent queue.
- [RTI_INT32 receive_buffer_size](#)
The size of the receive socket buffer.
- [RTI_INT32 message_size_max](#)
The maximum size of the message which can be received.
- struct [OSAPI_ThreadProperty rcv_thread_property](#)
Thread properties for each receive thread created by this NETIO interface.
- struct [DDS_ProductVersion](#) [pro_minimum_compatibility_version](#)
The minimum Connex Pro version that the shared memory transport should be compatible with.

7.143.1 Detailed Description

<<eXtension>> Properties for the SHMEM Transport.

7.143.2 Member Data Documentation

received_message_count_max

[RTI_INT32](#) NETIO_SHMEMInterfaceFactoryProperty::received_message_count_max

Max number of received message sizes that can be residing inside the shared memory transport concurrent queue.

receive_buffer_size

```
RTI_INT32 NETIO_SHMEMInterfaceFactoryProperty::receive_buffer_size
```

The size of the receive socket buffer.

message_size_max

```
RTI_INT32 NETIO_SHMEMInterfaceFactoryProperty::message_size_max
```

The maximum size of the message which can be received.

recv_thread_property

```
struct OSAPI_ThreadProperty NETIO_SHMEMInterfaceFactoryProperty::recv_thread_property
```

Thread properties for each receive thread created by this NETIO interface.

pro_minimum_compatibility_version

```
struct DDS_ProductVersion NETIO_SHMEMInterfaceFactoryProperty::pro_minimum_compatibility_version
```

The minimum Connex Pro version that the shared memory transport should be compatible with.

7.144 OSAPI_LogProperty Struct Reference

Configuration of logging functionality.

```
#include <osapi_log.h>
```

Public Attributes

- [RTI_SIZE_T max_buffer_size](#)
The maximum number of bytes allocated to the log buffer.
- [RTI_UINT32 log_detail](#)
Bitmap to control the fidelity of what is stored in the log entry.
- [OSAPI_Log_write_buffer_T write_buffer](#)
Function pointer to output a log buffer.

7.144.1 Detailed Description

Configuration of logging functionality.

7.144.2 Member Data Documentation

max_buffer_size

`RTI_SIZE_T OSAPI_LogProperty::max_buffer_size`

The maximum number of bytes allocated to the log buffer.

Log entries are of variable length. When this limit has been reached, no more log entries can be stored unless the buffer is cleared.

log_detail

`RTI_UINT32 OSAPI_LogProperty::log_detail`

Bitmap to control the fidelity of what is stored in the log entry.

OSAPI_LOG_DETAIL_MODULENAME - Log modulename

OSAPI_LOG_DETAIL_SOURCEFILE - Log source file

OSAPI_LOG_DETAIL_LINENUMBER - Log line number

OSAPI_LOG_DETAIL_FUNCTIONNAME - Log function name

write_buffer

`OSAPI_Log_write_buffer_T OSAPI_LogProperty::write_buffer`

Function pointer to output a log buffer.

7.145 OSAPI_SharedMemorySegmentHandle Struct Reference

```
#include <netio_zcopy_shm_segment.h>
```

Public Attributes

- struct `OSAPI_SharedMemorySegmentHeader` * `ptr_header`
- void * `ptr_user_data`
- `OSAPI_SHMEM_MODE` mode

7.145.1 Detailed Description

Opaque handle initialized by the create and attach methods and used to interact with a shared memory segment.

7.145.2 Member Data Documentation

ptr_header

```
struct OSAPI\_SharedMemorySegmentHeader* OSAPI_SharedMemorySegmentHandle::ptr_header
```

Pointer to segment header in shared memory

ptr_user_data

```
void* OSAPI_SharedMemorySegmentHandle::ptr_user_data
```

Pointer to where user data can be stored in shared memory

mode

```
OSAPI\_SHMEM\_MODE OSAPI_SharedMemorySegmentHandle::mode
```

Mode requested by the caller of create or attach

7.146 OSAPI_SharedMemorySegmentHeader Struct Reference

```
#include <netio_zcopy_shm_segment.h>
```

Public Attributes

- [RTI_UINT64](#) size
- [OSAPI_SHMEM_MODE](#) mode

7.146.1 Detailed Description

Opaque header at beginning of shared memory segment to share information about the segment between processes.

7.146.2 Member Data Documentation

size

```
RTI\_UINT64 OSAPI_SharedMemorySegmentHeader::size
```

Size requested by the caller of create

mode

[OSAPI_SHMEM_MODE](#) OSAPI_SharedMemorySegmentHeader::mode

Mode requested by the caller of create

7.147 OSAPI_SystemI Struct Reference

System API.

```
#include <osapi_system.h>
```

Public Attributes

- [OSAPI_System_start_timer_T start_timer](#)
Implementation of function to start a timer.
- [OSAPI_System_get_timer_resolution_T get_timer_resolution](#)
Implementation of function to get timer resolution.
- [OSAPI_System_get_time_T get_time](#)
Implementation of function to get the current time.
- [OSAPI_System_generate_uuid_T generate_uuid](#)
Implementation of function to generate a UUID.
- [OSAPI_System_get_hostname_T get_hostname](#)
Implementation of function to get the hostname.
- [OSAPI_System_get_ticktime_T get_ticktime](#)
Implementation of function to get the tick time.

7.147.1 Detailed Description

System API.

The methods in this interface are called by the corresponding OSAPI_System function. For example, OSAPI_System↵_get_time() calls system->[get_time\(\)](#).

7.147.2 Member Data Documentation

start_timer

[OSAPI_System_start_timer_T](#) OSAPI_SystemI::start_timer

Implementation of function to start a timer.

get_timer_resolution

`OSAPI_System_get_timer_resolution_T OSAPI_SystemI::get_timer_resolution`

Implementation of function to get timer resolution.

get_time

`OSAPI_System_get_time_T OSAPI_SystemI::get_time`

Implementation of function to get the current time.

generate_uuid

`OSAPI_System_generate_uuid_T OSAPI_SystemI::generate_uuid`

Implementation of function to generate a UUID.

get_hostname

`OSAPI_System_get_hostname_T OSAPI_SystemI::get_hostname`

Implementation of function to get the hostname.

get_ticktime

`OSAPI_System_get_ticktime_T OSAPI_SystemI::get_ticktime`

Implementation of function to get the tick time.

7.148 OSAPI_SystemProperty Struct Reference

The System Properties.

```
#include <osapi_system.h>
```

Public Attributes

- struct `OSAPI_TimerProperty timer_property`
Timer properties.
- char `hostname [OSAPI_SYSTEM_MAX_HOSTNAME]`
The hostname for the node.
- `RTI_INT32 max_user_blocking_threads`
The maximum number of threads which can block at the same time.
- `RTI_INT32 max_timers`
The maximum number timers the OS platform support library is required to support.
- struct `OSAPI_TaskProperty task_scheduler`

7.148.1 Detailed Description

The System Properties.

System properties

7.148.2 Member Data Documentation

timer_property

```
struct OSAPI\_TimerProperty OSAPI_SystemProperty::timer_property
```

Timer properties.

The timer properties control the time thread running the internal RTI Connex DDS Micro timer tick. Note that each port of RTI Connex DDS Micro may choose to implement the timer tick differently. Please refer to platform specific information for details.

hostname

```
char OSAPI_SystemProperty::hostname[OSAPI\_SYSTEM\_MAX\_HOSTNAME]
```

The hostname for the node.

RTI Connex DDS Micro uses this name when it announces itself on the network. This name is used by other RTI products. The hostname must not exceed [OSAPI_SYSTEM_MAX_HOSTNAME](#), including the \0 character.

max_user_blocking_threads

```
RTI\_INT32 OSAPI_SystemProperty::max_user_blocking_threads
```

The maximum number of threads which can block at the same time.

RTI Connex DDS Micro supports blocking operations, such as [DDS_KEEP_ALL_HISTORY_QOS](#), and may block the user thread in certain conditions, e.g. if out of resources. The maximum number of threads that RTI Connex DDS Micro can block at the same time is controlled by this property.

If a blocking operation is unable to acquire a blocking resource, the operation will return with [DDS_RETCODE_OUT_OF_RESOURCES](#).

Note that receive threads created by a transport and threads calling DDS APIs are both considered user threads.

[default] 32

[range] [1, INT_MAX]

max_timers

```
RTI_INT32 OSAPI_SystemProperty::max_timers
```

The maximum number timers the OS platform support library is required to support.

RTI Connex DDS Micro implements its own software timers to keep track of Qos policies such as [DDS_DeadlineQosPolicy](#). These software timers are driven by the underlying platform RTI Connex DDS Micro is executing on. This property configures the maximum number of timers that the OS is required to support.

RTI Connex DDS Micro allocated uses these timers as follows:

- One timer is allocated per [DDSDomainParticipant](#)
- One timer is allocated per [DDSDomainParticipant](#) for each [Zero Copy v2 API](#) transport enabled on a [DDSDomainParticipant](#).

The default value supports 8 DomainParticipants without [Zero Copy v2 API](#) enabled and 4 DomainParticipants with `netio_zcopy` enabled.

[default] 8

[range] [1, INT_MAX]

task_scheduler

```
struct OSAPI_TaskProperty OSAPI_SystemProperty::task_scheduler
```

Properties for the task scheduler

7.149 OSAPI_SystemTime Struct Reference

System Time.

```
#include <osapi_time.h>
```

Public Attributes

- [RTI_INT64 sec](#)
Seconds.
- [RTI_UINT32 nanosec](#)
Nanoseconds.

7.149.1 Detailed Description

System Time.

Expresses time in seconds and nanoseconds. Assumes that Nanoseconds is less than 1000000000.

7.149.2 Member Data Documentation

sec

`RTI_INT64 OSAPI_SystemTime::sec`

Seconds.

nanosec

`RTI_UINT32 OSAPI_SystemTime::nanosec`

Nanoseconds.

7.150 OSAPI_TaskProperty Struct Reference

Task properties.

```
#include <osapi_system.h>
```

Public Attributes

- struct `OSAPI_ThreadProperty thread`
Thread properties for tasks.
- `RTI_UINT32 rate`
The rate at which periodic tasks are run in nanosec.

7.150.1 Detailed Description

Task properties.

Properties for task creation used by the system.

7.150.2 Member Data Documentation

thread

`struct OSAPI_ThreadProperty OSAPI_TaskProperty::thread`

Thread properties for tasks.

The thread properties to use when creating task threads.

Task thread properties

rate

`RTI_UINT32 OSAPI_TaskProperty::rate`

The rate at which periodic tasks are run in nanosec.

The rate at which periodic tasks are run in nanosec

7.151 OSAPI_ThreadProperty Struct Reference

Properties for thread creation.

```
#include <osapi_thread.h>
```

Public Attributes

- [RTI_UINT32 stack_size](#)

Hint on required stack-size in bytes. The special value OSAPI_THREAD_USE_OSDEFAULT_STACKSIZE means do not set the stack-size and rely on the OS defaults.

- [RTI_INT32 priority](#)

Hint on required priority. A value ≥ 0 is used as is, and negative value is mapped according to the platform documentation.

- [OSAPI_ThreadOptions options](#)

Hint for thread creation to the OS.

7.151.1 Detailed Description

Properties for thread creation.

7.151.2 Member Data Documentation

stack_size

`RTI_UINT32 OSAPI_ThreadProperty::stack_size`

Hint on required stack-size in bytes. The special value OSAPI_THREAD_USE_OSDEFAULT_STACKSIZE means do not set the stack-size and rely on the OS defaults.

priority

`RTI_INT32 OSAPI_ThreadProperty::priority`

Hint on required priority. A value ≥ 0 is used as is, and negative value is mapped according to the platform documentation.

options

`OSAPI_ThreadOptions OSAPI_ThreadProperty::options`

Hint for thread creation to the OS.

7.152 OSAPI_TimerProperty Struct Reference

Timer properies.

```
#include <osapi_timer.h>
```

Public Attributes

- struct `OSAPI_ThreadProperty thread`
The thread settings for the timer thread, if a thread is created.

7.152.1 Detailed Description

Timer properies.

7.152.2 Member Data Documentation

thread

`struct OSAPI_ThreadProperty OSAPI_TimerProperty::thread`

The thread settings for the timer thread, if a thread is created.

7.153 PSK_ErrListener Struct Reference

A structure for holding callback pointers in case of an error.

```
#include <psk_oss1_transform.h>
```

Public Attributes

- `int(* callback)(const char *msg, size_t msg_len, void *user_ptr)`
Prototype for callback to be invoked in case of error.
- `void * user_ptr`
Pointer to be used in error callback.

7.153.1 Detailed Description

A structure for holding callback pointers in case of an error.

7.153.2 Member Data Documentation

callback

```
int (* PSK_ErrListener::callback) (const char *msg, size_t msg_len, void *user_ptr)
```

Prototype for callback to be invoked in case of error.

This method is invoked for any error that has been recorded in the PSK Transformation PSL. If its return value is not 0, the queue of errors continues to be processed (with the associated callback invocations). If its return value is 0, the walking over errors is cancelled.

user_ptr

```
void* PSK_ErrListener::user_ptr
```

Pointer to be used in error callback.

7.154 PSK_FilePassTrackerErrListener Struct Reference

A structure for holding callback pointers in case of an error.

```
#include <psk_file_pass_tracker.h>
```

Public Attributes

- `int(* callback)(const char *msg, size_t msg_len, void *user_ptr)`
Prototype for callback to be invoked in case of error.
- `void * user_ptr`
Pointer to be used in error callback.

7.154.1 Detailed Description

A structure for holding callback pointers in case of an error.

7.154.2 Member Data Documentation

callback

```
int (* PSK_FilePassTrackerErrListener::callback) (const char *msg, size_t msg_len, void *user_ptr)
```

Prototype for callback to be invoked in case of error.

This method is invoked for any error that has been recorded in the PSK Transformation PSL. If its return value is not 0, the queue of errors continues to be processed (with the associated callback invocations). If its return value is 0, the walking over errors is cancelled.

user_ptr

```
void* PSK_FilePassTrackerErrListener::user_ptr
```

Pointer to be used in error callback.

7.155 PSK_PassTrackerConfiguration Struct Reference

Configuration structure for file-based PSK pass tracker.

```
#include <psk_file_pass_tracker.h>
```

Public Attributes

- int [polling_interval_ms](#)
Polling interval in milliseconds for checking file changes.
- [RTI_BOOL enable_init_read](#)
Whether to read the passphrase file during initialization.
- struct [PSK_FilePassTrackerErrListener](#) [err_listener](#)
Configuration items for an error callback mechanism.

7.155.1 Detailed Description

Configuration structure for file-based PSK pass tracker.

7.155.2 Member Data Documentation

polling_interval_ms

```
int PSK_PassTrackerConfiguration::polling_interval_ms
```

Polling interval in milliseconds for checking file changes.

enable_init_read

```
RTI_BOOL PSK_PassTrackerConfiguration::enable_init_read
```

Whether to read the passphrase file during initialization.

err_listener

```
struct PSK_FilePassTrackerErrListener PSK_PassTrackerConfiguration::err_listener
```

Configuration items for an error callback mechanism.

7.156 RHSMHistoryFactory Class Reference

<<eXtension>> A reader history factory that will be used by the DataReader to create a history cache.

```
#include <dds_cpp_rh_sm.hxx>
```

Static Public Member Functions

- static struct RT_ComponentFactoryI * [get_interface](#) ()
Gets the singleton instance of the RHSM reader history factory.

7.156.1 Detailed Description

<<eXtension>> A reader history factory that will be used by the DataReader to create a history cache.

7.156.2 Member Function Documentation

get_interface()

```
static struct RT_ComponentFactoryI * RHSMHistoryFactory::get_interface () [static]
```

Gets the singleton instance of the RHSM reader history factory.

This function gets the singleton instance of the RHSM history plugin factory that is used by the middleware to create a ReaderHistory plugin. The singleton instance of the factory must be registered with the name "rh" in the DomainParticipantFactory.

Returns

Pointer to RHSM history plugin factory

MT Safety:

This operation is thread safe.

7.157 RTPS_Checksum Union Reference

<<eXtension>> RTPS checksum type

```
#include <rtps_checksum.h>
```

Public Attributes

- [RTI_UINT32 checksum32](#)
A 32-bit unsigned checksum in host endianness order.
- [RTI_UINT64 checksum64](#)
A 64-bit unsigned integer checksum in host endianness order.
- [RTI_UINT8 checksum128](#) [16]
A 128-bit checksum as a 16-byte array.

7.157.1 Detailed Description

<<eXtension>> RTPS checksum type

7.158 RTPS_ChecksumClass Struct Reference

<<eXtension>> Definition of a checksum function

```
#include <rtps_checksum.h>
```

Public Attributes

- [RTPS_ChecksumClassId_T class_id](#)
The checksum class ID.
- void * [context](#)
User defined context.
- [RTPS_ChecksumCalculate_T checksum_calculate](#)
User defined function to calculate a checksum.

7.158.1 Detailed Description

<<eXtension>> Definition of a checksum function

7.159 RTPS_ChecksumProperty Struct Reference

<<eXtension>> Checksum properties for the RTPS plugin

```
#include <rtps_checksum.h>
```

Public Attributes

- [RTPS_ChecksumClass_T builtin_checksum32_class](#)
The builtin 32-bit checksum class.
- [RTPS_ChecksumClass_T builtin_checksum64_class](#)
The builtin 64-bit checksum class.
- [RTPS_ChecksumClass_T builtin_checksum128_class](#)
The builtin 128-bit checksum class.
- [RTPS_ChecksumTxMode_T checksum_tx_mode](#)
The transmit mode for checksums.
- [RTI_BOOL allow_built_in_override](#)
If TRUE, allow overriding the builtin checksum functions.

7.159.1 Detailed Description

<<[eXtension](#)>> Checksum properties for the RTPS plugin

7.160 RTPS_InterfaceFactoryProperty Struct Reference

<<[eXtension](#)>> Properties for the RTPS Transport.

```
#include <rtps_rtps.h>
```

Public Attributes

- struct [RTPS_ChecksumProperty checksum](#)
The checksum properties for RTPS.

7.160.1 Detailed Description

<<[eXtension](#)>> Properties for the RTPS Transport.

7.161 RTRegistry Class Reference

<<[eXtension](#)>> A run-time component factory.

```
#include <dds_cpp_rt.hxx>
```

Public Member Functions

- bool [register_component](#) (const char *name, RT_ComponentFactoryI *intf, RT_ComponentFactoryProperty *property, RT_ComponentFactoryListener *listener)
Register a factory of RT components
- bool [unregister](#) (const char *name, RT_ComponentFactoryProperty **property, RT_ComponentFactoryListener **listener)
Remove a component factory from the registry

7.161.1 Detailed Description

<<*eXtension*>> A run-time component factory.

7.161.2 Member Function Documentation

register_component()

```
bool RTRegistry::register_component (
    const char * name,
    RT_ComponentFactoryI * intf,
    RT_ComponentFactoryProperty * property,
    RT_ComponentFactoryListener * listener)
```

Register a factory of RT components

In order to make a component available, it is necessary to register a factory for the components. The factory is responsible for creating and deleting components.

The name of the factory must be unique. Otherwise, if a factory already exists with the the same name, the registration will fail.

Parameters

<i>name</i>	<< <i>in</i> >> Name of the component factory
<i>intf</i>	<< <i>in</i> >> Pointer to the implementation of the factory interface
<i>property</i>	<< <i>in</i> >> Properties with which to create the factory
<i>listener</i>	<< <i>in</i> >> The listener to install with the factory

Returns

RTI_TRUE on success, RTI_FALSE on failure

See also

[RTRegistry::unregister](#)

unregister()

```
bool RTRegistry::unregister (
    const char * name,
    RT_ComponentFactoryProperty ** property,
    RT_ComponentFactoryListener ** listener)
```

Remove a component factory from the registry

After a component factory is unregistered, it will no longer be possible to create or delete components with the factory.

Parameters

<i>name</i>	<< <i>in</i> >> Name of the component factory
<i>property</i>	<< <i>in</i> >> Properties with which the factory was created
<i>listener</i>	<< <i>in</i> >> The listener installed with the factory

Returns

RTI_TRUE on success, RTI_FALSE on failure

See also

[RTRegistry::register_component](#)

7.162 SHMEMInterfaceFactory Class Reference

<<*eXtension*>> A SHMEM transport factory that will be used by a DomainParticipant to create a SHMEM interface. Only one SHMEM interface can be registered with the DomainParticipantFactory.

```
#include <dds_cpp_netio_shmem.hxx>
```

Static Public Member Functions

- static struct RT_ComponentFactoryI * [get_interface](#) ()
Gets the singleton instance of the SHMEM interface factory.

7.162.1 Detailed Description

<<*eXtension*>> A SHMEM transport factory that will be used by a DomainParticipant to create a SHMEM interface. Only one SHMEM interface can be registered with the DomainParticipantFactory.

7.162.2 Member Function Documentation**get_interface()**

```
static struct RT_ComponentFactoryI * SHMEMInterfaceFactory::get_interface () [static]
```

Gets the singleton instance of the SHMEM interface factory.

This function gets the singleton instance of the SHMEM factory that is used by the middleware to create a SHMEM transport instance.

Returns

Pointer to SHMEM factory instance

7.163 UDP_InterfaceFactoryProperty Struct Reference

<<*eXtension*>> Properties for the UDP Transport.

```
#include <netio_udp.h>
```

Public Attributes

- struct REDA_StringSeq [allow_interface](#)
Sequence of allowed interface names.
- struct REDA_StringSeq [deny_interface](#)
Sequence of denied interface names. This list is checked after the [allow_interface](#) list.
- RTI_INT32 [max_send_buffer_size](#)
The size of the send socket buffer.
- RTI_INT32 [max_receive_buffer_size](#)
The size of the receive socket buffer.
- RTI_INT32 [max_message_size](#)
The maximum size of the message which can be sent or received.
- RTI_INT32 [max_send_message_size](#)
The maximum size of the message which can be sent. This is only a hint and not enforced.
- RTI_INT32 [multicast_ttl](#)
The maximum TTL.
- struct UDP_NatEntrySeq [nat](#)
Configure network address translation (NAT).
- struct UDP_InterfaceTableEntrySeq [if_table](#)
The interface table if interfaces are added manually.
- REDA_String_T [multicast_interface](#)
The network interface to use to send to multicast.
- RTI_BOOL [is_default_interface](#)
If this should be considered the default UDP interface if no other UDP interface is found to handle a route.
- RTI_BOOL [disable_auto_interface_config](#)
Disable reading of available network interfaces using system information and instead rely on the manually configured interface table.
- RTI_BOOL [multicast_loopback_disabled](#)
Prevents the transport plugin from putting multicast packets onto the loopback interface.
- struct OSAPI_ThreadProperty [recv_thread](#)
Thread properties for each receive thread created by this NETIO interface.
- RTI_BOOL [enable_interface_bind](#)
Bind receive sockets to specific interfaces.
- RTI_BOOL [disable_multicast_bind](#)
Disable a multicast receive socket binding to a multicast address.
- RTI_BOOL [disable_multicast_interface_select](#)
Do not select outgoing interfaces when sending to multicast groups.
- struct UDP_TransformRuleSeq [source_rules](#)
Rules for how to transform received UDP payloads based on the source address.
- struct UDP_TransformRuleSeq [destination_rules](#)
Rules for how to transform sent UDP payloads based on the destination address.

- [UDP_TransformUdpMode_T transform_udp_mode](#)
Determines how regular UDP is supported when transformations are supported.
- [RTI_INT32 transform_locator_kind](#)
The locator to use for locators that have transformations.
- [RTI_INT32 transport_priority_mapping_low](#)
Set low value of output range to IPv4 TOS.
- [RTI_INT32 transport_priority_mapping_high](#)
Highest transport priority value for DSCP mapping.
- [RTI_UINT32 transport_priority_mask](#)
Set mask for use of transport priority field.
- [RTI_UINT32 max_unicast_send_sockets](#)
The number of unicast send sockets to create per transport instance.

7.163.1 Detailed Description

<<[eXtension](#)>> Properties for the UDP Transport.

NOTE: some default values (or entire data fields) may not be used on certain platforms. Refer to the Platform Notes in the User's Manual for more details.

7.163.2 Member Data Documentation

allow_interface

```
struct REDA_StringSeq UDP_InterfaceFactoryProperty::allow_interface
```

Sequence of allowed interface names.

deny_interface

```
struct REDA_StringSeq UDP_InterfaceFactoryProperty::deny_interface
```

Sequence of denied interface names. This list is checked after the `allow_interface` list.

max_send_buffer_size

```
RTI\_INT32 UDP_InterfaceFactoryProperty::max_send_buffer_size
```

The size of the send socket buffer.

[default] 256 * 1024, 65535 for QNX ([note](#))

max_receive_buffer_size

```
RTI_INT32 UDP_InterfaceFactoryProperty::max_receive_buffer_size
```

The size of the receive socket buffer.

[default] 256 * 1024, 65535 for QNX ([note](#))

max_message_size

```
RTI_INT32 UDP_InterfaceFactoryProperty::max_message_size
```

The maximum size of the message which can be sent or received.

[default] 8 * 1024 ([note](#)) **[range]** 1 - UDP_MAX_PACKET_SIZE (65507) ([note](#))

max_send_message_size

```
RTI_INT32 UDP_InterfaceFactoryProperty::max_send_message_size
```

The maximum size of the message which can be sent. This is only a hint and not enforced.

This parameters is useful to UDP transformation functions which must allocate a buffer to put the result in.

[default] -1, but note that this is a place holder for UDP_MAX_MESSAGE_SIZE_UNLIMITED ([note](#))

multicast_ttl

```
RTI_INT32 UDP_InterfaceFactoryProperty::multicast_ttl
```

The maximum TTL.

[default] 1 ([note](#))

nat

```
struct UDP_NatEntrySeq UDP_InterfaceFactoryProperty::nat
```

Configure network address translation (NAT).

Configure network address translation (NAT). This feature only supports translation between a private and public IP address; UDP ports are not translated. Does not support any hole punching technique or WAN server, thus is only useful when the private and public address mapping is static. Please refer to [UDP_NatEntrySeq](#) for the definition of the sequence element.

if_table

```
struct UDP_InterfaceTableEntrySeq UDP_InterfaceFactoryProperty::if_table
```

The interface table if interfaces are added manually.

multicast_interface

```
REDA_String_T UDP_InterfaceFactoryProperty::multicast_interface
```

The network interface to use to send to multicast.

is_default_interface

```
RTI_BOOL UDP_InterfaceFactoryProperty::is_default_interface
```

If this should be considered the default UDP interface if no other UDP interface is found to handle a route.

[default] 1 (RTI_TRUE) ([note](#))

disable_auto_interface_config

```
RTI_BOOL UDP_InterfaceFactoryProperty::disable_auto_interface_config
```

Disable reading of available network interfaces using system information and instead rely on the manually configured interface table.

[default] 0 (RTI_FALSE) ([note](#))

multicast_loopback_disabled

```
RTI_BOOL UDP_InterfaceFactoryProperty::multicast_loopback_disabled
```

Prevents the transport plugin from putting multicast packets onto the loopback interface.

If multicast loopback is disabled (this value is set to 1), then when sending multicast packets, RTI Connext DDS Micro will not put a copy of the packets on the loopback interface. This prevents applications on the same node (including itself) from receiving those packets.

This value is set to 0 by default, meaning multicast loopback is enabled.

Disabling multicast loopback (setting this value to 1) may result in minor performance gains when using multicast.

[default] 0 (RTI_FALSE) ([note](#))

recv_thread

```
struct OSAPI_ThreadProperty UDP_InterfaceFactoryProperty::recv_thread
```

Thread properties for each receive thread created by this NETIO interface.

[default] See ::DDS_UDP_THREAD_PROPERTY_DEFAULT for more detail ([note](#))

enable_interface_bind

```
RTI_BOOL UDP_InterfaceFactoryProperty::enable_interface_bind
```

Bind receive sockets to specific interfaces.

When this is set to 1, the UDP transport binds each receive port to a specific interface when the allow_interface/deny_interface lists are non-empty.

[default] 0 ([note](#))

disable_multicast_bind

```
RTI_BOOL UDP_InterfaceFactoryProperty::disable_multicast_bind
```

Disable a multicast receive socket binding to a multicast address.

NOTE: This option has no effect on Windows. On Windows a multicast socket is only bound to the UDP port.

The default value is 0, meaning that a multicast socket is bound to both the UDP port and multicast address.

When this is set to 1, the UDP transport binds a multicast receive socket only to the UDP port.

[default] 0 ([note](#))

disable_multicast_interface_select

```
RTI_BOOL UDP_InterfaceFactoryProperty::disable_multicast_interface_select
```

Do not select outgoing interfaces when sending to multicast groups.

The default value is 0 and RTI Connex DDS Micro will select the outgoing interfaces when sending to multicast either using the allow_interface and deny_interface properties, or the multicast_interface property.

When this is set to 1, RTI Connex DDS Micro will not select the outgoing interface when sending to multicast groups. This may be useful if the OS has fine grained control to select which interface should be used to send to multicast.

[default] 0 ([note](#))

source_rules

```
struct UDP_TransformRuleSeq UDP_InterfaceFactoryProperty::source_rules
```

Rules for how to transform received UDP payloads based on the source address.

destination_rules

```
struct UDP_TransformRuleSeq UDP_InterfaceFactoryProperty::destination_rules
```

Rules for how to transform sent UDP payloads based on the destination address.

transform_udp_mode

```
UDP_TransformUdpMode_T UDP_InterfaceFactoryProperty::transform_udp_mode
```

Determines how regular UDP is supported when transformations are supported.

[default] UDP_TRANSFORM_UDP_MODE_DISABLED ([note](#))

transform_locator_kind

```
RTI_INT32 UDP_InterfaceFactoryProperty::transform_locator_kind
```

The locator to use for locators that have transformations.

When transformation rules have been enabled, they are announced as a vendor-specific locator. This property overrides this value.

NOTE: Changing this value may prevent communication.

NOTE: Must be greater or equal than NETIO_ADDRESS_KIND_USER and not equal to NETIO_ADDRESS_KIND_SHARED or NETIO_ADDRESS_KIND_INTRA.

[default] NETIO_ADDRESS_KIND_TUDPv4 ([note](#))

transport_priority_mapping_low

```
RTI_INT32 UDP_InterfaceFactoryProperty::transport_priority_mapping_low
```

Set low value of output range to IPv4 TOS.

This is used in conjunction with [transport_priority_mask](#) and [transport_priority_mapping_high](#) to define the mapping from the DDS transport priority to the IPv4 TOS field. Defines the low value of the output range for scaling.

Note that IPv4 TOS is generally an 8-bit value. The default value is 0.

transport_priority_mapping_high

`RTI_INT32 UDP_InterfaceFactoryProperty::transport_priority_mapping_high`

Highest transport priority value for DSCP mapping.

When [transport_priority_mask](#) is non-zero, this value defines the highest transport priority value that will be mapped to DSCP. Transport priority values higher than this value will be mapped to DSCP 63.

The default value is 255.

transport_priority_mask

`RTI_UINT32 UDP_InterfaceFactoryProperty::transport_priority_mask`

Set mask for use of transport priority field.

This is used in conjunction with [transport_priority_mapping_low](#) and [transport_priority_mapping_high](#) to define the mapping from the DDS transport priority to the IPv4 TOS field. Defines a contiguous region of bits in the 32-bit transport priority value that is used to generate values for the IPv4 TOS field on an outgoing socket. For example, the value 0x0000ff00 causes bits 9-16 (8 bits) to be used in the mapping. The value will be scaled from the mask range (0x0000 - 0xff00 in this case) to the range specified by low and high. If the mask is set to zero, then the transport will not set IPv4 TOS for send sockets.

max_unicast_send_sockets

`RTI_UINT32 UDP_InterfaceFactoryProperty::max_unicast_send_sockets`

The number of unicast send sockets to create per transport instance.

By default, the transport uses a single socket per instance for outgoing unicast communication.

When this value is greater than 1, the transport creates the specified number of send sockets per instance for outgoing communication.

This setting is particularly useful when used with the [TRANSPORT_PRIORITY](#) QoS policy. When the transport creates multiple sockets, it maintains an internal priority-to-socket mapping. For each outgoing message, the transport computes the effective priority (after applying the configured priority mask and mapping range) and selects a preferred socket based on this mapping.

If the selected socket is already configured with the required priority, the transport sends the message immediately. Otherwise, the transport updates the socket's IP_TOS value before transmission. Increasing the number of sockets therefore reduces the likelihood of runtime priority reconfiguration and can improve performance in systems that use multiple distinct priorities.

You can configure the value to match the total number of distinct effective priorities assigned to all [DDSDataWriter](#) and [DDSDataReader](#) instances using the interface. When the number of sockets is greater than or equal to the number of distinct effective priorities, each priority can be associated with a dedicated socket, eliminating the need for priority updates at runtime.

If the number of sockets is smaller than the number of distinct effective priorities, multiple priorities will share sockets. In this case, the transport distributes priorities evenly across the available sockets. However, because priorities are numeric values, certain structured priority sets (for example, DSCP codepoints that are spaced at regular intervals) may cluster onto the same socket depending on the configured socket count. For best distribution, avoid choosing a socket count that shares common divisors with typical priority spacing (for example, powers of 2 when using DSCP values that are multiples of 8).

NOTE: Increasing the number of sockets increases memory usage and file descriptor consumption. You should balance performance benefits against memory and resource overhead when selecting an appropriate value. **[default]** 1 ([note](#))

7.164 UDP_InterfaceTableEntry Struct Reference

Generic structure to describe a network interface.

```
#include <netio_udp.h>
```

Public Attributes

- [RTI_UINT32 flags](#)
Flags to indicate if the interface is up etc.
- [RTI_UINT32 address](#)
The address of the interface as configured by the OS.
- [RTI_UINT32 netmask](#)
The netmask of the interface as configured by the OS.
- char [ifname](#) [UDP_INTERFACE_MAX_IFNAME]
The name of the interface as configured by the OS.

7.164.1 Detailed Description

Generic structure to describe a network interface.

7.164.2 Member Data Documentation

flags

```
RTI_UINT32 UDP_InterfaceTableEntry::flags
```

Flags to indicate if the interface is up etc.

address

```
RTI_UINT32 UDP_InterfaceTableEntry::address
```

The address of the interface as configured by the OS.

netmask

```
RTI_UINT32 UDP_InterfaceTableEntry::netmask
```

The netmask of the interface as configured by the OS.

ifname

```
char UDP_InterfaceTableEntry::ifname[UDP_INTERFACE_MAX_IFNAME]
```

The name of the interface as configured by the OS.

7.165 UDP_InterfaceTableEntrySeq Struct Reference

<<*eXtension*>> Declares IDL sequence<UDP_InterfaceTableEntry>

```
#include <netio_udp.h>
```

7.165.1 Detailed Description

<<*eXtension*>> Declares IDL sequence<UDP_InterfaceTableEntry>

7.166 UDP_NatEntry Struct Reference

<<*eXtension*>> Properties for a Network Address Translation entry

```
#include <netio_udp.h>
```

7.166.1 Detailed Description

<<*eXtension*>> Properties for a Network Address Translation entry

Adding [UDP_NatEntry](#) entries to the [UDP_InterfaceFactoryProperty::nat](#) sequence enables translation between local and public network addresses. Please refer to [Sequence Support](#) for more information on how to manages sequences in RTI Connex DDS Micro.

7.167 UDP_NatEntrySeq Struct Reference

<<*eXtension*>> Declares IDL sequence<UDP_NatEntry>

```
#include <netio_udp.h>
```

7.167.1 Detailed Description

<<*eXtension*>> Declares IDL sequence<UDP_NatEntry>

7.168 UDPInterfaceFactory Class Reference

<<[eXtension](#)>> A UDP transport factory that will be used by a DomainParticipant to create a UDP interface. Only one UDP interface can be registered with the DomainParticipantFactory.

```
#include <netio_udp_cpp.hxx>
```

Static Public Member Functions

- static struct RT_ComponentFactoryI * [get_interface](#) ()
Gets the singleton instance of the UDP interface factory.

7.168.1 Detailed Description

<<[eXtension](#)>> A UDP transport factory that will be used by a DomainParticipant to create a UDP interface. Only one UDP interface can be registered with the DomainParticipantFactory.

7.168.2 Member Function Documentation

[get_interface\(\)](#)

```
static struct RT_ComponentFactoryI * UDPInterfaceFactory::get_interface () [static]
```

Gets the singleton instance of the UDP interface factory.

This function gets the singleton instance of the UDP factory that is used by the middleware to create a UDP transport instance. [UDP_InterfaceFactoryProperty](#)

Returns

Pointer to UDP factory instance()

7.169 UDPInterfaceTable Class Reference

```
#include <netio_udp_cpp.hxx>
```

Static Public Member Functions

- static bool [add_entry](#) (struct [UDP_InterfaceTableEntrySeq](#) *seq, [RTI_UINT32](#) address, [RTI_UINT32](#) netmask, const char *ifname, [RTI_UINT32](#) flags)
Add an entry to the list of available interfaces.

7.169.1 Detailed Description

7.170 WHSMHistoryFactory Class Reference

<<*eXtension*>> A writer history factory that will be used by the DataWriter to create a history cache.

```
#include <dds_cpp_wh_sm.hxx>
```

Static Public Member Functions

- static struct RT_ComponentFactoryI * [get_interface](#) ()
Gets the singleton instance of the WHSM writer history factory.

7.170.1 Detailed Description

<<*eXtension*>> A writer history factory that will be used by the DataWriter to create a history cache.

7.170.2 Member Function Documentation

get_interface()

```
static struct RT_ComponentFactoryI * WHSMHistoryFactory::get_interface () [static]
```

Gets the singleton instance of the WHSM writer history factory.

This function gets the singleton instance of the WHSM history plugin factory that is used by the middleware to create a WriterHistory plugin. The singleton instance of the factory must be registered with the name "wh" in the DomainParticipantFactory.

Returns

Pointer to WHSM history plugin factory

7.171 ZCOPY_NotifInterfaceFactoryProperty Struct Reference

Notification interface property.

```
#include <netio_zcopy_notif_interface.h>
```

Public Attributes

- [RTI_UINT32 max_samples_per_notif](#)
Maximum samples to process per notification per DataReader.
- struct [ZCOPY_NotifUserInterfaceI](#) * [user_intf](#)
The downstream transport interface.
- void * [user_property](#)
Pass-through pointer to the downstream transport. This must be valid for the life-cycle of the transport.

7.171.1 Detailed Description

Notification interface property.

7.171.2 Member Data Documentation

max_samples_per_notif

```
RTI_UINT32 ZCOPY_NotifInterfaceFactoryProperty::max_samples_per_notif
```

Maximum samples to process per notification per DataReader.

This setting optimizes performance when sending large amounts of data. However, setting it to an excessively high value may prevent other threads from running effectively.

[default] 1.

user_intf

```
struct ZCOPY_NotifUserInterfaceI* ZCOPY_NotifInterfaceFactoryProperty::user_intf
```

The downstream transport interface.

[default] NULL.

user_property

```
void* ZCOPY_NotifInterfaceFactoryProperty::user_property
```

Pass-through pointer to the downstream transport. This must be valid for the life-cycle of the transport.

[default] NULL.

7.172 ZCOPY_NotifMechanismProperty Struct Reference

Notification mechanism property for the default implementation.

```
#include <netio_zcopy_default_notif_mech.h>
```

Public Attributes

- [RTI_UINT32 intf_addr](#)
The interface address for this implementation. The value should be globally unique.
- struct [OSAPI_ThreadProperty thread_prop](#)
The thread property for the receive thread in the notification mechanism.
- [RTI_UINT32 max_receive_ports](#)
The maximum number of receive ports.
- [RTI_UINT32 max_routes](#)
The maximum number of outgoing routes this notification mechanism can handle.

7.172.1 Detailed Description

Notification mechanism property for the default implementation.

7.172.2 Member Data Documentation

intf_addr

```
RTI_UINT32 ZCOPY_NotifMechanismProperty::intf_addr
```

The interface address for this implementation. The value should be globally unique.

[default] 0.

thread_prop

```
struct OSAPI_ThreadProperty ZCOPY_NotifMechanismProperty::thread_prop
```

The thread property for the receive thread in the notification mechanism.

[default] [OSAPI_THREAD_PROPERTY_DEFAULT](#).

max_receive_ports

```
RTI_UINT32 ZCOPY_NotifMechanismProperty::max_receive_ports
```

The maximum number of receive ports.

[default] 2.

max_routes

`RTI_UINT32 ZCOPY_NotifMechanismProperty::max_routes`

The maximum number of outgoing routes this notification mechanism can handle.

[default] 32.

7.173 ZCOPY_NotifUserInterface Struct Reference

Notification mechanism user interface.

```
#include <netio_zcopy_notif_mechanism_intf.h>
```

Public Attributes

- [ZCOPY_NotifUserInterface_createFunc create_instance](#)
Create an instance of the notification mechanism interface with upstream as the owner.
- [ZCOPY_NotifUserInterface_deleteFunc delete_instance](#)
Delete an instance of the notification mechanism interface.
- [NETIO_Interface_resolve_addressFunc resolve_address](#)
Resolve an address string from the notification mechanism interface.
- [NETIO_Interface_get_route_tableFunc get_route_table](#)
Get the notification mechanism interface route table.
- [ZCOPY_NotifUserInterface_reserve_addressFunc reserve_address](#)
Reserve an address with the notification mechanism interface.
- [ZCOPY_NotifUserInterface_release_addressFunc release_address](#)
Release addresses to the notification mechanism interface.
- [ZCOPY_NotifUserInterface_add_routeFunc add_route](#)
Add a route on the notification mechanism interface.
- [ZCOPY_NotifUserInterface_delete_routeFunc delete_route](#)
Delete a route on the notification mechanism interface.
- [ZCOPY_NotifUserInterface_bindFunc bind](#)
Bind on the notification mechanism interface.
- [ZCOPY_NotifUserInterface_unbindFunc unbind](#)
Unbind on the notification mechanism interface.
- [ZCOPY_NotifUserInterface_sendFunc send](#)
Send a notification using the notification mechanism interface.
- [ZCOPY_NotifUserInterface_notify_portFunc notify_rcv_port](#)
Notify the receive port.

7.173.1 Detailed Description

Notification mechanism user interface.

Interface functions for the notification mechanism user interface. This allows the user to specify custom implementations of the notification mechanism. The Default Notification Mechanism provided with /ndds can be accessed using `::DDSZCOPY_NotifUserInterface::get_interface`.

See also

`::DDSZCOPY_NotifMechanism::register`

7.173.2 Member Data Documentation

create_instance

`ZCOPY_NotifUserInterface_createFunc` `ZCOPY_NotifUserInterfaceI::create_instance`

Create an instance of the notification mechanism interface with upstream as the owner.

Parameters

<i>upstream[in]</i>	The upstream notification interface.
<i>user_property[in]</i>	The user property.

Returns

The user interface instance or NULL if an error occurred

delete_instance

`ZCOPY_NotifUserInterface_deleteFunc` `ZCOPY_NotifUserInterfaceI::delete_instance`

Delete an instance of the notification mechanism interface.

Not available in CERT.

Parameters

<i>user_intf[in]</i>	The user interface instance.
----------------------	------------------------------

resolve_address

`NETIO_Interface_resolve_addressFunc` `ZCOPY_NotifUserInterfaceI::resolve_address`

Resolve an address string from the notification mechanism interface.

Instruct the notification mechanism interface specified by `user_intf` to determine if the address string `address_string` is a valid address and return the result in `address_value`.

Parameters

in	<i>netio_intf</i>	Interface.
in	<i>netio_intf</i>	The interface entry requested to resolve the address.
in	<i>address_string</i>	The address to convert.
out	<i>address_value</i>	The converted address.
out	<i>invalid</i>	If the address is valid or not.

Returns

RTI_TRUE on success, RTI_FALSE on failure.

get_route_table

```
NETIO_Interface_get_route_tableFunc ZCOPY_NotifUserInterfaceI::get_route_table
```

Get the notification mechanism interface route table.

Instruct the notification mechanism interface *netio_intf* to return a sequence of address and netmask pairs this interface can send to.

Parameters

in	<i>netio_intf</i>	The Notif interface.
in, out	<i>address</i>	Sequence of NETIO addresses this interface understands.
in, out	<i>netmask</i>	Sequence of the corresponding netmasks.

Returns

RTI_TRUE on success, RTI_FALSE on failure.;

reserve_address

```
ZCOPY_NotifUserInterface_reserve_addressFunc ZCOPY_NotifUserInterfaceI::reserve_address
```

Reserve an address with the notification mechanism interface.

Instruct the notification mechanism interface specified by *user_intf* to set up resources for listening to messages on the address *src_addr* and return a *port_entry_out*. The *port_entry_out* will be provided to a bind call.

Parameters

in	<i>user_intf</i>	The user interface instance.
in	<i>src_addr</i>	The address to reserve.
out	<i>port_entry_out</i>	The port entry associated with the address.

release_address

`ZCOPY_NotifUserInterface_release_addressFunc` `ZCOPY_NotifUserInterfaceI::release_address`

Release addresses to the notification mechanism interface.

Instruct the notification mechanism interface specified by `user_intf` to release resources for listening to messages on the address `src_addr`.

Parameters

in	<i>user_intf</i>	The user interface instance.
in	<i>src_addr</i>	The address to release.
in	<i>port_entry</i>	The port entry associated with the address.

Returns

RTI_TRUE on success, RTI_FALSE otherwise

add_route

`ZCOPY_NotifUserInterface_add_routeFunc` `ZCOPY_NotifUserInterfaceI::add_route`

Add a route on the notification mechanism interface.

Instruct the notification mechanism interface specified by `user_intf` to add a route from the source to the destination. The `route_entry_out` will be provided to the user later when calling `send`.

Parameters

in	<i>user_intf</i>	The user interface instance.
in	<i>source</i>	The source address.
in	<i>destination</i>	The destination address.
out	<i>route_entry_out</i>	The route entry associated with the route.

Returns

RTI_TRUE on success, RTI_FALSE otherwise

delete_route

`ZCOPY_NotifUserInterface_delete_routeFunc` `ZCOPY_NotifUserInterfaceI::delete_route`

Delete a route on the notification mechanism interface.

Instruct the notification mechanism interface specified by `user_intf` to remove a route from the source to the destination.

Parameters

in	<i>user_intf</i>	The user interface instance.
in	<i>source</i>	The source address.
in	<i>destination</i>	The destination address.
in	<i>route_entry</i>	The route entry associated with the route.

Returns

RTI_TRUE on success, RTI_FALSE otherwise.

bind

`ZCOPY_NotifUserInterface_bindFunc` `ZCOPY_NotifUserInterfaceI::bind`

Bind on the notification mechanism interface.

Instruct the notification mechanism interface specified by `user_intf` to start listening for messages on the address specified by `src_addr` on the given `port_entry`.

Parameters

in	<i>user_intf</i>	The user interface instance.
in	<i>src_addr</i>	The remote address.
in	<i>dst_addr</i>	The local address.
in	<i>port_entry</i>	The port entry associated with the local address.
in	<i>bind_entry</i>	The bind entry associated with the bind.

Returns

RTI_TRUE on success, RTI_FALSE otherwise

unbind

`ZCOPY_NotifUserInterface_unbindFunc` `ZCOPY_NotifUserInterfaceI::unbind`

Unbind on the notification mechanism interface.

Instruct the notification mechanism interface specified by `user_intf` to stop listening for messages on the address specified by `src_addr`.

Parameters

in	<i>user_intf</i>	The user interface instance.
in	<i>src_addr</i>	The remote address.
in	<i>dst_addr</i>	The local address.
in	<i>port_entry</i>	The port entry associated with the local address.
out	<i>bind_entry_out</i>	The bind entry associated with the bind.

Returns

RTI_TRUE on success, RTI_FALSE otherwise.

send

`ZCOPY_NotifUserInterface_sendFunc` `ZCOPY_NotifUserInterfaceI::send`

Send a notification using the notification mechanism interface.

Instruct the notification mechanism interface specified by `user_intf` to send a notification to the destination using the `route_entry`.

Parameters

in	<i>user_intf</i>	The user interface instance.
in	<i>source</i>	The source address.
in	<i>destination</i>	The destination address.
in	<i>route_entry</i>	The route entry associated with the route.

Returns

RTI_TRUE on success, RTI_FALSE otherwise.

notify_rcv_port

```
ZCOPY_NotifUserInterface_notify_portFunc ZCOPY_NotifUserInterfaceI::notify_rcv_port
```

Notify the receive port.

Instruct the notification mechanism interface specified by *user_intf* to notify the receive port specified by *port_entry*.

Used by the reader to send notification on a specific receive port.

Parameters

in	<i>user_intf</i>	The user interface instance.
in	<i>port_entry</i>	The port entry to notify.

Returns

RTI_TRUE on success, RTI_FALSE otherwise.

8 File Documentation

8.1 app_gen_log.h File Reference

```
#include "osapi/osapi_log.h"
```

Macros

- #define `APPGEN_LOG_ALLOCATE_EC` (`APPGEN_LOG_BASE` + 1)
Failed to allocate memory for an Application Generator interface.
- #define `APPGEN_LOG_COPY_ENTITY_NAME_EC` (`APPGEN_LOG_BASE` + 2)
Cannot copy a name into an entity name. Name too long.
- #define `APPGEN_LOG_CREATE_ENTITY_NAME_EC` (`APPGEN_LOG_BASE` + 3)
Cannot generate an entity name. Name too long.
- #define `APPGEN_MULTIPPLICITY_EC` (`APPGEN_LOG_BASE` + 4)
Cannot create entity name because multiplicity is wrong (0)
- #define `APPGEN_FACTORY_REGISTER_EC` (`APPGEN_LOG_BASE` + 5)
Cannot register a factory.
- #define `APPGEN_LOG_FACTORY_TOPIC_NOT_FOUND_EC` (`APPGEN_LOG_BASE` + 6)
Topic not found. Wrong topic name in writer configuration.
- #define `APPGEN_LOG_DDS_ENTITY_EC` (`APPGEN_LOG_BASE` + 7)
Failed to create/delete/lookup DDS entity.
- #define `APPGEN_LOG_API_ERROR_EC` (`APPGEN_LOG_BASE` + 8)
A call to a DDS api returned an error.
- #define `APPGEN_LOG_APP_NAME_ERROR_EC` (`APPGEN_LOG_BASE` + 9)
Wrong application or participant name. Not a qualified name.
- #define `APPGEN_LOG_LIB_NOT_FOUND_EC` (`APPGEN_LOG_BASE` + 10)
Participant cannot be created because the library name is not found in the configuration.
- #define `APPGEN_APP_PARTICIPANT_NOT_FOUND_EC` (`APPGEN_LOG_BASE` + 11)
Participant cannot be created because the participant name is not found in the configuration.
- #define `APPGEN_LOG_APP_CONFIG_NOT_CORRECT_EC` (`APPGEN_LOG_BASE` + 12)
Application generation configuration is not correct or not consistent.
- #define `APPGEN_LOG_APP_CONFIG_POINTER_NOT_CORRECT_EC` (`APPGEN_LOG_BASE` + 13)
Application generation configuration is not correct or not consistent.
- #define `APPGEN_FACTORY_UNREGISTER_EC` (`APPGEN_LOG_BASE` + 14)
Cannot unregister a factory.

8.2 cdr_log.h File Reference

```
#include "osapi/osapi_log.h"
```

Macros

- #define `CDR_LOG_STREAM_ALLOC_EC` (`CDR_LOG_BASE` + 1)
Failed to allocate stream or stream buffer.
- #define `CDR_LOG_SET_OFFSET_EC` (`CDR_LOG_BASE` + 2)
Failed to set stream's current offset.
- #define `CDR_LOG_INCR_OFFSET_EC` (`CDR_LOG_BASE` + 3)
Failed to increment stream's current offset.
- #define `CDR_LOG_UNSUPPORTED_OPTIONS_EC` (`CDR_LOG_BASE` + 4)
Unsupported CDR encapsulations were received. The sample was dropped.

8.3 db_log.h File Reference

DB module log codes.

```
#include "osapi/osapi_log.h"
```

Macros

- #define [DB_LOG_SORTED_ALLOC_EC](#) (DB_LOG_BASE + 1)
Not sufficient memory to allocate sorted list for an index.
- #define [DB_LOG_NAME_TOO_LONG_EC](#) (DB_LOG_BASE + 2)
The specified database name is too long.
- #define [DB_LOG_ILLEGAL_TABLE_SIZE_EC](#) (DB_LOG_BASE + 3)
Illegal table size specified.
- #define [DB_LOG_ILLEGAL_LOCK_MODE_EC](#) (DB_LOG_BASE + 4)
Illegal combination of lock mode and mutex given.
- #define [DB_LOG_MUTEX_ALLOC_EC](#) (DB_LOG_BASE + 5)
Failed to allocate database mutex.
- #define [DB_LOG_ALLOC_TABLE_POOL_EC](#) (DB_LOG_BASE + 6)
Failed to allocate buffer pool.
- #define [DB_LOG_TABLES_INUSE_EC](#) (DB_LOG_BASE + 7)
Table is in use.
- #define [DB_LOG_TABLE_NAME_TOO_LONG_EC](#) (DB_LOG_BASE + 8)
Specified table name too long.
- #define [DB_LOG_ILLEGAL_RECORD_COUNT_EC](#) (DB_LOG_BASE + 9)
Illegal record count specified.
- #define [DB_LOG_TABLE_EXISTS_EC](#) (DB_LOG_BASE + 10)
Specified table already exists.
- #define [DB_LOG_OUT_OF_TABLES_EC](#) (DB_LOG_BASE + 11)
Table resources exceeded.
- #define [DB_LOG_OUT_OF_RECORDS_EC](#) (DB_LOG_BASE + 12)
Table record resources exceeded.
- #define [DB_LOG_OUT_OF_INDICES_EC](#) (DB_LOG_BASE + 13)
Table index resources exceeded.
- #define [DB_LOG_OUT_OF_CURSORS_EC](#) (DB_LOG_BASE + 14)
Table cursor resources exceeded.
- #define [DB_LOG_RECORDS_INUSE_EC](#) (DB_LOG_BASE + 15)
Cannot delete table, records are still in table.
- #define [DB_LOG_CURSORS_INUSE_EC](#) (DB_LOG_BASE + 16)
Cannot delete table, cursors are still in use.
- #define [DB_LOG_INDEX_INUSE_EC](#) (DB_LOG_BASE + 17)
Cannot delete table, indices are still in use.
- #define [DB_LOG_ALLOC_CURSOR_POOL_EC](#) (DB_LOG_BASE + 18)
Failed to allocate cursor buffer pool.
- #define [DB_LOG_ALLOC_INDEX_POOL_EC](#) (DB_LOG_BASE + 19)
Failed to allocate index buffer pool.

- #define `DB_LOG_ALLOC_RECORD_POOL_EC` (`DB_LOG_BASE` + 20)
Failed to allocate record buffer pool.
- #define `DB_LOG_ALLOC_DATABASE_EC` (`DB_LOG_BASE` + 21)
Failed to allocate database.
- #define `DB_LOG_TABLE_NOT_INUSE_EC` (`DB_LOG_BASE` + 22)
An attempt was made to delete a table not in use.
- #define `DB_LOG_RECORD_ALREADY_EXISTS_EC` (`DB_LOG_BASE` + 23)
An attempt was made to insert an already existing record.
- #define `DB_LOG_RECORD_DOES_NOT_EXIST_EC` (`DB_LOG_BASE` + 24)
An attempt was made to delete a record that does not exist.
- #define `DB_LOG_CURSOR_INVALIDATED_EC` (`DB_LOG_BASE` + 25)
An attempt was made to use a cursor that has been invalidated.
- #define `DB_LOG_LOCK_FAILURE_EC` (`DB_LOG_BASE` + 26)
Failed to lock the database.
- #define `DB_LOG_UNLOCK_FAILURE_EC` (`DB_LOG_BASE` + 27)
Failed to unlock the database.
- #define `DB_LOG_OUT_OF_CTORTDOR_EC` (`DB_LOG_BASE` + 28)
Constructor/Destructor memory could not be allocated.

8.3.1 Detailed Description

DB module log codes.

This file defines log codes for all messages logged by the database modules. All the log codes in this file are publicly documented.

8.4 dds_c_config.h File Reference

DDS C configuration file.

```
#include "netio/netio_config.h"
#include "osapi/osapi_config.h"
```

8.4.1 Detailed Description

DDS C configuration file.

8.5 dds_c_flowcontroller.h File Reference

DDS Flow Controller.

```
#include "netio/netio_flowcontroller.h"
#include "dds_c/dds_c_common.h"
#include "dds_c/dds_c_infrastructure.h"
```

Classes

- struct [DDS_FlowControllerTokenBucketProperty_t](#)
DDSFlowController uses the popular token bucket approach for open loop network flow control. The flow control characteristics are determined by the token bucket properties.
- struct [DDS_FlowControllerProperty_t](#)
Determines the flow control characteristics of the DDSFlowController.

Enumerations

- enum [DDS_FlowControllerSchedulingPolicy](#) { [DDS_RR_FLOW_CONTROLLER_SCHED_POLICY](#) , [DDS_EDF_FLOW_CONTROLLER_SCHED_POLICY](#) , [DDS_HPF_FLOW_CONTROLLER_SCHED_POLICY](#) }
Kinds of flow controller scheduling policy.

Variables

- const char *const [DDS_DEFAULT_FLOW_CONTROLLER_NAME](#)
[default] Special value of DDS_PublishModeQosPolicy::flow_controller_name that refers to the built-in default flow controller.
- const char *const [DDS_FIXED_RATE_FLOW_CONTROLLER_NAME](#)
Special value of DDS_PublishModeQosPolicy::flow_controller_name that refers to the built-in fixed-rate flow controller.
- const char *const [DDS_ON_DEMAND_FLOW_CONTROLLER_NAME](#)
Special value of DDS_PublishModeQosPolicy::flow_controller_name that refers to the built-in on-demand flow controller.
- const struct [DDS_FlowControllerProperty_t](#) [DDS_FLOW_CONTROLLER_PROPERTY_DEFAULT](#)
<<eXtension>> Special value for creating a DDSFlowController with default property.

8.5.1 Detailed Description

DDS Flow Controller.

8.6 dds_c_log.h File Reference

DDS_C module log codes.

```
#include "dds_c/dds_c_config.h"
#include "osapi/osapi_log.h"
```

Macros

- #define `DDSC_LOG_INVALID_DURATION_EC` (`DDSC_LOG_BASE` + 1)
The specified duration is not valid.
- #define `DDSC_LOG_INVALID_PARTICIPANT_NAME_EC` (`DDSC_LOG_BASE` + 2)
An invalid participant name was specified with an unknown GUID.
- #define `DDSC_LOG_INVALID_PARTICIPANT_GUID_PREFIX_EC` (`DDSC_LOG_BASE` + 3)
An invalid participant GUID prefix was specified, typically the GUID prefix does not match an already detected participant.
- #define `DDSC_LOG_SYS_GETTIME_EC` (`DDSC_LOG_BASE` + 4)
A call to OSAPI_System_get_time failed.
- #define `DDSC_LOG_GET_NEXT_OBJECT_ID_EC` (`DDSC_LOG_BASE` + 5)
Failed to get the next automatically generated object ID for an entity's GUID. This typically means the object id pool has been exhausted.
- #define `DDSC_LOG_SET_ENTITY_NAME_EC` (`DDSC_LOG_BASE` + 6)
Failed to set name string for a DDS entity in the DomainParticipantQos entity_name policy.
- #define `DDSC_LOG_SYS_GET_HOSTNAME_EC` (`DDSC_LOG_BASE` + 7)
A call to OSAPI_System_get_hostname failed.
- #define `DDSC_LOG_IO_SNPRINTF_FAILED_EC` (`DDSC_LOG_BASE` + 8)
A call to OSAPI_Stdio_snprintf failed. Typically this means the destination buffer was too small.
- #define `DDSC_LOG_TOPIC_FIND_EC` (`DDSC_LOG_BASE` + 9)
Failed to find a topic created by a DomainParticipant.
- #define `DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_NAME_EC` (`DDSC_LOG_BASE` + 10)
Endpoint discovery failed because the name of the remote participant parent for an endpoint was not found.
- #define `DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_KEY_EC` (`DDSC_LOG_BASE` + 11)
Endpoint discovery failed because the key of the remote participant parent for an endpoint was not found. Note that the key is logged as 4 integers in host endianness format.
- #define `DDSC_LOG_REMOTE_PARTICIPANT_KEY_NOT_EQUAL_EC` (`DDSC_LOG_BASE` + 12)
Failed endpoint discovery when key does not match the remote participant.
- #define `DDSC_LOG_INVALID_ENDPOINT_GUID_EC` (`DDSC_LOG_BASE` + 13)
Failed endpoint discovery due to an invalid or unknown endpoint GUID.
- #define `DDSC_LOG_ENDPOINT_NOT_CHILD_OF_PARTICIPANT_EC` (`DDSC_LOG_BASE` + 14)
Failed endpoint discovery when an endpoint is determined to belong to a different remote participant.
- #define `DDSC_LOG_PARTICIPANT_DOES_NOT_EXIST_EC` (`DDSC_LOG_BASE` + 15)
Failed participant discovery because a remote participant that should have been already asserted locally was not found.
- #define `DDSC_LOG_REFRESH_REM_PARTICIPANT_EC` (`DDSC_LOG_BASE` + 16)
Did not find a remote participant when asserting participant liveliness for it. Note that the key is logged as 4 integers in host endianness format.
- #define `DDSC_LOG_REFRESH_REM_PARTICIPANT_TIMEOUT_EC` (`DDSC_LOG_BASE` + 17)
Failed to assert participant liveliness to a remote participant.
- #define `DDSC_LOG_PARTICIPANT_LOOKUP_EC` (`DDSC_LOG_BASE` + 18)
Failed to find a remote participant as previously discovered.
- #define `DDSC_LOG_REMOVE_PUBLICATION_EC` (`DDSC_LOG_BASE` + 19)
Failed to remove resources for a remote publication.
- #define `DDSC_LOG_REMOVE_SUBSCRIPTION_EC` (`DDSC_LOG_BASE` + 20)
Failed to remove resources for a remote subscription.
- #define `DDSC_LOG_FIND_PUBLICATION_PARENT_EC` (`DDSC_LOG_BASE` + 21)
Cannot determine the participant of a remote publication.
- #define `DDSC_LOG_FIND_SUBSCRIPTION_PARENT_EC` (`DDSC_LOG_BASE` + 22)

- Cannot determine the participant of a remote subscription.*

 - #define `DDSC_LOG_MAX_PARTICIPANT_ID_REACHED_EC` (`DDSC_LOG_BASE` + 23)
Failed to create DomainParticipant due to running out of participant IDs. This is typically caused by all UDP ports being used.
 - #define `DDSC_LOG_RESERVE_LOCATORS_EC` (`DDSC_LOG_BASE` + 24)
Failed to reserve endpoint locators.
 - #define `DDSC_LOG_TIMER_CREATE_TIMEOUT_EC` (`DDSC_LOG_BASE` + 25)
Failed to create a timeout.
 - #define `DDSC_LOG_ILLEGAL_OBJECTID_EC` (`DDSC_LOG_BASE` + 26)
Illegal object id specified.
 - #define `DDSC_LOG_TOPIC_NARROW_EC` (`DDSC_LOG_BASE` + 27)
Failed to narrow a TopicDescription to the named Topic.
 - #define `DDSC_LOG_FAILED_UPDATE_STATUS_CONDITION_EC` (`DDSC_LOG_BASE` + 28)
Failed to update state of StatusCondition.
 - #define `DDSC_LOG_WS_REMOVE_COND_REFERENCE_EC` (`DDSC_LOG_BASE` + 29)
Failed to remove a condition reference from a waitset.
 - #define `DDSC_LOG_WS_ADD_COND_REFERENCE_EC` (`DDSC_LOG_BASE` + 30)
Failed to add a condition reference to a waitset.
 - #define `DDSC_LOG_RELEASE_META_MC_EC` (`DDSC_LOG_BASE` + 31)
Failed to release resources for multicast discovery locators.
 - #define `DDSC_LOG_RELEASE_META_UC_EC` (`DDSC_LOG_BASE` + 32)
Failed to release resources for unicast discovery locators.
 - #define `DDSC_LOG_RELEASE_USER_MC_EC` (`DDSC_LOG_BASE` + 33)
Failed to release resources for multicast user locators.
 - #define `DDSC_LOG_RELEASE_USER_UC_EC` (`DDSC_LOG_BASE` + 34)
Failed to release resources for unicast user locators.
 - #define `DDSC_LOG_INVALID_DOMAINID_EC` (`DDSC_LOG_BASE` + 35)
The domain ID specified exceeds what is allowed based on the parameters specified in `dp_qos.protocol.rtps_well_known_ports`.
 - #define `DDSC_LOG_ON_TYPE_REGISTERED_FAILURE_EC` (`DDSC_LOG_BASE` + 36)
Error occurred during the `on_type_registered` call back.
 - #define `DDSC_LOG_ON_TYPE_UNREGISTERED_FAILURE_EC` (`DDSC_LOG_BASE` + 37)
Error occurred during the `on_type_unregistered` call back.
 - #define `DDSC_LOG_GET_SERIALIZED_KEY_SIZE_EC` (`DDSC_LOG_BASE` + 38)
Error occurred during the `on_type_unregistered` call back.
 - #define `DDSC_LOG_MIN_DISCOVERY_MTU_EC` (`DDSC_LOG_BASE` + 39)
Minimum MTU across all Discovery transports must be greater or equal to a packet containing serialized Participant↔BuiltinTopicData.
 - #define `DDSC_LOG_MIN_MTU_SIZE_EC` (`DDSC_LOG_BASE` + 40)
MTU for a transport set lower or equal to Protocol Overhead of 448 bytes.
 - #define `DDSC_LOG_GET_MTU_NO_ROUTES_EC` (`DDSC_LOG_BASE` + 41)
MTU could not be found because of no routes.
 - #define `DDSC_LOG_RESOLVE_LOCATORS_PRIORITY_EC` (`DDSC_LOG_BASE` + 42)
Failed to resolve locators based on priority.
 - #define `DDSC_LOG_REMOTE_PUBLICATION_DATA_CHANGED_EC` (`DDSC_LOG_BASE` + 43)
The publication builtin topic data for a remote publication changed after the remote publication was asserted.
 - #define `DDSC_LOG_REMOTE_SUBSCRIPTION_DATA_CHANGED_EC` (`DDSC_LOG_BASE` + 44)

Fields in subscription builtin topic data for a remote subscription which are not supported to dynamically change were updated after the remote subscription was asserted.

- #define `DDSC_LOG_GET_MTU_EC` (`DDSC_LOG_BASE` + 45)
Error getting the mtu from the route resolver.
- #define `DDSC_LOG_DATABASE_CREATE_EC` (`DDSC_LOG_BASE` + 100)
Failed to create database.
- #define `DDSC_LOG_DATABASE_DELETE_EC` (`DDSC_LOG_BASE` + 101)
Failed to delete database.
- #define `DDSC_LOG_TABLE_CREATE_EC` (`DDSC_LOG_BASE` + 102)
Failed to create database table of the specified name.
- #define `DDSC_LOG_TABLE_INUSE_EC` (`DDSC_LOG_BASE` + 103)
Failed to delete a database table because it is not empty.
- #define `DDSC_LOG_TABLE_DELETE_EC` (`DDSC_LOG_BASE` + 104)
Failed to delete a database table.
- #define `DDSC_LOG_TABLE_SELECT_EC` (`DDSC_LOG_BASE` + 105)
A selection operation failed on the specified database table.
- #define `DDSC_LOG_CREATE_INDEX_EC` (`DDSC_LOG_BASE` + 106)
Failed to create an index on a database table.
- #define `DDSC_LOG_DELETE_INDEX_EC` (`DDSC_LOG_BASE` + 107)
Failed to delete an index on a database table.
- #define `DDSC_LOG_DB_CURSOR_INVALIDATED_EC` (`DDSC_LOG_BASE` + 108)
A database table cursor was invalidated while in use.
- #define `DDSC_LOG_SUBSCRIPTION_RECORD` 1
Database record for a Subscription.
- #define `DDSC_LOG_PUBLICATION_RECORD` 2
Database record for a Publication.
- #define `DDSC_LOG_PARTICIPANT_RECORD` 3
Database record for a Remote Participant.
- #define `DDSC_LOG_TOPIC_RECORD` 4
Database record for a Topic.
- #define `DDSC_LOG_DATAWRITER_RECORD` 5
Database record for a DataWriter.
- #define `DDSC_LOG_DATAREADER_RECORD` 6
Database record for a DataReader.
- #define `DDSC_LOG_PUBLISHER_RECORD` 7
Database record for a Publisher.
- #define `DDSC_LOG_SUBSCRIBER_RECORD` 8
Database record for a Subscriber.
- #define `DDSC_LOG_ROUTE_RECORD` 9
Database record for a route record.
- #define `DDSC_LOG_BIND_RECORD` 10
Database record for a bind record.
- #define `DDSC_LOG_TYPE_RECORD` 11
Database record for a type plugin.
- #define `DDSC_LOG_REMOTE_ENDPOINT_TRUST_STATE_RECORD` 12
Database record for a remote trust endpoint.
- #define `DDSC_LOG_MATCHED_DW_RECORD` 13

- *Database record for a matched remote data writer.*
- #define `DDSC_LOG_STRING_RECORD` 14
- *Database record for a string property.*
- #define `DDSC_LOG_RECORD_CREATE_EC` (`DDSC_LOG_BASE` + 109)
- *Failed to create a database record of the specified kind.*
- #define `DDSC_LOG_RECORD_DELETE_EC` (`DDSC_LOG_BASE` + 110)
- *Failed to delete a database record of the specified kind.*
- #define `DDSC_LOG_RECORD_INSERT_EC` (`DDSC_LOG_BASE` + 111)
- *Failed to insert a database record of the specified kind.*
- #define `DDSC_LOG_RECORD_ERROR_EC` (`DDSC_LOG_BASE` + 112)
- *Unknown error for database record of the specified kind.*
- #define `DDSC_LOG_RECORD_EXISTS_EC` (`DDSC_LOG_BASE` + 113)
- *A database record of the specified kind already exists.*
- #define `DDSC_LOG_RECORD_LOOKUP_EC` (`DDSC_LOG_BASE` + 114)
- *A lookup of a database record of the specified kind failed.*
- #define `DDSC_LOG_RECORD_NOT_EXISTS_EC` (`DDSC_LOG_BASE` + 115)
- *A database record of the specified kind does not exist.*
- #define `DDSC_LOG_RECORD_SELECT_EC` (`DDSC_LOG_BASE` + 116)
- *A database select on the specified record kind failed.*
- #define `DDSC_LOG_RECORD_REMOVE_EC` (`DDSC_LOG_BASE` + 117)
- *Removal a database record of the specified kind failed.*
- #define `DDSC_LOG_RECORD_INITIALIZE_EC` (`DDSC_LOG_BASE` + 118)
- *A database record of the specified kind could not be initialized.*
- #define `DDSC_LOG_RECORD_FINALIZE_EC` (`DDSC_LOG_BASE` + 119)
- *A database record of the specified kind could not be finalized.*
- #define `DDSC_LOG_RECORD_RESET_EC` (`DDSC_LOG_BASE` + 120)
- *A database record of the specified kind could not be reset.*
- #define `DDSC_LOG_PUBLICATIONDATA_OBJECT` 1
- *DDS PublicationBuiltinTopicData object.*
- #define `DDSC_LOG_SUBSCRIPTIONDATA_OBJECT` 2
- *DDS SubscriptionBuiltinTopicData object.*
- #define `DDSC_LOG_PARTICIPANTDATA_OBJECT` 3
- *DDS ParticipantBuiltinTopicData object.*
- #define `DDSC_LOG_DATAREADERQOS_OBJECT` 4
- *DDS DataReaderQos object.*
- #define `DDSC_LOG_DATAWRITERQOS_OBJECT` 5
- *DDS DataWriterQos object.*
- #define `DDSC_LOG_TOPICQOS_OBJECT` 6
- *DDS TopicQos object.*
- #define `DDSC_LOG_PUBLISHERQOS_OBJECT` 7
- *DDS PublisherQos object.*
- #define `DDSC_LOG_SUBSCRIBERQOS_OBJECT` 8
- *DDS SubscriberQos object.*
- #define `DDSC_LOG_PARTICIPANTQOS_OBJECT` 9
- *DDS DomainParticipantQos object.*
- #define `DDSC_LOG_ENTITY_OBJECT` 10
- *DDS Entity object.*

- #define [DDSC_LOG_PARTICIPANTFACTORYQOS_OBJECT](#) 11
DDS DomainParticipantFactoryQos object.
- #define [DDSC_LOG_TYPE_OBJECT](#) 12
A DDS Type object.
- #define [DDSC_LOG_BINDRESOLVER_OBJECT](#) 13
A NETIO BindResolver object.
- #define [DDSC_LOG_ROUTERESOLVER_OBJECT](#) 14
A NETIO RouteResolver object.
- #define [DDSC_LOG_ADDRESSRESOLVER_OBJECT](#) 15
A NETIO AddressResolver object.
- #define [DDSC_LOG_DATAWRITERIO_OBJECT](#) 16
A NETIO DataWriterInterface object.
- #define [DDSC_LOG_DATAREADERIO_OBJECT](#) 17
A NETIO DataReaderInterface object.
- #define [DDSC_LOG_LOG_OBJECT](#) 18
A OSAPI Log object.
- #define [DDSC_LOG_SYSTEM_OBJECT](#) 19
A OSAPI system object.
- #define [DDSC_LOG_MUTEX_OBJECT](#) 20
A OSAPI mutex object.
- #define [DDSC_LOG_CONDREF_OBJECT](#) 21
A DDS Waitset condition reference object.
- #define [DDSC_LOG_WSREF_OBJECT](#) 22
A DDS Condition waitset reference object.
- #define [DDSC_LOG_MD5STREAM_OBJECT](#) 23
A MD5 stream object.
- #define [DDSC_LOG_CONDITION_OBJECT](#) 24
A DDS Condition object.
- #define [DDSC_LOG_RT_OBJECT](#) 25
A RT Condition object.
- #define [DDSC_LOG_PARTICIPANT_POOL_OBJECT](#) 26
A Participant pool object.
- #define [DDSC_LOG_TIMER_OBJECT](#) 27
A OSAPI_Timer object.
- #define [DDSC_LOG_TIMEROUT_OBJECT](#) 28
A OSAPI_Timer timeout object.
- #define [DDSC_LOG_TOPIC_OBJECT](#) 29
A DDS Topic object.
- #define [DDSC_LOG_WAITSET_OBJECT](#) 30
A DDS WaitSet object.
- #define [DDSC_LOG_UUID_OBJECT](#) 31
A UUID object.
- #define [DDSC_LOG_MEMPOOL_OBJECT](#) 32
A REDA_MemPool object.
- #define [DDSC_LOG_PARTICIPANT_PROPERTIES](#) 33
A DomainParticipant's string properties sequence.
- #define [DDSC_LOG_PARTICIPANT_BINARY_PROPERTIES](#) 34

- *A DomainParticipant's binary properties sequence.*
- #define [DDSC_LOG_PARTICIPANT_BUILTIN_PUBLISHER](#) 35
- *A DomainParticipant's built-in Publisher.*
- #define [DDSC_LOG_PARTICIPANT_BUILTIN_SUBSCRIBER](#) 36
- *A DomainParticipant's built-in Subscriber.*
- #define [DDSC_LOG_PARTICIPANT_GENERIC_MESSAGE_OBJECT](#) 37
- *A built-in Inter-Participant Channel data type object.*
- #define [DDSC_LOG_DATA HOLDER_SEQ_OBJECT](#) 38
- *A Trust data holder sequence object.*
- #define [DDSC_LOG_PROPERTYQOSPOLICY_OBJECT](#) 39
- *A Property QoS policy object.*
- #define [DDSC_LOG_STRING_OBJECT](#) 40
- *A String object.*
- #define [DDSC_LOG_AUTHPOOL_OBJECT](#) 41
- *An authentication pool object.*
- #define [DDSC_LOG_BUFFER_OBJECT](#) 42
- *A Log buffer object.*
- #define [DDSC_LOG_STRINGMANAGER_OBJECT](#) 43
- *A String Manager object.*
- #define [DDSC_LOG_OBJECT_INITIALIZE_EC](#) (DDSC_LOG_BASE + 200)
- *Out of resources to initialize object of the specified kind.*
- #define [DDSC_LOG_OBJECT_ALLOCATE_EC](#) (DDSC_LOG_BASE + 201)
- *Out of resources to allocate an object of the specified kind.*
- #define [DDSC_LOG_OBJECT_FINALIZE_EC](#) (DDSC_LOG_BASE + 202)
- *Failed to finalize object of specified kind.*
- #define [DDSC_LOG_OBJECT_DELETE_EC](#) (DDSC_LOG_BASE + 203)
- *Failed to delete object of specified kind.*
- #define [DDSC_LOG_OBJECT_COPY_EC](#) (DDSC_LOG_BASE + 204)
- *Failed to copy object of specified kind.*
- #define [DDSC_LOG_OBJECT_REFCOUNT_EC](#) (DDSC_LOG_BASE + 205)
- *Failed to delete/finalize an object because other objects are referencing it.*
- #define [DDSC_LOG_OBJECT_GET_PROPERTY_EC](#) (DDSC_LOG_BASE + 206)
- *Failed to get the object properties.*
- #define [DDSC_LOG_OBJECT_SET_PROPERTY_EC](#) (DDSC_LOG_BASE + 207)
- *Failed to set the object properties.*
- #define [DDSC_LOG_OBJECT_EMPTY_EC](#) (DDSC_LOG_BASE + 208)
- *An object is empty, typically applies only to buffer-pool objects.*
- #define [DDSC_LOG_OBJECT_INUSECOUNT_EC](#) (DDSC_LOG_BASE + 209)
- *Failed to delete/finalize an object because other objects are using it.*
- #define [DDSC_LOG_OBJECT_CREATE_EC](#) (DDSC_LOG_BASE + 210)
- *Failed to create an object.*
- #define [DDSC_LOG_NETMASK_SEQUENCE](#) 1
- *A NETIO Netmask sequence.*
- #define [DDSC_LOG_ROUTE_SEQUENCE](#) 2
- *A NETIO Route sequence.*
- #define [DDSC_LOG_RESERVED_SEQUENCE](#) 3
- *A NETIO Reserved address sequence.*

- `#define DDSC_LOG_ENABLED_USER_TRANSPORT_SEQUENCE 4`
A DDS sequence of enabled user NETIO transports.
- `#define DDSC_LOG_ENABLED_TRANSPORT_SEQUENCE 5`
A DDS sequence of enabled transports.
- `#define DDSC_LOG_ENABLED_DISCOVERY_TRANSPORT_SEQUENCE 6`
A DDS sequence of enabled discovery transports.
- `#define DDSC_LOG_INITIAL_PEER_SEQUENCE 7`
A DDS sequence of initial peers.
- `#define DDSC_LOG_DESTINATION_SEQUENCE 8`
A NETIO sequence of destinations to send to.
- `#define DDSC_LOG_METAUNICAST_SEQUENCE 9`
A DDS sequence of meta-traffic unicast locators.
- `#define DDSC_LOG_METAMULTICAST_SEQUENCE 10`
A DDS sequence of meta-traffic multicast locators.
- `#define DDSC_LOG_USERUNICAST_SEQUENCE 11`
A DDS sequence of user-traffic unicast locators.
- `#define DDSC_LOG_USERMULTICAST_SEQUENCE 12`
A DDS sequence of user-traffic multicast locators.
- `#define DDSC_LOG_READTAKE_SEQUENCE 13`
A DDS sequence used in a read/take call.
- `#define DDSC_LOG_DATAHOLDER_SEQUENCE 14`
A DDS sequence of DataHolder objects.
- `#define DDSC_LOG_COOKIE_PAYLOAD_SEQUENCE 15`
A cookie octet sequence.
- `#define DDSC_LOG_REPRESENTATION_ID_SEQUENCE 16`
a sequence for data representation IDs
- `#define DDSC_LOG_SEQUENCE_SETMAX_EC (DDSC_LOG_BASE + 300)`
Failed to set the maximum length of a sequence of the specified kind.
- `#define DDSC_LOG_SEQUENCE_SETLENGTH_EC (DDSC_LOG_BASE + 301)`
Failed to set the length of a sequence of the specified kind.
- `#define DDSC_LOG_SEQUENCE_GETREF_EC (DDSC_LOG_BASE + 302)`
Failed to get a reference at the specified index for a sequence of the specified kind.
- `#define DDSC_LOG_SEQUENCE_INITIALIZE_EC (DDSC_LOG_BASE + 303)`
Failed to initialize a sequence of the specified kind.
- `#define DDSC_LOG_SEQUENCE_FINALIZE_EC (DDSC_LOG_BASE + 304)`
Failed to finalize a sequence of the specified kind.
- `#define DDSC_LOG_SEQUENCE_COPY_EC (DDSC_LOG_BASE + 305)`
Failed to copy a sequence of the specified kind.
- `#define DDSC_LOG_SEQUENCE_INVALID_EC (DDSC_LOG_BASE + 306)`
The sequence of the specified kind was invalid in the context it is used.
- `#define DDSC_LOG_DISCOVERY_COMPONENT 1`
A component of discovery kind.
- `#define DDSC_LOG_RTPS_COMPONENT 2`
A component of RTPS kind.
- `#define DDSC_LOG_READERHISTORY_COMPONENT 3`
A component of reader-history kind.
- `#define DDSC_LOG_WRITERHISTORY_COMPONENT 4`

- *A component of writer-history kind.*
- #define `DDSC_LOG_DATAREADERIO_COMPONENT` 5
- *A component of DataReaderInterface kind.*
- #define `DDSC_LOG_DATAWRITERIO_COMPONENT` 6
- *A component of DataWriterInterface kind.*
- #define `DDSC_LOG_NETIO_COMPONENT` 7
- *A component of NETIO kind.*
- #define `DDSC_LOG_TRANSPORT_COMPONENT` 8
- *A component of transport kind.*
- #define `DDSC_LOG_APPGEN_COMPONENT` 9
- *A component of application generation kind.*
- #define `DDSC_LOG_COMPONENT_LOOKUP_EC` (`DDSC_LOG_BASE` + 400)
- *Did not find a component factory with the given name in the registry.*
- #define `DDSC_LOG_COMPONENT_CREATE_EC` (`DDSC_LOG_BASE` + 401)
- *Could not create a component of the specified kind using the specified factory.*
- #define `DDSC_LOG_COMPONENT_DELETE_EC` (`DDSC_LOG_BASE` + 402)
- *Could not delete a component of the specified kind using the specified factory.*
- #define `DDSC_LOG_HISTORY_RESOURCE` 1
- *Exceeded resource limits for writer history queue.*
- #define `DDSC_LOG_SAMPLE_RESOURCES` 2
- *Failed to get resource for new sample due to resource limit.*
- #define `DDSC_LOG_PARTICIPANT_RESOURCES` 3
- *Failed to allocate a new participant.*
- #define `DDSC_LOG_RESOURCE_EXCEEDED_EC` (`DDSC_LOG_BASE` + 500)
- *Could not allocate a resource of the specified kind.*
- #define `DDSC_LOG_TOPIC_QOS` 1
- *The TopicQos kind.*
- #define `DDSC_LOG_DATAREADER_QOS` 2
- *The DataReaderQos kind.*
- #define `DDSC_LOG_DATAWRITER_QOS` 3
- *The DataWriterQos kind.*
- #define `DDSC_LOG_SUBSCRIBER_QOS` 4
- *The SubscriberQos kind.*
- #define `DDSC_LOG_PUBLISHER_QOS` 5
- *The PublisherQos kind.*
- #define `DDSC_LOG_PARTICIPANT_QOS` 6
- *The DomainParticipantQos kind.*
- #define `DDSC_LOG_PARTICIPANTFACTORY_QOS` 7
- *The DomainParticipantFactoryQos kind.*
- #define `DDSC_LOG_DEFAULTTOPIC_QOS` 8
- *The default TopicQos kind.*
- #define `DDSC_LOG_DEFAULTDATAREADER_QOS` 9
- *The default DataReaderQos kind.*
- #define `DDSC_LOG_DEFAULTDATAWRITER_QOS` 10
- *The default DataWriterQos kind.*
- #define `DDSC_LOG_DEFAULTSUBSCRIBER_QOS` 11
- *The default SubscriberQos kind.*

- #define [DDSC_LOG_DEFAULTPUBLISHER_QOS](#) 12
The default PublisherQos kind.
- #define [DDSC_LOG_DEFAULTPARTICIPANT_QOS](#) 13
The default PublisherQos kind.
- #define [DDSC_LOG_DEADLINE_QOS_POLICY](#) 14
The deadline qos policy kind.
- #define [DDSC_LOG_LIVELINESS_QOS_POLICY](#) 15
The liveliness qos policy kind.
- #define [DDSC_LOG_HISTORY_QOS_POLICY](#) 16
The history qos policy kind.
- #define [DDSC_LOG_RESOURCE_LIMIT_QOS_POLICY](#) 17
The resource limits qos policy kind.
- #define [DDSC_LOG_PROTOCOL_QOS_POLICY](#) 18
The protocol qos policy kind.
- #define [DDSC_LOG_TYPE_SUPPORT_QOS_POLICY](#) 19
The type-support qos policy kind.
- #define [DDSC_LOG_RELIABILITY_QOS_POLICY](#) 20
The reliability qos policy kind.
- #define [DDSC_LOG_DURABILITY_QOS_POLICY](#) 21
The durability qos policy kind.
- #define [DDSC_LOG_OWNERSHIP_QOS_POLICY](#) 22
The ownership qos policy kind.
- #define [DDSC_LOG_OWNERSHIP_STRENGTH_QOS_POLICY](#) 23
The ownership-strength qos policy kind.
- #define [DDSC_LOG_TRANSPORT_QOS_POLICY](#) 24
The transport qos policy kind.
- #define [DDSC_LOG_PARTICIPANT_ID_QOS_POLICY](#) 25
The participant id qos policy kind.
- #define [DDSC_LOG_HEARTBEATS_QOS_POLICY](#) 26
The heartbeat qos policy kind.
- #define [DDSC_LOG_DATAWRITER_RESOURCE_QOS_POLICY](#) 27
The DataWriterQos writer_resource_limits policy.
- #define [DDSC_LOG_DATAREADER_RESOURCE_QOS_POLICY](#) 28
The DataReaderQos reader_resource_limits policy.
- #define [DDSC_LOG_DESTINATION_ORDER_POLICY](#) 29
The DestinationOrderQos destination_order policy.
- #define [DDSC_LOG_PROPERTY_QOS_POLICY](#) 30
The PropertyQos policy.
- #define [DDSC_LOG_PUBLISH_MODE_QOS_POLICY](#) 31
The PublishModeQos policy.
- #define [DDSC_LOG_DATA_REPRESENTATION_QOS_POLICY](#) 33
The DataRepresentationQos policy.
- #define [DDSC_LOG_LATENCY_BUDGET_QOS_POLICY](#) 34
The LatencyBudgetQos policy.
- #define [DDSC_LOG_CONTENT_FILTER_QOS_POLICY](#) 35
The ContentFilterQos policy.
- #define [DDSC_LOG_QOS_INCONSISTENT_POLICY_EC](#) ([DDSC_LOG_BASE](#) + 501)

- An inconsistent Qos policy for the specified Qos kind was found.*

 - #define `DDSC_LOG_QOS_INCONSISTENT_POLICIES_EC` (`DDSC_LOG_BASE` + 502)
- Inconsistency between two Qos policies for the specified Qos kind was found.*

 - #define `DDSC_LOG_QOS_INCONSISTENT_EC` (`DDSC_LOG_BASE` + 503)
- Failed to create an entity or set a qos due to inconsistent policy.*

 - #define `DDSC_LOG_QOS_COPY_EC` (`DDSC_LOG_BASE` + 504)
- Failed to copy a Qos of the specified kind.*

 - #define `DDSC_LOG_QOS_INITIALIZE_EC` (`DDSC_LOG_BASE` + 505)
- Failed to initialize a Qos of the specified kind.*

 - #define `DDSC_LOG_QOS_FINALIZE_EC` (`DDSC_LOG_BASE` + 506)
- Failed to finalize a Qos of the specified kind.*

 - #define `DDSC_LOG_QOS_SET_EC` (`DDSC_LOG_BASE` + 507)
- Failed to set a Qos of the specified kind.*

 - #define `DDSC_LOG_QOS_SET_ON_ENABLED_EC` (`DDSC_LOG_BASE` + 508)
- Failed to set a Qos of the specified kind because the entity is already enabled.*

 - #define `DDSC_LOG_QOS_IMMUTABLE_EC` (`DDSC_LOG_BASE` + 509)
- Failed to set a Qos of the specified kind the immutable Qos policies have been changed.*

 - #define `DDSC_LOG_QOS_CHANGED_EC` (`DDSC_LOG_BASE` + 510)
- A discovered Qos changed (the entity already existed)*

 - #define `DDSC_LOG_QOS_GET_EC` (`DDSC_LOG_BASE` + 511)
- Failed to get a Qos of the specified kind.*

 - #define `DDSC_LOG_TOPIC_LISTENER` 1
- The topic listener kind.*

 - #define `DDSC_LOG_DATAREADER_LISTENER` 2
- The datareader listener kind.*

 - #define `DDSC_LOG_DATAWRITER_LISTENER` 3
- The datawriter listener kind.*

 - #define `DDSC_LOG_SUBSCRIBER_LISTENER` 4
- The subscriber listener kind.*

 - #define `DDSC_LOG_PUBLISHER_LISTENER` 5
- The publisher listener kind.*

 - #define `DDSC_LOG_PARTICIPANT_LISTENER` 6
- The participant listener kind.*

 - #define `DDSC_LOG_PARTICIPANTFACTORY_LISTENER` 7
- The participant-factory listener kind.*

 - #define `DDSC_LOG_LISTENER_INCONSISTENT_EC` (`DDSC_LOG_BASE` + 512)
- Failed to create an entity due to inconsistent listener and status mask.*

 - #define `DDSC_LOG_LISTENER_SET_EC` (`DDSC_LOG_BASE` + 513)
- Failed to set the listener of the specified kind.*

 - #define `DDSC_LOG_LISTENER_GET_EC` (`DDSC_LOG_BASE` + 514)
- Failed to get the listener of the specified kind.*

 - #define `DDSC_LOG_LISTENER_SET_ILLEGAL_NULL_EC` (`DDSC_LOG_BASE` + 515)
- Illegal combination of NULL listener and non-NONE status mask when setting a listener for an Entity.*

 - #define `DDSC_LOG_TOPIC_ENTITY` 1
- The topic entity kind.*

 - #define `DDSC_LOG_DATAWRITER_ENTITY` 2
- The datawriter entity kind.*

- #define [DDSC_LOG_DATAREADER_ENTITY](#) 3
The datareader entity kind.
- #define [DDSC_LOG_PUBLISHER_ENTITY](#) 4
The publisher entity kind.
- #define [DDSC_LOG_SUBSCRIBER_ENTITY](#) 5
The subscriber entity kind.
- #define [DDSC_LOG_PARTICIPANT_ENTITY](#) 6
The participant entity kind.
- #define [DDSC_LOG_PUBLICATION_ENTITY](#) 7
The publication entity kind.
- #define [DDSC_LOG_SUBSCRIPTION_ENTITY](#) 8
The subscription entity kind.
- #define [DDSC_LOG_ENTITY_ENABLE_EC](#) (DDSC_LOG_BASE + 600)
Failed to enable an entity of the specified kind.
- #define [DDSC_LOG_ENTITY_NOT_EMPTY_EC](#) (DDSC_LOG_BASE + 601)
Failed to an delete/finalize an entity of the specified kind because it is not empty.
- #define [DDSC_LOG_ENTITY_FINALIZE_EC](#) (DDSC_LOG_BASE + 602)
Failed to finalize an entity of the specified kind.
- #define [DDSC_LOG_ENTITY_INITIALIZE_EC](#) (DDSC_LOG_BASE + 603)
Failed to initialize an entity of the specified kind.
- #define [DDSC_LOG_ENTITY_NOT_ENABLED_EC](#) (DDSC_LOG_BASE + 604)
An operation was attempted on an entity that is not enabled.
- #define [DDSC_LOG_ENTITY_DIFFERENT_FACTORY_EC](#) (DDSC_LOG_BASE + 605)
Entities are in different factories.
- #define [DDSC_LOG_ENTITY_INVALID_PROPERTY_EC](#) (DDSC_LOG_BASE + 606)
An invalid property was specified for the entity.
- #define [DDSC_LOG_DATAWRITER_CDR](#) 1
The datawriter CDR kind.
- #define [DDSC_LOG_DATAREADER_CDR](#) 2
The datareader CDR kind.
- #define [DDSC_LOG_DATAWRITER_INLINE](#) 3
The datawriter inline kind.
- #define [DDSC_LOG_CDR_POOL_ALLOC_EC](#) (DDSC_LOG_BASE + 700)
Failed to allocate a pool of the specified kind.
- #define [DDSC_LOG_CDR_BUFFER_SET_EC](#) (DDSC_LOG_BASE + 701)
Failed to set the CDR buffer for a packet.
- #define [DDSC_LOG_CDR_POOL_DELETE_EC](#) (DDSC_LOG_BASE + 702)
Failed to delete the CDR pool.
- #define [DDSC_LOG_CDR_SERIALIZE_PID_EC](#) (DDSC_LOG_BASE + 703)
Failed to serialize a parameter ID.
- #define [DDSC_LOG_CDR_SERIALIZE_PID_LENGTH_EC](#) (DDSC_LOG_BASE + 704)
Failed to serialize a parameter length.
- #define [DDSC_LOG_CDR_SERIALIZE_KEYHASH_EC](#) (DDSC_LOG_BASE + 705)
Failed to serialize a key-hash.
- #define [DDSC_LOG_CDR_SERIALIZE_DATA_EC](#) (DDSC_LOG_BASE + 706)
Failed to serialize payload data.
- #define [DDSC_LOG_DESERIALIZE_BAD_PID_LENGTH_EC](#) (DDSC_LOG_BASE + 707)

- Deserialized an invalid parameter length for a specific parameter ID.*

 - #define `DDSC_LOG_CDR_DESERIALIZE_PID_EC` (`DDSC_LOG_BASE` + 708)
- Failed to deserialize the ID of an inline parameter.*

 - #define `DDSC_LOG_CDR_DESERIALIZE_PID_LENGTH_EC` (`DDSC_LOG_BASE` + 709)
- Failed to deserialize the length of an inline parameter.*

 - #define `DDSC_LOG_CDR_INCREMENT_POS_EC` (`DDSC_LOG_BASE` + 710)
- Failed to increment to the position of the next inline parameter.*

 - #define `DDSC_LOG_CDR_SET_POS_EC` (`DDSC_LOG_BASE` + 711)
- Failed to set the reception stream position.*

 - #define `DDSC_LOG_CDR_DESERIALIZE_HEADER_EC` (`DDSC_LOG_BASE` + 712)
- Failed to deserialize the encapsulation header.*

 - #define `DDSC_LOG_CDR_DESERIALIZE_DATA_EC` (`DDSC_LOG_BASE` + 713)
- Failed to deserialize CDR payload data.*

 - #define `DDSC_LOG_CDR_INITIALIZE_SAMPLE_EC` (`DDSC_LOG_BASE` + 714)
- Failed to initialize CDR sample.*

 - #define `DDSC_LOG_CDR_FINALIZE_SAMPLE_EC` (`DDSC_LOG_BASE` + 715)
- Failed to finalize sample.*

 - #define `DDSC_LOG_CDR_SERIALIZE_STATUS_INFO_EC` (`DDSC_LOG_BASE` + 716)
- Failed to serialize the status info parameter.*

 - #define `DDSC_LOG_CDR_DESERIALIZE_KEYHASH_EC` (`DDSC_LOG_BASE` + 717)
- Failed to deserialize a key-hash.*

 - #define `DDSC_LOG_CDR_DESERIALIZE_KEY_EC` (`DDSC_LOG_BASE` + 718)
- Failed to deserialize CDR payload key.*

 - #define `DDSC_LOG_NETIO_ADD_ANON_TOPIC_ROUTE_EC` (`DDSC_LOG_BASE` + 800)
- Failed to add a route to an anonymous participant discovery datawriter.*

 - #define `DDSC_LOG_NETIO_ADD_TOPIC_ROUTE_EC` (`DDSC_LOG_BASE` + 801)
- Failed to add a route to a topic from a datawriter.*

 - #define `DDSC_LOG_NETIO_DELETE_TOPIC_ROUTE_EC` (`DDSC_LOG_BASE` + 802)
- Failed to delete a route to a topic.*

 - #define `DDSC_LOG_NETIO_FORWARD_TOPIC_EC` (`DDSC_LOG_BASE` + 803)
- Failed to forward a topic.*

 - #define `DDSC_LOG_NETIO_NETIO_KIND` 1
- Any NETIO interface kind, typically UDP.*

 - #define `DDSC_LOG_INTRA_NETIO_KIND` 2
- The intra interface kind.*

 - #define `DDSC_LOG_RTPS_NETIO_KIND` 3
- The RTPS interface kind.*

 - #define `DDSC_LOG_DATAREADER_NETIO_KIND` 4
- The DataReader interface kind.*

 - #define `DDSC_LOG_DATAWRITER_NETIO_KIND` 5
- The DataWriter interface kind.*

 - #define `DDSC_LOG_NETIO_BIND_EXTERNAL_EC` (`DDSC_LOG_BASE` + 804)
- Failed to bind two external interface of the specified kind.*

 - #define `DDSC_LOG_NETIO_UNBIND_EXTERNAL_EC` (`DDSC_LOG_BASE` + 805)
- Failed to unbind two external interfaces of the specified kind.*

 - #define `DDSC_LOG_NETIO_BIND_EC` (`DDSC_LOG_BASE` + 806)
- Failed to bind an interface to a peer interface.*

- #define `DDSC_LOG_NETIO_UNBIND_EC` (`DDSC_LOG_BASE` + 807)
Failed to unbind an interface from a peer interface.
- #define `DDSC_LOG_NETIO_ADD_ROUTE_EC` (`DDSC_LOG_BASE` + 808)
Failed to add a route from an interface to a peer interface.
- #define `DDSC_LOG_NETIO_DELETE_ROUTE_EC` (`DDSC_LOG_BASE` + 809)
Failed to delete a route from an interface to a peer interface.
- #define `DDSC_LOG_NETIO_GET_EXTERNAL_INTF_EC` (`DDSC_LOG_BASE` + 810)
Failed to get an external interface for the specified interface kind.
- #define `DDSC_LOG_NETIO_NO_ROUTE_EC` (`DDSC_LOG_BASE` + 811)
A DataReader failed a bind due to no existing route.
- #define `DDSC_LOG_NETIO_ROUTE_LOOKUP_FAILED_EC` (`DDSC_LOG_BASE` + 812)
Lookup a route to a destination failed.
- #define `DDSC_LOG_NETIO_GET_ROUTE_TABLE_FAILED_EC` (`DDSC_LOG_BASE` + 813)
Failed to get the route table for an interface.
- #define `DDSC_LOG_NETIO_SEND_FAILED_EC` (`DDSC_LOG_BASE` + 814)
Failure when sending on an interface.
- #define `DDSC_LOG_NETIO_SET_STATE_EC` (`DDSC_LOG_BASE` + 815)
Failed to set an interface state.
- #define `DDSC_LOG_NETIO_PEER_LOOKUP_EC` (`DDSC_LOG_BASE` + 816)
Datawriter did not find a peer.
- #define `DDSC_LOG_NETIO_FORCED_REMOVE_EC` (`DDSC_LOG_BASE` + 817)
Forced removal of sample downstream failed.
- #define `DDSC_LOG_PACKET_INIT_EC` (`DDSC_LOG_BASE` + 818)
Failed to initialize a packet.
- #define `DDSC_LOG_PACKET_SET_HEAD_EC` (`DDSC_LOG_BASE` + 819)
Failed to set the head of a packet.
- #define `DDSC_LOG_PACKET_SET_TAIL_EC` (`DDSC_LOG_BASE` + 820)
Failed to set the tail of a packet.
- #define `DDSC_LOG_NETIO_DISCOVERY_ENABLED_TRANSPORT_EC` (`DDSC_LOG_BASE` + 821)
Configured discovery enabled transport is not valid.
- #define `DDSC_LOG_NETIO_PACKETPOOL_RETURN_EC` (`DDSC_LOG_BASE` + 822)
Failed to return a packet to the packet pool.
- #define `DDSC_LOG_NETIO_PACKETPOOL_GET_EC` (`DDSC_LOG_BASE` + 823)
Failed to get a packet from the packet pool.
- #define `DDSC_LOG_NETIO_PACKETPOOL_CREATE_EC` (`DDSC_LOG_BASE` + 824)
Failed to create packet pool.
- #define `DDSC_LOG_NETIO_PACKETPOOL_DELETE_EC` (`DDSC_LOG_BASE` + 825)
Failed to delete packet pool.
- #define `DDSC_LOG_DW_ACKNACK_FAILED_EC` (`DDSC_LOG_BASE` + 900)
Failed to ACKNACK sample in the writer history.
- #define `DDSC_LOG_DW_COMMIT_EC` (`DDSC_LOG_BASE` + 901)
Failed to commit a sample to the writer queue.
- #define `DDSC_LOG_DW_KEYHASH_CREATE_EC` (`DDSC_LOG_BASE` + 902)
Failed to create keyhash of instance handle.
- #define `DDSC_LOG_DW_ILLEGAL_KEY_KIND_EC` (`DDSC_LOG_BASE` + 903)
Failed a write due to an invalid key kind for the type being written.
- #define `DDSC_LOG_DW_CREATE_TYPED_WRITER_EC` (`DDSC_LOG_BASE` + 904)

- Failed to create a typed writer.*

 - #define `DDSC_LOG_DW_HISTORY_REGISTER_KEY_EC` (`DDSC_LOG_BASE` + 905)
- Failed to register the key of an instance.*

 - #define `DDSC_LOG_DW_UNKNOWN_FLOW_CONTROLLER_EC` (`DDSC_LOG_BASE` + 906)
- Failed to create a flow-controller.*

 - #define `DDSC_LOG_DW_FLOW_CONTROLLER_REQUIRED_EC` (`DDSC_LOG_BASE` + 907)
- The data-sample is larger then supported by the transport, but the flow-controller has been compiled out.*

 - #define `DDSC_LOG_DW_INSTANCE_ASSERTION_FAILED_EC` (`DDSC_LOG_BASE` + 908)
- Failed to reserve an instance to publish discovery data for a user created data writer. This typically means DomainParticipantQos.resource_limits.remote_writer_allocation is too small.*

 - #define `DDSC_LOG_DW_INCOMPATIBLE_PADDING_EC` (`DDSC_LOG_BASE` + 909)
- A DataReader incompatible with padding bits in encapsulation header was discovered.*

 - #define `DDSC_LOG_DR_CREATE_TYPED_READER_EC` (`DDSC_LOG_BASE` + 1000)
- Failed to create a typed datareader.*

 - #define `DDSC_LOG_DR_COPY_DATA_SAMPLE_EC` (`DDSC_LOG_BASE` + 1001)
- Failed to copy a sample upon reception, read, or take.*

 - #define `DDSC_LOG_DR_COMMIT_SAMPLE_EC` (`DDSC_LOG_BASE` + 1002)
- Failed to commit a sample to be made available to be read or taken.*

 - #define `DDSC_LOG_DR_FILTER_ERROR_EC` (`DDSC_LOG_BASE` + 1003)
- A datareader filter function failed.*

 - #define `DDSC_LOG_DR_DESERIALIZE_KEYHASH_EC` (`DDSC_LOG_BASE` + 1004)
- Failed to deserialize a key-hash parameter.*

 - #define `DDSC_LOG_DR_GET_ENTRY_FAILED_EC` (`DDSC_LOG_BASE` + 1005)
- Failed to get a Reader History entry for a received sample.*

 - #define `DDSC_LOG_DR_COMMIT_ENTRY_EC` (`DDSC_LOG_BASE` + 1006)
- Failed to commit a receive sample to Reader History to be read or taken.*

 - #define `DDSC_LOG_DR_UNREGISTER_KEY_EC` (`DDSC_LOG_BASE` + 1007)
- A DataReader failed to unregister an instance.*

 - #define `DDSC_LOG_DR_DISPOSE_KEY_EC` (`DDSC_LOG_BASE` + 1008)
- A DataReader failed to dispose an instance.*

 - #define `DDSC_LOG_DR_READ_TAKE_FAILURE_EC` (`DDSC_LOG_BASE` + 1009)
- A call to a reader/take function failed.*

 - #define `DDSC_LOG_DR_INSTANCE_TO_KEYHASH_EC` (`DDSC_LOG_BASE` + 1010)
- Failed to create keyhash of instance handle.*

 - #define `DDSC_LOG_DR_INSTANCE_MAPPING_EXHAUSTED_EC` (`DDSC_LOG_BASE` + 1011)
- Failed to create keyhash of instance handle.*

 - #define `DDSC_LOG_DR_INSTANCE_ASSERTION_FAILED_EC` (`DDSC_LOG_BASE` + 1012)
- Failed to reserve an instance to publish discovery data for a user created data reader. This typically means DomainParticipantQos.resource_limits.remote_reader_allocation is too small.*

 - #define `DDSC_LOG_DR_ON_SAMPLE_REMOVED_EC` (`DDSC_LOG_BASE` + 1013)
- Failed to remove a sample when it was being pushed out from the Reader History.*

 - #define `DDSC_LOG_TYPE_NAME_CMP_EC` (`DDSC_LOG_BASE` + 1100)
- Two type names are incompatible.*

 - #define `DDSC_LOG_TOPIC_NAME_CMP_EC` (`DDSC_LOG_BASE` + 1101)
- Two topic names are incompatible.*

 - #define `DDSC_LOG_TYPE_FUNCTION_GET_SERIALIZED_SAMPLE_MAX_SIZE` 1
- The get_serialized_sample_max_size function kind.*

- `#define DDSC_LOG_TYPE_FUNCTION_SERIALIZE_DATA 2`
The serialize_data function kind.
- `#define DDSC_LOG_TYPE_FUNCTION_DESERIALIZE_DATA 3`
The deserialize_data function kind.
- `#define DDSC_LOG_TYPE_FUNCTION_CREATE_SAMPLE 4`
The create_sample function kind.
- `#define DDSC_LOG_TYPE_FUNCTION_COPY_SAMPLE 5`
The copy_sample function kind.
- `#define DDSC_LOG_TYPE_FUNCTION_DELETE_SAMPLE 6`
The delete_sample function kind.
- `#define DDSC_LOG_TYPE_FUNCTION_GET_KEY_KIND 7`
The get_key_kind function kind.
- `#define DDSC_LOG_TYPE_FUNCTION_INSTANCE_TO_KEYHASH 8`
The instance_to_keyhash function kind.
- `#define DDSC_LOG_TYPE_FUNCTION_NULL_EC (DDSC_LOG_BASE + 1102)`
Invalid type plugin, The specified function pointer is NULL.
- `#define DDSC_LOG_TOPIC_TOO_LONG_EC (DDSC_LOG_BASE + 1103)`
Failed to create a topic because the name exceeded the maximum length of 255 octets (excluding the terminating NUL)
- `#define DDSC_LOG_TYPE_TOO_LONG_EC (DDSC_LOG_BASE + 1104)`
Failed to create a type because the name exceeded the maximum length of 255 octets (excluding the terminating NUL)
- `#define DDSC_LOG_LOOKUP_TYPE_PLUGIN_EC (DDSC_LOG_BASE + 1105)`
A type-plugin for the given type could not be found.
- `#define DDSC_LOG_TYPE_KEY_TYPE_EC (DDSC_LOG_BASE + 1106)`
Two types have incompatible keys.
- `#define DDSC_LOG_TYPE_PLUGIN_GET_BUFFER_EC (DDSC_LOG_BASE + 1107)`
Type-plugin cannot return a buffer.
- `#define DDSC_LOG_TYPE_PLUGIN_INCONSISTENT_DR_EC (DDSC_LOG_BASE + 1108)`
Inconsistent type-plugin DataReader.
- `#define DDSC_LOG_DUPLICATE_GUID_PREFIX_EC (DDSC_LOG_BASE + 1109)`
A DomainParticipant with the specified GUID already exists in the DomainParticipant factory.
- `#define DDSC_LOG_TYPE_MANAGED_POOL_FAILURE_EC (DDSC_LOG_BASE + 1110)`
Managed pool function failed.
- `#define DDSC_LOG_TYPE_GET_SAMPLE_ID_EC (DDSC_LOG_BASE + 1111)`
Managed pool failed to get sample id.
- `#define DDSC_LOG_PARTICIPANT_NAME_EMPTY_EC (DDSC_LOG_BASE + 1112)`
A DomainParticipant with enable_participant_discovery_by_name set to to TRUE but an empty name is created.
- `#define DDSC_LOG_DUPLICATE_PARTICIPANT_NAME_EC (DDSC_LOG_BASE + 1113)`
A DomainParticipant with enable_participant_discovery_by_name set to to TRUE, but a local participant with the same DomainParticipantQos.participant_name.name already exists.
- `#define DDSC_LOG_DUPLICATE_REMOTE_PARTICIPANT_NAME_EC (DDSC_LOG_BASE + 1114)`
A remote participant is discovered with the same name as the a local participant name in the same domain ID.
- `#define DDSC_LOG_REMOTE_PARTICIPANT_NAME_EMPTY_EC (DDSC_LOG_BASE + 1115)`
A remote participant without a name is discovered.
- `#define DDSC_LOG_REMOTE_PARTICIPANT_RESTART_EC (DDSC_LOG_BASE + 1116)`
Failed to reset a remote participant.
- `#define DDSC_LOG_DISC_LOCAL_PARTICIPANT_ENABLED_EC (DDSC_LOG_BASE + 1200)`
Discovery plugin failed its update after a local DomainParticipant was enabled.

- #define `DDSC_LOG_DISC_BEFORE_LOCAL_PARTICIPANT_CREATED_EC` (`DDSC_LOG_BASE` + 1201)
Discovery plugin failed its update before a local DomainParticipant was created.
- #define `DDSC_LOG_DISC_AFTER_LOCAL_PARTICIPANT_CREATED_EC` (`DDSC_LOG_BASE` + 1202)
Discovery plugin failed its update after a local DomainParticipant was created.
- #define `DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_ENABLED_EC` (`DDSC_LOG_BASE` + 1203)
Discovery plugin failed its update after a local DataReader was enabled.
- #define `DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_DELETED_EC` (`DDSC_LOG_BASE` + 1204)
Discovery plugin failed its update after a local DataReader was deleted.
- #define `DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_ENABLED_EC` (`DDSC_LOG_BASE` + 1205)
Discovery plugin failed its update after a local DataWriter was enabled.
- #define `DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_DELETED_EC` (`DDSC_LOG_BASE` + 1206)
Discovery plugin failed its update after a local DataWriter was deleted.
- #define `DDSC_LOG_DISC_BEFORE_REMOTE_PARTICIPANT_DELETED_EC` (`DDSC_LOG_BASE` + 1207)
Discovery plugin failed its update after being notified that a remote DomainParticipant was about to be deleted.
- #define `DDSC_LOG_DISC_ADD_PEER_EC` (`DDSC_LOG_BASE` + 1208)
Failed to add a peer with discovery plugin.
- #define `DDSC_LOG_DESERIALIZE_UNKNOWN_PID_EC` (`DDSC_LOG_BASE` + 1209)
Deserialized a parameter of unknown ID.
- #define `DDSC_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC` (`DDSC_LOG_BASE` + 1210)
Failed to serialize Builtin Endpoint Mask parameter.
- #define `DDSC_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC` (`DDSC_LOG_BASE` + 1211)
Failed to deserialize Builtin Endpoint Mask parameter.
- #define `DDSC_LOG_SERIALIZE_TOPIC_NAME_EC` (`DDSC_LOG_BASE` + 1212)
Failed to serialize Topic Name parameter.
- #define `DDSC_LOG_CDR_SET_OFFSET_EC` (`DDSC_LOG_BASE` + 1213)
Failed to set current offset of stream.
- #define `DDSC_LOG_SERIALIZE_ENTITY_NAME_EC` (`DDSC_LOG_BASE` + 1214)
Failed to serialize Entity Name parameter.
- #define `DDSC_LOG_SERIALIZE_TYPE_NAME_EC` (`DDSC_LOG_BASE` + 1215)
Failed to serialize Type Name parameter.
- #define `DDSC_LOG_SERIALIZE_GUID_EC` (`DDSC_LOG_BASE` + 1216)
Failed to serialize GUID key.
- #define `DDSC_LOG_SERIALIZE_DEFAULT_UNICAST_EC` (`DDSC_LOG_BASE` + 1217)
Failed to serialize Default Unicast Locator parameter.
- #define `DDSC_LOG_DESERIALIZE_LOCATOR_EC` (`DDSC_LOG_BASE` + 1218)
Failed to deserialize a locator of the specified kind.
- #define `DDSC_LOG_LOCATORS_FULL_EC` (`DDSC_LOG_BASE` + 1219)
A locator sequence is full, the remaining locators of the specified kind are dropped.
- #define `DDSC_LOG_UNSUPPORTED_LOCATOR_EC` (`DDSC_LOG_BASE` + 1220)
The locator kind is not supported by any transport registered with the domain participant and is dropped.
- #define `DDSC_LOG_SERIALIZE_PROTOCOL_VERSION_EC` (`DDSC_LOG_BASE` + 1221)
Failed to serialize Protocol Version parameter.
- #define `DDSC_LOG_DESERIALIZE_PROTOCOL_VERSION_EC` (`DDSC_LOG_BASE` + 1222)
Failed to deserialize Protocol Version parameter.
- #define `DDSC_LOG_DESERIALIZE_VENDOR_ID_EC` (`DDSC_LOG_BASE` + 1223)
Failed to deserialize Vendor ID parameter.
- #define `DDSC_LOG_SERIALIZE_VENDOR_ID_EC` (`DDSC_LOG_BASE` + 1224)

- Failed to serialize Vendor ID parameter.*

 - #define `DDSC_LOG_SERIALIZE_PRODUCT_VERSION_EC` (`DDSC_LOG_BASE` + 1225)
- Failed to serialize Product Version parameter.*

 - #define `DDSC_LOG_SERIALIZE_LEASE_DURATION_EC` (`DDSC_LOG_BASE` + 1226)
- Failed to serialize Lease Duration parameter.*

 - #define `DDSC_LOG_DATAWRITER_ADD_PEER_FAILED_EC` (`DDSC_LOG_BASE` + 1227)
- Failed to add a remote peer to a local DataWriter.*

 - #define `DDSC_LOG_DATAREADER_ADD_PEER_FAILED_EC` (`DDSC_LOG_BASE` + 1228)
- Failed to add a remote peer to a local DataReader.*

 - #define `DDSC_LOG_DISC_REMOTE_PARTICIPANT_REMOVE_EC` (`DDSC_LOG_BASE` + 1229)
- Failed to remote/reset remote participant after liveliness expired.*

 - #define `DDSC_LOG_DESERIALIZE_PARTITION_SEQ_EC` (`DDSC_LOG_BASE` + 1230)
- Failed to deserialize partition seq.*

 - #define `DDSC_LOG_INITIALIZE_DISCOVERY_QUEUE_EC` (`DDSC_LOG_BASE` + 1231)
- Failed to initialize discovery queue.*

 - #define `DDSC_LOG_QUEUE_PUBLICATION_DISCOVERY_EC` (`DDSC_LOG_BASE` + 1232)
- Failed to queue a publication discovery message.*

 - #define `DDSC_LOG_QUEUE_SUBSCRIPTION_DISCOVERY_EC` (`DDSC_LOG_BASE` + 1233)
- Failed to queue a subscription discovery message.*

 - #define `DDSC_LOG_QUEUE_FINALIZE_EC` (`DDSC_LOG_BASE` + 1234)
- Failed finalize discovery queue.*

 - #define `DDSC_LOG_IPC_INIT_FAILED_EC` (`DDSC_LOG_BASE` + 1300)
- Failed to initialize Builtin IPC entities.*

 - #define `DDSC_LOG_IPC_ALLOC_FAILED_EC` (`DDSC_LOG_BASE` + 1301)
- Failed to allocate memory for one of the Builtin IPC entities.*

 - #define `DDSC_LOG_IPC_FINALIZE_FAILED_EC` (`DDSC_LOG_BASE` + 1302)
- Failed to finalize all resources A potential memory leak might have been caused by unexpected errors during finalization of acquired resources.*

 - #define `DDSC_LOG_IPC_CHANNEL_TYPE_REGISTER_EC` (`DDSC_LOG_BASE` + 1303)
- Failed to register type for a Builtin IPC channel.*

 - #define `DDSC_LOG_IPC_CHANNEL_TOPIC_CREATE_EC` (`DDSC_LOG_BASE` + 1304)
- Failed to create topic for a Builtin IPC channel.*

 - #define `DDSC_LOG_IPC_CHANNEL_READER_CREATE_EC` (`DDSC_LOG_BASE` + 1305)
- Failed to create reader for a Builtin IPC channel.*

 - #define `DDSC_LOG_IPC_CHANNEL_WRITER_CREATE_EC` (`DDSC_LOG_BASE` + 1306)
- Failed to create writer for a Builtin IPC channel.*

 - #define `DDSC_LOG_IPC_CHANNEL_TYPE_PLUGIN_EC` (`DDSC_LOG_BASE` + 1307)
- Failed to find type plugin for a Builtin IPC channel.*

 - #define `DDSC_LOG_IPC_CHANNEL_UNKNOWN_EC` (`DDSC_LOG_BASE` + 1308)
- Unknown builtin IPC channel requested.*

 - #define `DDSC_LOG_IPC_UPDATE_QOS_FAILED_EC` (`DDSC_LOG_BASE` + 1309)
- Failed to update DomainParticipantQos to account for built-in IPC endpoints.*

 - #define `DDSC_LOG_IPC_CHANNEL_ADD_ANON_ROUTE_EC` (`DDSC_LOG_BASE` + 1310)
- Failed to add an anonymous route to a built-in DataWriter or DataReader.*

 - #define `DDSC_LOG_IPC_CHANNEL_DELETE_ANON_ROUTE_EC` (`DDSC_LOG_BASE` + 1311)
- Failed to delete an anonymous route to a built-in DataWriter or DataReader.*

 - #define `DDSC_LOG_IPC_CHANNEL_ADD_PEER_EC` (`DDSC_LOG_BASE` + 1312)

- Failed to add an inter-participant channel peer.*

 - #define `DDSC_LOG_IPC_CHANNEL_REMOVE_PEER_EC` (`DDSC_LOG_BASE` + 1313)
- Failed to remove an inter-participant channel peer.*

 - #define `DDSC_LOG_IPC_CHANNEL_RETURN_LOAN_EC` (`DDSC_LOG_BASE` + 1314)
- Failed to return a loaned inter-participant sample.*

 - #define `DDSC_LOG_IPC_CHANNEL_PROCESS_SAMPLE_EC` (`DDSC_LOG_BASE` + 1315)
- Failed to process inter-participant message.*

 - #define `DDS_LOG_IPC_ASSERT_ROUTES_EC` (`DDSC_LOG_BASE` + 1316)
- Failed to assert inter-participant routes.*

 - #define `DDSC_LOG_IPC_CHANNEL_WRITE_SAMPLE_EC` (`DDSC_LOG_BASE` + 1317)
- Failed to write inter-participant message.*

 - #define `DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_AUTH_HANDSHAKE_FAILED_EC` (`DDSC_LOG_BASE` + 1318)
- Failed to process handshake authentication.*

 - #define `DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_PARTICIPANT_INTERCEPTOR_STATE_FAILED_EC` (`DDSC_LOG_BASE` + 1319)
- Failed to process participant interceptor tokens.*

 - #define `DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_ENDPOINT_INTERCEPTOR_STATE_FAILED_EC` (`DDSC_LOG_BASE` + 1320)
- Failed to process endpoint interceptor tokens.*

 - #define `DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_AUTH_REQUEST_FAILED_EC` (`DDSC_LOG_BASE` + 1321)
- Failed to process authentication request.*

 - #define `DDSC_LOG_IPC_CHANNEL_CONFIG_INITIALIZE_FAILED_EC` (`DDSC_LOG_BASE` + 1322)
- Failed to initialize inter-participant channel.*

 - #define `DDSC_LOG_IPC_CHANNEL_CONFIG_FINALIZE_FAILED_EC` (`DDSC_LOG_BASE` + 1323)
- Failed to set inter-participant channel configuration.*

 - #define `DDSC_LOG_IPC_CHANNEL_CONFIG_SET_FAILED_EC` (`DDSC_LOG_BASE` + 1324)
- Failed to get inter-participant channel configuration.*

 - #define `DDSC_LOG_IPC_CHANNEL_SET_LISTENERS_FAILED_EC` (`DDSC_LOG_BASE` + 1325)
- Failed to set inter-participant channel listeners.*

 - #define `DDSC_LOG_IPC_CHANNEL_ENABLE_FAILED_EC` (`DDSC_LOG_BASE` + 1326)
- Failed to enable inter-participant channel entities.*

 - #define `DDSC_LOG_IPC_CHANNEL_CREATE_FAILED_EC` (`DDSC_LOG_BASE` + 1327)
- Failed to create inter-participant channel.*

 - #define `DDSC_LOG_TRUST_UNKNOWN_GMCLASSID_EC` (`DDSC_LOG_BASE` + 1401)
- Unknown generic message class id.*

 - #define `DDSC_LOG_TRUST_UNKNOWN_SERVICEID_EC` (`DDSC_LOG_BASE` + 1402)
- Unknown builtin service id.*

 - #define `DDSC_LOG_TRUST_IGNORED_REMOTE_PARTICIPANT_EC` (`DDSC_LOG_BASE` + 1403)
- Ignored remote participant.*

 - #define `DDSC_LOG_TRUST_IGNORED_HANDSHAKE_MESSAGE_EC` (`DDSC_LOG_BASE` + 1404)
- Ignored handshake message from remote participant.*

 - #define `DDSC_LOG_TRUST_UNEXPECTED_VALIDATION_RESULT_EC` (`DDSC_LOG_BASE` + 1405)
- Unexpected validation result.*

 - #define `DDSC_LOG_TRUST_UNAUTHORIZED_REMOTE_PARTICIPANT_EC` (`DDSC_LOG_BASE` + 1406)
- A remote participant failed the authorization process.*

 - #define `DDSC_LOG_TRUST_FAILED_VALIDATE_LOCAL_IDENTITY_EC` (`DDSC_LOG_BASE` + 1407)
- Failed to validate local identity.*

- #define `DDSC_LOG_TRUST_INVALID_IDENTITY_HANDLE_EC` (`DDSC_LOG_BASE` + 1408)
Invalid local trust identity.
- #define `DDSC_LOG_TRUST_GET_IDENTITY_TOKEN_FAILED_EC` (`DDSC_LOG_BASE` + 1409)
Failed to get identity token.
- #define `DDSC_LOG_TRUST_INVALID_PERMISSIONS_HANDLE_EC` (`DDSC_LOG_BASE` + 1410)
Invalid local trust permissions.
- #define `DDSC_LOG_TRUST_INVALID_PARTICIPANT_GUID_EC` (`DDSC_LOG_BASE` + 1411)
Invalid DomainParticipant GUID configuration.
- #define `DDSC_LOG_TRUST_GET_PARTICIPANT_TRUST_ATTRIBUTES_FAILED_EC` (`DDSC_LOG_BASE` + 1412)
Failed to get DomainParticipant trust attributes.
- #define `DDSC_LOG_TRUST_CHECK_CREATE_PARTICIPANT_FAILED_EC` (`DDSC_LOG_BASE` + 1413)
Failed to create access control DomainParticipant.
- #define `DDSC_LOG_TRUST_GET_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC` (`DDSC_LOG_BASE` + 1414)
Failed to get permissions credential token.
- #define `DDSC_LOG_TRUST_GET_PERMISSIONS_TOKEN_FAILED_EC` (`DDSC_LOG_BASE` + 1415)
Failed to get permissions token.
- #define `DDSC_LOG_TRUST_INVALID_INTERCEPTOR_HANDLE_EC` (`DDSC_LOG_BASE` + 1416)
Invalid interceptor handle.
- #define `DDSC_LOG_TRUST_REGISTER_LOCAL_PARTICIPANT_FAILED_EC` (`DDSC_LOG_BASE` + 1417)
Failed to register local DomainParticipant.
- #define `DDSC_LOG_TRUST_RETURN_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC` (`DDSC_LOG_BASE` + 1418)
Failed to return permissions credential token.
- #define `DDSC_LOG_TRUST_RETURN_PERMISSIONS_TOKEN_FAILED_EC` (`DDSC_LOG_BASE` + 1419)
Failed to return permissions token.
- #define `DDSC_LOG_TRUST_RETURN_IDENTITY_TOKEN_FAILED_EC` (`DDSC_LOG_BASE` + 1420)
Failed to return identity token.
- #define `DDSC_LOG_TRUST_UNREGISTER_PARTICIPANT_FAILED_EC` (`DDSC_LOG_BASE` + 1421)
Failed to unregister local participant.
- #define `DDSC_LOG_TRUST_RETURN_PERMISSIONS_HANDLE_FAILED_EC` (`DDSC_LOG_BASE` + 1422)
Failed to return permissions handle.
- #define `DDSC_LOG_TRUST_RETURN_IDENTITY_HANDLE_FAILED_EC` (`DDSC_LOG_BASE` + 1423)
Failed to return identity handle.
- #define `DDSC_LOG_TRUST_RETURN_SHAREDSECRET_HANDLE_FAILED_EC` (`DDSC_LOG_BASE` + 1424)
Failed to return shared secret handle.
- #define `DDSC_LOG_TRUST_GET_AUTHENTICATED_PEER_CREDENTIAL_TOKEN_FAILED_EC` (`DDSC_LOG_BASE` + 1425)
Failed to get authenticated peer credential token.
- #define `DDSC_LOG_TRUST_CHECK_REMOTE_PARTICIPANT_FAILED_EC` (`DDSC_LOG_BASE` + 1426)
Failed to check access control remote DomainParticipant.
- #define `DDSC_LOG_TRUST_GET_SHARED_SECRET_FAILED_EC` (`DDSC_LOG_BASE` + 1427)
Failed to get shared secret.
- #define `DDSC_LOG_TRUST_PREPARE_LOCAL_PARTICIPANT_STATE_FAILED_EC` (`DDSC_LOG_BASE` + 1428)
Failed to validate local DomainParticipant trust.

- `#define DDSC_LOG_TRUST_DELETE_TRUST_PLUGINS_EC (DDSC_LOG_BASE + 1429)`
Failed to delete trust plugins.
- `#define DDSC_LOG_TRUST_SET_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1430)`
Failed to set permissions credential and token.
- `#define DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_PARTICIPANT_FAILED_EC (DDSC_LOG_BASE + 1431)`
Failed to register matched remote DomainParticipant.
- `#define DDSC_LOG_TRUST_GET_TOPIC_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1432)`
Failed to get topic trust attributes.
- `#define DDSC_LOG_TRUST_CHECK_CREATE_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1433)`
Failed to create trust check DataWriter.
- `#define DDSC_LOG_TRUST_RETURN_DATAWRITER_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1434)`
Failed to return DataWriter trust attributes.
- `#define DDSC_LOG_TRUST_GET_DATAWRITER_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1435)`
Failed to get DataWriter trust attributes.
- `#define DDSC_LOG_TRUST_GET_DATAREADER_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1436)`
Failed to get DataReader trust attributes.
- `#define DDSC_LOG_TRUST_CHECK_CREATE_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1437)`
Failed to create trust check DataReader.
- `#define DDSC_LOG_TRUST_BEGIN_HANDSHAKE_REPLY_EC (DDSC_LOG_BASE + 1438)`
Handshake begin reply failed.
- `#define DDSC_LOG_TRUST_CREATE_LOCAL_PARTICIPANT_INTERCEPTOR_TOKENS_EC (DDSC_LOG_BASE + 1439)`
Failed to create local DomainParticipant interceptor tokens.
- `#define DDSC_LOG_TRUST_CREATE_LOCAL_DW_TOKEN_EC (DDSC_LOG_BASE + 1440)`
Failed to create local DataWriter interceptor token.
- `#define DDSC_LOG_TRUST_CREATE_LOCAL_DR_TOKEN_EC (DDSC_LOG_BASE + 1441)`
Failed to create local DataReader interceptor token.
- `#define DDSC_LOG_TRUST_RETURN_HANDSHAKE_HANDLE_FAILED_EC (DDSC_LOG_BASE + 1442)`
Failed to return handshake handle.
- `#define DDSC_LOG_TRUST_SET_REMOTE_PARTICIPANT_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1443)`
Failed to set remote DomainParticipant interceptor tokens.
- `#define DDSC_LOG_TRUST_SET_REMOTE_DATAREADER_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1444)`
Failed to set remote DataReader interceptor tokens.
- `#define DDSC_LOG_TRUST_SET_REMOTE_DATAWRITER_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1445)`
Failed to set remote DataWriter interceptor tokens.
- `#define DDSC_LOG_TRUST_REGISTER_LOCAL_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1446)`
Failed to register local DataReader.
- `#define DDSC_LOG_TRUST_REGISTER_LOCAL_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1447)`
Failed to register local DataWriter.
- `#define DDSC_LOG_TRUST_CHECK_REMOTE_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1448)`

- Failed to check remote DataWriter.*

 - #define `DDSC_LOG_TRUST_CHECK_REMOTE_DATAREADER_FAILED_EC` (`DDSC_LOG_BASE + 1449`)
- Failed to check remote DataReader.*

 - #define `DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_DATAWRITER_FAILED_EC` (`DDSC_LOG_BASE + 1450`)
- Failed to register matched remote DataWriter.*

 - #define `DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_DATAREADER_FAILED_EC` (`DDSC_LOG_BASE + 1451`)
- Failed to register matched remote DataReader.*

 - #define `DDSC_LOG_TRUST_UNREGISTER_DATAREADER_FAILED_EC` (`DDSC_LOG_BASE + 1452`)
- Failed to unregister DataReader.*

 - #define `DDSC_LOG_TRUST_UNREGISTER_DATAWRITER_FAILED_EC` (`DDSC_LOG_BASE + 1453`)
- Failed to unregister DataWriter.*

 - #define `DDSC_LOG_TRUST_VALIDATE_REMOTE_IDENTITY_EC` (`DDSC_LOG_BASE + 1454`)
- Failed to validate remote identity.*

 - #define `DDSC_LOG_TRUST_BEGIN_HANDSHAKE_REQUEST_EC` (`DDSC_LOG_BASE + 1455`)
- Handshake begin request failed.*

 - #define `DDSC_LOG_TRUST_PROCESS_HANDSHAKE_EC` (`DDSC_LOG_BASE + 1456`)
- Handshake process failed.*

 - #define `DDSC_LOG_TRUST_CREATE_TRUST_PLUGINS_FAILED_EC` (`DDSC_LOG_BASE + 1457`)
- Failed to create trust plugins.*

 - #define `DDSC_LOG_TRUST_PARSE_PROPERTY_FAILED_EC` (`DDSC_LOG_BASE + 1458`)
- Failed to parse trust property.*

 - #define `DDSC_LOG_TRUST_AFTER_REMOTE_PARTICIPANT_READY_FAILED_EC` (`DDSC_LOG_BASE + 1459`)
- Error after remote trust DomainParticipant is ready.*

 - #define `DDSC_LOG_TRUST_REMOVE_REMOTE_PARTICIPANT_EC` (`DDSC_LOG_BASE + 1460`)
- Failed to remove remote DomainParticipant.*

 - #define `DDS_LOG_TRUST_ASSERT_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC` (`DDSC_LOG_BASE + 1461`)
- Failed to assert DomainParticipant generic message.*

 - #define `DDSC_LOG_TRUST_SEND_HANDSHAKE_EC` (`DDSC_LOG_BASE + 1462`)
- Failed to send handshake message.*

 - #define `DDSC_LOG_VALIDATE_REMOTE_PARTICIPANT_TRUST_FAILED_EC` (`DDSC_LOG_BASE + 1463`)
- Failed to validate trust remote DomainParticipant.*

 - #define `DDSC_LOG_TRUST_BIND_REMOTE_DATAREADER_FAILED_EC` (`DDSC_LOG_BASE + 1464`)
- Failed to get remote DataReader bind properties.*

 - #define `DDSC_LOG_TRUST_REGISTER_REMOTE_DATAREADER_FAILED_EC` (`DDSC_LOG_BASE + 1465`)
- Failed to register remote DataReader.*

 - #define `DDSC_LOG_TRUST_BIND_REMOTE_DATAWRITER_FAILED_EC` (`DDSC_LOG_BASE + 1466`)
- Failed to get remote DataWriter bind properties.*

 - #define `DDSC_LOG_TRUST_REGISTER_REMOTE_DATAWRITER_FAILED_EC` (`DDSC_LOG_BASE + 1467`)
- Failed to register remote DataWriter.*

 - #define `DDSC_LOG_TRUST_VALIDATE_REMOTE_DATAWRITER_TRUST_FAILED_EC` (`DDSC_LOG_BASE + 1468`)
- Failed to validate remote DataWriter.*

 - #define `DDSC_LOG_TRUST_VALIDATE_REMOTE_DATAREADER_TRUST_FAILED_EC` (`DDSC_LOG_BASE + 1469`)

- Failed to validate remote DataReader.*

 - #define DDSC_LOG_TRUST_VALIDATE_LOCAL_DATAWRITER_TRUST_FAILED_EC (DDSC_LOG_BASE + 1470)
- Failed to validate local DataWriter.*

 - #define DDSC_TRUST_INVALID_PARTICIPANT_GENERIC_MESSAGE_EC (DDSC_LOG_BASE + 1471)

Invalid DomainParticipant trust generic message.

 - #define DDSC_LOG_TRUST_PREPARE_AUTH_REQUEST_EC (DDSC_LOG_BASE + 1472)

Failed to prepare authentication request.

 - #define DDSC_LOG_TRUST_PREPARE_AUTH_HANDSHAKE_MESSAGE_EC (DDSC_LOG_BASE + 1473)

Failed to prepare authentication handshake message.

 - #define DDSC_LOG_TRUST_PREPARE_ENDPOINT_INTERCEPTOR_MESSAGE_EC (DDSC_LOG_BASE + 1474)

Failed to prepare endpoint interceptor message.

 - #define DDSC_LOG_TRUST_INVALID_GENERIC_MESSAGE_REPLY_EC (DDSC_LOG_BASE + 1475)

Invalid trust generic message reply.

 - #define DDSC_LOG_FINALIZE_TRUST_FAILED_EC (DDSC_LOG_BASE + 1476)

Failed to finalize trust.

 - #define DDSC_LOG_TRUST_DATA HOLDER_COPY_FAILED_EC (DDSC_LOG_BASE + 1477)

Failed to copy trust data holder.

 - #define DDSC_LOG_TRUST_INVALID_SAMPLE_MAX_SERIALIZED_SIZE_EC (DDSC_LOG_BASE + 1478)

Invalid sample serialized size.

 - #define DDSC_LOG_TRUST_ALLOC_FAILED_EC (DDSC_LOG_BASE + 1479)

Failed to allocate memory.

 - #define DDSC_LOG_TRUST_SERIALIZE_DATA_FAILED_EC (DDSC_LOG_BASE + 1480)

Failed to serialize trust data.

 - #define DDSC_LOG_TRUST_INITIALIZE_PARTICIPANT_TRUST_FAILED_EC (DDSC_LOG_BASE + 1481)

Failed to initialize trust.

 - #define DDSC_LOG_TRUST_INITIALIZE_PARTICIPANT_BUILTIN_DATA_FAILED_EC (DDSC_LOG_BASE + 1482)

Failed to initialize DomainParticipant trust data.

 - #define DDSC_LOG_TRUST_INVALID_PARTICIPANT_GENERIC_MESSAGE_EC (DDSC_LOG_BASE + 1483)

Invalid DomainParticipant trust generic message.

 - #define DDSC_LOG_TRUST_ADVANCE_SN_FAILED_EC (DDSC_LOG_BASE + 1484)

Failed to advance trust sequence number.

 - #define DDSC_LOG_TRUST_WRITE_SAMPLE_FAILED_EC (DDSC_LOG_BASE + 1485)

Failed to write sample.

 - #define DDSC_LOG_TRUST_RESEND_HANDSHAKE_IN_PROGRESS_EC (DDSC_LOG_BASE + 1486)

Failed to resend handshake because it is already in progress.

 - #define DDSC_LOG_TRUST_RESEND_HANDSHAKE_STOP_FAILED_EC (DDSC_LOG_BASE + 1487)

Failed to stop periodic handshake.

 - #define DDSC_LOG_TRUST_RELEASE_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC (DDSC_LOG_BASE + 1488)

Failed to release DomainParticipant trust generic message.

 - #define DDSC_LOG_TRUST_RESEND_HANDSHAKE_MESSAGE_START_FAILED_EC (DDSC_LOG_BASE + 1489)

Failed to start periodic handshake.

- #define `DDSC_LOG_TRUST_UNEXPECTED_REMOTE_PARTICIPANT_STATUS_EC` (`DDSC_LOG_BASE` + 1490)
Invalid remote DomainParticipant trust status.
- #define `DDSC_LOG_TRUST_SEND_PARTICIPANT_INTERCEPTOR_STATE_FAILED_EC` (`DDSC_LOG_BASE` + 1491)
Failed to send DomainParticipant interceptor state.
- #define `DDSC_LOG_TRUST_PARTICIPANT_AUTHENTICATION_FAILED_EC` (`DDSC_LOG_BASE` + 1492)
DomainParticipant authentication failed.
- #define `DDSC_LOG_TRUST_ASSERT_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC` (`DDSC_LOG_BASE` + 1493)
Failed to assert DomainParticipant generic message.
- #define `DDSC_LOG_TRUST_SEND_HANDSHAKE_MESSAGE_FAILED_EC` (`DDSC_LOG_BASE` + 1494)
Failed to send handshake message.
- #define `DDSC_LOG_TRUST_INVALID_INTERCEPTOR_TOKENS_EC` (`DDSC_LOG_BASE` + 1495)
Invalid interceptor tokens.
- #define `DDSC_LOG_TRUST_INVALID_INTERCEPTOR_TOKENS_DESTINATION_EC` (`DDSC_LOG_BASE` + 1496)
Invalid interceptor token destination.
- #define `DDSC_LOG_TRUST_ENDPOINT_INTERNAL_ATTRIBUTES_CREATE_FAILED_EC` (`DDSC_LOG_BASE` + 1497)
Failed to copy endpoint trust attributes.
- #define `DDSC_LOG_TRUST_SEND_ENDPOINT_INTERCEPTOR_STATE_FAILED_EC` (`DDSC_LOG_BASE` + 1498)
Failed to send tokens to remote DomainParticipant.
- #define `DDSC_LOG_TRUST_CREATE_LOCAL_DATAWRITER_INTERCEPTOR_TOKENS_FAILED_EC` (`DDSC_LOG_BASE` + 1499)
Failed to create tokens for the remote DataReader.
- #define `DDSC_LOG_TRUST_CREATE_LOCAL_DATAREADER_INTERCEPTOR_TOKENS_FAILED_EC` (`DDSC_LOG_BASE` + 1500)
Failed to create tokens for the remote DataWriter.
- #define `DDSC_LOG_TRUST_RETURN_INTERCEPTOR_TOKENS_FAILED_EC` (`DDSC_LOG_BASE` + 1501)
Failed to return interceptor tokens.
- #define `DDSC_LOG_TRUST_RETURN_AUTHENTICATED_PEER_CREDENTIAL_TOKEN_FAILED_EC` (`DDSC_LOG_BASE` + 1502)
Failed to return authenticated peer credential token.
- #define `DDSC_LOG_TRUST_PROCESS_REMOTE_INTERCEPTOR_TOKENS_FAILED_EC` (`DDSC_LOG_BASE` + 1503)
Failed to process interceptor tokens.
- #define `DDSC_LOG_TRUST_UNREGISTER_REMOTE_DATAWRITER_FAILED_EC` (`DDSC_LOG_BASE` + 1504)
Failed to unregister remote DataWriter.
- #define `DDSC_LOG_TRUST_UNREGISTER_REMOTE_DATAREADER_FAILED_EC` (`DDSC_LOG_BASE` + 1505)
Failed to unregister remote DataReader.
- #define `DDSC_LOG_TRUST_INVALID_SHARED_SECRET_EC` (`DDSC_LOG_BASE` + 1506)
Invalid shared secret.
- #define `DDSC_LOG_TRUST_PARTICIPANT_UNEXPECTED_AUTHENTICATION_ERROR_EC` (`DDSC_LOG_BASE` + 1507)
Unexpected DomainParticipant authentication error.

- #define DDSC_LOG_TRUST_PREPARE_LOCAL_DP_TOKENS_EC (DDSC_LOG_BASE + 1508)
Failed to prepare local DomainParticipant interceptor tokens.
- #define DDSC_LOG_TRUST_ASSERT_PROPERTY_FAILED_EC (DDSC_LOG_BASE + 1509)
Failed to assert trust property.
- #define DDSC_LOG_TRUST_PARTICIPANT_INTERNAL_ATTRIBUTES_CREATE_FAILED_EC (DDSC_LOG_BASE + 1510)
Failed to copy DomainParticipant trust attributes.
- #define DDSC_LOG_TRUST_INVALID_PARTICIPANT_INTERNAL_ATTRIBUTES_EC (DDSC_LOG_BASE + 1511)
Invalid DomainParticipant trust attributes.
- #define DDSC_LOG_TRUST_TRANSFORM_OUTGOING_SERIALIZED_PAYLOAD_FAILED_EC (DDSC_LOG_BASE + 1512)
Failed to encode outgoing serialized payload.
- #define DDSC_LOG_TRUST_TRANSFORM_INCOMING_SERIALIZED_PAYLOAD_FAILED_EC (DDSC_LOG_BASE + 1513)
Failed to decode incoming serialized payload.
- #define DDSC_LOG_TRUST_FINALIZE_AC_PLUGIN_PROPERTIES_FAILED_EC (DDSC_LOG_BASE + 1514)
Failed to finalize trust properties.
- #define DDSC_LOG_TRUST_AFTER_REMOTE_PARTICIPANT_AUTHENTICATED_FAILED_EC (DDSC_LOG_BASE + 1515)
Error after remote trust DomainParticipant is authenticated.
- #define DDSC_LOG_TRUST_CACHE_REMOTE_PARTICIPANT_SAMPLE_FAILED_EC (DDSC_LOG_BASE + 1516)
Failed to copy remote DomainParticipant sample.
- #define DDSC_LOG_TRUST_FINALIZE_AUTH_HANDSHAKE_STATE_FAILED_EC (DDSC_LOG_BASE + 1517)
Failed to finalize authentication handshake.
- #define DDSC_LOG_TRUST_FINALIZE_REMOTE_PARTICIPANT_TRUST_FAILED_EC (DDSC_LOG_BASE + 1518)
Failed to finalize remote trust DomainParticipant record.
- #define DDSC_LOG_TRUST_RESET_REMOTE_PARTICIPANT_TRUST_FAILED_EC (DDSC_LOG_BASE + 1519)
Failed to finalize remote trust DomainParticipant.
- #define DDSC_LOG_TRUST_ASSERT_AUTH_HANDSHAKE_STATE_FAILED_EC (DDSC_LOG_BASE + 1520)
Failed to assert authentication handshake state.
- #define DDSC_LOG_TRUST_INVALID_AUTH_HANDSHAKE_STATE_EC (DDSC_LOG_BASE + 1521)
Invalid authentication handshake state.
- #define DDSC_LOG_TRUST_SEND_AUTH_REQUEST_FAILED_EC (DDSC_LOG_BASE + 1522)
Failed to send authentication request.
- #define DDSC_LOG_TRUST_GET_AUTH_HANDSHAKE_STATE_FAILED_EC (DDSC_LOG_BASE + 1523)
Failed to get authentication handshake state.
- #define DDSC_LOG_TRUST_SET_AUTH_HANDSHAKE_STATE_FAILED_EC (DDSC_LOG_BASE + 1524)
Failed to set authentication handshake state.
- #define DDSC_LOG_TRUST_REAUTH_REMOTE_PARTICIPANT_FAILED_EC (DDSC_LOG_BASE + 1525)
Failed to reauthenticate a remote DomainParticipant.
- #define DDSC_LOG_TRUST_RESEND_HANDSHAKE_MESSAGE_FAILED_EC (DDSC_LOG_BASE + 1526)

- Failed to resend handshake message.*

 - #define `DDSC_LOG_TRUST_PBUFS_MAX_EXCEEDED_EC` (`DDSC_LOG_BASE` + 1527)

Maximum number of buffers exceeded.
- #define `DDSC_LOG_TRUST_CREATE_TRUST_PLUGIN_EC` (`DDSC_LOG_BASE` + 1528)

Failed to create trust plugin.
- #define `DDSC_LOG_TRUST_LOOKUP_FACTORY_FAILED_EC` (`DDSC_LOG_BASE` + 1529)

Failed to lookup trust plugin factory.
- #define `DDSC_LOG_NAME_LOOKUP_EC` (`DDSC_LOG_BASE` + 1700)

Fully qualified name does not contain substring "..."
- #define `DDSC_LOG_ENTITY_NAME_TOO_LONG_EC` (`DDSC_LOG_BASE` + 1701)

Failed to get an entity because the specified name exceeds the maximum length of 255 octets (excluding the terminating NUL)
- #define `DDSC_LOG_MESSAGE_DATA_UNKNOWN_LIVELINESS_KIND_EC` (`DDSC_LOG_BASE` + 1800)

Unknown inter-participant message liveliness kind received.
- #define `DDSC_LOG_MESSAGE_DATA_WRONG_LIVELINESS_KIND_EC` (`DDSC_LOG_BASE` + 1801)

Wrong inter-participant message liveliness kind received.
- #define `DDSC_LOG_UPDATE_LEASE_DURATION_TIMER_EC` (`DDSC_LOG_BASE` + 1802)

Failed to update DataWriter lease duration.
- #define `DDSC_LOG_PARTMESSAGE_WRITE_SAMPLE_FAILED_EC` (`DDSC_LOG_BASE` + 1803)

Failed to write inter-participant liveliness message.
- #define `DDSC_LOG_SEND_LIVELINESS_FAILED_EC` (`DDSC_LOG_BASE` + 1804)

Failed to send inter-participant liveliness message.
- #define `DDSC_LOG_UPDATE_LIVELINESS_FAILED_EC` (`DDSC_LOG_BASE` + 1805)

Failed to update DataWriter liveliness.
- #define `DDSC_LOG_REFRESH_LIVELINESS_FAILED_EC` (`DDSC_LOG_BASE` + 1806)

Failed to refresh endpoint liveliness.
- #define `DDSC_LOG_TRUST_INCOMPATIBLE_ENDPOINT_TRUST_ATTRIBUTES_EC` (`DDSC_LOG_BASE` + 1807)

Two security endpoints are incompatible.
- #define `DDSC_LOG_FLOW_CONTROLLER_CREATE_EC` (`DDSC_LOG_BASE` + 1900)

Failed to create flow controller.
- #define `DDSC_LOG_FLOW_CONTROLLER_DELETE_EC` (`DDSC_LOG_BASE` + 1901)

Failed to delete flow controller.
- #define `DDSC_LOG_XTYPES_LOANED_SAMPLE_STATE_ERROR_EC` (`DDSC_LOG_BASE` + 2000)

Extensible types sample state error.
- #define `DDSC_LOG_INTERPRETER_SUPPORT_UNEXPECTED_ERROR_EC` (`DDSC_LOG_BASE` + 2001)

Interpreter unexpected error.
- #define `DDSC_LOG_MEMORY_MANAGER_OUT_OF_RESOURCES_EC` (`DDSC_LOG_BASE` + 2002)

Memory manager out of resources.
- #define `DDSC_LOG_MEMORY_MANAGER_NOT_OWNER_EC` (`DDSC_LOG_BASE` + 2003)

Memory manager is not the owner of the data passed in.
- #define `DDSC_LOG_MEMORY_MANAGER_DELETE_EC` (`DDSC_LOG_BASE` + 2004)

The wire manager was not able to delete a resource.
- #define `DDSC_LOG_BUFFER_MAX_EXCEEDED_EC` (`DDSC_LOG_BASE` + 2101)

Maximum size of buffer exceeded.
- #define `DDSC_LOG_INVALID_BUFFER_LENGTH_EC` (`DDSC_LOG_BASE` + 2102)

Invalid buffer length.

- #define `DDSC_LOG_INCOMPATIBLE_PRESENTATION_QOS_EC` (`DDSC_LOG_BASE` + 2201)
Matching error because Presentation QoS is not compatible.
- #define `DDSC_LOG_INCOMPATIBLE_PARTITION_QOS_EC` (`DDSC_LOG_BASE` + 2202)
Two endpoints did not match because of incompatible partition QoS.
- #define `DDSC_LOG_CHECKSUM_REQUIRED_EC` (`DDSC_LOG_BASE` + 2301)
A participant requires checksum, but the discovered participant does not send a checksum.
- #define `DDSC_LOG_CHECKSUM_INCOMPATIBLE_EC` (`DDSC_LOG_BASE` + 2302)
A participant does not support checksum's sent by a discovered participant.
- #define `DDSC_LOG_CHECKSUM_INCONSISTENT_CLASS_EC` (`DDSC_LOG_BASE` + 2303)
The checksum class id for a custom checksum does match.
- #define `DDSC_LOG_CHECKSUM_INCONSISTENT_COMPUTE_EC` (`DDSC_LOG_BASE` + 2304)
The computed_crc_kind is inconsistent with what is allowed or supported.
- #define `DDSC_LOG_CHECKSUM_INCONSISTENT_ALLOW_EC` (`DDSC_LOG_BASE` + 2305)
The allowed_crc_mask is inconsistent with what is supported.
- #define `DDSC_LOG_FLOWCONTROLLER_INCONSISTENT_PROP_EC` (`DDSC_LOG_BASE` + 2306)
The token bucket properties for the flow controller are inconsistent.
- #define `DDSC_LOG_LOCK_EC` (`DDSC_LOG_BASE` + 2401)
Failed to take a lock.
- #define `DDSC_LOG_UNLOCK_EC` (`DDSC_LOG_BASE` + 2402)
Failed to give a lock.
- #define `DDSC_LOG_FILTER_PLUGIN_FACTORY_NOT_FOUND_EC` (`DDSC_LOG_BASE` + 2501)
Failed to find the filter plugin factory specified in a DomainParticipant's QoS.
- #define `DDSC_LOG_ENABLE_WRITER_FILTERING_EC` (`DDSC_LOG_BASE` + 2502)
A local writer failed to enable writer filtering for a reader.
- #define `DDSC_LOG_CREATE_WRITER_FILTER_EC` (`DDSC_LOG_BASE` + 2503)
A local writer failed to create its writer filter.
- #define `DDSC_LOG_EVALUATE_WRITER_FILTER_EC` (`DDSC_LOG_BASE` + 2504)
A local writer failed to evaluate a sample against the filters of its matched readers.
- #define `DDSC_LOG_APPLY_READER_FILTER_EC` (`DDSC_LOG_BASE` + 2505)
A local writer failed to check if a reader had filtered a sample.
- #define `DDSC_LOG_COMPILE_CONTENT_FILTER_EC` (`DDSC_LOG_BASE` + 2506)
Failed to compile the content filter configured in a DataReader's QoS.
- #define `DDSC_LOG_CONTENT_FILTERING_NOT_ENABLED_EC` (`DDSC_LOG_BASE` + 2507)
Reader configured a content filter but content filtering is not enabled.
- #define `DDSC_LOG_EVALUATE_CONTENT_FILTER_EC` (`DDSC_LOG_BASE` + 2508)
Failed to evaluate a sample against a reader's content filter.
- #define `DDSC_LOG_DESERIALIZE_CONTENT_FILTER_INFO_EC` (`DDSC_LOG_BASE` + 2509)
Failed to deserialize the content filter info from the in-line QoS of a sample.
- #define `DDSC_LOG_CHANGE_CONTENT_FILTER_EC` (`DDSC_LOG_BASE` + 2510)
Failed to change the configured content filter of a DataReader.
- #define `DDSC_LOG_UPDATE_REMOTE_CONTENT_FILTER_EC` (`DDSC_LOG_BASE` + 2511)
Failed to update the content filter of a remote subscription.
- #define `DDSC_LOG_CREATE_FILTER_PLUGIN_EC` (`DDSC_LOG_BASE` + 2512)
DomainParticipant failed to create its filter plugin.
- #define `DDSC_LOG_FINALIZE_FILTER_PLUGIN_EC` (`DDSC_LOG_BASE` + 2513)
DomainParticipant failed to finalize its filter plugin.
- #define `DDSC_LOG_DESERIALIZE_REMOTE_CONTENT_FILTER_EC` (`DDSC_LOG_BASE` + 2514)
Failed to deserialize the content filter property in the subscription built-in topic data for a remote subscription.

8.6.1 Detailed Description

DDS_C module log codes.

8.7 dds_filter_log.h File Reference

DDS Filter module log codes.

```
#include "osapi/osapi_log.h"
```

Macros

- `#define DDS_FILTER_LOG_FILTER_CLASS_RECORD 1`
Database record for a Filter class.
- `#define DDS_FILTER_LOG_COMPILED_FILTER_RECORD 2`
Database record for a compiled filter.
- `#define DDS_FILTER_LOG_READER_RECORD 3`
Database record for a reader entry.
- `#define DDS_FILTER_LOG_ROUTE_RECORD 4`
Database record for a route entry.
- `#define DDS_FILTER_LOG_RECORD_SELECT_EC (DDS_FILTER_LOG_BASE + 1)`
A database select on the specified record kind failed.
- `#define DDS_FILTER_LOG_RECORD_CREATE_EC (DDS_FILTER_LOG_BASE + 2)`
Failed to create a database record of the specified kind.
- `#define DDS_FILTER_LOG_RECORD_DELETE_EC (DDS_FILTER_LOG_BASE + 3)`
Failed to delete a database record of the specified kind.
- `#define DDS_FILTER_LOG_RECORD_INSERT_EC (DDS_FILTER_LOG_BASE + 4)`
Failed to insert a database record of the specified kind.
- `#define DDS_FILTER_LOG_RECORD_REMOVE_EC (DDS_FILTER_LOG_BASE + 5)`
Failed to remove a database record of the specified kind.
- `#define DDS_FILTER_LOG_TABLE_CREATE_EC (DDS_FILTER_LOG_BASE + 6)`
Failed to create a database table of the specified kind.
- `#define DDS_FILTER_LOG_TABLE_DELETE_EC (DDS_FILTER_LOG_BASE + 7)`
Failed to delete a database table of the specified kind.
- `#define DDS_FILTER_LOG_FILTER_CREATE_INSTANCE_EC (DDS_FILTER_LOG_BASE + 10)`
Failed to create an instance of the specified filter class.
- `#define DDS_FILTER_LOG_FILTER_DELETE_INSTANCE_EC (DDS_FILTER_LOG_BASE + 11)`
Failed to delete an instance of the specified filter class.
- `#define DDS_FILTER_LOG_FILTER_COMPILE_EC (DDS_FILTER_LOG_BASE + 12)`
Failed to compile a filter of the specified class.
- `#define DDS_FILTER_LOG_FILTER_EVALUATE_EC (DDS_FILTER_LOG_BASE + 13)`
Failed to evaluate a filter of the specified class.
- `#define DDS_FILTER_LOG_WRITER_DETACH_EC (DDS_FILTER_LOG_BASE + 14)`
Failed to detach a writer because it still has remote readers attached.
- `#define DDS_FILTER_LOG_FILTER_EXPRESSION_MAX_COUNT_EC (DDS_FILTER_LOG_BASE + 15)`

No more filters can be compiled because the configured maximum number of filter expressions has been reached.

- #define `DDS_FILTER_LOG_FILTER_CLASS_REGISTER_EC` (`DDS_FILTER_LOG_BASE` + 20)
Failed to register a filter class because one with the same name has already been registered or because the provided interface did not contain the required function pointers.
- #define `DDS_FILTER_LOG_FILTER_CLASS_NOT_FOUND_EC` (`DDS_FILTER_LOG_BASE` + 21)
Failed to find a filter class registered with the specified name.
- #define `DDS_FILTER_LOG_FILTER_CLASS_UNREGISTER_EC` (`DDS_FILTER_LOG_BASE` + 22)
Failed to unregister a filter class because it is still in use.
- #define `DDS_FILTER_LOG_CONTENT_FILTERED_TOPIC_NAME_STRING` 1
Content filter topic name string.
- #define `DDS_FILTER_LOG_RELATED_TOPIC_NAME_STRING` 2
Content filter related topic name string.
- #define `DDS_FILTER_LOG_FILTER_CLASS_NAME_STRING` 3
Filter class name string.
- #define `DDS_FILTER_LOG_FILTER_EXPRESSION_STRING` 4
Filter expression string.
- #define `DDS_FILTER_LOG_FILTER_EXPRESSION_PARAMETER_STRING` 5
Filter expression parameter string.
- #define `DDS_FILTER_LOG_CONTENT_FILTER_NAME_STRING` 6
Content filter QoS expression name string.
- #define `DDS_FILTER_LOG_STRING_ASSERT_EC` (`DDS_FILTER_LOG_BASE` + 30)
Failed to assert a string in the string manager of a specified kind.
- #define `DDS_FILTER_LOG_STRING_DELETE_EC` (`DDS_FILTER_LOG_BASE` + 31)
Failed to delete a string in the string manager of a specified kind.
- #define `DDS_FILTER_LOG_CONTENT_FILTER_NAME_TOO_LONG_EC` (`DDS_FILTER_LOG_BASE` + 32)
Failed to construct a content filter topic name from the combination of the related topic name and the filter name because it exceeded the maximum topic name length.
- #define `DDS_FILTER_LOG_STRING_MANAGER_CREATE_EC` (`DDS_FILTER_LOG_BASE` + 33)
Failed to create a string manager for a specified string kind, max_string_size, and memory_allocation.
- #define `DDS_FILTER_LOG_FILTER_PARAMETER_COUNT_EC` (`DDS_FILTER_LOG_BASE` + 34)
Failed to deserialize a content filter property because it contained too many expression parameters.
- #define `DDS_FILTER_LOG_TYPE_CODE_NOT_FOUND_EC` (`DDS_FILTER_LOG_BASE` + 40)
Content filtering is not supported on this data type because its type support does not include a type code.
- #define `DDS_FILTER_LOG_FILTER_PLUGIN_FACTORY_BUFFER` 1
Heap memory for a filter plugin factory instance.
- #define `DDS_FILTER_LOG_FILTER_PLUGIN_BUFFER` 2
Heap memory for a filter plugin instance.
- #define `DDS_FILTER_LOG_WRITER_FILTER_BUFFER` 3
Heap memory for a writer filter instance.
- #define `DDS_FILTER_LOG_INLINE_QOS_BUFFER` 4
Heap memory for a writer to serialize a sample's in-line QoS.
- #define `DDS_FILTER_LOG_HEAP_ALLOC_EC` (`DDS_FILTER_LOG_BASE` + 50)
Failed to allocate heap memory for a buffer of a given type.
- #define `DDS_FILTER_LOG_FILTER_PLUGIN_INIT_EC` (`DDS_FILTER_LOG_BASE` + 51)
Failed to initialize a filter plugin.
- #define `DDS_FILTER_LOG_FILTER_PLUGIN_FINALIZE_EC` (`DDS_FILTER_LOG_BASE` + 52)
Failed to finalize a filter plugin.

- #define `DDS_FILTER_LOG_FILTER_PLUGIN_FACTORY_IN_USE_EC` (`DDS_FILTER_LOG_BASE` + 53)
Failed to finalize a filter plugin factory because it is still in use.
- #define `DDS_FILTER_LOG_WRITER_FILTER_COMPILE_EC` (`DDS_FILTER_LOG_BASE` + 61)
A writer filter failed to compile a filter for a discovered reader.
- #define `DDS_FILTER_LOG_WRITER_FILTER_REMOVE_READER_EC` (`DDS_FILTER_LOG_BASE` + 62)
Failed to remove a reader from a writer filter because it still has active routes.
- #define `DDS_FILTER_LOG_WRITER_FILTER_SEND_GAP_EC` (`DDS_FILTER_LOG_BASE` + 63)
Writer failed to send a reader a GAP to indicate filtered data.
- #define `DDS_FILTER_LOG_WRITER_SERIALIZE_INLINE_QOS_EC` (`DDS_FILTER_LOG_BASE` + 64)
Writer failed to serialize the content filter info into the in-line QoS.
- #define `DDS_FILTER_LOG_WRITER_FILTER_EVALUATE_OUT_OF_ORDER_EC` (`DDS_FILTER_LOG_BASE` + 65)
Writer filter was evaluated out of order which requires invalidating a previously results.
- #define `DDS_FILTER_LOG_WRITER_FILTER_STORE_RESULT_EC` (`DDS_FILTER_LOG_BASE` + 66)
Writer filter unexpectedly failed to store the result of a filter evaluation.
- #define `DDS_FILTER_LOG_SQL_UNSUPPORTED_TYPE_CODE_EC` (`DDS_FILTER_LOG_BASE` + 100)
Unsupported type code for SQL filtering.
- #define `DDS_FILTER_LOG_SQL_UNEXPECTED_TOKEN_EC` (`DDS_FILTER_LOG_BASE` + 101)
Found an unexpected token while parsing an SQL filter expression.
- #define `DDS_FILTER_LOG_SQL_MAX_PREDICATES_EXCEEDED_EC` (`DDS_FILTER_LOG_BASE` + 102)
Failed to parse an SQL filter expression because the number of predicates in the expression exceeded the configured maximum.
- #define `DDS_FILTER_LOG_SQL_INVALID_TOKEN_EC` (`DDS_FILTER_LOG_BASE` + 103)
Found an invalid token while parsing an SQL filter expression.
- #define `DDS_FILTER_LOG_SQL_PARSE_PARAMETER_EC` (`DDS_FILTER_LOG_BASE` + 104)
Failed to parse a parameter in an SQL filter expression.
- #define `DDS_FILTER_LOG_SQL_FIELD_NOT_FOUND_EC` (`DDS_FILTER_LOG_BASE` + 105)
Failed to parse an SQL filter expression because a given field name did not match any field in the type code of the data being filtered.
- #define `DDS_FILTER_LOG_SQL_UNSUPPORTED_FIELD_TYPE_EC` (`DDS_FILTER_LOG_BASE` + 106)
A field specified in an SQL filter expression is not a supported type for use in an SQL filter.
- #define `DDS_FILTER_LOG_SQL_ENUM_LABEL_NOT_FOUND_EC` (`DDS_FILTER_LOG_BASE` + 107)
Failed to find a given enum label in an SQL filter expression.
- #define `DDS_FILTER_LOG_SQL_INCOMPATIBLE_TYPES_EC` (`DDS_FILTER_LOG_BASE` + 108)
Comparison between two incompatible types in an SQL filter expression.
- #define `DDS_FILTER_LOG_SQL_INTEGER_OUT_OF_RANGE_EC` (`DDS_FILTER_LOG_BASE` + 109)
Integer constant in an SQL expression is out of range for the type.

8.7.1 Detailed Description

DDS Filter module log codes.

8.7.2 Macro Definition Documentation

DDS_FILTER_LOG_TYPE_CODE_NOT_FOUND_EC

```
#define DDS_FILTER_LOG_TYPE_CODE_NOT_FOUND_EC (DDS_FILTER_LOG_BASE + 40)
```

Content filtering is not supported on this data type because its type support does not include a type code.

Content filtering requires the type code to be able to evaluate the filter expression. The type code can be automatically generated by rtiddsgen either by annotating the type in its IDL with `@interpreted(true)` or by using the "-interpreted 1" flag.

DDS_FILTER_LOG_FILTER_PLUGIN_FACTORY_BUFFER

```
#define DDS_FILTER_LOG_FILTER_PLUGIN_FACTORY_BUFFER 1
```

Heap memory for a filter plugin factory instance.

DDS_FILTER_LOG_FILTER_PLUGIN_BUFFER

```
#define DDS_FILTER_LOG_FILTER_PLUGIN_BUFFER 2
```

Heap memory for a filter plugin instance.

DDS_FILTER_LOG_WRITER_FILTER_BUFFER

```
#define DDS_FILTER_LOG_WRITER_FILTER_BUFFER 3
```

Heap memory for a writer filter instance.

DDS_FILTER_LOG_INLINE_QOS_BUFFER

```
#define DDS_FILTER_LOG_INLINE_QOS_BUFFER 4
```

Heap memory for a writer to serialize a sample's in-line QoS.

8.8 dds_psk_log.h File Reference

DDS_PSK module log codes.

```
#include "osapi/osapi_log.h"
```

Macros

- #define `DDS_PSK_LOG_TRUST_EXCEPTION_EC` (`SEC_CORE_LOG_BASE` + 3)
Log trust exceptions.

8.8.1 Detailed Description

DDS_PSK module log codes.

8.8.2 Macro Definition Documentation

DDS_PSK_LOG_TRUST_EXCEPTION_EC

```
#define DDS_PSK_LOG_TRUST_EXCEPTION_EC (SEC_CORE_LOG_BASE + 3)
```

Log trust exceptions.

8.9 disc_dpde_log.h File Reference

```
#include "osapi/osapi_log.h"
```

Macros

- #define `DPDE_LOG_CDR_SET_OFFSET_EC` (`DPDE_LOG_BASE` + 1)
Failed to set current offset of stream.
- #define `DPDE_LOG_SERIALIZE_ENTITY_NAME_EC` (`DPDE_LOG_BASE` + 2)
Failed to serialize Entity Name parameter.
- #define `DPDE_LOG_SERIALIZE_TOPIC_NAME_EC` (`DPDE_LOG_BASE` + 3)
Failed to serialize Topic Name parameter.
- #define `DPDE_LOG_SERIALIZE_TYPE_NAME_EC` (`DPDE_LOG_BASE` + 4)
Failed to serialize Type Name parameter.
- #define `DPDE_LOG_SERIALIZE_GUID_EC` (`DPDE_LOG_BASE` + 5)
Failed to serialize GUID key.
- #define `DPDE_LOG_SERIALIZE_DEFAULT_UNICAST_EC` (`DPDE_LOG_BASE` + 6)
Failed to serialize Default Unicast Locator parameter.
- #define `DPDE_LOG_SERIALIZE_PROTOCOL_VERSION_EC` (`DPDE_LOG_BASE` + 7)
Failed to serialize Protocol Version parameter.
- #define `DPDE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC` (`DPDE_LOG_BASE` + 8)
Failed to deserialize Protocol Version parameter.
- #define `DPDE_LOG_DESERIALIZE_VENDOR_ID_EC` (`DPDE_LOG_BASE` + 9)
Failed to deserialize Vendor ID parameter.
- #define `DPDE_LOG_SERIALIZE_VENDOR_ID_EC` (`DPDE_LOG_BASE` + 10)
Failed to serialize Vendor ID parameter.

- #define DPDE_LOG_SERIALIZE_PRODUCT_VERSION_EC (DPDE_LOG_BASE + 11)
Failed to serialize Product Version parameter.
- #define DPDE_LOG_SERIALIZE_LEASE_DURATION_EC (DPDE_LOG_BASE + 12)
Failed to serialize Lease Duration parameter.
- #define DPDE_LOG_CREATE_PUBLISHER_EC (DPDE_LOG_BASE + 13)
Failed to create Publisher for built-in endpoint DataWriters.
- #define DPDE_LOG_CREATE_SUBSCRIBER_EC (DPDE_LOG_BASE + 14)
Failed to create Subscriber for built-in endpoint DataReaders.
- #define DPDE_LOG_CREATE_PARTICIPANT_DISCOVERY_EC (DPDE_LOG_BASE + 15)
Failed to create built-in endpoint for participant discovery after creating local user DomainParticipant.
- #define DPDE_LOG_CREATE_PUBLICATION_DISCOVERY_EC (DPDE_LOG_BASE + 16)
Failed to create built-in publication endpoint after creating local user DomainParticipant.
- #define DPDE_LOG_CREATE_SUBSCRIPTION_DISCOVERY_EC (DPDE_LOG_BASE + 17)
Failed to create built-in subscription endpoint after creating local DomainParticipant.
- #define DPDE_LOG_DELETE_SUBSCRIPTION_TOPIC_EC (DPDE_LOG_BASE + 18)
Failed to delete subscription built-in topic.
- #define DPDE_LOG_DELETE_PUBLICATION_TOPIC_EC (DPDE_LOG_BASE + 19)
Failed to delete publication built-in topic.
- #define DPDE_LOG_DELETE_PARTICIPANT_TOPIC_EC (DPDE_LOG_BASE + 20)
Failed to delete participant built-in topic.
- #define DPDE_LOG_DISPOSE_PARTICIPANT_EC (DPDE_LOG_BASE + 21)
Failed to dispose built-in participant when deleting DomainParticipant.
- #define DPDE_LOG_ALLOCATE_DPDE_EC (DPDE_LOG_BASE + 22)
Out of memory to allocate discovery plugin.
- #define DPDE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC (DPDE_LOG_BASE + 23)
Failed to serialize Builtin Endpoint Mask parameter.
- #define DPDE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC (DPDE_LOG_BASE + 24)
Failed to deserialize Builtin Endpoint Mask parameter.
- #define DPDE_LOG_DESERIALIZE_UNKNOWN_PID_EC (DPDE_LOG_BASE + 25)
Failed to deserialize a parameter of unknown ID.
- #define DPDE_LOG_ANNOUNCEMENT_EC (DPDE_LOG_BASE + 26)
Failed to write a dynamic participant discovery message.
- #define DPDE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC (DPDE_LOG_BASE + 27)
Failed to schedule an event to assert the next participant discovery announcement.
- #define DPDE_LOG_ADVANCE_SN_EC (DPDE_LOG_BASE + 28)
Failed to advance the sequence number of a participant discovery announcement.
- #define DPDE_LOG_ANNOUNCE_WRITE_EC (DPDE_LOG_BASE + 29)
Failed to write initial participant discovery announcement.
- #define DPDE_LOG_SCHEDULE_FAST_ASSERTION_EC (DPDE_LOG_BASE + 30)
Failed to schedule event for asserting participant discovery announcements.
- #define DPDE_LOG_REGISTER_TYPE_EC (DPDE_LOG_BASE + 32)
Failed to register a built-in type for discovery.
- #define DPDE_LOG_CREATE_TOPIC_EC (DPDE_LOG_BASE + 33)
Failed to create a topic for discovery.
- #define DPDE_LOG_CREATE_WRITER_EC (DPDE_LOG_BASE + 34)
Failed to create a built-in DataWriter for discovery.
- #define DPDE_LOG_CREATE_READER_EC (DPDE_LOG_BASE + 35)

- Failed to create a built-in DataReader for discovery.*

 - #define `DPDE_LOG_ASSERT_REMOTE_PARTICIPANT_EC` (`DPDE_LOG_BASE` + 36)
- Failed to assert and complete discovery of a remote participant.*

 - #define `DPDE_LOG_ENABLE_REMOTE_PARTICIPANT_EC` (`DPDE_LOG_BASE` + 37)
- Failed to enable and complete discovery of a remote participant.*

 - #define `DPDE_LOG_REFRESH_REMOTE_PARTICIPANT_EC` (`DPDE_LOG_BASE` + 38)
- Failed to refresh liveliness for a discovered remote participant.*

 - #define `DPDE_LOG_TAKE_PARTICIPANT_SAMPLE_EC` (`DPDE_LOG_BASE` + 39)
- Failed to take a sample from a participant discovery DataReader.*

 - #define `DPDE_LOG_INVALID_PARTICIPANT_SAMPLE_EC` (`DPDE_LOG_BASE` + 40)
- Received a participant discovery announcement with invalid state.*

 - #define `DPDE_LOG_RETURN_PARTICIPANT_SAMPLE_EC` (`DPDE_LOG_BASE` + 41)
- Failed to return a loan on a participant discovery sample.*

 - #define `DPDE_LOG_DISPOSE_PUBLICATION_EC` (`DPDE_LOG_BASE` + 42)
- Failed to dispose an instance for a publication.*

 - #define `DPDE_LOG_DISPOSE_SUBSCRIPTION_EC` (`DPDE_LOG_BASE` + 43)
- Failed to dispose an instance of a subscription.*

 - #define `DPDE_LOG_ASSERT_REMOTE_PUBLICATION_EC` (`DPDE_LOG_BASE` + 44)
- Failed to assert and complete discovery of a remote publication.*

 - #define `DPDE_LOG_ASSERT_REMOTE_SUBSCRIPTION_EC` (`DPDE_LOG_BASE` + 45)
- Failed to assert and complete discovery of a remote subscription. Logs the topic name.*

 - #define `DPDE_LOG_TAKE_PUBLICATION_SAMPLE_EC` (`DPDE_LOG_BASE` + 46)
- Failed to take a sample from a publication discovery DataReader.*

 - #define `DPDE_LOG_REMOVE_REMOTE_PUBLICATION_EC` (`DPDE_LOG_BASE` + 47)
- Failed to dispose or unregister a remote publication instance.*

 - #define `DPDE_LOG_INVALID_PUBLICATION_SAMPLE_EC` (`DPDE_LOG_BASE` + 48)
- Received a publication discovery announcement with invalid state.*

 - #define `DPDE_LOG_RETURN_PUBLICATION_SAMPLE_EC` (`DPDE_LOG_BASE` + 49)
- Failed to return a loan on a publication discovery sample.*

 - #define `DPDE_LOG_TAKE_SUBSCRIPTION_SAMPLE_EC` (`DPDE_LOG_BASE` + 51)
- Failed to take a sample from a subscription discovery DataReader.*

 - #define `DPDE_LOG_INVALID_SUBSCRIPTION_SAMPLE_EC` (`DPDE_LOG_BASE` + 52)
- Received a subscription discovery sample with invalid state.*

 - #define `DPDE_LOG_RETURN_SUBSCRIPTION_SAMPLE_EC` (`DPDE_LOG_BASE` + 53)
- Failed to return a loan on a subscription discovery sample.*

 - #define `DPDE_LOG_REMOVE_REMOTE_SUBSCRIPTION_EC` (`DPDE_LOG_BASE` + 54)
- Failed to dispose or unregister a remote subscription instance.*

 - #define `DPDE_LOG_LOCATORS_FULL_EC` (`DPDE_LOG_BASE` + 55)
- A locator sequence is full, the remaining locators of the specified kind are dropped.*

 - #define `DPDE_LOG_DESERIALIZE_LOCATOR_EC` (`DPDE_LOG_BASE` + 56)
- Failed to deserialize a locator of the specified kind.*

 - #define `DPDE_LOG_UNSUPPORTED_LOCATOR_EC` (`DPDE_LOG_BASE` + 57)
- The locator kind is not supported by any transport registered with the domain participant and is dropped.*

 - #define `DPDE_LOG_ADD_PEER_EC` (`DPDE_LOG_BASE` + 58)
- Failed to add the specified entity as a peer.*

 - #define `DPDE_LOG_REMOVE_PEER_EC` (`DPDE_LOG_BASE` + 59)
- Failed to add the specified entity as a peer.*

- `#define DPDE_LOG_SERIALIZE_PRESENTATION_EC (DPDE_LOG_BASE + 60)`
Failed to serialize Presentation parameter.
- `#define DPDE_LOG_UNREGISTER_TYPE_EC (DPDE_LOG_BASE + 61)`
Failed to unregister a built-in type for discovery.
- `#define DPDE_LOG_ADD_REMOTE_PARTICIPANT_ROUTES_FAILED_EC (DPDE_LOG_BASE + 62)`
Failed to add routes to a remote participant.
- `#define DPDE_LOG_DELETE_REMOTE_PARTICIPANT_ROUTES_FAILED_EC (DPDE_LOG_BASE + 63)`
Failed to delete routes to a remote participant.
- `#define DPDE_LOG_REMOVE_REMOTE_PARTICIPANT_EC (DPDE_LOG_BASE + 64)`
Failed remove a remote participant.
- `#define DPDE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC (DPDE_LOG_BASE + 65)`
A participant with incompatible checksum was discovered and ignored.

8.10 disc_dpse_log.h File Reference

```
#include "osapi/osapi_log.h"
```

Macros

- `#define DPSE_LOG_INVALID_PEER_ADDRESS_EC (DPSE_LOG_BASE + 1)`
A peer address string specifies an invalid address.
- `#define DPSE_LOG_CREATE_DISCOVERY_PUBLISHER_EC (DPSE_LOG_BASE + 2)`
Failed to create the publisher for a participant discovery writer.
- `#define DPSE_LOG_CREATE_DISCOVERY_SUBSCRIBER_EC (DPSE_LOG_BASE + 3)`
Failed to create the subscriber for a participant discovery datareader.
- `#define DPSE_LOG_REGISTER_TYPE_EC (DPSE_LOG_BASE + 4)`
Failed to register a built-in participant discovery type.
- `#define DPSE_LOG_CREATE_TOPIC_EC (DPSE_LOG_BASE + 5)`
Failed to create a topic for the built-in participant discovery topic.
- `#define DPSE_LOG_CREATE_WRITER_EC (DPSE_LOG_BASE + 6)`
Failed to create a DataWriter for participant discovery.
- `#define DPSE_LOG_CREATE_READER_EC (DPSE_LOG_BASE + 7)`
Failed to create a DataReader for participant discovery.
- `#define DPSE_LOG_DISPOSE_EC (DPSE_LOG_BASE + 8)`
Failed to dispose a participant discovery instance.
- `#define DPSE_LOG_ANNOUNCE_LOCAL_PARTICIPANT_DELETION_EC (DPSE_LOG_BASE + 9)`
Failed to dispose a participant discovery instance upon deletion.
- `#define DPSE_LOG_DELETE_WRITER_EC (DPSE_LOG_BASE + 10)`
Failed to delete a participant discovery DataWriter.
- `#define DPSE_LOG_DELETE_PUBLISHER_EC (DPSE_LOG_BASE + 11)`
Failed to delete a participant discovery Publisher.
- `#define DPSE_LOG_DELETE_READER_EC (DPSE_LOG_BASE + 12)`
Failed to delete a participant discovery DataReader.
- `#define DPSE_LOG_DELETE_TOPIC_EC (DPSE_LOG_BASE + 13)`

- Failed to delete a participant discovery Topic.*

 - #define `DPSE_LOG_DELETE_SUBSCRIBER_EC` (`DPSE_LOG_BASE` + 14)
- Failed to delete a participant discovery Subscriber.*

 - #define `DPSE_KEYLIST_OBJECT` (1)
- The list keeping track of asserted participant keys.*

 - #define `DPSE_PARTICIPANTMAMES_OBJECT` (2)
- The list keeping track of asserted participant names.*

 - #define `DPSE_PARTICIPANTNAMEINDEX_OBJECT` (3)
- The index keeping track of asserted participant names.*

 - #define `DPSE_PARTICIPANTANNOUCEMENT_OBJECT` (4)
- The participant announcement timeout object.*

 - #define `DPSE_DISCOVERYPLUGIN_OBJECT` (5)
- A discovery plugin object.*

 - #define `DPSE_PARTICIPANTBUILTINTOPICDATA_OBJECT` (6)
- ParticipantBuiltinTopicData.*

 - #define `DPSE_PARTICIPANTENTITYNAME_OBJECT` (7)
- ParticipantBuiltinTopicData entity-name object.*

 - #define `DPSE_ENABLEDPARTICIPANTINDEX_OBJECT` (8)
- The index keeping track of enabled participant names.*

 - #define `DPSE_PARTICIPANTPOOL_OBJECT` (10)
- The pool of asserted participants.*

 - #define `DPSE_LOG_OBJECT_ALLOCATE_EC` (`DPSE_LOG_BASE` + 15)
- Failed to allocate an object of the specified kind.*

 - #define `DPSE_LOG_OBJECT_INITIALIZE_EC` (`DPSE_LOG_BASE` + 16)
- Failed to initialize an object of the specified kind.*

 - #define `DPSE_LOG_OBJECT_FINALIZE_EC` (`DPSE_LOG_BASE` + 17)
- Failed to finalize an object of the specified kind.*

 - #define `DPSE_LOG_OBJECT_DELETE_EC` (`DPSE_LOG_BASE` + 18)
- Failed to delete an object of the specified kind.*

 - #define `DPSE_LOG_OBJECT_INVALID_EC` (`DPSE_LOG_BASE` + 19)
- The object was invalid in the context it was used.*

 - #define `DPSE_LOG_CDR_SET_POSITION_EC` (`DPSE_LOG_BASE` + 20)
- Failed to set the CDR stream position.*

 - #define `DPSE_LOG_CDR_LONG_KIND` 1
- CDR Long kind.*

 - #define `DPSE_LOG_CDR_PID_KIND` 2
- CDR PID kind, the pid argument specifies which pid.*

 - #define `DPSE_LOG_CDR_SERIALIZE_EC` (`DPSE_LOG_BASE` + 21)
- Failed to serialize the specified kind.*

 - #define `DPSE_LOG_CDR_DESERIALIZE_EC` (`DPSE_LOG_BASE` + 22)
- Failed to deserialize the specified kind.*

 - #define `DPSE_LOG_SERIALIZE_GUID_EC` (`DPSE_LOG_BASE` + 23)
- Failed to serialize a GUID key parameter.*

 - #define `DPSE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC` (`DPSE_LOG_BASE` + 24)
- Failed to serialize a Builtin Endpoint Mask parameter.*

 - #define `DPSE_LOG_SERIALIZE_PROTOCOL_VERSION_EC` (`DPSE_LOG_BASE` + 25)
- Failed to serialize a Protocol Version parameter.*

- `#define DPSE_LOG_SERIALIZE_VENDOR_ID_EC (DPSE_LOG_BASE + 26)`
Failed to serialize a Vendor ID parameter.
- `#define DPSE_LOG_SERIALIZE_DEFAULT_UNICAST_EC (DPSE_LOG_BASE + 27)`
Failed to serialize a Default Unicast Locator parameter.
- `#define DPSE_LOG_SERIALIZE_META_UNICAST_EC (DPSE_LOG_BASE + 28)`
Failed to serialize a Meta Unicast Locator parameter.
- `#define DPSE_LOG_SERIALIZE_META_MULTICAST_EC (DPSE_LOG_BASE + 29)`
Failed to serialize a Meta Multicast Locator parameter.
- `#define DPSE_LOG_SERIALIZE_LEASE_DURATION_EC (DPSE_LOG_BASE + 30)`
Failed to serialize a Lease Duration parameter.
- `#define DPSE_LOG_SERIALIZE_PRODUCT_VERSION_EC (DPSE_LOG_BASE + 31)`
Failed to serialize a Product Version parameter.
- `#define DPSE_LOG_DESERIALIZE_PRODUCT_VERSION_EC (DPSE_LOG_BASE + 32)`
Failed to serialize a Product Version parameter.
- `#define DPSE_LOG_SERIALIZE_PARTICIPANT_NAME_EC (DPSE_LOG_BASE + 33)`
Failed to serialize a Participant Name parameter.
- `#define DPSE_LOG_DESERIALIZE_GUID_EC (DPSE_LOG_BASE + 34)`
Failed to deserialize a GUID key parameter.
- `#define DPSE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC (DPSE_LOG_BASE + 35)`
Failed to deserialize a Builtin Endpoint Mask parameter.
- `#define DPSE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC (DPSE_LOG_BASE + 36)`
Failed to deserialize a Protocol Version parameter.
- `#define DPSE_LOG_DESERIALIZE_VENDOR_ID_EC (DPSE_LOG_BASE + 37)`
Failed to deserialize a Vendor ID parameter.
- `#define DPSE_LOG_DESERIALIZE_TOO_MANY_LOCATORS_EC (DPSE_LOG_BASE + 38)`
Cannot deserialize another locator parameter, having reached maximum of 4 unicast or 4 multicast locators.
- `#define DPSE_LOG_DESERIALIZE_PARTICIPANT_NAME_EC (DPSE_LOG_BASE + 39)`
Failed to deserialize a Participant Name parameter.
- `#define DPSE_LOG_DESERIALIZE_UNKNOWN_PID_EC (DPSE_LOG_BASE + 40)`
Failed to deserialize a parameter with an unknown ID.
- `#define DPSE_LOG_ANNOUNCEMENT_EC (DPSE_LOG_BASE + 41)`
Failed to send a participant discovery announcement.
- `#define DPSE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC (DPSE_LOG_BASE + 42)`
Failed to schedule an event to send the next participant discovery announcement.
- `#define DPSE_LOG_ADVANCE_SN_EC (DPSE_LOG_BASE + 43)`
Failed to advance the sequence number of a participant discovery announcement.
- `#define DPSE_LOG_ANNOUNCE_WRITE_EC (DPSE_LOG_BASE + 44)`
Failed to write a participant discovery announcement message.
- `#define DPSE_LOG_SCHEDULE_FAST_ASSERTION_EC (DPSE_LOG_BASE + 45)`
Failed to schedule an event to send the next participant discovery announcement.
- `#define DPSE_LOG_PARTICIPANT_TAKE_EC (DPSE_LOG_BASE + 46)`
Failed to take a sample from a participant discovery DataReader.
- `#define DPSE_LOG_ASSERT_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 47)`
Failed to assert remote participant.
- `#define DPSE_LOG_ON_ASSERT_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 48)`
Failed to assert and complete discovery of a remote participant.
- `#define DPSE_LOG_ADD_ANONYMOUS_ROUTE_EC (DPSE_LOG_BASE + 49)`

- Failed to add an anonymous route.*

 - #define `DPSE_LOG_DELETE_ANONYMOUS_ROUTE_EC` (`DPSE_LOG_BASE` + 50)
- Failed to delete an anonymous route.*

 - #define `DPSE_LOG_MAX_REMOTE_PARTICIPANT_EC` (`DPSE_LOG_BASE` + 51)
- Exceeded resource limit, remote participant allocation.*

 - #define `DPSE_LOG_REFRESH_REMOTE_PARTICIPANT_EC` (`DPSE_LOG_BASE` + 52)
- Failed to refresh liveliness for a remote participant.*

 - #define `DPSE_LOG_RESET_REMOTE_PARTICIPANT_EC` (`DPSE_LOG_BASE` + 53)
- Failed to reset liveliness for a remote participant.*

 - #define `DPSE_LOG_INVALID_DISCOVERY_SAMPLE_EC` (`DPSE_LOG_BASE` + 54)
- Received a participant discovery announcement with invalid state.*

 - #define `DPSE_LOG_RETURN_DISCOVERY_SAMPLE_EC` (`DPSE_LOG_BASE` + 55)
- Failed to return a loan on a participant discovery announcement.*

 - #define `DPSE_LOG_SERIALIZE_MULTICAST_EC` (`DPSE_LOG_BASE` + 56)
- Failed to serialize a Multicast Locator parameter.*

 - #define `DPSE_LOG_GET_DDS_PROPERTIES_EC` (`DPSE_LOG_BASE` + 57)
- Failed to serialize a Multicast Locator parameter.*

 - #define `DPSE_TOPIC_ENTITY` 1
- A DDS Topic entity.*

 - #define `DPSE_PUBLISHER_ENTITY` 2
- A DDS Publisher entity.*

 - #define `DPSE_SUBSCRIBER_ENTITY` 3
- A DDS Subscriber entity.*

 - #define `DPSE_LOG_ENTITY_ENABLE_EC` (`DPSE_LOG_BASE` + 58)
- Failed to enable the specified entity kind.*

 - #define `DPSE_LOG_DEFAULT_UNICAST_LOCATOR_SEQUENCE` 1
- The default unicast locator sequence.*

 - #define `DPSE_LOG_DEFAULT_MULTICAST_LOCATOR_SEQUENCE` 2
- The default multicast locator sequence.*

 - #define `DPSE_LOG_META_UNICAST_LOCATOR_SEQUENCE` 3
- The meta unicast locator sequence.*

 - #define `DPSE_LOG_META_MULTICAST_LOCATOR_SEQUENCE` 4
- The meta multicast locator sequence.*

 - #define `DPSE_LOG_SEQUENCE_SETMAX_EC` (`DPSE_LOG_BASE` + 59)
- Failed to set the maximum length of a sequence of the specified kind.*

 - #define `DPSE_LOG_SEQUENCE_SETLENGTH_EC` (`DPSE_LOG_BASE` + 60)
- Failed to set the length of a sequence of the specified kind.*

 - #define `DPSE_LOG_SEQUENCE_GETREF_EC` (`DPSE_LOG_BASE` + 61)
- Failed to get a reference at the specified index for a sequence of the specified kind.*

 - #define `DPSE_LOG_SEQUENCE_INITIALIZE_EC` (`DPSE_LOG_BASE` + 62)
- Failed to initialize a sequence of the specified kind.*

 - #define `DPSE_LOG_SEQUENCE_FINALIZE_EC` (`DPSE_LOG_BASE` + 63)
- Failed to finalize a sequence of the specified kind.*

 - #define `DPSE_LOG_SEQUENCE_COPY_EC` (`DPSE_LOG_BASE` + 64)
- Failed to copy a sequence of the specified kind.*

 - #define `DPSE_LOG_GET_TIMER_EC` (`DPSE_LOG_BASE` + 65)
- Failed to get timer.*

- #define `DPSE_LOG_UNSUPPORTED_LOCATOR_EC` (`DPSE_LOG_BASE` + 66)
The locator kind is not supported by any transport registered with the domain participant and is dropped.
- #define `DPSE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC` (`DPSE_LOG_BASE` + 67)
A participant with incompatible checksum was discovered and ignored.
- #define `DPSE_LOG_SERIALIZE_PROP_EC` (`DPSE_LOG_BASE` + 68)
A Participant failed to serialize its properties.

8.10.1 Macro Definition Documentation

DPSE_KEYLIST_OBJECT

```
#define DPSE_KEYLIST_OBJECT (1)
```

The list keeping track of asserted participant keys.

DPSE_PARTICIPANTNAMES_OBJECT

```
#define DPSE_PARTICIPANTNAMES_OBJECT (2)
```

The list keeping track of asserted participant names.

DPSE_PARTICIPANTNAMEINDEX_OBJECT

```
#define DPSE_PARTICIPANTNAMEINDEX_OBJECT (3)
```

The index keeping track of asserted participant names.

DPSE_PARTICIPANTANNOUNCEMENT_OBJECT

```
#define DPSE_PARTICIPANTANNOUNCEMENT_OBJECT (4)
```

The participant announcement timeout object.

DPSE_DISCOVERYPLUGIN_OBJECT

```
#define DPSE_DISCOVERYPLUGIN_OBJECT (5)
```

A discovery plugin object.

DPSE_PARTICIPANTBUILTINTOPICDATA_OBJECT

```
#define DPSE_PARTICIPANTBUILTINTOPICDATA_OBJECT (6)
```

ParticipantBuiltinTopicData.

DPSE_PARTICIPANTENTITYNAME_OBJECT

```
#define DPSE_PARTICIPANTENTITYNAME_OBJECT (7)
```

ParticipantBuiltinTopicData entity-name object.

DPSE_ENABLEDPARTICIPANTINDEX_OBJECT

```
#define DPSE_ENABLEDPARTICIPANTINDEX_OBJECT (8)
```

The index keeping track of enabled participant names.

DPSE_PARTICIPANTPOOL_OBJECT

```
#define DPSE_PARTICIPANTPOOL_OBJECT (10)
```

The pool of asserted participants.

DPSE_LOG_CDR_LONG_KIND

```
#define DPSE_LOG_CDR_LONG_KIND 1
```

CDR Long kind.

DPSE_LOG_CDR_PID_KIND

```
#define DPSE_LOG_CDR_PID_KIND 2
```

CDR PID kind, the pid argument specifies which pid.

DPSE_TOPIC_ENTITY

```
#define DPSE_TOPIC_ENTITY 1
```

A DDS Topic entity.

DPSE_PUBLISHER_ENTITY

```
#define DPSE_PUBLISHER_ENTITY 2
```

A DDS Publisher entity.

DPSE_SUBSCRIBER_ENTITY

```
#define DPSE_SUBSCRIBER_ENTITY 3
```

A DDS Subscriber entity.

DPSE_LOG_DEFAULT_UNICAST_LOCATOR_SEQUENCE

```
#define DPSE_LOG_DEFAULT_UNICAST_LOCATOR_SEQUENCE 1
```

The default unicast locator sequence.

DPSE_LOG_DEFAULT_MULTICAST_LOCATOR_SEQUENCE

```
#define DPSE_LOG_DEFAULT_MULTICAST_LOCATOR_SEQUENCE 2
```

The default multicast locator sequence.

DPSE_LOG_META_UNICAST_LOCATOR_SEQUENCE

```
#define DPSE_LOG_META_UNICAST_LOCATOR_SEQUENCE 3
```

The meta unicast locator sequence.

DPSE_LOG_META_MULTICAST_LOCATOR_SEQUENCE

```
#define DPSE_LOG_META_MULTICAST_LOCATOR_SEQUENCE 4
```

The meta multicast locator sequence.

8.11 netio_log.h File Reference

NETIO module log codes.

```
#include "osapi/osapi_log.h"
```

Macros

- #define `NETIO_LOG_GETHOST_BYNAME_EC` (`NETIO_LOG_BASE` + 1)
Failed to get host address from a string representation.
- #define `NETIO_LOG_BIND_NEW_EC` (`NETIO_LOG_BASE` + 2)
Failed to allocate memory for a bind-resolver.
- #define `NETIO_LOG_BIND_NEW_RTABLE_EC` (`NETIO_LOG_BASE` + 3)
Failed to create a database table for routes of a bind-resolver.
- #define `NETIO_LOG_BIND_CREATE_RTABLE_EC` (`NETIO_LOG_BASE` + 4)
Failed to create route record.
- #define `NETIO_LOG_BIND_DELETE_RTABLE_EC` (`NETIO_LOG_BASE` + 5)
Failed to delete a database table for routes of a bind_resolver.
- #define `NETIO_LOG_BIND_SET_LENGTH_EC` (`NETIO_LOG_BASE` + 6)
Failed to set the length for a sequence of resolved addresses.
- #define `NETIO_LOG_BIND_SET_MAX_EC` (`NETIO_LOG_BASE` + 7)
Failed to set the maximum length of an internal sequence of addresses.
- #define `NETIO_LOG_BIND_EXTERNAL_FAILED_EC` (`NETIO_LOG_BASE` + 8)
An interface failed on bind_external.
- #define `NETIO_LOG_UNBIND_EXTERNAL_FAILED_EC` (`NETIO_LOG_BASE` + 10)
An interface failed on unbind_external.
- #define `NETIO_LOG_ROUTE_RTABLE_ALLOC_EC` (`NETIO_LOG_BASE` + 11)
Failed to allocate memory for a route resolver.
- #define `NETIO_LOG_ROUTE_RTABLE_CREATE_EC` (`NETIO_LOG_BASE` + 12)
Failed to create a database table for routes of a route-resolver.
- #define `NETIO_LOG_ROUTE_RTABLE_DELETE_EC` (`NETIO_LOG_BASE` + 13)
Failed to delete a database table for routes of a route-resolver.
- #define `NETIO_LOG_ROUTE_RTABLE_ADD_EC` (`NETIO_LOG_BASE` + 14)
Failed to create route resolver record.
- #define `NETIO_LOG_ROUTE_RTABLE_UPDATE_EC` (`NETIO_LOG_BASE` + 15)
Failed to find routes to update for a route-resolver.
- #define `NETIO_LOG_AR_ALLOC_FAILED_EC` (`NETIO_LOG_BASE` + 16)
Failed to allocate memory for address resolver.
- #define `NETIO_LOG_AR_ADD_INVALID_INTERFACE_EC` (`NETIO_LOG_BASE` + 17)
An attempt was made to add an existing interface with a different port resolver to the address resolver.
- #define `NETIO_LOG_AR_ADDRESS_RESOLVE_FAILED_EC` (`NETIO_LOG_BASE` + 18)
An interface failed to resolve an address.
- #define `NETIO_LOG_AR_PORT_RESOLVE_FAILED_EC` (`NETIO_LOG_BASE` + 19)
An interface failed to resolve a port.
- #define `NETIO_LOG_AR_CONTEXT_UNINITIALIZED_EC` (`NETIO_LOG_BASE` + 21)
NETIO_AddressResolver_resolve/get_next was called with an uninitialized context.
- #define `NETIO_LOG_AR_INVALID_TOKEN_EC` (`NETIO_LOG_BASE` + 22)
An invalid token was encountered in the address string.
- #define `NETIO_LOG_AR_ADDRESS_STRING_EXCEEDED_EC` (`NETIO_LOG_BASE` + 23)
The length of the address exceeded the address buffer passed in.
- #define `NETIO_LOG_INVALID_ADDRESS_INDEX_EC` (`NETIO_LOG_BASE` + 24)
An address index was not parsed correctly.
- #define `NETIO_LOG_ILLEGAL_ROUTE_OPERATION_EC` (`NETIO_LOG_BASE` + 25)

- Illegal route operation attempted.*

 - #define `NETIO_LOG_SET_NAME_EC` (`UDP_LOG_BASE` + 26)

Failed to set the name of a runtime component interface.
- #define `NETIO_LOG_PACKET_SET_HEAD_EC` (`UDP_LOG_BASE` + 27)

Failed to set packet's head.
- #define `NETIO_LOG_PACKET_SET_TAIL_EC` (`UDP_LOG_BASE` + 28)

Failed to set packet's tail.
- #define `NETIO_LOG_PACKET_INITIALIZE_EC` (`UDP_LOG_BASE` + 29)

Failed to initialize packet.
- #define `NETIO_LOG_PACKET_OVERFLOW_EC` (`UDP_LOG_BASE` + 30)

Packet payload length overflow detected.
- #define `NETIO_LOG_LOOP_DELETE_TABLE_EC` (`NETIO_LOG_BASE` + 100)

Failed to delete a database table for a NETIO loopback interface.
- #define `NETIO_LOG_LOOP_CREATE_TABLE_EC` (`NETIO_LOG_BASE` + 101)

Failed to create a database table for a NETIO loopback interface.
- #define `NETIO_LOG_LOOP_SELECT_TABLE_EC` (`NETIO_LOG_BASE` + 102)

Failed to select a valid record when sending with the NETIO loopback interface.
- #define `NETIO_LOG_LOOP_FWD_EC` (`NETIO_LOG_BASE` + 103)

Failed to send forward with the NETIO loopback interface.
- #define `NETIO_LOG_LOOP_BINDX_DB_EC` (`NETIO_LOG_BASE` + 104)

Failed to bind an external interface with the NETIO loopback interface.
- #define `NETIO_LOG_LOOP_UNBINDX_DB_EC` (`NETIO_LOG_BASE` + 105)

Failed to unbind an external interface with the NETIO loopback interface.
- #define `NETIO_LOG_LOOP_SET_LENGTH_EC` (`NETIO_LOG_BASE` + 106)

Failed to set the length of an address sequence.
- #define `NETIO_LOG_LOOP_INVALID_PROPERTY_EC` (`NETIO_LOG_BASE` + 107)

Invalid property passed in when creating a loopback interface.
- #define `NETIO_LOG_LOOP_INITIALIZE_FAILED_EC` (`NETIO_LOG_BASE` + 108)

Failed to initialize the loopback interface.
- #define `NETIO_LOG_LOOP_CURSOR_ERROR_EC` (`NETIO_LOG_BASE` + 109)

An error occurred with a cursor while iterating over a table.
- #define `NETIO_LOG_LOOP_INVALID_FACTORY_EC` (`NETIO_LOG_BASE` + 110)

An invalid factory was passed in to create a loopback interface.
- #define `NETIO_LOG_LOOP_INVALID_COMPONENT_EC` (`NETIO_LOG_BASE` + 111)

An invalid component was passed in to delete a loopback interface.
- #define `UDP_LOG_SOCKET_INIT_EC` (`UDP_LOG_BASE` + 200)

Failed to initialize use of sockets.
- #define `UDP_LOG_GETHOSTNAME_EC` (`UDP_LOG_BASE` + 201)

Failed to get the host address given the host name.
- #define `UDP_LOG_SOCKET_SET_MCAST_EC` (`UDP_LOG_BASE` + 202)

Failed to set a socket for multicast loopback.
- #define `UDP_LOG_SOCKET_SET_MCASTIF_EC` (`UDP_LOG_BASE` + 203)

Failed to set a socket for multicast bound to a specific interface.
- #define `UDP_LOG_SOCKET_CREATE_EC` (`UDP_LOG_BASE` + 204)

Failed to create a new socket, the sysrc code is the OS return code and reason for failure.
- #define `UDP_LOG_SOCKET_SETFD_EC` (`UDP_LOG_BASE` + 205)

Failed to set the close-on-exec flag for a socket.

- #define `UDP_LOG_SOCKET_SNDBUF_EC` (`UDP_LOG_BASE` + 206)
Failed to set the socket send buffer size.
- #define `UDP_LOG_SOCKET_TTL_EC` (`UDP_LOG_BASE` + 207)
Failed to set multicast TTL for a socket.
- #define `UDP_LOG_PACKET_INIT_EC` (`UDP_LOG_BASE` + 208)
Failed to initialize a receive packet.
- #define `UDP_LOG_PACKET_HEAD_EC` (`UDP_LOG_BASE` + 209)
Failed to set the head of a receive packet.
- #define `UDP_LOG_PACKET_FWD_EC` (`UDP_LOG_BASE` + 210)
Failed reception upstream for a received packet.
- #define `UDP_LOG_NAT_DELETE_EC` (`UDP_LOG_BASE` + 211)
Failed to delete a database record or index from an internal NAT.
- #define `UDP_LOG_FINALIZE_EC` (`UDP_LOG_BASE` + 212)
Failed to finalize the parent NETIO interface.
- #define `UDP_LOG_SOCKET_REUSE_PORT_EC` (`UDP_LOG_BASE` + 213)
Failed to set socket to reuse a port or address.
- #define `UDP_LOG_SOCKET_BIND_EC` (`UDP_LOG_BASE` + 214)
Failed a socket bind.
- #define `UDP_LOG_SOCKET_ADDGRP_EC` (`UDP_LOG_BASE` + 215)
Failed to add membership to a multicast group for a socket.
- #define `UDP_LOG_PORT_ENTRY_EC` (`UDP_LOG_BASE` + 216)
Failed to create a bind entry for a receive port.
- #define `UDP_LOG_PORT_ALLOC_EC` (`UDP_LOG_BASE` + 217)
Failed to allocate memory for a receive buffer of a specific port.
- #define `UDP_LOG_SOCKET_RXBUF_EC` (`UDP_LOG_BASE` + 218)
Failed to set the receive buffer size of a socket.
- #define `UDP_LOG_PORT_ADD_EC` (`UDP_LOG_BASE` + 219)
Failed to add a port entry record to a database table.
- #define `UDP_LOG_PORT_THREAD_EC` (`UDP_LOG_BASE` + 220)
Failed to create a receive thread for a receive port.
- #define `UDP_LOG_SOCKET_IFLIST_EC` (`UDP_LOG_BASE` + 221)
Failed to get a list of interface addresses.
- #define `UDP_LOG_SOCKET_IFFLAGS_EC` (`UDP_LOG_BASE` + 222)
Failed to get the network interface flags or mask of a socket.
- #define `UDP_LOG_SOCKET_CLOSE_EC` (`UDP_LOG_BASE` + 223)
Failed to close a socket.
- #define `UDP_LOG_LOADLIB_EC` (`UDP_LOG_BASE` + 224)
Failed to load a dynamic library.
- #define `UDP_LOG_GET_BUF_SIZE_EC` (`UDP_LOG_BASE` + 225)
Failed to get interface buffer size.
- #define `UDP_LOG_GET_BUF_ALLOC_EC` (`UDP_LOG_BASE` + 226)
Out of memory to allocate interface buffer.
- #define `UDP_LOG_GET_IFLIST_EC` (`UDP_LOG_BASE` + 227)
Failed to get an interface list.
- #define `UDP_LOG_GET_NAMEINFO_EC` (`UDP_LOG_BASE` + 228)
Failed getnameinfo()
- #define `UDP_LOG_MULTICAST_ENABLE_EC` (`UDP_LOG_BASE` + 229)

- Failed to enable multicast loopback.*

 - #define `UDP_LOG_UDP_CREATE_TABLE_EC` (`UDP_LOG_BASE` + 230)
- Failed to create a database table for a UDP interface.*

 - #define `UDP_LOG_UDP_CREATE_INDEX_EC` (`UDP_LOG_BASE` + 231)
- Failed to create an index for a UDP interface.*

 - #define `UDP_LOG_RECORD_EC` (`UDP_LOG_BASE` + 232)
- Failed to create or insert a database record for a UDP interface.*

 - #define `UDP_LOG_PROPERTY_FINALIZE_EC` (`UDP_LOG_BASE` + 233)
- Failed to finalize UDP interface factory property.*

 - #define `UDP_LOG_ALLOC_EC` (`UDP_LOG_BASE` + 234)
- Failed to allocate memory for a UDP interface.*

 - #define `UDP_LOG_SEND_PING_EC` (`UDP_LOG_BASE` + 235)
- Failed sending a ping message, upon adding a route or waking up a receive thread.*

 - #define `UDP_LOG_DROPGRP_EC` (`UDP_LOG_BASE` + 236)
- Failed to drop membership from a multicast group.*

 - #define `UDP_LOG_GET_LENGTH_EC` (`UDP_LOG_BASE` + 237)
- Length of address sequence exceeds its maximum.*

 - #define `UDP_LOG_SET_LENGTH_EC` (`UDP_LOG_BASE` + 240)
- Failed to set the length of an address sequence.*

 - #define `UDP_LOG_WSA_STARTUP_EC` (`UDP_LOG_BASE` + 241)
- Failed WSStartup()*

 - #define `UDP_LOG_WINSOCK_INCOMPATIBLE_EC` (`UDP_LOG_BASE` + 242)
- Incompatible version of Winsock, must not be older than 2.0.*

 - #define `UDP_LOG_INCONSISTENT_MAX_MESSAGE_SIZE_EC` (`UDP_LOG_BASE` + 243)
- Inconsistent max_message_size.*

 - #define `UDP_LOG_INITIALIZE_FAILED_EC` (`UDP_LOG_BASE` + 244)
- Failed to initialize the loopback interface.*

 - #define `UDP_LOG_PORT_NOT_FOUND_EC` (`UDP_LOG_BASE` + 245)
- Did not find the specified thread port.*

 - #define `UDP_LOG_CURSOR_ERROR_EC` (`UDP_LOG_BASE` + 246)
- An error occurred while iterating over a cursor.*

 - #define `UDP_LOG_SEND_ERROR_EC` (`UDP_LOG_BASE` + 247)
- Failed to send message.*

 - #define `UDP_LOG_MULTICAST_NOT_ENABLED_EC` (`UDP_LOG_BASE` + 248)
- Multicast address ignored because multicast supported is not compiled in.*

 - #define `UDP_LOG_TRANSPORT_NO_INTERFACES_EC` (`UDP_LOG_BASE` + 249)
- Transport does not have any interfaces.*

 - #define `UDP_LOG_RECV_ERROR_EC` (`UDP_LOG_BASE` + 250)
- Failed to receive message.*

 - #define `UDP_LOG_SOCKET_PRIORITY_EC` (`UDP_LOG_BASE` + 251)
- Failed to set socket priority.*

 - #define `UDP_LOG_TOS_NOT_SUPPORTED_EC` (`UDP_LOG_BASE` + 252)
- IP_TOS is not supported.*

 - #define `UDP_TRANSFORM_LOG_KIND_RULE` (1)
- Rule object.*

 - #define `UDP_TRANSFORM_LOG_KIND_ENTRY` (2)
- Entry object.*

- `#define UDP_TRANSFORM_LOG_KIND_FACTORY` (3)
Factory object.
- `#define UDP_TRANSFORM_LOG_KIND_FACTORY_INDEX` (4)
Factory object.
- `#define UDP_TRANSFORM_LOG_KIND_DST_INDEX` (5)
Factory object.
- `#define UDP_TRANSFORM_LOG_KIND_SRC_INDEX` (6)
Factory object.
- `#define UDP_TRANSFORM_LOG_KIND_TRANSFORM_SOURCE` (7)
Rule object.
- `#define UDP_TRANSFORM_LOG_KIND_TRANSFORM_DESTINATION` (8)
Entry object.
- `#define UDP_TRANSFORM_LOG_KIND_TRANSFORM_NETMASK_INDEX` (9)
Mask object.
- `#define UDP_TRANSFORM_LOG_INVALID_FACTORY_REFERENCE_EC` (`UDP_LOG_BASE` + 300)
The returned factory reference is NULL.
- `#define UDP_TRANSFORM_LOG_FACTORY_NOT_FOUND_EC` (`UDP_LOG_BASE` + 301)
Could not find the factory used as part of a transform rule.
- `#define UDP_TRANSFORM_LOG_ALLOC_EC` (`UDP_LOG_BASE` + 302)
Failed to allocate entry of the specified kind.
- `#define UDP_TRANSFORM_LOG_CREATE_EC` (`UDP_LOG_BASE` + 303)
Failed to create a transformation of the specified kind.
- `#define UDP_TRANSFORM_LOG_ADD_EC` (`UDP_LOG_BASE` + 304)
Failed to add object of specified kind.
- `#define UDP_TRANSFORM_LOG_FIND_EC` (`UDP_LOG_BASE` + 305)
Failed to find a transformation.
- `#define UDP_TRANSFORM_LOG_DUPLICATE_EC` (`UDP_LOG_BASE` + 306)
Duplicate object.
- `#define UDP_TRANSFORM_LOG_TRANSFORM_EC` (`UDP_LOG_BASE` + 307)
Source or destination transformation function failed.
- `#define UDP_TRANSFORM_LOG_INVALID_UDP_MODE_EC` (`UDP_LOG_BASE` + 308)
Invalid UDP mode specified.
- `#define UDP_TRANSFORM_LOG_INVALID_TUDP_LOCATOR_KIND_EC` (`UDP_LOG_BASE` + 309)
Invalid transformation UDP locator kind specified.
- `#define UDP_PRIORITY_MAP_ALLOC_EC` (`UDP_LOG_BASE` + 310)
Failed to allocate memory for priority map.
- `#define UDP_PRIORITY_MAP_INVALID_MAPPING_EC` (`UDP_LOG_BASE` + 311)
Invalid priority mapping specified.
- `#define UDP_PRIORITY_MAP_NEGATIVE_MAPPING_EC` (`UDP_LOG_BASE` + 312)
Negative priority mapping values specified.
- `#define UDP_PRIORITY_MAP_PRIORITY_IGNORED_EC` (`UDP_LOG_BASE` + 313)
Transport priority ignored.

8.11.1 Detailed Description

NETIO module log codes.

8.11.2 Macro Definition Documentation

UDP_TRANSFORM_LOG_KIND_RULE

```
#define UDP_TRANSFORM_LOG_KIND_RULE (1)
```

Rule object.

UDP_TRANSFORM_LOG_KIND_ENTRY

```
#define UDP_TRANSFORM_LOG_KIND_ENTRY (2)
```

Entry object.

UDP_TRANSFORM_LOG_KIND_FACTORY

```
#define UDP_TRANSFORM_LOG_KIND_FACTORY (3)
```

Factory object.

UDP_TRANSFORM_LOG_KIND_FACTORY_INDEX

```
#define UDP_TRANSFORM_LOG_KIND_FACTORY_INDEX (4)
```

Factory object.

UDP_TRANSFORM_LOG_KIND_DST_INDEX

```
#define UDP_TRANSFORM_LOG_KIND_DST_INDEX (5)
```

Factory object.

UDP_TRANSFORM_LOG_KIND_SRC_INDEX

```
#define UDP_TRANSFORM_LOG_KIND_SRC_INDEX (6)
```

Factory object.

UDP_TRANSFORM_LOG_KIND_TRANSFORM_SOURCE

```
#define UDP_TRANSFORM_LOG_KIND_TRANSFORM_SOURCE (7)
```

Rule object.

UDP_TRANSFORM_LOG_KIND_TRANSFORM_DESTINATION

```
#define UDP_TRANSFORM_LOG_KIND_TRANSFORM_DESTINATION (8)
```

Entry object.

UDP_TRANSFORM_LOG_KIND_TRANSFORM_NETMASK_INDEX

```
#define UDP_TRANSFORM_LOG_KIND_TRANSFORM_NETMASK_INDEX (9)
```

Mask object.

UDP_PRIORITY_MAP_PRIORITY_IGNORED_EC

```
#define UDP_PRIORITY_MAP_PRIORITY_IGNORED_EC (UDP_LOG_BASE + 313)
```

Transport priority ignored.

The DataReader or DataWriter transport priority is ignored by the UDP transport because the priority mask is not configured.

8.12 netio_shmem_log.h File Reference

NETIO Zero Copy Shared Data module log codes.

```
#include "osapi/osapi_log.h"
```

Macros

- #define [NETIO_SHMEM_LOG_DELETE_TABLE_EC](#) ([SHMEM_LOG_BASE](#) + 1)
Failed to delete a database table for a NETIO shared memory interface.
- #define [NETIO_SHMEM_LOG_CREATE_TABLE_EC](#) ([SHMEM_LOG_BASE](#) + 2)
Failed to create a database table for a NETIO shared memory interface.
- #define [NETIO_SHMEM_LOG_SELECT_TABLE_EC](#) ([SHMEM_LOG_BASE](#) + 3)
Failed to select a valid record when sending with the NETIO shared memory.
- #define [NETIO_SHMEM_LOG_FWD_EC](#) ([SHMEM_LOG_BASE](#) + 4)
Failed to send forward with the NETIO shared memory interface.
- #define [NETIO_SHMEM_LOG_BINDX_DB_EC](#) ([SHMEM_LOG_BASE](#) + 5)
Failed to bind an external interface with the NETIO shared memory interface.
- #define [NETIO_SHMEM_LOG_UNBINDX_DB_EC](#) ([SHMEM_LOG_BASE](#) + 6)
Failed to unbind an external interface with the NETIO shared memory interface.
- #define [NETIO_SHMEM_LOG_SET_LENGTH_EC](#) ([SHMEM_LOG_BASE](#) + 7)
Failed to set the length of an address sequence.
- #define [NETIO_SHMEM_LOG_INVALID_PROPERTY_EC](#) ([SHMEM_LOG_BASE](#) + 8)

- Invalid property passed in when creating a shared memory interface.*

 - #define NETIO_SHMEM_LOG_INITIALIZE_FAILED_EC (SHMEM_LOG_BASE + 9)
- Failed to initialize the shared memory interface.*

 - #define NETIO_SHMEM_LOG_CURSOR_ERROR_EC (SHMEM_LOG_BASE + 10)
- An error occurred with a cursor while iterating over a table.*

 - #define NETIO_SHMEM_LOG_INVALID_FACTORY_EC (SHMEM_LOG_BASE + 11)
- An invalid factory was passed in to create a shared memory interface.*

 - #define NETIO_SHMEM_LOG_INVALID_COMPONENT_EC (SHMEM_LOG_BASE + 12)
- An invalid component was passed in to delete a shared memory interface.*

 - #define NETIO_SHMEM_LOG_INCOMPATIBLE_COOKIE_EC (SHMEM_LOG_BASE + 13)
- Found incompatible cookie in shared memory header.*

 - #define NETIO_SHMEM_LOG_INCOMPATIBLE_VERSION_EC (SHMEM_LOG_BASE + 14)
- Found incompatible major version in shared memory header.*

 - #define NETIO_SHMEM_LOG_INCOMPATIBLE_HEADER_SIZE_EC (SHMEM_LOG_BASE + 15)
- Found incompatible header size in shared memory header.*

 - #define NETIO_SHMEM_LOG_INCOMPATIBLE_HEADER_EC (SHMEM_LOG_BASE + 16)
- Attempted to attach to shared memory header transport concurrent queue that is incompatible.*

 - #define NETIO_SHMEM_LOG_ADDRESS_IN_USE_EC (SHMEM_LOG_BASE + 17)
- Failed to create or attach to a shared memory segment.*

 - #define NETIO_SHMEM_LOG_CHECK_SYSTEM_SETTINGS_EC (SHMEM_LOG_BASE + 18)
- Possible failure due to misconfigured system settings on Darwin operating systems.*

 - #define NETIO_SHMEM_LOG_SHMEM_MUTEX_LOCK_FAILED_EC (SHMEM_LOG_BASE + 19)
- Failed to lock shared memory mutex protecting the shared memory segment.*

 - #define NETIO_SHMEM_LOG_SHMEM_MUTEX_UNLOCK_FAILED_EC (SHMEM_LOG_BASE + 20)
- Failed to unlock shared memory mutex protecting the shared memory segment.*

 - #define NETIO_SHMEM_LOG_FAILED_TO_ALLOCATE_STRUCT_EC (SHMEM_LOG_BASE + 21)
- Failed to allocate memory needed by shared memory transport.*

 - #define NETIO_SHMEM_LOG_FAILED_TO_INITIALIZE_EC (SHMEM_LOG_BASE + 22)
- Failed to initialize shared memory transport.*

 - #define NETIO_SHMEM_LOG_CREATE_ATTACH_INFINITE_EC (SHMEM_LOG_BASE + 23)
- Failed to create or attach. Possible shared memory key conflict.*

 - #define NETIO_SHMEM_LOG_FAILED_TO_INIT_MUTEX_EC (SHMEM_LOG_BASE + 24)
- Failed to create a new shared memory mutex or failed to attach to an existing one.*

 - #define NETIO_SHMEM_LOG_FAILED_TO_INIT_SIG_SEM_EC (SHMEM_LOG_BASE + 25)
- Failed to create signaling semaphore. A Domain Participant's shared memory transport blocks on a signaling semaphore when waiting for data.*

 - #define NETIO_SHMEM_LOG_FAILED_TAKE_SIGN_SEM_EC (SHMEM_LOG_BASE + 27)
- Failed to take a shared memory signaling semaphore.*

 - #define NETIO_SHMEM_LOG_SIG_SEM_SIGNAL_FAILED_EC (SHMEM_LOG_BASE + 28)
- Failed to give a shared memory signaling semaphore.*

 - #define NETIO_SHMEM_LOG_FAILED_TO_INIT_SEGMENT_ADDR_EC (SHMEM_LOG_BASE + 29)
- Failed to attach to a shared memory segment because it doesn't exist.*

 - #define NETIO_SHMEM_LOG_PACKET_INIT_EC (SHMEM_LOG_BASE + 30)
- Failed to initialize a receive packet.*

 - #define NETIO_SHMEM_LOG_FAILED_TO_INIT_CONCURRENT_Q_EC (SHMEM_LOG_BASE + 31)
- Failed to create a concurrent queue.*

 - #define NETIO_SHMEM_LOG_FAILED_TO_ATTACH_CONCURRENT_Q_EC (SHMEM_LOG_BASE + 32)

- Failed to attach to a concurrent queue.*

 - #define `NETIO_SHMEM_LOG_INCOMPATIBLE_CONCURRENT_Q_EC` (`SHMEM_LOG_BASE` + 33)
- Attempting to attach to an incompatible concurrent queue with incompatible message_size_max.*

 - #define `NETIO_SHMEM_LOG_FAILED_TO_GET_TIMESTAMP_EC` (`SHMEM_LOG_BASE` + 33)
- Failed to get timestamp.*

 - #define `NETIO_SHMEM_LOG_NONFATAL_SIG_SEM_RC_EC` (`SHMEM_LOG_BASE` + 34)
- Failed to unlock shared memory mutex.*

 - #define `NETIO_SHMEM_LOG_INCONSISTENT_MESSAGE_SIZE_EC` (`SHMEM_LOG_BASE` + 35)
- Failed to unlock shared memory mutex.*

 - #define `NETIO_SHMEM_LOG_RECORD_EC` (`SHMEM_LOG_BASE` + 36)
- Database operation on record failed.*

 - #define `NETIO_SHMEM_LOG_PACKET_HEAD_EC` (`SHMEM_LOG_BASE` + 37)
- Failed to set packet head.*

 - #define `NETIO_SHMEM_LOG_PACKET_FWD_EC` (`SHMEM_LOG_BASE` + 38)
- Failed reception upstream for a received packet.*

 - #define `NETIO_SHMEM_LOG_QUEUE_FULL_EC` (`SHMEM_LOG_BASE` + 39)
- Failed reception upstream for a received packet.*

 - #define `NETIO_SHMEM_LOG_ROUTE_LOOKUP_FAILED_EC` (`SHMEM_LOG_BASE` + 40)
- Failed to evict find an entry in the route table.*

 - #define `NETIO_SHMEM_LOG_ROUTE_DELETE_FAILED_EC` (`SHMEM_LOG_BASE` + 41)
- Failed to delete an entry from the route table.*

 - #define `NETIO_SHMEM_LOG_IGNORE_TRANSPORT_COMPATIBILITY_EC` (`SHMEM_LOG_BASE` + 42)
- The shared memory transport is ignoring the requested transport minimum compatibility version because it cannot be backward compatible.*

 - #define `NETIO_SHMEM_LOG_UNSUPPORTED_TRANSPORT_COMPATIBILITY_EC` (`SHMEM_LOG_BASE` + 43)
- Unsupported minimum compatibility version because it requires a version of the shared memory transport which is not supported.*

 - #define `NETIO_SHMEM_LOG_CQ_INT_REPRESENTATION_EC` (`SHMEM_LOG_BASE` + 200)
- Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.*

 - #define `NETIO_SHMEM_LOG_CQ_UNLINKED_EC` (`SHMEM_LOG_BASE` + 201)
- Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.*

 - #define `NETIO_SHMEM_LOG_CQ_INVALID_SIGNATURE_EC` (`SHMEM_LOG_BASE` + 202)
- Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.*

 - #define `NETIO_SHMEM_LOG_CQ_INCOMPATIBLE_VERSION_EC` (`SHMEM_LOG_BASE` + 203)
- Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.*

 - #define `NETIO_SHMEM_LOG_CQ_INCONSISTENT_STATE_EC` (`SHMEM_LOG_BASE` + 204)
- Found an inconsistent concurrent queue state while performing a a read or write. The queue memory may have been corrupted or otherwise tampered with.*

8.12.1 Detailed Description

NETIO Zero Copy Shared Data module log codes.

8.13 netio_zcopy_log.h File Reference

ZeroCopy module log codes.

```
#include "osapi/osapi_log.h"
```

Macros

- #define [ZCOPY_LOG_NOTIF_GUID_BUFFER_TOO_SMALL_EC](#) (NETIO_ZCOPY_LOG_BASE + 1)
Failed to convert GUID to string representation.
- #define [ZCOPY_LOG_NOTIF_ALREADY_CONNECTED_EC](#) (NETIO_ZCOPY_LOG_BASE + 2)
Failed to connect because already connected.
- #define [ZCOPY_LOG_NOTIF_NOT_CONNECTED_EC](#) (NETIO_ZCOPY_LOG_BASE + 3)
Failed to perform operation because notif is not connected.
- #define [ZCOPY_LOG_NOTIF_INCOMPATIBLE_VERSION_EC](#) (NETIO_ZCOPY_LOG_BASE + 4)
Failed to connect because of incompatible ZCV2 version.
- #define [ZCOPY_LOG_NOTIF_SLOT_ALREADY_ASSIGNED_EC](#) (NETIO_ZCOPY_LOG_BASE + 5)
Failed to assign a slot because it is already assigned.
- #define [ZCOPY_LOG_NOTIF_NO_FREE_SLOT_EC](#) (NETIO_ZCOPY_LOG_BASE + 6)
Failed to assign a slot because there is no free slot.
- #define [ZCOPY_LOG_NOTIF_INVALID_NUM_SLOTS_EC](#) (NETIO_ZCOPY_LOG_BASE + 7)
Failed to connect because of invalid number of slots.
- #define [ZCOPY_LOG_NOTIF_INVALID_PORT_NUMBER_EC](#) (NETIO_ZCOPY_LOG_BASE + 8)
Failed to convert GUID to string representation because of an invalid port number.
- #define [ZCOPY_LOG_NOTIF_OPEN_FAILED_EC](#) (NETIO_ZCOPY_LOG_BASE + 9)
Failed to open notification interface.
- #define [ZCOPY_LOG_NOTIF_DB_SELECT_RECORD_EC](#) (NETIO_ZCOPY_LOG_BASE + 10)
Notification interface failed to select a database record.
- #define [ZCOPY_LOG_NOTIF_DB_CREATE_RECORD_EC](#) (NETIO_ZCOPY_LOG_BASE + 11)
Notification interface failed to create a database record.
- #define [ZCOPY_LOG_NOTIF_DB_INSERT_RECORD_EC](#) (NETIO_ZCOPY_LOG_BASE + 12)
Notification interface failed to insert a database record.
- #define [ZCOPY_LOG_NOTIF_DB_REMOVE_RECORD_EC](#) (NETIO_ZCOPY_LOG_BASE + 13)
Notification interface failed to remove a database record.
- #define [ZCOPY_LOG_NOTIF_DB_DELETE_RECORD_EC](#) (NETIO_ZCOPY_LOG_BASE + 14)
Notification interface failed to delete a database record.
- #define [ZCOPY_LOG_NOTIF_DB_CURSOR_NEXT_EC](#) (NETIO_ZCOPY_LOG_BASE + 15)
Notification interface failed to get the next database record from a cursor.
- #define [ZCOPY_LOG_NOTIF_DB_LOCK_EC](#) (NETIO_ZCOPY_LOG_BASE + 16)
Notification interface failed to lock or unlock its database.
- #define [ZCOPY_LOG_NOTIF_DB_TABLE_CREATE_EC](#) (NETIO_ZCOPY_LOG_BASE + 17)
Notification interface failed to create a table.
- #define [ZCOPY_LOG_NOTIF_DB_TABLE_DELETE_EC](#) (NETIO_ZCOPY_LOG_BASE + 18)
Notification interface failed to delete a table.
- #define [ZCOPY_LOG_NOTIF_INVALID_PROPERTY_EC](#) (NETIO_ZCOPY_LOG_BASE + 19)
Notification interface initialized with an invalid property.

- #define ZCOPY_LOG_NOTIF_SQ_OPEN_EC (NETIO_ZCOPY_LOG_BASE + 20)
Notification interface failed to open a shared queue reader.
- #define ZCOPY_LOG_NOTIF_SQ_CLOSE_EC (NETIO_ZCOPY_LOG_BASE + 21)
Notification interface failed to close a shared queue reader.
- #define ZCOPY_LOG_NOTIF_NO_MORE_NOTIFIERS_EC (NETIO_ZCOPY_LOG_BASE + 22)
Notification interface ran out of notifiers in its pool of notifiers.
- #define ZCOPY_LOG_NOTIF_NOTIFIER_CONNECT_EC (NETIO_ZCOPY_LOG_BASE + 23)
Notification interface failed to connect a notifier to a notifiee.
- #define ZCOPY_LOG_NOTIF_NOTIFIER_DISCONNECT_EC (NETIO_ZCOPY_LOG_BASE + 24)
Notification interface failed to disconnect a notifier from a notifiee.
- #define ZCOPY_LOG_NOTIF_NOTIFIER_NOTIFY_EC (NETIO_ZCOPY_LOG_BASE + 25)
Notification interface failed to notify with a notifier.
- #define ZCOPY_LOG_NOTIF_NOTIFIER_ALLOW_EC (NETIO_ZCOPY_LOG_BASE + 26)
Notification interface failed to allow a notifier to notify.
- #define ZCOPY_LOG_NOTIF_NOTIFIEE_DESTROY_EC (NETIO_ZCOPY_LOG_BASE + 27)
Notification interface failed to destroy a notifiee.
- #define ZCOPY_LOG_NOTIF_NOTIFIEE_RAISE_FLAG_EC (NETIO_ZCOPY_LOG_BASE + 28)
Notification interface failed to raise a flag.
- #define ZCOPY_LOG_NOTIF_NOTIFIEE_NEXT_EC (NETIO_ZCOPY_LOG_BASE + 29)
Notification interface failed to get next notification.
- #define ZCOPY_LOG_NOTIF_THREAD_CREATE_EC (NETIO_ZCOPY_LOG_BASE + 30)
Notification interface failed to create a receive thread.
- #define ZCOPY_LOG_NOTIF_THREAD_DESTROY_EC (NETIO_ZCOPY_LOG_BASE + 31)
Notification interface failed to destroy a receive thread.
- #define ZCOPY_LOG_NOTIF_INTF_RECEIVE_EC (NETIO_ZCOPY_LOG_BASE + 32)
Notification interface failed to forward a packet upstream to a DRI.
- #define ZCOPY_LOG_NOTIF_PORT_IN_USE_EC (NETIO_ZCOPY_LOG_BASE + 33)
Notification interface tried to release a port that was still in use.
- #define ZCOPY_LOG_NOTIF_ALLOC_EC (NETIO_ZCOPY_LOG_BASE + 34)
Notification interface factory failed to allocate memory.
- #define ZCOPY_LOG_NOTIF_INTF_INIT_EC (NETIO_ZCOPY_LOG_BASE + 35)
Notification interface factory failed to initialize a notification interface.
- #define ZCOPY_LOG_NOTIF_INTF_FINALIZE_EC (NETIO_ZCOPY_LOG_BASE + 36)
Notification interface factory failed to finalize a notification interface.
- #define ZCOPY_LOG_NOTIF_FACTORY_IN_USE_EC (NETIO_ZCOPY_LOG_BASE + 37)
Tried to finalize a notification interface factory which was still in use.
- #define ZCOPY_LOG_NOTIF_MECHANISM_CREATE_INST_EC (NETIO_ZCOPY_LOG_BASE + 38)
Notification mechanism failed to create instance.
- #define ZCOPY_LOG_NOTIF_MECHANISM_RESERVE_ADDR_EC (NETIO_ZCOPY_LOG_BASE + 39)
Notification mechanism failed to reserve an address.
- #define ZCOPY_LOG_NOTIF_MECHANISM_RELEASE_ADDR_EC (NETIO_ZCOPY_LOG_BASE + 40)
Notification mechanism failed to release an address.
- #define ZCOPY_LOG_NOTIF_MECHANISM_ADD_ROUTE_EC (NETIO_ZCOPY_LOG_BASE + 41)
Notification mechanism failed to add route.
- #define ZCOPY_LOG_NOTIF_MECHANISM_DELETE_ROUTE_EC (NETIO_ZCOPY_LOG_BASE + 42)
Notification mechanism failed to delete route.
- #define ZCOPY_LOG_NOTIF_MECHANISM_BIND_EC (NETIO_ZCOPY_LOG_BASE + 43)

- *Notification mechanism failed to bind.*
- #define ZCOPY_LOG_NOTIF_MECHANISM_UNBIND_EC (NETIO_ZCOPY_LOG_BASE + 44)
- *Notification mechanism failed to unbind.*
- #define ZCOPY_LOG_NOTIF_MECHANISM_SEND_EC (NETIO_ZCOPY_LOG_BASE + 45)
- *Notification mechanism failed to send.*
- #define ZCOPY_LOG_WH_REGISTER_FAILED_EC (NETIO_ZCOPY_LOG_BASE + 46)
- *Writer history wrap failed.*
- #define ZCOPY_LOG_NOTIF_TIMER_CREATE_EC (NETIO_ZCOPY_LOG_BASE + 47)
- *Timer creation failed.*
- #define ZCOPY_LOG_NOTIF_RECEIVE_ON_EVENT_EC (NETIO_ZCOPY_LOG_BASE + 48)
- *Receive on Writer match event failed.*
- #define ZCOPY_LOG_NOTIF_TIMEOUT_DELETE_EC (NETIO_ZCOPY_LOG_BASE + 49)
- *Failed to delete timeout.*
- #define ZCOPY_LOG_NOTIF_TIMEOUT_CREATE_EC (NETIO_ZCOPY_LOG_BASE + 50)
- *Timeout creation failed.*
- #define ZCOPY_LOG_ADD_ENTRY_EC (NETIO_ZCOPY_LOG_BASE + 51)
- *failed to add an indexer entry*
- #define ZCOPY_LOG_REMOVE_ENTRY_EC (NETIO_ZCOPY_LOG_BASE + 52)
- *failed to remove an indexer entry*
- #define ZCOPY_LOG_DELETE_POOL_EC (NETIO_ZCOPY_LOG_BASE + 53)
- *The memory manager was not able to delete a resource.*
- #define ZCOPY_LOG_SHARED_MEM_POOL_OWNER_DEAD_EC (NETIO_ZCOPY_LOG_BASE + 54)
- *A shared memory pool is no longer usable because a process died while holding the pool's lock.*

8.13.1 Detailed Description

ZeroCopy module log codes.

Log codes of the ZeroCopy module

8.14 netio_zcopy_notif_interface.h File Reference

```
#include "rt/rt_rt.h"
#include "netio_zcopy/netio_zcopy_notif_mechanism_intf.h"
```

Classes

- struct ZCOPY_NotifInterfaceFactoryProperty
- *Notification interface property.*

Macros

- #define ZCOPY_NotifInterfaceFactoryProperty_INITIALIZER
- *Constant to initialize a ZCOPY_NotifInterfaceFactoryProperty.*

Functions

- [RTI_BOOL ZCOPY_NotifInterfaceFactory_register](#) (RT_Registry_T *registry, const char *name, struct ZCOPY_NotifInterfaceFactoryProperty *property)
Register the notification interface factory with the registry.
- [RTI_BOOL ZCOPY_NotifInterfaceFactory_unregister](#) (RT_Registry_T *registry, const char *name)
Unregister the notification interface factory.

8.14.1 Function Documentation

ZCOPY_NotifInterfaceFactory_register()

```
RTI_BOOL ZCOPY_NotifInterfaceFactory_register (
    RT_Registry_T * registry,
    const char * name,
    struct ZCOPY_NotifInterfaceFactoryProperty * property) [extern]
```

Register the notification interface factory with the registry.

Parameters

in	<i>registry</i>	The registry used to register the factory.
in	<i>name</i>	The name used to register the factory.
in	<i>property</i>	The property used to register the factory.

Returns

RTI_TRUE if the factory was registered successfully, RTI_FALSE otherwise.

ZCOPY_NotifInterfaceFactory_unregister()

```
RTI_BOOL ZCOPY_NotifInterfaceFactory_unregister (
    RT_Registry_T * registry,
    const char * name) [extern]
```

Unregister the notification interface factory.

Parameters

in	<i>registry</i>	The registry used to unregister the factory.
in	<i>name</i>	The name used to unregister the factory.

Returns

RTI_TRUE if the factory was unregistered successfully, RTI_FALSE otherwise.

8.15 netio_zcopy_whsq_log.h File Reference

WHSQ module log codes.

```
#include "osapi/osapi_log.h"
```

Macros

- #define [WHSQ_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC](#) (WHSQ_LOG_BASE + 1)
KEEP_ALL History kind is not supported.
- #define [WHSQ_LOG_UNLIMITED_HISTORY_NOT_SUPPORTED_EC](#) (WHSQ_LOG_BASE + 2)
Unlimited length resource limits unsupported.
- #define [WHSQ_LOG_MAX_SAMPLES_TOO_SMALL_EC](#) (WHSQ_LOG_BASE + 3)
DataWriterQos.resource_limits.max_samples set too small.
- #define [WHSQ_LOG_HISTORY_OBJECT](#) 1
A history object.
- #define [WHSQ_LOG_OBJECT_ALLOCATE_EC](#) (WHSQ_LOG_BASE + 4)
Failed to allocate object of the specified kind.
- #define [WHSQ_LOG_PURGE_FAILED_EC](#) (WHSQ_LOG_BASE + 5)
Failed to purge a sample from the shared queue.
- #define [WHSQ_LOG_WH_CREATE_FAILED_EC](#) (WHSQ_LOG_BASE + 6)
Failed to create the underlying writer history.
- #define [WHSQ_LOG_OBJECT_DELETE_EC](#) (WHSQ_LOG_BASE + 7)
Failed to delete object of the specified kind.
- #define [WHSQ_LOG_CONFIGURATION_EC](#) (WHSQ_LOG_BASE + 8)
Listener or Property not configured when creating the component.
- #define [WHSQ_LOG_NO_DDS_FACTORY_EC](#) (WHSQ_LOG_BASE + 9)
No factory for forwarding when creating a writer history instance.
- #define [WHSQ_LOG_COMMIT_FAILED_EC](#) (WHSQ_LOG_BASE + 10)
Committing a Sample failed.

8.15.1 Detailed Description

WHSQ module log codes.

8.15.2 Macro Definition Documentation

WHSQ_LOG_HISTORY_OBJECT

```
#define WHSQ_LOG_HISTORY_OBJECT 1
```

A history object.

8.16 osapi_log.h File Reference

OSAPI Log API.

```
#include "osapi/osapi_dll.h"
#include "osapi/osapi_heap.h"
#include "osapi/osapi_time.h"
#include "osapi/osapi_log_impl.h"
```

Classes

- struct [OSAPI_LogProperty](#)
Configuration of logging functionality.

Macros

- #define [OSAPI_LOG_BASE](#) (0)
Log Id base-number.
- #define [REDA_LOG_BASE](#) (1 << 16)
REDA Module.
- #define [DB_LOG_BASE](#) (2 << 16)
DB Module.
- #define [RT_LOG_BASE](#) (3 << 16)
RT Module.
- #define [NETIO_LOG_BASE](#) (4 << 16)
NETIO Module.
- #define [UDP_LOG_BASE](#) [NETIO_LOG_BASE](#)
UDP Module, same as NETIO.
- #define [SHMEM_LOG_BASE](#) [NETIO_LOG_BASE](#) + 10000
NETIO_SHMEM Module, same as NETIO.
- #define [SDM_LOG_BASE](#) [NETIO_LOG_BASE](#) + 20000
SDM Module, same as NETIO.
- #define [CDR_LOG_BASE](#) (5 << 16)
CDR Module.
- #define [XCDR_LOG_BASE](#) [CDR_LOG_BASE](#) + 20000
CDR Module.
- #define [RTPS_LOG_BASE](#) (6 << 16)
RTPS Module.
- #define [DDSC_LOG_BASE](#) (7 << 16)
DDS_C Module.
- #define [RHSM_LOG_BASE](#) (8 << 16)
RHSM Module.
- #define [WHSM_LOG_BASE](#) (9 << 16)
WHSM Module.
- #define [DPSE_LOG_BASE](#) (10 << 16)

- *DPSE Module.*
- #define `DPDE_LOG_BASE` (11 << 16)
- *DPDE Module.*
- #define `SEC_CORE_LOG_BASE` (12 << 16)
- *DDDS Security plugin.*
- #define `APPGEN_LOG_BASE` (13 << 16)
- *APPGEN Module.*
- #define `NETIO_ZCOPY_LOG_BASE` (14 << 16)
- *NETIO Zero Copy Module.*
- #define `DDS_FILTER_LOG_BASE` (15 << 16)
- *DDS Filter Module.*
- #define `OSAPI_LOG_GET_NEXT_OBJECT_ID_EC` (`OSAPI_LOG_BASE` + 1)
- *Retrieve the next error code failed.*
- #define `OSAPI_LOG_SYSTEM_SET_PROPERTY_EC` (`OSAPI_LOG_BASE` + 2)
- *An error occurred while setting the system properties.*
- #define `OSAPI_LOG_SYSTEM_TIMER_START_EC` (`OSAPI_LOG_BASE` + 4)
- *An error occurred when starting the system timer.*
- #define `OSAPI_LOG_SYSTEM_TIMER_STOP_EC` (`OSAPI_LOG_BASE` + 5)
- *An error occurred when stopping the system timer.*
- #define `OSAPI_LOG_THREAD_NEW_EC` (`OSAPI_LOG_BASE` + 6)
- *An error when allocating the a thread object.*
- #define `OSAPI_LOG_THREAD_CREATE_EC` (`OSAPI_LOG_BASE` + 7)
- *An error when creating the thread object.*
- #define `OSAPI_LOG_THREAD_SEM_EC` (`OSAPI_LOG_BASE` + 8)
- *An error when creating thread sync semaphore.*
- #define `OSAPI_LOG_THREAD_EXEC_CREATE_EC` (`OSAPI_LOG_BASE` + 9)
- *Failed to signal that a thread has been created.*
- #define `OSAPI_LOG_THREAD_EXEC_START_EC` (`OSAPI_LOG_BASE` + 10)
- *Failed to signal the start a created thread.*
- #define `OSAPI_LOG_THREAD_START_EC` (`OSAPI_LOG_BASE` + 11)
- *Failed to start a thread.*
- #define `OSAPI_LOG_THREAD_DESTROY_EC` (`OSAPI_LOG_BASE` + 12)
- *Failed to destroy a thread.*
- #define `OSAPI_LOG_THREAD_DESTROY_NO_START_EC` (`OSAPI_LOG_BASE` + 13)
- *Failed to start an unstarted thread being destroyed.*
- #define `OSAPI_LOG_THREAD_DESTROY_NO_WAKEUP_EC` (`OSAPI_LOG_BASE` + 14)
- *Failed to wake up a thread that's being destroyed.*
- #define `OSAPI_LOG_THREAD_INIT_EC` (`OSAPI_LOG_BASE` + 15)
- *Failed initializing a thread.*
- #define `OSAPI_LOG_THREAD_SCHEDPARAM_EC` (`OSAPI_LOG_BASE` + 16)
- *Failed to set scheduling policy of a thread.*
- #define `OSAPI_LOG_THREAD_GET_POLICY_EC` (`OSAPI_LOG_BASE` + 17)
- *Failed to get the scheduling policy of a thread.*
- #define `OSAPI_LOG_THREAD_POLICY_DIFFER_EC` (`OSAPI_LOG_BASE` + 18)
- *Scheduling policy mismatch between a created thread and the application thread.*
- #define `OSAPI_LOG_THREAD_PRIORITY_MAP_EC` (`OSAPI_LOG_BASE` + 19)
- *Failed to map to native thread priority values.*

- #define `OSAPI_LOG_TIMER_DELETE_EC` (`OSAPI_LOG_BASE` + 20)
Failed to delete the Timer object.
- #define `OSAPI_LOG_TIMER_TICK_MUTEX_EC` (`OSAPI_LOG_BASE` + 22)
Failed taking or giving the Timer mutex.
- #define `OSAPI_LOG_TIMER_GET_USER_DATA_EPOCH_EC` (`OSAPI_LOG_BASE` + 27)
Failed to return user data for a timeout due to mismatched epochs.
- #define `OSAPI_LOG_TIMER_NEW_EC` (`OSAPI_LOG_BASE` + 28)
Failed to allocate memory for a new Timer.
- #define `OSAPI_LOG_TIMER_NEW_ENTRY_EC` (`OSAPI_LOG_BASE` + 29)
Failed to allocate memory for a new Timer entry.
- #define `OSAPI_LOG_TIMER_NEW_WHEEL_EC` (`OSAPI_LOG_BASE` + 30)
Failed to allocate memory for a new Timer wheel.
- #define `OSAPI_LOG_TIMER_NEW_MUTEX_EC` (`OSAPI_LOG_BASE` + 31)
Failed to create a new Timer mutex.
- #define `OSAPI_LOG_TIMER_NEW_START_TIMER_EC` (`OSAPI_LOG_BASE` + 32)
Failed to start a new Timer being created.
- #define `OSAPI_LOG_TIMER_DELETE_STOP_TIMER_EC` (`OSAPI_LOG_BASE` + 33)
Failed to stop a Timer being deleted.
- #define `OSAPI_LOG_TIMER_DELETE_MUTEX_EC` (`OSAPI_LOG_BASE` + 34)
Failed to delete the Timer mutex.
- #define `OSAPI_LOG_TIMER_MUTEX_EC` (`OSAPI_LOG_BASE` + 35)
Failed to take or give a Timer mutex.
- #define `OSAPI_LOG_SEMAPHORE_DELETE_EC` (`OSAPI_LOG_BASE` + 36)
Failed to delete a semaphore.
- #define `OSAPI_LOG_SEMAPHORE_NEW_EC` (`OSAPI_LOG_BASE` + 37)
Failed to create a semaphore.
- #define `OSAPI_LOG_SEMAPHORE_NEW_INIT_EC` (`OSAPI_LOG_BASE` + 38)
Failed to initialize a new semaphore.
- #define `OSAPI_LOG_SEMAPHORE_GIVE_EC` (`OSAPI_LOG_BASE` + 39)
Failed to give a semaphore.
- #define `OSAPI_LOG_SEMAPHORE_TAKE_EC` (`OSAPI_LOG_BASE` + 40)
Failed to take a semaphore.
- #define `OSAPI_LOG_MUTEX_DELETE_EC` (`OSAPI_LOG_BASE` + 41)
Failed to delete a mutex.
- #define `OSAPI_LOG_MUTEX_NEW_EC` (`OSAPI_LOG_BASE` + 42)
Failed to create a mutex.
- #define `OSAPI_LOG_MUTEX_TAKE_EC` (`OSAPI_LOG_BASE` + 43)
Failed to take a mutex.
- #define `OSAPI_LOG_MUTEX_GIVE_EC` (`OSAPI_LOG_BASE` + 44)
Failed to give a mutex.
- #define `OSAPI_LOG_MUTEX_INIT_EC` (`OSAPI_LOG_BASE` + 45)
Failed to initialize a mutex.
- #define `OSAPI_LOG_HEAP_INTERNAL_ALLOCATE_EC` (`OSAPI_LOG_BASE` + 46)
Failed to allocate a buffer from the heap.
- #define `OSAPI_LOG_SYSTEM_GET_TIME_EC` (`OSAPI_LOG_BASE` + 48)
Failed to get current system time.
- #define `OSAPI_LOG_LAST_RECORDED_ERROR_EC` (`OSAPI_LOG_BASE` + 49)

Return the last recorded error-code for the calling thread.

- #define `OSAPI_LOG_SET_THREAD_NAME_EC` (`OSAPI_LOG_BASE` + 50)
Failed to set the thread name in the OS.
- #define `OSAPI_LOG_SEM_OPEN_EC` (`OSAPI_LOG_BASE` + 52)
Failure during call VxWorks `semOpen(...)` or Posix `sem_open(...)` when managing shared memory segments, semaphores, or mutexes.
- #define `OSAPI_LOG_SEM_INFO_GET_EC` (`OSAPI_LOG_BASE` + 53)
Failure during VxWorks call `semInfoGet` when managing shared memory segments, semaphores, or mutexes.
- #define `OSAPI_LOG_SD_UNMAP_EC` (`OSAPI_LOG_BASE` + 54)
Failure during VxWorks call `sdUnmap` when managing shared memory segments, semaphores, or mutexes.
- #define `OSAPI_LOG_SD_OPEN_EC` (`OSAPI_LOG_BASE` + 55)
Failure during VxWorks call `sdOpen` when managing shared memory segments, semaphores, or mutexes.
- #define `OSAPI_LOG_SEM_GIVE_EC` (`OSAPI_LOG_BASE` + 56)
Failure during VxWorks call `semGive` when managing shared memory segments, semaphores, or mutexes.
- #define `OSAPI_LOG_SEM_TAKE_EC` (`OSAPI_LOG_BASE` + 57)
Failure during VxWorks call `semTake` when managing shared memory segments, semaphores, or mutexes.
- #define `OSAPI_LOG_UNKNOWN_SEMAPHORE_TYPE_EC` (`OSAPI_LOG_BASE` + 58)
Trying to perform an operation an unknown semmutex type.
- #define `OSAPI_LOG_SEGMENT_KEY_ALREADY_EXISTS_EC` (`OSAPI_LOG_BASE` + 59)
Failed to create segment due to a segment already existing.
- #define `OSAPI_LOG_SEGMENT_KEY_DOESNT_EXIST_EC` (`OSAPI_LOG_BASE` + 60)
Segment trying to be attached to does not exist.
- #define `OSAPI_LOG_SEGMENT_DETACH_FAILED_EC` (`OSAPI_LOG_BASE` + 61)
Failed to detach from shared memory segment.
- #define `OSAPI_LOG_SHMEM_ATTACH_FAILED_EC` (`OSAPI_LOG_BASE` + 62)
Attaching to shared memory segment/mutex/signaling semaphore failed.
- #define `OSAPI_LOG_SHMEM_GIVE_FAILED_EC` (`OSAPI_LOG_BASE` + 63)
Failed to give shared memory mutex/signaling semaphore.
- #define `OSAPI_LOG_SHMEM_TAKE_FAILED_EC` (`OSAPI_LOG_BASE` + 64)
Failed to take shared memory mutex/signaling semaphore.
- #define `OSAPI_LOG_SHMEM_SEM_DELETE_FAILED_EC` (`OSAPI_LOG_BASE` + 65)
Failed to delete shared memory semaphore or mutex.
- #define `OSAPI_LOG_SHMEM_SEM_MUTEX_CREATED_EC` (`OSAPI_LOG_BASE` + 66)
Error during shared memory semaphore creation/deletion.
- #define `OSAPI_LOG_SHMEM_SEGMENT_FILE_MAP_FAILED_EC` (`OSAPI_LOG_BASE` + 67)
Windows specific API `MapViewOfFile` failed.
- #define `OSAPI_LOG_SHMEM_SET_DESCRIPTOR_EC` (`OSAPI_LOG_BASE` + 68)
Windows specific error during setting security descriptor.
- #define `OSAPI_LOG_SHMEM_INIT_DESCRIPTOR_EC` (`OSAPI_LOG_BASE` + 69)
Windows specific error during initialization of security descriptor.
- #define `OSAPI_LOG_SHMEM_FTRUNCATE_EC` (`OSAPI_LOG_BASE` + 70)
Posix specific error during `ftruncate(...)` call.
- #define `OSAPI_LOG_SHMEM_MMAP_EC` (`OSAPI_LOG_BASE` + 71)
Posix specific error during `mmap(...)` call.
- #define `OSAPI_LOG_SHMEM_SHM_OPEN_EC` (`OSAPI_LOG_BASE` + 72)
Posix specific error during `shm_open(...)` call.
- #define `OSAPI_LOG_SHMEM_MUNMAP_EC` (`OSAPI_LOG_BASE` + 73)

- Posix specific error during munmap(...) call.*

 - #define `OSAPI_LOG_CLOSE_EC` (`OSAPI_LOG_BASE` + 74)
- Posix specific error during close(...) call.*

 - #define `OSAPI_LOG_SHMEM_UNLINK_EC` (`OSAPI_LOG_BASE` + 76)
- Posix specific error during shared memory unlink(...) call.*

 - #define `OSAPI_LOG_EXHAUSTED_POSIX_SEM_LIMIT_EC` (`OSAPI_LOG_BASE` + 77)
- System ran out of space to create semaphores. Consider increasing Posix sem limit.*

 - #define `OSAPI_LOG_SHMEM_SEM_WAIT_EC` (`OSAPI_LOG_BASE` + 78)
- Posix specific error during shared memory wait call.*

 - #define `OSAPI_LOG_SHMEM_SEM_UNLINK_EC` (`OSAPI_LOG_BASE` + 79)
- Posix specific error during shared memory sem unlink call.*

 - #define `OSAPI_LOG_SHMEM_POSIX_GIVE_EC` (`OSAPI_LOG_BASE` + 80)
- Posix specific error during os specific implementation of a semaphore mutex give.*

 - #define `OSAPI_LOG_THREAD_SEMPOOL_EXCEEDED_EC` (`OSAPI_LOG_BASE` + 90)
- Failed to add a semaphore to the thread semaphore pool.*

 - #define `OSAPI_LOG_THREAD_SEMPOOL_INVALID_SEM_EC` (`OSAPI_LOG_BASE` + 91)
- Failed to delete a semaphore from the thread semaphore pool.*

 - #define `OSAPI_LOG_SHMEM_SIZE_EXCEEDED_EC` (`OSAPI_LOG_BASE` + 92)
- Trying to create a shared memory segment that is greater than $(2^{31}) - 1$. The middleware cannot support shared memory segments created than ~ 2 GB even though the operating system may.*

 - #define `OSAPI_LOG_MATH_OFV_EC` (`OSAPI_LOG_BASE` + 93)
- A math overflow occurred.*

 - #define `OSAPI_LOG_INVALID_TIMER_RES_EC` (`OSAPI_LOG_BASE` + 94)
- An invalid timer resolution was specified.*

 - #define `OSAPI_LOG_THREAD_JOIN_EC` (`OSAPI_LOG_BASE` + 95)
- Failed to join a thread.*

 - #define `OSAPI_LOG_THREAD_GET_MAX_PRIORITY_EC` (`OSAPI_LOG_BASE` + 96)
- Failed to get the maximum priority value for a scheduling policy.*

 - #define `OSAPI_LOG_SHMEM_FSTAT_EC` (`OSAPI_LOG_BASE` + 97)
- POSIX specific error during fstat(...) call.*

 - #define `OSAPI_LOG_SHMEM_MUTEX_INIT_TIMEOUT_EC` (`OSAPI_LOG_BASE` + 97)
- Timed out waiting for a shared memory mutex to be initialized.*

 - #define `OSAPI_LOG_AUTOSAR_SETEVENT_CALL_EC` (`OSAPI_LOG_BASE` + 100)
- Autosar SetEvent system call failed.*

 - #define `OSAPI_LOG_AUTOSAR_CLEAREVENT_CALL_EC` (`OSAPI_LOG_BASE` + 101)
- Autosar ClearEvent system call failed.*

 - #define `OSAPI_LOG_AUTOSAR_WAITEVENT_CALL_EC` (`OSAPI_LOG_BASE` + 102)
- Autosar WaitEvent system call failed.*

 - #define `OSAPI_LOG_AUTOSAR_GETEVENT_CALL_EC` (`OSAPI_LOG_BASE` + 103)
- Autosar GetEvent system call failed.*

 - #define `OSAPI_LOG_AUTOSAR_TERMINATETASK_CALL_EC` (`OSAPI_LOG_BASE` + 104)
- Autosar TerminateTask system call failed.*

 - #define `OSAPI_LOG_AUTOSAR_ACTIVATETASK_CALL_EC` (`OSAPI_LOG_BASE` + 105)
- Autosar ActivateTask system call failed.*

 - #define `OSAPI_LOG_AUTOSAR_SCHEDULE_CALL_EC` (`OSAPI_LOG_BASE` + 106)
- Autosar Schedule system call failed.*

 - #define `OSAPI_LOG_AUTOSAR_SETRELALARM_CALL_EC` (`OSAPI_LOG_BASE` + 107)

- Autosar SetRelAlarm system call failed.*

 - #define `OSAPI_LOG_AUTOSAR_CANCELALARM_CALL_EC` (`OSAPI_LOG_BASE` + 108)
- Autosar CancelAlarm system call failed.*

 - #define `OSAPI_LOG_SEGMENT_ALREADY_EXISTS_EC` (`OSAPI_LOG_BASE` + 150)
- Shared memory segment already exists.*

 - #define `OSAPI_LOG_SEGMENT_DOESNT_EXIST_EC` (`OSAPI_LOG_BASE` + 151)
- Shared memory segment does not exist.*

 - #define `OSAPI_LOG_SHMEM_INVALID_SEGMENT_NAME_EC` (`OSAPI_LOG_BASE` + 152)
- Invalid shared memory segment name.*

 - #define `OSAPI_LOG_SHMEM_INVALID_SEGMENT_MODE_EC` (`OSAPI_LOG_BASE` + 153)
- Invalid shared memory segment mode.*

 - #define `OSAPI_LOG_SHMEM_SEGMENT_NOT_LOCKABLE_EC` (`OSAPI_LOG_BASE` + 154)
- Shared memory segment does not support locking.*

 - #define `OSAPI_LOG_SHMEM_SEGMENT_NOT_ROBUST_EC` (`OSAPI_LOG_BASE` + 155)
- Shared memory segment does not support robust mutexes.*

 - #define `OSAPI_LOG_SHMEM_ALREADY_ATTACHED_EC` (`OSAPI_LOG_BASE` + 156)
- Shared memory segment is already attached.*

 - #define `OSAPI_LOG_SHMEM_NOT_ATTACHED_EC` (`OSAPI_LOG_BASE` + 157)
- Shared memory segment is not attached.*

 - #define `OSAPI_LOG_SHMEM_CHECK_UNLINKED_EC` (`OSAPI_LOG_BASE` + 158)
- Checking if shared memory segment has been unlinked.*

 - #define `OSAPI_SHMEM_SEGMENT_NOT_INITIALIZED_EC` (`OSAPI_LOG_BASE` + 159)
- Shared memory segment is not initialized.*

 - #define `OSAPI_LOG_SHMEM_SEGMENT_NAME_ALREADY_EXISTS_EC` (`OSAPI_LOG_BASE` + 160)
- Shared memory segment with specified name already exists.*

 - #define `OSAPI_LOG_SHMEM_SEGMENT_NAME_DOESNT_EXIST_EC` (`OSAPI_LOG_BASE` + 161)
- Shared memory segment with specified name does not exist.*

 - #define `OSAPI_LOG_PTHREAD_MUTEXATTR_INIT_EC` (`OSAPI_LOG_BASE` + 200)
- Failed to create a pthread mutex attribute.*

 - #define `OSAPI_LOG_PTHREAD_MUTEXATTR_SETPSHARED_EC` (`OSAPI_LOG_BASE` + 201)
- Failed to set the pthread mutex attribute to be process shared.*

 - #define `OSAPI_LOG_PTHREAD_MUTEXATTR_SETROBUST_EC` (`OSAPI_LOG_BASE` + 202)
- Failed to set the pthread mutex attribute to be robust.*

 - #define `OSAPI_LOG_PTHREAD_MUTEXATTR_DESTROY_EC` (`OSAPI_LOG_BASE` + 203)
- Failed to destroy the pthread mutex attribute.*

 - #define `OSAPI_LOG_PTHREAD_MUTEX_INIT_EC` (`OSAPI_LOG_BASE` + 204)
- Failed to initialize a pthread mutex.*

 - #define `OSAPI_LOG_PTHREAD_MUTEX_DESTROY_EC` (`OSAPI_LOG_BASE` + 205)
- Failed to destroy a pthread mutex.*

 - #define `OSAPI_LOG_PTHREAD_MUTEX_LOCK_EC` (`OSAPI_LOG_BASE` + 206)
- Failed to lock a pthread mutex.*

 - #define `OSAPI_LOG_PTHREAD_MUTEX_UNLOCK_EC` (`OSAPI_LOG_BASE` + 207)
- Failed to unlock a pthread mutex.*

 - #define `OSAPI_LOG_PTHREAD_MUTEX_CONSISTENT_EC` (`OSAPI_LOG_BASE` + 208)
- Failed to set the pthread mutex to be consistent.*

 - #define `OSAPI_LOG_PTHREAD_CONDATTR_INIT_EC` (`OSAPI_LOG_BASE` + 209)
- Failed to create a pthread condition variable attribute.*

- #define `OSAPI_LOG_PTHREAD_CONDATTR_SETPSHARED_EC` (`OSAPI_LOG_BASE` + 210)
Failed to set the pthread condition variable attribute to be process shared.
- #define `OSAPI_LOG_PTHREAD_CONDATTR_DESTROY_EC` (`OSAPI_LOG_BASE` + 211)
Failed to destroy the pthread condition variable attribute.
- #define `OSAPI_LOG_PTHREAD_COND_INIT_EC` (`OSAPI_LOG_BASE` + 212)
Failed to initialize a pthread condition variable.
- #define `OSAPI_LOG_PTHREAD_COND_DESTROY_EC` (`OSAPI_LOG_BASE` + 213)
Failed to destroy a pthread condition variable.
- #define `OSAPI_LOG_PTHREAD_COND_WAIT_EC` (`OSAPI_LOG_BASE` + 214)
Failed to wait on a pthread condition variable.
- #define `OSAPI_LOG_PTHREAD_COND_SIGNAL_EC` (`OSAPI_LOG_BASE` + 215)
Failed to signal a pthread condition variable.
- #define `OSAPI_LOG_PTHREAD_MUTEXATTR_SETTYPE_EC` (`OSAPI_LOG_BASE` + 216)
Failed to set the type of a pthread mutex attribute.
- #define `OSAPI_LOG_PTHREAD_MUTEXATTR_SETPROTOCOL_EC` (`OSAPI_LOG_BASE` + 217)
Failed to set the protocol of a pthread mutex attribute.
- #define `OSAPI_LOG_PTHREAD_MUTEXATTR_SETPRIOCEILING_EC` (`OSAPI_LOG_BASE` + 218)
Failed to set the priority ceiling of a pthread mutex attribute.
- #define `OSAPI_LOG_WIN_WAIT_FAILED_EC` (`OSAPI_LOG_BASE` + 300)
WaitForSingleObject returned an error while waiting for the timer semaphore to time out. Micro continues to run, but it may indicate a platform problem.
- #define `OSAPI_LOG_DETAIL_MODULENAME` (0x80000000UL)
Bitmap for enabling saving the module name in the log buffer.
- #define `OSAPI_LOG_DETAIL_SOURCEFILE` (0x40000000UL)
Bitmap for enabling saving the file name in the log buffer.
- #define `OSAPI_LOG_DETAIL_LINENUMBER` (0x20000000UL)
Bitmap for enabling saving the line number in the log buffer.
- #define `OSAPI_LOG_DETAIL_FUNCTIONNAME` (0x10000000UL)
Bitmap for enabling saving the function name in the log buffer.
- #define `OSAPI_LogProperty_INITIALIZER`
Initializer for the `OSAPI_LogProperty`.

Typedefs

- typedef void(* `OSAPI_Log_write_buffer_T`) (const char *buffer, `RTI_SIZE_T` length)
Optional user-defined function to output a log/trace buffer.

Enumerations

- enum `OSAPI_LogVerbosity_T` { `OSAPI_LOG_VERBOSITY_SILENT` = 0 , `OSAPI_LOG_VERBOSITY_ERROR` = 1 , `OSAPI_LOG_VERBOSITY_WARNING` = 2 , `OSAPI_LOG_VERBOSITY_DEBUG` = 3 }
Logging verbosity.

Functions

- [RTI_BOOL OSAPI_Log_set_log_handler](#) (OSAPI_LogHandler_T handler, void *param)
Install a log handler.
- [RTI_BOOL OSAPI_Log_get_log_handler](#) (OSAPI_LogHandler_T *handler, void **param)
Return the current log handler.
- [RTI_BOOL OSAPI_Log_set_display_handler](#) (OSAPI_LogDisplay_T handler, void *param)
Install a display handler.
- [RTI_BOOL OSAPI_Log_get_display_handler](#) (OSAPI_LogDisplay_T *handler, void **param)
Return the current display function.
- void [OSAPI_Log_set_verbosity](#) (OSAPI_LogVerbosity_T verbosity)
Set the log verbosity.
- void [OSAPI_Trace_set_trace_mask](#) (RTI_UINT32 mask)
Set the trace mask.
- [RTI_BOOL OSAPI_Log_clear](#) (void)
Clear the log buffer.
- void [OSAPI_Log_get_property](#) (struct OSAPI_LogProperty *property)
Get the current log properties.
- [RTI_BOOL OSAPI_Log_set_property](#) (struct OSAPI_LogProperty *property)
Set the current log properties.
- [OSAPI_LogVerbosity_T OSAPI_Log_get_verbosity](#) (void)
Get the current log verbosity.
- [RTI_INT32 OSAPI_Log_get_last_error_code](#) (void)
Returns the error code for a function that failed.

8.16.1 Detailed Description

OSAPI Log API.

8.16.2 Macro Definition Documentation

OSAPI_LOG_BASE

```
#define OSAPI_LOG_BASE (0)
```

Log Id base-number.

REDA_LOG_BASE

```
#define REDA_LOG_BASE (1 << 16)
```

REDA Module.

DB_LOG_BASE

```
#define DB_LOG_BASE (2 << 16)
```

DB Module.

RT_LOG_BASE

```
#define RT_LOG_BASE (3 << 16)
```

RT Module.

NETIO_LOG_BASE

```
#define NETIO_LOG_BASE (4 << 16)
```

NETIO Module.

UDP_LOG_BASE

```
#define UDP_LOG_BASE NETIO_LOG_BASE
```

UDP Module, same as NETIO.

SHMEM_LOG_BASE

```
#define SHMEM_LOG_BASE NETIO_LOG_BASE + 10000
```

NETIO_SHMEM Module, same as NETIO.

SDM_LOG_BASE

```
#define SDM_LOG_BASE NETIO_LOG_BASE + 20000
```

SDM Module, same as NETIO.

CDR_LOG_BASE

```
#define CDR_LOG_BASE (5 << 16)
```

CDR Module.

XCDR_LOG_BASE

```
#define XCDR_LOG_BASE CDR_LOG_BASE + 20000
```

CDR Module.

RTPS_LOG_BASE

```
#define RTPS_LOG_BASE (6 << 16)
```

RTPS Module.

DDSC_LOG_BASE

```
#define DDSC_LOG_BASE (7 << 16)
```

DDS_C Module.

RHSM_LOG_BASE

```
#define RHSM_LOG_BASE (8 << 16)
```

RHSM Module.

WHSM_LOG_BASE

```
#define WHSM_LOG_BASE (9 << 16)
```

WHSM Module.

DPSE_LOG_BASE

```
#define DPSE_LOG_BASE (10 << 16)
```

DPSE Module.

DPDE_LOG_BASE

```
#define DPDE_LOG_BASE (11 << 16)
```

DPDE Module.

SEC_CORE_LOG_BASE

```
#define SEC_CORE_LOG_BASE (12 << 16)
```

DDDS Security plugin.

APPGEN_LOG_BASE

```
#define APPGEN_LOG_BASE (13 << 16)
```

APPGEN Module.

NETIO_ZCOPY_LOG_BASE

```
#define NETIO_ZCOPY_LOG_BASE (14 << 16)
```

NETIO Zero Copy Module.

DDS_FILTER_LOG_BASE

```
#define DDS_FILTER_LOG_BASE (15 << 16)
```

DDS Filter Module.

OSAPI_LOG_MATH_OFV_EC

```
#define OSAPI_LOG_MATH_OFV_EC (OSAPI_LOG_BASE + 93)
```

A math overflow occurred.

OSAPI_LOG_INVALID_TIMER_RES_EC

```
#define OSAPI_LOG_INVALID_TIMER_RES_EC (OSAPI_LOG_BASE + 94)
```

An invalid timer resolution was specified.

8.16.3 Function Documentation**OSAPI_Log_set_log_handler()**

```
RTI_BOOL OSAPI_Log_set_log_handler (  
    OSAPI_LogHandler_T handler,  
    void * param)
```

Install a log handler.

The log functionality allows the user to specify a log handler. The log handler is a function which is called for every logged event. It is up to the user to decide what to do with the log message. The handler is a global function pointer.

Parameters

<i>handler</i>	<<in>> Pointer to log handler function
<i>param</i>	<<in>> Parameter passed to the log handler function. This parameter is transparent to the log functionality.

Returns

RTI_TRUE on success, RTI_FALSE on failure

OSAPI_Log_get_log_handler()

```
RTI_BOOL OSAPI_Log_get_log_handler (  
    OSAPI_LogHandler_T * handler,  
    void ** param)
```

Return the current log handler.

Parameters

<i>handler</i>	<<out>> Pointer to store log handler function
<i>param</i>	<<out>> Pointer to store the current log handler parameter.

Returns

RTI_TRUE on success, RTI_FALSE on failure

OSAPI_Log_set_display_handler()

```
RTI_BOOL OSAPI_Log_set_display_handler (  
    OSAPI_LogDisplay_T handler,  
    void * param)
```

Install a display handler.

The display handler is responsible for outputting log messages to a console.

Parameters

<i>handler</i>	<<in>> Pointer to display function
<i>param</i>	<<in>> Parameter passed to the display handler function. This parameter is transparent to the log functionality.

Returns

RTI_TRUE on success, RTI_FALSE on failure

OSAPI_Log_get_display_handler()

```
RTI_BOOL OSAPI_Log_get_display_handler (
    OSAPI_LogDisplay_T * handler,
    void ** param)
```

Return the current display function.

Return the current log handler.

Parameters

<i>handler</i>	<<in>> Pointer to store display handler function
<i>param</i>	<<in>> Pointer to store the current display handler parameters.

Returns

RTI_TRUE on success, RTI_FALSE on failure

OSAPI_Log_set_verbosity()

```
void OSAPI_Log_set_verbosity (
    OSAPI_LogVerbosity_T verbosity)
```

Set the log verbosity.

Change the log verbosity. The new setting takes immediate effect.

Parameters

<i>verbosity</i>	<<in>> New log verbosity
------------------	--------------------------

OSAPI_Trace_set_trace_mask()

```
void OSAPI_Trace_set_trace_mask (
    RTI_UINT32 mask)
```

Set the trace mask.

Parameters

<i>mask</i>	<<in>> New trace mask.
-------------	------------------------

0 - Disable trace

0xffffffff - Enable trace

References [RTI_UINT32](#).

OSAPI_Log_clear()

```
RTI_BOOL OSAPI_Log_clear (
    void )
```

Clear the log buffer.

Clear the log buffer, all the current entries are lost.

NOTE: This function must only be called inside a log-handler as it is not thread-safe.

Returns

RTI_TRUE on success, RTI_FALSE on failure

OSAPI_Log_get_property()

```
void OSAPI_Log_get_property (
    struct OSAPI_LogProperty * property)
```

Get the current log properties.

Return the current log properties

Parameters

<i>property</i>	<<out>> Current log properties
-----------------	--------------------------------

See Also ::OSAPI_Log::set_property

OSAPI_Log_set_property()

```
RTI_BOOL OSAPI_Log_set_property (
    struct OSAPI_LogProperty * property)
```

Set the current log properties.

Set the current log properties. It is not possible to set new properties after ::OSAPI_Log::initialize has been called.

Parameters

<i>property</i>	<<in>> New log properties
-----------------	---------------------------

See Also ::OSAPI_Log::get_property

OSAPI_Log_get_verbosity()

```
OSAPI_LogVerbosity_T OSAPI_Log_get_verbosity (
    void )
```

Get the current log verbosity.

Set the current log properties. It is not possible to set new properties after ::OSAPI_Log::initialize has been called.

Returns

Current verbosity

References [RTI_INT32](#), and [RTI_UINT32](#).

8.17 osapi_process.h File Reference

process related utility

```
#include "osapi/osapi_dll.h"
#include "osapi/osapi_types.h"
```

Functions

- [RTI_BOOL OSAPI_Process_is_alive](#) (OSAPI_ProcessId pid)
Returns true if a process is alive.
- OSAPI_ProcessId [OSAPI_Process_getpid](#) (void)
Return the process ID, which is unique for the application.

8.17.1 Detailed Description

process related utility

8.18 osapi_semaphore.h File Reference

Semaphore interface definition.

```
#include "osapi/osapi_dll.h"
#include "osapi/osapi_types.h"
```

Macros

- `#define OSAPI_SEMAPHORE_TIMEOUT_INFINITE -1`
If `OSAPI_Semaphore_take` is called with `OSAPI_SEMAPHORE_TIMEOUT_INFINITE` as timeout, `OSAPI_Semaphore_take` will not return until the semaphore is signaled.
- `#define OSAPI_SEMAPHORE_RESULT_OK 0`
If `OSAPI_Semaphore_take` succeeds and the semaphore is signaled within the timeout period, `OSAPI_Semaphore_take` returns `TRUE` and sets the failure reason to `OSAPI_SEMAPHORE_RESULT_OK`.
- `#define OSAPI_SEMAPHORE_RESULT_TIMEOUT 1`
If `OSAPI_Semaphore_take` was called with a timeout value different from `OSAPI_SEMAPHORE_TIMEOUT_INFINITE`, and the semaphore was not signaled before the timeout expired, `OSAPI_Semaphore_take` returns `TRUE` and sets the failure reason to `OSAPI_SEMAPHORE_RESULT_TIMEOUT`.
- `#define OSAPI_SEMAPHORE_RESULT_ERROR 2`
If `OSAPI_Semaphore_take` fails for an unknown reason, `OSAPI_Semaphore_take` returns `FALSE` and sets the failure reason to `OSAPI_SEMAPHORE_RESULT_ERROR`.

Typedefs

- `typedef struct OSAPI_Semaphore OSAPI_Semaphore_T`
Abstract Semaphore type.

Functions

- `OSAPI_Semaphore_T * OSAPI_Semaphore_new (void)`
Create a Semaphore.
- `RTI_BOOL OSAPI_Semaphore_delete (OSAPI_Semaphore_T *self)`
Delete a semaphore.
- `RTI_BOOL OSAPI_Semaphore_take (OSAPI_Semaphore_T *self, RTI_INT32 timeout_ms, RTI_INT32 *fail_↔ reason)`
Take a Semaphore.
- `RTI_BOOL OSAPI_Semaphore_give (OSAPI_Semaphore_T *self)`
Give a Semaphore.

8.18.1 Detailed Description

Semaphore interface definition.

8.18.2 Function Documentation

OSAPI_Semaphore_new()

```
OSAPI_Semaphore_T * OSAPI_Semaphore_new (
    void )
```

Create a Semaphore.

Create a new binary semaphore with an initial count of 0 and a maximum count of 1. A call to `OSAPI_Semaphore_take` will block until the semaphore is signaled with a call to `OSAPI_Semaphore_give`.

The created semaphore must be deleted using `OSAPI_Semaphore_delete` when no longer needed (only available when `RTI_CERT` is not defined).

Returns

Pointer to a semaphore with an initial count of 0 on success, `NULL` on failure.

OSAPI_Semaphore_take()

```
RTI_BOOL OSAPI_Semaphore_take (
    OSAPI_Semaphore_T * self,
    RTI_INT32 timeout_ms,
    RTI_INT32 * fail_reason)
```

Take a Semaphore.

Wait for the semaphore to be signaled. If the semaphore count is greater than 0, this function decrements the count and returns immediately. Otherwise, it blocks until the semaphore is signaled by [OSAPI_Semaphore_give](#) or the timeout expires.

Parameters

<i>self</i>	<< <i>in</i> >> Semaphore previously created with OSAPI_Semaphore_new . Cannot be NULL.
<i>timeout_ms</i>	<< <i>in</i> >> Timeout in milliseconds. The take call will wait at most <i>timeout_ms</i> milliseconds before returning. If OSAPI_SEMAPHORE_TIMEOUT_INFINITE is specified, the call will not return until the semaphore count is 1 or higher.
<i>fail_reason</i>	<< <i>out</i> >> Additional information when the call returns. Cannot be NULL. If RTI_TRUE is returned and the semaphore was acquired, <i>fail_reason</i> is set to OSAPI_SEMAPHORE_RESULT_OK. If RTI_TRUE is returned but the operation timed out, <i>fail_reason</i> is set to OSAPI_SEMAPHORE_RESULT_TIMEOUT. If RTI_FALSE is returned, <i>fail_reason</i> is set to OSAPI_SEMAPHORE_RESULT_ERROR.

Returns

RTI_TRUE on success (including timeout), RTI_FALSE on error.

See Also [OSAPI_Semaphore_give](#)

References [RTI_INT32](#).

OSAPI_Semaphore_give()

```
RTI_BOOL OSAPI_Semaphore_give (
    OSAPI_Semaphore_T * self)
```

Give a Semaphore.

Signal the semaphore by incrementing its count. If any threads are blocked waiting on this semaphore via [OSAPI_Semaphore_take](#), one of them will be unblocked. A semaphore can only be given if it is owned by the calling thread.

Parameters

<i>self</i>	<< <i>in</i> >> Semaphore previously created with OSAPI_Semaphore_new . Cannot be NULL.
-------------	---

Returns

RTI_TRUE on success, RTI_FALSE on failure.

See Also [OSAPI_Semaphore_take](#)

References [RTI_INT32](#), and [RTI_UINT32](#).

8.19 osapi_stdio.h File Reference

Standard I/O interface definition.

```
#include "osapi/osapi_dll.h"
#include "osapi/osapi_types.h"
```

8.19.1 Detailed Description

Standard I/O interface definition.

8.20 osapi_task.h File Reference

Task interface definition.

```
#include "osapi/osapi_config.h"
#include "osapi/osapi_dll.h"
#include "osapi/osapi_types.h"
#include "osapi/osapi_system.h"
#include "osapi/osapi_time.h"
#include "osapi/osapi_thread.h"
```

8.20.1 Detailed Description

Task interface definition.

8.21 osapi_time.h File Reference

Time API definition.

```
#include "osapi/osapi_dll.h"
#include "osapi/osapi_types.h"
```

Classes

- struct [OSAPI_SystemTime](#)
System Time.

Typedefs

- typedef struct OSAPI_SystemTime [OSAPI_SystemTime](#)
System Time.

8.21.1 Detailed Description

Time API definition.

This file implements functions to convert between time formats.

8.22 reda_log.h File Reference

REDA module log codes.

```
#include "osapi/osapi_log.h"
```

Macros

- #define REDA_LOG_HEAP_MGR_SAMPLE_BUFFERPOOL 1
Sample buffer pool object.
- #define REDA_LOG_HEAP_MGR_LISTNODE_BUFFERPOOL 2
Listnode buffer pool object.
- #define REDA_LOG_HEAP_MGR_OBJECT_MEMORY 3
Heap manager object.
- #define REDA_LOG_HEAP_MGR_INDEXER 4
REDA indexer object.
- #define REDA_LOG_HEAP_MGR_MUTEX 5
REDA heap manager object.
- #define REDA_LOG_BUFFERPOOL_OUT_OF_RESOURCES_EC (REDA_LOG_BASE + 1)
Not sufficient memory to allocate buffer-pool.
- #define REDA_LOG_BUFFERPOOL_BUFFER_INITIALIZATION_FAILED_EC (REDA_LOG_BASE + 2)
The buffer initialization method failed.
- #define REDA_LOG_BUFFERPOOL_NOT_EMPTY_EC (REDA_LOG_BASE + 3)
Cannot delete buffer-pool due to buffer(s) not returned to pool.
- #define REDA_LOG_BUFFERPOOL_DOUBLE_FREE_EC (REDA_LOG_BASE + 4)
A buffer was freed more than once.
- #define REDA_LOG_MEMPOOL_INCONSISTENT_PROPERTIES_EC (REDA_LOG_BASE + 5)
Inconsistent properties specified for a REDA_MemPool.
- #define REDA_LOG_MEMPOOL_OUT_OF_RESOURCES_EC (REDA_LOG_BASE + 6)
Insufficient memory to allocate a new REDA_MemPool.
- #define REDA_LOG_MEMPOOL_POOL_ALLOCATION_EC (REDA_LOG_BASE + 7)
Error during allocation of the memory pool used by a REDA_MemPool.
- #define REDA_LOG_BUFFERPOOL_NULL_POINTER_EC (REDA_LOG_BASE + 8)
A pointer in a pool's linked list was NULL.
- #define REDA_LOG_SEQUENCE_COPY_FAILED_EC (REDA_LOG_BASE + 100)
An error occurred while copying a sequence.
- #define REDA_LOG_SEQUENCE_ALLOC_FAILED_EC (REDA_LOG_BASE + 101)
An error occurred while allocating space for a sequence.
- #define REDA_LOG_SEQUENCE_SET_MAX_FAILED_EC (REDA_LOG_BASE + 102)

- An error occurred while setting the maximum sequence size.*

 - #define REDA_LOG_SEQUENCE_REALLOCATION_EC (REDA_LOG_BASE + 103)
- An attempt was made to resize an already allocated sequence.*

 - #define REDA_LOG_SEQUENCE_INVALID_OPERATION_EC (REDA_LOG_BASE + 104)
- An error occurred while operating on two sequences (copy/compare)*

 - #define REDA_LOG_SEQUENCE_INVALID_LENGTH_EC (REDA_LOG_BASE + 105)
- An attempt was made to set an illegal length (length < 0 or length > max_length)*

 - #define REDA_LOG_SEQUENCE_INDEX_OUT_OF_BOUNDS_EC (REDA_LOG_BASE + 106)
- An illegal sequence index was specified (index < 0 or index > length)*

 - #define REDA_LOG_STRING_ALLOC_FAILED_EC (REDA_LOG_BASE + 200)
- An error occurred while allocating memory for a string.*

 - #define REDA_LOG_INDEX_FULL_EC (REDA_LOG_BASE + 300)
- An attempt was made to add an element to a full index.*

 - #define REDA_LOG_INDEX_ENTRY_EXISTS_EC (REDA_LOG_BASE + 301)
- An attempt was made to add an existing element to an index.*

 - #define REDA_LOG_INVALID_LIST_NODE_EC (REDA_LOG_BASE + 302)
- An invalid list node was encountered.*

 - #define REDA_LOG_BUFFER_OUT_OF_MEMORY_EC (REDA_LOG_BASE + 307)
- Failed to allocate buffer of a specified kind.*

 - #define REDA_LOG_HEAP_MGR_ALLOC_EC (REDA_LOG_BASE + 308)
- Allocation error in REDA Heap Manager.*

 - #define REDA_LOG_HEAP_MGR_GET_BUFFER_EC (REDA_LOG_BASE + 309)
- Error when there are no more available loanable buffers.*

 - #define REDA_LOG_HEAP_MGR_MUTEX_FAILURE_EC (REDA_LOG_BASE + 310)
- Error during locking or unlocking of a mutex.*

8.22.1 Detailed Description

REDA module log codes.

8.23 rh_sm_log.h File Reference

RH module log codes.

```
#include "osapi/osapi_log.h"
```

Macros

- #define `RHSM_LOG_OUTSTANDING_SAMPLE_EC` (`RHSM_LOG_BASE` + 1)
Failed to delete Reader History due to outstanding, unremoved sample.
- #define `RHSM_LOG_NO_PROPERTY_QOS_EC` (`RHSM_LOG_BASE` + 2)
Failed to create Reader History due to unspecified property Qos.
- #define `RHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC` (`RHSM_LOG_BASE` + 3)
KEEP_ALL History kind is not supported.
- #define `RHSM_LOG_MAX_SAMPLE_TOO_SMALL_EC` (`RHSM_LOG_BASE` + 4)
DataReaderQos.resource_limits.max_samples set too small.
- #define `RHSM_LOG_TBF_NOT_SUPPORTED_EC` (`RHSM_LOG_BASE` + 5)
Time-based filtering is not supported.
- #define `RHSM_LOG_SAMPLE_INFO_POOL_EC` (`RHSM_LOG_BASE` + 6)
Failed to allocate a buffer pool for sample info.
- #define `RHSM_LOG_SAMPLE_POOL_EC` (`RHSM_LOG_BASE` + 7)
Failed to allocate a buffer pool for sample pointers.
- #define `RHSM_LOG_SAMPLE_PTR_ARRAY_EC` (`RHSM_LOG_BASE` + 8)
Failed to get sample buffer.
- #define `RHSM_LOG_INFO_ARRAY_EC` (`RHSM_LOG_BASE` + 9)
Failed to get sample info buffer.
- #define `RHSM_LOG_GET_RECEPTION_TIMESTAMP_EC` (`RHSM_LOG_BASE` + 10)
Failed to get time for reception timestamp.
- #define `RHSM_LOG_INVALID_INSTANCE_REPLACEMENT_EC` (`RHSM_LOG_BASE` + 11)
An invalid instance replacement policy was specified.
- #define `RHSM_LOG_FAILED_TO_REMOVE_OLDEST_EC` (`RHSM_LOG_BASE` + 12)
Failed to remove the oldest sample.
- #define `RHSM_LOG_SAMPLE_POOL_EMPTY_EC` (`RHSM_LOG_BASE` + 13)
Sample pool empty, may have exceeded.
- #define `RHSM_LOG_RW_PRUNE_FAILED_EC` (`RHSM_LOG_BASE` + 14)
Failed to prune remote writer entry.
- #define `RHSM_LOG_ENTRY_RESERVATION_FAILED_EC` (`RHSM_LOG_BASE` + 15)
Failed to reserve an entry.
- #define `RHSM_LOG_RETURN_SAMPLE_EC` (`RHSM_LOG_BASE` + 16)
Failed to return a sample.
- #define `RHSM_LOG_HISTORY_UPDATE_EC` (`RHSM_LOG_BASE` + 17)
Failed to update the history queue.
- #define `RHSM_LOG_GET_TIME_EC` (`RHSM_LOG_BASE` + 18)
Failed to get the system time.
- #define `RHSM_LOG_HISTORY_OBJECT` 1
A history object.
- #define `RHSM_LOG_REMOVALIST_OBJECT` 2
A removal list object.
- #define `RHSM_LOG_RWPOOL_OBJECT` 3
A remote writer pool object.
- #define `RHSM_LOG_RWINDEX_OBJECT` 4
A remote writer index object.
- #define `RHSM_LOG_KEY_OBJECT` 5

- *A Key object.*
- #define `RHSM_LOG_RW_OBJECT` 6
- *A Key object.*
- #define `RHSM_LOG_KEYPOOL_OBJECT` 7
- *A Key pool object.*
- #define `RHSM_LOG_OWNER_OBJECT` 8
- *A key owner object.*
- #define `RHSM_LOG_SAMPLEPOOL_OBJECT` 9
- *A sample pool object.*
- #define `RHSM_LOG_KEYINDEXPOOL_OBJECT` 10
- *A key index pool.*
- #define `RHSM_LOG_SAMPLEPTPOOL_OBJECT` 11
- *A sample ptr pool object.*
- #define `RHSM_LOG_SAMPLEINFO_OBJECT` 12
- *A sample info pool.*
- #define `RHSM_LOG_INSTANCEHANDLE_OBJECT` 13
- *A sample info pool.*
- #define `RHSM_LOG_OBJECT_ALLOCATE_EC` (`RHSM_LOG_BASE` + 19)
- *Failed to allocate object of the specified kind.*
- #define `RHSM_LOG_OBJECT_DELETE_EC` (`RHSM_LOG_BASE` + 20)
- *Failed to delete object of the specified kind.*
- #define `RHSM_LOG_OBJECT_INDEX_EC` (`RHSM_LOG_BASE` + 21)
- *Failed to index an object of the specified kind.*
- #define `RHSM_LOG_OBJECT_INVALID_EC` (`RHSM_LOG_BASE` + 22)
- *Invalid object specified.*
- #define `RHSM_LOG_NO_PROPERTY_EC` (`RHSM_LOG_BASE` + 23)
- *No property when creating a reader history instance.*
- #define `RHSM_LOG_DESTINATION_BY_SOURCE_NOT_SUPPORTED_EC` (`RHSM_LOG_BASE` + 24)
- *Destination ordering by `SOURCE_TIMESTAMP` ordering is not supported.*
- #define `RHSM_LOG_RECOMMIT_FAILURE_EC` (`RHSM_LOG_BASE` + 25)
- *Failed to recommit sample.*

8.23.1 Detailed Description

RH module log codes.

8.23.2 Macro Definition Documentation

`RHSM_LOG_HISTORY_OBJECT`

```
#define RHSM_LOG_HISTORY_OBJECT 1
```

A history object.

RHSM_LOG_REMOVELIST_OBJECT

```
#define RHSM_LOG_REMOVELIST_OBJECT 2
```

A removal list object.

RHSM_LOG_RWPOOL_OBJECT

```
#define RHSM_LOG_RWPOOL_OBJECT 3
```

A remote writer pool object.

RHSM_LOG_RWINDEX_OBJECT

```
#define RHSM_LOG_RWINDEX_OBJECT 4
```

A remote writer index object.

RHSM_LOG_KEY_OBJECT

```
#define RHSM_LOG_KEY_OBJECT 5
```

A Key object.

RHSM_LOG_RW_OBJECT

```
#define RHSM_LOG_RW_OBJECT 6
```

A Key object.

RHSM_LOG_KEYPOOL_OBJECT

```
#define RHSM_LOG_KEYPOOL_OBJECT 7
```

A Key pool object.

RHSM_LOG_OWNER_OBJECT

```
#define RHSM_LOG_OWNER_OBJECT 8
```

A key owner object.

RHSM_LOG_SAMPLEPOOL_OBJECT

```
#define RHSM_LOG_SAMPLEPOOL_OBJECT 9
```

A sample pool object.

RHSM_LOG_KEYINDEXPOOL_OBJECT

```
#define RHSM_LOG_KEYINDEXPOOL_OBJECT 10
```

A key index pool.

RHSM_LOG_SAMPLEPTPOOL_OBJECT

```
#define RHSM_LOG_SAMPLEPTPOOL_OBJECT 11
```

A sample ptr pool object.

RHSM_LOG_SAMPLEINFO_OBJECT

```
#define RHSM_LOG_SAMPLEINFO_OBJECT 12
```

A sample info pool.

RHSM_LOG_INSTANCEHANDLE_OBJECT

```
#define RHSM_LOG_INSTANCEHANDLE_OBJECT 13
```

A sample info pool.

8.24 rt_log.h File Reference

```
#include "osapi/osapi_log.h"
```

Macros

- #define `RT_LOG_SET_IMMUTABLE_PROPERTY_EC` (`RT_LOG_BASE` + 1)
Failed to set registry property due to registry already being enabled.
- #define `RT_LOG_REGISTRY_INIT_FAILURE_EC` (`RT_LOG_BASE` + 2)
Failed to initialize registry due to failed table creation.
- #define `RT_LOG_REGISTRY_FINALIZE_EC` (`RT_LOG_BASE` + 3)
Failed to finalize registry due to failed table deletion.
- #define `RT_LOG_REGISTRY_EXISTS_EC` (`RT_LOG_BASE` + 4)
An attempt was made to register a factory that already existed.
- #define `RT_LOG_REGISTRY_REGISTER_EC` (`RT_LOG_BASE` + 5)
Error registering a component factory. May have exceeded RT_RegistryProperty.max_factories.
- #define `RT_LOG_REGISTRY_INIT_FACTORY_EC` (`RT_LOG_BASE` + 7)
A registered factory failed to initialize.
- #define `RT_LOG_REGISTRY_NAME_TOO_LONG_EC` (`RT_LOG_BASE` + 8)
Factory name longer than maximum length of 8.
- #define `RT_LOG_REGISTRY_INCONSISTENT_CID_EC` (`RT_LOG_BASE` + 9)
Factory name exists, but the class ID is not of the requested type.
- #define `RT_LOG_REGISTRY_NOT_INITIALIZED_EC` (`RT_LOG_BASE` + 10)
An attempt was made to operate on an uninitialized registry.

8.25 rtps_config.h File Reference

RTPS Configuration.

8.25.1 Detailed Description

RTPS Configuration.

Configurable build flags for RTPS functionality

8.26 rtps_dll.h File Reference

RTPS Configuration.

```
#include "osapi/osapi_config.h"
#include "rtps/rtps_config.h"
```

8.26.1 Detailed Description

RTPS Configuration.

Configurable build flags for RTPS functionality

8.27 rtps_log.h File Reference

RTPS module log codes.

```
#include "osapi/osapi_log.h"
```

Macros

- #define [RTPS_LOG_INITIALIZE_INTERFACE_EC](#) (RTPS_LOG_BASE + 1)
Failed to initialize an RTPS interface.
- #define [RTPS_LOG_ALLOCATE_EC](#) (RTPS_LOG_BASE + 2)
Failed to allocate heap memory for internal resources.
- #define [RTPS_LOG_CREATE_ROUTE_TABLE_EC](#) (RTPS_LOG_BASE + 3)
Failed to create a database table for storing RTPS route info.
- #define [RTPS_LOG_DB_CREATE_ROUTE_PEER_INDEX_EC](#) (RTPS_LOG_BASE + 4)
Failed to create database index for route-peer info.
- #define [RTPS_LOG_DB_CREATE_ROUTE_XPORT_INDEX_EC](#) (RTPS_LOG_BASE + 5)
Failed to create database index for route-transport info.
- #define [RTPS_LOG_DB_CREATE_BIND_PEER_INDEX_EC](#) (RTPS_LOG_BASE + 6)
Failed to create database index for bind-peer info.
- #define [RTPS_LOG_INDEX_ADD_ENTRY_EC](#) (RTPS_LOG_BASE + 8)
Failed to add an entry to a database index.
- #define [RTPS_LOG_DB_SELECT_ALL_EC](#) (RTPS_LOG_BASE + 9)
Failed to select all entries of a database table.
- #define [RTPS_LOG_DB_SELECT_MATCH_EC](#) (RTPS_LOG_BASE + 10)
Matching entry not found in a database table.
- #define [RTPS_LOG_DB_SELECT_RANGE_EC](#) (RTPS_LOG_BASE + 11)
No matching entries found in a database table.
- #define [RTPS_LOG_DB_CREATE_ROUTE_ENTRY_EC](#) (RTPS_LOG_BASE + 12)
Failed to create database entry for route.
- #define [RTPS_LOG_DB_INSERT_ROUTE_ENTRY_EC](#) (RTPS_LOG_BASE + 13)
Failed to insert entry into route table.
- #define [RTPS_LOG_DB_REMOVE_ROUTE_ENTRY_EC](#) (RTPS_LOG_BASE + 14)
Failed to remove entry from route table.
- #define [RTPS_LOG_DB_DELETE_ROUTE_ENTRY_EC](#) (RTPS_LOG_BASE + 15)
Failed to delete entry from route table.
- #define [RTPS_LOG_DB_CREATE_BIND_ENTRY_EC](#) (RTPS_LOG_BASE + 16)
Failed to create database entry for bind.
- #define [RTPS_LOG_DB_INSERT_BIND_ENTRY_EC](#) (RTPS_LOG_BASE + 17)
Failed to insert an entry into the bind table.
- #define [RTPS_LOG_DB_REMOVE_BIND_ENTRY_EC](#) (RTPS_LOG_BASE + 18)
Failed to remove an entry from the bind table.
- #define [RTPS_LOG_DB_REMOVE_EXTERNAL_BIND_ENTRY_EC](#) (RTPS_LOG_BASE + 19)
Failed to remove an entry from the external bind table.
- #define [RTPS_LOG_DB_DELETE_EXTERNAL_BIND_ENTRY_EC](#) (RTPS_LOG_BASE + 20)
Failed to delete entry from external bind table.

- #define `RTPS_LOG_SEND_EC` (`RTPS_LOG_BASE` + 21)
Failed to send a packet due to a lower module failing to send the packet.
- #define `RTPS_LOG_RECEIVE_EC` (`RTPS_LOG_BASE` + 22)
Failed to receive a packet due to a higher module failing to receive the packet.
- #define `RTPS_LOG_ROUTE_PACKET_EC` (`RTPS_LOG_BASE` + 23)
Failed to send a packet, when routing to a lower module or peer.
- #define `RTPS_LOG_FORWARD_UPSTREAM_EC` (`RTPS_LOG_BASE` + 24)
Failed to receive a packet, when forwarding to a higher upstream module.
- #define `RTPS_LOG_BAD_PARAMETER_EC` (`RTPS_LOG_BASE` + 25)
Bad parameter to an RTPS interface function.
- #define `RTPS_LOG_NOT_ENABLED_EC` (`RTPS_LOG_BASE` + 26)
Failed because interface is not enabled.
- #define `RTPS_LOG_READER_UNSUPPORTED_EC` (`RTPS_LOG_BASE` + 27)
RTPS reader does not support send.
- #define `RTPS_LOG_INTERFACE_MISMATCH_EC` (`RTPS_LOG_BASE` + 28)
Upstream interface does not match source or destination of packet being sent or received.
- #define `RTPS_LOG_FULL_SEND_WINDOW_EC` (`RTPS_LOG_BASE` + 29)
Failed to send due to reaching maximum send window size.
- #define `RTPS_LOG_NETIO_PACKET_SET_TAIL_EC` (`RTPS_LOG_BASE` + 31)
Failed to set tail of packet to send.
- #define `RTPS_LOG_FUNC_UNSUPPORTED_EC` (`RTPS_LOG_BASE` + 32)
RTPS interface does not support this function.
- #define `RTPS_LOG_EXCEEDED_LIMIT_TRANSPORTS_EC` (`RTPS_LOG_BASE` + 33)
Out of transport entries.
- #define `RTPS_LOG_EXCEEDED_LIMIT_PEERS_EC` (`RTPS_LOG_BASE` + 34)
Out of peer entries.
- #define `RTPS_LOG_ASSERT_PEER_EC` (`RTPS_LOG_BASE` + 35)
Failed to assert a remote writer or reader.
- #define `RTPS_LOG_ASSERT_TRANSPORT_EC` (`RTPS_LOG_BASE` + 36)
Failed to assert a downstream transport.
- #define `RTPS_LOG_FIND_EXTERNAL_INTERFACE_EC` (`RTPS_LOG_BASE` + 37)
Failed to lookup an external interface.
- #define `RTPS_LOG_NONEXISTENT_ROUTE_EC` (`RTPS_LOG_BASE` + 38)
Failed to delete a nonexistent route.
- #define `RTPS_LOG_NONEXISTENT_BIND_EC` (`RTPS_LOG_BASE` + 39)
Failed to remove a nonexistent bind entry.
- #define `RTPS_LOG_NONEXISTENT_EXTERNAL_BIND_EC` (`RTPS_LOG_BASE` + 40)
Failed to remove a nonexistent external bind entry.
- #define `RTPS_LOG_STALE_ACK_EPOCH_EC` (`RTPS_LOG_BASE` + 41)
Dropped an ACKNACK submessage with an old epoch.
- #define `RTPS_LOG_STALE_HB_EPOCH_EC` (`RTPS_LOG_BASE` + 42)
Dropped a HEARTBEAT submessage with an old epoch.
- #define `RTPS_LOG_ACK_EC` (`RTPS_LOG_BASE` + 43)
Failed an acknack() upstream.
- #define `RTPS_LOG_REQUEST_EC` (`RTPS_LOG_BASE` + 45)
Failed a request() upstream.
- #define `RTPS_LOG_RETURN_LOAN_EC` (`RTPS_LOG_BASE` + 46)

- Failed a return_loan() upstream.*

 - #define RTPS_LOG_NETIO_PACKET_SET_HEAD_EC (RTPS_LOG_BASE + 47)
- Failed to set the head of a packet.*

 - #define RTPS_LOG_SEND_HEARTBEAT_EC (RTPS_LOG_BASE + 48)
- Failed to send a HEARTBEAT submessage.*

 - #define RTPS_LOG_SHIFT_BITMAP_EC (RTPS_LOG_BASE + 49)
- Failed to shift a bitmap.*

 - #define RTPS_LOG_FULLY_ACKED_READER_EC (RTPS_LOG_BASE + 50)
- A Reader is fully acknowledged and does not need to respond to a final HEARTBEAT.*

 - #define RTPS_LOG_DATA_OUT_OF_RANGE_EC (RTPS_LOG_BASE + 51)
- Dropping a DATA submessage whose sequence number is outside of a Reader's receive window.*

 - #define RTPS_LOG_DATA_ALREADY_RECEIVED_EC (RTPS_LOG_BASE + 52)
- Dropping a DATA submessage whose sequence number was previously received.*

 - #define RTPS_LOG_INTERFACE_NOT_ENABLED_EC (RTPS_LOG_BASE + 53)
- Failed to receive a message because interface is not enabled.*

 - #define RTPS_LOG_INVALID_PACKET_EC (RTPS_LOG_BASE + 54)
- Dropped a message with an invalid packet header.*

 - #define RTPS_LOG_UNSUPPORTED_INTERFACE_EC (RTPS_LOG_BASE + 55)
- Received a message on an unsupported interface.*

 - #define RTPS_LOG_UNKNOWN_SUBMESSAGE_EC (RTPS_LOG_BASE + 56)
- Received a submessage with an unknown ID.*

 - #define RTPS_LOG_PROCESS_ACKNACK_EC (RTPS_LOG_BASE + 57)
- Failed to process received ACKNACK submessage.*

 - #define RTPS_LOG_PROCESS_DATA_EC (RTPS_LOG_BASE + 58)
- Failed to process received DATA submessage.*

 - #define RTPS_LOG_PROCESS_GAP_EC (RTPS_LOG_BASE + 59)
- Failed to process received GAP submessage.*

 - #define RTPS_LOG_PROCESS_HEARTBEAT_EC (RTPS_LOG_BASE + 60)
- Failed to process received HEARTBEAT submessage.*

 - #define RTPS_LOG_CREATE_HB_EVENT_EC (RTPS_LOG_BASE + 61)
- Failed to create periodic HEARTBEAT event.*

 - #define RTPS_LOG_CREATE_EVENT_TIMER_EC (RTPS_LOG_BASE + 62)
- Failed to create periodic HEARTBEAT event timer.*

 - #define RTPS_LOG_SEQ_NUM_OUT_OF_RANGE_EC (RTPS_LOG_BASE + 63)
- Failed to set bitmap bit due to out-of-range sequence number.*

 - #define RTPS_LOG_FIRST_SN_GREATER_LAST_SN_EC (RTPS_LOG_BASE + 65)
- Invalid range of sequence numbers to fill in bitmap.*

 - #define RTPS_LOG_BITCOUNT_OUT_OF_BOUNDS_EC (RTPS_LOG_BASE + 66)
- Deserialized an invalid out-of-bounds bitcount for a bitmap.*

 - #define RTPS_LOG_SHIFT_SN_OUT_OF_RANGE_EC (RTPS_LOG_BASE + 67)
- Failed to shift bitmap due to invalid starting sequence number.*

 - #define RTPS_LOG_SERIALIZE_HOST_ID_EC (RTPS_LOG_BASE + 68)
- Failed to serialize host ID of GUID.*

 - #define RTPS_LOG_SERIALIZE_APP_ID_EC (RTPS_LOG_BASE + 69)
- Failed to serialize app ID of GUID.*

 - #define RTPS_LOG_SERIALIZE_INSTANCE_ID_EC (RTPS_LOG_BASE + 70)
- Failed to serialized instance ID of GUID.*

- #define `RTPS_LOG_SERIALIZE_OBJECT_ID_EC` (`RTPS_LOG_BASE` + 71)
Failed to serialize object ID of GUID.
- #define `RTPS_LOG_DESERIALIZE_HOST_ID_EC` (`RTPS_LOG_BASE` + 72)
Failed to deserialize host ID of GUID.
- #define `RTPS_LOG_DESERIALIZE_APP_ID_EC` (`RTPS_LOG_BASE` + 73)
Failed to deserialize app ID of GUID.
- #define `RTPS_LOG_DESERIALIZE_INSTANCE_ID_EC` (`RTPS_LOG_BASE` + 74)
Failed to deserialize instance ID of GUID.
- #define `RTPS_LOG_DESERIALIZE_OBJECT_ID_EC` (`RTPS_LOG_BASE` + 75)
Failed to deserialize object ID of GUID.
- #define `RTPS_LOG_DB_CREATE_EXT_BIND_ENTRY_EC` (`RTPS_LOG_BASE` + 76)
Failed to create database entry for external bind table.
- #define `RTPS_LOG_BITMAP_SET_BIT_EC` (`RTPS_LOG_BASE` + 77)
Failed to set bit in bitmap.
- #define `RTPS_LOG_UNSUPPORTED_INFO_REPLY_EC` (`RTPS_LOG_BASE` + 78)
Received and dropped currently unsupported INFO_REPLY submessage.
- #define `RTPS_LOG_UNSUPPORTED_INFO_REPLY_IP4_EC` (`RTPS_LOG_BASE` + 79)
Received and dropped currently unsupported INFO_REPLY_IP4 submessage.
- #define `RTPS_LOG_UNSUPPORTED_INFO_SRC_EC` (`RTPS_LOG_BASE` + 80)
Received and dropped currently unsupported INFO_SRC submessage.
- #define `RTPS_LOG_DELETE_INDEX_EC` (`RTPS_LOG_BASE` + 81)
Failed to delete index of a database table.
- #define `RTPS_LOG_DELETE_TABLE_EC` (`RTPS_LOG_BASE` + 82)
Failed to delete a database table.
- #define `RTPS_LOG_DB_LOCK_EC` (`RTPS_LOG_BASE` + 83)
Failed to take database lock.
- #define `RTPS_LOG_DB_UNLOCK_EC` (`RTPS_LOG_BASE` + 84)
Failed to give database lock.
- #define `RTPS_LOG_CREATE_BIND_TABLE_EC` (`RTPS_LOG_BASE` + 85)
Failed to create a database table for storing RTPS bind info.
- #define `RTPS_LOG_STATUS_CHANGE_EC` (`RTPS_LOG_BASE` + 86)
Failed to update status for reliable reader activity changed.
- #define `RTPS_LOG_SET_GAP_EC` (`RTPS_LOG_BASE` + 87)
Failed to update status for reliable reader activity changed.
- #define `RTPS_LOG_WINDOW_POOL_ALLOC_EC` (`RTPS_LOG_BASE` + 88)
Out of memory to allocate send window pool.
- #define `RTPS_LOG_GET_WINDOW_BUFFER_EC` (`RTPS_LOG_BASE` + 89)
Failed to get buffer from send window pool.
- #define `RTPS_LOG_INSERT_WINDOW_FULL_EC` (`RTPS_LOG_BASE` + 90)
Cannot insert sample into full window.
- #define `RTPS_LOG_WINDOW_INSERT_EC` (`RTPS_LOG_BASE` + 91)
Failed to insert sample into send window.
- #define `RTPS_LOG_TIMER_DELETE_TIMEOUT_EC` (`RTPS_LOG_BASE` + 92)
Failed to delete a timeout event.
- #define `RTPS_LOG_BUFFERPOOL_DELETE_EC` (`RTPS_LOG_BASE` + 93)
Failed to delete a buffer pool.
- #define `RTPS_LOG_DIRECT_SEND_EC` (`RTPS_LOG_BASE` + 94)

- Failed to send a message to a specific peer.*

 - #define `RTPS_LOG_PACKET_INITIALIZE_EC` (`RTPS_LOG_BASE` + 95)
- RTPS writer failed to initialize a packet.*

 - #define `RTPS_LOG_WRITER_REQUEST_SENT_HISTORY_EC` (`RTPS_LOG_BASE` + 96)
- RTPS reliable writer failed to request and resend history.*

 - #define `RTPS_LOG_BITMAP_SHIFT_EC` (`RTPS_LOG_BASE` + 97)
- Failed to shift bitmap to new lead sequence number.*

 - #define `RTPS_LOG_WINDOW_ADVANCE_EC` (`RTPS_LOG_BASE` + 98)
- Send window of a remote reader failed to advance.*

 - #define `RTPS_LOG_WINDOW_ADVANCE_UNACKED_EC` (`RTPS_LOG_BASE` + 99)
- Failed when advancing window ahead of unacknowledged sequence number.*

 - #define `RTPS_LOG_LOOKUP_PEER_EC` (`RTPS_LOG_BASE` + 100)
- Failed to find peer that should exist.*

 - #define `RTPS_LOG_READER_SEND_ACKNACK_EC` (`RTPS_LOG_BASE` + 101)
- RTPS reader failed to send an ACKNACK message.*

 - #define `RTPS_LOG_FINALIZE_INTERFACE_EC` (`RTPS_LOG_BASE` + 102)
- Failed to finalize an RTPS interface.*

 - #define `RTPS_LOG_INDEXER_REMOVE_ENTRY_EC` (`RTPS_LOG_BASE` + 103)
- Failed to remove index entry for external interface upon deleting RTPS interface.*

 - #define `RTPS_LOG_LOOKUP_EXT_BIND_ENTRY_EC` (`RTPS_LOG_BASE` + 104)
- Failed when looking up external bind entry from database.*

 - #define `RTPS_LOG_SEQ_INIT_EC` (`RTPS_LOG_BASE` + 105)
- Failed to initialize a local sequence.*

 - #define `RTPS_LOG_SEQ_MAX_EC` (`RTPS_LOG_BASE` + 106)
- Failed to set maximum of a local sequence.*

 - #define `RTPS_LOG_SEQ_LEN_EC` (`RTPS_LOG_BASE` + 107)
- Failed to set length of a local sequence.*

 - #define `RTPS_LOG_INTF_MODE_UNDEF_EC` (`RTPS_LOG_BASE` + 108)
- Initializing an RTPS interface with undefined mode.*

 - #define `RTPS_LOG_DELETE_BUFFER_POOL_EC` (`RTPS_LOG_BASE` + 109)
- Failed to delete buffer pool.*

 - #define `RTPS_LOG_CREATE_BUFFER_POOL_EC` (`RTPS_LOG_BASE` + 110)
- Failed to create buffer pool.*

 - #define `RTPS_LOG_CREATE_PEERS_INDEX_EC` (`RTPS_LOG_BASE` + 111)
- Failed to create index of peers.*

 - #define `RTPS_LOG_DELETE_INDEXER_EC` (`RTPS_LOG_BASE` + 112)
- Failed to delete indexer.*

 - #define `RTPS_LOG_GET_PEER_BUFFER_EC` (`RTPS_LOG_BASE` + 113)
- Failed to get buffer from peer buffer pool.*

 - #define `RTPS_LOG_DB_CREATE_DIRECT_INDEX_EC` (`RTPS_LOG_BASE` + 114)
- Failed to create database index for direct sends.*

 - #define `RTPS_LOG_DB_CREATE_GROUP_INDEX_EC` (`RTPS_LOG_BASE` + 115)
- Failed to create database index for group sends.*

 - #define `RTPS_LOG_NETIO_PACKET_INSUFFICIENT_BUFFER_EC` (`RTPS_LOG_BASE` + 116)
- Insufficient buffer in packet.*

 - #define `RTPS_LOG_PROCESS_DATA_BATCH_EC` (`RTPS_LOG_BASE` + 136)
- Failed to process received DATA BATCH submessage.*

- #define [RTPS_LOG_PROCESS_HEARTBEAT_BATCH_EC](#) (RTPS_LOG_BASE + 137)
Failed to process received HEARTBEAT_BATCH submessage.
- #define [RTPS_LOG_NEGATIVE_RESERVATION_COUNT_EC](#) (RTPS_LOG_BASE + 138)
The reservation count is zero.
- #define [RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_LOOKUP_FAILED_EC](#) (RTPS_LOG_BASE + 139)
Failed to lookup a remote participant's interceptor handle.
- #define [RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_CREATE_FAILED_EC](#) (RTPS_LOG_BASE + 140)
Failed to create a record to store a remote participant's interceptor handle.
- #define [RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_INSERT_FAILED_EC](#) (RTPS_LOG_BASE + 141)
Failed to insert a record in the table storing interceptor handles of matched remote participants.
- #define [RTPS_LOG_TRUST_INTERCEPTOR_HANDLE_LIST_FULL_EC](#) (RTPS_LOG_BASE + 142)
Interceptor handle list out of resources.
- #define [RTPS_LOG_TRUST_INCONSISTENT_INTERCEPTOR_HANDLE_EC](#) (RTPS_LOG_BASE + 143)
Inconsistent handles found for remote peer.
- #define [RTPS_LOG_TRUST_BIND_FAILED_EC](#) (RTPS_LOG_BASE + 144)
Failed to bind resources for trust behavior.
- #define [RTPS_LOG_TRUST_SUBMSG_CONVERSION_FAILED_EC](#) (RTPS_LOG_BASE + 145)
Failed to bind resources for trust behavior.
- #define [RTPS_LOG_TRUST_INVALID_SUBMSG_EC](#) (RTPS_LOG_BASE + 146)
A malformed trust message was received.
- #define [RTPS_LOG_TRUST_UNEXPECTED_READER_STATE_EC](#) (RTPS_LOG_BASE + 147)
The RTPS reader was in an unexpected state.
- #define [RTPS_LOG_DB_CREATE_SELECTED_GROUP_INDEX_EC](#) (RTPS_LOG_BASE + 148)
Failed to create database index for the selected route group.
- #define [RTPS_LOG_LOCK_EC](#) (RTPS_LOG_BASE + 149)
Failed to take a lock.
- #define [RTPS_LOG_UNLOCK_EC](#) (RTPS_LOG_BASE + 150)
Failed to give a lock.
- #define [RTPS_LOG_SCHEDULE_PACKET_EC](#) (RTPS_LOG_BASE + 151)
Failed to schedule a packet.
- #define [RTPS_LOG_SEQ_FINALIZE_EC](#) (RTPS_LOG_BASE + 152)
Failed to finalize a sequence.
- #define [RTPS_LOG_TRANSFORM_RTPS_EC](#) (RTPS_LOG_BASE + 153)
Failed to transform rtps message.
- #define [RTPS_LOG_RESTORE_TRUST_LOAN_BUFFER_EC](#) (RTPS_LOG_BASE + 154)
Failed to restore trust loan buffer.
- #define [RTPS_LOG_CREATE_REASSEMBLY_TABLE_EC](#) (RTPS_LOG_BASE + 155)
Failed to create reassembly table.
- #define [RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM128_EC](#) (RTPS_LOG_BASE + 200)
The property tries to override the BUILTIN128 checksum without setting allow_builtin_override to TRUE.
- #define [RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM32_EC](#) (RTPS_LOG_BASE + 201)
The property tries to override the BUILTIN32 checksum without setting allow_builtin_override to TRUE.
- #define [RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM64_EC](#) (RTPS_LOG_BASE + 202)
The property tries to override the BUILTIN64 checksum without setting allow_builtin_override to TRUE.

- #define `RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN128_EC` (`RTPS_LOG_BASE` + 203)
The property override of the BUILTIN128 checksum is invalid. It is not legal to reuse any of the built-in functions.
- #define `RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN32_EC` (`RTPS_LOG_BASE` + 204)
The property override of the BUILTIN32 checksum is invalid. It is not legal to reuse any of the built-in functions functions.
- #define `RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN64_EC` (`RTPS_LOG_BASE` + 205)
The property override of the BUILTIN64 checksum is invalid. It is not legal to reuse any of the built-in functions functions.
- #define `RTPS_LOG_CHECKSUM_CHECKSUM_CALCULATION_FAILED_EC` (`RTPS_LOG_BASE` + 210)
RTPS_Interface_calculate_checksum() failed to calculate the checksum.
- #define `RTPS_LOG_UNSUPPORTED_MANDATORY_HDR_EXT_PID_EC` (`RTPS_LOG_BASE` + 211)
The HeaderExtension contained a mandatory PID that was not understood.
- #define `RTPS_LOG_INVALID_HDR_EXT_PID_LIST_EC` (`RTPS_LOG_BASE` + 212)
An invalid HeaderExtension PID was detected.
- #define `RTPS_LOG_INVALID_HDR_EXT_PID_ERROR_EC` (`RTPS_LOG_BASE` + 213)
Failed to decode the HeaderExtension PID list.
- #define `RTPS_LOG_INCONSISTENT_HDR_EXT_LENGTH_ERROR_EC` (`RTPS_LOG_BASE` + 214)
The HeaderExtension length is inconsistent with the decoded encoded HeaderExtension length.
- #define `RTPS_LOG_MESSAGE_EXCEED_LENGTH_ERROR_EC` (`RTPS_LOG_BASE` + 215)
A RTPS message which was longer than the indicated message length in the RTPS header was received. This causes the rest of the RTPS message to be invalidated and the remaining submessages to be ignored.
- #define `RTPS_LOG_CHECKSUM_INVALID_CRC32_FLAGS_EC` (`RTPS_LOG_BASE` + 216)
The CRC32 submsg flags are invalid.
- #define `RTPS_LOG_CHECKSUM_MISSING_CHECKSUM_LENGTH_EC` (`RTPS_LOG_BASE` + 217)
A header extension was received, but the checksum length is not present.
- #define `RTPS_LOG_CHECKSUM_UNSUPPORTED_EC` (`RTPS_LOG_BASE` + 218)
Unsupported checksum bits received, message dropped.
- #define `RTPS_LOG_CHECKSUM_LENGTH_DESERIALIZE_ERROR_EC` (`RTPS_LOG_BASE` + 219)
Checksum length could not be deserialized.
- #define `RTPS_LOG_CHECKSUM_INCONSISTENT_LENGTH_EC` (`RTPS_LOG_BASE` + 220)
The payload length is less than the deserialized checksum length.
- #define `RTPS_LOG_CHECKSUM_CHECKSUM_DESERIALIZE_ERROR_EC` (`RTPS_LOG_BASE` + 221)
Checksum length could not be deserialized.
- #define `RTPS_LOG_CHECKSUM_CHECKSUM_ERROR_EC` (`RTPS_LOG_BASE` + 222)
The checksum is invalid.
- #define `RTPS_LOG_CHECKSUM_TOO_MANY_SG_BUFFERS_EC` (`RTPS_LOG_BASE` + 223)
Found too many scatter-gather buffers when calculating checksum.
- #define `RTPS_LOG_SEND_LIVELINESS_EC` (`RTPS_LOG_BASE` + 224)
Failure to send Liveliness packets.
- #define `RTPS_LOG_DELETE_RECORD_EC` (`RTPS_LOG_BASE` + 225)
Failure to delete a record.
- #define `RTPS_LOG_APPLY_CONTENT_FILTER_EC` (`RTPS_LOG_BASE` + 300)
Failed to apply a content filter to a message.
- #define `RTPS_LOG_UPDATE_RELIABLE_PEERS_FILTER_EC` (`RTPS_LOG_BASE` + 301)
Failed to update the state of reliable peers while applying filtering.
- #define `RTPS_LOG_ADD_FILTERED_ROUTE_EC` (`RTPS_LOG_BASE` + 302)
Failed to add a filtered route.
- #define `RTPS_LOG_DELETE_FILTERED_ROUTE_EC` (`RTPS_LOG_BASE` + 303)
Failed to delete a filtered route.

8.27.1 Detailed Description

RTPS module log codes.

Log codes of the RTPS module

8.27.2 Macro Definition Documentation

RTPS_LOG_DB_REMOVE_ROUTE_ENTRY_EC

```
#define RTPS_LOG_DB_REMOVE_ROUTE_ENTRY_EC (RTPS_LOG_BASE + 14)
```

Failed to remove entry from route table.

Failure reason given by database return code (dbrc)

8.28 rtps_rtps_impl.h File Reference

Implementation of RTPS interface functions and types.

```
#include "reda/reda_sequenceNumber.h"  
#include "osapi/osapi_config.h"
```

8.28.1 Detailed Description

Implementation of RTPS interface functions and types.

RTPS protocol defined types, implemented in C.

8.29 wh_sm_log.h File Reference

WH module log codes.

```
#include "osapi/osapi_log.h"
```

Macros

- #define `WHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC` (`WHSM_LOG_BASE` + 1)
KEEP_ALL History kind is not supported.
- #define `WHSM_LOG_UNLIMITED_HISTORY_NOT_SUPPORTED_EC` (`WHSM_LOG_BASE` + 2)
Unlimited length resource limits unsupported.
- #define `WHSM_LOG_MAX_SAMPLES_TOO_SMALL_EC` (`WHSM_LOG_BASE` + 3)
DataWriterQos.resource_limits.max_samples set too small.
- #define `WHSM_LOG_HISTORY_OBJECT` 1
A history object.
- #define `WHSM_LOG_KEY_OBJECT` 2
A key object.
- #define `WHSM_LOG_KEYPOOL_OBJECT` 3
A key-pool object.
- #define `WHSM_LOG_SAMPLEPOOL_OBJECT` 4
A sample-pool object.
- #define `WHSM_LOG_KEYINDEX_OBJECT` 5
A key index pool.
- #define `WHSM_LOG_HISTORYINDEX_OBJECT` 6
A key index pool.
- #define `WHSM_LOG_SAMPLEINDEX_OBJECT` 7
A key index pool.
- #define `WHSM_LOG_SAMPLE_OBJECT` 9
A sample object.
- #define `WHSM_LOG_OBJECT_ALLOCATE_EC` (`WHSM_LOG_BASE` + 4)
Failed to allocate object of the specified kind.
- #define `WHSM_LOG_OBJECT_DELETE_EC` (`WHSM_LOG_BASE` + 5)
Failed to delete object of the specified kind.
- #define `WHSM_LOG_OBJECT_INDEX_EC` (`WHSM_LOG_BASE` + 6)
Failed to index an object of the specified kind.
- #define `WHSM_LOG_NO_PROPERTY_EC` (`WHSM_LOG_BASE` + 7)
No property when creating a writer history instance.
- #define `WHSM_LOG_INVALID_INDEX_OBJECT_EC` (`WHSM_LOG_BASE` + 8)
The sample removed from an index did not match sample's key.

8.29.1 Detailed Description

WH module log codes.

8.29.2 Macro Definition Documentation

WHSM_LOG_HISTORY_OBJECT

```
#define WHSM_LOG_HISTORY_OBJECT 1
```

A history object.

WHSM_LOG_KEY_OBJECT

```
#define WHSM_LOG_KEY_OBJECT 2
```

A key object.

WHSM_LOG_KEYPOOL_OBJECT

```
#define WHSM_LOG_KEYPOOL_OBJECT 3
```

A key-pool object.

WHSM_LOG_SAMPLEPOOL_OBJECT

```
#define WHSM_LOG_SAMPLEPOOL_OBJECT 4
```

A sample-pool object.

WHSM_LOG_KEYINDEX_OBJECT

```
#define WHSM_LOG_KEYINDEX_OBJECT 5
```

A key index pool.

WHSM_LOG_HISTORYINDEX_OBJECT

```
#define WHSM_LOG_HISTORYINDEX_OBJECT 6
```

A key index pool.

WHSM_LOG_SAMPLEINDEX_OBJECT

```
#define WHSM_LOG_SAMPLEINDEX_OBJECT 7
```

A key index pool.

WHSM_LOG_SAMPLE_OBJECT

```
#define WHSM_LOG_SAMPLE_OBJECT 9
```

A sample object.

Index

- `_contiguous_buffer`
 - `FooSeq`, [852](#)
 - `_element_size`
 - `FooSeq`, [853](#)
 - `_flags`
 - `FooSeq`, [853](#)
 - `_length`
 - `FooSeq`, [853](#)
 - `_maximum`
 - `FooSeq`, [852](#)
 - `_token1`
 - `FooSeq`, [853](#)
 - `_token2`
 - `FooSeq`, [853](#)
 - `~FooSeq`
 - `FooSeq`, [843](#)
- `accept_unknown_peers`
 - `DDS_DiscoveryQosPolicy`, [545](#)
- `access_scope`
 - `DDS_PresentationQosPolicy`, [602](#)
- `active_count`
 - `DDS_ReliableReaderActivityChangedStatus`, [623](#)
- `active_count_change`
 - `DDS_ReliableReaderActivityChangedStatus`, [624](#)
- `add_entry`
 - UDP Transport API, [148](#)
- `add_peer`
 - `DDSDomainParticipant`, [735](#)
- `add_property`
 - `DDSPropertyQosPolicyHelper`, [768](#)
- `add_route`
 - `ZCOPY_NotifUserInterfaceI`, [908](#)
- `address`
 - `DDS_Locator_t`, [586](#)
 - `NETIO_AddressValue`, [863](#)
 - `NETIO_DGRAM_InterfaceRouteEntry`, [868](#)
 - `NETIO_DGRAM_InterfaceTableEntry`, [870](#)
 - `UDP_InterfaceTableEntry`, [898](#)
- `address_high`
 - `NETIO_DGRAM_InterfaceMultiCastGroup`, [867](#)
- `address_low`
 - `NETIO_DGRAM_InterfaceMultiCastGroup`, [867](#)
- `alive_count`
 - `DDS_LivelinessChangedStatus`, [580](#)
- `alive_count_change`
 - `DDS_LivelinessChangedStatus`, [581](#)
- `allow_builtin_override`
 - RTPS Transport API, [153](#)
- `allow_interface`
 - `UDP_InterfaceFactoryProperty`, [892](#)
- `allowed_crc_mask`
 - `DDS_ChecksumProperty_t`, [512](#)
 - `DDS_WireProtocolQosPolicy`, [668](#)
- `app_gen_log.h`, [910](#)
- `APPGEN`, [496](#)
 - `APPGEN_APP_PARTICIPANT_NOT_FOUND_EC`, [499](#)
 - `APPGEN_FACTORY_REGISTER_EC`, [498](#)
 - `APPGEN_FACTORY_UNREGISTER_EC`, [499](#)
 - `APPGEN_LOG_ALLOCATE_EC`, [497](#)
 - `APPGEN_LOG_API_ERROR_EC`, [498](#)
 - `APPGEN_LOG_APP_CONFIG_NOT_CORRECT_EC`, [499](#)
 - `APPGEN_LOG_APP_CONFIG_POINTER_NOT_CORRECT_EC`, [499](#)
 - `APPGEN_LOG_APP_NAME_ERROR_EC`, [498](#)
 - `APPGEN_LOG_COPY_ENTITY_NAME_EC`, [497](#)
 - `APPGEN_LOG_CREATE_ENTITY_NAME_EC`, [498](#)
 - `APPGEN_LOG_DDS_ENTITY_EC`, [498](#)
 - `APPGEN_LOG_FACTORY_TOPIC_NOT_FOUND_EC`, [498](#)
 - `APPGEN_LOG_LIB_NOT_FOUND_EC`, [499](#)
 - `APPGEN_MULTIPLICITY_EC`, [498](#)
- `APPGEN_APP_PARTICIPANT_NOT_FOUND_EC`
 - `APPGEN`, [499](#)
- `APPGEN_Factory_get_interface`
 - Application Generation API, [222](#)
- `APPGEN_Factory_register`
 - Application Generation API, [222](#)
- `APPGEN_FACTORY_REGISTER_EC`
 - `APPGEN`, [498](#)
- `APPGEN_Factory_unregister`
 - Application Generation API, [222](#)
- `APPGEN_FACTORY_UNREGISTER_EC`
 - `APPGEN`, [499](#)
- `APPGEN_LOG_ALLOCATE_EC`
 - `APPGEN`, [497](#)
- `APPGEN_LOG_API_ERROR_EC`
 - `APPGEN`, [498](#)
- `APPGEN_LOG_APP_CONFIG_NOT_CORRECT_EC`
 - `APPGEN`, [499](#)
- `APPGEN_LOG_APP_CONFIG_POINTER_NOT_CORRECT_EC`
 - `APPGEN`, [499](#)
- `APPGEN_LOG_APP_NAME_ERROR_EC`
 - `APPGEN`, [498](#)
- `APPGEN_LOG_BASE`
 - `osapi_log.h`, [979](#)
- `APPGEN_LOG_COPY_ENTITY_NAME_EC`
 - `APPGEN`, [497](#)
- `APPGEN_LOG_CREATE_ENTITY_NAME_EC`
 - `APPGEN`, [498](#)

- APPGEN_LOG_DDS_ENTITY_EC
 - APPGEN, [498](#)
- APPGEN_LOG_FACTORY_TOPIC_NOT_FOUND_EC
 - APPGEN, [498](#)
- APPGEN_LOG_LIB_NOT_FOUND_EC
 - APPGEN, [499](#)
- APPGEN_MULTIPLICITY_EC
 - APPGEN, [498](#)
- Application Generation API, [221](#)
 - APPGEN_Factory_get_interface, [222](#)
 - APPGEN_Factory_register, [222](#)
 - APPGEN_Factory_unregister, [222](#)
- as_datawriter
 - FooDataWriter, [827](#)
- as_entity
 - DDSDDataReader, [673](#)
 - DDSDomainParticipant, [718](#)
 - DDSTopic, [792](#)
- as_topicdescription
 - DDSTopic, [792](#)
- as_uint32
 - NETIO_AddressValue, [863](#)
- assert_liveliness
 - DDSDDataWriter, [696](#)
 - DDSDomainParticipant, [730](#)
- assert_property
 - DDSPROPERTYQosPolicyHelper, [767](#)
- attach_condition
 - DDSWaitSet, [802](#)
- autoenable_created_entities
 - DDS_EntityFactoryQosPolicy, [564](#)
- bind
 - ZCOPY_NotifUserInterfaceI, [908](#)
- bind_address
 - NETIO_DGRAM_InterfaceI, [866](#)
- bits
 - NETIO_Netmask, [872](#)
- builtin_checksum128_class
 - RTPS Transport API, [153](#)
- builtin_checksum32_class
 - RTPS Transport API, [152](#)
- builtin_checksum64_class
 - RTPS Transport API, [152](#)
- builtin_endpoint_reader_nack_period
 - DPDE_DiscoveryPluginProperty, [808](#)
- builtin_multicast_port_offset
 - DDS_RtpsWellKnownPorts_t, [637](#)
- builtin_unicast_port_offset
 - DDS_RtpsWellKnownPorts_t, [638](#)
- builtin_writer_heartbeat_period
 - DPDE_DiscoveryPluginProperty, [808](#)
- builtin_writer_heartbeats_per_max_samples
 - DPDE_DiscoveryPluginProperty, [808](#)
- builtin_writer_max_heartbeat_retries
 - DPDE_DiscoveryPluginProperty, [807](#)
- bytes_per_token
 - DDS_FlowControllerTokenBucketProperty_t, [573](#)
- cache_serialized_samples
 - DPDE_DiscoveryPluginProperty, [806](#)
- callback
 - PSK_ErrListener, [884](#)
 - PSK_FilePassTrackerErrListener, [885](#)
- CDR, [280](#)
 - CDR_LOG_INCR_OFFSET_EC, [281](#)
 - CDR_LOG_SET_OFFSET_EC, [280](#)
 - CDR_LOG_STREAM_ALLOC_EC, [280](#)
 - CDR_LOG_UNSUPPORTED_OPTIONS_EC, [281](#)
- CDR Stream API, [136](#)
 - CDR_Stream_check_size, [138](#)
 - CDR_Stream_get_current_position_offset, [137](#)
 - CDR_Stream_get_current_position_ptr, [138](#)
 - CDR_Stream_increment_current_position, [138](#)
 - CDR_Stream_is_byte_swapped, [140](#)
 - CDR_Stream_set_current_position_offset, [137](#)
- cdr_log.h, [911](#)
- CDR_LOG_BASE
 - osapi_log.h, [977](#)
- CDR_LOG_INCR_OFFSET_EC
 - CDR, [281](#)
- CDR_LOG_SET_OFFSET_EC
 - CDR, [280](#)
- CDR_LOG_STREAM_ALLOC_EC
 - CDR, [280](#)
- CDR_LOG_UNSUPPORTED_OPTIONS_EC
 - CDR, [281](#)
- CDR_Stream_check_size
 - CDR Stream API, [138](#)
- CDR_Stream_get_current_position_offset
 - CDR Stream API, [137](#)
- CDR_Stream_get_current_position_ptr
 - CDR Stream API, [138](#)
- CDR_Stream_increment_current_position
 - CDR Stream API, [138](#)
- CDR_Stream_is_byte_swapped
 - CDR Stream API, [140](#)
- CDR_Stream_set_current_position_offset
 - CDR Stream API, [137](#)
- CDR_Stream_t, [510](#)
- check_crc
 - DDS_WireProtocolQosPolicy, [667](#)
- checksum
 - DDS_ParticipantBuiltinTopicData, [596](#)
 - RTPS Transport API, [153](#)
- checksum128
 - RTPS Transport API, [152](#)
- checksum32

- RTPS Transport API, [152](#)
- checksum64
 - RTPS Transport API, [152](#)
- checksum_calculate
 - RTPS Transport API, [152](#)
- checksum_tx_mode
 - RTPS Transport API, [153](#)
- class_id
 - RTPS Transport API, [152](#)
- coherent_access
 - DDS_PresentationQosPolicy, [602](#)
- Compliance Configuration, [134](#)
- compute_crc
 - DDS_WireProtocolQosPolicy, [666](#)
- computed_crc_kind
 - DDS_ChecksumProperty_t, [512](#)
 - DDS_WireProtocolQosPolicy, [667](#)
- Conditions and WaitSets, [126](#)
- Content Filter Plugin, [31](#)
 - DDS_SQL_CONTENT_FILTER_CLASS, [31](#)
- CONTENT_FILTER, [99](#)
- content_filter
 - DDS_DataReaderQos, [522](#)
 - DDS_SubscriptionBuiltinTopicData, [653](#)
- content_filtered_topic_name
 - DDS_ContentFilterProperty, [513](#)
- context
 - RTPS Transport API, [152](#)
- copy_data
 - FooTypeSupport, [856](#)
- copy_no_alloc
 - FooSeq, [845](#)
- count
 - DDS_QosPolicyCount, [620](#)
- create_data
 - FooDataWriter, [839](#)
 - FooTypeSupport, [855](#)
- create_datareader
 - DDSSubscriber, [783](#)
- create_datawriter
 - DDSPublisher, [771](#)
- create_flowcontroller
 - DDSDomainParticipant, [737](#)
- create_instance
 - NETIO_DGRAM_Interfacel, [864](#)
 - ZCOPY_NotifUserInterfacel, [905](#)
- create_participant
 - DDSDomainParticipantFactory, [742](#)
- create_participant_from_config
 - DDSDomainParticipantFactory, [744](#)
- create_publisher
 - DDSDomainParticipant, [718](#)
- create_subscriber
 - DDSDomainParticipant, [719](#)

- create_topic
 - DDSDomainParticipant, [721](#)
- current_count
 - DDS_PublicationMatchedStatus, [616](#)
 - DDS_SubscriptionMatchedStatus, [654](#)
- current_count_change
 - DDS_PublicationMatchedStatus, [616](#)
 - DDS_SubscriptionMatchedStatus, [654](#)
- Data Sample, [68](#)
- DATA_READER_PROTOCOL, [119](#)
- DATA_REPRESENTATION, [106](#)
 - DDS_AUTO_DATA_REPRESENTATION, [108](#)
 - DDS_DataRepresentationId_t, [108](#)
 - DDS_XCDR2_DATA_REPRESENTATION, [107](#)
 - DDS_XCDR_DATA_REPRESENTATION, [107](#)
 - DDS_XML_DATA_REPRESENTATION, [107](#)
- DATA_WRITER_PROTOCOL, [119](#)
- DATA_WRITER_TRANSFER_MODE, [126](#)
 - transfer_mode, [126](#)
- DataReader, [65](#)
 - DDS_NOT_REJECTED, [67](#)
 - DDS_REJECTED_BY_INSTANCES_LIMIT, [67](#)
 - DDS_REJECTED_BY_REMOTE_WRITERS_LIMIT, [67](#)
 - DDS_REJECTED_BY_SAMPLES_LIMIT, [67](#)
 - DDS_REJECTED_BY_SAMPLES_PER_INSTANCE_LIMIT, [67](#)
 - DDS_REJECTED_BY_SAMPLES_PER_REMOTE_WRITER_LIMIT, [67](#)
 - DDS_SAMPLE_LOST_BY_DATAWRITER, [68](#)
 - DDS_SAMPLE_LOST_BY_HISTORY_DEPTH_LIMIT, [68](#)
 - DDS_SAMPLE_LOST_BY_MANAGEDPOOL_OVERWRITE, [68](#)
 - DDS_SAMPLE_LOST_BY_MAX_SAMPLES_LIMIT, [68](#)
 - DDS_SAMPLE_LOST_BY_META_SAMPLE_LIMIT, [68](#)
 - DDS_SAMPLE_LOST_BY_NOT_READ_ON_CACHE_DELETION, [68](#)
 - DDS_SAMPLE_LOST_NOT_LOST, [67](#)
 - DDS_SampleLostStatusKind, [67](#)
 - DDS_SampleRejectedStatusKind, [66](#)
- DataReader Resource Limits, [113](#)
 - DDS_DataReaderResourceLimitsInstanceReplacementKind, [113](#)
 - DDS_NO_INSTANCE_REPLACEMENT_QOS, [114](#)
 - DDS_REPLACE_OLDEST_INSTANCE_REPLACEMENT_QOS, [114](#)
- DataReaderQos, [231](#)
- DataWriter, [61](#)
 - DDS_DataWriterListener_ReliableReaderActivityChangedCallback, [62](#)

- DDS_DataWriterListener_ReliableSampleUnacknowledgedCallback, [63](#)
- DDS_DataWriterListener_SampleRemovedCallback, [63](#)
- DDS_ReliableReaderActivityChangedStatus_INITIALIZED, [62](#)
- DataWriter Resource Limits, [114](#)
- DataWriterQos, [236](#)
- DB, [272](#)
 - DB_LOG_ALLOC_CURSOR_POOL_EC, [276](#)
 - DB_LOG_ALLOC_DATABASE_EC, [277](#)
 - DB_LOG_ALLOC_INDEX_POOL_EC, [276](#)
 - DB_LOG_ALLOC_RECORD_POOL_EC, [277](#)
 - DB_LOG_ALLOC_TABLE_POOL_EC, [275](#)
 - DB_LOG_CURSOR_INVALIDATED_EC, [277](#)
 - DB_LOG_CURSORS_INUSE_EC, [276](#)
 - DB_LOG_ILLEGAL_LOCK_MODE_EC, [274](#)
 - DB_LOG_ILLEGAL_RECORD_COUNT_EC, [275](#)
 - DB_LOG_ILLEGAL_TABLE_SIZE_EC, [274](#)
 - DB_LOG_INDEX_INUSE_EC, [276](#)
 - DB_LOG_LOCK_FAILURE_EC, [277](#)
 - DB_LOG_MUTEX_ALLOC_EC, [274](#)
 - DB_LOG_NAME_TOO_LONG_EC, [274](#)
 - DB_LOG_OUT_OF_CTORDTOR_EC, [278](#)
 - DB_LOG_OUT_OF_CURSORS_EC, [276](#)
 - DB_LOG_OUT_OF_INDICES_EC, [276](#)
 - DB_LOG_OUT_OF_RECORDS_EC, [275](#)
 - DB_LOG_OUT_OF_TABLES_EC, [275](#)
 - DB_LOG_RECORD_ALREADY_EXISTS_EC, [277](#)
 - DB_LOG_RECORD_DOES_NOT_EXIST_EC, [277](#)
 - DB_LOG_RECORDS_INUSE_EC, [276](#)
 - DB_LOG_SORTED_ALLOC_EC, [274](#)
 - DB_LOG_TABLE_EXISTS_EC, [275](#)
 - DB_LOG_TABLE_NAME_TOO_LONG_EC, [275](#)
 - DB_LOG_TABLE_NOT_INUSE_EC, [277](#)
 - DB_LOG_TABLES_INUSE_EC, [275](#)
 - DB_LOG_UNLOCK_FAILURE_EC, [278](#)
- db_log.h, [912](#)
- DB_LOG_ALLOC_CURSOR_POOL_EC, [276](#)
- DB_LOG_ALLOC_DATABASE_EC, [277](#)
- DB_LOG_ALLOC_INDEX_POOL_EC, [276](#)
- DB_LOG_ALLOC_RECORD_POOL_EC, [277](#)
- DB_LOG_ALLOC_TABLE_POOL_EC, [275](#)
- DB_LOG_BASE
 - osapi_log.h, [976](#)
- DB_LOG_CURSOR_INVALIDATED_EC, [277](#)
- DB_LOG_CURSORS_INUSE_EC, [276](#)
- DB_LOG_ILLEGAL_LOCK_MODE_EC, [274](#)
- DB_LOG_ILLEGAL_RECORD_COUNT_EC, [275](#)
- DB_LOG_ILLEGAL_TABLE_SIZE_EC, [274](#)
- DB_LOG_INDEX_INUSE_EC, [276](#)
- DB_LOG_LOCK_FAILURE_EC, [277](#)
- DB_LOG_MUTEX_ALLOC_EC, [274](#)
- DB_LOG_NAME_TOO_LONG_EC, [274](#)
- DB_LOG_OUT_OF_CTORDTOR_EC, [278](#)
- DB_LOG_OUT_OF_CURSORS_EC, [276](#)
- DB_LOG_OUT_OF_INDICES_EC, [276](#)
- DB_LOG_OUT_OF_RECORDS_EC, [275](#)
- DB_LOG_OUT_OF_TABLES_EC, [275](#)
- DB_LOG_RECORD_ALREADY_EXISTS_EC, [277](#)
- DB_LOG_RECORD_DOES_NOT_EXIST_EC, [277](#)
- DB_LOG_RECORDS_INUSE_EC, [276](#)
- DB_LOG_SORTED_ALLOC_EC, [274](#)
- DB_LOG_TABLE_EXISTS_EC, [275](#)
- DB_LOG_TABLE_NAME_TOO_LONG_EC, [275](#)
- DB_LOG_TABLE_NOT_INUSE_EC, [277](#)
- DB_LOG_TABLES_INUSE_EC, [275](#)
- DB_LOG_UNLOCK_FAILURE_EC, [278](#)
- DDS API, [27](#)
- DDS-Specific Primitive Types, [43](#)
 - DDS_Boolean, [47](#)
 - DDS_BOOLEAN_FALSE, [44](#)
 - DDS_BOOLEAN_TRUE, [44](#)
 - DDS_Char, [45](#)
 - DDS_Double, [47](#)
 - DDS_Enum, [48](#)
 - DDS_Float, [47](#)
 - DDS_Long, [46](#)
 - DDS_LongDouble, [47](#)
 - DDS_LongLong, [46](#)

- DDS_Octet, [45](#)
- DDS_Short, [45](#)
- DDS_String, [48](#)
- DDS_UnsignedLong, [46](#)
- DDS_UnsignedLongLong, [46](#)
- DDS_UnsignedShort, [45](#)
- DDS_Wchar, [45](#)
- DDS_Wstring, [48](#)
- DDS_ALIVE_INSTANCE_STATE
 - Instance States, [74](#)
- DDS_ANY_INSTANCE_STATE
 - Instance States, [74](#)
- DDS_ANY_SAMPLE_STATE
 - Sample States, [70](#)
- DDS_ANY_VIEW_STATE
 - View States, [72](#)
- DDS_ASYNCHRONOUS_PUBLISH_MODE_QOS
 - PUBLISH_MODE, [123](#)
- DDS_AUTO_DATA_REPRESENTATION
 - DATA_REPRESENTATION, [108](#)
- DDS_AUTOMATIC_LIVELINESS_QOS
 - LIVELINESS, [103](#)
- DDS_AUTOMATIC_PUBLISH_MODE_QOS
 - PUBLISH_MODE, [123](#)
- DDS_BEST_EFFORT_RELIABILITY_QOS
 - RELIABILITY, [104](#)
- DDS_Boolean
 - DDS-Specific Primitive Types, [47](#)
- DDS_BOOLEAN_FALSE
 - DDS-Specific Primitive Types, [44](#)
- DDS_BOOLEAN_TRUE
 - DDS-Specific Primitive Types, [44](#)
- dds_builtin_endpoints
 - DDS_ParticipantBuiltinTopicData, [595](#)
- DDS_BuiltinTopicKey_t, [510](#)
 - Discovery, [38](#)
 - value, [511](#)
- DDS_BY_RECEPTION_TIMESTAMP_DESTINATIONORDER_QOS
 - DESTINATION_ORDER, [111](#)
- DDS_BY_SOURCE_TIMESTAMP_DESTINATIONORDER_QOS
 - DESTINATION_ORDER, [112](#)
- DDS_C, [340](#)
 - DDS_LOG_IPC_ASSERT_ROUTES_EC, [425](#)
 - DDS_LOG_TRUST_ASSERT_PARTICIPANT_GENERIC_MESSAGE_FILTERING_NOT_ENABLED_EC, [436](#)
 - DDSC_LOG_ADDRESSRESOLVER_OBJECT, [380](#)
 - DDSC_LOG_APPGEN_COMPONENT, [390](#)
 - DDSC_LOG_APPLY_READER_FILTER_EC, [450](#)
 - DDSC_LOG_AUTHPOOL_OBJECT, [384](#)
 - DDSC_LOG_BIND_RECORD, [375](#)
 - DDSC_LOG_BINDRESOLVER_OBJECT, [380](#)
 - DDSC_LOG_BUFFER_MAX_EXCEEDED_EC, [448](#)
 - DDSC_LOG_BUFFER_OBJECT, [384](#)
 - DDSC_LOG_CDR_BUFFER_SET_EC, [402](#)
 - DDSC_LOG_CDR_DESERIALIZE_DATA_EC, [404](#)
 - DDSC_LOG_CDR_DESERIALIZE_HEADER_EC, [404](#)
 - DDSC_LOG_CDR_DESERIALIZE_KEY_EC, [405](#)
 - DDSC_LOG_CDR_DESERIALIZE_KEYHASH_EC, [405](#)
 - DDSC_LOG_CDR_DESERIALIZE_PID_EC, [403](#)
 - DDSC_LOG_CDR_DESERIALIZE_PID_LENGTH_EC, [404](#)
 - DDSC_LOG_CDR_FINALIZE_SAMPLE_EC, [404](#)
 - DDSC_LOG_CDR_INCREMENT_POS_EC, [404](#)
 - DDSC_LOG_CDR_INITIALIZE_SAMPLE_EC, [404](#)
 - DDSC_LOG_CDR_POOL_ALLOC_EC, [402](#)
 - DDSC_LOG_CDR_POOL_DELETE_EC, [403](#)
 - DDSC_LOG_CDR_SERIALIZE_DATA_EC, [403](#)
 - DDSC_LOG_CDR_SERIALIZE_KEYHASH_EC, [403](#)
 - DDSC_LOG_CDR_SERIALIZE_PID_EC, [403](#)
 - DDSC_LOG_CDR_SERIALIZE_PID_LENGTH_EC, [403](#)
 - DDSC_LOG_CDR_SERIALIZE_STATUS_INFO_EC, [405](#)
 - DDSC_LOG_CDR_SET_OFFSET_EC, [420](#)
 - DDSC_LOG_CDR_SET_POS_EC, [404](#)
 - DDSC_LOG_CHANGE_CONTENT_FILTER_EC, [451](#)
 - DDSC_LOG_CHECKSUM_INCOMPATIBLE_EC, [449](#)
 - DDSC_LOG_CHECKSUM_INCONSISTENT_ALLOW_EC, [449](#)
 - DDSC_LOG_CHECKSUM_INCONSISTENT_CLASS_EC, [449](#)
 - DDSC_LOG_CHECKSUM_INCONSISTENT_COMPUTE_EC, [449](#)
 - DDSC_LOG_CHECKSUM_REQUIRED_EC, [448](#)
 - DDSC_LOG_COMPILE_CONTENT_FILTER_EC, [450](#)
 - DDSC_LOG_COMPONENT_CREATE_EC, [391](#)
 - DDSC_LOG_COMPONENT_DELETE_EC, [391](#)
 - DDSC_LOG_COMPONENT_LOOKUP_EC, [390](#)
 - DDSC_LOG_CONDITION_OBJECT, [381](#)
 - DDSC_LOG_CONDREF_OBJECT, [381](#)
 - DDSC_LOG_CONTENT_FILTER_QOS_POLICY, [396](#)
 - DDSC_LOG_CREATE_FILTER_PLUGIN_EC, [451](#)
 - DDSC_LOG_CREATE_INDEX_EC, [373](#)
 - DDSC_LOG_CREATE_WRITER_FILTER_EC, [450](#)
 - DDSC_LOG_DATA_HOLDER_SEQ_OBJECT, [383](#)
 - DDSC_LOG_DATA_REPRESENTATION_QOS_POLICY, [396](#)
 - DDSC_LOG_DATABASE_CREATE_EC, [373](#)
 - DDSC_LOG_DATABASE_DELETE_EC, [373](#)

- DDSC_LOG_DATAHOLDER_SEQUENCE, [388](#)
 DDSC_LOG_DATAREADER_ADD_PEER_FAILED_EC, [422](#)
 DDSC_LOG_DATAREADER_CDR, [402](#)
 DDSC_LOG_DATAREADER_ENTITY, [400](#)
 DDSC_LOG_DATAREADER_LISTENER, [398](#)
 DDSC_LOG_DATAREADER_NETIO_KIND, [406](#)
 DDSC_LOG_DATAREADER_QOS, [392](#)
 DDSC_LOG_DATAREADER_RECORD, [375](#)
 DDSC_LOG_DATAREADER_RESOURCE_QOS_POLICY, [396](#)
 DDSC_LOG_DATAREADERIO_COMPONENT, [390](#)
 DDSC_LOG_DATAREADERIO_OBJECT, [380](#)
 DDSC_LOG_DATAREADERQOS_OBJECT, [379](#)
 DDSC_LOG_DATAWRITER_ADD_PEER_FAILED_EC, [422](#)
 DDSC_LOG_DATAWRITER_CDR, [402](#)
 DDSC_LOG_DATAWRITER_ENTITY, [400](#)
 DDSC_LOG_DATAWRITER_INLINE, [402](#)
 DDSC_LOG_DATAWRITER_LISTENER, [398](#)
 DDSC_LOG_DATAWRITER_NETIO_KIND, [406](#)
 DDSC_LOG_DATAWRITER_QOS, [392](#)
 DDSC_LOG_DATAWRITER_RECORD, [374](#)
 DDSC_LOG_DATAWRITER_RESOURCE_QOS_POLICY, [395](#)
 DDSC_LOG_DATAWRITERIO_COMPONENT, [390](#)
 DDSC_LOG_DATAWRITERIO_OBJECT, [380](#)
 DDSC_LOG_DATAWRITERQOS_OBJECT, [379](#)
 DDSC_LOG_DB_CURSOR_INVALIDATED_EC, [374](#)
 DDSC_LOG_DEADLINE_QOS_POLICY, [394](#)
 DDSC_LOG_DEFAULTDATAREADER_QOS, [393](#)
 DDSC_LOG_DEFAULTDATAWRITER_QOS, [393](#)
 DDSC_LOG_DEFAULTPARTICIPANT_QOS, [393](#)
 DDSC_LOG_DEFAULTPUBLISHER_QOS, [393](#)
 DDSC_LOG_DEFAULTSUBSCRIBER_QOS, [393](#)
 DDSC_LOG_DEFAULTTOPIC_QOS, [393](#)
 DDSC_LOG_DELETE_INDEX_EC, [374](#)
 DDSC_LOG_DESERIALIZE_BAD_PID_LENGTH_EC, [403](#)
 DDSC_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC, [420](#)
 DDSC_LOG_DESERIALIZE_CONTENT_FILTER_INFO_EC, [451](#)
 DDSC_LOG_DESERIALIZE_LOCATOR_EC, [421](#)
 DDSC_LOG_DESERIALIZE_PARTITION_SEQ_EC, [422](#)
 DDSC_LOG_DESERIALIZE_PROTOCOL_VERSION_EC, [421](#)
 DDSC_LOG_DESERIALIZE_REMOTE_CONTENT_FILTER_EC, [451](#)
 DDSC_LOG_DESERIALIZE_UNKNOWN_PID_EC, [419](#)
 DDSC_LOG_DESERIALIZE_VENDOR_ID_EC, [421](#)
 DDSC_LOG_DESTINATION_ORDER_POLICY, [396](#)
 DDSC_LOG_DESTINATION_SEQUENCE, [387](#)
 DDSC_LOG_DISC_ADD_PEER_EC, [419](#)
 DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_DELETED_EC, [419](#)
 DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_ENABLED_EC, [418](#)
 DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_DELETED_EC, [419](#)
 DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_ENABLED_EC, [419](#)
 DDSC_LOG_DISC_AFTER_LOCAL_PARTICIPANT_CREATED_EC, [418](#)
 DDSC_LOG_DISC_BEFORE_LOCAL_PARTICIPANT_CREATED_EC, [418](#)
 DDSC_LOG_DISC_BEFORE_REMOTE_PARTICIPANT_DELETED_EC, [419](#)
 DDSC_LOG_DISC_LOCAL_PARTICIPANT_ENABLED_EC, [418](#)
 DDSC_LOG_DISC_REMOTE_PARTICIPANT_REMOVE_EC, [422](#)
 DDSC_LOG_DISCOVERY_COMPONENT, [389](#)
 DDSC_LOG_DR_COMMIT_ENTRY_EC, [412](#)
 DDSC_LOG_DR_COMMIT_SAMPLE_EC, [412](#)
 DDSC_LOG_DR_COPY_DATA_SAMPLE_EC, [412](#)
 DDSC_LOG_DR_CREATE_TYPED_READER_EC, [411](#)
 DDSC_LOG_DR_DESERIALIZE_KEYHASH_EC, [412](#)
 DDSC_LOG_DR_DISPOSE_KEY_EC, [413](#)
 DDSC_LOG_DR_FILTER_ERROR_EC, [412](#)
 DDSC_LOG_DR_GET_ENTRY_FAILED_EC, [412](#)
 DDSC_LOG_DR_INSTANCE_ASSERTION_FAILED_EC, [413](#)
 DDSC_LOG_DR_INSTANCE_MAPPING_EXHAUSTED_EC, [413](#)
 DDSC_LOG_DR_INSTANCE_TO_KEYHASH_EC, [413](#)
 DDSC_LOG_DR_ON_SAMPLE_REMOVED_EC, [413](#)
 DDSC_LOG_DR_READ_TAKE_FAILURE_EC, [413](#)
 DDSC_LOG_DR_UNREGISTER_KEY_EC, [412](#)
 DDSC_LOG_DUPLICATE_GUID_PREFIX_EC, [416](#)
 DDSC_LOG_DUPLICATE_PARTICIPANT_NAME_EC, [417](#)
 DDSC_LOG_DUPLICATE_REMOTE_PARTICIPANT_NAME_EC, [417](#)
 DDSC_LOG_DURABILITY_QOS_POLICY, [395](#)
 DDSC_LOG_DW_ACKNACK_FAILED_EC, [410](#)
 DDSC_LOG_DW_COMMIT_EC, [410](#)
 DDSC_LOG_DW_CREATE_TYPED_WRITER_EC, [410](#)
 DDSC_LOG_DW_FLOW_CONTROLLER_REQUIRED_EC, [411](#)
 DDSC_LOG_DW_HISTORY_REGISTER_KEY_EC,

- 410
 DDSC_LOG_DW_ILLEGAL_KEY_KIND_EC, 410
 DDSC_LOG_DW_INCOMPATIBLE_PADDING_EC, 411
 DDSC_LOG_DW_INSTANCE_ASSERTION_FAILED_EC, 411
 DDSC_LOG_DW_KEYHASH_CREATE_EC, 410
 DDSC_LOG_DW_UNKNOWN_FLOW_CONTROLLER_EC, 410
 DDSC_LOG_ENABLE_WRITER_FILTERING_EC, 450
 DDSC_LOG_ENABLED_DISCOVERY_TRANSPORT_SEQUENCE, 387
 DDSC_LOG_ENABLED_TRANSPORT_SEQUENCE, 386
 DDSC_LOG_ENABLED_USER_TRANSPORT_SEQUENCE, 386
 DDSC_LOG_ENDPOINT_NOT_CHILD_OF_PARTICIPANT_EC, 368
 DDSC_LOG_ENTITY_DIFFERENT_FACTORY_EC, 402
 DDSC_LOG_ENTITY_ENABLE_EC, 401
 DDSC_LOG_ENTITY_FINALIZE_EC, 401
 DDSC_LOG_ENTITY_INITIALIZE_EC, 401
 DDSC_LOG_ENTITY_INVALID_PROPERTY_EC, 402
 DDSC_LOG_ENTITY_NAME_TOO_LONG_EC, 446
 DDSC_LOG_ENTITY_NOT_EMPTY_EC, 401
 DDSC_LOG_ENTITY_NOT_ENABLED_EC, 401
 DDSC_LOG_ENTITY_OBJECT, 379
 DDSC_LOG_EVALUATE_CONTENT_FILTER_EC, 451
 DDSC_LOG_EVALUATE_WRITER_FILTER_EC, 450
 DDSC_LOG_FAILED_UPDATE_STATUS_CONDITION_EC, 370
 DDSC_LOG_FILTER_PLUGIN_FACTORY_NOT_FOUND_EC, 450
 DDSC_LOG_FINALIZE_FILTER_PLUGIN_EC, 451
 DDSC_LOG_FINALIZE_TRUST_FAILED_EC, 438
 DDSC_LOG_FIND_PUBLICATION_PARENT_EC, 369
 DDSC_LOG_FIND_SUBSCRIPTION_PARENT_EC, 369
 DDSC_LOG_FLOW_CONTROLLER_CREATE_EC, 447
 DDSC_LOG_FLOW_CONTROLLER_DELETE_EC, 447
 DDSC_LOG_FLOWCONTROLLER_INCONSISTENT_PROPERTIES_EC, 449
 DDSC_LOG_GET_MTU_EC, 372
 DDSC_LOG_GET_MTU_NO_ROUTES_EC, 372
 DDSC_LOG_GET_NEXT_OBJECT_ID_EC, 367
 DDSC_LOG_GET_SERIALIZED_KEY_SIZE_EC, 371
 DDSC_LOG_HEARTBEATS_QOS_POLICY, 395
 DDSC_LOG_HISTORY_QOS_POLICY, 394
 DDSC_LOG_HISTORY_RESOURCE, 391
 DDSC_LOG_ILLEGAL_OBJECTID_EC, 370
 DDSC_LOG_INCOMPATIBLE_PARTITION_QOS_EC, 448
 DDSC_LOG_INCOMPATIBLE_PRESENTATION_QOS_EC, 448
 DDSC_LOG_INITIAL_PEER_SEQUENCE, 387
 DDSC_LOG_INITIALIZE_DISCOVERY_QUEUE_EC, 422
 DDSC_LOG_INTERPRETER_SUPPORT_UNEXPECTED_ERROR_EC, 447
 DDSC_LOG_INTRA_NETIO_KIND, 406
 DDSC_LOG_INVALID_BUFFER_LENGTH_EC, 448
 DDSC_LOG_INVALID_DOMAINID_EC, 371
 DDSC_LOG_INVALID_DURATION_EC, 366
 DDSC_LOG_INVALID_ENDPOINT_GUID_EC, 368
 DDSC_LOG_INVALID_PARTICIPANT_GUID_PREFIX_EC, 366
 DDSC_LOG_INVALID_PARTICIPANT_NAME_EC, 366
 DDSC_LOG_IO_SNPRINTF_FAILED_EC, 367
 DDSC_LOG_IPC_ALLOC_FAILED_EC, 423
 DDSC_LOG_IPC_CHANNEL_ADD_ANON_ROUTE_EC, 424
 DDSC_LOG_IPC_CHANNEL_ADD_PEER_EC, 425
 DDSC_LOG_IPC_CHANNEL_CONFIG_FINALIZE_FAILED_EC, 426
 DDSC_LOG_IPC_CHANNEL_CONFIG_INITIALIZE_FAILED_EC, 426
 DDSC_LOG_IPC_CHANNEL_CONFIG_SET_FAILED_EC, 426
 DDSC_LOG_IPC_CHANNEL_CREATE_FAILED_EC, 427
 DDSC_LOG_IPC_CHANNEL_DELETE_ANON_ROUTE_EC, 425
 DDSC_LOG_IPC_CHANNEL_ENABLE_FAILED_EC, 427
 DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_AUTH_HANDSHAKE_EC, 426
 DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_AUTH_REQUEST_EC, 426
 DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_ENDPOINT_INTERACTION_EC, 426
 DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_PARTICIPANT_INTERACTION_EC, 426
 DDSC_LOG_IPC_CHANNEL_PROCESS_SAMPLE_EC, 425
 DDSC_LOG_IPC_CHANNEL_READER_CREATE_EC, 424
 DDSC_LOG_IPC_CHANNEL_REMOVE_PEER_EC, 425

- DDSC_LOG_IPC_CHANNEL_RETURN_LOAN_EC, 425
- DDSC_LOG_IPC_CHANNEL_SET_LISTENERS_FAILED_EC, 427
- DDSC_LOG_IPC_CHANNEL_TOPIC_CREATE_EC, 424
- DDSC_LOG_IPC_CHANNEL_TYPE_PLUGIN_EC, 424
- DDSC_LOG_IPC_CHANNEL_TYPE_REGISTER_EC, 423
- DDSC_LOG_IPC_CHANNEL_UNKNOWN_EC, 424
- DDSC_LOG_IPC_CHANNEL_WRITE_SAMPLE_EC, 425
- DDSC_LOG_IPC_CHANNEL_WRITER_CREATE_EC, 424
- DDSC_LOG_IPC_FINALIZE_FAILED_EC, 423
- DDSC_LOG_IPC_INIT_FAILED_EC, 423
- DDSC_LOG_IPC_UPDATE_QOS_FAILED_EC, 424
- DDSC_LOG_LATENCY_BUDGET_QOS_POLICY, 396
- DDSC_LOG_LISTENER_GET_EC, 399
- DDSC_LOG_LISTENER_INCONSISTENT_EC, 399
- DDSC_LOG_LISTENER_SET_EC, 399
- DDSC_LOG_LISTENER_SET_ILLEGAL_NULL_EC, 400
- DDSC_LOG_LIVELINESS_QOS_POLICY, 394
- DDSC_LOG_LOCATORS_FULL_EC, 421
- DDSC_LOG_LOCK_EC, 449
- DDSC_LOG_LOG_OBJECT, 381
- DDSC_LOG_LOOKUP_TYPE_PLUGIN_EC, 415
- DDSC_LOG_MATCHED_DW_RECORD, 376
- DDSC_LOG_MAX_PARTICIPANT_ID_REACHED_EC, 369
- DDSC_LOG_MD5STREAM_OBJECT, 381
- DDSC_LOG_MEMORY_MANAGER_DELETE_EC, 448
- DDSC_LOG_MEMORY_MANAGER_NOT_OWNER_EC, 448
- DDSC_LOG_MEMORY_MANAGER_OUT_OF_RESOURCES_EC, 447
- DDSC_LOG_MEMPOOL_OBJECT, 383
- DDSC_LOG_MESSAGE_DATA_UNKNOWN_LIVELINESS_KIND_EC, 446
- DDSC_LOG_MESSAGE_DATA_WRONG_LIVELINESS_KIND_EC, 446
- DDSC_LOG_METAMULTICAST_SEQUENCE, 387
- DDSC_LOG_METAUNICAST_SEQUENCE, 387
- DDSC_LOG_MIN_DISCOVERY_MTU_EC, 371
- DDSC_LOG_MIN_MTU_SIZE_EC, 372
- DDSC_LOG_MUTEX_OBJECT, 381
- DDSC_LOG_NAME_LOOKUP_EC, 445
- DDSC_LOG_NETIO_ADD_ANON_TOPIC_ROUTE_EC, 405
- DDSC_LOG_NETIO_ADD_ROUTE_EC, 407
- DDSC_LOG_NETIO_ADD_TOPIC_ROUTE_EC, 405
- DDSC_LOG_NETIO_BIND_EC, 407
- DDSC_LOG_NETIO_BIND_EXTERNAL_EC, 406
- DDSC_LOG_NETIO_COMPONENT, 390
- DDSC_LOG_NETIO_DELETE_ROUTE_EC, 407
- DDSC_LOG_NETIO_DELETE_TOPIC_ROUTE_EC, 405
- DDSC_LOG_NETIO_DISCOVERY_ENABLED_TRANSPORT_EC, 409
- DDSC_LOG_NETIO_FORCED_REMOVE_EC, 408
- DDSC_LOG_NETIO_FORWARD_TOPIC_EC, 405
- DDSC_LOG_NETIO_GET_EXTERNAL_INTF_EC, 407
- DDSC_LOG_NETIO_GET_ROUTE_TABLE_FAILED_EC, 408
- DDSC_LOG_NETIO_NETIO_KIND, 406
- DDSC_LOG_NETIO_NO_ROUTE_EC, 407
- DDSC_LOG_NETIO_PACKETPOOL_CREATE_EC, 409
- DDSC_LOG_NETIO_PACKETPOOL_DELETE_EC, 409
- DDSC_LOG_NETIO_PACKETPOOL_GET_EC, 409
- DDSC_LOG_NETIO_PACKETPOOL_RETURN_EC, 409
- DDSC_LOG_NETIO_PEER_LOOKUP_EC, 408
- DDSC_LOG_NETIO_ROUTE_LOOKUP_FAILED_EC, 407
- DDSC_LOG_NETIO_SEND_FAILED_EC, 408
- DDSC_LOG_NETIO_SET_STATE_EC, 408
- DDSC_LOG_NETIO_UNBIND_EC, 407
- DDSC_LOG_NETIO_UNBIND_EXTERNAL_EC, 406
- DDSC_LOG_NETMASK_SEQUENCE, 386
- DDSC_LOG_OBJECT_ALLOCATE_EC, 384
- DDSC_LOG_OBJECT_COPY_EC, 385
- DDSC_LOG_OBJECT_CREATE_EC, 386
- DDSC_LOG_OBJECT_DELETE_EC, 385
- DDSC_LOG_OBJECT_EMPTY_EC, 385
- DDSC_LOG_OBJECT_FINALIZE_EC, 385
- DDSC_LOG_OBJECT_GET_PROPERTY_EC, 385
- DDSC_LOG_OBJECT_INITIALIZE_EC, 384
- DDSC_LOG_OBJECT_INUSECOUNT_EC, 386
- DDSC_LOG_OBJECT_REFCOUNT_EC, 385
- DDSC_LOG_OBJECT_SET_PROPERTY_EC, 385
- DDSC_LOG_ON_TYPE_REGISTERED_FAILURE_EC, 371
- DDSC_LOG_ON_TYPE_UNREGISTERED_FAILURE_EC, 371
- DDSC_LOG_OWNERSHIP_QOS_POLICY, 395
- DDSC_LOG_OWNERSHIP_STRENGTH_QOS_POLICY, 395
- DDSC_LOG_PACKET_INIT_EC, 408
- DDSC_LOG_PACKET_SET_HEAD_EC, 409
- DDSC_LOG_PACKET_SET_TAIL_EC, 409

- DDSC_LOG_PARTICIPANT_BINARY_PROPERTIES, 383
- DDSC_LOG_PARTICIPANT_BUILTIN_PUBLISHER, 383
- DDSC_LOG_PARTICIPANT_BUILTIN_SUBSCRIBER, 383
- DDSC_LOG_PARTICIPANT_DOES_NOT_EXIST_EC, 368
- DDSC_LOG_PARTICIPANT_ENTITY, 400
- DDSC_LOG_PARTICIPANT_GENERIC_MESSAGE_OBJECT, 383
- DDSC_LOG_PARTICIPANT_ID_QOS_POLICY, 395
- DDSC_LOG_PARTICIPANT_LISTENER, 399
- DDSC_LOG_PARTICIPANT_LOOKUP_EC, 368
- DDSC_LOG_PARTICIPANT_NAME_EMPTY_EC, 416
- DDSC_LOG_PARTICIPANT_POOL_OBJECT, 382
- DDSC_LOG_PARTICIPANT_PROPERTIES, 383
- DDSC_LOG_PARTICIPANT_QOS, 392
- DDSC_LOG_PARTICIPANT_RECORD, 374
- DDSC_LOG_PARTICIPANT_RESOURCES, 391
- DDSC_LOG_PARTICIPANTDATA_OBJECT, 378
- DDSC_LOG_PARTICIPANTFACTORY_LISTENER, 399
- DDSC_LOG_PARTICIPANTFACTORY_QOS, 393
- DDSC_LOG_PARTICIPANTFACTORYQOS_OBJECT, 380
- DDSC_LOG_PARTICIPANTQOS_OBJECT, 379
- DDSC_LOG_PARTMESSAGE_WRITE_SAMPLE_FAILED_EC, 446
- DDSC_LOG_PROPERTY_QOS_POLICY, 396
- DDSC_LOG_PROPERTYQOSPOLICY_OBJECT, 384
- DDSC_LOG_PROTOCOL_QOS_POLICY, 394
- DDSC_LOG_PUBLICATION_ENTITY, 401
- DDSC_LOG_PUBLICATION_RECORD, 374
- DDSC_LOG_PUBLICATIONDATA_OBJECT, 378
- DDSC_LOG_PUBLISH_MODE_QOS_POLICY, 396
- DDSC_LOG_PUBLISHER_ENTITY, 400
- DDSC_LOG_PUBLISHER_LISTENER, 399
- DDSC_LOG_PUBLISHER_QOS, 392
- DDSC_LOG_PUBLISHER_RECORD, 375
- DDSC_LOG_PUBLISHERQOS_OBJECT, 379
- DDSC_LOG_QOS_CHANGED_EC, 398
- DDSC_LOG_QOS_COPY_EC, 397
- DDSC_LOG_QOS_FINALIZE_EC, 397
- DDSC_LOG_QOS_GET_EC, 398
- DDSC_LOG_QOS_IMMUTABLE_EC, 398
- DDSC_LOG_QOS_INCONSISTENT_EC, 397
- DDSC_LOG_QOS_INCONSISTENT_POLICIES_EC, 397
- DDSC_LOG_QOS_INCONSISTENT_POLICY_EC, 397
- DDSC_LOG_QOS_INITIALIZE_EC, 397
- DDSC_LOG_QOS_SET_EC, 397
- DDSC_LOG_QOS_SET_ON_ENABLED_EC, 398
- DDSC_LOG_QUEUE_FINALIZE_EC, 423
- DDSC_LOG_QUEUE_PUBLICATION_DISCOVERY_EC, 423
- DDSC_LOG_QUEUE_SUBSCRIPTION_DISCOVERY_EC, 423
- DDSC_LOG_READERHISTORY_COMPONENT, 389
- DDSC_LOG_READTAKE_SEQUENCE, 388
- DDSC_LOG_RECORD_CREATE_EC, 376
- DDSC_LOG_RECORD_DELETE_EC, 376
- DDSC_LOG_RECORD_ERROR_EC, 377
- DDSC_LOG_RECORD_EXISTS_EC, 377
- DDSC_LOG_RECORD_FINALIZE_EC, 378
- DDSC_LOG_RECORD_INITIALIZE_EC, 378
- DDSC_LOG_RECORD_INSERT_EC, 377
- DDSC_LOG_RECORD_LOOKUP_EC, 377
- DDSC_LOG_RECORD_NOT_EXISTS_EC, 377
- DDSC_LOG_RECORD_REMOVE_EC, 377
- DDSC_LOG_RECORD_RESET_EC, 378
- DDSC_LOG_RECORD_SELECT_EC, 377
- DDSC_LOG_REFRESH_LIVELINESS_FAILED_EC, 447
- DDSC_LOG_REFRESH_REM_PARTICIPANT_EC, 368
- DDSC_LOG_REFRESH_REM_PARTICIPANT_TIMEOUT_EC, 368
- DDSC_LOG_RELEASE_META_MC_EC, 370
- DDSC_LOG_RELEASE_META_UC_EC, 370
- DDSC_LOG_RELEASE_USER_MC_EC, 371
- DDSC_LOG_RELEASE_USER_UC_EC, 371
- DDSC_LOG_RELIABILITY_QOS_POLICY, 394
- DDSC_LOG_REMOTE_ENDPOINT_TRUST_STATE_RECORD, 375
- DDSC_LOG_REMOTE_PARTICIPANT_KEY_NOT_EQUAL_EC, 368
- DDSC_LOG_REMOTE_PARTICIPANT_NAME_EMPTY_EC, 417
- DDSC_LOG_REMOTE_PARTICIPANT_RESTART_EC, 418
- DDSC_LOG_REMOTE_PUBLICATION_DATA_CHANGED_EC, 372
- DDSC_LOG_REMOTE_SUBSCRIPTION_DATA_CHANGED_EC, 372
- DDSC_LOG_REMOVE_PUBLICATION_EC, 369
- DDSC_LOG_REMOVE_SUBSCRIPTION_EC, 369
- DDSC_LOG_REPRESENTATION_ID_SEQUENCE, 388
- DDSC_LOG_RESERVE_LOCATORS_EC, 369
- DDSC_LOG_RESERVED_SEQUENCE, 386
- DDSC_LOG_RESOLVE_LOCATORS_PRIORITY_EC, 372
- DDSC_LOG_RESOURCE_EXCEEDED_EC, 392

- DDSC_LOG_RESOURCE_LIMIT_QOS_POLICY, [394](#)
- DDSC_LOG_ROUTE_RECORD, [375](#)
- DDSC_LOG_ROUTE_SEQUENCE, [386](#)
- DDSC_LOG_ROUTERESOLVER_OBJECT, [380](#)
- DDSC_LOG_RT_OBJECT, [382](#)
- DDSC_LOG_RTPS_COMPONENT, [389](#)
- DDSC_LOG_RTPS_NETIO_KIND, [406](#)
- DDSC_LOG_SAMPLE_RESOURCES, [391](#)
- DDSC_LOG_SEND_LIVELINESS_FAILED_EC, [446](#)
- DDSC_LOG_SEQUENCE_COPY_EC, [389](#)
- DDSC_LOG_SEQUENCE_FINALIZE_EC, [389](#)
- DDSC_LOG_SEQUENCE_GETREF_EC, [388](#)
- DDSC_LOG_SEQUENCE_INITIALIZE_EC, [389](#)
- DDSC_LOG_SEQUENCE_INVALID_EC, [389](#)
- DDSC_LOG_SEQUENCE_SETLENGTH_EC, [388](#)
- DDSC_LOG_SEQUENCE_SETMAX_EC, [388](#)
- DDSC_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC, [419](#)
- DDSC_LOG_SERIALIZE_DEFAULT_UNICAST_EC, [420](#)
- DDSC_LOG_SERIALIZE_ENTITY_NAME_EC, [420](#)
- DDSC_LOG_SERIALIZE_GUID_EC, [420](#)
- DDSC_LOG_SERIALIZE_LEASE_DURATION_EC, [422](#)
- DDSC_LOG_SERIALIZE_PRODUCT_VERSION_EC, [422](#)
- DDSC_LOG_SERIALIZE_PROTOCOL_VERSION_EC, [421](#)
- DDSC_LOG_SERIALIZE_TOPIC_NAME_EC, [420](#)
- DDSC_LOG_SERIALIZE_TYPE_NAME_EC, [420](#)
- DDSC_LOG_SERIALIZE_VENDOR_ID_EC, [421](#)
- DDSC_LOG_SET_ENTITY_NAME_EC, [367](#)
- DDSC_LOG_STRING_OBJECT, [384](#)
- DDSC_LOG_STRING_RECORD, [376](#)
- DDSC_LOG_STRINGMANAGER_OBJECT, [384](#)
- DDSC_LOG_SUBSCRIBER_ENTITY, [400](#)
- DDSC_LOG_SUBSCRIBER_LISTENER, [399](#)
- DDSC_LOG_SUBSCRIBER_QOS, [392](#)
- DDSC_LOG_SUBSCRIBER_RECORD, [375](#)
- DDSC_LOG_SUBSCRIBERQOS_OBJECT, [379](#)
- DDSC_LOG_SUBSCRIPTION_ENTITY, [401](#)
- DDSC_LOG_SUBSCRIPTION_RECORD, [374](#)
- DDSC_LOG_SUBSCRIPTIONDATA_OBJECT, [378](#)
- DDSC_LOG_SYS_GET_HOSTNAME_EC, [367](#)
- DDSC_LOG_SYS_GETTIME_EC, [366](#)
- DDSC_LOG_SYSTEM_OBJECT, [381](#)
- DDSC_LOG_TABLE_CREATE_EC, [373](#)
- DDSC_LOG_TABLE_DELETE_EC, [373](#)
- DDSC_LOG_TABLE_INUSE_EC, [373](#)
- DDSC_LOG_TABLE_SELECT_EC, [373](#)
- DDSC_LOG_TIMER_CREATE_TIMEOUT_EC, [369](#)
- DDSC_LOG_TIMER_OBJECT, [382](#)
- DDSC_LOG_TIMEROUT_OBJECT, [382](#)
- DDSC_LOG_TOPIC_ENTITY, [400](#)
- DDSC_LOG_TOPIC_FIND_EC, [367](#)
- DDSC_LOG_TOPIC_LISTENER, [398](#)
- DDSC_LOG_TOPIC_NAME_CMP_EC, [414](#)
- DDSC_LOG_TOPIC_NARROW_EC, [370](#)
- DDSC_LOG_TOPIC_OBJECT, [382](#)
- DDSC_LOG_TOPIC_QOS, [392](#)
- DDSC_LOG_TOPIC_RECORD, [374](#)
- DDSC_LOG_TOPIC_TOO_LONG_EC, [415](#)
- DDSC_LOG_TOPICQOS_OBJECT, [379](#)
- DDSC_LOG_TRANSPORT_COMPONENT, [390](#)
- DDSC_LOG_TRANSPORT_QOS_POLICY, [395](#)
- DDSC_LOG_TRUST_ADVANCE_SN_FAILED_EC, [439](#)
- DDSC_LOG_TRUST_AFTER_REMOTE_PARTICIPANT_AUTHENTICATED_EC, [443](#)
- DDSC_LOG_TRUST_AFTER_REMOTE_PARTICIPANT_READY_FAILED_EC, [435](#)
- DDSC_LOG_TRUST_ALLOC_FAILED_EC, [438](#)
- DDSC_LOG_TRUST_ASSERT_AUTH_HANDSHAKE_STATE_FAILED_EC, [444](#)
- DDSC_LOG_TRUST_ASSERT_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC, [440](#)
- DDSC_LOG_TRUST_ASSERT_PROPERTY_FAILED_EC, [442](#)
- DDSC_LOG_TRUST_BEGIN_HANDSHAKE_REPLY_EC, [432](#)
- DDSC_LOG_TRUST_BEGIN_HANDSHAKE_REQUEST_EC, [435](#)
- DDSC_LOG_TRUST_BIND_REMOTE_DATAREADER_FAILED_EC, [436](#)
- DDSC_LOG_TRUST_BIND_REMOTE_DATAWRITER_FAILED_EC, [436](#)
- DDSC_LOG_TRUST_CACHE_REMOTE_PARTICIPANT_SAMPLE_FAILED_EC, [443](#)
- DDSC_LOG_TRUST_CHECK_CREATE_DATAREADER_FAILED_EC, [432](#)
- DDSC_LOG_TRUST_CHECK_CREATE_DATAWRITER_FAILED_EC, [432](#)
- DDSC_LOG_TRUST_CHECK_CREATE_PARTICIPANT_FAILED_EC, [429](#)
- DDSC_LOG_TRUST_CHECK_REMOTE_DATAREADER_FAILED_EC, [434](#)
- DDSC_LOG_TRUST_CHECK_REMOTE_DATAWRITER_FAILED_EC, [434](#)
- DDSC_LOG_TRUST_CHECK_REMOTE_PARTICIPANT_FAILED_EC, [431](#)
- DDSC_LOG_TRUST_CREATE_LOCAL_DATAREADER_INTERCEPTOR_FAILED_EC, [441](#)
- DDSC_LOG_TRUST_CREATE_LOCAL_DATAWRITER_INTERCEPTOR_FAILED_EC, [441](#)
- DDSC_LOG_TRUST_CREATE_LOCAL_DR_TOKEN_EC, [433](#)
- DDSC_LOG_TRUST_CREATE_LOCAL_DW_TOKEN_EC, [433](#)

[433](#)
 DDSC_LOG_TRUST_CREATE_LOCAL_PARTICIPANT_INTERNAL_ATTRIBUTES_FAILED_EC, [438](#)
[432](#)
 DDSC_LOG_TRUST_CREATE_TRUST_PLUGIN_EC, [428](#)
[445](#)
 DDSC_LOG_TRUST_CREATE_TRUST_PLUGINS_FAILED_EC, [429](#)
[435](#)
 DDSC_LOG_TRUST_DATA_HOLDER_COPY_FAILED_EC, [441](#)
[438](#)
 DDSC_LOG_TRUST_DELETE_TRUST_PLUGINS_EC, [440](#)
[431](#)
 DDSC_LOG_TRUST_ENDPOINT_INTERNAL_ATTRIBUTES_CREATE_FAILED_EC, [439](#)
[441](#)
 DDSC_LOG_TRUST_FAILED_VALIDATE_LOCAL_IDENTITY_TOKEN_EC, [428](#)
[428](#)
 DDSC_LOG_TRUST_FINALIZE_AC_PLUGIN_PROPERTIES_FAILED_EC, [443](#)
[443](#)
 DDSC_LOG_TRUST_FINALIZE_AUTH_HANDSHAKE_STATE_FAILED_EC, [428](#)
[444](#)
 DDSC_LOG_TRUST_FINALIZE_REMOTE_PARTICIPANT_TRUST_FAILED_EC, [438](#)
[444](#)
 DDSC_LOG_TRUST_GET_AUTH_HANDSHAKE_STATE_FAILED_EC, [442](#)
[444](#)
 DDSC_LOG_TRUST_GET_AUTHENTICATED_PEER_CREDENTIALS_FAILED_EC, [445](#)
[430](#)
 DDSC_LOG_TRUST_GET_DATAREADER_TRUST_ATTRIBUTES_FAILED_EC, [435](#)
[432](#)
 DDSC_LOG_TRUST_GET_DATAWRITER_TRUST_ATTRIBUTES_FAILED_EC, [440](#)
[432](#)
 DDSC_LOG_TRUST_GET_IDENTITY_TOKEN_FAILED_EC, [443](#)
[428](#)
 DDSC_LOG_TRUST_GET_PARTICIPANT_TRUST_ATTRIBUTES_FAILED_EC, [442](#)
[429](#)
 DDSC_LOG_TRUST_GET_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC, [445](#)
[429](#)
 DDSC_LOG_TRUST_GET_PERMISSIONS_TOKEN_FAILED_EC, [437](#)
[429](#)
 DDSC_LOG_TRUST_GET_SHARED_SECRET_FAILED_EC, [437](#)
[431](#)
 DDSC_LOG_TRUST_GET_TOPIC_TRUST_ATTRIBUTES_FAILED_EC, [437](#)
[431](#)
 DDSC_LOG_TRUST_IGNORED_HANDSHAKE_MESSAGE_EC, [442](#)
[427](#)
 DDSC_LOG_TRUST_IGNORED_REMOTE_PARTICIPANT_ATTRIBUTES_EC, [431](#)
[427](#)
 DDSC_LOG_TRUST_INCOMPATIBLE_ENDPOINT_TRUST_ATTRIBUTES_EC, [435](#)
[447](#)
 DDSC_LOG_TRUST_INITIALIZE_PARTICIPANT_BUILTIN_ATTRIBUTES_FAILED_EC, [442](#)
[439](#)
 DDSC_LOG_TRUST_INITIALIZE_PARTICIPANT_TRUST_ATTRIBUTES_FAILED_EC, [445](#)
[438](#)
 DDSC_LOG_TRUST_INVALID_AUTH_HANDSHAKE_STATE_EC, [433](#)
[444](#)
 DDSC_LOG_TRUST_INVALID_GENERIC_MESSAGE_REPLY_EC, [434](#)
 DDSC_LOG_TRUST_INVALID_IDENTITY_HANDLE_EC, [438](#)
 DDSC_LOG_TRUST_INVALID_INTERCEPTOR_HANDLE_EC, [428](#)
 DDSC_LOG_TRUST_INVALID_INTERCEPTOR_TOKENS_DESTINATION_EC, [429](#)
 DDSC_LOG_TRUST_INVALID_PARTICIPANT_GENERIC_MESSAGE_EC, [441](#)
 DDSC_LOG_TRUST_INVALID_PARTICIPANT_GUID_EC, [440](#)
 DDSC_LOG_TRUST_INVALID_PARTICIPANT_INTERNAL_ATTRIBUTES_EC, [439](#)
 DDSC_LOG_TRUST_INVALID_PERMISSIONS_HANDLE_EC, [428](#)
 DDSC_LOG_TRUST_INVALID_SAMPLE_MAX_SERIALIZED_SIZE_EC, [443](#)
 DDSC_LOG_TRUST_INVALID_SHARED_SECRET_EC, [428](#)
 DDSC_LOG_TRUST_LOOKUP_FACTORY_FAILED_EC, [438](#)
 DDSC_LOG_TRUST_LOOKUP_FACTORY_PROPERTY_FAILED_EC, [442](#)
 DDSC_LOG_TRUST_LOOKUP_TOKEN_PARSE_EC, [445](#)
 DDSC_LOG_TRUST_PARTICIPANT_AUTHENTICATION_FAILED_EC, [435](#)
 DDSC_LOG_TRUST_PARTICIPANT_INTERNAL_ATTRIBUTES_CREATE_FAILED_EC, [440](#)
 DDSC_LOG_TRUST_PARTICIPANT_UNEXPECTED_AUTHENTICATION_EC, [443](#)
 DDSC_LOG_TRUST_PBUFS_MAX_EXCEEDED_EC, [442](#)
 DDSC_LOG_TRUST_PREPARE_AUTH_HANDSHAKE_MESSAGE_EC, [445](#)
 DDSC_LOG_TRUST_PREPARE_AUTH_REQUEST_EC, [437](#)
 DDSC_LOG_TRUST_PREPARE_ENDPOINT_INTERCEPTOR_MESSAGE_EC, [437](#)
 DDSC_LOG_TRUST_PREPARE_LOCAL_DP_TOKENS_EC, [437](#)
 DDSC_LOG_TRUST_PREPARE_LOCAL_PARTICIPANT_STATE_FAILED_EC, [442](#)
 DDSC_LOG_TRUST_PROCESS_HANDSHAKE_EC, [431](#)
 DDSC_LOG_TRUST_PROCESS_REMOTE_INTERCEPTOR_TOKENS_EC, [435](#)
 DDSC_LOG_TRUST_REAUTH_REMOTE_PARTICIPANT_FAILED_EC, [442](#)
 DDSC_LOG_TRUST_REGISTER_LOCAL_DATAREADER_FAILED_EC, [445](#)
 DDSC_LOG_TRUST_REGISTER_LOCAL_DATAWRITER_FAILED_EC, [433](#)
 DDSC_LOG_TRUST_REGISTER_LOCAL_PARTICIPANT_FAILED_EC, [434](#)

[429](#) DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_DATAREADER_FAILED_EC, [440](#) DDSC_LOG_FAILED_SERIALIZE_DATA_FAILED_EC,
[434](#) DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_DATAWRITER_FAILED_EC, [438](#) DDSC_LOG_FAILED_SET_AUTH_HANDSHAKE_STATE_FAILED_EC,
[434](#) DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_PARTICIPANT_FAILED_EC, [445](#) DDSC_LOG_FAILED_SET_PERMISSIONS_CREDENTIAL_TOKEN_F
[431](#) DDSC_LOG_TRUST_REGISTER_REMOTE_DATAREADER_FAILED_EC, [431](#) DDSC_LOG_FAILED_SET_REMOTE_DATAREADER_INTERCEPTOR
[436](#) DDSC_LOG_TRUST_REGISTER_REMOTE_DATAWRITER_FAILED_EC, [433](#) DDSC_LOG_FAILED_SET_REMOTE_DATAWRITER_INTERCEPTOR,
[436](#) DDSC_LOG_TRUST_RELEASE_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC, [433](#) DDSC_LOG_FAILED_SET_REMOTE_PARTICIPANT_INTERCEPTOR,
[439](#) DDSC_LOG_TRUST_REMOVE_REMOTE_PARTICIPANT_FAILED_EC, [433](#) DDSC_LOG_TRUST_TRANFORM_INCOMING_SERIALIZED_PAYLO
[435](#) DDSC_LOG_TRUST_RESEND_HANDSHAKE_IN_PROGRESS_FAILED_EC, [443](#) DDSC_LOG_TRUST_TRANFORM_OUTGOING_SERIALIZED_PAYLO
[439](#) DDSC_LOG_TRUST_RESEND_HANDSHAKE_MESSAGE_FAILED_EC, [443](#) DDSC_LOG_TRUST_UNAUTHORIZED_REMOTE_PARTICIPANT_EC,
[445](#) DDSC_LOG_TRUST_RESEND_HANDSHAKE_MESSAGE_START_FAILED_EC, [428](#) DDSC_LOG_FAILED_UNEXPECTED_REMOTE_PARTICIPANT_STAT
[440](#) DDSC_LOG_TRUST_RESEND_HANDSHAKE_STOP_FAILED_EC, [440](#) DDSC_LOG_TRUST_UNEXPECTED_VALIDATION_RESULT_EC,
[439](#) DDSC_LOG_TRUST_RESET_REMOTE_PARTICIPANT_TOKEN_FAILED_EC, [428](#) DDSC_LOG_FAILED_UNKNOWN_GMCLASSID_EC,
[444](#) DDSC_LOG_TRUST_RETURN_AUTHENTICATED_PEER_OBSOLETE_TOKEN_UNKNOWN_EC, [427](#) DDSC_LOG_FAILED_TOKEN_UNKNOWN_EC,
[441](#) DDSC_LOG_TRUST_RETURN_DATAWRITER_TRUST_ATTRIBUTES_FAILED_EC, [427](#) DDSC_LOG_FAILED_UNREGISTER_DATAREADER_FAILED_EC,
[432](#) DDSC_LOG_TRUST_RETURN_HANDSHAKE_HANDLE_FAILED_EC, [434](#) DDSC_LOG_TRUST_UNREGISTER_DATAWRITER_FAILED_EC,
[433](#) DDSC_LOG_TRUST_RETURN_IDENTITY_HANDLE_FAILED_EC, [434](#) DDSC_LOG_TRUST_UNREGISTER_PARTICIPANT_FAILED_EC,
[430](#) DDSC_LOG_TRUST_RETURN_IDENTITY_TOKEN_FAILED_EC, [430](#) DDSC_LOG_TRUST_UNREGISTER_REMOTE_DATAREADER_FAILED
[430](#) DDSC_LOG_TRUST_RETURN_INTERCEPTOR_TOKENS_FAILED_EC, [442](#) DDSC_LOG_TRUST_UNREGISTER_REMOTE_DATAWRITER_FAILED
[441](#) DDSC_LOG_TRUST_RETURN_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC, [442](#) DDSC_LOG_FAILED_VALIDATE_LOCAL_DATAWRITER_TRUST_FAIL
[429](#) DDSC_LOG_TRUST_RETURN_PERMISSIONS_HANDLE_FAILED_EC, [437](#) DDSC_LOG_TRUST_VALIDATE_REMOTE_DATAREADER_TRUST_F
[430](#) DDSC_LOG_TRUST_RETURN_PERMISSIONS_TOKEN_FAILED_EC, [437](#) DDSC_LOG_TRUST_VALIDATE_REMOTE_DATAWRITER_TRUST_FA
[430](#) DDSC_LOG_TRUST_RETURN_SHAREDSECRET_HANDLE_FAILED_EC, [437](#) DDSC_LOG_FAILED_VALIDATE_REMOTE_IDENTITY_EC,
[430](#) DDSC_LOG_TRUST_SEND_AUTH_REQUEST_FAILED_EC, [435](#) DDSC_LOG_FAILED_WRITE_SAMPLE_FAILED_EC,
[444](#) DDSC_LOG_TRUST_SEND_ENDPOINT_INTERCEPTOR_STAGE_FAILED_EC, [439](#) DDSC_LOG_TYPE_FUNCTION_COPY_SAMPLE,
[441](#) DDSC_LOG_TRUST_SEND_HANDSHAKE_EC, [414](#) DDSC_LOG_TYPE_FUNCTION_CREATE_SAMPLE,
[436](#) DDSC_LOG_TRUST_SEND_HANDSHAKE_MESSAGE_FAILED_EC, [414](#) DDSC_LOG_TYPE_FUNCTION_DELETE_SAMPLE,
[440](#) DDSC_LOG_TRUST_SEND_PARTICIPANT_INTERCEPTOR_STAGE_FAILED_EC, [414](#) DDSC_LOG_TYPE_FUNCTION_DESERIALIZE_DATA,

- 414
- DDSC_LOG_TYPE_FUNCTION_GET_KEY_KIND, [dds_c_log.h](#), 914
- 415 DDS_Char
- DDSC_LOG_TYPE_FUNCTION_GET_SERIALIZED_SAMPLE_HANDLE, [DDS-Specific Primitive Types](#), 45
- 414 DDS_CHECKSUM_BUILTIN128
- DDSC_LOG_TYPE_FUNCTION_INSTANCE_TO_KEY_HANDLE, [WIRE_PROTOCOL](#), 116
- 415 WIRE_PROTOCOL, 116
- DDSC_LOG_TYPE_FUNCTION_NULL_EC, 415 DDS_CHECKSUM_BUILTIN32
- DDSC_LOG_TYPE_FUNCTION_SERIALIZE_DATA, [WIRE_PROTOCOL](#), 116
- 414 DDS_CHECKSUM_BUILTIN64
- DDSC_LOG_TYPE_GET_SAMPLE_ID_EC, 416 [WIRE_PROTOCOL](#), 116
- DDSC_LOG_TYPE_KEY_TYPE_EC, 415 DDS_CHECKSUM_NONE
- DDSC_LOG_TYPE_MANAGED_POOL_FAILURE_EC, [WIRE_PROTOCOL](#), 116
- 416 DDS_ChecksumKind_t
- DDSC_LOG_TYPE_NAME_CMP_EC, 413 [WIRE_PROTOCOL](#), 117
- DDSC_LOG_TYPE_OBJECT, 380 DDS_ChecksumKindMask_t
- DDSC_LOG_TYPE_PLUGIN_GET_BUFFER_EC, [WIRE_PROTOCOL](#), 117
- 416 DDS_ChecksumProperty_t, 511
- DDSC_LOG_TYPE_PLUGIN_INCONSISTENT_DR_EC, [allowed_crc_mask](#), 512
- 416 [computed_crc_kind](#), 512
- DDSC_LOG_TYPE_RECORD, 375 [require_crc](#), 512
- DDSC_LOG_TYPE_SUPPORT_QOS_POLICY, 394 DDS_CONTENT_FILTER_CLASS_NAME_MAX_LENGTH
- DDSC_LOG_TYPE_TOO_LONG_EC, 415 [FILTER](#), 97
- DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_KEY_HANDLE, DDS_CONTENT_FILTER_EXPRESSION_MAX_LENGTH
- 367 [FILTER](#), 98
- DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_NAME_EC, DDS_CONTENT_FILTER_MAX_PARAMETERS
- 367 [FILTER](#), 98
- DDSC_LOG_UNLOCK_EC, 449 DDS_CONTENT_FILTER_NAME_MAX_LENGTH
- DDSC_LOG_UNSUPPORTED_LOCATOR_EC, 421 [FILTER](#), 97
- DDSC_LOG_UPDATE_LEASE_DURATION_TIMER_EC, 446 DDS_ContentFilterProperty, 512
- 446 [content_filtered_topic_name](#), 513
- DDSC_LOG_UPDATE_LIVELINESS_FAILED_EC, [expression_parameters](#), 514
- 446 [filter_class_name](#), 513
- DDSC_LOG_UPDATE_REMOTE_CONTENT_FILTER_EC, [filter_expression](#), 513
- 451 [related_topic_name](#), 513
- DDSC_LOG_USERMULTICAST_SEQUENCE, 387 DDS_ContentFilterQosPolicy, 514
- DDSC_LOG_USERUNICAST_SEQUENCE, 387 [expression_parameters](#), 516
- DDSC_LOG_UUID_OBJECT, 382 [filter_class_name](#), 515
- DDSC_LOG_VALIDATE_REMOTE_PARTICIPANT_TRUST_HANDLE_EXPRESSION, 516
- 436 [filter_name](#), 515
- DDSC_LOG_WAITSET_OBJECT, 382 DDS_DATA_AVAILABLE_STATUS
- DDSC_LOG_WRITERHISTORY_COMPONENT, [Status Kinds](#), 89
- 390 DDS_DATA_ON_READERS_STATUS
- DDSC_LOG_WS_ADD_COND_REFERENCE_EC, [Status Kinds](#), 89
- 370 DDS_DATA_WRITER_SAMPLE_REMOVED_STATUS
- DDSC_LOG_WS_REMOVE_COND_REFERENCE_EC, [Status Kinds](#), 92
- 370 DDS_DATAREADER_CPP
- DDSC_LOG_WSREF_OBJECT, 381 [User Data Type Support](#), 51
- DDSC_LOG_XTYPES_LOANED_SAMPLE_STATE_ERROR_EC, DDS_DATAREADER_ENTITY_KIND
- 447 [Entity Support](#), 128
- DDSC_TRUST_INVALID_PARTICIPANT_GENERIC_MESSAGE_ID, DDS_DATAREADER_QOS_DEFAULT
- 437 [Subscriber](#), 65
- dds_c_config.h, 913 DDS_DataReaderInstanceReplacedStatus, 516
- dds_c_flowcontroller.h, 913 [last_instance_handle](#), 517

- last_replacement_instance, 517
- lost_samples, 517
- publication_handle, 517
- total_count, 517
- total_count_change, 517
- DDS_DataReaderProtocolQosPolicy, 518
 - propagate_dispose_of_unregistered_instances, 519
 - rtps_object_id, 518
 - rtps_reliable_reader, 518
- DDS_DataReaderQos, 519
 - content_filter, 522
 - deadline, 521
 - destination_order, 521
 - durability, 521
 - history, 521
 - liveliness, 521
 - ownership, 521
 - property, 522
 - protocol, 522
 - reader_resource_limits, 522
 - reliability, 521
 - representation, 522
 - resource_limits, 521
 - subscription_name, 523
 - transport, 522
 - transport_priority, 523
 - user_data, 522
- DDS_DataReaderResourceLimitsInstanceReplacementKind
 - DataReader Resource Limits, 113
- DDS_DataReaderResourceLimitsQosPolicy, 523
 - instance_replacement, 525
 - max_fragmented_samples, 526
 - max_fragmented_samples_per_remote_writer, 526
 - max_outstanding_reads, 525
 - max_remote_writers, 524
 - max_remote_writers_per_instance, 524
 - max_routes_per_writer, 525
 - max_samples_per_remote_writer, 524
 - shmem_ref_transfer_mode_attached_segment_allocation, 526
- DDS_DataRepresentationId_t
 - DATA_REPRESENTATION, 108
- DDS_DataRepresentationIdSeq, 527
- DDS_DataRepresentationQosPolicy, 527
 - value, 528
- DDS_DATAWRITER_CPP
 - User Data Type Support, 51
- DDS_DATAWRITER_ENTITY_KIND
 - Entity Support, 128
- DDS_DATAWRITER_QOS_DEFAULT
 - Publisher, 61
- DDS_DataWriterListener_ReliableReaderActivityChangedCallback
 - DataWriter, 62
- DDS_DataWriterListener_ReliableSampleUnacknowledgedCallback
 - DataWriter, 63
- DDS_DataWriterListener_SampleRemovedCallback
 - DataWriter, 63
- DDS_DataWriterProtocolQosPolicy, 528
 - rtps_object_id, 529
 - rtps_reliable_writer, 529
 - serialize_on_write, 530
- DDS_DataWriterQos, 530
 - deadline, 532
 - destination_order, 533
 - durability, 533
 - history, 532
 - liveliness, 532
 - ownership, 533
 - ownership_strength, 533
 - property, 534
 - protocol, 533
 - publication_name, 534
 - publish_mode, 534
 - reliability, 533
 - representation, 533
 - resource_limits, 532
 - transport, 534
 - transport_priority, 534
 - user_data, 534
 - writer_resource_limits, 534
- DDS_DataWriterResourceLimitsQosPolicy, 535
 - initialize_writer_loaned_sample, 536
 - max_remote_reader_filters, 537
 - max_remote_readers, 536
 - max_routes_per_reader, 536
 - writer_loaned_sample_allocation, 536
- DDS_DataWriterShmemRefTransferModeSettings, 538
 - enable_data_consistency_check, 538
- DDS_DataWriterTransferModeQosPolicy, 538
 - shmem_ref_settings, 539
- DDS_DEADLINE_QOS_POLICY_ID
 - QoS Policies, 96
- DDS_DeadlineQosPolicy, 539
 - period, 541
- DDS_DEFAULT_FLOW_CONTROLLER_NAME
 - Flow Controllers, 57
- DDS_DestinationOrderQosPolicy, 541
 - kind, 541
 - source_timestamp_tolerance, 541
- DDS_DestinationOrderQosPolicyKind
 - DESTINATION_ORDER, 111
- DDS_DiscoveryComponent, 542
 - name, 542
- DDS_DiscoveryQosPolicy, 543
 - accept_unknown_peers, 545
 - discovery, 544
 - enable_endpoint_discovery_queue, 545
 - enable_participant_discovery_by_name, 545

- enabled_transports, [544](#)
- initial_peers, [544](#)
- metatraffic_transport_priority, [546](#)
- DDS_DomainId_t
 - Domain Module, [32](#)
- DDS_DomainParticipantFactoryQos, [546](#)
 - entity_factory, [547](#)
 - resource_limits, [547](#)
- DDS_DomainParticipantQos, [547](#)
 - discovery, [548](#)
 - entity_factory, [548](#)
 - filter, [550](#)
 - participant_name, [549](#)
 - property, [550](#)
 - protocol, [549](#)
 - resource_limits, [549](#)
 - transports, [549](#)
 - trust, [549](#)
 - user_data, [550](#)
 - user_traffic, [549](#)
- DDS_DomainParticipantResourceLimitsQosPolicy, [550](#)
 - local_publisher_allocation, [553](#)
 - local_reader_allocation, [552](#)
 - local_subscriber_allocation, [553](#)
 - local_topic_allocation, [553](#)
 - local_type_allocation, [553](#)
 - local_writer_allocation, [552](#)
 - matching_reader_writer_pair_allocation, [555](#)
 - matching_writer_reader_pair_allocation, [554](#)
 - max_destination_ports, [556](#)
 - max_partition_cumulative_characters, [560](#)
 - max_partition_string_allocation, [561](#)
 - max_partition_string_size, [560](#)
 - max_partitions, [560](#)
 - max_receive_ports, [555](#)
 - participant_user_data_max_count, [558](#)
 - participant_user_data_max_length, [557](#)
 - publisher_group_data_max_count, [558](#)
 - publisher_group_data_max_length, [558](#)
 - reader_user_data_max_count, [560](#)
 - reader_user_data_max_length, [559](#)
 - remote_participant_allocation, [554](#)
 - remote_reader_allocation, [554](#)
 - remote_writer_allocation, [554](#)
 - shmem_ref_transfer_mode_max_segments, [557](#)
 - subscriber_group_data_max_count, [559](#)
 - subscriber_group_data_max_length, [559](#)
 - topic_data_max_count, [558](#)
 - topic_data_max_length, [558](#)
 - writer_user_data_max_count, [559](#)
 - writer_user_data_max_length, [559](#)
- DDS_Double
 - DDS-Specific Primitive Types, [47](#)
- DDS_DurabilityQosPolicy, [561](#)
 - kind, [562](#)
- DDS_DurabilityQosPolicyKind
 - DURABILITY, [106](#)
- DDS_Duration_equal
 - Time Support, [78](#)
- DDS_DURATION_INFINITE
 - Time Support, [79](#)
- DDS_DURATION_INFINITE_NSEC
 - Time Support, [79](#)
- DDS_DURATION_INFINITE_SEC
 - Time Support, [79](#)
- DDS_Duration_is_infinite
 - Time Support, [78](#)
- DDS_Duration_is_zero
 - Time Support, [78](#)
- DDS_Duration_t, [562](#)
 - nanosec, [562](#)
 - sec, [562](#)
- DDS_DURATION_ZERO
 - Time Support, [79](#)
- DDS_DURATION_ZERO_NSEC
 - Time Support, [79](#)
- DDS_DURATION_ZERO_SEC
 - Time Support, [79](#)
- DDS_EDF_FLOW_CONTROLLER_SCHED_POLICY
 - Flow Controllers, [56](#)
- DDS_ENTITYFACTORY_QOS_POLICY_ID
 - QoS Policies, [96](#)
- DDS_EntityFactoryQosPolicy, [563](#)
 - autoenable_created_entities, [564](#)
- DDS_EntityKind_t
 - Entity Support, [128](#)
- DDS_ENTITYNAME_QOS_NAME_MAX
 - ENTITY_NAME, [120](#)
- DDS_EntityNameQosPolicy, [564](#)
 - name, [565](#)
- DDS_Enum
 - DDS-Specific Primitive Types, [48](#)
- DDS_EXCLUSIVE_OWNERSHIP_QOS
 - OWNERSHIP, [101](#)
- DDS_FILTER, [499](#)
 - DDS_FILTER_LOG_COMPILED_FILTER_RECORD, [502](#)
 - DDS_FILTER_LOG_CONTENT_FILTER_NAME_STRING, [506](#)
 - DDS_FILTER_LOG_CONTENT_FILTER_NAME_TOO_LONG_EC, [506](#)
 - DDS_FILTER_LOG_CONTENT_FILTERED_TOPIC_NAME_STRING, [505](#)
 - DDS_FILTER_LOG_FILTER_CLASS_NAME_STRING, [505](#)
 - DDS_FILTER_LOG_FILTER_CLASS_NOT_FOUND_EC, [505](#)
 - DDS_FILTER_LOG_FILTER_CLASS_RECORD, [502](#)

DDS_FILTER_LOG_FILTER_CLASS_REGISTER_EC, 504
 DDS_FILTER_LOG_FILTER_CLASS_UNREGISTER_EC, 505
 DDS_FILTER_LOG_FILTER_COMPILE_EC, 504
 DDS_FILTER_LOG_FILTER_CREATE_INSTANCE_EC, 503
 DDS_FILTER_LOG_FILTER_DELETE_INSTANCE_EC, 504
 DDS_FILTER_LOG_FILTER_EVALUATE_EC, 504
 DDS_FILTER_LOG_FILTER_EXPRESSION_MAX_COUNT_EC, 504
 DDS_FILTER_LOG_FILTER_EXPRESSION_PARAMETER_COUNT_EC, 505
 DDS_FILTER_LOG_FILTER_EXPRESSION_STRING, 505
 DDS_FILTER_LOG_FILTER_PARAMETER_COUNT_EC, 506
 DDS_FILTER_LOG_FILTER_PLUGIN_FACTORY_IN_USE_EC, 507
 DDS_FILTER_LOG_FILTER_PLUGIN_FINALIZE_EC, 507
 DDS_FILTER_LOG_FILTER_PLUGIN_INIT_EC, 507
 DDS_FILTER_LOG_HEAP_ALLOC_EC, 507
 DDS_FILTER_LOG_READER_RECORD, 502
 DDS_FILTER_LOG_RECORD_CREATE_EC, 503
 DDS_FILTER_LOG_RECORD_DELETE_EC, 503
 DDS_FILTER_LOG_RECORD_INSERT_EC, 503
 DDS_FILTER_LOG_RECORD_REMOVE_EC, 503
 DDS_FILTER_LOG_RECORD_SELECT_EC, 502
 DDS_FILTER_LOG_RELATED_TOPIC_NAME_STRING, 505
 DDS_FILTER_LOG_ROUTE_RECORD, 502
 DDS_FILTER_LOG_SQL_ENUM_LABEL_NOT_FOUND_EC, 509
 DDS_FILTER_LOG_SQL_FIELD_NOT_FOUND_EC, 509
 DDS_FILTER_LOG_SQL_INCOMPATIBLE_TYPES_EC, 510
 DDS_FILTER_LOG_SQL_INTEGER_OUT_OF_RANGE_EC, 510
 DDS_FILTER_LOG_SQL_INVALID_TOKEN_EC, 509
 DDS_FILTER_LOG_SQL_MAX_PREDICATES_EXCEEDED_EC, 508
 DDS_FILTER_LOG_SQL_PARSE_PARAMETER_EC, 509
 DDS_FILTER_LOG_SQL_UNEXPECTED_TOKEN_EC, 508
 DDS_FILTER_LOG_SQL_UNSUPPORTED_FIELD_TYPE_EC, 509
 DDS_FILTER_LOG_SQL_UNSUPPORTED_TYPE_CODE_EC, 508
 DDS_FILTER_LOG_STRING_ASSERT_EC, 506
 DDS_FILTER_LOG_STRING_DELETE_EC, 506
 DDS_FILTER_LOG_STRING_MANAGER_CREATE_EC, 506
 DDS_FILTER_LOG_TABLE_CREATE_EC, 503
 DDS_FILTER_LOG_TABLE_DELETE_EC, 503
 DDS_FILTER_LOG_WRITER_DETACH_EC, 504
 DDS_FILTER_LOG_WRITER_FILTER_COMPILE_EC, 507
 DDS_FILTER_LOG_WRITER_FILTER_EVALUATE_OUT_OF_ORDER_EC, 508
 DDS_FILTER_LOG_WRITER_FILTER_REMOVE_READER_EC, 507
 DDS_FILTER_LOG_WRITER_FILTER_SEND_GAP_EC, 507
 DDS_FILTER_LOG_WRITER_FILTER_STORE_RESULT_EC, 508
 DDS_FILTER_LOG_WRITER_SERIALIZE_INLINE_QOS_EC, 508
 DDS_FILTER_LOG_FILTER_PLUGIN_BUFFER, 944
 DDS_FILTER_LOG_FILTER_PLUGIN_FACTORY_BUFFER, 944
 DDS_FILTER_LOG_INLINE_QOS_BUFFER, 944
 DDS_FILTER_LOG_TYPE_CODE_NOT_FOUND_EC, 944
 DDS_FILTER_LOG_WRITER_FILTER_BUFFER, 944
 DDS_FILTER_LOG_BASE
 osapi_log.h, 979
 DDS_FILTER_LOG_COMPILED_FILTER_RECORD
 DDS_FILTER, 502
 DDS_FILTER_LOG_CONTENT_FILTER_NAME_STRING
 DDS_FILTER, 506
 DDS_FILTER_LOG_CONTENT_FILTER_NAME_TOO_LONG_EC
 DDS_FILTER, 506
 DDS_FILTER_LOG_CONTENT_FILTERED_TOPIC_NAME_STRING
 DDS_FILTER, 505
 DDS_FILTER_LOG_FILTER_CLASS_NAME_STRING
 DDS_FILTER, 505
 DDS_FILTER_LOG_FILTER_CLASS_NOT_FOUND_EC
 DDS_FILTER, 505
 DDS_FILTER_LOG_FILTER_CLASS_RECORD
 DDS_FILTER, 502
 DDS_FILTER_LOG_FILTER_CLASS_REGISTER_EC
 DDS_FILTER, 504
 DDS_FILTER_LOG_FILTER_CLASS_UNREGISTER_EC
 DDS_FILTER, 505
 DDS_FILTER_LOG_FILTER_COMPILE_EC
 DDS_FILTER, 504
 DDS_FILTER_LOG_FILTER_CREATE_INSTANCE_EC
 DDS_FILTER, 503
 DDS_FILTER_LOG_FILTER_DELETE_INSTANCE_EC
 DDS_FILTER, 504

- DDS_FILTER_LOG_FILTER_EVALUATE_EC
 - DDS_FILTER, [504](#)
- DDS_FILTER_LOG_FILTER_EXPRESSION_MAX_COUNT_EC
 - DDS_FILTER, [504](#)
- DDS_FILTER_LOG_FILTER_EXPRESSION_PARAMETER_COUNT_EC
 - DDS_FILTER, [505](#)
- DDS_FILTER_LOG_FILTER_EXPRESSION_STRING
 - DDS_FILTER, [505](#)
- DDS_FILTER_LOG_FILTER_PARAMETER_COUNT_EC
 - DDS_FILTER, [506](#)
- DDS_FILTER_LOG_FILTER_PLUGIN_BUFFER
 - dds_filter_log.h, [944](#)
- DDS_FILTER_LOG_FILTER_PLUGIN_FACTORY_BUFFER
 - dds_filter_log.h, [944](#)
- DDS_FILTER_LOG_FILTER_PLUGIN_FACTORY_IN_USE_EC
 - DDS_FILTER, [507](#)
- DDS_FILTER_LOG_FILTER_PLUGIN_FINALIZE_EC
 - DDS_FILTER, [507](#)
- DDS_FILTER_LOG_FILTER_PLUGIN_INIT_EC
 - DDS_FILTER, [507](#)
- DDS_FILTER_LOG_HEAP_ALLOC_EC
 - DDS_FILTER, [507](#)
- DDS_FILTER_LOG_INLINE_QOS_BUFFER
 - dds_filter_log.h, [944](#)
- DDS_FILTER_LOG_READER_RECORD
 - DDS_FILTER, [502](#)
- DDS_FILTER_LOG_RECORD_CREATE_EC
 - DDS_FILTER, [503](#)
- DDS_FILTER_LOG_RECORD_DELETE_EC
 - DDS_FILTER, [503](#)
- DDS_FILTER_LOG_RECORD_INSERT_EC
 - DDS_FILTER, [503](#)
- DDS_FILTER_LOG_RECORD_REMOVE_EC
 - DDS_FILTER, [503](#)
- DDS_FILTER_LOG_RECORD_SELECT_EC
 - DDS_FILTER, [502](#)
- DDS_FILTER_LOG_RELATED_TOPIC_NAME_STRING
 - DDS_FILTER, [505](#)
- DDS_FILTER_LOG_ROUTE_RECORD
 - DDS_FILTER, [502](#)
- DDS_FILTER_LOG_SQL_ENUM_LABEL_NOT_FOUND_EC
 - DDS_FILTER, [509](#)
- DDS_FILTER_LOG_SQL_FIELD_NOT_FOUND_EC
 - DDS_FILTER, [509](#)
- DDS_FILTER_LOG_SQL_INCOMPATIBLE_TYPES_EC
 - DDS_FILTER, [510](#)
- DDS_FILTER_LOG_SQL_INTEGER_OUT_OF_RANGE_EC
 - DDS_FILTER, [510](#)
- DDS_FILTER_LOG_SQL_INVALID_TOKEN_EC
 - DDS_FILTER, [509](#)
- DDS_FILTER_LOG_SQL_MAX_PREDICATES_EXCEEDED_EC
 - DDS_FILTER, [508](#)
- DDS_FILTER_LOG_SQL_PARSE_PARAMETER_EC
 - DDS_FILTER, [509](#)
- DDS_FILTER_LOG_SQL_UNEXPECTED_TOKEN_EC
 - DDS_FILTER, [508](#)
- DDS_FILTER_LOG_SQL_UNSUPPORTED_FIELD_TYPE_EC
 - DDS_FILTER, [509](#)
- DDS_FILTER_LOG_SQL_UNSUPPORTED_TYPE_CODE_EC
 - DDS_FILTER, [508](#)
- DDS_FILTER_LOG_STRING_ASSERT_EC
 - DDS_FILTER, [506](#)
- DDS_FILTER_LOG_STRING_DELETE_EC
 - DDS_FILTER, [506](#)
- DDS_FILTER_LOG_STRING_MANAGER_CREATE_EC
 - DDS_FILTER, [506](#)
- DDS_FILTER_LOG_TABLE_CREATE_EC
 - DDS_FILTER, [503](#)
- DDS_FILTER_LOG_TABLE_DELETE_EC
 - DDS_FILTER, [503](#)
- DDS_FILTER_LOG_TYPE_CODE_NOT_FOUND_EC
 - dds_filter_log.h, [944](#)
- DDS_FILTER_LOG_WRITER_DETACH_EC
 - DDS_FILTER, [504](#)
- DDS_FILTER_LOG_WRITER_FILTER_BUFFER
 - dds_filter_log.h, [944](#)
- DDS_FILTER_LOG_WRITER_FILTER_COMPILE_EC
 - DDS_FILTER, [507](#)
- DDS_FILTER_LOG_WRITER_FILTER_EVALUATE_OUT_OF_ORDER_EC
 - DDS_FILTER, [508](#)
- DDS_FILTER_LOG_WRITER_FILTER_REMOVE_READER_EC
 - DDS_FILTER, [507](#)
- DDS_FILTER_LOG_WRITER_FILTER_SEND_GAP_EC
 - DDS_FILTER, [507](#)
- DDS_FILTER_LOG_WRITER_FILTER_STORE_RESULT_EC
 - DDS_FILTER, [508](#)
- DDS_FILTER_LOG_WRITER_SERIALIZE_INLINE_QOS_EC
 - DDS_FILTER, [508](#)
- DDS_FILTER_PLUGIN_FACTORY_DEFAULT_NAME
 - FILTER, [98](#)
- DDS_FILTER_RESOURCE_LIMITS_DEFAULT
 - FILTER, [98](#)
- DDS_FilterQosPolicy, [565](#)
 - disable_builtin_sql_filter, [567](#)
 - disable_writer_filtering, [567](#)
 - name, [566](#)
 - resource_limits, [566](#)
- DDS_FilterResourceLimits, [567](#)
 - filter_class_max_count, [568](#)
 - filter_class_max_length, [568](#)
 - filter_expression_max_count, [569](#)
 - filter_expression_max_length, [568](#)
 - filter_parameter_max_count_per_expression, [569](#)
 - filter_parameter_max_length, [569](#)
 - sql_predicate_max_count_per_expression, [569](#)
- DDS_FIXED_RATE_FLOW_CONTROLLER_NAME
 - Flow Controllers, [58](#)
- DDS_Float

- DDS-Specific Primitive Types, [47](#)
- DDS_FLOW_CONTROLLER_PROPERTY_DEFAULT
 - DomainParticipant, [35](#)
- DDS_FlowControllerProperty_t, [570](#)
 - scheduling_policy, [571](#)
 - token_bucket, [571](#)
- DDS_FlowControllerSchedulingPolicy
 - Flow Controllers, [55](#)
- DDS_FlowControllerTokenBucketProperty_t, [571](#)
 - bytes_per_token, [573](#)
 - max_tokens, [572](#)
 - period, [573](#)
 - tokens_added_per_period, [572](#)
 - tokens_leaked_per_period, [573](#)
- DDS_GROUP_PRESENTATION_QOS
 - Presentation, [110](#)
- DDS_GroupDataQosPolicy, [574](#)
 - value, [575](#)
- DDS_GUID_t, [575](#)
 - GUID Support, [80](#)
 - value, [576](#)
- DDS_GUID_UNKNOWN
 - GUID Support, [81](#)
- DDS_HANDLE_NIL
 - User Data Type Support, [53](#)
- DDS_HISTORY_QOS_POLICY_ID
 - QoS Policies, [96](#)
- DDS_HistoryQosPolicy, [576](#)
 - depth, [577](#)
 - kind, [577](#)
- DDS_HistoryQosPolicyKind
 - HISTORY, [105](#)
- DDS_HPF_FLOW_CONTROLLER_SCHED_POLICY
 - Flow Controllers, [57](#)
- DDS_INCONSISTENT_TOPIC_STATUS
 - Status Kinds, [87](#)
- DDS_InconsistentTopicStatus, [577](#)
- DDS_INSTANCE_PRESENTATION_QOS
 - Presentation, [110](#)
- DDS_INSTANCE_REPLACED_STATUS
 - Status Kinds, [91](#)
- DDS_InstanceHandle_equals
 - User Data Type Support, [52](#)
- DDS_InstanceHandle_is_nil
 - User Data Type Support, [53](#)
- DDS_InstanceHandle_t
 - User Data Type Support, [52](#)
- DDS_InstanceHandleSeq, [578](#)
- DDS_InstanceId_t
 - Type Support, [129](#)
- DDS_InstanceStateKind
 - Instance States, [73](#)
- DDS_InstanceStateMask
 - Instance States, [73](#)

- DDS_INTEROPERABLE_RTPS_WELL_KNOWN_PORTS
 - Wire Protocol, [118](#)
- DDS_INVALID_QOS_POLICY_ID
 - QoS Policies, [96](#)
- DDS_KEEP_ALL_HISTORY_QOS
 - HISTORY, [105](#)
- DDS_KEEP_LAST_HISTORY_QOS
 - HISTORY, [105](#)
- DDS_LatencyBudgetQosPolicy, [578](#)
 - duration, [580](#)
- DDS_LENGTH_AUTO
 - Resource Limits, [109](#)
- DDS_LENGTH_UNLIMITED
 - Resource Limits, [109](#)
- DDS_LIVELINESS_CHANGED_STATUS
 - Status Kinds, [90](#)
- DDS_LIVELINESS_LOST_STATUS
 - Status Kinds, [90](#)
- DDS_LIVELINESS_QOS_POLICY_ID
 - QoS Policies, [96](#)
- DDS_LivelinessChangedStatus, [580](#)
 - alive_count, [580](#)
 - alive_count_change, [581](#)
 - last_publication_handle, [581](#)
 - not_alive_count, [580](#)
 - not_alive_count_change, [581](#)
- DDS_LivelinessLostStatus, [581](#)
 - total_count, [582](#)
 - total_count_change, [582](#)
- DDS_LivelinessQosPolicy, [582](#)
 - kind, [585](#)
 - lease_duration, [585](#)
- DDS_LivelinessQosPolicyKind
 - LIVELINESS, [102](#)
- DDS_LOCATOR_ADDRESS_LENGTH_MAX
 - Discovery, [38](#)
- DDS_LOCATOR_INVALID
 - Discovery, [38](#)
- DDS_LOCATOR_KIND_RESERVED
 - Discovery, [39](#)
- DDS_LOCATOR_KIND_SHMEM
 - Discovery, [39](#)
- DDS_LOCATOR_KIND_UDPv4
 - Discovery, [39](#)
- DDS_LOCATOR_KIND_UDPv6
 - Discovery, [39](#)
- DDS_Locator_t, [585](#)
 - address, [586](#)
 - kind, [586](#)
 - port, [586](#)
- DDS_LocatorSeq, [586](#)
- DDS_LOG_IPC_ASSERT_ROUTES_EC
 - DDS_C, [425](#)
- DDS_LOG_TRUST_ASSERT_PARTICIPANT_GENERIC_MESSAGE_FAILURE

- DDS_C, [436](#)
- DDS_Long
 - DDS-Specific Primitive Types, [46](#)
- DDS_LongDouble
 - DDS-Specific Primitive Types, [47](#)
- DDS_LongLong
 - DDS-Specific Primitive Types, [46](#)
- DDS_MANUAL_BY_PARTICIPANT_LIVELINESS_QOS
 - LIVELINESS, [103](#)
- DDS_MANUAL_BY_TOPIC_LIVELINESS_QOS
 - LIVELINESS, [103](#)
- DDS_NEW_VIEW_STATE
 - View States, [72](#)
- DDS_NO_INSTANCE_REPLACEMENT_QOS
 - DataReader Resource Limits, [114](#)
- DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE
 - Instance States, [74](#)
- DDS_NOT_ALIVE_INSTANCE_STATE
 - Instance States, [74](#)
- DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE
 - Instance States, [74](#)
- DDS_NOT_NEW_VIEW_STATE
 - View States, [72](#)
- DDS_NOT_READ_SAMPLE_STATE
 - Sample States, [70](#)
- DDS_NOT_REJECTED
 - DataReader, [67](#)
- DDS_Octet
 - DDS-Specific Primitive Types, [45](#)
- DDS_OFFERED_DEADLINE_MISSED_STATUS
 - Status Kinds, [87](#)
- DDS_OFFERED_INCOMPATIBLE_QOS_STATUS
 - Status Kinds, [88](#)
- DDS_OfferedDeadlineMissedStatus, [586](#)
 - last_instance_handle, [587](#)
 - total_count, [587](#)
 - total_count_change, [587](#)
- DDS_OfferedIncompatibleQosStatus, [588](#)
 - last_policy_id, [589](#)
 - policies, [589](#)
 - total_count, [588](#)
 - total_count_change, [588](#)
- DDS_ON_DEMAND_FLOW_CONTROLLER_NAME
 - Flow Controllers, [58](#)
- DDS_OWNERSHIP_QOS_POLICY_ID
 - QoS Policies, [96](#)
- DDS_OwnershipQosPolicy, [589](#)
 - kind, [592](#)
- DDS_OwnershipQosPolicyKind
 - OWNERSHIP, [101](#)
- DDS_OWNERSHIPSTRENGTH_QOS_POLICY_ID
 - QoS Policies, [96](#)
- DDS_OwnershipStrengthQosPolicy, [593](#)
 - value, [593](#)
- DDS_PARTICIPANT_ENTITY_KIND
 - Entity Support, [128](#)
- DDS_PARTICIPANT_QOS_DEFAULT
 - DomainParticipantFactory, [33](#)
- DDS_PARTICIPANT_TOPIC_NAME
 - Participant Built-in Topic, [41](#)
- DDS_ParticipantBuiltinTopicData, [594](#)
 - checksum, [596](#)
 - dds_builtin_endpoints, [595](#)
 - default_multicast_locators, [595](#)
 - default_unicast_locators, [595](#)
 - key, [595](#)
 - liveliness_lease_duration, [596](#)
 - metatraffic_multicast_locators, [596](#)
 - metatraffic_unicast_locators, [596](#)
 - participant_name, [595](#)
 - product_version, [596](#)
 - property, [596](#)
 - rtps_protocol_version, [595](#)
 - rtps_vendor_id, [595](#)
 - user_data, [596](#)
- DDS_PartitionQosPolicy, [597](#)
 - name, [599](#)
- DDS_PresentationQosPolicy, [599](#)
 - access_scope, [602](#)
 - coherent_access, [602](#)
 - ordered_access, [603](#)
- DDS_PresentationQosPolicyAccessScopeKind
 - PRESENTATION, [110](#)
- DDS_ProductVersion_t, [603](#)
 - major, [604](#)
 - minor, [604](#)
 - release, [604](#)
 - revision, [604](#)
- DDS_PRODUCTVERSION_UNKNOWN
 - Discovery, [38](#)
- DDS_Property_t, [604](#)
 - name, [605](#)
 - propagate, [605](#)
 - value, [605](#)
- DDS_PropertyQosPolicy, [605](#)
 - value, [606](#)
- DDS_PropertySeq, [606](#)
- DDS_PROTOCOLVERSION
 - Discovery, [40](#)
- DDS_PROTOCOLVERSION_1_0
 - Discovery, [39](#)
- DDS_PROTOCOLVERSION_1_1
 - Discovery, [39](#)
- DDS_PROTOCOLVERSION_1_2
 - Discovery, [39](#)
- DDS_PROTOCOLVERSION_2_0
 - Discovery, [40](#)
- DDS_PROTOCOLVERSION_2_1

- Discovery, [40](#)
- DDS_ProtocolVersion_t, [607](#)
 - major, [607](#)
 - minor, [607](#)
- DDS_PSK_DEFAULT_PLUGIN_NAME
 - Lightweight Security API, [224](#)
- dds_psk_log.h, [944](#)
 - DDS_PSK_LOG_TRUST_EXCEPTION_EC, [945](#)
- DDS_PSK_LOG_TRUST_EXCEPTION_EC
 - dds_psk_log.h, [945](#)
- DDS_PskPassTracker_InterfaceI, [607](#)
 - finalize_func, [608](#)
 - initialize_func, [608](#)
- DDS_PskServiceFactoryProperty, [608](#)
 - pass_tracker_config, [610](#)
 - pass_tracker_get_interface_func, [610](#)
 - psl_config, [609](#)
 - psl_get_interface_func, [609](#)
 - service_name, [609](#)
- DDS_PskServiceFactoryProperty_INITIALIZER
 - Lightweight Security API, [223](#)
- DDS_PUBLICATION_MATCHED_STATUS
 - Status Kinds, [91](#)
- DDS_PUBLICATION_PRIORITY_AUTOMATIC
 - PUBLISH_MODE, [122](#)
- DDS_PUBLICATION_PRIORITY_UNDEFINED
 - PUBLISH_MODE, [122](#)
- DDS_PUBLICATION_TOPIC_NAME
 - Publication Built-in Topic, [41](#)
- DDS_PublicationBuiltInTopicData, [611](#)
 - deadline, [613](#)
 - destination_order, [614](#)
 - durability, [614](#)
 - group_data, [614](#)
 - key, [612](#)
 - latency_budget, [613](#)
 - liveliness, [613](#)
 - ownership, [613](#)
 - ownership_strength, [613](#)
 - participant_key, [612](#)
 - partition, [614](#)
 - reliability, [613](#)
 - representation, [614](#)
 - topic_data, [615](#)
 - topic_name, [612](#)
 - type_name, [612](#)
 - unicast_locator, [614](#)
 - user_data, [614](#)
- DDS_PublicationMatchedStatus, [615](#)
 - current_count, [616](#)
 - current_count_change, [616](#)
 - last_subscription_handle, [616](#)
 - total_count, [616](#)
 - total_count_change, [616](#)
- DDS_Publisher
 - Publisher, [60](#)
- DDS_PUBLISHER_ENTITY_KIND
 - Entity Support, [128](#)
- DDS_PUBLISHER_QOS_DEFAULT
 - DomainParticipant, [34](#)
- DDS_PublisherQos, [616](#)
 - entity_factory, [617](#)
 - group_data, [617](#)
 - partition, [617](#)
 - publisher_name, [617](#)
- DDS_PublishModeQosPolicy, [618](#)
 - flow_controller_name, [619](#)
 - kind, [619](#)
 - priority, [619](#)
- DDS_PublishModeQosPolicyKind
 - PUBLISH_MODE, [122](#)
- DDS_QosPolicyCount, [620](#)
 - count, [620](#)
 - policy_id, [620](#)
- DDS_QosPolicyId_t
 - QoS Policies, [96](#)
- DDS_READ_SAMPLE_STATE
 - Sample States, [70](#)
- DDS_REJECTED_BY_INSTANCES_LIMIT
 - DataReader, [67](#)
- DDS_REJECTED_BY_REMOTE_WRITERS_LIMIT
 - DataReader, [67](#)
- DDS_REJECTED_BY_SAMPLES_LIMIT
 - DataReader, [67](#)
- DDS_REJECTED_BY_SAMPLES_PER_INSTANCE_LIMIT
 - DataReader, [67](#)
- DDS_REJECTED_BY_SAMPLES_PER_REMOTE_WRITER_LIMIT
 - DataReader, [67](#)
- DDS_RELIABILITY_QOS_POLICY_ID
 - QoS Policies, [96](#)
- DDS_ReliabilityQosPolicy, [621](#)
 - kind, [622](#)
 - max_blocking_time, [622](#)
- DDS_ReliabilityQosPolicyKind
 - RELIABILITY, [104](#)
- DDS_RELIABLE_READER_ACTIVITY_CHANGED_STATUS
 - Status Kinds, [92](#)
- DDS_RELIABLE_RELIABILITY_QOS
 - RELIABILITY, [104](#)
- DDS_ReliableReaderActivityChangedStatus, [623](#)
 - active_count, [623](#)
 - active_count_change, [624](#)
 - inactive_count, [623](#)
 - inactive_count_change, [624](#)
 - last_instance_handle, [624](#)
- DDS_ReliableReaderActivityChangedStatus_INITIALIZER
 - DataWriter, [62](#)
- DDS_ReliableSampleUnacknowledgedStatus, [624](#)

- instance_handle, [625](#)
- sequence_number, [625](#)
- unacknowledged_count, [625](#)
- DDS_REPLACE_OLDEST_INSTANCE_REPLACEMENT_QOS
 - DataReader Resource Limits, [114](#)
- DDS_REQUESTED_DEADLINE_MISSED_STATUS
 - Status Kinds, [88](#)
- DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS
 - Status Kinds, [88](#)
- DDS_RequestedDeadlineMissedStatus, [625](#)
 - last_instance_handle, [626](#)
 - total_count, [626](#)
 - total_count_change, [626](#)
- DDS_RequestedIncompatibleQosStatus, [626](#)
 - last_policy_id, [627](#)
 - policies, [627](#)
 - total_count, [627](#)
 - total_count_change, [627](#)
- DDS_ResourceLimitsQosPolicy, [627](#)
 - max_instances, [630](#)
 - max_samples, [629](#)
 - max_samples_per_instance, [630](#)
- DDS_RETCODE_ALREADY_DELETED
 - Return Codes, [83](#)
- DDS_RETCODE_BAD_PARAMETER
 - Return Codes, [83](#)
- DDS_RETCODE_ERROR
 - Return Codes, [83](#)
- DDS_RETCODE_ILLEGAL_OPERATION
 - Return Codes, [83](#)
- DDS_RETCODE_IMMUTABLE_POLICY
 - Return Codes, [83](#)
- DDS_RETCODE_INCONSISTENT_POLICY
 - Return Codes, [83](#)
- DDS_RETCODE_NO_DATA
 - Return Codes, [83](#)
- DDS_RETCODE_NOT_ALLOWED_BY_SEC
 - Return Codes, [83](#)
- DDS_RETCODE_NOT_ENABLED
 - Return Codes, [83](#)
- DDS_RETCODE_OK
 - Return Codes, [83](#)
- DDS_RETCODE_OUT_OF_RESOURCES
 - Return Codes, [83](#)
- DDS_RETCODE_PRECONDITION_NOT_MET
 - Return Codes, [83](#)
- DDS_RETCODE_TIMEOUT
 - Return Codes, [83](#)
- DDS_RETCODE_UNSUPPORTED
 - Return Codes, [83](#)
- DDS_ReturnCode_t
 - Return Codes, [82](#)
- DDS_RR_FLOW_CONTROLLER_SCHED_POLICY
 - Flow Controllers, [56](#)
- DDS_RTI_BACKWARDS_COMPATIBLE_RTPS_WELL_KNOWN_PORTS
 - WIRE_PROTOCOL, [118](#)
- DDS_RtpsReliableReaderProtocol_t, [631](#)
 - nack_period, [632](#)
- DDS_RtpsReliableWriterProtocol_t, [632](#)
 - heartbeat_period, [633](#)
 - heartbeats_per_max_samples, [633](#)
 - max_heartbeat_retries, [634](#)
 - max_send_window, [633](#)
- DDS_RtpsWellKnownPorts_t, [634](#)
 - builtin_multicast_port_offset, [637](#)
 - builtin_unicast_port_offset, [638](#)
 - domain_id_gain, [636](#)
 - participant_id_gain, [637](#)
 - port_base, [636](#)
 - user_multicast_port_offset, [638](#)
 - user_unicast_port_offset, [638](#)
- DDS_SAMPLE_LOST_BY_DATAWRITER
 - DataReader, [68](#)
- DDS_SAMPLE_LOST_BY_HISTORY_DEPTH_LIMIT
 - DataReader, [68](#)
- DDS_SAMPLE_LOST_BY_MANAGEDPOOL_OVERWRITE
 - DataReader, [68](#)
- DDS_SAMPLE_LOST_BY_MAX_SAMPLES_LIMIT
 - DataReader, [68](#)
- DDS_SAMPLE_LOST_BY_META_SAMPLE_LIMIT
 - DataReader, [68](#)
- DDS_SAMPLE_LOST_BY_NOT_READ_ON_CACHE_DELETION
 - DataReader, [68](#)
- DDS_SAMPLE_LOST_NOT_LOST
 - DataReader, [67](#)
- DDS_SAMPLE_LOST_STATUS
 - Status Kinds, [89](#)
- DDS_SAMPLE_REJECTED_STATUS
 - Status Kinds, [89](#)
- DDS_SampleIdentity_t, [639](#)
 - sequence_number, [639](#)
 - writer_guid, [639](#)
- DDS_SampleInfo, [639](#)
 - instance_handle, [641](#)
 - instance_state, [641](#)
 - publication_handle, [641](#)
 - publication_sequence_number, [642](#)
 - reception_timestamp, [642](#)
 - sample_state, [641](#)
 - source_timestamp, [642](#)
 - valid_data, [642](#)
 - view_state, [641](#)
- DDS_SampleInfoSeq, [643](#)
- DDS_SampleLostStatus, [643](#)
 - reason, [644](#)
 - sample_info, [644](#)
 - total_count, [644](#)
 - total_count_change, [644](#)

- DDS_SampleLostStatusKind
 - DataReader, [67](#)
- DDS_SampleRejectedStatus, [644](#)
 - last_instance_handle, [645](#)
 - last_reason, [645](#)
 - total_count, [645](#)
 - total_count_change, [645](#)
- DDS_SampleRejectedStatusKind
 - DataReader, [66](#)
- DDS_SampleStateKind
 - Sample States, [70](#)
- DDS_SampleStateMask
 - Sample States, [70](#)
- DDS_SecTransform_Configuration, [645](#)
 - err_listener, [646](#)
 - provider_name, [646](#)
- DDS_SequenceNumber_compare
 - Sequence Number Support, [81](#)
- DDS_SequenceNumber_t, [646](#)
 - high, [647](#)
 - low, [647](#)
- DDS_SHARED_OWNERSHIP_QOS
 - OWNERSHIP, [101](#)
- DDS_Short
 - DDS-Specific Primitive Types, [45](#)
- DDS_SIZE_AUTO
 - RESOURCE_LIMITS, [109](#)
- DDS_SQL_CONTENT_FILTER_CLASS
 - Content Filter Plugin, [31](#)
- DDS_STATUS_MASK_ALL
 - Status Kinds, [86](#)
- DDS_STATUS_MASK_NONE
 - Status Kinds, [86](#)
- DDS_StatusKind
 - Status Kinds, [87](#)
- DDS_StatusMask
 - Status Kinds, [86](#)
- DDS_String
 - DDS-Specific Primitive Types, [48](#)
- DDS_String_alloc
 - String Support, [131](#)
- DDS_String_cmp
 - String Support, [132](#)
- DDS_String_dup
 - String Support, [132](#)
- DDS_String_free
 - String Support, [132](#)
- DDS_String_length
 - String Support, [133](#)
- DDS_String_ncmp
 - String Support, [133](#)
- DDS_SUBSCRIBER_ENTITY_KIND
 - Entity Support, [128](#)
- DDS_SUBSCRIBER_QOS_DEFAULT
 - DomainParticipant, [34](#)
- DDS_SubscriberQos, [647](#)
 - entity_factory, [648](#)
 - group_data, [648](#)
 - partition, [648](#)
 - subscriber_name, [648](#)
- DDS_SUBSCRIPTION_MATCHED_STATUS
 - Status Kinds, [91](#)
- DDS_SUBSCRIPTION_TOPIC_NAME
 - Subscription Built-in Topic, [42](#)
- DDS_SubscriptionBuiltinTopicData, [648](#)
 - content_filter, [653](#)
 - deadline, [651](#)
 - destination_order, [652](#)
 - durability, [651](#)
 - group_data, [653](#)
 - key, [650](#)
 - latency_budget, [651](#)
 - liveliness, [651](#)
 - multicast_locator, [652](#)
 - ownership, [651](#)
 - participant_key, [650](#)
 - partition, [652](#)
 - presentation, [652](#)
 - reliability, [651](#)
 - representation, [652](#)
 - topic_data, [653](#)
 - topic_name, [650](#)
 - type_name, [650](#)
 - unicast_locator, [652](#)
 - user_data, [652](#)
- DDS_SubscriptionMatchedStatus, [653](#)
 - current_count, [654](#)
 - current_count_change, [654](#)
 - last_publication_handle, [654](#)
 - total_count, [654](#)
 - total_count_change, [654](#)
- DDS_SYNCHRONOUS_PUBLISH_MODE_QOS
 - PUBLISH_MODE, [122](#)
- DDS_SystemResourceLimitsQosPolicy, [655](#)
 - max_components, [656](#)
 - max_participants, [655](#)
- DDS_TIME_INVALID
 - Time Support, [77](#)
- DDS_TIME_INVALID_NSEC
 - Time Support, [77](#)
- DDS_TIME_INVALID_SEC
 - Time Support, [77](#)
- DDS_Time_is_zero
 - Time Support, [78](#)
- DDS_Time_t, [656](#)
 - nanosec, [657](#)
 - sec, [657](#)
- DDS_TIME_ZERO

- Time Support, [77](#)
- DDS_TOPIC_ENTITY_KIND
 - Entity Support, [128](#)
- DDS_TOPIC_PRESENTATION_QOS
 - PRESSENTATION, [110](#)
- DDS_TOPIC_QOS_DEFAULT
 - DomainParticipant, [34](#)
- DDS_TopicDataQosPolicy, [657](#)
 - value, [658](#)
- DDS_TopicQos, [658](#)
 - topic_data, [658](#)
- DDS_TRANSIENT_LOCAL_DURABILITY_QOS
 - DURABILITY, [106](#)
- DDS_TransportPriorityQosPolicy, [659](#)
 - value, [659](#)
- DDS_TransportQosPolicy, [659](#)
 - enabled_transports, [660](#)
- DDS_TrustQosPolicy, [660](#)
 - suite, [661](#)
- DDS_TYPESUPPORT_CPP
 - User Data Type Support, [52](#)
- DDS_UNKNOWN_ENTITY_KIND
 - Entity Support, [128](#)
- DDS_UnsignedLong
 - DDS-Specific Primitive Types, [46](#)
- DDS_UnsignedLongLong
 - DDS-Specific Primitive Types, [46](#)
- DDS_UnsignedShort
 - DDS-Specific Primitive Types, [45](#)
- DDS_UserDataQosPolicy, [661](#)
 - value, [662](#)
- DDS_UserTrafficQosPolicy, [663](#)
 - enabled_transports, [663](#)
- DDS_VENDOR_ID_LENGTH_MAX
 - Discovery, [38](#)
- DDS_VendorId_t, [664](#)
 - vendorId, [664](#)
- DDS_ViewStateKind
 - View States, [71](#)
- DDS_ViewStateMask
 - View States, [71](#)
- DDS_VOLATILE_DURABILITY_QOS
 - DURABILITY, [106](#)
- DDS_Wchar
 - DDS-Specific Primitive Types, [45](#)
- DDS_WireProtocolQosPolicy, [664](#)
 - allowed_crc_mask, [668](#)
 - check_crc, [667](#)
 - compute_crc, [666](#)
 - computed_crc_kind, [667](#)
 - participant_id, [665](#)
 - require_crc, [667](#)
 - rtps_app_id, [666](#)
 - rtps_host_id, [665](#)
 - rtps_instance_id, [666](#)
 - rtps_well_known_ports, [666](#)
- DDS_WRITEPARAMS_DEFAULT
 - Infrastructure Module, [76](#)
- DDS_WriteParams_t, [668](#)
 - handle, [669](#)
 - source_timestamp, [669](#)
- DDS_Wstring
 - DDS-Specific Primitive Types, [48](#)
- DDS_XCDR2_DATA_REPRESENTATION
 - DATA_REPRESENTATION, [107](#)
- DDS_XCDR_DATA_REPRESENTATION
 - DATA_REPRESENTATION, [107](#)
- DDS_XML_DATA_REPRESENTATION
 - DATA_REPRESENTATION, [107](#)
- DDS_XTYPES_COMPLIANCE_MASK_PROPERTY
 - Property Reference, [135](#)
- DDSC_LOG_ADDRESSRESOLVER_OBJECT
 - DDS_C, [380](#)
- DDSC_LOG_APPGEN_COMPONENT
 - DDS_C, [390](#)
- DDSC_LOG_APPLY_READER_FILTER_EC
 - DDS_C, [450](#)
- DDSC_LOG_AUTHPOOL_OBJECT
 - DDS_C, [384](#)
- DDSC_LOG_BASE
 - osapi_log.h, [978](#)
- DDSC_LOG_BIND_RECORD
 - DDS_C, [375](#)
- DDSC_LOG_BINDRESOLVER_OBJECT
 - DDS_C, [380](#)
- DDSC_LOG_BUFFER_MAX_EXCEEDED_EC
 - DDS_C, [448](#)
- DDSC_LOG_BUFFER_OBJECT
 - DDS_C, [384](#)
- DDSC_LOG_CDR_BUFFER_SET_EC
 - DDS_C, [402](#)
- DDSC_LOG_CDR_DESERIALIZE_DATA_EC
 - DDS_C, [404](#)
- DDSC_LOG_CDR_DESERIALIZE_HEADER_EC
 - DDS_C, [404](#)
- DDSC_LOG_CDR_DESERIALIZE_KEY_EC
 - DDS_C, [405](#)
- DDSC_LOG_CDR_DESERIALIZE_KEYHASH_EC
 - DDS_C, [405](#)
- DDSC_LOG_CDR_DESERIALIZE_PID_EC
 - DDS_C, [403](#)
- DDSC_LOG_CDR_DESERIALIZE_PID_LENGTH_EC
 - DDS_C, [404](#)
- DDSC_LOG_CDR_FINALIZE_SAMPLE_EC
 - DDS_C, [404](#)
- DDSC_LOG_CDR_INCREMENT_POS_EC
 - DDS_C, [404](#)
- DDSC_LOG_CDR_INITIALIZE_SAMPLE_EC

DDS_C, [404](#)
 DDSC_LOG_CDR_POOL_ALLOC_EC
 DDS_C, [402](#)
 DDSC_LOG_CDR_POOL_DELETE_EC
 DDS_C, [403](#)
 DDSC_LOG_CDR_SERIALIZE_DATA_EC
 DDS_C, [403](#)
 DDSC_LOG_CDR_SERIALIZE_KEYHASH_EC
 DDS_C, [403](#)
 DDSC_LOG_CDR_SERIALIZE_PID_EC
 DDS_C, [403](#)
 DDSC_LOG_CDR_SERIALIZE_PID_LENGTH_EC
 DDS_C, [403](#)
 DDSC_LOG_CDR_SERIALIZE_STATUS_INFO_EC
 DDS_C, [405](#)
 DDSC_LOG_CDR_SET_OFFSET_EC
 DDS_C, [420](#)
 DDSC_LOG_CDR_SET_POS_EC
 DDS_C, [404](#)
 DDSC_LOG_CHANGE_CONTENT_FILTER_EC
 DDS_C, [451](#)
 DDSC_LOG_CHECKSUM_INCOMPATIBLE_EC
 DDS_C, [449](#)
 DDSC_LOG_CHECKSUM_INCONSISTENT_ALLOW_EC
 DDS_C, [449](#)
 DDSC_LOG_CHECKSUM_INCONSISTENT_CLASS_EC
 DDS_C, [449](#)
 DDSC_LOG_CHECKSUM_INCONSISTENT_COMPUTE_EC
 DDS_C, [449](#)
 DDSC_LOG_CHECKSUM_REQUIRED_EC
 DDS_C, [448](#)
 DDSC_LOG_COMPILE_CONTENT_FILTER_EC
 DDS_C, [450](#)
 DDSC_LOG_COMPONENT_CREATE_EC
 DDS_C, [391](#)
 DDSC_LOG_COMPONENT_DELETE_EC
 DDS_C, [391](#)
 DDSC_LOG_COMPONENT_LOOKUP_EC
 DDS_C, [390](#)
 DDSC_LOG_CONDITION_OBJECT
 DDS_C, [381](#)
 DDSC_LOG_CONDREF_OBJECT
 DDS_C, [381](#)
 DDSC_LOG_CONTENT_FILTER_QOS_POLICY
 DDS_C, [396](#)
 DDSC_LOG_CONTENT_FILTERING_NOT_ENABLED_EC
 DDS_C, [450](#)
 DDSC_LOG_COOKIE_PAYLOAD_SEQUENCE
 DDS_C, [388](#)
 DDSC_LOG_CREATE_FILTER_PLUGIN_EC
 DDS_C, [451](#)
 DDSC_LOG_CREATE_INDEX_EC
 DDS_C, [373](#)
 DDSC_LOG_CREATE_WRITER_FILTER_EC
 DDS_C, [450](#)
 DDSC_LOG_DATA_HOLDER_SEQ_OBJECT
 DDS_C, [383](#)
 DDSC_LOG_DATA_REPRESENTATION_QOS_POLICY
 DDS_C, [396](#)
 DDSC_LOG_DATABASE_CREATE_EC
 DDS_C, [373](#)
 DDSC_LOG_DATABASE_DELETE_EC
 DDS_C, [373](#)
 DDSC_LOG_DATAHOLDER_SEQUENCE
 DDS_C, [388](#)
 DDSC_LOG_DATAREADER_ADD_PEER_FAILED_EC
 DDS_C, [422](#)
 DDSC_LOG_DATAREADER_CDR
 DDS_C, [402](#)
 DDSC_LOG_DATAREADER_ENTITY
 DDS_C, [400](#)
 DDSC_LOG_DATAREADER_LISTENER
 DDS_C, [398](#)
 DDSC_LOG_DATAREADER_NETIO_KIND
 DDS_C, [406](#)
 DDSC_LOG_DATAREADER_QOS
 DDS_C, [392](#)
 DDSC_LOG_DATAREADER_RECORD
 DDS_C, [375](#)
 DDSC_LOG_DATAREADER_RESOURCE_QOS_POLICY
 DDS_C, [396](#)
 DDSC_LOG_DATAREADERIO_COMPONENT
 DDS_C, [390](#)
 DDSC_LOG_DATAREADERIO_OBJECT
 DDS_C, [380](#)
 DDSC_LOG_DATAREADERQOS_OBJECT
 DDS_C, [379](#)
 DDSC_LOG_DATAWRITER_ADD_PEER_FAILED_EC
 DDS_C, [422](#)
 DDSC_LOG_DATAWRITER_CDR
 DDS_C, [402](#)
 DDSC_LOG_DATAWRITER_ENTITY
 DDS_C, [400](#)
 DDSC_LOG_DATAWRITER_INLINE
 DDS_C, [402](#)
 DDSC_LOG_DATAWRITER_LISTENER
 DDS_C, [398](#)
 DDSC_LOG_DATAWRITER_NETIO_KIND
 DDS_C, [406](#)
 DDSC_LOG_DATAWRITER_QOS
 DDS_C, [392](#)
 DDSC_LOG_DATAWRITER_RECORD
 DDS_C, [374](#)
 DDSC_LOG_DATAWRITER_RESOURCE_QOS_POLICY
 DDS_C, [395](#)
 DDSC_LOG_DATAWRITERIO_COMPONENT
 DDS_C, [390](#)
 DDSC_LOG_DATAWRITERIO_OBJECT

DDS_C, [380](#)
 DDSC_LOG_DATAWRITERQOS_OBJECT
 DDS_C, [379](#)
 DDSC_LOG_DB_CURSOR_INVALIDATED_EC
 DDS_C, [374](#)
 DDSC_LOG_DEADLINE_QOS_POLICY
 DDS_C, [394](#)
 DDSC_LOG_DEFAULTDATAREADER_QOS
 DDS_C, [393](#)
 DDSC_LOG_DEFAULTDATAWRITER_QOS
 DDS_C, [393](#)
 DDSC_LOG_DEFAULTPARTICIPANT_QOS
 DDS_C, [393](#)
 DDSC_LOG_DEFAULTPUBLISHER_QOS
 DDS_C, [393](#)
 DDSC_LOG_DEFAULTSUBSCRIBER_QOS
 DDS_C, [393](#)
 DDSC_LOG_DEFAULTTOPIC_QOS
 DDS_C, [393](#)
 DDSC_LOG_DELETE_INDEX_EC
 DDS_C, [374](#)
 DDSC_LOG_DESERIALIZE_BAD_PID_LENGTH_EC
 DDS_C, [403](#)
 DDSC_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC
 DDS_C, [420](#)
 DDSC_LOG_DESERIALIZE_CONTENT_FILTER_INFO_EC
 DDS_C, [451](#)
 DDSC_LOG_DESERIALIZE_LOCATOR_EC
 DDS_C, [421](#)
 DDSC_LOG_DESERIALIZE_PARTITION_SEQ_EC
 DDS_C, [422](#)
 DDSC_LOG_DESERIALIZE_PROTOCOL_VERSION_EC
 DDS_C, [421](#)
 DDSC_LOG_DESERIALIZE_REMOTE_CONTENT_FILTER_EC
 DDS_C, [451](#)
 DDSC_LOG_DESERIALIZE_UNKNOWN_PID_EC
 DDS_C, [419](#)
 DDSC_LOG_DESERIALIZE_VENDOR_ID_EC
 DDS_C, [421](#)
 DDSC_LOG_DESTINATION_ORDER_POLICY
 DDS_C, [396](#)
 DDSC_LOG_DESTINATION_SEQUENCE
 DDS_C, [387](#)
 DDSC_LOG_DISC_ADD_PEER_EC
 DDS_C, [419](#)
 DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_DELETED_EC
 DDS_C, [419](#)
 DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_ENABLED_EC
 DDS_C, [418](#)
 DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_DELETED_EC
 DDS_C, [419](#)
 DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_ENABLED_EC
 DDS_C, [419](#)
 DDSC_LOG_DISC_AFTER_LOCAL_PARTICIPANT_CREATED_EC
 DDS_C, [418](#)
 DDSC_LOG_DISC_BEFORE_LOCAL_PARTICIPANT_CREATED_EC
 DDS_C, [418](#)
 DDSC_LOG_DISC_BEFORE_REMOTE_PARTICIPANT_DELETED_EC
 DDS_C, [419](#)
 DDSC_LOG_DISC_LOCAL_PARTICIPANT_ENABLED_EC
 DDS_C, [418](#)
 DDSC_LOG_DISC_REMOTE_PARTICIPANT_REMOVE_EC
 DDS_C, [422](#)
 DDSC_LOG_DISCOVERY_COMPONENT
 DDS_C, [389](#)
 DDSC_LOG_DR_COMMIT_ENTRY_EC
 DDS_C, [412](#)
 DDSC_LOG_DR_COMMIT_SAMPLE_EC
 DDS_C, [412](#)
 DDSC_LOG_DR_COPY_DATA_SAMPLE_EC
 DDS_C, [412](#)
 DDSC_LOG_DR_CREATE_TYPED_READER_EC
 DDS_C, [411](#)
 DDSC_LOG_DR_DESERIALIZE_KEYHASH_EC
 DDS_C, [412](#)
 DDSC_LOG_DR_DISPOSE_KEY_EC
 DDS_C, [413](#)
 DDSC_LOG_DR_FILTER_ERROR_EC
 DDS_C, [412](#)
 DDSC_LOG_DR_GET_ENTRY_FAILED_EC
 DDS_C, [412](#)
 DDSC_LOG_DR_INSTANCE_ASSERTION_FAILED_EC
 DDS_C, [413](#)
 DDSC_LOG_DR_INSTANCE_MAPPING_EXHAUSTED_EC
 DDS_C, [413](#)
 DDSC_LOG_DR_INSTANCE_TO_KEYHASH_EC
 DDS_C, [413](#)
 DDSC_LOG_DR_ON_SAMPLE_REMOVED_EC
 DDS_C, [413](#)
 DDSC_LOG_DR_READ_TAKE_FAILURE_EC
 DDS_C, [413](#)
 DDSC_LOG_DR_UNREGISTER_KEY_EC
 DDS_C, [412](#)
 DDSC_LOG_DUPLICATE_GUID_PREFIX_EC
 DDS_C, [416](#)
 DDSC_LOG_DUPLICATE_PARTICIPANT_NAME_EC
 DDS_C, [417](#)
 DDSC_LOG_DUPLICATE_REMOTE_PARTICIPANT_NAME_EC
 DDS_C, [417](#)
 DDSC_LOG_DURABILITY_QOS_POLICY
 DDS_C, [395](#)
 DDSC_LOG_DW_ACKNACK_FAILED_EC
 DDS_C, [410](#)
 DDSC_LOG_DW_COMMIT_EC
 DDS_C, [410](#)
 DDSC_LOG_DW_CREATE_TYPED_WRITER_EC
 DDS_C, [410](#)
 DDSC_LOG_DW_FLOW_CONTROLLER_REQUIRED_EC
 DDS_C, [410](#)

DDS_C, 411	DDS_C, 369
DDSC_LOG_DW_HISTORY_REGISTER_KEY_EC	DDSC_LOG_FIND_SUBSCRIPTION_PARENT_EC
DDS_C, 410	DDS_C, 369
DDSC_LOG_DW_ILLEGAL_KEY_KIND_EC	DDSC_LOG_FLOW_CONTROLLER_CREATE_EC
DDS_C, 410	DDS_C, 447
DDSC_LOG_DW_INCOMPATIBLE_PADDING_EC	DDSC_LOG_FLOW_CONTROLLER_DELETE_EC
DDS_C, 411	DDS_C, 447
DDSC_LOG_DW_INSTANCE_ASSERTION_FAILED_EC	DDSC_LOG_FLOWCONTROLLER_INCONSISTENT_PROP_EC
DDS_C, 411	DDS_C, 449
DDSC_LOG_DW_KEYHASH_CREATE_EC	DDSC_LOG_GET_MTU_EC
DDS_C, 410	DDS_C, 372
DDSC_LOG_DW_UNKNOWN_FLOW_CONTROLLER_EC	DDSC_LOG_GET_MTU_NO_ROUTES_EC
DDS_C, 410	DDS_C, 372
DDSC_LOG_ENABLE_WRITER_FILTERING_EC	DDSC_LOG_GET_NEXT_OBJECT_ID_EC
DDS_C, 450	DDS_C, 367
DDSC_LOG_ENABLED_DISCOVERY_TRANSPORT_SEQUENCE	DDSC_LOG_GET_SERIALIZED_KEY_SIZE_EC
DDS_C, 387	DDS_C, 371
DDSC_LOG_ENABLED_TRANSPORT_SEQUENCE	DDSC_LOG_HEARTBEATS_QOS_POLICY
DDS_C, 386	DDS_C, 395
DDSC_LOG_ENABLED_USER_TRANSPORT_SEQUENCED	DDSC_LOG_HISTORY_QOS_POLICY
DDS_C, 386	DDS_C, 394
DDSC_LOG_ENDPOINT_NOT_CHILD_OF_PARTICIPANT_EC	DDSC_LOG_HISTORY_RESOURCE
DDS_C, 368	DDS_C, 391
DDSC_LOG_ENTITY_DIFFERENT_FACTORY_EC	DDSC_LOG_ILLEGAL_OBJECTID_EC
DDS_C, 402	DDS_C, 370
DDSC_LOG_ENTITY_ENABLE_EC	DDSC_LOG_INCOMPATIBLE_PARTITION_QOS_EC
DDS_C, 401	DDS_C, 448
DDSC_LOG_ENTITY_FINALIZE_EC	DDSC_LOG_INCOMPATIBLE_PRESENTATION_QOS_EC
DDS_C, 401	DDS_C, 448
DDSC_LOG_ENTITY_INITIALIZE_EC	DDSC_LOG_INITIAL_PEER_SEQUENCE
DDS_C, 401	DDS_C, 387
DDSC_LOG_ENTITY_INVALID_PROPERTY_EC	DDSC_LOG_INITIALIZE_DISCOVERY_QUEUE_EC
DDS_C, 402	DDS_C, 422
DDSC_LOG_ENTITY_NAME_TOO_LONG_EC	DDSC_LOG_INTERPRETER_SUPPORT_UNEXPECTED_ERROR_EC
DDS_C, 446	DDS_C, 447
DDSC_LOG_ENTITY_NOT_EMPTY_EC	DDSC_LOG_INTRA_NETIO_KIND
DDS_C, 401	DDS_C, 406
DDSC_LOG_ENTITY_NOT_ENABLED_EC	DDSC_LOG_INVALID_BUFFER_LENGTH_EC
DDS_C, 401	DDS_C, 448
DDSC_LOG_ENTITY_OBJECT	DDSC_LOG_INVALID_DOMAINID_EC
DDS_C, 379	DDS_C, 371
DDSC_LOG_EVALUATE_CONTENT_FILTER_EC	DDSC_LOG_INVALID_DURATION_EC
DDS_C, 451	DDS_C, 366
DDSC_LOG_EVALUATE_WRITER_FILTER_EC	DDSC_LOG_INVALID_ENDPOINT_GUID_EC
DDS_C, 450	DDS_C, 368
DDSC_LOG_FAILED_UPDATE_STATUS_CONDITION_EC	DDSC_LOG_INVALID_PARTICIPANT_GUID_PREFIX_EC
DDS_C, 370	DDS_C, 366
DDSC_LOG_FILTER_PLUGIN_FACTORY_NOT_FOUND_EC	DDSC_LOG_INVALID_PARTICIPANT_NAME_EC
DDS_C, 450	DDS_C, 366
DDSC_LOG_FINALIZE_FILTER_PLUGIN_EC	DDSC_LOG_IO_SNPRINTF_FAILED_EC
DDS_C, 451	DDS_C, 367
DDSC_LOG_FINALIZE_TRUST_FAILED_EC	DDSC_LOG_IPC_ALLOC_FAILED_EC
DDS_C, 438	DDS_C, 423
DDSC_LOG_FIND_PUBLICATION_PARENT_EC	DDSC_LOG_IPC_CHANNEL_ADD_ANON_ROUTE_EC

- DDS_C, [424](#)
- DDSC_LOG_IPC_CHANNEL_ADD_PEER_EC
 - DDS_C, [425](#)
- DDSC_LOG_IPC_CHANNEL_CONFIG_FINALIZE_FAILED_EC
 - DDS_C, [426](#)
- DDSC_LOG_IPC_CHANNEL_CONFIG_INITIALIZE_FAILED_EC
 - DDS_C, [426](#)
- DDSC_LOG_IPC_CHANNEL_CONFIG_SET_FAILED_EC
 - DDS_C, [426](#)
- DDSC_LOG_IPC_CHANNEL_CREATE_FAILED_EC
 - DDS_C, [427](#)
- DDSC_LOG_IPC_CHANNEL_DELETE_ANON_ROUTE_EC
 - DDS_C, [425](#)
- DDSC_LOG_IPC_CHANNEL_ENABLE_FAILED_EC
 - DDS_C, [427](#)
- DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_AUTH_HANDSHAKE_FAILED_EC
 - DDS_C, [426](#)
- DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_AUTH_REQUEST_MISMATCHED_DW_RECORD
 - DDS_C, [426](#)
- DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_ENDPOINT_NOT_ENDPOINT_STATE_FAILED_EC
 - DDS_C, [426](#)
- DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_PARTICIPANT_STATE_FAILED_EC
 - DDS_C, [426](#)
- DDSC_LOG_IPC_CHANNEL_PROCESS_SAMPLE_EC
 - DDS_C, [425](#)
- DDSC_LOG_IPC_CHANNEL_READER_CREATE_EC
 - DDS_C, [424](#)
- DDSC_LOG_IPC_CHANNEL_REMOVE_PEER_EC
 - DDS_C, [425](#)
- DDSC_LOG_IPC_CHANNEL_RETURN_LOAN_EC
 - DDS_C, [425](#)
- DDSC_LOG_IPC_CHANNEL_SET_LISTENERS_FAILED_EC
 - DDS_C, [427](#)
- DDSC_LOG_IPC_CHANNEL_TOPIC_CREATE_EC
 - DDS_C, [424](#)
- DDSC_LOG_IPC_CHANNEL_TYPE_PLUGIN_EC
 - DDS_C, [424](#)
- DDSC_LOG_IPC_CHANNEL_TYPE_REGISTER_EC
 - DDS_C, [423](#)
- DDSC_LOG_IPC_CHANNEL_UNKNOWN_EC
 - DDS_C, [424](#)
- DDSC_LOG_IPC_CHANNEL_WRITE_SAMPLE_EC
 - DDS_C, [425](#)
- DDSC_LOG_IPC_CHANNEL_WRITER_CREATE_EC
 - DDS_C, [424](#)
- DDSC_LOG_IPC_FINALIZE_FAILED_EC
 - DDS_C, [423](#)
- DDSC_LOG_IPC_INIT_FAILED_EC
 - DDS_C, [423](#)
- DDSC_LOG_IPC_UPDATE_QOS_FAILED_EC
 - DDS_C, [424](#)
- DDSC_LOG_LATENCY_BUDGET_QOS_POLICY
 - DDS_C, [396](#)
- DDSC_LOG_LISTENER_GET_EC
 - DDS_C, [399](#)
- DDSC_LOG_LISTENER_INCONSISTENT_EC
 - DDS_C, [399](#)
- DDSC_LOG_LISTENER_SET_EC
 - DDS_C, [399](#)
- DDSC_LOG_LISTENER_SET_ILLEGAL_NULL_EC
 - DDS_C, [400](#)
- DDSC_LOG_LIVELINESS_QOS_POLICY
 - DDS_C, [394](#)
- DDSC_LOG_LOCATORS_FULL_EC
 - DDS_C, [421](#)
- DDSC_LOG_LOCK_EC
 - DDS_C, [449](#)
- DDSC_LOG_LOG_OBJECT
 - DDS_C, [381](#)
- DDSC_LOG_MAX_PLUGIN_EC
 - DDS_C, [415](#)
- DDSC_LOG_MATCHED_DW_RECORD
 - DDS_C, [376](#)
- DDSC_LOG_MAX_PLUGIN_STATE_FAILED_EC
 - DDS_C, [369](#)
- DDSC_LOG_MONITOR_STATE_FAILED_EC
 - DDS_C, [381](#)
- DDSC_LOG_MEMORY_MANAGER_DELETE_EC
 - DDS_C, [448](#)
- DDSC_LOG_MEMORY_MANAGER_NOT_OWNER_EC
 - DDS_C, [448](#)
- DDSC_LOG_MEMORY_MANAGER_OUT_OF_RESOURCES_EC
 - DDS_C, [447](#)
- DDSC_LOG_MEMPOOL_OBJECT
 - DDS_C, [383](#)
- DDSC_LOG_MESSAGE_DATA_UNKNOWN_LIVELINESS_KIND_EC
 - DDS_C, [446](#)
- DDSC_LOG_MESSAGE_DATA_WRONG_LIVELINESS_KIND_EC
 - DDS_C, [446](#)
- DDSC_LOG_METAMULTICAST_SEQUENCE
 - DDS_C, [387](#)
- DDSC_LOG_METAUNICAST_SEQUENCE
 - DDS_C, [387](#)
- DDSC_LOG_MIN_DISCOVERY_MTU_EC
 - DDS_C, [371](#)
- DDSC_LOG_MIN_MTU_SIZE_EC
 - DDS_C, [372](#)
- DDSC_LOG_MUTEX_OBJECT
 - DDS_C, [381](#)
- DDSC_LOG_NAME_LOOKUP_EC
 - DDS_C, [445](#)
- DDSC_LOG_NETIO_ADD_ANON_TOPIC_ROUTE_EC
 - DDS_C, [405](#)
- DDSC_LOG_NETIO_ADD_ROUTE_EC
 - DDS_C, [407](#)
- DDSC_LOG_NETIO_ADD_TOPIC_ROUTE_EC
 - DDS_C, [405](#)
- DDSC_LOG_NETIO_BIND_EC
 - DDS_C, [399](#)

DDS_C, [407](#)
 DDSC_LOG_NETIO_BIND_EXTERNAL_EC
 DDS_C, [406](#)
 DDSC_LOG_NETIO_COMPONENT
 DDS_C, [390](#)
 DDSC_LOG_NETIO_DELETE_ROUTE_EC
 DDS_C, [407](#)
 DDSC_LOG_NETIO_DELETE_TOPIC_ROUTE_EC
 DDS_C, [405](#)
 DDSC_LOG_NETIO_DISCOVERY_ENABLED_TRANSPORT_EC
 DDS_C, [409](#)
 DDSC_LOG_NETIO_FORCED_REMOVE_EC
 DDS_C, [408](#)
 DDSC_LOG_NETIO_FORWARD_TOPIC_EC
 DDS_C, [405](#)
 DDSC_LOG_NETIO_GET_EXTERNAL_INTF_EC
 DDS_C, [407](#)
 DDSC_LOG_NETIO_GET_ROUTE_TABLE_FAILED_EC
 DDS_C, [408](#)
 DDSC_LOG_NETIO_NETIO_KIND
 DDS_C, [406](#)
 DDSC_LOG_NETIO_NO_ROUTE_EC
 DDS_C, [407](#)
 DDSC_LOG_NETIO_PACKETPOOL_CREATE_EC
 DDS_C, [409](#)
 DDSC_LOG_NETIO_PACKETPOOL_DELETE_EC
 DDS_C, [409](#)
 DDSC_LOG_NETIO_PACKETPOOL_GET_EC
 DDS_C, [409](#)
 DDSC_LOG_NETIO_PACKETPOOL_RETURN_EC
 DDS_C, [409](#)
 DDSC_LOG_NETIO_PEER_LOOKUP_EC
 DDS_C, [408](#)
 DDSC_LOG_NETIO_ROUTE_LOOKUP_FAILED_EC
 DDS_C, [407](#)
 DDSC_LOG_NETIO_SEND_FAILED_EC
 DDS_C, [408](#)
 DDSC_LOG_NETIO_SET_STATE_EC
 DDS_C, [408](#)
 DDSC_LOG_NETIO_UNBIND_EC
 DDS_C, [407](#)
 DDSC_LOG_NETIO_UNBIND_EXTERNAL_EC
 DDS_C, [406](#)
 DDSC_LOG_NETMASK_SEQUENCE
 DDS_C, [386](#)
 DDSC_LOG_OBJECT_ALLOCATE_EC
 DDS_C, [384](#)
 DDSC_LOG_OBJECT_COPY_EC
 DDS_C, [385](#)
 DDSC_LOG_OBJECT_CREATE_EC
 DDS_C, [386](#)
 DDSC_LOG_OBJECT_DELETE_EC
 DDS_C, [385](#)
 DDSC_LOG_OBJECT_EMPTY_EC
 DDS_C, [385](#)
 DDSC_LOG_OBJECT_FINALIZE_EC
 DDS_C, [385](#)
 DDSC_LOG_OBJECT_GET_PROPERTY_EC
 DDS_C, [385](#)
 DDSC_LOG_OBJECT_INITIALIZE_EC
 DDS_C, [384](#)
 DDSC_LOG_OBJECT_INUSECOUNT_EC
 DDS_C, [386](#)
 DDSC_LOG_OBJECT_REFCOUNT_EC
 DDS_C, [385](#)
 DDSC_LOG_OBJECT_SET_PROPERTY_EC
 DDS_C, [385](#)
 DDSC_LOG_ON_TYPE_REGISTERED_FAILURE_EC
 DDS_C, [371](#)
 DDSC_LOG_ON_TYPE_UNREGISTERED_FAILURE_EC
 DDS_C, [371](#)
 DDSC_LOG_OWNERSHIP_QOS_POLICY
 DDS_C, [395](#)
 DDSC_LOG_OWNERSHIP_STRENGTH_QOS_POLICY
 DDS_C, [395](#)
 DDSC_LOG_PACKET_INIT_EC
 DDS_C, [408](#)
 DDSC_LOG_PACKET_SET_HEAD_EC
 DDS_C, [409](#)
 DDSC_LOG_PACKET_SET_TAIL_EC
 DDS_C, [409](#)
 DDSC_LOG_PARTICIPANT_BINARY_PROPERTIES
 DDS_C, [383](#)
 DDSC_LOG_PARTICIPANT_BUILTIN_PUBLISHER
 DDS_C, [383](#)
 DDSC_LOG_PARTICIPANT_BUILTIN_SUBSCRIBER
 DDS_C, [383](#)
 DDSC_LOG_PARTICIPANT_DOES_NOT_EXIST_EC
 DDS_C, [368](#)
 DDSC_LOG_PARTICIPANT_ENTITY
 DDS_C, [400](#)
 DDSC_LOG_PARTICIPANT_GENERIC_MESSAGE_OBJECT
 DDS_C, [383](#)
 DDSC_LOG_PARTICIPANT_ID_QOS_POLICY
 DDS_C, [395](#)
 DDSC_LOG_PARTICIPANT_LISTENER
 DDS_C, [399](#)
 DDSC_LOG_PARTICIPANT_LOOKUP_EC
 DDS_C, [368](#)
 DDSC_LOG_PARTICIPANT_NAME_EMPTY_EC
 DDS_C, [416](#)
 DDSC_LOG_PARTICIPANT_POOL_OBJECT
 DDS_C, [382](#)
 DDSC_LOG_PARTICIPANT_PROPERTIES
 DDS_C, [383](#)
 DDSC_LOG_PARTICIPANT_QOS
 DDS_C, [392](#)
 DDSC_LOG_PARTICIPANT_RECORD

DDS_C, [374](#)
DDSC_LOG_PARTICIPANT_RESOURCES
 DDS_C, [391](#)
DDSC_LOG_PARTICIPANTDATA_OBJECT
 DDS_C, [378](#)
DDSC_LOG_PARTICIPANTFACTORY_LISTENER
 DDS_C, [399](#)
DDSC_LOG_PARTICIPANTFACTORY_QOS
 DDS_C, [393](#)
DDSC_LOG_PARTICIPANTFACTORYQOS_OBJECT
 DDS_C, [380](#)
DDSC_LOG_PARTICIPANTQOS_OBJECT
 DDS_C, [379](#)
DDSC_LOG_PARTMESSAGE_WRITE_SAMPLE_FAILED_EC
 DDS_C, [446](#)
DDSC_LOG_PROPERTY_QOS_POLICY
 DDS_C, [396](#)
DDSC_LOG_PROPERTYQOSPOLICY_OBJECT
 DDS_C, [384](#)
DDSC_LOG_PROTOCOL_QOS_POLICY
 DDS_C, [394](#)
DDSC_LOG_PUBLICATION_ENTITY
 DDS_C, [401](#)
DDSC_LOG_PUBLICATION_RECORD
 DDS_C, [374](#)
DDSC_LOG_PUBLICATIONDATA_OBJECT
 DDS_C, [378](#)
DDSC_LOG_PUBLISH_MODE_QOS_POLICY
 DDS_C, [396](#)
DDSC_LOG_PUBLISHER_ENTITY
 DDS_C, [400](#)
DDSC_LOG_PUBLISHER_LISTENER
 DDS_C, [399](#)
DDSC_LOG_PUBLISHER_QOS
 DDS_C, [392](#)
DDSC_LOG_PUBLISHER_RECORD
 DDS_C, [375](#)
DDSC_LOG_PUBLISHERQOS_OBJECT
 DDS_C, [379](#)
DDSC_LOG_QOS_CHANGED_EC
 DDS_C, [398](#)
DDSC_LOG_QOS_COPY_EC
 DDS_C, [397](#)
DDSC_LOG_QOS_FINALIZE_EC
 DDS_C, [397](#)
DDSC_LOG_QOS_GET_EC
 DDS_C, [398](#)
DDSC_LOG_QOS_IMMUTABLE_EC
 DDS_C, [398](#)
DDSC_LOG_QOS_INCONSISTENT_EC
 DDS_C, [397](#)
DDSC_LOG_QOS_INCONSISTENT_POLICIES_EC
 DDS_C, [397](#)
DDSC_LOG_QOS_INCONSISTENT_POLICY_EC
 DDS_C, [397](#)
DDSC_LOG_QOS_INITIALIZE_EC
 DDS_C, [397](#)
DDSC_LOG_QOS_SET_EC
 DDS_C, [397](#)
DDSC_LOG_QOS_SET_ON_ENABLED_EC
 DDS_C, [398](#)
DDSC_LOG_QUEUE_FINALIZE_EC
 DDS_C, [423](#)
DDSC_LOG_QUEUE_PUBLICATION_DISCOVERY_EC
 DDS_C, [423](#)
DDSC_LOG_QUEUE_SUBSCRIPTION_DISCOVERY_EC
 DDS_C, [423](#)
DDSC_LOG_READERHISTORY_COMPONENT
 DDS_C, [389](#)
DDSC_LOG_READTAKE_SEQUENCE
 DDS_C, [388](#)
DDSC_LOG_RECORD_CREATE_EC
 DDS_C, [376](#)
DDSC_LOG_RECORD_DELETE_EC
 DDS_C, [376](#)
DDSC_LOG_RECORD_ERROR_EC
 DDS_C, [377](#)
DDSC_LOG_RECORD_EXISTS_EC
 DDS_C, [377](#)
DDSC_LOG_RECORD_FINALIZE_EC
 DDS_C, [378](#)
DDSC_LOG_RECORD_INITIALIZE_EC
 DDS_C, [378](#)
DDSC_LOG_RECORD_INSERT_EC
 DDS_C, [377](#)
DDSC_LOG_RECORD_LOOKUP_EC
 DDS_C, [377](#)
DDSC_LOG_RECORD_NOT_EXISTS_EC
 DDS_C, [377](#)
DDSC_LOG_RECORD_REMOVE_EC
 DDS_C, [377](#)
DDSC_LOG_RECORD_RESET_EC
 DDS_C, [378](#)
DDSC_LOG_RECORD_SELECT_EC
 DDS_C, [377](#)
DDSC_LOG_REFRESH_LIVELINESS_FAILED_EC
 DDS_C, [447](#)
DDSC_LOG_REFRESH_REM_PARTICIPANT_EC
 DDS_C, [368](#)
DDSC_LOG_REFRESH_REM_PARTICIPANT_TIMEOUT_EC
 DDS_C, [368](#)
DDSC_LOG_RELEASE_META_MC_EC
 DDS_C, [370](#)
DDSC_LOG_RELEASE_META_UC_EC
 DDS_C, [370](#)
DDSC_LOG_RELEASE_USER_MC_EC
 DDS_C, [371](#)
DDSC_LOG_RELEASE_USER_UC_EC

DDS_C, [371](#)
 DDSC_LOG_RELIABILITY_QOS_POLICY
 DDS_C, [394](#)
 DDSC_LOG_REMOTE_ENDPOINT_TRUST_STATE_RECORD
 DDS_C, [375](#)
 DDSC_LOG_REMOTE_PARTICIPANT_KEY_NOT_EQUAL_EC
 DDS_C, [368](#)
 DDSC_LOG_REMOTE_PARTICIPANT_NAME_EMPTY_EC
 DDS_C, [417](#)
 DDSC_LOG_REMOTE_PARTICIPANT_RESTART_EC
 DDS_C, [418](#)
 DDSC_LOG_REMOTE_PUBLICATION_DATA_CHANGED_EC
 DDS_C, [372](#)
 DDSC_LOG_REMOTE_SUBSCRIPTION_DATA_CHANGED_EC
 DDS_C, [372](#)
 DDSC_LOG_REMOVE_PUBLICATION_EC
 DDS_C, [369](#)
 DDSC_LOG_REMOVE_SUBSCRIPTION_EC
 DDS_C, [369](#)
 DDSC_LOG_REPRESENTATION_ID_SEQUENCE
 DDS_C, [388](#)
 DDSC_LOG_RESERVE_LOCATORS_EC
 DDS_C, [369](#)
 DDSC_LOG_RESERVED_SEQUENCE
 DDS_C, [386](#)
 DDSC_LOG_RESOLVE_LOCATORS_PRIORITY_EC
 DDS_C, [372](#)
 DDSC_LOG_RESOURCE_EXCEEDED_EC
 DDS_C, [392](#)
 DDSC_LOG_RESOURCE_LIMIT_QOS_POLICY
 DDS_C, [394](#)
 DDSC_LOG_ROUTE_RECORD
 DDS_C, [375](#)
 DDSC_LOG_ROUTE_SEQUENCE
 DDS_C, [386](#)
 DDSC_LOG_ROUTERESOLVER_OBJECT
 DDS_C, [380](#)
 DDSC_LOG_RT_OBJECT
 DDS_C, [382](#)
 DDSC_LOG RTPS_COMPONENT
 DDS_C, [389](#)
 DDSC_LOG RTPS_NETIO_KIND
 DDS_C, [406](#)
 DDSC_LOG_SAMPLE_RESOURCES
 DDS_C, [391](#)
 DDSC_LOG_SEND_LIVELINESS_FAILED_EC
 DDS_C, [446](#)
 DDSC_LOG_SEQUENCE_COPY_EC
 DDS_C, [389](#)
 DDSC_LOG_SEQUENCE_FINALIZE_EC
 DDS_C, [389](#)
 DDSC_LOG_SEQUENCE_GETREF_EC
 DDS_C, [388](#)
 DDSC_LOG_SEQUENCE_INITIALIZE_EC
 DDS_C, [389](#)
 DDSC_LOG_SEQUENCE_INVALID_EC
 DDS_C, [389](#)
 DDSC_LOG_SEQUENCE_SETLENGTH_EC
 DDS_C, [388](#)
 DDSC_LOG_SEQUENCE_SETMAX_EC
 DDS_C, [388](#)
 DDSC_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC
 DDS_C, [419](#)
 DDSC_LOG_SERIALIZE_DEFAULT_UNICAST_EC
 DDS_C, [420](#)
 DDSC_LOG_SERIALIZE_ENTITY_NAME_EC
 DDS_C, [420](#)
 DDSC_LOG_SERIALIZE_GUID_EC
 DDS_C, [420](#)
 DDSC_LOG_SERIALIZE_LEASE_DURATION_EC
 DDS_C, [422](#)
 DDSC_LOG_SERIALIZE_PRODUCT_VERSION_EC
 DDS_C, [422](#)
 DDSC_LOG_SERIALIZE_PROTOCOL_VERSION_EC
 DDS_C, [421](#)
 DDSC_LOG_SERIALIZE_TOPIC_NAME_EC
 DDS_C, [420](#)
 DDSC_LOG_SERIALIZE_TYPE_NAME_EC
 DDS_C, [420](#)
 DDSC_LOG_SERIALIZE_VENDOR_ID_EC
 DDS_C, [421](#)
 DDSC_LOG_SET_ENTITY_NAME_EC
 DDS_C, [367](#)
 DDSC_LOG_STRING_OBJECT
 DDS_C, [384](#)
 DDSC_LOG_STRING_RECORD
 DDS_C, [376](#)
 DDSC_LOG_STRINGMANAGER_OBJECT
 DDS_C, [384](#)
 DDSC_LOG_SUBSCRIBER_ENTITY
 DDS_C, [400](#)
 DDSC_LOG_SUBSCRIBER_LISTENER
 DDS_C, [399](#)
 DDSC_LOG_SUBSCRIBER_QOS
 DDS_C, [392](#)
 DDSC_LOG_SUBSCRIBER_RECORD
 DDS_C, [375](#)
 DDSC_LOG_SUBSCRIBERQOS_OBJECT
 DDS_C, [379](#)
 DDSC_LOG_SUBSCRIPTION_ENTITY
 DDS_C, [401](#)
 DDSC_LOG_SUBSCRIPTION_RECORD
 DDS_C, [374](#)
 DDSC_LOG_SUBSCRIPTIONDATA_OBJECT
 DDS_C, [378](#)
 DDSC_LOG_SYS_GET_HOSTNAME_EC
 DDS_C, [367](#)
 DDSC_LOG_SYS_GETTIME_EC

DDS_C, [366](#)
 DDSC_LOG_SYSTEM_OBJECT
 DDS_C, [381](#)
 DDSC_LOG_TABLE_CREATE_EC
 DDS_C, [373](#)
 DDSC_LOG_TABLE_DELETE_EC
 DDS_C, [373](#)
 DDSC_LOG_TABLE_INUSE_EC
 DDS_C, [373](#)
 DDSC_LOG_TABLE_SELECT_EC
 DDS_C, [373](#)
 DDSC_LOG_TIMER_CREATE_TIMEOUT_EC
 DDS_C, [369](#)
 DDSC_LOG_TIMER_OBJECT
 DDS_C, [382](#)
 DDSC_LOG_TIMEROUT_OBJECT
 DDS_C, [382](#)
 DDSC_LOG_TOPIC_ENTITY
 DDS_C, [400](#)
 DDSC_LOG_TOPIC_FIND_EC
 DDS_C, [367](#)
 DDSC_LOG_TOPIC_LISTENER
 DDS_C, [398](#)
 DDSC_LOG_TOPIC_NAME_CMP_EC
 DDS_C, [414](#)
 DDSC_LOG_TOPIC_NARROW_EC
 DDS_C, [370](#)
 DDSC_LOG_TOPIC_OBJECT
 DDS_C, [382](#)
 DDSC_LOG_TOPIC_QOS
 DDS_C, [392](#)
 DDSC_LOG_TOPIC_RECORD
 DDS_C, [374](#)
 DDSC_LOG_TOPIC_TOO_LONG_EC
 DDS_C, [415](#)
 DDSC_LOG_TOPICQOS_OBJECT
 DDS_C, [379](#)
 DDSC_LOG_TRANSPORT_COMPONENT
 DDS_C, [390](#)
 DDSC_LOG_TRANSPORT_QOS_POLICY
 DDS_C, [395](#)
 DDSC_LOG_TRUST_ADVANCE_SN_FAILED_EC
 DDS_C, [439](#)
 DDSC_LOG_TRUST_AFTER_REMOTE_PARTICIPANT_AUDIT_FAILED_EC
 DDS_C, [443](#)
 DDSC_LOG_TRUST_AFTER_REMOTE_PARTICIPANT_READ_FAILED_EC
 DDS_C, [435](#)
 DDSC_LOG_TRUST_ALLOC_FAILED_EC
 DDS_C, [438](#)
 DDSC_LOG_TRUST_ASSERT_AUTH_HANDSHAKE_STATE_FAILED_EC
 DDS_C, [444](#)
 DDSC_LOG_TRUST_ASSERT_PARTICIPANT_GENERIC_MESSAGE_FAILURE_EC
 DDS_C, [440](#)
 DDSC_LOG_TRUST_ASSERT_PROPERTY_FAILED_EC
 DDS_C, [442](#)
 DDSC_LOG_TRUST_BEGIN_HANDSHAKE_REPLY_EC
 DDS_C, [432](#)
 DDSC_LOG_TRUST_BEGIN_HANDSHAKE_REQUEST_EC
 DDS_C, [435](#)
 DDSC_LOG_TRUST_BIND_REMOTE_DATAREADER_FAILED_EC
 DDS_C, [436](#)
 DDSC_LOG_TRUST_BIND_REMOTE_DATAWRITER_FAILED_EC
 DDS_C, [436](#)
 DDSC_LOG_TRUST_CACHE_REMOTE_PARTICIPANT_SAMPLE_FAILED_EC
 DDS_C, [443](#)
 DDSC_LOG_TRUST_CHECK_CREATE_DATAREADER_FAILED_EC
 DDS_C, [432](#)
 DDSC_LOG_TRUST_CHECK_CREATE_DATAWRITER_FAILED_EC
 DDS_C, [432](#)
 DDSC_LOG_TRUST_CHECK_CREATE_PARTICIPANT_FAILED_EC
 DDS_C, [429](#)
 DDSC_LOG_TRUST_CHECK_REMOTE_DATAREADER_FAILED_EC
 DDS_C, [434](#)
 DDSC_LOG_TRUST_CHECK_REMOTE_DATAWRITER_FAILED_EC
 DDS_C, [434](#)
 DDSC_LOG_TRUST_CHECK_REMOTE_PARTICIPANT_FAILED_EC
 DDS_C, [431](#)
 DDSC_LOG_TRUST_CREATE_LOCAL_DATAREADER_INTERCEPTOR_T
 DDS_C, [441](#)
 DDSC_LOG_TRUST_CREATE_LOCAL_DATAWRITER_INTERCEPTOR_T
 DDS_C, [441](#)
 DDSC_LOG_TRUST_CREATE_LOCAL_DR_TOKEN_EC
 DDS_C, [433](#)
 DDSC_LOG_TRUST_CREATE_LOCAL_DW_TOKEN_EC
 DDS_C, [433](#)
 DDSC_LOG_TRUST_CREATE_LOCAL_PARTICIPANT_INTERCEPTOR_T
 DDS_C, [432](#)
 DDSC_LOG_TRUST_CREATE_TRUST_PLUGIN_EC
 DDS_C, [445](#)
 DDSC_LOG_TRUST_CREATE_TRUST_PLUGINS_FAILED_EC
 DDS_C, [435](#)
 DDSC_LOG_TRUST_DATA_HOLDER_COPY_FAILED_EC
 DDS_C, [438](#)
 DDSC_LOG_TRUST_DELETE_TRUST_PLUGINS_EC
 DDS_C, [431](#)
 DDSC_LOG_TRUST_ENDPOINT_INTERNAL_ATTRIBUTES_CREATE_FA
 DDS_C, [441](#)
 DDSC_LOG_TRUST_ENDPOINT_INTERNAL_ATTRIBUTES_VALIDATE_LOCAL_IDENTITY_EC
 DDS_C, [428](#)
 DDSC_LOG_TRUST_FINALIZE_AC_PLUGIN_PROPERTIES_FAILED_EC
 DDS_C, [443](#)
 DDSC_LOG_TRUST_FINALIZE_AUTH_HANDSHAKE_STATE_FAILED_EC
 DDS_C, [444](#)
 DDSC_LOG_TRUST_FINALIZE_REMOTE_PARTICIPANT_TRUST_FAILED_EC
 DDS_C, [444](#)
 DDSC_LOG_TRUST_GET_AUTH_HANDSHAKE_STATE_FAILED_EC
 DDS_C, [444](#)
 DDSC_LOG_TRUST_GET_AUTHENTICATED_PEER_CREDENTIAL_TOK

DDS_C, [430](#)
 DDSC_LOG_TRUST_GET_DATAREADER_TRUST_ATTRIBUTES_FAILED_EC
 DDS_C, [432](#)
 DDSC_LOG_TRUST_GET_DATAWRITER_TRUST_ATTRIBUTES_FAILED_EC
 DDS_C, [432](#)
 DDSC_LOG_TRUST_GET_IDENTITY_TOKEN_FAILED_EC
 DDS_C, [428](#)
 DDSC_LOG_TRUST_GET_PARTICIPANT_TRUST_ATTRIBUTES_FAILED_EC
 DDS_C, [429](#)
 DDSC_LOG_TRUST_GET_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC
 DDS_C, [429](#)
 DDSC_LOG_TRUST_GET_PERMISSIONS_TOKEN_FAILED_EC
 DDS_C, [429](#)
 DDSC_LOG_TRUST_GET_SHARED_SECRET_FAILED_EC
 DDS_C, [431](#)
 DDSC_LOG_TRUST_GET_TOPIC_TRUST_ATTRIBUTES_FAILED_EC
 DDS_C, [431](#)
 DDSC_LOG_TRUST_IGNORED_HANDSHAKE_MESSAGE_EC
 DDS_C, [427](#)
 DDSC_LOG_TRUST_IGNORED_REMOTE_PARTICIPANT_EC
 DDS_C, [427](#)
 DDSC_LOG_TRUST_INCOMPATIBLE_ENDPOINT_TRUST_ATTRIBUTES_EC
 DDS_C, [447](#)
 DDSC_LOG_TRUST_INITIALIZE_PARTICIPANT_BUILTIN_DATA_FAILED_EC
 DDS_C, [439](#)
 DDSC_LOG_TRUST_INITIALIZE_PARTICIPANT_TRUST_FAILED_EC
 DDS_C, [438](#)
 DDSC_LOG_TRUST_INVALID_AUTH_HANDSHAKE_STATE_EC
 DDS_C, [444](#)
 DDSC_LOG_TRUST_INVALID_GENERIC_MESSAGE_REPLY_EC
 DDS_C, [438](#)
 DDSC_LOG_TRUST_INVALID_IDENTITY_HANDLE_EC
 DDS_C, [428](#)
 DDSC_LOG_TRUST_INVALID_INTERCEPTOR_HANDLE_EC
 DDS_C, [429](#)
 DDSC_LOG_TRUST_INVALID_INTERCEPTOR_TOKENS_EC
 DDS_C, [441](#)
 DDSC_LOG_TRUST_INVALID_INTERCEPTOR_TOKENS_EC
 DDS_C, [440](#)
 DDSC_LOG_TRUST_INVALID_PARTICIPANT_GENERIC_MESSAGE_EC
 DDS_C, [439](#)
 DDSC_LOG_TRUST_INVALID_PARTICIPANT_GUID_EC
 DDS_C, [428](#)
 DDSC_LOG_TRUST_INVALID_PARTICIPANT_INTERNAL_ATTRIBUTES_EC
 DDS_C, [443](#)
 DDSC_LOG_TRUST_INVALID_PERMISSIONS_HANDLE_EC
 DDS_C, [428](#)
 DDSC_LOG_TRUST_INVALID_SAMPLE_MAX_SERIALIZED_SIZE_EC
 DDS_C, [438](#)
 DDSC_LOG_TRUST_INVALID_SHARED_SECRET_EC
 DDS_C, [442](#)
 DDSC_LOG_TRUST_LOOKUP_FACTORY_FAILED_EC
 DDS_C, [445](#)
 DDSC_LOG_TRUST_PARSE_PROPERTY_FAILED_EC
 DDS_C, [435](#)
 DDSC_LOG_TRUST_PREPARE_AUTH_REQUEST_EC
 DDS_C, [442](#)
 DDSC_LOG_TRUST_PREPARE_AUTH_HANDSHAKE_MESSAGE_EC
 DDS_C, [445](#)
 DDSC_LOG_TRUST_PREPARE_ENDPOINT_INTERCEPTOR_MESSAGE_EC
 DDS_C, [437](#)
 DDSC_LOG_TRUST_PREPARE_LOCAL_DP_TOKENS_EC
 DDS_C, [437](#)
 DDSC_LOG_TRUST_PREPARE_LOCAL_PARTICIPANT_STATE_FAILED_EC
 DDS_C, [442](#)
 DDSC_LOG_TRUST_PROCESS_HANDSHAKE_EC
 DDS_C, [431](#)
 DDSC_LOG_TRUST_PROCESS_REMOTE_INTERCEPTOR_TOKENS_FAILED_EC
 DDS_C, [435](#)
 DDSC_LOG_TRUST_REGISTER_LOCAL_DATAREADER_FAILED_EC
 DDS_C, [442](#)
 DDSC_LOG_TRUST_REGISTER_LOCAL_DATAWRITER_FAILED_EC
 DDS_C, [445](#)
 DDSC_LOG_TRUST_REGISTER_LOCAL_PARTICIPANT_FAILED_EC
 DDS_C, [433](#)
 DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_DATAREADER_FAILED_EC
 DDS_C, [434](#)
 DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_DATAWRITER_FAILED_EC
 DDS_C, [434](#)
 DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_PARTICIPANT_FAILED_EC
 DDS_C, [431](#)
 DDSC_LOG_TRUST_REGISTER_REMOTE_DATAREADER_FAILED_EC
 DDS_C, [436](#)
 DDSC_LOG_TRUST_REGISTER_REMOTE_DATAWRITER_FAILED_EC
 DDS_C, [436](#)
 DDSC_LOG_TRUST_RELEASE_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC
 DDS_C, [439](#)
 DDSC_LOG_TRUST_REMOVE_REMOTE_PARTICIPANT_EC
 DDS_C, [435](#)
 DDSC_LOG_TRUST_RESEND_HANDSHAKE_IN_PROGRESS_EC
 DDS_C, [439](#)
 DDSC_LOG_TRUST_RESEND_HANDSHAKE_MESSAGE_FAILED_EC
 DDS_C, [445](#)
 DDSC_LOG_TRUST_RESEND_HANDSHAKE_MESSAGE_START_FAILED_EC
 DDS_C, [440](#)
 DDSC_LOG_TRUST_RESEND_HANDSHAKE_STOP_FAILED_EC
 DDS_C, [439](#)
 DDSC_LOG_TRUST_RESET_REMOTE_PARTICIPANT_TRUST_FAILED_EC
 DDS_C, [439](#)

DDS_C, [444](#)
 DDSC_LOG_TRUST_RETURN_AUTHENTICATED_PEER_CREDENTIAL_TOKEN_FAILED_EC
 DDS_C, [441](#)
 DDSC_LOG_TRUST_RETURN_DATAWRITER_TRUST_ATTRIBUTES_FAILED_EC
 DDS_C, [432](#)
 DDSC_LOG_TRUST_RETURN_HANDSHAKE_HANDLE_FAILED_EC
 DDS_C, [433](#)
 DDSC_LOG_TRUST_RETURN_IDENTITY_HANDLE_FAILED_EC
 DDS_C, [430](#)
 DDSC_LOG_TRUST_RETURN_IDENTITY_TOKEN_FAILED_EC
 DDS_C, [430](#)
 DDSC_LOG_TRUST_RETURN_INTERCEPTOR_TOKENS_FAILED_EC
 DDS_C, [441](#)
 DDSC_LOG_TRUST_RETURN_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC
 DDS_C, [429](#)
 DDSC_LOG_TRUST_RETURN_PERMISSIONS_HANDLE_FAILED_EC
 DDS_C, [430](#)
 DDSC_LOG_TRUST_RETURN_PERMISSIONS_TOKEN_FAILED_EC
 DDS_C, [430](#)
 DDSC_LOG_TRUST_RETURN_SHAREDSECRET_HANDLE_FAILED_EC
 DDS_C, [430](#)
 DDSC_LOG_TRUST_SEND_AUTH_REQUEST_FAILED_EC
 DDS_C, [444](#)
 DDSC_LOG_TRUST_SEND_ENDPOINT_INTERCEPTOR_STATE_FAILED_EC
 DDS_C, [441](#)
 DDSC_LOG_TRUST_SEND_HANDSHAKE_EC
 DDS_C, [436](#)
 DDSC_LOG_TRUST_SEND_HANDSHAKE_MESSAGE_FAILED_EC
 DDS_C, [440](#)
 DDSC_LOG_TRUST_SEND_PARTICIPANT_INTERCEPTOR_STATE_FAILED_EC
 DDS_C, [440](#)
 DDSC_LOG_TRUST_SERIALIZE_DATA_FAILED_EC
 DDS_C, [438](#)
 DDSC_LOG_TRUST_SET_AUTH_HANDSHAKE_STATE_FAILED_EC
 DDS_C, [445](#)
 DDSC_LOG_TRUST_SET_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC
 DDS_C, [431](#)
 DDSC_LOG_TRUST_SET_REMOTE_DATAREADER_INTERCEPTOR_STATE_FAILED_EC
 DDS_C, [433](#)
 DDSC_LOG_TRUST_SET_REMOTE_DATAWRITER_INTERCEPTOR_STATE_FAILED_EC
 DDS_C, [433](#)
 DDSC_LOG_TRUST_SET_REMOTE_PARTICIPANT_INTERCEPTOR_STATE_FAILED_EC
 DDS_C, [433](#)
 DDSC_LOG_TRUST_TRANSFORM_INCOMING_SERIALIZED_DATA_LOAD_FAILURE_EC
 DDS_C, [443](#)
 DDSC_LOG_TRUST_TRANSFORM_OUTGOING_SERIALIZED_DATA_LOAD_FAILURE_EC
 DDS_C, [443](#)
 DDSC_LOG_TRUST_UNAUTHORIZED_REMOTE_PARTICIPANT_STATE_EC
 DDS_C, [428](#)
 DDSC_LOG_TRUST_UNEXPECTED_REMOTE_PARTICIPANT_STATE_EC
 DDS_C, [440](#)
 DDSC_LOG_TRUST_UNEXPECTED_VALIDATION_RESULT_EC
 DDS_C, [428](#)
 DDSC_LOG_TRUST_UNKNOWN_GMCLASSID_EC
 DDS_C, [427](#)
 DDSC_LOG_UNREGISTER_DATAREADER_FAILED_EC
 DDS_C, [427](#)
 DDSC_LOG_UNREGISTER_DATAWRITER_FAILED_EC
 DDS_C, [434](#)
 DDSC_LOG_UNREGISTER_PARTICIPANT_FAILED_EC
 DDS_C, [434](#)
 DDSC_LOG_UNREGISTER_REMOTE_DATAREADER_FAILED_EC
 DDS_C, [442](#)
 DDSC_LOG_UNREGISTER_REMOTE_DATAWRITER_FAILED_EC
 DDS_C, [442](#)
 DDSC_LOG_VALIDATE_LOCAL_DATAWRITER_TRUST_FAILED_EC
 DDS_C, [437](#)
 DDSC_LOG_VALIDATE_REMOTE_DATAREADER_TRUST_FAILED_EC
 DDS_C, [437](#)
 DDSC_LOG_VALIDATE_REMOTE_DATAWRITER_TRUST_FAILED_EC
 DDS_C, [437](#)
 DDSC_LOG_VALIDATE_REMOTE_IDENTITY_EC
 DDS_C, [435](#)
 DDSC_LOG_WRITE_SAMPLE_FAILED_EC
 DDS_C, [439](#)
 DDS_C, [414](#)
 DDS_C, [414](#)
 DDS_C, [414](#)
 DDS_C, [414](#)
 DDS_C, [414](#)
 DDS_C, [414](#)
 DDS_C, [415](#)
 DDS_C, [414](#)
 DDS_C, [415](#)
 DDS_C, [415](#)
 DDS_C, [415](#)
 DDS_C, [415](#)
 DDS_C, [416](#)
 DDS_C, [415](#)
 DDS_C, [416](#)
 DDS_C, [413](#)
 DDS_C, [380](#)
 DDS_C, [416](#)
 DDS_C, [416](#)

- DDS_C, [416](#)
- DDSC_LOG_TYPE_RECORD
 - DDS_C, [375](#)
- DDSC_LOG_TYPE_SUPPORT_QOS_POLICY
 - DDS_C, [394](#)
- DDSC_LOG_TYPE_TOO_LONG_EC
 - DDS_C, [415](#)
- DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_KEY_EC
 - DDS_C, [367](#)
- DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_NAME_EC
 - DDS_C, [367](#)
- DDSC_LOG_UNLOCK_EC
 - DDS_C, [449](#)
- DDSC_LOG_UNSUPPORTED_LOCATOR_EC
 - DDS_C, [421](#)
- DDSC_LOG_UPDATE_LEASE_DURATION_TIMER_EC
 - DDS_C, [446](#)
- DDSC_LOG_UPDATE_LIVELINESS_FAILED_EC
 - DDS_C, [446](#)
- DDSC_LOG_UPDATE_REMOTE_CONTENT_FILTER_EC
 - DDS_C, [451](#)
- DDSC_LOG_USERMULTICAST_SEQUENCE
 - DDS_C, [387](#)
- DDSC_LOG_USERUNICAST_SEQUENCE
 - DDS_C, [387](#)
- DDSC_LOG_UUID_OBJECT
 - DDS_C, [382](#)
- DDSC_LOG_VALIDATE_REMOTE_PARTICIPANT_TRUST_FAILURE_EC
 - DDS_C, [436](#)
- DDSC_LOG_WAITSET_OBJECT
 - DDS_C, [382](#)
- DDSC_LOG_WRITERHISTORY_COMPONENT
 - DDS_C, [390](#)
- DDSC_LOG_WS_ADD_COND_REFERENCE_EC
 - DDS_C, [370](#)
- DDSC_LOG_WS_REMOVE_COND_REFERENCE_EC
 - DDS_C, [370](#)
- DDSC_LOG_WSREF_OBJECT
 - DDS_C, [381](#)
- DDSC_LOG_XTYPES_LOANED_SAMPLE_STATE_ERROR_EC
 - DDS_C, [447](#)
- DDSC_TRUST_INVALID_PARTICIPANT_GENERIC_MESSAGE_EC
 - DDS_C, [437](#)
- DDSCondition, [669](#)
- DDSConditionSeq, [670](#)
- DDSDataReader, [670](#)
 - as_entity, [673](#)
 - get_instance_replaced_missed_status, [687](#)
 - get_listener, [675](#)
 - get_liveliness_changed_status, [684](#)
 - get_matched_publication_data, [685](#)
 - get_matched_publications, [685](#)
 - get_qos, [674](#)
 - get_requested_deadline_missed_status, [686](#)
 - get_requested_incompatible_qos_status, [687](#)
 - get_sample_lost_status, [686](#)
 - get_sample_rejected_status, [686](#)
 - get_subscriber, [673](#)
 - get_subscription_matched_status, [684](#)
 - get_topicdescription, [673](#)
 - is_data_consistent_untyped, [682](#)
 - lookup_instance_untyped, [682](#)
 - read_instance_untyped, [679](#)
 - read_next_sample_untyped, [681](#)
 - read_untyped, [675](#)
 - return_loan_untyped, [683](#)
 - set_listener, [674](#)
 - set_qos, [673](#)
 - take_instance_untyped, [680](#)
 - take_next_sample_untyped, [681](#)
 - take_untyped, [676](#)
- DDSDataReaderListener, [688](#)
 - on_before_sample_commit, [691](#)
 - on_before_sample_deserialize, [690](#)
 - on_data_available, [689](#)
 - on_instance_replaced, [690](#)
 - on_liveliness_changed, [689](#)
 - on_requested_deadline_missed, [689](#)
 - on_requested_incompatible_qos, [689](#)
 - on_sample_lost, [690](#)
 - on_sample_rejected, [690](#)
 - on_subscription_matched, [690](#)
- DDSDataWriter, [692](#)
 - assert_liveliness, [696](#)
 - dispose_untyped, [703](#)
 - dispose_w_timestamp_untyped, [704](#)
 - get_listener, [698](#)
 - get_liveliness_lost_status, [708](#)
 - get_matched_subscription_data, [709](#)
 - get_matched_subscriptions, [708](#)
 - get_offered_deadline_missed_status, [709](#)
 - get_offered_incompatible_qos_status, [710](#)
 - get_publication_matched_status, [706](#)
 - get_publisher, [697](#)
 - get_qos, [697](#)
 - get_topic, [696](#)
 - register_instance_untyped, [699](#)
 - register_instance_w_timestamp_untyped, [700](#)
 - set_listener, [698](#)
 - set_qos, [697](#)
 - unregister_instance_untyped, [701](#)
 - unregister_instance_w_timestamp_untyped, [702](#)
 - write_untyped, [694](#)
 - write_w_timestamp_untyped, [705](#)
- DDSDataWriterListener, [710](#)
 - on_liveliness_lost, [712](#)
 - on_offered_deadline_missed, [711](#)
 - on_offered_incompatible_qos, [712](#)

- on_publication_matched, 712
 - on_reliable_reader_activity_changed, 713
 - on_reliable_sample_unacknowledged, 713
 - on_sample_removed, 713
- DDSDomainEntity, 714
- DDSDomainParticipant, 715
 - add_peer, 735
 - as_entity, 718
 - assert_liveliness, 730
 - create_flowcontroller, 737
 - create_publisher, 718
 - create_subscriber, 719
 - create_topic, 721
 - delete_contained_entities, 731
 - delete_flowcontroller, 738
 - delete_publisher, 719
 - delete_subscriber, 720
 - delete_topic, 722
 - find_topic, 722
 - get_current_time, 735
 - get_default_flowcontroller_property, 736
 - get_default_publisher_qos, 724
 - get_default_subscriber_qos, 725
 - get_default_topic_qos, 726
 - get_discovered_participant_data, 734
 - get_discovered_participants, 734
 - get_listener, 734
 - get_qos, 723
 - lookup_datareader_by_name, 730
 - lookup_datawriter_by_name, 729
 - lookup_flowcontroller, 739
 - lookup_publisher_by_name, 728
 - lookup_subscriber_by_name, 729
 - lookup_topicdescription, 728
 - register_type, 731
 - set_default_flowcontroller_property, 736
 - set_default_publisher_qos, 724
 - set_default_subscriber_qos, 726
 - set_default_topic_qos, 727
 - set_listener, 733
 - set_qos, 723
 - unregister_type, 732
- DDSDomainParticipantFactory, 739
 - create_participant, 742
 - create_participant_from_config, 744
 - delete_participant, 743
 - finalize_instance, 741
 - get_default_participant_qos, 746
 - get_instance, 741
 - get_qos, 746
 - get_registry, 747
 - lookup_participant, 744
 - lookup_participant_by_name, 745
 - set_default_participant_qos, 745
 - set_qos, 747
- DDSDomainParticipantListener, 748
 - on_before_sample_commit, 752
 - on_before_sample_deserialize, 751
 - on_data_available, 750
 - on_data_on_readers, 752
 - on_inconsistent_topic, 754
 - on_instance_replaced, 751
 - on_liveliness_changed, 750
 - on_liveliness_lost, 753
 - on_offered_deadline_missed, 753
 - on_offered_incompatible_qos, 753
 - on_publication_matched, 754
 - on_reliable_reader_activity_changed, 754
 - on_requested_deadline_missed, 750
 - on_requested_incompatible_qos, 750
 - on_sample_lost, 751
 - on_sample_rejected, 750
 - on_subscription_matched, 751
- DDSEntity, 756
 - enable, 758
 - get_instance_handle, 760
 - get_status_changes, 759
 - get_statuscondition, 759
- DDSFlowController, 761
 - get_name, 762
 - get_participant, 763
 - get_property, 762
 - set_property, 761
 - trigger_flow, 762
- DDSGuardCondition, 763
 - set_trigger_value, 764
- DDSListener, 764
- DDSPropertyQosPolicyHelper, 767
 - add_property, 768
 - assert_property, 767
 - lookup_property, 768
 - remove_property, 769
- DDSPublisher, 770
 - create_datawriter, 771
 - delete_contained_entities, 776
 - delete_datawriter, 772
 - get_default_datawriter_qos, 773
 - get_listener, 777
 - get_participant, 775
 - get_qos, 775
 - lookup_datawriter, 774
 - lookup_datawriter_by_name, 774
 - set_default_datawriter_qos, 773
 - set_listener, 776
 - set_qos, 775
- DDSPublisherListener, 777
- DDSStatusCondition, 779
 - get_enabled_statuses, 780

- set_enabled_statuses, 780
- DDSSubscriber, 781
 - create_datareader, 783
 - delete_contained_entities, 785
 - delete_datareader, 783
 - get_default_datareader_qos, 784
 - get_listener, 788
 - get_participant, 786
 - get_qos, 787
 - lookup_datareader, 785
 - lookup_datareader_by_name, 786
 - set_default_datareader_qos, 784
 - set_listener, 788
 - set_qos, 787
- DDSSubscriberListener, 789
 - on_data_on_readers, 790
- DDSTopic, 790
 - as_entity, 792
 - as_topicdescription, 792
 - get_inconsistent_topic_status, 794
 - get_listener, 795
 - get_name, 795
 - get_participant, 796
 - get_qos, 793
 - get_type_name, 795
 - narrow, 792
 - set_listener, 794
 - set_qos, 793
- DDSTopicDescription, 797
 - get_name, 798
 - get_participant, 798
 - get_type_name, 797
- DDSTopicListener, 799
 - on_inconsistent_topic, 799
- DDSWaitSet, 800
 - attach_condition, 802
 - detach_condition, 803
 - get_conditions, 803
 - wait, 802
- DEADLINE, 100
- deadline
 - DDS_DataReaderQos, 521
 - DDS_DataWriterQos, 532
 - DDS_PublicationBuiltinTopicData, 613
 - DDS_SubscriptionBuiltinTopicData, 651
- default_multicast_locators
 - DDS_ParticipantBuiltinTopicData, 595
- default_unicast_locators
 - DDS_ParticipantBuiltinTopicData, 595
- delete_contained_entities
 - DDSDomainParticipant, 731
 - DDSPublisher, 776
 - DDSSubscriber, 785
- delete_data
 - FooDataWriter, 839
 - FooTypeSupport, 856
- delete_datareader
 - DDSSubscriber, 783
- delete_datawriter
 - DDSPublisher, 772
- delete_flowcontroller
 - DDSDomainParticipant, 738
- delete_instance
 - NETIO_DGRAM_InterfaceI, 864
 - ZCOPY_NotifUserInterfaceI, 905
- delete_participant
 - DDSDomainParticipantFactory, 743
- delete_publisher
 - DDSDomainParticipant, 719
- delete_route
 - ZCOPY_NotifUserInterfaceI, 908
- delete_subscriber
 - DDSDomainParticipant, 720
- delete_topic
 - DDSDomainParticipant, 722
- deny_interface
 - UDP_InterfaceFactoryProperty, 892
- depth
 - DDS_HistoryQosPolicy, 577
- deserialize_data_from_cdr_buffer
 - FooTypeSupport, 859
- DESTINATION_ORDER, 110
 - DDS_BY_RECEPTION_TIMESTAMP_DESTINATIONORDER_QOS, 111
 - DDS_BY_SOURCE_TIMESTAMP_DESTINATIONORDER_QOS, 112
 - DDS_DestinationOrderQosPolicyKind, 111
- destination_order
 - DDS_DataReaderQos, 521
 - DDS_DataWriterQos, 533
 - DDS_PublicationBuiltinTopicData, 614
 - DDS_SubscriptionBuiltinTopicData, 652
- destination_rules
 - UDP_InterfaceFactoryProperty, 896
- detach_condition
 - DDSWaitSet, 803
- disable_auto_interface_config
 - UDP_InterfaceFactoryProperty, 894
- disable_builtin_sql_filter
 - DDS_FilterQosPolicy, 567
- disable_multicast_bind
 - UDP_InterfaceFactoryProperty, 895
- disable_multicast_interface_select
 - UDP_InterfaceFactoryProperty, 895
- disable_writer_filtering
 - DDS_FilterQosPolicy, 567
- disc_dpde_log.h, 945
- disc_dpse_log.h, 948

- DPSE_DISCOVERYPLUGIN_OBJECT, [952](#)
- DPSE_ENABLEDPARTICIPANTINDEX_OBJECT, [953](#)
- DPSE_KEYLIST_OBJECT, [952](#)
- DPSE_LOG_CDR_LONG_KIND, [953](#)
- DPSE_LOG_CDR_PID_KIND, [953](#)
- DPSE_LOG_DEFAULT_MULTICAST_LOCATOR_SEQUENCE, [954](#)
- DPSE_LOG_DEFAULT_UNICAST_LOCATOR_SEQUENCE, [954](#)
- DPSE_LOG_META_MULTICAST_LOCATOR_SEQUENCE, [954](#)
- DPSE_LOG_META_UNICAST_LOCATOR_SEQUENCE, [954](#)
- DPSE_PARTICIPANTANNOUCEMENT_OBJECT, [952](#)
- DPSE_PARTICIPANTBUILTINTOPICDATA_OBJECT, [952](#)
- DPSE_PARTICIPANTENTITYNAME_OBJECT, [952](#)
- DPSE_PARTICIPANTMAMES_OBJECT, [952](#)
- DPSE_PARTICIPANTNAMEINDEX_OBJECT, [952](#)
- DPSE_PARTICIPANTPOOL_OBJECT, [953](#)
- DPSE_PUBLISHER_ENTITY, [953](#)
- DPSE_SUBSCRIBER_ENTITY, [953](#)
- DPSE_TOPIC_ENTITY, [953](#)
- discard_loan
 - FooDataWriter, [839](#)
- DISCOVERY, [99](#)
- Discovery, [35](#)
 - DDS_BuiltinTopicKey_t, [38](#)
 - DDS_LOCATOR_ADDRESS_LENGTH_MAX, [38](#)
 - DDS_LOCATOR_INVALID, [38](#)
 - DDS_LOCATOR_KIND_RESERVED, [39](#)
 - DDS_LOCATOR_KIND_SHMEM, [39](#)
 - DDS_LOCATOR_KIND_UDPv4, [39](#)
 - DDS_LOCATOR_KIND_UDPv6, [39](#)
 - DDS_PRODUCTVERSION_UNKNOWN, [38](#)
 - DDS_PROTOCOLVERSION, [40](#)
 - DDS_PROTOCOLVERSION_1_0, [39](#)
 - DDS_PROTOCOLVERSION_1_1, [39](#)
 - DDS_PROTOCOLVERSION_1_2, [39](#)
 - DDS_PROTOCOLVERSION_2_0, [40](#)
 - DDS_PROTOCOLVERSION_2_1, [40](#)
 - DDS_VENDOR_ID_LENGTH_MAX, [38](#)
- discovery
 - DDS_DiscoveryQosPolicy, [544](#)
 - DDS_DomainParticipantQos, [548](#)
- dispose
 - FooDataWriter, [834](#)
- dispose_untyped
 - DDSDDataWriter, [703](#)
- dispose_w_timestamp
 - FooDataWriter, [835](#)
- dispose_w_timestamp_untyped
 - DDSDDataWriter, [704](#)
- Documentation Guide, [25](#)
- Domain Module, [31](#)
 - DDS_DomainId_t, [32](#)
- domain_id_gain
 - DDS_RtpsWellKnownPorts_t, [636](#)
- DOMAIN_PARTICIPANT_RESOURCE_LIMITS, [120](#)
- DomainParticipant, [33](#)
 - DDS_FLOW_CONTROLLER_PROPERTY_DEFAULT, [35](#)
 - DDS_PUBLISHER_QOS_DEFAULT, [34](#)
 - DDS_SUBSCRIBER_QOS_DEFAULT, [34](#)
 - DDS_TOPIC_QOS_DEFAULT, [34](#)
- DomainParticipantFactory, [32](#)
 - DDS_PARTICIPANT_QOS_DEFAULT, [33](#)
- DomainParticipantFactoryQos, [225](#)
- DomainParticipantQos, [225](#)
- DPDE, [484](#)
 - DPDE_LOG_ADD_PEER_EC, [495](#)
 - DPDE_LOG_ADD_REMOTE_PARTICIPANT_ROUTES_FAILED_EC, [496](#)
 - DPDE_LOG_ADVANCE_SN_EC, [491](#)
 - DPDE_LOG_ALLOCATE_DPDE_EC, [490](#)
 - DPDE_LOG_ANNOUNCE_WRITE_EC, [491](#)
 - DPDE_LOG_ANNOUNCEMENT_EC, [491](#)
 - DPDE_LOG_ASSERT_REMOTE_PARTICIPANT_EC, [492](#)
 - DPDE_LOG_ASSERT_REMOTE_PUBLICATION_EC, [493](#)
 - DPDE_LOG_ASSERT_REMOTE_SUBSCRIPTION_EC, [493](#)
 - DPDE_LOG_CDR_SET_OFFSET_EC, [487](#)
 - DPDE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC, [496](#)
 - DPDE_LOG_CREATE_PARTICIPANT_DISCOVERY_EC, [489](#)
 - DPDE_LOG_CREATE_PUBLICATION_DISCOVERY_EC, [489](#)
 - DPDE_LOG_CREATE_PUBLISHER_EC, [489](#)
 - DPDE_LOG_CREATE_READER_EC, [492](#)
 - DPDE_LOG_CREATE_SUBSCRIBER_EC, [489](#)
 - DPDE_LOG_CREATE_SUBSCRIPTION_DISCOVERY_EC, [490](#)
 - DPDE_LOG_CREATE_TOPIC_EC, [492](#)
 - DPDE_LOG_CREATE_WRITER_EC, [492](#)
 - DPDE_LOG_DELETE_PARTICIPANT_TOPIC_EC, [490](#)
 - DPDE_LOG_DELETE_PUBLICATION_TOPIC_EC, [490](#)
 - DPDE_LOG_DELETE_REMOTE_PARTICIPANT_ROUTES_FAILED_EC, [496](#)
 - DPDE_LOG_DELETE_SUBSCRIPTION_TOPIC_EC, [490](#)
 - DPDE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC, [490](#)

- 491
- DPDE_LOG_DESERIALIZE_LOCATOR_EC, 495
- DPDE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC, 488
- DPDE_LOG_DESERIALIZE_UNKNOWN_PID_EC, 491
- DPDE_LOG_DESERIALIZE_VENDOR_ID_EC, 488
- DPDE_LOG_DISPOSE_PARTICIPANT_EC, 490
- DPDE_LOG_DISPOSE_PUBLICATION_EC, 493
- DPDE_LOG_DISPOSE_SUBSCRIPTION_EC, 493
- DPDE_LOG_ENABLE_REMOTE_PARTICIPANT_EC, 492
- DPDE_LOG_INVALID_PARTICIPANT_SAMPLE_EC, 493
- DPDE_LOG_INVALID_PUBLICATION_SAMPLE_EC, 494
- DPDE_LOG_INVALID_SUBSCRIPTION_SAMPLE_EC, 494
- DPDE_LOG_LOCATORS_FULL_EC, 495
- DPDE_LOG_REFRESH_REMOTE_PARTICIPANT_EC, 492
- DPDE_LOG_REGISTER_TYPE_EC, 492
- DPDE_LOG_REMOVE_PEER_EC, 495
- DPDE_LOG_REMOVE_REMOTE_PARTICIPANT_EC, 496
- DPDE_LOG_REMOVE_REMOTE_PUBLICATION_EC, 494
- DPDE_LOG_REMOVE_REMOTE_SUBSCRIPTION_EC, 495
- DPDE_LOG_RETURN_PARTICIPANT_SAMPLE_EC, 493
- DPDE_LOG_RETURN_PUBLICATION_SAMPLE_EC, 494
- DPDE_LOG_RETURN_SUBSCRIPTION_SAMPLE_EC, 494
- DPDE_LOG_SCHEDULE_FAST_ASSERTION_EC, 491
- DPDE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC, 490
- DPDE_LOG_SERIALIZE_DEFAULT_UNICAST_EC, 488
- DPDE_LOG_SERIALIZE_ENTITY_NAME_EC, 487
- DPDE_LOG_SERIALIZE_GUID_EC, 488
- DPDE_LOG_SERIALIZE_LEASE_DURATION_EC, 489
- DPDE_LOG_SERIALIZE_PRESENTATION_EC, 495
- DPDE_LOG_SERIALIZE_PRODUCT_VERSION_EC, 489
- DPDE_LOG_SERIALIZE_PROTOCOL_VERSION_EC, 488
- DPDE_LOG_SERIALIZE_TOPIC_NAME_EC, 488
- DPDE_LOG_SERIALIZE_TYPE_NAME_EC, 488
- DPDE_LOG_SERIALIZE_VENDOR_ID_EC, 489
- DPDE_LOG_TAKE_PARTICIPANT_SAMPLE_EC, 493
- DPDE_LOG_TAKE_PUBLICATION_SAMPLE_EC, 494
- DPDE_LOG_TAKE_SUBSCRIPTION_SAMPLE_EC, 494
- DPDE_LOG_UNREGISTER_TYPE_EC, 496
- DPDE_LOG_UNSUPPORTED_LOCATOR_EC, 495
- DPDE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC, 491
- DPDE Dynamic Discovery API, 136
- DPDE_DiscoveryPluginProperty_INITIALIZER, 136
- DPDE_DiscoveryPluginProperty, 804
- builtin_endpoint_reader_nack_period, 808
- builtin_writer_heartbeat_period, 808
- builtin_writer_heartbeats_per_max_samples, 808
- builtin_writer_max_heartbeat_retries, 807
- cache_serialized_samples, 806
- initial_participant_announcement_period, 805
- initial_participant_announcements, 805
- max_locators_per_discovered_participant, 806
- max_participant_locators, 806
- max_samples_per_builtin_endpoint_reader, 807
- max_samples_per_remote_builtin_endpoint_writer, 807
- participant_liveliness_assert_period, 805
- participant_liveliness_lease_duration, 805
- participant_message_reader_reliability_kind, 808
- DPDE_DiscoveryPluginProperty_INITIALIZER
- DPDE Dynamic Discovery API, 136
- DPDE_LOG_ADD_PEER_EC
- DPDE, 495
- DPDE_LOG_ADD_REMOTE_PARTICIPANT_ROUTES_FAILED_EC
- DPDE, 496
- DPDE_LOG_ADVANCE_SN_EC
- DPDE, 491
- DPDE_LOG_ALLOCATE_DPDE_EC
- DPDE, 490
- DPDE_LOG_ANNOUNCE_WRITE_EC
- DPDE, 491
- DPDE_LOG_ANNOUNCEMENT_EC
- DPDE, 491
- DPDE_LOG_ASSERT_REMOTE_PARTICIPANT_EC
- DPDE, 492
- DPDE_LOG_ASSERT_REMOTE_PUBLICATION_EC
- DPDE, 493
- DPDE_LOG_ASSERT_REMOTE_SUBSCRIPTION_EC
- DPDE, 493
- DPDE_LOG_BASE
- osapi_log.h, 978
- DPDE_LOG_CDR_SET_OFFSET_EC
- DPDE, 487
- DPDE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC
- DPDE, 496
- DPDE_LOG_CREATE_PARTICIPANT_DISCOVERY_EC

DPDE, [489](#)
 DPDE_LOG_CREATE_PUBLICATION_DISCOVERY_EC DPDE, [489](#)
 DPDE_LOG_CREATE_PUBLISHER_EC DPDE, [489](#)
 DPDE_LOG_CREATE_READER_EC DPDE, [492](#)
 DPDE_LOG_CREATE_SUBSCRIBER_EC DPDE, [489](#)
 DPDE_LOG_CREATE_SUBSCRIPTION_DISCOVERY_EC DPDE, [490](#)
 DPDE_LOG_CREATE_TOPIC_EC DPDE, [492](#)
 DPDE_LOG_CREATE_WRITER_EC DPDE, [492](#)
 DPDE_LOG_DELETE_PARTICIPANT_TOPIC_EC DPDE, [490](#)
 DPDE_LOG_DELETE_PUBLICATION_TOPIC_EC DPDE, [490](#)
 DPDE_LOG_DELETE_REMOTE_PARTICIPANT_ROUTES_DPDE, [496](#)
 DPDE_LOG_DELETE_SUBSCRIPTION_TOPIC_EC DPDE, [490](#)
 DPDE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC DPDE, [491](#)
 DPDE_LOG_DESERIALIZE_LOCATOR_EC DPDE, [495](#)
 DPDE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC DPDE, [488](#)
 DPDE_LOG_DESERIALIZE_UNKNOWN_PID_EC DPDE, [491](#)
 DPDE_LOG_DESERIALIZE_VENDOR_ID_EC DPDE, [488](#)
 DPDE_LOG_DISPOSE_PARTICIPANT_EC DPDE, [490](#)
 DPDE_LOG_DISPOSE_PUBLICATION_EC DPDE, [493](#)
 DPDE_LOG_DISPOSE_SUBSCRIPTION_EC DPDE, [493](#)
 DPDE_LOG_ENABLE_REMOTE_PARTICIPANT_EC DPDE, [492](#)
 DPDE_LOG_INVALID_PARTICIPANT_SAMPLE_EC DPDE, [493](#)
 DPDE_LOG_INVALID_PUBLICATION_SAMPLE_EC DPDE, [494](#)
 DPDE_LOG_INVALID_SUBSCRIPTION_SAMPLE_EC DPDE, [494](#)
 DPDE_LOG_LOCATORS_FULL_EC DPDE, [495](#)
 DPDE_LOG_REFRESH_REMOTE_PARTICIPANT_EC DPDE, [492](#)
 DPDE_LOG_REGISTER_TYPE_EC DPDE, [492](#)
 DPDE_LOG_REMOVE_PEER_EC DPDE, [495](#)
 DPDE_LOG_REMOVE_REMOTE_PARTICIPANT_EC DPDE, [496](#)
 DPDE_LOG_REMOVE_REMOTE_PUBLICATION_EC DPDE, [494](#)
 DPDE_LOG_REMOVE_REMOTE_SUBSCRIPTION_EC DPDE, [495](#)
 DPDE_LOG_RETURN_PARTICIPANT_SAMPLE_EC DPDE, [493](#)
 DPDE_LOG_RETURN_PUBLICATION_SAMPLE_EC DPDE, [494](#)
 DPDE_LOG_RETURN_SUBSCRIPTION_SAMPLE_EC DPDE, [494](#)
 DPDE_LOG_SCHEDULE_FAST_ASSERTION_EC DPDE, [491](#)
 DPDE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC DPDE, [490](#)
 DPDE_LOG_SERIALIZE_DEFAULT_UNICAST_EC DPDE, [488](#)
 DPDE_LOG_SERIALIZE_ENTITY_NAME_EC DPDE, [487](#)
 DPDE_LOG_SERIALIZE_GUID_EC DPDE, [488](#)
 DPDE_LOG_SERIALIZE_LEASE_DURATION_EC DPDE, [489](#)
 DPDE_LOG_SERIALIZE_PRESENTATION_EC DPDE, [495](#)
 DPDE_LOG_SERIALIZE_PRODUCT_VERSION_EC DPDE, [489](#)
 DPDE_LOG_SERIALIZE_PROTOCOL_VERSION_EC DPDE, [488](#)
 DPDE_LOG_SERIALIZE_TOPIC_NAME_EC DPDE, [488](#)
 DPDE_LOG_SERIALIZE_TYPE_NAME_EC DPDE, [488](#)
 DPDE_LOG_SERIALIZE_VENDOR_ID_EC DPDE, [489](#)
 DPDE_LOG_TAKE_PARTICIPANT_SAMPLE_EC DPDE, [493](#)
 DPDE_LOG_TAKE_PUBLICATION_SAMPLE_EC DPDE, [494](#)
 DPDE_LOG_TAKE_SUBSCRIPTION_SAMPLE_EC DPDE, [494](#)
 DPDE_LOG_UNREGISTER_TYPE_EC DPDE, [496](#)
 DPDE_LOG_UNSUPPORTED_LOCATOR_EC DPDE, [495](#)
 DPDE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC DPDE, [491](#)
 DPDEDiscoveryFactory, [809](#)
 get_interface, [809](#)
 DPSE, [471](#)
 DPSE_LOG_ADD_ANONYMOUS_ROUTE_EC, [481](#)
 DPSE_LOG_ADVANCE_SN_EC, [481](#)

- DPSE_LOG_ANNOUNCE_LOCAL_PARTICIPANT_DELETION_EC, [481](#)
- [476](#)
- DPSE_LOG_ANNOUNCE_WRITE_EC, [481](#)
- DPSE_LOG_ANNOUNCEMENT_EC, [480](#)
- DPSE_LOG_ASSERT_REMOTE_PARTICIPANT_EC, [481](#)
- DPSE_LOG_CDR_DESERIALIZE_EC, [478](#)
- DPSE_LOG_CDR_SERIALIZE_EC, [477](#)
- DPSE_LOG_CDR_SET_POSITION_EC, [477](#)
- DPSE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_INDEX_EC, [484](#)
- DPSE_LOG_CREATE_DISCOVERY_PUBLISHER_EC, [475](#)
- DPSE_LOG_CREATE_DISCOVERY_SUBSCRIBER_EC, [475](#)
- DPSE_LOG_CREATE_READER_EC, [475](#)
- DPSE_LOG_CREATE_TOPIC_EC, [475](#)
- DPSE_LOG_CREATE_WRITER_EC, [475](#)
- DPSE_LOG_DELETE_ANONYMOUS_ROUTE_EC, [482](#)
- DPSE_LOG_DELETE_PUBLISHER_EC, [476](#)
- DPSE_LOG_DELETE_READER_EC, [476](#)
- DPSE_LOG_DELETE_SUBSCRIBER_EC, [476](#)
- DPSE_LOG_DELETE_TOPIC_EC, [476](#)
- DPSE_LOG_DELETE_WRITER_EC, [476](#)
- DPSE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC, [479](#)
- DPSE_LOG_DESERIALIZE_GUID_EC, [479](#)
- DPSE_LOG_DESERIALIZE_PARTICIPANT_NAME_EC, [480](#)
- DPSE_LOG_DESERIALIZE_PRODUCT_VERSION_EC, [479](#)
- DPSE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC, [480](#)
- DPSE_LOG_DESERIALIZE_TOO_MANY_LOCATORS_EC, [480](#)
- DPSE_LOG_DESERIALIZE_UNKNOWN_PID_EC, [480](#)
- DPSE_LOG_DESERIALIZE_VENDOR_ID_EC, [480](#)
- DPSE_LOG_DISPOSE_EC, [476](#)
- DPSE_LOG_ENTITY_ENABLE_EC, [483](#)
- DPSE_LOG_GET_DDS_PROPERTIES_EC, [483](#)
- DPSE_LOG_GET_TIMER_EC, [484](#)
- DPSE_LOG_INVALID_DISCOVERY_SAMPLE_EC, [482](#)
- DPSE_LOG_INVALID_PEER_ADDRESS_EC, [475](#)
- DPSE_LOG_MAX_REMOTE_PARTICIPANT_EC, [482](#)
- DPSE_LOG_OBJECT_ALLOCATE_EC, [477](#)
- DPSE_LOG_OBJECT_DELETE_EC, [477](#)
- DPSE_LOG_OBJECT_FINALIZE_EC, [477](#)
- DPSE_LOG_OBJECT_INITIALIZE_EC, [477](#)
- DPSE_LOG_OBJECT_INVALID_EC, [477](#)
- DPSE_LOG_ON_ASSERT_REMOTE_PARTICIPANT_INDEX_EC, [481](#)
- DPSE_LOG_PARTICIPANT_TAKE_EC, [481](#)
- DPSE_LOG_REFRESH_REMOTE_PARTICIPANT_EC, [482](#)
- DPSE_LOG_REGISTER_TYPE_EC, [475](#)
- DPSE_LOG_RESET_REMOTE_PARTICIPANT_EC, [482](#)
- DPSE_LOG_RETURN_DISCOVERY_SAMPLE_EC, [482](#)
- DPSE_LOG_SCHEDULE_FAST_ASSERTION_EC, [481](#)
- DPSE_LOG_SEQUENCE_COPY_EC, [484](#)
- DPSE_LOG_SEQUENCE_FINALIZE_EC, [483](#)
- DPSE_LOG_SEQUENCE_GETREF_EC, [483](#)
- DPSE_LOG_SEQUENCE_INITIALIZE_EC, [483](#)
- DPSE_LOG_SEQUENCE_SETLENGTH_EC, [483](#)
- DPSE_LOG_SEQUENCE_SETMAX_EC, [483](#)
- DPSE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC, [478](#)
- DPSE_LOG_SERIALIZE_DEFAULT_UNICAST_EC, [478](#)
- DPSE_LOG_SERIALIZE_GUID_EC, [478](#)
- DPSE_LOG_SERIALIZE_LEASE_DURATION_EC, [479](#)
- DPSE_LOG_SERIALIZE_META_MULTICAST_EC, [479](#)
- DPSE_LOG_SERIALIZE_META_UNICAST_EC, [478](#)
- DPSE_LOG_SERIALIZE_MULTICAST_EC, [482](#)
- DPSE_LOG_SERIALIZE_PARTICIPANT_NAME_EC, [479](#)
- DPSE_LOG_SERIALIZE_PRODUCT_VERSION_EC, [479](#)
- DPSE_LOG_SERIALIZE_PROP_EC, [484](#)
- DPSE_LOG_SERIALIZE_PROTOCOL_VERSION_EC, [478](#)
- DPSE_LOG_SERIALIZE_VENDOR_ID_EC, [478](#)
- DPSE_LOG_UNSUPPORTED_LOCATOR_EC, [484](#)
- DPSE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC, [480](#)
- DPSE Static Discovery API, [135](#)
- DPSE_DiscoveryPluginProperty_INITIALIZER, [135](#)
- DPSE_DISCOVERYPLUGIN_OBJECT
 - disc_dpse_log.h, [952](#)
- DPSE_DiscoveryPluginProperty, [809](#)
 - initial_participant_announcement_period, [811](#)
 - initial_participant_announcements, [810](#)
 - max_locators_per_discovered_participant, [811](#)
 - max_participant_locators, [811](#)
 - participant_liveliness_assert_period, [810](#)
 - participant_liveliness_lease_duration, [810](#)
 - participant_message_reader_reliability_kind, [811](#)
- DPSE_DiscoveryPluginProperty_INITIALIZER
 - DPSE Static Discovery API, [135](#)
- DPSE_ENABLEDPARTICIPANTINDEX_OBJECT

disc_dpse_log.h, [953](#)
 DPSE_KEYLIST_OBJECT
 disc_dpse_log.h, [952](#)
 DPSE_LOG_ADD_ANONYMOUS_ROUTE_EC
 DPSE, [481](#)
 DPSE_LOG_ADVANCE_SN_EC
 DPSE, [481](#)
 DPSE_LOG_ANNOUNCE_LOCAL_PARTICIPANT_DELETION_EC
 DPSE, [476](#)
 DPSE_LOG_ANNOUNCE_WRITE_EC
 DPSE, [481](#)
 DPSE_LOG_ANNOUNCEMENT_EC
 DPSE, [480](#)
 DPSE_LOG_ASSERT_REMOTE_PARTICIPANT_EC
 DPSE, [481](#)
 DPSE_LOG_BASE
 osapi_log.h, [978](#)
 DPSE_LOG_CDR_DESERIALIZE_EC
 DPSE, [478](#)
 DPSE_LOG_CDR_LONG_KIND
 disc_dpse_log.h, [953](#)
 DPSE_LOG_CDR_PID_KIND
 disc_dpse_log.h, [953](#)
 DPSE_LOG_CDR_SERIALIZE_EC
 DPSE, [477](#)
 DPSE_LOG_CDR_SET_POSITION_EC
 DPSE, [477](#)
 DPSE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_DESCRIPTOR_EC
 DPSE, [484](#)
 DPSE_LOG_CREATE_DISCOVERY_PUBLISHER_EC
 DPSE, [475](#)
 DPSE_LOG_CREATE_DISCOVERY_SUBSCRIBER_EC
 DPSE, [475](#)
 DPSE_LOG_CREATE_READER_EC
 DPSE, [475](#)
 DPSE_LOG_CREATE_TOPIC_EC
 DPSE, [475](#)
 DPSE_LOG_CREATE_WRITER_EC
 DPSE, [475](#)
 DPSE_LOG_DEFAULT_MULTICAST_LOCATOR_SEQUENCE
 disc_dpse_log.h, [954](#)
 DPSE_LOG_DEFAULT_UNICAST_LOCATOR_SEQUENCE
 disc_dpse_log.h, [954](#)
 DPSE_LOG_DELETE_ANONYMOUS_ROUTE_EC
 DPSE, [482](#)
 DPSE_LOG_DELETE_PUBLISHER_EC
 DPSE, [476](#)
 DPSE_LOG_DELETE_READER_EC
 DPSE, [476](#)
 DPSE_LOG_DELETE_SUBSCRIBER_EC
 DPSE, [476](#)
 DPSE_LOG_DELETE_TOPIC_EC
 DPSE, [476](#)
 DPSE_LOG_DELETE_WRITER_EC
 DPSE, [476](#)
 DPSE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC
 DPSE, [479](#)
 DPSE_LOG_DESERIALIZE_GUID_EC
 DPSE, [479](#)
 DPSE_LOG_DESERIALIZE_PARTICIPANT_NAME_EC
 DPSE, [480](#)
 DPSE_LOG_DESERIALIZE_PRODUCT_VERSION_EC
 DPSE, [479](#)
 DPSE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC
 DPSE, [480](#)
 DPSE_LOG_DESERIALIZE_TOO_MANY_LOCATORS_EC
 DPSE, [480](#)
 DPSE_LOG_DESERIALIZE_UNKNOWN_PID_EC
 DPSE, [480](#)
 DPSE_LOG_DESERIALIZE_VENDOR_ID_EC
 DPSE, [480](#)
 DPSE_LOG_DISPOSE_EC
 DPSE, [476](#)
 DPSE_LOG_ENTITY_ENABLE_EC
 DPSE, [483](#)
 DPSE_LOG_GET_DDS_PROPERTIES_EC
 DPSE, [483](#)
 DPSE_LOG_GET_TIMER_EC
 DPSE, [484](#)
 DPSE_LOG_INVALID_DISCOVERY_SAMPLE_EC
 DPSE, [482](#)
 DPSE_LOG_INVALID_PEER_ADDRESS_EC
 DPSE, [475](#)
 DPSE_LOG_MAX_REMOTE_PARTICIPANT_EC
 DPSE, [482](#)
 DPSE_LOG_META_MULTICAST_LOCATOR_SEQUENCE
 disc_dpse_log.h, [954](#)
 DPSE_LOG_META_UNICAST_LOCATOR_SEQUENCE
 disc_dpse_log.h, [954](#)
 DPSE_LOG_OBJECT_ALLOCATE_EC
 DPSE, [477](#)
 DPSE_LOG_OBJECT_DELETE_EC
 DPSE, [477](#)
 DPSE_LOG_OBJECT_FINALIZE_EC
 DPSE, [477](#)
 DPSE_LOG_OBJECT_INITIALIZE_EC
 DPSE, [477](#)
 DPSE_LOG_OBJECT_INVALID_EC
 DPSE, [477](#)
 DPSE_LOG_ON_ASSERT_REMOTE_PARTICIPANT_EC
 DPSE, [481](#)
 DPSE_LOG_PARTICIPANT_TAKE_EC
 DPSE, [481](#)
 DPSE_LOG_REFRESH_REMOTE_PARTICIPANT_EC
 DPSE, [482](#)
 DPSE_LOG_REGISTER_TYPE_EC
 DPSE, [475](#)
 DPSE_LOG_RESET_REMOTE_PARTICIPANT_EC

- DPSE, [482](#)
- DPSE_LOG_RETURN_DISCOVERY_SAMPLE_EC
 - DPSE, [482](#)
- DPSE_LOG_SCHEDULE_FAST_ASSERTION_EC
 - DPSE, [481](#)
- DPSE_LOG_SEQUENCE_COPY_EC
 - DPSE, [484](#)
- DPSE_LOG_SEQUENCE_FINALIZE_EC
 - DPSE, [483](#)
- DPSE_LOG_SEQUENCE_GETREF_EC
 - DPSE, [483](#)
- DPSE_LOG_SEQUENCE_INITIALIZE_EC
 - DPSE, [483](#)
- DPSE_LOG_SEQUENCE_SETLENGTH_EC
 - DPSE, [483](#)
- DPSE_LOG_SEQUENCE_SETMAX_EC
 - DPSE, [483](#)
- DPSE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC
 - DPSE, [478](#)
- DPSE_LOG_SERIALIZE_DEFAULT_UNICAST_EC
 - DPSE, [478](#)
- DPSE_LOG_SERIALIZE_GUID_EC
 - DPSE, [478](#)
- DPSE_LOG_SERIALIZE_LEASE_DURATION_EC
 - DPSE, [479](#)
- DPSE_LOG_SERIALIZE_META_MULTICAST_EC
 - DPSE, [479](#)
- DPSE_LOG_SERIALIZE_META_UNICAST_EC
 - DPSE, [478](#)
- DPSE_LOG_SERIALIZE_MULTICAST_EC
 - DPSE, [482](#)
- DPSE_LOG_SERIALIZE_PARTICIPANT_NAME_EC
 - DPSE, [479](#)
- DPSE_LOG_SERIALIZE_PRODUCT_VERSION_EC
 - DPSE, [479](#)
- DPSE_LOG_SERIALIZE_PROP_EC
 - DPSE, [484](#)
- DPSE_LOG_SERIALIZE_PROTOCOL_VERSION_EC
 - DPSE, [478](#)
- DPSE_LOG_SERIALIZE_VENDOR_ID_EC
 - DPSE, [478](#)
- DPSE_LOG_UNSUPPORTED_LOCATOR_EC
 - DPSE, [484](#)
- DPSE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC
 - DPSE, [480](#)
- DPSE_PARTICIPANTANNOUCEMENT_OBJECT
 - disc_dpse_log.h, [952](#)
- DPSE_PARTICIPANTBUILTINTOPICDATA_OBJECT
 - disc_dpse_log.h, [952](#)
- DPSE_PARTICIPANTENTITYNAME_OBJECT
 - disc_dpse_log.h, [952](#)
- DPSE_PARTICIPANTMMAMES_OBJECT
 - disc_dpse_log.h, [952](#)
- DPSE_PARTICIPANTNAMEINDEX_OBJECT
 - disc_dpse_log.h, [952](#)
- disc_dpse_log.h, [952](#)
- DPSE_PARTICIPANTPOOL_OBJECT
 - disc_dpse_log.h, [953](#)
- DPSE_PUBLISHER_ENTITY
 - disc_dpse_log.h, [953](#)
- DPSE_SUBSCRIBER_ENTITY
 - disc_dpse_log.h, [953](#)
- DPSE_TOPIC_ENTITY
 - disc_dpse_log.h, [953](#)
- DPSEDiscoveryFactory, [812](#)
 - get_interface, [812](#)
- DPSEDiscoveryPlugin, [813](#)
 - RemoteParticipant_assert, [813](#)
 - RemotePublication_assert, [813](#)
 - RemoteSubscription_assert, [813](#)
- DURABILITY, [106](#)
 - DDS_DurabilityQosPolicyKind, [106](#)
 - DDS_TRANSIENT_LOCAL_DURABILITY_QOS, [106](#)
 - DDS_VOLATILE_DURABILITY_QOS, [106](#)
- durability
 - DDS_DataReaderQos, [521](#)
 - DDS_DataWriterQos, [533](#)
 - DDS_PublicationBuiltinTopicData, [614](#)
 - DDS_SubscriptionBuiltinTopicData, [651](#)
- duration
 - DDS_LatencyBudgetQosPolicy, [580](#)
- Dynamic Participant Dynamic Endpoint (DPDE), [240](#)
- Dynamic Participant Static Endpoint (DPSE), [239](#)
- enable
 - DDSEntity, [758](#)
- enable_data_consistency_check
 - DDS_DataWriterShmemRefTransferModeSettings, [538](#)
- enable_endpoint_discovery_queue
 - DDS_DiscoveryQosPolicy, [545](#)
- enable_init_read
 - PSK_PassTrackerConfiguration, [885](#)
- enable_interface_bind
 - UDP_InterfaceFactoryProperty, [895](#)
- enable_participant_discovery_by_name
 - DDS_DiscoveryQosPolicy, [545](#)
- enabled_transports
 - DDS_DiscoveryQosPolicy, [544](#)
 - DDS_TransportQosPolicy, [660](#)
 - DDS_UserTrafficQosPolicy, [663](#)
- ensure_length
 - FooSeq, [848](#)
- Entity Support, [127](#)
 - DDS_DATAREADER_ENTITY_KIND, [128](#)
 - DDS_DATAWRITER_ENTITY_KIND, [128](#)
 - DDS_EntityKind_t, [128](#)
 - DDS_PARTICIPANT_ENTITY_KIND, [128](#)

- DDS_PUBLISHER_ENTITY_KIND, [128](#)
- DDS_SUBSCRIBER_ENTITY_KIND, [128](#)
- DDS_TOPIC_ENTITY_KIND, [128](#)
- DDS_UNKNOWN_ENTITY_KIND, [128](#)
- ENTITY_FACTORY, [112](#)
- entity_factory
 - DDS_DomainParticipantFactoryQos, [547](#)
 - DDS_DomainParticipantQos, [548](#)
 - DDS_PublisherQos, [617](#)
 - DDS_SubscriberQos, [648](#)
- ENTITY_NAME, [120](#)
 - DDS_ENTITYNAME_QOS_NAME_MAX, [120](#)
 - set_name, [121](#)
- err_listener
 - DDS_SecTransform_Configuration, [646](#)
 - PSK_PassTrackerConfiguration, [886](#)
- Exception codes, [128](#)
- expression_parameters
 - DDS_ContentFilterProperty, [514](#)
 - DDS_ContentFilterQosPolicy, [516](#)
- Extended Qos Support, [112](#)
- FILTER, [97](#)
 - DDS_CONTENT_FILTER_CLASS_NAME_MAX_LENGTH, [97](#)
 - DDS_CONTENT_FILTER_EXPRESSION_MAX_LENGTH, [98](#)
 - DDS_CONTENT_FILTER_MAX_PARAMETERS, [98](#)
 - DDS_CONTENT_FILTER_NAME_MAX_LENGTH, [97](#)
 - DDS_FILTER_PLUGIN_FACTORY_DEFAULT_NAME, [98](#)
 - DDS_FILTER_RESOURCE_LIMITS_DEFAULT, [98](#)
- filter
 - DDS_DomainParticipantQos, [550](#)
- filter_class_max_count
 - DDS_FilterResourceLimits, [568](#)
- filter_class_max_length
 - DDS_FilterResourceLimits, [568](#)
- filter_class_name
 - DDS_ContentFilterProperty, [513](#)
 - DDS_ContentFilterQosPolicy, [515](#)
- filter_expression
 - DDS_ContentFilterProperty, [513](#)
 - DDS_ContentFilterQosPolicy, [516](#)
- filter_expression_max_count
 - DDS_FilterResourceLimits, [569](#)
- filter_expression_max_length
 - DDS_FilterResourceLimits, [568](#)
- filter_name
 - DDS_ContentFilterQosPolicy, [515](#)
- filter_parameter_max_count_per_expression
 - DDS_FilterResourceLimits, [569](#)
- filter_parameter_max_length
 - DDS_FilterResourceLimits, [569](#)
- finalize_data
 - FooTypeSupport, [855](#)
- finalize_func
 - DDS_PskPassTracker_InterfaceI, [608](#)
- finalize_instance
 - DDSDomainParticipantFactory, [741](#)
- find_topic
 - DDSDomainParticipant, [722](#)
- flags
 - NETIO_DGRAM_InterfaceTableEntry, [869](#)
 - UDP_InterfaceTableEntry, [898](#)
- FlatData Topic-Types, [49](#)
- Flow Controllers, [54](#)
 - DDS_DEFAULT_FLOW_CONTROLLER_NAME, [57](#)
 - DDS_EDF_FLOW_CONTROLLER_SCHED_POLICY, [56](#)
 - DDS_FIXED_RATE_FLOW_CONTROLLER_NAME, [58](#)
 - DDS_FlowControllerSchedulingPolicy, [55](#)
 - DDS_HPF_FLOW_CONTROLLER_SCHED_POLICY, [57](#)
 - DDS_ON_DEMAND_FLOW_CONTROLLER_NAME, [58](#)
 - DDS_RR_FLOW_CONTROLLER_SCHED_POLICY, [56](#)
- flow_controller_name
 - DDS_PublishModeQosPolicy, [619](#)
- FooDataReader, [814](#)
 - is_data_consistent, [825](#)
 - lookup_instance, [824](#)
 - narrow, [815](#)
 - read, [817](#)
 - read_instance, [822](#)
 - read_next_sample, [816](#)
 - return_loan, [815](#)
 - take, [818](#)
 - take_instance, [823](#)
 - take_next_sample, [817](#)
- FooDataWriter, [826](#)
 - as_datawriter, [827](#)
 - create_data, [839](#)
 - delete_data, [839](#)
 - discard_loan, [839](#)
 - dispose, [834](#)
 - dispose_w_timestamp, [835](#)
 - get_loan, [837](#)
 - narrow, [827](#)
 - register_instance, [829](#)
 - register_instance_w_timestamp, [830](#)
 - unregister_instance, [831](#)
 - unregister_instance_w_timestamp, [832](#)
 - write, [828](#)
 - write_w_timestamp, [836](#)

- FooSeq, 841
 - _contiguous_buffer, 852
 - _element_size, 853
 - _flags, 853
 - _length, 853
 - _maximum, 852
 - _token1, 853
 - _token2, 853
 - ~FooSeq, 843
 - copy_no_alloc, 845
 - ensure_length, 848
 - FooSeq, 843
 - from_array, 845
 - get_contiguous_buffer, 852
 - has_ownership, 852
 - length, 847, 848
 - loan_contiguous, 850
 - max_alloc_size, 853
 - maximum, 849
 - operator=, 844
 - operator[], 847
 - to_array, 846
 - unloan, 851
- FooTypeSupport, 854
 - copy_data, 856
 - create_data, 855
 - delete_data, 856
 - deserialize_data_from_cdr_buffer, 859
 - finalize_data, 855
 - initialize_data, 854
 - register_type, 857
 - serialize_data_to_cdr_buffer, 859
 - serialize_data_to_cdr_buffer_ex, 858
 - unregister_type, 857
- from_array
 - FooSeq, 845
- General Utilities and Compliance Configuration, 134
- generate_uuid
 - OSAPI_SystemI, 878
- get_conditions
 - DDSWaitSet, 803
- get_contiguous_buffer
 - FooSeq, 852
- get_current_time
 - DDSDomainParticipant, 735
- get_default_datareader_qos
 - DDSSubscriber, 784
- get_default_datawriter_qos
 - DDSPublisher, 773
- get_default_flowcontroller_property
 - DDSDomainParticipant, 736
- get_default_participant_qos
 - DDSDomainParticipantFactory, 746
- get_default_publisher_qos
 - DDSDomainParticipant, 724
- get_default_subscriber_qos
 - DDSDomainParticipant, 725
- get_default_topic_qos
 - DDSDomainParticipant, 726
- get_discovered_participant_data
 - DDSDomainParticipant, 734
- get_discovered_participants
 - DDSDomainParticipant, 734
- get_enabled_statuses
 - DDSStatusCondition, 780
- get_hostname
 - OSAPI_SystemI, 878
- get_inconsistent_topic_status
 - DDSTopic, 794
- get_instance
 - DDSDomainParticipantFactory, 741
- get_instance_handle
 - DDSEntity, 760
- get_instance_replaced_missed_status
 - DDSDataReader, 687
- get_interface
 - DPDEDDiscoveryFactory, 809
 - DPSEDDiscoveryFactory, 812
 - RHSMHistoryFactory, 886
 - SHMEMInterfaceFactory, 890
 - UDPIInterfaceFactory, 900
 - WHSMHistoryFactory, 901
- get_interface_list
 - NETIO_DGRAM_InterfaceI, 864
- get_listener
 - DDSDataReader, 675
 - DDSDataWriter, 698
 - DDSDomainParticipant, 734
 - DDSPublisher, 777
 - DDSSubscriber, 788
 - DDSTopic, 795
- get_liveliness_changed_status
 - DDSDataReader, 684
- get_liveliness_lost_status
 - DDSDataWriter, 708
- get_loan
 - FooDataWriter, 837
- get_matched_publication_data
 - DDSDataReader, 685
- get_matched_publications
 - DDSDataReader, 685
- get_matched_subscription_data
 - DDSDataWriter, 709
- get_matched_subscriptions
 - DDSDataWriter, 708
- get_name
 - DDSFlowController, 762

- DDSTopic, [795](#)
- DDSTopicDescription, [798](#)
- get_offered_deadline_missed_status
 - DDSDataWriter, [709](#)
- get_offered_incompatible_qos_status
 - DDSDataWriter, [710](#)
- get_participant
 - DDSFlowController, [763](#)
 - DDSPublisher, [775](#)
 - DDSSubscriber, [786](#)
 - DDSTopic, [796](#)
 - DDSTopicDescription, [798](#)
- get_property
 - DDSFlowController, [762](#)
- get_publication_matched_status
 - DDSDataWriter, [706](#)
- get_publisher
 - DDSDataWriter, [697](#)
- get_qos
 - DDSDataReader, [674](#)
 - DDSDataWriter, [697](#)
 - DDSDomainParticipant, [723](#)
 - DDSDomainParticipantFactory, [746](#)
 - DDSPublisher, [775](#)
 - DDSSubscriber, [787](#)
 - DDSTopic, [793](#)
- get_registry
 - DDSDomainParticipantFactory, [747](#)
- get_requested_deadline_missed_status
 - DDSDataReader, [686](#)
- get_requested_incompatible_qos_status
 - DDSDataReader, [687](#)
- get_route_table
 - NETIO_DGRAM_Interface, [866](#)
 - ZCOPY_NotifUserInterface, [906](#)
- get_sample_lost_status
 - DDSDataReader, [686](#)
- get_sample_rejected_status
 - DDSDataReader, [686](#)
- get_status_changes
 - DDSEntity, [759](#)
- get_statuscondition
 - DDSEntity, [759](#)
- get_subscriber
 - DDSDataReader, [673](#)
- get_subscription_matched_status
 - DDSDataReader, [684](#)
- get_ticktime
 - OSAPI_SystemI, [878](#)
- get_time
 - OSAPI_SystemI, [878](#)
- get_timer_resolution
 - OSAPI_SystemI, [877](#)
- get_topic
 - DDSDataWriter, [696](#)
- get_topicdescription
 - DDSDataReader, [673](#)
- get_type_name
 - DDSTopic, [795](#)
 - DDSTopicDescription, [797](#)
- GROUP_DATA, [125](#)
- group_data
 - DDS_PublicationBuiltinTopicData, [614](#)
 - DDS_PublisherQos, [617](#)
 - DDS_SubscriberQos, [648](#)
 - DDS_SubscriptionBuiltinTopicData, [653](#)
- GUID Support, [80](#)
 - DDS_GUID_t, [80](#)
 - DDS_GUID_UNKNOWN, [81](#)
- handle
 - DDS_WriteParams_t, [669](#)
- has_ownership
 - FooSeq, [852](#)
- heartbeat_period
 - DDS_RtpsReliableWriterProtocol_t, [633](#)
- heartbeats_per_max_samples
 - DDS_RtpsReliableWriterProtocol_t, [633](#)
- high
 - DDS_SequenceNumber_t, [647](#)
- HISTORY, [104](#)
 - DDS_HistoryQosPolicyKind, [105](#)
 - DDS_KEEP_ALL_HISTORY_QOS, [105](#)
 - DDS_KEEP_LAST_HISTORY_QOS, [105](#)
- history
 - DDS_DataReaderQos, [521](#)
 - DDS_DataWriterQos, [532](#)
- hostname
 - OSAPI_SystemProperty, [879](#)
- if_table
 - UDP_InterfaceFactoryProperty, [893](#)
- ifname
 - NETIO_DGRAM_InterfaceTableEntry, [870](#)
 - UDP_InterfaceTableEntry, [898](#)
- inactive_count
 - DDS_ReliableReaderActivityChangedStatus, [623](#)
- inactive_count_change
 - DDS_ReliableReaderActivityChangedStatus, [624](#)
- Infrastructure Module, [74](#)
 - DDS_WRITEPARAMS_DEFAULT, [76](#)
- initial_participant_announcement_period
 - DPDE_DiscoveryPluginProperty, [805](#)
 - DPSE_DiscoveryPluginProperty, [811](#)
- initial_participant_announcements
 - DPDE_DiscoveryPluginProperty, [805](#)
 - DPSE_DiscoveryPluginProperty, [810](#)
- initial_peers
 - DDS_DiscoveryQosPolicy, [544](#)

initialize_data
 FooTypeSupport, 854
 initialize_func
 DDS_PskPassTracker_InterfaceI, 608
 initialize_writer_loaned_sample
 DDS_DataWriterResourceLimitsQosPolicy, 536
 Instance States, 72
 DDS_ALIVE_INSTANCE_STATE, 74
 DDS_ANY_INSTANCE_STATE, 74
 DDS_InstanceStateKind, 73
 DDS_InstanceStateMask, 73
 DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE, 74
 DDS_NOT_ALIVE_INSTANCE_STATE, 74
 DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE, 74
 instance_handle
 DDS_ReliableSampleUnacknowledgedStatus, 625
 DDS_SampleInfo, 641
 instance_replacement
 DDS_DataReaderResourceLimitsQosPolicy, 525
 instance_state
 DDS_SampleInfo, 641
 intf_addr
 ZCOPY_NotifMechanismProperty, 903
 is_data_consistent
 FooDataReader, 825
 is_data_consistent_untyped
 DDSDDataReader, 682
 is_default_interface
 UDP_InterfaceFactoryProperty, 894
 key
 DDS_ParticipantBuiltinTopicData, 595
 DDS_PublicationBuiltinTopicData, 612
 DDS_SubscriptionBuiltinTopicData, 650
 kind
 DDS_DestinationOrderQosPolicy, 541
 DDS_DurabilityQosPolicy, 562
 DDS_HistoryQosPolicy, 577
 DDS_LivelinessQosPolicy, 585
 DDS_Locator_t, 586
 DDS_OwnershipQosPolicy, 592
 DDS_PublishModeQosPolicy, 619
 DDS_ReliabilityQosPolicy, 622
 NETIO_Address, 861
 last_instance_handle
 DDS_DataReaderInstanceReplacedStatus, 517
 DDS_OfferedDeadlineMissedStatus, 587
 DDS_ReliableReaderActivityChangedStatus, 624
 DDS_RequestedDeadlineMissedStatus, 626
 DDS_SampleRejectedStatus, 645
 last_policy_id
 DDS_OfferedIncompatibleQosStatus, 589
 DDS_RequestedIncompatibleQosStatus, 627
 last_publication_handle
 DDS_LivelinessChangedStatus, 581
 DDS_SubscriptionMatchedStatus, 654
 last_reason
 DDS_SampleRejectedStatus, 645
 last_replacement_instance
 DDS_DataReaderInstanceReplacedStatus, 517
 last_subscription_handle
 DDS_PublicationMatchedStatus, 616
 LATENCY_BUDGET, 100
 latency_budget
 DDS_PublicationBuiltinTopicData, 613
 DDS_SubscriptionBuiltinTopicData, 651
 lease_duration
 DDS_LivelinessQosPolicy, 585
 length
 FooSeq, 847, 848
 Lightweight Security API, 223
 DDS_PSK_DEFAULT_PLUGIN_NAME, 224
 DDS_PskServiceFactoryProperty_INITIALIZER, 223
 LIVELINESS, 102
 DDS_AUTOMATIC_LIVELINESS_QOS, 103
 DDS_LivelinessQosPolicyKind, 102
 DDS_MANUAL_BY_PARTICIPANT_LIVELINESS_QOS, 103
 DDS_MANUAL_BY_TOPIC_LIVELINESS_QOS, 103
 liveliness
 DDS_DataReaderQos, 521
 DDS_DataWriterQos, 532
 DDS_PublicationBuiltinTopicData, 613
 DDS_SubscriptionBuiltinTopicData, 651
 liveliness_lease_duration
 DDS_ParticipantBuiltinTopicData, 596
 loan_contiguous
 FooSeq, 850
 local_publisher_allocation, 226
 DDS_DomainParticipantResourceLimitsQosPolicy, 553
 local_reader_allocation, 227
 DDS_DomainParticipantResourceLimitsQosPolicy, 552
 local_subscriber_allocation, 227
 DDS_DomainParticipantResourceLimitsQosPolicy, 553
 local_topic_allocation, 226
 DDS_DomainParticipantResourceLimitsQosPolicy, 553
 local_type_allocation, 226
 DDS_DomainParticipantResourceLimitsQosPolicy, 553
 local_writer_allocation, 227

- DDS_DomainParticipantResourceLimitsQosPolicy, 552
- locator_kind
 - NETIO_DGRAM_InterfaceTableEntry, 869
- Log API, 218
 - OSAPI_LOG_DETAIL_FUNCTIONNAME, 219
 - OSAPI_LOG_DETAIL_LINENUMBER, 219
 - OSAPI_LOG_DETAIL_MODULENAME, 219
 - OSAPI_LOG_DETAIL_SOURCEFILE, 219
 - OSAPI_Log_get_last_error_code, 221
 - OSAPI_LOG_VERBOSITY_DEBUG, 221
 - OSAPI_LOG_VERBOSITY_ERROR, 221
 - OSAPI_LOG_VERBOSITY_SILENT, 221
 - OSAPI_LOG_VERBOSITY_WARNING, 221
 - OSAPI_Log_write_buffer_T, 220
 - OSAPI_LogProperty_INITIALIZER, 220
 - OSAPI_LogVerbosity_T, 220
- Log Codes, 242
- log_detail
 - OSAPI_LogProperty, 875
- lookup_datareader
 - DDSSubscriber, 785
- lookup_datareader_by_name
 - DDSDomainParticipant, 730
 - DDSSubscriber, 786
- lookup_datawriter
 - DDSPublisher, 774
- lookup_datawriter_by_name
 - DDSDomainParticipant, 729
 - DDSPublisher, 774
- lookup_flowcontroller
 - DDSDomainParticipant, 739
- lookup_instance
 - FooDataReader, 824
- lookup_instance_untyped
 - DDSDataReader, 682
- lookup_participant
 - DDSDomainParticipantFactory, 744
- lookup_participant_by_name
 - DDSDomainParticipantFactory, 745
- lookup_property
 - DDSPropertyQosPolicyHelper, 768
- lookup_publisher_by_name
 - DDSDomainParticipant, 728
- lookup_subscriber_by_name
 - DDSDomainParticipant, 729
- lookup_topicdescription
 - DDSDomainParticipant, 728
- lost_samples
 - DDS_DataReaderInstanceReplacedStatus, 517
- low
 - DDS_SequenceNumber_t, 647
- major
 - DDS_ProductVersion_t, 604
 - DDS_ProtocolVersion_t, 607
- majorRev
 - NDDS_Type_PluginVersion, 860
- mask
 - NETIO_Netmask, 872
- matching_reader_writer_pair_allocation
 - DDS_DomainParticipantResourceLimitsQosPolicy, 555
- matching_writer_reader_pair_allocation, 227
 - DDS_DomainParticipantResourceLimitsQosPolicy, 554
- max_alloc_size
 - FooSeq, 853
- max_blocking_time
 - DDS_ReliabilityQosPolicy, 622
- max_buffer_size, 238
 - OSAPI_LogProperty, 875
- max_components, 225
 - DDS_SystemResourceLimitsQosPolicy, 656
- max_destination_ports, 229
 - DDS_DomainParticipantResourceLimitsQosPolicy, 556
- max_fragmented_samples, 235
 - DDS_DataReaderResourceLimitsQosPolicy, 526
- max_fragmented_samples_per_remote_writer, 235
 - DDS_DataReaderResourceLimitsQosPolicy, 526
- max_heartbeat_retries
 - DDS_RtpsReliableWriterProtocol_t, 634
- max_instances, 232, 237
 - DDS_ResourceLimitsQosPolicy, 630
- max_locators_per_discovered_participant, 240, 241
 - DPDE_DiscoveryPluginProperty, 806
 - DPSE_DiscoveryPluginProperty, 811
- max_message_size, 238
 - UDP_InterfaceFactoryProperty, 893
- max_outstanding_reads, 235
 - DDS_DataReaderResourceLimitsQosPolicy, 525
- max_participant_locators, 239, 240
 - DPDE_DiscoveryPluginProperty, 806
 - DPSE_DiscoveryPluginProperty, 811
- max_participants, 225
 - DDS_SystemResourceLimitsQosPolicy, 655
- max_partition_cumulative_characters
 - DDS_DomainParticipantResourceLimitsQosPolicy, 560
- max_partition_string_allocation
 - DDS_DomainParticipantResourceLimitsQosPolicy, 561
- max_partition_string_size
 - DDS_DomainParticipantResourceLimitsQosPolicy, 560
- max_partitions

- DDS_DomainParticipantResourceLimitsQosPolicy, 560
- max_receive_buffer_size
 - UDP_InterfaceFactoryProperty, 892
- max_receive_ports, 230
 - DDS_DomainParticipantResourceLimitsQosPolicy, 555
 - ZCOPY_NotifMechanismProperty, 903
- max_remote_reader_filters
 - DDS_DataWriterResourceLimitsQosPolicy, 537
- max_remote_readers, 238
 - DDS_DataWriterResourceLimitsQosPolicy, 536
- max_remote_writers, 234
 - DDS_DataReaderResourceLimitsQosPolicy, 524
- max_remote_writers_per_instance, 232
 - DDS_DataReaderResourceLimitsQosPolicy, 524
- max_routes
 - ZCOPY_NotifMechanismProperty, 903
- max_routes_per_reader, 237
 - DDS_DataWriterResourceLimitsQosPolicy, 536
- max_routes_per_writer, 234
 - DDS_DataReaderResourceLimitsQosPolicy, 525
- max_samples, 233, 236
 - DDS_ResourceLimitsQosPolicy, 629
- max_samples_per_builtin_endpoint_reader, 241
 - DPDE_DiscoveryPluginProperty, 807
- max_samples_per_instance, 233, 237
 - DDS_ResourceLimitsQosPolicy, 630
- max_samples_per_notif
 - ZCOPY_NotifInterfaceFactoryProperty, 902
- max_samples_per_remote_builtin_endpoint_writer
 - DPDE_DiscoveryPluginProperty, 807
- max_samples_per_remote_writer, 234
 - DDS_DataReaderResourceLimitsQosPolicy, 524
- max_send_buffer_size
 - UDP_InterfaceFactoryProperty, 892
- max_send_message_size
 - UDP_InterfaceFactoryProperty, 893
- max_send_window
 - DDS_RtpsReliableWriterProtocol_t, 633
- max_timers
 - OSAPI_SystemProperty, 879
- max_tokens
 - DDS_FlowControllerTokenBucketProperty_t, 572
- max_unicast_send_sockets
 - UDP_InterfaceFactoryProperty, 897
- max_user_blocking_threads
 - OSAPI_SystemProperty, 879
- maximum
 - FooSeq, 849
- message_size_max
 - NETIO_SHMEMInterfaceFactoryProperty, 874
- metatraffic_multicast_locators
 - DDS_ParticipantBuiltinTopicData, 596
- metatraffic_transport_priority
 - DDS_DiscoveryQosPolicy, 546
- metatraffic_unicast_locators
 - DDS_ParticipantBuiltinTopicData, 596
- minor
 - DDS_ProductVersion_t, 604
 - DDS_ProtocolVersion_t, 607
- minorRev
 - NDDS_Type_PluginVersion, 860
- mode
 - OSAPI_SharedMemorySegmentHandle, 876
 - OSAPI_SharedMemorySegmentHeader, 876
- mtu
 - NETIO_DGRAM_InterfaceTableEntry, 869
- multicast_group
 - NETIO_DGRAM_InterfaceTableEntry, 870
- multicast_interface
 - UDP_InterfaceFactoryProperty, 894
- multicast_locator
 - DDS_SubscriptionBuiltinTopicData, 652
- multicast_loopback_disabled
 - UDP_InterfaceFactoryProperty, 894
- multicast_ttl
 - UDP_InterfaceFactoryProperty, 893
- nack_period
 - DDS_RtpsReliableReaderProtocol_t, 632
- name
 - DDS_DiscoveryComponent, 542
 - DDS_EntityNameQosPolicy, 565
 - DDS_FilterQosPolicy, 566
 - DDS_PartitionQosPolicy, 599
 - DDS_Property_t, 605
- nanosec
 - DDS_Duration_t, 562
 - DDS_Time_t, 657
 - OSAPI_SystemTime, 881
- narrow
 - DDSTopic, 792
 - FooDataReader, 815
 - FooDataWriter, 827
- nat
 - UDP_InterfaceFactoryProperty, 893
- NDDS_Discovery_Property, 860
- NDDS_Type_PluginVersion, 860
 - majorRev, 860
 - minorRev, 860
 - Type Support, 129
- NDDS_TYPEPLUGIN_NO_KEY
 - Type Support, 130
- NDDS_TYPEPLUGIN_USER_KEY
 - Type Support, 130
- NDDS_TypePluginKeyKind
 - Type Support, 130

NETIO, 281

NETIO_LOG_AR_ADD_INVALID_INTERFACE_EC, 290

NETIO_LOG_AR_ADDRESS_RESOLVE_FAILED_EC, 290

NETIO_LOG_AR_ADDRESS_STRING_EXCEEDED_EC, 290

NETIO_LOG_AR_ALLOC_FAILED_EC, 290

NETIO_LOG_AR_CONTEXT_UNINITIALIZED_EC, 290

NETIO_LOG_AR_INVALID_TOKEN_EC, 290

NETIO_LOG_AR_PORT_RESOLVE_FAILED_EC, 290

NETIO_LOG_BIND_CREATE_RTABLE_EC, 288

NETIO_LOG_BIND_DELETE_RTABLE_EC, 288

NETIO_LOG_BIND_EXTERNAL_FAILED_EC, 289

NETIO_LOG_BIND_NEW_EC, 288

NETIO_LOG_BIND_NEW_RTABLE_EC, 288

NETIO_LOG_BIND_SET_LENGTH_EC, 288

NETIO_LOG_BIND_SET_MAX_EC, 288

NETIO_LOG_GETHOST_BYNAME_EC, 288

NETIO_LOG_ILLEGAL_ROUTE_OPERATION_EC, 291

NETIO_LOG_INVALID_ADDRESS_INDEX_EC, 291

NETIO_LOG_LOOP_BINDX_DB_EC, 292

NETIO_LOG_LOOP_CREATE_TABLE_EC, 292

NETIO_LOG_LOOP_CURSOR_ERROR_EC, 293

NETIO_LOG_LOOP_DELETE_TABLE_EC, 292

NETIO_LOG_LOOP_FWD_EC, 292

NETIO_LOG_LOOP_INITIALIZE_FAILED_EC, 293

NETIO_LOG_LOOP_INVALID_COMPONENT_EC, 293

NETIO_LOG_LOOP_INVALID_FACTORY_EC, 293

NETIO_LOG_LOOP_INVALID_PROPERTY_EC, 293

NETIO_LOG_LOOP_SELECT_TABLE_EC, 292

NETIO_LOG_LOOP_SET_LENGTH_EC, 293

NETIO_LOG_LOOP_UNBINDX_DB_EC, 293

NETIO_LOG_PACKET_INITIALIZE_EC, 291

NETIO_LOG_PACKET_OVERFLOW_EC, 292

NETIO_LOG_PACKET_SET_HEAD_EC, 291

NETIO_LOG_PACKET_SET_TAIL_EC, 291

NETIO_LOG_ROUTE_RTABLE_ADD_EC, 289

NETIO_LOG_ROUTE_RTABLE_ALLOC_EC, 289

NETIO_LOG_ROUTE_RTABLE_CREATE_EC, 289

NETIO_LOG_ROUTE_RTABLE_DELETE_EC, 289

NETIO_LOG_ROUTE_RTABLE_UPDATE_EC, 289

NETIO_LOG_SET_NAME_EC, 291

NETIO_LOG_UNBIND_EXTERNAL_FAILED_EC, 289

NETIO_SHMEM_LOG_ADDRESS_IN_USE_EC, 305

NETIO_SHMEM_LOG_BINDX_DB_EC, 304

NETIO_SHMEM_LOG_CHECK_SYSTEM_SETTINGS_EC, 306

NETIO_SHMEM_LOG_CREATE_ATTACH_INFINITE_EC, 306

NETIO_SHMEM_LOG_CREATE_TABLE_EC, 303

NETIO_SHMEM_LOG_CURSOR_ERROR_EC, 304

NETIO_SHMEM_LOG_DELETE_TABLE_EC, 303

NETIO_SHMEM_LOG_FAILED_TAKE_SIGN_SEM_EC, 307

NETIO_SHMEM_LOG_FAILED_TO_ALLOCATE_STRUCT_EC, 306

NETIO_SHMEM_LOG_FAILED_TO_ATTACH_CONCURRENT_Q_EC, 307

NETIO_SHMEM_LOG_FAILED_TO_GET_TIMESTAMP_EC, 308

NETIO_SHMEM_LOG_FAILED_TO_INIT_CONCURRENT_Q_EC, 307

NETIO_SHMEM_LOG_FAILED_TO_INIT_MUTEX_EC, 306

NETIO_SHMEM_LOG_FAILED_TO_INIT_SEGMENT_ADDR_EC, 307

NETIO_SHMEM_LOG_FAILED_TO_INIT_SIG_SEM_EC, 307

NETIO_SHMEM_LOG_FAILED_TO_INITIALIZE_EC, 306

NETIO_SHMEM_LOG_FWD_EC, 304

NETIO_SHMEM_LOG_IGNORE_TRANSPORT_COMPATIBILITY_EC, 309

NETIO_SHMEM_LOG_INCOMPATIBLE_CONCURRENT_Q_EC, 308

NETIO_SHMEM_LOG_INCOMPATIBLE_COOKIE_EC, 305

NETIO_SHMEM_LOG_INCOMPATIBLE_HEADER_EC, 305

NETIO_SHMEM_LOG_INCOMPATIBLE_HEADER_SIZE_EC, 305

NETIO_SHMEM_LOG_INCOMPATIBLE_VERSION_EC, 305

NETIO_SHMEM_LOG_INCONSISTENT_MESSAGE_SIZE_EC, 308

NETIO_SHMEM_LOG_INITIALIZE_FAILED_EC, 304

NETIO_SHMEM_LOG_INVALID_COMPONENT_EC, 305

NETIO_SHMEM_LOG_INVALID_FACTORY_EC, 305

NETIO_SHMEM_LOG_INVALID_PROPERTY_EC, 304

NETIO_SHMEM_LOG_NONFATAL_SIG_SEM_RC_EC, 308

NETIO_SHMEM_LOG_PACKET_FWD_EC, 308

NETIO_SHMEM_LOG_PACKET_HEAD_EC, 308

NETIO_SHMEM_LOG_PACKET_INIT_EC, 307

NETIO_SHMEM_LOG_QUEUE_FULL_EC, 309

NETIO_SHMEM_LOG_RECORD_EC, 308

- NETIO_SHMEM_LOG_ROUTE_DELETE_FAILED_EC, 309
- NETIO_SHMEM_LOG_ROUTE_LOOKUP_FAILED_EC, 309
- NETIO_SHMEM_LOG_SELECT_TABLE_EC, 303
- NETIO_SHMEM_LOG_SET_LENGTH_EC, 304
- NETIO_SHMEM_LOG_SHMEM_MUTEX_LOCK_FAILED_EC, 306
- NETIO_SHMEM_LOG_SHMEM_MUTEX_UNLOCK_FAILED_EC, 306
- NETIO_SHMEM_LOG_SIG_SEM_SIGNAL_FAILED_EC, 307
- NETIO_SHMEM_LOG_UNBINDX_DB_EC, 304
- NETIO_SHMEM_LOG_UNSUPPORTED_TRANSPORT_COMPATIBLE_EC, 309
- UDP_LOG_ALLOC_EC, 298
- UDP_LOG_CURSOR_ERROR_EC, 300
- UDP_LOG_DROPGRP_EC, 299
- UDP_LOG_FINALIZE_EC, 295
- UDP_LOG_GET_BUF_ALLOC_EC, 297
- UDP_LOG_GET_BUF_SIZE_EC, 297
- UDP_LOG_GET_IFLIST_EC, 297
- UDP_LOG_GET_LENGTH_EC, 299
- UDP_LOG_GET_NAMEINFO_EC, 298
- UDP_LOG_GETHOSTNAME_EC, 294
- UDP_LOG_INCONSISTENT_MAX_MESSAGE_SIZE_EC, 299
- UDP_LOG_INITIALIZE_FAILED_EC, 300
- UDP_LOG_LOADLIB_EC, 297
- UDP_LOG_MULTICAST_ENABLE_EC, 298
- UDP_LOG_MULTICAST_NOT_ENABLED_EC, 300
- UDP_LOG_NAT_DELETE_EC, 295
- UDP_LOG_PACKET_FWD_EC, 295
- UDP_LOG_PACKET_HEAD_EC, 295
- UDP_LOG_PACKET_INIT_EC, 295
- UDP_LOG_PORT_ADD_EC, 296
- UDP_LOG_PORT_ALLOC_EC, 296
- UDP_LOG_PORT_ENTRY_EC, 296
- UDP_LOG_PORT_NOT_FOUND_EC, 300
- UDP_LOG_PORT_THREAD_EC, 296
- UDP_LOG_PROPERTY_FINALIZE_EC, 298
- UDP_LOG_RECORD_EC, 298
- UDP_LOG_RECV_ERROR_EC, 301
- UDP_LOG_SEND_ERROR_EC, 300
- UDP_LOG_SEND_PING_EC, 299
- UDP_LOG_SET_LENGTH_EC, 299
- UDP_LOG_SOCKET_ADDGRP_EC, 296
- UDP_LOG_SOCKET_BIND_EC, 296
- UDP_LOG_SOCKET_CLOSE_EC, 297
- UDP_LOG_SOCKET_CREATE_EC, 294
- UDP_LOG_SOCKET_IFFLAGS_EC, 297
- UDP_LOG_SOCKET_IFLIST_EC, 297
- UDP_LOG_SOCKET_INIT_EC, 294
- UDP_LOG_SOCKET_PRIORITY_EC, 301
- UDP_LOG_SOCKET_REUSE_PORT_EC, 295
- UDP_LOG_SOCKET_RXBUF_EC, 296
- UDP_LOG_SOCKET_SET_MCAST_EC, 294
- UDP_LOG_SOCKET_SET_MCASTIF_EC, 294
- UDP_LOG_SOCKET_SETFD_EC, 294
- UDP_LOG_SOCKET_SNDBUF_EC, 294
- UDP_LOG_SOCKET_TTL_EC, 295
- UDP_LOG_TOS_NOT_SUPPORTED_EC, 301
- UDP_LOG_TRANSPORT_NO_INTERFACES_EC, 300
- UDP_LOG_UDP_CREATE_INDEX_EC, 298
- UDP_LOG_UDP_CREATE_TABLE_EC, 298
- UDP_LOG_WINSOCK_INCOMPATIBLE_EC, 299
- UDP_PRIORITY_MAP_CREATE_EC, 299
- UDP_PRIORITY_MAP_ALLOC_EC, 303
- UDP_PRIORITY_MAP_INVALID_MAPPING_EC, 303
- UDP_PRIORITY_MAP_NEGATIVE_MAPPING_EC, 303
- UDP_TRANSFORM_LOG_ADD_EC, 302
- UDP_TRANSFORM_LOG_ALLOC_EC, 301
- UDP_TRANSFORM_LOG_CREATE_EC, 302
- UDP_TRANSFORM_LOG_DUPLICATE_EC, 302
- UDP_TRANSFORM_LOG_FACTORY_NOT_FOUND_EC, 301
- UDP_TRANSFORM_LOG_FIND_EC, 302
- UDP_TRANSFORM_LOG_INVALID_FACTORY_REFERENCE_EC, 301
- UDP_TRANSFORM_LOG_INVALID_TUDP_LOCATOR_KIND_EC, 302
- UDP_TRANSFORM_LOG_INVALID_UDP_MODE_EC, 302
- UDP_TRANSFORM_LOG_TRANSFORM_EC, 302
- NETIO Address API, 144
 - NETIO_Address_INITIALIZER, 145
 - NETIO_ADDRESS_KIND_UDPv4, 145
 - NETIO_ADDRESS_KIND_UDPv6, 145
 - NETIO_ADDRESS_RTPS_RESERVED_HIGH, 145
 - NETIO_ADDRESS_RTPS_RESERVED_LOW, 145
 - NETIO_Netmask_INITIALIZER, 145
- NETIO API, 141
- NETIO DGRAM API, 173
 - NETIO_ADDRESS_UDPv4_ANY_MULTICAST_ROUTE, 179
 - NETIO_ADDRESS_UDPv4_ANY_UNICAST_ROUTE, 179
 - NETIO_ADDRESS_UDPv6_ANY_MULTICAST_ROUTE, 179
 - NETIO_ADDRESS_UDPv6_ANY_UNICAST_ROUTE, 179
 - NETIO_DGRAM_Interface_createFunc, 177
 - NETIO_DGRAM_INTERFACE_MAX_ADDRESSES, 175
 - NETIO_DGRAM_INTERFACE_MAX_INTERFACES,

- 175
- NETIO_DGRAM_INTERFACE_SHARED_PORT_FLAG, 175
- NETIO_DGRAM_InterfaceFactory_register, 178
- NETIO_DGRAM_InterfaceI, 178
- NETIO_DGRAM_InterfaceMultiCastGroup_INITIALIZER, 175
- NETIO_DGRAM_InterfaceMultiCastGroup_Invalid, 176
- NETIO_DGRAM_InterfaceMultiCastGroup_UDPv4, 176
- NETIO_DGRAM_InterfaceMultiCastGroup_UDPv6, 176
- NETIO_DGRAM_InterfaceTableEntry_INITIALIZER, 176
- NETIO_DGRAM_InterfaceTableEntrySeq_INITIALIZER, 177
- NETIO_DGRAMUserInterface_bind_addressFunc, 177
- NETIO_DGRAMUserInterface_deleteFunc, 178
- NETIO_DGRAMUserInterface_get_interface_listFunc, 177
- NETIO_DGRAMUserInterface_resolve_addressFunc, 177
- NETIO Interface API, 142
 - NETIO_Interface_receiveFunc, 143
 - NETIO_Interface_reserve_addressFunc, 143
 - NETIO_Interface_T, 143
- NETIO Packet API, 142
 - NETIO_Packet_T, 142
- NETIO Shared Memory API, 154
- NETIO_Address, 861
 - kind, 861
 - port, 861
 - value, 861
- NETIO_Address_INITIALIZER
 - NETIO Address API, 145
- NETIO_ADDRESS_KIND_UDPv4
 - NETIO Address API, 145
- NETIO_ADDRESS_KIND_UDPv6
 - NETIO Address API, 145
- NETIO_ADDRESS_RTPS_RESERVED_HIGH
 - NETIO Address API, 145
- NETIO_ADDRESS_RTPS_RESERVED_LOW
 - NETIO Address API, 145
- NETIO_ADDRESS_UDPv4_ANY_MULTICAST_ROUTE
 - NETIO DGRAM API, 179
- NETIO_ADDRESS_UDPv4_ANY_UNICAST_ROUTE
 - NETIO DGRAM API, 179
- NETIO_ADDRESS_UDPv6_ANY_MULTICAST_ROUTE
 - NETIO DGRAM API, 179
- NETIO_ADDRESS_UDPv6_ANY_UNICAST_ROUTE
 - NETIO DGRAM API, 179
- NETIO_AddressSeq, 862
- NETIO_AddressUInt32, 862
 - value, 862
- NETIO_AddressValue, 862
 - address, 863
 - as_uint32, 863
- NETIO_DEFAULT_NOTIF_NAME
 - Notification Interface, 184
- NETIO_DGRAM_Interface_createFunc
 - NETIO DGRAM API, 177
- NETIO_DGRAM_INTERFACE_MAX_ADDRESSES
 - NETIO DGRAM API, 175
- NETIO_DGRAM_INTERFACE_MAX_INTERFACES
 - NETIO DGRAM API, 175
- NETIO_DGRAM_INTERFACE_SHARED_PORT_FLAG
 - NETIO DGRAM API, 175
- NETIO_DGRAM_InterfaceFactory_register
 - NETIO DGRAM API, 178
- NETIO_DGRAM_InterfaceI, 863
 - bind_address, 866
 - create_instance, 864
 - delete_instance, 864
 - get_interface_list, 864
 - get_route_table, 866
 - NETIO DGRAM API, 178
 - release_address, 865
 - resolve_address, 865
 - send, 865
- NETIO_DGRAM_InterfaceMultiCastGroup, 867
 - address_high, 867
 - address_low, 867
 - netmask, 867
- NETIO_DGRAM_InterfaceMultiCastGroup_INITIALIZER
 - NETIO DGRAM API, 175
- NETIO_DGRAM_InterfaceMultiCastGroup_Invalid
 - NETIO DGRAM API, 176
- NETIO_DGRAM_InterfaceMultiCastGroup_UDPv4
 - NETIO DGRAM API, 176
- NETIO_DGRAM_InterfaceMultiCastGroup_UDPv6
 - NETIO DGRAM API, 176
- NETIO_DGRAM_InterfaceRouteEntry, 868
 - address, 868
 - netmask, 868
- NETIO_DGRAM_InterfaceTableEntry, 869
 - address, 870
 - flags, 869
 - ifname, 870
 - locator_kind, 869
 - mtu, 869
 - multicast_group, 870
- NETIO_DGRAM_InterfaceTableEntry_INITIALIZER
 - NETIO DGRAM API, 176
- NETIO_DGRAM_InterfaceTableEntrySeq, 870
- NETIO_DGRAM_InterfaceTableEntrySeq_INITIALIZER
 - NETIO DGRAM API, 177

- NETIO_DGRAMUserInterface_bind_addressFunc
NETIO DGRAM API, [177](#)
- NETIO_DGRAMUserInterface_deleteFunc
NETIO DGRAM API, [178](#)
- NETIO_DGRAMUserInterface_get_interface_listFunc
NETIO DGRAM API, [177](#)
- NETIO_DGRAMUserInterface_resolve_addressFunc
NETIO DGRAM API, [177](#)
- NETIO_Interface, [871](#)
- NETIO_Interface_receiveFunc
NETIO Interface API, [143](#)
- NETIO_Interface_reserve_addressFunc
NETIO Interface API, [143](#)
- NETIO_Interface_T
NETIO Interface API, [143](#)
- NETIO_InterfaceI, [871](#)
- netio_log.h, [954](#)
 - UDP_PRIORITY_MAP_PRIORITY_IGNORED_EC,
[961](#)
 - UDP_TRANSFORM_LOG_KIND_DST_INDEX, [960](#)
 - UDP_TRANSFORM_LOG_KIND_ENTRY, [960](#)
 - UDP_TRANSFORM_LOG_KIND_FACTORY, [960](#)
 - UDP_TRANSFORM_LOG_KIND_FACTORY_INDEX,
[960](#)
 - UDP_TRANSFORM_LOG_KIND_RULE, [960](#)
 - UDP_TRANSFORM_LOG_KIND_SRC_INDEX, [960](#)
 - UDP_TRANSFORM_LOG_KIND_TRANSFORM_DESTINATION,
[960](#)
 - UDP_TRANSFORM_LOG_KIND_TRANSFORM_NETWORK_INDEX,
[961](#)
 - UDP_TRANSFORM_LOG_KIND_TRANSFORM_SOURCE,
[960](#)
- NETIO_LOG_AR_ADD_INVALID_INTERFACE_EC
NETIO, [290](#)
- NETIO_LOG_AR_ADDRESS_RESOLVE_FAILED_EC
NETIO, [290](#)
- NETIO_LOG_AR_ADDRESS_STRING_EXCEEDED_EC
NETIO, [290](#)
- NETIO_LOG_AR_ALLOC_FAILED_EC
NETIO, [290](#)
- NETIO_LOG_AR_CONTEXT_UNINITIALIZED_EC
NETIO, [290](#)
- NETIO_LOG_AR_INVALID_TOKEN_EC
NETIO, [290](#)
- NETIO_LOG_AR_PORT_RESOLVE_FAILED_EC
NETIO, [290](#)
- NETIO_LOG_BASE
osapi_log.h, [977](#)
- NETIO_LOG_BIND_CREATE_RTABLE_EC
NETIO, [288](#)
- NETIO_LOG_BIND_DELETE_RTABLE_EC
NETIO, [288](#)
- NETIO_LOG_BIND_EXTERNAL_FAILED_EC
NETIO, [289](#)
- NETIO_LOG_BIND_NEW_EC
NETIO, [288](#)
- NETIO_LOG_BIND_NEW_RTABLE_EC
NETIO, [288](#)
- NETIO_LOG_BIND_SET_LENGTH_EC
NETIO, [288](#)
- NETIO_LOG_BIND_SET_MAX_EC
NETIO, [288](#)
- NETIO_LOG_GETHOST_BYNAME_EC
NETIO, [288](#)
- NETIO_LOG_ILLEGAL_ROUTE_OPERATION_EC
NETIO, [291](#)
- NETIO_LOG_INVALID_ADDRESS_INDEX_EC
NETIO, [291](#)
- NETIO_LOG_LOOP_BINDX_DB_EC
NETIO, [292](#)
- NETIO_LOG_LOOP_CREATE_TABLE_EC
NETIO, [292](#)
- NETIO_LOG_LOOP_CURSOR_ERROR_EC
NETIO, [293](#)
- NETIO_LOG_LOOP_DELETE_TABLE_EC
NETIO, [292](#)
- NETIO_LOG_LOOP_FWD_EC
NETIO, [292](#)
- NETIO_LOG_LOOP_INITIALIZE_FAILED_EC
NETIO, [293](#)
- NETIO_LOG_LOOP_INVALID_COMPONENT_EC
NETIO, [293](#)
- NETIO_LOG_LOOP_INVALID_FACTORY_EC
NETIO, [293](#)
- NETIO_LOG_LOOP_INVALID_PROPERTY_EC
NETIO, [293](#)
- NETIO_LOG_LOOP_SELECT_TABLE_EC
NETIO, [292](#)
- NETIO_LOG_LOOP_SET_LENGTH_EC
NETIO, [293](#)
- NETIO_LOG_LOOP_UNBINDX_DB_EC
NETIO, [293](#)
- NETIO_LOG_PACKET_INITIALIZE_EC
NETIO, [291](#)
- NETIO_LOG_PACKET_OVERFLOW_EC
NETIO, [292](#)
- NETIO_LOG_PACKET_SET_HEAD_EC
NETIO, [291](#)
- NETIO_LOG_PACKET_SET_TAIL_EC
NETIO, [291](#)
- NETIO_LOG_ROUTE_RTABLE_ADD_EC
NETIO, [289](#)
- NETIO_LOG_ROUTE_RTABLE_ALLOC_EC
NETIO, [289](#)
- NETIO_LOG_ROUTE_RTABLE_CREATE_EC
NETIO, [289](#)
- NETIO_LOG_ROUTE_RTABLE_DELETE_EC
NETIO, [289](#)

- NETIO_LOG_ROUTE_RTABLE_UPDATE_EC
 - NETIO, [289](#)
- NETIO_LOG_SET_NAME_EC
 - NETIO, [291](#)
- NETIO_LOG_UNBIND_EXTERNAL_FAILED_EC
 - NETIO, [289](#)
- NETIO_Netmask, [871](#)
 - bits, [872](#)
 - mask, [872](#)
- NETIO_Netmask_INITIALIZER
 - NETIO Address API, [145](#)
- NETIO_NetmaskSeq, [872](#)
- NETIO_Packet, [872](#)
- NETIO_Packet_T
 - NETIO Packet API, [142](#)
- NETIO_SharedMemoryMutex_attach
 - Shared Memory Mutex API, [156](#)
- NETIO_SharedMemoryMutex_create
 - Shared Memory Mutex API, [156](#)
- NETIO_SharedMemoryMutex_create_or_attach
 - Shared Memory Mutex API, [157](#)
- NETIO_SharedMemoryMutex_delete
 - Shared Memory Mutex API, [160](#)
- NETIO_SharedMemoryMutex_detach
 - Shared Memory Mutex API, [160](#)
- NETIO_SharedMemoryMutex_lock
 - Shared Memory Mutex API, [159](#)
- NETIO_SharedMemoryMutex_unlock
 - Shared Memory Mutex API, [158](#)
- NETIO_SharedMemorySegment_attach
 - Shared Memory Segment API, [164](#)
- NETIO_SharedMemorySegment_create
 - Shared Memory Segment API, [162](#)
- NETIO_SharedMemorySegment_create_or_attach
 - Shared Memory Segment API, [162](#)
- NETIO_SharedMemorySegment_delete
 - Shared Memory Segment API, [163](#)
- NETIO_SharedMemorySegment_detach
 - Shared Memory Segment API, [164](#)
- NETIO_SharedMemorySegment_get_address
 - Shared Memory Segment API, [165](#)
- NETIO_SharedMemorySegment_get_max_size
 - Shared Memory Segment API, [166](#)
- NETIO_SharedMemorySegment_get_size
 - Shared Memory Segment API, [165](#)
- NETIO_SharedMemorySegmentHeader, [873](#)
- NETIO_SharedMemorySignalingSemaphore_attach
 - Shared Memory Signaling Semaphore, [168](#)
- NETIO_SharedMemorySignalingSemaphore_create
 - Shared Memory Signaling Semaphore, [167](#)
- NETIO_SharedMemorySignalingSemaphore_create_or_attach
 - Shared Memory Signaling Semaphore, [169](#)
- NETIO_SharedMemorySignalingSemaphore_delete
 - Shared Memory Signaling Semaphore, [172](#)
- NETIO_SharedMemorySignalingSemaphore_detach
 - Shared Memory Signaling Semaphore, [171](#)
- NETIO_SharedMemorySignalingSemaphore_signal
 - Shared Memory Signaling Semaphore, [170](#)
- NETIO_SharedMemorySignalingSemaphore_wait
 - Shared Memory Signaling Semaphore, [171](#)
- netio_shmem_log.h, [961](#)
- NETIO_SHMEM_LOG_ADDRESS_IN_USE_EC
 - NETIO, [305](#)
- NETIO_SHMEM_LOG_BINDX_DB_EC
 - NETIO, [304](#)
- NETIO_SHMEM_LOG_CHECK_SYSTEM_SETTINGS_EC
 - NETIO, [306](#)
- NETIO_SHMEM_LOG_CQ_INCOMPATIBLE_VERSION_EC
 - REDA, [272](#)
- NETIO_SHMEM_LOG_CQ_INCONSISTENT_STATE_EC
 - REDA, [272](#)
- NETIO_SHMEM_LOG_CQ_INT_REPRESENTATION_EC
 - REDA, [272](#)
- NETIO_SHMEM_LOG_CQ_INVALID_SIGNATURE_EC
 - REDA, [272](#)
- NETIO_SHMEM_LOG_CQ_UNLINKED_EC
 - REDA, [272](#)
- NETIO_SHMEM_LOG_CREATE_ATTACH_INFINITE_EC
 - NETIO, [306](#)
- NETIO_SHMEM_LOG_CREATE_TABLE_EC
 - NETIO, [303](#)
- NETIO_SHMEM_LOG_CURSOR_ERROR_EC
 - NETIO, [304](#)
- NETIO_SHMEM_LOG_DELETE_TABLE_EC
 - NETIO, [303](#)
- NETIO_SHMEM_LOG_FAILED_TAKE_SIGN_SEM_EC
 - NETIO, [307](#)
- NETIO_SHMEM_LOG_FAILED_TO_ALLOCATE_STRUCT_EC
 - NETIO, [306](#)
- NETIO_SHMEM_LOG_FAILED_TO_ATTACH_CONCURRENT_Q_EC
 - NETIO, [307](#)
- NETIO_SHMEM_LOG_FAILED_TO_GET_TIMESTAMP_EC
 - NETIO, [308](#)
- NETIO_SHMEM_LOG_FAILED_TO_INIT_CONCURRENT_Q_EC
 - NETIO, [307](#)
- NETIO_SHMEM_LOG_FAILED_TO_INIT_MUTEX_EC
 - NETIO, [306](#)
- NETIO_SHMEM_LOG_FAILED_TO_INIT_SEGMENT_ADDR_EC
 - NETIO, [307](#)
- NETIO_SHMEM_LOG_FAILED_TO_INIT_SIG_SEM_EC
 - NETIO, [307](#)
- NETIO_SHMEM_LOG_FAILED_TO_INITIALIZE_EC
 - NETIO, [306](#)
- NETIO_SHMEM_LOG_FWD_EC
 - NETIO, [304](#)
- NETIO_SHMEM_LOG_IGNORE_TRANSPORT_COMPATIBILITY_EC
 - NETIO, [309](#)
- NETIO_SHMEM_LOG_INCOMPATIBLE_CONCURRENT_Q_EC

- NETIO, [308](#)
- NETIO_SHMEM_LOG_INCOMPATIBLE_COOKIE_EC
 - NETIO, [305](#)
- NETIO_SHMEM_LOG_INCOMPATIBLE_HEADER_EC
 - NETIO, [305](#)
- NETIO_SHMEM_LOG_INCOMPATIBLE_HEADER_SIZE_EC
 - NETIO, [305](#)
- NETIO_SHMEM_LOG_INCOMPATIBLE_VERSION_EC
 - NETIO, [305](#)
- NETIO_SHMEM_LOG_INCONSISTENT_MESSAGE_SIZE_EC
 - NETIO, [308](#)
- NETIO_SHMEM_LOG_INITIALIZE_FAILED_EC
 - NETIO, [304](#)
- NETIO_SHMEM_LOG_INVALID_COMPONENT_EC
 - NETIO, [305](#)
- NETIO_SHMEM_LOG_INVALID_FACTORY_EC
 - NETIO, [305](#)
- NETIO_SHMEM_LOG_INVALID_PROPERTY_EC
 - NETIO, [304](#)
- NETIO_SHMEM_LOG_NONFATAL_SIG_SEM_RC_EC
 - NETIO, [308](#)
- NETIO_SHMEM_LOG_PACKET_FWD_EC
 - NETIO, [308](#)
- NETIO_SHMEM_LOG_PACKET_HEAD_EC
 - NETIO, [308](#)
- NETIO_SHMEM_LOG_PACKET_INIT_EC
 - NETIO, [307](#)
- NETIO_SHMEM_LOG_QUEUE_FULL_EC
 - NETIO, [309](#)
- NETIO_SHMEM_LOG_RECORD_EC
 - NETIO, [308](#)
- NETIO_SHMEM_LOG_ROUTE_DELETE_FAILED_EC
 - NETIO, [309](#)
- NETIO_SHMEM_LOG_ROUTE_LOOKUP_FAILED_EC
 - NETIO, [309](#)
- NETIO_SHMEM_LOG_SELECT_TABLE_EC
 - NETIO, [303](#)
- NETIO_SHMEM_LOG_SET_LENGTH_EC
 - NETIO, [304](#)
- NETIO_SHMEM_LOG_SHMEM_MUTEX_LOCK_FAILED_EC
 - NETIO, [306](#)
- NETIO_SHMEM_LOG_SHMEM_MUTEX_UNLOCK_FAILED_EC
 - NETIO, [306](#)
- NETIO_SHMEM_LOG_SIG_SEM_SIGNAL_FAILED_EC
 - NETIO, [307](#)
- NETIO_SHMEM_LOG_UNBINDX_DB_EC
 - NETIO, [304](#)
- NETIO_SHMEM_LOG_UNSUPPORTED_TRANSPORT_COMPATIBILITY_EC
 - NETIO, [309](#)
- NETIO_SHMEMInterfaceFactoryProperty, [873](#)
 - message_size_max, [874](#)
 - pro_minimum_compatibility_version, [874](#)
 - receive_buffer_size, [873](#)
 - received_message_count_max, [873](#)
 - recv_thread_property, [874](#)
- netio_zcopy_log.h, [964](#)
- NETIO_ZCOPY_LOG_BASE
 - osapi_log.h, [979](#)
 - netio_zcopy_notif_interface.h, [966](#)
 - ZCOPY_NotifInterfaceFactory_register, [967](#)
 - ZCOPY_NotifInterfaceFactory_unregister, [967](#)
- netio_zcopy_whsq_log.h, [968](#)
 - WHSQ_LOG_HISTORY_OBJECT, [968](#)
- mask
 - NETIO_DGRAM_InterfaceMultiCastGroup, [867](#)
 - NETIO_DGRAM_InterfaceRouteEntry, [868](#)
 - UDP_InterfaceTableEntry, [898](#)
- not_alive_count
 - DDS_LivelinessChangedStatus, [580](#)
- not_alive_count_change
 - DDS_LivelinessChangedStatus, [581](#)
- Notification Interface, [183](#)
 - NETIO_DEFAULT_NOTIF_NAME, [184](#)
- notify_recv_port
 - ZCOPY_NotifUserInterfaceI, [910](#)
- on_before_sample_commit
 - DDSDataReaderListener, [691](#)
 - DDSDomainParticipantListener, [752](#)
- on_before_sample_deserialize
 - DDSDataReaderListener, [690](#)
 - DDSDomainParticipantListener, [751](#)
- on_data_available
 - DDSDataReaderListener, [689](#)
 - DDSDomainParticipantListener, [750](#)
- on_data_on_readers
 - DDSDomainParticipantListener, [752](#)
 - DDSSubscriberListener, [790](#)
- on_inconsistent_topic
 - DDSDomainParticipantListener, [754](#)
 - DDSTopicListener, [799](#)
- on_instance_replaced
 - DDSDataReaderListener, [690](#)
 - DDSDomainParticipantListener, [751](#)
- on_liveliness_changed
 - DDSDataReaderListener, [689](#)
 - DDSDomainParticipantListener, [750](#)
- on_liveliness_lost
 - DDSDataWriterListener, [712](#)
 - DDSDomainParticipantListener, [753](#)
- on_offered_deadline_missed
 - DDSDataWriterListener, [711](#)
 - DDSDomainParticipantListener, [753](#)
- on_offered_incompatible_qos
 - DDSDataWriterListener, [712](#)
 - DDSDomainParticipantListener, [753](#)
- on_publication_matched
 - DDSDataWriterListener, [712](#)

- DDSDomainParticipantListener, [754](#)
- on_reliable_reader_activity_changed
 - DDSDataWriterListener, [713](#)
 - DDSDomainParticipantListener, [754](#)
- on_reliable_sample_unacknowledged
 - DDSDataWriterListener, [713](#)
- on_requested_deadline_missed
 - DDSDataReaderListener, [689](#)
 - DDSDomainParticipantListener, [750](#)
- on_requested_incompatible_qos
 - DDSDataReaderListener, [689](#)
 - DDSDomainParticipantListener, [750](#)
- on_sample_lost
 - DDSDataReaderListener, [690](#)
 - DDSDomainParticipantListener, [751](#)
- on_sample_rejected
 - DDSDataReaderListener, [690](#)
 - DDSDomainParticipantListener, [750](#)
- on_sample_removed
 - DDSDataWriterListener, [713](#)
- on_subscription_matched
 - DDSDataReaderListener, [690](#)
 - DDSDomainParticipantListener, [751](#)
- operator=
 - FooSeq, [844](#)
- operator[]
 - FooSeq, [847](#)
- options
 - OSAPI_ThreadProperty, [882](#)
- ordered_access
 - DDS_PresentationQosPolicy, [603](#)
- OS API, [184](#)
- OSAPI, [238](#), [242](#)
 - OSAPI_LOG_AUTOSAR_ACTIVATETASK_CALL_EC, [260](#)
 - OSAPI_LOG_AUTOSAR_CANCELALARM_CALL_EC, [260](#)
 - OSAPI_LOG_AUTOSAR_CLEAREVENT_CALL_EC, [259](#)
 - OSAPI_LOG_AUTOSAR_GETEVENT_CALL_EC, [260](#)
 - OSAPI_LOG_AUTOSAR_SCHEDULE_CALL_EC, [260](#)
 - OSAPI_LOG_AUTOSAR_SETEVENT_CALL_EC, [259](#)
 - OSAPI_LOG_AUTOSAR_SETRELALARM_CALL_EC, [260](#)
 - OSAPI_LOG_AUTOSAR_TERMINATETASK_CALL_EC, [260](#)
 - OSAPI_LOG_AUTOSAR_WAITEVENT_CALL_EC, [259](#)
 - OSAPI_LOG_CLOSE_EC, [257](#)
 - OSAPI_LOG_EXHAUSTED_POSIX_SEM_LIMIT_EC, [257](#)
 - OSAPI_LOG_GET_NEXT_OBJECT_ID_EC, [248](#)
 - OSAPI_LOG_HEAP_INTERNAL_ALLOCATE_EC, [253](#)
 - OSAPI_LOG_LAST_RECORDED_ERROR_EC, [254](#)
 - OSAPI_LOG_MUTEX_DELETE_EC, [253](#)
 - OSAPI_LOG_MUTEX_GIVE_EC, [253](#)
 - OSAPI_LOG_MUTEX_INIT_EC, [253](#)
 - OSAPI_LOG_MUTEX_NEW_EC, [253](#)
 - OSAPI_LOG_MUTEX_TAKE_EC, [253](#)
 - OSAPI_LOG_PTHREAD_COND_DESTROY_EC, [265](#)
 - OSAPI_LOG_PTHREAD_COND_INIT_EC, [265](#)
 - OSAPI_LOG_PTHREAD_COND_SIGNAL_EC, [265](#)
 - OSAPI_LOG_PTHREAD_COND_WAIT_EC, [265](#)
 - OSAPI_LOG_PTHREAD_CONDATTR_DESTROY_EC, [264](#)
 - OSAPI_LOG_PTHREAD_CONDATTR_INIT_EC, [264](#)
 - OSAPI_LOG_PTHREAD_CONDATTR_SETPSHARED_EC, [264](#)
 - OSAPI_LOG_PTHREAD_MUTEX_CONSISTENT_EC, [264](#)
 - OSAPI_LOG_PTHREAD_MUTEX_DESTROY_EC, [264](#)
 - OSAPI_LOG_PTHREAD_MUTEX_INIT_EC, [263](#)
 - OSAPI_LOG_PTHREAD_MUTEX_LOCK_EC, [264](#)
 - OSAPI_LOG_PTHREAD_MUTEX_UNLOCK_EC, [264](#)
 - OSAPI_LOG_PTHREAD_MUTEXATTR_DESTROY_EC, [263](#)
 - OSAPI_LOG_PTHREAD_MUTEXATTR_INIT_EC, [263](#)
 - OSAPI_LOG_PTHREAD_MUTEXATTR_SETPRIOCEILING_EC, [265](#)
 - OSAPI_LOG_PTHREAD_MUTEXATTR_SETPROTOCOL_EC, [265](#)
 - OSAPI_LOG_PTHREAD_MUTEXATTR_SETPSHARED_EC, [263](#)
 - OSAPI_LOG_PTHREAD_MUTEXATTR_SETROBUST_EC, [263](#)
 - OSAPI_LOG_PTHREAD_MUTEXATTR_SETTYPE_EC, [265](#)
 - OSAPI_LOG_SD_OPEN_EC, [254](#)
 - OSAPI_LOG_SD_UNMAP_EC, [254](#)
 - OSAPI_LOG_SEGMENT_ALREADY_EXISTS_EC, [260](#)
 - OSAPI_LOG_SEGMENT_DETACH_FAILED_EC, [255](#)
 - OSAPI_LOG_SEGMENT_DOESNT_EXIST_EC, [261](#)
 - OSAPI_LOG_SEGMENT_KEY_ALREADY_EXISTS_EC, [255](#)
 - OSAPI_LOG_SEGMENT_KEY_DOESNT_EXIST_EC, [255](#)

- OSAPI_LOG_SEM_GIVE_EC, [255](#)
- OSAPI_LOG_SEM_INFO_GET_EC, [254](#)
- OSAPI_LOG_SEM_OPEN_EC, [254](#)
- OSAPI_LOG_SEM_TAKE_EC, [255](#)
- OSAPI_LOG_SEMAPHORE_DELETE_EC, [252](#)
- OSAPI_LOG_SEMAPHORE_GIVE_EC, [252](#)
- OSAPI_LOG_SEMAPHORE_NEW_EC, [252](#)
- OSAPI_LOG_SEMAPHORE_NEW_INIT_EC, [252](#)
- OSAPI_LOG_SEMAPHORE_TAKE_EC, [252](#)
- OSAPI_LOG_SET_THREAD_NAME_EC, [254](#)
- OSAPI_LOG_SHMEM_ALREADY_ATTACHED_EC, [262](#)
- OSAPI_LOG_SHMEM_ATTACH_FAILED_EC, [255](#)
- OSAPI_LOG_SHMEM_CHECK_UNLINKED_EC, [262](#)
- OSAPI_LOG_SHMEM_FSTAT_EC, [259](#)
- OSAPI_LOG_SHMEM_FTRUNCATE_EC, [257](#)
- OSAPI_LOG_SHMEM_GIVE_FAILED_EC, [256](#)
- OSAPI_LOG_SHMEM_INIT_DESCRIPTOR_EC, [256](#)
- OSAPI_LOG_SHMEM_INVALID_SEGMENT_MODE_EC, [261](#)
- OSAPI_LOG_SHMEM_INVALID_SEGMENT_NAME_EC, [261](#)
- OSAPI_LOG_SHMEM_MMAP_EC, [257](#)
- OSAPI_LOG_SHMEM_MUNMAP_EC, [257](#)
- OSAPI_LOG_SHMEM_MUTEX_INIT_TIMEOUT_EC, [259](#)
- OSAPI_LOG_SHMEM_NOT_ATTACHED_EC, [262](#)
- OSAPI_LOG_SHMEM_POSIX_GIVE_EC, [258](#)
- OSAPI_LOG_SHMEM_SEGMENT_FILE_MAP_FAILED_EC, [256](#)
- OSAPI_LOG_SHMEM_SEGMENT_NAME_ALREADY_EXISTS_EC, [262](#)
- OSAPI_LOG_SHMEM_SEGMENT_NAME_DOESNT_EXIST_EC, [263](#)
- OSAPI_LOG_SHMEM_SEGMENT_NOT_LOCKABLE_EC, [261](#)
- OSAPI_LOG_SHMEM_SEGMENT_NOT_ROBUST_EC, [261](#)
- OSAPI_LOG_SHMEM_SEM_DELETE_FAILED_EC, [256](#)
- OSAPI_LOG_SHMEM_SEM_MUTEX_CREATED_EC, [256](#)
- OSAPI_LOG_SHMEM_SEM_UNLINK_EC, [258](#)
- OSAPI_LOG_SHMEM_SEM_WAIT_EC, [258](#)
- OSAPI_LOG_SHMEM_SET_DESCRIPTOR_EC, [256](#)
- OSAPI_LOG_SHMEM_SHM_OPEN_EC, [257](#)
- OSAPI_LOG_SHMEM_SIZE_EXCEEDED_EC, [258](#)
- OSAPI_LOG_SHMEM_TAKE_FAILED_EC, [256](#)
- OSAPI_LOG_SHMEM_UNLINK_EC, [257](#)
- OSAPI_LOG_SYSTEM_GET_TIME_EC, [253](#)
- OSAPI_LOG_SYSTEM_SET_PROPERTY_EC, [248](#)
- OSAPI_LOG_SYSTEM_TIMER_START_EC, [248](#)
- OSAPI_LOG_SYSTEM_TIMER_STOP_EC, [248](#)
- OSAPI_LOG_THREAD_CREATE_EC, [248](#)
- OSAPI_LOG_THREAD_DESTROY_EC, [249](#)
- OSAPI_LOG_THREAD_DESTROY_NO_START_EC, [249](#)
- OSAPI_LOG_THREAD_DESTROY_NO_WAKEUP_EC, [249](#)
- OSAPI_LOG_THREAD_EXEC_CREATE_EC, [249](#)
- OSAPI_LOG_THREAD_EXEC_START_EC, [249](#)
- OSAPI_LOG_THREAD_GET_MAX_PRIORITY_EC, [259](#)
- OSAPI_LOG_THREAD_GET_POLICY_EC, [250](#)
- OSAPI_LOG_THREAD_INIT_EC, [250](#)
- OSAPI_LOG_THREAD_JOIN_EC, [259](#)
- OSAPI_LOG_THREAD_NEW_EC, [248](#)
- OSAPI_LOG_THREAD_POLICY_DIFFER_EC, [250](#)
- OSAPI_LOG_THREAD_PRIORITY_MAP_EC, [250](#)
- OSAPI_LOG_THREAD_SCHEDPARAM_EC, [250](#)
- OSAPI_LOG_THREAD_SEM_EC, [249](#)
- OSAPI_LOG_THREAD_SEMPOOL_EXCEEDED_EC, [258](#)
- OSAPI_LOG_THREAD_SEMPOOL_INVALID_SEM_EC, [258](#)
- OSAPI_LOG_THREAD_START_EC, [249](#)
- OSAPI_LOG_TIMER_DELETE_EC, [250](#)
- OSAPI_LOG_TIMER_DELETE_MUTEX_EC, [252](#)
- OSAPI_LOG_TIMER_DELETE_STOP_TIMER_EC, [251](#)
- OSAPI_LOG_TIMER_GET_USER_DATA_EPOCH_EC, [251](#)
- OSAPI_LOG_TIMER_MUTEX_EC, [252](#)
- OSAPI_LOG_TIMER_NEW_EC, [251](#)
- OSAPI_LOG_TIMER_NEW_ENTRY_EC, [251](#)
- OSAPI_LOG_TIMER_NEW_MUTEX_EC, [251](#)
- OSAPI_LOG_TIMER_NEW_START_TIMER_EC, [251](#)
- OSAPI_LOG_TIMER_NEW_WHEEL_EC, [251](#)
- OSAPI_LOG_TIMER_TICK_MUTEX_EC, [250](#)
- OSAPI_LOG_UNKNOWN_SEMAPHORE_TYPE_EC, [255](#)
- OSAPI_LOG_WIN_WAIT_FAILED_EC, [266](#)
- OSAPI_SHMEM_SEGMENT_NOT_INITIALIZED_EC, [262](#)
- OSAPI Heap API, [185](#)
 - OSAPI_ALIGNMENT_DEFAULT, [186](#)
 - OSAPI_get_alignment_of, [186](#)
 - OSAPI_Heap_align_size_up, [186](#)
 - OSAPI_Heap_allocate, [187](#)
 - OSAPI_Heap_allocate_buffer, [189](#)
 - OSAPI_Heap_allocate_struct, [188](#)
 - OSAPI_Heap_free, [188](#)
 - OSAPI_Heap_free_buffer, [190](#)
 - OSAPI_Heap_free_struct, [189](#)

- OSAPI_Heap_is_address_aligned, [186](#)
- OSAPI_Heap_realloc, [188](#)
- OSAPI Memory API, [195](#)
- OSAPI Mutex API, [190](#)
 - OSAPI_Mutex_delete, [191](#)
 - OSAPI_Mutex_give, [192](#)
 - OSAPI_Mutex_new, [191](#)
 - OSAPI_Mutex_T, [191](#)
 - OSAPI_Mutex_take, [191](#)
- OSAPI Process API, [192](#)
 - OSAPI_Process_getpid, [193](#)
 - OSAPI_Process_is_alive, [193](#)
- OSAPI Semaphore API, [193](#)
 - OSAPI_Semaphore_delete, [195](#)
 - OSAPI_SEMAPHORE_RESULT_ERROR, [194](#)
 - OSAPI_SEMAPHORE_RESULT_OK, [194](#)
 - OSAPI_SEMAPHORE_RESULT_TIMEOUT, [194](#)
 - OSAPI_Semaphore_T, [195](#)
 - OSAPI_SEMAPHORE_TIMEOUT_INFINITE, [194](#)
- OSAPI String API, [195](#)
- OSAPI System API, [195](#)
 - OSAPI_System_generate_uuid_T, [199](#)
 - OSAPI_System_get_hostname_T, [199](#)
 - OSAPI_System_get_ticktime_T, [200](#)
 - OSAPI_System_get_time_T, [198](#)
 - OSAPI_System_get_timer_resolution_T, [198](#)
 - OSAPI_System_initialize_T, [199](#)
 - OSAPI_SYSTEM_MAX_HOSTNAME, [197](#)
 - OSAPI_System_start_timer_T, [198](#)
 - OSAPI_SystemI, [200](#)
 - OSAPI_SystemProperty_INITIALIZER, [197](#)
 - OSAPI_TaskProperty_INITIALIZER, [197](#)
- OSAPI Thread API, [200](#)
 - OSAPI_THREAD_DEFAULT_OPTIONS, [202](#)
 - OSAPI_THREAD_FLOATING_POINT, [202](#)
 - OSAPI_THREAD_PRIORITY_ABOVE_NORMAL, [202](#)
 - OSAPI_THREAD_PRIORITY_BELOW_NORMAL, [201](#)
 - OSAPI_THREAD_PRIORITY_HIGH, [202](#)
 - OSAPI_THREAD_PRIORITY_INHERIT, [202](#)
 - OSAPI_THREAD_PRIORITY_LOW, [201](#)
 - OSAPI_THREAD_PRIORITY_NORMAL, [202](#)
 - OSAPI_THREAD_PROPERTY_DEFAULT, [203](#)
 - OSAPI_THREAD_REALTIME_PRIORITY, [203](#)
 - OSAPI_THREAD_STDIO, [203](#)
 - OSAPI_THREAD_USE_OSDEFAULT_STACKSIZE, [202](#)
 - OSAPI_ThreadOptions, [203](#)
 - OSAPI_ThreadProperty_INITIALIZER, [203](#)
- OSAPI time API, [204](#)
 - OSAPI_SystemTime, [204](#)
- OSAPI Timer API, [204](#)
- OSAPI Types, [205](#)
 - RTI_BOOL, [207](#)
 - RTI_FALSE, [207](#)
 - RTI_INT16, [206](#)
 - RTI_INT32, [206](#)
 - RTI_INT64, [206](#)
 - RTI_INT8, [205](#)
 - RTI_SIZE_T, [207](#)
 - RTI_TRUE, [207](#)
 - RTI_UINT16, [206](#)
 - RTI_UINT32, [206](#)
 - RTI_UINT64, [207](#)
 - RTI_UINT8, [205](#)
- OSAPI_ALIGNMENT_DEFAULT
 - OSAPI Heap API, [186](#)
- OSAPI_get_alignment_of
 - OSAPI Heap API, [186](#)
- OSAPI_Heap_align_size_up
 - OSAPI Heap API, [186](#)
- OSAPI_Heap_allocate
 - OSAPI Heap API, [187](#)
- OSAPI_Heap_allocate_buffer
 - OSAPI Heap API, [189](#)
- OSAPI_Heap_allocate_struct
 - OSAPI Heap API, [188](#)
- OSAPI_Heap_free
 - OSAPI Heap API, [188](#)
- OSAPI_Heap_free_buffer
 - OSAPI Heap API, [190](#)
- OSAPI_Heap_free_struct
 - OSAPI Heap API, [189](#)
- OSAPI_Heap_is_address_aligned
 - OSAPI Heap API, [186](#)
- OSAPI_Heap_realloc
 - OSAPI Heap API, [188](#)
- osapi_log.h, [969](#)
 - APPGEN_LOG_BASE, [979](#)
 - CDR_LOG_BASE, [977](#)
 - DB_LOG_BASE, [976](#)
 - DDS_FILTER_LOG_BASE, [979](#)
 - DDSC_LOG_BASE, [978](#)
 - DPDE_LOG_BASE, [978](#)
 - DPSE_LOG_BASE, [978](#)
 - NETIO_LOG_BASE, [977](#)
 - NETIO_ZCOPY_LOG_BASE, [979](#)
 - OSAPI_LOG_BASE, [976](#)
 - OSAPI_Log_clear, [981](#)
 - OSAPI_Log_get_display_handler, [980](#)
 - OSAPI_Log_get_log_handler, [980](#)
 - OSAPI_Log_get_property, [982](#)
 - OSAPI_Log_get_verbosity, [982](#)
 - OSAPI_LOG_INVALID_TIMER_RES_EC, [979](#)
 - OSAPI_LOG_MATH_OFV_EC, [979](#)
 - OSAPI_Log_set_display_handler, [980](#)
 - OSAPI_Log_set_log_handler, [979](#)

- OSAPI_Log_set_property, [982](#)
- OSAPI_Log_set_verbosity, [981](#)
- OSAPI_Trace_set_trace_mask, [981](#)
- REDA_LOG_BASE, [976](#)
- RHSM_LOG_BASE, [978](#)
- RT_LOG_BASE, [977](#)
- RTPS_LOG_BASE, [978](#)
- SDM_LOG_BASE, [977](#)
- SEC_CORE_LOG_BASE, [978](#)
- SHMEM_LOG_BASE, [977](#)
- UDP_LOG_BASE, [977](#)
- WHSM_LOG_BASE, [978](#)
- XCDR_LOG_BASE, [977](#)
- OSAPI_LOG_AUTOSAR_ACTIVATETASK_CALL_EC
 - OSAPI, [260](#)
- OSAPI_LOG_AUTOSAR_CANCELALARM_CALL_EC
 - OSAPI, [260](#)
- OSAPI_LOG_AUTOSAR_CLEAREVENT_CALL_EC
 - OSAPI, [259](#)
- OSAPI_LOG_AUTOSAR_GETEVENT_CALL_EC
 - OSAPI, [260](#)
- OSAPI_LOG_AUTOSAR_SCHEDULE_CALL_EC
 - OSAPI, [260](#)
- OSAPI_LOG_AUTOSAR_SETEVENT_CALL_EC
 - OSAPI, [259](#)
- OSAPI_LOG_AUTOSAR_SETRELALARM_CALL_EC
 - OSAPI, [260](#)
- OSAPI_LOG_AUTOSAR_TERMINATETASK_CALL_EC
 - OSAPI, [260](#)
- OSAPI_LOG_AUTOSAR_WAITEVENT_CALL_EC
 - OSAPI, [259](#)
- OSAPI_LOG_BASE
 - osapi_log.h, [976](#)
- OSAPI_Log_clear
 - osapi_log.h, [981](#)
- OSAPI_LOG_CLOSE_EC
 - OSAPI, [257](#)
- OSAPI_LOG_DETAIL_FUNCTIONNAME
 - Log API, [219](#)
- OSAPI_LOG_DETAIL_LINENUMBER
 - Log API, [219](#)
- OSAPI_LOG_DETAIL_MODULENAME
 - Log API, [219](#)
- OSAPI_LOG_DETAIL_SOURCEFILE
 - Log API, [219](#)
- OSAPI_LOG_EXHAUSTED_POSIX_SEM_LIMIT_EC
 - OSAPI, [257](#)
- OSAPI_Log_get_display_handler
 - osapi_log.h, [980](#)
- OSAPI_Log_get_last_error_code
 - Log API, [221](#)
- OSAPI_Log_get_log_handler
 - osapi_log.h, [980](#)
- OSAPI_LOG_GET_NEXT_OBJECT_ID_EC
 - OSAPI, [248](#)
- OSAPI_Log_get_property
 - osapi_log.h, [982](#)
- OSAPI_Log_get_verbosity
 - osapi_log.h, [982](#)
- OSAPI_LOG_HEAP_INTERNAL_ALLOCATE_EC
 - OSAPI, [253](#)
- OSAPI_LOG_INVALID_TIMER_RES_EC
 - osapi_log.h, [979](#)
- OSAPI_LOG_LAST_RECORDED_ERROR_EC
 - OSAPI, [254](#)
- OSAPI_LOG_MATH_OFV_EC
 - osapi_log.h, [979](#)
- OSAPI_LOG_MUTEX_DELETE_EC
 - OSAPI, [253](#)
- OSAPI_LOG_MUTEX_GIVE_EC
 - OSAPI, [253](#)
- OSAPI_LOG_MUTEX_INIT_EC
 - OSAPI, [253](#)
- OSAPI_LOG_MUTEX_NEW_EC
 - OSAPI, [253](#)
- OSAPI_LOG_MUTEX_TAKE_EC
 - OSAPI, [253](#)
- OSAPI_LOG_PTHREAD_COND_DESTROY_EC
 - OSAPI, [265](#)
- OSAPI_LOG_PTHREAD_COND_INIT_EC
 - OSAPI, [265](#)
- OSAPI_LOG_PTHREAD_COND_SIGNAL_EC
 - OSAPI, [265](#)
- OSAPI_LOG_PTHREAD_COND_WAIT_EC
 - OSAPI, [265](#)
- OSAPI_LOG_PTHREAD_CONDATTR_DESTROY_EC
 - OSAPI, [264](#)
- OSAPI_LOG_PTHREAD_CONDATTR_INIT_EC
 - OSAPI, [264](#)
- OSAPI_LOG_PTHREAD_CONDATTR_SETPSHARED_EC
 - OSAPI, [264](#)
- OSAPI_LOG_PTHREAD_MUTEX_CONSISTENT_EC
 - OSAPI, [264](#)
- OSAPI_LOG_PTHREAD_MUTEX_DESTROY_EC
 - OSAPI, [264](#)
- OSAPI_LOG_PTHREAD_MUTEX_INIT_EC
 - OSAPI, [263](#)
- OSAPI_LOG_PTHREAD_MUTEX_LOCK_EC
 - OSAPI, [264](#)
- OSAPI_LOG_PTHREAD_MUTEX_UNLOCK_EC
 - OSAPI, [264](#)
- OSAPI_LOG_PTHREAD_MUTEXATTR_DESTROY_EC
 - OSAPI, [263](#)
- OSAPI_LOG_PTHREAD_MUTEXATTR_INIT_EC
 - OSAPI, [263](#)
- OSAPI_LOG_PTHREAD_MUTEXATTR_SETPRIOCEILING_EC
 - OSAPI, [265](#)
- OSAPI_LOG_PTHREAD_MUTEXATTR_SETPROTOCOL_EC

OSAPI, [265](#)
 OSAPI_LOG_PTHREAD_MUTEXATTR_SETPSHARED_EC [OSAPI_LOG_SHMEM_FSTAT_EC](#)
 OSAPI, [263](#)
 OSAPI_LOG_PTHREAD_MUTEXATTR_SETROBUST_EC [OSAPI_LOG_SHMEM_FTRUNCATE_EC](#)
 OSAPI, [263](#)
 OSAPI_LOG_PTHREAD_MUTEXATTR_SETTYPE_EC [OSAPI_LOG_SHMEM_GIVE_FAILED_EC](#)
 OSAPI, [265](#)
 OSAPI_LOG_SD_OPEN_EC [OSAPI_LOG_SHMEM_INIT_DESCRIPTOR_EC](#)
 OSAPI, [254](#)
 OSAPI_LOG_SD_UNMAP_EC [OSAPI_LOG_SHMEM_INVALID_SEGMENT_MODE_EC](#)
 OSAPI, [254](#)
 OSAPI_LOG_SEGMENT_ALREADY_EXISTS_EC [OSAPI_LOG_SHMEM_INVALID_SEGMENT_NAME_EC](#)
 OSAPI, [260](#)
 OSAPI_LOG_SEGMENT_DETACH_FAILED_EC [OSAPI_LOG_SHMEM_MMAP_EC](#)
 OSAPI, [255](#)
 OSAPI_LOG_SEGMENT_DOESNT_EXIST_EC [OSAPI_LOG_SHMEM_MUNMAP_EC](#)
 OSAPI, [261](#)
 OSAPI_LOG_SEGMENT_KEY_ALREADY_EXISTS_EC [OSAPI_LOG_SHMEM_MUTEX_INIT_TIMEOUT_EC](#)
 OSAPI, [255](#)
 OSAPI_LOG_SEGMENT_KEY_DOESNT_EXIST_EC [OSAPI_LOG_SHMEM_NOT_ATTACHED_EC](#)
 OSAPI, [255](#)
 OSAPI_LOG_SEM_GIVE_EC [OSAPI_LOG_SHMEM_POSIX_GIVE_EC](#)
 OSAPI, [255](#)
 OSAPI_LOG_SEM_INFO_GET_EC [OSAPI_LOG_SHMEM_SEGMENT_FILE_MAP_FAILED_EC](#)
 OSAPI, [254](#)
 OSAPI_LOG_SEM_OPEN_EC [OSAPI_LOG_SHMEM_SEGMENT_NAME_ALREADY_EXISTS_EC](#)
 OSAPI, [254](#)
 OSAPI_LOG_SEM_TAKE_EC [OSAPI_LOG_SHMEM_SEGMENT_NAME_DOESNT_EXIST_EC](#)
 OSAPI, [255](#)
 OSAPI_LOG_SEMAPHORE_DELETE_EC [OSAPI_LOG_SHMEM_SEGMENT_NOT_LOCKABLE_EC](#)
 OSAPI, [252](#)
 OSAPI_LOG_SEMAPHORE_GIVE_EC [OSAPI_LOG_SHMEM_SEGMENT_NOT_ROBUST_EC](#)
 OSAPI, [252](#)
 OSAPI_LOG_SEMAPHORE_NEW_EC [OSAPI_LOG_SHMEM_SEM_DELETE_FAILED_EC](#)
 OSAPI, [252](#)
 OSAPI_LOG_SEMAPHORE_NEW_INIT_EC [OSAPI_LOG_SHMEM_SEM_MUTEX_CREATED_EC](#)
 OSAPI, [252](#)
 OSAPI_LOG_SEMAPHORE_TAKE_EC [OSAPI_LOG_SHMEM_SEM_UNLINK_EC](#)
 OSAPI, [252](#)
 OSAPI_Log_set_display_handler [OSAPI_LOG_SHMEM_SEM_WAIT_EC](#)
 osapi_log.h, [980](#)
 OSAPI_Log_set_log_handler [OSAPI_LOG_SHMEM_SET_DESCRIPTOR_EC](#)
 osapi_log.h, [979](#)
 OSAPI_Log_set_property [OSAPI_LOG_SHMEM_SHM_OPEN_EC](#)
 osapi_log.h, [982](#)
 OSAPI_LOG_SET_THREAD_NAME_EC [OSAPI_LOG_SHMEM_SIZE_EXCEEDED_EC](#)
 OSAPI, [254](#)
 OSAPI_Log_set_verbosity [OSAPI_LOG_SHMEM_TAKE_FAILED_EC](#)
 osapi_log.h, [981](#)
 OSAPI_LOG_SHMEM_ALREADY_ATTACHED_EC [OSAPI_LOG_SHMEM_UNLINK_EC](#)
 OSAPI, [262](#)
 OSAPI_LOG_SHMEM_ATTACH_FAILED_EC [OSAPI_LOG_SYSTEM_GET_TIME_EC](#)
 OSAPI, [255](#)
 OSAPI_LOG_SHMEM_CHECK_UNLINKED_EC [OSAPI_LOG_SYSTEM_SET_PROPERTY_EC](#)

OSAPI, [248](#)
 OSAPI_LOG_SYSTEM_TIMER_START_EC
 OSAPI, [248](#)
 OSAPI_LOG_SYSTEM_TIMER_STOP_EC
 OSAPI, [248](#)
 OSAPI_LOG_THREAD_CREATE_EC
 OSAPI, [248](#)
 OSAPI_LOG_THREAD_DESTROY_EC
 OSAPI, [249](#)
 OSAPI_LOG_THREAD_DESTROY_NO_START_EC
 OSAPI, [249](#)
 OSAPI_LOG_THREAD_DESTROY_NO_WAKEUP_EC
 OSAPI, [249](#)
 OSAPI_LOG_THREAD_EXEC_CREATE_EC
 OSAPI, [249](#)
 OSAPI_LOG_THREAD_EXEC_START_EC
 OSAPI, [249](#)
 OSAPI_LOG_THREAD_GET_MAX_PRIORITY_EC
 OSAPI, [259](#)
 OSAPI_LOG_THREAD_GET_POLICY_EC
 OSAPI, [250](#)
 OSAPI_LOG_THREAD_INIT_EC
 OSAPI, [250](#)
 OSAPI_LOG_THREAD_JOIN_EC
 OSAPI, [259](#)
 OSAPI_LOG_THREAD_NEW_EC
 OSAPI, [248](#)
 OSAPI_LOG_THREAD_POLICY_DIFFER_EC
 OSAPI, [250](#)
 OSAPI_LOG_THREAD_PRIORITY_MAP_EC
 OSAPI, [250](#)
 OSAPI_LOG_THREAD_SCHEDPARAM_EC
 OSAPI, [250](#)
 OSAPI_LOG_THREAD_SEM_EC
 OSAPI, [249](#)
 OSAPI_LOG_THREAD_SEMPOOL_EXCEEDED_EC
 OSAPI, [258](#)
 OSAPI_LOG_THREAD_SEMPOOL_INVALID_SEM_EC
 OSAPI, [258](#)
 OSAPI_LOG_THREAD_START_EC
 OSAPI, [249](#)
 OSAPI_LOG_TIMER_DELETE_EC
 OSAPI, [250](#)
 OSAPI_LOG_TIMER_DELETE_MUTEX_EC
 OSAPI, [252](#)
 OSAPI_LOG_TIMER_DELETE_STOP_TIMER_EC
 OSAPI, [251](#)
 OSAPI_LOG_TIMER_GET_USER_DATA_EPOCH_EC
 OSAPI, [251](#)
 OSAPI_LOG_TIMER_MUTEX_EC
 OSAPI, [252](#)
 OSAPI_LOG_TIMER_NEW_EC
 OSAPI, [251](#)
 OSAPI_LOG_TIMER_NEW_ENTRY_EC
 OSAPI, [251](#)
 OSAPI_LOG_TIMER_NEW_MUTEX_EC
 OSAPI, [251](#)
 OSAPI_LOG_TIMER_NEW_START_TIMER_EC
 OSAPI, [251](#)
 OSAPI_LOG_TIMER_NEW_WHEEL_EC
 OSAPI, [251](#)
 OSAPI_LOG_TIMER_TICK_MUTEX_EC
 OSAPI, [250](#)
 OSAPI_LOG_UNKNOWN_SEMAPHORE_TYPE_EC
 OSAPI, [255](#)
 OSAPI_LOG_VERBOSITY_DEBUG
 Log API, [221](#)
 OSAPI_LOG_VERBOSITY_ERROR
 Log API, [221](#)
 OSAPI_LOG_VERBOSITY_SILENT
 Log API, [221](#)
 OSAPI_LOG_VERBOSITY_WARNING
 Log API, [221](#)
 OSAPI_LOG_WIN_WAIT_FAILED_EC
 OSAPI, [266](#)
 OSAPI_Log_write_buffer_T
 Log API, [220](#)
 OSAPI_LogProperty, [874](#)
 log_detail, [875](#)
 max_buffer_size, [875](#)
 write_buffer, [875](#)
 OSAPI_LogProperty_INIIALIZER
 Log API, [220](#)
 OSAPI_LogVerbosity_T
 Log API, [220](#)
 OSAPI_Mutex_delete
 OSAPI Mutex API, [191](#)
 OSAPI_Mutex_give
 OSAPI Mutex API, [192](#)
 OSAPI_Mutex_new
 OSAPI Mutex API, [191](#)
 OSAPI_Mutex_T
 OSAPI Mutex API, [191](#)
 OSAPI_Mutex_take
 OSAPI Mutex API, [191](#)
 osapi_process.h, [983](#)
 OSAPI_Process_getpid
 OSAPI Process API, [193](#)
 OSAPI_Process_is_alive
 OSAPI Process API, [193](#)
 osapi_semaphore.h, [983](#)
 OSAPI_Semaphore_give, [985](#)
 OSAPI_Semaphore_new, [984](#)
 OSAPI_Semaphore_take, [984](#)
 OSAPI_Semaphore_delete
 OSAPI Semaphore API, [195](#)
 OSAPI_Semaphore_give
 osapi_semaphore.h, [985](#)

- OSAPI_Semaphore_new
 - osapi_semaphore.h, [984](#)
- OSAPI_SEMAPHORE_RESULT_ERROR
 - OSAPI Semaphore API, [194](#)
- OSAPI_SEMAPHORE_RESULT_OK
 - OSAPI Semaphore API, [194](#)
- OSAPI_SEMAPHORE_RESULT_TIMEOUT
 - OSAPI Semaphore API, [194](#)
- OSAPI_Semaphore_T
 - OSAPI Semaphore API, [195](#)
- OSAPI_Semaphore_take
 - osapi_semaphore.h, [984](#)
- OSAPI_SEMAPHORE_TIMEOUT_INFINITE
 - OSAPI Semaphore API, [194](#)
- OSAPI_SharedMemory_is_robust_mutex_supported
 - OSAPI_SharedMemorySegment, [218](#)
- OSAPI_SharedMemoryMonitor, [208](#)
 - OSAPI_SharedMemoryMonitor_acquire, [209](#)
 - OSAPI_SharedMemoryMonitor_finalize, [209](#)
 - OSAPI_SharedMemoryMonitor_get_size, [208](#)
 - OSAPI_SharedMemoryMonitor_initialize, [208](#)
 - OSAPI_SharedMemoryMonitor_mark_consistent, [211](#)
 - OSAPI_SharedMemoryMonitor_release, [210](#)
 - OSAPI_SharedMemoryMonitor_signal, [210](#)
 - OSAPI_SharedMemoryMonitor_wait, [210](#)
- OSAPI_SharedMemoryMonitor_acquire
 - OSAPI_SharedMemoryMonitor, [209](#)
- OSAPI_SharedMemoryMonitor_finalize
 - OSAPI_SharedMemoryMonitor, [209](#)
- OSAPI_SharedMemoryMonitor_get_size
 - OSAPI_SharedMemoryMonitor, [208](#)
- OSAPI_SharedMemoryMonitor_initialize
 - OSAPI_SharedMemoryMonitor, [208](#)
- OSAPI_SharedMemoryMonitor_mark_consistent
 - OSAPI_SharedMemoryMonitor, [211](#)
- OSAPI_SharedMemoryMonitor_release
 - OSAPI_SharedMemoryMonitor, [210](#)
- OSAPI_SharedMemoryMonitor_signal
 - OSAPI_SharedMemoryMonitor, [210](#)
- OSAPI_SharedMemoryMonitor_wait
 - OSAPI_SharedMemoryMonitor, [210](#)
- OSAPI_SharedMemorySegment, [211](#)
 - OSAPI_SharedMemory_is_robust_mutex_supported, [218](#)
 - OSAPI_SharedMemorySegment_attach_impl, [216](#)
 - OSAPI_SharedMemorySegment_create_impl, [215](#)
 - OSAPI_SharedMemorySegment_delete_impl, [215](#)
 - OSAPI_SharedMemorySegment_detach_impl, [216](#)
 - OSAPI_SharedMemorySegment_get_max_size, [214](#)
 - OSAPI_SharedMemorySegment_is_owner_alive, [214](#)
 - OSAPI_SharedMemorySegment_lock_impl, [216](#)
 - OSAPI_SharedMemorySegment_mark_consistent_impl, [217](#)
 - OSAPI_SharedMemorySegment_unlock_impl, [217](#)
 - OSAPI_SharedMemorySegmentHandle_get_size, [214](#)
 - OSAPI_SharedMemorySegmentHeader_get_size, [214](#)
 - OSAPI_SHMEM_ATTACH_STATUS, [213](#)
 - OSAPI_SHMEM_ATTACH_STATUS_OK, [213](#)
 - OSAPI_SHMEM_ATTACH_STATUS_SEGMENT_NOT_FOUND, [213](#)
 - OSAPI_SHMEM_MODE, [213](#)
 - OSAPI_SHMEM_MODE_LOCKABLE, [213](#)
 - OSAPI_SHMEM_MODE_READ, [213](#)
 - OSAPI_SHMEM_MODE_ROBUST, [213](#)
 - OSAPI_SHMEM_MODE_UNKNOWN, [213](#)
 - OSAPI_SHMEM_MODE_WRITE, [213](#)
 - OSAPI_SHMEM_STATUS, [213](#)
 - OSAPI_SHMEM_STATUS_OK, [213](#)
 - OSAPI_SHMEM_STATUS_OWNER_DEAD, [213](#)
- OSAPI_SharedMemorySegment_attach_impl
 - OSAPI_SharedMemorySegment, [216](#)
- OSAPI_SharedMemorySegment_create_impl
 - OSAPI_SharedMemorySegment, [215](#)
- OSAPI_SharedMemorySegment_delete_impl
 - OSAPI_SharedMemorySegment, [215](#)
- OSAPI_SharedMemorySegment_detach_impl
 - OSAPI_SharedMemorySegment, [216](#)
- OSAPI_SharedMemorySegment_get_max_size
 - OSAPI_SharedMemorySegment, [214](#)
- OSAPI_SharedMemorySegment_is_owner_alive
 - OSAPI_SharedMemorySegment, [214](#)
- OSAPI_SharedMemorySegment_lock_impl
 - OSAPI_SharedMemorySegment, [216](#)
- OSAPI_SharedMemorySegment_mark_consistent_impl
 - OSAPI_SharedMemorySegment, [217](#)
- OSAPI_SharedMemorySegment_unlock_impl
 - OSAPI_SharedMemorySegment, [217](#)
- OSAPI_SharedMemorySegmentHandle, [875](#)
 - mode, [876](#)
 - ptr_header, [876](#)
 - ptr_user_data, [876](#)
- OSAPI_SharedMemorySegmentHandle_get_size
 - OSAPI_SharedMemorySegment, [214](#)
- OSAPI_SharedMemorySegmentHeader, [876](#)
 - mode, [876](#)
 - size, [876](#)
- OSAPI_SharedMemorySegmentHeader_get_size
 - OSAPI_SharedMemorySegment, [214](#)
- OSAPI_SHMEM_ATTACH_STATUS
 - OSAPI_SharedMemorySegment, [213](#)
- OSAPI_SHMEM_ATTACH_STATUS_OK
 - OSAPI_SharedMemorySegment, [213](#)
- OSAPI_SHMEM_ATTACH_STATUS_SEGMENT_NOT_FOUND

- OSAPI_SharedMemorySegment, 213
- OSAPI_SHMEM_MODE
 - OSAPI_SharedMemorySegment, 213
- OSAPI_SHMEM_MODE_LOCKABLE
 - OSAPI_SharedMemorySegment, 213
- OSAPI_SHMEM_MODE_READ
 - OSAPI_SharedMemorySegment, 213
- OSAPI_SHMEM_MODE_ROBUST
 - OSAPI_SharedMemorySegment, 213
- OSAPI_SHMEM_MODE_UNKNOWN
 - OSAPI_SharedMemorySegment, 213
- OSAPI_SHMEM_MODE_WRITE
 - OSAPI_SharedMemorySegment, 213
- OSAPI_SHMEM_SEGMENT_NOT_INITIALIZED_EC
 - OSAPI, 262
- OSAPI_SHMEM_STATUS
 - OSAPI_SharedMemorySegment, 213
- OSAPI_SHMEM_STATUS_OK
 - OSAPI_SharedMemorySegment, 213
- OSAPI_SHMEM_STATUS_OWNER_DEAD
 - OSAPI_SharedMemorySegment, 213
- osapi_stdio.h, 986
- OSAPI_System_generate_uuid_T
 - OSAPI System API, 199
- OSAPI_System_get_hostname_T
 - OSAPI System API, 199
- OSAPI_System_get_ticktime_T
 - OSAPI System API, 200
- OSAPI_System_get_time_T
 - OSAPI System API, 198
- OSAPI_System_get_timer_resolution_T
 - OSAPI System API, 198
- OSAPI_System_initialize_T
 - OSAPI System API, 199
- OSAPI_SYSTEM_MAX_HOSTNAME
 - OSAPI System API, 197
- OSAPI_System_start_timer_T
 - OSAPI System API, 198
- OSAPI_SystemI, 877
 - generate_uuid, 878
 - get_hostname, 878
 - get_ticktime, 878
 - get_time, 878
 - get_timer_resolution, 877
 - OSAPI System API, 200
 - start_timer, 877
- OSAPI_SystemProperty, 878
 - hostname, 879
 - max_timers, 879
 - max_user_blocking_threads, 879
 - task_scheduler, 880
 - timer_property, 879
- OSAPI_SystemProperty_INITIALIZER
 - OSAPI System API, 197
- OSAPI_SystemTime, 880
 - nanosec, 881
 - OSAPI time API, 204
 - sec, 881
- osapi_task.h, 986
- OSAPI_TaskProperty, 881
 - rate, 881
 - thread, 881
- OSAPI_TaskProperty_INITIALIZER
 - OSAPI System API, 197
- OSAPI_THREAD_DEFAULT_OPTIONS
 - OSAPI Thread API, 202
- OSAPI_THREAD_FLOATING_POINT
 - OSAPI Thread API, 202
- OSAPI_THREAD_PRIORITY_ABOVE_NORMAL
 - OSAPI Thread API, 202
- OSAPI_THREAD_PRIORITY_BELOW_NORMAL
 - OSAPI Thread API, 201
- OSAPI_THREAD_PRIORITY_HIGH
 - OSAPI Thread API, 202
- OSAPI_THREAD_PRIORITY_INHERIT
 - OSAPI Thread API, 202
- OSAPI_THREAD_PRIORITY_LOW
 - OSAPI Thread API, 201
- OSAPI_THREAD_PRIORITY_NORMAL
 - OSAPI Thread API, 202
- OSAPI_THREAD_PROPERTY_DEFAULT
 - OSAPI Thread API, 203
- OSAPI_THREAD_REALTIME_PRIORITY
 - OSAPI Thread API, 203
- OSAPI_THREAD_STDIO
 - OSAPI Thread API, 203
- OSAPI_THREAD_USE_OSDEFAULT_STACKSIZE
 - OSAPI Thread API, 202
- OSAPI_ThreadOptions
 - OSAPI Thread API, 203
- OSAPI_ThreadProperty, 882
 - options, 882
 - priority, 882
 - stack_size, 882
- OSAPI_ThreadProperty_INITIALIZER
 - OSAPI Thread API, 203
- osapi_time.h, 986
- OSAPI_TimerProperty, 883
 - thread, 883
- OSAPI_Trace_set_trace_mask
 - osapi_log.h, 981
- OWNERSHIP, 101
 - DDS_EXCLUSIVE_OWNERSHIP_QOS, 101
 - DDS_OwnershipQosPolicyKind, 101
 - DDS_SHARED_OWNERSHIP_QOS, 101
- ownership
 - DDS_DataReaderQos, 521
 - DDS_DataWriterQos, 533

- DDS_PublicationBuiltinTopicData, 613
- DDS_SubscriptionBuiltinTopicData, 651
- OWNERSHIP_STRENGTH, 102
- ownership_strength
 - DDS_DataWriterQos, 533
 - DDS_PublicationBuiltinTopicData, 613
- Participant Built-in Topic, 40
 - DDS_PARTICIPANT_TOPIC_NAME, 41
- participant_id
 - DDS_WireProtocolQosPolicy, 665
- participant_id_gain
 - DDS_RtpsWellKnownPorts_t, 637
- participant_key
 - DDS_PublicationBuiltinTopicData, 612
 - DDS_SubscriptionBuiltinTopicData, 650
- participant_liveliness_assert_period
 - DPDE_DiscoveryPluginProperty, 805
 - DPSE_DiscoveryPluginProperty, 810
- participant_liveliness_lease_duration
 - DPDE_DiscoveryPluginProperty, 805
 - DPSE_DiscoveryPluginProperty, 810
- participant_message_reader_reliability_kind
 - DPDE_DiscoveryPluginProperty, 808
 - DPSE_DiscoveryPluginProperty, 811
- participant_name
 - DDS_DomainParticipantQos, 549
 - DDS_ParticipantBuiltinTopicData, 595
- participant_user_data_max_count
 - DDS_DomainParticipantResourceLimitsQosPolicy, 558
- participant_user_data_max_length
 - DDS_DomainParticipantResourceLimitsQosPolicy, 557
- PARTITION, 123
- partition
 - DDS_PublicationBuiltinTopicData, 614
 - DDS_PublisherQos, 617
 - DDS_SubscriberQos, 648
 - DDS_SubscriptionBuiltinTopicData, 652
- pass_tracker_config
 - DDS_PskServiceFactoryProperty, 610
- pass_tracker_get_interface_func
 - DDS_PskServiceFactoryProperty, 610
- period
 - DDS_DeadlineQosPolicy, 541
 - DDS_FlowControllerTokenBucketProperty_t, 573
- policies
 - DDS_OfferedIncompatibleQosStatus, 589
 - DDS_RequestedIncompatibleQosStatus, 627
- policy_id
 - DDS_QosPolicyCount, 620
- polling_interval_ms
 - PSK_PassTrackerConfiguration, 885
- port
 - DDS_Locator_t, 586
 - NETIO_Address, 861
- port_base
 - DDS_RtpsWellKnownPorts_t, 636
- PRESENTATION, 109
 - DDS_GROUP_PRESENTATION_QOS, 110
 - DDS_INSTANCE_PRESENTATION_QOS, 110
 - DDS_PresentationQosPolicyAccessScopeKind, 110
 - DDS_TOPIC_PRESENTATION_QOS, 110
- presentation
 - DDS_SubscriptionBuiltinTopicData, 652
- priority
 - DDS_PublishModeQosPolicy, 619
 - OSAPI_ThreadProperty, 882
- pro_minimum_compatibility_version
 - NETIO_SHMEMInterfaceFactoryProperty, 874
- product_version
 - DDS_ParticipantBuiltinTopicData, 596
- propagate
 - DDS_Property_t, 605
- propagate_dispose_of_unregistered_instances
 - DDS_DataReaderProtocolQosPolicy, 519
- PROPERTY, 123
- property
 - DDS_DataReaderQos, 522
 - DDS_DataWriterQos, 534
 - DDS_DomainParticipantQos, 550
 - DDS_ParticipantBuiltinTopicData, 596
- Property Reference, 134
 - DDS_XTYPES_COMPLIANCE_MASK_PROPERTY, 135
- protocol
 - DDS_DataReaderQos, 522
 - DDS_DataWriterQos, 533
 - DDS_DomainParticipantQos, 549
- provider_name
 - DDS_SecTransform_Configuration, 646
- PSK_ErrListener, 883
 - callback, 884
 - user_ptr, 884
- PSK_FilePassTrackerErrListener, 884
 - callback, 885
 - user_ptr, 885
- PSK_PassTrackerConfiguration, 885
 - enable_init_read, 885
 - err_listener, 886
 - polling_interval_ms, 885
- psl_config
 - DDS_PskServiceFactoryProperty, 609
- psl_get_interface_func
 - DDS_PskServiceFactoryProperty, 609
- ptr_header
 - OSAPI_SharedMemorySegmentHandle, 876

- ptr_user_data
 - OSAPI_SharedMemorySegmentHandle, [876](#)
- Publication Built-in Topic, [41](#)
 - DDS_PUBLICATION_TOPIC_NAME, [41](#)
- Publication Module, [54](#)
- publication_handle
 - DDS_DataReaderInstanceReplacedStatus, [517](#)
 - DDS_SampleInfo, [641](#)
- publication_name
 - DDS_DataWriterQos, [534](#)
- publication_sequence_number
 - DDS_SampleInfo, [642](#)
- PUBLISH_MODE, [121](#)
 - DDS_ASYNCHRONOUS_PUBLISH_MODE_QOS, [123](#)
 - DDS_AUTOMATIC_PUBLISH_MODE_QOS, [123](#)
 - DDS_PUBLICATION_PRIORITY_AUTOMATIC, [122](#)
 - DDS_PUBLICATION_PRIORITY_UNDEFINED, [122](#)
 - DDS_PublishModeQosPolicyKind, [122](#)
 - DDS_SYNCHRONOUS_PUBLISH_MODE_QOS, [122](#)
- publish_mode
 - DDS_DataWriterQos, [534](#)
- Publisher, [59](#)
 - DDS_DATAWRITER_QOS_DEFAULT, [61](#)
 - DDS_Publisher, [60](#)
- publisher_group_data_max_count
 - DDS_DomainParticipantResourceLimitsQosPolicy, [558](#)
- publisher_group_data_max_length
 - DDS_DomainParticipantResourceLimitsQosPolicy, [558](#)
- publisher_name
 - DDS_PublisherQos, [617](#)
- QoS Policies, [92](#)
 - DDS_DEADLINE_QOS_POLICY_ID, [96](#)
 - DDS_ENTITYFACTORY_QOS_POLICY_ID, [96](#)
 - DDS_HISTORY_QOS_POLICY_ID, [96](#)
 - DDS_INVALID_QOS_POLICY_ID, [96](#)
 - DDS_LIVELINESS_QOS_POLICY_ID, [96](#)
 - DDS_OWNERSHIP_QOS_POLICY_ID, [96](#)
 - DDS_OWNERSHIPSTRENGTH_QOS_POLICY_ID, [96](#)
 - DDS_QosPolicyId_t, [96](#)
 - DDS_RELIABILITY_QOS_POLICY_ID, [96](#)
- rate
 - OSAPI_TaskProperty, [881](#)
- read
 - FooDataReader, [817](#)
- read_instance
 - FooDataReader, [822](#)
- read_instance_untyped
 - DDSDDataReader, [679](#)
- read_next_sample
 - FooDataReader, [816](#)
- read_next_sample_untyped
 - DDSDDataReader, [681](#)
- read_untyped
 - DDSDDataReader, [675](#)
- reader_resource_limits
 - DDS_DataReaderQos, [522](#)
- reader_user_data_max_count
 - DDS_DomainParticipantResourceLimitsQosPolicy, [560](#)
- reader_user_data_max_length
 - DDS_DomainParticipantResourceLimitsQosPolicy, [559](#)
- reason
 - DDS_SampleLostStatus, [644](#)
- receive_buffer_size
 - NETIO_SHMEMInterfaceFactoryProperty, [873](#)
- received_message_count_max
 - NETIO_SHMEMInterfaceFactoryProperty, [873](#)
- reception_timestamp
 - DDS_SampleInfo, [642](#)
- recv_thread
 - UDP_InterfaceFactoryProperty, [894](#)
- recv_thread_property
 - NETIO_SHMEMInterfaceFactoryProperty, [874](#)
- REDA, [266](#)
 - NETIO_SHMEM_LOG_CQ_INCOMPATIBLE_VERSION_EC, [272](#)
 - NETIO_SHMEM_LOG_CQ_INCONSISTENT_STATE_EC, [272](#)
 - NETIO_SHMEM_LOG_CQ_INT_REPRESENTATION_EC, [272](#)
 - NETIO_SHMEM_LOG_CQ_INVALID_SIGNATURE_EC, [272](#)
 - NETIO_SHMEM_LOG_CQ_UNLINKED_EC, [272](#)
 - REDA_LOG_BUFFER_OUT_OF_MEMORY_EC, [271](#)
 - REDA_LOG_BUFFERPOOL_BUFFER_INITIALIZATION_FAILED_EC, [268](#)
 - REDA_LOG_BUFFERPOOL_DOUBLE_FREE_EC, [269](#)
 - REDA_LOG_BUFFERPOOL_NOT_EMPTY_EC, [269](#)
 - REDA_LOG_BUFFERPOOL_NULL_POINTER_EC, [269](#)
 - REDA_LOG_BUFFERPOOL_OUT_OF_RESOURCES_EC, [268](#)
 - REDA_LOG_HEAP_MGR_ALLOC_EC, [271](#)
 - REDA_LOG_HEAP_MGR_GET_BUFFER_EC, [271](#)
 - REDA_LOG_HEAP_MGR_INDEXER, [268](#)
 - REDA_LOG_HEAP_MGR_LISTNODE_BUFFERPOOL, [268](#)
 - REDA_LOG_HEAP_MGR_MUTEX, [268](#)

REDA_LOG_HEAP_MGR_MUTEX_FAILURE_EC, [271](#)
 REDA_LOG_HEAP_MGR_OBJECT_MEMORY, [268](#)
 REDA_LOG_HEAP_MGR_SAMPLE_BUFFERPOOL, [268](#)
 REDA_LOG_INDEX_ENTRY_EXISTS_EC, [271](#)
 REDA_LOG_INDEX_FULL_EC, [271](#)
 REDA_LOG_INVALID_LIST_NODE_EC, [271](#)
 REDA_LOG_MEMPOOL_INCONSISTENT_PROPERTIES_EC, [269](#)
 REDA_LOG_MEMPOOL_OUT_OF_RESOURCES_EC, [269](#)
 REDA_LOG_MEMPOOL_POOL_ALLOCATION_EC, [269](#)
 REDA_LOG_SEQUENCE_ALLOC_FAILED_EC, [270](#)
 REDA_LOG_SEQUENCE_COPY_FAILED_EC, [269](#)
 REDA_LOG_SEQUENCE_INDEX_OUT_OF_BOUNDS_EC, [270](#)
 REDA_LOG_SEQUENCE_INVALID_LENGTH_EC, [270](#)
 REDA_LOG_SEQUENCE_INVALID_OPERATION_EC, [270](#)
 REDA_LOG_SEQUENCE_REALLOCATION_EC, [270](#)
 REDA_LOG_SEQUENCE_SET_MAX_FAILED_EC, [270](#)
 REDA_LOG_STRING_ALLOC_FAILED_EC, [270](#)
 reda_log.h, [987](#)
 REDA_LOG_BASE
 osapi_log.h, [976](#)
 REDA_LOG_BUFFER_OUT_OF_MEMORY_EC
 REDA, [271](#)
 REDA_LOG_BUFFERPOOL_BUFFER_INITIALIZATION_FAILED_EC
 REDA, [268](#)
 REDA_LOG_BUFFERPOOL_DOUBLE_FREE_EC
 REDA, [269](#)
 REDA_LOG_BUFFERPOOL_NOT_EMPTY_EC
 REDA, [269](#)
 REDA_LOG_BUFFERPOOL_NULL_POINTER_EC
 REDA, [269](#)
 REDA_LOG_BUFFERPOOL_OUT_OF_RESOURCES_EC
 REDA, [268](#)
 REDA_LOG_HEAP_MGR_ALLOC_EC
 REDA, [271](#)
 REDA_LOG_HEAP_MGR_GET_BUFFER_EC
 REDA, [271](#)
 REDA_LOG_HEAP_MGR_INDEXER
 REDA, [268](#)
 REDA_LOG_HEAP_MGR_LISTNODE_BUFFERPOOL
 REDA, [268](#)
 REDA_LOG_HEAP_MGR_MUTEX
 REDA, [268](#)
 REDA_LOG_HEAP_MGR_MUTEX_FAILURE_EC
 REDA, [271](#)
 REDA_LOG_HEAP_MGR_OBJECT_MEMORY
 REDA, [268](#)
 REDA_LOG_HEAP_MGR_SAMPLE_BUFFERPOOL
 REDA, [268](#)
 REDA_LOG_INDEX_ENTRY_EXISTS_EC
 REDA, [271](#)
 REDA_LOG_INDEX_FULL_EC
 REDA, [271](#)
 REDA_LOG_INVALID_LIST_NODE_EC
 REDA, [271](#)
 REDA_LOG_MEMPOOL_INCONSISTENT_PROPERTIES_EC
 REDA, [269](#)
 REDA_LOG_MEMPOOL_OUT_OF_RESOURCES_EC
 REDA, [269](#)
 REDA_LOG_MEMPOOL_POOL_ALLOCATION_EC
 REDA, [269](#)
 REDA_LOG_SEQUENCE_ALLOC_FAILED_EC
 REDA, [270](#)
 REDA_LOG_SEQUENCE_COPY_FAILED_EC
 REDA, [269](#)
 REDA_LOG_SEQUENCE_INDEX_OUT_OF_BOUNDS_EC
 REDA, [270](#)
 REDA_LOG_SEQUENCE_INVALID_LENGTH_EC
 REDA, [270](#)
 REDA_LOG_SEQUENCE_INVALID_OPERATION_EC
 REDA, [270](#)
 REDA_LOG_SEQUENCE_REALLOCATION_EC
 REDA, [270](#)
 REDA_LOG_SEQUENCE_SET_MAX_FAILED_EC
 REDA, [270](#)
 REDA_LOG_STRING_ALLOC_FAILED_EC
 REDA, [270](#)
 RTComponent
 RTRegistry, [889](#)
 register_instance
 FooDataWriter, [829](#)
 register_instance_untyped
 DDSDDataWriter, [699](#)
 register_instance_w_timestamp
 FooDataWriter, [830](#)
 register_instance_w_timestamp_untyped
 DDSDDataWriter, [700](#)
 register_type
 DDSDomainParticipant, [731](#)
 FooTypeSupport, [857](#)
 related_topic_name
 DDS_ContentFilterProperty, [513](#)
 release
 DDS_ProductVersion_t, [604](#)
 release_address
 NETIO_DGRAM_InterfaceI, [865](#)
 ZCOPY_NotifUserInterfaceI, [906](#)
 RELIABILITY, [103](#)

- DDS_BEST_EFFORT_RELIABILITY_QOS, [104](#)
- DDS_ReliabilityQosPolicyKind, [104](#)
- DDS_RELIABLE_RELIABILITY_QOS, [104](#)
- reliability
 - DDS_DataReaderQos, [521](#)
 - DDS_DataWriterQos, [533](#)
 - DDS_PublicationBuiltinTopicData, [613](#)
 - DDS_SubscriptionBuiltinTopicData, [651](#)
- remote_participant_allocation, [228](#)
 - DDS_DomainParticipantResourceLimitsQosPolicy, [554](#)
- remote_reader_allocation, [229](#)
 - DDS_DomainParticipantResourceLimitsQosPolicy, [554](#)
- remote_writer_allocation, [229](#)
 - DDS_DomainParticipantResourceLimitsQosPolicy, [554](#)
- RemoteParticipant_assert
 - DPSEDiscoveryPlugin, [813](#)
- RemotePublication_assert
 - DPSEDiscoveryPlugin, [813](#)
- RemoteSubscription_assert
 - DPSEDiscoveryPlugin, [813](#)
- remove_property
 - DDSPROPERTY_QOS_POLICY_HELPER, [769](#)
- representation
 - DDS_DataReaderQos, [522](#)
 - DDS_DataWriterQos, [533](#)
 - DDS_PublicationBuiltinTopicData, [614](#)
 - DDS_SubscriptionBuiltinTopicData, [652](#)
- require_crc
 - DDS_ChecksumProperty_t, [512](#)
 - DDS_WireProtocolQosPolicy, [667](#)
- reserve_address
 - ZCOPY_NotifUserInterfaceI, [906](#)
- resolve_address
 - NETIO_DGRAM_InterfaceI, [865](#)
 - ZCOPY_NotifUserInterfaceI, [905](#)
- Resource Limits, [224](#)
- RESOURCE_LIMITS, [108](#)
 - DDS_LENGTH_AUTO, [109](#)
 - DDS_LENGTH_UNLIMITED, [109](#)
 - DDS_SIZE_AUTO, [109](#)
- resource_limits
 - DDS_DataReaderQos, [521](#)
 - DDS_DataWriterQos, [532](#)
 - DDS_DomainParticipantFactoryQos, [547](#)
 - DDS_DomainParticipantQos, [549](#)
 - DDS_FilterQosPolicy, [566](#)
- Return Codes, [82](#)
 - DDS_RETCODE_ALREADY_DELETED, [83](#)
 - DDS_RETCODE_BAD_PARAMETER, [83](#)
 - DDS_RETCODE_ERROR, [83](#)
 - DDS_RETCODE_ILLEGAL_OPERATION, [83](#)
 - DDS_RETCODE_IMMUTABLE_POLICY, [83](#)
 - DDS_RETCODE_INCONSISTENT_POLICY, [83](#)
 - DDS_RETCODE_NO_DATA, [83](#)
 - DDS_RETCODE_NOT_ALLOWED_BY_SEC, [83](#)
 - DDS_RETCODE_NOT_ENABLED, [83](#)
 - DDS_RETCODE_OK, [83](#)
 - DDS_RETCODE_OUT_OF_RESOURCES, [83](#)
 - DDS_RETCODE_PRECONDITION_NOT_MET, [83](#)
 - DDS_RETCODE_TIMEOUT, [83](#)
 - DDS_RETCODE_UNSUPPORTED, [83](#)
 - DDS_ReturnCode_t, [82](#)
- return_loan
 - FooDataReader, [815](#)
- return_loan_untyped
 - DDSDDataReader, [683](#)
- revision
 - DDS_ProductVersion_t, [604](#)
- RH, [464](#)
 - RHSM_LOG_DESTINATION_BY_SOURCE_NOT_SUPPORTED_EC, [469](#)
 - RHSM_LOG_ENTRY_RESERVATION_FAILED_EC, [468](#)
 - RHSM_LOG_FAILED_TO_REMOVE_OLDEST_EC, [467](#)
 - RHSM_LOG_GET_RECEPTION_TIMESTAMP_EC, [467](#)
 - RHSM_LOG_GET_TIME_EC, [468](#)
 - RHSM_LOG_HISTORY_UPDATE_EC, [468](#)
 - RHSM_LOG_INFO_ARRAY_EC, [467](#)
 - RHSM_LOG_INVALID_INSTANCE_REPLACEMENT_EC, [467](#)
 - RHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC, [466](#)
 - RHSM_LOG_MAX_SAMPLE_TOO_SMALL_EC, [466](#)
 - RHSM_LOG_NO_PROPERTY_EC, [469](#)
 - RHSM_LOG_NO_PROPERTY_QOS_EC, [466](#)
 - RHSM_LOG_OBJECT_ALLOCATE_EC, [468](#)
 - RHSM_LOG_OBJECT_DELETE_EC, [468](#)
 - RHSM_LOG_OBJECT_INDEX_EC, [469](#)
 - RHSM_LOG_OBJECT_INVALID_EC, [469](#)
 - RHSM_LOG_OUTSTANDING_SAMPLE_EC, [466](#)
 - RHSM_LOG_RECOMMIT_FAILURE_EC, [469](#)
 - RHSM_LOG_RETURN_SAMPLE_EC, [468](#)
 - RHSM_LOG_RW_PRUNE_FAILED_EC, [468](#)
 - RHSM_LOG_SAMPLE_INFO_POOL_EC, [466](#)
 - RHSM_LOG_SAMPLE_POOL_EC, [467](#)
 - RHSM_LOG_SAMPLE_POOL_EMPTY_EC, [467](#)
 - RHSM_LOG_SAMPLE_PTR_ARRAY_EC, [467](#)
 - RHSM_LOG_TBF_NOT_SUPPORTED_EC, [466](#)
- rh_sm_log.h, [988](#)
 - RHSM_LOG_HISTORY_OBJECT, [990](#)
 - RHSM_LOG_INSTANCEHANDLE_OBJECT, [992](#)
 - RHSM_LOG_KEY_OBJECT, [991](#)

- RHSM_LOG_KEYINDEXPOOL_OBJECT, 992
- RHSM_LOG_KEYPOOL_OBJECT, 991
- RHSM_LOG_OWNER_OBJECT, 991
- RHSM_LOG_REMOVELIST_OBJECT, 990
- RHSM_LOG_RW_OBJECT, 991
- RHSM_LOG_RWINDEX_OBJECT, 991
- RHSM_LOG_RWPOOL_OBJECT, 991
- RHSM_LOG_SAMPLEINFO_OBJECT, 992
- RHSM_LOG_SAMPLEPOOL_OBJECT, 991
- RHSM_LOG_SAMPLEPTPOOL_OBJECT, 992
- RHSM Reader History API, 141
- RHSM_LOG_BASE
 - osapi_log.h, 978
- RHSM_LOG_DESTINATION_BY_SOURCE_NOT_SUPPORTED_EC
 - RH, 469
- RHSM_LOG_ENTRY_RESERVATION_FAILED_EC
 - RH, 468
- RHSM_LOG_FAILED_TO_REMOVE_OLDEST_EC
 - RH, 467
- RHSM_LOG_GET_RECEPTION_TIMESTAMP_EC
 - RH, 467
- RHSM_LOG_GET_TIME_EC
 - RH, 468
- RHSM_LOG_HISTORY_OBJECT
 - rh_sm_log.h, 990
- RHSM_LOG_HISTORY_UPDATE_EC
 - RH, 468
- RHSM_LOG_INFO_ARRAY_EC
 - RH, 467
- RHSM_LOG_INSTANCEHANDLE_OBJECT
 - rh_sm_log.h, 992
- RHSM_LOG_INVALID_INSTANCE_REPLACEMENT_EC
 - RH, 467
- RHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC
 - RH, 466
- RHSM_LOG_KEY_OBJECT
 - rh_sm_log.h, 991
- RHSM_LOG_KEYINDEXPOOL_OBJECT
 - rh_sm_log.h, 992
- RHSM_LOG_KEYPOOL_OBJECT
 - rh_sm_log.h, 991
- RHSM_LOG_MAX_SAMPLE_TOO_SMALL_EC
 - RH, 466
- RHSM_LOG_NO_PROPERTY_EC
 - RH, 469
- RHSM_LOG_NO_PROPERTY_QOS_EC
 - RH, 466
- RHSM_LOG_OBJECT_ALLOCATE_EC
 - RH, 468
- RHSM_LOG_OBJECT_DELETE_EC
 - RH, 468
- RHSM_LOG_OBJECT_INDEX_EC
 - RH, 469
- RHSM_LOG_OBJECT_INVALID_EC
 - RH, 469
- RHSM_LOG_OUTSTANDING_SAMPLE_EC
 - RH, 466
- RHSM_LOG_OWNER_OBJECT
 - rh_sm_log.h, 991
- RHSM_LOG_RECOMMIT_FAILURE_EC
 - RH, 469
- RHSM_LOG_REMOVELIST_OBJECT
 - rh_sm_log.h, 990
- RHSM_LOG_RETURN_SAMPLE_EC
 - RH, 468
- RHSM_LOG_RW_OBJECT
 - rh_sm_log.h, 991
- RHSM_LOG_RW_PRUNE_FAILED_EC
 - RH, 468
- RHSM_LOG_RWINDEX_OBJECT
 - rh_sm_log.h, 991
- RHSM_LOG_RWPOOL_OBJECT
 - rh_sm_log.h, 991
- RHSM_LOG_SAMPLE_INFO_POOL_EC
 - RH, 466
- RHSM_LOG_SAMPLE_POOL_EC
 - RH, 467
- RHSM_LOG_SAMPLE_POOL_EMPTY_EC
 - RH, 467
- RHSM_LOG_SAMPLE_PTR_ARRAY_EC
 - RH, 467
- RHSM_LOG_SAMPLEINFO_OBJECT
 - rh_sm_log.h, 992
- RHSM_LOG_SAMPLEPOOL_OBJECT
 - rh_sm_log.h, 991
- RHSM_LOG_SAMPLEPTPOOL_OBJECT
 - rh_sm_log.h, 992
- RHSM_LOG_TBF_NOT_SUPPORTED_EC
 - RH, 466
- RHSMHistoryFactory, 886
 - get_interface, 886
- RT, 278
 - RT_LOG_REGISTRY_EXISTS_EC, 279
 - RT_LOG_REGISTRY_FINALIZE_EC, 279
 - RT_LOG_REGISTRY_INCONSISTENT_CID_EC, 280
 - RT_LOG_REGISTRY_INIT_FACTORY_EC, 279
 - RT_LOG_REGISTRY_INIT_FAILURE_EC, 279
 - RT_LOG_REGISTRY_NAME_TOO_LONG_EC, 279
 - RT_LOG_REGISTRY_NOT_INITIALIZED_EC, 280
 - RT_LOG_REGISTRY_REGISTER_EC, 279
 - RT_LOG_SET_IMMUTABLE_PROPERTY_EC, 279
- RT Run-Time API, 140
 - rt_log.h, 992
- RT_LOG_BASE
 - osapi_log.h, 977
- RT_LOG_REGISTRY_EXISTS_EC
 - RT, 279

- RT_LOG_REGISTRY_FINALIZE_EC
 - RT, [279](#)
- RT_LOG_REGISTRY_INCONSISTENT_CID_EC
 - RT, [280](#)
- RT_LOG_REGISTRY_INIT_FACTORY_EC
 - RT, [279](#)
- RT_LOG_REGISTRY_INIT_FAILURE_EC
 - RT, [279](#)
- RT_LOG_REGISTRY_NAME_TOO_LONG_EC
 - RT, [279](#)
- RT_LOG_REGISTRY_NOT_INITIALIZED_EC
 - RT, [280](#)
- RT_LOG_REGISTRY_REGISTER_EC
 - RT, [279](#)
- RT_LOG_SET_IMMUTABLE_PROPERTY_EC
 - RT, [279](#)
- RTI Connex Micro C++ API, [1](#)
- RTI_BOOL
 - OSAPI Types, [207](#)
- RTI_FALSE
 - OSAPI Types, [207](#)
- RTI_INT16
 - OSAPI Types, [206](#)
- RTI_INT32
 - OSAPI Types, [206](#)
- RTI_INT64
 - OSAPI Types, [206](#)
- RTI_INT8
 - OSAPI Types, [205](#)
- RTI_SIZE_T
 - OSAPI Types, [207](#)
- RTI_TRUE
 - OSAPI Types, [207](#)
- RTI_UINT16
 - OSAPI Types, [206](#)
- RTI_UINT32
 - OSAPI Types, [206](#)
- RTI_UINT64
 - OSAPI Types, [207](#)
- RTI_UINT8
 - OSAPI Types, [205](#)
- RTPS, [309](#)
 - RTPS_LOG_ACK_EC, [322](#)
 - RTPS_LOG_ADD_FILTERED_ROUTE_EC, [339](#)
 - RTPS_LOG_ALLOCATE_EC, [317](#)
 - RTPS_LOG_APPLY_CONTENT_FILTER_EC, [339](#)
 - RTPS_LOG_ASSERT_PEER_EC, [321](#)
 - RTPS_LOG_ASSERT_TRANSPORT_EC, [321](#)
 - RTPS_LOG_BAD_PARAMETER_EC, [320](#)
 - RTPS_LOG_BITCOUNT_OUT_OF_BOUNDS_EC, [325](#)
 - RTPS_LOG_BITMAP_SET_BIT_EC, [327](#)
 - RTPS_LOG_BITMAP_SHIFT_EC, [330](#)
 - RTPS_LOG_BUFFERPOOL_DELETE_EC, [329](#)
 - RTPS_LOG_CHECKSUM_CHECKSUM_CALCULATION_FAILED_EC, [337](#)
 - RTPS_LOG_CHECKSUM_CHECKSUM_DESERIALIZE_ERROR_EC, [338](#)
 - RTPS_LOG_CHECKSUM_CHECKSUM_ERROR_EC, [339](#)
 - RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM128_EC, [336](#)
 - RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM32_EC, [336](#)
 - RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM64_EC, [336](#)
 - RTPS_LOG_CHECKSUM_INCONSISTENT_LENGTH_EC, [338](#)
 - RTPS_LOG_CHECKSUM_INVALID_CRC32_FLAGS_EC, [338](#)
 - RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN128_EC, [336](#)
 - RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN32_EC, [337](#)
 - RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN64_EC, [337](#)
 - RTPS_LOG_CHECKSUM_LENGTH_DESERIALIZE_ERROR_EC, [338](#)
 - RTPS_LOG_CHECKSUM_MISSING_CHECKSUM_LENGTH_EC, [338](#)
 - RTPS_LOG_CHECKSUM_TOO_MANY_SG_BUFFERS_EC, [339](#)
 - RTPS_LOG_CHECKSUM_UNSUPPORTED_EC, [338](#)
 - RTPS_LOG_CREATE_BIND_TABLE_EC, [328](#)
 - RTPS_LOG_CREATE_BUFFER_POOL_EC, [332](#)
 - RTPS_LOG_CREATE_EVENT_TIMER_EC, [325](#)
 - RTPS_LOG_CREATE_HB_EVENT_EC, [325](#)
 - RTPS_LOG_CREATE_PEERS_INDEX_EC, [332](#)
 - RTPS_LOG_CREATE_REASSEMBLY_TABLE_EC, [336](#)
 - RTPS_LOG_CREATE_ROUTE_TABLE_EC, [317](#)
 - RTPS_LOG_DATA_ALREADY_RECEIVED_EC, [324](#)
 - RTPS_LOG_DATA_OUT_OF_RANGE_EC, [323](#)
 - RTPS_LOG_DB_CREATE_BIND_ENTRY_EC, [318](#)
 - RTPS_LOG_DB_CREATE_BIND_PEER_INDEX_EC, [317](#)
 - RTPS_LOG_DB_CREATE_DIRECT_INDEX_EC, [332](#)
 - RTPS_LOG_DB_CREATE_EXT_BIND_ENTRY_EC, [327](#)
 - RTPS_LOG_DB_CREATE_GROUP_INDEX_EC, [333](#)
 - RTPS_LOG_DB_CREATE_ROUTE_ENTRY_EC, [318](#)
 - RTPS_LOG_DB_CREATE_ROUTE_PEER_INDEX_EC, [317](#)
 - RTPS_LOG_DB_CREATE_ROUTE_XPORT_INDEX_EC, [317](#)

- [317](#)
 RTPS_LOG_DB_CREATE_SELECTED_GROUP_INDEX_EC, [317](#)
[335](#)
 RTPS_LOG_DB_DELETE_EXTERNAL_BIND_ENTRY_EC, [319](#)
[318](#)
 RTPS_LOG_DB_DELETE_ROUTE_ENTRY_EC, [318](#)
 RTPS_LOG_DB_INSERT_BIND_ENTRY_EC, [319](#)
 RTPS_LOG_DB_INSERT_ROUTE_ENTRY_EC, [318](#)
 RTPS_LOG_DB_LOCK_EC, [328](#)
 RTPS_LOG_DB_REMOVE_BIND_ENTRY_EC, [319](#)
 RTPS_LOG_DB_REMOVE_EXTERNAL_BIND_ENTRY_EC, [319](#)
 RTPS_LOG_DB_SELECT_ALL_EC, [317](#)
 RTPS_LOG_DB_SELECT_MATCH_EC, [318](#)
 RTPS_LOG_DB_SELECT_RANGE_EC, [318](#)
 RTPS_LOG_DB_UNLOCK_EC, [328](#)
 RTPS_LOG_DELETE_BUFFER_POOL_EC, [332](#)
 RTPS_LOG_DELETE_FILTERED_ROUTE_EC, [340](#)
 RTPS_LOG_DELETE_INDEX_EC, [328](#)
 RTPS_LOG_DELETE_INDEXER_EC, [332](#)
 RTPS_LOG_DELETE_RECORD_EC, [339](#)
 RTPS_LOG_DELETE_TABLE_EC, [328](#)
 RTPS_LOG_DESERIALIZE_APP_ID_EC, [326](#)
 RTPS_LOG_DESERIALIZE_HOST_ID_EC, [326](#)
 RTPS_LOG_DESERIALIZE_INSTANCE_ID_EC, [327](#)
 RTPS_LOG_DESERIALIZE_OBJECT_ID_EC, [327](#)
 RTPS_LOG_DIRECT_SEND_EC, [329](#)
 RTPS_LOG_EXCEEDED_LIMIT_PEERS_EC, [321](#)
 RTPS_LOG_EXCEEDED_LIMIT_TRANSPORTS_EC, [321](#)
 RTPS_LOG_FINALIZE_INTERFACE_EC, [331](#)
 RTPS_LOG_FIND_EXTERNAL_INTERFACE_EC, [322](#)
 RTPS_LOG_FIRST_SN_GREATER_LAST_SN_EC, [325](#)
 RTPS_LOG_FORWARD_UPSTREAM_EC, [320](#)
 RTPS_LOG_FULL_SEND_WINDOW_EC, [320](#)
 RTPS_LOG_FULLY_ACKED_READER_EC, [323](#)
 RTPS_LOG_FUNC_UNSUPPORTED_EC, [321](#)
 RTPS_LOG_GET_PEER_BUFFER_EC, [332](#)
 RTPS_LOG_GET_WINDOW_BUFFER_EC, [329](#)
 RTPS_LOG_INCONSISTENT_HDR_EXT_LENGTH_ERROR_EC, [337](#)
 RTPS_LOG_INDEX_ADD_ENTRY_EC, [317](#)
 RTPS_LOG_INDEXER_REMOVE_ENTRY_EC, [331](#)
 RTPS_LOG_INITIALIZE_INTERFACE_EC, [317](#)
 RTPS_LOG_INSERT_WINDOW_FULL_EC, [329](#)
 RTPS_LOG_INTERFACE_MISMATCH_EC, [320](#)
 RTPS_LOG_INTERFACE_NOT_ENABLED_EC, [324](#)
 RTPS_LOG_INTF_MODE_UNDEF_EC, [331](#)
 RTPS_LOG_INVALID_HDR_EXT_PID_ERROR_EC, [337](#)
 RTPS_LOG_INVALID_HDR_EXT_PID_LIST_EC, [337](#)
 RTPS_LOG_INVALID_PACKET_EC, [324](#)
 RTPS_LOG_LOCK_EC, [335](#)
 RTPS_LOG_LOOKUP_EXT_BIND_ENTRY_EC, [331](#)
 RTPS_LOG_LOOKUP_PEER_EC, [330](#)
 RTPS_LOG_MESSAGE_EXCEED_LENGTH_ERROR_EC, [338](#)
 RTPS_LOG_NEGATIVE_RESERVATION_COUNT_EC, [333](#)
 RTPS_LOG_NETIO_PACKET_INSUFFICIENT_BUFFER_EC, [333](#)
 RTPS_LOG_NETIO_PACKET_SET_HEAD_EC, [323](#)
 RTPS_LOG_NETIO_PACKET_SET_TAIL_EC, [321](#)
 RTPS_LOG_NONEXISTENT_BIND_EC, [322](#)
 RTPS_LOG_NONEXISTENT_EXTERNAL_BIND_EC, [322](#)
 RTPS_LOG_NONEXISTENT_ROUTE_EC, [322](#)
 RTPS_LOG_NOT_ENABLED_EC, [320](#)
 RTPS_LOG_PACKET_INITIALIZE_EC, [330](#)
 RTPS_LOG_PROCESS_ACKNACK_EC, [324](#)
 RTPS_LOG_PROCESS_DATA_BATCH_EC, [333](#)
 RTPS_LOG_PROCESS_DATA_EC, [324](#)
 RTPS_LOG_PROCESS_GAP_EC, [325](#)
 RTPS_LOG_PROCESS_HEARTBEAT_BATCH_EC, [333](#)
 RTPS_LOG_PROCESS_HEARTBEAT_EC, [325](#)
 RTPS_LOG_READER_SEND_ACKNACK_EC, [330](#)
 RTPS_LOG_READER_UNSUPPORTED_EC, [320](#)
 RTPS_LOG_RECEIVE_EC, [319](#)
 RTPS_LOG_REQUEST_EC, [323](#)
 RTPS_LOG_RESTORE_TRUST_LOAN_BUFFER_EC, [336](#)
 RTPS_LOG_RETURN_LOAN_EC, [323](#)
 RTPS_LOG_ROUTE_PACKET_EC, [320](#)
 RTPS_LOG_SCHEDULE_PACKET_EC, [335](#)
 RTPS_LOG_SEND_EC, [319](#)
 RTPS_LOG_SEND_HEARTBEAT_EC, [323](#)
 RTPS_LOG_SEND_LIVELINESS_EC, [339](#)
 RTPS_LOG_SEQ_FINALIZE_EC, [335](#)
 RTPS_LOG_SEQ_INIT_EC, [331](#)
 RTPS_LOG_SEQ_LEN_EC, [331](#)
 RTPS_LOG_SEQ_MAX_EC, [331](#)
 RTPS_LOG_SEQ_NUM_OUT_OF_RANGE_EC, [325](#)
 RTPS_LOG_SERIALIZE_APP_ID_EC, [326](#)
 RTPS_LOG_SERIALIZE_HOST_ID_EC, [326](#)
 RTPS_LOG_SERIALIZE_INSTANCE_ID_EC, [326](#)
 RTPS_LOG_SERIALIZE_OBJECT_ID_EC, [326](#)
 RTPS_LOG_SET_GAP_EC, [328](#)
 RTPS_LOG_SHIFT_BITMAP_EC, [323](#)

- RTPS_LOG_SHIFT_SN_OUT_OF_RANGE_EC, 326
- RTPS_LOG_STALE_ACK_EPOCH_EC, 322
- RTPS_LOG_STALE_HB_EPOCH_EC, 322
- RTPS_LOG_STATUS_CHANGE_EC, 328
- RTPS_LOG_TIMER_DELETE_TIMEOUT_EC, 329
- RTPS_LOG_TRANSFORM_RTPS_EC, 336
- RTPS_LOG_TRUST_BIND_FAILED_EC, 334
- RTPS_LOG_TRUST_INCONSISTENT_INTERCEPTOR_HANDLE_EC, 334
- RTPS_LOG_TRUST_INTERCEPTOR_HANDLE_LIST_FULL_EC, 334
- RTPS_LOG_TRUST_INVALID_SUBMSG_EC, 335
- RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_INVALID_EC, 334
- RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_INSERT_FAILED_EC, 334
- RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_LOOKUP_FAILED_EC, 333
- RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_LOOKUP_FAILED_IP4_EC, 333
- RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_LOOKUP_FAILED_IP6_EC, 333
- RTPS_LOG_TRUST_SUBMSG_CONVERSION_FAILED_EC, 334
- RTPS_LOG_TRUST_UNEXPECTED_READER_STATE_EC, 335
- RTPS_LOG_UNKNOWN_SUBMESSAGE_EC, 324
- RTPS_LOG_UNLOCK_EC, 335
- RTPS_LOG_UNSUPPORTED_INFO_REPLY_EC, 327
- RTPS_LOG_UNSUPPORTED_INFO_REPLY_IP4_EC, 327
- RTPS_LOG_UNSUPPORTED_INFO_SRC_EC, 327
- RTPS_LOG_UNSUPPORTED_INTERFACE_EC, 324
- RTPS_LOG_UNSUPPORTED_MANDATORY_HDR_EXTENSION_EC, 337
- RTPS_LOG_UPDATE_RELIABLE_PEERS_FILTER_EC, 339
- RTPS_LOG_WINDOW_ADVANCE_EC, 330
- RTPS_LOG_WINDOW_ADVANCE_UNACKED_EC, 330
- RTPS_LOG_WINDOW_INSERT_EC, 329
- RTPS_LOG_WINDOW_POOL_ALLOC_EC, 329
- RTPS_LOG_WRITER_REQUEST_SENT_HISTORY_EC, 330
- RTPS Transport API, 149
 - allow_builtin_override, 153
 - builtin_checksum128_class, 153
 - builtin_checksum32_class, 152
 - builtin_checksum64_class, 152
 - checksum, 153
 - checksum128, 152
 - checksum32, 152
 - checksum64, 152
 - checksum_calculate, 152
 - checksum_tx_mode, 153
 - class_id, 152
 - context, 152
 - RTPS_Checksum_T, 150
 - RTPS_CHECKSUM_TXMODE_OMG, 151
 - RTPS_CHECKSUM_TXMODE_RTICRC32, 151
 - RTPS_ChecksumCalculate_T, 150
 - RTPS_ChecksumClass_T, 151
 - RTPS_ChecksumClassId_T, 150
 - RTPS_ChecksumTxMode_T, 151
 - RTPS_INTERFACE_FACTORY_DEFAULT, 153
 - RTPS_InterfaceFactoryProperty_T, 151
 - rtps_app_id
 - DDS_WireProtocolQosPolicy, 666
 - RTPS_Transport_API, 150
 - RTPS_CHECKSUM_TXMODE_OMG, 151
 - RTPS_CHECKSUM_TXMODE_RTICRC32, 151
 - RTPS Transport API, 151
 - RTPS_ChecksumCalculate_T, 150
 - RTPS Transport API, 150
 - RTPS_ChecksumClass, 887
 - RTPS_ChecksumClass_T, 151
 - RTPS Transport API, 151
 - RTPS_ChecksumClassId_T, 150
 - RTPS Transport API, 150
 - RTPS_ChecksumProperty, 887
 - RTPS_ChecksumTxMode_T, 151
 - RTPS Transport API, 151
 - rtps_config.h, 993
 - rtps_dll.h, 993
 - rtps_instance_id
 - DDS_WireProtocolQosPolicy, 665
 - rtps_instance_id
 - DDS_WireProtocolQosPolicy, 666
 - RTPS_INTERFACE_FACTORY_DEFAULT, 153
 - RTPS Transport API, 153
 - RTPS_InterfaceFactoryProperty, 888
 - RTPS_InterfaceFactoryProperty_T, 151
 - RTPS Transport API, 151
 - rtps_log.h, 994
 - RTPS_LOG_DB_REMOVE_ROUTE_ENTRY_EC, 1001
 - RTPS_LOG_ACK_EC, 322
 - RTPS_LOG_ADD_FILTERED_ROUTE_EC, 339
 - RTPS_LOG_ALLOCATE_EC, 317
 - RTPS_LOG_APPLY_CONTENT_FILTER_EC, 339
 - RTPS_LOG_ASSERT_PEER_EC, 321

RTPS_LOG_ASSERT_TRANSPORT_EC	RTPS_LOG_CREATE_REASSEMBLY_TABLE_EC
RTPS, 321	RTPS, 336
RTPS_LOG_BAD_PARAMETER_EC	RTPS_LOG_CREATE_ROUTE_TABLE_EC
RTPS, 320	RTPS, 317
RTPS_LOG_BASE	RTPS_LOG_DATA_ALREADY_RECEIVED_EC
osapi_log.h, 978	RTPS, 324
RTPS_LOG_BITCOUNT_OUT_OF_BOUNDS_EC	RTPS_LOG_DATA_OUT_OF_RANGE_EC
RTPS, 325	RTPS, 323
RTPS_LOG_BITMAP_SET_BIT_EC	RTPS_LOG_DB_CREATE_BIND_ENTRY_EC
RTPS, 327	RTPS, 318
RTPS_LOG_BITMAP_SHIFT_EC	RTPS_LOG_DB_CREATE_BIND_PEER_INDEX_EC
RTPS, 330	RTPS, 317
RTPS_LOG_BUFFERPOOL_DELETE_EC	RTPS_LOG_DB_CREATE_DIRECT_INDEX_EC
RTPS, 329	RTPS, 332
RTPS_LOG_CHECKSUM_CHECKSUM_CALCULATION_FAILURE_EC	RTPS_LOG_DB_CREATE_EXT_BIND_ENTRY_EC
RTPS, 337	RTPS, 327
RTPS_LOG_CHECKSUM_CHECKSUM_DESERIALIZE_ERROR_EC	RTPS_LOG_DB_CREATE_GROUP_INDEX_EC
RTPS, 338	RTPS, 333
RTPS_LOG_CHECKSUM_CHECKSUM_ERROR_EC	RTPS_LOG_DB_CREATE_ROUTE_ENTRY_EC
RTPS, 339	RTPS, 318
RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM_LENGTH_EC	RTPS_LOG_DB_CREATE_ROUTE_PEER_INDEX_EC
RTPS, 336	RTPS, 317
RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM_VALUE_EC	RTPS_LOG_DB_CREATE_ROUTE_XPORT_INDEX_EC
RTPS, 336	RTPS, 317
RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM_VERSION_EC	RTPS_LOG_DB_CREATE_SELECTED_GROUP_INDEX_EC
RTPS, 336	RTPS, 335
RTPS_LOG_CHECKSUM_INCONSISTENT_LENGTH_EC	RTPS_LOG_DB_DELETE_EXTERNAL_BIND_ENTRY_EC
RTPS, 338	RTPS, 319
RTPS_LOG_CHECKSUM_INVALID_CRC32_FLAGS_EC	RTPS_LOG_DB_DELETE_ROUTE_ENTRY_EC
RTPS, 338	RTPS, 318
RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN128_EC	RTPS_LOG_DB_INSERT_BIND_ENTRY_EC
RTPS, 336	RTPS, 319
RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN32_EC	RTPS_LOG_DB_INSERT_ROUTE_ENTRY_EC
RTPS, 337	RTPS, 318
RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN64_EC	RTPS_LOG_DB_LOCK_EC
RTPS, 337	RTPS, 328
RTPS_LOG_CHECKSUM_LENGTH_DESERIALIZE_ERROR_EC	RTPS_LOG_DB_REMOVE_BIND_ENTRY_EC
RTPS, 338	RTPS, 319
RTPS_LOG_CHECKSUM_MISSING_CHECKSUM_LENGTH_EC	RTPS_LOG_DB_REMOVE_EXTERNAL_BIND_ENTRY_EC
RTPS, 338	RTPS, 319
RTPS_LOG_CHECKSUM_TOO_MANY_SG_BUFFERS_EC	RTPS_LOG_DB_REMOVE_ROUTE_ENTRY_EC
RTPS, 339	rtps_log.h, 1001
RTPS_LOG_CHECKSUM_UNSUPPORTED_EC	RTPS_LOG_DB_SELECT_ALL_EC
RTPS, 338	RTPS, 317
RTPS_LOG_CREATE_BIND_TABLE_EC	RTPS_LOG_DB_SELECT_MATCH_EC
RTPS, 328	RTPS, 318
RTPS_LOG_CREATE_BUFFER_POOL_EC	RTPS_LOG_DB_SELECT_RANGE_EC
RTPS, 332	RTPS, 318
RTPS_LOG_CREATE_EVENT_TIMER_EC	RTPS_LOG_DB_UNLOCK_EC
RTPS, 325	RTPS, 328
RTPS_LOG_CREATE_HB_EVENT_EC	RTPS_LOG_DELETE_BUFFER_POOL_EC
RTPS, 325	RTPS, 332
RTPS_LOG_CREATE_PEERS_INDEX_EC	RTPS_LOG_DELETE_FILTERED_ROUTE_EC
RTPS, 332	RTPS, 340

RTPS_LOG_DELETE_INDEX_EC
 RTPS, [328](#)
 RTPS_LOG_DELETE_INDEXER_EC
 RTPS, [332](#)
 RTPS_LOG_DELETE_RECORD_EC
 RTPS, [339](#)
 RTPS_LOG_DELETE_TABLE_EC
 RTPS, [328](#)
 RTPS_LOG_DESERIALIZE_APP_ID_EC
 RTPS, [326](#)
 RTPS_LOG_DESERIALIZE_HOST_ID_EC
 RTPS, [326](#)
 RTPS_LOG_DESERIALIZE_INSTANCE_ID_EC
 RTPS, [327](#)
 RTPS_LOG_DESERIALIZE_OBJECT_ID_EC
 RTPS, [327](#)
 RTPS_LOG_DIRECT_SEND_EC
 RTPS, [329](#)
 RTPS_LOG_EXCEEDED_LIMIT_PEERS_EC
 RTPS, [321](#)
 RTPS_LOG_EXCEEDED_LIMIT_TRANSPORTS_EC
 RTPS, [321](#)
 RTPS_LOG_FINALIZE_INTERFACE_EC
 RTPS, [331](#)
 RTPS_LOG_FIND_EXTERNAL_INTERFACE_EC
 RTPS, [322](#)
 RTPS_LOG_FIRST_SN_GREATER_LAST_SN_EC
 RTPS, [325](#)
 RTPS_LOG_FORWARD_UPSTREAM_EC
 RTPS, [320](#)
 RTPS_LOG_FULL_SEND_WINDOW_EC
 RTPS, [320](#)
 RTPS_LOG_FULLY_ACKED_READER_EC
 RTPS, [323](#)
 RTPS_LOG_FUNC_UNSUPPORTED_EC
 RTPS, [321](#)
 RTPS_LOG_GET_PEER_BUFFER_EC
 RTPS, [332](#)
 RTPS_LOG_GET_WINDOW_BUFFER_EC
 RTPS, [329](#)
 RTPS_LOG_INCONSISTENT_HDR_EXT_LENGTH_ERROR_EC
 RTPS, [337](#)
 RTPS_LOG_INDEX_ADD_ENTRY_EC
 RTPS, [317](#)
 RTPS_LOG_INDEXER_REMOVE_ENTRY_EC
 RTPS, [331](#)
 RTPS_LOG_INITIALIZE_INTERFACE_EC
 RTPS, [317](#)
 RTPS_LOG_INSERT_WINDOW_FULL_EC
 RTPS, [329](#)
 RTPS_LOG_INTERFACE_MISMATCH_EC
 RTPS, [320](#)
 RTPS_LOG_INTERFACE_NOT_ENABLED_EC
 RTPS, [324](#)
 RTPS_LOG_INTF_MODE_UNDEF_EC
 RTPS, [331](#)
 RTPS_LOG_INVALID_HDR_EXT_PID_ERROR_EC
 RTPS, [337](#)
 RTPS_LOG_INVALID_HDR_EXT_PID_LIST_EC
 RTPS, [337](#)
 RTPS_LOG_INVALID_PACKET_EC
 RTPS, [324](#)
 RTPS_LOG_LOCK_EC
 RTPS, [335](#)
 RTPS_LOG_LOOKUP_EXT_BIND_ENTRY_EC
 RTPS, [331](#)
 RTPS_LOG_LOOKUP_PEER_EC
 RTPS, [330](#)
 RTPS_LOG_MESSAGE_EXCEED_LENGTH_ERROR_EC
 RTPS, [338](#)
 RTPS_LOG_NEGATIVE_RESERVATION_COUNT_EC
 RTPS, [333](#)
 RTPS_LOG_NETIO_PACKET_INSUFFICIENT_BUFFER_EC
 RTPS, [333](#)
 RTPS_LOG_NETIO_PACKET_SET_HEAD_EC
 RTPS, [323](#)
 RTPS_LOG_NETIO_PACKET_SET_TAIL_EC
 RTPS, [321](#)
 RTPS_LOG_NONEXISTENT_BIND_EC
 RTPS, [322](#)
 RTPS_LOG_NONEXISTENT_EXTERNAL_BIND_EC
 RTPS, [322](#)
 RTPS_LOG_NONEXISTENT_ROUTE_EC
 RTPS, [322](#)
 RTPS_LOG_NOT_ENABLED_EC
 RTPS, [320](#)
 RTPS_LOG_PACKET_INITIALIZE_EC
 RTPS, [330](#)
 RTPS_LOG_PROCESS_ACKNACK_EC
 RTPS, [324](#)
 RTPS_LOG_PROCESS_DATA_BATCH_EC
 RTPS, [333](#)
 RTPS_LOG_PROCESS_DATA_EC
 RTPS, [324](#)
 RTPS_LOG_PROCESS_GAP_EC
 RTPS, [325](#)
 RTPS_LOG_PROCESS_HEARTBEAT_BATCH_EC
 RTPS, [333](#)
 RTPS_LOG_PROCESS_HEARTBEAT_EC
 RTPS, [325](#)
 RTPS_LOG_READER_SEND_ACKNACK_EC
 RTPS, [330](#)
 RTPS_LOG_READER_UNSUPPORTED_EC
 RTPS, [320](#)
 RTPS_LOG_RECEIVE_EC
 RTPS, [319](#)
 RTPS_LOG_REQUEST_EC
 RTPS, [323](#)

- RTPS_LOG_RESTORE_TRUST_LOAN_BUFFER_EC
 - RTPS, [336](#)
- RTPS_LOG_RETURN_LOAN_EC
 - RTPS, [323](#)
- RTPS_LOG_ROUTE_PACKET_EC
 - RTPS, [320](#)
- RTPS_LOG_SCHEDULE_PACKET_EC
 - RTPS, [335](#)
- RTPS_LOG_SEND_EC
 - RTPS, [319](#)
- RTPS_LOG_SEND_HEARTBEAT_EC
 - RTPS, [323](#)
- RTPS_LOG_SEND_LIVELINESS_EC
 - RTPS, [339](#)
- RTPS_LOG_SEQ_FINALIZE_EC
 - RTPS, [335](#)
- RTPS_LOG_SEQ_INIT_EC
 - RTPS, [331](#)
- RTPS_LOG_SEQ_LEN_EC
 - RTPS, [331](#)
- RTPS_LOG_SEQ_MAX_EC
 - RTPS, [331](#)
- RTPS_LOG_SEQ_NUM_OUT_OF_RANGE_EC
 - RTPS, [325](#)
- RTPS_LOG_SERIALIZE_APP_ID_EC
 - RTPS, [326](#)
- RTPS_LOG_SERIALIZE_HOST_ID_EC
 - RTPS, [326](#)
- RTPS_LOG_SERIALIZE_INSTANCE_ID_EC
 - RTPS, [326](#)
- RTPS_LOG_SERIALIZE_OBJECT_ID_EC
 - RTPS, [326](#)
- RTPS_LOG_SET_GAP_EC
 - RTPS, [328](#)
- RTPS_LOG_SHIFT_BITMAP_EC
 - RTPS, [323](#)
- RTPS_LOG_SHIFT_SN_OUT_OF_RANGE_EC
 - RTPS, [326](#)
- RTPS_LOG_STALE_ACK_EPOCH_EC
 - RTPS, [322](#)
- RTPS_LOG_STALE_HB_EPOCH_EC
 - RTPS, [322](#)
- RTPS_LOG_STATUS_CHANGE_EC
 - RTPS, [328](#)
- RTPS_LOG_TIMER_DELETE_TIMEOUT_EC
 - RTPS, [329](#)
- RTPS_LOG_TRANSFORM_RTPS_EC
 - RTPS, [336](#)
- RTPS_LOG_TRUST_BIND_FAILED_EC
 - RTPS, [334](#)
- RTPS_LOG_TRUST_INCONSISTENT_INTERCEPTOR_HANDLE_EC
 - RTPS, [334](#)
- RTPS_LOG_TRUST_INTERCEPTOR_HANDLE_LIST_FULL_EC
 - RTPS, [334](#)
- RTPS_LOG_TRUST_INVALID_SUBMSG_EC
 - RTPS, [335](#)
- RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_CREATE_FAILED_EC
 - RTPS, [334](#)
- RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_INSERT_FAILED_EC
 - RTPS, [334](#)
- RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_LOOKUP_FAILED_EC
 - RTPS, [333](#)
- RTPS_LOG_TRUST_SUBMSG_CONVERSION_FAILED_EC
 - RTPS, [334](#)
- RTPS_LOG_TRUST_UNEXPECTED_READER_STATE_EC
 - RTPS, [335](#)
- RTPS_LOG_UNKNOWN_SUBMESSAGE_EC
 - RTPS, [324](#)
- RTPS_LOG_UNLOCK_EC
 - RTPS, [335](#)
- RTPS_LOG_UNSUPPORTED_INFO_REPLY_EC
 - RTPS, [327](#)
- RTPS_LOG_UNSUPPORTED_INFO_REPLY_IP4_EC
 - RTPS, [327](#)
- RTPS_LOG_UNSUPPORTED_INFO_SRC_EC
 - RTPS, [327](#)
- RTPS_LOG_UNSUPPORTED_INTERFACE_EC
 - RTPS, [324](#)
- RTPS_LOG_UNSUPPORTED_MANDTORY_HDR_EXT_PID_EC
 - RTPS, [337](#)
- RTPS_LOG_UPDATE_RELIABLE_PEERS_FILTER_EC
 - RTPS, [339](#)
- RTPS_LOG_WINDOW_ADVANCE_EC
 - RTPS, [330](#)
- RTPS_LOG_WINDOW_ADVANCE_UNACKED_EC
 - RTPS, [330](#)
- RTPS_LOG_WINDOW_INSERT_EC
 - RTPS, [329](#)
- RTPS_LOG_WINDOW_POOL_ALLOC_EC
 - RTPS, [329](#)
- RTPS_LOG_WRITER_REQUEST_SENT_HISTORY_EC
 - RTPS, [330](#)
- rtps_object_id
 - DDS_DataReaderProtocolQosPolicy, [518](#)
 - DDS_DataWriterProtocolQosPolicy, [529](#)
- rtps_protocol_version
 - DDS_ParticipantBuiltinTopicData, [595](#)
- rtps_reliable_reader
 - DDS_DataReaderProtocolQosPolicy, [518](#)
- rtps_reliable_writer
 - DDS_DataWriterProtocolQosPolicy, [529](#)
- rtps_rtps_impl.h, [1001](#)
- rtps_vendor_id
 - DDS_ParticipantBuiltinTopicData, [595](#)
- rtps_wellknown_ports
 - DDS_WireProtocolQosPolicy, [666](#)
- Registry, [888](#)
- register_component, [889](#)

- unregister, 889
- Sample States, 69
 - DDS_ANY_SAMPLE_STATE, 70
 - DDS_NOT_READ_SAMPLE_STATE, 70
 - DDS_READ_SAMPLE_STATE, 70
 - DDS_SampleStateKind, 70
 - DDS_SampleStateMask, 70
- sample_info
 - DDS_SampleLostStatus, 644
- sample_state
 - DDS_SampleInfo, 641
- scheduling_policy
 - DDS_FlowControllerProperty_t, 571
- SDM_LOG_BASE
 - osapi_log.h, 977
- sec
 - DDS_Duration_t, 562
 - DDS_Time_t, 657
 - OSAPI_SystemTime, 881
- SEC_CORE_LOG_BASE
 - osapi_log.h, 978
- send
 - NETIO_DGRAM_Interfacel, 865
 - ZCOPY_NotifUserInterfacel, 909
- Sequence Number Support, 81
 - DDS_SequenceNumber_compare, 81
- Sequence Support, 128
- sequence_number
 - DDS_ReliableSampleUnacknowledgedStatus, 625
 - DDS_SampleIdentity_t, 639
- serialize_data_to_cdr_buffer
 - FooTypeSupport, 859
- serialize_data_to_cdr_buffer_ex
 - FooTypeSupport, 858
- serialize_on_write
 - DDS_DataWriterProtocolQosPolicy, 530
- service_name
 - DDS_PskServiceFactoryProperty, 609
- set_default_datareader_qos
 - DDSSubscriber, 784
- set_default_datawriter_qos
 - DDSPublisher, 773
- set_default_flowcontroller_property
 - DDSDomainParticipant, 736
- set_default_participant_qos
 - DDSDomainParticipantFactory, 745
- set_default_publisher_qos
 - DDSDomainParticipant, 724
- set_default_subscriber_qos
 - DDSDomainParticipant, 726
- set_default_topic_qos
 - DDSDomainParticipant, 727
- set_enabled_statuses
 - DDSStatusCondition, 780
- set_listener
 - DDSDataReader, 674
 - DDSDataWriter, 698
 - DDSDomainParticipant, 733
 - DDSPublisher, 776
 - DDSSubscriber, 788
 - DDSTopic, 794
- set_name
 - ENTITY_NAME, 121
- set_property
 - DDSFlowController, 761
- set_qos
 - DDSDataReader, 673
 - DDSDataWriter, 697
 - DDSDomainParticipant, 723
 - DDSDomainParticipantFactory, 747
 - DDSPublisher, 775
 - DDSSubscriber, 787
 - DDSTopic, 793
- set_trigger_value
 - DDSGuardCondition, 764
- Shared Memory Mutex API, 155
 - NETIO_SharedMemoryMutex_attach, 156
 - NETIO_SharedMemoryMutex_create, 156
 - NETIO_SharedMemoryMutex_create_or_attach, 157
 - NETIO_SharedMemoryMutex_delete, 160
 - NETIO_SharedMemoryMutex_detach, 160
 - NETIO_SharedMemoryMutex_lock, 159
 - NETIO_SharedMemoryMutex_unlock, 158
- Shared Memory Segment API, 161
 - NETIO_SharedMemorySegment_attach, 164
 - NETIO_SharedMemorySegment_create, 162
 - NETIO_SharedMemorySegment_create_or_attach, 162
 - NETIO_SharedMemorySegment_delete, 163
 - NETIO_SharedMemorySegment_detach, 164
 - NETIO_SharedMemorySegment_get_address, 165
 - NETIO_SharedMemorySegment_get_max_size, 166
 - NETIO_SharedMemorySegment_get_size, 165
- Shared Memory Signaling Semaphore, 167
 - NETIO_SharedMemorySignalingSemaphore_attach, 168
 - NETIO_SharedMemorySignalingSemaphore_create, 167
 - NETIO_SharedMemorySignalingSemaphore_create_or_attach, 169
 - NETIO_SharedMemorySignalingSemaphore_delete, 172
 - NETIO_SharedMemorySignalingSemaphore_detach, 171
 - NETIO_SharedMemorySignalingSemaphore_signal, 170

- NETIO_SharedMemorySignalingSemaphore_wait, 171
- Shared Memory Transport API, 153
- SHMEM_LOG_BASE
 - osapi_log.h, 977
- shmem_ref_settings
 - DDS_DataWriterTransferModeQosPolicy, 539
- shmem_ref_transfer_mode_attached_segment_allocation
 - DDS_DataReaderResourceLimitsQosPolicy, 526
- shmem_ref_transfer_mode_max_segments
 - DDS_DomainParticipantResourceLimitsQosPolicy, 557
- SHMEMInterfaceFactory, 890
 - get_interface, 890
- size
 - OSAPI_SharedMemorySegmentHeader, 876
- source_rules
 - UDP_InterfaceFactoryProperty, 895
- source_timestamp
 - DDS_SampleInfo, 642
 - DDS_WriteParams_t, 669
- source_timestamp_tolerance
 - DDS_DestinationOrderQosPolicy, 541
- sql_predicate_max_count_per_expression
 - DDS_FilterResourceLimits, 569
- stack_size
 - OSAPI_ThreadProperty, 882
- start_timer
 - OSAPI_SystemI, 877
- Status Kinds, 83
 - DDS_DATA_AVAILABLE_STATUS, 89
 - DDS_DATA_ON_READERS_STATUS, 89
 - DDS_DATA_WRITER_SAMPLE_REMOVED_STATUS, 92
 - DDS_INCONSISTENT_TOPIC_STATUS, 87
 - DDS_INSTANCE_REPLACED_STATUS, 91
 - DDS_LIVELINESS_CHANGED_STATUS, 90
 - DDS_LIVELINESS_LOST_STATUS, 90
 - DDS_OFFERED_DEADLINE_MISSED_STATUS, 87
 - DDS_OFFERED_INCOMPATIBLE_QOS_STATUS, 88
 - DDS_PUBLICATION_MATCHED_STATUS, 91
 - DDS_RELIABLE_READER_ACTIVITY_CHANGED_STATUS, 92
 - DDS_REQUESTED_DEADLINE_MISSED_STATUS, 88
 - DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS, 88
 - DDS_SAMPLE_LOST_STATUS, 89
 - DDS_SAMPLE_REJECTED_STATUS, 89
 - DDS_STATUS_MASK_ALL, 86
 - DDS_STATUS_MASK_NONE, 86
 - DDS_StatusKind, 87
 - DDS_StatusMask, 86
 - DDS_SUBSCRIPTION_MATCHED_STATUS, 91
- String Support, 130
 - DDS_String_alloc, 131
 - DDS_String_cmp, 132
 - DDS_String_dup, 132
 - DDS_String_free, 132
 - DDS_String_length, 133
 - DDS_String_ncmp, 133
- Subscriber, 65
 - DDS_DATAREADER_QOS_DEFAULT, 65
- subscriber_group_data_max_count
 - DDS_DomainParticipantResourceLimitsQosPolicy, 559
- subscriber_group_data_max_length
 - DDS_DomainParticipantResourceLimitsQosPolicy, 559
- subscriber_name
 - DDS_SubscriberQos, 648
- Subscription Built-in Topic, 42
 - DDS_SUBSCRIPTION_TOPIC_NAME, 42
- Subscription Module, 63
- subscription_name
 - DDS_DataReaderQos, 523
- suite
 - DDS_TrustQosPolicy, 661
- SYSTEM_RESOURCE_LIMITS, 114
- take
 - FooDataReader, 818
- take_instance
 - FooDataReader, 823
- take_instance_untyped
 - DDSDDataReader, 680
- take_next_sample
 - FooDataReader, 817
- take_next_sample_untyped
 - DDSDDataReader, 681
- take_untyped
 - DDSDDataReader, 676
- task_scheduler
 - OSAPI_SystemProperty, 880
- thread
 - OSAPI_TaskProperty, 881
 - OSAPI_TimerProperty, 883
- thread_prop
 - ZCOPY_NotifMechanismProperty, 903
- Time Support, 76
 - DDS_Duration_equal, 78
 - DDS_DURATION_INFINITE, 79
 - DDS_DURATION_INFINITE_NSEC, 79
 - DDS_DURATION_INFINITE_SEC, 79
 - DDS_Duration_is_infinite, 78
 - DDS_Duration_is_zero, 78
 - DDS_DURATION_ZERO, 79

- DDS_DURATION_ZERO_NSEC, [79](#)
- DDS_DURATION_ZERO_SEC, [79](#)
- DDS_TIME_INVALID, [77](#)
- DDS_TIME_INVALID_NSEC, [77](#)
- DDS_TIME_INVALID_SEC, [77](#)
- DDS_Time_is_zero, [78](#)
- DDS_TIME_ZERO, [77](#)
- timer_property
 - OSAPI_SystemProperty, [879](#)
- to_array
 - FooSeq, [846](#)
- token_bucket
 - DDS_FlowControllerProperty_t, [571](#)
- tokens_added_per_period
 - DDS_FlowControllerTokenBucketProperty_t, [572](#)
- tokens_leaked_per_period
 - DDS_FlowControllerTokenBucketProperty_t, [573](#)
- Topic, [42, 49](#)
- TOPIC_DATA, [125](#)
- topic_data
 - DDS_PublicationBuiltinTopicData, [615](#)
 - DDS_SubscriptionBuiltinTopicData, [653](#)
 - DDS_TopicQos, [658](#)
- topic_data_max_count
 - DDS_DomainParticipantResourceLimitsQosPolicy, [558](#)
- topic_data_max_length
 - DDS_DomainParticipantResourceLimitsQosPolicy, [558](#)
- topic_name
 - DDS_PublicationBuiltinTopicData, [612](#)
 - DDS_SubscriptionBuiltinTopicData, [650](#)
- total_count
 - DDS_DataReaderInstanceReplacedStatus, [517](#)
 - DDS_LivelinessLostStatus, [582](#)
 - DDS_OfferedDeadlineMissedStatus, [587](#)
 - DDS_OfferedIncompatibleQosStatus, [588](#)
 - DDS_PublicationMatchedStatus, [616](#)
 - DDS_RequestedDeadlineMissedStatus, [626](#)
 - DDS_RequestedIncompatibleQosStatus, [627](#)
 - DDS_SampleLostStatus, [644](#)
 - DDS_SampleRejectedStatus, [645](#)
 - DDS_SubscriptionMatchedStatus, [654](#)
- total_count_change
 - DDS_DataReaderInstanceReplacedStatus, [517](#)
 - DDS_LivelinessLostStatus, [582](#)
 - DDS_OfferedDeadlineMissedStatus, [587](#)
 - DDS_OfferedIncompatibleQosStatus, [588](#)
 - DDS_PublicationMatchedStatus, [616](#)
 - DDS_RequestedDeadlineMissedStatus, [626](#)
 - DDS_RequestedIncompatibleQosStatus, [627](#)
 - DDS_SampleLostStatus, [644](#)
 - DDS_SampleRejectedStatus, [645](#)
 - DDS_SubscriptionMatchedStatus, [654](#)
- Trace API, [190](#)
- transfer_mode
 - DATA_WRITER_TRANSFER_MODE, [126](#)
- transform_locator_kind
 - UDP_InterfaceFactoryProperty, [896](#)
- transform_udp_mode
 - UDP_InterfaceFactoryProperty, [896](#)
- TRANSPORT, [119](#)
- transport
 - DDS_DataReaderQos, [522](#)
 - DDS_DataWriterQos, [534](#)
- TRANSPORT_PRIORITY, [124](#)
- transport_priority
 - DDS_DataReaderQos, [523](#)
 - DDS_DataWriterQos, [534](#)
- transport_priority_mapping_high
 - UDP_InterfaceFactoryProperty, [896](#)
- transport_priority_mapping_low
 - UDP_InterfaceFactoryProperty, [896](#)
- transport_priority_mask
 - UDP_InterfaceFactoryProperty, [897](#)
- transports
 - DDS_DomainParticipantQos, [549](#)
- trigger_flow
 - DDSFlowController, [762](#)
- TRUST, [100](#)
- trust
 - DDS_DomainParticipantQos, [549](#)
- Type Support, [129](#)
 - DDS_InstanceId_t, [129](#)
 - NDDS_Type_PluginVersion, [129](#)
 - NDDS_TYPEPLUGIN_NO_KEY, [130](#)
 - NDDS_TYPEPLUGIN_USER_KEY, [130](#)
 - NDDS_TypePluginKeyKind, [130](#)
- type_name
 - DDS_PublicationBuiltinTopicData, [612](#)
 - DDS_SubscriptionBuiltinTopicData, [650](#)
- UDP Transform API, [149](#)
- UDP Transport, [238](#)
- UDP Transport API, [146](#)
 - add_entry, [148](#)
 - UDP_INTERFACE_INTERFACE_MULTICAST_FLAG, [147](#)
 - UDP_INTERFACE_INTERFACE_UP_FLAG, [147](#)
 - UDP_INTERFACE_MAX_IFNAME, [147](#)
 - UDP_INTERFACE_MAX_NETMASK_BITS, [147](#)
 - UDP_InterfaceFactoryProperty_finalize, [148](#)
 - UDP_InterfaceFactoryProperty_initialize, [147](#)
 - UDP_INTERFACE_INTERFACE_MULTICAST_FLAG
 - UDP Transport API, [147](#)
 - UDP_INTERFACE_INTERFACE_UP_FLAG
 - UDP Transport API, [147](#)
 - UDP_INTERFACE_MAX_IFNAME

- UDP Transport API, [147](#)
- UDP_INTERFACE_MAX_NETMASK_BITS
 - UDP Transport API, [147](#)
- UDP_InterfaceFactoryProperty, [891](#)
 - allow_interface, [892](#)
 - deny_interface, [892](#)
 - destination_rules, [896](#)
 - disable_auto_interface_config, [894](#)
 - disable_multicast_bind, [895](#)
 - disable_multicast_interface_select, [895](#)
 - enable_interface_bind, [895](#)
 - if_table, [893](#)
 - is_default_interface, [894](#)
 - max_message_size, [893](#)
 - max_receive_buffer_size, [892](#)
 - max_send_buffer_size, [892](#)
 - max_send_message_size, [893](#)
 - max_unicast_send_sockets, [897](#)
 - multicast_interface, [894](#)
 - multicast_loopback_disabled, [894](#)
 - multicast_ttl, [893](#)
 - nat, [893](#)
 - recv_thread, [894](#)
 - source_rules, [895](#)
 - transform_locator_kind, [896](#)
 - transform_udp_mode, [896](#)
 - transport_priority_mapping_high, [896](#)
 - transport_priority_mapping_low, [896](#)
 - transport_priority_mask, [897](#)
- UDP_InterfaceFactoryProperty_finalize
 - UDP Transport API, [148](#)
- UDP_InterfaceFactoryProperty_initialize
 - UDP Transport API, [147](#)
- UDP_InterfaceTableEntry, [898](#)
 - address, [898](#)
 - flags, [898](#)
 - ifname, [898](#)
 - netmask, [898](#)
- UDP_InterfaceTableEntrySeq, [899](#)
- UDP_LOG_ALLOC_EC
 - NETIO, [298](#)
- UDP_LOG_BASE
 - osapi_log.h, [977](#)
- UDP_LOG_CURSOR_ERROR_EC
 - NETIO, [300](#)
- UDP_LOG_DROPGRP_EC
 - NETIO, [299](#)
- UDP_LOG_FINALIZE_EC
 - NETIO, [295](#)
- UDP_LOG_GET_BUF_ALLOC_EC
 - NETIO, [297](#)
- UDP_LOG_GET_BUF_SIZE_EC
 - NETIO, [297](#)
- UDP_LOG_GET_IFLIST_EC
 - NETIO, [297](#)
- UDP_LOG_GET_LENGTH_EC
 - NETIO, [299](#)
- UDP_LOG_GET_NAMEINFO_EC
 - NETIO, [298](#)
- UDP_LOG_GETHOSTNAME_EC
 - NETIO, [294](#)
- UDP_LOG_INCONSISTENT_MAX_MESSAGE_SIZE_EC
 - NETIO, [299](#)
- UDP_LOG_INITIALIZE_FAILED_EC
 - NETIO, [300](#)
- UDP_LOG_LOADLIB_EC
 - NETIO, [297](#)
- UDP_LOG_MULTICAST_ENABLE_EC
 - NETIO, [298](#)
- UDP_LOG_MULTICAST_NOT_ENABLED_EC
 - NETIO, [300](#)
- UDP_LOG_NAT_DELETE_EC
 - NETIO, [295](#)
- UDP_LOG_PACKET_FWD_EC
 - NETIO, [295](#)
- UDP_LOG_PACKET_HEAD_EC
 - NETIO, [295](#)
- UDP_LOG_PACKET_INIT_EC
 - NETIO, [295](#)
- UDP_LOG_PORT_ADD_EC
 - NETIO, [296](#)
- UDP_LOG_PORT_ALLOC_EC
 - NETIO, [296](#)
- UDP_LOG_PORT_ENTRY_EC
 - NETIO, [296](#)
- UDP_LOG_PORT_NOT_FOUND_EC
 - NETIO, [300](#)
- UDP_LOG_PORT_THREAD_EC
 - NETIO, [296](#)
- UDP_LOG_PROPERTY_FINALIZE_EC
 - NETIO, [298](#)
- UDP_LOG_RECORD_EC
 - NETIO, [298](#)
- UDP_LOG_RECV_ERROR_EC
 - NETIO, [301](#)
- UDP_LOG_SEND_ERROR_EC
 - NETIO, [300](#)
- UDP_LOG_SEND_PING_EC
 - NETIO, [299](#)
- UDP_LOG_SET_LENGTH_EC
 - NETIO, [299](#)
- UDP_LOG_SOCKET_ADDGRP_EC
 - NETIO, [296](#)
- UDP_LOG_SOCKET_BIND_EC
 - NETIO, [296](#)
- UDP_LOG_SOCKET_CLOSE_EC
 - NETIO, [297](#)
- UDP_LOG_SOCKET_CREATE_EC

- NETIO, [294](#)
- UDP_LOG_SOCKET_IFFLAGS_EC
 - NETIO, [297](#)
- UDP_LOG_SOCKET_IFLIST_EC
 - NETIO, [297](#)
- UDP_LOG_SOCKET_INIT_EC
 - NETIO, [294](#)
- UDP_LOG_SOCKET_PRIORITY_EC
 - NETIO, [301](#)
- UDP_LOG_SOCKET_REUSE_PORT_EC
 - NETIO, [295](#)
- UDP_LOG_SOCKET_RXBUF_EC
 - NETIO, [296](#)
- UDP_LOG_SOCKET_SET_MCAST_EC
 - NETIO, [294](#)
- UDP_LOG_SOCKET_SET_MCASTIF_EC
 - NETIO, [294](#)
- UDP_LOG_SOCKET_SETFD_EC
 - NETIO, [294](#)
- UDP_LOG_SOCKET_SNDBUF_EC
 - NETIO, [294](#)
- UDP_LOG_SOCKET_TTL_EC
 - NETIO, [295](#)
- UDP_LOG_TOS_NOT_SUPPORTED_EC
 - NETIO, [301](#)
- UDP_LOG_TRANSPORT_NO_INTERFACES_EC
 - NETIO, [300](#)
- UDP_LOG_UDP_CREATE_INDEX_EC
 - NETIO, [298](#)
- UDP_LOG_UDP_CREATE_TABLE_EC
 - NETIO, [298](#)
- UDP_LOG_WINSOCK_INCOMPATIBLE_EC
 - NETIO, [299](#)
- UDP_LOG_WSA_STARTUP_EC
 - NETIO, [299](#)
- UDP_NatEntry, [899](#)
- UDP_NatEntrySeq, [899](#)
- UDP_PRIORITY_MAP_ALLOC_EC
 - NETIO, [303](#)
- UDP_PRIORITY_MAP_INVALID_MAPPING_EC
 - NETIO, [303](#)
- UDP_PRIORITY_MAP_NEGATIVE_MAPPING_EC
 - NETIO, [303](#)
- UDP_PRIORITY_MAP_PRIORITY_IGNORED_EC
 - netio_log.h, [961](#)
- UDP_TRANSFORM_LOG_ADD_EC
 - NETIO, [302](#)
- UDP_TRANSFORM_LOG_ALLOC_EC
 - NETIO, [301](#)
- UDP_TRANSFORM_LOG_CREATE_EC
 - NETIO, [302](#)
- UDP_TRANSFORM_LOG_DUPLICATE_EC
 - NETIO, [302](#)
- UDP_TRANSFORM_LOG_FACTORY_NOT_FOUND_EC
 - NETIO, [301](#)
- UDP_TRANSFORM_LOG_FIND_EC
 - NETIO, [302](#)
- UDP_TRANSFORM_LOG_INVALID_FACTORY_REFERENCE_EC
 - NETIO, [301](#)
- UDP_TRANSFORM_LOG_INVALID_TUDP_LOCATOR_KIND_EC
 - NETIO, [302](#)
- UDP_TRANSFORM_LOG_INVALID_UDP_MODE_EC
 - NETIO, [302](#)
- UDP_TRANSFORM_LOG_KIND_DST_INDEX
 - netio_log.h, [960](#)
- UDP_TRANSFORM_LOG_KIND_ENTRY
 - netio_log.h, [960](#)
- UDP_TRANSFORM_LOG_KIND_FACTORY
 - netio_log.h, [960](#)
- UDP_TRANSFORM_LOG_KIND_FACTORY_INDEX
 - netio_log.h, [960](#)
- UDP_TRANSFORM_LOG_KIND_RULE
 - netio_log.h, [960](#)
- UDP_TRANSFORM_LOG_KIND_SRC_INDEX
 - netio_log.h, [960](#)
- UDP_TRANSFORM_LOG_KIND_TRANSFORM_DESTINATION
 - netio_log.h, [960](#)
- UDP_TRANSFORM_LOG_KIND_TRANSFORM_NETMASK_INDEX
 - netio_log.h, [961](#)
- UDP_TRANSFORM_LOG_KIND_TRANSFORM_SOURCE
 - netio_log.h, [960](#)
- UDP_TRANSFORM_LOG_TRANSFORM_EC
 - NETIO, [302](#)
- UDPInterfaceFactory, [900](#)
 - get_interface, [900](#)
- UDPInterfaceTable, [900](#)
- unacknowledged_count
 - DDS_ReliableSampleUnacknowledgedStatus, [625](#)
- unbind
 - ZCOPY_NotifUserInterfaceI, [909](#)
- unicast_locator
 - DDS_PublicationBuiltinTopicData, [614](#)
 - DDS_SubscriptionBuiltinTopicData, [652](#)
- unloan
 - FooSeq, [851](#)
- unregister
 - RTRegistry, [889](#)
- unregister_instance
 - FooDataWriter, [831](#)
- unregister_instance_untyped
 - DDSDataWriter, [701](#)
- unregister_instance_w_timestamp
 - FooDataWriter, [832](#)
- unregister_instance_w_timestamp_untyped
 - DDSDataWriter, [702](#)
- unregister_type
 - DDSDomainParticipant, [732](#)
 - FooTypeSupport, [857](#)

- User Data Type Support, [50](#)
 - DDS_DATAREADER_CPP, [51](#)
 - DDS_DATAWRITER_CPP, [51](#)
 - DDS_HANDLE_NIL, [53](#)
 - DDS_InstanceHandle_equals, [52](#)
 - DDS_InstanceHandle_is_nil, [53](#)
 - DDS_InstanceHandle_t, [52](#)
 - DDS_TYPESUPPORT_CPP, [52](#)
- USER_DATA, [124](#)
- user_data
 - DDS_DataReaderQos, [522](#)
 - DDS_DataWriterQos, [534](#)
 - DDS_DomainParticipantQos, [550](#)
 - DDS_ParticipantBuiltinTopicData, [596](#)
 - DDS_PublicationBuiltinTopicData, [614](#)
 - DDS_SubscriptionBuiltinTopicData, [652](#)
- user_intf
 - ZCOPY_NotifInterfaceFactoryProperty, [902](#)
- user_multicast_port_offset
 - DDS_RtpsWellKnownPorts_t, [638](#)
- user_property
 - ZCOPY_NotifInterfaceFactoryProperty, [902](#)
- user_ptr
 - PSK_ErrListener, [884](#)
 - PSK_FilePassTrackerErrListener, [885](#)
- USER_TRAFFIC, [99](#)
- user_traffic
 - DDS_DomainParticipantQos, [549](#)
- user_unicast_port_offset
 - DDS_RtpsWellKnownPorts_t, [638](#)
- valid_data
 - DDS_SampleInfo, [642](#)
- value
 - DDS_BuiltinTopicKey_t, [511](#)
 - DDS_DataRepresentationQosPolicy, [528](#)
 - DDS_GroupDataQosPolicy, [575](#)
 - DDS_GUID_t, [576](#)
 - DDS_OwnershipStrengthQosPolicy, [593](#)
 - DDS_Property_t, [605](#)
 - DDS_PropertyQosPolicy, [606](#)
 - DDS_TopicDataQosPolicy, [658](#)
 - DDS_TransportPriorityQosPolicy, [659](#)
 - DDS_UserDataQosPolicy, [662](#)
 - NETIO_Address, [861](#)
 - NETIO_AddressUInt32, [862](#)
- vendorId
 - DDS_VendorId_t, [664](#)
- View States, [70](#)
 - DDS_ANY_VIEW_STATE, [72](#)
 - DDS_NEW_VIEW_STATE, [72](#)
 - DDS_NOT_NEW_VIEW_STATE, [72](#)
 - DDS_ViewStateKind, [71](#)
 - DDS_ViewStateMask, [71](#)
- view_state
 - DDS_SampleInfo, [641](#)
- wait
 - DDSWaitSet, [802](#)
- WH, [469](#)
 - WHSM_LOG_INVALID_INDEX_OBJECT_EC, [471](#)
 - WHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC, [470](#)
 - WHSM_LOG_MAX_SAMPLES_TOO_SMALL_EC, [470](#)
 - WHSM_LOG_NO_PROPERTY_EC, [471](#)
 - WHSM_LOG_OBJECT_ALLOCATE_EC, [470](#)
 - WHSM_LOG_OBJECT_DELETE_EC, [471](#)
 - WHSM_LOG_OBJECT_INDEX_EC, [471](#)
 - WHSM_LOG_UNLIMITED_HISTORY_NOT_SUPPORTED_EC, [470](#)
- wh_sm_log.h, [1001](#)
 - WHSM_LOG_HISTORY_OBJECT, [1002](#)
 - WHSM_LOG_HISTORYINDEX_OBJECT, [1003](#)
 - WHSM_LOG_KEY_OBJECT, [1002](#)
 - WHSM_LOG_KEYINDEX_OBJECT, [1003](#)
 - WHSM_LOG_KEYPOOL_OBJECT, [1003](#)
 - WHSM_LOG_SAMPLE_OBJECT, [1003](#)
 - WHSM_LOG_SAMPLEINDEX_OBJECT, [1003](#)
 - WHSM_LOG_SAMPLEPOOL_OBJECT, [1003](#)
- WHSM Writer History API, [141](#)
- WHSM_LOG_BASE
 - osapi_log.h, [978](#)
- WHSM_LOG_HISTORY_OBJECT
 - wh_sm_log.h, [1002](#)
- WHSM_LOG_HISTORYINDEX_OBJECT
 - wh_sm_log.h, [1003](#)
- WHSM_LOG_INVALID_INDEX_OBJECT_EC
 - WH, [471](#)
- WHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC
 - WH, [470](#)
- WHSM_LOG_KEY_OBJECT
 - wh_sm_log.h, [1002](#)
- WHSM_LOG_KEYINDEX_OBJECT
 - wh_sm_log.h, [1003](#)
- WHSM_LOG_KEYPOOL_OBJECT
 - wh_sm_log.h, [1003](#)
- WHSM_LOG_MAX_SAMPLES_TOO_SMALL_EC
 - WH, [470](#)
- WHSM_LOG_NO_PROPERTY_EC
 - WH, [471](#)
- WHSM_LOG_OBJECT_ALLOCATE_EC
 - WH, [470](#)
- WHSM_LOG_OBJECT_DELETE_EC
 - WH, [471](#)
- WHSM_LOG_OBJECT_INDEX_EC
 - WH, [471](#)
- WHSM_LOG_SAMPLE_OBJECT

- wh_sm_log.h, [1003](#)
- WHSM_LOG_SAMPLEINDEX_OBJECT
 - wh_sm_log.h, [1003](#)
- WHSM_LOG_SAMPLEPOOL_OBJECT
 - wh_sm_log.h, [1003](#)
- WHSM_LOG_UNLIMITED_HISTORY_NOT_SUPPORTED_EC
 - WH, [470](#)
- WHSMHistoryFactory, [901](#)
 - get_interface, [901](#)
- WHSQ, [462](#)
 - WHSQ_LOG_COMMIT_FAILED_EC, [464](#)
 - WHSQ_LOG_CONFIGURATION_EC, [464](#)
 - WHSQ_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC, [463](#)
 - WHSQ_LOG_MAX_SAMPLES_TOO_SMALL_EC, [463](#)
 - WHSQ_LOG_NO_DDS_FACTORY_EC, [464](#)
 - WHSQ_LOG_OBJECT_ALLOCATE_EC, [463](#)
 - WHSQ_LOG_OBJECT_DELETE_EC, [464](#)
 - WHSQ_LOG_PURGE_FAILED_EC, [463](#)
 - WHSQ_LOG_UNLIMITED_HISTORY_NOT_SUPPORTED_EC, [463](#)
 - WHSQ_LOG_WH_CREATE_FAILED_EC, [463](#)
- WHSQ_LOG_COMMIT_FAILED_EC
 - WHSQ, [464](#)
- WHSQ_LOG_CONFIGURATION_EC
 - WHSQ, [464](#)
- WHSQ_LOG_HISTORY_OBJECT
 - netio_zcopy_whsq_log.h, [968](#)
- WHSQ_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC
 - WHSQ, [463](#)
- WHSQ_LOG_MAX_SAMPLES_TOO_SMALL_EC
 - WHSQ, [463](#)
- WHSQ_LOG_NO_DDS_FACTORY_EC
 - WHSQ, [464](#)
- WHSQ_LOG_OBJECT_ALLOCATE_EC
 - WHSQ, [463](#)
- WHSQ_LOG_OBJECT_DELETE_EC
 - WHSQ, [464](#)
- WHSQ_LOG_PURGE_FAILED_EC
 - WHSQ, [463](#)
- WHSQ_LOG_UNLIMITED_HISTORY_NOT_SUPPORTED_EC
 - WHSQ, [463](#)
- WHSQ_LOG_WH_CREATE_FAILED_EC
 - WHSQ, [463](#)
- WIRE_PROTOCOL, [115](#)
 - DDS_CHECKSUM_AUTO, [116](#)
 - DDS_CHECKSUM_BUILTIN128, [116](#)
 - DDS_CHECKSUM_BUILTIN32, [116](#)
 - DDS_CHECKSUM_BUILTIN64, [116](#)
 - DDS_CHECKSUM_NONE, [116](#)
 - DDS_ChecksumKind_t, [117](#)
 - DDS_ChecksumKindMask_t, [117](#)
- DDS_INTEROPERABLE_RTPS_WELL_KNOWN_PORTS, [118](#)
- DDS_RTI_BACKWARDS_COMPATIBLE_RTPS_WELL_KNOWN_PORTS, [118](#)
- write
 - FooDataWriter, [828](#)
 - write_buffer
 - OSAPI_LogProperty, [875](#)
 - write_untyped
 - DDSDDataWriter, [694](#)
 - write_w_timestamp
 - FooDataWriter, [836](#)
 - write_w_timestamp_untyped
 - DDSDDataWriter, [705](#)
 - writer_guid
 - DDS_SampleIdentity_t, [639](#)
 - writer_loaned_sample_allocation
 - DDS_DataWriterResourceLimitsQosPolicy, [536](#)
 - writer_resource_limits
 - DDS_DataWriterQos, [534](#)
 - writer_user_data_max_count
 - DDS_DomainParticipantResourceLimitsQosPolicy, [559](#)
 - writer_user_data_max_length
 - DDS_DomainParticipantResourceLimitsQosPolicy, [559](#)
- XCDR_LOG_BASE
 - osapi_log.h, [977](#)
- ZCOPY_LOG_ADD_ENTRY_EC
 - ZeroCopy, [461](#)
- ZCOPY_LOG_DELETE_POOL_EC
 - ZeroCopy, [462](#)
- ZCOPY_LOG_NOTIF_ALLOC_EC
 - ZeroCopy, [459](#)
- ZCOPY_LOG_NOTIF_ALREADY_CONNECTED_EC
 - ZeroCopy, [454](#)
- ZCOPY_LOG_NOTIF_DB_CREATE_RECORD_EC
 - ZeroCopy, [456](#)
- ZCOPY_LOG_NOTIF_DB_CURSOR_NEXT_EC
 - ZeroCopy, [456](#)
- ZCOPY_LOG_NOTIF_DB_DELETE_RECORD_EC
 - ZeroCopy, [456](#)
- ZCOPY_LOG_NOTIF_DB_INSERT_RECORD_EC
 - ZeroCopy, [456](#)
- ZCOPY_LOG_NOTIF_DB_LOCK_EC
 - ZeroCopy, [456](#)
- ZCOPY_LOG_NOTIF_DB_REMOVE_RECORD_EC
 - ZeroCopy, [456](#)
- ZCOPY_LOG_NOTIF_DB_SELECT_RECORD_EC
 - ZeroCopy, [456](#)
- ZCOPY_LOG_NOTIF_DB_TABLE_CREATE_EC
 - ZeroCopy, [457](#)
- ZCOPY_LOG_NOTIF_DB_TABLE_DELETE_EC

- ZeroCopy, [457](#)
- ZCOPY_LOG_NOTIF_FACTORY_IN_USE_EC
 - ZeroCopy, [459](#)
- ZCOPY_LOG_NOTIF_GUID_BUFFER_TOO_SMALL_EC
 - ZeroCopy, [454](#)
- ZCOPY_LOG_NOTIF_INCOMPATIBLE_VERSION_EC
 - ZeroCopy, [455](#)
- ZCOPY_LOG_NOTIF_INTF_FINALIZE_EC
 - ZeroCopy, [459](#)
- ZCOPY_LOG_NOTIF_INTF_INIT_EC
 - ZeroCopy, [459](#)
- ZCOPY_LOG_NOTIF_INTF_RECEIVE_EC
 - ZeroCopy, [459](#)
- ZCOPY_LOG_NOTIF_INVALID_NUM_SLOTS_EC
 - ZeroCopy, [455](#)
- ZCOPY_LOG_NOTIF_INVALID_PORT_NUMBER_EC
 - ZeroCopy, [455](#)
- ZCOPY_LOG_NOTIF_INVALID_PROPERTY_EC
 - ZeroCopy, [457](#)
- ZCOPY_LOG_NOTIF_MECHANISM_ADD_ROUTE_EC
 - ZeroCopy, [460](#)
- ZCOPY_LOG_NOTIF_MECHANISM_BIND_EC
 - ZeroCopy, [460](#)
- ZCOPY_LOG_NOTIF_MECHANISM_CREATE_INST_EC
 - ZeroCopy, [460](#)
- ZCOPY_LOG_NOTIF_MECHANISM_DELETE_ROUTE_EC
 - ZeroCopy, [460](#)
- ZCOPY_LOG_NOTIF_MECHANISM_RELEASE_ADDR_EC
 - ZeroCopy, [460](#)
- ZCOPY_LOG_NOTIF_MECHANISM_RESERVE_ADDR_EC
 - ZeroCopy, [460](#)
- ZCOPY_LOG_NOTIF_MECHANISM_SEND_EC
 - ZeroCopy, [461](#)
- ZCOPY_LOG_NOTIF_MECHANISM_UNBIND_EC
 - ZeroCopy, [460](#)
- ZCOPY_LOG_NOTIF_NO_FREE_SLOT_EC
 - ZeroCopy, [455](#)
- ZCOPY_LOG_NOTIF_NO_MORE_NOTIFIERS_EC
 - ZeroCopy, [457](#)
- ZCOPY_LOG_NOTIF_NOT_CONNECTED_EC
 - ZeroCopy, [455](#)
- ZCOPY_LOG_NOTIF_NOTIFIEE_DESTROY_EC
 - ZeroCopy, [458](#)
- ZCOPY_LOG_NOTIF_NOTIFIEE_NEXT_EC
 - ZeroCopy, [458](#)
- ZCOPY_LOG_NOTIF_NOTIFIEE_RAISE_FLAG_EC
 - ZeroCopy, [458](#)
- ZCOPY_LOG_NOTIF_NOTIFIER_ALLOW_EC
 - ZeroCopy, [458](#)
- ZCOPY_LOG_NOTIF_NOTIFIER_CONNECT_EC
 - ZeroCopy, [457](#)
- ZCOPY_LOG_NOTIF_NOTIFIER_DISCONNECT_EC
 - ZeroCopy, [458](#)
- ZCOPY_LOG_NOTIF_NOTIFIER_NOTIFY_EC
 - ZeroCopy, [458](#)
- ZCOPY_LOG_NOTIF_OPEN_FAILED_EC
 - ZeroCopy, [455](#)
- ZCOPY_LOG_NOTIF_PORT_IN_USE_EC
 - ZeroCopy, [459](#)
- ZCOPY_LOG_NOTIF_RECEIVE_ON_EVENT_EC
 - ZeroCopy, [461](#)
- ZCOPY_LOG_NOTIF_SLOT_ALREADY_ASSIGNED_EC
 - ZeroCopy, [455](#)
- ZCOPY_LOG_NOTIF_SQ_CLOSE_EC
 - ZeroCopy, [457](#)
- ZCOPY_LOG_NOTIF_SQ_OPEN_EC
 - ZeroCopy, [457](#)
- ZCOPY_LOG_NOTIF_THREAD_CREATE_EC
 - ZeroCopy, [458](#)
- ZCOPY_LOG_NOTIF_THREAD_DESTROY_EC
 - ZeroCopy, [459](#)
- ZCOPY_LOG_NOTIF_TIMEOUT_CREATE_EC
 - ZeroCopy, [461](#)
- ZCOPY_LOG_NOTIF_TIMEOUT_DELETE_EC
 - ZeroCopy, [461](#)
- ZCOPY_LOG_NOTIF_TIMER_CREATE_EC
 - ZeroCopy, [461](#)
- ZCOPY_LOG_REMOVE_ENTRY_EC
 - ZeroCopy, [462](#)
- ZCOPY_LOG_SHARED_MEM_POOL_OWNER_DEAD_EC
 - ZeroCopy, [462](#)
- ZCOPY_LOG_WH_REGISTER_FAILED_EC
 - ZeroCopy, [461](#)
- ZCOPY_NotifInterfaceFactory_register
 - netio_zcopy_notif_interface.h, [967](#)
- ZCOPY_NotifInterfaceFactory_unregister
 - netio_zcopy_notif_interface.h, [967](#)
- ZCOPY_NotifInterfaceFactoryProperty, [901](#)
 - max_samples_per_notif, [902](#)
 - user_intf, [902](#)
 - user_property, [902](#)
- ZCOPY_NotifInterfaceFactoryProperty_INITIALIZER
 - Zero Copy v2 API, [181](#)
- ZCOPY_NotifMechanismProperty, [902](#)
 - intf_addr, [903](#)
 - max_receive_ports, [903](#)
 - max_routes, [903](#)
 - thread_prop, [903](#)
- ZCOPY_NotifUserInterface_add_routeFunc
 - Zero Copy v2 API, [182](#)
- ZCOPY_NotifUserInterface_bindFunc
 - Zero Copy v2 API, [182](#)
- ZCOPY_NotifUserInterface_createFunc
 - Zero Copy v2 API, [181](#)
- ZCOPY_NotifUserInterface_delete_routeFunc
 - Zero Copy v2 API, [182](#)
- ZCOPY_NotifUserInterface_deleteFunc
 - Zero Copy v2 API, [181](#)

- ZCOPY_NotifUserInterface_notify_portFunc
 - Zero Copy v2 API, [183](#)
- ZCOPY_NotifUserInterface_release_addressFunc
 - Zero Copy v2 API, [181](#)
- ZCOPY_NotifUserInterface_reserve_addressFunc
 - Zero Copy v2 API, [181](#)
- ZCOPY_NotifUserInterface_sendFunc
 - Zero Copy v2 API, [182](#)
- ZCOPY_NotifUserInterface_unbindFunc
 - Zero Copy v2 API, [182](#)
- ZCOPY_NotifUserInterfaceI, [904](#)
 - add_route, [908](#)
 - bind, [908](#)
 - create_instance, [905](#)
 - delete_instance, [905](#)
 - delete_route, [908](#)
 - get_route_table, [906](#)
 - notify_rcv_port, [910](#)
 - release_address, [906](#)
 - reserve_address, [906](#)
 - resolve_address, [905](#)
 - send, [909](#)
 - unbind, [909](#)
 - Zero Copy v2 API, [183](#)
- Zero Copy Transfer Over Shared Memory, [50](#)
- Zero Copy v2 API, [179](#)
 - ZCOPY_NotifInterfaceFactoryProperty_INITIALIZER, [181](#)
 - ZCOPY_NotifUserInterface_add_routeFunc, [182](#)
 - ZCOPY_NotifUserInterface_bindFunc, [182](#)
 - ZCOPY_NotifUserInterface_createFunc, [181](#)
 - ZCOPY_NotifUserInterface_delete_routeFunc, [182](#)
 - ZCOPY_NotifUserInterface_deleteFunc, [181](#)
 - ZCOPY_NotifUserInterface_notify_portFunc, [183](#)
 - ZCOPY_NotifUserInterface_release_addressFunc, [181](#)
 - ZCOPY_NotifUserInterface_reserve_addressFunc, [181](#)
 - ZCOPY_NotifUserInterface_sendFunc, [182](#)
 - ZCOPY_NotifUserInterface_unbindFunc, [182](#)
 - ZCOPY_NotifUserInterfaceI, [183](#)
- ZeroCopy, [452](#)
 - ZCOPY_LOG_ADD_ENTRY_EC, [461](#)
 - ZCOPY_LOG_DELETE_POOL_EC, [462](#)
 - ZCOPY_LOG_NOTIF_ALLOC_EC, [459](#)
 - ZCOPY_LOG_NOTIF_ALREADY_CONNECTED_EC, [454](#)
 - ZCOPY_LOG_NOTIF_DB_CREATE_RECORD_EC, [456](#)
 - ZCOPY_LOG_NOTIF_DB_CURSOR_NEXT_EC, [456](#)
 - ZCOPY_LOG_NOTIF_DB_DELETE_RECORD_EC, [456](#)
 - ZCOPY_LOG_NOTIF_DB_INSERT_RECORD_EC, [456](#)
 - ZCOPY_LOG_NOTIF_DB_LOCK_EC, [456](#)
 - ZCOPY_LOG_NOTIF_DB_REMOVE_RECORD_EC, [456](#)
 - ZCOPY_LOG_NOTIF_DB_SELECT_RECORD_EC, [456](#)
 - ZCOPY_LOG_NOTIF_DB_TABLE_CREATE_EC, [457](#)
 - ZCOPY_LOG_NOTIF_DB_TABLE_DELETE_EC, [457](#)
 - ZCOPY_LOG_NOTIF_FACTORY_IN_USE_EC, [459](#)
 - ZCOPY_LOG_NOTIF_GUID_BUFFER_TOO_SMALL_EC, [454](#)
 - ZCOPY_LOG_NOTIF_INCOMPATIBLE_VERSION_EC, [455](#)
 - ZCOPY_LOG_NOTIF_INTF_FINALIZE_EC, [459](#)
 - ZCOPY_LOG_NOTIF_INTF_INIT_EC, [459](#)
 - ZCOPY_LOG_NOTIF_INTF_RECEIVE_EC, [459](#)
 - ZCOPY_LOG_NOTIF_INVALID_NUM_SLOTS_EC, [455](#)
 - ZCOPY_LOG_NOTIF_INVALID_PORT_NUMBER_EC, [455](#)
 - ZCOPY_LOG_NOTIF_INVALID_PROPERTY_EC, [457](#)
 - ZCOPY_LOG_NOTIF_MECHANISM_ADD_ROUTE_EC, [460](#)
 - ZCOPY_LOG_NOTIF_MECHANISM_BIND_EC, [460](#)
 - ZCOPY_LOG_NOTIF_MECHANISM_CREATE_INST_EC, [460](#)
 - ZCOPY_LOG_NOTIF_MECHANISM_DELETE_ROUTE_EC, [460](#)
 - ZCOPY_LOG_NOTIF_MECHANISM_RELEASE_ADDR_EC, [460](#)
 - ZCOPY_LOG_NOTIF_MECHANISM_RESERVE_ADDR_EC, [460](#)
 - ZCOPY_LOG_NOTIF_MECHANISM_SEND_EC, [461](#)
 - ZCOPY_LOG_NOTIF_MECHANISM_UNBIND_EC, [460](#)
 - ZCOPY_LOG_NOTIF_NO_FREE_SLOT_EC, [455](#)
 - ZCOPY_LOG_NOTIF_NO_MORE_NOTIFIERS_EC, [457](#)
 - ZCOPY_LOG_NOTIF_NOT_CONNECTED_EC, [455](#)
 - ZCOPY_LOG_NOTIF_NOTIFIEE_DESTROY_EC, [458](#)
 - ZCOPY_LOG_NOTIF_NOTIFIEE_NEXT_EC, [458](#)
 - ZCOPY_LOG_NOTIF_NOTIFIEE_RAISE_FLAG_EC, [458](#)
 - ZCOPY_LOG_NOTIF_NOTIFIER_ALLOW_EC, [458](#)
 - ZCOPY_LOG_NOTIF_NOTIFIER_CONNECT_EC, [457](#)
 - ZCOPY_LOG_NOTIF_NOTIFIER_DISCONNECT_EC, [458](#)

ZCOPY_LOG_NOTIF_NOTIFIER_NOTIFY_EC, [458](#)
ZCOPY_LOG_NOTIF_OPEN_FAILED_EC, [455](#)
ZCOPY_LOG_NOTIF_PORT_IN_USE_EC, [459](#)
ZCOPY_LOG_NOTIF_RECEIVE_ON_EVENT_EC,
[461](#)
ZCOPY_LOG_NOTIF_SLOT_ALREADY_ASSIGNED_EC,
[455](#)
ZCOPY_LOG_NOTIF_SQ_CLOSE_EC, [457](#)
ZCOPY_LOG_NOTIF_SQ_OPEN_EC, [457](#)
ZCOPY_LOG_NOTIF_THREAD_CREATE_EC, [458](#)
ZCOPY_LOG_NOTIF_THREAD_DESTROY_EC,
[459](#)
ZCOPY_LOG_NOTIF_TIMEOUT_CREATE_EC, [461](#)
ZCOPY_LOG_NOTIF_TIMEOUT_DELETE_EC, [461](#)
ZCOPY_LOG_NOTIF_TIMER_CREATE_EC, [461](#)
ZCOPY_LOG_REMOVE_ENTRY_EC, [462](#)
ZCOPY_LOG_SHARED_MEM_POOL_OWNER_DEAD_EC,
[462](#)
ZCOPY_LOG_WH_REGISTER_FAILED_EC, [461](#)